PUBLIC
2026-10-27

SAP BTP Connectivity

© 2026 SAP SE or an SAP affiliate company. All rights reserved.

Content

1 What is SAP BTP Connectivity? 4

1.1 Connectivity for the Multi-Cloud Foundation 11

Getting Started 12

Concepts 16

Connectivity Scenarios 27

What's New for Connectivity 142

Administration 1300

Developing Applications 1306

Connectivity in the Kyma Environment 1307

1.2 Connectivity Service 1308

Connectivity Service: Administration 1309

Consuming the Connectivity Service 1314

1.3 Destination Service 1347

Destination Service: Administration 1348

Consuming the Destination Service 1550

1.4 Cloud Connector 1602

Installation 1608

Configuration 1669

Administration 1970

Security 2057

Upgrade 2069

Update the Java VM 2070

Uninstallation 2072

Frequently Asked Questions 2073

REST APIs 2084

1.5 Connectivity Proxy for Kubernetes 2088

Concepts 2089

Lifecycle Management 2120

Verification and Testing 2143

Monitoring 2145

Using the Connectivity Proxy 2148

Troubleshooting 2152

Frequently Asked Questions 2159

Connectivity Proxy in the Kyma Environment 2160

1.6 Transparent Proxy for Kubernetes 2164

Using the Transparent Proxy 2166

Integration with SAP BTP Connectivity. 2247
Lifecycle Management. 2252
Monitoring. 2294
Troubleshooting. 2297
Recommended Actions. 2326
Resilience. 2328
Verification and Testing. 2330
Transparent Proxy in the Kyma Environment. 2332
Concepts. 2336
Local Development 2356
Transparent Proxy Operator. 2358
Security Considerations. 2359
1.7 Connectivity via Reverse Proxy. 2360
1.8 Connectivity Support. 2362
Release and Maintenance Strategy. 2364
1.9 Submitting Improvement Requests. 2365

1 What is SAP BTP Connectivity?

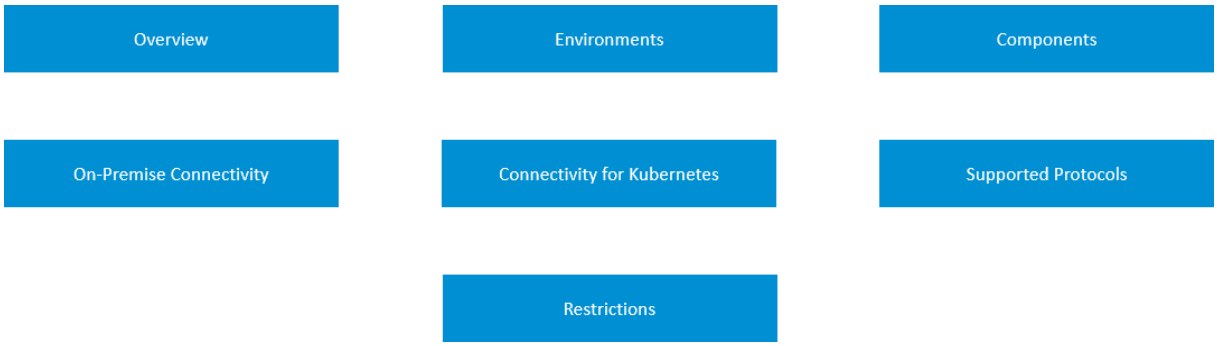
SAP BTP Connectivity: overview, features, restrictions.

> **Note**
>
> This documentation refers to SAP BTP, multi-cloud foundation. If you are looking for information about the Neo environment, see Connectivity for the Neo Environment.
>
> For more information on SAP BTP, multi-cloud Foundation, see also What Is the Multi-Cloud Foundation?.

Content

In this Topic

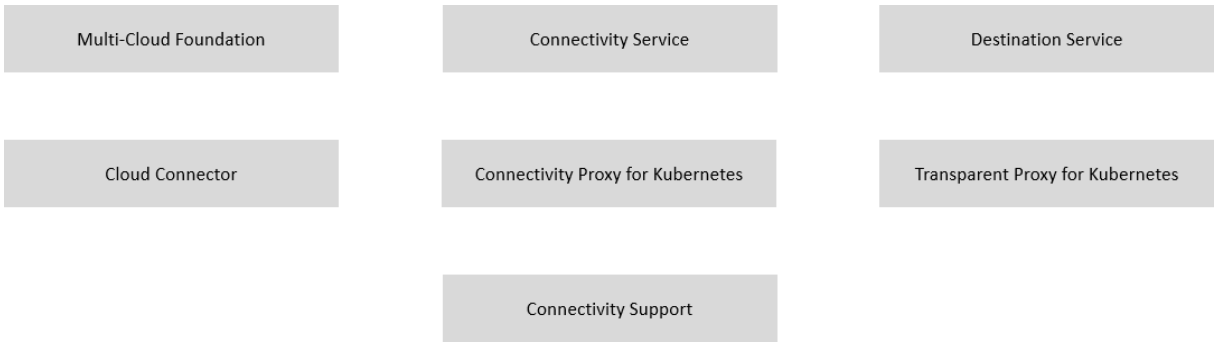
Hover over the elements for a description. Click an element for more information.



- [page 5]
- [page 6]
- Components and services for SAP BTP Connectivity [page 7]
- On-Premise connectivity: basic scenario [page 7]
- Connectivity for Kubernetes: components and architecture [page 8]
- Supported Protocols [page 8]
- Restrictions [page 9]

In this Guide

Hover over the elements for a description. Click an element for more information.



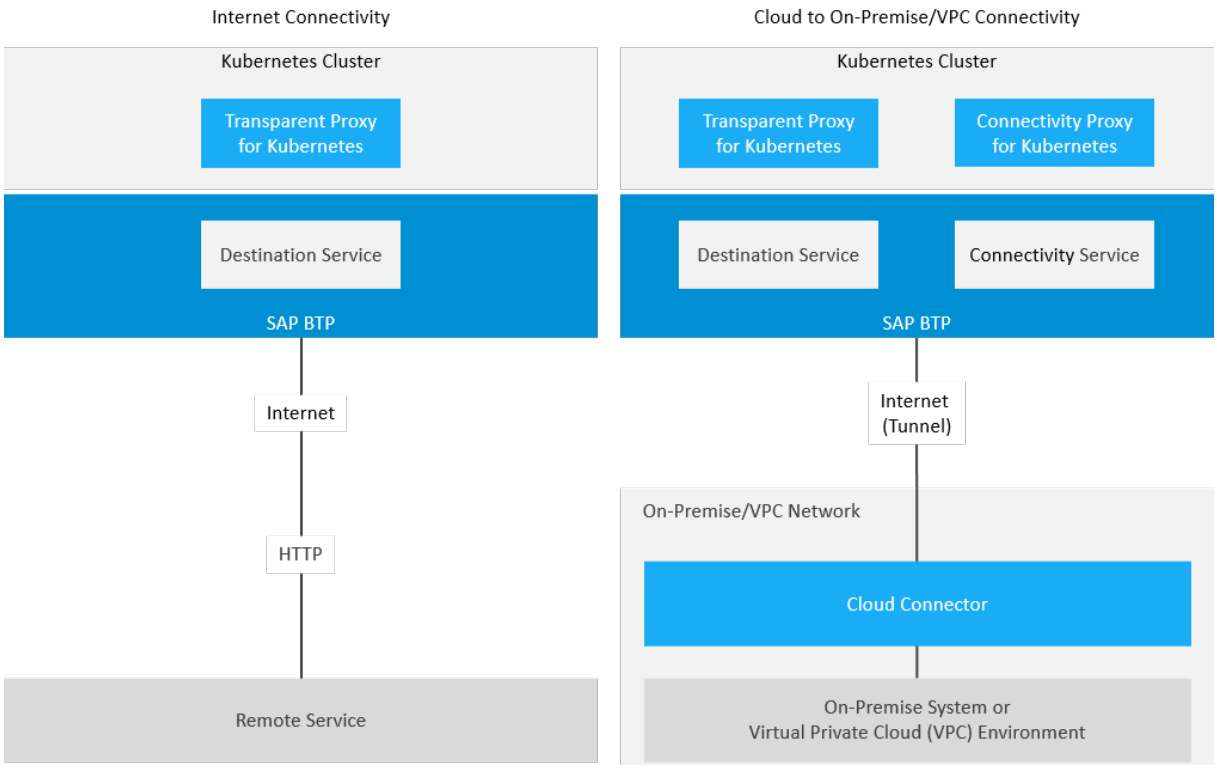
- Consuming SAP BTP Connectivity for your application in the Multi-Cloud Foundation: Overview. [page 11]
- Learn how to manage and consume the Connectivity service in SAP BTP, Multi-Cloud Foundation. [page 1308]
- Learn how to manage and consume the Destination service in SAP BTP, Multi-Cloud Foundation. [page 1347]
- Learn more about the Cloud Connector: features, scenarios and setup. [page 1602]
- Use the Connectivity Proxy for Kubernetes and the Kyma environment to connect workloads on a Kubernetes cluster to on-premise systems. [page 2088]
- Use the Transparent Proxy for Kubernetes and the Kyma environment to connect workloads on a Kubernetes cluster to Internet and on-premise applications. [page 2164]
- Support information for SAP BTP Connectivity. [page 2362]

Overview

SAP BTP Connectivity offers a set of services and software components which allow SAP cloud applications to securely access remote services that run on the Internet or on-premise. This framework:

- Allows subaccount-specific configuration of application connections via destinations.
- Allows you to make connections to on-premise systems, using the Cloud Connector.
- Lets you establish a secure tunnel from your on-premise network to applications on SAP BTP, while you keep full control and auditability of what is exposed to the cloud.
- Supports both the Neo environment and the multi-cloud foundation for application development on SAP BTP.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.



- For Internet communication, the multi-cloud foundation provides the Destination service to establish a connection from your application to another remote service.
  - The Transparent Proxy lets you connect workloads on a Kubernetes cluster.
- To connect to an on-premsie system or service, you can additionally use the Connectivity service and the Connectivity Proxy (for Kubernetes workloads).
  - On-premise scenarios also require the Cloud Connector, which is installed within your local network and routes the requests from and to SAP BTP.
- For inbound connections (calling an application or service on SAP BTP from an external source), you can use Cloud Connector service channels [page 1949] (on-premise connections) or the respective API endpoints of your SAP BTP region (Internet connections).

For more information, see Connectivity Scenarios [page 27].

Back to Content [page 4]

Environments

SAP BTP, multi-cloud foundation covers three environments:

- **Cloud Foundry**
  For more information, see Getting Started in the Cloud Foundry Environment.
- **ABAP**
  For more information, see Getting Started in the ABAP Environment.

- **Kyma**
  For more information, see Getting Started in the Kyma Environment.

Components

SAP BTP Connectivity provides the following services and components that you can use in different combinations to configure your specific connectivity scenario:

- Connectivity Service [page 1308]
- Destination Service [page 1347]
- Cloud Connector [page 1602]
- Connectivity Proxy [page 2088] (for Kubernetes and SAP BTP, Kyma Environment)
- Transparent Proxy [page 2164] (for Kubernetes and SAP BTP, Kyma Environment)

For more information, see Getting Started [page 12].

On-Premise Connectivity

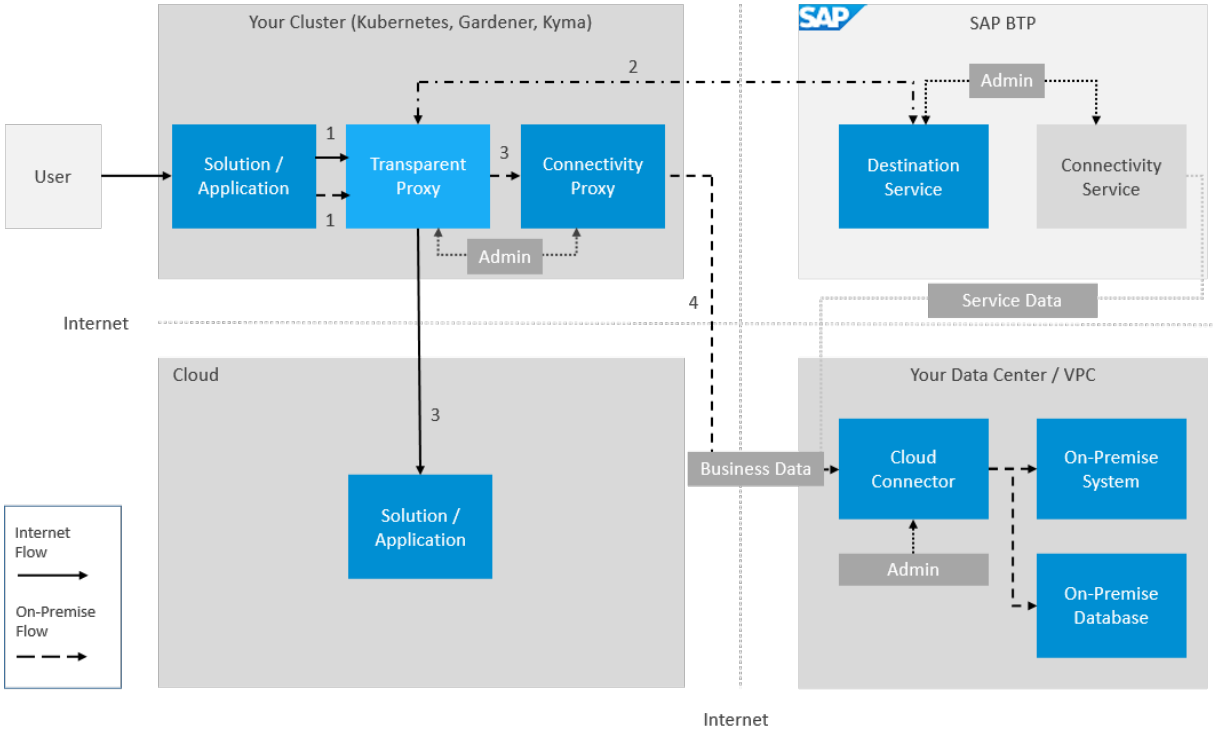
> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

A typical scenario for connecting your on-premise network to SAP BTP looks like this:

- Your company owns a global account on SAP BTP and one or more subaccounts that are assigned to this global account.
- Using SAP BTP, you subscribe to or deploy your own applications.
- To connect to these applications from your on-premise network, the Cloud Connector administrator sets up a secure tunnel to your company's subaccount on SAP BTP.
- The platform ensures that the tunnel can only be used by applications that are assigned to your subaccount.
- Applications assigned to other (sub)accounts cannot access the tunnel. It is encrypted via transport layer security (TLS), which guarantees connection privacy.

For more information, see Connectivity Scenarios [page 27].

Connectivity for Kubernetes



You can use dedicated components to connect Kubernetes workloads to remote systems on the Internet or on-premise.

- The Transparent Proxy lets you connect your Kubernetes workloads to remote systems using SAP BTP destinations easily. It provides authentication, simplifies user propagation, automated Connectivity Proxy integration, and unified access to the destination target systems.
- The Connectivity Proxy for Kubernetes connects workloads running on a Kubernetes cluster to on-premise systems, which are exposed via the Cloud Connector.

You can deploy the Transparent Proxy and the Connectivity Proxy either as standalone components, or as modules in SAP BTP, Kyma environment.

For more information, see:

- Connectivity Proxy for Kubernetes [page 2088]
- Transparent Proxy for Kubernetes [page 2164]
- Connectivity in the Kyma Environment [page 1307]
- Connectivity Scenarios [page 27]

Supported Protocols

SAP BTP Connectivity supports the following protocols and scenarios:

Protocol	Scenario
HTTP(S)	Exchange data between your cloud application and Internet services or on-premise systems. • Create and configure HTTP destinations to make Web connections. • Connect to on-premise systems via HTTP, using the Cloud Connector.
RFC	Invoke on-premise ABAP function modules via RFC. • Create and configure RFC destinations. • Make connections to back-end systems via RFC, using the Cloud Connector.
TCP	Access Internet or on-premise systems, generically on TCP level.
LDAP	Enables LDAP-based user management if you are operating an LDAP server within your network.
Mail Protocols	The e-mail provider and e-mail account are configured using MAIL destinations. • The **SMTP** protocol allows you to send e-mail messages from your Web applications using e-mail providers that are accessible on the Internet, such as Google Mail (Gmail). • The **IMAP** and **POP3** protocols let you retrieve e-mails from the mailbox of your e-mail account.

Back to Content [page 4]

Restrictions

General [page 9]

Protocols [page 10]

multi-cloud foundation [page 10]

Cloud Connector [page 11]

> ⓘ Note
>
> For information about general SAP BTP restrictions, see Prerequisites and Restrictions.

General

Topic	Restriction
Java Connector for RFC	To develop a Java Connector (JCo) application for RFC communication, your SDK local runtime must be hosted by a 64-bit JVM, on a x86_64 operating system (Microsoft Windows OS, Linux OS, or Mac OS X). On Windows platforms, you must install the **Microsoft Visual Studio C++ 2013** runtime libraries (vcredist_x64.exe), see Visual C++ Redistributable Packages for Visual Studio 2013 .
Ports	For Internet connections, you are allowed to use any port >`1024`. For cloud to on-premise scenarios, there are no port limitations.
Destination Configuration	You can use destination configuration files with extension `.props`, `.properties`, `.jks`, and `.txt`, as well as files with no extension.

Back to Restrictions [page 9]

Protocols

For the cloud to on-premise connectivity scenario, the following protocols are currently supported:

Protocol	Info
HTTP	HTTPS is not needed, since the tunnel used by the Cloud Connector is TLS-encrypted.
RFC	You can communicate with SAP systems down to SAP R/3 release 4.6C. Supported runtime environment is SAP Java Buildpack with a minimal version of 1.8.0.
LDAP	
TCP	You can use TCP-based communication for any client that supports TCP.

Back to Restrictions [page 9]

Multi-Cloud Foundation

Topic	Restriction
Service Channels	Service channels are supported only for RFC and for Kubernetes clusters. The service channel for Kubernetes uses TCP, which enables a variety of protocol-specific connections. For information, see Using Service Channels [page 1949].

Back to Restrictions [page 9]

Cloud Connector

Topic	Restriction
Scenarios	To learn in which system landscapes you can set up the Cloud Connector, see Extended Scenarios [page 1607].
Installation	To check all software and hardware restrictions for working with the Cloud Connector, see Prerequisites [page 1610].

Back to Restrictions [page 9]

Back to Content [page 4]

Related Information

Connectivity for the Multi-Cloud Foundation [page 11]
Connectivity Service [page 1308]
Destination Service [page 1347]
Cloud Connector [page 1602]
Connectivity Proxy for Kubernetes [page 2088]
Transparent Proxy for Kubernetes [page 2164]
Connectivity via Reverse Proxy [page 2360]
Connectivity Support [page 2362]
Submitting Improvement Requests [page 2365]

1.1 Connectivity for the Multi-Cloud Foundation

Consuming SAP BTP Connectivity for your application in the multi-cloud foundation: Overview.

> **Note**
>
> This documentation refers to SAP BTP, multi-cloud foundation. If you are looking for information about the Neo environment, see Connectivity for the Neo Environment.
>
> For more information on SAP BTP, multi-cloud foundation, see also What Is the Multi-Cloud Foundation?.

Getting Started [page 12]	Use SAP BTP Connectivity for your application in the multi-cloud foundation: available services, components, and use cases.
Concepts [page 16]	Find information on basic concepts of SAP BTP Connectivity.
Connectivity Scenarios [page 27]	Connections between SAP BTP applications and their target applications (inside or outsideSAP BTP) can be established in different ways - depending on the location of the target application and on the direction of communication.
What's New for Connectivity [page 142]	Find the latest features, enhancements and bug fixes for SAP BTP Connectivity .
Administration [page 1300]	Learn about Connectivty administration tools, user roles, monitoring, troubleshooting, and configuration backup for destinations.
Developing Applications [page 1306]	Find information on Connectivity concepts, scenarios, and components to develop your SAP BTP application.
Connectivity in the Kyma Environment [page 1307]	Find an overview of Connectivity components you can use for different purposes in the Kyma environment.

1.1.1 Getting Started

Use SAP BTP Connectivity for your application in the multi-cloud foundation: available services, components and use cases.

> ⓘ Note
>
> You can use SAP BTP Connectivity for the multi-cloud foundation and for the Neo environment. This documentation refers to SAP BTP, multi-cloud foundation. If you are looking for information about the Neo environment, see Connectivity for the Neo Environment.

Overview

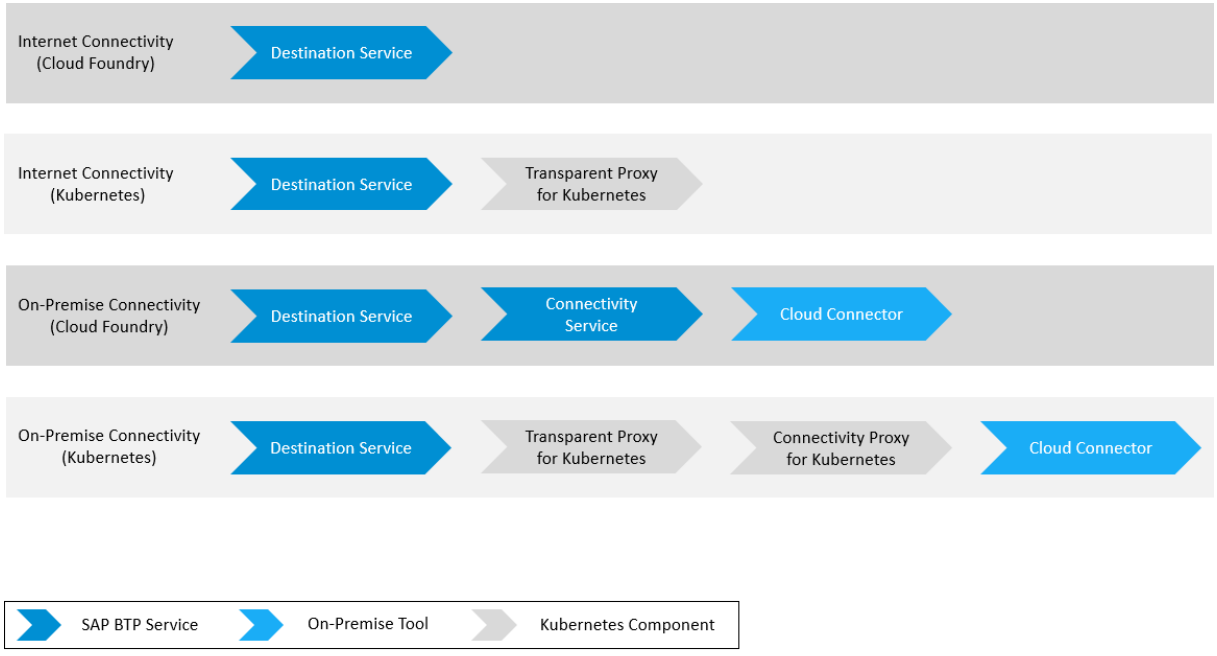
SAP BTP provides robust connectivity features that let you establish reliable communication channels, ensuring data flows smoothly and securely across different environments. This capability is essential for enterprise applications and typically complex, needing to consume or push data to a variety of sources or destinations, including systems that are accessible directly, hosted in public or private clouds, or located on customer premises. SAP BTP Connectivity empowers businesses to extend, innovate, and optimize their digital landscapes effectively.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Services and Components

Basic Use Cases

Hover over the elements for a description. Click an element for more information.



- Learn how to manage destinations and consume the Destination service in SAP BTP, multi-cloud foundation. [page 1347]
- Learn how to manage destinations and consume the Destination service in SAP BTP, multi-cloud foundation. [page 1347]
- Learn how to manage destinations and consume the Destination service in SAP BTP, multi-cloud foundation. [page 1347]
- Learn how to manage destinations and consume the Destination service in SAP BTP, multi-cloud foundation. [page 1347]
- Learn how to manage and consume the Connectivity service in SAP BTP, multi-cloud foundation. [page 1308]

- Use the transparent proxy for Kubernetes to connect workloads on a Kubernetes cluster to Internet and on-premise applications. [page 2164]
- Use the transparent proxy for Kubernetes to connect workloads on a Kubernetes cluster to Internet and on-premise applications. [page 2164]
- Use the connectivity proxy for Kubernetes to connect workloads on a Kubernetes cluster to on-premise systems. [page 2088]
- Learn more about the Cloud Connector: features, scenarios and setup. [page 1602]
- Learn more about the Cloud Connector: features, scenarios and setup. [page 1602]

Service/Component	Use Cases

Destination Service	The Destination service lets you find the information that is required to access a remote service or system from your cloud application.
	• For the connection to an on-premise system, you can use this service together with the Connectivity service.
	  • For Kubernetes environments, use the Destination service, the Connectivity Proxy and the Transparent Proxy for connections to the target system.
	• For the connection to any other Web application, you can use the Destination service without the Connectivity service.
	  • For Kubernetes environments, use the Destination service and the Transparent Proxy for automated and seamless connections to the target system.
	• Manages routing and authentication details, as well as custom scenario-specific parameters.
	• Performs authentication flows based on the configured details.

Connectivity Service	The Connectivity service offers a set of services and software components which allow SAP BTP applications to securely access remote services that run on premises. It: • Allows you to make connections to on-premise systems. • Provides, together with the Cloud Connector, a secure tunnel from your on-premise network to applications on SAP BTP, while you keep full control and auditability of what is exposed to the cloud. This way, no network ports need to be exposed on the on-premise side. • Supports various communication protocols towards the target system.
Connectivity Proxy for Kubernetes	The Connectivity Proxy is a Kubernetes component that connects workloads running on a Kubernetes cluster to on-premise systems, which are exposed via the Cloud Connector. The Connectivity Proxy must be paired to an SAP BTP region to grant access to the Cloud Connectors connected to that region. The SAP BTP domain model (subaccounts) is used to target a particular Cloud Connector.

Service/Component	Use Cases
Transparent Proxy for Kubernetes	The Transparent Proxy lightens the way your Kubernetes workloads connect to remote systems using SAP BTP destinations. It provides authentication, user principal propagation, automated Connectivity Proxy handshakes, and easy access to the destination target systems by exposing them as Kubernetes services and routing the traffic to them: • Directly for Internet-accessible solutions/applications/systems • Via the Connectivity Proxy and the Cloud Connector for on-premise systems
Cloud Connector	The Cloud Connector serves as a secure link between cloud applications and on-premise systems, allowing controlled access to them. It functions as a reverse invoke proxy within a secured network, providing fine-grained control over both accessible on-premise resources and cloud applications using it.

> **Note**
>
> - The Connectivity service and the Destination service are built-in SAP BTP services in the multi-cloud environment.
> - The Transparent Proxy and the Connectivity Proxy are available in two different versions: as modules (to be deployed) in SAP BTP, Kyma environment, and as standalone components.
> - The Cloud Connector is a standalone component to be installed and configured locally (in your on-premise network).

Recommendations

- Use the Destination service to store and manage your connection configuration data (including credentials, certificates, URL, headers, queries, etc.) at design time, and automate the OAuth2 token retrieval process at runtime for your application.
- Use the Transparent Proxy for Kubernetes to enhance developers' user experience and simplify the consumption of end systems by automating the retrieval of destinations from the Destination service, as well as integration with the Connectivity Proxy and authentication with end systems.
- Use the Cloud Connector and the Connectivity service to establish a secure connection between Cloud Foundry applications and on-premise systems.
- Use the Cloud Connector and the Connectivity Proxy (for Kubernetes environments) to establish a secure connection between Kubernetes applications and on-premise systems.

Scenarios

To find examples of typical Connectivity scenarios, see Connectivity Scenarios [page 27].

Related Information

Connectivity Service [page 1308]
Destination Service [page 1347]
Cloud Connector [page 1602]
Connectivity Proxy for Kubernetes [page 2088]
Transparent Proxy for Kubernetes [page 2164]

1.1.2 Concepts

Find information on basic concepts of SAP BTP Connectivity.

Hover over the elements for a description. Click an element for more information.

- Find an overview of the key Destination service entities used to set up connectivity for SAP BTP applications. [page 16]
- Using multitenancy for applications that require a connection to a remote service or on-premise application. [page 18]
- Find an overview of recommended security measures for SAP BTP Connectivity. [page 23]
- Improve resilience of your SAP BTP applications. [page 24]

1.1.2.1 Connectivity Entities

Find an overview of the key Destination service entities used to set up connectivity for SAP BTP applications.

> ### Note
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Destinations

An SAP BTP destination stores your connection configuration data (including credentials, certificates, URL, etc.) that is needed to address the corresponding cloud or on-premise target system or service.

For more information, see Managing Destinations [page 1349].

Destination Fragments

Destination fragments are objects used to override and extend destination properties through the Destination service REST API.

You can use destination fragments to override and/or extend destination properties as result of the “Find a destination” REST API request. These properties are key-value based objects which contain a name for identification and additional configurable properties.

For more information, see Manage Destination Fragments [page 1503].

Certificates

Certificates are standardized data sets used to authenticate certain properties of a person or object in a cryptographic procedure. Certificates are usually issued by a Certification Authority (CA).

A common standard are X.509 public-key certificates, which confirm the identity of the certificate owner as well as other properties of a public cryptographic key.

In the SAP BTP Connectivity context, certificates can be stored in a destination to authenticate users or applications calling a specific target system or service.

For more information, see Manage Destination Certificates [page 1504].

Trust

Scenarios involving user propagation from a multi-cloud application to other systems, to SAP BTP environments, or to another subaccount in the same SAP BTP environment require a trust configuration between the involved systems (sender and receiver).

This includes exchanging public keys and certificates between those systems and configuring trust between them.

For more information, see Manage Trust [page 1514].

1.1.2.2 Multitenancy in SAP BTP Connectivity

Find information about the connectivity-specific usage of multitenancy in SAP BTP, multi-cloud foundation.

You can use multitenancy for applications that require a connection to a remote service or on-premise application, specified by the consumer account.

> **Note**
>
> Multitenancy connectivity scenarios are fully applicable also for *cloud-to-on-premise* connectivity. The on-premise use cases described in this guide include connections to virtual private cloud (VPC) environments.

In a multitenancy scenario, the individual (consumer) tenants are based on the context of the used token to call the respective services.

Endpoint Configuration

Applications that require a connection to a remote service can use the Destination service to configure HTTP or RFC endpoints for the individual consumers or read the ones configured by the consumer. Endpoints on the cloud side (SAP BTP) are configured as destinations. On-premise destinations correspond to the backend systems exposed in the Cloud Connector.

Destination Levels

You can manage configurations (destinations, certificates, and so on) simultaneously on three levels: *subaccount*, *service instance*, and *subscription*. This means that it is possible to have one and the same destination configuration on more than one configuration level. For more information, see Managing Destinations [page 1349].

Here is how lookup is performed according to the level, when configured:

Level	Lookup
Subaccount	Looked up on subaccount level, no matter which destination service instance is used.
Service instance	Lookup via particular service instance (in provider or consumer subaccount associated with this service instance).
Subscription	Lookup via a particular service instance, with a token in the context of a subscriber subaccount (a different subscription level exists for each subscriber subaccount). For more information, see Accessing and Managing Destination Service Configurations on Subscription Level [page 22].

When the application accesses the destination at runtime using the "Find API", the Destination service does the following:

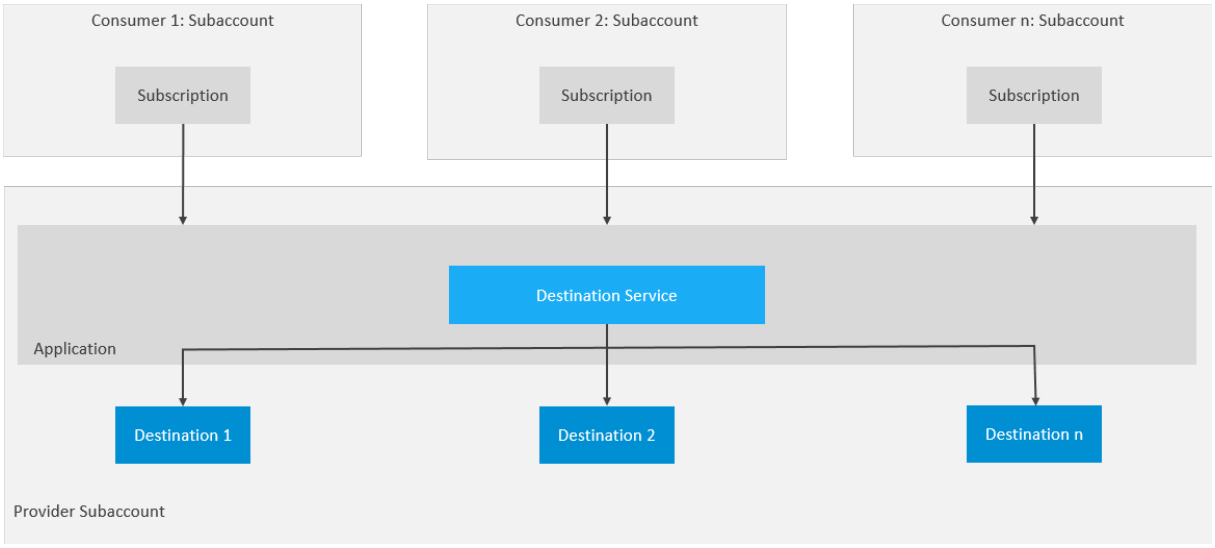
- For a destination associated with a **provider** subaccount:
  1. Checks if the destination is available on the *service instance* level. If there is no destination found, it
  2. Searches the destination on *subaccount* level.
- For a destination associated with a **consumer** subaccount:
  1. Checks if the destination is available on the *subscription* level. If there is no destination found, it
  2. Searches the destination on *subaccount* level.

For the "Consume API" [page 1569], the level should be explicitly specified.

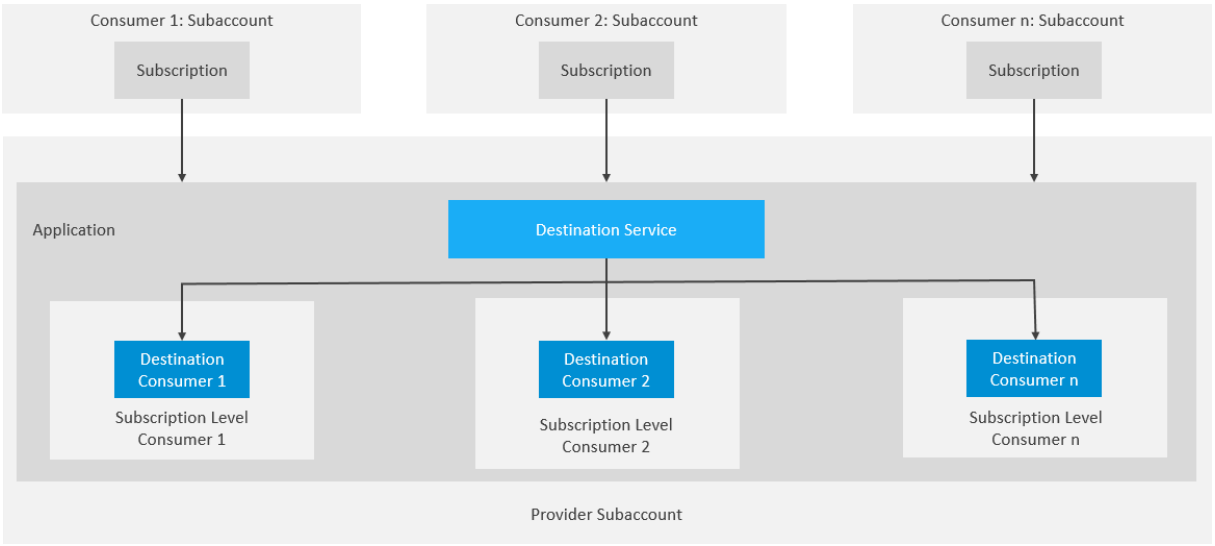
Back to Top [page 18]

Provider-Managed Configurations

Generic configurations (not specific for the consumer subaccounts)

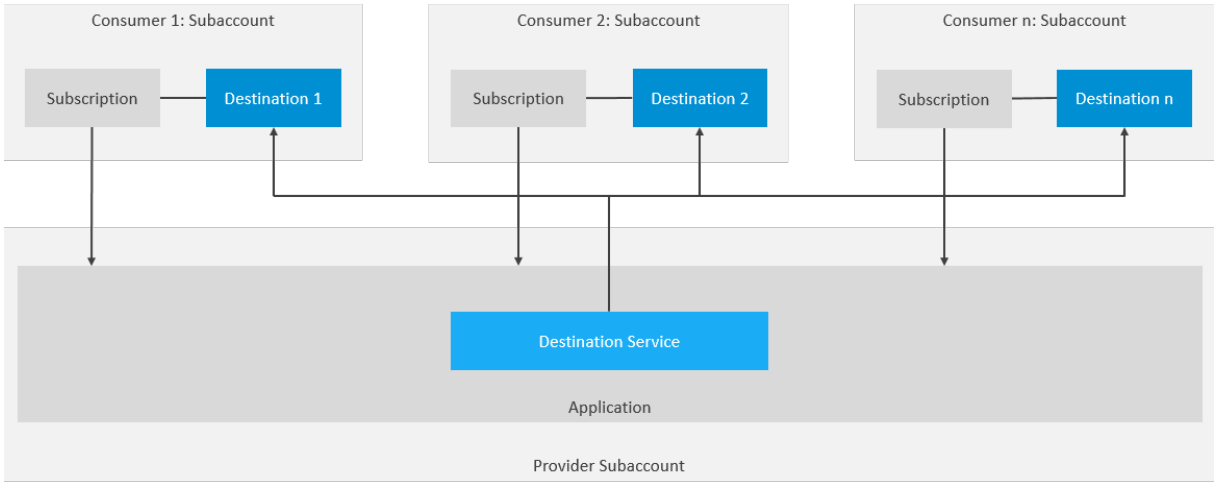


Consumer-specific configurations



Back to Top [page 18]

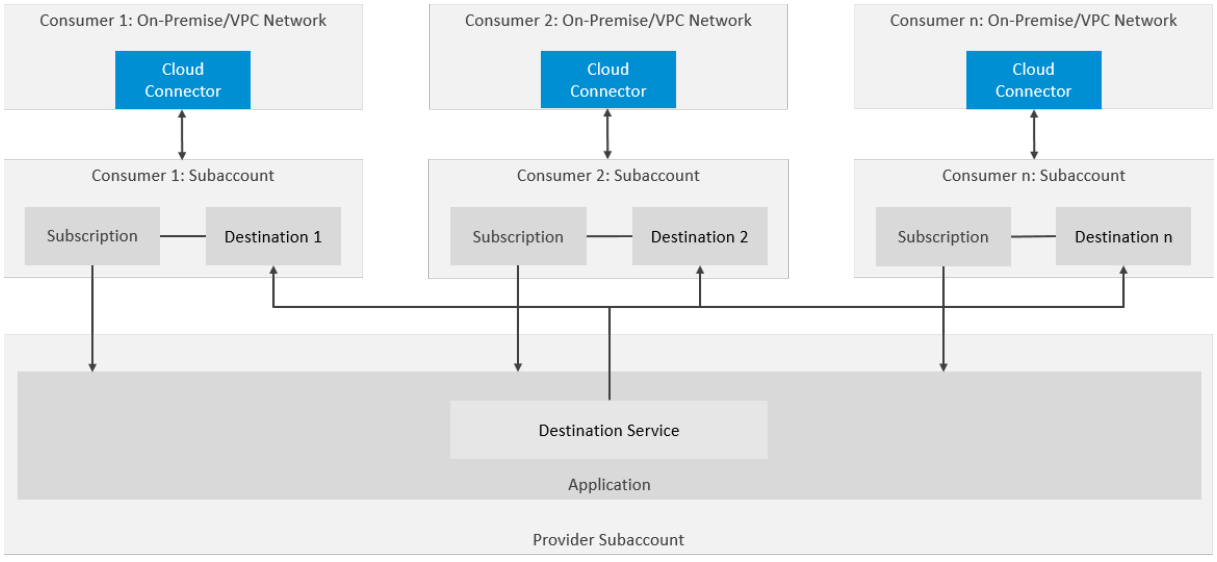
Consumer-Managed Configurations



> ### Note
>
> In this scenario, consumer configurations on subaccount level are shared with the provider application.

Back to Top [page 18]

On-Premise Connection



Back to Top [page 18]

Related Information

Developing Multitenant Applications in the Cloud Foundry Environment
Accessing and Managing Destination Service Configurations on Subscription Level [page 22]

1.1.2.2.1 Accessing and Managing Destination Service Configurations on Subscription Level

Allow subscriber subaccount administrators of your SaaS application to manage Destination service configurations on subscription level.

By default, only the SaaS application itself can manage Destination service configurations (destinations, certificates, and so on) on any of the subscription levels that are associated with its service instance. This is done by fetching an access token using the instance credentials in the context of the desired subscriber account and calling the Destination Service REST API [page 1497].

However, every SaaS application provider can decide to allow users with specific roles in the context of the consumer subaccount to manage configurations on subscription level. This makes it easy for SaaS application consumers to share configurations with the application.

Enablement Process for the SaaS Application

The enablement is done via properties of the Destination service instance that the SaaS application configures as a dependency. During creation or update, using a config JSON, a specification for each configuration type can be provided. Within this specification, for each operation a list of scopes needed to perform the operation can be provided.

For more information, see Use a Config.JSON to Create or Update a Destination Service Instance [page 1525].

To perform these operations, we recommended that you use dedicated scopes in the `xs-app.json` of the SaaS application, assigned to dedicated roles, but you can associate pre-existing scopes with the operations as well.

Subscription Level Zones

The subscription level is divided into two virtual zones: one that is shared with the consumer subaccount and one that is strictly private to the SaaS application.

Only the shared zone is shown to the consumer subaccount and anything managed by the consumer is located in this zone.

When an SaaS application manages configurations via the REST API, by default the created configuration will be placed in the private zone. However, there is also an option to create it in the shared zone, allowing for the

consumer to read and/or modify it. This is done via the `$metadata.shared_with_subscriber` property in the request body of the POST/PUT API. If set to **`true`**, the configuration will be available via the subscription level of the UIs.

When the SaaS application fetches configurations, the lookup is done in both zones.

> ⚠ Restriction
>
> It is not allowed to have configurations with the same name in both zones.

Subscription Level Access for Consumer Subaccount Users

The application will become visible to all consumer subaccounts as an available subscription level. Users will see what operations they can perform (*read*, *read & write*, or *no access*). This is based on their assigned roles and whether they match the scopes defined during enablement.

For more information, see Access the Destinations Editor [page 1473].

> ⓘ Note
>
> It may take some time after enablement for the consumer subaccounts to see the changes taking effect. This is due to caching on the Destination service side.

1.1.2.3 Security

Find an overview of recommended security measures for SAP BTP Connectivity.

Topic	More Information
Enable single sign-on by forwarding the identity of cloud users to a remote system or service.	Principal Propagation [page 30]
Set up and run the Cloud Connector according to the highest security standards.	Security Guidelines [page 2065]
Choose between different options to encrypt your data when using the Destination service.	Using Encryption Keys for the Destination Service [page 24]

More Information

SAP BTP Security Recommendations collects information that lets you secure the configuration and operation of SAP BTP services in your landscape.

1.1.2.3.1 Using Encryption Keys for the Destination Service

Choose between different options to encrypt your data when using the Destination service.

> **⚠ Restriction**
>
> This feature is supported only in regions providing the *SAP Credential Store*.

When using the Destination service, there are two options to encrypt your data:

- **Customer-specific encryption keys (CSEK)**:
  This option encrypts your data with individual (customer-specific) encryption keys. The keys, however, are still managed by SAP. This is the default setting, and you do not need to do anything to enable it.
  Customer-specific encryption is a prerequisite for using *customer-managed keys (CMK)*.
- **Customer-managed keys (CMK)**: This option applies only if you want to bring your own key for encryption. When you choose this option, data are double-encrypted: Using the default CSEK encryption, and in addition using encryption with your own key.
  To use this option, you must provision your own key management service. For this purpose, SAP offers the *Data Custodian Key Management service*. Using this service, you can control your own keys (grant, revoke, rotate, and delete keys).
  For more information, see What is Key Management Service?

When using CSEK/CMK, all destination properties (both main and additional) are encrypted, except for (destination) `Name` and `FragmentName`.

> **ⓘ Note**
>
> Destination properties are encrypted regardless of their sensitivity.

> **⚠ Restriction**
>
> This service does not encrypt the following data:
>
> - (Destination) `Name` and `FragmentName`
> - All **metadata** like creation and update dates, tools that created the destination, owner subaccount, and so on

1.1.2.4 Resilience Recommendations

Improve resilience of your SAP BTP applications.

While SAP strives to ensure the highest possible availability of the provided services, true resilience actually is a two-way collaboration between client and server. Thus, it is important to implement any applications using

the Connectivity and/or Destination services (or any other services for that matter) in a resilient way. This page gives suggestions on measures to take in order to endure short disruptions, network issues, slowdowns or other abnormal situations that might arise. By doing this, these application can withstand such disruptions, ensuring business continuity in the face of underlying issues in the platform.

> **Note**
>
> When using client libraries like the BTP security library , the Cloud SDK , and so on, many of these recommendations may already be included. However, we recommend that you double-check these features, as they might require additional configuration.

Caching

Caching is an important pillar of resilience. It is the act of storing data (acquired from an external resource or as a result of a heavy computation) for future use. Caching operations must be done within a reasonable timespan to avoid the risk of data becoming invalid or outdated. So, caching lets you reduce the number of risky or heavy operations that go over the network towards an external resource. By doing this, the risk of failure is reduced, and caching additionally improves the performance of the application.

> **Note**
>
> Be careful what kind of data you cache and for how long. It is especially important to consider the handling of sensitive data (personal data, security objects, and so on).

> **Caution**
>
> Make sure you limit the size of the caches to avoid memory issues. There are different options when the maximum number of entities in the cache is reached. For example, you can drop the oldest entity from the cache in favor of the new one.

Token Caching

Caching access tokens to the services is highly recommended for the period of their validity. Access tokens to the service include the timestamp on which the token is no longer valid (as per JWT specification). Thus the tokens can be reused at any time before said timestamp. Keep in mind that tokens are issued for the specific clients and in the context of the specific tenant. Thus this should be taken into account when designing the caching mechanism.

Destination/Certificate Caching

The entities of the Destination service (be it destination configurations or certificate configuration) can also be cached. This includes both responses for single entities, as well as the list of entities responses. The drawback of this is that any changes in the configurations will not become immediately available to the application, thus a reasonable caching time (3-5 minutes is usually a good choice) must be set.

If you do not expect time critical changes in the destination configuration, you can set the caching time even higher (for example, an hour). Also, we recommend that you refresh the cache on-demand instead of calling the Destination service every 3-5 minutes.

By caching applications, they can withstand temporary issues with the service or the network. We also recommend that in cases where the cache has expired but retrieval of the updated entities does not succeed to still use the cached value as a further resilience measure.

Retry Logic

When requests (whether it is the token call, the service call or the business call) fail, it is always a good idea to retry the call over a reasonable time. It also makes sense to have some delay between retries, for example, at least 100 milliseconds. You can even make this delay increase with each subsequent retry. By doing this, you can handle short intermittent issues without having business impact.

Pagination

The APIs of the Destination service that return multiple entities support pagination. We highly recommended that you make use of pagination if you use these APIs and have a high number of entities (more than 100 entities). By doing this, you ease the pressure on the service but also make processing lighter on client side as the list can be handled in batches. See more about pagination in the documentation of the Destination Service REST API [page 1497].

Timeouts

Connect and read timeouts help you keep your application resources from getting stuck in abnormally long processing. In some cases, one attempt to get data might get stuck, but abandoning it and trying again might immediately succeed.

We recommend you use appropriate *connect* and *read* timeouts when communicating with the services. The *connect* timeout is recommended to be on the low end - a couple of seconds. The *read* timeout would depend on the semantic of the call being made. A call to the Destination service REST API would not require a high read timeout. Around half a minute would be sufficient. The same is true for token retrieval calls. For business calls, including calls to on-premise systems via the Connectivity service, the timeout would be based on the type of the call and is heavily scenario-specific.

Circuit Breaker

The circuit-breaker architecture pattern can be a powerful tool for handling a misbehaving dependency. It reduces the effort done by your application when communicating to a service which is identified as not working or unstable, while waiting for the dependency to come back online.

For more information on this pattern, see https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker .

Cloud Connector High Availability

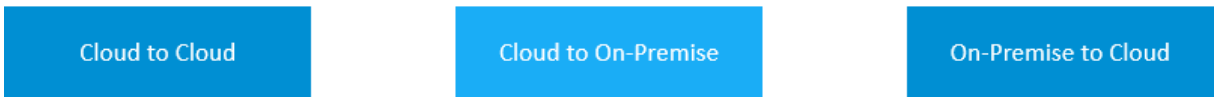
See High Availability Setup [page 1995].

1.1.3 Connectivity Scenarios

Basic connectivity scenarios in SAP BTP, multi-cloud foundation.

Connections between SAP BTP applications and their target applications (inside or outside SAP BTP) can be established in different ways - depending on the location of the target application and on the direction of communication.

The communication can be classified into three basic scenarios:



- In the cloud-to-cloud scenario, an SAP BTP application connects to a target application running on a cloud-based infrastructure. The target can be an SAP BTP application or any other cloud-based application. [page 28]
- In this scenario, you perform calls from your SAP BTP application to a target application located in your local on-premise network. [page 29]
- The on-premise-to-cloud scenario corresponds to the cloud-to-on-premise connection, but the communication direction is different: You perform a call from an on-premise application to SAP BTP. [page 29]

Hover over the elements for a description. Click an element for more information.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

In this section, we describe the key elements and procedures of these scenarios, indicating the relevant parts in documentation for more information.

Scenario: Cloud to Cloud

In the *cloud-to-cloud* scenario, an SAP BTP application connects to a target application running on a cloud-based infrastructure (platform, server). The target can be an SAP BTP application or any other cloud-based application.

For this scenario, you must configure at least one destination containing the essential communication data, such as target host (URL), communication protocol, and authentication type.

> **Note**
>
> A commonly used authentication method is *principal propagation* (also known as *user propagation*), which lets you propagate the identity of the logged-on user to the target application.
>
> For more information on authentication methods, see Configuring Authentication [page 1351].

For more information on these procedures, see:

- Managing Destinations [page 1349]
- Principal Propagation [page 30]
  - Cloud to Cloud [page 32]
    - Manage Trust [page 1514]

Some common *principal propagation* scenarios for communication between an SAP BTP multi-cloud (Cloud Foundry) application and different target applications are described in detail here:

- JWT Bearer Flow between Two Applications in SAP Cloud Identity Services (App to App) [page 34]
- JWT Bearer Flow from the Multi-Cloud Environment to SuccessFactors [page 36]
- SAML Bearer Assertion Flow from the Cloud Foundry Environment to SAP S/4HANA Cloud [page 52]
- SAML Bearer Assertion Flow from the Cloud Foundry Environment to SAP SuccessFactors [page 60]
- SAML Bearer Assertion Flow between Cloud Foundry Applications [page 37]
- SAML Bearer Assertion Flow from the Cloud Foundry Environment to the Neo Environment [page 45]

Scenario: Cloud to On-Premise

In this scenario, you want to perform calls from your SAP BTP application to a target application located in your local *on-premise* network. While the basic procedures are similar to the *cloud-to-cloud* scenario (including some configuration that is specific to on-premise communication), this connection type requires the Cloud Connector as additional component, to be installed and managed within your local on-premise network. The Cloud Connector provides a secure tunnel for communication between your local apps and SAP BTP.

> ### Note
>
> A commonly used authentication method is *principal propagation* (also known as *user propagation*), which lets you propagate the identity of the logged-on user to the target application.
>
> For more information on authentication methods, see Configuring Authentication [page 1351].

Find more information about the basic procedures for cloud-to-on-premise communication in these sections:

- Cloud Connector [page 1602]
- Managing Destinations [page 1349]
- Principal Propagation [page 30]
  - Cloud to On-Premise [page 30]

Scenario: On-Premise to Cloud

The *on-premise-to-cloud* scenario corresponds to the *cloud-to-on-premise* connection in terms of the app location (one app/service is residing on SAP BTP, and the connected app in your local network). However, the communication direction is different: In this case, you want to perform a call from an on-premise application to SAP BTP, for example, to

- Connect your local database tools to an SAP HANA database residing on SAP BTP.

Other examples for on-premise-to-cloud connections are:

- Calls from your on-premise system to SAP BTP ABAP environment using the RFC protocol
- Connections to a service running in an SAP BTP Kubernetes cluster

On-premise-to-cloud communication requires a special setup that can be configured using the so-called Cloud Connector *service channels*. These service channels provide access from your local network to specific services on SAP BTP, or to an ABAP Cloud system, while the called services are not exposed to direct access from the Internet.

To learn more about specific use cases and procedures for the on-premise-to-cloud scenario, see:

- Cloud Connector [page 1602]
  - Using Service Channels [page 1949]

1.1.3.1 Principal Propagation

Enable single sign-on (SSO) by forwarding the identity of cloud users to a remote system or service.

The Connectivity and Destination services let you forward the identity of a cloud user to a remote system. This process is called principal propagation (also known as *user propagation* or *user principal propagation*). It uses a JSON Web token (JWT) as exchange format for the user information.

Two scenarios are supported: Cloud to on-premise (using the Connectivity service) and cloud to cloud (using the Destination service).

- Cloud to On-Premise [page 30]: The user is propagated from a cloud application to an on-premise system using a destination configuration with authentication type `PrincipalPropagation`.

  > ### Note
  >
  > This scenario requires the Cloud Connector to connect to your on-premise system.

  > ### Note
  >
  > The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

- Cloud to Cloud [page 32]: The user is propagated from a cloud application to another remote (cloud) system using a destination configuration with authentication types `OAuth2SAMLBearerAssertion` or `OAuth2JWTBearer`.

For more information on setting up destinations, see:

- Create HTTP Destinations [page 1476]
- Create RFC Destinations [page 1478]

1.1.3.1.1 Cloud to On-Premise

Forward the identity of cloud users from SAP BTP to on-premise systems using principal propagation.

> ### Note
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Concept

The Connectivity service lets you connect your cloud applications to on-premise systems through the Cloud Connector and forward the identity of a cloud user. This process is called principal propagation (also known as

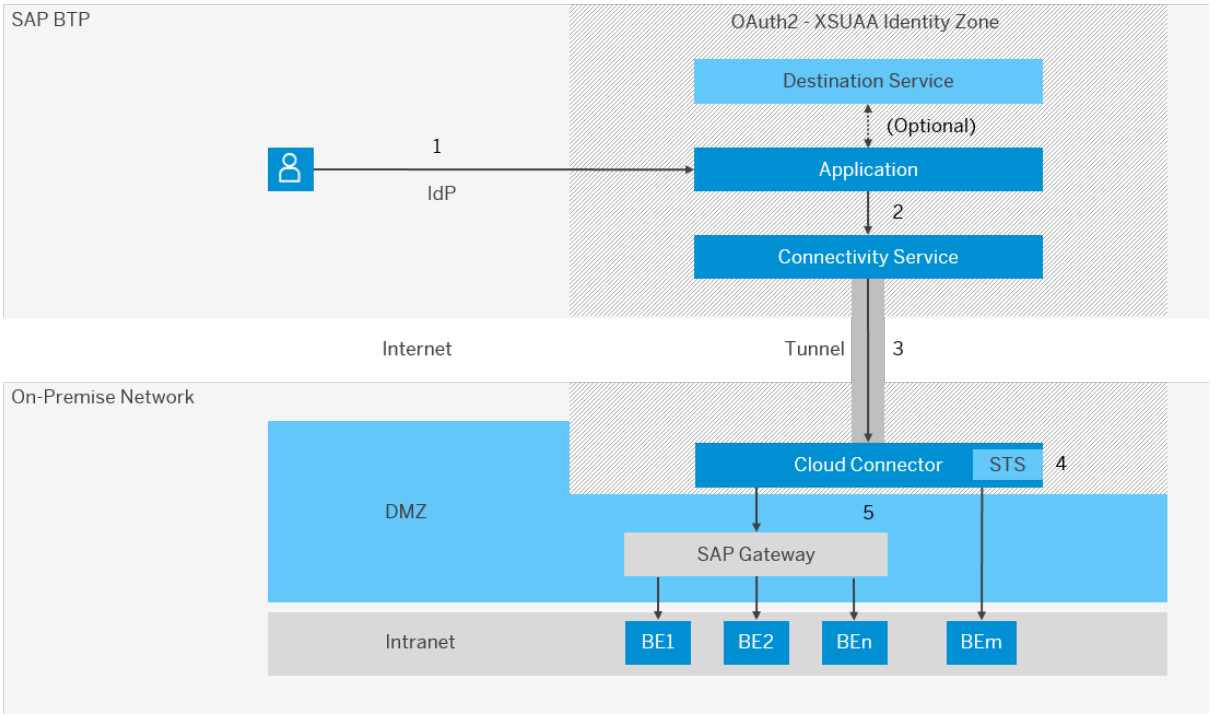
user propagation). The JSON Web token (JWT) representing the cloud user identity is forwarded to the Cloud Connector, which verifies it, and propagates the user identity via either an X.509 certificate or Kerberos.

Optionally, you can configure and use a destination configuration by setting the authentication type as `PrincipalPropagation`. For more information, see Managing Destinations [page 1349].

> **Note**
>
> This scenario is only applicable if the on-premise system is exposed to the cloud via the Cloud Connector.

You can configure principal propagation for HTTP or RFC communication.

Scenario: Cloud to On-Premise



1. A user logs in to the cloud application. Its identity is established by an identity provider (this can be the default IdP for the subaccount or another trusted IdP).
2. The cloud application then uses a user exchange token (or a designated secondary header) to propagate the user to the Connectivity service. See also Configure Principal Propagation via User Exchange Token [page 1327].
   - Optionally, the application may use the Destination service to externalize the connection configuration that points to the target on-premise system. See also Consuming the Destination Service [page 1550].
   - If you are using RFC as communication protocol with the *SAP Java Buildpack*, this step is already done by the Java Connector (JCo).
3. The Connectivity service forwards the JWT (that represents the user) to the Cloud Connector.

4. The Cloud Connector receives the JWT, verifies it, extracts the attributes, and uses its STS (security token service) component to issue a new token (for example, an X.509 certificate) with the same or similar attributes to assert the identity to the backend (BE1-BEm). The Cloud Connector and the cloud application share the same trust settings, see Set Up Trust [page 1702].
5. The Cloud Connector sends the new token (for example, an X.509 certificate) to the backend system.

Configuration: Cloud to On-Premise

Task Type	Task
Operator	Configuring Principal Propagation [page 1702] (Cloud Connector)
Developer	Use cases: • HTTP communication: Configure Principal Propagation via User Exchange Token [page 1327] (Connectivity service) • RFC communication: Configure Principal Propagation for RFC [page 123]

Related Information

Principal Propagation from SAP BTP Applications to On-Premise Systems Using Corporate IdP Tokens

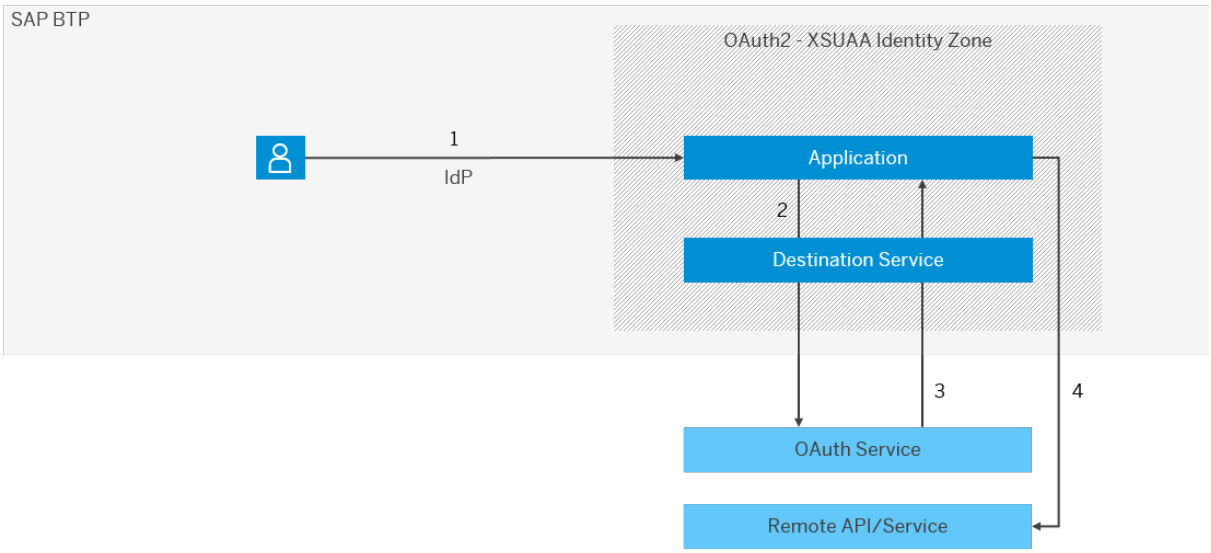
1.1.3.1.2 Cloud to Cloud

Forward the identity of cloud users from SAP BTP to remote systems on the Internet, enabling single sign-on (SSO).

Concept

The Destination service provides a secure way of forwarding the identity of a cloud user to another remote system or service using a destination configuration with authentication type `OAuth2SAMLBearerAssertion`. This enables the cloud application to consume OAuth-protected APIs exposed by the target remote system.

Scenario: Cloud to Cloud



1. A user logs in to the cloud application. Its identity is established by an identity provider (this can be the default IdP for the subaccount or another trusted IdP).
2. When the application retrieves an `OAuthSAMLBearer` destination, the user is made available to the Destination Service by means of a user exchange JWT. The service then wraps the user identity in a SAML assertion, signs it with the subaccount's private key and sends it to the specified OAuth token service.
3. The OAuth token service accepts the SAML assertion and returns an OAuth access token. In turn, the Destination service returns both the destination and the access token to the requesting application.
4. The application uses the destination properties and the access token to consume the remote API.

You can set up user propagation for connections to applications in different cloud systems or environments.

Configuration: Cloud to Cloud

Task Type	Task
Operator	Manage Trust [page 1514]

Task Type	Task
Operator and/or Developer	User Propagation via SAML 2.0 Bearer Assertion Flow [page 1558] (Destination service)

Use Cases: Cloud to Cloud

- SAML Bearer Assertion Flow from the Cloud Foundry Environment to SAP S/4HANA Cloud [page 52]
- SAML Bearer Assertion Flow from the Cloud Foundry Environment to SAP SuccessFactors [page 60]
- SAML Bearer Assertion Flow between Cloud Foundry Applications [page 37]
- SAML Bearer Assertion Flow from the Cloud Foundry Environment to the Neo Environment [page 45]

Related Information

Setting up Trust Between Systems (for SAML Bearer Assertion Flows) [page 66]

1.1.3.1.2.1 JWT Bearer Flow between Two Applications in SAP Cloud Identity Services (App to App)

Configure principal propagation between two applications in *SAP Cloud Identity Services*.

Scenario

You have an application protected with *SAP Cloud Identity Services*. You perform user-based login flows (for example, authorization code grant) to have a named user authenticated in the application, and the application then works on behalf of that user. You want to call another application, protected with *SAP Cloud Identity Services* as well, maintaining the logged-in user context (principal propagation). The called application will then perform its work on behalf of the same user without requiring a separate login.

This guide shows how to model and delegate to the Destination service the operation described in Consume an API from a Provider Application (example *Principal Propagation* in section *Procedure*).

Prerequisites

The integration between applications is established and relevant parameters are known, see Consume an API from a Provider Application (section *Prerequisites*).

Concept Overview

The concept is described in Consuming APIs from Other Applications .

Configuring the Destination

The interaction is modeled as an *OAuth JWT Bearer Authentication* destination.

For detailed information on specific flavors (like using mTLS instead of a client secret for the token fetching), see OAuth JWT Bearer Authentication [page 1395].

Sample Code

```
{
    "Name": "app2app",
    "Type": "HTTP",
    "Description": "",
    "URL": "https://<app2 url>",
    "ProxyType": "Internet",
    "Authentication": "OAuth2JWTBearer",
    "clientId": "<app1 client id>",
    "clientSecret": "<app1 client secret>",
    "tokenServiceURL": "https://<SAP Cloud Identity Services tenant url>/
oauth2/token",
    "tokenServiceURLType": "Dedicated",
    "tokenService.body.resource": "<app2 reference>"
}
```

Where:

- `<app2 url>` is the URL of the target application (the provider application). This is what the first application will call with the resulting token from the exchange to perform its request on behalf of the user.
- `<app1 client id>` is the client ID of the first application (the one making the call / the consumer application).
- `<app1 client secret>` is the client secret of the first application (the one making the call / the consumer application).
- `<SAP Cloud Identity Services tenant url>` is the host of the *SAP Cloud Identity Services* tenant. The operations are performed in the context of this host.
- `<app2 reference>` is used to identify the dependency (the target application / the provider application) consumed by the first application (the one making the call / the consumer application). It can be:

- `urn:sap:identity:application:provider:name:<Dependency_name>`, where `<Dependency name>` is the name that the tenant administrator configures for the dependency between the consumer application and the provider application.
- `urn:sap:identity:application:provider:clientid:<client_id>`, where `<client_id>` is client ID of the provider application.

Consuming the Destination and Executing the Scenario

To consume the created destination, use the procedure described in Exchanging User JWTs via OAuth2JWTBearer Destinations [page 1579].

1.1.3.1.2.2 JWT Bearer Flow from the Multi-Cloud Environment to SuccessFactors

Configure *principal propagation via OAuth JWT Bearer authentication* for connections from SAP BTP to SuccessFactors.

SuccessFactors supports OIDC for authentication. This includes principal propagation, implemented via OAuth JWT Bearer. This flow should be used when you need to propagate the identity of the logged in user towards SuccessFactors.

Prerequisites

Follow the SuccessFactors guide to complete the OIDC setup: Register Your Own Application to Communicate with SAP SuccessFactors HCM Suite with OpenID Connect. Perform the options for technical user access.

Configuring the Destination

The interaction is modeled as an *OAuth JWT Bearer Authentication* destination.

For more information on details about specific flavors (like using mTLS instead of a client secret for the token fetching), see OAuth JWT Bearer Authentication [page 1395].

Sample Code

```
{
    "Name": "sfsftechuser",
```

```
    "Type": "HTTP",
    "Description": "",
    "URL": "https://<sfsf url>",
    "ProxyType": "Internet",
    "Authentication": "OAuth2JWTBearer",
    "clientId": "<oidc client id>",
    "clientSecret": "<oidc client secret>",
    "tokenServiceURL": "https://<SAP Cloud Identity Services tenant url>/
oauth2/token",
    "tokenServiceURLType": "Dedicated",
    "tokenService.body.resource":
"urn:sap:identity:application:provider:name:<sfsf dependency name>"
}
```

Where:

- `<sfsf url>` is the URL of the SuccessFactors system.
- `<oidc client id>` is the client ID of the OIDC application, created during the setup.
- `<oidc client secret>` is the client secret of the OIDC application, created during the setup.
- `<SAP Cloud Identity Services tenant url>` is the host of the *SAP Cloud Identity services* tenant. The operations are performed in the context of this host.
- `<sfsf dependency name>` is the name of the dependency created from the OIDC application towards SuccessFactors during the setup phase.

Consuming the Destination and Executing the Scenario

Use the Find a Destination REST API [page 1562] or the Consume REST API [page 1569] to fetch the destination, including the access token that the Destination service will fetch for you.

1.1.3.1.2.3 SAML Bearer Assertion Flow between Cloud Foundry Applications

Propagate the identity of a user between Cloud Foundry applications that are located in different subaccounts or regions.

Steps

Scenario [page 38]

Prerequisites [page 38]

Concept [page 38]

Procedure [page 40]

1. Assemble IdP Metadata for Subaccount 1 [page 40]
2. Establish Trust between Subaccount 1 and Subaccount 2 [page 42]
3. Create an OAuthSAMLBearerAssertion Destination for Application 1 [page 43]
4. Consume the Destination and Execute the Scenario [page 45]

Scenario

- You have deployed an application in a Cloud Foundry environment (**application 1**).
- You want to call another Cloud Foundry application (**application 2**) in a different subaccount, in the same or another region.
- You want to propagate the identity of the user that is logged in to application 1, to application 2.

Back to Steps [page 37]

Prerequisites

- You have two applications (application 1 and application 2) deployed in Cloud Foundry spaces in different subaccounts in the same region or even in different regions.
- You have an instance of the Destination service bound to application 1.
- You have a user JWT (JSON Web Token) in application 1 where the call to application 2 is performed.

Back to Steps [page 37]

Concept

The identity of a user logged in to application 1 is established by an identity provider (IdP) of the respective subaccount (**subaccount 1**).

> **Note**
>
> You can use the default IdP for the Cloud Foundry subaccount or a custom-configured IdP.

When the application retrieves an OAuthSAMLBearer destination, the user is made available to the Cloud Foundry Destination service by means of a *user exchange* JWT. See also User Propagation via SAML 2.0 Bearer Assertion Flow [page 1558].

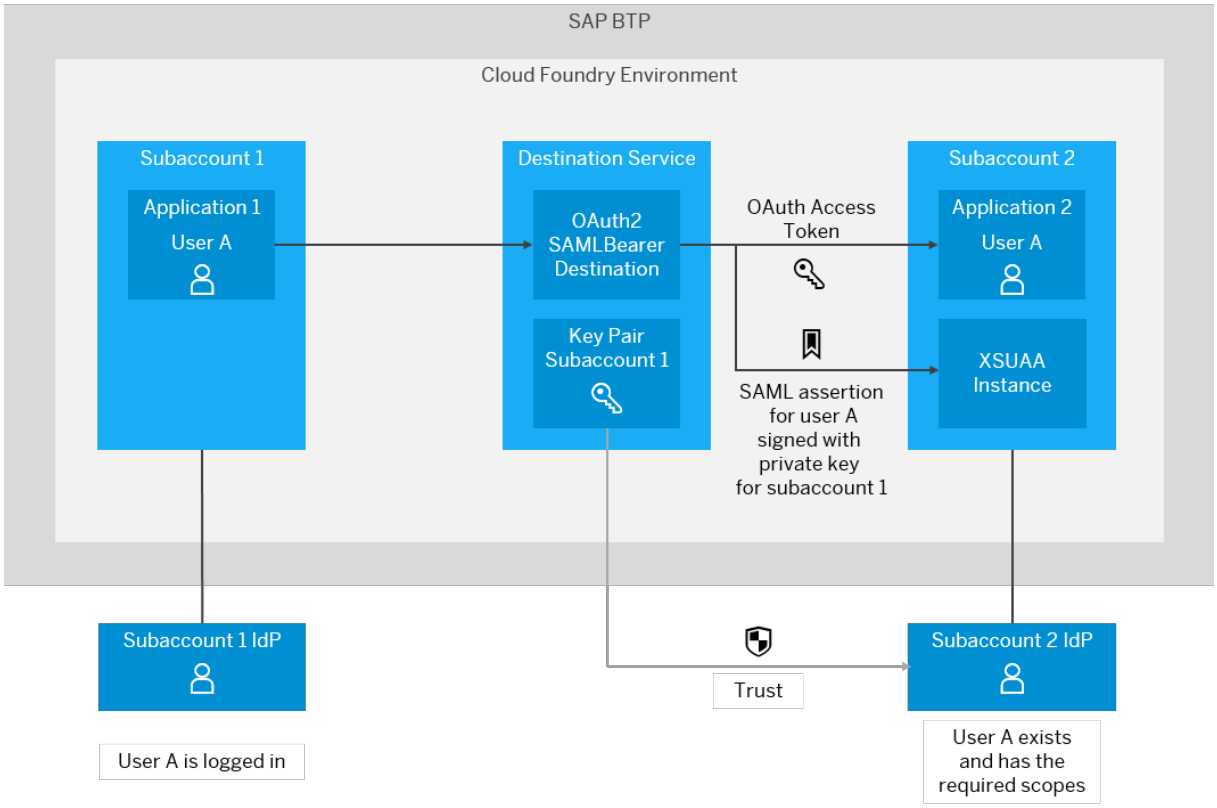
The service then wraps the user identity in a SAML assertion, signs it with subaccount 1's private key (which is part of the special key pair for the subaccount, maintained by the Destination service) and sends it to the authentication endpoint of **subaccount 2**, which hosts application 2.

To make the authentication endpoint accept the SAML assertion and return an access token, you must set up a trust relationship between the two subaccounts, by using subaccount 1's public key. You can achieve this by assembling the SAML IdP metadata, using subaccount 1's public key and setting up a new trust configuration for subaccount 2, which is based on that metadata.

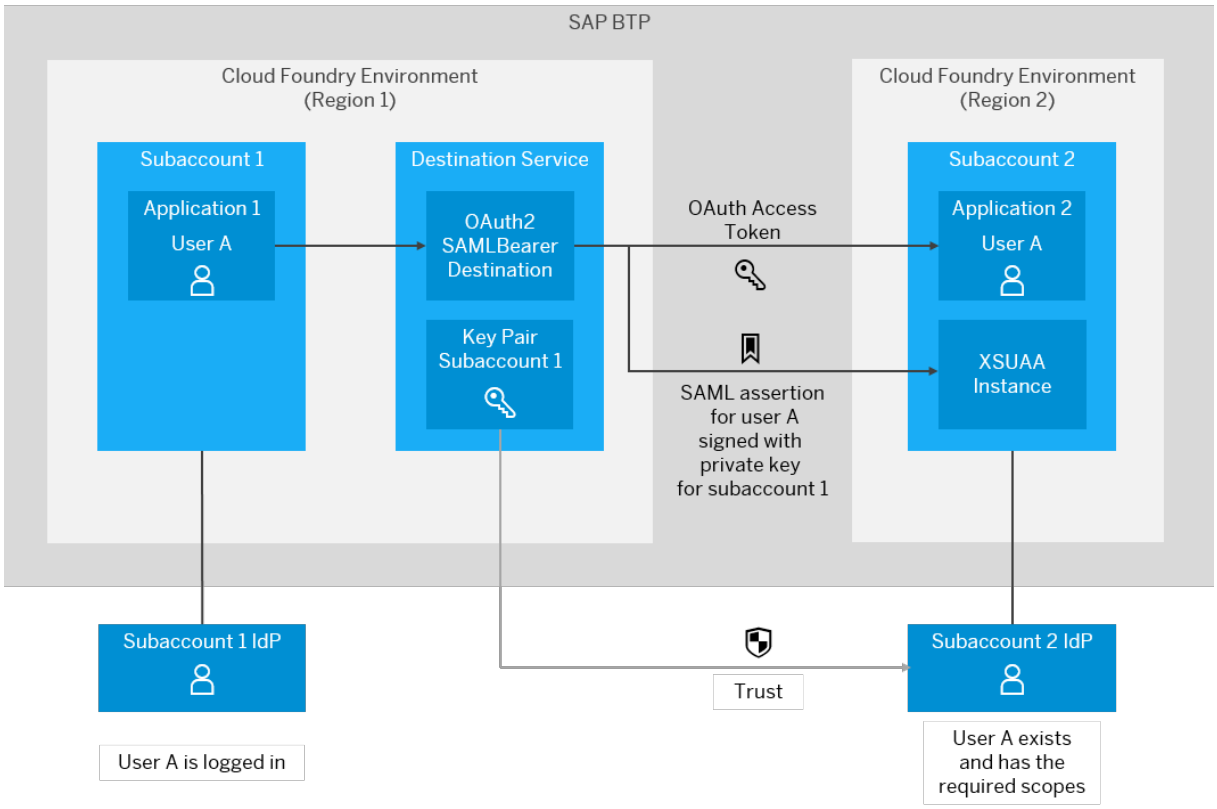
This way, users propagated from application 1 can be verified by subaccount 2's IdP before granting them access tokens with their respective scopes in the context of subaccount 2.

The authentication endpoint accepts the SAML assertion and returns an OAuth access token. In turn, the Destination service returns both the destination configuration and the access token to the requesting application (application 1). Application 1 then uses the destination properties and the access token to call application 2.

Option 1 - Setting up Trust between Subaccounts in the Same Region



Option 2 - Setting up Trust between Subaccounts in Different Regions



Back to Steps [page 37]

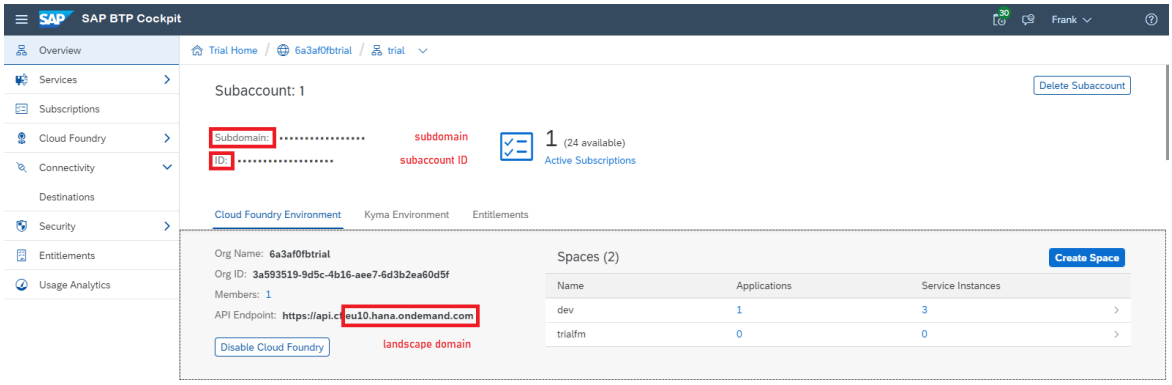
Procedure

Assemble IdP Metadata for Subaccount 1

1. Download the X.509 certificate of subaccount 1. For instructions, see Manage Trust [page 1514]. The content of the file is shown as:

   ```
   -----BEGIN CERTIFICATE-----<content>-----END CERTIFICATE-----
   ```

   Below, we refer to the value of <content> as `${S1_CERTIFICATE}`.
2. In the cockpit, navigate to the overview page of subaccount 1. For details, see Navigate in the Cockpit. Here you can see the landscape domain, subaccount ID and subdomain. Below, we refer to the landscape domain as `${S1_LANDSCAPE_DOMAIN}`, to the subaccount ID as `${S1_SUBACCOUNT_ID}` and to the subdomain as `${S1_SUBDOMAIN}`.



3. In your browser, call `https://${S1_SUBDOMAIN}.authentication.${S1_LANDSCAPE_DOMAIN}/saml/metadata` and download the XML file. Within the XML file you can find the following structure:

> ### Note
>
> The value for `${S1_LANDSCAPE_DOMAIN}` must contain the *main* landscape only. For example, if the landscape is *cf.***us10-001***.hana.ondemand.com*, the URL would look like this: *https://${S1_SUBDOMAIN}.authentication.***us10***.hana.ondenmand.com/saml/metadata*.

> ### Sample Code
>
> ```
> <?xml version="1.0" encoding="UTF-8"?>
> ...
> <md:AssertionConsumerService
> Binding="urn:oasis:names:tc:SAML:2.0:bindings:URI"
> Location="https://${S1_SUBDOMAIN}.authentication.${S1_LANDSCAPE_DOMAIN}/
> oauth/token/alias/<alias>" index="1"/>
> ...
> ```

Below, we refer to the value of <alias> as `${S1_ALIAS}`.

4. Assemble the new IdP metadata for subaccount 1 by replacing the `${...}` placeholders in the following template with the values determined in the previous steps:

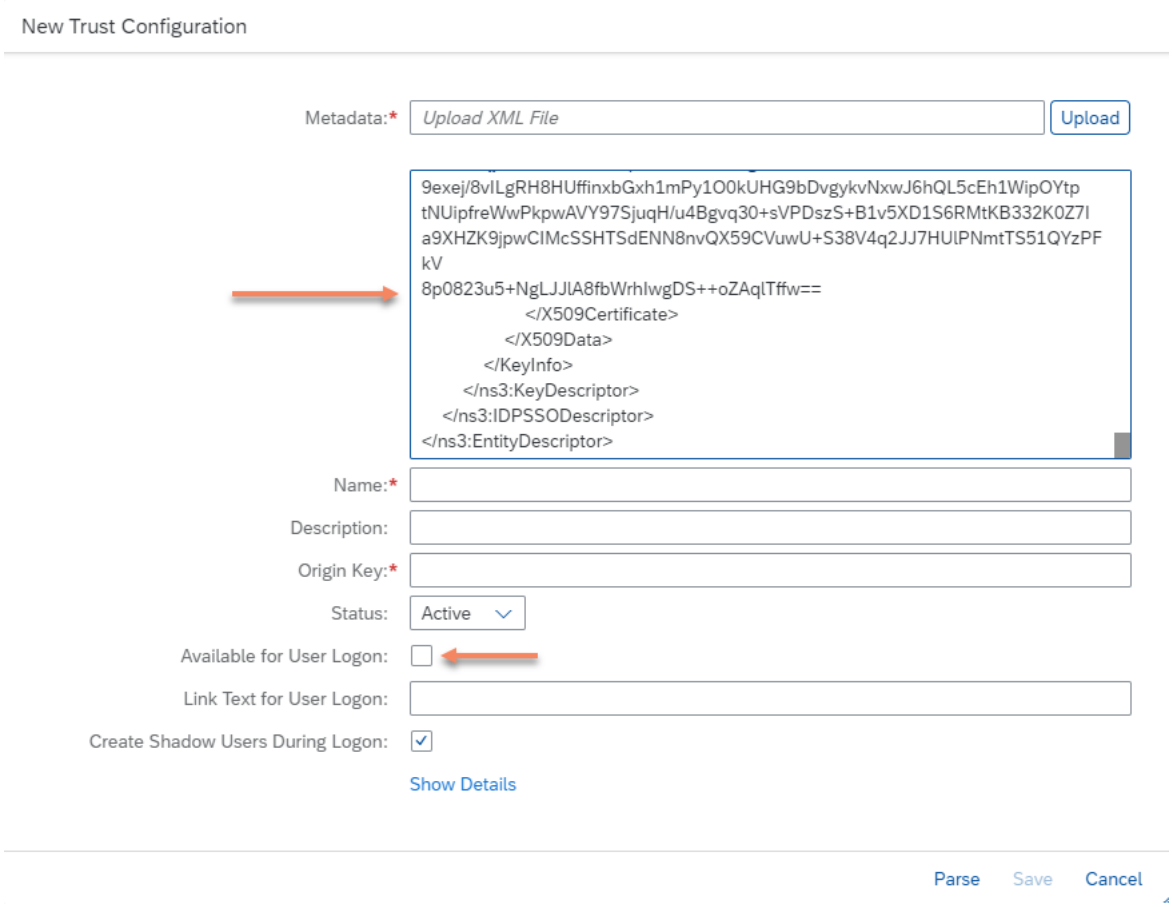
> ### Sample Code
>
> ```
> <ns3:EntityDescriptor
>     ID="cfapps.${S1_LANDSCAPE_HOST}/${S1_SUBACCOUNT_ID}"
>     entityID="cfapps.${S1_LANDSCAPE_HOST}/${S1_SUBACCOUNT_ID}"
>     xmlns="http://www.w3.org/2000/09/xmldsig#"
>     xmlns:ns2="http://www.w3.org/2001/04/xmlenc#"
>     xmlns:ns4="urn:oasis:names:tc:SAML:2.0:assertion"
>     xmlns:ns3="urn:oasis:names:tc:SAML:2.0:metadata">
>     <ns3:SPSSODescriptor AuthnRequestsSigned="true"
>         protocolSupportEnumeration="urn:oasis:names:tc:SAML:2.0:protocol">
>         <ns3:KeyDescriptor use="signing">
>             <KeyInfo>
>                 <KeyName>${S1_ALIAS}</KeyName>
>                 <X509Data>
>                     <X509Certificate>
>                         ${S1_CERTIFICATE}
>                     </X509Certificate>
>                 </X509Data>
>             </KeyInfo>
>         </ns3:KeyDescriptor>
>     </ns3:SPSSODescriptor>
>     <ns3:IDPSSODescriptor
> ```

```
        WantAuthnRequestsSigned="true"
        protocolSupportEnumeration="urn:oasis:names:tc:SAML:2.0:protocol">
        <ns3:KeyDescriptor use="signing">
            <KeyInfo>
                <KeyName>${S1_ALIAS}</KeyName>
                <X509Data>
                    <X509Certificate>
                        ${S1_CERTIFICATE}
                    </X509Certificate>
                </X509Data>
            </KeyInfo>
            <ns3:SingleSignOnService
Binding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST" Location="https://
sap.com"/>
        </ns3:KeyDescriptor>
    </ns3:IDPSSODescriptor>
</ns3:EntityDescriptor>
```

Back to Steps [page 37]

Establish Trust between Subaccount 1 and Subaccount 2

1. In the cockpit, navigate to the overview page for subaccount 2.
2. From the left panel, select *Security* > *Trust Configuration*. Choose *New Trust Configuration*. For details, see Establish Trust and Federation with UAA Using Any SAML Identity Provider.
3. Paste the assembled IdP metadata for subaccount 1 in the `<Metadata>` text box and uncheck *Available for User Logon*.



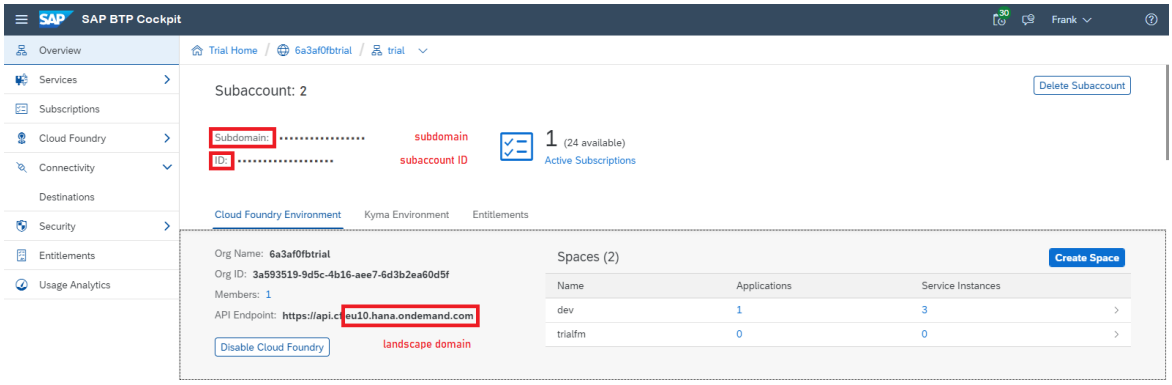
4. Choose *Parse*.
5. Enter a `<Name>` for the trust configuration and choose *Save*.

> **Note**
>
> Additionally, you must add users to this new trust configuration and assign appropriate scopes to them.

Back to Steps [page 37]

Create an OAuthSAMLBearerAssertion Destination for Application 1

1. In the cockpit, navigate to the overview page for subaccount 2.
2. Here you can see the landscape domain, subaccount ID and subdomain of subaccount 2. Below, we refer to the landscape domain as `${S2_LANDSCAPE_DOMAIN}`, to the subaccount ID as `${S2_SUBACCOUNT_ID}` and to the subdomain as `${S2_SUBDOMAIN}`.



3. In your browser, call `https://${S2_SUBDOMAIN}.authentication.${S2_LANDSCAPE_DOMAIN}/saml/metadata` and download the XML file. Within the XML file, you can find the following structure. It contains the <audience> and the <alias> variables:

> **Sample Code**
>
> ```
> <?xml version="1.0" encoding="UTF-8"?>
> <md:EntityDescriptor entityID="<audience>" ...>
> ...
>     <md:AssertionConsumerService
> Binding="urn:oasis:names:tc:SAML:2.0:bindings:URI" Location="https://
> ${S2_SUBDOMAIN}.authentication.${S2_LANDSCAPE_DOMAIN}/oauth/token/alias/
> <alias>" index="1"/>
> ...
> ```

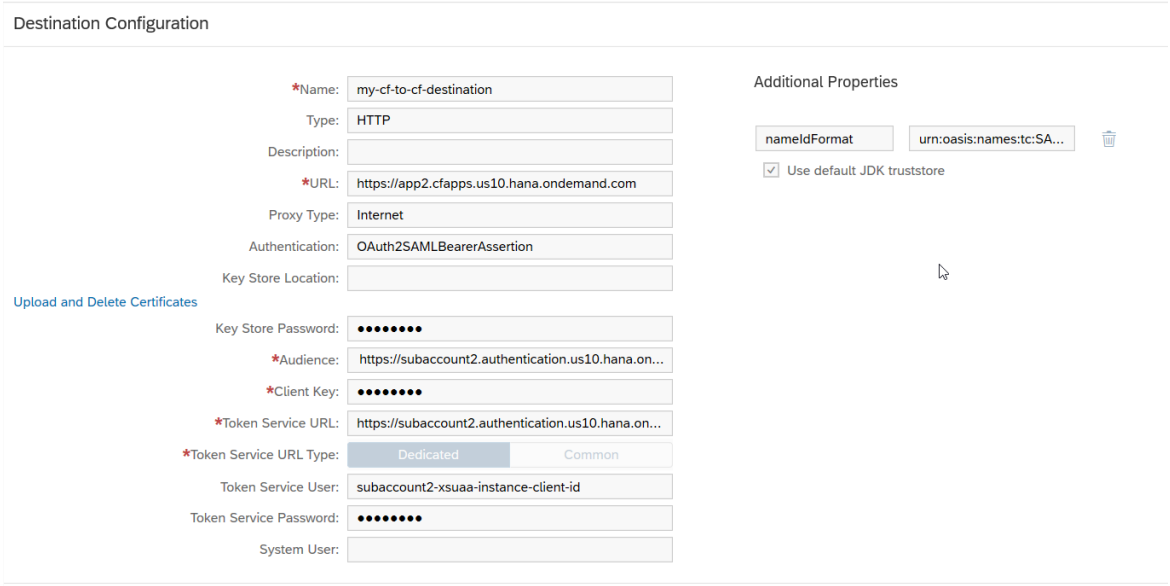
   Below, we refer to the value of <alias> as `${S2_ALIAS}` and <audience> as `${S2_AUDIENCE}`.
4. In cockpit, navigate to subaccount 1.
5. From the left panel, select *Connectivity* > *Destinations*.
6. Choose *New Destination* and configure the values as described below. Replace the `${...}` placeholders with the values you determined in the previous steps and sections.

Property	Value
`Name`	Choose any name for your destination. You will use this name to request the destination from the Destination service.
`Type`	**`HTTP`**
`URL`	The URL of application 2, identifying the resource you want to consume.
`Proxy Type`	**`Internet`**
`Authentication`	**`OAuth2SAMLBearerAssertion`**
`Audience`	**`${S2_AUDIENCE}`**
`Client Key`	The *clientid* of the XSUAA instance in subaccount 2. Can be acquired via a binding or service key.
`Token Service URL`	**`https://${S2_SUBDOMAIN}.authentication.${S2_LANDSCAPE_DOMAIN}/oauth/token/alias/${S2_ALIAS}`**
`Token Service URL Type`	**`Dedicated`**
`Token Service User`	The *clientid* of the XSUAA instance in subaccount 2. Can be acquired via a binding or service key.
`Token Service Password`	The *clientsecret* of the XSUAA instance in subaccount 2. Can be acquired via a binding or service key.
`authnContextClassRef`	**`urn:oasis:names:tc:SAML:2.0:ac:classes:PreviousSession`**

Additional Properties

Property	Value
`nameIdFormat`	**`urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress`**

Example



7. Choose *Save*.

Back to Steps [page 37]

Consume the Destination and Execute the Scenario

To perform the scenario and execute the request from application 1, targeting application 2, proceed as follows:

1. Decide on where the user identity will be located when calling the Destination service. For details, see User Propagation via SAML 2.0 Bearer Assertion Flow [page 1558]. This will determine how exactly you will perform step 2.
2. Execute a "find destination" request from application 1 to the Destination service. For details, see Consuming the Destination Service [page 1550] and the REST API documentation.
3. From the Destination service response, extract the access token and URL, and construct your request to application 2. See "Find a Destination" Response Structure [page 1565] for details on the structure of the response from the Destination service.

Back to Steps [page 37]

1.1.3.1.2.4 SAML Bearer Assertion Flow from the Cloud Foundry Environment to the Neo Environment

Propagate the identity of a user from a Cloud Foundry application to a Neo application.

Steps

Scenario [page 46]

Prerequisites [page 46]

Concept [page 46]

Procedure [page 48]

1. Configure a Local Service Provider for the Neo Subaccount [page 48]
2. Establish Trust between Cloud Foundry and Neo Subaccounts [page 48]
3. Create an OAuth Client for the Neo Application [page 50]
4. Create an OAuth2SAMLBearerAssertion Destination for the Cloud Foundry Application [page 50]
5. Consume the Destination and Execute the Scenario [page 52]

Scenario

- You have deployed an application in the Cloud Foundry environment.
- You want to consume OAuth protected APIs exposed by an application deployed in the Neo environment.
- You want to propagate the identity of the user logged in to the Cloud Foundry application, to the Neo application.

Back to Steps [page 46]

Prerequisites

- You have deployed an application in the Cloud Foundry environment.
- You have bound an instance of the Destination Service to the application.
- You have bound an instance of the XSUAA service with the `application` plan to the application.
- You have deployed an application in the Neo environment.

Back to Steps [page 46]

Concept

The identity of a user logged in to the Cloud Foundry application is established by an identity provider (IdP).

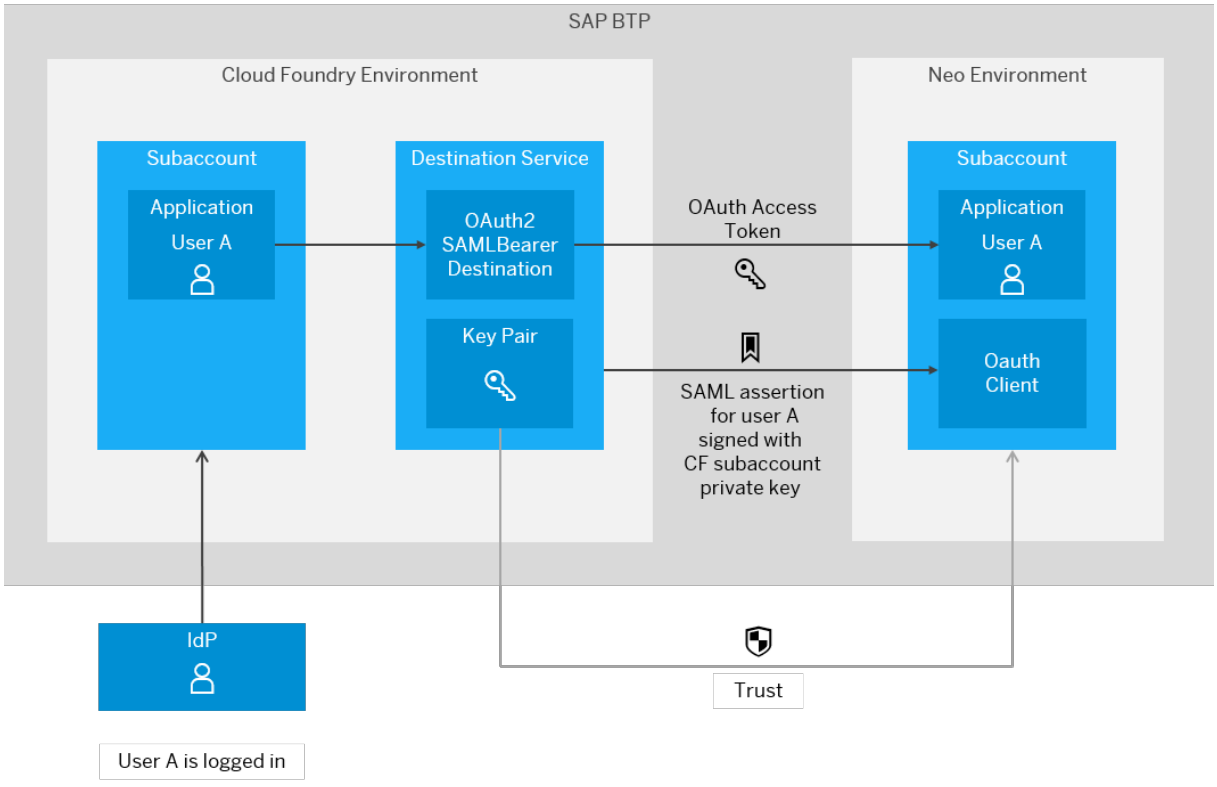
> **Note**
>
> You can use the default IdP of the Cloud Foundry subaccount or any trusted IdP, for example, the Neo subaccount IdP.

When the application retrieves an OAuthSAMLBearer destination, the user is made available to Cloud Foundry Destination service by means of a *user exchange* JWT (JSON Web Token).

The service then wraps the user identity in a SAML assertion, signs it with the Cloud Foundry subaccount private key and sends it to the token endpoint of the OAuth service for the Neo application.

To make the Neo application accept the SAML assertion, you must set up a trust relationship between the Neo subaccount and the Cloud Foundry subaccount public key. You can achieve this by adding the Cloud Foundry subaccount X.509 certificate as trusted IdP in the Neo subaccount. Thus, the Cloud Foundry application starts acting as an IdP and any users propagated by it are accepted by the Neo application, even users that do not exist in the IdP.

The OAuth service accepts the SAML assertion and returns an OAuth access token. In turn, the Destination service returns both the destination and the access token to the requesting application. The application then uses the destination properties and the access token to consume the remote API.

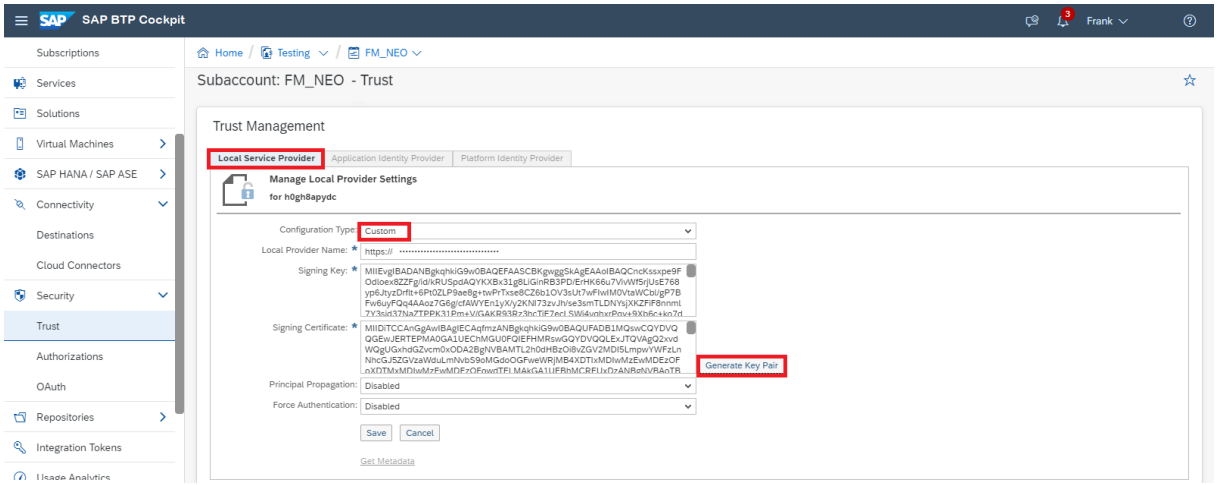


Back to Steps [page 46]

Procedure

Configure a Local Service Provider for the Neo Subaccount

1. In the cockpit, navigate to your Neo subaccount, choose *Security* > *Trust* from the left menu, and go to tab *Local Service Provider* on the right. For `<Configuration Type>`, select **`Custom`** and choose *Generate Key Pair*.
2. *Save* the configuration.

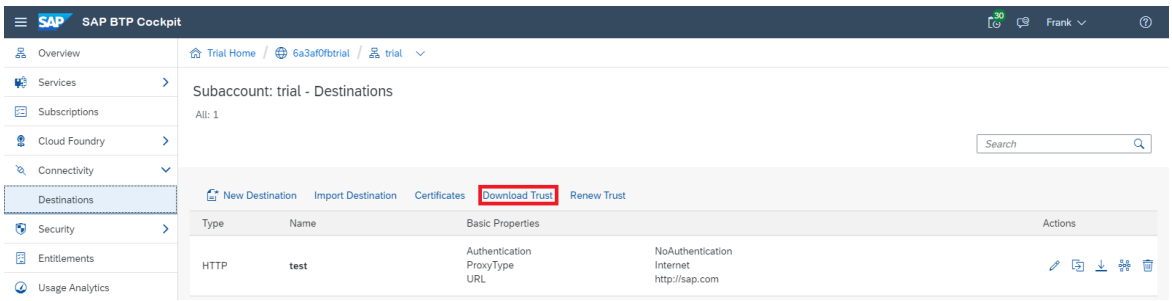


> **Note**
>
> IMPORTANT: When you choose **`Custom`** for the *Local Service Provider* type, the default IdP (SAP ID service) will no longer be available. If your scenario requires login to the SAP ID service as well, you can safely skip this step and leave the default settings for the Local Service Provider.

Back to Steps [page 46]

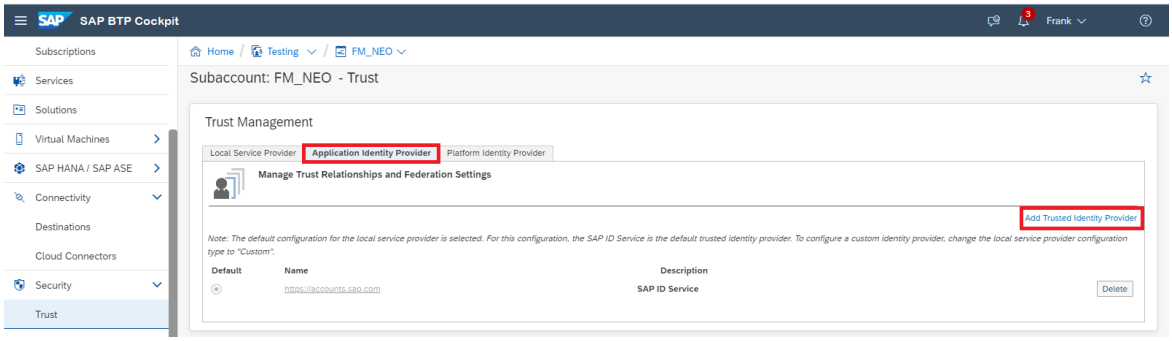
Establish Trust between Cloud Foundry and Neo Subaccounts

1. Download the X.509 certificate of the Cloud Foundry subaccount:
   In the cockpit, navigate to your Cloud Foundry subaccount and choose *Connectivity* > *Destinations*. Press *Download Trust* to get the certificate for this subaccount.

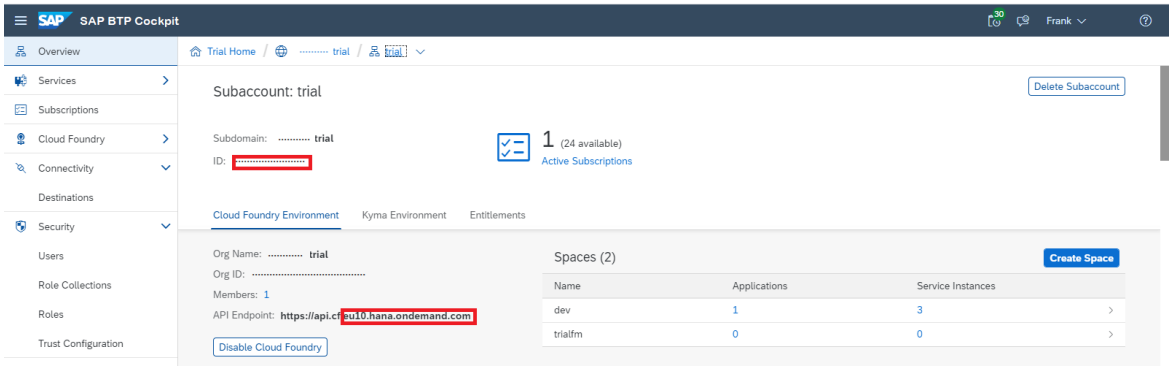


2. Configure trust in the Neo subaccount:

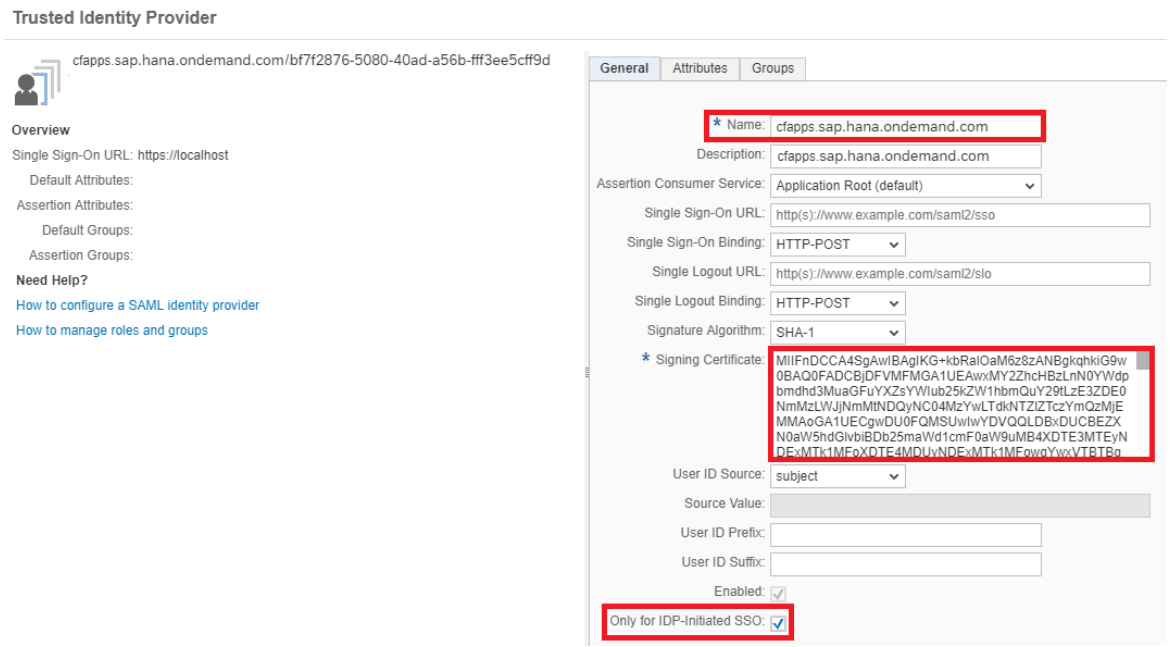
In the cockpit, navigate to your Neo subaccount, choose *Trust* from the left menu, and select the *Application Identity Provider* tab on the right. Then choose *Add Trusted Identity Provider*.



In the `<Name>` field, enter the *cfapps* host followed by the subaccount GUID, for example `cfapps.sap.hana.ondemand.com/bf7f2876-5080-40ad-a56b-fff3ee5cff9d`. This information is available in the cockpit, on the overview page of your Cloud Foundry subaccount:



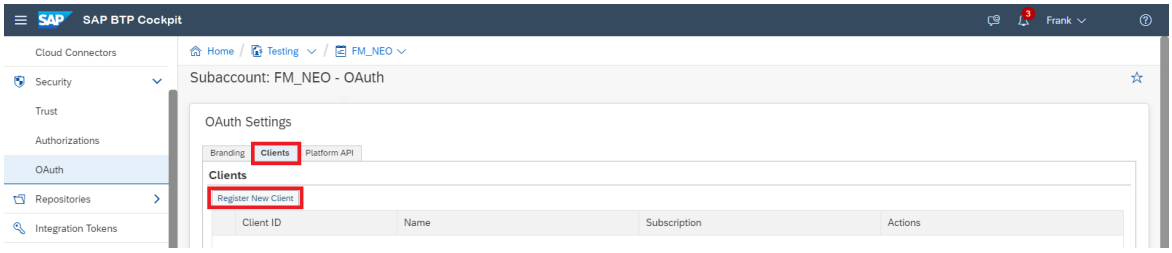
In the `<Signing Certificate>` field, paste the X.509 certificate you downloaded in step 1. Make sure you remove the *BEGIN CERTIFICATE* and *END CERTIFICATE* strings. Then check *Only for IDP-Initiated SSO* and save the configuration:



Back to Steps [page 46]

Create an OAuth Client for the Neo Application

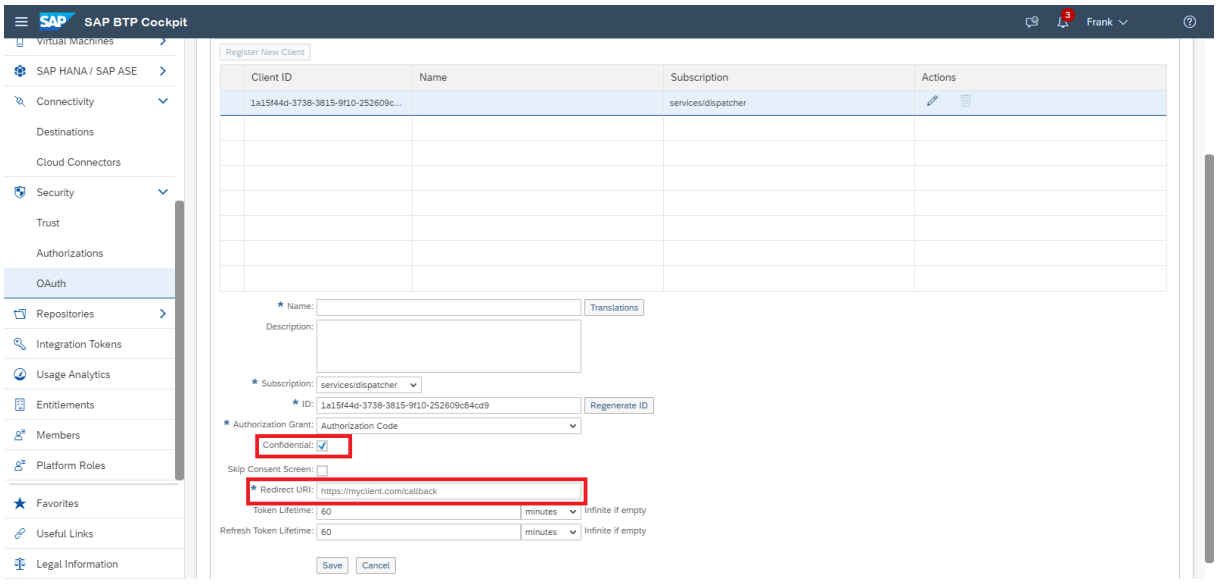
1. In the cockpit, navigate to the Neo subaccount, choose *Security* > *OAuth* from the left menu, select tab *Client*, and choose *Register New Client*:



2. Enter a `<Name>` for the client.
3. In the `<Subscription>` field, select your Neo application.
4. For `<Authorization Grant>` select **`Authorization Code`**.
5. Check the *Confidential* checkbox and provide a secret for the OAuth client.

> **Note**
>
> Make sure you remember the secret, because it will not be visible later.

6. `<Redirect URI>` is irrelevant for the OAuth SAML Bearer Assertion flow, so you can provide any URL in the Cloud Foundry application.

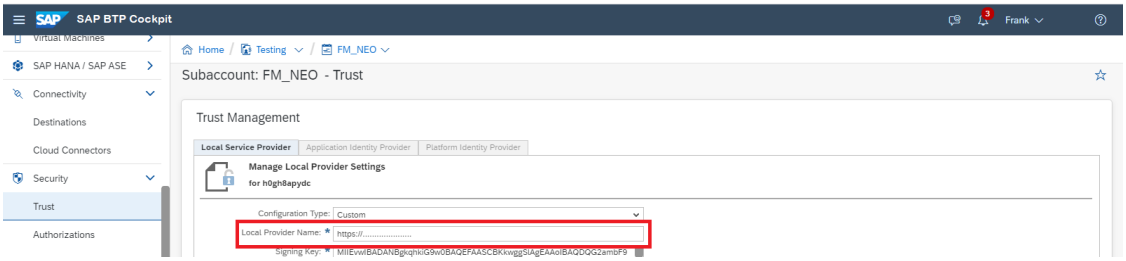


Back to Steps [page 46]

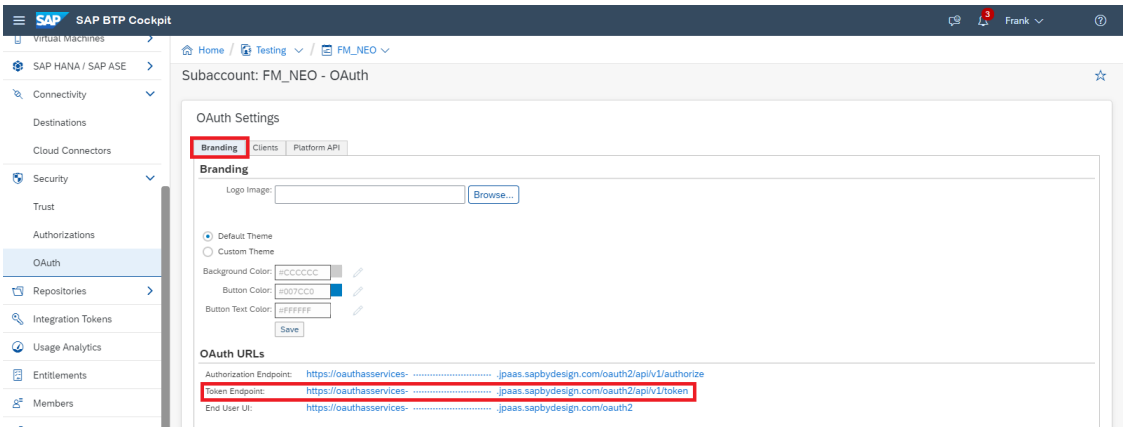
Create an OAuth2SAMLBearerAssertion Destination for the Cloud Foundry Application

1. In the cockpit, navigate to the Cloud Foundry subaccount, choose *Connectivity* > *Destinations* from the left menu, select the *Client* tab and press *New Destination*.
2. Enter a `<Name>` for the destination, then provide:
   - `<URL>`: the URL of the Neo application/API you want to consume.

- `<Authentication>`: **`OAuth2SAMLBearerAssertion`**
- `<Audience>`: can be taken from the Neo subaccount, if you choose *Security* > *Trust* form the left menu, go to the *Local Service Provider* tab, and copy the value of `<Local Provider Name>`:



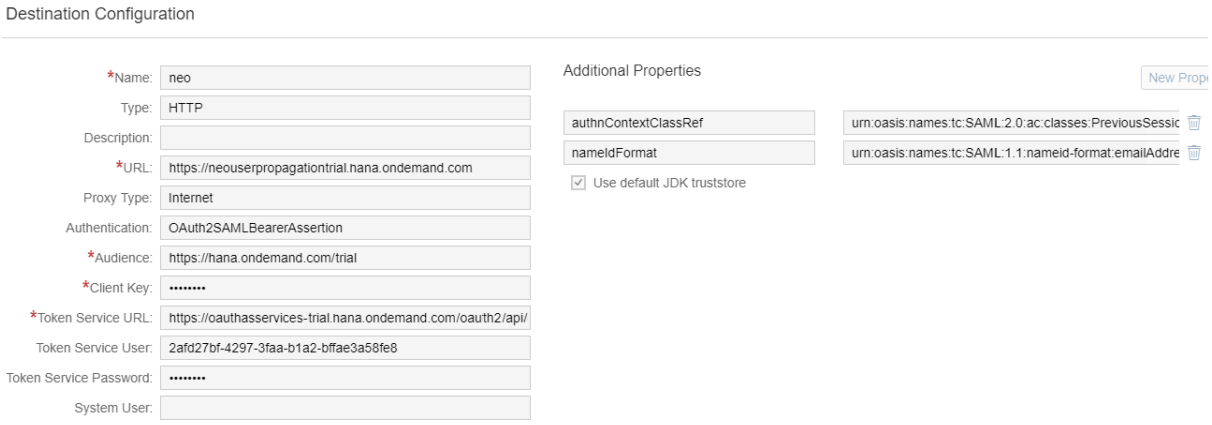
- `<Client Key>`: the ID of the OAuth client for the Neo application
- `<Token Service URL>`: can be taken from the *Branding* tab in the Neo subaccount (choose *Security* > *OAuth* from the left menu):



- `<Token Service User>`: again the ID of the OAuth client for the Neo application.
- `<Token Service Password>`: the OAuth client secret.

Enter two additional properties:

- **authnContextClassRef**: **`urn:oasis:names:tc:SAML:2.0:ac:classes:PreviousSession`**
- **nameIdFormat**:
  - **`urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified`**, if the user ID is propagated to the Neo application, or
  - **`urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress`**, if the user email is propagated to the Neo application.



Back to Steps [page 46]

Consume the Destination and Execute the Scenario

To perform the scenario and execute the request from the source application towards the target application, proceed as follows:

1. Decide on where the user identity will be located when calling the Destination service. For details, see User Propagation via SAML 2.0 Bearer Assertion Flow [page 1558]. This will determine how exactly you will perform step 2.
2. Execute a "find destination" request from the source application to the Destination service. For details, see Consuming the Destination Service [page 1550] and the REST API documentation.
3. From the Destination service response, extract the access token and URL, and construct your request to the target application. See "Find a Destination" Response Structure [page 1565] for details on the structure of the response from the Destination service.

Back to Steps [page 46]

1.1.3.1.2.5 SAML Bearer Assertion Flow from the Cloud Foundry Environment to SAP S/4HANA Cloud

Configure user propagation (single sign-on), using OAuth communication from the SAP BTP Cloud Foundry environment to S/4HANA Cloud. As OAuth mechanism, you use the *OAuth 2.0 SAML Bearer Assertion Flow*.

Steps

Scenario [page 53]

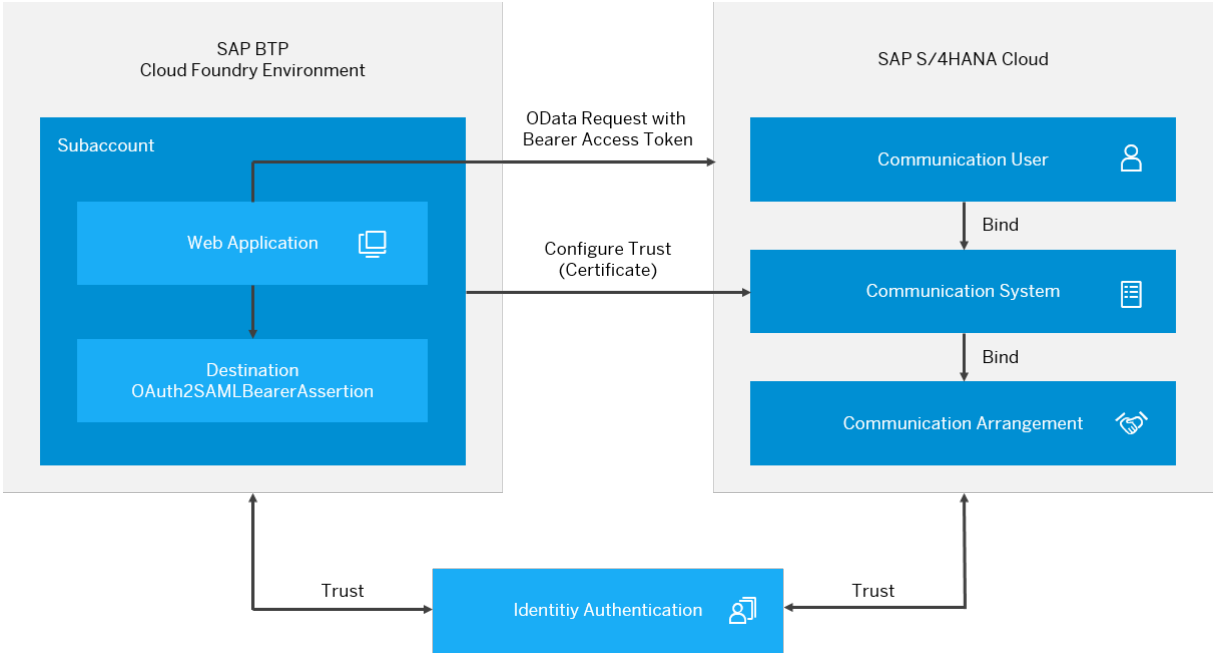
Prerequisites [page 53]

Configuration Tasks [page 54]

Scenario

As a customer, you own an SAP BTP global account and have created at least one subaccount therein. Within the subaccount, you have deployed a Web application. Authentication against the Web application is based on a trusted identity provider (IdP) that you need to configure for the subaccount.

On the S/4HANA Cloud side, you own an S/4HANA ABAP tenant. Authentication against the S/4HANA ABAP tenant is based on the trusted IdP which is always your Identity Authentication Service (IAS) tenant. Typically, you will configure this S/4HANA Cloud Identity tenant to forward authentication requests to your corporate IdP.



Prerequisites

- You have an S/4HANA Cloud tenant and a user with the following business catalogs assigned:

Business Role ID	Area
SAP_BCR_CORE_COM	Communication Management
SAP_BCR_CORE_IAM	Identity and Access Management
SAP_BCR_CORE_EXT	Extensibility

- You have administrator permission for the configured S/4HANA Cloud IAS tenant.
- You have a subaccount and PaaS tenant in the SAP BTP Cloud Foundry environment.

Next Step

- Configuration Tasks [page 54]

1.1.3.1.2.5.1 Configuration Tasks

Perform these steps to set up user propagation between S/4HANA Cloud and the SAP BTP Cloud Foundry environment.

Tasks

1. Configure Single Sign-On between S/4HANA Cloud and the Cloud Foundry Organization on SAP BTP [page 54]
2. Configure OAuth Communication [page 54]
3. Configure Communication Settings in S/4HANA Cloud [page 55]
4. Configure Communication Settings in SAP BTP [page 58]
5. Consume the Destination and Execute the Scenario [page 60]

Configure Single Sign-On between S/4HANA Cloud and the Cloud Foundry Organization on SAP BTP

To configure SSO with S/4HANA you must configure trust between the S/4HANA IAS tenant and theCloud Foundry organization, see Manually Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services.

Configure OAuth Communication

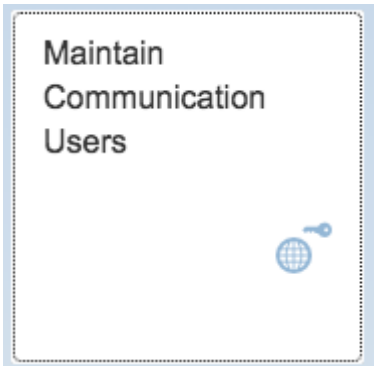
Export the X.509 certificate (trust certificate) from your Cloud Foundry subaccount.

For more information, see Manage Trust [page 1514].

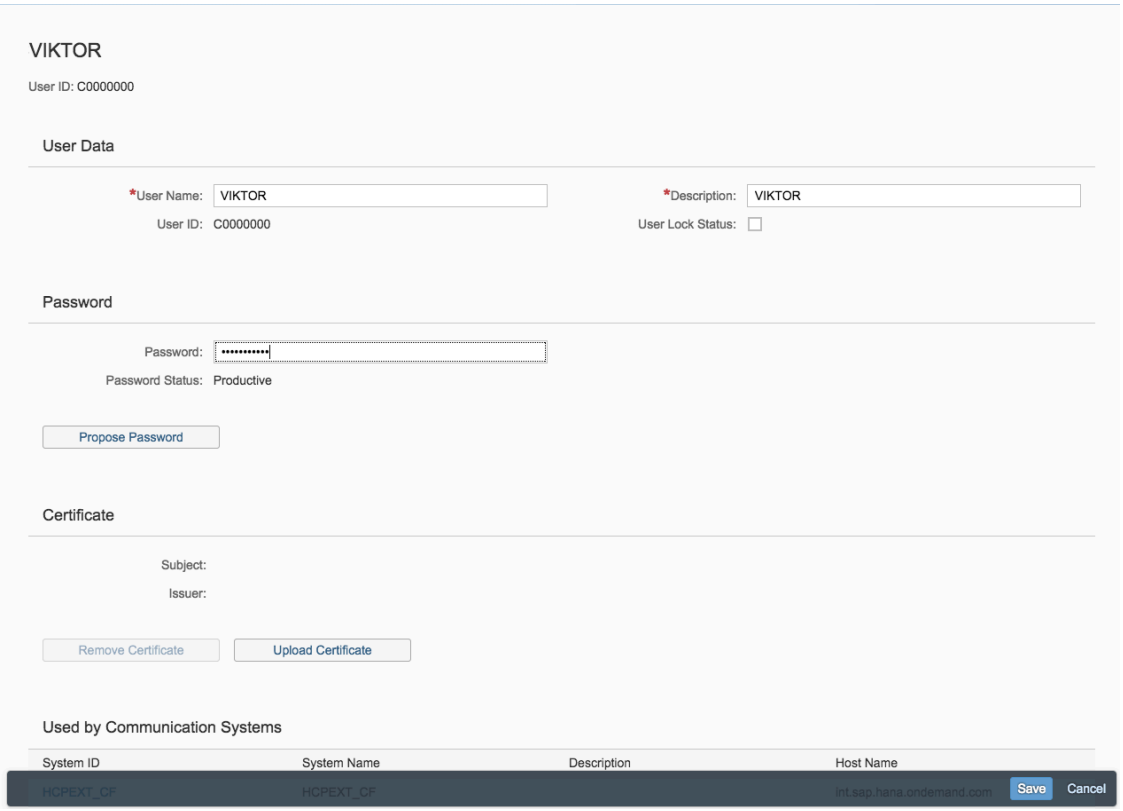
Back to Tasks [page 54]

Configure Communication Settings in S/4HANA Cloud

1. **Create a Communication User**
   1. In your S/4HANA Cloud launchpad, choose the application *Maintain Communication Users*.



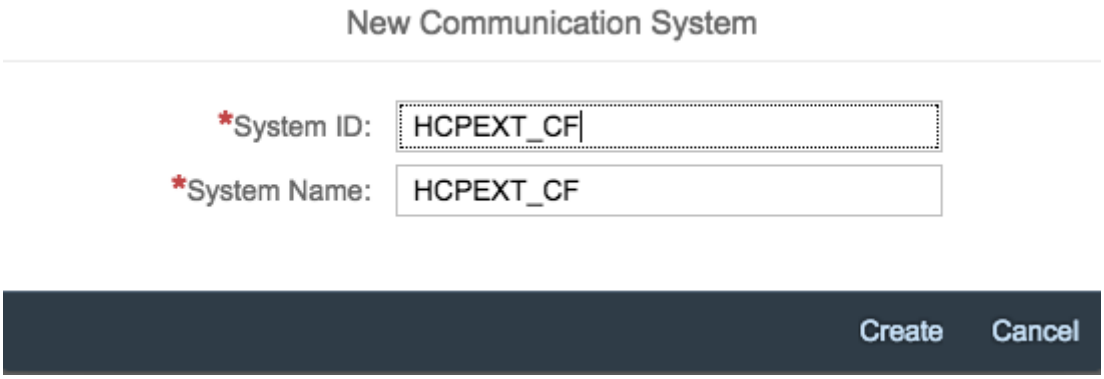
   2. From the *User List* view, create a new user.
   3. Set `<User Name>`, `<Password>` and `<Description>`.
   4. Copy this password, you will need it in a later step.
   5. Press the *Save* on the bottom of the screen.



   6. Close the *Communication Users* application.

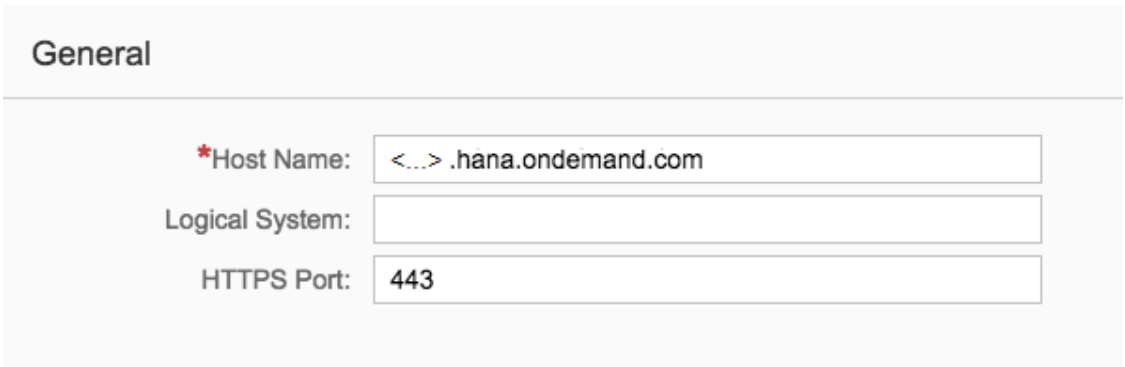
2. **Set up a Communication System for OAuth**
    1. From the launchpad, choose the application *Communication Systems*.

    2. From the list view, select *New*.
    3. A popup window appears. Enter the `<System ID>` and the `<System Name>`, then choose *Create*.



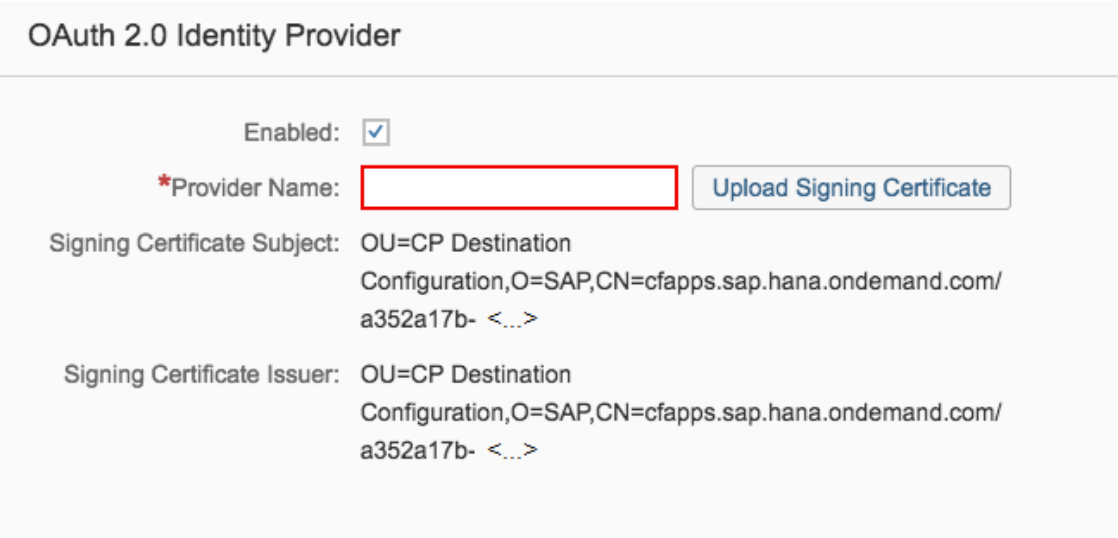
    4. Enter the host name. This is your Cloud Foundry region, for example: `cf.eu10.hana.ondemand.com` for Europe (Frankfurt).

> **Note**
>
> For the complete list of standard regions, see Regions.

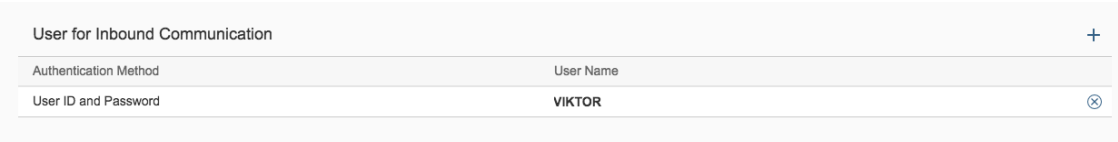


    5. Enable the OAuth Identity Provider.

6. Upload the subaccount certificate that you have downloaded before from the SAP BTP cockpit.



7. From `<Signing Certificate Subject>`, copy the `CN` value (for example: `cfapps.sap.hana.ondemand.com/a352a17b-<...>`) and paste it in the field `<Provider Name>`.
8. Add the *Communication User* you have created in the previous step.

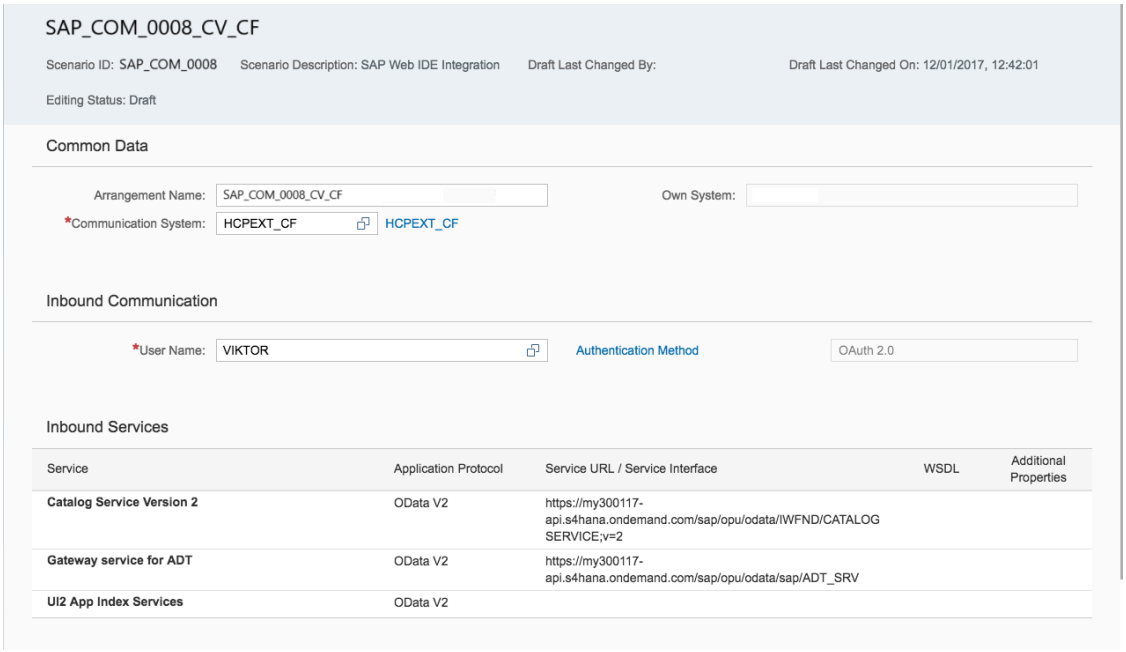


9. Save your settings and go back to the launchpad.

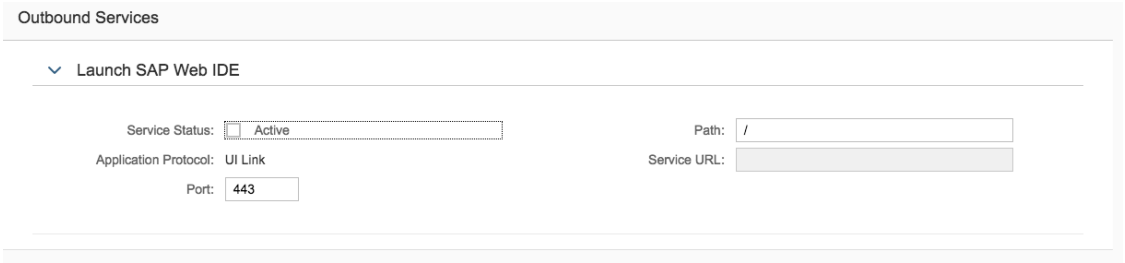
3. **Create a Communication Arrangement**
   1. Start the *Communication Arrangements* application.

   2. From the list view, select *New*.

3. In the popup, choose a scenario. For our example, we use `SAP_COM_0008`. Set the arrangement name, for example **`SAP_COM_0008_MY_TEST`**.
4. In the *Common Data* section of the configuration screen, select the `<Communication System>` that you have created in the step before. The communication user is added automatically in the *Inbound Communication* section, and the `<Authentication Method>` is set to `OAuth 2.0`.



5. In the *Outbound Services* section, go to *Launch SAP Web IDE* and uncheck the *Active* checkbox of the field `<Service Status>`.



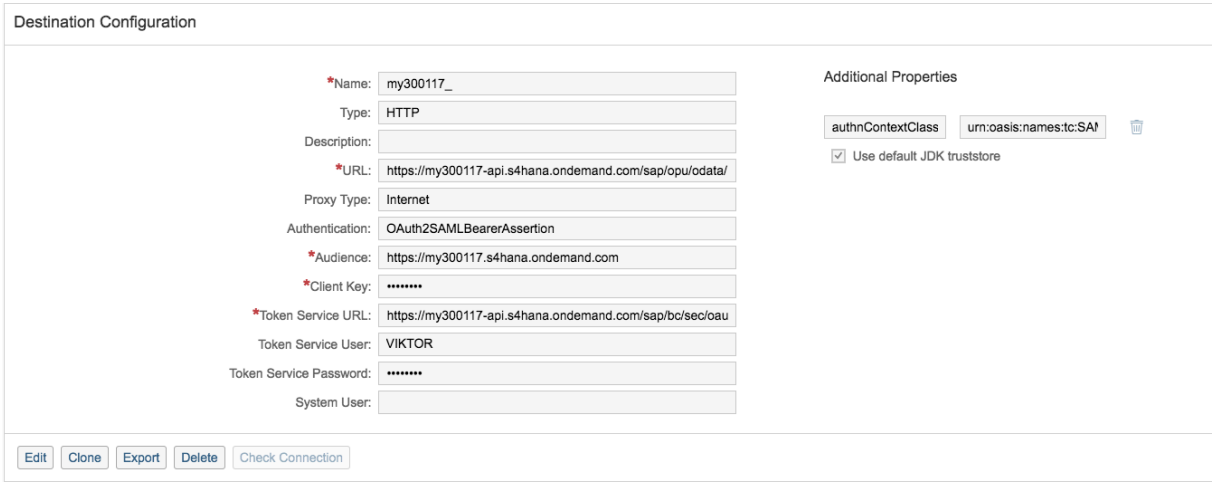
6. Save your settings and go back to the launchpad.

Back to Tasks [page 54]

Configure Communication Settings in SAP BTP

1. From the SAP BTP cockpit, choose *Cloud Foundry environment* > *your global account*.
2. Choose your subaccount, and from the left-side subaccount menu, go to *Connectivity* > *Destinations*.
3. Press the *New Destination* button.
4. Enter the following parameters for your destination:

Parameter	Value
`Name`	Enter a meaningful name.
`Type`	**`HTTP`**
`Description`	(Optional) Enter a meaningful description.
`URL`	The OData URL, for example `https://my300117-api.s4hana.ondemand.com/sap/opu/odata/IWFND/CATALOGSERVICE;v=2?$format=json`
`Proxy Type`	**`Internet`**
`Authentication`	**`OAuth2SAMLBearerAssertion`**
`Audience`	The URL of your SAP S/4HANA Cloud account. To get it, log on to your SAP S/4HANA Cloud account. Select the profile picture. Then choose *Settings* and copy the value from the `<Server>` field. Add **`https://`** to the beginning of this string, for example, **`https://my300117.s4hana.ondemand.com`**. **Note** This URL does not contain **`my300117`**-*api*, but only **`my300117`**.
`Client Key`	The name of the communication user you have in the SAP S/4HANA ABAP tenant, e.g VIKTOR.
`Token Service URL`	For this field, you need the part of the URL before `/sap/...` that you copied before from *Communications Arrangements* service URL/service interface: `https://my300117-api.s4hana.ondemand.com/sap/bc/sec/oauth2/token`
`Token Service User`	The same user as for the `Client Key` parameter.
`Token Service Password`	The password for the communication user.
`System User`	This parameter is not used, leave the field empty.
`authnContextClassRef`	**`urn:oasis:names:tc:SAML:2.0:ac:classes:X509`**



Back to Tasks [page 54]

Consume the Destination and Execute the Scenario

To perform the scenario and execute the request from the source application towards the target application, proceed as follows:

1. Decide on where the user identity will be located when calling the Destination service. For details, see User Propagation via SAML 2.0 Bearer Assertion Flow [page 1558]. This will determine how exactly you will perform step 2.
2. Execute a "find destination" request from the source application to the Destination service. For details, see Consuming the Destination Service [page 1550] and the REST API documentation.
3. From the Destination service response, extract the access token and URL, and construct your request to the target application. See "Find a Destination" Response Structure [page 1565] for details on the structure of the response from the Destination service.

Back to Tasks [page 54]

1.1.3.1.2.6 SAML Bearer Assertion Flow from the Cloud Foundry Environment to SAP SuccessFactors

Configure user propagation from the SAP BTP Cloud Foundry environment to SAP SuccessFactors.

Steps

Scenario [page 61]

Prerequisites [page 61]

Concept Overview [page 61]

Create an OAuth Client in SAP SuccessFactors [page 62]

Create and Consume a Destination for the Cloud Foundry Application [page 64]

Scenario

- From an application in the SAP BTP Cloud Foundry environment, you want to consume OData APIs exposed by SuccessFactors modules.
- To enable single sign-on, you want to propagate the identity of the application's logged-in user to SuccessFactors.

Prerequisites

- In your Cloud Foundry space, you have a deployed application.
- You have an instance of the Destination Service that is bound to the application.
- An instance of the xsuaa service with `application` plan is bound to the application.

Concept Overview

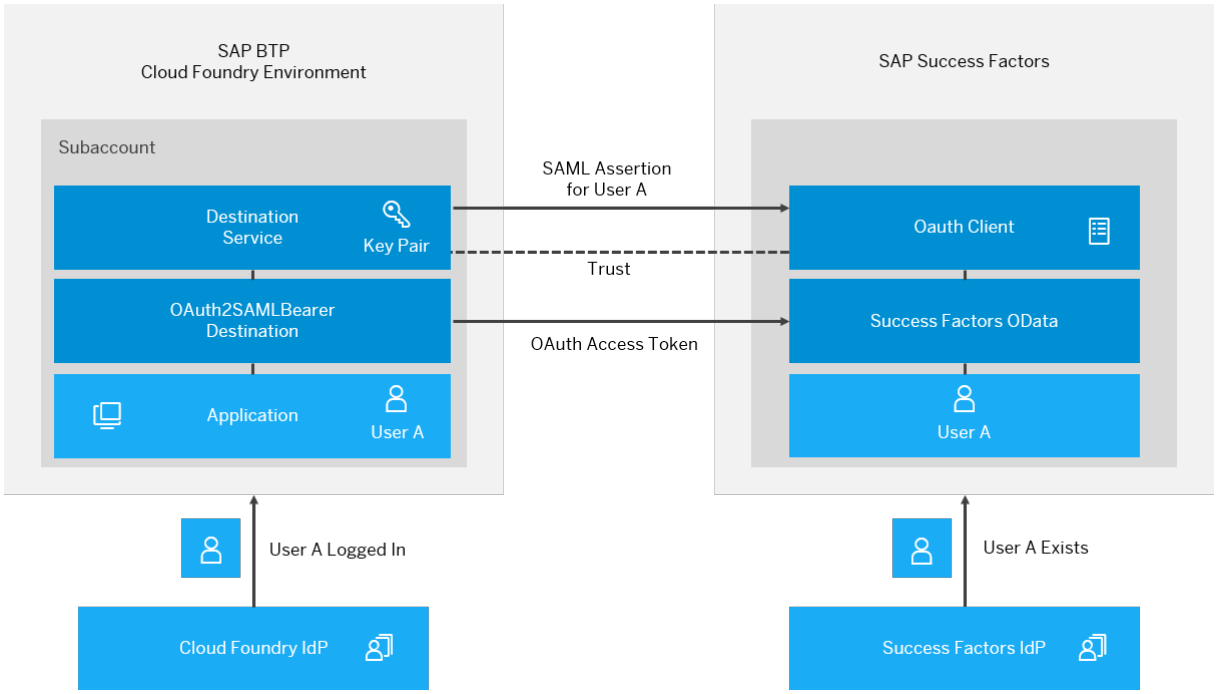
A user logs in to the Cloud Foundry application. Its identity is established by an **Identity Provider** (IdP). This could be the default IdP for the Cloud Foundry subaccount or a trusted IdP, for example the SuccessFactors IdP.

When the application retrieves an `OAuth2SAMLBearer` destination, the user is made available to the Cloud Foundry Destination service by means of a **user exchange token**, represented by a JSON Web Token (JWT). The service then wraps the user identity in a SAML assertion, signs it with the Cloud Foundry subaccount private key and sends it to the token endpoint of the SuccessFactors OAuth server.

To accept the SAML assertion and return an **access token**, a **trust** relationship must be set up between SuccessFactors and the Cloud Foundry subaccount public key. You can achieve this by providing the Cloud Foundry subaccount **X.509 certificate** when creating the OAuth client in SuccessFactors.

Users that are propagated from the Cloud Foundry application, are verified by the SuccessFactors OAuth server before granting them access tokens. This means, users that do not exist in the SuccessFactors user store will be rejected.

For valid users, the OAuth server accepts the SAML assertion and returns an OAuth access token. In turn, the Destination service returns both the destination and the access token to the requesting application. The application then uses the destination properties and the access token to consume SuccessFactors APIs.



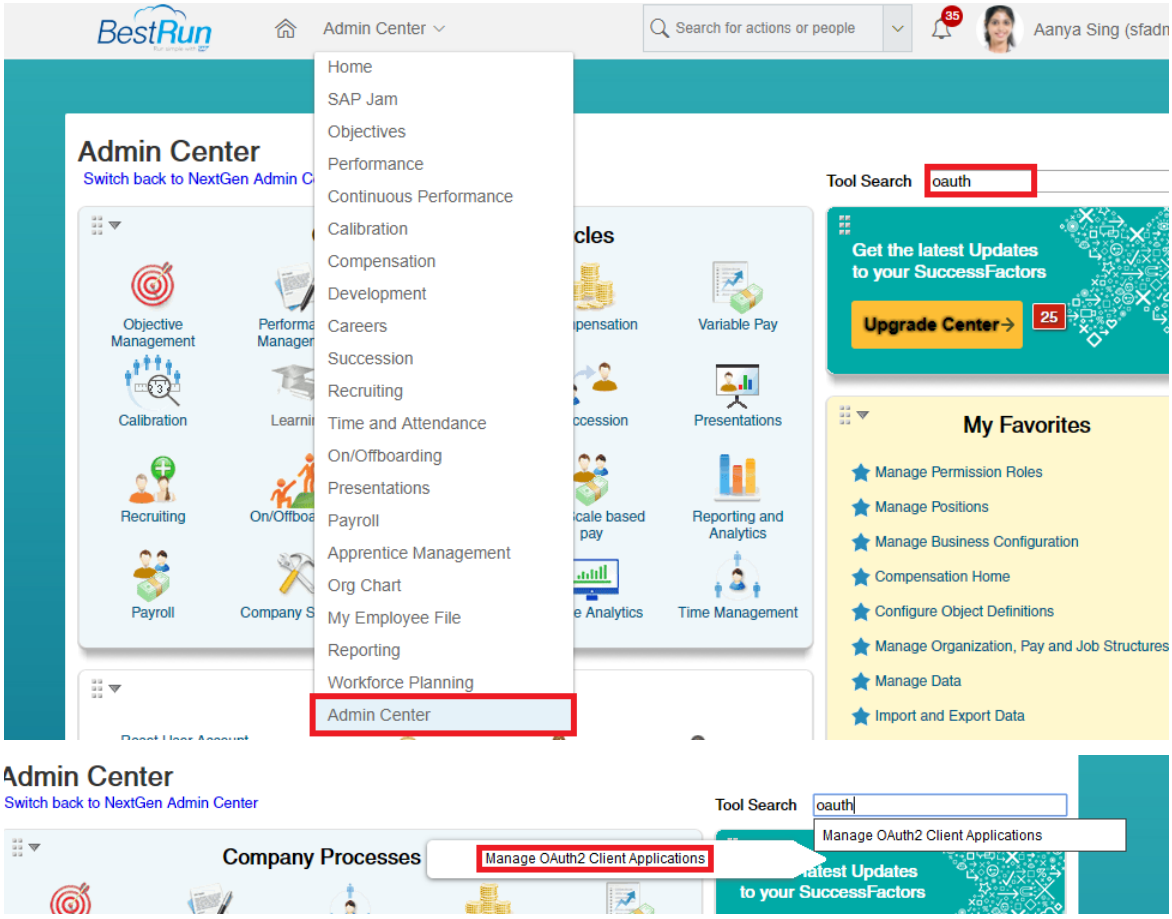
Next Steps

- Create an OAuth Client in SAP SuccessFactors [page 62]
- Create and Consume a Destination for the Cloud Foundry Application [page 64]

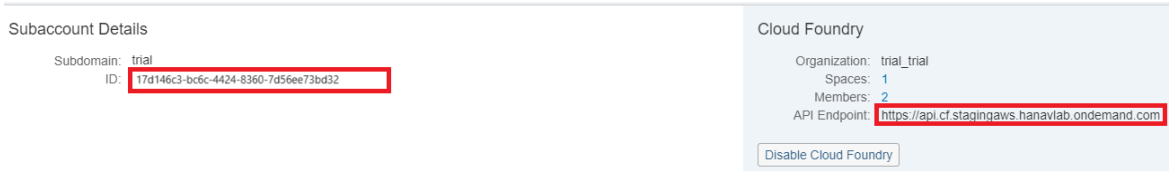
1.1.3.1.2.6.1 Create an OAuth Client in SAP SuccessFactors

Create an OAuth client in SuccessFactors for user propagation from the SAP BTP Cloud Foundry environment.

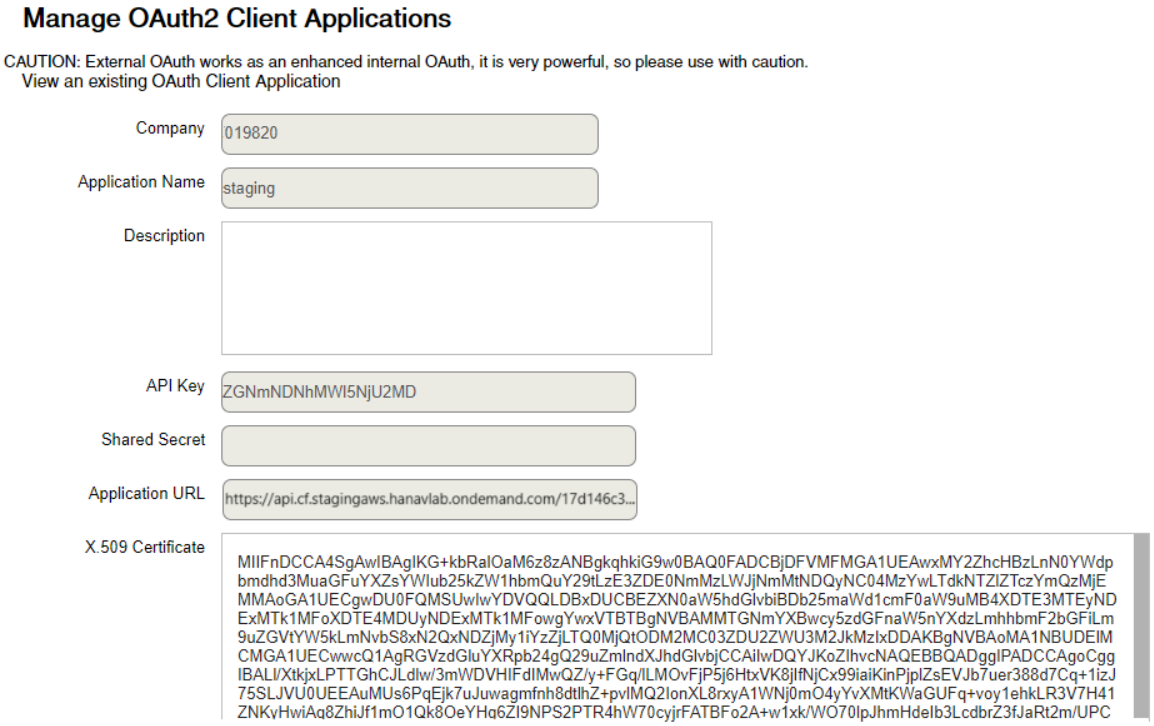
1. Export the X.509 certificate (trust certificate) from your Cloud Foundry subaccount.
   For more information, see Manage Trust [page 1514].
2. Create a SuccessFactors OAuth Client:
   In SuccessFactors, go to the *Admin Center* and search for **`OAuth`**. Choose **`Manage OAuth2 Client Applications`**.



3. Press the *Register Client Application* button on the right. In the `<Application Name>` field, provide some arbitrary descriptive name for the client. For `<Application URL>`, enter the Cloud Foundry API endpoint of the client SAP BTP subaccount, followed by the subaccount GUID, for example `https://api.cf.stagingaws.hanavlab.ondemand.com/17d146c3-bc6c-4424-8360-7d56ee73bd32`. This information is available in the cloud cockpit under subaccount details:



4. In the field `<X.509 Certificate>`, paste the certificate that you downloaded in step 1.
5. Choose *Register* to save the OAuth client.
6. Now, locate your client in the list by its application name, choose *View* in the *Actions* column and take note of the `<API Key>` that has been generated for it. You will use this key later in the OAuth2SAMLBearer destination in the Cloud Foundry environment.



Next Step

- Create and Consume a Destination for the Cloud Foundry Application [page 64]

1.1.3.1.2.6.2 Create and Consume a Destination for the Cloud Foundry Application

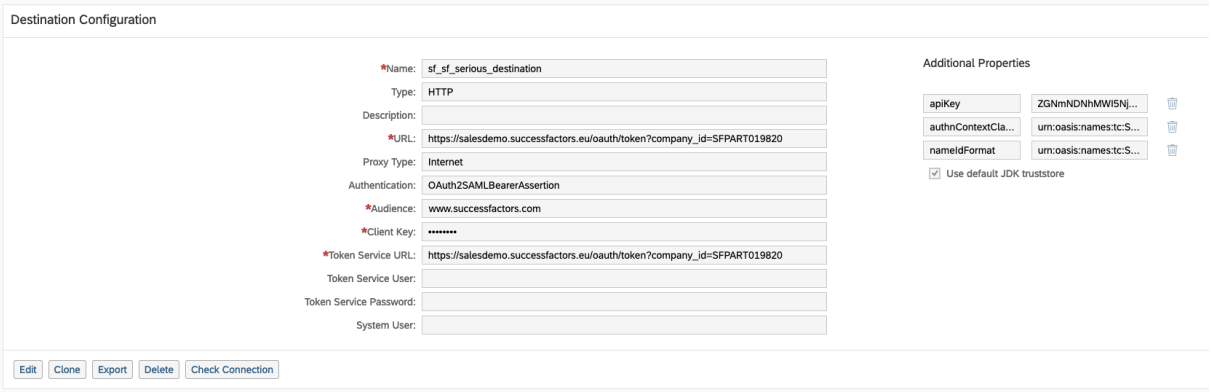
Create and consume an OAuth2SAMLBearerAssertion destination for your Cloud Foundry application.

Create the Destination

1. In the cloud cockpit, navigate to your Cloud Foundry subaccount and from the left-side subaccount menu, choose *Connectivity* > *Destinations*. Choose *New Destination* and enter a name Then provide the following settings:
   - `<URL>`: URL of the SuccessFactors OData API you want to consume.
   - `<Authentication>`: **`OAuth2SAMLBearerAssertion`**
   - `<Audience>`: **`www.successfactors.com`**
   - `<Client Key>`: API Key of the OAuth client you created in SuccessFactors.

- `<Token Service URL>`: API endpoint URL for the SuccessFactors instance, followed by `/oauth/token` and the URL parameter `company_id` with the company ID, for example `https://apisalesdemo2.successfactors.eu/oauth/token?company_id=SFPART019820`.

2. Enter three additional properties:
   - **apiKey**: the API Key of the OAuth client you created in SuccessFactors.
   - **authnContextClassRef**: `urn:oasis:names:tc:SAML:2.0:ac:classes:PreviousSession`
   - **nameIdFormat**:
     - `urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified`, if the **user ID** will be propagated to a SuccessFactors application, or
     - `urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress`, if the **user e-mail** will be propagated to SuccessFactors.



Consume the Destination and Execute the Scenario

To perform the scenario and execute the request from the source application towards the target application, proceed as follows:

1. Decide on where the user identity will be located when calling the Destination service. For details, see User Propagation via SAML 2.0 Bearer Assertion Flow [page 1558]. This will determine how exactly you will perform step 2.
2. Execute a "find destination" request from the source application to the Destination service. For details, see Consuming the Destination Service [page 1550] and the REST API documentation.
3. From the Destination service response, extract the access token and URL, and construct your request to the target application. See "Find a Destination" Response Structure [page 1565] for details on the structure of the response from the Destination service.

1.1.3.1.2.7 Setting up Trust Between Systems (for SAML Bearer Assertion Flows)

Establish trust for principal propagation using an *SAML Bearer Assertion* flow.

For detailed information, see Manage Trust [page 1514].

1.1.3.2 Invoking ABAP Function Modules via RFC

Call a remote-enabled function module in an on-premise or cloud ABAP server from your application, using the RFC protocol.

Find the tasks and prerequisites that are required to consume an ABAP function module via RFC, using the Java Connector (JCo) API as a built-in feature of SAP BTP.

Tasks

Task Type	Task
Operator	Prerequisites [page 66]
Operator and/or Developer	About JCo [page 67]
	Installation Prerequisites for JCo Applications [page 67]
	Consume Connectivity via RFC [page 68]
	Restrictions [page 68]

Prerequisites

Before you can use RFC communication for an SAP BTP application, you must configure:

- A destination on SAP BTP to use RFC.
  For more information, see RFC Destinations [page 1443].

- (Only for on-premise backend systems) RFC connectivity between a backend system and the application. To do this, you must install the Cloud Connector [page 1602] in your internal network and configure it to expose a remote-enabled function module in an ABAP system.
  For more information, see Initial Configuration (RFC) [page 1678] and Configure Access Control (RFC) [page 1752].

Back to Tasks [page 66]

About JCo

To learn in detail about the SAP JCo API, see the JCo 3.1 documentation on SAP Support Portal.

> **Note**
>
> Some sections of this documentation are not applicable to SAP BTP:
>
> - **Architecture**: CPIC is only used in the last mile from your Cloud Connector to an *on-premise* ABAP backend. From SAP BTP to the Cloud Connector, TLS-protected communication is used.
> - **Installation**: SAP BTP runtimes already include all required artifacts.
> - **Customizing and Integration**: On SAP BTP, the integration is already done by the runtime. You can concentrate on your business application logic.
> - **Server Programming**: The programming model of JCo on SAP BTP does not include server-side RFC communication.
> - **IDoc Support for External Java Applications**: Currently, there is no IDocLibrary for JCo available on SAP BTP

Back to Tasks [page 66]

Installation Prerequisites for JCo Applications

- For connections to an *on-premise* ABAP backend, you have downloaded the Cloud Connector installation archive from SAP Development Tools for Eclipse and connected the Cloud Connector to your subaccount.
- You have downloaded and set up your Eclipse IDE and the Eclipse Tools for Cloud Foundry .
- You have downloaded the Cloud Foundry CLI, see Tools.

Back to Tasks [page 66]

Consuming Connectivity via RFC

You can call a service from a fenced customer network using an application that consumes a remote-enabled function module.

Invoking function modules via RFC is enabled by a JCo API that is comparable to the one available in SAP NetWeaver Application Server Java 7.50, and in JCo standalone 3.1. If you are an experienced JCo developer, you can easily develop a Web application using JCo: you simply consume the APIs like you do in other Java environments. Restrictions that apply in the cloud environment are mentioned in the **Restrictions** section below.

Find a sample Web application in Invoke ABAP Function Modules in On-Premise ABAP Systems [page 69].

Back to Tasks [page 66]

Restrictions

- The supported **runtime environment** is SAP Java Buildpack as of version 1.8.0.

  > ⓘ Note
  >
  > You must use the Tomcat or TomEE runtime offered by the build pack to make JCo work correctly. You cannot bring a container of your own. This can only be achieved by using WAR deployments.

- Your application must not bundle the JCo 3.1 standalone Java archives nor the native library. JCo is already embedded properly in the build pack.
- **JCoServer** functionality cannot be used within SAP BTP.
- **Environment embedding**, such as offered by JCo standalone 3.1, is not possible. This is, however, similar to SAP NetWeaver AS Java.
- **Logon authentication** only supports user/password credentials (basic authentication) and principal propagation. See Authentication to the On-Premise System [page 1322].
- The supported set of **configuration properties** is restricted. For details, see RFC Destinations [page 1443].
- By default, a **stateful sequence of function module invocations** must be done in a single HTTP request/response cycle.
  To make a stateful sequence work across several request/response cycles, you must either set the parameter `USE_JCO_SESSIONS` to `true` in the *manifest.yml* file of your application, or do this after

deploying your application to Cloud Foundry. To do this in the Cloud Foundry environment, perform the following steps:

1. Open the cloud cockpit.
2. Navigate to your subaccount.
3. Go to *Cloud Foundry* and choose the space where your application is deployed.
4. Select the deployed application from the application list.
5. From the left-side menu, choose *User-Provided Variables*.
6. Here, you can create the parameter `USE_JCO_SESSIONS` with value **`true`**.

Back to Tasks [page 66]

Related Information

- Use Cases [page 69]
- Developing Java in the Cloud Foundry Environment
- SAP Java Connector

1.1.3.2.1 Use Cases

Find instructions for typical RFC end-to-end scenarios that use the Connectivity service and/or the Destination service.

Invoke ABAP Function Modules in On-Premise ABAP Systems [page 69]	Call a function module in an on-premise ABAP system via RFC, using a sample Web application.
Invoke ABAP Function Modules in Cloud ABAP Systems [page 94]	Call a function module in a cloud ABAP system via RFC, using a sample Web application.
Multitenancy for JCo Applications (Advanced) [page 112]	Learn about the required steps to make your JCo application tenant-aware.
Configure Principal Propagation for RFC [page 123]	Enable single sign-on (SSO) via RFC by forwarding the identity of cloud users to an on-premise system.

1.1.3.2.1.1 Invoke ABAP Function Modules in On-Premise ABAP Systems

Call a function module in an on-premise ABAP system via RFC, using a sample Web application.

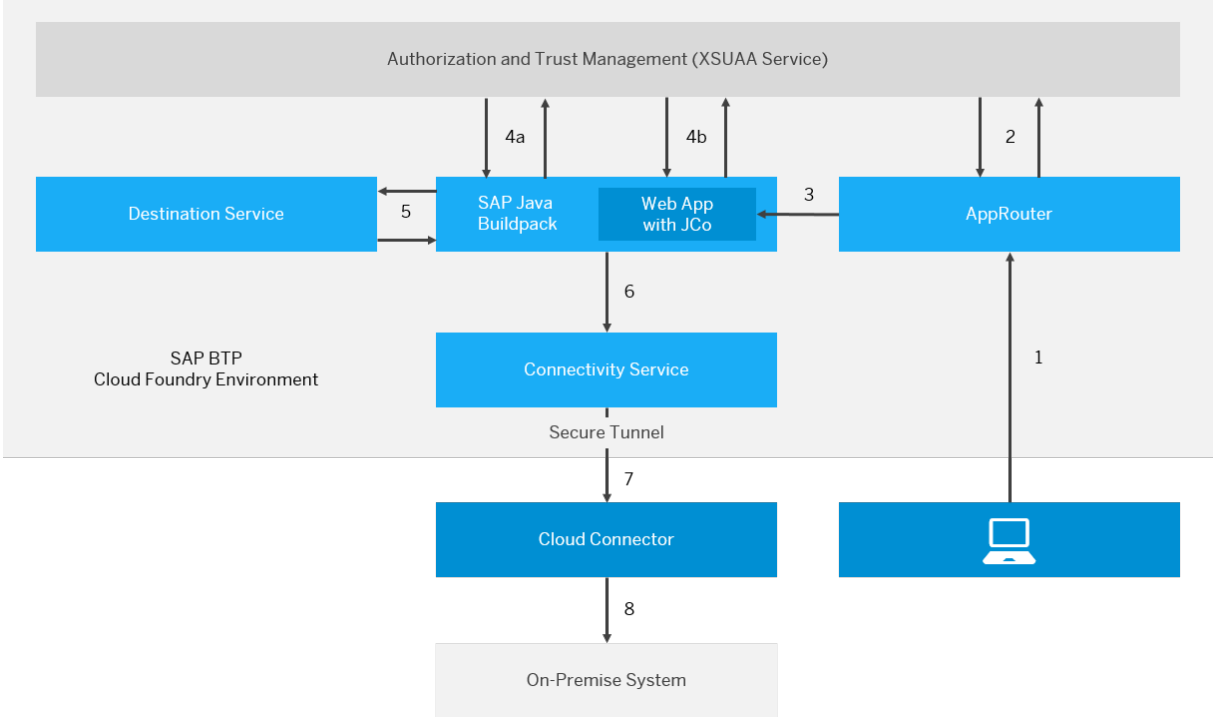
This scenario performs a remote function call to invoke an ABAP function module, by using the Connectivity service and the Destination service, as well as an application router.

Tasks

Task Type	Task
Operator and/or Developer	Scenario Overview [page 71]
	Connectivity User Roles [page 72]
	Installation Prerequisites [page 73]
	Used Values [page 72]
Developer	Develop a Sample Web Application [page 73]
Operator and/or Developer	Create and Bind Service Instances [page 78]
Developer	Deploy the Application [page 81]
Operator	Configure Roles and Trust [page 83]
Operator and/or Developer	Set Up an Application Router [page 83]
Operator	Configure the RFC Destination [page 88]
	Configure the Cloud Connector [page 89]
Operator and/or Developer	Monitoring Your Web Application [page 93] (Optional)

Scenario Overview

Control Flow for Using the Java Connector (JCo) with Basic Authentication



Process Steps:

1. Call through AppRouter (entry point for business applications).

   > **Note**
   >
   > AppRouter is only required you want to use multitenancy or perform user-specific service calls. In all other cases, JCo uses cloud-security-xsuaa-integration with `ClientCredentialFlow`.

2. Redirect to XSUAA for login. JSON Web Token (JWT1) is sent to AppRouter and cached there.
3. AppRouter calls Web app and sends JWT1 with credentials.
4. Buildpack/XSUAA interaction:
   1. Buildpack requests JWT2 to access the Destination service instance (JCo call).
   2. Buildpack requests JWT3 to access the Connectivity service instance.
5. Buildpack requests destination configuration (JWT2).
6. Buildpack sends request to the Connectivity service instance (with JWT3 and Authorization Header).
7. Connectivity service forwards request to the Cloud Connector.
8. Cloud Connector sends request to on-premise system.

Since token exchanges are handled by the buildpack, you must only create and bind the service instances, see Create and Bind Service Instances [page 78].

Back to Tasks [page 70]

Used Values

This scenario uses:

- A subaccount in region **`Europe (Frankfurt)`**, for which the API endpoint is `api.cf.eu10.hana.ondemand.com`.
- The application name **`jco-demo-<subaccount name>`**, where `<subaccount name>` is the subdomain name of the subaccount. For this example, we use **`p1234`**.

Back to Tasks [page 70]

Connectivity User Roles

Different user roles are involved in the cloud to on-premise connectivity end-to-end scenario. The particular steps for the relevant roles are described below:

IT Administrator

Sets up and configures the Cloud Connector. Scenario steps:

1. Downloads the Cloud Connector from https://tools.hana.ondemand.com/#cloud
2. Installs the Cloud Connector.
3. Establishes a TLS tunnel from the connector to an SAP BTP subaccount.
4. Configures the exposed back-end systems and resources.

Application Developer

Develops Web applications using destinations. Scenario steps:

1. Installs Eclipse IDE, the Cloud Foundry Plugin for Eclipse and the Cloud Foundry CLI.
2. Develops a Java EE application using the JCo APIs.
3. Configures connectivity destinations via the SAP BTP cockpit.
4. Deploys and tests the Java EE application on SAP BTP.

Account Operator

Deploys Web applications, creates application routers, creates and binds service instances, conducts tests. Scenario steps:

1. Obtains a ready Java EE application WAR file.
2. Deploys an application router with respective routes to the application.
3. Creates an XSUAA service instance and binds it to the router.
4. Deploys the Java EE application to an SAP BTP subaccount.
5. Creates a Connectivity service and Destination service instance, and binds them to the application.
6. Creates and manages roles and role collections.

Back to Tasks [page 70]

Installation Prerequisites

- You have downloaded the Cloud Connector installation archive from SAP Development Tools for Eclipse and connected the Cloud Connector to your subaccount.
- You have downloaded and set up your Eclipse IDE and the Eclipse Tools for Cloud Foundry .
- You have downloaded the Cloud Foundry CLI, see Tools.

Back to Tasks [page 70]

Next Steps

- Develop a Sample Web Application [page 73]
- Create and Bind Service Instances [page 78]
- Deploy the Application [page 81]
- Configure Roles and Trust [page 83]
- Set Up an Application Router [page 83]
- Configure the RFC Destination [page 88]
- Configure the Cloud Connector [page 89]
- Monitoring Your Web Application [page 93] (Optional)

Related Information

Multitenancy for JCo Applications (Advanced) [page 112]

1.1.3.2.1.1.1 Develop a Sample Web Application

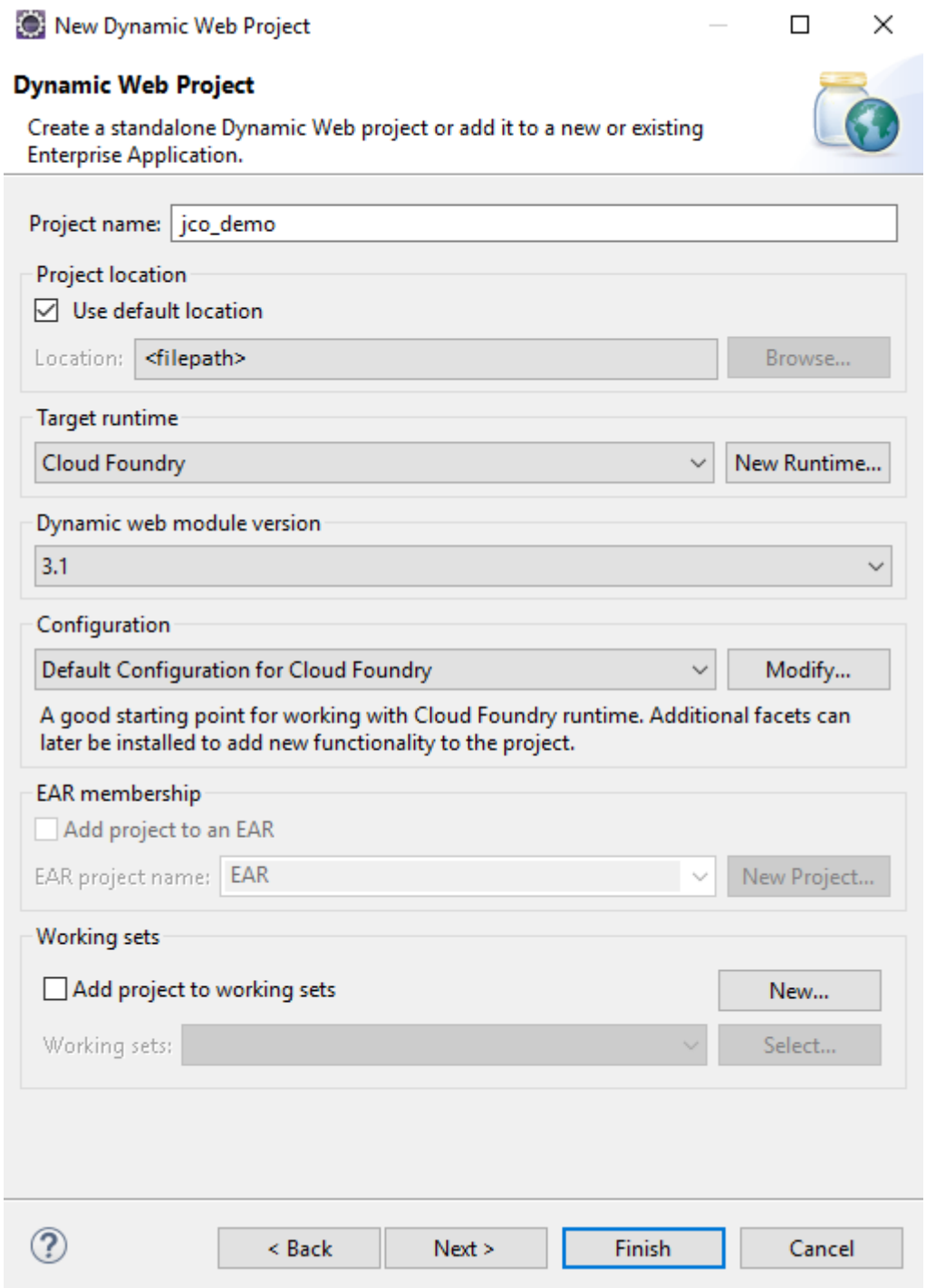
Create a Web application to call an ABAP function module via RFC.

Steps

1. Create a Dynamic Web Project [page 74]
2. Include JCo Dependencies [page 75]
3. Create a Sample Servlet [page 76]

Create a Dynamic Web Project

1. Open the *Java EE* perspective of the Eclipse IDE.
2. On the *Project Explorer* view, choose *New* > *Dynamic Web Project* in the context menu.
3. Enter **`jco_demo`** as the project name.
4. In the *Target Runtime* pane, select **`Cloud Foundry`** . If it is not yet in the list of available runtimes, choose *New Runtime* and select it from there.
5. In the *Configuration* pane, leave the default configuration.
6. Choose *Finish*.



Back to Steps [page 73]

Include JCo Dependencies

To use JCo functionality seamlessly at compile time in Eclipse, you must include the JCo dependencies into your web project. Therefore, you must convert it into a maven project.

1. In the *Project Explorer* view, right-click on the project `jco-demo` and choose *Configure* > *Convert to Maven Project*.
2. In the dialog window, leave the default settings unchanged and choose *Finish*.
3. Open the `pom.xml` file and include the following dependency:

```
<dependencies>
  <dependency>
    <groupId>com.sap.cloud</groupId>
    <artifactId>neo-java-web-api</artifactId>
    <version>[4.108.9,5.0.0)</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

Back to Steps [page 73]

Create a Sample Servlet

1. From the *jco_demo* project node, choose *New* > *Servlet* in the context menu.
2. Enter **`com.sap.demo.jco`** as the <> and **`ConnectivityRFCExample`**Java as the `<Class name>`. Choose *Next*.
3. Choose *Finish* to create the servlet and open it in the Java editor.
4. Replace the entire servlet class to make use of the JCo API. The JCo API is visible by default for cloud applications. You do not need to add it explicitly to the application class path.

Sample Code

```
package com.sap.demo.jco;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.sap.conn.jco.AbapException;
import com.sap.conn.jco.JCoDestination;
import com.sap.conn.jco.JCoDestinationManager;
import com.sap.conn.jco.JCoException;
import com.sap.conn.jco.JCoFunction;
import com.sap.conn.jco.JCoParameterList;
import com.sap.conn.jco.JCoRepository;

/**
 * Sample application that uses the Connectivity
                                    service. In particular, it is
 * making use of the capability to invoke a function module in an ABAP
system
 * via RFC
```

```
 *
 * Note: The JCo APIs are available under <code>com.sap.conn.jco</code>.
 */
@WebServlet("/ConnectivityRFCExample/*")
public class ConnectivityRFCExample extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        PrintWriter responseWriter = response.getWriter();
        try {
            // access the RFC Destination "JCoDemoSystem"
            JCoDestination destination =
JCoDestinationManager.getDestination("JCoDemoSystem");

            // make an invocation of STFC_CONNECTION in the backend;
            JCoRepository repo = destination.getRepository();
            JCoFunction stfcConnection =
repo.getFunction("STFC_CONNECTION");

            JCoParameterList imports =
stfcConnection.getImportParameterList();
            imports.setValue("REQUTEXT", "SAP BTP Connectivity runs with
JCo");
            stfcConnection.execute(destination);
            JCoParameterList exports =
stfcConnection.getExportParameterList();
            String echotext = exports.getString("ECHOTEXT");
            String resptext = exports.getString("RESPTEXT");
            response.addHeader("Content-type", "text/html");
            responseWriter.println("<html><body>");
            responseWriter.println("<h1>Executed STFC_CONNECTION in system
JCoDemoSystem</h1>");
            responseWriter.println("<p>Export parameter ECHOTEXT of
STFC_CONNECTION:<br>");
            responseWriter.println(echotext);
            responseWriter.println("<p>Export parameter RESPTEXT of
STFC_CONNECTION:<br>");
            responseWriter.println(resptext);
            responseWriter.println("</body></html>");
        } catch (AbapException ae) {
            // just for completeness: As this function module does not
have an exception
            // in its signature, this exception cannot occur. But you
should always
            // take care of AbapExceptions
        } catch (JCoException e) {
            response.addHeader("Content-type", "text/html");
            responseWriter.println("<html><body>");
            responseWriter
                    .println("<h1>Exception occurred while executing
STFC_CONNECTION in system JCoDemoSystem</h1>");
            responseWriter.println("<pre>");
            e.printStackTrace(responseWriter);
            responseWriter.println("</pre>");
            responseWriter.println("</body></html>");
        }
    }
}
```

5. Save the Java editor and make sure that the project compiles without errors.

Back to Steps [page 73]

Next Steps

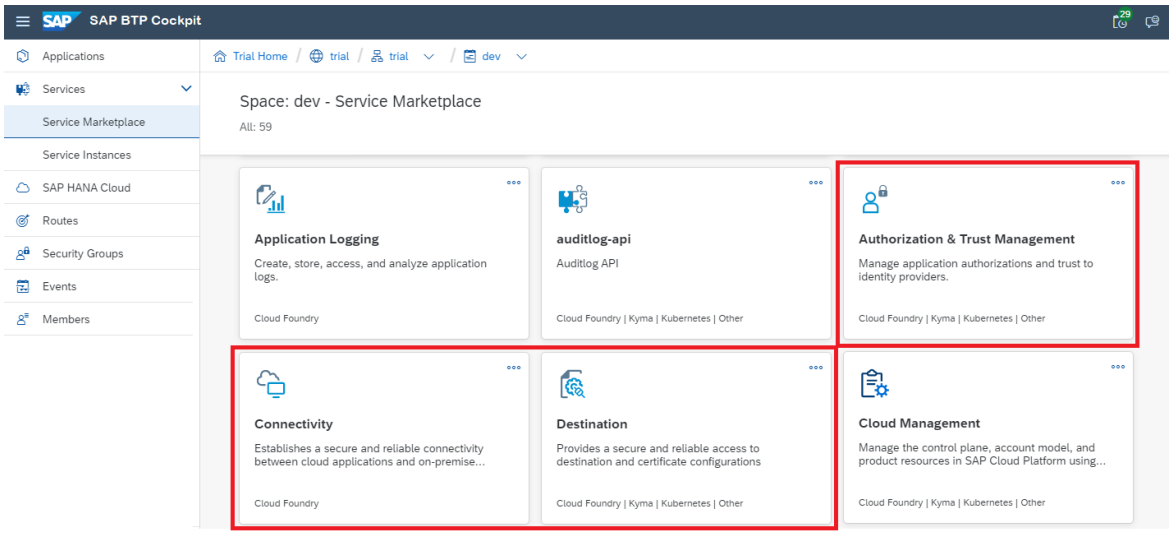
- Create and Bind Service Instances [page 78]
- Deploy the Application [page 81]
- Configure Roles and Trust [page 83]
- Set Up an Application Router [page 83]
- Configure the RFC Destination [page 88]
- Configure the Cloud Connector [page 89]
- Monitoring Your Web Application [page 93] (Optional)

1.1.3.2.1.1.2 Create and Bind Service Instances

You must create and bind several service instances, before you can use your application.

Procedure

1. Logon to the cloud cockpit and choose your subaccount.
2. Choose the space where you want to deploy your demo application.
3. Choose *Service Marketplace* and find these 3 services:
   - *Connectivity*
   - *Destination*
   - *Authorization & Trust Management* (XSUAA)

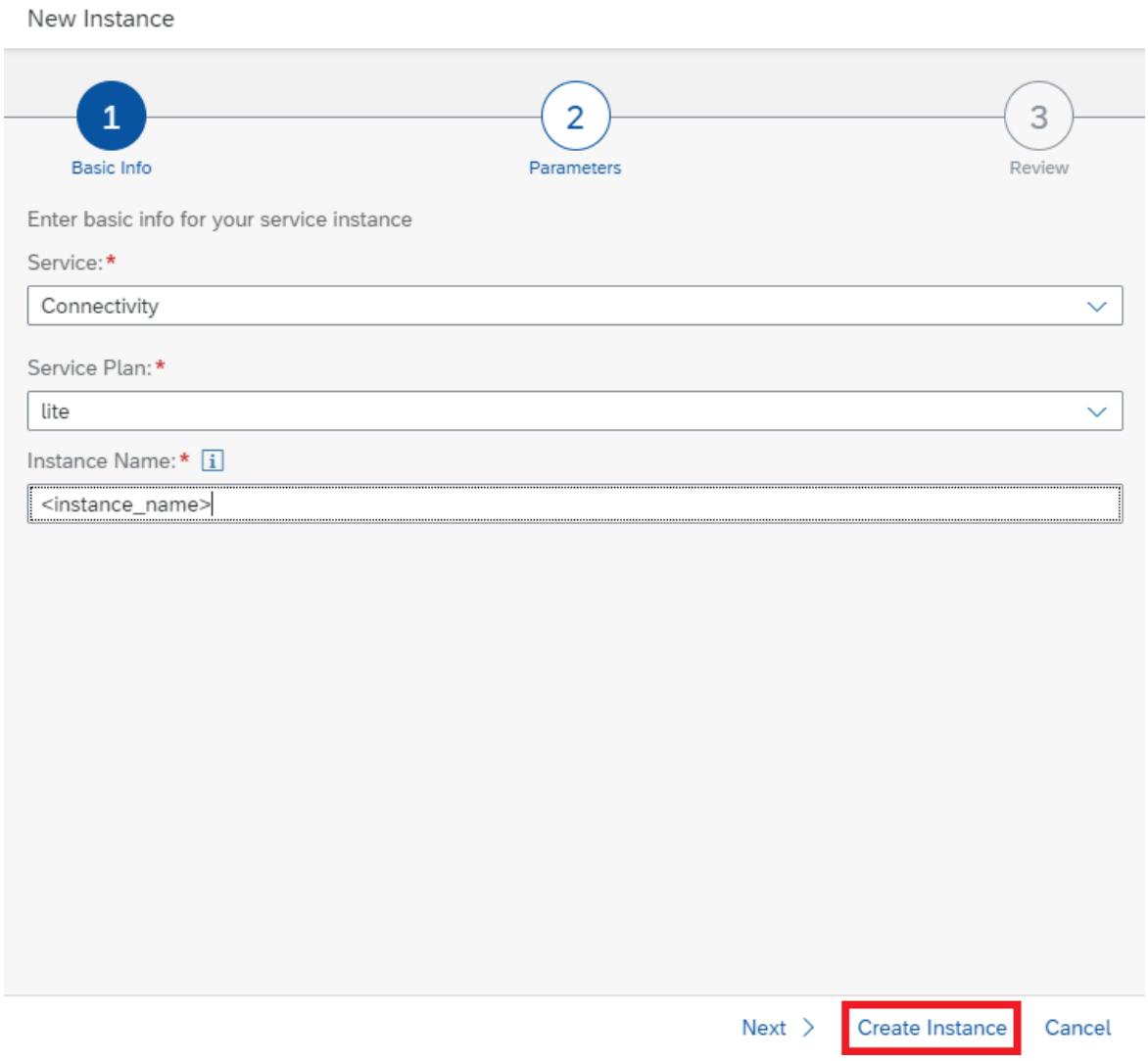


4. Create and bind a service instance for each of these services.
   - Connectivity service [page 79]
   - Destination service [page 79]

- Authorization & Trust Management (XSUAA service) [page 80]

Connectivity Service

1. Choose *Connectivity* > *Create Instance* .
2. Insert an instance name (for example, **`connectivity_jco`**) and choose *Create Instance* .



Back to Procedure [page 78]

Destination Service

1. Choose *Destination* > *Create Instance* .
2. Insert an instance name (for example, **`destination_jco`**) and choose *Create Instance* .

Back to Procedure [page 78]

Authorization & Trust Management (XSUAA Service)

1. Choose *Authorization & Trust Management* > *Create Instance* .
2. Select `<Service Plan>` **`application`**.
3. Enter an `<Instance Name>` and choose *Next*.

> **Note**
>
> The instance name must match the one defined in the manifest file.

4. In the next tab *Parameters*, insert the following as a JSON file:

> **Sample Code**

```
{
    "xsappname" : "jco-demo-p1234",
    "tenant-mode": "dedicated",
    "scopes": [
        {
            "name": "$XSAPPNAME.all",
            "description": "all"
        }
    ],
    "role-templates": [
        {
            "name": "all",
            "description": "all",
            "scope-references": [
                "$XSAPPNAME.all"
            ]
        }
    ]
}
```

5. Go to tab *Review* and choose *Create Instance*.

Back to Procedure [page 78]

Next Steps

- Deploy the Application [page 81]
- Configure Roles and Trust [page 83]
- Set Up an Application Router [page 83]

- Configure the RFC Destination [page 88]
- Configure the Cloud Connector [page 89]
- Monitoring Your Web Application [page 93] (Optional)

1.1.3.2.1.1.3 Deploy the Application

Deploy your Cloud Foundry application to call an ABAP function module via RFC.

Prerequisites

You have created and bound the required service instances, see Create and Bind Service Instances [page 78].

Procedure

1. To deploy your Web application, you can use the following two alternative procedures:
   - Deploying from the Eclipse IDE
   - Deploying from the CLI, see Developing Java in the Cloud Foundry Environment
2. In the following, we publish it with the CLI.
3. To do this, create a `manifest.yml` file. The key parameter is `USE_JCO: true`, which must be set to include JCo into the buildpack during deployment.

   > **Note**
   >
   > JCo supports the usage of X.509 secrets for communication with the Destination/Connectivity service. If you want to use it, you must specify the binding accordingly.
   >
   > For more information, see Binding Parameters of SAP Authorization and Trust Management Service.

   manifest.yml

   > **Sample Code**

```
---
applications:

- name: jco-demo-p1234
  buildpacks:
  - sap_java_buildpack
  env:
    USE_JCO: true
    # This is necessary only if more than one instance is bound
    xsuaa_connectivity_instance_name: "xsuaa_jco"
    connectivity_instance_name: "connectivity_jco"
    destination_instance_name: "destination_jco"
```

```
services:
  - xsuaa_jco
  - connectivity_jco
  - destination_jco
```

> ⚠ Caution
>
> The client libraries (java-security, spring-xsuaa, and container security api for node.js as of version 3.0.6) have been updated. When using these libraries, setting the parameter `SAP_JWT_TRUST_ACL` has become obsolete. This update comes with a change regarding scopes:
>
> - For a business application A calling an application B, it is now mandatory that application B grants at least one scope to the calling business application A.
> - Business application A must accept these granted scopes or authorities as part of the application security descriptor.
>
> You can grant scopes using the `xs-security.json` file.
>
> For more information, see Application Security Descriptor Configuration Syntax, specifically the sections *Referencing the Application* and *Authorities*.

> ⓘ Note
>
> If you have more than one instance of those three services bound to your application, you must specify which one JCo should use with the respective env parameters:
>
> - `xsuaa_connectivity_instance_name`
> - `connectivity_instance_name`
> - `destination_instance_name`.

4. In Eclipse, right-click on the project and navigate to *Export* > *WAR file*.
5. Choose a destination by pressing the *Browse...* button next to the `manifest.yml` you created before, for example as `jcodemo.war`.
6. Leave the other default settings unchanged and choose *Finish* to export the WAR file.
7. Perform a CLI login via `cf login -a api.cf.eu10.hana.ondemand.com -u <your_email_address>` (password, org and space are prompted after a successful login).
8. Push the application with `cf push -f manifest.yml -p jcodemo.war`.
9. Now, the application should be deployed successfully.

Next Steps

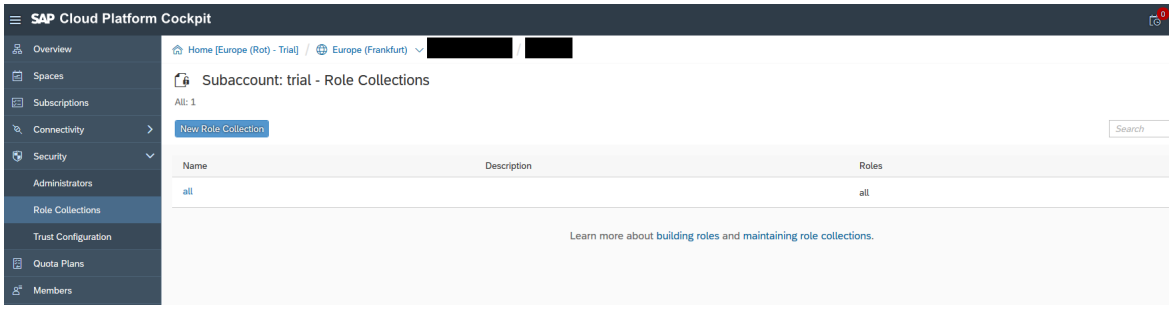
- Configure Roles and Trust [page 83]
- Set Up an Application Router [page 83]
- Configure the RFC Destination [page 88]
- Configure the Cloud Connector [page 89]

- Monitoring Your Web Application [page 93] (Optional)

1.1.3.2.1.1.4 Configure Roles and Trust

Configure a role that enables your user to access your Web application.

1. From the subaccount menu, choose *Trust Configuration*.
2. If you don't have a trust configuration, follow the steps in Manually Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services.
3. To add and assign roles, choose *Security* > *Role Collections* from the subaccount menu:



4. Create a *New Role Collection* with the name **`all`**.
5. Select *all* > *Edit* and add yourself as user.
6. Choose *Save*.

Next Steps

- Set Up an Application Router [page 83]
- Configure the RFC Destination [page 88]
- Configure the Cloud Connector [page 89]
- Monitoring Your Web Application [page 93] (Optional)

Related Information

Working with Role Collections

1.1.3.2.1.1.5 Set Up an Application Router

For authentication purposes, configure and deploy an application router for your test application.

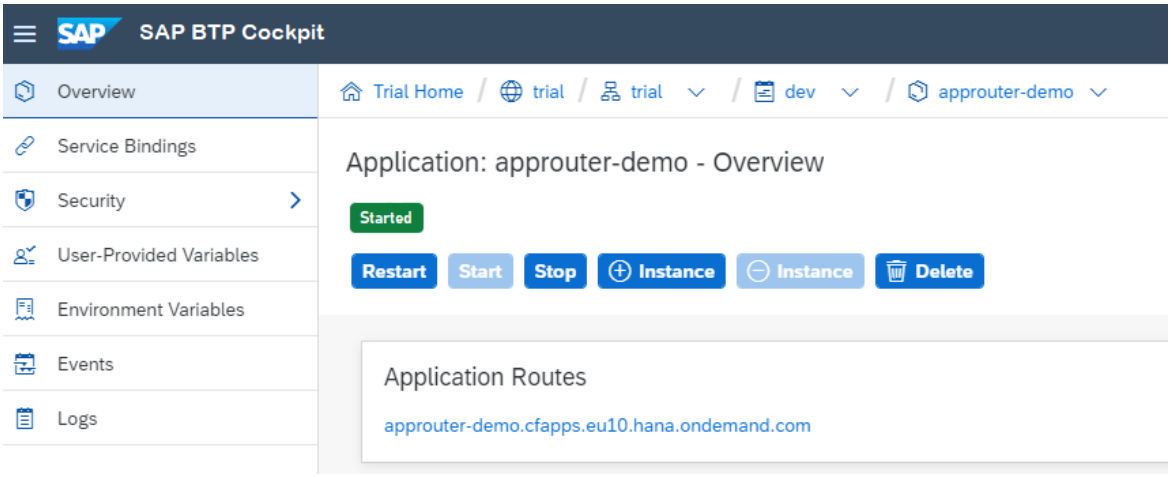
> ### Note
>
> AppRouter is only required if you want to use multitenancy or perform user-specific service calls. In all other cases, JCo uses cloud-security-xsuaa-integration with `ClientCredentialFlow`.

1. To set up an application router, follow the steps in Application Router or use the demo file *approuter.zip* (download).
2. For deployment, you need a manifest file, similar to this one:

> ### Sample Code
>
> ```
> ---
> applications:
>
> - name: approuter-jco-demo-p1234
>   path: ./
>   buildpacks:
>   - nodejs_buildpack
>   memory: 120M
>   routes:
>   - route: approuter-jco-demo-p1234.cfapps.eu10.hana.ondemand.com
>   env:
>     NODE_TLS_REJECT_UNAUTHORIZED: 0
>     destinations: >
>       [
>          {"name":"dest-to-example", "url" :"https://jco-
> demo-p1234.cfapps.eu10.hana.ondemand.com/ConnectivityRFCExample",
> "forwardAuthToken": true }
>       ]
>   services:
>     - xsuaa_jco
> ```

> ### Note
>
> - The routes and destination URLs need to fit your test application.
> - In this example, we already bound our XSUAA instance to the application router. Alternatively, you could also do this via the cloud cockpit.

3. Push the approuter with `cf push -f manifest.yml -p approuter.zip`.
4. To navigate to the `approuter` application in the cloud cockpit, choose *<your_space>* > *Applications* > *<your application>* > *Overview*.



5. When choosing the application route, you are requested to login. Provide the credentials known by the IdP you configured in *Roles & Trust*.
6. After successful login, you are routed to the test application which is then executed.
7. If the application issues an exception, saying that the `JCoDemoSystem` destination has not yet been specified, you must configure the `JCoDemoSystem` destination first.

```
Exception occurred while executing STFC_CONNECTION in system JCoDemoSystem
com.sap.conn.jco.JCoException: (106) JCO_ERROR_RESOURCE: Destination
JCoDemoSystem does not exist
    at
com.sap.conn.jco.rt.DefaultDestinationManager.update(DefaultDestinationManager
.java:223)
    at
com.sap.conn.jco.rt.DefaultDestinationManager.searchDestination(DefaultDestina
tionManager.java:377)
    at
com.sap.conn.jco.rt.DefaultDestinationManager.getDestinationInstance(DefaultDe
stinationManager.java:96)
    at
com.sap.conn.jco.JCoDestinationManager.getDestination(JCoDestinationManager.ja
va:52)
    at
com.sap.demo.jco.ConnectivityRFCExample.doGet(ConnectivityRFCExample.java:47)
        ..... (cut rest of the call stack)
```

> **Note**
>
> Make sure you **don't** include this dependency
>
> ```
> <dependency>
>     <groupId>com.sap.cloud.security</groupId>
>     <artifactId>java-security</artifactId>
> </dependency>
> ```
>
> or any of its dependencies such as `java-api` with scope `compile` directly or transitively with any other jar.

Calling JCo APIs from Newly Created Threads

If you are using an Application Router and it is mandatory for you to call JCo APIs from a different thread than the one which is executing your servlet function, make sure the thread local information of the cloud-security-xsuaa-integration API , used by JCo internally, is set again within your newly created thread.

To do this, add the following dependency to your project:

```
<dependency>
    <groupId>com.sap.cloud.security</groupId>
    <artifactId>java-api</artifactId>
    <version>2.7.7</version>
    <scope>provided</scope>
</dependency>
```

Adjust your code from the step Develop a Sample Web Application [page 73] in the following way:

Sample Code

```
package com.sap.demo.jco;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import com.sap.cloud.security.token.SecurityContext;
import com.sap.cloud.security.token.Token;
import com.sap.conn.jco.AbapException;
import com.sap.conn.jco.JCoDestination;
import com.sap.conn.jco.JCoDestinationManager;
import com.sap.conn.jco.JCoException;
import com.sap.conn.jco.JCoFunction;
import com.sap.conn.jco.JCoParameterList;
import com.sap.conn.jco.JCoRepository;
/**
 *
 * Sample application that uses the connectivity service. In particular, it is
 * making use of the capability to invoke a function module in an ABAP system
 * via RFC
 *
 * Note: The JCo APIs are available under <code>com.sap.conn.jco</code>.
 */
@WebServlet("/ConnectivityRFCExample/*")
public class ConnectivityRFCExample extends HttpServlet {
    private static final long serialVersionUID = 1L;
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        PrintWriter responseWriter = response.getWriter();
        // access the token from the thread which is executing the servlet
        Token token = SecurityContext.getToken();
        Thread runThread = new Thread(() -> {
            // set the information in the newly created thread
            SecurityContext.setToken(token);
            try {
                // access the RFC Destination "JCoDemoSystem"
                JCoDestination destination =
JCoDestinationManager.getDestination("JCoDemoSystem_Normal");
                // make an invocation of STFC_CONNECTION in the backend
```

```
                JCoRepository repo = destination.getRepository();
                JCoFunction stfcConnection =
repo.getFunction("STFC_CONNECTION");
                JCoParameterList imports =
stfcConnection.getImportParameterList();
                imports.setValue("REQUTEXT", "SAP BTP Connectivity runs with
JCo");
                stfcConnection.execute(destination);
                JCoParameterList exports =
stfcConnection.getExportParameterList();
                String echotext = exports.getString("ECHOTEXT");
                String resptext = exports.getString("RESPTEXT");
                response.addHeader("Content-type", "text/html");
                responseWriter.println("<html><body>");
                responseWriter.println("<h1>Executed STFC_CONNECTION in
system JCoDemoSystem</h1>");
                responseWriter.println("<p>Export parameter ECHOTEXT of
STFC_CONNECTION:<br>");
                responseWriter.println(echotext);
                responseWriter.println("<p>Export parameter RESPTEXT of
STFC_CONNECTION:<br>");
                responseWriter.println(resptext);
                responseWriter.println("</body></html>");
            } catch (AbapException ae) {
                // just for completeness: As this function module does not
have an exception
                // in its signature, this exception cannot occur. But you
should always
                // take care of AbapExceptions
            } catch (JCoException e) {
                response.addHeader("Content-type", "text/html");
                responseWriter.println("<html><body>");
                responseWriter
                        .println("<h1>Exception occurred while executing
STFC_CONNECTION in system JCoDemoSystem</h1>");
                responseWriter.println("<pre>");
                e.printStackTrace(responseWriter);
                responseWriter.println("</pre>");
                responseWriter.println("</body></html>");
            } finally {
                // after execution clear the token again
                SecurityContext.clearToken();
            }
        });
        runThread.start();
        // wait to be finished
        try {
            runThread.join();
        } catch (InterruptedException e) {
            e.printStackTrace(responseWriter);
        }
    }
}
```

Note

If you want to use thread pools, make sure that in your thread pool implementation this information is set correctly in the thread which is about to be (re)used, and removed as soon as the thread is put back into the pool.

Next Steps

- Configure the RFC Destination [page 88]
- Configure the Cloud Connector [page 89]
- Monitoring Your Web Application [page 93] (Optional)

1.1.3.2.1.1.6 Configure the RFC Destination

Configure an RFC destination on SAP BTP that you can use in your Web application to call the on-premise ABAP system.

To configure the destination, you must use a virtual application server host name (**`abapserver.hana.cloud`**) and a virtual system number (**`42`**) that you will expose later in the Cloud Connector. Alternatively, you could use a load balancing configuration with a message server host and a system ID.

1. Create a `.properties` file with the following settings:

```
Name=JCoDemoSystem
 Type=RFC
 jco.client.ashost=abapserver.hana.cloud
 jco.client.sysnr=42
 jco.destination.proxy_type=OnPremise
 jco.client.user=<DEMOUSER>
 jco.client.passwd=<Password>
 jco.client.client=000
 jco.client.lang=EN
 jco.destination.pool_capacity=5
```

2. Go to your subaccount in the cloud cockpit.
   1. From the subaccount menu, choose *Connectivity* > *Destinations* > *Import Destination* and upload this file.
   2. Alternatively, you can create a destination for the service instance **`destination_jco`** to make it visible only for this instance. To do this, go to *<your space>* > *Services* > *Service Instances* > *<your destination_instance>* and choose *Destinations*.
3. Call again the URL that references the cloud application in the Web browser. The Web application should now return a different exception:

```
Exception occurred while executing STFC_CONNECTION in system JCoDemoSystem
com.sap.conn.jco.JCoException: (102) JCO_ERROR_COMMUNICATION: Opening
connection to backend failed: Opening connection to
abapserver.hana.cloud:sapgw42 denied. Expose the system in your Cloud
Connector in case it was a valid request.
    at
com.sap.conn.jco.rt.MiddlewareJavaRfc.generateJCoException(MiddlewareJavaRfc.j
ava:487)
    at
com.sap.conn.jco.rt.MiddlewareJavaRfc$JavaRfcClient.connect(MiddlewareJavaRfc.
java:1216)
    at com.sap.conn.jco.rt.ClientConnection.connect(ClientConnection.java:700)
    at
com.sap.conn.jco.rt.RepositoryConnection.connect(RepositoryConnection.java:72)
    at com.sap.conn.jco.rt.PoolingFactory.init(PoolingFactory.java:113)
```

```
    at
com.sap.conn.jco.rt.ConnectionManager.createFactory(ConnectionManager.java:426
)
    at
com.sap.conn.jco.rt.DefaultConnectionManager.createFactory(DefaultConnectionMa
nager.java:46)
    at
com.sap.conn.jco.rt.ConnectionManager.getFactory(ConnectionManager.java:376)
    at
com.sap.conn.jco.rt.RfcDestination.getSystemID(RfcDestination.java:1101)
        ..... (cut rest of the call stack)
```

4. This means that the Cloud Connector denied opening a connection to this system. As a next step, you must configure the system in your installed Cloud Connector.

Next Steps

- Configure the Cloud Connector [page 89]
- Monitoring Your Web Application [page 93] (Optional)

Related Information

Target System Configuration [page 1452]

1.1.3.2.1.1.7 Configure the Cloud Connector

Configure the system mapping and the function module in the Cloud Connector.

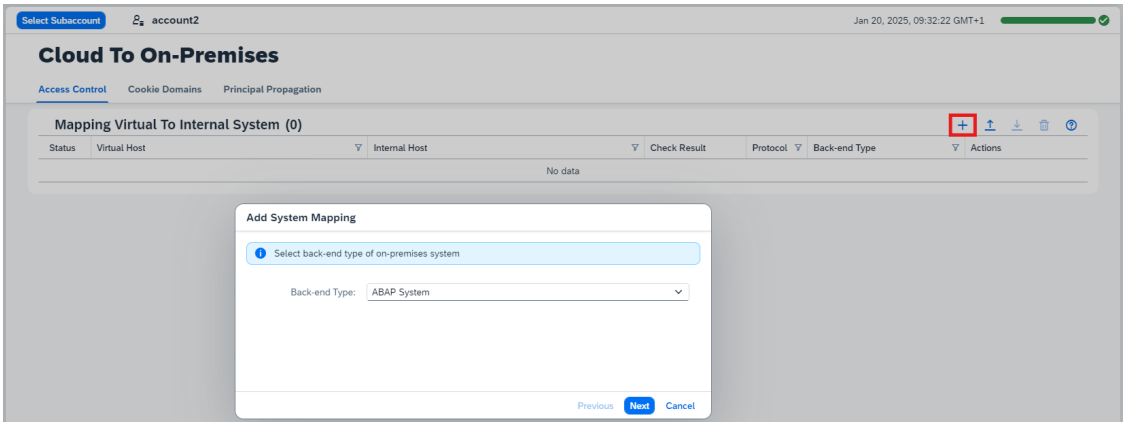
Steps

1. Configure Host Mapping [page 89]
2. Configure the Function Module [page 92]

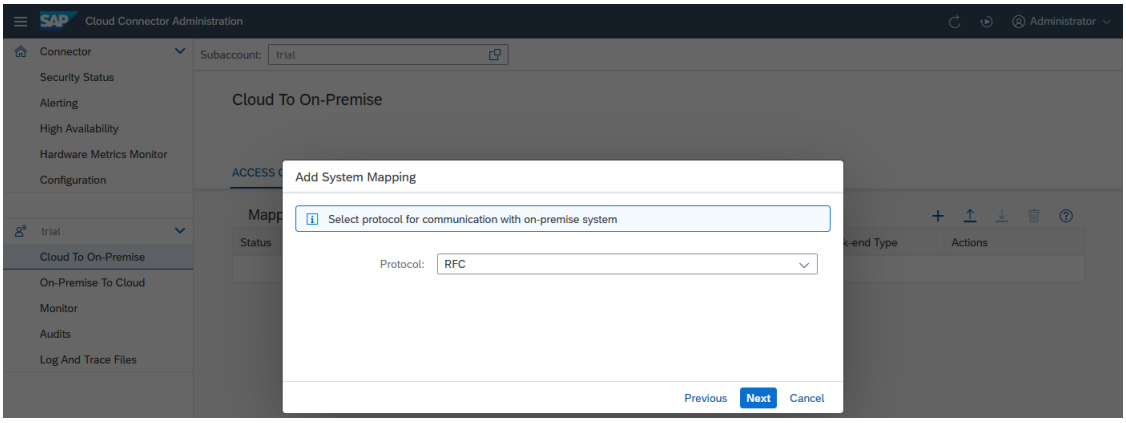
Configure Host Mapping

The Cloud Connector only allows access to trusted backend systems. To configure this, follow the steps below:

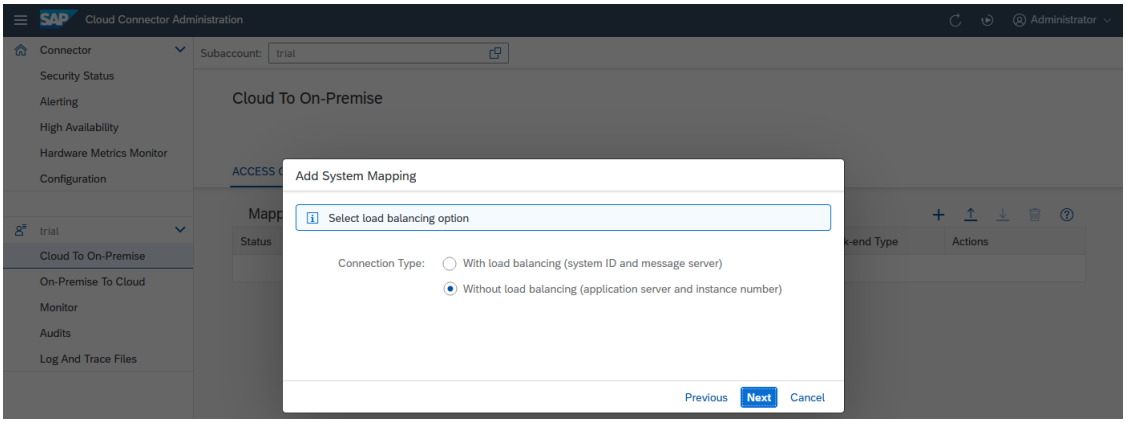
1. Optional: In the Cloud Connector administration UI, you can check under *Audits* whether access has been denied:
   ```
   Denying access for user DEMOUSER to system abapserver.hana.cloud:sapgw42
   [connectionId=-1547299395]
   ```
2. In the Cloud Connector administration UI, choose *Cloud To On-Premise* from your *Subaccount* menu, tab *Access Control*.
3. In section *Mapping Virtual To Internal System* choose *Add* to define a new system.
   1. For *Backend Type*, select **`ABAP System`** and choose *Next*.



   2. For *Protocol*, select **RFC** and choose *Next*.

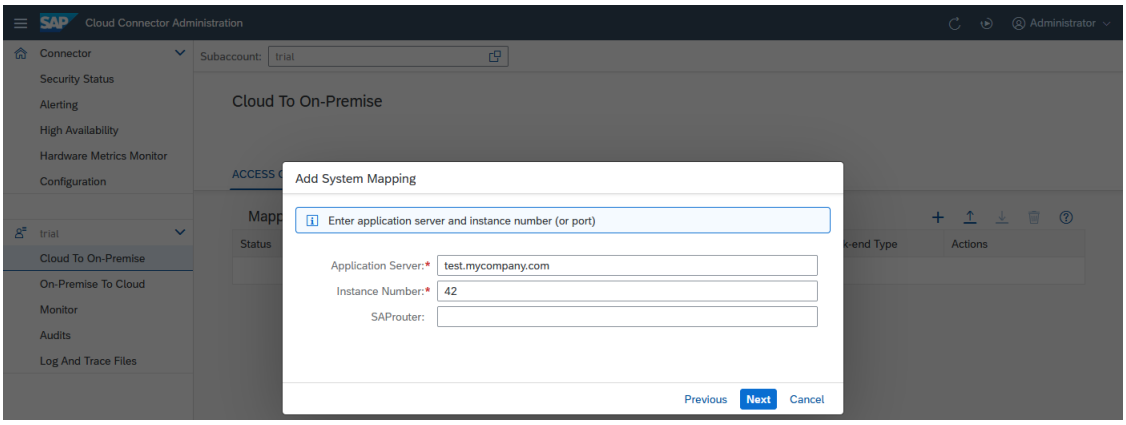


   3. Choose option *Without load balancing*.



   4. Enter application server and instance number. The *Application Server* entry must be the physical host name of the machine on which the ABAP application server is running. Choose *Next*.

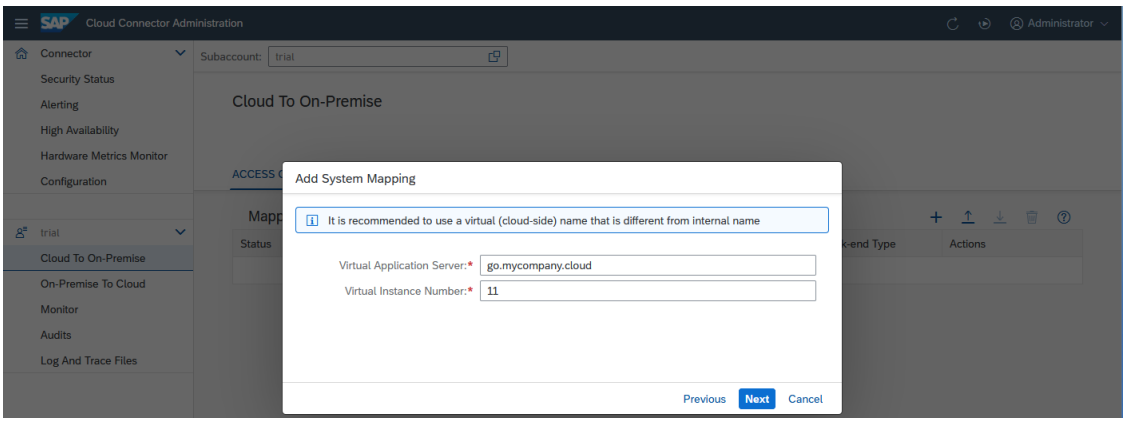
Example:



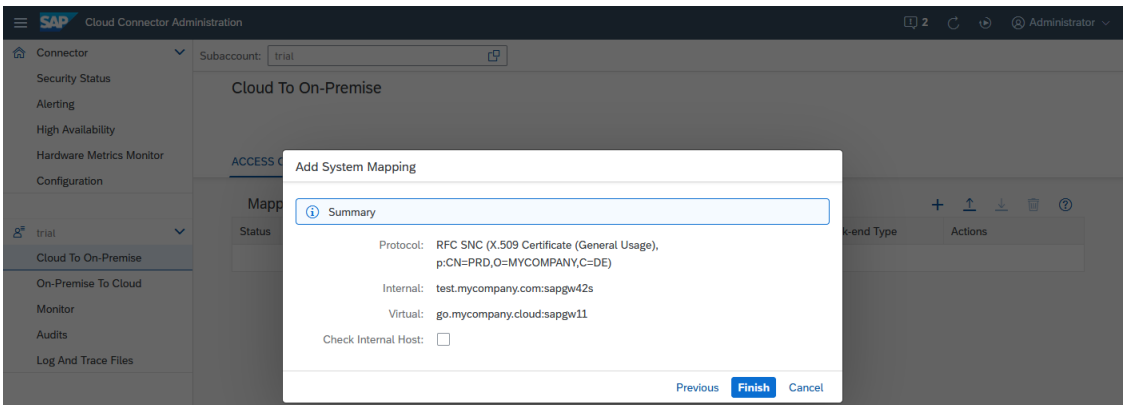
5. Enter server and instance number for virtual mapping.

> ### Note
>
> The values must match with the ones of the destination configuration in the cloud cockpit.

Example:



6. Summary (example):



4. Call again the URL that references the cloud application in the Web browser. The application should now throw a different exception:

```
om.sap.conn.jco.JCoException: (102) JCO_ERROR_COMMUNICATION:  Partner
signaled an error: Access denied for STFC_CONNECTION on
```

```
abapserver.hana.cloud:sapgw42. Expose the function module in your Cloud
Connector in case it was a valid request.
    at
com.sap.conn.jco.rt.MiddlewareJavaRfc.generateJCoException(MiddlewareJavaRfc.j
ava:632)
    at
com.sap.conn.jco.rt.MiddlewareJavaRfc$JavaRfcClient.execute(MiddlewareJavaRfc.
java:1764)
    at
com.sap.conn.jco.rt.ClientConnection.execute(ClientConnection.java:1110)
    at com.sap.conn.jco.rt.ClientConnection.execute(ClientConnection.java:943)
    at com.sap.conn.jco.rt.RfcDestination.execute(RfcDestination.java:1307)
    at com.sap.conn.jco.rt.RfcDestination.execute(RfcDestination.java:1278)
    at com.sap.conn.jco.rt.AbapFunction.execute(AbapFunction.java:295)
    at
com.sap.demo.jco.ConnectivityRFCExample.doGet(ConnectivityRFCExample.java:55)
        ..... (cut rest of the call stack)
```

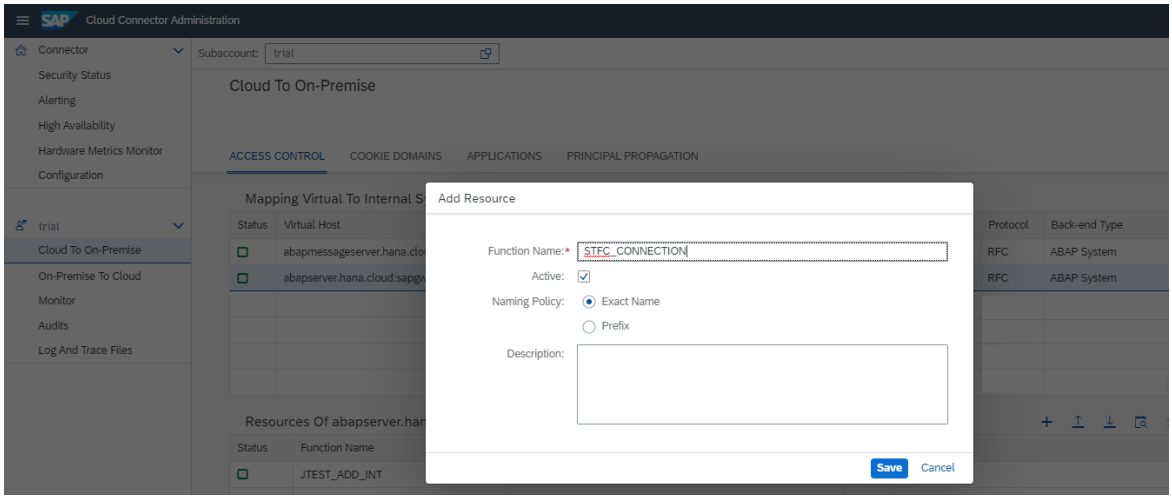
5. This means that the Cloud Connector denied invoking STFC_CONNECTION in this system. As a final step, you must provide access to this function module.

Back to Steps [page 89]

Configure the Function Module

The Cloud Connector only allows access to explicitly allowed resources (which, in an RFC scenario, are defined on the basis of function module names). To configure the function module, follow the steps below:

1. Optional: In the Cloud Connector administration UI, you can check under *Monitor* > *Audit* whether access has been denied:
   `Denying access for user DEMOUSER to resource STFC_CONNECTION on system abapserver.hana.cloud:sapgw42 [connectionId=609399452]`
2. In the Cloud Connector administration UI, choose again *Cloud To On-Premise* from your *Subaccount* menu, and go to tab *Access Control*.
3. For the specified internal system referring to *abapserver.hana.cloud*, add a new resource. To do this, select the system in the table.
4. Add a new function name under the list of exposed resources. In section *Resources Accessible On abapserver.hana.cloud:sapgw42*, choose the *Add* button and specify STFC_CONNECTION as accessible resource, as shown in the screenshot below. Make sure that you have selected the *Exact Name* option to only expose this specific function module.



5. Call again the URL that references the cloud application in the Web browser. The application should now return a message showing the export parameters of the function module.

See also Configure Access Control (RFC) [page 1752].

Back to Steps [page 89]

Next Step (Optional)

- Monitoring Your Web Application [page 93]

1.1.3.2.1.1.8 Monitoring Your Web Application

Monitor the state and logs of your Web application deployed on SAP BTP, using the Application Logging service.

For this purpose, create an instance of the Application Logging service (as you did for the Destination and Connectivity service) and bind it to your application, see Create and Bind Service Instances [page 78].

To activate JCo logging, set the following property in the *env* section of your manifest file:

```
SET_LOGGING_LEVEL: '{com.sap.conn.jco: INFO, com.sap.core.connectivity.jco:
INFO}'
```

Now you can see and open the logs in the cloud cockpit or in the Kibana Dashboard in the tab *Logs*, if you are within your application.

For error analysis, you can activate more detailed JCo debug tracing:

```
SET_LOGGING_LEVEL: '{com.sap.conn.jco: DEBUG, com.sap.core.connectivity.jco:
DEBUG}'
```

To include other relevant components for debug tracing, set:

```
SET_LOGGING_LEVEL: '{com.sap.conn.jco: DEBUG, com.sap.core.connectivity.jco:
DEBUG, com.sap.core.connectivity.apiext: DEBUG, com.sap.cloud.security: DEBUG,
com.sap.xs.security: DEBUG, com.sap.xs.env: DEBUG}'
```

1.1.3.2.1.2 Invoke ABAP Function Modules in Cloud ABAP Systems

Call a function module in a cloud ABAP system via RFC, using a sample Web application.

This scenario performs a remote function call to invoke an ABAP function module, by using the Destination service in the Cloud Foundry environment, as well as a Cloud Foundry application router.

> ⓘ Note
>
> For more information on invoking ABAP function modules via WebSocket RFC in SAP BTP, ABAP environment, see tutorial WebSocket RFC to Cloud.

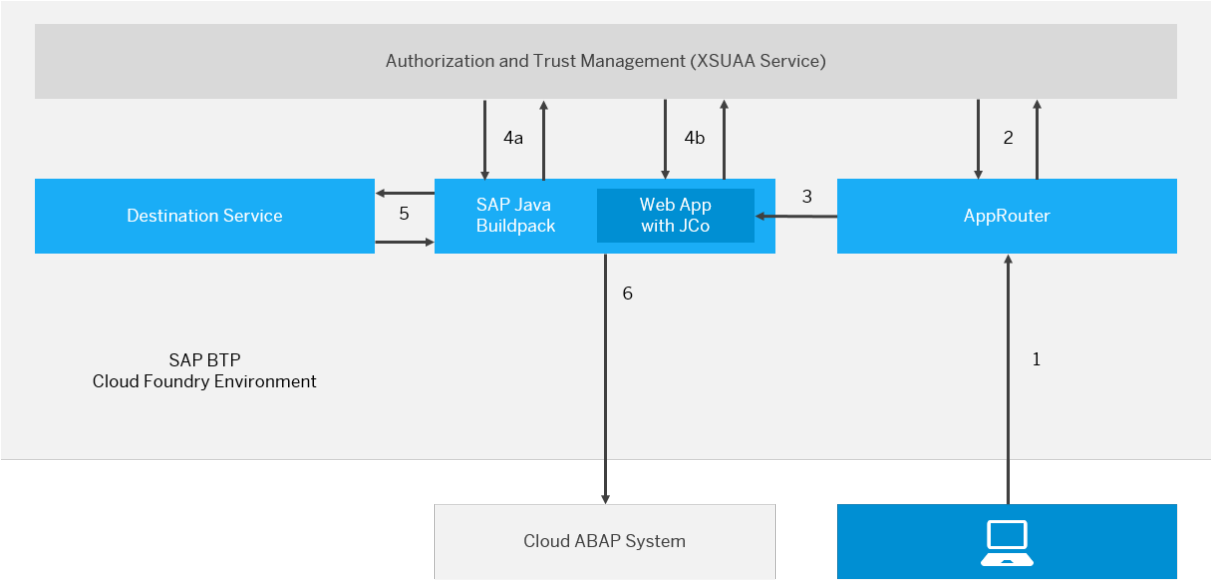
Tasks

Task Type	Task
Operator and/or Developer	Scenario Overview [page 95]
	Used Values [page 96]
	Connectivity User Roles [page 97]
	Installation Prerequisites [page 97]
Developer	Develop a Sample Web Application [page 98]

Task Type	Task
Operator and/or Developer	Create and Bind Service Instances [page 102]
Developer	Deploy the Application [page 104]
Operator	Configure Roles and Trust [page 106]
Operator and/or Developer	Set Up an Application Router [page 107]
Operator	Configure the RFC Destination [page 111]
Operator and/or Developer	Monitoring Your Web Application [page 112] (Optional)

Scenario Overview

Control Flow for Using the Java Connector (JCo) with Basic Authentication



Process Steps:

1. Call through AppRouter (entry point for business applications).

   > ### Note
   >
   > AppRouter is only required you want to use multitenancy or perform user-specific service calls. In all other cases, JCo uses cloud-security-xsuaa-integration with `ClientCredentialFlow`.

2. Redirect to XSUAA for login. JSON Web Token (JWT1) is sent to AppRouter and cached there.
3. AppRouter calls Web app and sends JWT1 with credentials.
4. Buildpack/XSUAA interaction: Buildpack requests JWT2 to access the Destination service instance (JCo call).
5. Buildpack requests destination configuration (JWT2).
6. Buildpack sends request to the cloud ABAP system (with JWT2 and Authorization Header).

Since token exchanges are handled by the buildpack, you must only create and bind the service instances, see Create and Bind Service Instances [page 78].

Back to Tasks [page 94]

Used Values

This scenario uses:

- A subaccount in region **`Europe (Frankfurt)`**, for which the API endpoint is `api.cf.eu10.hana.ondemand.com`.
- The application name **`jco-demo-<subaccount name>`**, where `<subaccount name>` is the subdomain name of the subaccount. For this example, we use **`p1234`**.

Back to Tasks [page 94]

Connectivity User Roles

Different user roles are involved in the cloud-to-cloud connectivity scenario. The particular steps for the relevant roles are described below:

Application Developer

Develops Web applications using destinations. Scenario steps:

1. Installs Eclipse IDE, the Cloud Foundry Plugin for Eclipse and the Cloud Foundry CLI.
2. Develops a Java EE application using the JCo APIs.
3. Configures connectivity destinations via the SAP BTP cockpit.
4. Deploys and tests the Java EE application on SAP BTP.

Account Operator

Deploys Web applications, creates application routers, creates and binds service instances, conducts tests. Scenario steps:

1. Obtains a ready Java EE application WAR file.
2. Deploys an application router with respective routes to the application.
3. Creates an XSUAA service instance and binds it to the router.
4. Deploys the Java EE application to an SAP BTP subaccount.
5. Creates a Destination service instance, and binds it to the application.
6. Creates and manages roles and role collections.

Back to Tasks [page 94]

Installation Prerequisites

- You have downloaded and set up your Eclipse IDE and the Eclipse Tools for Cloud Foundry .
- You have downloaded the Cloud Foundry CLI, see Tools.

Back to Tasks [page 94]

Next Steps

- Develop a Sample Web Application [page 98]
- Create and Bind Service Instances [page 102]

- Deploy the Application [page 104]
- Configure Roles and Trust [page 106]
- Set Up an Application Router [page 107]
- Configure the RFC Destination [page 111]
- Monitoring Your Web Application [page 112] (Optional)

Related Information

Multitenancy for JCo Applications (Advanced) [page 112]

1.1.3.2.1.2.1 Develop a Sample Web Application

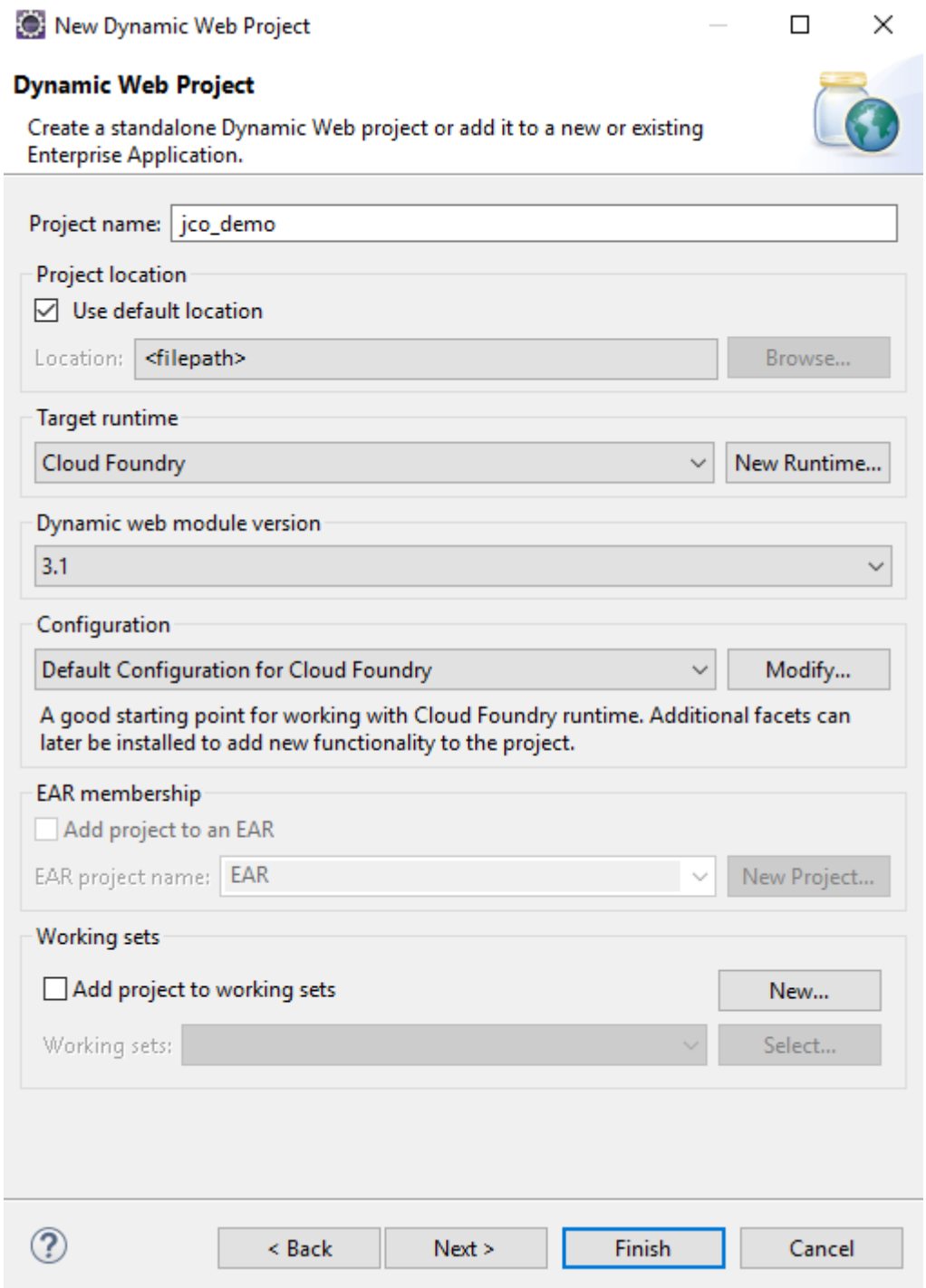
Create a Web application to call an ABAP function module via RFC.

Steps

1. Create a Dynamic Web Project [page 74]
2. Include JCo Dependencies [page 75]
3. Create a Sample Servlet [page 76]

Create a Dynamic Web Project

1. Open the *Java EE* perspective of the Eclipse IDE.
2. On the *Project Explorer* view, choose *New* > *Dynamic Web Project* in the context menu.
3. Enter **`jco_demo`** as the project name.
4. In the *Target Runtime* pane, select **`Cloud Foundry`** . If it is not yet in the list of available runtimes, choose *New Runtime* and select it from there.
5. In the *Configuration* pane, leave the default configuration.
6. Choose *Finish*.



Back to Steps [page 73]

Include JCo Dependencies

To use JCo functionality seamlessly at compile time in Eclipse, you must include the JCo dependencies into your web project. Therefore, you must convert it into a maven project.

1. In the *Project Explorer* view, right-click on the project `jco-demo` and choose *Configure* > *Convert to Maven Project*.
2. In the dialog window, leave the default settings unchanged and choose *Finish*.
3. Open the `pom.xml` file and include the following dependency:

```
<dependencies>
  <dependency>
    <groupId>com.sap.cloud</groupId>
    <artifactId>neo-java-web-api</artifactId>
    <version>[3.71.8,4.0.0)</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

Back to Steps [page 73]

Create a Sample Servlet

1. From the *jco_demo* project node, choose *New* > *Servlet* in the context menu.
2. Enter **`com.sap.demo.jco`** as the `<package>` and **`ConnectivityRFCExample`**Java as the `<Class name>`. Choose *Next*.
3. Choose *Finish* to create the servlet and open it in the Java editor.
4. Replace the entire servlet class to make use of the JCo API. The JCo API is visible by default for cloud applications. You do not need to add it explicitly to the application class path.

Sample Code

```
package com.sap.demo.jco;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.sap.conn.jco.AbapException;
import com.sap.conn.jco.JCoDestination;
import com.sap.conn.jco.JCoDestinationManager;
import com.sap.conn.jco.JCoException;
import com.sap.conn.jco.JCoFunction;
import com.sap.conn.jco.JCoParameterList;
import com.sap.conn.jco.JCoRepository;

/**
 * Sample application that makes use of the capability to invoke a
function module in an ABAP system
 * via RFC
 *
 * Note: The JCo APIs are available under <code>com.sap.conn.jco</code>.
```

```
 */
@WebServlet("/ConnectivityRFCExample/*")
public class ConnectivityRFCExample extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        PrintWriter responseWriter = response.getWriter();
        try {
            // access the RFC Destination "JCoDemoSystem"
            JCoDestination destination =
JCoDestinationManager.getDestination("JCoDemoSystem");

            // make an invocation of STFC_CONNECTION in the backend;
            JCoRepository repo = destination.getRepository();
            JCoFunction stfcConnection =
repo.getFunction("STFC_CONNECTION");

            JCoParameterList imports =
stfcConnection.getImportParameterList();
            imports.setValue("REQUTEXT", "SAP BTP Connectivity runs with
JCo");
            stfcConnection.execute(destination);
            JCoParameterList exports =
stfcConnection.getExportParameterList();
            String echotext = exports.getString("ECHOTEXT");
            String resptext = exports.getString("RESPTEXT");
            response.addHeader("Content-type", "text/html");
            responseWriter.println("<html><body>");
            responseWriter.println("<h1>Executed STFC_CONNECTION in system
JCoDemoSystem</h1>");
            responseWriter.println("<p>Export parameter ECHOTEXT of
STFC_CONNECTION:<br>");
            responseWriter.println(echotext);
            responseWriter.println("<p>Export parameter RESPTEXT of
STFC_CONNECTION:<br>");
            responseWriter.println(resptext);
            responseWriter.println("</body></html>");
        } catch (AbapException ae) {
            // just for completeness: As this function module does not
have an exception
            // in its signature, this exception cannot occur. But you
should always
            // take care of AbapExceptions
        } catch (JCoException e) {
            response.addHeader("Content-type", "text/html");
            responseWriter.println("<html><body>");
            responseWriter
                    .println("<h1>Exception occurred while executing
STFC_CONNECTION in system JCoDemoSystem</h1>");
            responseWriter.println("<pre>");
            e.printStackTrace(responseWriter);
            responseWriter.println("</pre>");
            responseWriter.println("</body></html>");
        }
    }
}
```

5. Save the Java editor and make sure that the project compiles without errors.

Back to Steps [page 73]

Next Steps

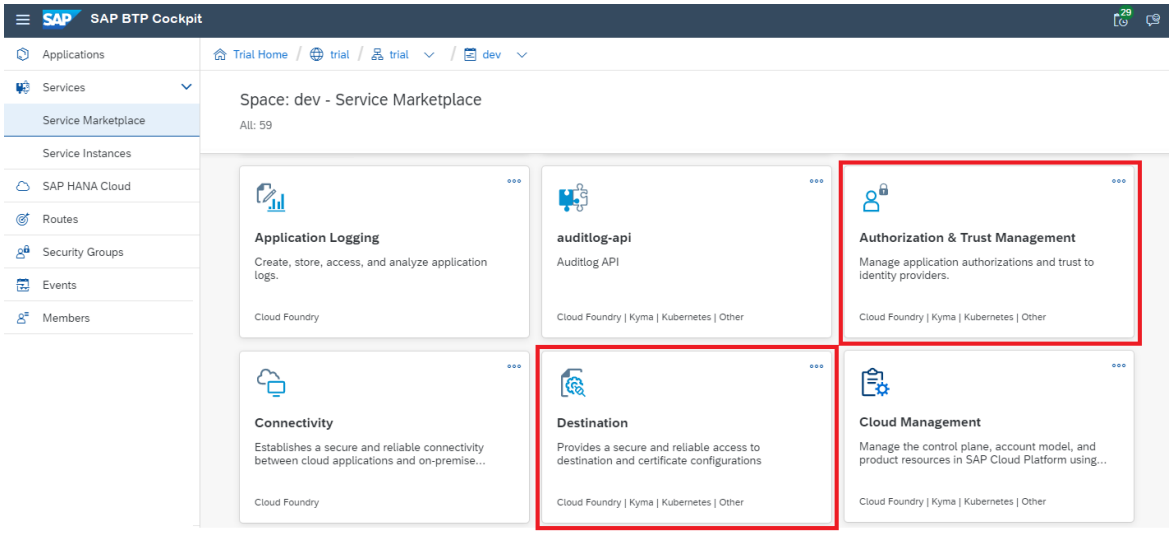
- Create and Bind Service Instances [page 102]
- Deploy the Application [page 104]
- Configure Roles and Trust [page 106]
- Set Up an Application Router [page 107]
- Configure the RFC Destination [page 111]
- Monitoring Your Web Application [page 112] (Optional)

1.1.3.2.1.2.2 Create and Bind Service Instances

You must create and bind several service instances, before you can use your application.

Procedure

1. Logon to the cloud cockpit and choose your subaccount.
2. Choose the space where you want to deploy your demo application.
3. Choose *Service Marketplace* and find these 2 services:
   - *Destination*
   - *Authorization & Trust Management* (XSUAA)



4. Create and bind a service instance for each of these services.
   - Destination service [page 79]
   - Authorization & Trust Management (XSUAA service) [page 80]

Destination Service

1. Choose *Destination* > *Create Instance*.
2. Insert an instance name (for example, **`destination_jco`**) and choose *Create Instance*.

Back to Procedure [page 78]

Authorization & Trust Management (XSUAA Service)

1. Choose *Authorization & Trust Management* > *Create Instance*
2. Select `<Service Plan>` **`application`**.
3. Enter an `<Instance Name>` and choose *Next*.

> **Note**
>
> The instance name must match the one defined in the manifest file.

4. In the next tab *Parameters* , insert the following as a JSON file:

> **Sample Code**

```
{
    "xsappname" : "jco-demo-p1234",
    "tenant-mode": "dedicated",
    "scopes": [
        {
            "name": "$XSAPPNAME.all",
            "description": "all"
        }
    ],
    "role-templates": [
        {
            "name": "all",
            "description": "all",
            "scope-references": [
                "$XSAPPNAME.all"
            ]
        }
    ]
}
```

5. Go to tab *Review* and choose *Create Instance*.

Back to Procedure [page 78]

Next Steps

- Deploy the Application [page 104]
- Configure Roles and Trust [page 106]
- Set Up an Application Router [page 107]
- Configure the RFC Destination [page 111]
- Monitoring Your Web Application [page 112] (Optional)

1.1.3.2.1.2.3 Deploy the Application

Deploy your Cloud Foundry application to call an ABAP function module via RFC.

Prerequisites

You have created and bound the required service instances, see Create and Bind Service Instances [page 78].

Procedure

1. To deploy your Web application, you can use the following two alternative procedures:
   - Deploying from the Eclipse IDE
   - Deploying from the CLI, see Developing Java in the Cloud Foundry Environment
2. In the following, we publish it with the CLI.
3. To do this, create a `manifest.yml` file. The key parameter is `USE_JCO: true`, which must be set to include JCo into the buildpack during deployment.

   > **Note**
   >
   > JCo supports the usage of X.509 secrets for communication with the Destination/Connectivity service. If you want to use it, you must specify the binding accordingly.
   >
   > For more information, see Binding Parameters of SAP Authorization and Trust Management Service.

   manifest.yml

   Sample Code

   ```
   ---
   applications:

   - name: jco-demo-p1234
     buildpacks:
   ```

```
  - sap_java_buildpack
env:
  USE_JCO: true
  # This is necessary only if more than one instance is bound
  xsuaa_connectivity_instance_name: "xsuaa_jco"
  connectivity_instance_name: "connectivity_jco"
  destination_instance_name: "destination_jco"
services:
  - xsuaa_jco
  - connectivity_jco
  - destination_jco
```

> ⚠ Caution
>
> The client libraries (java-security, spring-xsuaa, and container security api for node.js as of version 3.0.6) have been updated. When using these libraries, setting the parameter `SAP_JWT_TRUST_ACL` has become obsolete. This update comes with a change regarding scopes:
>
> - For a business application A calling an application B, it is now mandatory that application B grants at least one scope to the calling business application A.
> - Business application A must accept these granted scopes or authorities as part of the application security descriptor.
>
> You can grant scopes using the `xs-security.json` file.
>
> For more information, see Application Security Descriptor Configuration Syntax, specifically the sections *Referencing the Application* and *Authorities*.

> ⓘ Note
>
> If you have more than one instance of those three services bound to your application, you must specify which one JCo should use with the respective env parameters:
>
> - `xsuaa_connectivity_instance_name`
> - `connectivity_instance_name`
> - `destination_instance_name`.

4. In Eclipse, right-click on the project and navigate to *Export* > *WAR file*.
5. Choose a destination by pressing the *Browse...* button next to the `manifest.yml` you created before, for example as `jcodemo.war`.
6. Leave the other default settings unchanged and choose *Finish* to export the WAR file.
7. Perform a CLI login via `cf login -a api.cf.eu10.hana.ondemand.com -u <your_email_address>` (password, org and space are prompted after a successful login).
8. Push the application with `cf push -f manifest.yml -p jcodemo.war`.
9. Now, the application should be deployed successfully.

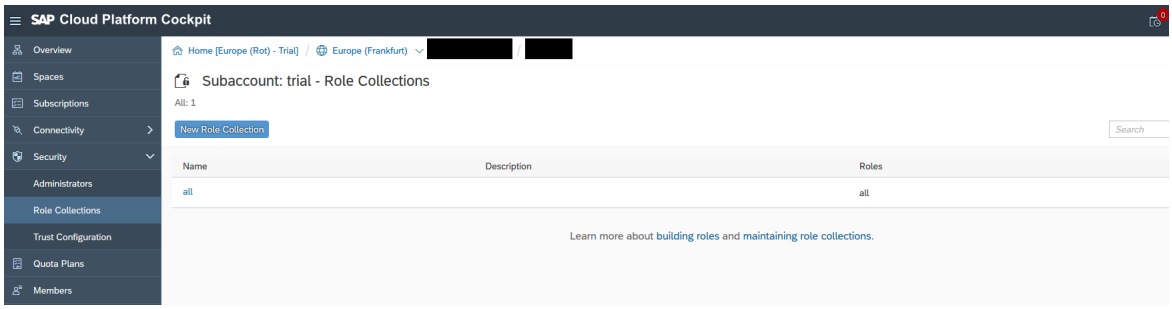
Next Steps

- Configure Roles and Trust [page 106]
- Set Up an Application Router [page 107]
- Configure the RFC Destination [page 111]
- Monitoring Your Web Application [page 112] (Optional)

1.1.3.2.1.2.4 Configure Roles and Trust

Configure a role that enables your user to access your Web application.

1. From the subaccount menu, choose *Trust Configuration*.
2. If you don't have a trust configuration, follow the steps in Manually Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services.
3. To add and assign roles, choose *Security* > *Role Collections* from the subaccount menu:



4. Create a *New Role Collection* with the name **`all`**.
5. Select *all* > *Edit* and add yourself as user.
6. Choose *Save*.

Next Steps

- Set Up an Application Router [page 107]
- Configure the RFC Destination [page 111]
- Monitoring Your Web Application [page 112] (Optional)

Related Information

Working with Role Collections

1.1.3.2.1.2.5 Set Up an Application Router

For authentication purposes, configure and deploy an application router for your test application.

> **Note**
>
> AppRouter is only required if you want to use multitenancy or perform user-specific service calls. In all other cases, JCo uses cloud-security-xsuaa-integration with `ClientCredentialFlow`.

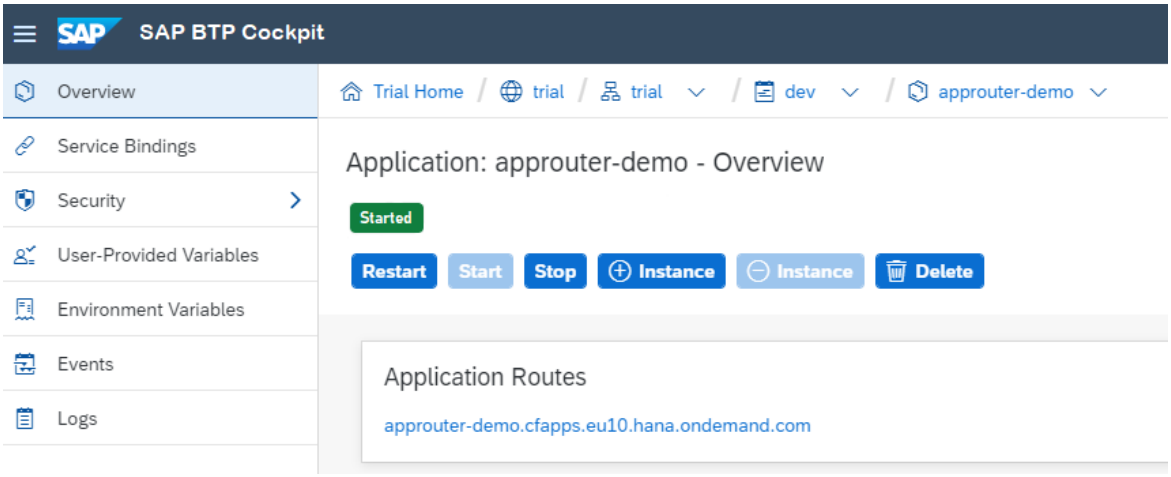
1. To set up an application router, follow the steps in Application Router or use the demo file *approuter.zip* (download).
2. For deployment, you need a manifest file, similar to this one:

   > **Sample Code**
   >
   ```
   ---
   applications:

   - name: approuter-jco-demo-p1234
     path: ./
     buildpacks:
     - nodejs_buildpack
     memory: 120M
     routes:
     - route: approuter-jco-demo-p1234.cfapps.eu10.hana.ondemand.com
     env:
       NODE_TLS_REJECT_UNAUTHORIZED: 0
       destinations: >
         [
            {"name":"dest-to-example", "url" :"https://jco-
   demo-p1234.cfapps.eu10.hana.ondemand.com/ConnectivityRFCExample",
   "forwardAuthToken": true }
         ]
     services:
       - xsuaa_jco
   ```

   > **Note**
   >
   > - The routes and destination URLs need to fit your test application.
   > - In this example, we already bound our XSUAA instance to the application router. Alternatively, you could also do this via the cloud cockpit.

3. Push the approuter with `cf push -f manifest.yml -p approuter.zip`.
4. To navigate to the `approuter` application in the cloud cockpit, choose *<your_space>* > *Applications* > *<your application>* > *Overview*.



5. When choosing the application route, you are requested to login. Provide the credentials known by the IdP you configured in *Roles & Trust*.
6. After successful login, you are routed to the test application which is then executed.
7. If the application issues an exception, saying that the `JCoDemoSystem` destination has not yet been specified, you must configure the `JCoDemoSystem` destination first.

```
Exception occurred while executing STFC_CONNECTION in system JCoDemoSystem
com.sap.conn.jco.JCoException: (106) JCO_ERROR_RESOURCE: Destination
JCoDemoSystem does not exist
    at
com.sap.conn.jco.rt.DefaultDestinationManager.update(DefaultDestinationManager
.java:223)
    at
com.sap.conn.jco.rt.DefaultDestinationManager.searchDestination(DefaultDestina
tionManager.java:377)
    at
com.sap.conn.jco.rt.DefaultDestinationManager.getDestinationInstance(DefaultDe
stinationManager.java:96)
    at
com.sap.conn.jco.JCoDestinationManager.getDestination(JCoDestinationManager.ja
va:52)
    at
com.sap.demo.jco.ConnectivityRFCExample.doGet(ConnectivityRFCExample.java:47)
        ..... (cut rest of the call stack)
```

> **Note**
>
> Make sure you **don't** include this dependency
>
> ```
> <dependency>
>     <groupId>com.sap.cloud.security</groupId>
>     <artifactId>java-security</artifactId>
> </dependency>
> ```
>
> or any of its dependencies such as `java-api` with scope `compile` directly or transitively with any other jar.

Calling JCo APIs from Newly Created Threads

If you are using an Application Router and it is mandatory for you to call JCo APIs from a different thread than the one which is executing your servlet function, make sure the thread local information of the cloud-security-xsuaa-integration API , used by JCo internally, is set again within your newly created thread.

To do this, add the following dependency to your project:

```
<dependency>
    <groupId>com.sap.cloud.security</groupId>
    <artifactId>java-api</artifactId>
    <version>2.7.7</version>
    <scope>provided</scope>
</dependency>
```

Adjust your code from the step Develop a Sample Web Application [page 98] in the following way:

Sample Code

```
package com.sap.demo.jco;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import com.sap.cloud.security.token.SecurityContext;
import com.sap.cloud.security.token.Token;
import com.sap.conn.jco.AbapException;
import com.sap.conn.jco.JCoDestination;
import com.sap.conn.jco.JCoDestinationManager;
import com.sap.conn.jco.JCoException;
import com.sap.conn.jco.JCoFunction;
import com.sap.conn.jco.JCoParameterList;
import com.sap.conn.jco.JCoRepository;
/**
 *
 * Sample application that uses the connectivity service. In particular, it is
 * making use of the capability to invoke a function module in an ABAP system
 * via RFC
 *
 * Note: The JCo APIs are available under <code>com.sap.conn.jco</code>.
 */
@WebServlet("/ConnectivityRFCExample/*")
public class ConnectivityRFCExample extends HttpServlet {
    private static final long serialVersionUID = 1L;
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        PrintWriter responseWriter = response.getWriter();
        // access the token from the thread which is executing the servlet
        Token token = SecurityContext.getToken();
        Thread runThread = new Thread(() -> {
            // set the information in the newly created thread
            SecurityContext.setToken(token);
            try {
                // access the RFC Destination "JCoDemoSystem"
                JCoDestination destination =
JCoDestinationManager.getDestination("JCoDemoSystem_Normal");
                // make an invocation of STFC_CONNECTION in the backend
```

```
                JCoRepository repo = destination.getRepository();
                JCoFunction stfcConnection =
repo.getFunction("STFC_CONNECTION");
                JCoParameterList imports =
stfcConnection.getImportParameterList();
                imports.setValue("REQUTEXT", "SAP BTP Connectivity runs with
JCo");
                stfcConnection.execute(destination);
                JCoParameterList exports =
stfcConnection.getExportParameterList();
                String echotext = exports.getString("ECHOTEXT");
                String resptext = exports.getString("RESPTEXT");
                response.addHeader("Content-type", "text/html");
                responseWriter.println("<html><body>");
                responseWriter.println("<h1>Executed STFC_CONNECTION in
system JCoDemoSystem</h1>");
                responseWriter.println("<p>Export parameter ECHOTEXT of
STFC_CONNECTION:<br>");
                responseWriter.println(echotext);
                responseWriter.println("<p>Export parameter RESPTEXT of
STFC_CONNECTION:<br>");
                responseWriter.println(resptext);
                responseWriter.println("</body></html>");
            } catch (AbapException ae) {
                // just for completeness: As this function module does not
have an exception
                // in its signature, this exception cannot occur. But you
should always
                // take care of AbapExceptions
            } catch (JCoException e) {
                response.addHeader("Content-type", "text/html");
                responseWriter.println("<html><body>");
                responseWriter
                        .println("<h1>Exception occurred while executing
STFC_CONNECTION in system JCoDemoSystem</h1>");
                responseWriter.println("<pre>");
                e.printStackTrace(responseWriter);
                responseWriter.println("</pre>");
                responseWriter.println("</body></html>");
            } finally {
                // after execution clear the token again
                SecurityContext.clearToken();
            }
        });
        runThread.start();
        // wait to be finished
        try {
            runThread.join();
        } catch (InterruptedException e) {
            e.printStackTrace(responseWriter);
        }
    }
}
```

Note

If you want to use thread pools, make sure that in your thread pool implementation this information is set correctly in the thread which is about to be (re)used, and removed as soon as the thread is put back into the pool.

Next Steps

- Configure the RFC Destination [page 111]
- Monitoring Your Web Application [page 112] (Optional)

1.1.3.2.1.2.6 Configure the RFC Destination

Configure an RFC destination on SAP BTP that you can use in your Web application to call the cloud ABAP system.

To configure the destination, you must use a WebSocket application server host name (**`<id>.abap.eu10.hana.ondemand.com`**) and a WebSocket port (**`443`**).

1. Create a `.properties` file with the following settings:

```
Name=JCoDemoSystem
Type=RFC
jco.client.wshost= <id>.abap.eu10.hana.ondemand.com
jco.client.wsport=443
jco.client.alias_user=<DEMOUSER>
jco.client.passwd=<Password>
jco.client.client=100
jco.client.lang=EN
jco.destination.pool_capacity=5
jco.destination.proxy_type=Internet
```

2. Go to your subaccount in the cloud cockpit.
   1. From the subaccount menu, choose *Connectivity* > *Destinations* > *Import Destination* and upload this file.
   2. Alternatively, you can create a destination for the service instance **`destination_jco`** to make it visible only for this instance. To do this, go to *<your space>* > *Services* > *Service Instances* > *<your destination_instance>* and choose *Destinations*.
3. Specify a trust/key store or security information, see WebSocket Connection [page 1456].

Next Steps

- Monitoring Your Web Application [page 112] (Optional)

Related Information

Target System Configuration [page 1452]

1.1.3.2.1.2.7 Monitoring Your Web Application

Monitor the state and logs of your Web application deployed on SAP BTP, using the Application Logging service.

For this purpose, create an instance of the Application Logging service (as you did for the Destination service) and bind it to your application, see Create and Bind Service Instances [page 78].

To activate JCo logging, set the following property in the *env* section of your manifest file:

```
SET_LOGGING_LEVEL: '{com.sap.conn.jco: INFO, com.sap.core.connectivity.jco:
INFO}'
```

Now you can see and open the logs in the cloud cockpit or in the Kibana Dashboard in the tab *Logs*, if you are within your application.

For error analysis, you can activate more detailed JCo debug tracing:

```
SET_LOGGING_LEVEL: '{com.sap.conn.jco: DEBUG, com.sap.core.connectivity.jco:
DEBUG}'
```

To include other relevant components for debug tracing, set:

```
SET_LOGGING_LEVEL: '{com.sap.conn.jco: DEBUG, com.sap.core.connectivity.jco:
DEBUG, com.sap.core.connectivity.apiext: DEBUG, com.sap.cloud.security: DEBUG,
com.sap.xs.security: DEBUG, com.sap.xs.env: DEBUG}'
```

1.1.3.2.1.3 Multitenancy for JCo Applications (Advanced)

Learn about the required steps to make your JCo application tenant-aware.

Using this procedure, you can enable the sample JCo application created in Invoke ABAP Function Modules in On-Premise ABAP Systems [page 69] or Invoke ABAP Function Modules in Cloud ABAP Systems [page 94], for multitenancy.

Steps

1. Prerequisites [page 113]
2. Adjust the Application Router [page 113]
3. Adjust the XSUAA Service Instance and Roles [page 114]
4. Make the Application Subscribable [page 115]
5. Create the SAAS Provisioning Service [page 120]

6. Subscribe to the Application [page 121]
7. Create a New Route [page 123]

Prerequisites

- Your runtime environment uses SAP Java Buildpack version 1.9.0 or higher.
- You have successfully completed one of these precedures:
  Invoke ABAP Function Modules in On-Premise ABAP Systems [page 69]
  Invoke ABAP Function Modules in Cloud ABAP Systems [page 94]
- You have created a second subaccount (in the same global account), that is used to subscribe to your application.

Back to Steps [page 112]

Adjust the Application Router

The application router needs to be tenant-aware with a `TENANT_HOST_PATTERN` to recognize different tenants from the URL, see Multitenancy. `TENANT_HOST_PATTERN` should have the following format: "`^(.*).<application domain>`". The application router extracts the token captured by "`(.*)`" to use it as the subscriber tenant. The manifest file might look like this:

Sample Code

```
manifest.yml
---
applications:

- name: approuter-jco-demo-p42424242
  path: ./
  buildpacks:
  - nodejs_buildpack
  memory: 120M
  routes:
  - route: approuter-jco-demo-p42424242.cfapps.eu10.hana.ondemand.com
  env:
    TENANT_HOST_PATTERN: "^(.*).cfapps.eu10.hana.ondemand.com"
    NODE_TLS_REJECT_UNAUTHORIZED: 0
    destinations: >
      [
         {"name":"dest-to-example", "url" :"https://jco-
demo-p42424242.cfapps.eu10.hana.ondemand.com/ConnectivityRFCExample",
"forwardAuthToken": true }
      ]
  services:
    - xsuaa_jco
```

Back to Steps [page 112]

Adjust the XSUAA Service Instance and Roles

To call the XSUAA in a tenant-aware way, you must adjust the configuration JSON file. The tenant mode must now have the value "shared". Also, you must allow calling the previously defined REST APIs (callbacks).

Sample Code

```
{
    "xsappname" : "jco-demo-p42424242",
    "tenant-mode": "shared",
    "scopes": [
        {
         "name":"$XSAPPNAME.Callback",
         "description":"With this scope set, the callbacks for tenant
onboarding, offboarding and getDependencies can be called.",
         "grant-as-authority-to-apps":[
            "$XSAPPNAME(application,sap-provisioning,tenant-onboarding)"
         ]
          },
        {
            "name": "$XSAPPNAME.access",
            "description": "app access"
        },
        {
            "name": "uaa.user",
            "description": "uaa.user"
        }
    ],
    "role-templates": [
        {
         "name":"MultitenancyCallbackRoleTemplate",
         "description":"Call callback-services of applications",
         "scope-references":[
            "$XSAPPNAME.Callback"
         ]
          },
        {
            "name": "UAAaccess",
            "description": "UAA user access",
            "scope-references": [
                "uaa.user",
                "$XSAPPNAME.access"
            ]
        }
    ]
}
```

Add Roles

1. In the cloud cockpit, navigate to the subaccount view and go the tab *Role Collections* under *Security* (see step Configure Roles and Trust [page 83] from the previous procedure).
2. Click on the role collection name.
3. Choose *Add Role*.
4. In the popup window, select the demo application as `<Application Identifier>`.
5. For `<Role Template>` and `<Role>`, use **`MultitenancyCallbackRoleTemplate`** and choose *Save*.
6. Choose *Add Role* again.
7. Select the demo application as `<Application Identifier>`.

8. For *Role Template* and *Role* use **`UAAaccess`** and choose *Save*.

Back to Steps [page 112]

Make the Application Subscribable

Firstly, in order to make the application subscribable, it must provide at least the following REST APIs:

- GET dependent services of an application [page 115]
- PUT tenant subscription to an application [page 118]

In our sample application, we implement new servlets for each of these APIs.

The following servlets need additional maven dependencies:

Sample Code

```
<dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.8.5</version>
</dependency>
<dependency>
    <groupId>com.unboundid.components</groupId>
    <artifactId>json</artifactId>
    <version>1.0.0</version>
</dependency>
<dependency>
    <groupId>javax.ws.rs</groupId>
    <artifactId>javax.ws.rs-api</artifactId>
    <version>2.1.1</version>
</dependency>
```

GET Dependencies

The current JCo dependencies are the Connectivity and Destination service. Thus, the GET API must return information about these two services:

Sample Code

```
import java.io.IOException;
import java.util.Arrays;
import java.util.List;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.ws.rs.core.MediaType;

import org.json.JSONException;
import org.json.JSONObject;
```

```
import com.google.gson.Gson;

@WebServlet("/callback/v1.0/dependencies")
public class GetDependencyServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    private static final String DESTINATION_SERVICE_NAME = "destination";
    private static final String CONNECTIVITY_SERVICE_NAME = "connectivity";
    private static final String XSAPPNAME_PROPERTY = "xsappname";

    protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
        try {
            DependantServiceDto destinationService =
createLPSDependency(DESTINATION_SERVICE_NAME);
            DependantServiceDto connectivityService =
createLPSDependency(CONNECTIVITY_SERVICE_NAME);
            List<DependantServiceDto> dependenciesList =
Arrays.asList(destinationService, connectivityService);

            response.setStatus(200);
            response.setContentType(MediaType.APPLICATION_JSON);

            String json = new Gson().toJson(dependenciesList);
            response.getWriter().println(json);
        } catch (JSONException e) {
            response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR,
e.getMessage());
        }
    }

    private static DependantServiceDto createLPSDependency(String
serviceName) throws JSONException {
        JSONObject credentials =
EnvironmentVariableAccessor.getServiceCredentials(serviceName);
        String xsappname = credentials.getString(XSAPPNAME_PROPERTY);
        return new DependantServiceDto(serviceName, xsappname);
    }
}
```

Find the code of the two helper classes below:

DependantServiceDto.java

```
public class DependantServiceDto {
    private String appName;
    private String appId;
    public DependantServiceDto() {}
    public DependantServiceDto(String appName, String appId) {
        this.appName = appName;
        this.appId = appId;
    }
    public String getAppName() {
        return appName;
    }
    public void setAppName(String appName) {
        this.appName = appName;
    }
    public String getAppId() {
        return appId;
    }
    public void setAppId(String appId) {
        this.appId = appId;
    }
```

```
    @Override public boolean equals(Object o) {
        if (this == o)
            return true;
        if (!(o instanceof DependantServiceDto))
            return false;
        DependantServiceDto that = (DependantServiceDto) o;
        if (!appName.equals(that.appName))
            return false;
        return appId.equals(that.appId);
    }
    @Override public int hashCode() {
        int result = appName.hashCode();
        result = 31 * result + appId.hashCode();
        return result;
    }
}
```

EnvironmentVariableAccessor.java

```
import java.text.MessageFormat;
import org.json.JSONArray;
import org.json.JSONException;
import org.json.JSONObject;
/**
 * Methods for extracting configurations from the environment variables
 */
public final class EnvironmentVariableAccessor
{
    public static final String BEARER_WITH_TRAILING_SPACE="Bearer ";
    private static final String VCAP_SERVICES=System.getenv("VCAP_SERVICES");
    private static final String VCAP_SERVICES_CREDENTIALS="credentials";
    private static final String VCAP_SERVICES_NAME="name";
    private static final String
PROP_XSUAA_CONNECTIVITY_INSTANCE_NAME="xsuaa_connectivity_instance_name";
    private static final String DEFAULT_XSUAA_CONNECTIVITY_INSTANCE_NAME="conn-
xsuaa";
    private EnvironmentVariableAccessor()
    {
    }
    /**
     * Returns service credentials for a given service from VCAP_SERVICES
     *
     * @see <a href= "https://docs.run.pivotal.io/devguide/deploy-apps/
environment-variable.html#VCAP-SERVICES">VCAP_SERVICES</a>
     */
    public static JSONObject getServiceCredentials(String serviceName) throws
JSONException
    {
        return new
JSONObject(VCAP_SERVICES).getJSONArray(serviceName).getJSONObject(0).getJSONObjec
t(VCAP_SERVICES_CREDENTIALS);
    }
    /**
     * Returns service credentials for a given service instance from
VCAP_SERVICES
     *
     * @see <a href= "https://docs.run.pivotal.io/devguide/deploy-apps/
environment-variable.html#VCAP-SERVICES">VCAP_SERVICES</a>
     */
    public static JSONObject getServiceCredentials(String serviceName, String
serviceInstanceName) throws JSONException
    {
        JSONArray jsonarr=new
JSONObject(VCAP_SERVICES).getJSONArray(serviceName);
        for (int i=0; i<jsonarr.length(); i++)
        {
```

```
            JSONObject serviceInstanceObject=jsonarr.getJSONObject(i);
            String
instanceName=serviceInstanceObject.getString(VCAP_SERVICES_NAME);
            if (instanceName.equals(serviceInstanceName))
            {
                return
serviceInstanceObject.getJSONObject(VCAP_SERVICES_CREDENTIALS);
            }
        }
        throw new RuntimeException(MessageFormat.format("Service instance {0} of
service {1} not bound to application", serviceInstanceName, serviceName));
    }
    /**
     * Returns service credentials attribute for a given service from
VCAP_SERVICES
     *
     * @see <a href= "https://docs.run.pivotal.io/devguide/deploy-apps/
environment-variable.html#VCAP-SERVICES">VCAP_SERVICES</a>
     */
    public static String getServiceCredentialsAttribute(String serviceName,
String attributeName) throws JSONException
    {
        return getServiceCredentials(serviceName).getString(attributeName);
    }
    /**
     * Returns the name of the xsuaa service for connectivity service.
     */
    public static String getXsuaaConnectivityInstanceName()
    {
        String
xsuaaConnectivityInstanceName=System.getenv(PROP_XSUAA_CONNECTIVITY_INSTANCE_NAME
);
        return xsuaaConnectivityInstanceName!=null?
xsuaaConnectivityInstanceName:DEFAULT_XSUAA_CONNECTIVITY_INSTANCE_NAME;
    }
}
```

Back to Make the Application Subscribable [page 115]

PUT Tenant Subscription

This API is called whenever a tenant is subscribing. In our example, we just read the received JSON, and return the tenant-aware URL of the application router which points to our application. Also, if a tenant wants to unsubscribe, DELETE does currently nothing.

Sample Code

SubscribeServlet

```
import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.google.gson.Gson;

@WebServlet("/callback/v1.0/tenants/*")
public class SubscribeServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
```

```
    protected void doPut(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
        PayloadDataDto payload = new Gson().fromJson(request.getReader(),
PayloadDataDto.class);
        response.getWriter().println("https://" +
payload.getSubscribedSubdomain() + ".approuter-jco-
demo.cfapps.eu10.hana.ondemand.com");
    }

    @Override
    protected void doDelete(HttpServletRequest req, HttpServletResponse resp)
throws ServletException, IOException {
        super.doDelete(req, resp);
    }
}
```

Here is the helper class:

PayloadDataDto.java

```
import java.util.Map;
public class PayloadDataDto {
    private String subscriptionAppName;
    private String subscriptionAppId;
    private String subscribedTenantId;
    private String subscribedSubdomain;
    private String subscriptionAppPlan;
    private long subscriptionAppAmount;
    private String[] dependantServiceInstanceAppIds = null;
    private Map<String, String> additionalInformation;
    public PayloadDataDto() {}
    public PayloadDataDto(String subscriptionAppName, String subscriptionAppId,
            String subscribedTenantId, String subscribedSubdomain, String
subscriptionAppPlan,
            Map<String, String> additionalInformation) {
        this.subscriptionAppName = subscriptionAppName;
        this.subscriptionAppId = subscriptionAppId;
        this.subscribedTenantId = subscribedTenantId;
        this.subscribedSubdomain = subscribedSubdomain;
        this.subscriptionAppPlan = subscriptionAppPlan;
        this.additionalInformation = additionalInformation;
    }
    public String getSubscriptionAppName() {
        return subscriptionAppName;
    }
    public void setSubscriptionAppName(String subscriptionAppName) {
        this.subscriptionAppName = subscriptionAppName;
    }
    public String getSubscriptionAppId() {
        return subscriptionAppId;
    }
    public void setSubscriptionAppId(String subscriptionAppId) {
        this.subscriptionAppId = subscriptionAppId;
    }
    public String getSubscribedTenantId() {
        return subscribedTenantId;
    }
    public void setSubscribedTenantId(String subscribedTenantId) {
        this.subscribedTenantId = subscribedTenantId;
    }
    public String getSubscribedSubdomain() {
        return subscribedSubdomain;
    }
    public void setSubscribedSubdomain(String subscribedSubdomain) {
```

```
        this.subscribedSubdomain = subscribedSubdomain;
    }
    public String getSubscriptionAppPlan() {
        return subscriptionAppPlan;
    }
    public void setSubscriptionAppPlan(String subscriptionAppPlan) {
        this.subscriptionAppPlan = subscriptionAppPlan;
    }
    public Map<String, String> getAdditionalInformation() {
        return additionalInformation;
    }
    public void setAdditionalInformation(Map<String, String>
additionalInformation) {
        this.additionalInformation = additionalInformation;
    }
    public long getSubscriptionAppAmount() {
        return subscriptionAppAmount;
    }
    public void setSubscriptionAppAmount(long subscriptionAppAmount) {
        this.subscriptionAppAmount = subscriptionAppAmount;
    }
    public String[] getDependantServiceInstanceAppIds() {
        return dependantServiceInstanceAppIds;
    }
    public void setDependantServiceInstanceAppIds(
            String[] dependantServiceInstanceAppIds) {
        this.dependantServiceInstanceAppIds = dependantServiceInstanceAppIds;
    }
    public String toString() {
        return String.format("Payload data: subscriptionAppName=%s,
subscriptionAppId=%s, subscribedTenantId=%s,"
                + "subscribedSubdomain=%s subscriptionAppPlan=%s
subscriptionAppAmount=%s dependantServiceInstanceAppIds=%s",
this.subscriptionAppName, this.subscriptionAppId,
                this.subscribedTenantId, this.subscribedSubdomain,
this.subscriptionAppPlan, this.getSubscriptionAppAmount(),
dependantServiceInstanceAppIds);
    }
}
```

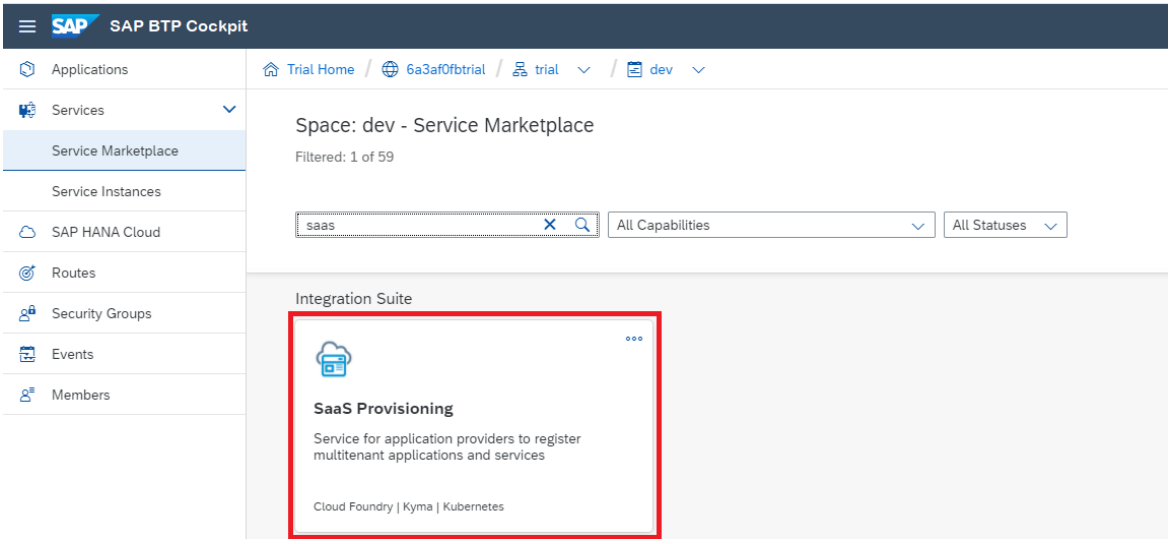
Back to Make the Application Subscribable [page 115]

Back to Steps [page 112]

Create the SAAS Provisioning Service

For the subscription of other tenants, your application must have a bound SAAS provisioning service instance. You can do this using the cockpit:

1. Go to the *Service Marketplace* in the cloud cockpit:



2. Choose *Saas Provisioning* > *Instances* > *New Instance*.
3. Select **`application`** as `<Service Plan>` and choose *Next*.
4. In the step *Specify Parameters (Optional)*, insert the following as a JSON file:

Sample Code

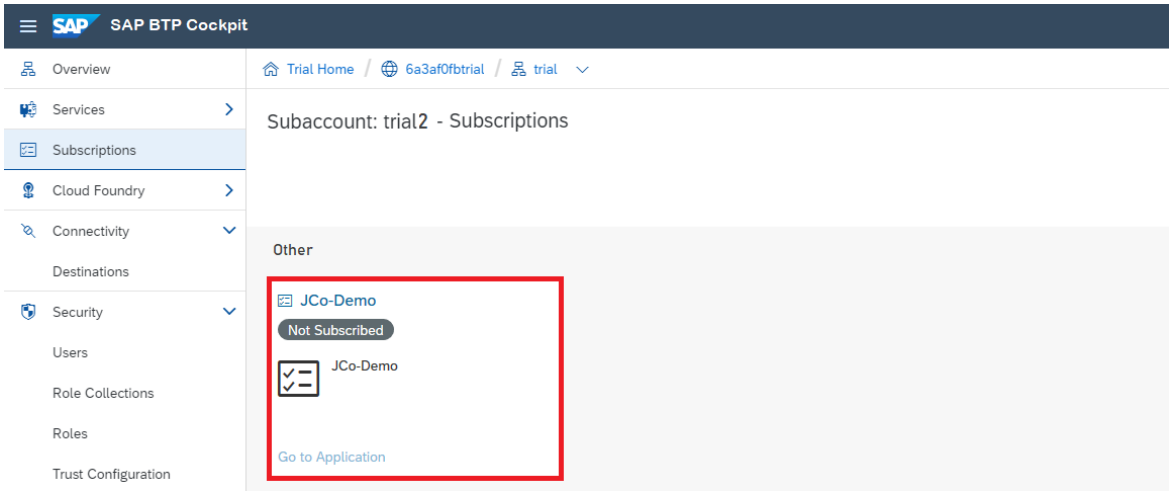
```
{
    "xsappname" : "jco-demo-p42424242",
    "appName" : "JCo-Demo",
    "appUrls": {
            "getDependencies": "https://jco-demo-
p42424242.cfapps.eu10.hana.ondemand.com/callback/v1.0/dependencies",
            "onSubscription" : "https://jco-demo-
p42424242.cfapps.eu10.hana.ondemand.com/callback/v1.0/tenants/{tenantId}"
    }
}
```

5. Choose *Next*, and select the sample application **`jco-demo-p42424242`** in the drop-down menu to assign the SAAS service to it.
6. Choose *Next*, insert an instance name, for example, **`saas_jco`**, and confirm the creation by pressing *Finish*.

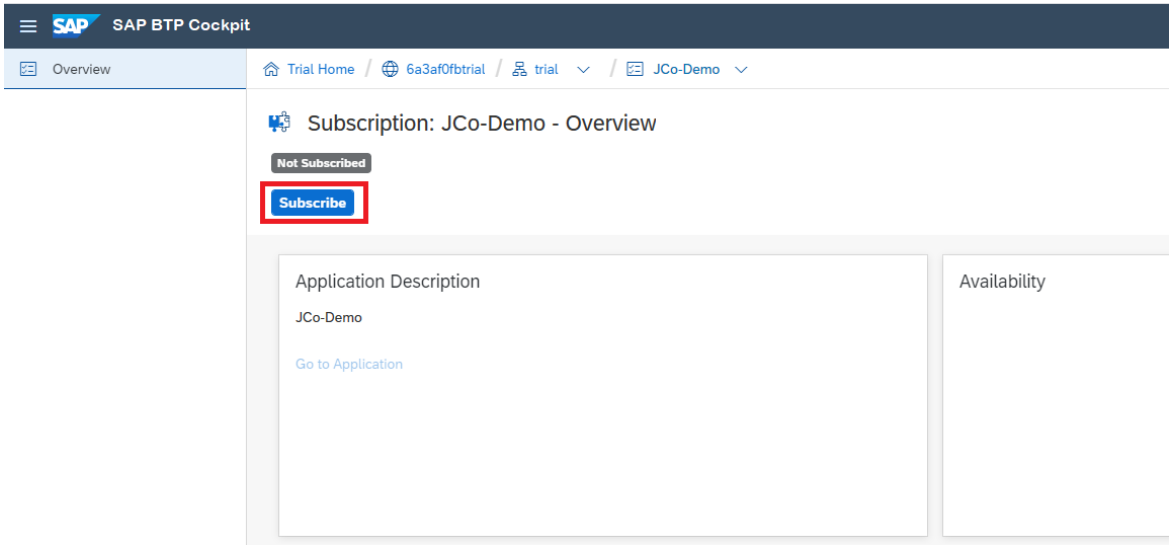
Back to Steps [page 112]

Subscribe to the Application

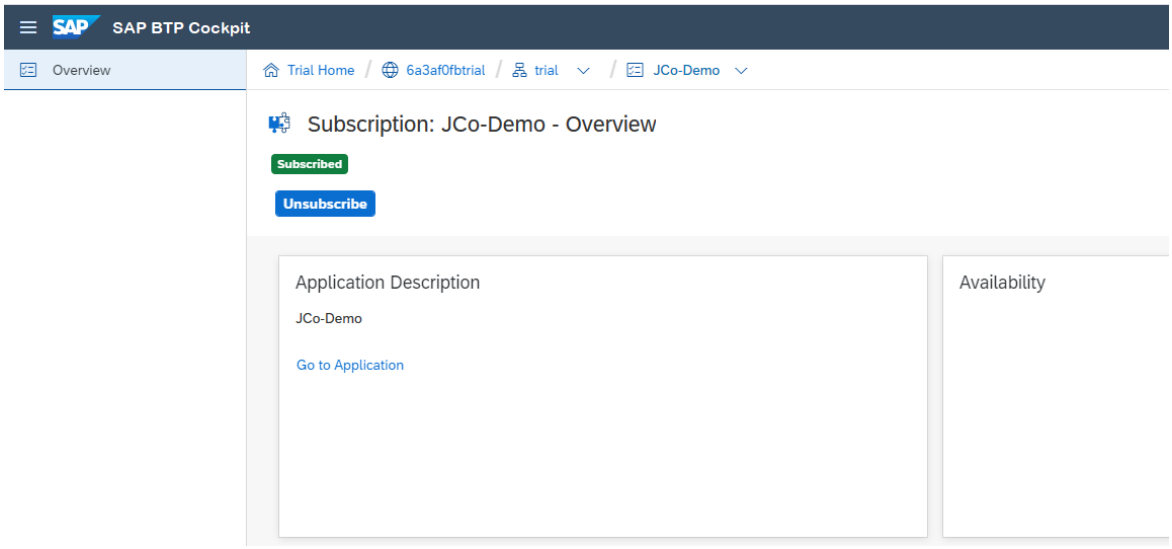
1. To subscribe the new application from a different subaccount, go to *Subscriptions* in the cockpit:



2. Click on *JCo-Demo*.
3. In the next window, choose *Subscribe*:



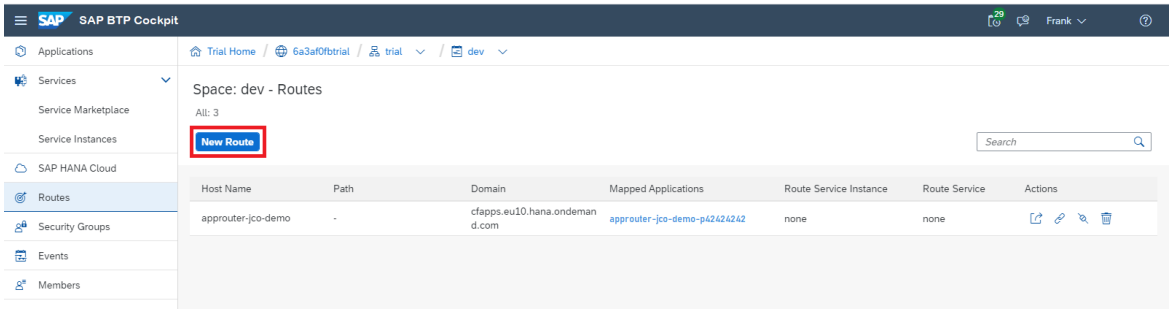
4. If the subscription was successful, your window should look like that:



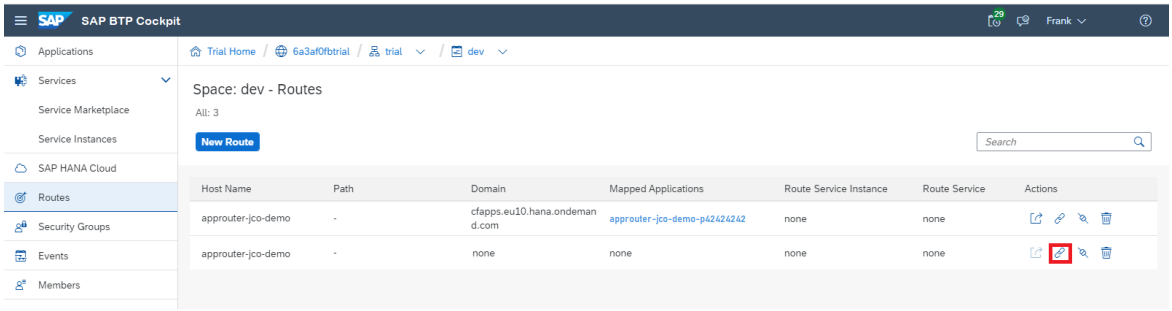
Back to Steps [page 112]

Create a New Route

1. To call the application with a new tenant, you must create a new route (URL). In the cockpit, choose *Routes* > *New Route*:



2. For `<Domain>`, select the landscape your application is deployed in (e.g. `cfapps.eu10.hana.ondemand.com`).
3. For `<Host Name>`, choose the tenant-specific link. In our example it would be `<tenant-subdomain-name>.approuter-jco-demo-p42424242`.
4. Choose *Save*.
5. Choose *Map Route* of the newly created route where the field `<Mapped Applications>` is empty (value `none`):



6. Select the approuter application and choose *Save*.
7. Congratulations, you are done, now you are able to call the sample application with this newly created route from the other subaccount!

Back to Steps [page 112]

1.1.3.2.1.4 Configure Principal Propagation for RFC

Enable single sign-on (SSO) via RFC by forwarding the identity of cloud users to an on-premise system.

Prerequisites

You have set up the Cloud Connector and the relevant backend system for principal propagation. For more informatioon, see Configuring Principal Propagation [page 1702].

Procedure

- Make sure your RFC destination is configured with `jco.destination.auth_type=PrincipalPropagation`. For more informatioon, see User Logon Properties [page 1445].
- For your Java application, use an application router to forward a user token which is used by the Java Connector (JCo) for principal propagation. For more informatioon, see Set Up an Application Router [page 83].

1.1.3.3 Token Retrieval Using IAS-Signed SAML2.0 Assertions

Retrieve access tokens from token servers using IAS (Identity Authentication service)-signed SAML2.0 assertions.

The Destination service lets you retrieve access tokens from token servers using IAS-signed SAML2.0 assertions. This process can be simplified by the use of Destination Chaining [page 1538], which automates the generation of the IAS-signed SAML2.0 assertion and its subsequent consumption by sending a single request to the Destination service. The specific chain used for this scenario is the IAS-generated SAML Assertion to OAuth2 Bearer Token Chain [page 1540].

> **Note**
>
> For more information on identity authentication, see SAP Cloud Identity Services.

Find below the step-by-step description of a sample procedure using an *OpenID Connect* (OIDC) application.

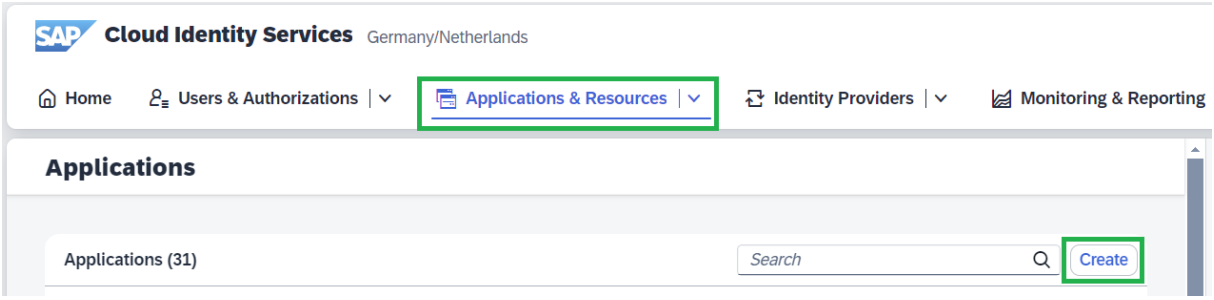
> **Note**
>
> For more information on OIDC, see OpenID Specifications .

Prerequisites

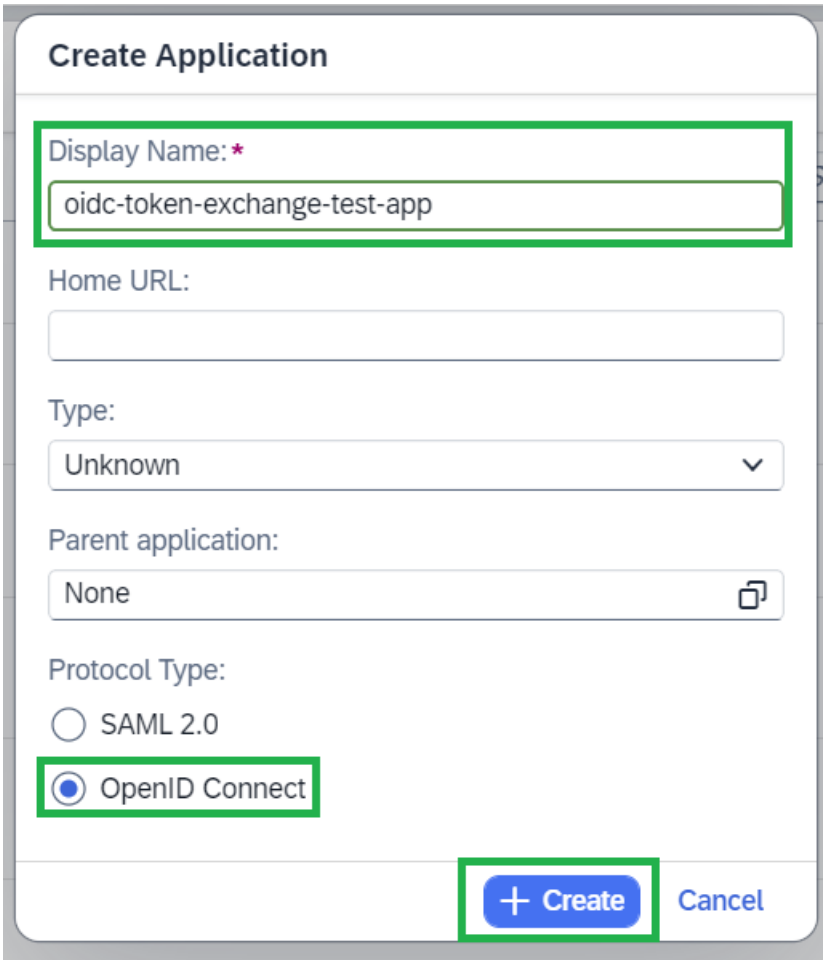
IAS

Create a New OIDC Application

In your IAS account, choose the *Applications & Resources* tab and press the *Create* button.



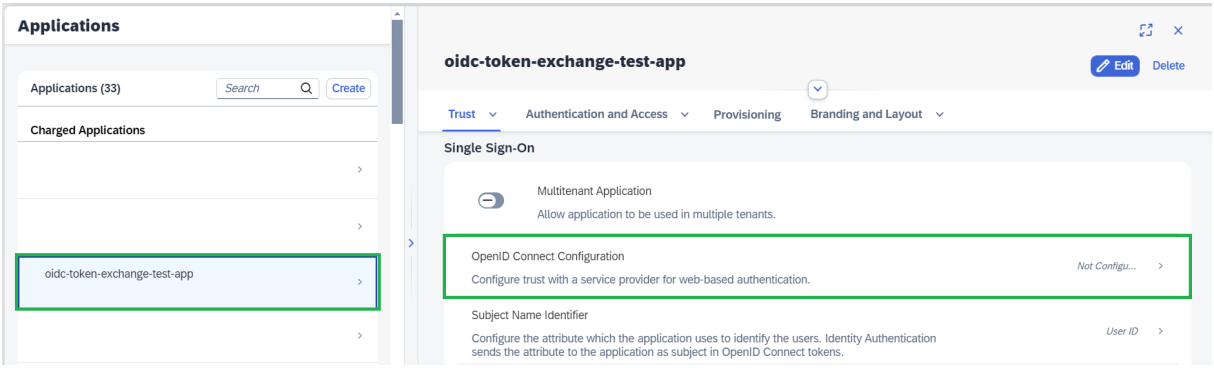
Enter any **Display Name** for your application, and select **`OpenID Connect`** as your `Protocol Type`.



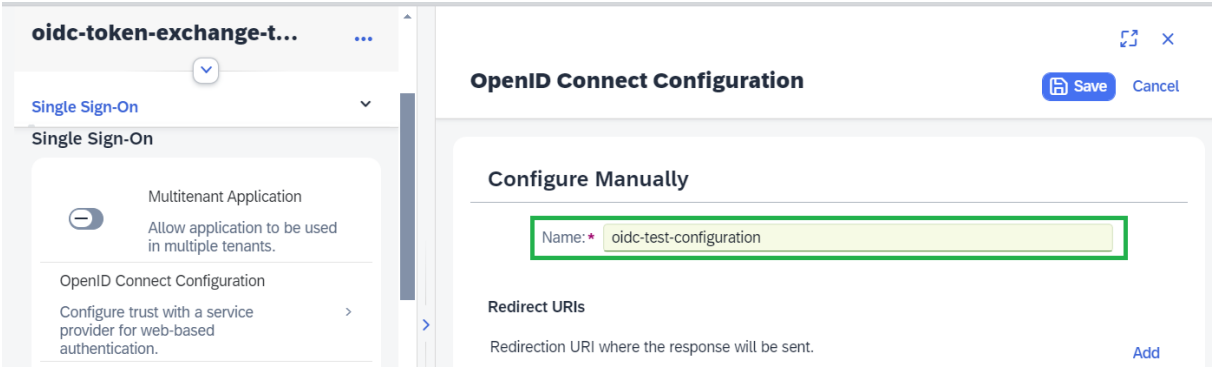
Press the *Create* button to finish creating your application.

Configure the OIDC Application

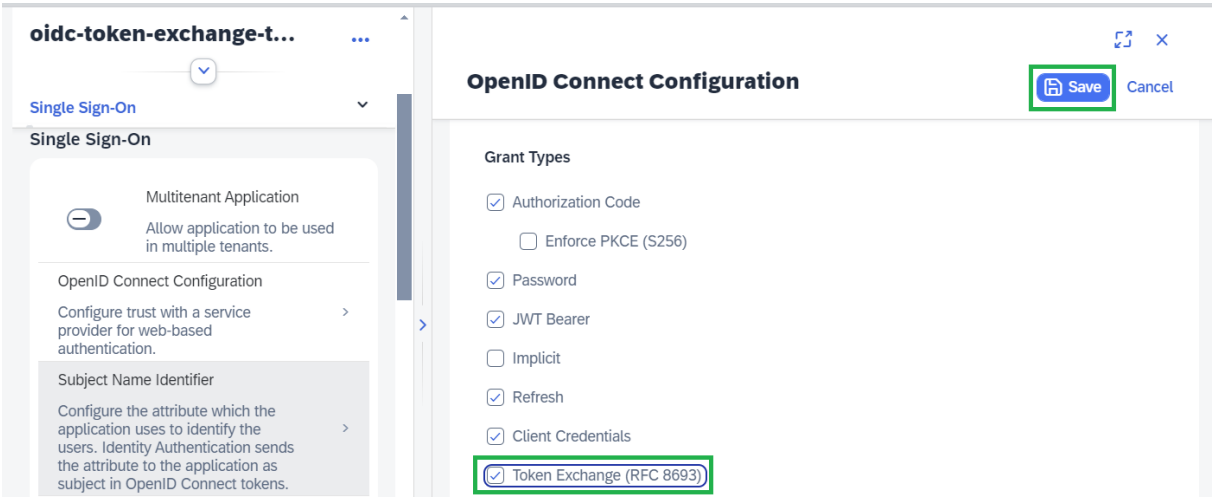
In the *Applications* section, select your new OIDC application. A window for your application will open on the right side of the screen. In the *Single Sign-On* section, select *OpenID Connect Configuration*.



Enter any name for the new configuration.

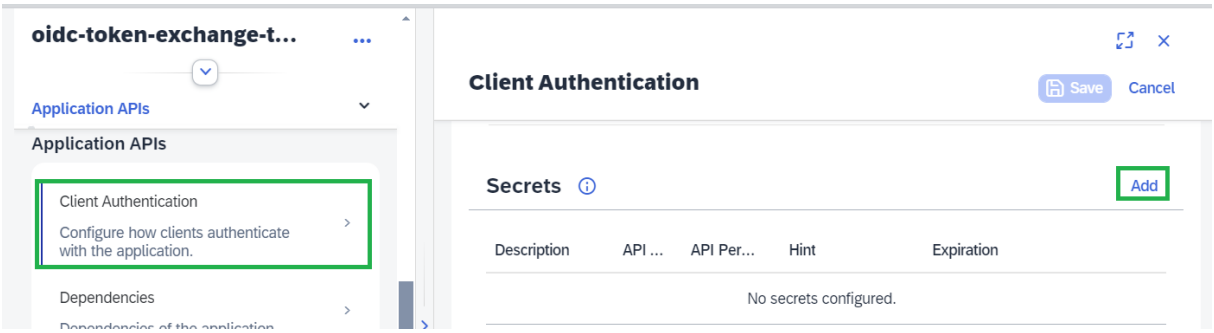


Scroll down to the **Grant Types** section. Select the *Token Exchange (RFC 8693)* checkbox, and press the *Save* button to save your configuration.

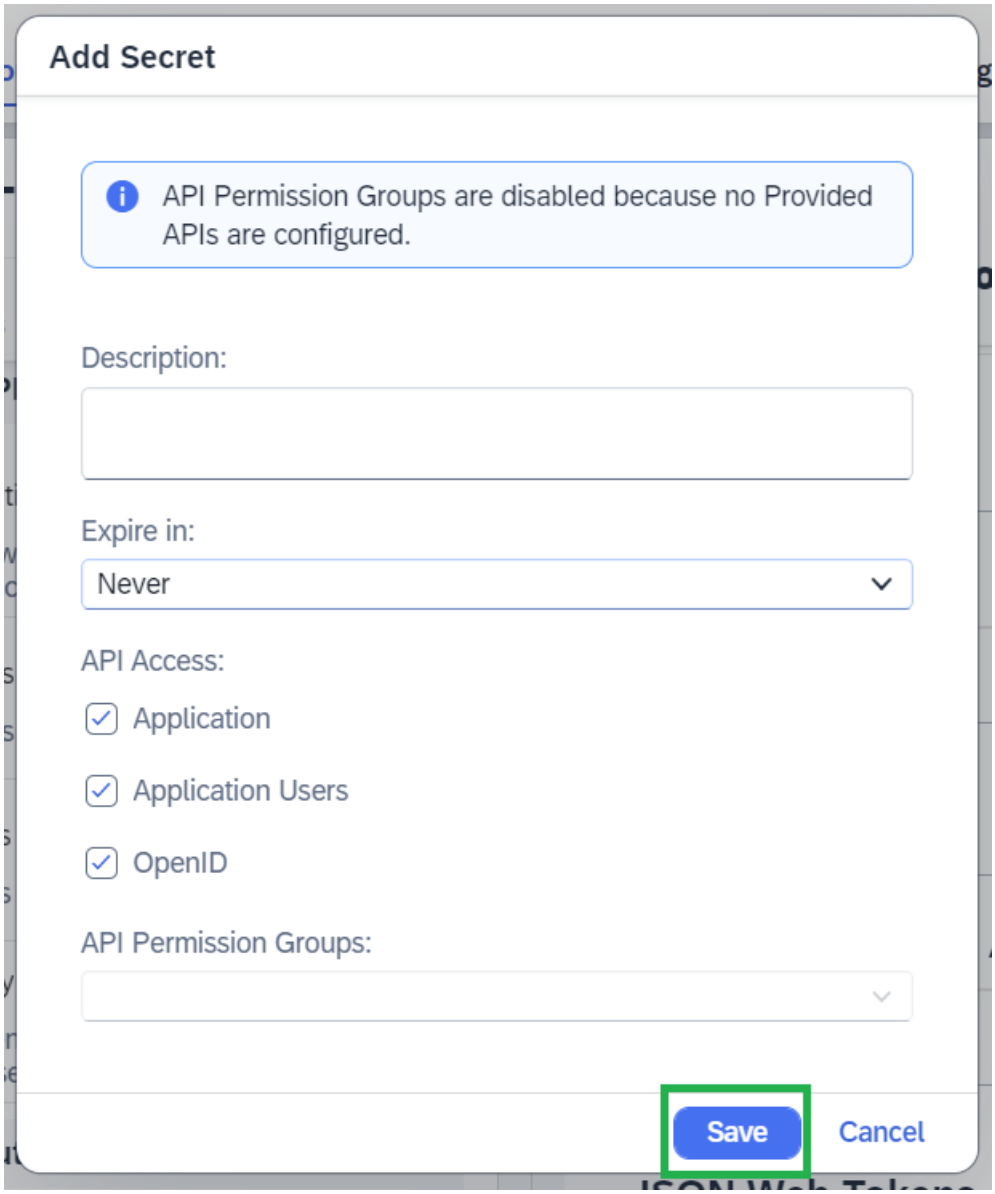


Create a Client Secret for the OIDC Application

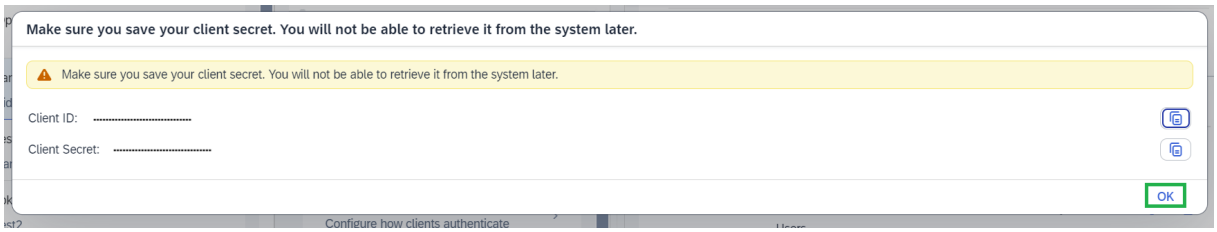
In the application window, go to the *Application APIs* section and select *Client Authentication*. Then press the *Add* button to open the secret creation form.



No configurations are necessary here. Press the *Save* button to create the client secret.

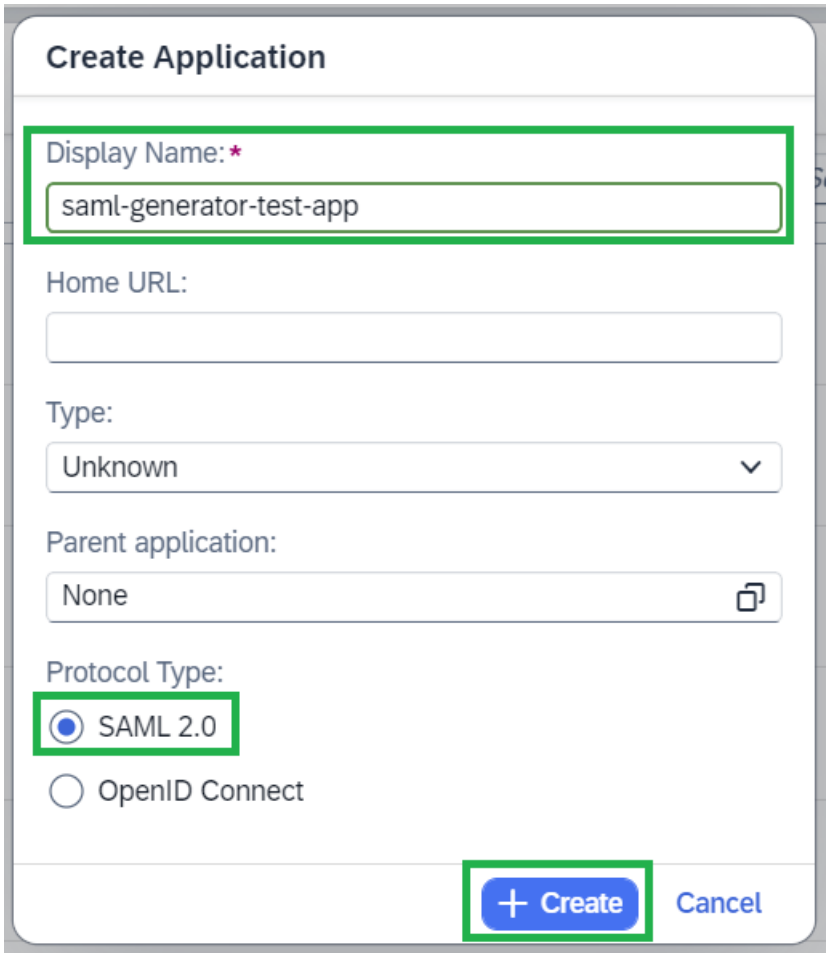


A window will open, displaying the `Client ID` and the `Client Secret`. Save them as the window suggests, and then press *OK* to close the window.



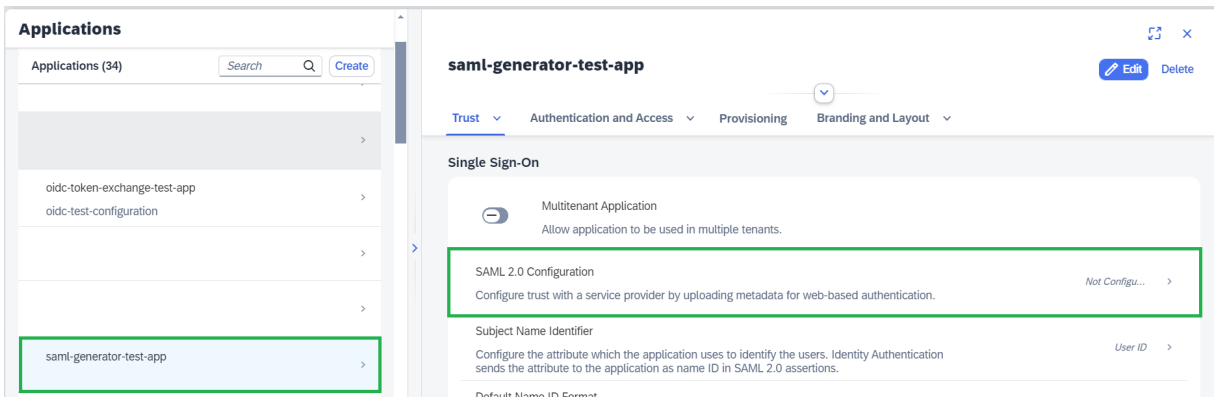
Create a New SAML2.0 Application

In your IAS account, under the *Applications & Resources* tab, select the *Create* button in the same way as in the step for *OpenId Connect* application creation. Enter any **Display Name** for your application, and select SAML2.0 as your **Protocol Type**. Then press the *Create* button to finish creating your application.



Configure the SAML2.0 Application

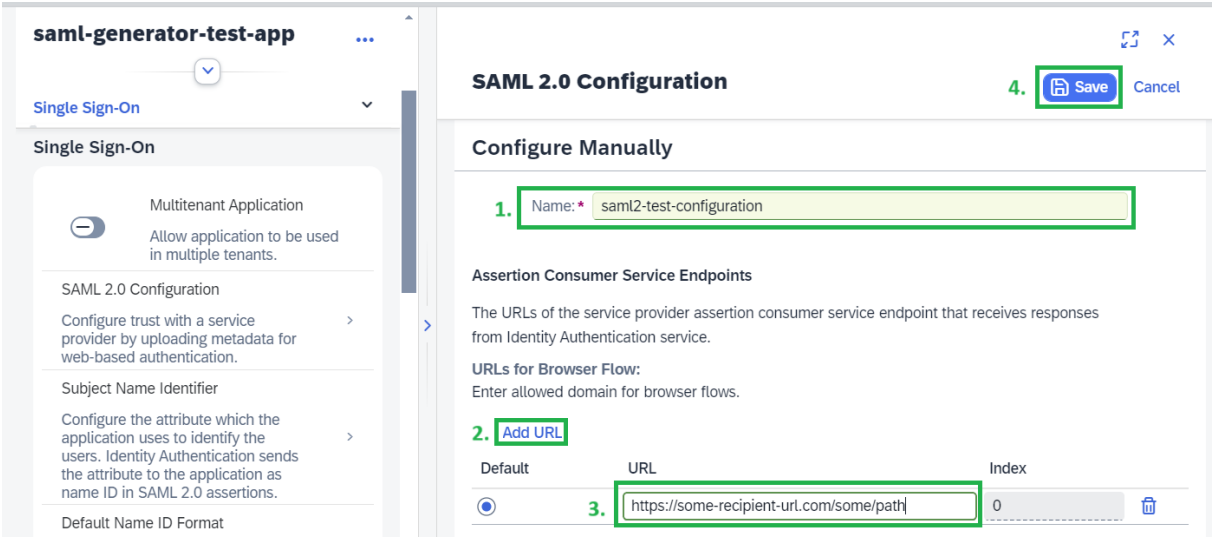
Select your new SAML2.0 application from the *Applications* list. From the application window on the right, in the *Single Sign-On* section, select **`SAML2.0 Configuration`**.



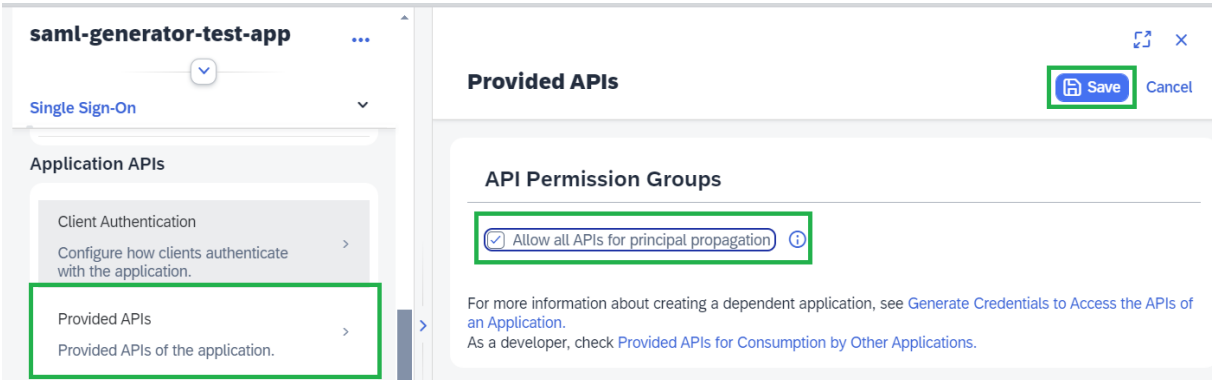
Set any name for the configuration, press *Add URL*, and set the recipient endpoint of the generated SAML2.0 assertions.

> **Note**
>
> **SAML2.0 Recipient Info**
>
> The recipient of a SAML2.0 assertion represents the token endpoint URL of the authorization server (or an acceptable alias). For more information, see RFC7522 .

Once the configuration is complete, press the *Save* button to apply it.

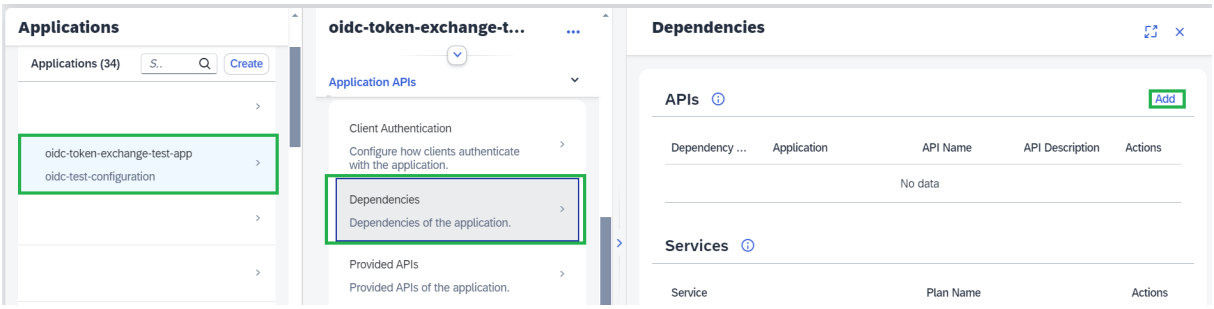


In the application window, go to the *Application APIs* section and select **`Provided APIs`**. In the newly opened window, select the **`Allow all APIs for principal propagation`** checkbox, and press the *Save* button to apply your change.



Create a SAML2.0 Dependency

Select your OIDC application. In the application window, go to the *Application APIs* section and select **`Dependencies`**. In the newly opened window, press the *Add* button.



In the *Add API* form, choose any name for your dependency, select your application from the drop down menu, and press the *Save* button to create the dependency.

Destinations Service

Create an IAS-signed SAML2.0 Assertion Provider Destination

> ⚠ Caution
>
> The *OAuth Token Exchange* authentication is currently supported only by the Destination service REST API. For information on how to access it, see Calling the Destination Service REST API [page 1562].

Request to the Destination service REST API:

> {·} Sample Code

```
curl https://<destination-service-host>/destination-configuration/v1/
subaccountDestinations --request POST \
  --header 'Authorization: Bearer <destination-service-access-token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "Name": "samlAssertionProviderDest",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": "<any-backend-url>",
    "Authentication": "OAuth2TokenExchange",
    "tokenServiceURL": "<URL-of-your-IAS-tenant>/oauth2/token",
    "clientId": "<IAS-client-id>",
    "clientSecret": "<IAS-client-secret>",
    "tokenServiceURL.queries.requested_token_type" :
"urn:ietf:params:oauth:token-type:saml2",
```

```
    "tokenServiceURL.queries.resource" :
"urn:sap:identity:application:provider:name:<SAML-application-dependency-
name>"
}'
```

Create an IAS-signed SAML2.0 Assertion Consumer Destination

Request to the Destination service REST API:

Sample Code

```
curl https://<destination-service-host>/destination-configuration/v1/
subaccountDestinations --request POST \
  --header 'Authorization: Bearer <destination-service-access-token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "Name": "samlAssertionConsumerDest",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": "<backend-system-host>",
    "Authentication": "OAuth2SAMLBearerAssertion",
    "tokenServiceURLType": "Dedicated",
    "tokenServiceURL": "<SAML-assertion-consumer-token-service-URL>",
    "clientKey": "<client-key>",
    "SAMLAssertionProvider": "ClientProvided"
}'
```

Execution

To run the flow, the *Find a Destination* endpoint of the Destination service REST API must be called. For information on how to access the REST API, see Calling the Destination Service REST API [page 1562].

Request to the Destination service REST API:

Sample Code

```
curl https://<destination-service-host>/destination-configuration/v1/
destinations/samlAssertionConsumerDest --request GET \
  --header 'Accept: */*' \
  --header 'Authorization: Bearer <destination-service-access-token>' \
  --header 'X-chain-name: com.sap.iasGeneratedOAuth2SamlBearerAssertion' \
  --header 'X-chain-var-samlProviderDestinationName:
samlAssertionProviderDest' \
  --header 'X-chain-var-subjectToken: <subject-token-value>' \
  --header 'X-chain-var-subjectTokenType: urn:ietf:params:oauth:token-
type:access_token'
```

Where:

- `X-chain-name`: The name of the chain, which will be processed as part of the request flow. The value will always equal "*com.sap.iasGeneratedOAuth2SamlBearerAssertion*".

This chain is a predefined chain configured to support the successful execution of this particular flow. For more information, see Predefined Destination Chains [page 1540].

- `X-chain-var-samlProviderDestinationName`: The name of the `OAuth2TokenExchange` authentication destination, which will be processed in order to generate the IAS-signed SAML2.0 assertion.
- `X-chain-var-subjectToken`: Represents the value of the subject token (an access token usually associated with the user principal), which will be provided to the IAS token server endpoint and exchanged for an IAS-signed SAML2.0 assertion.
- `X-chain-var-subjectTokenType`: Represents the type of the subject token, which will be provided to the IAS token server endpoint and exchanged for an IAS-signed SAML2.0 assertion.

Example response:

Sample Code

```
{
    "owner": {
        "SubaccountId": "<subaccount-id>",
        "InstanceId": "<instance-id>"
    },
    "destinationConfiguration": {
        "Name": "samlAssertionConsumerDest",
        "Type": "HTTP",
        "ProxyType": "Internet",
        "URL": "https://example.com",
        "Authentication": "OAuth2SAMLBearerAssertion",
        "tokenServiceURLType": "Dedicated",
        "tokenServiceURL": "https://example-token-service.com/oauth/token",
        "clientKey": "test-client-key",
        "SAMLAssertionProvider": "ClientProvided"
    },
    "authTokens": [
        {
            "type": "Bearer",
            "value": "<token-value>",
            "http_header": {
                "key": "Authorization",
                "value": "Bearer <token-value>"
            },
            "expires_in": "3600"
        }
    ]
}
```

Related Information

Destination Chaining [page 1538]

IAS-generated SAML Assertion to OAuth2 Bearer Token Chain [page 1540]

Configure Secrets for API Authentication

Consuming APIs from Other Applications

Create OpenID Connect Application for Token Exchange

Configure OpenID Connect Application for Token Exchange

Calling the Destination Service REST API [page 1562]

OAuth Token Exchange Authentication [page 1433]

OAuth SAML Bearer Assertion Authentication [page 1363]

1.1.3.4 Manage Certificates Issued by the SAP Cloud Root CA

Generate SAP Cloud Root CA certificates via the Destination service REST API, based on a locally generated and signed CSR (certificate signing request). In this scenario, the private key is not transferred over the Internet, and stored in the service.

Prerequisites

You have an SAP BTP subaccount. There are no specific requirements for the subaccount region.

Steps

> **Note**
>
> Each step is marked with "API", "UI" or "API and UI", to indicate for which administration scenario it is relevant.

API: Getting Access to the API

1. First, you need a service instance of the Destination service with service plan lite. You can do this in a Cloud Foundry space (using the cockpit or the Cloud Foundry CLI), or directly in the subaccount (the so called "Other Environment", using the cockpit or the SAP BTP CLI), or in Kubernetes/Kyma (using the SAP BTP service operator).
2. Create a binding and service key for the instance. We recommend that you use x509 bindings, but for quick tests you can also take client secrets (which is the default).
3. In the binding, you will see the base Destination service URL (field `uri`), the subaccount's token retrieval base URL (field `url`), the client ID (field `clientid`), and either the client secret (field `clientsecret`) or the mTLS properties.
4. Use the credentials to fetch a token which will then be used to call the REST API (the example uses `clientid` and `clientsecret`):

Sample Code

```
curl -X POST \
        "<url>/oauth/token" \
        -H "Content-Type: application/x-www-form-urlencoded" \
        -d "grant_type=client_credentials" \
        --data-urlencode "client_id=<clientid>" \
        --data-urlencode "client_secret=<clientsecret>"
```

5. We will use the `uri` from the binding and the `access_token` from the response of the token request later.

For more information, see also Calling the Destination Service REST API [page 1562].

UI: Navigate to the Destination Certificates UI

1. Open your subaccount in the SAP BTP cockpit.
2. Select *Destination Certificates* from the left panel (subaccount menu).

For more information, see Manage Destination Certificates [page 1504].

API and UI: Generating a Private Key and a CSR

1. We will use *openssl* for the purposes of the example to get the CSR, signed by a newly generated private key.
2. Execute:

Sample Code

```
openssl req -utf8 -nodes -sha256 -newkey rsa:3072 -keyout genExample.key
-out genExample.csr
```

Note

All the attributes of the CSR will get overridden during generation, so don't spend too much time on them.

3. The private key will result in a `genExample.key`. You will need it when you make use of the certificate. It will not be needed for the next steps of the certificate generation.
4. Get the CSR in base64-encoded form:

Note

`-w0` is used to avoid line breaks, but some flavors of base64 don't offer this option.

Sample Code

```
base64 -w0 genExample.csr
```

5. We will use the output of this command later.

API: Call the Destination Service REST API to Generate the Certificate

1. Execute:

   Sample Code

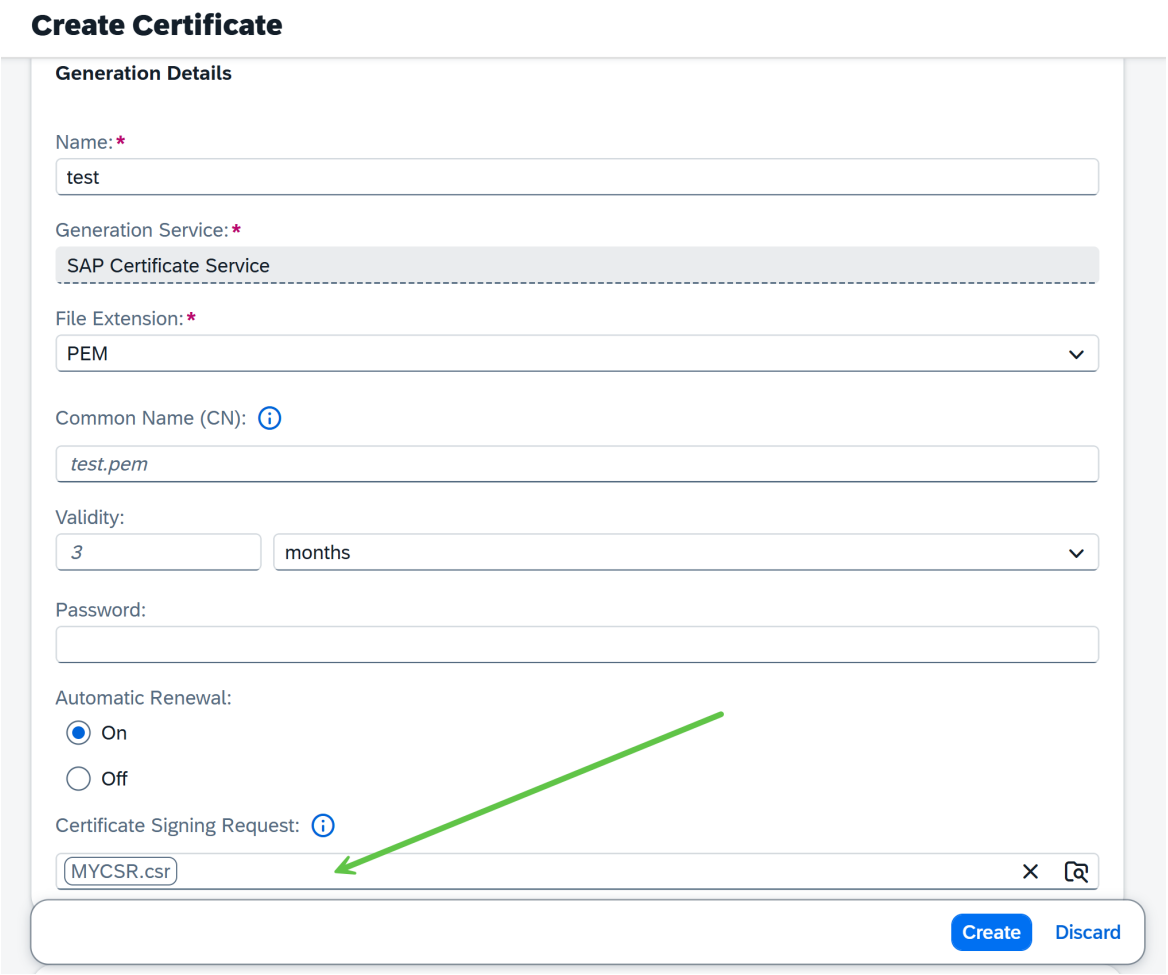
   ```
   curl --request POST \
   --url "${uri}/destination-configuration/v1/subaccountCertificates" \
   --header "Authorization: Bearer ${token}" \
   --header 'content-type: application/json' \
   --data """
   {
       \"Name\": \"example.pem\",
       \"Attributes\": {
           \"CN\": \"myExampleCertCn\",
           \"Validity\": {
               \"TimeUnit\": \"DAYS\",
               \"Value\": 30
           },
           \"AutomaticRenew\": true,
           \"CSR\": \"${csr}\"
       }
   }
   """
   ```

2. Note that the `${...}` values are the data collected in the previous steps.
3. You can specify the `Validity` of the certificate.
4. You can set an `AutomaticRenew` flag for the certificate.
   For more information, see Automatic Renewal of Generated Certificates [page 1513].
   If you do not use the auto-renew function, you may want to get notified about the certificate expiration.
   For more information, see Destination Service Notifications [page 1528].

For more information, see also SAP Business Accelerator Hub.

UI: Use the *Destination Certificates* Screen to Generate the Certificate

1. On the *Destination Certificates* screen, choose the *Create* button to generate a new certificate.
2. Specify `Name`, `CN`, expiration, renewal and format details.
3. Use the *Certificate Signing Request* file upload option to select your local CSR file (*genExample.csr*).



4. Choose *Create* to generate the certificate.

API and UI: Fetch your Newly Generated Certificate

> **Note**
>
> If you enabled automatic renewal, you should perform this fetching step periodically or when the renewal happens in order to be able to acquire the new certificate.

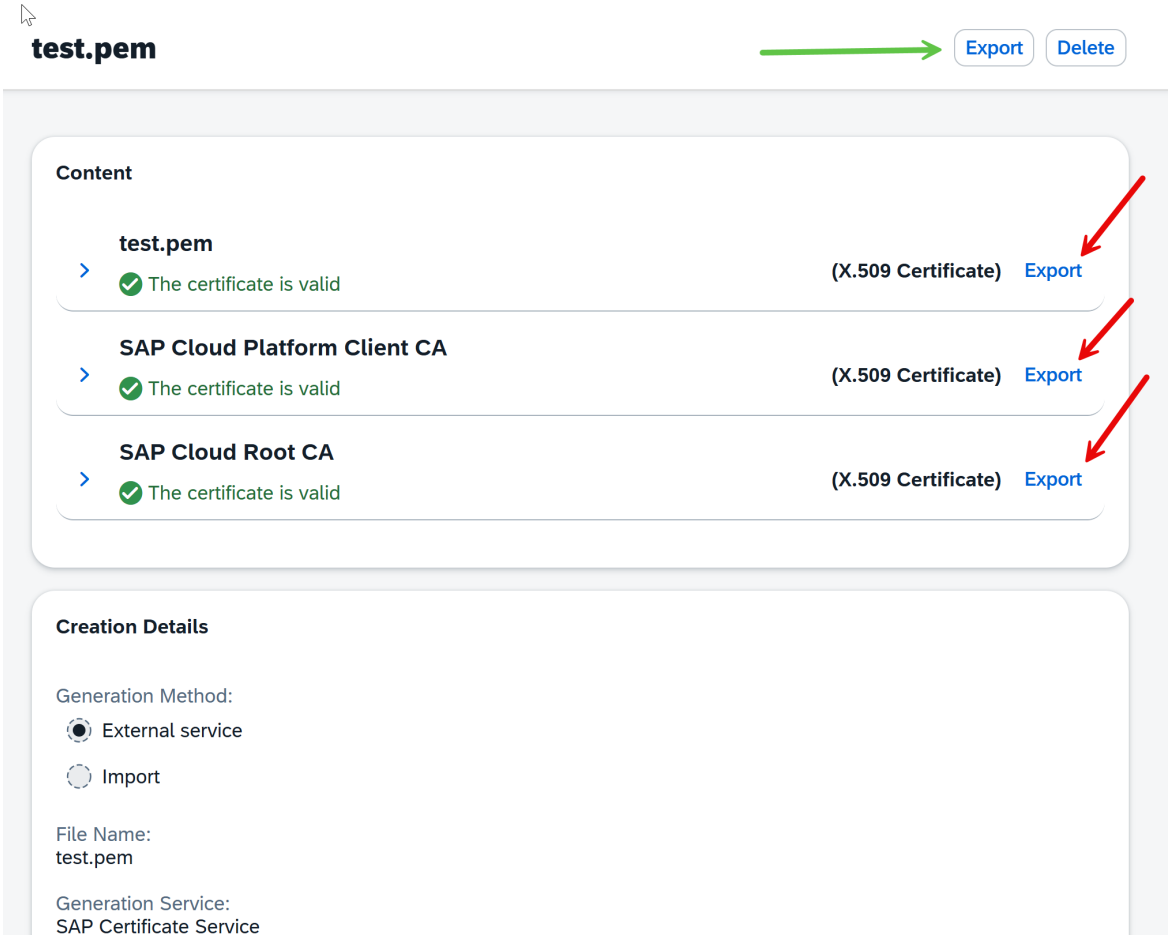
- Via REST API
  1. Execute:

     > **Sample Code**
     >
     > ```
     > curl --request GET \
     >     --url "${uri}/destination-configuration/v1/subaccountCertificates/
     > example.pem" \
     >     --header "Authorization: Bearer ${token}"
     > ```

  2. You will find the needed value in the `Content` field of the response JSON.
  3. Base64-decode the value of this field and you will get the certificate chain.

For more information, see also SAP Business Accelerator Hub (REST API).

- Via UI:
  1. On the *Destination Certificates* screen, select the generated certificate.
  2. Choose the main *Export* button to get the whole chain (green arrow) or the *Export* button on the individual nodes to get only one of the certificates in the chain (red arrows).



For more information, see also Manage Destination Certificates [page 1504] (cockpit).

1.1.3.5 Technical User Access to SuccessFactors

Configure *technical user propagation* for connections from SAP BTP to SuccessFactors.

SuccessFactors supports OIDC (OpenID Connect) for authentication. This includes technical users, implemented via *OAuth Client Credentials*. This flow should be used when you do not need to propagate the identity of the logged-in user towards *SuccessFactors*. Instead, all calls will be made on behalf of the same user, represented by an OAuth client.

Prerequisites

Follow the SuccessFactors guide to complete the OIDC setup: Register Your Own Application to Communicate with SAP SuccessFactors HCM Suite with OpenID Connect. Perform the options for technical user access.

Configuring the Destination

The interaction is modeled as an *OAuth Client Credentials Authentication* destination.

For more information on details about specific flavors (like using mTLS instead of a client secret for the token fetching), see OAuth Client Credentials Authentication [page 1375].

Sample Code

```
{
    "Name": "sfsftechuser",
    "Type": "HTTP",
    "Description": "",
    "URL": "https://<sfsf url>",
    "ProxyType": "Internet",
    "Authentication": "OAuth2ClientCredentials",
    "clientId": "<oidc client id>",
    "clientSecret": "<oidc client secret>",
    "tokenServiceURL": "https://<SAP Cloud Identity Services tenant url>/
oauth2/token",
    "tokenServiceURLType": "Dedicated",
    "tokenService.body.resource":
"urn:sap:identity:application:provider:name:<sfsf dependency name>"
}
```

Where:

- `<sfsf url>` is the URL of the SuccessFactors system.
- `<oidc client id>` is the client ID of the OIDC application, created during the setup.
- `<oidc client secret>` is the client secret of the OIDC application, created during the setup.
- `<SAP Cloud Identity Services tenant url>` is the host of the *SAP Cloud Identity services* tenant. The operations are performed in the context of this host.
- `<sfsf dependency name>` is the name of the dependency created from the OIDC application towards SuccessFactors during the setup phase.

Consuming the Destination and Executing the Scenario

Use the Find a Destination REST API [page 1562] or the Consume REST API [page 1569] to fetch the destination, including the access token that the Destination service will fetch for you.

1.1.3.6 Connectivity Scenarios: Examples

Find some typical examples for Connectivity scenarios and the corresponding Connectivity services, components and processes needed.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Cloud to Cloud

Connecting Cloud Foundry Applications to Cloud Systems

You can consume a cloud service or system from SAP BTP, Cloud Foundry environment. Use the Destination service to store and manage your connection configuration data (including credentials, certificates, URL, headers, queries, etc.) at design time, and automate the OAuth2 token retrieval process at runtime for your application.

For more information, see Destination Service [page 1347].

Connecting Kubernetes Applications to Cloud Systems

You can consume a cloud service or system from your Kubernetes environment. Use the Transparent Proxy for Kubernetes to enhance developers' user experience and simplify the consumption of end systems. For example:

- Developing an application that requires consumption of an SAP HANA database.
- Adding new functionality that requires the consumption of an SAP BTP service.
- Creating new SAP Fiori-based user interfaces that require consumption of an OData service.
- Adding new functionality that requires integration with third-party APIs.
- An application requires data exchange or service consumption via a RESTful API.

For more information, see Transparent Proxy for Kubernetes [page 2164].

Cloud to On-Premise

Connecting Cloud Foundry Applications to On-Premise Systems

You can consume an on-premise system from SAP BTP, Cloud Foundry environment. Use the Connectivity service and the Cloud Connector to implement a specific scenario. For example:

- Creating new SAP Fiori-based user interfaces that require consumption of an OData service.

- An application requires data exchange or service consumption via a RESTful API.
- Extending an application that requires consumption of an ABAP system via RFC.
- Develop an application that requires interaction with a mail server for E-mail operations.
- Develop an application that requires interaction with a relational or non-relational database (e.g. SAP HANA, MySQL, PostgreSQL, MongoDB).
- A new functionality in an application requires user authentication and directory services integration using an LDAP server.
- An application needs to transfer files to and from an FTP server.
- Enable single sign-on (SSO) by forwarding the identity of cloud users to a remote system or service (user propagation / principal propagation).

For more information, see Connectivity Service [page 1308], Destination Service [page 1347], Cloud Connector [page 1602], Principal Propagation [page 30].

Connecting Kubernetes Applications to On-Premise Systems

You can consume an on-premise system from your SAP Kyma, or any other Kubernetes environment. Use the Connectivity Service, Connectivity Proxy, Transparent Proxy, and Cloud Connector to implement a specific scenario. For example:

- Creating new SAP Fiori-based user interfaces that require consumption of an OData service.
- An application requires data exchange or service consumption via a RESTful API.
- Extending an application that requires consumption of APIs in an ABAP system via RFC.
- Develop an application that requires interaction with a mail server for E-mail operations.
- Develop an application that requires interaction with a relational or non-relational database (e.g. HANA, MySQL, PostgreSQL, MongoDB).
- A new functionality in an application requires user authentication and directory services integration using an LDAP server.
- An application needs to transfer files to and from an FTP server.
- Enable single sign-on (SSO) by forwarding the identity of cloud users to a remote system or service (user propagation / principal propagation).

For more information, see Connectivity Service [page 1308], Destination Service [page 1347], Connectivity Proxy for Kubernetes [page 2088], Transparent Proxy for Kubernetes [page 2164], Cloud Connector [page 1602], Principal Propagation [page 30].

On-Premise to Cloud (Service Channels)

You can connect to a cloud service from your on-premise network using service channels of the Cloud Connector. For example:

- Configure an RFC connection from your on-premise system to SAP S/4HANA Cloud.
  For scenarios that need to call from on-premise systems to SAP BTP, ABAP environment using RFC, you can establish a connection to an ABAP Cloud tenant host.
- Configure a connection to a service in a Kubernetes cluster.

Create a service channel to establish a connection to a service in a Kubernetes cluster that is not directly exposed to external access.

For more information, see Using Service Channels [page 1949].

1.1.4 What's New for Connectivity

Find the latest features, enhancements and bug fixes for SAP BTP Connectivity .

What's New for Connectivity

Related Information

2022-2024 Connectivity (Archive) [page 142]
2021 Connectivity (Archive) [page 1258]
2020 Connectivity (Archive) [page 1276]
2019 Connectivity (Archive) [page 1285]
2018 Connectivity (Archive) [page 1292]
2017 Connectivity (Archive) [page 1296]

1.1.4.1 2022-2024 Connectivity (Archive)

2022

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.17.2 - Bug Fixes	Release of Cloud Connector version 2.17.2 provides the following bug fixes: • The connection check for access	Info only	General Availability	Changed	Technology	Not applicable		2024-10-31	2024-10-31	2.17.2	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			scontrolentriescouldfailevenifthecon nectionwouldwor										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			kfinein productive usages. This was caused by a changed de										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			fault value introduced in Tomcat 9.0.90. With the upgrade										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			to Tomcat 9.0.95, the problem has been resolved. This was a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			regression in 2.17.1. • When upgrading a shadow instance										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			, the master high availability (HA) portgot lost during the u										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			pgrade. This was a regression in 2.17.1. • Pressing Caps L										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ockwhenenteringapasswordintoCloudConnector'slogin										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			screendeletedthepasswordcharacterstha twerenter										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			edsofar.Thiswascausedbyaregressioninui5Core1.108										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			.30thatwasusedin2.17.1. • Intheconnectionmonitordeta										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ilsviewtheactiverequeststswerenotshowingtheusertha t										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			is invoking the service, even if the user is actually known										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			. This was a regression introduced in 2.16.0. • Due to a fau										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			It consistency check, the customregion list contained										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			all the known regions that were in use on the Cloud Connect										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			or. This is legal state has been corrected. • When using added										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			icatedHAportandresettingtheHAsettingsonashadow,th										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			eshadow's UI port within the HA information was wrongly se										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t t o t h e H A p o r t. A s a c o n s e q u e n c e , a f t e r c o n n e c t i n g a g a i n t										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			oamaster,itwasnotpossibletoopenthesharedowUlusingth										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e b u t t o n i n t h e m a s t e r U I i n t h e *H i g h A v a i l a b i l i t y* s c r e e n .										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.17.2 - Enhancements	Release of Cloud Connector version 2.17.2 provides the following enhancements: • Introduced a Caps Lock warn	Info only	General Availability	New	Technology	Not applicable		2024-10-31	2024-10-31	2.17.2	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ingonallpasswordinputfields.Previouslly,suchawarni										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ngwasonlyshownonthelogiinscreen. • CloudConnectordo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			esnolongerallowtouploadUIcertificatesfromaP12filethh										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ati nclude the Key Usage certs igning. Such certifi cates a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rejected by all browsers for identifying a server. • Th										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e c o n n e c t i o n c h e c k f o r a c c e s s c o n t r o l e n t r i e s i s n o w a l l o w e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			for all authenticated users. • When opening TCP connection										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			successfully, an audit log entry is written that a access to o										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			the system has been allowed.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Cloud Connector - New Regions	Additional regions were added to SAP BTP: • ap30: Australia (Sydney) - Google Cloud • br2	Info only	General Availability	Announcement	Technology	Not applicable		2024-10-28	2024-10-28	2410b	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			0: Brazil (São Paulo) - Azure • sa30: KSA (Dammam) - Google C										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			loud For more information, see Cloud Connector: Prerequisites. In case of questions please contact SAP Support.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - REST API	The "Get All Destinations" endpoint of the service's REST API now supports suffix-based (`endsWith`) filtering on the `Name` property.	Info only	General Availability	New	Technology	Not applicable		2024-10-28	2024-10-31	2410b	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Connectivity Proxy for Kubernetes - New Features	Connectivity Proxy version 2.13.0 provides the following new features: • The Connectivity Proxy now exposes	Info only	General Availability	Announcement	Technology	Not applicable		2024-10-17	2024-10-17	2.13.0	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			various JVM and component-specific *Prometheus* metrics.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			• [Helm] You can now configure the topology key for the pod a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nti-affinity of each pod related to the Connectivity Proxy. Th										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			is gives you more control over pod scheduling. • [Helm] You										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			cannowconfiguretherequestandlimitresourceconsumption										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ts for the *Region Configuration Controller* microservice										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			of the Connectivity Proxy. • Support for custom PKI (public										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			eyinfrastructure): The Connectivity Proxy now provides a

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			list of certificate authorities which it should trust in addi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ontothedefaultonewhenperformingoutboundHTTPScall										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sisnowsupported.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Connectivity Proxy for Kubernetes - Security Fixes	Connectivity Proxy version 2.13.0 provides the following security fixes: • Multiple OS vulnerabilities by	Info only	General Availability	Changed	Technology	Not applicable		2024-10-17	2024-10-17	2.13.0	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			removing or upgrading packages on the image were										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			fixed. • Multiple Java vulnerabilities by upgrading d										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e p e n d e n c i e s t o a n e w e r v e r s i o n w e r e f i x e d .										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Cloud Connector - IP Addresses	Additional IP addresses will be added to all SAP BTP regions running on Amazon Web Services (AWS) and Microsoft Azure. **Action:** If you restrict system access by *allow-listing*	Required	General Availability	Announcement	Technology	Not applicable		2024-10-14	2024-10-14	2410a	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			IPs in firewall rules, **make sure you update your configuration** as soon as possible. **The additional IP addresses will be used after January 19, 2025.** ⓘ No										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			te Current IP addresses will continue to function (a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ndthereforemustnotberemoved)forsometimeaft										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			erthatdate.Afollow-upannouncementwillbeprovide										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			doncethe oldIPscanberemovedsafely.										
			For more information, see Cloud Connector:										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Prerequisites. In case of questions please contact SAP Support.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - New Features	Transparent Proxy version 1.6.0 provides the following new features: • You can now define added capped	Info only	General Availability	New	Technology	Not applicable		2024-09-05	2024-09-05	1.6.0	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ortonwhichadestinationisconsumableinsideaKu										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			between test cluster. For more information, see Destin										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ationCustomResource. • Use the *Transparent Proxy*										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Operator in regular Kubernetes cluster with basi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ngle-file installation. The *Transparent Proxy Oper*										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			a t o r p r o v i d e s e n h a n c e d m o n i t o r i n g o f t h e T r a n s p a r										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			entProxy,ensuringitrunsoptimallyaccordingtoth										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e p r o v i d e d c o n f i g u r a t i o n . F o r m o r e i n f o r m a t i o n , s e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e Installation with Operator. • Seamless consumption o										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			fsystemsdefinedasLDAPdestinations.Workwithsys										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t e m s t h r o u g h t h e L D A P p r o t o c o l , l e v e r a g i n g t h e f e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			atureprovidedbytheTransparentProxy.Formo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			reinformation, see LDAP Destinations. • Seamlessco										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nsumptionofsystemsdefinedasMALLdestinations.W										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			orkwithsystemsthroughtheSMTP,POP3,andIMAPp										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rotocols, leveraging the feature provided by t										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			he Transparent Proxy. For more information, see M										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			All Destinations.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - Enhancements	Transparent Proxy version 1.6.0 provides the following enhancements: • The Transparent Proxy now re	Info only	General Availability	Changed	Technology	Not applicable		2024-09-05	2024-09-05	1.6.0	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sponds to Connectivity Proxy updates, providinge										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nhancedsynchronizationbetweencomponents. • Im										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			provided user experience in the Key management by s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			etting the initially created Destination service instances										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tance as the default. This is eliminate the need for rm										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			anualconfigurationandreducesthecomplexityofs										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			etting up the Transparent Proxy. For more information										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tion, see Destination Service Integration. • Improve										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			d u s e r e x p e r i e n c e i n t h e K y m a e n v i r o n m e n t b y c r e a t i n										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ga "gateway" destination customer resource by default										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			It, allowing you to start using destinations immedia										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tely without additional configuration in the Key main										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			stance. For more information, see Destination Gate										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			way. • Added extension to the Key add a shboard for c										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onfiguration management. You can now create, updat										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e, and deletedestination customresource updata										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t e t h e T r a n s p a r e n t P r o x y c o n f i g u r a t i o n u s i n g t h e n										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ewuserinterfaceviews. • Destinationscannowbeco										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nsumeddirectlyfromthedestinationcustomresourc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			enamespacewithoutspecifyingtheTransparentPr										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			oxyname space in the URL. For more information, se										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e Using the Transparent Proxy.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - Bug Fixes	Transparent Proxy version 1.6.0 provides the following bug fixes: • Fixed an issue with incorrect typ	Info only	General Availability	Changed	Technology	Not applicable		2024-09-05	2024-09-05	1.6.0	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			arsedd otsin the *Addr esss* field of SAP BTP destinatio										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nsoftypeTCP.Youcannowdefineaddressesswithout										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			anycharacterlimitation.Formoreinformation,see										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			TCP Destinations. • Fixed multiple Golang vulnerabil										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			it i e s b y u p g r a d i n g d e p e n d e n c i e s t o a n e w e r v e r s i o										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			n.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.18.0 - OS Versions	With the next planned feature release of the Cloud Connector (2.18.0), we will discontinue support of the following OS versions: • Windows 7, 8.1	Info only	General Availability	Announcement	Technology	Not applicable		2024-08-08	2024-08-08	2.18.0	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			• Windows Server 2012, 2012 R2 • SUSE Linux Enterprise Server										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			r 12 • RedHat Enterprise Linux 7 • macOS 10.14, 10.15, 11 (x86										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			_64) The following OS versions do no longer work for existing releases already: • SUSE Linux Enterprise Se										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rver11 • RedHatEnterpriseLinux6 • macOS10.7-10.13(x86_										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			64)										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.17.1 - Bug Fixes	Release of Cloud Connector version 2.17.1 provides the following bug fixes: • The subaccount authentica	Info only	General Availability	Changed	Technology	Not applicable		2024-08-08	2024-08-08	2.17.1	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tiondataexpirationwasnotcalculatedcorrectlyan										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			d r e p o r t e d a s e x p i r e d , e v e n i f s t i l l v a l i d . Re-gres-sions in 2.17.0 : • L D										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			A P s e c u r e h o s t f l a g s (*S e c u r e H o s t a n d S e c u r e A l t e r n*										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ateHost)werenneithherstoredinthheLDAPdraftconf										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			i g u r a t i o n n o r i n t h e f i n a l c o nf i g u r a t i o n w h e n p r e s s i										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ng the *Save Draft* or *Activate* button. • On the subacc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ountoverviewpage,thereg ionhostcouldhavebe										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			enshownasnotreachablewithmessage*Proxycould*										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			not be reached, event though this subaccount is sho										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			w n a s c o n n e c t e d a n d t h e r e i s n o p r o x y i n u s e . P r o d u										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ctivecon nectionsw erenot affected. • Adding as subac										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			c o u n t w a s n o t p o s s i b l e , if B T P i s r e a c h e d o n l y v i a a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			proxy and the proxy returns an unexpected status										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			code on check. • CPI Ctr acefiles were neither shown n										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ordownloadablefromtheUI. • The *OpenShadow* but to										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			n a t t h e t o p o f t h e *H i g h A v a il a b il it y* s c r e e n d i d n o t r e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sultinanyaction. • Ifaddingan RFCSNCaccesscontro										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			lentrywithoutloadbalancinglogon,thevirtualp										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			orthadanaddition al's 'atthheend.Suchanentryc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ould not be used from the cloud side. These issues have been fixed.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.17.1 - Enhancements	Release of Cloud Connector version 2.17.1 provides the following enhancement: • Additional filters were added i	Info only	General Availability	New	Technology	Not applicable		2024-08-08	2024-08-08	2.17.1	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nthe access control table in which the mappings of virtua										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			l t o i n t e r n a l s y s t e m s a r e l i s t e d .										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Java Connector 3.1.10.0 - Bug Fixes	Release of Java Connector (JCo) version 3.1.10.0 provides the following bug fixes: • If a remote-enabled func	Info only	General Availability	Changed	Technology	Not applicable		2024-07-11	2024-07-11	3.1.10.0	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tionmodule(RFM)hasmultipleTABLESparameters,										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			the content of one or more received JCoTable objects										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ectscouldhavebeeninincorrect,allrowsoffaffected										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tablesbeingonlypartlyfilled.Inthiscase,eachr										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			owwaspopulatedwithdataaonlyuptoacertaaincolu										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			m n a n d f o ll o w i n g fi e l d s w e r e e m p t y o r c o n t a i n e d t h e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			irtype-specificinitialvalue.Thefullcontentoft										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			h e f i r s t T A B L E S p a r a m e t e r w a s a l w a y s c o r r e c t. T h i s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			p r o g r a m e r r o r o c c u r r e d u n d e r s p e c i a l c o m p l e x p r										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			econditions, for which the affected TABLESparame										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t e r s n e e d e d t o h a v e a c e r t a i n l a y o u t s t r u c t u r e o f										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			fieldtypes, while the affected table contentwas be										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ingtransferredincompressedrow-basedserializ										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ation format.										

> **Note**
>
> This was a regression bug w

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			hichhadbeeniintroducedwithJCo3.1.9.0 • Changing										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			edvaluesofpropertyjco.destination.repos										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			itory_destination,whichwerereturnedfromt										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			hecurrentlyactiveDestinationDataProvideri										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nstanceforagivendestinationnname,werenotproc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			essed correctly by the JCo runtime, and already exis										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ting JCoDestinationinstancesforthatdestinati										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onnamewerenotupdatedwiththenewvalue.Suchex										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			isting JCoDestination instance still returned										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			hold JCo Repository instance, although heavy

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			entonemightbeassociatedwiththenewlyconfigur										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			edrepositorydestination. These issues have been fixed.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Cloud Connector - SAP BTP Cockpit - Download Authentication Data	You can now download an authentication data file from your subaccount in the SAP BTP cockpit (*Connectivity* > *Cloud Connectors* > *Download Authentication Data*) to simplify	Info only	General Availability	New	Technology	Not applicable		2024-06-27	2024-06-27	2406a	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sub-account configuration in the Cloud Connector administration UI. For more information, see also Set up Connection Parameters and HTTPS Proxy (steps 2 + 4: file-based										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sub-account configuration).										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - Hotfix	Transparent Proxy version 1.5.2 provides the following hotfix: • Fixed an issue that could potentiall	Info only	General Availability	Changed	Technology	Not applicable		2024-05-29	2024-05-29	1.5.2	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			yincreaseCPUusageforTransparentProxypodsbeyondt										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			hethreshold, possibly resulting in failed requests. • Fix										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			edanissueinthheautomatedlstiointegrationwhichcould										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			previously prevent the Transparent Proxy from starting.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Java Connector 3.1.9.0 - Bug Fixes	Release of Java Connector (JCo) version 3.1.9.0 provides the following bug fixes: • Setting numeric string valu	Info only	General Availability	Changed	Technology	Not applicable		2024-05-30	2024-05-30	3.1.9.0	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			esintofieldsoftypeBCDwithmethodJCoRecord.s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			etValue([name|index],String)resultedinaC										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onversionExceptionifthenumericstringusedsc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			identification with an exponent. The same Converters										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ionExceptionoccurredwhendeserializingsuchs										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t r i n g s f o r f i e l d t y p e B C D f r o m J S O N i n p u t d a t a v i a m e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			thodJCoRecord.fromJSON([Readder|String]										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			). • Setting declinal number values into fields of type

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			peBCDwithmethodJCoRecord.setValue([name|i										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ndex],BigDecimal)did not throw a ConversionException										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Exception if the fractionpart of the provided jav										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			a.math.BigDecimal value had to many digits to										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			fully fit into the BCD field. The BigDecimal value w										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			a s r o u n d e d a n d s t o r e d i n t h e B C D fi e l d n e v e r t h e l e s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			s. Thus, the provided value's precision was lost										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			unnoticed. • If a JCoFunction uses a TABLES parame										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			terwithcompatibly-extendedmetadatahavingapp										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e n d e d a d d i t i o n a l c o l u m n s c o m p a r e d t o t h e p a r a m e t										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e r m e t a d a t a u s e d o n R F C c o m m u n i c a t i o n p a r t n e r s i d										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e, the first additional field of each transferred ro										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			w c o u l d c o n t a i n N U L L - b y t e s a n d / o r g a r b a g e c h a r a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			c t e r s . T h e o c c u r r e n c e o f t h i s p r o g r a m e r r o r d e p e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nded on the table row's fields structure and when the

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			r a l i g n m e n t b y t e s a r e r e q u i r e d f o r i n t e r n a l l y o r g a n										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			izingthetableecontentdataaornot. These issues have been fixed.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Java Connector 3.1.9.0 - Enhancements	Release of Java Connector (JCo) version 3.1.9.0 provides the following enhancements: • JCo 3.1.9.0 now supports	Info only	General Availability	New	Technology	Not applicable		2024-05-30	2024-05-30	3.1.9.0	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tsSapMachine21. • WebSocketRFCclientcommunica										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tionhasbeenimprovedbyaddingaping/pongpacke										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t e x c h a n g e m e c h a n i s m f o r k e e p i n g W e b S o c k e t R F C c										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onnectionsaliveswhileswaitingforanRFCresponsefr										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			o m t h e c o m m u n i c a t i o n p a r t n e r. H e n c e , a n u n d e r l y i n g										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			TCP/IP network connections should not get closed a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nymorebyfirewallsorothernetworkdevicesduetoo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			apotentialnetworktraffic idletimeoutwhilewaiting										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			for long running requests being processed on the										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			WebSocketRFCcommunicationpartnerserverside										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e. Corresponding pinging periods and pong timeouts ar										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			econfigurable.Findmoredetailsaboutthenewpro										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			p e r t i e s j c o . d e s t i n a t i o n . w s _ p i n g _ p e r i o d a n										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			d j c o . d e s t i n a t i o n . w s _ p o n g _ t i m e o u t i n T a r g										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			etSystemConfiguration(CloudFoundryenvironmen										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t).										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Neo	Java Connector 3.1.9.0 - Bug Fixes	Release of Java Connector (JCo) version 3.1.9.0 provides the following bug fixes: • Setting numeric string valu	Info only	General Availability	Changed	Technology	Not applicable		2024-05-16	2024-05-16	3.1.9.0	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			esintofieldsoftypeBCDwithmethodJCoRecord.s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			etValue([name|index],String)resultedinaC										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onversionExceptionifthenumericstringusedsc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			identification with an exponent. These same Converters										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ionExceptionoccurredwhendeserializingsuchs										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t r i n g s f o r f i e l d t y p e B C D f r o m J S O N i n p u t d a t a v i a m e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			thodJCoRecoord.fromJSON([Readder|String]										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			). • Setting decimal number values into fields of type

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			peBCDwithmethodJCoRecord.setValue([name|i										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ndex],BigDecimal)did not throw a ConversionException										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Exception if the fractionpart of the provided jav										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			a.math.BigDecimal value had too many digits to										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			fully fit into the BCD field. The **BigDecimal** value w										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			a s r o u n d e d a n d s t o r e d i n t h e B C D fi e l d n e v e r t h e l e s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			s. Thus, the provided value's precision was lost										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			unnoticed. • If a JCoFunction uses a TABLES parame										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			terwithcompatiblyextendedmetadatahavingapp										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			endedadditionalcolumnscomparedtotheparamet										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e r m e t a d a t a u s e d o n R F C c o m m u n i c a t i o n p a r t n e r s i d										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e , t h e fi r s t a d d it i o n a l fi e l d o f e a c h t r a n s f e r r e d r o										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			w c o u l d c o n t a i n N U L L - b y t e s a n d / o r g a r b a g e c h a r a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			c t e r s . T h e o c c u r r e n c e o f t h i s p r o g r a m e r r o r d e p e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nded on the table row's fields structure and when the										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			r a l i g n m e n t b y t e s a r e r e q u i r e d f o r i n t e r n a l l y o r g a n										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			izingthetablecontentdataaornot. These issues have been fixed.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Neo	Java Connector 3.1.9.0 - Enhancements	Release of Java Connector (JCo) version 3.1.9.0 provides the following enhancements: • JCo 3.1.9.0 now requir e	Info only	General Availability	New	Technology	Not applicable		2024-05-16	2024-05-16	3.1.9.0	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sthe Microsoft Visual Studio 2015-2022 C/C++ ru										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			n t i m e l i b r a r i e s t o b e i n s t a l l e d o n t h e s y s t e m . T h i s i s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			relevant when using the SAP BTP SDK for the Neo env										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ironmentonWindowsoperatingsystems. • JCo3.1.9.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			0 nowsupportsSapMachine21. • WebSocketRFCClien										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tcommunicationhasbeenimprovedbyaddingapiing										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			/pongpacketexchangemechanismforkeepingWeb										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			SocketRFCconnectionsalivewhilewaitingforanRFC										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			r e s p o n s e f r o m t h e c o m m u n i c a t i o n p a r t n e r. H e n c e , a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nunderlyingTCP/IPnetworkconnectionshouldnot										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			getclosedanymorebyfirewallsorothernetworkde										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			vicesduetoapotentialnetworktraffic idletimeout										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			while waiting for long running ingress requests being proc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			essedonthe WebSocket RFC communicationpartner										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			server side. Corresponding pinging periods and pooling										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			timeout sare conf igurable. Find more detail s about										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t h e n e w p r o p e r t i e s j c o . d e s t i n a t i o n . w s _ p i n g _										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			period and jco.destination.ws_ping_timeo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			utin TargetSystemConfiguration(Neoenvironmen										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t).										
Connectivity	Cloud Foundry Neo	Cloud Connector - Direct Upgrades	**Important information**: Direct Cloud Connector upgrades require a start release of at least 2.13. For versions before 2.13, a two-step upgrade via 2.16 must be done.	Info only	General Availability	Announcement	Technology	Not applicable		2024-05-02	2024-05-02	2.17.0	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.17.0 - Bug Fixes	Release of Cloud Connector version 2.17.0 provides the following bug fixes: • APATCH operation for */api/v1/*	Info only	General Availability	Changed	Technology	Not applicable		2024-05-02	2024-05-02	2.17.0	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			configuration/connector/ui/uiCertificate is no longer used										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			is allowed on the shadow instance. • The shadow has available b										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ility (HA) state is now also stored when end is connecting, sot										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			hat inconsistent states in the HA setup can be avoided.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.17.0 - Enhancements	Release of Cloud Connector version 2.17.0 provides the following enhancements: • CloudConnector2.17isbas	Info only	General Availability	New	Technology	Not applicable		2024-05-02	2024-05-02	2.17.0	

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e d o n a d i f f e r e n t r u n t i m e c o n t a i n e r . S o f a r , i t w a s b a s e d o n t h e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			JavaWeb3.xruntimeonTomcat8.5,nowitswitchedtoJavaWe										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			b4.x, which is based on Tomcat 9. • The Cloud Connector can										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nowuseSAPMachine21asJavaruntime.Formoreinforma										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tion, see Prerequisites. • CloudConnector 2.17 supp or										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t s u p t o 3 L D A P s e r v e r s f o r a u t h e n t i c a t i o n . T h i s a l l o w s t o s u										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			p p o r t s e t u p s i n w h i c h t h e u s e r b a s e i s n o t c o m p l e t e l y a v a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ilableinasingleLDAPuserstore,orifthereareemultipleu										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			serbasesinasingleLDAPuserstore.Formoreinforma										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tion, see Use LDAP for User Administration. • You can c										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onfigurease parate port for the high availability (HA) -rel										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			atedcommunicationbetweenthetwoinstancesmasteran										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			dshadow). This let s you use HA to get her with certificate-ba										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sedauthentication. For more information, see Install										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			a Failover Instance for High Availability. • Additional										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			hardware monitoring RESTAPIs for disk and CPU status have										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			been provided. For more information, see Monitoring A										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Pls. • You can now use the hardware remote monitor on the shadow in										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			stance as well. For more information, see Hardware Me										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			trics. • A description for subject patterns was introduce										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			d. For more information, see Configure Subject Pat										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			terns for Principal Propagation. • Access control data										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			anowcontainsthecreationtimestamp.Formoreinforma										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tion, see Configure Access Control. • You can now add a su										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			b a c c o u n t u s i n g a n a u t h e n t i c a t i o n d a t a f i l e d o w n l o a d e d f r o										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			mSAPBTP.Formoreinformation,seeSetupConnectio										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nParameters and HTTPSProxy (steps 2+4: file-bas										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			edsubaccountconfiguration). • TheCloudConnectorUIn										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			owprovidesasessionexpirationprogressbarinthetopright										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			hticornerofeachscreen,indicatinghowlongyourcurren										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tsessionistillvaliduntilyouneedtologinagainbefore p										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			roceeding. For more information, see Initial Config										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			uration. • You can now run the new script `resetCiphers.sh` to reset the ciphers for the										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			mthescc folder in the Cloud Connector installation directory										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ory to reset the cipher sif UI access is blocked due to wron										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			gcipher settings. For more information on on cipher suit										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			es, see Encryption Ciphers.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - New Features	Transparent Proxy version 1.5.0 provides the following new features: • The Transparent Proxy module	Info only	General Availability	New	Technology	Not applicable			2024-04-18		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			inKymanowautomaticallydetectsandintegratesw										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			with the Connectivity Proxy module, reducing the need

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			d f o r m a n u a l c o n f i g u r a t i o n . F o r m o r e i n f o r m a t i o n , s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ee Transparent Proxy in the Kyma Environment. • Train										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			s p a r e n t P r o x y s u p p o r t s t h e *D e s t i n a t i o n F r a g m e n t* f										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			eature, allowing for the extension of additestination										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			with a fragment. The destination fragments themselv										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			esenablemoreflexibletechnicalconnectionconfi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			guration management for the solution administrat										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ors. Now, using the Transparent Proxy, you can ea										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			silyconsumetargetsystemsdefinedasdestination-										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			fragment pairs. For more information, see Extendi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ngDestinationswithFragmentsandDestinationFragm										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ents. • Transparent Proxy now supports an integ										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ration with Connectivity Proxy in the so-called multi-r										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			egionsetup.Thisallowsyoutolinkthemulti-regionsetup.Thisallowsyoutolinkthemulti-region										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			related configuration between the two components										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			, resulting in simplified consumption at runtime. For										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			more information, see Connectivity Proxy Integrati										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			on.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - Enhancements	Transparent Proxy version 1.5.0 provides the following enhancements: • Enhanced Integration with Con	Info only	General Availability	New	Technology	Not applicable			2024-04-18		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nectivityProxyinKubernetesCluster. This is upda										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t e a l l o w s f o r s i m p l e r i n t e g r a t i o n w i t h t h e C o n n e c t i v

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ity Proxy outside of the Key maenvironment. Users can

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nowlinkthetwocomponentsbysimplypointingtoth										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			eConnectivityProxyservicename.Formoreinform										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ation, see Connectivity Proxy Integration. • Optimize										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			d r e c o n c i l i a t i o n p r o c e s s i n T r a n s p a r e n t P r o x y . T h										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			isoptimizationallowsforquickersynchronization										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			with the SAP Destination service.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - Bug Fixes	Transparent Proxy version 1.5.0 provides the following bug fixes: • Fixed multiple OS vulnerabilities by	Info only	General Availability	Changed	Technology	Not applicable			2024-04-18		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			removing or upgrading packages on the image. • Fixed multi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			pleGolangvulnerabilitiesbyupgradingdependenciesto										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			answerversion.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - Enhancement	The automatic token retrieval functionality of the "find destination" API now supports MAIL destinations with authentication `BasicAuthentication` and all available types of OAuth au-	Info only	General Availability	New	Technology	Not applicable			2024-03-21		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			thentication. For more information, see Create Mail Destinations.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - New Feature	A new entity called *destination fragment* has been introduced. Each fragment is a key:value pair and is used to extend and override the properties of a destination at runtime. This lets you avoid	Info only	General Availability	New	Technology	Not applicable			2024-03-12		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			configuration duplication by keeping common properties within a destination and extracting various specific properties into dedicated fragments. At runtime the application can decide										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			which fragment to apply to the base destination, based on the particular scenario. You can use destination fragments through the Destination service REST API. For more infor-										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			mation, see Destination Fragments.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Connectivity Proxy for Kubernetes - Kyma Integration	As previously announced (February 26, 2024), on-premise connectivity in the Kyma environment will be handled by a Kyma module called *connectivity-proxy*. This module is now available and	Info only	General Availability	Announcement	Technology	Not applicable			2024-03-12		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			can be activated by customers. The result is backward-compatible to the previous built-in solution, but the activation involves a short migration down time (see previous announ										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			cement).										
			Note Keep in mind that all remaining users usin										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			gthepreviousssolutionwillbemigratedcentrallyonMarc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			h16 within the announced maintenance window. For more information,										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			see On-Premise Connectivity in the Kyma Environment.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Connectivity Proxy for Kubernetes 2.12.1 - Enhancements	Release of Connectivity Proxy version 2.12.1 provides the following enhancement: You can now configure a `nodeSelector` section for each pod related to the Connectivity Proxy.	Info only	General Availability	New	Technology	Not applicable			2024-03-21		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			As an operator, this lets you control on which worker nodes these pods will be scheduled. For more information, see Configuration Guide.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Connectivity Proxy for Kubernetes 2.12.1 - Bug Fixes	Release of Connectivity Proxy version 2.12.1 provides the following bug fixes: • [Helm] Changes in the port of t	Info only	General Availability	Changed	Technology	Not applicable			2024-03-21		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			hebusinessdataatunnelwerenotproperlytakenintoacco										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			unt, resulting in inconsistent configurations. • [Helm] Cha										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ngesinthelstiogatewayselectorwerenotproperlytaken										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			into account, resulting in inconsistent configurations.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			• Multiple OS vulnerabilities by removing or upgrading pack										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ages on the image were detected. • Multiple Java vulnerabil										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			itiesbyupgradingdependenciestoanewerversionweredeployed										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			etected. These issues haave been fixed.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Connectivity Proxy for Kubernetes - Announcement	The Connectivity Proxy in the Kyma runtime will be **converted to a standard Kyma third party module**. All existing instances of the Connectivity Proxy must be converted to the new module-based	Info only	General Availability	New	Technology	Not applicable			2024-02-26		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			solution. The **migration is automated** and happens upon enabling the Connectivity Proxy module. After the migration, the Connectivity Proxy will be consumable in the same way										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			as before. The migration has two stages. • As of **March 2, 2024**, the module will be										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			available via the 'fast' channel. **All customers can switch**										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			enenableit and market hes witch (it will also be availa										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			before new installations). • Then, on **March 16, 2024**,										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			during maintenance window, **all remaining connec**										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tivityproxiesprovisionedviatheoldapproach										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			will automatically be migrated to the module by SA										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			P. While the migration results in a backward-compatible deployment of the Connectivity Proxy, the process involves **a downtime of around 2 minutes** (depending on cluster										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			load and other factors). In case of questions or issues, please raise a support ticket on the BC-CP-CON-K8S-PROXY component.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Cloud Connector - IP Addresses	ⓘ Note This release note refers to the same note	Required	General Availability	New	Technology	Not applicable			2024-02-22		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			from November 16, 2023, now including a concrete timeli										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			mit(date)forupdatingyourfirewallrules.										
			Additional IP addresses were added to										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			SAP BTP regions running on Amazon Web Services (AWS). The following regions are affected by this change: • br10 – Brazil (São										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Paulo) • jp10–Japan(Tokyo) • ap10–Australia(Sydn										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ey) • ap11_AsiaPacific(Singapore) • ap12_AsiaPac										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ific(Seoul) • ca10–Canada(Montreal) • eu10–Europe(										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Frankfurt) • eu11 – Europe (Frankfurt) • us10 – USEa										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			st (VA) **Action:** If you restrict system access by *allow-listing IPs* in firewall rules, **make sure you update your configuration** as soon as possible. **The additional**										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			IP addresses will be used after March 31, 2024. For more information, see Cloud Connector: Prerequisites. In case of questions please contact SAP Support.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.16.2 - Bug Fixes	Release of Cloud Connector version 2.16.2 provides the following bug fixes: • Multiple issues were fixed for c	Info only	General Availability	Changed	Technology	Not applicable			2024-02-08		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onfiguring certificate-based authentication: • Authenticat										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				ionwasnolongerworkingafteranupgradefromto									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				2.16.1, because the configuration was modified wrongly									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				uringthe upgrade. • Authentication was no longer working in									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				gafterchangingLDAPsettings,becausethecon figurati									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				onwasmodifiedwrongly whenpersistingtheLDAPchange									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				s. • The list of trusted certificates could not be adjusted									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				anymoreafteractivatingcertificate-basedauthentic									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				ation.									
			• Compatibility mode in bgRFC for sending units to old S										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			APNetWeaver7.01systemscontainsdummyRFC_PINGinvo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			cationstoreflectsomeconstraintsinsuchascenario.The										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			CloudConnector, however, required RFC_PING to be part of t										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			h e a c c e s s c o n t r o l i s t a n d h e n c e d i d n o t a u t o m a t i c a l l y a l l o w										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			it, like it should be for such infrastructure-related, rem										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			o t e - e n a b l e d f u n c t i o n s . T h i s i s s u e c o u l d o c c u r i n l B P										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			integrationscenarios. • The proxychecksouldwrongly										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			indicate that the HTTP proxy is not reachable. • Avuliner										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ability could occur in the Cloud Connector due to improper										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			checking of certificates. For more information, see SA										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Pseecuritynote3424246100 . These issues have been fixed.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.16.2 - Enhancements	Release of Cloud Connector version 2.16.2 provides the following enhancements: • The Cloud Connector now supp	Info only	General Availability	New	Technology	Not applicable			2024-02-08		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ports clients that use a SAP router string in their configura										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tion in order to connect to an SNC-protected service channel										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nel to an ABAP Cloud system. • The Cloud Connector now prov										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			idesachangeRolescriptthatallowstoswitchthehigha va										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ilabilityrolefrommasterto shadowandviceversa.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - Hotfix	Transparent Proxy version 1.4.3 provides the following hotfix: An issue was fixed that could potentially cause misconfiguration of the Transparent Proxy upon restart, and could result in fail-	Recommended	General Availability	Changed	Technology	Not applicable			2024-02-08		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ing requests. **We strongly recommend that you pull the new version and upgrade as soon as possible.**										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector - Reindexing of Security Recommendations	In the *SAP BTP Security Recommendations*, the index name for the Cloud Connector has been changed from **BTP-CLC-xxxx** to **BTP-SCC-xxxx** to comply with current naming con-	Info only	General Availability	Changed	Technology	Not applicable			2024-01-25		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ventions. No other changes were made to these recommendations. For more information, see SAP BTP Security Recommendations.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - Changes	Transparent Proxy version 1.4.0 provides the following changes: • Enhanced performance on request	Info only	General Availability	Changed	Technology	Not applicable			2024-01-11		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			stprocessingforfaster,moreefficientargetsys										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			temconsumption. • Enhancedrequesttroubleshooti										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ng, providing detailed error information and sour										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ceidentification. • Improved summary of destination c										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			customresourcesforKubernetesadministrators										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			, providing detailed status without the need for order										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			scriptions.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - New Features	A new version of the Transparent Proxy for Kubernetes (1.4.0) has been released. Transparent Proxy version 1.4.0 provides the following new features: • Multitenanc	Info only	General Availability	New	Technology	Not applicable			2024-01-11		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			y s u p p o r t f o r s e a m l e s s c o n s u m p t i o n o f t a r g e t s y s t e m s d e f i										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			needs TCP destinations in the different tenants and exposes										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			dlocallyonthelevelofTCPprotocol.Theactualcommunic										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ationprotocolmaybeanyTCP-basedprotocol,forexample										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			, ODBC/JDBC, SMTP, etc. Formore information, see Mult										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			itenancy. • Simplearbitraryaccesstodestinations.Dynam										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ically access the target systems via a single destination c										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			u s t o m r e s o u r c e , i n s t e a d o f c r e a t i n g s u c h f o r e v e r y l o c a l l										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			yexposeddestination.Formoreinformation,seeDyna										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			micLookup of Destinations. • Native integration with Ls tio										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			. Easily, via a single configuration, add the target systems										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			orserviceendpointsexposedviaTransparentProxytot										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			helloservicemesh. Formore information, see Confi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			guration Guide. • Native integration with the Connectivity P										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			roxy in multi-tenant trusted mode. You can now easily consu										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			metargetsystemswithoutsendingidentitytokenstothec										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onnectivity Proxy. • Automated update of service keys, inu										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sebyTransparentProxy,uponKubernetessecretschang										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e. • The Transparent Proxy module in Kyma now auto-detect										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sandloadsallDestinationserviceinstancebindingswithi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nitsnamespace,reducingtheneedformanualconfiguratio										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			n. For more information, see Transparent Proxy in the Key										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			a Environment. • The Transparent Proxy module in Kyma now, b										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ydefault,createsaDestinationserviceinstanceandbind										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ingupoonenablingthemodule,signifi cantlysimplifyingth										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			einitialsetup.Formoreinformation,seeTranspare										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nt Proxy in the Kyma Environment.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Connectivity Proxy for Kubernetes 2.11.0 - Enhancements	Release of Connectivity Proxy version 2.11.0 (2.10 intentionally skipped) provides the following enhancements: • You can now use a multi	Info only	General Availability	New	Technology	Not applicable			2023-12-14		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			i-registrationsetupforasingleConnectivityProxyinstallation										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			. This setup lets you use one proxy installation with several a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ISAPBTPregions(orwithdifferentsubaccountswithinthe										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			same region). For more information, see Installing t										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			he Connectivity Proxy in Multi-Region Mode. • The *Digi C*										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ertG5RootCA was added to the trust store of the Connectiv										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ityProxy.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Connectivity Proxy for Kubernetes 2.11.0 - Changes:	Release of Connectivity Proxy version 2.11.0 (2.10 intentionally skipped) provides the following changes: • The "utilitydockerimag	Info only	General Availability	Changed	Technology	Not applicable			2023-12-14		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			es", used by Helm for execution of pre- and post-deploy jo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			bscannowbeconfigured,allowingtheuseofprivaterepositories										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ories. • The configured image sent in the helm chart can now be co										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nfigured with adding gest instead of tagging order to be more espe										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			cificinreferencingthedesiredimage.Tagisstillthedefa										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ult. • The Connectivity Proxy is now running on SAP Machine 17										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			, instead of the previously used SAP Machine 11. • There is										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nowautomaticperiodicchecking intheConnectivityProxyc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			omponentsfornewCAs,usedbytheConnectivityservicetoo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			issueCloudConnectorcertificates.Asaresult,youunoion										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			generated to periodically perform a hel mupgrade to refresh										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			the list of trusted CAs.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Connectivity Proxy for Kubernetes 2.11.0 - Security	Release of Connectivity Proxy version 2.11.0 (2.10 intentionally skipped) provides the following security updates: • Multiopen-sou	Recommended	General Availability	Changed	Technology	Not applicable			2023-12-14		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rcecomponentshavebeenupgraded. • Thisreleasecontai										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nsseverallversionbumpsofdependencylibraries,includ										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ingvarioussecurityfixes.**Werecommendthatyouupgradet**										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			he Connectivity Proxy to the latest version.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Connectivity Proxy for Kubernetes 2.11.0 - Bug Fixes	Release of Connectivity Proxy version 2.11.0 (2.10 intentionally skipped) provides the following bug fix: The pod limit configurations of one of the operators of the Connectivity	Info only	General Availability	Changed	Technology	Not applicable			2023-12-14		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Proxy were adjusted in order to prevent potential startup issues.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Cloud Connector - IP Addresses	Additional IP addresses were added to SAP BTP regions running on Amazon Web Services (AWS). The following regions are affected by this change: • br10 – B	Required	General Availability	New	Technology	Not applicable			2023-11-16		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			razil (São Paulo) • jp10 – Japan (Tokyo) • ap10 – Austr										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			alia(Sydney) • ap11–AsiaPacific(Singapore) • ap12										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			– Asia Pacific (Seoul) • ca10 – Canada (Montreal) • eu10										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			– Europe (Frankfurt) • eu11 – Europe (Frankfurt)										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			• us10–USEast (VA) **Action:** If you restrict system access by *allow-listing IPs* in firewall rules, **make sure you update your configura-**										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tion as soon as possible. The new IP addresses *may be used as of late Q1, 2024*. For more information, see Cloud Connector: Prerequisites. In case of questions please contact										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			SAP Support .										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.16.1 - Bug Fixes	Release of Cloud Connector version 2.16.1 provides the following bug fixes: • LDAP rolenames were rewrongly d	Info only	General Availability	Changed	Technology	Not applicable			2023-11-16		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			isplayedinthead										
			ministrationUI.A										
			ctually,thepos										
			itionnow										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			as shifted by one.										

> **Note**
>
> The runtime was not affect

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ed. This was a regression in version 2.16.0. The issue										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			has been fixed. • Passwords with leading or trailing space										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sidnotworkneitherintheloginscreennorintheLDAPte										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			stUI.Whencreatingabackupintheadministrationui withap										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			assword containing special characters, it could not be										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			stored. This was a regression introduced with version 2										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			.15.1.Theissuehasbeenfixed. Note Creatin										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				gbackupswiththeRESTAPIwasnotaffected.									
			• Selec										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ting as subaccount in the administration UI might have failed										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			d. This happened if there are more than 100 subaccounts sho										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			stedonthe CloudConnectorinstanceandwhenfiltering the										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			subaccountlist.Inthiscase,acompletelydifferentonew										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			a s s e l e c t e d w h e n c h o o s i n g t h e s u b a c c o u n t. T h i s w a s a r e g r										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			esioninversion2.16.0.Theissuehasbeenfixed. • Du										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ringanupgradeviaRPMitcouldhappenthatthetheUIaftertheeee										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			initial login screen was not loading. In the trace of the										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			CloudConnector,youcouldseemanymessagestellingthat										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			a JAR file could not be found that was existing only before

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			he upgrade. This could be resolved by a restart of the daemon.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			monoonly. The issue was caused by another issue in upgrad										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			eprocessing, which has been implemented to avoid the error on										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			eousstiuatiion. • Avoida *NullPointerExceptio*nwhenreceivi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ngacl/osserequestinRFCprotocolprocessing.Thiscoul										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			dleadtowrongvaluesforthecconnectionmonitorforRFCco										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nnections. The call stack of the exception started with *java*										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			.lang.NullPointerException: while trying to invoke the me										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			thodcom.sap.core.connectivity.spi.processing.Target										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tHost.connectionProtocol() of a null object loaded from lo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			callvariable 't arget' at com.sap.scc.metering.backend.B										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			backendCallStatisticsCollector.updateNonRequestBased(B										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ackendCallStatisticsCollector.java:109) at com.sap.scc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			.metering.backend.BackendCallStatisticsCollector.repo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rtCloseConnection(BackendCallStatisticsCollector.java:										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			96) • Corrupted subaccounts could not be loaded properly.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			As a consequence, it was impossible to delete them from th										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e a d m i n i s t r a t i o n U I . I n s o m e c a s e s , t h e y e v e n d i d n o t a p p e a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rinthelistofsubaccounts,butwereexistingt										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			oaddsuchasubaccountwouldfail.Thisissuehasbeenfi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			xed. • Self-signed system certificates containing SANs ne										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			edtohaveallCAattributes,whichisnotdesiredforthesys										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tem certificate, and hence it does not have them. As a consequ										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			uence, such a certificate cannot be added to the trust store										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e o f a s e r v e r, w h i c h i s n e e d e d t o e s t a b l i s h t r u s t. B y n o t a l l o w										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ingtoaddSANelementstoself-signedsystemcertificates										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			, this problem cannot occur. • A dedicated service vul										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nerability in the Cloud Connector was fixed. For more info										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rmation, see security note https://me.sap.com/n										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			otes/3362463										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.16.1 - Enhancements	Release of Cloud Connector version 2.16.1 provides the following enhancement: The subaccount dashboard now allows to sort the entries based on the display name.	Info only	General Availability	New	Technology	Not applicable			2023-11-16		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - Kyma Environment	The Transparent Proxy is now part of the Kyma environment as a module. You can benefit from the SAP managed product lifecycle and enhanced support for the Transparent Proxy in your Kyma runtime.	Info only	General Availability	Changed	Technology	Not applicable			2023-10-05		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			For more information, see Transparent Proxy in the Kyma Environment.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - Bug Fix	Release of Transparent Proxy for Kubernetes 1.3.1 provides the following bug fix: An issue originating from Istio 1.18.1 (respectively Kyma 2.17) could cause the Transparent Proxy	Info only	General Availability	Changed	Technology	Not applicable			2023-08-24		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			to fail upon startup. This issue has been fixed.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Java Connector 3.1.8.0 - Bug Fixes	Release of Java Connector (JCo) version 3.1.8.0 provides the following bug fixes: • Calls to RFMs (RFC-enable	Info only	General Availability	Changed	Technology	Not applicable			2023-08-24		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			dfunctionmodules)didnotworkasexpectedR										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			FMwas defined with *Interface Contract*" Fast seriali										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			zationrequired", althoughfastserializationwasconf										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			igured for the used JCoDestination via logon pr										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			operty jco.client.serialization_format=										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			columnBased.Inthiscase,asystemdumpwithrun										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			timeerrorRFC_NOTSUPPORTED_SERIALIZATIONnocc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			urredatABAPsystemsideandacorresponding JC										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			oExceptionwithherrorgroupJCO_ERROR_SYSTEM_										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			FAILUREwasthrownatJCoside. • Whenthe JCoappli c										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ationwasrunningincertaintimezones,settingaSt										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			r i n g o r c h a r [ ] o b j e c t v a l u e i n t o a t y p e T I M E f i e l d o										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rparameterresultedinaConversionException										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			, although the specified time value string was valid. F										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			or example, when running in time zone America/Maza										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tlan,theninvokingJCoRecord.setValue([inde										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			x|name],"18:30:00")causedanexceptionsimil										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			artifollowingone:com.sap.conn.jco.Converter										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sionException:(122)JCO_ERROR_CONVERSION:Ca										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nnotconvertthevalue'18:30:00'fromtypejava										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			.lang.String to type TLMEfield <name> in record <re										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			coordName> • When getting a `java.util.Date` object										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t f r o m a **J C o R e c o r d** fi e l d o f t y p e **C H A R** , w h o s e c o n t e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ntrepresentsadateortimevalueinISOformat(YY										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			YY-MM-DDformatesorHH:MM:SSformatimes), a Conve										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rsionExceptionwasthrown,althoughsuchatype										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			conversion is possible and allowed. Note Th										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				is was a registration bug which had been introduce									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			dwithJCo3.1.7.0. • JCoDestinationinstances										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			with authentication type PrincipalPropagation										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			were unnecessarily used by a JCoRepository for q										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			uerying RFC metadata, which might have ended up i										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			n a J C o E x c e p t i o n w i t h e r r o r g r o u p J C O _ E R R O R _ C O N										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			FIGURATION, if currently logged-on user identit										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			y w a s a v a i l a b l e .										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - Alert Notifications	An event will be produced in the Alert Notification service when the subaccount trust certificate of the Destination service (used for SAML-based scenarios) is close to expiring, allowing administrators to react ac-	Info only	General Availability	New	Technology	Not applicable			2023-08-10		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			cordingly ahead of time. For more information, see Notifications and Alerts.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.16.0 - Removed Feature	Release of Cloud Connector version 2.16.0 no longer provides the following feature: Due to the discontinuation of the *Enhanced Disaster Recovery Service*, the related	Info only	General Availability	Changed	Technology	Not applicable			2023-07-27		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			functionality in the Cloud Connector has been dropped. For more information, see the corresponding Enhanced Disaster Recovery Service release note.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.16.0 - Bug Fix	Release of Cloud Connector version 2.16.0 provides the following bug fix: The usage monitor data is no longer lost when doing a master-shadow switch.	Info only	General Availability	Changed	Technology	Not applicable			2023-07-27		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.16.0 - Enhancements	Release of Cloud Connector version 2.16.0 provides the following enhancements: • The Cloud Connector now supp	Info only	General Availability	New	Technology	Not applicable			2023-07-27		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			portsOracleLinux9asadditionalOSversion.Formorei										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nformation, see Product Availability Matrix. • macOS on										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			aarch64 is added as a supported platform for the Cloud Co										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nnector.Formoreinformation,seeProductAvailabil										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ity Matrix. • Service channels for RFC that are exposing cl										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			asicRFCendpointstoABAPcloudsystems,suchasS/4HA										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			NA Cloud, IBP, or SAP BTP ABAP environment, can now be prote										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			cted with SNC. For more information, see Configuration as										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ervice Channel for RFC. • The Cloud Connector now allow										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			s the logon to the UI and the REST APIs with a client certificat										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e. For more information, see Logon to the Cloud Con										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nectoriaClientCertificate. • The connectionmonitoran										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			d t h e u s a g e m o n i t o r n o w a l s o s u p p o r t a c c e s s c o n t r o l e n t r i e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			s f o r t h e T C P p r o t o c o l . F o r m o r e i n f o r m a t i o n , s e e B a c k										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			endConnectionsandUsageStatistics. • The hardwaremo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nitorinformationofthelast24hoursisnowsurvivingare										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			start. For more information, see Hardware Metrics.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			• You can now export the selected audit log entries to a csv fil										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e f o r e x t e r n a l a n a l y s i s . F o r m o r e i n f o r m a t i o n , s e e M a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nageAuditLogs.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - New Version	A new version of the Transparent Proxy for Kubernetes (1.3.0) has been released. Changed features: • TCP connectivity is now allowed	Info only	General Availability	Changed	Technology	Not applicable			2023-07-13		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			by default in Istio mesh. For more information, see Transparent Proxy for Kubernetes.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - New Version	A new version of the Transparent Proxy for Kubernetes (1.3.0) has been released. New features: • Multitenancy support for seaml	Info only	General Availability	New	Technology	Not applicable			2023-07-13		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			esconsumptionofHTTPtargetsystemsdefinedasdestinati										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onsindifferententenants.Formoreinformation,seeMu										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			lt itenancy. • You can now work with multiple Destinations service i										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ceinstances,evenifLocatedonadifferentBTPregion.Fo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			r m o r e i n f o r m a t i o n , s e e L i f e c y c l e M a n a g e m e n t. • T r a n s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			parenthandlingofclientassertionauthenticationflowfort										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			herespectivedestinationauthenticationtypes.Seealsoth										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e c o r r e s p o n d i n g Destinationservicereleasenote. Form										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			or information, see HTTP Authentication. • Lifecycle										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			management via Landscaper is now supported. Formor										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			information, see Lifecycle Management. • You can now										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			removeobsoleteHelmconfigurations.Formoreinforma										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tion, see Lifecycle Management.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - Changes	• You can now use the API for retrieving only the public part o	Info only	General Availability	Changed	Technology	Not applicable			2023-07-13		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			fastoredcertificatealsoforpasswordprotectedkeyst										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ores, as long as the password is known to the Destination service										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rvice(thatis,generatedviathesservice). • Usingthequ										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ryparameterofthe"Find"APIforskippingtokenretriev										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			alonOAuthdestinationswithmTLSnowincludesthecertifica										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t e f o r t h e t o k e n s e r v i c e a s p a r t o f t h e A P I r e s p o n s e b o d y .										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - Service Instance Transformation	Service instance transformation for the Document Management service and SAP Build Process Automation is now supported.	Info only	General Availability	New	Technology	Not applicable			2023-07-13		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - Alert Notifications	The Destination service is now integrated with the Alert Notification service (ANS). Currently, you can get notifications for expiring certificate entities which are not enabled for auto-	Info only	General Availability	New	Technology	Not applicable			2023-05-18		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			matic re-newal. For more infor-mat-ion, see Noti-fica-tions and Alerts.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Connectivity Proxy for Kubernetes 2.9 - Bug Fixes	Release of Connectivity Proxy version 2.9 provides the following bug fixes: • The Helm chart templates were re	Info only	General Availability	Changed	Technology	Not applicable			2023-05-03		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			not compatible with the latest listed versions. • MultipleOSv										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			unerabilities were caused by removing or upgrading pack										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ages on the image. • Multiple Java vulnerabilities were caus										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			edbyupgradingdependenciestoanewerversion. Theseissueshave										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			been fixed.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Connectivity Proxy for Kubernetes 2.9 - Enhancements	Release of Connectivity Proxy version 2.9 provides the following enhancements: • You can now disable the "ser	Info only	General Availability	Changed	Technology	Not applicable			2023-05-03		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			vicechannels" feature to install multiple connectivity pr										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			oxiesonthesameclluster. • HostOSversions,containingc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			groupv2, are now available. • You can now install the Transp										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			arentProxytogetherwiththeConnectivityProxyviaHelmc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			hart. • You can now use embedded corporate JWTs (JSON web to										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			kens) for principal propagation. For more information, see Connectivity Proxy for Kubernetes.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector - OS Versions	The Cloud Connector supports Oracle Linux 8 as additional OS version. For more information, see Product Availability Matrix.	Info only	General Availability	New	Technology	Not applicable			2023-04-28		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.15.2 - Bug Fixes	Release of Cloud Connector version 2.15.2 provides the following bug fixes: • The connection check for LDAPS	Info only	General Availability	Changed	Technology	Not applicable			2023-04-20		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			access control entries was always failing. This issue has										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			beenfixed. Note Theactualruntimewasnotaffe										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			cted. • Subject patterns could not be modified. Only delete										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tingandaddingwaspossible.Thiswasaregressionin										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			2.15.1 only. • High availability master/shadow configura										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tionsynchronizationcouldbepermanentlydisruptedifana										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			d m i n i s t r a t o r t r i g g e r e d a r o l e s w i t c h . O n s h a d o w s i d e , t										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			hetracewasshowingreoccurringNullPointerExce										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			p										
			ti										
			o										
			n										
			s										
			a										
			t										
			m										
			e										
			t										
			h										
			o										
			d										
			c										
			o										
			m										
			.										
			s										
			a										
			p										
			.										
			s										
			c										
			c										
			.i										
			n										
			f										
			o										
			.										
			S										
			u										
			p										
			p										
			o										
			r										
			tI										
			n										
			f										
			o										
			.										
			g										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			etScc Version(). This issue has been fixed. • Disabling										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			trustedOAuthserversisnolongerignoredwhenverifyin										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			gJSONwebtokens(JWTs)forprincipalpropagation.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.15.2 - Enhancement	Release of Cloud Connector version 2.15.2 provides the following enhancement: The lists for *Most Recent Requests* and *Top Time Consumers* in the monitor now contain a *User*	Info only	General Availability	New	Technology	Not applicable			2023-04-20		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			column.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - Client Assertions	You can now use client assertions that are automatically fetched by the Destination service, as an OAuth client authentication method. This extends the existing method for caller-provided assertions via HTTP	Info only	General Availability	New	Technology	Not applicable			2023-05-04		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			header when calling the "Find Destination" API. For more information, see Using Client Assertion with OAuth Flows.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - Changes	Transparent Proxy version 1.2.0 includes the following changes: • Improved handling for the stream in	Info only	General Availability	Changed	Technology	Not applicable			2023-03-23		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			gofl argepayloads,ensuringefficientdatatransferbetw										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			enclient and server. • Applied server all performance improv										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ements. • Fixed OS Vulnerabilities by removing or upgradi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ngrelatedpackagesonthehimage.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - Enhancements	Transparent Proxy version 1.2.0 introduces the following features: • Support and transparent handling o	Info only	General Availability	New	Technology	Not applicable			2023-03-23		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			f t h e *O A u t h 2 T e c h n i c a l U s e r P r o p a g a t i o n* a u t h e n t i c a t i o n t y										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			pe. • Added an option for declarative control over the access										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			s (scope of visibility) to the exposed destinations in the Ku										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			bernetescluster. • If default scope is set to “clusterWid										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				e " - e v e r y w o r k l o a d f r o m e v e r y n a m e s p a c e c a n a c c e s s t h e e								

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				xposeddestination. • If default scope is set to “namespa									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				ced" – the Transparent Proxy limits the access (scope of									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				visibility) to destination CR solely to workload shoste									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				d i n t h e s a m e n a m e s p a c e w h e r e t h e d e s t i n a t i o n C R s a r e m a n									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			aged. • Enables administrators to configure interlanhan										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			dling of secure communication between Transparent Proxy										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			'sinternalmicro-componentsbyencryptingtheirconnec										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tionsusingmutualTLS(mTLS). • Enableshorizontalautosca										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ling of the Transparent Proxy's components based on their

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			load. • Enables vertical autoscaling of the Transparent Pr										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			oxy's components based on their load. • Added support for form										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			anualdeploymentinSAPKymaruntime,enablingtheTransp										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			arentProxy to be configured and run seamlessly. For more information,										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			see Transparent Proxy for Kubernetes.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Connectivity Service - Authentication	As new option for principal propagation, you can now use an embedded corporate IdP token for scenarios where the user token sent to the Connectivity service has an embedded JWT (JSON Web to-	Info only	General Availability	New	Technology	Not applicable			2023-03-09		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ken) from the original corporate IdP. This token will be sent to the Cloud Connector and will be used to issue the short-lived certificate. For more information, see Configure Principal Propagation										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			via Corporate IdP Embedded Token.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - Certificates	You can now use a *CSR* (certificate signing request) to generate certificates that are signed by the *SAP Cloud Root CA*, in addition to the already supported method of providing just parameters for the generation.	Info only	General Availability	New	Technology	Not applicable			2023-02-09		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			This new option lets you use your own local private key.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Java Connector 3.1.7.0 - Bug Fixes	Release of Java Connector (JCo) version 3.1.7.0 provides the following bug fixes: • basXMLdeserialization: When enc	Info only	General Availability	Changed	Technology	Not applicable			2023-01-26		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			allinganRFC-enabledfunctionmodulea ndusingba										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sXMLserialization, an exception at deserializing the										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			RFC responsed at a mi g h t h a v e o c c u r r e d if s k i p p i n g										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			someunneededbasXMLcontentwasrequired.Theex										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ception was similar to: *BasXMLParser.parsethreewe*										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			exceptionwhileparsingparameter<parameterName>whe										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			n p r o c e s s i n g f i e l d <f i e l d N a m e > c o m . s a p . c o n n .j c o .										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			XMLParserException:(130)JCO_ERROR_XML_PARS										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ER: Unexpected token while discarding <fieldName										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			>incontentoftheparameter<parameterName>ofthecurrr										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			entBXMLdocument:'<?>'atposition<#####>atcom										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			.sap.conn.jco.rt.BasXMLParser.parse(BasXMLPars										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			er.java:536)atcom.sap.conn.jco.rt.BasXMLParser.setC										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ompressedBytes(BasXMLParser.java:1588)atcom.sap.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			conn.rfc.engine.RfcImp.receiveBasXMLCompres										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sedData(Rfc Imp.java:377)atcom.sap.conn.rfc.engi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ne.RfcGet.ab_rfcget(RfcGet.java:425)at com.sap.co										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nn.rfc.engine.RfcRcv.ab_rfcreceive(RfcRcv.java										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			va:41)atcom.sap.connn.rfc.engine.RfcIoOpenCntl.Rf										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			cReceive(RfcIoOpenCntl.java:2078)... • TypeTIMEfieldval										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			uconversionsfrom/tojava.util.Dateobjects:Whe										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ngetting or setting a java.util.Date object from

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ortoaJCoRecordfieldoftypeTLME,aConversion										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Exception might have been thrown during ongoing										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			h e c l o c k t i m e v a l u e a n d t h e t i m e z o n e , i n w h i c h t h e J C										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			o a p p l i c a t i o n w a s r u n n i n g . F o r e x a m p l e , w h e n r u n n i										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ngintimezoneAmerica/Mazatlan, invoking JCoReco										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rd.getTime(...)couldhavecausedanexceptions										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			similar to the following one: *com.sap.conn.jco.Con*										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			versionException: Unparseable date: "00000000" i										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nrecord<name>atfield<name> • Type **UTCLONG** field val										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			uconversionsfrom/tojava.util.Dateobjects:Whe										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ngettingorsettingajavautil.Dateobjectfrom										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ortoaJCoRecordfieldoftypeUTCLONG, theconve										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			r t e d t i m e s t a m p v a l u e w a s w r o n g b e c a u s e t h e d e f a u lt										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			timezone was used for the conversion instead of the

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			eUTCtimezone.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Java Connector 3.1.7.0 - Enhancements	Release of Java Connector (JCo) version 3.1.7.0 introduces the following enhancements: • JCoRepository: A new API	Info only	General Availability	New	Technology	Not applicable			2023-01-26		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			l m e t h o d J C o R e p o s i t o r y . r e m o v e O u t d a t e d M e t a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			DataFromCache() has been introduced to identify										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			and remove outdated metadata from the local cache										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e.Moreover,anewdestinationpropertyjco.destii										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nation.repository.check_interval has been										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			added to execute this new cache remove all feature rei										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nregulartimeintervalls.Itallowsamoreconvenien										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t u s a g e o f J C o i n d e v e l o p m e n t e n v i r o n m e n t s , w h e r e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ABAP remote function interface access, structures, an										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			d t a b l e s c h a n g e f r e q u e n t l y , s o t h a t i t i s n o t n e c e s s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			arytoclearthewholeJCoRepositorycachemanu										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			allyanymore,ortorestartacentrallyrunningJCo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			webserverapplicationreachtimeafterdynamicallyr										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			etrievedRFCmetadatahasbeenmodifiedintheorigi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nSAPbackendsystem.Formoreinformation,seeth										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e API documentation. • JCoRecord: New API methods sha										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			v e b e e n i n t r o d u c e d t o t h e J C o R e c o r d i n t e r f a c e :										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			J C o R e c o r d . s e t V a l u e([i n d e x | n a m e] , j a v a . u t										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			il.Date) and JCoRecord.setValue([index|na									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			me],java.math.BigInteger).ThesenewAPImeth										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			odsservicetype-safeandmoreperformantreplacement										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			of method JCoRecord.setValue([index|name], java.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			lang.Object) forsettingfieldvaluesfromjava.util.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Date and java.math.BigInteger objects. • JCoRequest										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t: The JCoRequest interface has been enhanced w										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			it had dditional execute(...) methods to support al										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sotRFCandqRFCexecutionswhenusingtherequest										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			/responseprogrammingmodel.Formoreinformationa										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			bout the additional methods, see the JavaDoc of interfac										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			eJCoRequest.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - Certificates	You can now use a REST API to manage *active* and *passive* sub-account certificates, and switch their roles. This lets you perform a non-disruptive rotation of the signing certificate (for ex-	Info only	General Availability	New	Technology	Not applicable			2023-01-26		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ample, when the active certificate is close to the expiry date). For more information, see Rotate Certificates.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.15.1 - Bug Fixes	Release of Cloud Connector version 2.15.1 provides the following bug fixes: • Principal propagation scenario	Info only	General Availability	Changed	Technology	Not applicable			2022-12-15		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			s c o u l d f a i l p r o d u c i n g a t r a c e e n t r y t h a t c o n t a i n s t h e m e s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sage "SSO token validation failed: com.sap.core.connecti										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			vity.tunnel.client.sso.InvalidSSOTokenException: No princi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			palisextracted, as the principal type UNKNOWN is invalid										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			."This issue has been fixed. • In the context of *high ava*										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			il a b il it y , a m a s t e r i n s t a n c e c o u l d p e r m a n e n t l y s h o w a s h a d o										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			winstanceasconnectedsinceJanuary1,1970,eveniftheе										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			shadow was disconnected. This issue has been fixed.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			• It was not possible to edit Kubernetes cluster service cha										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nnelsinthheadministrationUl.Thiswrongbehaviorha										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sbeenfixed. • When performing an upgrade avoiding downti										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			me, the subject patter nfor principal propagation was res										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			et to the default value. This issue has been fixed. • Java										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			17 supportwasincompletesothatcertainoperationsweer										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			efailing. For example, when synchronizing trusted identen										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tity providers, an HTTP 500 error was shown in the admin is										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tration URL. This issue has been fixed. • The *registration* a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rbagecollectionhistoryfileofSAPJVM8wasnolongercrea										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tedwhenusingtheWindowsportableversion.Thisissu										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e h a s b e e n fi x e d .										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Connectivity Service - Bug Fix	When making **HTTP HEAD** requests in certain error cases (like a path not being allowed in the Cloud Connector), the response contained a body. This issue has been fixed.	Info only	General Availability	Changed	Technology	Not applicable			2022-12-15		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes - New Version	Transparent Proxy for Kubernetes version 1.1.1 is now available. **New** • Added support and transparent h	Info only	General Availability	New	Technology	Not applicable			2022-11-17		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			and handling of: • *OAuthRefreshToken* and *OAuth Authorization Code* authentication types • Customheadersandq										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				uerypararameterscon figuredinadestinationHTTP									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				timeout configurations configured in a destination									
			• Support for the mTLS-based s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ervice instance key for technical access to the Destinati										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onservice • Improved error handling **Security** • Fixed OSS vulner										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			abilitiesbyremovingorupgradingrelatedpackagesonth										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e i m a g e For more information, see Transparent Proxy for Kubernetes.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - New Features	• You can now configure timeouts towards the token service f	Info only	General Availability	New	Technology	Not applicable			2022-11-09		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			or automatic token retrieval (via destination properties).										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			For more information, see HTTP Destinations. • A new au										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			thentication type for *technical user propagation* is now av										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ailable(onlyforon-premiseconnectionsviatheConnecti										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			vity service). For more information, see Authenticati										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onto the On-Premise System. • A new API endpoint for re										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			trievingonlythepublicpartofacertificateconfigurationi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			snowavailable.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Neo	Cloud Connector - New IP Addresses for the Neo Environment	⚠ Caution The planned IP changes of the Connec	Required	General Availability	Announcement	Technology	Not applicable			2022-10-20		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tivity service (Neo environment) for the Cloud Connec										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			to r in Q 4 2 0 2 2 (s e e r el a t e d n o t e fr o m S e p t e m b e r 1 9, 2 0										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			22) particularly affect Cloud Connector versions prior										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			to 2.12.2. **Required Action:** *If you use IP-based firewall*										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			wall rules and one of these Cloud Connector versions,										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			makes sure you include **both** the old and new IP address										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			esforreachconnectedregiontha tshowsan**OLD**and										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			NEW IP address (Cloud Connector documentation) *a*										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			s s o o n a s p o s si bl e, t o a v oi d o u t a g e s. O t h e r w is e, a r e s t a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rt of the Cloud Connector will be required after the IPc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			hanges are effective. For more information, see Prer										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			equisites.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Connectivity Proxy for Kubernetes - New Version	Version (2.8.0) is now available for the Connectivity Proxy for Kubernetes. **New** • You can now configure the allowe	Info only	General Availability	Announcement	Technology	Not applicable			2022-10-20		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			dcyphersuitesfortheIngressendpointoftheConnectivit										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			y Proxy via the Helm chart. • Added support for granular access										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			esscontrolforservicemappingresources(fortheon-p										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			remiset oclou dscena rio), based onthe Clou dCo nnec tor lo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			cationID. **Bug Fixes** • Fixed an issue where custom values for the p										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			roxyportswerenotfullytakenintoaccount. • Fixedanissu										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e w h e r e u p d a t i n g a s e r v i c e m a p p i n g r e s o u r c e w o u l d r e s u l t i										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			norphaned entries in the Connectivity Proxy. • Fixed an issu										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ewhereupdatingsomeofthecconfigurationsofthe Connectiv										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ity Proxy could result in the inability to create new service										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			mapping resources. • Fixed an issue where the uninstallati										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			one of the Connectivity Proxy while still having ingress service mapping										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ngresourceswouldonlyresultinapartialuninstallation.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Now the uninstall would fail without removing anything ins										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			uchcases. • Fixedanissuewhereservicechannelsoftype										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ABAP Cloud System could not be established via Cloud Conn										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ector2.15 towards service mapping soft type *RFC*. **Security Fixes** • Fi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			xedmultipleOSvulnerabilitiesbyremovingorupgradingp										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ackages on the image. • Fixed multiple Java vulnerabilities										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			by upgrading dependencies to a newer version. For more information, see Con-										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nectivity Proxy for Kubernetes.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.15.0 - Bug Fixes	Release of Cloud Connector version 2.15.0 provides the following bug fixes: • Logini nfocont entisn owincl	Info only	General Availability	Changed	Technology	Not applicable			2022-10-06		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			udedinthebackup. • SolutionManagerintegrationviathehttpdestinationtypeisnotsupported.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ostagentfailed,whenusingnon-ASCIIcharactersindesc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			riptionsofaccesscontrolentries(forexample,an*uml aut*										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			like "ä"), as the LMDBxml was generated using the wrong enc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			oding. This issue has been fixed. • Principal propagatio										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nforRFCisnowworkingevenwhennotprovidingadummySN										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Cpartnernameforloadbalancingconnections.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.15.0 - Enhancements	Release of Cloud Connector version 2.15.0 provides the following enhancements: • The Cloud Connector can now	Info only	General Availability	New	Technology	Not applicable			2022-10-06		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			use SAP Machine 17 as Java runtime. For more information										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			, see JDKs. • The Cloud Connector supports Windows 11 and										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			RedHatEnterpriseLinux9additionalOSversions.Form										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			or information, see Product Availability Matrix. • Th										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e C l o u d C o n n e c t o r p r o v i d e s a n e w t y p e o f *s* *e* *r* *v* *i* *c* *e* *c* *h* *a* *n* *n* *e* *l*										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			for service endpoints within Kubernetes clusters that i										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			n c l u d e a d e p l o y m e n t o f t h e C o n n e c t i v i t y P r o x y f o r K u b e r n e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tes. For more information, see Configure a Service C										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			hannelforaKubernetesCluster. • You can now forward										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			not only business user identities, but also technical use										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			r s i d e n t i f i e d b y O A u t h c l i e n t c r e d e n t i a l t o k e n s . O n t h e C l o u										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			d Connectors ide, you can do th is via enhanced confi gurati										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			on of the subject patterns for principal propagation. For										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			more information, see Configure a Subject Pattern										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			for Principal Propagation. • Certificate signing requests										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tsforUlcertificatesletyouchoosebetweenn2diffferentke										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ysizes:2048or4096.Formoreinformation,seeExch										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			angeUICertificatesinthesAdministrationUI. • Enhanc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			edtruststorerconfigurationcapabilitiesallowtoswitchthe										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			truststoremodifneeded. Thedefaultpropertyofanempt										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			y t r u s t s t o r e h a s b e e n c h a n g e d t o *d o n o t t r u s t a n y c e r t i f i c a*										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			te. For more information, see Configure Trust. • Supp o										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rtforSLSprotocolversion3.0whenusingaremoteCAfor										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			principal propagation. For more information, see Conf										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			igureaCACertificateforPrincipalPropagation. • Im										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			provided configuration for the principal type for HTTP accesses										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			scontrolentriestoprovidemoreclarityaboutoverallimp										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			actoncredentialprocessing.Formoreinformation,s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ee Initial Configuration (HTTP). • Additional admin istr										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ationRESTAPIsforciphersandtruststoreareenowavailab										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			le. For more information, see Authentication and UI										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Settings and Trust store CA Certificates. • An additional										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			almonitoringAPIforopenconnectionsviaservicechannel										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sisnowavailable. Formoreinformation, seeMonitorin										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			gAPIs. • An additional monitoring API is available, that atche										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			cksifthe addressedinstancehasthemasterrole.ThisAP										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			l d o e s n o t r e q u ir e a u t h e n ti c a ti o n . F o r m o r e i n f o r m a ti o n ,										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			see Monitoring APIs. • Enhanced ciphers configuration with b										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			etterfilteringcapabilitiesisnowavailable.Formoreinfo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rmation, see Recommendations for Secure Setup.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - Authentication Types	For the Destination service, the new authentication type *OAuth Authorization Code* is now available. For more information, see OAuth Authorization Code Authentication.	Info only	General Availability	New	Technology	Not applicable			2022-10-06		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Neo	Cloud Connector - New IP Addresses for the Neo Environment	⚠ Caution Due to a planned network update, the	Required	General Availability	Announcement	Technology	Not applicable			2022-09-19		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Paddressesofthe Connectivityservicefortheclou										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			dConnector will change for many regions (Neo environ										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nment) by **end of September 2022. Required Action:**										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			If you use IP-based firewall rules, make sure you include										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e **both** the old and new IP address es for reach connec

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tedregiontha tshowsan **OLD** and **NEW** IPaddress (Clo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			udConnectordocumentation). Formoreinformation										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			, see Prerequisites.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Kyma	Transparent Proxy for Kubernetes	The Transparent Proxy for Kubernetes is now available. The Transparent Proxy lightens the way your Kubernetes workloads connect to Internet and on-premise systems, modeled	Info only	General Availability	New	Technology	Not applicable			2022-09-27		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			as destination configurations. It automates technical client authentication, Connectivity Proxy handshake, facilitates and significantly simplifies principal propagation, as well as general										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			access to the target systems. For more information, see Transparent Proxy for Kubernetes.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Java Connector - X.509 Secrets	JCo supports the usage of X.509 secrets for communication with the Destination and Connectivity services. For more information, see Binding Parameters of SAP Authoriza-	Info only	General Availability	New	Technology	Not applicable			2022-09-08		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tion and Trust Management Service.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Kubernetes Connectivity Proxy - New Version	A new version (2.7.0) has been released for the Kubernetes Connectivity Proxy. **New** • Out-of-the-box support in	Info only	General Availability	Announcement	Technology	Not applicable			2022-07-28		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			the Helm chart for listing ingress gateway as a sanalter										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			native to the Nginx ingress controller. • Support for										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			TCP and JDBC service mapping types - requires a not										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			yetreleasedversionoftheCloudConnector(2.1										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			5). • The Connectivity Proxy can now be used with official										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			support on non-Gardener clusters. **Changed** • The timeoutp										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			eriodforwhichtheConnectivityProxywaitsforaclienttotore										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			add a t a f r o m i t s p r o x y e n d p o i n t s i s n o w c o n f i g u r a b l e .										
			Security										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			• Fixed multiple OS vulnerabilities by removing or up										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			grading packages on the image • Fixed multiple Java										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			vulnerabilities by upgrading dependencies to a ne										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			server version.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Java Connector 3.1.6.0 - Bug Fixes	Release of Java Connector (JCo) version 3.1.6.0 provides the following bug fixes: • When setting numeric value	Info only	General Availability	Changed	Technology	Not applicable			2022-07-28		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sintofieldsoftypeNUMwithmethodsJCoRecord.s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			etValue([name|index],char) or JCoRecord.s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			etValue([name|index],char[]),theresulting c										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ontentwasswrongng.Thefieldwasnotacorrectlyform										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			attednumericfieldafterwardsasthepassedvalue										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			was left aligned instead of right aligned and hence,										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			also not leading zero digits were added.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				te Method JCoRecord.setValue([name|index									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			`],String)` was not affected by this bug. • A `JCoRepositorymetadataquerymi`										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ghthavefailedwhenloopingovermultipledestinati										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onsfortrying to obtain the requested RFC metada										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ta, if a destination configuration got corrupted. Now										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			, J C o c o r r e c t l y i g n o r e s s u c h a f a i l u r e a n d u s e s t h e n										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			extworkingdestinationforthequery. • If a function m										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			odulecontainingaparameterwithanamestartingw										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ithXMLwasexecuted,ashortdumpoccurredinABAP										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			indicated by a JCoException with a text similar to

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			(104)JCO_ERROR_SYSTEM_FAILURE: Internal error										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			or: Unexpected status in the ABAP runtime environment										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nt.(Remoteshortdump:RUNT_INTERNAL_ERRORins										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			system[ABC|abcashost.corp|42]). These same is true										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			if there is a structure or table parameter contain										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ingafieldwithsuchanname.Thisshappeneddduetoami										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			s s i n g s p e c i a l e s c a p e s e q u e n c e t h a t s h o u l d b e u s e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			dasthestandardrecommendstoreservethisprefi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			xforlaterstandardextensions. • When using jco										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			.client.serializationformat=columnBas										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			edtogetherwithjco.client.networkLANmode										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			, it could happen that the response was not compres										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sed by the backend, although the data size was larg										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e e n o u g h . T h i s h a p p e n e d d u e t o a n i n v a l i d i n i t i a l i z a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tion of the RFC communication.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Java Connector 3.1.6.0 - Enhancements	Release of Java Connector (JCo) version 3.1.6.0 introduces the following enhancements: • `JCo.queryMetaDat`	Info only	General Availability	New	Technology	Not applicable			2022-07-28		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			aSet() was enhanced to only request the metadata										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			from the backend for orthose function modules and data										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tatypesthatarenotyetcachedinthe JCoRepositi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tory, for which the query is performed. • JCo was e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nhancedtointerpretallsostringsconsistingonly										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			of blanks as an initial value for fields of type NUM.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service	• You can now skip the user_attributes.prefix when adding in	Info only	General Availability	New	Technology	Not applicable			2022-07-14		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			gadditionalIdPAttributestothegeneratedSAMLassertio										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nXMLs.Formoreinformation,seeUserPropagationv										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			iaSAML2.0BearerAssertionFlow. • Added support for										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			automatic token retrieval for the OAuth2 refresh token au										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			thentication type. For more information, see OAuthRe										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			refreshTokenAuthentication.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service	Additional user attributes are now fetched even if the user token is sent to the Destination service via the `X-User-Token` header in the OAuth SAML Bearer and SAML Assertion flows.	Info only	General Availability	Changed	Technology	Not applicable			2022-07-14		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			For more information, see User Propagation via SAML 2.0 Bearer Assertion Flow.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service	Destination Java APIs now use the login context to determine from which tenant to retrieve a destination. For more information, see Destination Java APIs.	Info only	General Availability	Changed	Technology	Not applicable			2022-06-16		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service	You can update certificates with new content via the REST API. Up to now, the only way to update a certificate was to delete it first and then create it again with the same name and the new	Info only	General Availability	New	Technology	Not applicable			2022-06-02		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			content.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Feature Scope Description	The feature scope description for Connectivity has been integrated into the central feature scope description for SAP BTP. For more information, see Connectivity product page.	Info only	General Availability	Changed	Technology	Not applicable			2022-06-02		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.14.2 - Bug Fixes	Release of Cloud Connector version 2.14.2 provides the following bug fixes: • Server certificate without	Info only	General Availability	Changed	Technology	Not applicable			2022-06-02		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			a subject area now accepted as valid in TLS backend dc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ommunication, if a fitting *Subject Alternate Name* entent										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ryisavailable. • TheCloudConnectorviewinthecl										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			oudcockpitisupdatedwithshadowinstanceinform										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ationwheninitentionallyswitchingrolesfromtheCl										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			oud Connector UI's high availability screen. • Starti n										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			gafreshlyextractedportableinstallationallsow										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			orksproperlyonWindows,ifthepathtothepor tabl										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e i n s t a ll a ti o n c o n t a i n s a s p a c e c h a r a c t e r.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Kubernetes Connectivity Proxy	Version 2.6.1 has been released for the Kubernetes Connectivity Proxy. **New** You can now use the on-premise-to-cloud connectivity scenario, a.k.a. service channels. It enables RFC-	Info only	General Availability	Announcement	Technology	Not applicable			2022-05-19		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			based connections from the customer premise through the Cloud Connector, towards a private cloud endpoint (not exposed to the Internet) that serves RFC. **Fixed** • An iss										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			u e w i t h t h e a l l o w R e m o t e C o n n e c t i o n s c o n f i g u r a t i										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onproperty was fixed. • The Upgrade HTTP header va										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Issue was not handled correctly when it was "WebSocke										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			t " i n s t e a d o f " w e b s o c k e t " . T h i s i s s u e h a s b e e n fi x										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ed. **Security** • Fixed multiple OS vulnerabilities by removi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ngorupgradingpackagesonthe image. • Fixedmulti										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			pleJavavulnerabilitiesbyupgradingdependenci										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			est oa new ver sion .										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - Certificates	• You can create destinations using the MTA descripto	Info only	General Availability	New	Technology	Not applicable			2022-05-19		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			r w h e n t h e s e r v i c e k e y h a s a n X . 5 0 9 b i n d i n g . F o r m o										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			reinformation, see: Creating a Destination Using the										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			MTA Descriptor. • You can update certificates in case of

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			f c o nf li c t (a lr e a d y e x i s ti n g c e r tif i c a t e w it h t h e s a m e n										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ame) whencreatingentitiesviaaconfig.jsondurin										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			g i n s t a n c e c r e a t i o n / u p d a t e . F o r m o r e i n f o r m a t i o n										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			, see: Use a Config.JSON to Create or Update a Destin										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ationService Instance.										
Connectivity	Cloud Foundry	Destination Service - IdP Metadata	Destination service-generated IdP metadata now include `entityID` by default.	Info only	General Availability	Changed	Technology	Not applicable			2022-05-19		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Connectivity Proxy for Kubernetes (Cloud Foundry Environment) - Version 2.5.0	Connectivity Proxy version 2.5.0 includes the following features: • automatic detection of changes	Info only	General Availability	Announcement	Technology	Not applicable			2022-03-24		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			inconfigurationsandsecretwithoutthenneedforma										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nualrestarts. • scalablethreadpoolsizesforth										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e p r o x y a n d b u s i n e s s d a t a t u n n e l s e r v e r s , b o t h a u										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tomatic and via explicit values. • change d the min Av										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			availableconfigurationforthePodDistributionBu										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			dgetischangedfrom90%to50%. • multipleopensou										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			r c e s o f t w a r e c o m p o n e n t s r e p o r t e d a s v u l n e r a b l e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			has been replaced.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - Automatic Certificate Renewal	You can now choose automatic renewal for certificates, generated via the Destination service (requires opt-in during generation). The service then gets a new certificate with the same	Info only	General Availability	New	Technology	Not applicable			2022-03-10		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			subject as soon as the current one is about to expire. Once renewed, any subsequent attempt to fetch the certificate will result in returning the new certificate. For more information, see Use										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Destination Certificates.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.14.1 - Bug Fixes	Release of Cloud Connector version 2.14.1 provides the following bug fixes: • If a Cloud Connector needed	Info only	General Availability	Changed	Technology	Not applicable			2022-03-10		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			more than one minute to load its configuration, if fail										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			edtostartthe administrationUIandraisedexcep										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tion java.util.concurrent.ExecutionExcep										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tion:org.apache.catalina.LifecycleExce										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ption:Failedtostartcomponent[StandardE										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ngine[Catalina].StandardHost[localhost].										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Standard Context[]] which is eventually used										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			by an exception com.sap.scc.servlets.Critic										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			alScException:Couldnotgetinitializedc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onfiguration-lockedbyanotherthreadatcom										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			.sap.scc.config.SccConfig.getInstance(										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			`cc Config.java:240) This issue has been fixed.`										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			• When upgrading high-availability (HA) installation										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			s, initialization could fail after request to one of t										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			heinstancesswithaNullPointerException,res										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ultinginafailedstartofthheadministrationUI,bec										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			auseoldercloudconnectormasterserverwrotteth										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			eshadowsettingwithheemptyvalue.Theexception										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			c a ll s t a c k w o u l d c o n t a i n a j a v a . l a n g . N u l l P o i n										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			terException: whiletryingtoinvoketheemethod										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			java.io.File.equals(java.lang.Object)of										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			a n u l l o b j e c t r e t u r n e d f r o m j a v a . i o . F i l e . g e t										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ParentFile()atcom.sap.scc.util.SecTools										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			.getValidFile(SecTools.java:1311)at com.s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			a p . s c c . u t i l . S e c T o o l s . g e t V a l i d F i l e(S e c T o										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ols.javax:1144)atcom.sap.scc.audit.file.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			FileSystemAuditLogger.<init>(FileSystem										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			AuditLogger.java:87) This issue has been fixed										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			d. • Avoid 100% CPU in one thread when using SAP JVM 8										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			to prevent issues that may occur due to a missing b										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			upgfixinthecurrentlyavailableSAPJVM8patchlevel										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ls. • After tramaster-shadowswitch, usagestatistics										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			w e r e s h o w i n g 0 f o r t h e e x c h a n g e d b y t e s i n s t e a d o f										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			the correct value. This issue has been fixed. • Unde										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			r c e r t a i n c o n d i t i o n s , t h e m a s t e r - m a s t e r c h e c k e r										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			job was not started when starting the Cloud Connec										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			tor. As a consequence, this situation was not recognized										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ized and not corrected. This issue has been fixed.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Cloud Connector 2.14.1 - Enhancements	Release of Cloud Connector version 2.14.1 introduces the following enhancements: • System behavior of the	Info only	General Availability	New	Technology	Not applicable			2022-03-10		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			daemon with regard to automatic restart and configu										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ration of maximal threads has been improved. • SNCc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onfigurationwithSAPCryptographicLibraryissim										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			plifiedbyscriptsincludedintheddeliverry.Formore										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			information, see Initial Configuration (RFC).										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - Destination Properties	You can now explicitly set the assertion recipient via a destination property for `OAuth2SAMLBearerAssertion` and `SAMLAssertion` authentication types. For more information,	Info only	General Availability	New	Technology	Not applicable			2022-02-24		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			see HTTP Destinations.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry	Destination Service - OAuth Parameters	• The Destination service now supports the definition	Info only	General Availability	New	Technology	Not applicable			2022-02-10		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			of custombodyparameters for the token request i										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nOAuthflows. • You can send client credentials in th										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e t o k e n r e q u e s t b o d y f o r O A u t h fl o w s (c o nf i g u r a b l e										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			vi a de sti na tio n pr op er ty). For more information, see HTTP Destinations.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Java Connector 3.1.5.2 - Bug Fixes	Release of Java Connector (JCo) version 3.1.5.2 provides the following bug fixes: • Created `JCoCustomDest`	Info only	General Availability	Changed	Technology	Not applicable			2022-02-10		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			initiation instance programmatically ignored exi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			stringUserDatavaluesandusedaSPACEstring("")										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			for reach modified value, which chr esulted in destinati										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onconfigurationerrors, RFC logonfailures, and ot										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			h e r u n d e s i r e d b e h a v i o r b e c a u s e o f m i s s i n g o r w r o n

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			gRFC logon parameters, for example, usage of the

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			wrong logon language. This issue has been fixed.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope

> **Note**
>
> This was a regression bug, introduced

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				with JCo 3.1.5.1.									
			• When cleaning a previously init										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ialized nested IMPORT/EXPORT/CHANGING table										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			before executing a function module, a NullPointe										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rExceptionwasthrownshowingacallstacksimila										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rtojava.lang.NullPointerException: Cannot invo										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ke"com.sap.conn.jco.rt.DefaultTable.getRecord										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			MetaData()"becauseTable"isnullatcom.sap.co										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nn.jco.rt.ComplexParameter.estimateNumBytes(Com										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			pl exParameter.javax:73)atcom.sap.connn.jcco.rt.C										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			complexParameter.<init>(ComplexParameter.java:										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			31)atcom.sap.connn.jco.rt.CompIexTabIeParamet										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			er.<init>(ComplexTableParameter.java:15) due to										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			a missing null check. This issue has been fixed.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope

> **Note**
>
> This was a regression bug, introduced wi

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
				thJCo3.1.5.1									

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Java Connector 3.1.5.1 - Enhancements	Release of Java Connector (JCo) version 3.1.5.1 introduces the following improvement: • When using jco.client	Info only	General Availability	New	Technology	Not applicable			2022-01-13		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			.serialization_format=columnBasedandjc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			o.client.network=LAN, and the data amount to										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			b e s e n t i s s m a ll e r t h a n 8 K B , c o m p r e s s i o n i s t u r n e d										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			of f t o a v o i d t h e i n i t i a l i z a t i o n t i m e o f t h e L Z 4 c o m p r e s										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sionenginee.										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
Connectivity	Cloud Foundry Neo	Java Connector 3.1.5.1 - Bug Fixes	Release of Java Connector (JCo) version 3.1.5.1 provides the following bug fixes: • When running an applicatio	Info only	General Availability	Changed	Technology	Not applicable			2022-01-13		

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nforalongerperiodoftime,JCocouldrunoutofcon										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			nectionhandles, indicated by a JCoException, w										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			hichwasthrownonconnectshowingacallstacksim										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ilartocom.sap.conn.jco.JCoException:(10										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			6)JCO_ERROR_RESOURCE:MaximumnumberofRF										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			Cconnectionsreached[32768](remotesyste										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			mis[]). This issue has been fixed. • If setting allarg										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			eprimitive Java add double value into a parameter, a tab										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			leorastructurefieldoftypeBCD,theautomaticall										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			y c o n v e r t e d v a l u e w a s w r o n g d u e t o a n o v e r f l o w i n t h										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			eJCointernaldatatypeconversionroutines.Thi										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			sissuehasbeenfixed. • When using jco.client.se										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			rialization_format=columnBasedwithparam										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			e t e r s c o n t a i n i n g n e s t e d s t r u c t u r e s w i t h n e s t e d t										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ables, the communication failed with various user errors										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			a t b a c k e n d s i d e i n t h e d e s e r i a l i z e r o f t h e f a s t s e r i a										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			lization. This issue has been fixed. • When using jco										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			.client.pcs=2andjco.client.serializati										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			on_format=columnBased inadestinationconfigu										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			ration, a NullPointerException was thrown onc										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			onnect, due to a wrong integration initialization. This is										

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of	Version	Scope
			issue has been fixed.										

1.1.4.2 2021 Connectivity (Archive)

2021

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry	Connectivity Service - Cloud Connector - SAP Cloud PKI	SAP Cloud PKI (public key infrastructure) is enabled for technical communication between Cloud Connector and Connectivity service. • The change is transparent for the Cloud Connector - as soon as you renew your subaccount certificate in the Cloud Connector, the newly issued X.509 client certificate will be part of SAP Cloud PKI. • If you are using Connectivity proxy software components as part of your solution, make sure you use version 2.4.1 or higher. For scenarios with termination in the ingress, version 2.3.1 of the Connectivity proxy is sufficient.	Info only	New	2021-12-02
Connectivity	Integration Suite	Cloud Foundry Neo	Destination Service - Mail Destinations	You can now configure destinations of type MAIL with any OAuth-based authentication type as available for HTTP destinations, including the option for mTLS via X.509 client certificate. **⚠ Restriction** The Mail Java API (Neo environment) does not provide the `javax.mail.Session` object out of the box. It must be configured manually.	Info only	New	2021-12-02

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry Neo	Cloud Connector 2.14.0.1 - Enhancements	Release of Cloud Connector version 2.14.0.1 introduces the following enhancements: • Cloud Connector can now use SAPMachine 11 as Java runtime. For more information, see Prerequisites. • Cloud Connector supports Windows Server 2022 as additional OS version. For more information, see Prerequisites. • Monitoring was extended to show usage information for service channels (on-premise to cloud scenarios). For more information, see Monitoring. • An administrator can now configure more connectivity-related parameters on the *Configuration* > *Advanced* screen instead of modifying the configuration files on OS level. For more information, see Configure Tunnel Connections. • You can define a different location for audit log and trace files. For more information, see Manage Audit Logs and Troubleshooting • Additional configuration REST APIs let you configure the Cloud Connector remotely. For more information, see Configuration REST APIs. • The additional role *Subaccount Administrator* lets you define authorizations limited to subaccount-related tasks. Cross-subaccount configuration can only be viewed by users having this role. For more information, see Use LDAP for Authentication. • Access control usage monitor data is now persisted and will survive a restart. • The connection check for HTTPS access control entries now reveals information about the causes for a failing check and potential configuration issues if principal propagation with x.509 certificates is used.	Recommended	New	2021-12-02

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
				Action: We recommend that you always use the latest Cloud Connector version. For more information, see Upgrade.			
Connectivity	Integration Suite	Cloud Foundry Neo	Cloud Connector 2.14.0.1 - Fixes	Release of Cloud Connector version 2.14.0.1 provides the following bug fixes: • Audit log selection was not working correctly if the end of the time interval was before noon. This issue has been fixed. • If an RFC connection is broken while waiting for data from the ABAP system, the processing engine could get into an inconsistent state, causing wrong processing for succeeding requests sent from the cloud application, which eventually could make the cloud application hang. This issue has been fixed. • Fixed a race condition that could occur if many requests were sent from the cloud application over the same RFC connection, and if the network from the cloud application to the Cloud Connector was very fast. In such a situation, two threads were processing this single RFC connection, causing an inconsistency that could lead to a `NullPointerException` in `RfcBlock.populateRequestStatistics` when trying to access the field `<performanceStatistics>`. • When configuring an RFC SNC acccess control entry, but overall SNC configuration is incomplete, Cloud Connector now reports the configuration error at runtime instead of falling back to plain RFC, if offered by the backend.	Info only	Changed	2021-12-02

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Features	• The *Destinations* UI in the cockpit lets you configure an X.509 client certificate for automatic token retrieval when using the relevant OAuth-based authentication types. The `AuthenticationHeaderProvider` Java client library, part of SAP Java Buildpack, was adapted as well. The REST API already supports it. • The `ProxyType` attribute now offers the new option `PrivateLink`, allowing you to configure a destination with URL, and optionally a token service URL, pointing to services consumed via the SAP Private Link service (beta). For more information, see also What Is SAP Private Link Service (Beta)?. **Note** The `CheckConnection` functionality as well as automatic token retrieval are not yet supported.	Info only	New	2021-11-18
Connectivity	Integration Suite	Neo	Destination Service - Features	The *Destinations* UI in the cockpit lets you configure an X.509 client certificate for automatic token retrieval when using the relevant OAuth-based authentication types. The `AuthenticationHeaderProvider` Java client library, part of the Neo runtimes, was adapted as well.	Info only	New	2021-11-18
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Bug Fix	• A change has been applied that improves the stability and availability of the service during startup, for example, in case of a rolling update. • A request processing change has been applied, improving asyncronous handling of the processing load, ultimately improving overall service stability and availability.	Info only	Changed	2021-11-18

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry	Destination Service - OAuth	• Destinations with `ProxyType` set to `OnPremise` can now be configured with OAuth-based authentication types, both via the BTP cockpit UI and the Destination service REST API. • Automated token retrieval from OAuth servers residing on premise, exposed via Connectivity service and Cloud Connector, is now supported.	Info only	New	2021-11-04
Connectivity	Integration Suite	Cloud Foundry Kyma	Connectivity Proxy Version 2.4.1	Connectivity proxy version 2.4.1 is now available. • Connectivity CA is now automatically downloaded during help deployment. • Applies critical preparation for adoption of SAP Cloud PKI. • Improved server certificate validation towards remote targets. • Multiple open source software components reported as vulnerable have been replaced.	Info only	Announcement	2021-10-21
Connectivity	Integration Suite	Cloud Foundry	Connectivity Service - Bug Fix	An internal service exception could result in a *502 Bad Gateway* error on the client side, which was visible in the Cloud Connector logs during automatic reconnect. This issue has been fixed.		Changed	2021-10-07
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Bug Fixes	• In some cases, a Destination service instance could not be deleted. The operation now works as expected. • A performance optimisation reduces the overall amount of remote calls to XSUAA when the Destination service is called with a user token. As a result, less load is put on XSUAA, and the Destination service responds faster. This fix contributes to improving overall stability.		Changed	2021-10-07
Connectivity	Integration Suite	Cloud Foundry	SAP Java Buildpack	SAP Java Buildpack has been updated from version 1.38.0 to 1.39.0. • The `com.sap.cloud.security.xsuaa` API has been updated to version 2.10.5. • The Connectivity API extension has been updated to version 3.12.0.		New	2021-09-13

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Authentication Types	When using authentication type `OAuth2ClientCredentials`, you can choose a tenant to perform automated token retrieval that is different from the tenant used to look up the destination configuration. This feature is especially useful for automated service scenarios, like running offline jobs, and so on. **Note** You cannot use this feature in combination with a passed user context. In this case, the tenant used to perform automated token retrieval is exclusively determined by the user context. For more information, see OAuth Client Credentials Authentication.		New	2021-08-26
Connectivity	Integration Suite	Neo	Destinations - Timeout Properties	You can configure timeout properties in the destination configuration, following a documented naming convention. This feature lets you manage timeouts externally, regardless of the cloud application's lifecycle. Using HttpDestination library version 2.15, timeout properties are processed at runtime when pre-configuring the HTTP client instance for the cloud application. For more information, see HTTP Destinations.		New	2021-08-26
Connectivity	Integration Suite	Cloud Foundry Neo	Cloud Connector - Security Fixes	Multiple vulnerabilities in the Cloud Connector have been fixed. For more information, see SAP security note 3058553.		Changed	2021-08-12
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Client Certificates	When generating an X.509 client certificate (as announced on July 1), you can set a password to protect the private key. If you choose a PKCS12 file format, also the keystore is protected by the same password.		New	2021-08-12

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Neo	Destination Service - HTTP Headers	As of HttpDestination version 2.14.0 , any defined HTTP header in the destination configuration (see HTTP Destinations) is processed at runtime, that is, it is added to the request that is sent to the target server.		New	2021-08-12
Connectivity	Integration Suite	Neo	Connectivity Service - Bug Fix	A few stabilisation fixes have been applied in the Connectivity service to handle rare cases in which an abnormal amount of metering data was received by the Cloud Connector. This could cause a partial blockage of the service.		Changed	2021-08-12
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Bug Fix	A few stabilisation fixes have been applied on the REST server-side logic, allowing more efficient parallel request handling under load.		Changed	2021-08-12
Connectivity	Integration Suite	Cloud Foundry Neo	Cloud Connector 2.13.2 - Enhancements	Release of Cloud Connector version 2.13.2 introduces the following improvement: • The HTML validation of login information has been improved.		New	2021-07-15

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry Neo	Cloud Connector 2.13.2 - Fixes	Release of Cloud Connector version 2.13.2 provides the following bug fixes: • A regression prevented that links provided in the login info widget (login screen) could be clicked. This issue has been fixed. • When rewriting a location header for redirect responses (status codes 30x), the lookup to determine the virtual host that replaces the internal one is now case insensitive. • When using custom attributes in JWTs (JSON web tokens) for principal propagation, the value can now be extracted even if represented as single element array. • A slow network could prevent a successful initial push, caused by an unintended timeout. As a consequence, the configuration on the shadow instance could be incomplete, even though the shadow showed a successful connection. This issue has been fixed. • CPIC traces can now be turned on and off multiple times without the need to restart. • Issues with restoring a 2.13.x backup into a fresh installation on Linux have been fixed.		Changed	2021-07-15

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry	Destination Service	• Using the Destination service REST API, you can configure a service-side issued X.509 client certificate as part of the SAP Cloud PKI (public key infrastructure) and formally choose *automatic renewal* of the certificate. • The same feature is available in the *Destinations* editor of the cloud cockpit (*Connectivity* > *Destinations*). • The *SAP Java Buildpack* now includes Java APIs as part of a client library for the Destination service. You can use the `ConnectivityConfiguration` Java API to retrieve destination and certificate configurations, and `AuthenticationHeaderProvider` Java API to provide prepared HTTP headers holding authentication tokens for various scenarios. For more information, see Destination Java APIs.		New	2021-07-01
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Destinations Editor	The *Destinations* UI (editor) in the cockpit lets you create a certificate configuration entry containing an X.509 client certificate part of SAP Cloud PKI (public key infrastructure). Optionally, you can specify values for the certificate common name (CN) as well as for the validity period (minimum value: one day, maximum value: one year). This feature has already been available via the Destination service REST API.		New	2021-06-17
Connectivity	Integration Suite	Cloud Foundry Neo	Destination Service - Destinations Editor	The *Destinations* UI (editor) in the cockpit has introduced a warning message as a reminder that the user's personal password should not be used when configuring the authentication type of a destination, for example, `BasicAuthentication` or `OAuth2Password`. The reason behind is that by design, destination configurations are meant to be used by one or more cloud applications which tipically are used by more than one person.		New	2021-06-17

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry Neo	Connectivity Service - Bug Fix	In rare cases, the Cloud Connector version shown in the cockpit (*Connectivity* → *Cloud Connectors*) got lost. This issue has been fixed.		Changed	2021-06-17
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Bug Fix	An issue has been resolved which prevented updating a service instance (previously created in the cockpit) via the Cloud Foundry CLI.		Changed	2021-06-17
Connectivity	Integration Suite	Cloud Foundry	Connectivity Service - Principal Propagation	You can configure a principal propagation scenario using a user access token issued by the Identity Authentication service (IAS), in addition to the scenarios based on XSUAA. For more information, see Principal Propagation via IAS Token.		New	2021-06-03
Connectivity	Integration Suite	Cloud Foundry	Destination Service - HTTP Destinations	A naming convention has been introduced, specifying how to properly configure HTTP headers and queries in a destination configuration. For more information, see HTTP Destinations.		New	2021-06-03
Connectivity	Integration Suite	Cloud Foundry Neo	Connectivity Service - Tunnel Connections	On the cloud side, the tunnel connection idle threshold has been increased to better match both older (yet supported) and latest Cloud Connector versions (versions lower or equal to 2.13). This ensures the internal heartbeat mechanism would work properly even in some special cases in which short interruptions have been observed.		Changed	2021-06-03
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Bugfix	The service could have experienced delays in case of parallel load which caused requests to be executed unexpectedly slower on random basis. This issue has been fixed.		Changed	2021-06-03

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry Neo	Java Connector 3.1.4.0 - Enhancements	Release of Java Connector (JCo) version 3.1.4.0 introduces the following features and improvements: • JCo now offers the ABAP server processing time for JCo client scenarios via method `JCoThrougput.getServerTime()`. • `JCoRepository` methods were enhanced to ignore trailing blanks in passed structure, table, and function module names which are supposed to be looked up. • Performance was improved for setting `DATE` and `TIME` datatype fields when using *strings* as input values.		New	2021-05-20

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry Neo	Java Connector 3.1.4.0 - Bug Fixes	Release of Java Connector (JCo) version 3.1.4.0 provides the following bug fixes: • When invoking remote function modules (RFMs) via the t/q/bgRFC protocol to the same `JCoDestination` in multiple threads simultaneously, used TIDs, queue names and unit IDs could have been overwritten and used in the wrong thread context, which might have led to data loss in the target system. For example, IDocs that seemed to have been transferred correctly without an error, were not stored in the target system, because the TID contract was broken and several IDocs were erroneously sent at the same time with the same TID although different ones had been specified. This issue has been fixed. **Note** This regression bug was introduced with JCo 3.1.3. • When the new reentrance ticket technology, introduced as of S/4HANA 1909, was used to log on to the communication partner system, and `JCoCustomRepository` was configured to use query mode `DISABLE_REPOSITORY_POOL`, querying RFC metadata resulted in a logon failure (*invalid logon ticket*). This issue has been fixed. • If a JSON document contained numeric fields with negative values, for example, for a field of type `BCD` or `INT`, the JSON parser did not accept the sign character when analyzing the value for the given field. In this case, `JCoRecord.fromJSON()` threw an exception similar to *JCoSerializationException: (191) JCO_ERROR_SERIALIZATION: Digit(s) expected near position <###>*. This issue has been fixed.		Changed	2021-05-20

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry	Cockpit: Cloud Connectors View	The *Cloud Connectors* view in the SAP BTP cockpit is now also available if your account is running on cloud management tools feature set B. For more information, see Monitoring (section *Monitoring from the Cockpit*).		New	2021-04-22
Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.13.1 - Enhancements	Release of Cloud Connector version 2.13.1 introduces the following features and improvements: • Additional audit log entries for changing the trace level are available. • You can open the support log assistant directly from the *Log And Trace Files* screen. For more information, see Troubleshooting, section *Log And Trace Files*. • The dependency on ping checks for connections to the LDAP system, which is used for UI authentication, has been minimized to avoid unnecessary role switches in high availability mode.		New	2021-03-25

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of

Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.13.1 - Fixes	Release of Cloud Connector version 2.13.1 provides the following bug fixes:
				• Connecting a subaccount with several hundred access control entries is working again.
				• With release 2.13.1, restoring a backup created in version 2.13.0 works properly again for a Cloud Connector running on Linux.
				• Backups created in version 2.12.5 and older can be restored properly. Failures on restore led to a non-usable Cloud Connector setup.
				• The following high availability issues have been fixed:
				  • Improved implementation ensures that a high availability setup does not end up in a shadow/ shadow situation. This issue could occur under rare circumstances.
				  • Errors could occur if subaccounts have a larger number of access control entries.
				  • Network issues could prevent the individual replication of configuration changes.
				  • After switching roles, connections can now always be reestablished correctly.
				• The connection test for LDAPS access control entries now works correctly.
				• A memory leak in the comprised netty library has been fixed by upgrading to a newer version.
				• A subaccount display issue has been fixed:
				In version 2.13.0, subaccounts on *eu2.hana.ondemand.com* were displayed as belonging to region Europe (Rot) instead of Europe (Frankfurt).
						Changed	2021-03-25

Connectivity	Integration Suite	Cloud Foundry	Destination Service - Mobile Service Instances	You can create a destination configuration pointing to a mobile service instance, resulting in a fully functional destination configuration, including automatic token retrieval for the respective OAuth flows supported by the mobile service.		New	2021-02-25

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Kyma Other	Destination Service - Consumption from Kyma or Kubernetes Environments	Consumption of the Destination service from the Kyma or Kubernetes environments has been officially documented. For more information, see Create and Bind a Destination Service Instance.		New	2021-02-25
Connectivity	Integration Suite	Cloud Foundry	Principal Propagation Authentication - OpenID Connect	When sending a user principal via the HTTP header `X-user-token`, you can use any *OpenID Connect*-compliant OAuth server and a related OpenID access token for passing the user identity. To enable this feature, you must specify either `x_user_token.jwks_uri` `x_user_token.jwks` as additional attribute, as described in the respective authentication type. or For more information, see HTTP Destinations.		New	2021-02-11
Connectivity	Integration Suite	Cloud Foundry	OAuth - X.509 Client Certificates	You can use X.509 client certificates for OAuth flows supported by the respective authentication types, see OAuth with X.509 Client Certificates.		New	2021-02-11
Connectivity	Integration Suite	Cloud Foundry	"Find Destination" REST API - Skip Credentials	The "Find Destination" REST API endpoint has been enhanced with a new feature enabling the client application to initiate a skip of credentials in the returned response. This parameter is useful especially for OAuth destinations (such as *OAuth2 User Token Exchange*, *OAuth2 JWT Bearer*, *OAuth2 SAML Bearer Assertion*). The client application may actually need only the auto-retrieved token by the service, which makes the credentials optional for the application, and in certain cases they are preferred not to be returned in the response. For more information, see SAP API Business Hub.		New	2021-02-11

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry	Destinations - Authentication Types	• The property `SystemUser` is deprecated. The cockpit now shows an alert if this feature is still in use, suggesting what to do instead. Alternatives for technical user authentication are *Basic Authentication*, *OAuth2 Client Credentials*, or *Client Certificate Authentication*. See also OAuth SAML Bearer Assertion Authentication. **Note** In general, we recommend that you work on behalf of specific (named) users rather than working with a technical user. To extend an OAuth access token's validity, consider using an OAuth refresh token. • Authentication type *SAP Assertion SSO* is deprecated. The cockpit now shows an alert if this feature is still in use, suggesting what to do instead. Authentication types *Principal Propagation* (for on-premise connections), *OAuth2 SAML Bearer Assertion* (Internet connections) or *SAML Assertion* (Internet connections) are the recommended mechanisms for establishing single sign-on (SSO). See SAP Assertion SSO Authentication.		Changed	2021-02-11
Connectivity	Integration Suite	Neo	Password Storage API	The Password Storage API documentation on SAP API Business Hub has been moved from the deprecated API package to SAP Cloud Platform Credential Store.		Changed	2021-01-28

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.13.0 - Enhancements	Release of Cloud Connector version 2.13.0 introduces the following features and improvements: • Cloud Connector 2.13 is based on a different runtime container. Up to version 2.12.x, the JavaWeb 1.x runtime based on Tomcat 7 was used. It now switches to JavaWeb 3.x based on Tomcat 8.5. As a consequence, the internal structure has changed and works differently. The upgrade will adjust these changes as much as possible for versions 2.9 and higher. • Linux on *ppc64 little endian* (ppc64le) is added as a supported platform for the Cloud Connector. For more information, see Prerequisites. • A set of new configuration REST APIs has been added. For more information, see Configuration REST APIs. • An additional screen in the subaccount-specific monitoring provides usage statistics of the various access control entries. Alternatively, you can access the same data using a new monitoring REST API. For more information, see Monitoring. • For access control entries of type TCP, you can configure a port range instead of a single port. For more information, see Configure Access Control (TCP). • You can configure a widget that shows information about the Cloud Connector on the login screen. For more information, see Configure Login Screen Information. • Improved high availability communication supersedes applying SAP note 2915578. • For new Cloud Connector versions, a notification and alert is shown to help you schedule the update. • The Cloud Connector supports JSON Web tokens (JWTs) based on OpenID-Connect (OIDC) for principal propgation authentication (Cloud Foundry environment).		New	2021-01-14

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
				For more information, see Configure Principal Propagation via OIDC Token. • Client-side load balancing based on round-robin was introduced for Cloud Connector connections to SAP Cloud Platform to address its endpoints which are exposed on multiple IP addresses for high availability. • Scenarios based on the HTTP header *Expect: 100-continue* and response code *HTTP 100* are now supported.			
Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.13.0 - Fixes	Release of Cloud Connector version 2.13.0 provides the following bug fixes: • When doing a rollover at midnight, the initial audit log entry for a new file was not created correctly and the audit log checker wrongly assessed such files as corrupted. This issue has been fixed. • Incorrect host information could be used in audit logs related to access control audit entries. This issue has been fixed.		Changed	2021-01-14
Connectivity	Integration Suite	Cloud Foundry	Cloud Connector - Subaccount Configuration	For a subaccount that uses a custom identity provider (IDP), you can choose this IDP for authentication instead of the (default) SAP ID service when configuring the subaccount in the Cloud Connector. For more information, see Use a Custom IDP for Subaccount Configuration.		New	2021-01-14

1.1.4.3 2020 Connectivity (Archive)

2020

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration Suite	Neo Cloud Foundry	Java Connector (JCo) - Client Certificates	JCo provides the new property `jco.client.tls_client_certificate_logon` to support the usage of a TLS client certificate for logging on to an ABAP system via WebSocket RFC. For more information, see: User Logon Properties (Cloud Foundry environment) User Logon Properties (Neo environment) For more information on WebSocket RFC, see also: WebSocket RFC	New	2020-12-17
Connectivity	Integration Suite	Cloud Foundry	HTTP Destinations - Authentication Types	Authentication type *SAP Assertion SSO* is deprecated. It will soon be removed as a feature from the Destination service. Use Principal Propagation SSO Authentication instead, which is the recommended mechanism for establishing single sign-on (SSO).	Deprecated	2020-12-17
Connectivity	Integration Suite	Neo	HTTP Destinations - Authentication Types	Authentication type *SAP Assertion SSO* is deprecated. Use Principal Propagation SSO Authentication instead, which is the recommended mechanism for establishing single sign-on (SSO).	Deprecated	2020-12-17
Connectivity	Integration Suite	Cloud Foundry	HTTP Destinations - Destination Properties	The destination property `SystemUser` for the authentication types: • OAuth SAML Bearer Assertion Authentication • SAP Assertion SSO Authentication will be removed soon. More information on timelines and required actions will be published in the release notes at a later stage. See also: OAuth SAML Bearer Assertion Authentication SAP Assertion SSO Authentication	Announcement	2020-12-03

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration Suite	Neo Cloud Foundry	JCo Runtime - Enhancement	JCo Runtime 3.1.3.0 introduces the following enhancement: If the backend is known to be new enough, JCo does not check for the existence of RFC_METADATA_GET, thus avoiding the need to provide additional authorizations for the repository user.	New	2020-11-05
Connectivity	Integration Suite	Neo Cloud Foundry	JCo Runtime - Bug Fix	JCo Runtime 3.1.3.0 provides the following bug fix: Up to JCo 3.1.2, the initial value for fields of type STRING and XSTRING was *null*. Since the initial value check in ABAP is different, JCo now behaves the same way and uses an emtpy string and an empty byte array, respectively.	Changed	2020-11-05
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Automatic Token Retrieval	The Destination service offers a new feature related to the automatic token retrieval functionality, which lets the destination administrator define HTTP headers and query parameters as additional configuration properties, used at runtime when requesting the token service to obtain an access token. See HTTP Destinations.	New	2020-11-05
Connectivity	Integration Suite	Cloud Foundry	Documentation - Principal Propagation Scenarios	The documentation of principal propagation (user propagation) scenarios provides improved information on the basic concept and guidance on how to set up different scenarios. See Principal Propagation.	Changed	2020-10-22
Connectivity	Integration Suite	Cloud Foundry	Cloud Connector 2.12.5 - Enhancements	Release of Cloud Connector version 2.12.5 introduces the following improvements: • For principal propagation scenarios, custom attributes stored in `xs.user.attributes` of the JWT (JSON Web token) are now accessible for the subject pattern. See Configure a Subject Pattern for Principal Propagation. • Improved resolving for DNS names with multiple IP addresses by adding randomness to the choice of the IP to use. This is relevant for many connectivity endpoints in SAP Cloud Platform, Cloud Foundry environment.	New	2020-10-22

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.12.5 - Fixes	Release of Cloud Connector version 2.12.5 provides the following bug fixes: • After actively performing a master-shadow switch for a disaster recovery subaccount, a zombie connection could cause a timeout of all application requests to on-premise systems. This issue has been fixed. • When refreshing the subaccount certificate in an high availability setup, transferring the changed certificate to the shadow was not immediately triggered, and the updated certificate could get lost. This issue has been fixed. • If many RFC connections were canceled at the same time, the Cloud Connector could crash in the native layer, causing the process to die. This issue has been fixed. • The LDAP configuration test now supports all possible configuration parameters.	Changed	2020-10-22
Connectivity	Integration Suite	Cloud Foundry	Connectivity Service - Service Instances - Quota Management	When using service plan “lite”, quota management is no longer required for this service. From any subaccount you can consume the service using service instances without restrictions on the instance count. Previously, access to service plan “lite” has been granted via entitlement and quota management of the application runtime. It has now become an integral service offering of SAP Cloud Platform to simplify its usage. See also Create and Bind a Connectivity Service Instance.	Changed	2020-10-08
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Service Instances - Quota Management	When using service plan “lite”, quota management is no longer required for this service. From any subaccount you can consume the service using service instances without restrictions on the instance count. Previously, access to service plan “lite” has been granted via entitlement and quota management of the application runtime. It has now become an integral service offering of SAP Cloud Platform to simplify its usage. See also Create and Bind a Destination Service Instance.	Changed	2020-10-08

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry	SAP Java Buildpack - Java Connector (JCo)	The SAP Java Buildpack has been updated from 1.27.3. to 1.28.0. • TomEE Tomcat has been updated from 7.0.104 to 7.0.105. • SAPJVM has been updated to 81.65.65. • The *com.sap.cloud.security.xsuaa* API has been updated from 2.7.5 to 2.7.6. • The SAP HANA driver has been updated from 2.5.49 to 2.5.52. • **JCo-corresponding libraries** have been updated: *connectivity* to 3.3.3, *connectivity apiext* to 0.1.37. • The **activation process for the JCo component** in the SAP Java Buildpack has been changed. Starting with this release, it is activated by setting the following environment variable: **<USE_JCO=true>**. **Note** The **previous activation process** for the JCo component is deprecated and **will expire after a transition period**.	Changed	2020-09-24
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Error Handling	Error handling has been improved for updating service instances via the Cloud Foundry CLI and the cloud cockpit when providing the configuration JSON data.	Changed	2020-09-10
Connectivity	Integration Suite	Neo	Connectivity Service - Bug Fix	A synchronization issue has been fixed on cloud side that in very rare cases could lead to a *zombie* tunnel from the Cloud Connector to SAP Cloud Platform, which required to reconnect the Cloud Connector.	Changed	2020-09-10
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Bug Fix	During *Check Connection* processing of a destination with basic authentication, the Destination service now uses the user credentials for both the HTTP HEAD and HTTP GET requests to verify the connection on HTTP level.	Changed	2020-09-10
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Bug Fix	Using authentication type `OAuth2SAMLBearerAssertion`, an issue could occur when adding the user's SAML group attributes into the resulting SAML assertion that is sent to the target token service. This issue has been fixed.	Changed	2020-08-13

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry	Destination Service REST API - Pagination Feature	The REST API pagination feature provides improved error handling in case of issues with the pagination, for example, if an invalid page number is provided.	Changed	2020-08-13
Connectivity	Integration Suite	Neo	HttpDestination Library - New Version	The `HttpDestination` v2 library has been officially released in the Maven Central Repository . It enables the usage in Tomcat and TomEE-based runtimes the same way as in the deprecated *JavaWeb* and *Java EE 6 Web Profile* runtimes. See also HttpDestination Library.	New	2020-07-30
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Bug Fix	An error handling issue has been fixed in the Destination service, which is related to the recently introduced *SAP Assertion SSO authentication* type. If a wrong input was provided, you can now see the error properly, and recover it.	Changed	2020-07-30
Connectivity	Integration Suite	Cloud Foundry	Destinations - Authentication Types	You can use authentication type `OAuth2JWTBearer` when configuring a Destination. It is a simplified version of the authentication type `OAuth2UserTokenExchange` and represents the official OAuth grant type for exchanging OAuth tokens. See HTTP Destinations.	New	2020-07-02
Connectivity	Integration Suite	Cloud Foundry	Destination Service - HTTP Header	The Destination service provides a prepared HTTP header that simplifies application and service development. See HTTP Destinations (code samples).	New	2020-07-02
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Bug Fix	A concurrency issue in the Destination service, related to parallel auth token retrieval in the token cache functionality, could result in partial request failures. This issue has been fixed.	Changed	2020-07-02
Connectivity	Integration Suite	Cloud Foundry	HTTP Destinations - Authentication Types	The Cloud Foundry environment supports `SAP Assertion SSO` as authentication type for configuring destinations in the Destination service. See HTTP Destinations.	New	2020-06-18

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry	Destination Service REST API	The "Find Destination" REST API now includes the scopes of the automatically retrieved access token in the response that is returned to the caller. See "Find Destination" Response Structure.	New	2020-06-04
Connectivity	Integration Suite	Cloud Foundry	Destinations for Service Instances	For subscription-based scenarios, you can use an automated procedure to create a destination that points to your service instance. See Managing Destinations.	New	2020-06-04
Connectivity	Integration Suite	Neo Cloud Foundry	Connectivity Service - Bug Fix	In rare cases, establishing a secure tunnel between Cloud Connector (version 2.12.3 or older) and the Connectivity service could cause an issue that requires to manually disconnect and connect the Cloud Connector. This issue has been fixed. The fix requires Cloud Connector version 2.12.4 or higher.	Changed	2020-05-21
Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.12.4 - Features	Release of Cloud Connector version 2.12.4 introduces the following features and enhancements: • You can activate the SSL trace in the Cloud Connector administration UI also for the shadow instance.	New	2020-05-07
Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.12.4 - Fixes	Release of Cloud Connector version 2.12.4 provides the following bug fixes: • You can edit and delete domain mappings in the Cloud Connector administration UI correctly. • The REST API does no longer return an empty configuration. • REST API DELETE operations do not require setting a content-type application/json to function properly. • If more than 2000 audit log entries match a selection, redefining the search and getting a shorter list now works as expected. • A potential leak of HTTP backend connections has been closed.	Changed	2020-05-07
Connectivity	Integration Suite	Cloud Foundry	Connectivity Service - Bug Fix	A fix has been applied in the Connectivity service internal load balancers, enabling the sending of TCP keep-alive packets on client and server side. This change mainly affects SOCKS5-based communication scenarios.	Changed	2020-03-26

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Service Instances	You can create a service instance specifying an update policy. This allows you to avoid name conflicts with existing destinations. See Create and Bind a Destination Service Instance.	New	2020-03-26
Connectivity	Integration Suite	Cloud Foundry	Cockpit - Destination Management	The *Destinations* editor in the cockpit is available for accounts running on the cloud management tools feature set B. See Managing Destinations.	New	2020-03-12
Connectivity	Integration Suite	Neo	Connectivity Service - Bug Fix	When creating or editing a destination with authentication type `OAuth2ClientCredentials` in the cockpit, the parameter `Audience` could not be added as additional property. This issue has been fixed.	Changed	2020-03-12
Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.12.3 - Features	Release of Cloud Connector version 2.12.3 introduces the following features and enhancements: • When using the SAP JVM as runtime, the thread dump includes additional information about currently executed RFC function modules. • The hardware monitor includes a Java Heap history, showing the usage in the last 24 hours. • If you are using the file `scc_daemon_extension.sh` to extend the daemon in a Linux installation, the content is included in the initialization section of the daemon. This lets you make custom extensions to the daemon that survive an upgrade. See Installation on Linux OS, section *Installer Scenario*.	New	2020-02-27

Technical Component	Capability	Environment	Title	Description	Type	Available as of

Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.12.3 - Fixes	Release of Cloud Connector version 2.12.3 provides the following bug fixes:
				• When switching roles between master and shadow instance in a high availability setup, the switch is no longer blocked by active RFC function module invocations.
				• A fix in the backend HTTP connection handling prevents issues when the backend tries to send the HTTP response before completely reading the HTTP request.
				• When sending large amounts of data to an on-premise system, and using RFC with a network that provides large bandwidth, the Cloud Connector could fail with the error message *Received invalid block with negative size*. This issue has been fixed.
				• The Cloud Connector admin UI now shows the correct user information for installed Cloud Connector instances in the *About* window.
				• Fixes in the context of disaster recovery:
				  • The location ID is now handled properly when setting it *after* adding the recovery subaccount.
				  • Application trust settings and application-specific connections are applied in the disaster case.
				  • Principal propagation settings are applied in the disaster case
					Changed	2020-02-27

Connectivity	Integration Suite	Neo Cloud Foundry	JCo Runtime - WebSocket RFC	The JCo runtime in SAP Cloud Platform lets you use WebSocket RFC (RFC over Internet) with ABAP servers as of S/4HANA (on-premise) version 1909. In the RFC destination configuration, this is reflected by new configuration properties and by the option to choose between different proxy types. See Target System Configuration (Cloud Foundry environment), or Target System Configuration (Neo environment).	New	2020-02-13
Connectivity	Integration Suite	Cloud Foundry	Connectivity Service for Trial Accounts - Bug Fix	The Connectivity service is operational again for trial accounts. A change in the Cloud Foundry Core component caused the service not be accessible by applications hosted in DiegoCell that are dedicated for trial usage in a separate VPC (virtual private cloud) account. This issue has been fixed.	Changed	2020-01-30

1.1.4.4 2019 Connectivity (Archive)

2019

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.12.2 - Features	Release of Cloud Connector version 2.12.2 introduces the following features and enhancements: • You can turn on the TLS trace from the Cloud Connector administration UI instead of modifying the `props.ini` file on OS level. See Troubleshooting. • The status of the used subaccount certificate is shown on the *Subaccount* overview page of the Cloud Connector administration UI, in addition to expiring certificates shown in the *Alerting* view. See Establish Connections to SAP Cloud Platform.	New	2019-12-05
Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.12.2 - Fixes	Release of Cloud Connector version 2.12.2 provides the following bug fixes: • Subject values for certificates requiring escaping are treated correctly. • Establishing a connection to the master is now possible when being logged on to the shadow with a user that has a space in its name. • Performance statistics could show too long total execution times. This issue has been fixed. • IP address changes for the connectivity service hosts are recognized properly. • The Cloud Connector could crash on Windows, when trying to enable the payload trace with 4-eyes-principle without the required user permissions. This issue has been fixed.	Changed	2019-12-05
Connectivity	Integration Suite	Cloud Foundry	Connectivity Service - Bug Fix	Applications sending a significant amount of data payload during OAuth authorization processing could cause an out-of-memory error on the Connectivity service side. This issue has been fixed.	Changed	2019-11-21

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration Suite	Neo	Region Europe (Frankfurt) - Change of Connectivity Service Hosts	The following IP addresses of the Connectivity service hosts for region Europe/Frankfurt (`eu2.hana.ondemand.com`) will change on **26 October 2019**: • `connectivitynotification.eu2.hana.ondemand.com`: from 157.133.70.140 (current) to 157.133.206.143 (new) • `connectivitycertsigning.eu2.hana.ondemand.com`: from 157.133.70.132 (current) to 157.133.205.174 (new) • `connectivitytunnel.eu2.hana.ondemand.com`: from 157.133.70.141 (current) to 157.133.205.233 (new) If you have allowed the current addresses or IP ranges in your firewall rules, make sure you also include the new values **before 26 October 2019**. See also: Prerequisites: Network.	Announcement	2019-10-03
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Connection Check	Using the *Destinations* editor in the cockpit, you can check connections also for on-premise destinations. See Check the Availability of a Destination.	Changed	2019-09-26
Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector - Java Runtime	The support for using Cloud Connector with Java runtime version 7 will end on December 31, 2019. Any Cloud Connector version released after that date may contain Java byte code requiring at least a JVM 8. We therefore strongly recommend that you perform **fresh** installations only with **Java 8**, and **update** existing installations running with Java 7, to Java 8 as of now. See SAP Cloud Connector – Java 7 support will phase out and Update the Java VM.	Announcement	2019-09-13

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.12.1 - Features	Release of Cloud Connector version 2.12.1 introduces the following features and enhancements: • Subject Alternative Names are separated from the subject definition and provide enhanced configuration options. You can configure complex values easily when creating a certificate signing request. See Exchange UI Certificates in the Administration UI. • In a high availability setup, the master instance detection no longer switches automatically if the configuration between the two instances is inconsistent. • Disaster recovery switch back to main subaccount is periodically checked (if not successful) every 6 hours. • Communication to on-premise systems supports SNI (Server Name Indication).	New	2019-08-15
Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.12.1 - Fixes	Release of Cloud Connector version 2.12.1 provides the following bug fixes: • The communication between master and shadow instance no longer ends up in unusable clients that show 403 results due to CSRF (Cross-Site Request Forgery) failures, which could cause undesired role switches. • When restoring a backup, the administrator password check works with all LDAP servers. • The LDAP configuration test utility properly supports secure communication. • The Refresh Subaccount Certificate dialog is no longer hanging when the refresh action fails due to some authentication or authorization issue.	Changed	2019-08-15
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Scope Attribute for OAuth-based Authentication Types	You can use the `scope` destination attribute for the OAuth-based authentication types `OAuth2ClientCredentials`, `OAuth2UserTokenExchange` and `OAuth2SAMLBearerAssertion`. This additional attribute provides flexibility on destination configuration level, letting you specify what scopes are selected when the OAuth access token is automatically retrieved by the service. See HTTP Destinations.	New	2019-08-15

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration Suite	Neo	JCo Runtime for SAP Cloud Platform - Features	• Additional APIs have been added to `JCoBackgroundUnitAttributes`. See API documentation for details. • If a structure or table contains only char-like fields, new APIs let you read or modify all of them at once for the structure or the current table row. See API documentation of `JCoTable` and `JCoStructure`.	New	2019-07-18
Connectivity	Integration Suite	Neo	JCo Runtime for SAP Cloud Platform - Fixes	• qRFC and tRFC requests sent to an ABAP system by JCo can be monitored again by AIF. • Structure fields of type STRING are no longer truncated if there is a white space at the end of the field.	Changed	2019-07-18
Connectivity	Integration Suite	Cloud Foundry	Connectivity Service - JCo Multitenancy	The Connectivity service supports multitenancy for JCo applications. This feature requires a runtime environment with SAP Java Buildpack version 1.9.0 or higher. See Scenario: Multitenancy for JCo Applications (Advanced).	New	2019-06-20
Connectivity	Integration Suite	Cloud Foundry	Cloud Cockpit - Cloud Connector View	The Cloud Connector view is available also for Cloud Foundry regions. It lets you see which Cloud Connectors are connected to a subaccount.	New	2019-04-25

Technical Component	Capability	Environment	Title	Description	Type	Available as of

Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.12 - Features	Release of Cloud Connector version 2.12 introduces the following features and enhancements:
				• The administration UI is now accessible not only with an administrator role, but also with a display and a support role. See Configure Named Cloud Connector Users and Use LDAP for Authentication.
				• For HTTP access control entries, you can
				  • allow a protocol upgrade, e.g. to WebSockets, for exposed resources. See Limit the Accessible Services for HTTP(S).
				  • define which host (virtual or internal) is sent in the host header. See Expose Intranet Systems, Step 8.
				• A disaster recovery subaccount in disaster recovery mode can be converted into a standard subaccount, if a disaster recovery region replaces the original region permanently. See Convert a Disaster Recovery Subaccount into a Standard Subaccount.
				• A service channel overview lets you check at a glance, which server ports are used by a Cloud Connector installation. See Service Channels: Port Overview.
				• Important subaccount configuration can be exported, and imported into another subaccount. See Copy a Subaccount Configuration.
				• An LDAP authentication configuration check lets you analyze and fix configuration issues before activating the LDAP authentication. See Use LDAP for Authentication.
				• You can use different user roles to access the Cloud Connector configuration REST APIs. See Configuration REST APIs.
				• REST APIs for shadow instance configuration have been added. See Shadow Instance Configuration.
				• You can define scenarios for resources. Such a scenario can be exported, and imported into other hosts. See Configure Accessible Resources.
					New	2019-04-25

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.12 - Fixes	Release of Cloud Connector version 2.12 provides the following bug fixes: • The SAN (subjectAlternativeName) usage in certificates can be defined in a better way and is stored correctly in the certificate. See Exchange UI Certificates in the Administration UI. • `IllegalArgumentException` does not occur any more in HTTP processing, if the backend closes a connection and data are streamed. • DNS caching is now recognized in reconnect situations if the IP of a DNS entry has changed. • SNC with load balancing now works correctly for RFC SNC-based access control entries. • A master-master situation is also recognized if, at startup of the former master instance, the new master (the former shadow instance) is not reachable. • Solution management model generation works correctly for a shadow instance. • The daemon is started properly on SLES 12 standard installations at system startup.	Changed	2019-04-25
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Authentication Types	Authentication type `OAuth2SAMLBearerAssertion` provides two different types of `Token Service URL`: • `Dedicated`: used in the context of a single tenant, or • `Common`: used in the context of multiple tenants. For type `Common`, the tenant subdomain is automatically set to the target `Token Service URL`. In addition, cloud applications can use the `x-user-token` HTTP header to propagate the user access token to the external target service at runtime. By default, the user principal is processed via the authorization HTTP header. See SAML Bearer Assertion Authentication.	New	2019-04-11
Connectivity	Integration Suite	Neo Cloud Foundry	Connectivity Service - Fix	When an on-premise system closed a connection that uses an RFC or SOCKS5 proxy, the Connectivity service kept the connection to the cloud application alive. This issue has been fixed. The connection is now always closed right after sending the response.	Changed	2019-04-11

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration Suite	Cloud Foundry	Connectivity Service - Protocols	The Connectivity service supports TCP connections to on-premise systems, exposing a SOCKS5 proxy to cloud applications. This feature follows the concept of binding the credentials of a Connectivity service instance. See Using the TCP Protocol for Cloud Applications.	New	2019-03-14
Connectivity	Integration Suite	Neo	Connectivity Service - Fix	After receiving an on-premise system response with HTTP header *Connection: close*, the Connectivity service kept the HTTP connection to the cloud application alive. This issue has been fixed. The connection is now always closed right after sending the response.	Changed	2019-03-14
Connectivity	Integration Suite	Neo	Cloud Connector - Certificate Update	For the Connectivity service (Neo environment), **a new, region-specific certificate authority** (X.509 certificate) is being introduced. If you use the Cloud Connector for on-premise connections to the Neo environment, you must import the new certificate authority into your trust configuration. • After the **next month** (concrete notification will be rolled out), the current certificate authority will no longer be used to issue client certificates for Cloud Connector deployments, and only the new one will be used. The Connectivity service will still trust client certificates of Cloud Connector deployments that were already issued. • After a **three-month period** (concrete notification will be rolled out), that trust will be removed and your Cloud Connector deployment must be configured to use the new client certificates. See Update the Certificate for a Subaccount.	Announcement	2019-02-28
Connectivity	Integration Suite	Cloud Foundry	Destination Service - Authentication Types	The new authentication type `OAuth2UserTokenExchange` lets your applications use an automated exchange of user access tokens when accessing other applications or services. The feature supports single-tenant and multi-tenant scenarios. See OAuth User Token Exchange Authentication.	New	2019-02-14
Connectivity	Integration Suite	Neo	RFC - Stateful Sequences	You can make a stateful sequence of function module invocations work across several request/response cycles. See Invoking ABAP Function Modules via RFC.	Changed	2019-01-31

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration Suite	Neo Cloud Foundry	Cloud Connector 2.11.3	A security note for Cloud Connector version 2.11.3 has been issued. See SAP note 2696233.	Changed	2019-01-15
Connectivity	Integration Suite	Cloud Foundry	Protocols - RFC Communication	You can use the RFC protocol to set up communication with on-premise ABAP systems for applications in the Cloud Foundry environment. This feature requires a runtime environment with SAP Java Buildpack version 1.8.0 or higher. See Invoking ABAP Function Modules via RFC.	New	2019-01-17
Connectivity	Integration Suite	Cloud Foundry	Destinations - Renew Certificates	A button in the *Destinations* editor lets you update the validity period of an X.509 certificate. See Set up Trust Between Systems.	New	2019-01-17

1.1.4.5 2018 Connectivity (Archive)

2018

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration	Neo	Connectivity Service - Performance	A change in the SAP Cloud Platform Connectivity service improves performance of data upload (on-premise to cloud) and data download (cloud to on-premise) up to 4 times and 15-30 times respectively.	Changed	2018-12-20

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration	Neo	Connectivity Service - Resilience	The Connectivity service has a better protection against zombie connections, which improves resilience and overall availability for the cloud applications consuming it.	Changed	2018-12-20
Connectivity	Integration	Neo	Password Storage Service	A Password Storage REST API is available in the SAP API Business Hub, see Password Storage (Neo Environment).	New	2018-12-06
Connectivity	Integration	Neo	Destination Configuration Service	A Destination Configuration service REST API is available in the SAP API Business Hub.	New	2018-12-06
Connectivity	Integration	Cloud Foundry	Destination Service	A Destination service REST API is available in the SAP API Business Hub.	New	2018-12-06
Connectivity	Integration	Neo	JCo Runtime for SAP Cloud Platform - Fixes	• When using `JCoRecord.fromJSON()` for a structure parameter, the data is now always sent to the backend system. Also, you do not need to append the number of provided rows for table parameters before parsing the JSON document any more. • Depending on the configuration of certain JCo properties, an internally managed connection pool could throw a `JCoException` (error group `JCO_ERROR_RESOURCE`). In a thread waiting for a free connection from this pool, an error message then erroneously reported that the pool was exhausted . This error situation could occur if the used destination was not configured with the property `jco.destination.max_get_client_time` set to 0 and the destination's `jco.destination.peak_limit` value was set higher than the `jco.destination.pool_capacity`. This issue has been fixed.	Changed	2018-12-06

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration	Neo	JCo Runtime for SAP Cloud Platform - Features	Support of the RFC fast serialization. Depending on the exchanged parameter and data types, the performance improvements for RFC communication can reach multiple factors. See SAP note 2372888 (prerequisites) and Parameters Influencing Communication Behavior (JCo configuration in SAP Cloud Platform).	Changed	2018-12-06
Connectivity	Integration	Neo	JCo Runtime for SAP Cloud Platform - Information	Local runtimes on Windows must install the VS 2013 redistributables for x64, instead of VS 2010.	Changed	2018-12-06
Connectivity	Integration	Neo Cloud Foundry	Cloud Connector Fixes	Release of Cloud Connector 2.11.3: • An issue in RFC communication could cause the trace entry *com.sap.scc.jni.CpicCommunicationException: no SAP ErrInfo available* when the network is slow. This issue has been fixed. • The Windows service no longer runs in *error 1067* when stopped by an administrator. • In previous releases, the connection between a shadow and a master instance occasionally failed at startup and produced an empty error message. This issue has been fixed. • The Cloud Connector does not cache Kerberos tokens in the protocol handler any more, as they are one-time tokens and cannot be reused. • For HTTP access control entries, you can configure resources containing a # character.	Changed	2018-12-06

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration	Neo Cloud Foundry	Cloud Connector Enhancements	Release of Cloud Connector 2.11.3: • If the user *sapadm* exists on a system, the installation on Linux assigns it to the *sccgroup*, which is a prerequisite for solution management integration to work properly, see Configure Solution Management Integration [page 1959]. • Restoring a backup has been improved. See Configuration Backup [page 1964]. • The HTTP session store size has been reduced. You can handle higher loads with a given heap size. • Cipher suite configuration has been improved. Also, there is a new security status entry for cipher suites, see Recommendations for Secure Setup [page 1665].	Changed	2018-12-06
Connectivity	Integration	Neo	HTTP Destinations	The *OAuth2 Client Credentials* grant type is supported by the *Destinations* editor in the SAP Cloud Platform cockpit as well as by the client Java APIs `ConnectivityConfiguration`, `AuthenticationHeaderProvider` and `HttpDestination`, available in SAP Cloud Platform Neo runtimes. See OAuth Client Credentials Authentication.	Changed	2018-10-11
Connectivity	Integration	Cloud Foundry	User Propagation	The connectivity service supports the SaaS application subscription flow and can be declared as a dependency in the get dependencies subscription callback, also via MTA (multi-target)-bundled applications. See Consuming the Connectivity Service (Cloud Foundry Environment) and Configure Principal Propagation via User Exchange Token (Cloud Foundry Environment).	Changed	2018-09-27
Connectivity	Integration	Neo Cloud Foundry	Cloud Connector 2.11.2	Release of Cloud Connector 2.11.2 • SNC configuration now provides the value of the environment variable `SECUDIR`, which you need for the usage of the SAP Cryptographic Library (SAPCRYPTOLIB). See Initial Configuration (RFC). • On Linux, the RPM (Red Hat Package Manager) now ensures that the configuration of the interaction with the SAP Host Agent (used for the Solution Manager integration) is adjusted. See Configure Solution Management Integration. • The Cloud Connector shadow instance now provides a configuration option for the connection and request timeout that may occur during health check against the master instance. See Master and Shadow Administration.	Changed	2018-08-16

Technical Component	Capability	Environment	Title	Description	Type	Available as of
Connectivity	Integration	Neo Cloud Foundry	Cloud Connector 2.11.2	Fixes of Cloud Connector 2.11.2 • In a high availability setup, the switch from the master instance to the shadow instance occasionally caused communication errors towards on-premise systems. This issue has now been fixed. • You can now import multiple certificates with the same subject to the trust store. Details about expiration date and issuer are displayed in the tool tip. See Set Up Trust, section *Trust Store*. • You can now configure also the MOC (Multiple Origin Composition) OData service paths as resources. • The `Location` header is now adjusted correctly according to your access control settings in case of a redirect. • Principal propagation now also works with SAML assertions that contain an empty attribute element. • SAP Cloud Platform applications occasionally got an HTTP 500 (internal server error) response when an HTTP connection was closed. The applications are now always informed properly.	Changed	2018-08-16
Connectivity	Integration	Neo	HttpDestination Library	The SAP `HttpDestination` library (available in the SDK and cloud runtime "Java EE 6 Web Profile") now creates Apache `HttpClient` instances which work with strict SNI (Server Name Indication) servers. Use cases with strict SNI configuration on the server side will no longer get the error message *Failure reason: "peer not authenticated"*, that was raised either at runtime or while performing a connection test via the SAP Cloud Platform cockpit *Destinations* editor (*Check Connection* function).	Changed	2018-08-16

1.1.4.6 2017 Connectivity (Archive)

Archived release notes for 2017 and older.

28 September 2017 - Connectivity

New

The destination service (Beta) is available in the Cloud Foundry environment. See Consuming the Destination Service [page 1550].

3 August 2017 - Connectivity

Enhancement

Cloud Connector

Release of SAP Cloud Platform Cloud Connector 2.10.1.

- The URLs of HTTP requests can now be longer than 4096 bytes.
- SAP Solution Manager can be integrated with one click of a button if the host agent is installed on a Cloud Connector machine. See the *Solution Management* section in Monitoring [page 2006].
- The limitation that only 100 subaccounts could be managed with the administration UI has been removed. See Adding and Managing Subaccounts [page 1682].

Fix

Cloud Connector

- The regression of 2.10.0 has been fixed, as principal propagation now works for RFC.
- The cloud user store works with group names that contain a backslash (\) or a slash (/).
- Proxy challenges for NT LAN Manager (NTLM) authentication are ignored in favor of Basic authentication.
- The back-end connection monitor works when using a JVM 7 as a runtime of Cloud Connector.

25 May 2017 - Connectivity

Enhancement

Cloud Connector

Release of SAP HANA Cloud connector 2.10.0.1

- Support of connectivity to an SAP Cloud Platform Cloud Foundry environment.
- Support of direct connectivity with S/4HANA Cloud systems. You can open a Service Channel to an S/4HANA Cloud system in order to use Communication Scenarios requiring RFC communication to S/4HANA Cloud. See Configure a Service Channel for RFC [page 1950].
- Support of arbitrary protocols via the possibility to configure a TCP access control entry. SAP Cloud Platform Connectivity is offering a SOCKS5 proxy, with which you can address such exposed hosts. See Using the TCP Protocol for Cloud Applications.
- Support for disaster recovery events of SAP Cloud Platform regions. For each subaccount you can configure a disaster recovery subaccount for a disaster region. In case of a disaster, the disaster recovery account can be switched active immediately using the exact same configuration. See Configure a Disaster Recovery Subaccount.
- In the access control settings you can add further constraints for RFC based communication to ABAP systems: an administrator can configure, which clients shall be exposed and can define which users should not be able to access the system via Cloud Connector. See Configure Access Control (RFC) [page 1752].
- You can generate self-signed certificates for CA and system certificate so that you can setup demo scenarios with principal propagation without the need of using lengthy openSSL or keytool command sequences. See Configure a CA Certificate [page 1706] and Initial Configuration (HTTP) [page 1676].
- A first set of monitoring HTTP APIs have been introduced: The state of all subaccount connections, a back-end connection monitor and a performance overview monitor. See Monitoring APIs [page 2016].
- On Windows platforms, Cloud Connector 2.10 now requires Visual Studio 2013 runtime libraries.

Fix

Cloud Connector

- The is no longer a bottleneck that could lengthen the processing times of requests to exposed back-end systems, after many hours under high load when using principal propagation, connection pooling, and many concurrent sessions.
- Session management is no longer terminating early active sessions in principal propagation scenarios.
- On Windows 10 hardware metering in virtualized environments shows hard disk and CPU data.

11 May 2017 - Connectivity

New

In case the remote server supports only TLS 1.2, use this property to ensure that your scenario will work. As TLS 1.2 is more secure than TLS 1.1, the default version used by HTTP destinations, consider switching to TLS 1.2.

30 March 2017 - Connectivity

Enhancement

The release of SAP Cloud Platform Cloud Connector 2.9.1 includes the following improvements:

- UI renovations based on collected customer feedback. The changes include rounding offs, fixes of wrong/odd behaviors, and adjustments of controls. For example, in some places tables were replaced by *sap.ui.table.Table* for better experience with many entries.
- You can trigger the creation of a thread dump from the *Log and Trace Files* view.
- The connection monitor graphic for idle connections was made easier to understand.

Fix

- When configuring authentication for LDAP, the alternate host settings are no longer ignored.
- The email configuration for alerts is processing correctly the user and password for access to the email server.
- Some servers used to fail to process HTTP requests when using the HTTP proxy approach (HTTP Proxy for On-Premise Connectivity) on the SAP Cloud Platform side.
- A bottleneck was removed that could lengthen the processing times of requests to exposed back-end systems under high load when using principal propagation.
- The Cloud Connector accepts passwords that contain the '§' character when using authentication-mode password.

16 March 2017 - Connectivity

Enhancement

Update of JCo runtime for SAP Cloud Platform. See What is SAP BTP Connectivity? [page 4].

Fix

A `java.lang.NullPointerException` might have occurred when using a `JCoRepository` instance in roundtrip optimization mode (will be used if the JCo property *jco.use_repository_roundtrip_optimization* was set to 1 at its creation time). The `NullPointerException` was either thrown when trying to execute a *JCoFunction* object, which has been created by such a repository instance, or even earlier when querying the meta data for a *JCoFunction*, *JCoFunctionTemplate* or a *JCoRecordMetaData* object from an AS ABAP back-end system. Only certain complex data structures and table parameter definitions were affected by this bug.

Older Release Notes

- 2016
- 2015

- 2014
- 2013

1.1.5 Administration

Learn about administration tools, user roles, monitoring, and troubleshooting in SAP BTP, multi-cloud foundation.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Task	Description
Administration Tools [page 1300]	Overview of administration tools for SAP BTP Connectivity.
User Roles [page 1301]	User roles and role collections for SAP BTP Connectivity.
Monitoring and Troubleshooting [page 1305]	Find information on monitoring tools, platform status, and SAP support.

1.1.5.1 Administration Tools

Find an overview of administration tools for SAP BTP Connectivity.

Tool	Description
Destinations Editor [page 1472]	Use the Destinations editor in the SAP BTP cockpit to create and configure SAP BTP destinations.
Destination Service REST API [page 1497]	Find information about the Destination service REST API for SAP BTP.
Cloud Connector Administration UI [page 1970] (local network)	Learn more about operating the Cloud Connector, using its administration UI.
Cloud Connector UI in the Cockpit [page 2006]	The cockpit includes a *Connectivity* section, where you can check the status of the Cloud Connector(s) attached in the current subaccount, if any, and get some detail information about these Cloud Connectors.

1.1.5.2 User Roles

Find information about user roles for SAP BTP Connectivity.

> **Note**
>
> This section refers to SAP BTP, Cloud Foundry environment. For information on role assignment in Kubernetes/Kyma environment, see Assign Roles in the Kyma Environment.

In this document, we refer to different types of user roles – *responsibility roles* and *technical roles*. Responsibility roles describe the required user groups and their general tasks in the end-to-end setup process. Configuring technical roles, you can control access to the dedicated cloud management tools by assigning specific permissions to users.

Responsibility Roles [page 1301]

Technical Roles [page 1301]

Responsibility Roles

The end-to-end use of the Connectivity service and the Destination service requires these **user groups**:

- *Application operators* - are responsible for productive deployment and operation of an application on SAP BTP. Application operators are also responsible for configuring the remote connections (destination and trust management) that an application might need, see Administration [page 1300].
- *Application developers* - develop a connectivity-enabled SAP BTP application by consuming the Connectivity service and/or the Destination service, see Developing Applications [page 1306].
- *IT administrators* - set up the connectivity to SAP BTP in your on-premise network, using the Cloud Connector [page 1602].

Some procedures on the SAP BTP can be done by developers as well as by application operators. Others may include a mix of development and operation tasks. These procedures are labeled using icons for the respective task type.

Task Types		
Operator	Developer	Operator and/or Developer

Technical Roles

To perform connectivity tasks in the Cloud Foundry environment, the following **technical roles** apply:

The Cloud Foundry environment provides **dedicated roles** for specific operations. They can be assigned to **custom role collections**, but some of them are also available in **default role collections**.

Technical Connectivity Roles and Operations [page 1302]

Default Role Collections [page 1305]

> **Note**
>
> To see the Destination editor on subaccount level, you must have at least the *Destination Viewer* role, or both the *Destination Configuration Viewer* and the *Destination Certificate Viewer* roles.
>
> For the Destination editor on service instance level, the corresponding roles apply: You need at least the *Destination Viewer Instance* role, or both the *Destination Configuration Instance Viewer* and the *Destination Certificate Instance Viewer* roles.

Technical Connectivity Roles and Operations

Level	Operation (SAP BTP Cockpit or Cloud Connector)	Role
Subaccount	**Connect a Cloud Connector** to a subaccount (Cloud Connector)	Cloud Connector Administrator
	Manage destinations (all CRUD operations) on subaccount level (cockpit) **Note** CRUD (*Create, Read, Update, Delete*) operations include actions like create destinations, display (view) destinations, edit destinations, and delete destinations. For more information, see Managing Destinations [page 1349].	One of these roles: • Destination Administrator • Destination Configuration Administrator
	View destinations (read operations) on subaccount level (cockpit)	One of these roles: • Destination Viewer • Destination Configuration Viewer
	Manage certificates (all CRUD operations) on subaccount level (cockpit) **Note** CRUD (*Create, Read, Update, Delete*) operations include actions like create destinations, display (view) destinations, edit destinations, and delete destinations. For more information, see Managing Destinations [page 1349].	One of these roles: • Destination Administrator • Destination Certificate Administrator

Level	Operation (SAP BTP Cockpit or Cloud Connector)	Role
	View certificates (read operations) on subaccount level (cockpit)	One of these roles: • Destination Viewer • Destination Certificate Viewer
	Generate or renew the subaccount key pair for trust management (cockpit)	One of these roles: • Destination Administrator • Destination Subaccount Trust Administrator
	Download the subaccount key pair for trust management (cockpit)	One of these roles: • Destination Viewer • Destination Subaccount Trust Viewer
Subaccount	**View Cloud Connectors** connected to a subaccount (cockpit)	A role containing the permission `readSCCTunnels`, for example, the predefined role `Cloud Connector Administrator`.
Service instance	**Manage destinations** (all CRUD operations) on service instance level (cockpit) **Note** CRUD (*Create, Read, Update, Delete*) operations include actions like create destinations, display (view) destinations, edit destinations, and delete destinations. For more information, see Managing Destinations [page 1349].	One of these roles: • Destination Administrator • Destination Configuration Administrator • Destination Administrator Instance • Destination Configuration Instance Administrator *plus* one of these roles: • *Org Manager* • *Space Manager* • *Space Developer* See User and Member Management.

Level	Operation (SAP BTP Cockpit or Cloud Connector)	Role
	View destinations (read operations) on service instance level (cockpit)	One of these roles: • Destination Viewer • Destination Configuration Viewer • Destination Viewer Instance • Destination Configuration Instance Viewer *plus* one of these roles: • *Org Manager* • *Space Manager* • *Space Developer* See User and Member Management.
	Manage certificates (all CRUD operations) on service instance level (cockpit) **Note** CRUD (*Create, Read, Update, Delete*) operations include actions like create destinations, display (view) destinations, edit destinations, and delete destinations. For more information, see Managing Destinations [page 1349].	One of these roles: • Destination Administrator • Destination Certificate Administrator • Destination Administrator Instance • Destination Certificate Instance Administrator *plus* one of these roles: • *Org Manager* • *Space Manager* • *Space Developer* See User and Member Management.
	View certificates (read operations) on service instance level (cockpit)	One of these roles: • Destination Viewer • Destination Certificate Viewer • Destination Viewer Instance • Destination Certificate Instance Viewer *plus* one of these roles: • *Org Manager* • *Space Manager* • *Space Developer* See User and Member Management.

Default Role Collections

Default Role Collection	Connectivity Roles Included
Subaccount Administrator	• Cloud Connector Administrator • Destination Administrator
Subaccount Viewer	• Cloud Connector Auditor • Destination Viewer
Cloud Connector Administrator	Cloud Connector Administrator
Destination Administrator	Destination Administrator
Connectivity and Destination Administrator	• Cloud Connector Administrator • Destination Administrator

> ### Note
>
> You can access subaccount-level destinations in two ways:
>
> - Via the cockpit (as described above)
> - Via the Destination service REST API
>
> If a user has access to the Destination service REST API (via service instance binding credentials or a service key), he has full access to the destination and certificate configurations managed by that instance of the Destination service.
>
> For more information, see About Roles in the Cloud Foundry Environment and check the activity *Instantiate and bind services to apps* in the linked Cloud Foundry documentation (*docs.cloudfoundry.org*).
>
> Additionally, applications have access to the REST API of the Destination service instance they are bound to.

1.1.5.3 Monitoring and Troubleshooting

Find information on monitoring and troubleshooting for SAP BTP Connectivity.

Getting Support

If you encounter an issue with this service, we recommend to follow the procedure below:

Check Platform Status

- Check the availability of the service at SAP Trust Center.
- To find information about region updates and SAP BTP notifications, see also Platform Updates and Notifications.
- For more information about selected platform incidents, see Root Cause Analyses.

Contact SAP Support

You can report an incident or error through the SAP Support Portal.

To find the relevant component for your for SAP BTP Connectivity incident, see Connectivity Support [page 2362] (section *SAP Support Information*).

When submitting the incident, we recommend including the following information:

- Region information (for example: Canary, EU10, US10)
- Subaccount technical name
- The URL of the page where the incident or error occurs
- The steps or clicks used to replicate the error
- Screenshots, videos, or the code entered

More Information

Topic	More Information
Monitor the Cloud Connector from the SAP BTP cockpit and from the Cloud Connector administration UI.	Monitoring [page 2006]
Troubleshoot connection problems and view different types of logs and traces in the Cloud Connector.	Monitoring, Logging, and Troubleshooting [page 2046]
Detailed support information for SAP Connectivity service and the Cloud Connector.	Connectivity Support [page 2362]

1.1.6 Developing Applications

Find information on Connectivity concepts, scenarios, and components to develop your SAP BTP application.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Task	Description
Concepts [page 16]	Find information on basic concepts of SAP BTP Connectivity.
Connectivity Scenarios	
Principal Propagation [page 30]	Forward the identity of a cloud user to a remote system. This process is called principal propagation (also known as user propagation or user principal propagation).
Invoking ABAP Function Modules via RFC [page 66]	Call a remote-enabled function module (RFM) in an on-premise or cloud ABAP server from your Cloud Foundry application, using the RFC protocol.
Components	
Consuming the Connectivity Service [page 1314]	Connect your Cloud Foundry application to an on-premise system.
Consuming the Destination Service [page 1550]	Retrieve and store technical information about the destination that is required to consume a target remote service from your application.
Using the Connectivity Proxy [page 2148]	Use the Connectivity Proxy for Kubernetes to connect workloads on a Kubernetes cluster to on-premise systems. The Connectivity Proxy offers multiple proxy endpoints which are communication protocol-specific: TCP (via SOCKS5), HTTP, RFC (invoking ABAP functions), and LDAP.
Using the Transparent Proxy [page 2166]	Use the Transparent Proxy for Kubernetes to connect workloads on a Kubernetes cluster to Internet or on-premise applications. The Transparent Proxy lightens the way your Kubernetes workloads connect to remote systems using BTP destinations of type `Internet` and `OnPremise`.

1.1.7 Connectivity in the Kyma Environment

Find an overview of Connectivity components you can use for different purposes in the Kyma environment.

Connectivity features are available for native consumption by workloads within the Kyma environment. Destinations and related resources are managed centrally and technical connectivity is facilitated with the help of Transparent Proxy and Connectivity Proxy which are both offered as Kyma modules. This approach simplifies lifecycle management operations based on the *Kubernetes Operator* pattern.

> **Note**
>
> The feature set of Transparent Proxy and Connectivity Proxy provided as Kyma modules may differ from their general offering as Kuberenetes-based deployments, which are fully operated by you.

Destination Service

Use the Destination service to centrally manage destinations and related resources, and delegate actions like access token retrieval and X.509 client certificate renewal.

Transparent Proxy

Use the Transparent Proxy to achieve unified, virtually transparent technical connectivity to any remote target application, service or system, hosted in the cloud, on premises or in a VPC (virtual private cloud) environment on hyperscalers.

Connectivity Proxy

Use the Connectivity Proxy to enable secure technical connectivity from cloud workloads to systems hosted on premises/VPC, securely exposed via the Cloud Connector.

Related Information

Getting Started in the Kyma Environment

Add and Delete a Kyma Module

Transparent Proxy in the Kyma Environment [page 2332]

Connectivity Proxy in the Kyma Environment [page 2160]

Managing Destinations [page 1349]

1.2 Connectivity Service

Learn how to manage and consume the Connectivity service in SAP BTP, multi-cloud foundation.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

The Connectivity service offers a set of services and software components which allow SAP BTP applications to securely access remote services that run on premises. It:

- Allows you to make connections to on-premise systems.
- Provides, together with the Cloud Connector, a secure tunnel from your on-premise network to applications on SAP BTP, while you keep full control and auditability of what is exposed to the cloud. This way, no network ports need to be exposed on the on-premise side.
- Supports various communication protocols towards the target system.

Connectivity Service: Administration [page 1309]	Manage the Connectivity service in SAP BTP, multi-cloud foundation.
Consuming the Connectivity Service [page 1314]	Connect your application to an on-premise system.

Related Information

Getting Started [page 12]
Destination Service [page 1347]
Cloud Connector [page 1602]

1.2.1 Connectivity Service: Administration

Manage the Connectivity service in SAP BTP, multi-cloud foundation.

Create and Bind a Connectivity Service Instance [page 1309]	To use the Connectivity service in your application, you need an instance of the service.
Download Authentication Data [page 1313]	Download an authentication data file for subaccount configuration in the Cloud Connector.
Disconnect the Cloud Connector [page 1314]	Learn about options to disconnect your Cloud Connector(s) from a subaccount.

1.2.1.1 Create and Bind a Connectivity Service Instance

To use the Connectivity service in your application, you need an instance of the service.

> **Note**
>
> This section refers to SAP BTP, Cloud Foundry environment. For information on service instances in the Kyma environment, see Using SAP BTP Services in the Kyma Environment.

Prerequisites

When using service plan “lite”, quota management is no longer required for this service. From any subaccount you can consume the service using service instances without restrictions on the instance count.

Previously, access to service plan “lite” has been granted via entitlement and quota management of the application runtime. It has now become an integral service offering of SAP BTP to simplify its usage. See also Entitlements and Quotas.

Procedure

You have two options for creating a service instance – using the CLI or using the SAP BTP cockpit:

- Create and Bind a Service Instance from the CLI [page 1310]
  - Example [page 1310]
  - Result [page 1312]
- Create and Bind a Service Instance from the Cockpit [page 1311]
  - Result [page 1312]

For more information on using X.509 certificates, see:

X.509 Bindings [page 1312]

Create and Bind a Service Instance from the CLI

Use the following CLI commands to create a service instance and bind it to an application:

1. *cf marketplace*
2. *cf create-service connectivity <service-plan> <service-name>*
3. *cf bind-service <app-name> <service-name> [-c <config json>]*

Back to Procedure [page 1310]

Example

To bind an instance of the Connectivity service "lite" plan to application "myapp", use following commands on the Cloud Foundry command line:

cf create-service connectivity lite myinstance

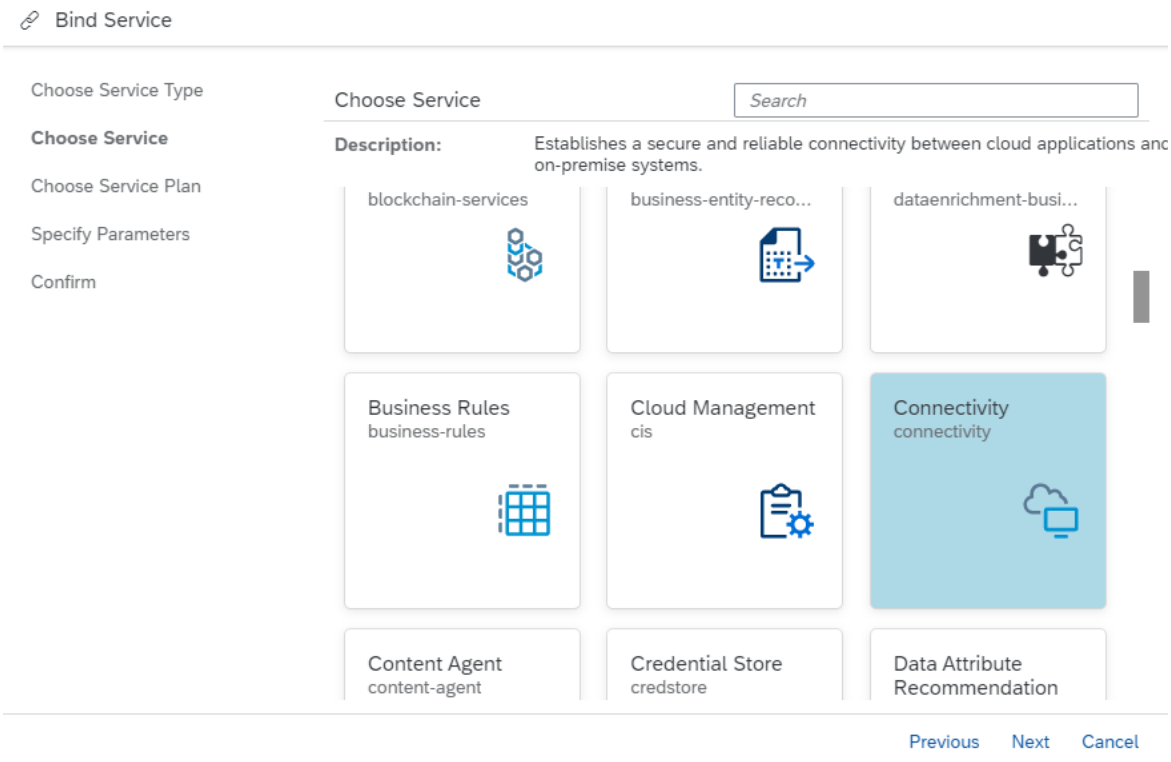
cf bind-service myapp myinstance

Back to Procedure [page 1310]

Create and Bind a Service Instance from the Cockpit

Assuming that you have already deployed your application to the platform, follow these steps to create a service instance and bind it to an application:

1. In the SAP BTP cockpit, navigate to your application.
2. From the left navigation menu, choose *Service Bindings*.
3. Choose the *Bind Service* button.
4. The *Bind Service* wizard appears.
5. Select the *Service from the catalog* radio button, then choose *Next*.
6. From the list of available services, select *Connectivity*, then choose *Next*.



7. On the next page of the wizard, select the *Create new instance* radio button.
8. Leave `<Plan>` `lite` selected and choose *Next*.
9. The next page is used for specifying user-provided parameters in JSON format. If you do not want to do that, skip this step by choosing *Next*.
10. In the `<Instance Name>` textbox, enter an unique name for your service instance.
11. Choose *Finish*.

Back to Procedure [page 1310]

Result

When the binding is created, the application gets the corresponding connectivity credentials in its environment variables:

Sample Code

```
"VCAP_SERVICES": {
 "connectivity": [
 {
 "credentials": {
 "onpremise_proxy_host": "10.0.85.1",
 "onpremise_proxy_port": "20003",
 "onpremise_proxy_http_port": "20003",
 "clientid": "sb-connectivity-app",
 "clientsecret": "KXqObiN6d9gLA4cS2rOVAahPCX0=",
 "token_service_url": "<token_service_url>",
 },
 "label": "connectivity",
 "name": "conn-lite",
 "plan": "default",
 "provider": null,
 "syslog_drain_url": null,
 "tags": [
 "connectivity",
 "conn",
 "connsvc"
 ],
 "volume_mounts": []
 }
 ],
```

Note

`"onpremise_proxy_http_port"` replaces the deprecated variable `"onpremise_proxy_port"`, which will be removed soon. Same goes for `"token_service_url"`, which replaces `"url"`.

Back to Procedure [page 1310]

X.509 Bindings

Note

Older instances of the service do not support the X.509 credentials binding type and an attempt to create one will result in an error. To overcome this, you just need to update the service instance, so it picks up the X.509 support.

The service supports X.509 bindings as described in Retrieving Access Tokens with Mutual Transport Layer Security (mTLS). This lets you choose if your binding / service key will be with a client secret X.509 certificate generated by SAP or an X.509 certificate provided by you. To do so, add a config JSON when creating your binding:

```
cf bind-service <app-name> <service-name> [-c <config json>]
```

The structure of the config JSON should be like this:

```
{
    "xsuaa":
    {
        <X.509 properties>
    },
    <other config parameters>
}
```

For a list of the supported X.509 properties, see:

- Parameters for X.509 Certificates Managed by SAP Authorization and Trust Management Service
- Parameters for Self-Managed X.509 Certificates

> **Note**
>
> By default, without providing a config JSON, a binding with client secret will be created.

Back to Procedure [page 1310]

1.2.1.2 Download Authentication Data

Download an authentication data file for subaccount configuration in the Cloud Connector.

You can download an authentication data file from your subaccount in the SAP BTP cockpit to simplify and accelerate subaccount configuration in the Cloud Connector.

To do so, perform the follwing steps:

1. Log on to the SAP BTP cockpit and choose the subaccount you'd like to connect to.
2. Go to section *Connectivity* > *Cloud Connectors* and press *Download Authentication Data*.
3. Choose the file containing the authentication data and choose *Next*.
4. Upload the authentication data file in the Cloud Connector.

For more information, see Initial Configuration [page 1670] (Cloud Connector), step 4.

Authentication metadata can also be obtained from the Connectivity service REST API.

For more information, see Connectivity Service REST API [page 1344].

1.2.1.3 Disconnect the Cloud Connector

Learn about options to disconnect your Cloud Connector(s) from a subaccount.

To disconnect a Cloud Connector from your SAP BTP subaccount, you have two different options:

- You can forcible disconnect it from the SAP BTP cockpit. To do so,
  1. Log on to the cockpit and choose the subaccount you'd like to connect to.
  2. Go to section *Connectivity* > *Cloud Connectors* and press *Force Disconnect* for the Cloud Connector you want to disconnect.
- Disconnect it in the Cloud Connector administration UI. To do so,
  1. Log on to the Cloud Connector administration UI and choose *Connector* from the main menu.
  2. In section **Subaccount Dashboard**, choose the *Disconnect* icon (column Actions) for the subaccount you want to disconnect.

Related Information

Connect and Disconnect a Cloud Subaccount [page 2003]
Monitoring [page 2006]

1.2.2 Consuming the Connectivity Service

Connect your cloud application to an on-premise system via HTTP.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

> **Note**
>
> To use the Connectivity service with a protocol other than HTTP, see
>
> - Invoking ABAP Function Modules via RFC [page 66]
> - Using the TCP Protocol for Cloud Applications [page 1335]

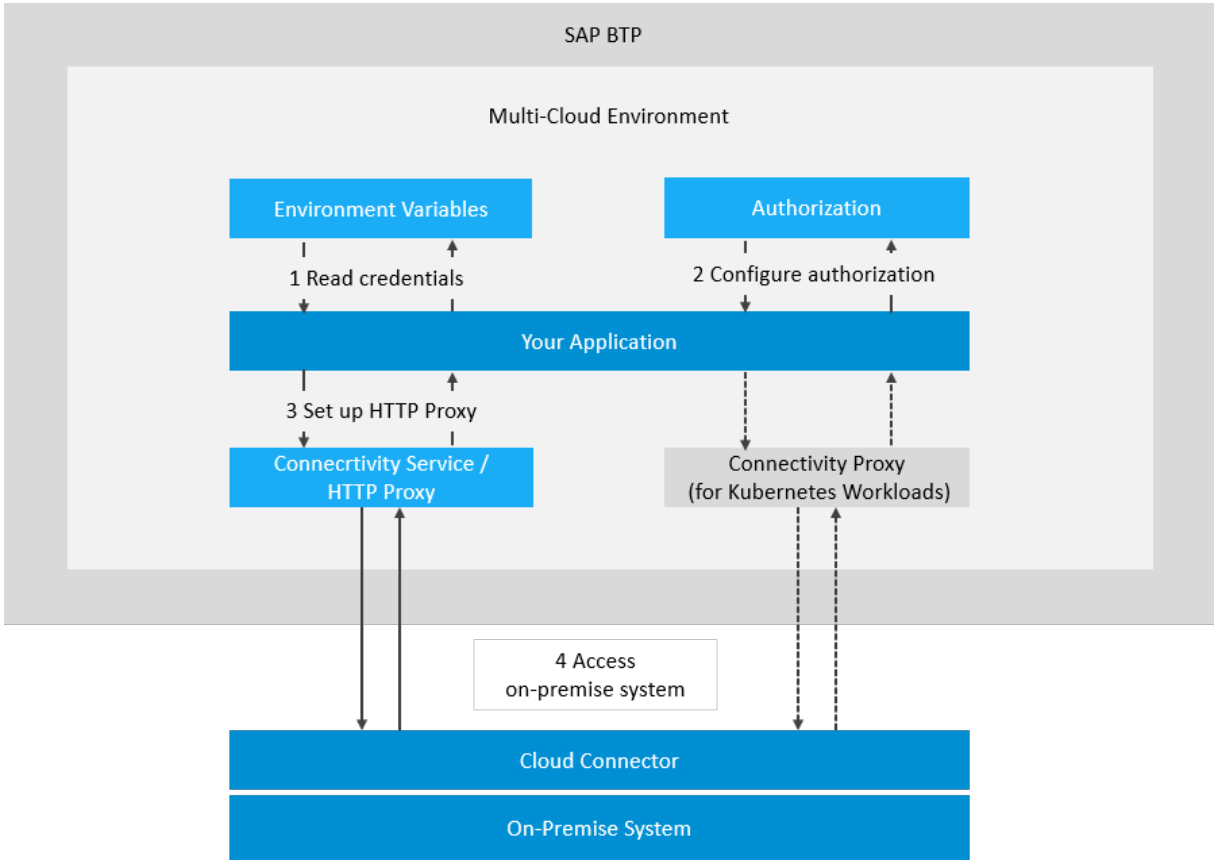
Tasks

Task Type	Task
Operator and/or Developer	Overview [page 1315] Prerequisites [page 1316]
Developer	Basic Steps [page 1317] 1. Read Credentials from the Environment Variables [page 1317] 2. Configure Authorization [page 1318] 3. Set up the HTTP Proxy for On-Premise Connectivity [page 1320]
Operator and/or Developer	Additional Steps [page 1321] • Authentication against the On-Premise System [page 1321] • Specify a Cloud Connector Location ID [page 1321] • Multitenancy in the Connectivity Service [page 1322]

Overview

Using the Connectivity service, you can connect your Cloud Foundry application to an on-premise system through the Cloud Connector. To achieve this, you must provide the required information about the target system (destination), and set up an HTTP proxy that lets your application access the on-premise system.

> ⓘ Note
>
> For more information on connecting Kubernetes workloads to on-premise systems, see Connectivity Proxy for Kubernetes [page 2088].



Back to Tasks [page 1314]

Prerequisites

- You have installed and configured a Cloud Connector in your on-premise landscape for the scenario you want to use. See Installation [page 1608] and Configuration [page 1669].
- You have deployed an application in a landscape of the Cloud Foundry environment that complies with the Business Application Pattern.
- The Connectivity service is a regular service in the Cloud Foundry environment. Therefore, to consume the Connectivity service from an application, you must create a service instance and bind it to the application. See Create and Bind a Connectivity Service Instance [page 1309].

> **Note**
>
> When connecting to HTTP(S) on-premise resources, the call from the cloud application must always use HTTP. If HTTPS is used, a 405 response will be returned. The tunnel used by the Cloud Connector is

TLS-encrypted, hence the traffic between cloud and on-premise network is protected regardless of the used protocol.

> ⚠ Caution
>
> There is a limit of **8192** bytes for the size of the HTTP lines (for example, request line or header) that you send via the Connectivity service. If this limit is exceeded, you receive an HTTP error of type 4xx. This issue is usually caused by the size of the *path + query* string of the request.

Back to Tasks [page 1314]

Basic Steps

To consume the Connectivity service from your Cloud Foundry application, perform the following basic steps:

1. Read Credentials from the Environment Variables [page 1317]
2. Configure Authorization [page 1318]
3. Set up the HTTP Proxy for On-Premise Connectivity [page 1320]

Back to Tasks [page 1314]

Read Credentials from the Environment Variables

Consuming the Connectivity service requires credentials from a *connectivity* or an *identity* service instance (see Authorization [page 1318]) which are bound to the application. By binding the application to service instances as described in the prerequisites, these credentials become part of the environment variables of the application. You can access them like this:

> {·} Sample Code

```
JSONObject jsonObj = new JSONObject(System.getenv("VCAP_SERVICES"));
JSONArray jsonArr = jsonObj.getJSONArray("<service name, not the instance
name>");
JSONObject credentials =
jsonArr.getJSONObject(0).getJSONObject("credentials");
```

> ### Note
>
> If you have multiple instances of the same service bound to the application, you must perform additional filtering to extract the correct credential from `jsonArr`. You must go through the elements of `jsonArr` and find the one matching the correct instance name.

This code stores a JSON object in the credentials variable. Additional parsing is required to extract the value for a specific key.

Back to Tasks [page 1314]

Configure Authorization

To make calls to on-premise systems configured in the Cloud Connector, the cloud application must be authorized at the proxy endpoint(s). The Connectivity service supports authorization tokens issued by Authorization and Trust Management service (XSUAA) and Identity Authentication service (IAS).

Authorization and Trust Management Service (XSUAA)

XSUAA-based authentication is enabled for all service instances out of the box without preparation steps. The needed credentials for obtaining the token are within the service key / binding of the Connectivity service instance that you want to use.

For more information, see Create and Bind a Connectivity Service Instance [page 1309].

Example: Obtaining an XSAA proxy authorization token

Request:

```
POST <token_service_url for the connectivity service binding>/oauth/token
Accept: application/json
Content-Type: application/x-www-form-urlencoded

client_id=<clientid>
client_secret=<clientsecret>
grant_type=client_credentials
```

Response:

```
{
    "access_token" : "<JWT>",
    "token_type" : "Bearer",
    "expires_in" : 3600
}
```

> ### Note
>
> "token_service_url" replaces the deprecated property "url", which will be removed soon.

Identity Authentication Service (IAS)

To enable your application for consuming the Connectivity service via IAS, you need to perform the following steps:

1. Establish trust between your subaccount and your tenant of the Cloud Identity Services service using OpenID Connect.
   For more information, see Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services.
2. Create or update an existing Connectivity service instance and bind it to the application.
   For more information, see Create and Bind a Connectivity Service Instance [page 1309].
3. (Optional) Verify that the Connectivity binding JSON contains the property "token-type" with value ["xsuaa", "ias"].
4. Create or update an existing Cloud Identity Services instance and bind it to the application. During the creation/update you must set the "consumed-services" property to an array that contains the name of the Connectivity service instance.
   For more information, see Getting Started with the Identity Service of SAP Business AI Platform and Reference Information for the Identity Service of SAP Business AI Platform.

For obtaining an IAS proxy authorization token you should use the Cloud Identity Services binding information. The token request must contain an additional parameter - "resource" with value "urn:sap:identity:application:service:id:<instance_id>", where <instance_id> is the value of the root-level property "instance_guid" in the Connectivity binding JSON .

Example: Obtaining an IAS proxy authorization token

Request:

```
POST <ias url>/oauth2/token
Accept: application/json
Content-Type: application/x-www-form-urlencoded

client_id=<clientid>
client_secret=<clientsecret>
grant_type=client_credentials
resource=urn:sap:identity:application:service:id:<instance_id>
```

Response:

```
{
    "access_token" : "<JWT>",
    "token_type" : "Bearer",
    "expires_in" : 3600
}
```

Back to Tasks [page 1314]

Set up the HTTP Proxy for On-Premise Connectivity

Required Information

To consume the connectivity service, you must be aware of some information about your on-premise system and the system mappings for it in the Cloud Connector. You need:

- The endpoint in the Cloud Connector (virtual host and virtual port) and accessible URL paths on it (destinations). For more information, see Configure Access Control (HTTP) [page 1744].
- The required authentication type for the on-premise system. For more information, see HTTP Destinations [page 1350].
- Depending on the authentication type, you might need a username and password for accessing the on-premise system. For more details, see HTTP Destinations [page 1350].
- Optionally, you can use a `LocationId`. For more details on this, see Specify a Cloud Connector Location ID [page 1321].

We recommend that you use the Destination service (see Consuming the Destination Service [page 1550]) to procure this information. However, this is optional and you can also provide it in a different way.

Proxy Setup

The Connectivity service provides a standard HTTP proxy for on-premise connectivity that is accessible by any application. Proxy host and port are available as the environment variables `<onpremise_proxy_host>` and `<onpremise_proxy_http_port>`. You can set up the on-premise HTTP proxy like this:

Sample Code

```
// get value of "onpremise_proxy_host" and "onpremise_proxy_http_port" from
the environment variables
// and create on-premise HTTP proxy
String connProxyHost =
connectivityCredentials.getString("onpremise_proxy_host");
int connProxyPort =
Integer.parseInt(credentials.getString("onpremise_proxy_http_port"));
Proxy proxy = new Proxy(Proxy.Type.HTTP, new InetSocketAddress(connProxyHost,
connProxyPort));


// create URL to the remote endpoint you like to call:
// virtualhost:1234 is defined as an endpoint in the Cloud Connector, as
described in the Required Information section
URL url = new URL("http://virtualhost:1234");


// create the connection object to the endpoint using the proxy
// this does not open a connection but only creates a connection object,
which can be later modified before actually connecting
urlConnection = (HttpURLConnection) url.openConnection(proxy);
urlConnection.setRequestProperty("Proxy-Authorization",  "Bearer " + "<token
for proxy authorization, as described in the Authorization section>");
```

> ### Note
>
> `"onpremise_proxy_http_port"` replaces the deprecated variable `"onpremise_proxy_port"`, which will be removed soon.

Back to Tasks [page 1314]

Authentication against the On-Premise System

Depending on the required authentication type for the desired on-premise resource, you may have to set an additional header in your request. This header provides the required information for the authentication process against the on-premise resource. See Authentication to the On-Premise System [page 1322].

Back to Tasks [page 1314]

Specify a Cloud Connector Location ID

> ### Note
>
> This is an advanced option when using more than one Cloud Connector for a subaccount. For more information how to set the `location ID` in the Cloud Connector, see Adding and Managing Subaccounts [page 1682], step 4 in section *Subaccount Dashboard*.

You can connect multiple Cloud Connectors to a subaccount if their `location ID` is different. Using the header `SAP-Connectivity-SCC-Location_ID` you can specify the Cloud Connector over which the connection should be opened. If this header is not specified, the connection is opened to the Cloud Connector that is connected without any `location ID`. For example:

> ### Sample Code
>
```
// Optionally, if configured, add the SCC location ID.
urlConnection.setRequestProperty("SAP-Connectivity-SCC-Location_ID",
"orlando");
```

Back to Tasks [page 1314]

Multitenancy in the Connectivity Service

To consume the Connectivity service from an SaaS application in a multitenant way, the only requirement is that the SaaS application returns the Connectivity service as a dependent service in its dependencies list.

For more information about the subscription flow, see Develop the Multitenant Application.

Back to Tasks [page 1314]

Related Information

Cloud Connector [page 1602]
Set Up an Application as a Sample Backend System
Create and Bind a Connectivity Service Instance [page 1309]
Authentication to the On-Premise System [page 1322]
Consuming the Destination Service [page 1550]
What Is the SAP Authorization and Trust Management Service?
Multitarget Applications in the Cloud Foundry Environment
Invoking ABAP Function Modules via RFC [page 66]
Using the TCP Protocol for Cloud Applications [page 1335]
Connectivity Service REST API [page 1344]

1.2.2.1 Authentication to the On-Premise System

Provide authentication information for the authentication type you use.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

You can use the Connectivity service in different authentication scenarios:

- **No authentication** to the on-premise system
- **Principal propagation** (user propagation) to the on-premise system
- **Basic authentication** to the on-premise system

Procedure

For each authentication type, you must provide specific information in the request to the virtual host:

- The SAP-Connectivity-Authentication Header [page 1323]
- Authentication Types [page 1323]

The SAP-Connectivity-Authentication Header

> **Note**
>
> For the principal propagation scenario, the `SAP-Connectivity-Authentication` header is only required if you do not use the user exchange token flow, see Configure Principal Propagation via User Exchange Token [page 1327].

Applications must propagate the user JWT token (`userToken`) using the `SAP-Connectivity-Authentication` header. This is required for the Connectivity service to open a tunnel to the subaccount for which a configuration is made in the Cloud Connector. The following example shows you how to do this using the *Spring* framework:

> **Sample Code**

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
if (auth == null) {
        throw new UserInfoException("User not authenticated");
}
OAuth2AuthenticationDetails details = (OAuth2AuthenticationDetails)
auth.getDetails();
String userToken = details.getTokenValue();
urlConnection.setRequestProperty("SAP-Connectivity-Authentication", "Bearer "
+ userToken);
```

Back to Procedure [page 1323]

Authentication Types

The required setup for each of the authentication type is as follows:

No Authentication

If the on-premise system does not need to identify the user, you should use this authentication type. It requires no additional information to be passed with the request.

Principal Propagation

When you open the application router to access your cloud application, you are prompted to log in. Doing so means that the cloud application now knows your identity. Principal propagation forwards this identity via the Cloud Connector to the on-premise system. This information is then used to grant access without additional input from the user. To achieve this, you do not need to send any additional information from your application, but you must set up the Cloud Connector for principal propagation. See Configuring Principal Propagation [page 1702].

> ⓘ Note
>
> Do not use the `Authorization` header in *principal propagation* scenarios.

Technical User Propagation

As of Cloud Connector version 2.15, consumers of the Connectivity service can propagate technical users from the cloud application towards the on-premise systems. To make use of this feature, provide a JWT (usually obtained via `client_credentials` OAuth flow), representing the technical user, via the `SAP-Connectivity-Technical-Authentication` header. This is similar to *principal propagation*, but in this case, a technical user is propagated instead of a business user.

```
urlConnection.setRequestProperty("SAP-Connectivity-Technical-Authentication",
"Bearer " + technicalUserToken);
```

For more information, see Configuring Principal Propagation [page 1702].

> ⓘ Note
>
> Do not use the `Authorization` header in *technical user propagation* scenarios.

Basic Authentication

If the on-premise system requires username and password to grant access, the cloud application must provide these data using the *Authorization* header. The following example shows how to do this:

> {·} Sample Code

```
// Basic authentication to backend system
String credentials = MessageFormat.format("{0}:{1}", backendUser,
backendPassword);
byte[] encodedBytes = Base64.encodeBase64(credentials.getBytes());
urlConnection.setRequestProperty("Authorization", "Basic " + new
String(encodedBytes));
```

Back to Procedure [page 1323]

Related Information

Configure Principal Propagation via Corporate IdP-Embedded Token [page 1325]
Configure Principal Propagation via User Exchange Token [page 1327]
Configure Principal Propagation via IAS Token [page 1332]
Configure Principal Propagation via OIDC Token [page 1333]

1.2.2.1.1 Configure Principal Propagation via Corporate IdP-Embedded Token

Configure a corporate IdP-embedded token for principal propagation (user propagation) from your application to an on-premise system.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

> **Restriction**
>
> Principal Propagation via corporate IdP-embedded token is not supported with IAS proxy authorization. In this case, the corporate IdP token should be sent directly with the "SAP-Connectivity-Authentication" header.
>
> For more information, see Authentication to the On-Premise System [page 1322].
>
> The IAS login token can be exchanged for a corporate IdP token via the automation functionalities of the Destination service.
>
> For more information, see Using an IAS User Token for Corporate IdP Token Principal Propagation to On-Premise Systems [page 1599].

Tasks

Task Type	Task
Operator and/or Developer	Scenario [page 1326]
Operator	Prerequisites [page 1326]

Task Type	Task
Developer	Solutions [page 1327]

Scenario

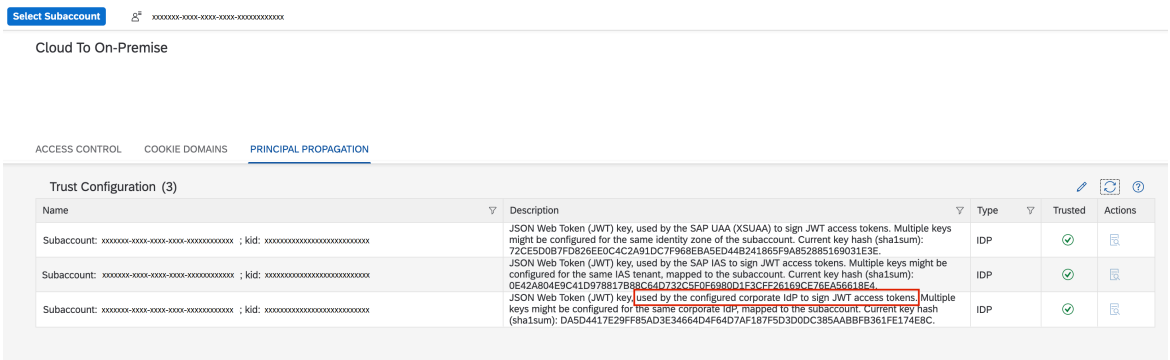
For a Cloud Foundry application that uses the Connectivity service, you want the currently logged-in user to be propagated to an on-premise system via token from your trusted corporate IdP (identity provider).

For more information, see Principal Propagation [page 30].

Back to Tasks [page 1325]

Prerequisites

- Cloud Connector 2.13 (or newer) must be used.
- The Cloud Connector must be connected to a subaccount that is configured with the corporate IdP, issuing the tokens.
  After the configuration, the issued XSUAA token contains an embedded token, which is extracted and propagated by the Connectivity service.
  For more information, see Include Tokens from Corporate Identity Providers or Identity Authentication in Tokens of the SAP Authorization and Trust Management Service.
- After the configuration, the issued XSUAA token contains an embedded token, which is extracted and propagated by the Connectivity service.
- Trust configuration for that subaccount must be synchronized in the Cloud Connector to obtain the JSON web key set from the configured corporate IdP that is used to verify the token.
  For more information, see Set Up Trust [page 1702].



Back to Tasks [page 1325]

Solutions

You have two options to implement the user propagation via embedded corporate IdP token:

1. **Recommended**. The application sends one header containing the user exchange token to the Connectivity proxy:
   - The application sends one HTTP header `Proxy-Authorization`.
   - The original *user token* is exchanged for a special *user exchange access token* following the OAuth2 JWT Bearer grant type.
     For more information, see Using JWTs as Authorization Grants .
   - The *user exchange access token* is send through the `Proxy-Authorization` header to consume the Connectivity service.
   - The embedded corporate token which contains the user details must be present in the obtained *user exchange access token*.

   ```
   Header: "Proxy-Authorization" : "Bearer <userExchangeAccessToken>"
   ```

   For more information on how to obtain a user exchange access token with an embedded IAS corporate IdP token, see SAP Authorization and Trust Management Service in the Cloud Foundry Environment and Generate the Authentication Token [page 1329].
2. The application sends two headers to the Connectivity proxy:
   - The application sends two HTTP headers: `SAP-Connectivity-Authentication` and `Proxy-Authorization`.
   - The embedded corporate token which contains the user details must be present in the user token provided via `SAP-Connectivity-Authentication`.
   - The access token is provided via `Proxy-Authorization`.

   ```
   Header: "SAP-Connectivity-Authentication" : "Bearer <userToken>"
   Header: "Proxy-Authorization" : "Bearer <accessToken>"
   ```

The Cloud Connector validates the token, extracts the available user data, and enables further processing through a configured subject pattern for the resulting short-lived X.509 client certificate.

Back to Tasks [page 1325]

1.2.2.1.2 Configure Principal Propagation via User Exchange Token

Configure a user exchange token for principal propagation (user propagation) from your application to an on-premise system.

> ### Note
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

> ### Restriction
>
> Principal Propagation via user exchange token is not supported with IAS proxy authorization. In this case, the principal should be propagated through the "SAP-Connectivity-Authentication" header.
>
> For more information, see Authentication to the On-Premise System [page 1322].

Tasks

Task Type	Task
Operator and/or Developer	Scenario [page 1328]
Developer	Solutions [page 1329]
	Generate the Authentication Token [page 1329]

Scenario

For a Cloud Foundry application that uses the Connectivity service, you want the currently logged-in user to be propagated to an on-premise system. For more information, see Principal Propagation [page 30].

> ### Note
>
> When performing a token exchange, the resulting token may lose some of its fields.

Back to Tasks [page 1328]

Solutions

You have two options to implement user propagation:

1. **Recommended**: The application sends one header containing the user exchange token to the Connectivity proxy:

   ```
   Header: "Proxy-Authorization" : "Bearer <userExchangeAcessToken>"
   ```

   This option is described in detail in Generate the Authentication Token [page 1329] below.
2. The application sends two headers to the Connectivity proxy:

   ```
   Header: "SAP-Connectivity-Authentication" : "Bearer <userToken>"
   Header: "Proxy-Authorization" : "Bearer <accessToken>"
   ```

   For more information about this solution, see also Consuming the Connectivity Service [page 1314].

   > ### Note
   >
   > This solution is supported to guarantee backward compatibility.

Back to Tasks [page 1328]

Generate the Authentication Token

To propagate a user to an on-premise system, you must call the Connectivity proxy using a special JWT (JSON Web token). This token is obtained by exchanging a valid user token following the OAuth2 JWT Bearer grant type .

> ### Note
>
> When using this type of exchange, some of the attributes inside the original user token may not be present in the created JWT.

- Example: Obtaining a User Token Following the JWT Bearer Grant Type [page 1329]
- Example: Calling the Connectivity Proxy with the Exchanged User Token [page 1331]

Example: Obtaining a User Token Following the JWT Bearer Grant Type

Request:

Sample Code

```
POST https://<host: the value of 'url' from Connectivity service credentials
in VCAP_SERVICES>/oauth/token
Accept: application/json
Content-Type: application/x-www-form-urlencoded
client_id=<connectivity_service_client_id>
client_secret=<connectivity_service_client_secret>
grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer
token_format=jwt
response_type=token
assertion=<logged-in-user-JWT>
```

Response:

Sample Code

```
{
    "access_token" : "<new-user-JWT>",
    "token_type" : "bearer",
    "expires_in" : 43199,
    "scope" : "<token-scopes>",
    "jti" : "7cc917b8bf6347a2aa18d7ac8f38a1c2"
}
```

The JWT in `access_token`, also referred to as *user exchange token*, now contains the user details. It is used to consume the Connectivity service.

In case of a Java application, you can use a library that implements the user exchange OAuth flow. Here is an example of how the `userExchangeAcessToken` can be obtained using the XSUAA Token Client and Token Flow API :

Caution

Make sure you get the latest API version.

A sample Maven dependency declaration:

```
<dependency>
    <groupId>com.sap.cloud.security.xsuaa</groupId>
    <artifactId>token-client</artifactId>
    <version><latest version (e.g.: 2.7.7)></version>
</dependency>
```

→ Remember

The XSUAA Token Client library works with multiple HTTP client libraries. Make sure you have one as Maven dependency.

The following sample uses the Apache REST client:

Sample Code

```
// service instance specific OAuth client credentials shall be used
String connectivityServiceClientId = credentials.getString(CLIENT_ID);
String connectivityServiceClientSecret = credentials.getString(CLIENT_SECRET);
// get the URL to xsuaa from the environment variables
URI xsuaaUri = new URI(xsuaaCredentials.getString("token_service_url"));

// use the XSUAA client library to ease the implementation of the user token
exchange flow
XsuaaTokenFlows tokenFlows = new XsuaaTokenFlows(new
DefaultOAuth2TokenService(), new XsuaaDefaultEndpoints(xsUaaUri.toString()),
new ClientCredentials(connectivityServiceClientId,
connectivityServiceClientSecret));
String userExchangeAcessToken =
tokenFlows.userTokenFlow().token(<jwtToken_to_exchange>).execute().getAccessTo
ken();
```

For more information about caching, see also XSUAA Token Client and Token Flow API - Cache .

Note

`xsuaaCredentials.getString("token_service_url")` replaces the deprecated property `xsuaaCredentials.getString("url")`, which will be removed soon.

See also: XSUAA Token Client and Token Flow API .

After obtaining the `userExchangeAcessToken`, you can use it to consume the Connectivity service.

Back to Generate the Authentication Token [page 1329]

Example: Calling the Connectivity Proxy with the Exchanged User Token

As a prerequisite to this step, you must configure the Connectivity proxy to be used by your client, see Set up the HTTP Proxy for On-Premise Connectivity [page 1320].

Once the application has retrieved the user exchange token, it must pass the token to the Connectivity proxy via the `Proxy-Authorization` header. In this example, we use `urlConnection` as a client.

Sample Code

```
// set user exchange token as Proxy-Authorization header in the URL connection
urlConnection.setRequestProperty("Proxy-Authorization", "Bearer " +
userExchangeAcessToken);
```

Note

Alternatively, you can use the *basic authentication* scheme:

Sample Code

```
// Basic authentication to Connectivity Proxy
String credentials = MessageFormat.format("{0}:{1}", userExchangeAcessToken,
"");
byte[] encodedBytes = Base64.encodeBase64(credentials.getBytes());
urlConnection.setRequestProperty("Proxy-Authorization", "Basic " + new
String(encodedBytes));
```

Back to Generate the Authentication Token [page 1329]

Back to Tasks [page 1328]

1.2.2.1.3 Configure Principal Propagation via IAS Token

Configure an Identity Authentication service (IAS) token for principal propagation (user propagation) from your application to an on-premise system.

Note

For more information on identity authentication, see SAP Cloud Identity Services.

Note

The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Scenario

For a Cloud Foundry application that uses the Connectivity service, you want the currently logged-in user to be propagated via the Cloud Connector to an on-premise system. This user is represented in the cloud application by an IAS token.

For more information, see Principal Propagation [page 30] and Getting Started with the Identity Service of SAP BTP.

Prerequisites

- Cloud Connector 2.13 (or newer) must be used.
- The Cloud Connector must be connected to a subaccount that is configured with the IAS tenant issuing the tokens.

For more information, see Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services.

Solution

The application sends two headers to the Connectivity Proxy:

Header: `"SAP-Connectivity-Authentication" : "Bearer <IAS-Token>"`

Header: `"Proxy-Authorization" : "Bearer <accessToken>"`

Additional Resources

For information on obtaining the IAS token, see Configure Authorization [page 1318].

More Information

Identity Authentication

1.2.2.1.4 Configure Principal Propagation via OIDC Token

Configure an OpenID-Connect (OIDC) token for principal propagation (user propagation) from your application to an on-premise system.

> **Note**
>
> For more information on OIDC, see OpenID Specifications .

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Tasks

Task Type	Task
Operator and/or Developer	Scenario [page 1334]
Developer	Solution [page 1334]

Scenario

For a Cloud Foundry application that uses the Connectivity service, you want the currently logged-in user to be propagated via the Cloud Connector to an on-premise system. This user is represented in the cloud application by an OIDC token. For more information, see Principal Propagation [page 30] and the OIDC specification .

Back to Tasks [page 1334]

Solution

The Cloud Connector supports principal propagation for OIDC tokens. If on the cloud application side the user is represented by an OIDC token, the application can send the user principal to the Connectivity service (thus reaching the Cloud Connector), using the `SAP-Connectivity-Authentication` HTTP header.

The Cloud Connector validates the token, extracts the available user data, and enables further processing through a configured subject pattern for the resulting short-lived X.509 client certificate.

By default, the user principal is identified by one of the following JWT (JSON web token) attributes:

- `user_name`
- `email`

- `mail`
- `user_uuid`
- `sub`

This list specifies the priority (in descending order from top to bottom) for the default value of `${name}` in the subject pattern of the X.509 client certificate. If a token has more than one of the above claims, the value of `${name}` is extracted from the claim with the highest priority by default.

For the example token below, the default value of `${name}` is `test@user.com`:

Sample Code

```
{
  "aud": "111111111111-2222-3333-444444444444",
  "iss": "https://issuer.com",
  "exp": 2091269073,
  "iat": 1601901108,
  "jti": "11122233344455566677788899900 0",
  "sub": "test",
  "email": "test@users.com"
}
```

The Cloud Connector administrator can control the exact value to be used as user principal for the *subject CN* of the X.509 client certificate by configuring a subject pattern. For more information, see Configure Subject Patterns [page 1726].

Back to Tasks [page 1334]

1.2.2.2 Using the TCP Protocol for Cloud Applications

Access on-premise systems from an application via TCP-based protocols, using a SOCKS5 Proxy.

Note

The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Content

Concept [page 1336]

Restrictions [page 1337]

Example [page 1337]

Troubleshooting [page 1343]

Concept

SAP BTP Connectivity provides a SOCKS5 proxy that you can use to access on-premise systems via TCP-based protocols. SOCKS5 is the industry standard for proxying TCP-based traffic (for more information, see IETF RFC 1928).

The SOCKS5 proxy host and port are accessible through the environment variables, which are generated after binding an application to a Connectivity service instance. For more information, see Consuming the Connectivity Service [page 1314].

You can access the host under `onpremise_proxy_host`, and the port through `onpremise_socks5_proxy_port`, obtained from the Connectivity service instance.

Authentication to the SOCKS5 proxy is mandatory. It involves the usage of a JWT (JSON Web token) access token (for more information, see IETF RFC 7519). The JWT can be retrieved through the `client_id` and `client_secret`, obtained from the Connectivity service instance. For more information, see Set up the HTTP Proxy for On-Premise Connectivity [page 1320], section **Authorization**.

The value of the SOCKS5 protocol authentication method is defined as **0x80** (defined as *X'80'* in IETF, refer to the official specification SOCKS Protocol Version 5). This value should be sent as part of the authentication method's negotiation request (known as *Initial Request* in SOCKS5). The server then confirms with a response containing its decimal representation (either **128** or **-128**, depending on the client implementation).

After a successful SOCKS5 *Initial Request*, the authentication procedure follows the standard SOCKS5 authentication sub-procedure, that is SOCKS5 *Authentication Request*. The request bytes, in sequence, should look like this:

Bytes	Description
1 byte	Authentication method version - currently `1`
4 bytes	Length of the JWT
X bytes	X - The actual value of the JWT in its encoded form
1 byte	Length of the Cloud Connector location ID (`0` if no Cloud Connector location ID is used)
Y bytes	Optional. Y - The value of the Cloud Connector location ID in base64-encoded form (if the the value of the location ID is not `0`)

The Cloud Connector location ID identifies Cloud Connector instances that are deployed in various locations of a customer's premises and connected to the same subaccount. Since the location ID is an optional property, you should include it in the request only if it has already been configured in the Cloud Connector. For more information, see Set up Connection Parameters and HTTPS Proxy [page 1674] (Step 4).

If not set in the Cloud Connector, the byte representing the length of the location ID in the *Authentication Request* should have the value `0`, without including any value for the Cloud Connector location ID (`sccLocationId`).

Back to Content [page 1335]

Restrictions

- You cannot use the provided SOCKS5 proxy as general-purpose proxy.
- Proxying UDP traffic is not supported.

Back to Content [page 1335]

Example

The following code snippet demonstrates an example based on the `Apache Http Client` library and Java code, which represents a way to replace the standard socket used in the Apache HTTP client with one that is responsible for authenticating with the Connectivity SOCKS5 proxy:

Sample Code

```
@Override
public void connect(SocketAddress endpoint, int timeout) throws IOException {
    super.connect(getProxyAddress(), timeout);

    OutputStream outputStream = getOutputStream();

    executeSOCKS5InitialRequest(outputStream); // 1. Negotiate authentication
method, i.e. 0x80 (128)

    executeSOCKS5AuthenticationRequest(outputStream); // 2. Negotiate
authentication sub-version and send the JWT (and optionally the Cloud
Connector Location ID)

    executeSOCKS5ConnectRequest(outputStream, (InetSocketAddress)
endpoint); // 3. Initiate connection to target on-premise backend system
}
```

Sample Code

```
private byte[] createInitialSOCKS5Request() throws IOException {
    ByteArrayOutputStream byteArraysStream = new ByteArrayOutputStream();
    try {
        byteArraysStream.write(SOCKS5_VERSION);
        byteArraysStream.write(SOCKS5_AUTHENTICATION_METHODS_COUNT);
        byteArraysStream.write(SOCKS5_JWT_AUTHENTICATION_METHOD);
        return byteArraysStream.toByteArray();
    } finally {
        byteArraysStream.close();
    }
}


private void executeSOCKS5InitialRequest(OutputStream outputStream) throws
IOException {
    byte[] initialRequest = createInitialSOCKS5Request();
    outputStream.write(initialRequest);

    assertServerInitialResponse();
}
```

Sample Code

```
private byte[] createJWTAuthenticationRequest() throws IOException {
    ByteArrayOutputStream byteArraysStream = new ByteArrayOutputStream();
    try {
        byteArraysStream.write(SOCKS5_JWT_AUTHENTICATION_METHOD_VERSION);

byteArraysStream.write(ByteBuffer.allocate(4).putInt(jwtToken.getBytes().lengt
h).array());
        byteArraysStream.write(jwtToken.getBytes());
        byteArraysStream.write(ByteBuffer.allocate(1).put((byte)
sccLocationId.getBytes().length).array());
        byteArraysStream.write(sccLocationId.getBytes());
        return byteArraysStream.toByteArray();
    } finally {
        byteArraysStream.close();
    }
}

private void executeSOCKS5AuthenticationRequest(OutputStream outputStream)
throws IOException {
    byte[] authenticationRequest = createJWTAuthenticationRequest();
    outputStream.write(authenticationRequest);

    assertAuthenticationResponse();
}
```

In version 4.2.6 of the Apache HTTP client, the class responsible for connecting the socket is `DefaultClientConnectionOperator`. By extending the class and replacing the standard socket with the complete example code below, which implements a Java Socket, you can handle the SOCKS5 authentication with ID **0x80**. It is based on a JWT and supports the Cloud Connector location ID.

Sample Code

```
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.InetAddress;
import java.net.InetSocketAddress;
import java.net.Socket;
import java.net.SocketAddress;
import java.net.SocketException;
import java.nio.ByteBuffer;

import java.util.Base64; // or any other library for base64 encoding
import org.json.JSONArray; // or any other library for JSON objects
import org.json.JSONObject; // or any other library for JSON objects
import org.json.JSONException; // or any other library for JSON objects

public class ConnectivitySocks5ProxySocket extends Socket {

    private static final byte SOCKS5_VERSION = 0x05;
    private static final byte SOCKS5_JWT_AUTHENTICATION_METHOD = (byte) 0x80;
    private static final byte SOCKS5_JWT_AUTHENTICATION_METHOD_VERSION = 0x01;
    private static final byte SOCKS5_COMMAND_CONNECT_BYTE = 0x01;
    private static final byte SOCKS5_COMMAND_REQUEST_RESERVED_BYTE = 0x00;
    private static final byte SOCKS5_COMMAND_ADDRESS_TYPE_IPv4_BYTE = 0x01;
    private static final byte SOCKS5_COMMAND_ADDRESS_TYPE_DOMAIN_BYTE = 0x03;
    private static final byte SOCKS5_AUTHENTICATION_METHODS_COUNT = 0x01;
    private static final int SOCKS5_JWT_AUTHENTICATION_METHOD_UNSIGNED_VALUE
= 0x80 & 0xFF;
```

```
    private static final byte SOCKS5_AUTHENTICATION_SUCCESS_BYTE = 0x00;

    private static final String SOCKS5_PROXY_HOST_PROPERTY =
"onpremise_proxy_host";
    private static final String SOCKS5_PROXY_PORT_PROPERTY =
"onpremise_socks5_proxy_port";

    private final String jwtToken;
    private final String sccLocationId;

    public ConnectivitySocks5ProxySocket(String jwtToken, String
sccLocationId) {
        this.jwtToken = jwtToken;
        this.sccLocationId = sccLocationId != null ?
Base64.getEncoder().encodeToString(sccLocationId.getBytes()) : "";
    }

    protected InetSocketAddress getProxyAddress() {
        try {
            JSONObject credentials = extractEnvironmentCredentials();
            String proxyHost =
credentials.getString(SOCKS5_PROXY_HOST_PROPERTY);
            int proxyPort =
Integer.parseInt(credentials.getString(SOCKS5_PROXY_PORT_PROPERTY));
            return new InetSocketAddress(proxyHost, proxyPort);
        } catch (JSONException ex) {
            throw new IllegalStateException("Unable to extract the SOCKS5
proxy host and port", ex);
        }
    }

    private JSONObject extractEnvironmentCredentials() throws JSONException {
        JSONObject jsonObj = new JSONObject(System.getenv("VCAP_SERVICES"));
        JSONArray jsonArr = jsonObj.getJSONArray("connectivity");
        return jsonArr.getJSONObject(0).getJSONObject("credentials");
    }

    @Override
    public void connect(SocketAddress endpoint, int timeout) throws
IOException {
        super.connect(getProxyAddress(), timeout);

        OutputStream outputStream = getOutputStream();

        executeSOCKS5InitialRequest(outputStream);

        executeSOCKS5AuthenticationRequest(outputStream);

        executeSOCKS5ConnectRequest(outputStream, (InetSocketAddress)
endpoint);
    }

    private void executeSOCKS5InitialRequest(OutputStream outputStream)
throws IOException {
        byte[] initialRequest = createInitialSOCKS5Request();
        outputStream.write(initialRequest);

        assertServerInitialResponse();
    }

    private byte[] createInitialSOCKS5Request() throws IOException {
        ByteArrayOutputStream byteArraysStream = new ByteArrayOutputStream();
        try {
            byteArraysStream.write(SOCKS5_VERSION);
            byteArraysStream.write(SOCKS5_AUTHENTICATION_METHODS_COUNT);
            byteArraysStream.write(SOCKS5_JWT_AUTHENTICATION_METHOD);
            return byteArraysStream.toByteArray();
        } finally {
```

```
            byteArraysStream.close();
        }
    }

    private void assertServerInitialResponse() throws IOException {
        InputStream inputStream = getInputStream();

        int versionByte = inputStream.read();
        if (SOCKS5_VERSION != versionByte) {
            throw new SocketException(String.format("Unsupported SOCKS
version - expected %s, but received %s", SOCKS5_VERSION, versionByte));
        }

        int authenticationMethodValue = inputStream.read();
        if (SOCKS5_JWT_AUTHENTICATION_METHOD_UNSIGNED_VALUE !=
authenticationMethodValue) {
            throw new SocketException(String.format("Unsupported
authentication method value - expected %s, but received %s",
                    SOCKS5_JWT_AUTHENTICATION_METHOD_UNSIGNED_VALUE,
authenticationMethodValue));
        }
    }

    private void executeSOCKS5AuthenticationRequest(OutputStream
outputStream) throws IOException {
        byte[] authenticationRequest = createJWTAuthenticationRequest();
        outputStream.write(authenticationRequest);

        assertAuthenticationResponse();
    }

    private byte[] createJWTAuthenticationRequest() throws IOException {
        ByteArrayOutputStream byteArraysStream = new ByteArrayOutputStream();
        try {
            byteArraysStream.write(SOCKS5_JWT_AUTHENTICATION_METHOD_VERSION);

byteArraysStream.write(ByteBuffer.allocate(4).putInt(jwtToken.getBytes().lengt
h).array());
            byteArraysStream.write(jwtToken.getBytes());
            byteArraysStream.write(ByteBuffer.allocate(1).put((byte)
sccLocationId.getBytes().length).array());
            byteArraysStream.write(sccLocationId.getBytes());
            return byteArraysStream.toByteArray();
        } finally {
            byteArraysStream.close();
        }
    }

    private void assertAuthenticationResponse() throws IOException {
        InputStream inputStream = getInputStream();

        int authenticationMethodVersion = inputStream.read();
        if (SOCKS5_JWT_AUTHENTICATION_METHOD_VERSION !=
authenticationMethodVersion) {
            throw new SocketException(String.format("Unsupported
authentication method version - expected %s, but received %s",
                    SOCKS5_JWT_AUTHENTICATION_METHOD_VERSION,
authenticationMethodVersion));
        }

        int authenticationStatus = inputStream.read();
        if (SOCKS5_AUTHENTICATION_SUCCESS_BYTE != authenticationStatus) {
            throw new SocketException("Authentication failed!");
        }
    }

    private void executeSOCKS5ConnectRequest(OutputStream outputStream,
InetSocketAddress endpoint) throws IOException {
```

```
        byte[] commandRequest = createConnectCommandRequest(endpoint);
        outputStream.write(commandRequest);

        assertConnectCommandResponse();
    }

    private byte[] createConnectCommandRequest(InetSocketAddress endpoint)
throws IOException {
        String host = endpoint.getHostName();
        int port = endpoint.getPort();
        ByteArrayOutputStream byteArraysStream = new ByteArrayOutputStream();
        try {
            byteArraysStream.write(SOCKS5_VERSION);
            byteArraysStream.write(SOCKS5_COMMAND_CONNECT_BYTE);
            byteArraysStream.write(SOCKS5_COMMAND_REQUEST_RESERVED_BYTE);
            byte[] hostToIPv4 = parseHostToIPv4(host);
            if (hostToIPv4 != null) {
                byteArraysStream.write(SOCKS5_COMMAND_ADDRESS_TYPE_IPv4_BYTE);
                byteArraysStream.write(hostToIPv4);
            } else {

byteArraysStream.write(SOCKS5_COMMAND_ADDRESS_TYPE_DOMAIN_BYTE);
                byteArraysStream.write(ByteBuffer.allocate(1).put((byte)
host.getBytes().length).array());
                byteArraysStream.write(host.getBytes());
            }
            byteArraysStream.write(ByteBuffer.allocate(2).putShort((short)
port).array());
            return byteArraysStream.toByteArray();
        } finally {
            byteArraysStream.close();
        }
    }

    private void assertConnectCommandResponse() throws IOException {
        InputStream inputStream = getInputStream();

        int versionByte = inputStream.read();
        if (SOCKS5_VERSION != versionByte) {
            throw new SocketException(String.format("Unsupported SOCKS
version - expected %s, but received %s", SOCKS5_VERSION, versionByte));
        }

        int connectStatusByte = inputStream.read();
        assertConnectStatus(connectStatusByte);

        readRemainingCommandResponseBytes(inputStream);
    }

    private void assertConnectStatus(int commandConnectStatus) throws
IOException {
        if (commandConnectStatus == 0) {
            return;
        }

        String commandConnectStatusTranslation;
        switch (commandConnectStatus) {
            case 1:
                commandConnectStatusTranslation = "FAILURE";
                break;
            case 2:
                commandConnectStatusTranslation = "FORBIDDEN";
                break;
            case 3:
                commandConnectStatusTranslation = "NETWORK_UNREACHABLE";
                break;
            case 4:
                commandConnectStatusTranslation = "HOST_UNREACHABLE";
```

```
                break;
            case 5:
                commandConnectStatusTranslation = "CONNECTION_REFUSED";
                break;
            case 6:
                commandConnectStatusTranslation = "TTL_EXPIRED";
                break;
            case 7:
                commandConnectStatusTranslation = "COMMAND_UNSUPPORTED";
                break;
            case 8:
                commandConnectStatusTranslation = "ADDRESS_UNSUPPORTED";
                break;
            default:
                commandConnectStatusTranslation = "UNKNOWN";
                break;
        }
        throw new SocketException("SOCKS5 command failed with status: " +
commandConnectStatusTranslation);
    }

    private byte[] parseHostToIPv4(String hostName) {
        byte[] parsedHostName = null;
        String[] virtualHostOctets = hostName.split("\\.", -1);
        int octetsCount = virtualHostOctets.length;
        if (octetsCount == 4) {
            try {
                byte[] ipOctets = new byte[octetsCount];
                for (int i = 0; i < octetsCount; i++) {
                    int currentOctet = Integer.parseInt(virtualHostOctets[i]);
                    if ((currentOctet < 0) || (currentOctet > 255)) {
                        throw new
IllegalArgumentException(String.format("Provided octet %s is not in the range
of [0-255]", currentOctet));
                    }
                    ipOctets[i] = (byte) currentOctet;
                }
                parsedHostName = ipOctets;
            } catch (IllegalArgumentException ex) {
                return null;
            }
        }

        return parsedHostName;
    }

    private void readRemainingCommandResponseBytes(InputStream inputStream)
throws IOException {
        inputStream.read(); // skipping over SOCKS5 reserved byte
        int addressTypeByte = inputStream.read();
        if (SOCKS5_COMMAND_ADDRESS_TYPE_IPv4_BYTE == addressTypeByte) {
            for (int i = 0; i < 6; i++) {
                inputStream.read();
            }
        } else if (SOCKS5_COMMAND_ADDRESS_TYPE_DOMAIN_BYTE ==
addressTypeByte) {
            int domainNameLength = inputStream.read();
            int portBytes = 2;
            inputStream.read(new byte[domainNameLength + portBytes], 0,
domainNameLength + portBytes);
        }
    }
}
```

Back to Example [page 1337]

Back to Content [page 1335]

Troubleshooting

If the handshake with the SOCKS5 proxy server fails, a SOCKS5 protocol error is returned, see IETF RFC 1928 . The table below shows the most common errors and their root cause in the scenario you use:

SOCSK5 Error Code	Technical Description	Client-Side Error Description	Scenario Error
0x00	SUCCESS		Success
0x01	FAILURE		Connection closed by backend or general scenario failure.
0x02	FORBIDDEN	`Connection not allowed by ruleset`	No matching host mapping found in Cloud Connector access control settings, see Configure Access Control (TCP) [page 1767].
0x03	NETWORK_UNREACHABLE		The Cloud Connector is not connected to the subaccount and the Cloud Connector `Location ID` that is used by the cloud application can't be identified. See Connect and Disconnect a Cloud Subaccount [page 2003] and Adding and Managing Subaccounts [page 1682], section *Procedure*.
0x04	HOST_UNREACHABLE		Cannot open connection to the backend, that is, the host is unreachable.
0x05	CONNECTION_REFUSED		Authentication failure
0x06	TTL_EXPIRED		*Not used*
0x07	COMMAND_UNSUPPORTED		Only the SOCKS5 `CONNECT` command is supported.
0x08	ADDRESS_UNSUPPORTED		Only the SOCKS5 `DOMAIN` and `IPv4` commands are supported.

Back to Content [page 1335]

1.2.2.3 Connectivity Service REST API

Use a REST API to read information about Cloud Connectors connected to a given SAP BTP subaccount.

The Connectivity service provides a REST API that lets you read information about Cloud Connectors connected to a given subaccount. This API is documented in the SAP Business Accelerator Hub. It shows all available endpoints, the supported operations, parameters, possible error cases, related status codes, and so on.

You can also try it out via executing requests using the credentials of your Connectivity Service instance [page 1309].

Related Information

Calling the Connectivity Service REST API [page 1344]

1.2.2.3.1 Calling the Connectivity Service REST API

Learn how to call the Connectivity service REST API to read information about Cloud Connectors connected to your SAP BTP subaccount.

Prerequisites

To enable your application for consuming the Connectivity service REST API, you must create a Connectivity service instance with service plan *api_access* and bind it to the application.

For more information, see Create and Bind a Connectivity Service Instance [page 1309].

Get an Access Token

The endpoints of the Connectivity service REST API are protected with OAuth tokens issued by XSUAA. In order to obtain one, you need to extract the value of `token_service_url` from the binding information and call it using the binding credentials.

The *Cloud Connector Info* API is protected by an access token, which can be obtained by the OAuth2 Client Credentials grant type .

The *Authentication Metadata Retrieval* API is protected by the user exchange JWT, which is obtained by exchanging a valid user token following the OAuth2 JWT Bearer grant type . A user token needed in this

flow has to be obtained by authenticating a platform user, which has the *Cloud Connector Administrator* role assigned in the subaccount.

Example: Obtaining an XSUAA access token

Request:

Sample Code

```
POST <token_service_url>
Accept: application/json
Content-Type: application/x-www-form-urlencoded

client_id=<clientid>
client_secret=<clientsecret>
grant_type=client_credentials
```

Response:

Sample Code

```
{
    "access_token" : "<JWT>",
    "token_type" : "Bearer",
    "expires_in" : 3600
}
```

Caution

For demo purposes, we are using the *clientid* and *clientsecret* for authentication. We recommend that you use mTLS, which includes an X.509 client certificate, as a more secure way of authenticating.

Call the Connectivity Service REST API

Once you have obtained an access token, you can call one of the Connectivity service's REST API endpoints. The `connectivity_service` object in the binding JSON contains API access properties such as the URL and supported paths.

For more information about the available endpoints, see *Connectivity service REST API reference*.

Example: Calling the *Cloud Connector Info* **REST API endpoint**

Request:

Sample Code

```
GET <url><scc_info_path>
Accept: application/json
```

```
Authorization: Bearer <access_token>
```

Response:

Sample Code

```
[
  {
    "attributes": {
      "connected_by": "admin@example.com",
      "connected_since": "1762174293176",
      "security_status":
"{\"summary\":\"low_risk\",\"cpic_trace\":\"ok\",\"trust_store\":\"ok\",\"ciph
ers\":\"low_risk\",\"ui_certificate\":\"low_risk\",\"payload_trace\":\"ok\",\"
authentication_type\":\"low_risk\",\"application_whitelist\":\"ok\"}",
      "agent_version": "2.18.0",
      "agent_type": "master",
      "client_id": "123456789",
      "exposed_systems":
"[{\"protocol\":\"HTTP\",\"host\":\"virtualhost:1234\",\"description\":\"\",\"
type\":\"nonSAPsys\",\"status\":\"exposed\"}]",
      "jvm_vendor_version": "OpenJDK 64-Bit Server VM (SAP SE, 17.0.14+7-LTS,
mixed mode, sharing)",
      "agent_recent_start_ms": "1762167520212",
      "java_version": "17.0.14 (SAP SE)"
    }
  }
]
```

Example: Calling the *Authentication Metadata Retrieval* **REST API endpoint**

Request:

Sample Code

```
PUT <url><authentication_metadata_path>
Accept: application/json
Authorization: Bearer <user_exchange_token>
```

Response:

Sample Code

```
{
  "authenticationMetadata": "Base64 encoded authentication metadata"
}
```

1.3 Destination Service

Learn how to manage destinations and consume the Destination service in SAP BTP, multi-cloud foundation.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

The Destination service lets you find the information that is required to access a remote service or system from your cloud application.

- For the connection to an on-premise system, you can use this service together with the Connectivity service.
  - For Kubernetes environments, use the Destination service, the Connectivity Proxy and the Transparent Proxy for connections to the target system.
- For the connection to any other Web application, you can use the Destination service without the Connectivity service.
  - For Kubernetes environments, use the Destination service and the Transparent Proxy for automated and seamless connections to the target system.
- Manages routing and authentication details, as well as custom scenario-specific parameters.
- Performs authentication flows based on the configured details.

> **Note**
>
> Each subaccount is limited to a maximum of 1,000 entities per configuration type (destination, certificate, fragment).

Destination Service: Administration [page 1348]	Manage the Destination service in SAP BTP, multi-cloud foundation.
Consuming the Destination Service [page 1550]	Retrieve and store technical information about the destination to consume a target remote service from your application.

Related Information

Getting Started [page 12]
Connectivity Service [page 1308]
Cloud Connector [page 1602]
Connectivity Proxy for Kubernetes [page 2088]
Transparent Proxy for Kubernetes [page 2164]

1.3.1 Destination Service: Administration

Manage the Destination service in SAP BTP, multi-cloud foundation.

Managing Destinations [page 1349]	To manage destinations for your application, choose a procedure that fits best your requirements.
Manage Destination Fragments [page 1503]	Destination fragments are objects used to override and extend destination properties through the Destination service REST API.
Manage Destination Certificates [page 1504]	Maintain certificates (X.509 client certificates, trusted certificates, CA certificates, trust store and key store certificates) in the *Destination Certificates* UI or the *Destinations* editor (SAP BTP cockpit).
Manage Trust [page 1514]	Download and configure X.509 certificates as a prerequisite for user propagation from the multi-cloud environment.
Create and Bind a Destination Service Instance [page 1523]	To use the Destination service in your application, you need an instance of the service.
Destination Service Notifications [page 1528]	Subscribe to the SAP Alert Notification service to receive notifications and alerts for the Destination service on SAP BTP.
Configure Destination Service IPs in Firewall Rules [page 1529]	Include Destination service IP addresses in your target system firewall rules for incoming traffic.
Destination Chaining [page 1538]	Invoke multiple destinations with a single *Find a Destination* request.
Labeling Destination Service Entities [page 1543]	You can assign labels to some of the configurations that you can manage within the Destination service. Currently, this is supported for destinations and certificates.
Configuring Backup [page 1549]	Create a backup of your destination configurations.

1.3.1.1 Replacement of the Old *Destinations* Editor with a New Set of UIs

Get information on the sunset of the old *Destinations* editor, which is replaced by a new set of UIs for destination management in the cockpit.

The *Destinations* editor, which is the administrative UI of the Destination service, has undergone a major overhaul in the last couple of years. During the second half of 2024, we started introducing some elements of the new UIs to the BTP cockpit, side by side with the existing editor:

- Initially, we released the Destination Trust [page 1514] UI, used for managing the signing certificates for the assertion XML in SAML-based flows.
- In a next step, we released the Destination Certificates [page 1504] page, used for managing certificates stored in the Destination service, including the generation of certificates and flagging them for auto-renewal.

- Finally, we introduced the actual new Destinations [page 1472] UI, which we called *Destinations (New)* in order to separate it from the old UI. The new *Destinations* UI lets you manage the destinations themselves, including creating them from templates (for example, from service instances [page 1489]).

Also, we brought these new UIs to the service instance level where applicable.

After the initial release, the new UIs received a lot of enhancements, eventually achieving feature parity with the old UI. On top of that, some new functionality is available in the new UIs. As such, we are now ready to announce the **shutdown of the old UI by end of October 2025**. Until then, the old UI will still be available under the *Destinations (Legacy)* navigation entry in the cockpit, while the new one is simply called *Destinations* again.

As of November 2025, the old UI will be removed from the cockpit.

1.3.1.2 Managing Destinations

To manage destinations for your application, choose a procedure that fits best your requirements.

There are various ways to manage destinations. Each method is characterized by different prerequisites and limitations. Before choosing a method, you should evaluate them and decide which one is the most appropriate for your particular scenario. The following table compares the available methods:

Method	Create from Scratch	Create from Template	Create from File (Import)	Save to File (Export)	Update	Delete	Clone	Check Connection
Using the Destinations Editor in the Cockpit [page 1472]	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Destination Service REST API [page 1497]	Yes	No	Yes	Yes	Yes	Yes	No	No
Create Destinations Using the MTA Descriptor [page 1497]	Yes	Limited	No	No	Yes	No	No	No
Create Destinations on Service Instance Creation [page 1502]	Yes	No	No	No	Yes	No	No	No

1.3.1.2.1 Destination Types

Find an overview of destinations types you can use to configure your SAP BTP destinations, as well detailed information on relevant configuration parameters.

HTTP Destinations [page 1350]

RFC Destinations [page 1443]

LDAP Destinations [page 1465]

TCP Destinations [page 1466]

MAIL Destinations [page 1467]

1.3.1.2.1.1 HTTP Destinations

Find information about HTTP destinations for Internet and on-premise connections.

> ⓘ Note
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Destinations for Subscribed Applications

In subscription-based (SaaS) scenarios, it is not always possible to consume a service instance by binding it to your application. In this case, you must create a destination pointing to your service instance. For more information, see Destinations Pointing to Service Instances [page 1489].

Proxy Types

The proxy types supported by the Connectivity service are:

- **Internet** - The application can connect to an external REST or SOAP service on the Internet.
- **OnPremise** - The application can connect to an on-premise backend system through the Cloud Connector.
- **PrivateLink** - Selected SAP BTP services can establish a private connection with selected services in your own IaaS provider accounts.
  For more information, see What Is SAP Private Link Service?.

The proxy type used for a destination is specified by the destination property `ProxyType`. The default value is `Internet`.

Proxy Settings for Your Local Runtime

If you work in your local development environment behind a proxy server and want to use a service from the Internet, you need to configure your proxy settings on JVM level. To do this, proceed as follows:

1. On the *Servers* view, double-click the added server and choose *Overview* to open the editor.
2. Click the *Open Launch Configuration* link.
3. Choose the *(x)=Arguments* tab page.
4. In the *VM Arguments* box, add the following row:

```
-Dhttp.proxyHost=yourproxyHost -Dhttp.proxyPort=yourProxyPort
-Dhttps.proxyHost=yourproxyHost -Dhttps.proxyPort=yourProxyPort
```

5. Choose *OK*.
6. Start or restart your SAP HANA Cloud local runtime.

Configuring Authentication

When creating an HTTP destination, you can use different authentication types for access control:

- No Authentication [page 1353]
- Basic Authentication [page 1353]
- Client Certificate Authentication [page 1355]
- Server Certificate Authentication [page 1356]
- Principal Propagation SSO Authentication for HTTP [page 1359]
- OAuth SAML Bearer Assertion Authentication [page 1363]
- OAuth Client Credentials Authentication [page 1375]
- OAuth User Token Exchange Authentication [page 1384]
- OAuth Password Authentication [page 1391]
- OAuth JWT Bearer Authentication [page 1395]
- SAML Assertion Authentication [page 1402]
- OAuth with X.509 Client Certificates [page 1408]
- OAuth Refresh Token Authentication [page 1409]
- OAuth Authorization Code Authentication [page 1416]
- OAuth Technical User Propagation Authentication [page 1424]
- OAuth Token Exchange Authentication [page 1433]
- Using Client Assertion with OAuth Flows [page 1439]

Custom Query Parameters and Headers

> ⚠ Restriction
>
> The procedure described below is not supported by all clients, and depends on the consumption method.

By default, the Destination service does not use URL-associated queries and header parameters.

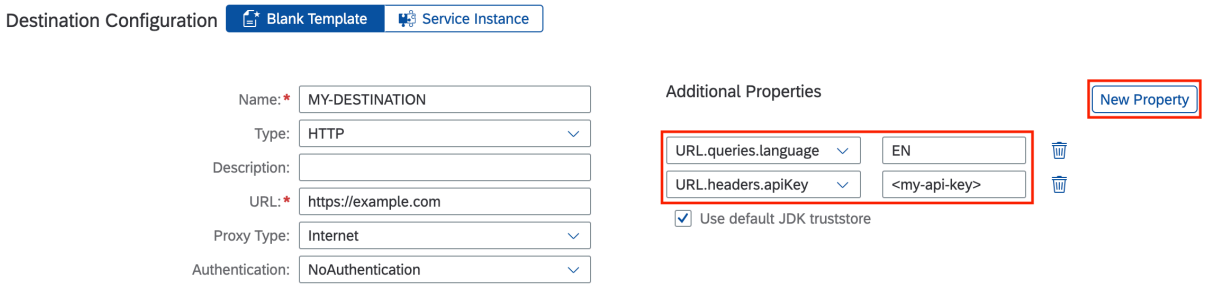
For most authentication types however, you can add them as custom parameters to the URL of a destination.

> → Tip
>
> You can use tools like the *Transparent Proxy for Kubernetes* to process those attributes automatically at runtime.
>
> For more information, see Using the Transparent Proxy [page 2166].

When using the *Destinations* editor for destination configuration, you can add these parameters as *additional properties*:

- `URL.headers.HEADER_KEY` for headers
- `URL.queries.QUERY_KEY` for query parameters

Replace `HEADER_KEY` and `QUERY_KEY` with the name of the headers or query parameters and set the respective values.



In the example above, the destination has a `language` query parameter with the value *EN* and an `apiKey` header with the value *<my-api-key>*.

Those additional headers and query parameters are added for every communication with the given destination.

For more information on additional properties, see Create HTTP Destinations [page 1476] (step 9) and the details of the respective authentication type [page 1351].

Related Information

OAuth with X.509 Client Certificates [page 1408]
Create HTTP Destinations [page 1476]

1.3.1.2.1.1.1 No Authentication

Create and configure a *No Authentication* destination for an application.

This authentication type is used for destinations that refer to a service on the Internet, an on-premise system, or a *Private Link* endpoint that does not require authentication. The relevant property value is:

```
Authentication=NoAuthentication
```

> **Note**
>
> When a destination is using HTTPS protocol to connect to a Web resource, the JDK truststore is used as truststore for the destination.

> **Note**
>
> `No Authentication` can be used in combination with `ProxyType=OnPremise`. In this case, also the `CloudConnectorLocationId` property can be specified. You can connect multiple Cloud Connectors to a subaccount as long as their location ID is different. The value defines the location ID identifying the Cloud Connector over which the connection shall be opened. The default value is the empty string identifying the Cloud Connector that is connected without any location ID.

1.3.1.2.1.1.2 Basic Authentication

Create and configure a *Basic Authentication* destination for an application.

Authentication type *Basic Authentication* is used for destinations that refer to a service on the Internet, an on-premise system, or a *Private Link* endpoint that requires basic authentication.

Properties

To configure a destination of this authentication type, you must specify all the required properties.

> **Caution**
>
> Do not use your *own personal credentials* in the `<User>` and `<Password>` fields. Always use a *technical user* instead.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Destination name. Must be unique for the destination level.
Type	`Type`	Destination type. Use **`HTTP`** as value for all HTTP(S) destinations.
URL	`URL`	URL of the protected resource on the called application.
Proxy Type	`ProxyType`	`Internet`, `OnPremise`, or `PrivateLink`.
Authentication	`Authentication`	Authentication type. Use **`BasicAuthentication`** as value.
User	`User`	User name of the *technical* user to be used.
Password	`Password`	Password of the *technical* user to be used.
Additional		
	Key	**Description**
	`Preemptive`	If this property is not set or is set to **`TRUE`** (that is, the default behavior is to use preemptive sending), the authentication token is sent preemptively. Otherwise, it relies on the challenge from the server (401 HTTP code). The default value (used if no value is explicitly specified) is **`TRUE`**. For more information about preemptiveness, see http://tools.ietf.org/html/rfc2617#section-3.3 .

Note

When a destination is using the HTTPS protocol to connect to a Web resource, the JDK truststore is used as truststore for the destination.

Note

`Basic Authentication` can be used in combination with `ProxyType=`**`OnPremise`**. In this case, also the `CloudConnectorLocationId` property can be specified. You can connect multiple Cloud Connectors to a subaccount as long as their location ID is different. The value defines the location ID identifying the Cloud Connector over which the connection shall be opened. The default value is the empty string identifying the Cloud Connector that is connected without any location ID.

The response for "find a destination" contains an `authTokens` object in the format given below. For more information on the fields in `authTokens`, see "Find a Destination" Response Structure [page 1565].

Sample Code

```
"authTokens": [
                        {
                        "type": "Basic",
                        "value": "dGVzdDpwYXNzMTIzNDU=",
```

```
                        "http_header": {
                        "key":"Authorization",
                        "value":"Basic dGVzdDpwYXNzMTIzNDU="
                        }
                        }
                        ]
```

1.3.1.2.1.1.3 Client Certificate Authentication

Create and configure a *Client Certificate* destination for an application.

> ⚠ Caution
>
> In the cockpit, this authentication type is shown in the selection list only if you have entered a URL that starts with **`https://`**.

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Destination name. Must be unique for the destination level.
Type	`Type`	Destination type. Use **`HTTP`** as value for all HTTP(S) destinations.
URL	`URL`	URL of the target endpoint.
Proxy Type	`ProxyType`	`Internet`, `OnPremise`, or `PrivateLink`.
Authentication	`Authentication`	Authentication type. Use **`ClientCertificateAuthentication`** as value.
Optional		
Key Store Source	`KeyStore.Source`	Optional. Specifies the storage location of the certificate to be used by the client. Supported values are: • `ClientProvided`: The key store is managed by the client (the application itself). • `DestinationService`: The key store is managed by the Destination service. If the property is not set, the key store is managed by the Destination service (default).
Key Store Location	`KeyStoreLocation`	The name of the key store file that contains the client certificate(s) for client certificate authentication against a remote server. This property is optional if `KeyStore.Source` is set to `ClientProvided`.

Cockpit Label	JSON Key	Description
Key Store Password	`KeyStorePassword`	Password for the key store file specified by `KeyStoreLocation`. Can be set empty.
Trust Store Location	`TrustStoreLocation`	Path to the keystore file which contains trusted certificates (Certificate Authorities) for authentication against a remote client. • When using a local environment: The relative path to the JKS file. The root path is the server's location on the file system. • When using a cloud environment: The name of the JKS file. Only shown and required if the default clilent trust store is not used. **Note** The default JDK truststore is appended to the truststore defined in the destination configuration. As a result, the destination simultaneously uses both truststores. If the `TrustStoreLocation` property is not specified, the JDK truststore is used as a default truststore for the destination.
Trust Store Password	`TrustStorePassword`	Only shown if the default clilent trust store is not used. Can be set empty.

> **Note**
>
> You can upload `KeyStore` files using the same command for uploading destination configuration property file. You only need to specify the `KeyStore` file instead of the destination configuration file.
>
> For more information on supported file formats, see Manage Destination Certificates [page 1504].

Related Information

Server Certificate Authentication [page 1356]

1.3.1.2.1.1.4 Server Certificate Authentication

Create and configure a *Server Certificate* destination for an application.

Context

The server certificate validation is applicable to all destinations with proxy type `Internet` and `PrivateLink` that use the HTTPS protocol.

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Optional		
Trust Store Location	`TrustStoreLocation`	Path to the keystore file which contains trusted certificates (Certificate Authorities) for authentication against a remote client. • When using a local environment: The relative path to the JKS file. The root path is the server's location on the file system. • When using a cloud environment: The name of the JKS file. **Note** The default JDK truststore is appended to the truststore defined in the destination configuration. As a result, the destination simultaneously uses both truststores. If the `TrustStoreLocation` property is not specified, the JDK truststore is used as a default truststore for the destination.
Trust Store Password	`TrustStorePassword`	Password for the JKS trust store file. This property is mandatory if `TrustStoreLocation` is used.
Additional		
	Key	**Description**

Cockpit Label	JSON Key	Description
	`TrustAll`	If this property is set to **TRUE** in the destination, the server certificate will not be checked for SSL connections. It is intended for test scenarios only, and should not be used in production (since the SSL server certificate is not checked, the server is not authenticated). The possible values are **TRUE** or **FALSE**; the default value is **FALSE** (that is, if the property is not present at all). If `TrustAll` = **TRUE**, the `TrustStoreLocation` property is ignored so you can omit it. If `TrustAll` = **FALSE**, the `TrustStoreLocation` property is mandatory to be used.
	`HostnameVerifier`	Optional property. It has two values: **`Strict`** and **`BrowserCompatible`**. This property specifies how the server hostname matches the names stored inside the server's X.509 certificate. This verifying process is only applied if TLS or SSL protocols are used and is not applied if the `TrustAll` property is specified. The default value (used if no value is explicitly specified) is **`Strict`**. • `Strict HostnameVerifier` works in the same way as Oracle Java 1.4, Oracle Java 5, and Oracle Java 6-rc. It is also similar to Microsoft Internet Explorer 6. This implementation appears to be compliant with RFC 2818 for dealing with wildcards. A wildcard such as "*.foo.com" matches only subdomains at the same level, for example "a.foo.com". It does not match deeper subdomains such as "a.b.foo.com". • `BrowserCompatible HostnameVerifier` works in the same way as Curl and Mozilla Firefox. The hostname must match either the first common name (CN) or any of the subject-alts. A wildcard can occur in the CN and in any of the subject-alts. The only difference between **`BrowserCompatible`** and **`Strict`** is that a wildcard (such as ".foo.com") with **`BrowserCompatible`** matches all subdomains, including "a.b.foo.com". For more information about these Java classes, see Package org.apache.http.conn.ssl . If `<TrustAll>` = **TRUE**, the `<HostnameVerifier>` property is ignored so you can omit it.

> **Note**
>
> Connections to remote services which require *Java Cryptography Extension (JCE) unlimited strength jurisdiction policy* are not supported.

Configuration

- Managing Destinations [page 1349]

Related Information

Client Certificate Authentication [page 1355]

1.3.1.2.1.1.5 Principal Propagation SSO Authentication for HTTP

Forward the identity of a cloud user from a Cloud Foundry application to a backend system via HTTP to enable single sign-on (SSO).

Context

A *PrincipalPropagation* destination enables single sign-on (SSO) by forwarding the identity of a cloud user to the Cloud Connector, and from there to the target on-premise system. In this way, the cloud user's identity can be provided without manual logon.

> **Note**
>
> This authentication type applies only for on-premise connectivity.

Configuration Steps

You can create and configure a *PrincipalPropagation* destination by using the properties listed below, and deploy it on SAP BTP. For more information, see Managing Destinations [page 1349].

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Destination name. Must be unique for the destination level.
Type	`Type`	Destination type. Use **`HTTP`** for all HTTP(S) destinations.
URL	`URL`	Virtual URL of the protected on-premise application.
Authentication	`Authentication`	Authentication type. Use **`PrincipalPropagation`** as value.
Proxy Type	`ProxyType`	You can only use proxy type **`OnPremise`**.
Optional		
Location ID	`CloudConnectorLocationId`	**Note** Relevant only if multiple Cloud Connectors are connected to a subaccount. You can connect multiple Cloud Connectors to a subaccount as long as their location ID is different. The location ID specifies the Cloud Connector over which the connection is opened. The default value is an empty string identifying the Cloud Connector that is connected without any location ID. For more information, see Adding a Subaccount [page 1684] (step 3e).
Additional		
	Key	**Description**

Cockpit Label	JSON Key	Description
	`URL.headers.<header-key>`	A static key prefix used as a namespace grouping of the URL's HTTP headers whose values will be sent to the target endpoint. For each HTTP header's key, you must add a `URL.headers` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.headers.<header-key-1>" : "<header-value-1>", ... "URL.headers.<header-key-N>": "<header-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.

Cockpit Label	JSON Key	Description
	`URL.queries.<query-key>`	A static key prefix used as a namespace grouping of URL's query parameters whose values will be sent to the target endpoint. For each query parameter's key, you must add a `URL.queries` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.queries.<query-key-1>" : "<query-value-1>", ... "URL.queries.<query-key-N>": "<query-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.

Example

```
Name=OnPremiseDestination
Type=HTTP
URL= http://virtualhost:80
Authentication=PrincipalPropagation
ProxyType=OnPremise
```

Related Information

Principal Propagation [page 30]

Principal Propagation from SAP BTP Applications to On-Premise Systems Using Corporate IdP Tokens

1.3.1.2.1.1.6 OAuth SAML Bearer Assertion Authentication

Create and configure an *OAuth SAML Bearer Assertion* destination for an application.

Context

You can call an OAuth2-protected remote system/API and propagate a user ID to the remote system by using the `OAuth2SAMLBearerAssertion` authentication type. The Destination service provides functionality for automatic token retrieval and caching, by automating the construction and sending of the SAML assertion. This simplifies application development, leaving you with only constructing the request to the remote system by providing the token, which is fetched for you by the Destination service. For more information, see User Propagation via SAML 2.0 Bearer Assertion Flow [page 1558].

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Name of the destination. Must be unique for the destination level.
Type	`Type`	Destination type. Choose **`HTTP`** for all HTTP(S) destinations.
URL	`URL`	URL of the target endpoint.
Proxy Type	`ProxyType`	You can only use proxy type **`Internet`** or **`OnPremise`**. If **`OnPremise`** is used, the OAuth server must be accessed through the Cloud Connector.
Authentication	`Authentication`	Authentication type. Use **`OAuth2SAMLBearerAssertion`** as value.

Cockpit Label	JSON Key	Description
Audience	`audience`	Intended audience for the assertion, which is verified by the OAuth authorization server. For more information, see SAML 2.0 Bearer Assertion Profiles for OAuth 2.0 . **Note** If the `SAMLAssertionProvider` property of the destination is set to `ClientProvided`, this property will not be used.
Client Key	`clientKey`	Key that identifies the consumer to the authorization server.

Cockpit Label	JSON Key	Description
Token Service URL	`tokenServiceURL`	The URL of the token service, against which the token exchange is performed. Depending on the `Token Service URL` type, this property is interpreted in different ways during the automatic token retrieval: • For **`Dedicated`**, the token service URL is taken as is. • For **`Common`**, the token service URL is searched for the tenant placeholder `{tenant}`. `{tenant}` is resolved as the subdomain of the subaccount on behalf of which the caller is performing the call. If the placeholder is not found, `{tenant}` is inserted as a subdomain of the token service URL. The subaccount subdomain is mandated during creation of the subaccount, see Create a Subaccount. **Examples** of interpreting the token service URL for the token service URL type **`Common`**, if the call to the Destination service is on behalf of a subaccount subdomain with value `mytenant`: • `https://authentication.us10.hana.ondemand.com/oauth/token` → `https://mytenant.authentication.us10.hana.ondemand.com/oauth/token` • `https://{tenant}.authentication.us10.hana.ondemand.com/oauth/token` → `https://mytenant.authentication.us10.hana.ondemand.com/oauth/token`

Cockpit Label	JSON Key	Description
		• `https://authentication.myauthserver.com/tenant/{tenant}/oauth/token` → `https://authentication.myauthserver.com/tenant/mytenant/oauth/token` • `https://oauth.{tenant}.myauthserver.com/token` → `https://oauth.mytenant.myauthserver.com/token`
AuthnContextClassRef	`authnContextClassRef`	Value of the `AuthnContextClassRef` tag, which is part of generated **`OAuth2SAMLBearerAssertion`** authentication. For more information, see SAML 2.0 specification .
Optional		
Token Service User	`tokenServiceUser`	User for basic authentication to OAuth server (if required).
Token Service Password	`tokenServicePassword`	Password for `tokenServiceUser` (if required).
SAML Assertion Provider	`SAMLAssertionProvider`	Specifies how the SAML assertion will be provided to the token service. There are two possible values: • `DestinationServiceGenerated`: specifies that Destination service will generate the SAML assertion, sign it using the subaccount trust certificate, and pass it to the token service. • `ClientProvided`: specifies that the client will provide the SAML assertion externally to the Destination service, and the service will pass it to the token service. If the property is missing, the Destination service will treat it by default as if the property was set to `DestinationServiceGenerated`.

Cockpit Label	JSON Key	Description
Name Qualifier	`nameQualifier`	Security domain of the user for which access token is requested.
Assertion Issuer	`assertionIssuer`	Issuer of the SAML assertion.
Assertion Recipient	`assertionRecipient`	Recipient of the SAML assertion. If not set, the token service URL will be the assertion's recipient.
Name Id Format	`nameIdFormat`	Value of the `NameIdFormat` tag, which is part of generated **`OAuth2SAMLBearerAssertion`** authentication. For more information, see SAML 2.0 specification .
User Id Source	`userIdSource`	When this property is set, the generated SAML2 assertion uses the currently logged-in user as a value for the `NameId` tag. See User Propagation via SAML 2.0 Bearer Assertion Flow [page 1558].
Token Service URL Type	`tokenServiceURLType`	Either **`Dedicated`** (if the `tokenServiceURL` serves only a single tenant), or **`Common`** (if the `tokenServiceURL` serves multiple tenants).
Token Service URL Proxy Type	`tokenServiceURL.ProxyType`	If set, it will use the specified `ProxyType` for the token service, independent from the `ProxyType` of the whole destination. This lets you configure Internet-based token services for on-premise systems.
Key Store Location	`KeyStoreLocation`	Contains the name of the certificate configuration to be used for *per-destination* SAML assertion signing. This certificate will be used instead of the standard subaccount-wide signing key. For more information, see Manage Trust [page 1514].
Key Store Password	`KeyStorePassword`	Contains the password for the certificate configuration (if one is needed) when using a *per-destination* SAML assertion signing certificate.
Token Service Key Store Location	`tokenService.KeyStoreLocation`	Contains the name of the certificate configuration to be used for mTLS towards the token service. This property is required when using OAuth with X.509 Client Certificates [page 1408].

Cockpit Label	JSON Key	Description
Token Service Key Store Password	`tokenService.KeyStorePassword`	Contains the password for the certificate configuration (if one is needed) when using OAuth with X.509 Client Certificates [page 1408].
Skip User Attributes Prefix In SAML Attributes	`skipUserAttributesPrefixInSAMLAttributes`	If set to *true*, any additional attributes taken from the OAuth server's user information endpoint, under the *user_attributes* section, will be added to the assertion without the prefix that the Destination service would usually add to them. For more information, see User Propagation via SAML 2.0 Bearer Assertion Flow [page 1558].
Include Signing Certificate In SAML Assertion	`includeSigningCertificateInSAMLAssertion`	If set to *true*, the public part of the signing certificate, used for signing the SAML assertion, will be included in the XML structure of the assertion. Some assertion recipients may require this to accept the assertion.
Skip User Uuid In SAML Attributes	`skipUserUuidInSAMLAttributes`	If set to *true*, any SAML assertion attribute with name 'user_uuid' will not be added to the resulting SAML assertion XML, even if such was found in the JWT, specifying the user's identity.
Additional		
	Key	**Description**

Cockpit Label	JSON Key	Description
	(Deprecated) `SystemUser`	User to be used when requesting an access token from the OAuth authorization server. If this property is not specified, the currently logged-in user is used. **⚠ Caution** This property is deprecated and will be removed by end of September 2026. We recommend that you work on behalf of specific (named) users instead of working with a technical user. As an alternative for technical user communication, we strongly recommend that you use one of these authentication types: • Basic Authentication (see Client Certificate Authentication [page 1355]) • Client Certificate Authentication (see Client Certificate Authentication [page 1355]) • OAuth Client Credentials Authentication [page 1375] To extend an OAuth access token's validity, consider using an OAuth refresh token. For *SuccessFactors* scenarios, see Technical User Access to SuccessFactors [page 138].
	`companyId`	Company identifier.
	`scope`	The value of the OAuth 2.0 scope parameter, expressed as a list of space-delimited, case-sensitive strings.

Cockpit Label	JSON Key	Description
	`tokenServiceURL.headers.<header-key>`	A static key prefix used as a namespace grouping of the `tokenServiceUrl`'s HTTP headers. Its values will be sent to the token service during token retrieval. For each HTTP header's key you must add a *'tokenServiceURL.headers'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.headers.<header-key-1>" : "<header-value-1>", ... "tokenServiceURL.headers.<header-key-N>": "<header-value-N>", }`
	`tokenServiceURL.ConnectionTimeoutInSeconds`	Defines the connection timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 60 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.
	`tokenServiceURL.SocketReadTimeoutInSeconds`	Defines the read timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 600 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.

Cockpit Label	JSON Key	Description
	`tokenServiceURL.queries.<query-key>`	A static key prefix used as a namespace grouping of `tokenServiceUrl`'s query parameters. Its values will be sent to the token service during token retrieval. For each query parameter's key you must add a *'tokenServiceURL.queries'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.queries.<query-key-1>" : "<query-value-1>", ... "tokenServiceURL.queries.<query-key-N>": "<query-value-N>", }`
	`tokenService.body.<param-key>`	A static key prefix used as a namespace grouping of parameters which are sent as part of the token request to the token service during token retrieval. For each request, a `tokenService.body` prefix must be added to the parameter key, separated by dot-delimiter. For example: **Sample Code** `{ ... "tokenService.body.<param-key-1>" : "<param-value-1>", ... "tokenService.body.<param-key-N>": "<param-value-N>", }`

Cockpit Label	JSON Key	Description
	`URL.headers.<header-key>`	A static key prefix used as a namespace grouping of the URL's HTTP headers whose values will be sent to the target endpoint. For each HTTP header's key, you must add a `URL.headers` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.headers.<header-key-1>" : "<header-value-1>", ... "URL.headers.<header-key-N>": "<header-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.

Cockpit Label	JSON Key	Description
	`URL.queries.<query-key>`	A static key prefix used as a namespace grouping of URL's query parameters whose values will be sent to the target endpoint. For each query parameter's key, you must add a `URL.queries` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.queries.<query-key-1>" : "<query-value-1>", ... "URL.queries.<query-key-N>": "<query-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.
	`x_user_token.jwks`	Base64-encoded *JSON web key set*, containing the signing keys which are used to validate the JWT provided in the *X-User-Token* header. For more information, see JWK Set Format .

Cockpit Label	JSON Key	Description
	`x_user_token.header.typ`	Specifies what is the token type of the JWT (JSON web token) provided in the `X-User-Token` header, which can be found in the decoded JWT header's "typ" JWT claim. This property is required when using user tokens which have not been generated by XSUAA. If this property is not provided, the Destination service will default to value `JWT` during runtime.
	`x_user_token.jwks_uri`	URI of the *JSON web key set*, containing the signing keys which are used to validate the JWT provided in the *X-User-Token* header. **⚠ Restriction** If the value is a private endpoint (for example, *localhost*), the Destination service will not be able to perform the verification of the X-User-Token header when using the "Find Destination" API. For more information, see OpenID Connect Discovery .
	`samlAssertionEncoding`	If set, this property controls what encoding will be used to encode the assertion XML prior to sending it to the configured token service. The possible values are "base64" (RFC 4648, section 4) and "base64url" (RFC 4648, section 5). If not set, the encoding will be "base64url".

Example

The connectivity destination below provides HTTP access to the OData API of the SuccessFactors Jam.

```
URL=https://demo.sapjam.com/OData/OData.svc
Name=sap_jam_odata
ProxyType=Internet
Type=HTTP
Authentication=OAuth2SAMLBearerAssertion
```

```
tokenServiceURL=https://demo.sapjam.com/api/v1/auth/token
clientKey=<unique_generated_string>
audience=cubetree.com
nameQualifier=www.successfactors.com
apiKey=<apiKey>
```

The response for "find destination" contains an `authTokens` object in the format given below. For more information on the fields in , see "Find a Destination" Response Structure [page 1565].

Sample Code

```
"authTokens": [
    {
        "type": "Bearer",
        "value": "eyJhbGciOiJSUzI1NiIsInR5cC...",
        "http_header": {
            "key":"Authorization",
            "value":"Bearer eyJhbGciOiJSUzI1NiIsInR5cC..."
        }
    }
]
```

Related Information

Create HTTP Destinations [page 1476]
Destination Examples [page 1486]
User Propagation via SAML 2.0 Bearer Assertion Flow [page 1558]
SAML Bearer Assertion Flow between Cloud Foundry Applications [page 37]
Exchanging User JWTs via OAuth2JWTBearer Destinations [page 1579]

1.3.1.2.1.1.7 OAuth Client Credentials Authentication

Create and configure an *OAuth2ClientCredentials* destination to consume OAuth-protected resources from an application.

SAP BTP supports applications to use the OAuth client credentials flow for consuming OAuth-protected resources.

The client credentials are used to request an access token from an OAuth authorization server.

Note

The retrieved access token is cached and auto-renovated. When a token is about to expire, a new token is created shortly before the expiration of the old one.

Configuration Steps

You can create and configure an *OAuth2ClientCredentials* destination using the properties listed below, and deploy it on SAP BTP. To create and configure a destination, follow the steps described in Managing Destinations [page 1349].

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Destination name. Must be unique for the destination level.
Type	`Type`	Destination type. Use **`HTTP`** as value for all HTTP(S) destinations.
URL	`URL`	URL of the protected resource on the called application.
Proxy Type	`ProxyType`	You can only use proxy type **`Internet`** or **`OnPremise`**. If **`OnPremise`** is used, the OAuth server must be accessed through the Cloud Connector.
Authentication	`Authentication`	Authentication type. Use **`OAuth2ClientCredentials`** as value.
Client ID	`clientId`	Client ID used to retrieve the access token.

Cockpit Label	JSON Key	Description
Token Service URL	`tokenServiceURL`	URL of the token service, against which token retrieval is performed. Depending on the `tokenServiceURLType`, this property is interpreted in different ways during automatic token retrieval: • For **`Dedicated`**, the `tokenServiceURL` is used as is. • For **`Common`**, the `tokenServiceURL` is searched for the tenant placeholder *{tenant}*. It is resolved as subdomain of the subaccount on whose behalf the caller is performing the call to the Destination service API for fetching the destination. If the placeholder is not found, *{tenant}* is processed as a subdomain of the `tokenServiceURL`. See the Destination service REST API to learn how the subaccount's subdomain is specified. The subaccount subdomain is mandated during creation of the subaccount, see Create a Subaccount (SAP BTP Core documentation). Examples of interpreting of the `tokenServiceURL` for `tokenServiceURLType` **`Common`**, if the call to the Destination service is done on behalf of a subaccount with subdomain value *mytenant*: • https://authentication.us10.hana.ondemand.com/oauth/token → https://**mytenant**.authentication.us10.hana.ondemand.com/oauth/token • https://**{tenant}**.authentication.us10.hana.ondemand.com/oauth/token → https://**myten-**

Cockpit Label	JSON Key	Description
		ant.authentication.us10.hana.ondemand.com/oauth/token • https://authentication.myauthserver.com/tenant/**{tenant}**/oauth/token → https://authentication.myauthserver.com/tenant/**mytenant**/oauth/token • https://oauth.**{tenant}**.myauthserver.com/token → https://oauth.**myten-ant**.myauthserver.com/token
Optional		
Client Secret	`clientSecret`	Client secret for the Client ID.
Token Service User	`tokenServiceUser`	User for basic authentication to OAuth server (if required).
Token Service Password	`tokenServicePassword`	Password for `tokenServiceUser` (if required).
Token Service URL Type	`tokenServiceURLType`	Either **`Dedicated`** (if the `tokenServiceURL` serves only a single tenant), or **`Common`** (if the `tokenServiceURL` serves multiple tenants).
Token Service URL Proxy Type	`tokenServiceURL.ProxyType`	If set, it will use the specified `ProxyType` for the token service, independent from the `ProxyType` of the whole destination. This lets you configure Internet-based token services for on-premise systems.
Token Service Key Store Location	`tokenService.KeyStoreLocation`	Contains the name of the certificate configuration to be used. This property is required when using client certificates for authentication. See OAuth with X.509 Client Certificates [page 1408].
Token Service Key Store Password	`tokenService.KeyStorePassword`	Contains the password for the certificate configuration (if one is needed) when using client certificates for authentication. See OAuth with X.509 Client Certificates [page 1408].
Additional		
	Key	**Description**
	`scope`	The value of the OAuth 2.0 scope parameter expressed as a list of space-delimited, case-sensitive strings.

Cockpit Label	JSON Key	Description
	`tokenServiceURL.headers.<header-key>`	A static key prefix used as a namespace grouping of the `tokenServiceUrl`'s HTTP headers. Its values will be sent to the token service during token retrieval. For each HTTP header's key you must add a *'tokenServiceURL.headers'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.headers.<header-key-1>" : "<header-value-1>", ... "tokenServiceURL.headers.<header-key-N>": "<header-value-N>", }`
	`tokenServiceURL.ConnectionTimeoutInSeconds`	Defines the connection timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 60 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.
	`tokenServiceURL.SocketReadTimeoutInSeconds`	Defines the read timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 600 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.

Cockpit Label	JSON Key	Description
	`tokenServiceURL.queries.<query-key>`	A static key prefix used as a namespace grouping of `tokenServiceUrl`'s query parameters. Its values will be sent to the token service during token retrieval. For each query parameter's key you must add a *'tokenServiceURL.queries'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.queries.<query-key-1>" : "<query-value-1>", ... "tokenServiceURL.queries.<query-key-N>": "<query-value-N>", }`
	`tokenService.body.<param-key>`	A static key prefix used as a namespace grouping of parameters which are sent as part of the token request to the token service during token retrieval. For each request, a `tokenService.body` prefix must be added to the parameter key, separated by dot-delimiter. For example: **Sample Code** `{ ... "tokenService.body.<param-key-1>" : "<param-value-1>", ... "tokenService.body.<param-key-N>": "<param-value-N>", }`

Cockpit Label	JSON Key	Description
	`tokenService.addClientCredentialsInBody`	Specifies whether the client credentials should be placed in the request body of the token request, rather than the `Authorization` header. Default is **`true`**. **Note** If set to **`false`**, but `tokenServiceUser` / `tokenServicePassword` are set, `tokenServiceUser` / `tokenServicePassword` are taken with priority.
	`clientAssertion.type`	When using this destination as a client assertion provider, you can specify the format of the client assertion as defined by the authorization server. The supported values are: • "urn:ietf:params:oauth:client-assertion-type:saml2-bearer" => indicating a SAML Bearer assertion. • "urn:ietf:params:oauth:client-assertion-type:jwt-bearer" => indicating a JWT Bearer token. This is used in case of automated client assertion fetching by the service. For more information, see Client Assertion with Automated Assertion Fetching by the Service [page 1440].
	`clientAssertion.destinationName`	Name of the destination that provides client assertions when using client assertion authentication mechanism. Must be on the same subaccount or service instance as this destination. This is used in case of automated client assertion fetching by the service. For more information, see Client Assertion with Automated Assertion Fetching by the Service [page 1440].

Cockpit Label	JSON Key	Description
	`URL.headers.<header-key>`	A static key prefix used as a namespace grouping of the URL's HTTP headers whose values will be sent to the target endpoint. For each HTTP header's key, you must add a `URL.headers` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.headers.<header-key-1>" : "<header-value-1>", ... "URL.headers.<header-key-N>": "<header-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.

Cockpit Label	JSON Key	Description
	`URL.queries.<query-key>`	A static key prefix used as a namespace grouping of URL's query parameters whose values will be sent to the target endpoint. For each query parameter's key, you must add a `URL.queries` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.queries.<query-key-1>" : "<query-value-1>", ... "URL.queries.<query-key-N>": "<query-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.

Example: Neo OAuth Token Service

Sample Code

```
URL=https://api.{landscape-domain}/desired-service-path
Name=sapOAuthCC
ProxyType=Internet
Type=HTTP
Authentication=OAuth2ClientCredentials
tokenServiceURL=(https://api.{landscape-domain}/oauth2/apitoken/v1?
grant_type=client_credentials
tokenServiceUser=clientIdValue
```

```
tokenServicePassword=secretValue
clientId=clientIdValue
clientSecret=secretValue
```

Example: OAuth Token Service Accepting Client Credentials as Body

Sample Code

```
URL=https://demo.sapjam.com/OData/OData.svc
Name=sap_jam_odata
ProxyType=Internet
Type=HTTP
Authentication=OAuth2ClientCredentials
tokenServiceURL=http://demo.sapjam.com/api/v1/auth/token
tokenServiceUser=tokenserviceuser
tokenServicePassword=pass
clientId=clientId
clientSecret=secret
```

Example: AuthTokens Object Response

The response for "find destination" contains an `authTokens` object in the format given below. For more information on the fields in `authTokens`, see "Find a Destination" Response Structure [page 1565].

Sample Code

```
"authTokens": [
    {
        "type": "Bearer",
        "value": "eyJhbGciOiJSUzI1NiIsInR5cC...",
        "http_header": {
            "key":"Authorization",
            "value":"Bearer eyJhbGciOiJSUzI1NiIsInR5cC..."
        }
    }
]
```

1.3.1.2.1.1.8 OAuth User Token Exchange Authentication

Learn about the `OAuth2UserTokenExchange` authentication type for HTTP destinations: use cases, supported properties and ways to retrieve an access token in an automated way.

Content

Overview [page 1385]

Properties [page 1385]

Example: AuthTokens Object Response [page 1390]

Overview

When a user is logged into an application that needs to call another application and pass the user context, the caller application must perform a user token exchange.

The user token exchange is a sequence of steps during which the initial user token is handed over to the authorization server and, in exchange, another access token is returned.

The calling application first receives a refresh token out of which the actual user access token is created. The resulting user access token contains the user and tenant context as well as technical access metadata, like scopes, that are required for accessing the target application.

Using the `OAuth2UserTokenExchange` authentication type, the Destination service performs all these steps automatically, which lets you simplify your application development in the Cloud Foundry environment.

Back to Content [page 1385]

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Name of the destination. Must be unique for the destination level.
Type	`Type`	Choose **`HTTP`** (for HTTP or HTTPS communication).
URL	`URL`	URL of the target endpoint.
Proxy Type	`ProxyType`	You can only use proxy type **`Internet`** or **`OnPremise`**. If **`OnPremise`** is used, the OAuth server must be accessed through the Cloud Connector.
Authentication	`Authentication`	**`OAuth2UserTokenExchange`** in this case.
Client ID	`clientId`	OAuth 2.0 client ID to be used for the user access token exchange.

Cockpit Label	JSON Key	Description
Token Service URL	`tokenServiceURL`	The URL of the token service, against which the token exchange is performed. Depending on the `Token Service URL Type`, this property is interpreted in different ways during the automatic token retrieval: • For **`Dedicated`**, the token service URL is taken as is. • For **`Common`**, the token service URL is searched for the tenant placeholder `{tenant}`. `{tenant}` is resolved as the subdomain of the subaccount on behalf of which the caller is performing the call. If the placeholder is not found, `{tenant}` is inserted as a subdomain of the token service URL. See Automated Access Token Retrieval [page 1580] for information about how the tenant is determined. The subaccount subdomain is mandated during creation of the subaccount, see Create a Subaccount. **Examples** of interpreting the token service URL for the token service URL type **`Common`**, if the call to the Destination service is on behalf of a subaccount subdomain with value `mytenant`: • `https://authentication.us10.hana.ondemand.com/oauth/token` → `https://mytenant.authentication.us10.hana.ondemand.com/oauth/token` • `https://{tenant}.authentication.us10.hana.ondemand.com/oauth/token` → `https://mytenant.authentication.us10.hana.ondemand.com/oauth/token` • `https://authentication.myauthserver.com/tenant/{tenant}/oauth/token` → `https://authentication.myauthserver.com/tenant/mytenant/oauth/token` • `https://oauth.{tenant}.myauthserver.com/token` → `https://oauth.mytenant.myauthserver.com/token`
Optional		
Client Secret	`clientSecret`	OAuth 2.0 client secret to be used for the user access token exchange.
Description	`Description`	Description of the destination.
Token Service URL Type	`tokenServiceURLType`	• Choose **`Dedicated`** if the token service URL serves only a single tenant. • Choose **`Common`** if the token service URL serves multiple tenants.
Token Service URL Proxy Type	`tokenServiceURL.ProxyType`	If set, it will use the specified `ProxyType` for the token service, independent from the `ProxyType` of the whole destination. This lets you configure Internet-based token services for on-premise systems.

Cockpit Label	JSON Key	Description
Token Service Key Store Location	`tokenService.KeyStoreLocation`	Contains the name of the certificate configuration to be used. This property is required when using client certificates for authentication. See OAuth with X.509 Client Certificates [page 1408].
Token Service Key Store Password	`tokenService.KeyStorePassword`	Contains the password for the certificate configuration (if one is needed) when using client certificates for authentication. See OAuth with X.509 Client Certificates [page 1408].
Additional		
	Key	**Description**
	`scope`	The value of the OAuth 2.0 scope parameter expressed as a list of space-delimited, case-sensitive strings.
	`tokenServiceURL.headers.<header-key>`	A static key prefix used as a namespace grouping of the `tokenServiceUrl`'s HTTP headers. Its values will be sent to the token service during token retrieval. For each HTTP header's key you must add a *'tokenServiceURL.headers'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.headers.<header-key-1>" : "<header-value-1>", ... "tokenServiceURL.headers.<header-key-N>": "<header-value-N>", }`
	`tokenServiceURL.queries.<query-key>`	A static key prefix used as a namespace grouping of `tokenServiceUrl`'s query parameters. Its values will be sent to the token service during token retrieval. For each query paramester's key you must add a *'tokenServiceURL.queries'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.queries.<query-key-1>" : "<query-value-1>", ... "tokenServiceURL.queries.<query-key-N>": "<query-value-N>", }`

Cockpit Label	JSON Key	Description
	`tokenService.body.<param-key>`	A static key prefix used as a namespace grouping of parameters which are sent as part of the token request to the token service during token retrieval. For each request, a `tokenService.body` prefix must be added to the parameter key, separated by dot-delimiter. For example: **Sample Code** `{ ... "tokenService.body.<param-key-1>" : "<param-value-1>", ... "tokenService.body.<param-key-N>": "<param-value-N>", }`
	`URL.headers.<header-key>`	A static key prefix used as a namespace grouping of the URL's HTTP headers whose values will be sent to the target endpoint. For each HTTP header's key, you must add a `URL.headers` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.headers.<header-key-1>" : "<header-value-1>", ... "URL.headers.<header-key-N>": "<header-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.

Cockpit Label	JSON Key	Description
	`URL.queries.<query-key>`	A static key prefix used as a namespace grouping of URL's query parameters whose values will be sent to the target endpoint. For each query parameter's key, you must add a `URL.queries` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.queries.<query-key-1>" : "<query-value-1>", ... "URL.queries.<query-key-N>": "<query-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.
	`tokenService.addClientCredentialsInBody`	Specifies whether the client credentials should be placed in the request body of the token request, rather than the `Authorization` header. Default is **`true`**. **Note** If set to **`false`**, but `tokenServiceUser` / `tokenServicePassword` are set, `tokenServiceUser` / `tokenServicePassword` are taken with priority.
	`x_user_token.jwks`	Base64-encoded *JSON web key set*, containing the signing keys which are used to validate the JWT provided in the *X-User-Token* header. For more information, see JWK Set Format .
	`x_user_token.header.typ`	Specifies what is the token type of the JWT (JSON web token) provided in the `X-User-Token` header, which can be found in the decoded JWT header's "typ" JWT claim. This property is required when using user tokens which have not been generated by XSUAA. If this property is not provided, the Destination service will default to value **`JWT`** during runtime.

Cockpit Label	JSON Key	Description
	`x_user_token.jwks_uri`	URI of the *JSON web key set*, containing the signing keys which are used to validate the JWT provided in the *X-User-Token* header. **Restriction** If the value is a private endpoint (for example, *localhost*), the Destination service will not be able to perform the verification of the X-User-Token header when using the "Find Destination" API. For more information, see OpenID Connect Discovery .

Back to Content [page 1385]

Example: AuthTokens Object Response

The response for "find destination" contains an `authTokens` object in the format given below. For more information on the fields in `authTokens`, see "Find a Destination" Response Structure [page 1565].

Sample Code

```
"authTokens": [
    {
        "type": "Bearer",
        "value": "eyJhbGciOiJSUzI1NiIsInR5cC...",
        "http_header": {
            "key":"Authorization",
            "value":"Bearer eyJhbGciOiJSUzI1NiIsInR5cC..."
        }
    }
]
```

Back to Content [page 1385]

Related Information

Exchanging User JWTs via OAuth2JWTBearer Destinations [page 1579]

1.3.1.2.1.1.9 OAuth Password Authentication

Learn about the OAuth password authentication type for HTTP destinations: use cases, supported properties and examples.

Content

Overview [page 1391]

Properties [page 1391]

Example: OAuth Token Service [page 1395]

Overview

SAP BTP provides support for applications to use the OAuth password grant flow for consuming OAuth-protected resources.

The client credentials as well as the user name and password are used to request an access token from an OAuth server, referred to as *token service* below. Access token retrieval is performed automatically by the Destination service when using the "find destination" REST endpoint.

Back to Content [page 1391]

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Destination name. It must be the same as the destination name you use for the configuration tools, that is, the console client and *Destinations* editor (cockpit).
Type	`Type`	Destination type. Choose **`HTTP`** (for HTTP or HTTPS communication).
URL	`URL`	URL of the protected resource being accessed.
Proxy Type	`ProxyType`	You can only use proxy type **`Internet`** or **`OnPremise`**. If **`OnPremise`** is used, the OAuth server must be accessed through the Cloud Connector.

Cockpit Label	JSON Key	Description
Authentication	`Authentication`	Authentication type. Use **`OAuth2Password`** as value.
Client ID	`clientId`	Client ID used to retrieve the access token.
User	`User`	User name of the technical user trying to get a token.
Password	`Password`	Password of the technical user trying to get a token.
Token Service URL	`tokenServiceURL`	Token retrieval URL of the OAuth server.
Optional		
Client Secret	`clientSecret`	Client secret for the client ID.
Token Service URL Proxy Type	`tokenServiceURL.ProxyType`	If set, it will use the specified `ProxyType` for the token service, independent from the `ProxyType` of the whole destination. This lets you configure Internet-based token services for on-premise systems.
Token Service Key Store Location	`tokenService.KeyStoreLocation`	Contains the name of the certificate configuration to be used. This property is required when using client certificates for authentication. See OAuth with X.509 Client Certificates [page 1408].
Token Service Key Store Password	`tokenService.KeyStorePassword`	Contains the password for the certificate configuration (if one is needed) when using client certificates for authentication. See OAuth with X.509 Client Certificates [page 1408].
Additional		
	Key	**Description**
	`scope`	Value of the OAuth 2.0 `scope` parameter, expressed as a list of space-delimited, case-sensitive strings.
	`tokenServiceURL.headers.<header-key>`	Static key prefix used as a namespace grouping of the `tokenServiceUrl`'s HTTP headers. Its values will be sent to the token service during token retrieval. For each HTTP header's key you must add a *'tokenServiceURL.headers'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.headers.<header-key-1>" : "<header-value-1>", ... "tokenServiceURL.headers.<header-key-N>": "<header-value-N>", }`
	`tokenServiceURL.ConnectionTimeoutInSeconds`	Defines the connection timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 60 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.

Cockpit Label	JSON Key	Description
	`tokenServiceURL.SocketReadTimeoutInSeconds`	Defines the read timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 600 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.
	`tokenServiceURL.queries.<query-key>`	Static key prefix used as a namespace grouping of `tokenServiceUrl`'s query parameters. Its values will be sent to the token service during token retrieval. For each query paramester's key you must add a *'tokenServiceURL.queries'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.queries.<query-key-1>" : "<query-value-1>", ... "tokenServiceURL.queries.<query-key-N>": "<query-value-N>", }`
	`tokenService.body.<param-key>`	A static key prefix used as a namespace grouping of parameters which are sent as part of the token request to the token service during token retrieval. For each request, a `tokenService.body` prefix must be added to the parameter key, separated by dot-delimiter. For example: **Sample Code** `{ ... "tokenService.body.<param-key-1>" : "<param-value-1>", ... "tokenService.body.<param-key-N>": "<param-value-N>", }`
	`tokenService.addClientCredentialsInBody`	Specifies whether the client credentials should be placed in the request body of the token request, rather than the `Authorization` header. Default is **`true`**.
	`clientAssertion.destinationName`	Name of the destination that provides client assertions when using client assertion authentication mechanism. Must be on the same subaccount or service instance as this destination. This is used in case of automated client assertion fetching by the service. For more information, see Client Assertion with Automated Assertion Fetching by the Service [page 1440].

Cockpit Label	JSON Key	Description
	`URL.headers.<header-key>`	Static key prefix used as a namespace grouping of the URL's HTTP headers whose values will be sent to the target endpoint. For each HTTP header's key, you must add a `URL.headers` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.headers.<header-key-1>" : "<header-value-1>", ... "URL.headers.<header-key-N>": "<header-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.
	`URL.queries.<query-key>`	Static key prefix used as a namespace grouping of URL's query parameters whose values will be sent to the target endpoint. For each query parameter's key, you must add a `URL.queries` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.queries.<query-key-1>" : "<query-value-1>", ... "URL.queries.<query-key-N>": "<query-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.

Back to Content [page 1391]

Example: OAuth Token Service

Sample Code

```
{
    "Name": "SapOAuthPassGrant",
    "Type": "HTTP",
    "URL": "https://myapp.cfapps.sap.hana.ondemand.com/mypath",
    "ProxyType": "Internet",
    "Authentication": "OAuth2Password",
    "clientId": "my-client-id",
    "clientSecret": "my-client-pass",
    "User": "my-username",
    "Password": "my-password",
    "tokenServiceURL": "https://authentication.sap.hana.ondemand.com/oauth/
token"
}
```

The response for "find destination" contains an `authTokens` object in the format given below. For more information on the fields in `authTokens`, see "Find a Destination" Response Structure [page 1565].

Sample Code

```
"authTokens": [
    {
        "type": "Bearer",
        "value": "eyJhbGciOiJSUzI1NiIsInR5cC...",
        "http_header": {
            "key":"Authorization",
            "value":"Bearer eyJhbGciOiJSUzI1NiIsInR5cC..."
        }
    }
]
```

Back to Content [page 1391]

1.3.1.2.1.1.10 OAuth JWT Bearer Authentication

Learn about the OAuth JWT bearer authentication type for HTTP destinations: use cases, supported properties and examples.

Content

Overview [page 1396]

Properties [page 1396]

Example: AuthTokens Object Response [page 1401]

Overview

To allow an application to call another application, passing the user context, and fetch resources, the caller application must pass an access token. In this authorization flow, the initial user token is passed to the OAuth server as input data. This process is performed automatically by the Destination service, which helps simplifying the application development: You only have to construct the right request to the target URL, by using the outcome (another access token) of the service-side automation.

Back to Content [page 1395]

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Name of the destination. Must be unique for the destination level.
Type	`Type`	Choose **`HTTP`** (for HTTP or HTTPS communication).
URL	`URL`	URL of the target endpoint.
Proxy Type	`ProxyType`	You can only use proxy type **`Internet`** or **`OnPremise`**. If **`OnPremise`** is used, the OAuth server must be accessed through the Cloud Connector.
Authentication	`Authentication`	**`OAuth2JWTBearer`** in this case.
Client ID	`clientId`	OAuth 2.0 client ID to be used for the user access token exchange.

Cockpit Label	JSON Key	Description
Token Service URL	`tokenServiceURL`	The URL of the token service, against which the token exchange is performed. Depending on the `Token Service URL Type`, this property is interpreted in different ways during the automatic token retrieval: • For **`Dedicated`**, the token service URL is taken as is. • For **`Common`**, the token service URL is searched for the tenant placeholder `{tenant}`. `{tenant}` is resolved as the subdomain of the subaccount on behalf of which the caller is performing the call. If the placeholder is not found, `{tenant}` is inserted as a subdomain of the token service URL. See Automated Access Token Retrieval [page 1580] for information about how the tenant is determined. The subaccount subdomain is mandated during creation of the subaccount, see Create a Subaccount. **Examples** of interpreting the token service URL for the token service URL type **`Common`**, if the call to the Destination service is on behalf of a subaccount subdomain with value `mytenant`: • `https://authentication.us10.hana.ondemand.com/oauth/token` → `https://mytenant.authentication.us10.hana.ondemand.com/oauth/token` • `https://{tenant}.authentication.us10.hana.ondemand.com/oauth/token` → `https://mytenant.authentication.us10.hana.ondemand.com/oauth/token` • `https://authentication.myauthserver.com/tenant/{tenant}/oauth/token` → `https://authentication.myauthserver.com/tenant/mytenant/oauth/token` • `https://oauth.{tenant}.myauthserver.com/token` → `https://oauth.mytenant.myauthserver.com/token`
Optional		
Client Secret	`clientSecret`	OAuth 2.0 client secret to be used for the user access token exchange.
Description	`Description`	A human-readable description of the destination.
Token Service URL Type	`tokenServiceURLType`	• Choose **`Dedicated`** if the token service URL serves only a single tenant. • Choose **`Common`** if the token service URL serves multiple tenants.
Token Service URL Proxy Type	`tokenServiceURL.ProxyType`	If set, it will use the specified `ProxyType` for the token service, independent from the `ProxyType` of the whole destination. This lets you configure Internet-based token services for on-premise systems.

Cockpit Label	JSON Key	Description
Token Service Key Store Location	`tokenService.KeyStoreLocation`	Contains the name of the certificate configuration to be used. This property is required when using client certificates for authentication. See OAuth with X.509 Client Certificates [page 1408].
Token Service Key Store Password	`tokenService.KeyStorePassword`	Contains the password for the certificate configuration (if one is needed) when using client certificates for authentication. See OAuth with X.509 Client Certificates [page 1408].
Additional		
	Key	**Description**
	`scope`	The value of the OAuth 2.0 `scope` parameter, expressed as a list of space-delimited, case-sensitive strings.
	`tokenServiceURL.headers.<header-key>`	A static key prefix used as a namespace grouping of the `tokenServiceUrl`'s HTTP headers. Its values will be sent to the token service during token retrieval. For each HTTP header's key you must add a *'tokenServiceURL.headers'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.headers.<header-key-1>" : "<header-value-1>", ... "tokenServiceURL.headers.<header-key-N>": "<header-value-N>", }`
	`tokenServiceURL.ConnectionTimeoutInSeconds`	Defines the connection timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 60 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.
	`tokenServiceURL.SocketReadTimeoutInSeconds`	Defines the read timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 600 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.

Cockpit Label	JSON Key	Description
	`tokenServiceURL.queries.<query-key>`	A static key prefix used as a namespace grouping of `tokenServiceUrl`'s query parameters. Its values will be sent to the token service during token retrieval. For each query paramester's key you must add a *'tokenServiceURL.queries'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.queries.<query-key-1>" : "<query-value-1>", ... "tokenServiceURL.queries.<query-key-N>": "<query-value-N>", }`
	`tokenService.body.<param-key>`	A static key prefix used as a namespace grouping of parameters which are sent as part of the token request to the token service during token retrieval. For each request, a `tokenService.body` prefix must be added to the parameter key, separated by dot-delimiter. For example: **Sample Code** `{ ... "tokenService.body.<param-key-1>" : "<param-value-1>", ... "tokenService.body.<param-key-N>": "<param-value-N>", }`
	`x_user_token.jwks`	Base64-encoded *JSON web key set*, containing the signing keys which are used to validate the JWT provided in the *X-User-Token* header. For more information, see JWK Set Format .
	`x_user_token.header.typ`	Specifies what is the token type of the JWT (JSON web token) provided in the `X-User-Token` header, which can be found in the decoded JWT header's "typ" JWT claim. This property is required when using user tokens which have not been generated by XSUAA. If this property is not provided, the Destination service will default to value **JWT** during runtime.

Cockpit Label	JSON Key	Description
	`x_user_token.jwks_uri`	URI of the *JSON web key set*, containing the signing keys which are used to validate the JWT provided in the *X-User-Token* header. **⚠ Restriction** If the value is a private endpoint (for example, *localhost*), the Destination service will not be able to perform the verification of the X-User-Token header when using the "Find Destination" API. For more information, see OpenID Connect Discovery .
	`URL.headers.<header-key>`	A static key prefix used as a namespace grouping of the URL's HTTP headers whose values will be sent to the target endpoint. For each HTTP header's key, you must add a `URL.headers` prefix separated by dot-delimiter. For example: **{·} Sample Code** `{ ... "URL.headers.<header-key-1>" : "<header-value-1>", ... "URL.headers.<header-key-N>": "<header-value-N>", }` **ⓘ Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.

Cockpit Label	JSON Key	Description
	`URL.queries.<query-key>`	A static key prefix used as a namespace grouping of URL's query parameters whose values will be sent to the target endpoint. For each query parameter's key, you must add a `URL.queries` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.queries.<query-key-1>" : "<query-value-1>", ... "URL.queries.<query-key-N>": "<query-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.
	`tokenService.addClientCredentialsInBody`	Specifies whether the client credentials should be placed in the request body of the token request, rather than the `Authorization` header. Default is **`true`**.

Back to Content [page 1395]

Example: AuthTokens Object Response

The response for "find destination" contains an `authTokens` object in the format given below. For more information on the fields in `authTokens`, see "Find a Destination" Response Structure [page 1565].

Sample Code

```
"authTokens": [
    {
        "type": "Bearer",
        "value": "eyJhbGciOiJSUzI1NiIsInR5cC...",
        "http_header": {
            "key":"Authorization",
            "value":"Bearer eyJhbGciOiJSUzI1NiIsInR5cC..."
        }
    }
]
```

Back to Content [page 1395]

1.3.1.2.1.1.11 SAML Assertion Authentication

Create and configure an *SAML Assertion* destination for an application.

Context

The Destination service lets you generate SAML assertions as per SAML 2.0 specification. You can retrieve a generated SAML assertion from the Destination service by using the `SAMLAssertion` authentication type, whereas OAuth SAML Bearer Assertion Authentication [page 1363] sends the generated SAML assertion to an OAuth server to get a token. The Destination service provides functionality for caching the generated SAML assertion for later use, and caching by the app whenever needed, which helps simplifying application development.

> ⚠ Caution
>
> For SAP on-premise systems, this feature is supported as of S/4HANA (on-premise) release 2020.

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Name of the destination. Must be unique for the destination level.
Type	`Type`	Destination type. Choose **`HTTP`** for all HTTP(S) destinations.
URL	`URL`	URL of the target endpoint.
Proxy Type	`ProxyType`	Choose **`Internet`**, **`PrivateLink`**, or **`OnPremise`**.
Authentication	`Authentication`	Authentication type. Use **`SAMLAssertion`** as value.

Cockpit Label	JSON Key	Description
Audience	`audience`	Value of the `Audience` tag, which is part of the generated SAML assertion. For more information, see SAML 2.0 specification .
AuthnContextClassRef	`authnContextClassRef`	Value of the `AuthnContextClassRef` tag, which is part of generated SAML assertion. For more information, see SAML 2.0 specification .
Optional		
Location ID	`CloudConnectorLocationId`	(only if `ProxyType=OnPremise`) Starting with Cloud Connector 2.9.0, you can connect multiple Cloud Connectors to an account, as long as their location ID is different. The value defines the location ID identifying the Cloud Connector over which the connection is opened. The default value is an empty string identifying the Cloud Connector that is connected without any location ID, which is also the case for all Cloud Connector versions prior to 2.9.0.
Client Key	`clientKey`	Key that identifies the consumer to the authorization server.
Name Qualifier	`nameQualifier`	When this property is set, the `NameQualifier` under the `NameId` tag of the generated SAML assertion is determined in accordance to the value.
Assertion Issuer	`assertionIssuer`	Issuer of the SAML assertion.
Assertion Recipient	`assertionRecipient`	Recipient of the SAML assertion.
Name Id Format	`nameIdFormat`	Value of the `NameIdFormat` tag, which is part of generated SAML Assertion. For more information, see SAML 2.0 specification .
User Id Source	`userIdSource`	When this property is set, the user ID in the `NameId` tag of the generated SAML assertion is determined in accordance to the value of this attribute. For more information, see User Propagation via SAML 2.0 Bearer Assertion Flow [page 1558].

Cockpit Label	JSON Key	Description
Key Store Location	`KeyStoreLocation`	Contains the name of the certificate configuration to be used for *per-destination* SAML assertion signing. This certificate will be used instead of the standard subaccount-wide signing key. For more information, see Manage Trust [page 1514].
Key Store Password	`KeyStorePassword`	Contains the password for the certificate configuration (if one is needed) when using a *per-destination* SAML assertion signing certificate.
Skip User Attributes Prefix In SAML Attribute	`skipUserAttributesPrefixInSAMLAttributes`	If set to true, any additional attributes taken from the OAuth server's user information endpoint, under the *user_attributes* section, will be added to the assertion without the prefix that the Destination service would usually add to them. For more information, see User Propagation via SAML 2.0 Bearer Assertion Flow [page 1558].
Include Signing Certificate In SAML Assertion	`includeSigningCertificateInSAMLAssertion`	If set to true, the public part of the signing certificate, used for signing the SAML assertion, will be included in the XML structure of the assertion. Some assertion recipients may require this to accept the assertion.
Skip User Uuid In SAML Attributes	`skipUserUuidInSAMLAttributes`	If set to true, any SAML assertion attribute with name 'user_uuid' will not be added to the resulting SAML assertion XML, even if such was found in the JWT, specifying the user's identity.
Additional		
	Key	**Description**
	`companyId`	Company identifier.
	`x_user_token.jwks`	Base64-encoded *JSON web key set*, containing the signing keys which are used to validate the JWT provided in the *X-User-Token* header. For more information, see JWK Set Format .

Cockpit Label	JSON Key	Description
	`x_user_token.header.typ`	Specifies what is the token type of the JWT (JSON web token) provided in the `X-User-Token` header, which can be found in the decoded JWT header's "typ" JWT claim. This property is required when using user tokens which have not been generated by XSUAA. If this property is not provided, the Destination service will default to value `JWT` during runtime.
	`x_user_token.jwks_uri`	URI of the *JSON web key set*, containing the signing keys which are used to validate the JWT provided in the *X-User-Token* header. **⚠ Restriction** If the value is a private endpoint (for example, *localhost*), the Destination service will not be able to perform the verification of the X-User-Token header when using the "Find Destination" API. For more information, see OpenID Connect Discovery .

Cockpit Label	JSON Key	Description
	`URL.headers.<header-key>`	A static key prefix used as a namespace grouping of the URL's HTTP headers whose values will be sent to the target endpoint. For each HTTP header's key, you must add a `URL.headers` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.headers.<header-key-1>" : "<header-value-1>", ... "URL.headers.<header-key-N>": "<header-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.

Cockpit Label	JSON Key	Description
	`URL.queries.<query-key>`	A static key prefix used as a namespace grouping of URL's query parameters whose values will be sent to the target endpoint. For each query parameter's key, you must add a `URL.queries` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.queries.<query-key-1>" : "<query-value-1>", ... "URL.queries.<query-key-N>": "<query-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.
	`samlAssertionEncoding`	If set, this property controls what encoding will be used to encode the assertion XML prior to sending it to the configured token service. The possible values are "base64" (RFC 4648, section 4) and "base64url" (RFC 4648, section 5). If not set, the encoding will be "base64".

Example

The connectivity destination below provides HTTP access to the OData API of the SuccessFactors Jam.

```
Name=destinationSamlAssertion
Type=HTTP
URL=https://myXXXXXX-api.s4hana.ondemand.com
Authentication=SAMLAssertion
ProxyType=Internet
audience=https://myXXXXXX.s4hana.ondemand.com
authnContextClassRef=urn:oasis:names:tc:SAML:2.0:ac:classes:X509
```

The response for "find destination" contains an `authTokens` object in the format given below. For more information on the fields in `authTokens`, see "Find a Destination" Response Structure [page 1565].

Sample Code

```
"authTokens": [
    {
        "type": "SAML2.0",
        "value": "PD94bWwgdmVyc2lvbj0iMS4wIiBlbmNvZ...",
        "http_header": {
            "key":"Authorization",
            "value":"SAML2.0 PD94bWwgdmVyc2lvbj0iMS4wIiBlbmNvZ..."
        }
    }
]
```

Related Information

Create HTTP Destinations [page 1476]
Destination Examples [page 1486]
Exchanging User JWTs via OAuth2JWTBearer Destinations [page 1579]

1.3.1.2.1.1.12 OAuth with X.509 Client Certificates

Use an X.509 certificate instead of a secret to authenticate against the authentication server.

Caution

OAuth with X.509 is only supported for `<ProxyType>=Internet`.

To perform mutual TLS, you can use an X.509 client certificate instead of a client secret when connecting to the authorization server. To do so, you must create a certificate configuration containing a valid X.509 client certificate or a keystore, and link it to the destination configuration using these properties:

Properties

Cockpit Label	JSON Key	Description
Optional		
Token Service Key Store Location	`tokenService.KeyStoreLocation`	Contains the name of the certificate configuration to be used. This property is required.
Token Service Key Store Password	`tokenService.KeyStorePassword`	Contains the password for the certificate configuration (if one is needed).

> ⚠ Caution
>
> Mutual TLS with an X.509 client certificate is performed only if the `tokenService.KeyStoreLocation` property is set in the destination configuration. Otherwise, the client secret is used.

Supported Certificate Configuration Formats

- Java Keystore (.jks): Requires the `tokenService.KeyStorePassword` property.
- PKCS12 (.pfx or .p12): Requires the `tokenService.KeyStorePassword` property.
- PEM-encoded X.509 client certificate and private key (.crt, .cer and .pem): The certificate configuration can contain several valid X.509 certificates and private keys.

Supported OAuth Flows

- OAuth Client Credentials Authentication [page 1375]
- OAuth Password Authentication [page 1391]
- OAuth User Token Exchange Authentication [page 1384]
- OAuth JWT Bearer Authentication [page 1395]
- OAuth SAML Bearer Assertion Authentication [page 1363]

1.3.1.2.1.1.13 OAuth Refresh Token Authentication

Create and configure an *OAuth refresh token* destination for an application.

Overview

SAP BTP provides support for applications to use the *OAuth2 refresh token* flow for consuming OAuth-protected resources.

Refresh tokens are a common way to maintain certain levels of access, without requiring the use of credentials for getting a new access token. They have a longer validity compared to access tokens and can be used to fetch brand new access tokens without performing again the original flow.

The client credentials and a refresh token are used to request an access token from an OAuth server, referred to below as *token service*. This is automatically performed by the Destination service when using the "Find a destination" REST endpoint.

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Name of the destination. It must be the same as the destination name you use for the configuration tools, that is, the console client and *Destinations* editor (cockpit).
Type	`Type`	Destination type. Choose **`HTTP`** for all HTTP(S) destinations.
URL	`URL`	The URL of the protected target resource.
Proxy Type	`ProxyType`	Choose **`Internet`** or **`OnPremise`**. If **`OnPremise`** is used, the OAuth server must be accessed through the Cloud Connector.
Authentication	`Authentication`	Authentication type. Use **`OAuth2RefreshToken`** as value.
Client ID	`clientId`	Client ID used to retrieve the access token.
Token Service URL	`tokenServiceURL`	Token retrieval URL of the OAuth server.
Optional		
Client Secret	`clientSecret`	Client secret for the Client ID.

Cockpit Label	JSON Key	Description
Token Service URL Type	`tokenServiceURLType`	Either **`Dedicated`** (if the `tokenServiceURL` serves only a single tenant), or **`Common`** (if the `tokenServiceURL` serves multiple tenants).
Token Service URL Proxy Type	`tokenServiceURL.ProxyType`	If set, it will use the specified `ProxyType` for the token service, independent from the `ProxyType` of the whole destination. This lets you configure Internet-based token services for on-premise systems.
Token Service Key Store Location	`tokenService.KeyStoreLocation`	Contains the name of the certificate configuration to be used. This property is required when using client certificates for authentication. See OAuth with X.509 Client Certificates [page 1408].
Token Service Key Store Password	`tokenService.KeyStorePassword`	Contains the password for the certificate configuration (if one is needed) when using client certificates for authentication. See OAuth with X.509 Client Certificates [page 1408].
Additional		
	Key	**Description**
	`scope`	Value of the OAuth 2.0 `scope` parameter, expressed as a list of space-delimited, case-sensitive strings.

Cockpit Label	JSON Key	Description
	`tokenServiceURL.headers.<header-key>`	Static key prefix used as a namespace grouping of the `tokenServiceUrl`'s HTTP headers. Its values will be sent to the token service during token retrieval. For each HTTP header's key you must add a *'tokenServiceURL.headers'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.headers.<header-key-1>" : "<header-value-1>", ... "tokenServiceURL.headers.<header-key-N>": "<header-value-N>", }`
	`tokenServiceURL.queries.<query-key>`	Static key prefix used as a namespace grouping of `tokenServiceUrl`'s query parameters. Its values will be sent to the token service during token retrieval. For each query parameter's key you must add a *'tokenServiceURL.queries'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.queries.<query-key-1>" : "<query-value-1>", ... "tokenServiceURL.queries.<query-key-N>": "<query-value-N>", }`

Cockpit Label	JSON Key	Description
	`tokenService.body.<param-key>`	A static key prefix used as a name-space grouping of parameters which are sent as part of the token re-quest to the token service during token retrieval. For each request, a `tokenService.body` prefix must be added to the parameter key, sepa-rated by dot-delimiter. For example: Sample Code `{ ... "tokenService.body.<param-key-1>" : "<param-value-1>", ... "tokenService.body.<param-key-N>": "<param-value-N>", }`
	`tokenService.addClientCredentialsInBody`	Specifies whether the client credentials should be placed in the request body of the token request, rather than the `Authorization` header. Default is **`true`**.

Cockpit Label	JSON Key	Description
	`URL.headers.<header-key>`	Static key prefix used as a namespace grouping of the URL's HTTP headers whose values will be sent to the target endpoint. For each HTTP header's key, you must add a `URL.headers` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.headers.<header-key-1>" : "<header-value-1>", ... "URL.headers.<header-key-N>": "<header-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.
	`tokenServiceURL.ConnectionTimeoutInSeconds`	Defines the connection timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 60 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.
	`tokenServiceURL.SocketReadTimeoutInSeconds`	Defines the read timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 600 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.

Cockpit Label	JSON Key	Description
	`URL.queries.<query-key>`	Static key prefix used as a namespace grouping of URL's query parameters whose values will be sent to the target endpoint. For each query parameter's key, you must add a `URL.queries` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.queries.<query-key-1>" : "<query-value-1>", ... "URL.queries.<query-key-N>": "<query-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.

Note

When the OAuth server is called, the caller side trusts the server based on the trust settings of the destination. For more information, see Server Certificate Authentication [page 1356].

Example: OAuth2 RefreshToken Destination

Sample Code

```
{
```

```
    "Name": "SapOAuthPassGrant",
    "Type": "HTTP",
    "URL": "https://myapp.cfapps.sap.hana.ondemand.com/mypath",
    "ProxyType": "Internet",
    "Authentication": "OAuth2RefreshToken",
    "clientId": "my-client-id",
    "clientSecret": "my-client-pass",
    "tokenServiceURL": "https://authentication.sap.hana.ondemand.com/oauth/
token"
}
```

Calling "Find Destination"

When calling the destination an *X-refresh-token* is a required header parameter.

Curl call example

Sample Code

```
curl --location --request
GET 'https://<destination>.<environment>.hanavlab.ondemand.com/destination-
configuration/v1/destinations/<destination-name>' \
--header 'X-refresh-token: <refresh_token>' \ # mandatory parameter
--header 'Authorization: Bearer <destination_token>'
```

The response for *Find Destination* will contain an `authTokens` object in the format given below. For more information on the fields in `authTokens`, see "Find a Destination" Response Structure [page 1565].

Sample Code

```
"authTokens": [
    {
        "type": "Bearer",
        "value": "eyJhbGciOiJSUzI1NiIsInR5cC...",
        "http_header": {
            "key":"Authorization",
            "value":"Bearer eyJhbGciOiJSUzI1NiIsInR5cC..."
        }
    }
]
```

1.3.1.2.1.1.14 OAuth Authorization Code Authentication

Create and configure an *OAuth Authorization Code* destination for an application.

Overview

The *OAuth Authorization Code* flow is a standard mechanism for business user login. It is a two-step procedure. In a first step, business users authenticate themselves towards an authorization server, which grants users an authorization code. The second step exchanges the authorization code for an access token through a token service. Applications can use this flow to access OAuth-protected resources.

The client credentials and an authorization code are used to request an access token from an OAuth server, referred to below as *token service*. This is performed automatically by the Destination service when using the "Find a destination" REST endpoint.

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Name of the destination. It must be the same as the destination name you use for the configuration tools, that is, the console client and *Destinations* editor (cockpit).
Type	`Type`	Destination type. Choose **`HTTP`** for all HTTP(S) destinations.
URL	`URL`	The URL of the protected target resource.
Proxy Type	`ProxyType`	Choose **`Internet`** or **`OnPremise`**. If **`OnPremise`** is used, the OAuth server must be accessed through the Cloud Connector.
Authentication	`Authentication`	Authentication type. Use **`OAuth2AuthorizationCode`** as value.
Client ID	`clientId`	Client ID of the application.
Token Service URL	`tokenServiceURL`	Token retrieval URL of the OAuth server.
Optional		
Client Secret	`clientSecret`	Client secret for the Client ID.
Token Service URL Type	`tokenServiceURLType`	Either **`Dedicated`** (if the `tokenServiceURL` serves only a single tenant), or **`Common`** (if the `tokenServiceURL` serves multiple tenants).

Cockpit Label	JSON Key	Description
Token Service URL Proxy Type	`tokenServiceURL.ProxyType`	If set, it will use the specified `ProxyType` for the token service, independent from the `ProxyType` of the whole destination. This lets you configure Internet-based token services for on-premise systems.
Token Service Key Store Location	`tokenService.KeyStoreLocation`	Contains the name of the certificate configuration to be used. This property is required when using client certificates for authentication. See OAuth with X.509 Client Certificates [page 1408].
Token Service Key Store Password	`tokenService.KeyStorePassword`	Contains the password for the certificate configuration (if one is needed) when using client certificates for authentication. See OAuth with X.509 Client Certificates [page 1408].
Additional		
	Key	**Description**
	`scope`	Value of the OAuth 2.0 `scope` parameter, expressed as a list of space-delimited, case-sensitive strings.
	`tokenServiceURL.headers.<header-key>`	Static key prefix used as a namespace grouping of the `tokenServiceUrl`'s HTTP headers. Its values will be sent to the token service during token retrieval. For each HTTP header's key you must add a *'tokenServiceURL.headers'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.headers.<header-key-1>" : "<header-value-1>", ... "tokenServiceURL.headers.<header-key-N>": "<header-value-N>", }`

Cockpit Label	JSON Key	Description
	`tokenServiceURL.ConnectionTimeoutInSeconds`	Defines the connection timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 60 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.
	`tokenServiceURL.SocketReadTimeoutInSeconds`	Defines the read timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 600 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.
	`tokenServiceURL.queries.<query-key>`	Static key prefix used as a namespace grouping of `tokenServiceUrl`'s query parameters. Its values will be sent to the token service during token retrieval. For each query parameter's key you must add a *'tokenServiceURL.queries'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.queries.<query-key-1>" : "<query-value-1>", ... "tokenServiceURL.queries.<query-key-N>": "<query-value-N>", }`

Cockpit Label	JSON Key	Description
	`tokenService.body.<param-key>`	A static key prefix used as a name-space grouping of parameters which are sent as part of the token re-quest to the token service during token retrieval. For each request, a `tokenService.body` prefix must be added to the parameter key, sepa-rated by dot-delimiter. For example: **Sample Code** `{ ... "tokenService.body.<param-key-1>" : "<param-value-1>", ... "tokenService.body.<param-key-N>": "<param-value-N>", }`
	`tokenService.addClientCredentialsInBody`	Specifies whether the client credentials should be placed in the request body of the token request, rather than the `Authorization` header. Default is **`true`**.
	`clientAssertion.destinationName`	Name of the destination that provides client assertions when using client assertion authentication mechanism. Must be on the same subaccount or service instance as this destination. This is used in case of automated client assertion fetching by the service. For more information, see Client Assertion with Automated Assertion Fetching by the Service [page 1440].

Cockpit Label	JSON Key	Description
	`URL.headers.<header-key>`	Static key prefix used as a namespace grouping of the URL's HTTP headers whose values will be sent to the target endpoint. For each HTTP header's key, you must add a `URL.headers` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.headers.<header-key-1>" : "<header-value-1>", ... "URL.headers.<header-key-N>": "<header-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.

Cockpit Label	JSON Key	Description
	`URL.queries.<query-key>`	Static key prefix used as a namespace grouping of URL's query parameters whose values will be sent to the target endpoint. For each query parameter's key, you must add a `URL.queries` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.queries.<query-key-1>" : "<query-value-1>", ... "URL.queries.<query-key-N>": "<query-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.

Example: OAuth2 Authorization Code Destination

Sample Code

```
{
    "Name": "SapOAuth2AuthorizationCodeDestination",
    "Type": "HTTP",
    "URL": "https://myapp.cfapps.sap.hana.ondemand.com/mypath",
    "ProxyType": "Internet",
    "Authentication": "OAuth2AuthorizationCode",
    "clientId": "my-client-id",
    "clientSecret": "my-client-pass",
```

```
    "tokenServiceURL": "https://authentication.sap.hana.ondemand.com/oauth/
token"
}
```

Calling "Find Destination"

When calling the destination, an *X-code* is a required header parameter. *X-redirect-uri* and *X-code-verifier* are optional header parameters. They depend on the call for the authorization code fetch. If a redirect URI was specified in that call, the same redirect URI must be used as value for the *X-redirect-uri* header. If a code challenge was presented in the authorization code fetch request, a code verifier must be given as value for the *X-code-verifier* header.

Curl call example

Sample Code

```
curl --location --request
GET 'https://<destination>.<environment>.hanavlab.ondemand.com/destination-
configuration/v1/destinations/<destination-name>' \
--header 'X-code: <authorization_code>' \ # mandatory parameter
--header 'X-redirect-uri: <redirect_uri>' \ # optional parameter
--header 'X-code-verifier: <code_verifier>' \ # optional parameter
--header 'Authorization: Bearer <destination_token>'
```

The response for *Find Destination* will contain an `authTokens` object in the format given below. For more information on the fields in `authTokens`, see "Find a Destination" Response Structure [page 1565].

Sample Code

```
"authTokens": [
    {
        "type": "Bearer",
        "value": "eyJhbGciOiJSUzI1NiIsInR5cC...",
        "http_header": {
            "key":"Authorization",
            "value":"Bearer eyJhbGciOiJSUzI1NiIsInR5cC..."
        }
    }
]
```

1.3.1.2.1.1.15 OAuth Technical User Propagation Authentication

Learn about the *OAuth2TechnicalUserPropagation* authentication type for HTTP destinations: use cases, supported properties and examples.

Overview

SAP BTP supports the propagation of technical users from the cloud application towards on-premise systems. In the Destination service, an access token representing the technical user is retrieved, which can then be sent in a header to the Connectivity service. This is similar to *principal propagation*, but in this case, a technical user is propagated instead of a business user.

The retrieval of the access token performs the OAuth 2.0 client credentials flow, according to the token service configurations in the destination. The token service is called from the Internet, not from the Cloud Connector.

> **Note**
>
> The retrieved access token is cached for the duration of its validity.

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Name of the destination. It must be the same as the destination name you use for the configuration tools, that is, the console client and *Destinations* editor (cockpit).
Type	`Type`	Destination type. Choose **HTTP** for all HTTP(S) destinations.
URL	`URL`	The URL of the protected target resource.

Cockpit Label	JSON Key	Description
Proxy Type	`ProxyType`	You can only use proxy type **`OnPremise`**. → Remember The token service is not accessed through the Cloud Connector, but through the Internet.
Authentication	`Authentication`	Authentication type. Use **`OAuth2TechnicalUserPropagation`** as value.
Client ID	`clientId`	Client ID of the application.

Cockpit Label	JSON Key	Description
Token Service URL	`tokenServiceURL`	The URL of the token service, against which the client credentials flow is performed. Depending on the `Token Service URL Type`, this property is interpreted in different ways during the automatic token retrieval: • For **`Dedicated`**, the token service URL is taken as is. • For **`Common`**, the token service URL is searched for the tenant placeholder `{tenant}`. `{tenant}` is resolved as the subdomain of the subaccount on behalf of which the client credentials flow should be done, based on the `x-tenant` header provided during the call. If the placeholder is not found, `{tenant}` is inserted as a subdomain of the token service URL. Consult the Destination Service REST API [page 1497] to see how the subdomain of the subaccount is specified. The subaccount subdomain is mandated during creation of the subaccount, see Create a Subaccount. **Examples** of interpreting the `tokenServiceURL` for `tokenServiceURLType`=**`Common`**, if the call to the Destination service is done with the `x-tenant` header with value **`mytenant`**: • `https://authentication.us10.hana.ondemand.com/oauth/token` → `https://mytenant.authentication.us10.hana.ondemand.com/oauth/token` • `https://{tenant}.authentication.us10.hana.ondemand.com`

Cockpit Label	JSON Key	Description
		`/oauth/token` → `https://mytenant.authentication.us10.hana.ondemand.com/oauth/token` • `https://authentication.myauthserver.com/tenant/{tenant}/oauth/token` → `https://authentication.myauthserver.com/tenant/mytenant/oauth/token` • `https://oauth.{tenant}.myauthserver.com/token` → `https://oauth.mytenant.myauthserver.com/token`
Optional		
Client Secret	`clientSecret`	Client secret for the Client ID.
Token Service User	`tokenServiceUser`	User for basic authentication to the OAuth server (if required).
Token Service Password	`tokenServicePassword`	Password for `tokenServiceUser` (if required).
Token Service URL Type	`tokenServiceURLType`	Either **`Dedicated`** (if the `tokenServiceURL` serves only a single tenant), or **`Common`** (if the `tokenServiceURL` serves multiple tenants).
Token Service Key Store Location	`tokenService.KeyStoreLocation`	Contains the name of the certificate configuration to be used. This property is required when using client certificates for authentication. See OAuth with X.509 Client Certificates [page 1408].
Token Service Key Store Password	`tokenService.KeyStorePassword`	Contains the password for the certificate configuration (if one is needed) when using client certificates for authentication. See OAuth with X.509 Client Certificates [page 1408].
Additional		
	Key	**Description**

Cockpit Label	JSON Key	Description
	`scope`	Value of the OAuth 2.0 `scope` parameter, expressed as a list of space-delimited, case-sensitive strings.
	`tokenServiceURL.headers.<header-key>`	Static key prefix used as a namespace grouping of the `tokenServiceUrl`'s HTTP headers. Its values will be sent to the token service during token retrieval. For each HTTP header's key you must add a *'tokenServiceURL.headers'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.headers.<header-key-1>" : "<header-value-1>", ... "tokenServiceURL.headers.<header-key-N>": "<header-value-N>", }`
	`tokenServiceURL.ConnectionTimeoutInSeconds`	Defines the connection timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 60 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.
	`tokenServiceURL.SocketReadTimeoutInSeconds`	Defines the read timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 600 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.

Cockpit Label	JSON Key	Description
	`tokenServiceURL.queries.<query-key>`	Static key prefix used as a namespace grouping of `tokenServiceUrl`'s query parameters. Its values will be sent to the token service during token retrieval. For each query parameter's key you must add a *'tokenServiceURL.queries'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.queries.<query-key-1>" : "<query-value-1>", ... "tokenServiceURL.queries.<query-key-N>": "<query-value-N>", }`
	`tokenService.body.<param-key>`	A static key prefix used as a namespace grouping of parameters which are sent as part of the token request to the token service during token retrieval. For each request, a `tokenService.body` prefix must be added to the parameter key, separated by dot-delimiter. For example: **Sample Code** `{ ... "tokenService.body.<param-key-1>" : "<param-value-1>", ... "tokenService.body.<param-key-N>": "<param-value-N>", }`

Cockpit Label	JSON Key	Description
	`tokenService.addClientCredentialsInBody`	Specifies whether the client credentials should be placed in the request body of the token request, rather than the `Authorization` header. Default is **`true`**. **Note** If set to **`false`**, but `tokenServiceUser` / `tokenServicePassword` are also set, `tokenServiceUser` / `tokenServicePassword` will be taken with priority.
	`clientAssertion.destinationName`	Name of the destination that provides client assertions when using client assertion authentication mechanism. Must be on the same subaccount or service instance as this destination. This is used in case of automated client assertion fetching by the service. For more information, see Client Assertion with Automated Assertion Fetching by the Service [page 1440].

Cockpit Label	JSON Key	Description
	`URL.headers.<header-key>`	Static key prefix used as a namespace grouping of the URL's HTTP headers whose values will be sent to the target endpoint. For each HTTP header's key, you must add a `URL.headers` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.headers.<header-key-1>" : "<header-value-1>", ... "URL.headers.<header-key-N>": "<header-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.

Cockpit Label	JSON Key	Description
	`URL.queries.<query-key>`	Static key prefix used as a namespace grouping of URL's query parameters whose values will be sent to the target endpoint. For each query parameter's key, you must add a `URL.queries` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.queries.<query-key-1>" : "<query-value-1>", ... "URL.queries.<query-key-N>": "<query-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.

Note

When the OAuth authorization server is called, it accepts the trust settings of the destination. For more information, see Server Certificate Authentication [page 1356].

Caution

When using the OAuth service of the SAP BTP *Neo* environment (*https://api.{landscape-domain}/oauth2/apitoken/v1?grant_type=client_credentials or oauthasservices.{landscape-domain}/oauth2/apitoken/v1?grant_type=client_credentials*) as `TokenServiceURL`, or any other OAuth token service which accepts client credentials **only as Authorization header**, you must also set the `clientId` and `clientSecret` values for `tokenServiceUser` and `tokenServicePassword` properties.

Example: OAuth Technical User Propagation Destination

Sample Code

```
Name=technical-user-example
Type=HTTP
URL=http://protected-url.example.com
ProxyType=OnPremise
Authentication=OAuth2TechnicalUserPropagation
clientId=clientId
clientSecret=secret1234
tokenServiceURL=http://authserver.example.com/oauth/token
```

Example: authTokens Object in *Find Destination* Response

The response for *Find Destination* will contain an `authTokens` object in the format given below. For more information on the fields in `authTokens`, see "Find a Destination" Response Structure [page 1565].

For usage of the `SAP-Connectivity-Technical-Authentication` header, see Authentication Types [page 1323].

Sample Code

```
"authTokens": [
    {
        "type": "Bearer",
        "value": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiI...",
        "http_header": {
            "key":"SAP-Connectivity-Technical-Authentication",
            "value":"Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiI..."
        }
    }
]
```

1.3.1.2.1.1.16 OAuth Token Exchange Authentication

Learn about the `OAuth2TokenExchange` authentication type for HTTP destinations: use cases, supported properties and ways to retrieve an access token in an automated way.

Overview

To allow an application to call another application, passing the principal context, and fetch resources, the caller application needs to pass a security token. In this authorization flow, the initial security token is passed to the OAuth server as input data. This process is performed automatically by the Destination service, which helps simplify the application development - you only have to construct the right request to the target URL, by using the outcome (another access token) of the service side automation.

With this token exchange flow, any type of token can be used, provided that it is supported by the configured token service.

Properties

To configure a destination of this type, you must specify all the required properties.

> **⚠ Restriction**
>
> `OAuth2TokenExchange` destinations don't support dynamic tenant-specific token service URLs. The token service URL is always taken as is.

The table below lists the destination properties needed for the creation of destination of `OAuth2TokenExchange` authentication type.

Cockpit Label	JSON Key	Input/Description
Required		
Name	`Name`	Name of the destination. Must be unique for the destination level.
Type	`Type`	Choose **`HTTP`** (for HTTP or HTTPS communication).
URL	`URL`	URL of the target endpoint.
Proxy Type	`ProxyType`	You can only use proxy type **`Internet`** or **`OnPremise`**. If **`OnPremise`** is used, the OAuth server must be accessed through the Cloud Connector.
Authentication	`Authentication`	Use **`OAuth2TokenExchange`**.
Client ID	`clientId`	OAuth 2.0 client ID to be used for the security access token exchange.
Token Service URL	`tokenServiceURL`	The URL of the token service, against which the token exchange is performed. **ⓘ Note** Tenant-specific URLs are not supported. The token service URL will be taken as is.
Optional		

Cockpit Label	JSON Key	Input/Description
Description	`Description`	Human-readable description of the destination.
Client Secret	`clientSecret`	OAuth 2.0 client secret to be used for the security access token exchange.
Token Service URL Proxy Type	`tokenServiceURL.ProxyType`	If set, it will use the specified `ProxyType` for the token service, independent from the `ProxyType` of the whole destination. This lets you configure Internet-based token services for on-premise systems.
Token Service Key Store Location	`tokenService.KeyStoreLocation`	Contains the name of the certificate configuration to be used. This property is required when using client certificates for authentication. For more information, see OAuth with X.509 Client Certificates [page 1408].
Token Service Key Store Password	`tokenService.KeyStorePassword`	Contains the password for the certificate configuration (if one is needed) when using client certificates for authentication. For more information, see OAuth with X.509 Client Certificates [page 1408].
Additional		
	Key	**Description**
	`scope`	Value of the OAuth 2.0 scope parameter expressed as a list of space-delimited, case-sensitive strings.

	`tokenServiceURL.headers.<header-key>`	A static key prefix used as a namespace grouping of the `tokenServiceUrl`'s HTTP headers. Its values will be sent to the token service during token retrieval. For each HTTP header's key you must add a *'tokenServiceURL.headers'* prefix separated by dot delimiter. For example:
		
		Sample Code
		
		`{`
		`  ...`
		`  "tokenServiceURL.headers.<header-key-1>" : "<header-value-1>",`
		`  ...`
		`  "tokenServiceURL.headers.<header-key-N>": "<header-value-N>",`
		`}`

Cockpit Label	JSON Key	Input/Description
	`tokenServiceURL.queries.<query-key>`	A static key prefix used as a namespace grouping of `tokenServiceUrl`'s query parameters. Its values will be sent to the token service during token retrieval. For each query paramester's key you must add a *'tokenServiceURL.queries'* prefix separated by dot delimiter. For example: **Sample Code** `{ ... "tokenServiceURL.queries.<query-key-1>" : "<query-value-1>", ... "tokenServiceURL.queries.<query-key-N>": "<query-value-N>", }`
	`tokenService.body.<param-key>`	A static key prefix used as a namespace grouping of parameters which are sent as part of the token request to the token service during token retrieval. For each request, a `tokenService.body` prefix must be added to the parameter key, separated by dot-delimiter. For example: **Sample Code** `{ ... "tokenService.body.<param-key-1>" : "<param-value-1>", ... "tokenService.body.<param-key-N>": "<param-value-N>", }`

Cockpit Label	JSON Key	Input/Description
	`URL.headers.<header-key>`	A static key prefix used as a namespace grouping of the URL's HTTP headers whose values will be sent to the target endpoint. For each HTTP header's key, you must add a `URL.headers` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.headers.<header-key-1>" : "<header-value-1>", ... "URL.headers.<header-key-N>": "<header-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.
	`URL.queries.<query-key>`	A static key prefix used as a namespace grouping of URL's query parameters whose values will be sent to the target endpoint. For each query parameter's key, you must add a `URL.queries` prefix separated by dot-delimiter. For example: **Sample Code** `{ ... "URL.queries.<query-key-1>" : "<query-value-1>", ... "URL.queries.<query-key-N>": "<query-value-N>", }` **Note** This is a naming convention. As the call to the target endpoint is performed on the client side, the service only provides the configured properties. The expectation for the client-side processing logic is to parse and use them. If you are using higher-level libraries and tools, please check if they support this convention.
	`tokenService.addClientCredentialsInBody`	Specifies whether the client credentials should be placed in the request body of the token request, rather than the `Authorization` header. Default is **`true`**.

Cockpit Label	JSON Key	Input/Description
	`tokenServiceURL.ConnectionTimeoutInSeconds`	Defines the connection timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 60 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.
	`tokenServiceURL.SocketReadTimeoutInSeconds`	Defines the read timeout for the token service retrieval. The minimum value allowed is 0, the maximum is 600 seconds. If the value exceeds the allowed number, the default value (10 seconds) is used.
	`clientAssertion.destinationName`	Name of the destination that provides client assertions when using the *client assertion authentication* mechanism. Must be on the same subaccount or service instance as this destination. This key is used in case of automated client assertion fetching by the service. For more information, see Client Assertion with Automated Assertion Fetching by the Service [page 1440].

> **Note**
>
> To find or consume a destination via the Destination service REST API, the following headers are mandatory for this authentication type:
>
> - `X-Subject-Token`
> - `X-Subject-Token-Type`
>
> For more information, see the Destination service REST API documentation.

Example: AuthTokens Object Response

The response for "find destination" contains an `authTokens` object in the format given below. For more information on the fields in `authTokens`, see "Find a Destination" Response Structure [page 1565].

> **Sample Code**

```
"authTokens": [
    {
        "type": "Bearer",
        "value": "eyJhbGciOiJSUzI1NiIsInR5cC...",
        "issued_token_type": "urn:ietf:params:oauth:token-type:access_token",
        "http_header": {
            "key":"Authorization",
            "value":"Bearer eyJhbGciOiJSUzI1NiIsInR5cC..."
        }
    }
]
```

1.3.1.2.1.1.17 Using Client Assertion with OAuth Flows

Replace client secrets with client assertions in OAuth flows for destinations.

Client assertion is one of the possible client authentication mechanisms when requesting an access token from an authorization server. When using client assertion, you must provide the `client_assertion` and `client_assertion_type` parameters in the request to an authorization server that supports client assertion for client authentication.

The Destination service supports client assertion as a client authentication mechanism. It sends the client assertion to the authorization server whenever requesting an OAuth token. The assertion can be either generated externally and provided to the Destination service, or it can be retrieved by the Destination service via an additional destination.

For more information, see

- RFC 7521
- Provide Client Assertion Properties as Headers [page 1439].

Prerequisites

You must ensure that the target authorization server supports client assertion as authentication mechanism and accepts the provided client assertions after validation. This might require additional setup in the authorization server and the assertion provider.

1.3.1.2.1.1.17.1 Provide Client Assertion Properties as Headers

Provide client assertion properties as headers when using client assertion with OAuth flows for a destination.

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Additional Properties

Header	Value
`X-client-assertion-type`	Absolute URI
`X-client-assertion`	Token

Example of Client Assertion Properties as Headers

To use the client assertion mechanism in the *Find Destination* API, you must add two mandatory headers as shown in the following example:

Sample Code

```
curl --location --request GET 'https://<destination_service_host>/destination-
configuration/v1/destinations/<destination-name>' \
--header 'X-client-assertion-type: urn:ietf:params:oauth:client-assertion-
type:jwt-bearer' \ # mandatory parameter
--header 'X-client-assertion: eyJraWQiOiJKMWpC...' \ # mandatory parameter
--header 'Authorization: Bearer <access_token>'
```

1.3.1.2.1.1.17.2 Client Assertion with Automated Assertion Fetching by the Service

Use the *Find Destination* API to fetch client assertions automatically when using client assertion with OAuth flows for a destination.

The *Find Destination* API lets you fetch client assertions automatically and then use them for retrieving tokens from authorization servers that accept client assertions as a client authentication mechanism.

To apply this mechanism, you must use the following of destination configurations:

- Destination that provides the client assertion - with specified token service that issues client assertions
- Destination that uses the client assertion - with specified token service that uses client assertion as a client authentication mechanism

Caution

Only *OAuth2ClientCredentials* authentication type is allowed for destinations that provide client assertions.The following authentication types are supported for destinations that use client assertions: *OAuth2Password*, *OAuth2ClientCredentials*, *OAuth2AuthorizationCode*, *OAuth2TechnicalUserPropagation*, *OAuth2TokenExchange*.

Define the Client Assertion Type

The client assertion type must be defined as a property in the destination that provides the client assertion:

Property	Value	Description
`clientAssertion.type`	Absolute URI	The format of the assertion as defined by the authorization server. The supported values are: • "urn:ietf:params:oauth:client-assertion-type:saml2-bearer" => indicating a SAML Bearer Assertion. • "urn:ietf:params:oauth:client-assertion-type:jwt-bearer" => indicating a JWT Bearer Token.

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Additional Properties

Key	Description
`clientAssertion.type`	absolute URI
`clientAssertion.destinationName`	Name of a destination

Header	Value
`X-client-assertion-destination-name`	Name of a destination

> ⚠ Caution
>
> If headers `X-client-assertion` and `X-client-assertion-type` are specified in the *Find Destination* API call, the `clientAssertion.destinationName` property will not be used for an automated assertion fetching mechanism.

Use a Header to Specify the Destination that Provides the Client Assertion

Alternatively, the destination that provides the client assertion can also be specified in a header in the *Find Destination* API.

Header	Value	Description
`X-client-assertion-destination-name`	Name of a destination	Name of the destination that provides the client assertion. Must be on the same subaccount or service instance as the destination that uses client assertions.

If specified, this header overrides the property `clientAssertion.destinationName` in the destination that uses client assertions.

> ⚠ Caution
>
> Usage of headers `X-client-assertion` and `X-client-assertion-type` is not allowed if this header is present.

Example: Destination that Provides Client Assertions

> {·} Sample Code

```
Name=Provides_Client_Assertion_Destination
Type=HTTP
URL= https://xxxx.example.com
ProxyType=Internet
Authentication=OAuth2ClientCredentials
clientId=clientId
clientSecret=secret1234
tokenServiceURL=https://authserver1.example.com/oauth/token/
clientAssertion.type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
```

Example: Destination that Uses Client Assertions

> {·} Sample Code

```
Name=Uses_Client_Assertion_Destination
Type=HTTP
URL= https://xxxx.example.com
ProxyType=Internet
Authentication=OAuth2ClientCredentials
clientId=clientId
tokenServiceURL=https://authserver2.example.com/oauth/token/
clientAssertion.destinationName=Provides_Client_Assertion_Destination
```

Example: *Find Destination* API Call for Client Assertion Authentication with Automated Assertion Fetching

curl call:

Sample Code

```
curl --location --request
GET 'https://<destination_service_host>/destination-configuration/v1/
destinations/<name_of_destination_that_uses_client_assertions>' \
--header 'Authorization: Bearer <access_token>'
```

1.3.1.2.1.2 RFC Destinations

Configure RFC destinations in the SAP BTP cockpit.

RFC destinations provide the configuration required for communication with an on-premise ABAP system via Remote Function Call. The RFC destination data is used by the Java Connector (JCo) version that is available within SAP BTP to establish and manage the connection.

Note

The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

RFC Destination Properties

The RFC destination-specific configuration in SAP BTP consists of properties arranged in groups, as described below. The supported set of properties is a subset of the standard JCo properties in arbitrary environments. The configuration data is divided into the following groups:

- Main Properties [page 1444]
- User Logon Properties [page 1445]
- Pooling Configuration [page 1462]
- Repository Configuration [page 1459]
- Target System Configuration [page 1452]
- Parameters Influencing Communication Behavior [page 1464]

The tables with the configuration parameter information on the subpages include columns with the label in the destination editor, the corresponding property name to use when importing a destination from a file, and the description of the parameter itself.

For configuration properties that are not listed in the subpages because they are used, for example, for tagging purposes or beta features, you can add any key that starts with `jco.destination` or `jco.client` and arbitrary values in the panel *Additional Properties*.

The minimal configuration contains user logon properties and information identifying the target host. This means that you must provide at least a set of properties containing this information.

Example

```
Name=SalesSystem
Type=RFC
jco.client.client=000
jco.client.lang=EN
jco.client.user=consultant
jco.client.passwd=<password>
jco.client.ashost=sales-system.cloud
jco.client.sysnr=42
jco.destination.proxy_type=OnPremise
jco.destination.pool_capacity=5
jco.destination.peak_limit=10
```

Related Information

Invoking ABAP Function Modules via RFC [page 66]
Create RFC Destinations [page 1478]

1.3.1.2.1.2.1 Main Properties

Configure the main properties for RFC destinations in the SAP BTP cockpit.

In the *Destination Details* panel, you configure the general settings for a destination in the section *Main Properties*.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Name of the destination under which it is also accessed in the application code.
Type	`Type`	Type of the destination. Must be set to **RFC**.

Cockpit Label	JSON Key	Description
Proxy Type	`jco.destination.proxy_type`	The select box for the Proxy Type allows to choose between **`Internet`**, **`OnPremise`**, and **`Local`**. • When choosing **`OnPremise`**, the RFC communication is routed over a Cloud Connector that is connected to the subaccount. • When choosing **`Internet`**, the RFC communication is done over a WebSocket connection directly with the partner system. • **`Local`** is used to indicate that the destination is used within an *Edge Integration Cell* context.
Optional		
Description	`jco.destination.description`	Description that explains the purpose of the concrete destination (optional).
Location ID	`jco.client.cloud_connector_location_id`	You can connect multiple Cloud Connectors to a subaccount as long as their location ID is different. The value defines the location ID identifying the Cloud Connector over which the connection is opened. The default value is an empty string identifying the Cloud Connector that is connected without any location ID, which is the default behavior when adding a subaccount in the Cloud Connector.

1.3.1.2.1.2.2 User Logon Properties

Configure user logon properties for RFC destinations in the SAP BTP cockpit.

User logon properties for RFC destinations cover different types of user credentials, as well as the ABAP system client and the logon language.

The currently supported logon mechanism uses *user* and *password* as credentials.

In the *Destinations* editor, you can configure the user logon properties for a destination in section *Authentication* of the *Destination Details* panel.

Cockpit Label	JSON Key	Description
Required (depending on the authentication type and the chosen credentials)		

Cockpit Label	JSON Key	Description
Authentication Type	`jco.destination.auth_type`	**Note** SAP BTP supports the propagation of business users (principal propagation) and technical users from the cloud application towards on-premise systems. In both cases, a specific access token representing the business user or technical user is retrieved in the RFC runtime (for example, in JCo or SAP BTP ABAP environment), which can then be sent to the Connectivity service. For more information, see Authenticating Users against On-Premises Systems [page 1701]. • If the property is not provided in a file, its default value **`CONFIGURED_USER`** is used. In this case, you must specify *user*, *password*, or other credentials directly. • To enable single sign-on via principal propagation (an access token representing the business user logged on in the cloud application is forwarded to the on-premise system), set the value to **`PrincipalPropagation`**. In this case, you do not need to provide `jco.client.user` and `jco.client.passwd` in the configuration. • To enable technical user propagation (an access token representing the technical user is forwarded to the on-premise system), set the value to **`TechnicalUserPropagation`**. In this case, you do not need to provide `jco.client.user` and `jco.client.passwd` in the configuration.

Cockpit Label	JSON Key	Description
		Note For **`PrincipalPropagation`**, we recommend that you do not use the *RepositoryDefaultCredentials*, since there are special permissions needed (for metadata lookup in the backend) that not all business application users might have.
User	`jco.client.user`	Represents the user to be used for logging on to the ABAP system. **Note** *Authorization Type* must be set to **`CONFIGURED_USER`**. The value is case-insensitive and the maximum length is 12 characters. **Caution** When *Proxy Type* is set to **`Internet`** (that is, a WebSocket RFC connection), the checkbox *Use TLS client certificate login* must be unchecked to see the input field in the *Destinations* editor.

Cockpit Label	JSON Key	Description
Alias User	`jco.client.alias_user`	Represents the user to be used for logging on to the ABAP system. **Note** *Authorization Type* must be set to **`CONFIGURED_USER`**. Either `jco.client.user` or `jco.client.alias_user` must be specified. The alias user may be up to 40 characters long and is case-sensitive. **Caution** When *Proxy Type* is set to **`Internet`** (that is, a WebSocket RFC connection), the checkbox *Use TLS client certificate login* must be unchecked to see the input field in the *Destinations* editor.

Cockpit Label	JSON Key	Description
Password	`jco.client.passwd`	Represents the password of the user that is used. **Note** *Authorization Type* must be set to **`CONFIGURED_USER`**. **Note** Passwords in systems of SAP NetWeaver releases lower than 7.0 are case-insensitive and can be only eight characters long. For releases 7.0 and higher, passwords are case-sensitive with a maximum length of 40. **Caution** When *Proxy Type* is set to **`Internet`** (that is, using a WebSocket RFC connection), the checkbox *Use TLS client certificate login* must be unchecked to see the input field in the *Destinations* editor.
Use TLS client certificate login	`jco.client.tls_client_certificate_logon`	**Note** *Authorization Type* must be set to **`CONFIGURED_USER`**. If the checkbox is checked or the value in a property file is set to **`1`**, the client certificate provided by the key store (must be configured in addition) is used for authentication, and the *User*, *Alias User* and *Password* input fields will be hidden. If the checkbox is unchecked or the value in a property file is set to **`0`**, the basic credentials must be provided.

Cockpit Label	JSON Key	Description
Key Store Location	`KeyStoreLocation`	The name of the key store file that contains the client certificate for client certificate authentication against the remote WebSocket RFC server. Mandatory, if the checkbox *Use TLS client certificate login* is checked or `jco.client.tls_client_certificate_logon` is **`1`**.
Key Store Password	`KeyStorePassword`	The password for the key store file specified via `KeyStoreLocation`. **⚠ Caution** We strongly recommended that you use only key stores that are protected with a password.
Client ID	`jco.client.tech_user_id`	Client ID of the application. Mandatory when setting *Authorization Type*/ `jco.destination.auth_type` to **`TechnicalUserPropagation`**. The *Client ID* is used to retrieve the access token from the OAuth Server.
Client Secret	`jco.client.tech_user_secret`	Client secret for the Client ID. Mandatory when setting *Authorization Type*/ `jco.destination.auth_type` to **`TechnicalUserPropagation`**.

Cockpit Label	JSON Key	Description
Token Service URL	`jco.client.tech_user_service_url`	URL of the token service, against which the token exchange is performed. Mandatory when setting *Authorization Type*/ `jco.destination.auth_type` to **`TechnicalUserPropagation`**. → Remember The token service is not accessed through the Cloud Connector, but directly over the Internet. When using the Authorization and Trust Management service (xsuaa) as token service for *Technical User Propagation*, the URL should follow this example: `https://{tenant}.authentication.us10.hana.ondemand.com/oauth/token` → `https://mytenant.authentication.us10.hana.ondemand.com/oauth/token`
Optional		
Logon Language	`jco.client.lang`	Optional property. Represents the logon language. If the property is not provided, the user's or system's default language is used. Valid values are two-character ISO language codes or one-character SAP language codes.

1.3.1.2.1.2.3 Target System Configuration

Learn about the JCo properties you can use to configure the target sytem information in an RFC destination.

> ⓘ Note
>
> This documentation refers to SAP BTP, multi-cloud foundation. If you are looking for information about the Neo environment, see Target System Configuration (Neo environment).

Content

Overview [page 1453]

Direct Connection [page 1453]

Load Balancing Connection [page 1454]

WebSocket Connection [page 1456]

Overview

You can use the following configuration types alternatively:

- Direct connection to an ABAP application server via Cloud Connector.
- Load balancing connection to a group of ABAP application servers via a message server via Cloud Connector.
- WebSocket connection to an ABAP application server, either via Cloud Connector or directly (RFC over Internet). In the latter case, the target ABAP system must be exposed to the Internet.

> **Note**
>
> When using a WebSocket connection, the target ABAP system must be exposed to the Internet.

Depending on the configuration you use, different properties are mandatory or optional.

The set of available options depends on the *Proxy Type* chosen while editing the destination. In the *Destinations* editor in the cockpit, the configuration must be provided in the *Target System Configuration* panel.

> **Note**
>
> Target system configurations that are relevant for *Proxy Type* set to **`Local`** for the *Edge Integration Cell* use case are not covered in this documentation.

Back to Content [page 1453]

Direct Connection

To use a direct connection (connection without load balancing) to an application server over the Cloud Connector, you must set the value for *Proxy Type* to **`OnPremise`**. In addition, you must uncheck the checkbox *Use Load Balancing Connection*.

Cockpit Label	JSON Key	Description
Virtual Application Server Host	`jco.client.ashost`	Represents the application server host to be used. For configurations on SAP BTP, the property must match a virtual host entry in the Cloud Connector *Access Control* configuration. The property indicates that a direct connection is established.
System Number	`jco.client.sysnr`	Represents the so-called "system number" and has two digits. It identifies the logical port on which the application server is listening for incoming requests. For configurations on SAP BTP, the property must match a virtual port entry in the Cloud Connector *Access Control* configuration. **Note** The virtual port in the above access control entry must be named **`sapgw<##>`**, where `<##>` is the value of `sysnr`.
Client	`jco.client.client`	Represents the client to be used in the ABAP system. Valid format is a 3-digit number.

Back to Content [page 1453]

Load Balancing Connection

To use load balancing to a system over Cloud Connector, you must set the value for *Proxy Type* to **`OnPremise`**. In addition, you must check the checkbox *Use Load Balancing Connection*.

Cockpit Label	JSON Key	Description
Virtual Message Server Host	`jco.client.mshost`	Represents the message server host to be used. For configurations on SAP BTP, the property must match a virtual host entry in the Cloud Connector *Access Control* configuration. The property indicates that load balancing is used for establishing a connection.
Logon Group	`jco.client.group`	Optional property. Identifies the group of application servers that is used, the so-called "logon group". If the property is not specified, the group `PUBLIC` is used.

Cockpit Label	JSON Key	Description
System ID	`jco.client.r3name`	Represents the three-character system ID of the ABAP system to be addressed. For configurations on SAP BTP, the property must match a virtual port entry in the Cloud Connector *Access Control* configuration. **Note** The virtual port in the above access control entry must be named **`sapms<###>`**, where `<###>` is the value of `r3name`. **Note** In the *Destinations* editor, the check box *Use Message Server Port instead of System ID Connection* must be unchecked.
Virtual Message Server Port	`jco.client.msserv`	Represents the port on which the message server is listening for incoming requests. you can use this property as an alternative to `jco.client.r3name`. One of these two must be present. For configurations on SAP BTP, the property must match a virtual port entry in the Cloud Connector *Access Control* configuration. You can therefore avoid lookups in the `/etc/services` file (`<Install_Drive>\Windows\System32\drivers\etc\services`) on the Cloud Connector host if you want to use the same value for virtual and internal port.. **Note** In the *Destinations* editor, the check box *Use Message Server Port instead of System ID Connection* must be unchecked.
Client	`jco.client.client`	Represents the client to be used in the ABAP system. Valid format is a 3-digit number.

Back to Content [page 1453]

WebSocket Connection

> **Note**
>
> For WebSocket destinations, the connection check is not yet available.

To use a direct connection over WebSocket, you must set the value for *Proxy Type* to `Internet`.

Prerequisites

Your target system is one of the following:

- SAP S/4HANA Cloud system
- SAP BTP, ABAP environment system
- ABAP server as of SAP S/4HANA (on-premise) version 1909 (must be exposed to the Internet)

The trust-related configuration must be provided in the *Client Trust Store* configuration panel.

For using a WebSocket connection via Cloud Connector, the *Proxy Type* must be set to `OnPremise`.

Cockpit Label	JSON Key	Description
WebSocket RFC Server Host	`jco.client.wshost`	Represents the WebSocket RFC server host on which the target ABAP system is running. The system must be exposed to the Internet, when communicating directly. When calling via Cloud Connector, the virtual host in the host mapping of the Cloud Connector must be provided.
WebSocket RFC Server Port	`jco.client.wsport`	Represents the WebSocket RFC server port on which the target ABAP system is listening. When calling via Cloud Connector, the virtual port in the host mapping of the Cloud Connector must be provided.
Client	`jco.client.client`	Represents the client to be used in the ABAP system. Valid format is a 3-digit number. Optional for WebSocket RFC, if not provided, the default client associated with the host will be used.

Cockpit Label	JSON Key	Description
WebSocket RFC Ping Period	`jco.destination.ws_ping_period`	Optional property and not relevant when using *Proxy Type* `OnPremise`. Time period of a WebSocket client connection in seconds after which a keep alive WebSocket ping packet is sent while waiting for response data during a call. • Switching keep alive pinging from off [0] to on [greater than 10] and vice versa will only affect new RFC connections opened afterwards. • Default is the value of JCo property `jco.ws.ping_period`, which is 300 seconds, if not set to a different value. • Valid values are 0 [off] and a range from 10 [ten seconds] to 86400 [one day].
WebSocket RFC Pong Timeout	`jco.destination.ws_pong_timeout`	Optional property and not relevant when using *Proxy Type* `OnPremise`. Timeout for a WebSocket keep alive ping reply packet in seconds. If no such so-called pong packet is received from the communication partner as a reply to a previously sent WebSocket keep alive ping packet within this timeout period, the client connection is considered as broken and will be closed. • Switching a pong timeout from off [0] to on [greater than 10] and vice versa will only affect new RFC connections opened afterwards. • Default is the value of JCo property `jco.ws.pong_timeout`, which is 60 seconds, if not set to a different value. • Valid values are 0 [off] and a range from 10 [ten seconds] to 3600 [one hour].

Cockpit Label	JSON Key	Description
Trust All	`jco.client.tls_trust_all`	The checkbox *Use default client trust store* must be unchecked. • If the checkbox *Trust All* is checked, all server certificates are considered trusted during a TLS handshake. • If the checkbox *Trust All* is unchecked, either a dedicated trust store must be configured or the default client trust store will be used as default. In an imported configuration file, the respective values are 1 and 0. **Note** We *strongly* recommend that you **leave this checkbox unchecked in productive scenarios**, and mark it as checked *only* for demo/test purposes. Not relevant when using *Proxy Type* `OnPremise`.
Trust Store Location	`TrustStoreLocation`	You can choose a trust store location. This field indicates the name of the trust store on the *Destination Certificates* page which contains trusted certificates (certificate authorities) for authentication against a remote client. Not relevant when using *Proxy Type* `OnPremise`. **Note** You can only configure a trust store location, if both checkboxes *Use default client trust store* and *Trust All* are unchecked.
Trust Store Password	`TrustStorePassword`	Password for the trust store file specified via *Trust Store Location*. Not relevant when using *Proxy Type* `OnPremise`.

> **Note**
>
> You can upload trust store files using the same command as for uploading destination configuration property files. You only need to specify the trust store file instead of the destination configuration file.

> **Note**
>
> Connections to remote services which require *Java Cryptography Extension (JCE) unlimited strength jurisdiction policy* are not supported.

For more information on WebSocket RFC, see also WebSocket RFC (ABAP Platform documentation).

Back to Content [page 1453]

1.3.1.2.1.2.4 Repository Configuration

Configure repository properties for RFC destinations in the SAP BTP cockpit.

Repository properties let you define the behavior of the repository that dynamically retrieves function module metadata.

All properties below are optional. Alternatively, you can create the metadata in the application code, using the metadata factory methods within the `JCo` class, to avoid additional round-trips to the on-premise system.

For convenience, those metadata objects can be stored in a `CustomRepository` when using them in the application.

In the *Destinations* editor in the cockpit, the configuration must be provided in the *Repository Configuration* panel.

> **Note**
>
> **Repository Management: Background**
>
> The built-in comfortable repository management currently uses the system ID as the identifier of a system, so destinations pointing to the same system can share the data to avoid unnecessary round-trips and memory consumption. However, this approach includes a drawback (described also in SAP Note 1405466): if you merge multiple networks over multiple Cloud Connectors in a single scenario, the probability of duplicate system IDs increases. As a consequence, collisions of different metadata definitions in those systems may cause issues. If duplicates are likely to be expected, using a `CustomRepository` and an application-managed set of repositories is a recommended workaround for this limitation.

Cockpit Label	JSON Key	Description
Optional		

Cockpit Label	JSON Key	Description
Repository Destination	`jco.destination.repository_destination`	Specifies which destination should be used for repository queries. If the destination does not exist, an error occurs when trying to retrieve the repository. Defaults to itself. **Note** The selectbox is visible in the *Destinations* editor only if the selected option for *Repository Configuration Type* is **`Repository Destination`**.
Repository User	`jco.destination.repository.user`	Optional property. If this property is set, and the repository destination is not set, it is used as the user for repository queries. This configuration option allows using a different user for repository lookups with a single destination configuration, and restricting this user's permissions accordingly. See also SAP Note 460089. **Note** The input field is visible in the *Destinations* editor only if the selected option for *Repository Configuration Type* is *Repository Basic Credentials*. When selecting the option *Repository Default Credentials*, the credential settings for repository connections will be the same as for business connections.
Repository Password	`jco.destination.repository.passwd`	Represents the password for a repository user. If you use such a user, this property is mandatory. **Note** The input field is visible in the *Destinations* editor only, if the selected option for *Repository Configuration Type* is *Repository Basic Credentials*.

Cockpit Label	JSON Key	Description
Check Interval	`jco.destination.repository.check_interval`	Time interval in minutes after which the associated repository is regularly checked for outdated metadata in its local cache. If outdated metadata are identified, they will be removed from the cache. The default value is 0 (repository checking feature is disabled). If multiple repository destination configurations refer to the same repository instance, the smallest configured *non-zero* value of all destinations will be effective for the repository. Therefore, the repository checking feature is only switched off if *all* repository destinations for the repository instance are configured with value 0, or do not specify this property. **Note** The configured value will only be considered when creating a destination from a properties file, if the respective destination can be used for repository metadata queries, and it will be ignored if `jco.destination.repository_destination` has been configured. In the latter case, the property `jco.destination.repository.check_interval` must be configured in the referred repository destination instead in order to activate this checking feature. For the same reason, the input field is visible in the *Destinations* editor only if the selected option for *Repository Configuration Type* is not *Repository Destination*.

Related Information

Invoking ABAP Function Modules via RFC [page 66]

1.3.1.2.1.2.5 Pooling Configuration

Configure pooling properties for RFC destinations in the SAP BTP cockpit.

Overview

This group of JCo properties covers different settings for the behavior of the destination's connection pool. All properties are optional.

In the *Destinations* editor in the cockpit, the configuration must be provided in the *Pooling Configuration* panel.

Cockpit Label	JSON Key	Description
Optional		
Pool Capacity	`jco.destination.pool_capacity`	Represents the maximum number of idle connections kept open by the destination. A value of **`0`** has the effect of no connection pooling, that is, connections will be closed after each request. The default value is **`1`**.
Peak Limit	`jco.destination.peak_limit`	Represents the maximum number of active connections you can create for a destination simultaneously. Value **`0`** allows an unlimited number of active connections. Otherwise, if the value is less than the value of `jco.destination.pool_capacity`, it will be automatically increased to this value. Default setting is the value of `jco.destination.pool_capacity`. If `jco.destination.pool_capacity` is not specified, the default is **`0`** (unlimited).
Maximum Get Client Time	`jco.destination.max_get_client_time`	Represents the maximum time in milliseconds to wait for a free connection in case the maximum number of active connections is already allocated by applications. The default value is 30000 (30 seconds).

Cockpit Label	JSON Key	Description
Expiration Time	`jco.destination.expiration_time`	Represents the time in milliseconds after which idle connections that are available in the pool can be closed. The default value is 60000 (60 seconds).
Expiration Check Period	`jco.destination.expiration_check_period`	Represents the interval in milliseconds for the timeout checker thread to check the idle connections in the pool for expiration. The default value is 60000 (60 seconds).
Pool Check Connection	`jco.destination.pool_check_connection`	If the checkbox *Pool Check Connection* is checked or when setting the property in a file to be imported to **`1`**, a pooled connection will be checked for corruption before being used for the next function module execution. Thus, it is possible to recognize corrupted connections and avoid exceptions passed to applications when connectivity is working in principle (default is that the checkbox is unchecked or the value is **`0`**). **Note** Turning on this check has performance impact for stateless communication. This is due to an additional low-level ping to the server, which takes a certain amount of time for non-corrupted connections, depending on latency.

Pooling Details

- Each destination is associated with a connection factory and, if the pooling feature is used, with a connection pool.
- Initially, the destination's connection pool is empty, and the JCo runtime does not preallocate any connection. The first connection will be created when the first function module invocation is performed. The `peak_limit` property describes how many connections can be created simultaneously, if applications allocate connections in different sessions at the same time. A connection is allocated either when a stateless function call is executed, or when a connection for a stateful call sequence is reserved within a session.
- After the `<peak_limit>` number of connections has been allocated (in `<peak_limit>` number of sessions), the next session will wait for at most `<max_get_client_time>` milliseconds until a different

session releases a connection (either finishes a stateless call or ends a stateful call sequence). In case the waiting session does not get any connection during the `<max_get_client_time>` period, the function request will be aborted with *JCoException* with the key **`JCO_ERROR_RESOURCE`**.

- Connections that are no longer used by applications are returned to the destination pool. There is at most a `<pool_capacity>` number of connections kept open by the pool. Further connections (`<peak_limit>` - `<pool_capacity>`) will be closed immediately after usage. The pooled connections (open connections in the pool) are marked as expired if they are not used again during `<expiration_time>` milliseconds. All expired connections will be closed by a timeout checker thread which executes the check every `<expiration_check_period>` milliseconds.

1.3.1.2.1.2.6 Parameters Influencing Communication Behavior

Configure ABAP communication properties for RFC destinations in the SAP BTP cockpit.

This group of JCo properties allows you to control the connection to an ABAP system. All properties are optional.

In the *Destinations* editor in the cockpit, the configuration must be provided in the *Communication Behavior Configuration* panel.

Cockpit Label	JSON Key	Description
Optional		
Enable protocol traces	`jco.client.trace`	Defines whether protocol traces are created. • If the checkbox is checked or the value in a property file is **`1`**, protocol traces are turned on. • If the checkbox is unchecked or the value in a property file is **`0`**, protocol traces are turned off. The default behavior is that protocol traces are turned off.
SAP codepage	`jco.client.codepage`	Declares the 4-digit SAP codepage that is used when initiating the connection to the backend. The default value is **`1100`** (comparable to iso-8859-1). It is important to provide this property if the password that is used contains characters that cannot be represented in **`1100`**.

Cockpit Label	JSON Key	Description
Enable table parameter delta management	`jco.client.delta`	Enables or disables table parameter delta management. If the checkbox is checked or the value in a property file is **`1`**, delta management is enabled, and disabled if the checkbox is unchecked or the value in a property file is **`0`**. The default behavior is that delta management is *enabled*.
Serialization Format	`jco.client.serialization_format`	Defines the serialization format that is used when transferring function module data to the partner system. The property impacts the serialization behavior of function module data. Valid values are **`columnBased`** and **`rowBased`**. If you choose **`columnBased`**, the *fast RFC serialization* is used, as long as the partner system supports it, see SAP Note 2372888. When choosing the **`rowBased`** option, *classic* or *basXML* serialization are used. The default value is **`rowBased`**.
Network Type	`jco.client.network`	Defines which network type is expected to be used for the destination. The property impacts the serialization behavior of function module data, see SAP Note 2372888. Valid values are **`WAN`** and **`LAN`**. The default value is **`LAN`**.

1.3.1.2.1.3 LDAP Destinations

Find information about LDAP destinations for on-premise connections from an SAP BTP subaccount.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Name of the target LDAP server.
Type	`Type`	Choose **LDAP**.
URL	`ldap.url`	Address of the target LDAP server.
Proxy Type	`ldap.proxyType`	`OnPremise`
Authentication	`ldap.authentication`	`NoAuthentication` or `BasicAuthentication`
Optional		
Description	`Description`	Free text describing the LDAP destination.
Location ID	`CloudConnectorLocationId`	If more than one Cloud Connector is connected to your subaccount, enter the location ID of the Cloud Connector that is used to connect to the target LDAP server.

Related Information

Create LDAP Destinations [page 1480]

1.3.1.2.1.4 TCP Destinations

Create and manage SAP BTP destinations using the TCP protocol for communication.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Destination name
Type	`Type`	Destination type. Use `TCP`.
Address	`Address`	*<virtual_host>:<virtual_port>* of the on-premise application or or <host>:<port> of the Internet application. The address may use *tcp://* as protocol scheme.
Proxy Type	`ProxyType`	`Internet`, `OnPremise`, or `PrivateLink`
Additional		
	Key	**Description**
	`tcp.nodelay`	Activates or deactivates TCP delay.
	`tcp.socket.linger`	Sets the linger state of the associated socket. The linger state specifies whether and for how long a socket maintains the connection after calling the `Close()` method if there is still data to be sent.
	`tcp.socket.sndbuf`	*Send* buffer size in Bytes.
	`tcp.socket.rcvbuf`	*Receive* buffer size in Bytes.
	`tcp.socket.timeout`	Amount of time the TCP client waits for a *send* or *receive* operation to complete successfully.
	`tcp.socket.keepalive`	Specifies if *keepalive* messages can be sent.
	`tcp.socket.reuseaddr`	Specifies if a TCP server can bind to an address to prevent multiple servers from binding to the same address.

1.3.1.2.1.5 MAIL Destinations

Find information about MAIL destinations for Internet or on-premise connections from an SAP BTP subaccount.

You can use MAIL destinations to access target systems which use the SMTP, POP3 or IMAP protocol.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Find below an overview of available configuration parameters for each protocol:

- IMAP [page 1468]
- POP3 [page 1469]
- SMTP [page 1470]

Related Information

Create MAIL Destinations [page 1484]

1.3.1.2.1.5.1 IMAP

Configure IMAP destinations in SAP BTP, multi-cloud foundation.

Find below an overview of available configuration parameters for IMAP.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Name of the MAIL destination.
Type	`Type`	Choose `MAIL`.
Proxy Type	`ProxyType`	`Internet` or `OnPremise`
Authentication	`Authentication`	Choose an authentication method from the dropdown list and enter the required properties depending on the authentication type.
User	`mail.user`	Default user name for connecting to the mail server.

Cockpit Label	JSON Key	Description
Optional		
Description	`Description`	Free text describing the MAIL destination.
Password	`mail.password`	Password to be used for connecting to the IMAP server.
Additional		
	Key	**Description**
	`mail.imap4.host`	Address of the target mail server.
	`mail.imap4.auth`	Specifies if authentication is required. Can be *true* or *false*.
	`mail.imap4.port`	Port on which your mail server listens for connections.
	`mail.transport.protocol`	Transport protocol to be used.
	`mail.imap4.from`	E-mail address to be used as the "From" address.

Related Information

Create MAIL Destinations [page 1484]

1.3.1.2.1.5.2 POP3

Configure POP3 destinations in SAP BTP, multi-cloud foundation.

Find below an overview of available configuration parameters for POP3.

> ⓘ Note
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Name of the MAIL destination.
Type	`Type`	Choose `MAIL`.
Proxy Type	`ProxyType`	`Internet` or `OnPremise`
Authentication	`Authentication`	Choose an authentication method from the dropdown list and enter the required properties depending on the authentication type.
User	`mail.user`	Default user name for connecting to the mail server.
Optional		
Description	`Description`	Free text describing the MAIL destination.
Password	`mail.password`	Password to be used for connecting to the POP3 server.
Additional		
	Key	**Description**
	`mail.pop3.host`	Address of the target mail server.
	`mail.pop3.auth`	Specifies if authentication is required. Can be *true* or *false*.
	`mail.pop3.port`	Port on which your mail server listens for connections.
	`mail.transport.protocol`	Transport protocol to be used.
	`mail.pop3.from`	E-mail address to be used as the "From" address.

Related Information

Create MAIL Destinations [page 1484]

1.3.1.2.1.5.3 SMTP

Configure SMTP destinations in SAP BTP, multi-cloud foundation.

Find below an overview of available configuration parameters for SMTP.

> ### Note
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Properties

To configure a destination of this authentication type, you must specify all the required properties.

Cockpit Label	JSON Key	Description
Required		
Name	`Name`	Name of the MAIL destination.
Type	`Type`	Choose `MAIL`.
Proxy Type	`ProxyType`	`Internet` or `OnPremise`
Authentication	`Authentication`	Choose an authentication method from the dropdown list and enter the required properties depending on the authentication type.
User	`mail.user`	Default user name for connecting to the mail server.
Optional		
Description	`Description`	Free text describing the MAIL destination.
Password	`mail.password`	Password to be used for connecting to the SMTP server.
Additional		
	Key	**Description**
	`mail.smtp.host`	Address of the target mail server.
	`mail.smtp.auth`	Specifies if authentication is required. Can be *true* or *false*.
	`mail.smtp.port`	Port on which your mail server listens for connections.
	`mail.transport.protocol`	Transport protocol to be used.
	`mail.smtp.provider.class`	Class that implements the SMTP provider in your Java-based system.
	`mail.smtp.provider.vendor`	Vendor of the SMTP provider.
	`mail.smtp.provider.version`	Version of the SMTP provider used.

Cockpit Label	JSON Key	Description
	`mail.smtp.from`	E-mail address to be used as the "From" address.

Related Information

Create MAIL Destinations [page 1484]

1.3.1.2.2 Using the Destinations Editor in the Cockpit

Use the *Destinations* editor in the SAP BTP cockpit to configure HTTP, RFC, LDAP, TCP, or MAIL destinations.

The *Destinations* editor lets you manage destinations on subaccount or service instance level.

You can use a destination to:

- Connect your application to the **Internet**, to an **on-premise system**, or using a **PrivateLink** connection.
- Send and retrieve e-mails, configuring a mail destination.
- Create a destination for subscription-based scenarios, pointing to your service instance. For more information, see Destinations Pointing to Service Instances [page 1489].

Prerequisites

1. You are logged into the SAP BTP cockpit.
2. You have the required authorizations. See User Roles [page 1301].
3. Make sure the following is fulfilled:
   - Subscription level: the SaaS application has allowed users (of the subscribed consumer) to manage destinations and you have the roles defined by the SaaS application to perform the desired operation.
   - Service instance level: you must have created a Destination service instance, see Create and Bind a Destination Service Instance [page 1523].
   - Subaccount level: no specific prerequisites.

   For more information, see Access the Destinations Editor [page 1473].

Restrictions

- A destination name must be unique for the current application. It must contain only alphanumeric characters, underscores, and dashes. The maximum length is 200 characters.

- The currently supported destination types are **HTTP**, **RFC**, **LDAP**, **TCP**, and **MAIL**.
  - HTTP Destinations [page 1350]: provide data communication via the HTTP protocol and are used for both Internet and on-premise connections.
  - RFC Destinations [page 1443]: make connections to ABAP on-premise systems via RFC protocol using the Java Connector (JCo) as API.
  - LDAP Destinations [page 1465]: connect to to a local LDAP server for user management.
  - TCP Destinations [page 1466]: use the TCP protocol for communication.
  - MAIL Destinations [page 1467]: specify an e-mail provider for sending and retrieving e-mails.

Tasks

- Create Destinations from Scratch [page 1475]
- Create Destinations from a Template [page 1489]
- Check the Availability of a Destination [page 1491]
- Duplicate Destinations [page 1493]
- Edit and Delete Destinations [page 1494]
- Manage Destination Certificates [page 1504]
- Export Destinations [page 1496]

Related Information

Destination Examples [page 1486]

1.3.1.2.2.1 Access the Destinations Editor

Access the Destinations editor in the SAP BTP cockpit to create and manage destinations.

You can edit destinations at three different levels:

- Subaccount level
- Service instance level
- Subscription level

On *subaccount* level, you can specify a destination for the entire subaccount, defining the used communication protocol and more properties, like authentication method, proxy type and URL.

On *service instance* level, you can reuse this destination for a specific space and adjust the URL if required. You can also create a new destination only on service instance level that is specific to the selected service instance and its assigned applications.

On *subscription* level, you specify a destination only for an SaaS application you are subscribed to (if the SaaS application has allowed this).

Prerequisites

- You are logged into the SAP BTP cockpit.
- You have the required authorizations. See User Roles [page 1301].

Procedure

Access on Subaccount Level

1. In the cockpit, select your *Global Account* and your subaccount name from the *Subaccount* menu in the breadcrumbs.
2. From the left-side panel, choose *Connectivity* > *Destinations*.
3. The Destinations editor will load. The right panel lets you select the *Context Level* in which to work. The *Subaccount Level* is the default selection.

Access on Service Instance Level

> **Note**
>
> To perform these steps, you must have a created destination service instance.
>
> For more information, see Create and Bind a Destination Service Instance [page 1523].

1. In the cockpit, choose your *Global Account* from the *Region Overview* and select a *Subaccount*.

   > **Caution**
   >
   > The *Cloud Foundry Organization* must be enabled.

2. From the left-side menu, choose *Spaces* and select a space name.
3. From the left-side menu, choose *Services* > *Service Instances*.
4. Select a service instance and choose *Destinations* from the left-side panel.

Access on Subscription Level

1. In the cockpit, select your global account and your subaccount name from the subaccount menu in the breadcrumbs.
2. From the left-side panel, choose *Connectivity* > *Destinations*.
3. The Destinations editor will load. The right panel lets you select the *Context Level* in which to work. Under *Subscriptions* you will see all SaaS applications that support consumer management of destinations and the permissions your user has.

   > **Note**
   >
   > The needed permissions for each subscription level are defined by the respective SaaS application.

> **Note**
>
> If you are developing an SaaS application and want to support consumer management of destinations, see Accessing and Managing Destination Service Configurations on Subscription Level [page 22].

Related Information

Create Destinations from Scratch [page 1475]
Create Destinations from a Template [page 1489]
Check the Availability of a Destination [page 1491]
Duplicate Destinations [page 1493]
Edit and Delete Destinations [page 1494]
Manage Destination Certificates [page 1504]
Import Destinations [page 1495]
Export Destinations [page 1496]

1.3.1.2.2.2 Create Destinations from Scratch

Use the *Destinations* editor in the SAP BTP cockpit to configure destinations from scratch.

Configuring destinations from scratch provides the complete set of editing functions. While this requires deeper technical knowledge of the scenario and the required connection configuration, it is the most flexible procedure and lets you create any type of supported destination.

To Create Destinations from a Template [page 1489], in contrast, you do not need this deeper knowledge for configuration, but editing options are limited by the available templates.

Related Information

Create HTTP Destinations [page 1476]
Create RFC Destinations [page 1478]
Create LDAP Destinations [page 1480]
Create TCP Destinations [page 1482]
Create MAIL Destinations [page 1484]
Destination Examples [page 1486]

1.3.1.2.2.2.1 Create HTTP Destinations

Create HTTP destinations in the *Destinations* editor (SAP BTP cockpit).

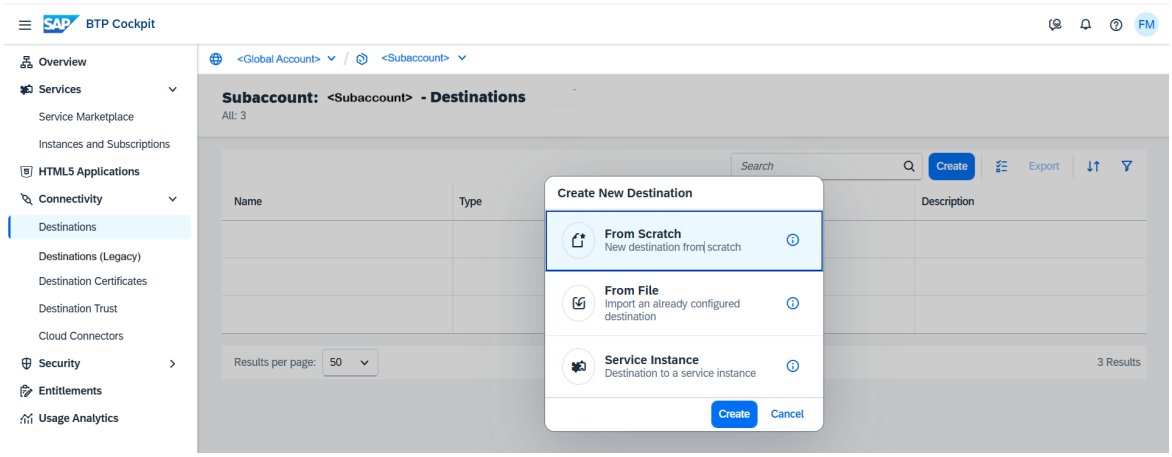
Prerequisites

You have logged into the cockpit and opened the *Destinations* editor from your subaccount menu (choose *Connectivity* > *Destinations*).

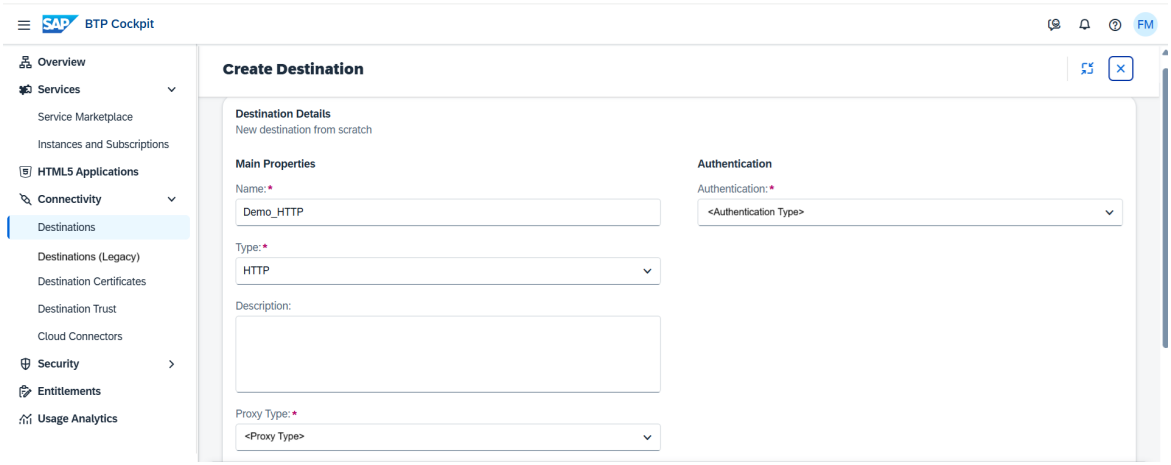
> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Procedure

1. Choose *Create* > *From Scratch* > *Create*.



2. Enter a destination name.
3. From the `<Type>` dropdown menu, choose **`HTTP`**.
4. The `<Description>` field is optional.
5. From the `<Proxy Type>` dropdown box, select **`Internet`**, **`OnPremise`**, or **`PrivateLink`**, depending on the connection type you want to provide for your application.



> **Note**
>
> For more information, see also HTTP Destinations [page 1350].

6. Specify the destination URL.
7. From the `<Authentication>` dropdown box, select the authentication type you want to use for the connection.

   For details, see HTTP Destinations [page 1350].

> **Note**
>
> If you set an **HTTPS** destination, you need to also add a Trust Store. For more information, see Manage Destination Certificates [page 1504].

8. (Optional) For *Internet* and *PrivateLink* connections only: You can enter a trust store location and password, or use the default client trust store.
9. (Optional) For *OnPremise* connections only: If you are using more than one Cloud Connector for your subaccount, you must enter the `<Location ID>` of the target Cloud Connector.
   See also Adding and Managing Subaccounts [page 1682] (section **Procedure**, step 4).
10. (Optional) You can enter additional properties.
    a. In the *Additional Properties* panel, choose *Add Property*.
    b. Enter a key (name) or choose one from the dropdown menu and specify a value for the property. You can add as many properties as you need.
    c. To delete a property, choose the *Delete* icon next to it.

> **Note**
>
> For a detailed description of specific properties for SAP Business Application Studio (formerly known as SAP Web IDE), see Connecting to External Systems.

11. When you are done, choose *Create*.

Related Information

Edit and Delete Destinations [page 1494]
Destination Examples [page 1486]

1.3.1.2.2.2.2 Create RFC Destinations

How to create RFC destinations in the *Destinations* editor (SAP BTP cockpit).

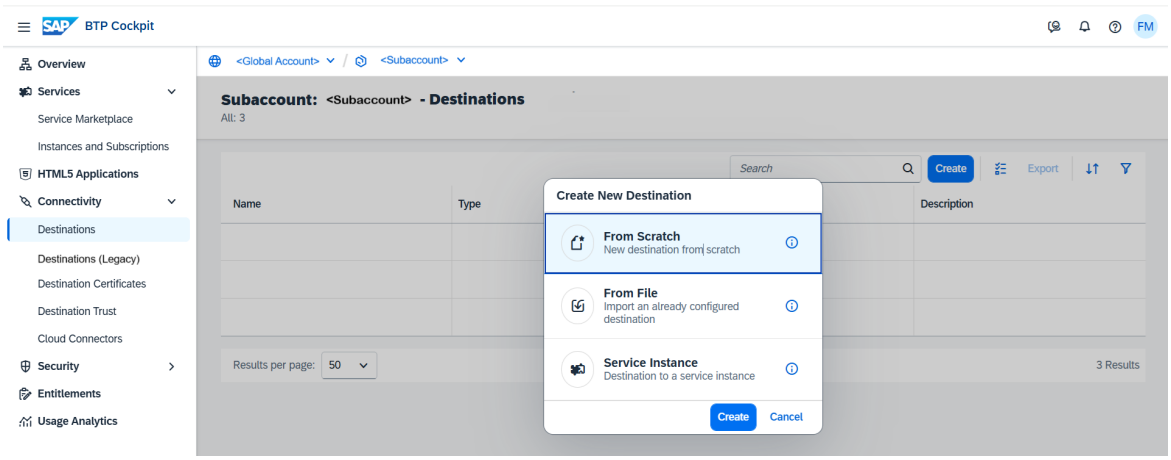
Prerequisites

You have logged into the cockpit and opened the *Destinations* editor from your subaccount menu (choose *Connectivity* > *Destinations*).

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

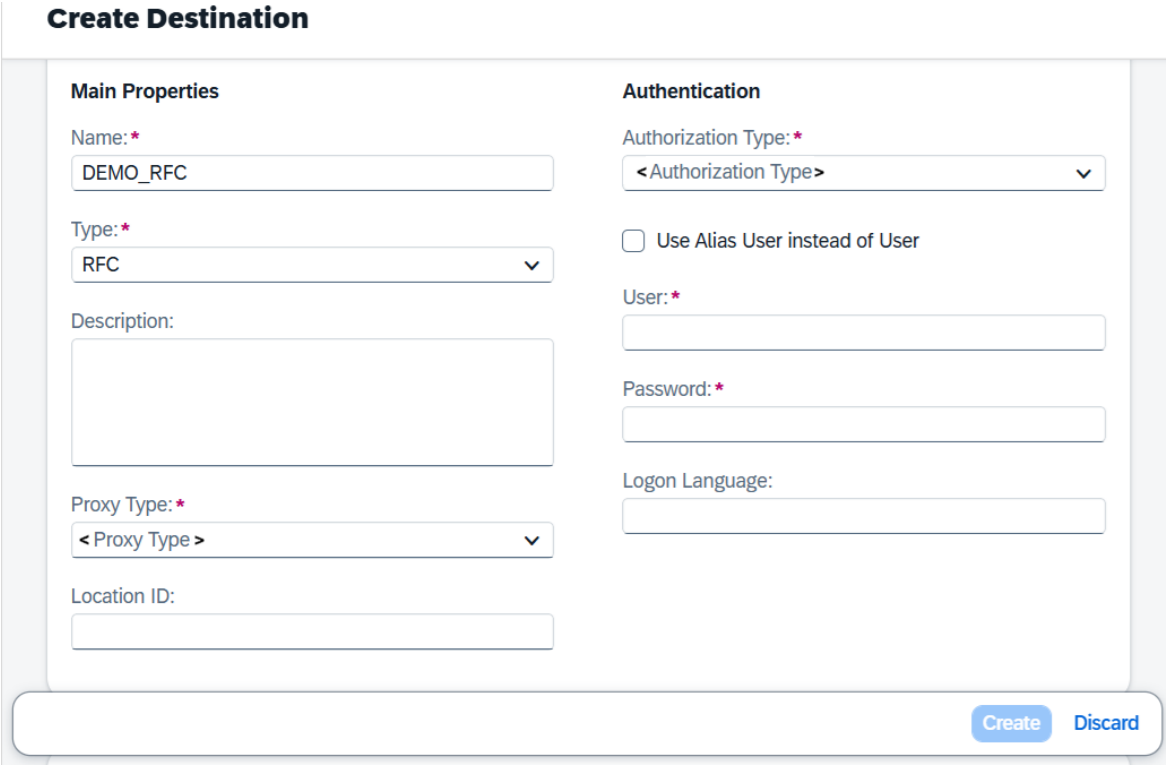
Procedure

1. Choose *Create* > *From Scratch* > *Create*.



2. Enter a destination name.
3. From the <Type> dropdown menu, choose **RFC**.

4. The `<Description>` field is optional.
5. From the `<Proxy Type>` dropdown box, select **`Internet`**, **`OnPremise`**, or **`Local`**, depending on the connection type you want to provide for your application.



> ### Note
>
> When using `<Proxy Type>` **`Internet`** , you can connect your application to any target service that is exposed to the Internet. `<Proxy Type>` **`OnPremise`** requires the Cloud Connector to access resources within your on-premise network.

6. (Optional) For *OnPremise* connections only: If you are using more than one Cloud Connector for your subaccount, you must enter the `<Location ID>` of the target Cloud Connector.
   See also Adding and Managing Subaccounts [page 1682] (section **Procedure**, step 4).
7. As *Authorization Type*, choose **`CONFIGURED_USER`**, **`PrincipalPropagation`** (for *OnPremise* connections only), or **`TechnicalUserPropagation`** (for *OnPremise* connections only), and enter the required parameters for the selected authorization type.
   See also User Logon Properties [page 1445].
8. In section *Target System Configuration*, enter the required target system information.
   See also Target System Configuration [page 1452].
9. (Optional) In section *Repository Configuration*, you can define the repository behavior.
   See also Repository Configuration [page 1459].
10. (Optional) In section *Pooling Configuration*, you can define the pooling behavior for the RFC destination.
    See also Pooling Configuration [page 1462].
11. (Optional) In section *Communication Behavior Configuration*, you can control the connection to an ABAP system.

See also Parameters Influencing Communication Behavior [page 1464].

12. (Optional) You can enter additional properties.
    a. In the *Additional Properties* panel, choose *Add Property*.
    b. Enter a key (name) or choose one from the dropdown menu and specify a value for the property. You can add as many properties as you need.
    c. To delete a property, choose the *Delete* icon next to it.

    For a detailed description of RFC-specific properties (JCo properties), see RFC Destinations [page 1443].

13. When you are done, choose *Create*.

Related Information

Edit and Delete Destinations [page 1494]
Destination Examples [page 1486]
Cloud Connector [page 1602]

1.3.1.2.2.2.3 Create LDAP Destinations

Create LDAP destinations in the *Destinations* editor (SAP BTP cockpit).

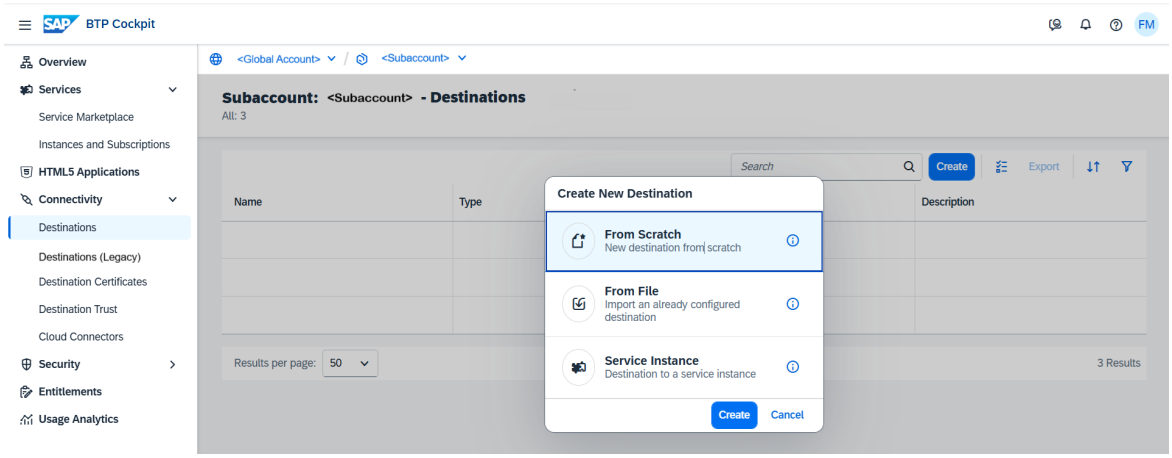
Prerequisites

You have logged into the cockpit and opened the *Destinations* editor from your subaccount menu (choose *Connectivity* > *Destinations*).

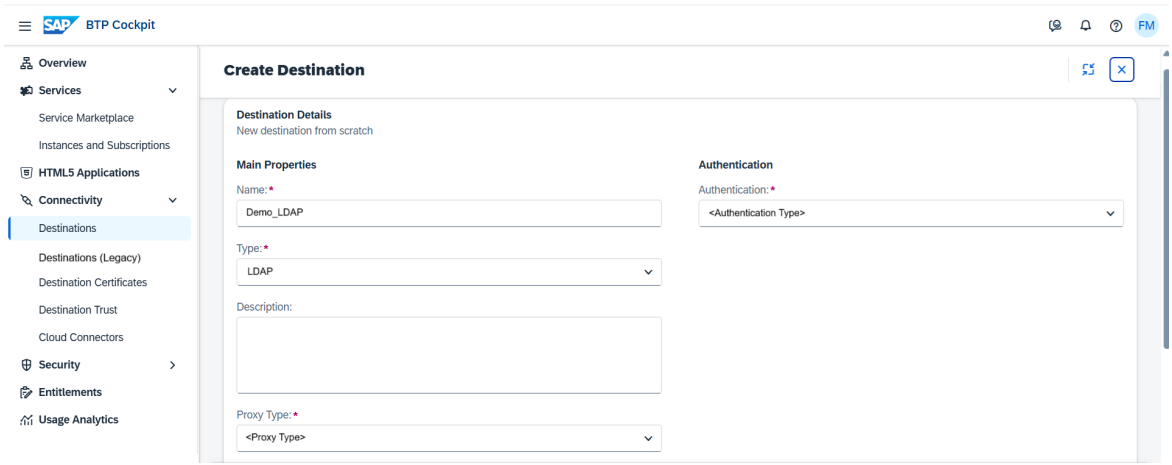
> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Procedure

1. Choose *Create* > *From Scratch* > *Create*.



2. Enter a destination name.
3. From the *Type* dropdown menu, choose **`LDAP`**.
4. The *Description* field is optional.
5. From the *Proxy Type* dropdown box, select **`Internet`**, **`OnPremise`**, or **`PrivateLink`**, depending on the connection type you want to provide for your application.



6. Specify the target *URL* of the LDAP server.
7. (Optional) For *OnPremise* connections only: If more than one Cloud Connector is connected to your subaccount, enter the location ID of the Cloud Connector that is used to connect to the target LDAP server.
8. Choose *Authentication* type **`NoAuthentication`** or **`BasicAuthentication`**. For a detailed parameter description, see Configuring Authentication [page 1351].
9. (Optional) You can enter additional properties.
    a. In the *Additional Properties* panel, choose *Add Property*.
    b. Enter a key (name) or choose one from the dropdown menu and specify a value for the property. You can add as many properties as you need.
    c. To delete a property, choose the *Delete* icon next to it.
10. When you are done, choose *Create*.

Related Information

Edit and Delete Destinations [page 1494]
Destination Examples [page 1486]
Cloud Connector [page 1602]

1.3.1.2.2.2.4 Create TCP Destinations

Create TCP destinations in the *Destinations* editor (SAP BTP cockpit).

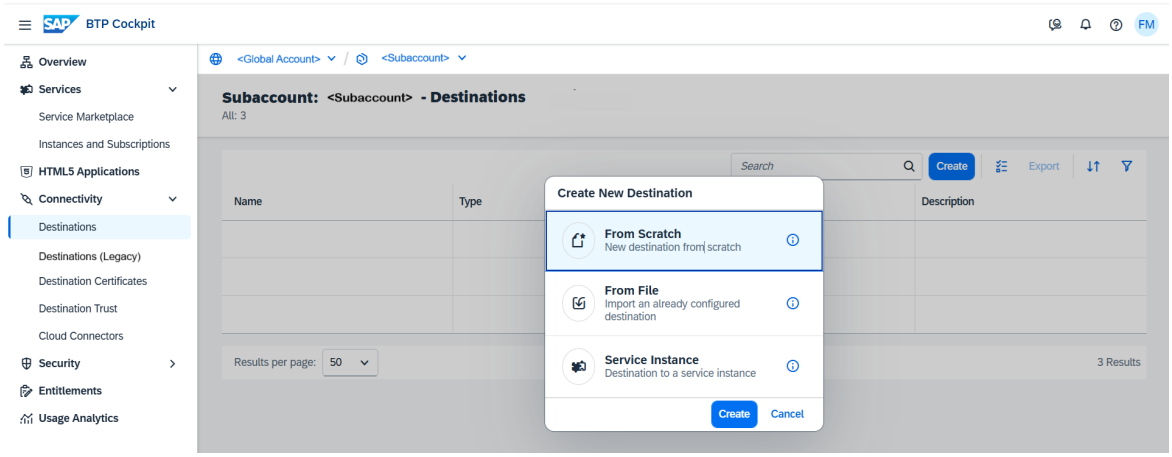
Prerequisites

You have logged into the cockpit and opened the *Destinations* editor from your subaccount menu (choose *Connectivity* > *Destinations*).

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

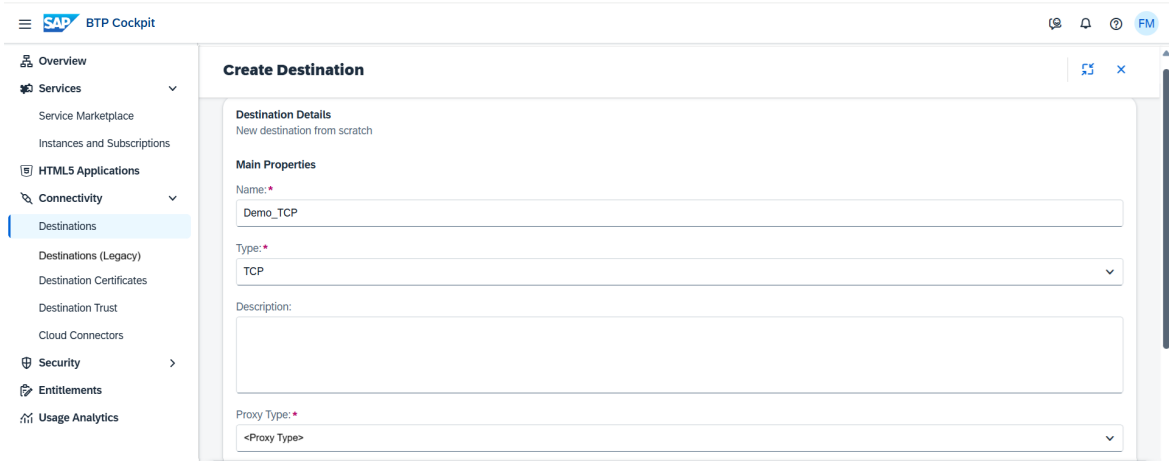
Procedure

1. Choose *Create* > *From Scratch* > *Create*.



2. Enter a destination name.

3. From the `<Type>` dropdown menu, choose **`TCP`**.
4. The `<Description>` field is optional.
5. From the `<Proxy Type>` dropdown box, select **`Internet`**, **`OnPremise`**, or **`PrivateLink`**, depending on the connection type you want to provide for your application.



> ### Note
>
> For more information, see also TCP Destinations [page 1466].

6. Specify the destination *Address*.
7. (Optional) If you selected `<Proxy Type>` **`OnPremise`**, and you are using more than one Cloud Connector for your subaccount, you must enter the `<Location ID>` of the target Cloud Connector.
   See also Adding and Managing Subaccounts [page 1682] (section **Procedure**, step 4).
8. (Optional) You can enter additional properties.
   a. In the *Additional Properties* panel, choose *Add Property*.
   b. Enter a key (name) or choose one from the dropdown menu and specify a value for the property. You can add as many properties as you need.
   c. To delete a property, choose the *Delete* icon next to it.
9. When you are done, choose *Create*.

Related Information

Edit and Delete Destinations [page 1494]
TCP Destinations [page 1466]

1.3.1.2.2.2.5 Create MAIL Destinations

Create MAIL destinations in the *Destinations* editor (SAP BTP cockpit).

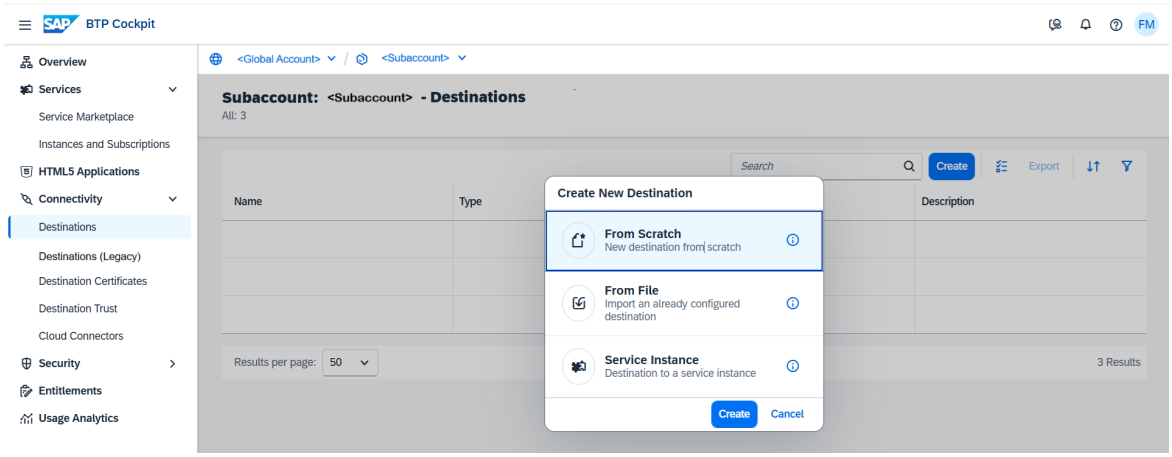
Prerequisites

You have logged into the cockpit and opened the *Destinations* editor from your subaccount menu (choose *Connectivity* > *Destinations*).

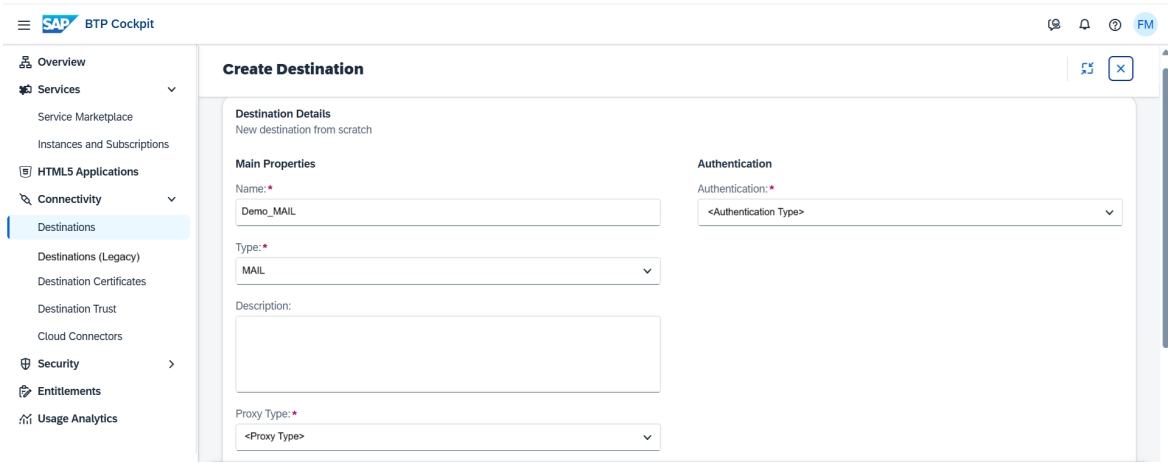
> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

Procedure

1. Choose *Create* > *From Scratch* > *Create*.



2. Enter a destination name.
3. From the *Type* dropdown menu, choose **`MAIL`**.
4. The *Description* field is optional.
5. From the *Proxy Type* dropdown box, select **`Internet`**, **`OnPremise`**, or **`PrivateLink`**, depending on the connection type you want to provide for your application.



> **Note**
>
> To access a mail server located in your own network (via Cloud Connector), choose **`OnPremise`**. To access an external mail server, choose **`Internet`**.

6. Choose the *Authentication* type to be used for the destination and enter the required parameters. For a detailed parameter description, see Configuring Authentication [page 1351].
7. Enter an *additional property* to specify the address of the target mail server.
   a. In the *Additional Properties* panel, choose *Add Property*.
   b. Depending on the protocol you want to use, enter *mail.imap4.host*, *mail.pop3.host*, or *mail.smtp.host* and specify a value for the property.
8. (Optional) You can enter more additional properties.
   a. In the *Additional Properties* panel, choose *Add Property*.
   b. Enter a key (name) or choose one from the dropdown menu and specify a value for the property. You can add as many properties as you need. Each key of an additional property must start with "`mail.`".
   c. To delete a property, choose the *Delete* icon next to it.
9. When you are done, choose *Create*.

Related Information

MAIL Destinations [page 1467]

1.3.1.2.2.2.6 Destination Examples

Find configuration examples for HTTP and RFC destinations in SAP BTP, using different authentication types.

Content

HTTP Destination (Internet, Client Certificate Authentication) [page 1486]

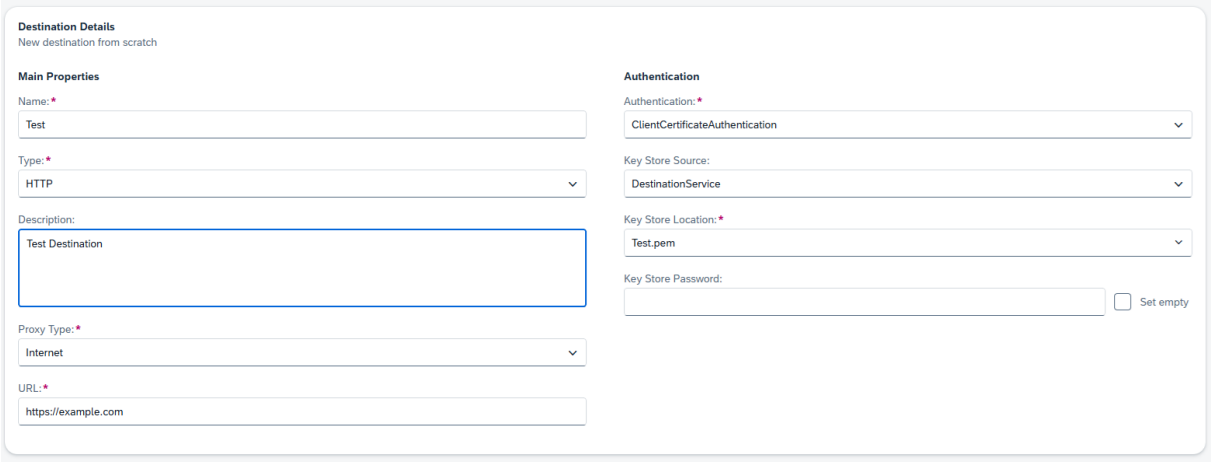
HTTP Destination (Internet, OAuth2SAMLBearerAssertion) [page 1487]

HTTP Destination (On-Premise) [page 1487]

RFC Destination [page 1488]

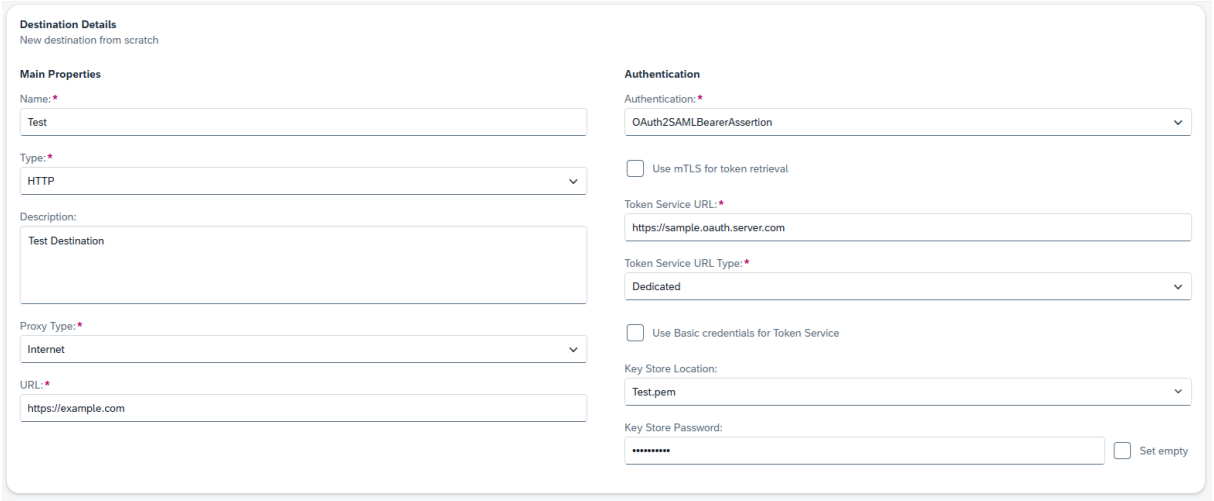
Mail Destination (Internet) [page 1488]

Example: HTTP Destination (Internet, Client Certificate Authentication)



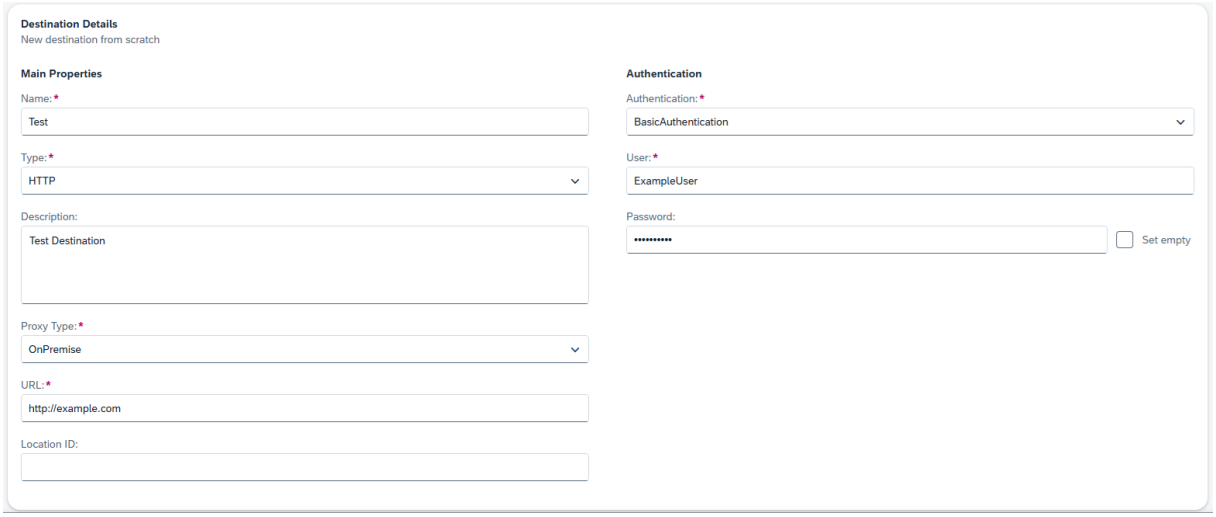
Back to Content [page 1486]

Example: HTTP Destination (Internet, OAuth2SAMLBearerAssertion)



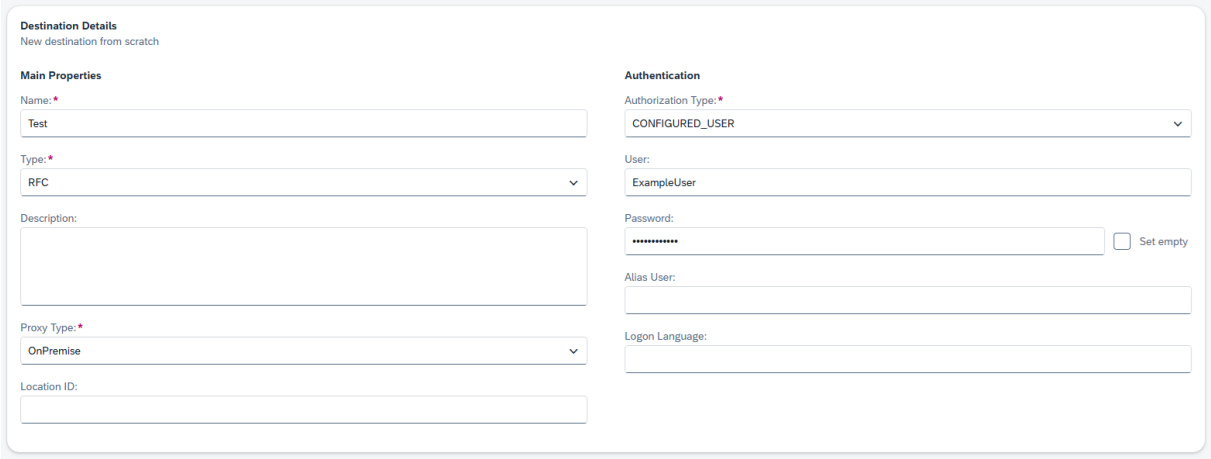
Back to Content [page 1486]

Example: HTTP Destination (On-Premise)



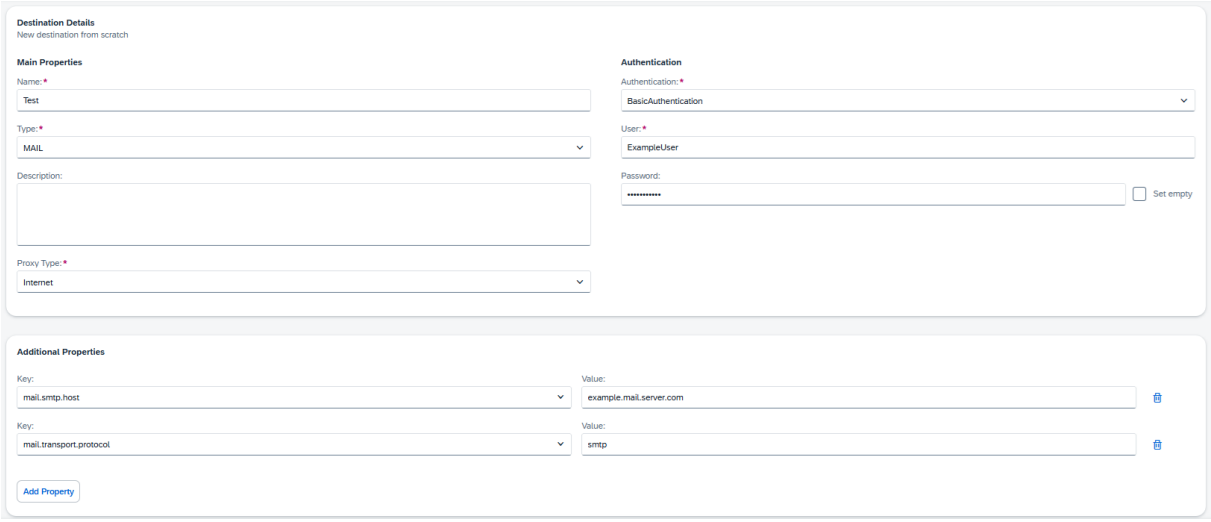
Back to Content [page 1486]

Example: RFC Destination



Back to Content [page 1486]

Example: Mail Destination (Internet)



Back to Content [page 1486]

Related Information

HTTP Destinations [page 1350]
RFC Destinations [page 1443]

MAIL Destinations [page 1467]

1.3.1.2.2.3 Create Destinations from a Template

Use a template to configure destinations with scenario-specific input data in the SAP BTP cockpit.

If you want to create several destinations for a common scenario, you can use a template that provides the scenario-specific input data. The Destination service uses the template to configure the destinations accordingly.

Currently, the following template is available for destination configuration:

Destinations Pointing to Service Instances [page 1489]

1.3.1.2.2.3.1 Destinations Pointing to Service Instances

Create a destination for subscription-based scenarios that points to your service instance.

> ⓘ Note
>
> This feature is applicable for a selected set of the most commonly used services (from a Destination service perspective). If you would like to use this feature for a service which is not yet supported, let us know by opening a support ticket, see Connectivity Support [page 2362].
>
> In the meantime, you can follow the steps described in Create Destinations from Scratch [page 1475].

Usually, in the Cloud Foundry environment, you consume service instances by binding them to your applications. However, in subscription-based scenarios this is not always possible. If you have purchased a subscription to an SaaS application that runs in a provider's subaccount, you cannot bind your service instance to this application.

In this case, you must create a destination that points to your service instance. Applications can consume this destination through a subscription to gain access to your service instance.

If you create such a destination from scratch, you must provide a service key for your instance, look up the credentials, and enter these values in the newly created destination.

Using the *Destinations Pointing to Service Instances* template, you only have to select the corresponding service instance.

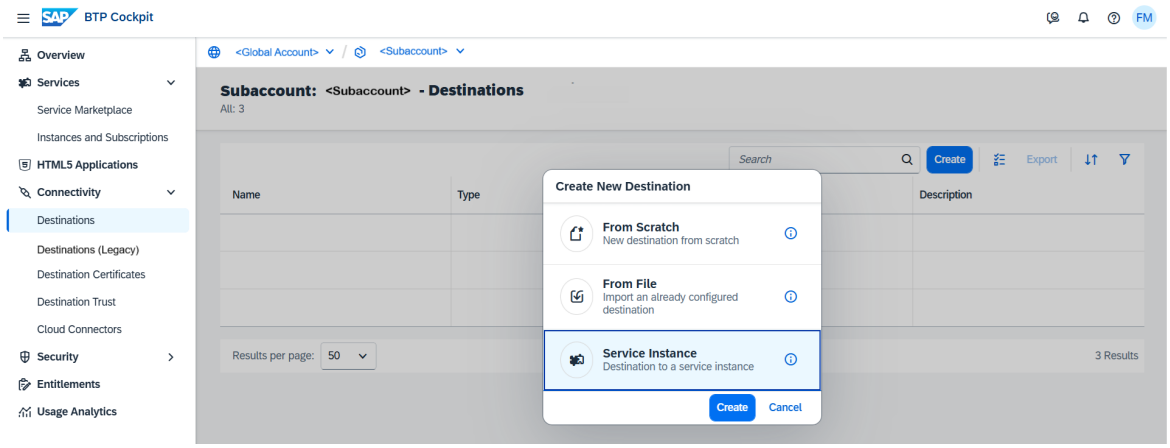
> ⓘ Note
>
> This procedure only applies for HTTP destinations on subaccount level.

Prerequisites

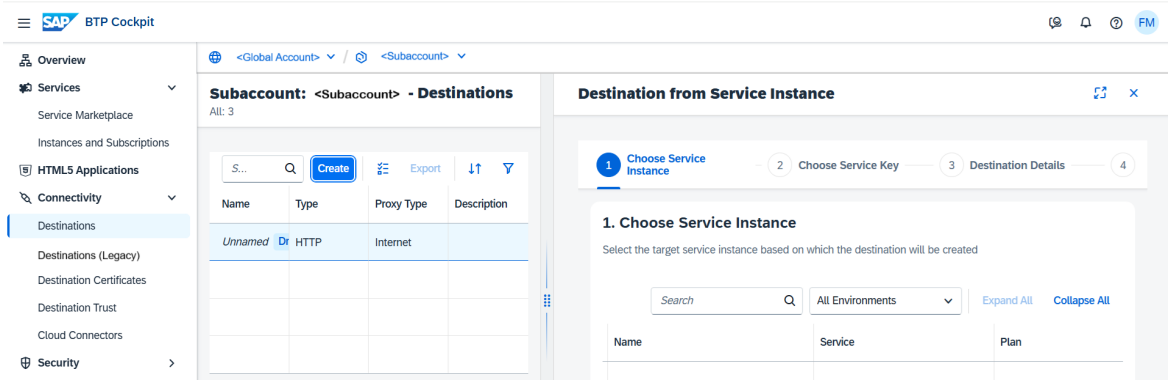
- You have a service instance which you want to make accessible to applications you are subscribed to.
- You have the Space Developer role in the space where this service instance resides (only relevant for Cloud Foundry environment).
- You have logged in to the cockpit and opened the *Destinations* editor on *subaccount* level. See Access the Destinations Editor [page 1473].

Procedure

1. Choose *Create* > *Service Instance* > *Create*.



2. Select a service instance and choose *Next*.



3. Choose an existing service key or generate a new one for the selected service instance. Choose *Next*.
4. Enter destination details and choose *Next*.
   For more information, see Create Destinations from Scratch [page 1475].
5. Review destination data.
6. Choose *Create*.

Result

You have a destination pointing to your service instance.

> ⚠ Caution
>
> If you delete the used service instance or its service key, the destination will stop working.

1.3.1.2.2.4 Check the Availability of a Destination

How to check the availability of a destination in the *Destinations* editor (SAP BTP cockpit).

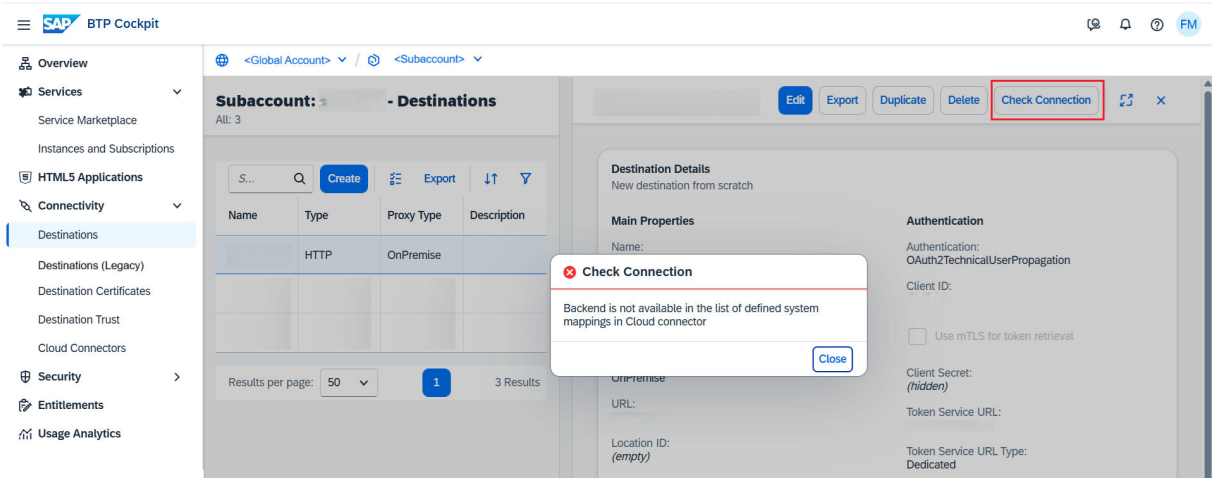
Prerequisites

You have logged into the cockpit and opened the *Destinations* editor.

Context

You can use the *Check Connection* button in the *Destinations* editor of the cockpit to verify if the **URL** configured for an HTTP Destination is reachable and if the connection to the specified system is possible.

For each destination, the check button is available in the destination detail view.



> ### Note
>
> The check does not guarantee that the target system or service is operational. It only verifies if a connection is possible.

This check is supported for destinations with `<Proxy Type>` **`Internet`** and **`OnPremise`**:

- For **`Internet`** destinations:
  - If the check receives an **HTTP status code above or equal to 500** from the targeted URL, the check is considered **failed**.
  - Every **HTTP status code below 500** is treated as **successful**.

  > ### Restriction
  >
  > The targeted URL cannot be a private endpoint (for example, *localhost*).

- For **`OnPremise`** destinations:
  - If the targeted backend is reached and returns an **HTTP status code below 500** the check is considered **successful**.

Error Messages for OnPremise Destinations

Error Message	Reason	Action
Backend status could not be determined.	• The Cloud Connector version is less than 2.7.1. • The Cloud Connector is not connected to the subaccount. • The backend returns an HTTP status code above or equal to 500 (server error). • The Cloud Connector is not configured properly.	• Upgrade the Cloud Connector to version 2.7.1 or higher. • Connect the Cloud Connector to the corresponding subaccount. • Check the server status (availability) of the backend system. • Check the basic Cloud Connector configuration steps: Initial Configuration [page 1670]
Backend is not available in the list of defined system mappings in Cloud Connector.	The Cloud Connector is not configured properly.	Check the basic Cloud Connector configuration steps: Initial Configuration [page 1670]
Resource is not accessible in Cloud Connector or backend is not reachable.	The Cloud Connector is not configured properly.	Check the basic Cloud Connector configuration steps: Initial Configuration [page 1670]
Backend is not reachable from Cloud Connector.	Cloud connector configuration is ok but the backend is not reachable.	Check the backend (server) availability.

1.3.1.2.2.5 Duplicate Destinations

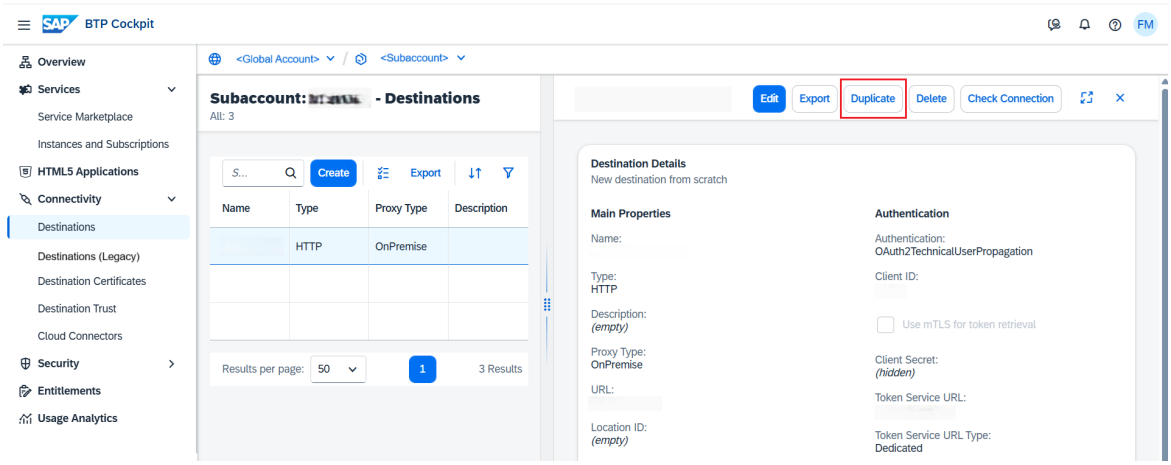
How to duplicate destinations in the *Destinations* editor (SAP BTP cockpit).

Prerequisites

You have previously created an HTTP destination in the *Destinations* editor of the cockpit.

Procedure

1. In the *Destinations* editor, select the existing destination which you want to duplicate, and open the detail view.
2. Choose *Duplicate*.



3. The editor automatically creates and opens a new destination that contains all the properties of the selected one.
4. You can modify some parameters if you need to.
5. When you are done, choose *Create*.

Related Information

Export Destinations [page 1496]
Destination Examples [page 1486]

1.3.1.2.2.6 Edit and Delete Destinations

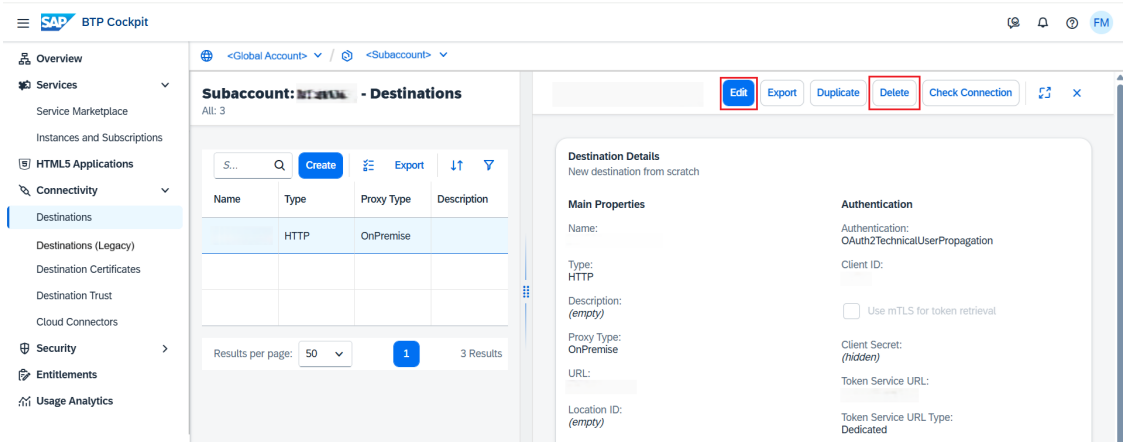
How to edit and delete destinations in the *Destinations* editor (SAP BTP cockpit).

Prerequisites

You have previously created an HTTP destination in the *Destinations* editor of the cockpit.

Procedure

- Edit a destination:
  1. To edit an existing destination, select the destination to open its detail view.
  2. Choose *Edit*.



  3. You can edit the main parameters as well as the additional properties of a destination.
  4. When you are done, choose *Save*. The changes will take effect in up to five minutes.

> → Tip
>
> For complete consistency, we recommend that you first stop your application, then apply your destination changes, and then start again the application. Also, bear in mind that these steps will cause an application downtime.

- Delete a destination:
  To remove an existing destination, choose *Delete* in the detail view. The changes will take effect in up to five minutes.

Related Information

Export Destinations [page 1496]

Destination Examples [page 1486]

1.3.1.2.2.7 Import Destinations

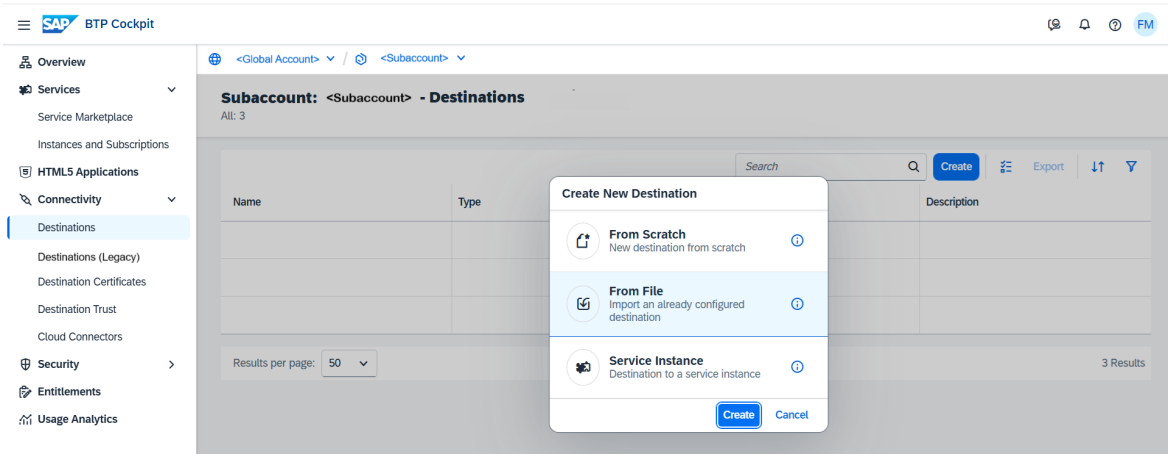
How to import destinations in the *Destinations* editor (SAP BTP cockpit).

Prerequisites

You have previously stored a destination configuration as JSON, YAML, or Properties file.

Procedure

1. Log into the cockpit and open the *Destinations* editor.
2. Choose *Create* > *From File* > *Create*.



3. Browse to a configuration file that contains the destination configuration.
4. Select the file and upload it.
5. Review and save the destination.

Related Information

Edit and Delete Destinations [page 1494]
Destination Examples [page 1486]

1.3.1.2.2.8 Export Destinations

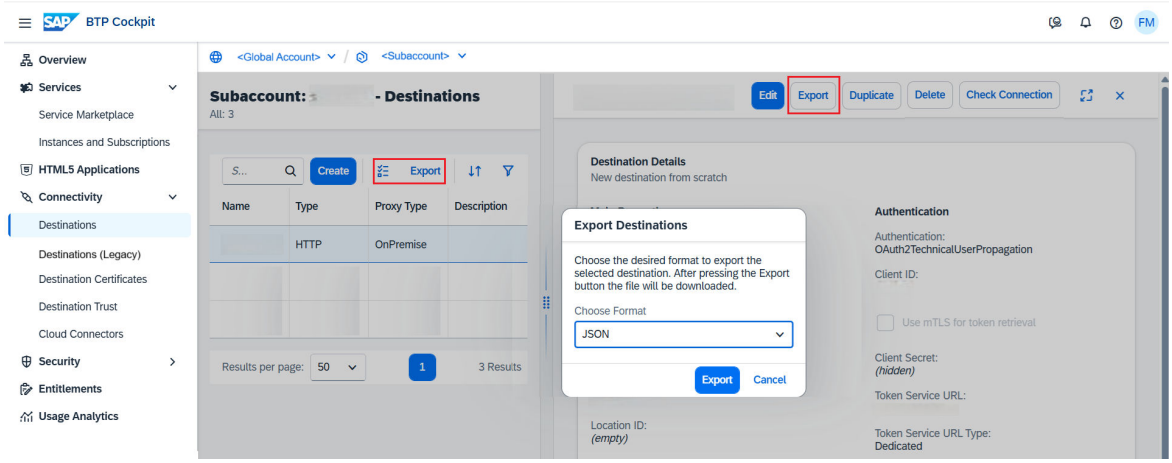
Export destinations from the *Destinations* editor in the SAP BTP cockpit to backup or reuse a destination configuration.

Prerequisites

You have created a destination in the *Destinations* editor.

Procedure

1. Log into the cockpit and open the *Destinations* editor.
2. To export a single destination, select it and choose *Export* in the detail view. To export several destinations at once, choose *Turn on bulk actions* and select all destinations you want to export. Then choose *Export* in the destination overview list.



3. Select the file format you want to use for download and choose *Export*. A file will be downloaded to your default download folder.
   The download includes the destination data in the selected file format. When downloading several destinations at once, their data will be downloaded as an archive file.

Related Information

Edit and Delete Destinations [page 1494]
Destination Examples [page 1486]

1.3.1.2.3 Destination Service REST API

Destination service REST API specification for SAP BTP.

The Destination service provides a REST API that you can use to read and manage resources like destinations, certificates and destination fragments on all available levels. This API is documented in the SAP Business Accelerator Hub.

It shows all available endpoints, the supported operations, parameters, possible error cases and related status codes, etc. You can also execute requests using the credentials (for example, the service key) of your Destination service instance, see Create and Bind a Destination Service Instance [page 1523].

Related Information

Calling the Destination Service REST API [page 1562]
"Find a Destination" Response Structure [page 1565]
Referring Resources Using the Consume REST API [page 1569]
Calling the Destination Service REST API via IAS Token [page 1575]

1.3.1.2.4 Create Destinations Using the MTA Descriptor

Use the multitarget-application (MTA) descriptor to manage destinations for complex deployments.

Content

- Concept [page 1497]
- Content Deployment [page 1498]
- Create a Destination on Service Instance Creation [page 1502]

Concept

When modeling a multitarget application (MTA), you can create and update destinations from your MTA descriptor.

For more information on MTA, see Multitarget Applications in the Cloud Foundry Environment.

Back to Content [page 1497]

Content Deployment

When modeling MTAs, you can configure content deployments (for more information, see Deploying Content with Generic Application Content Deployment). The Destinations service supports such content deployments, which lets you create or update destinations by modeling them in the MTA descriptor. Other operations, like deleting a destination, are not supported by this method.

Parameters

The parameters of the content deployment have the following structure:

```
content:
  subaccount:
    existing_destinations_policy: <policy> # optional, default value is "fail".
See "Existing destinations policy" for more details
    destinations:
    - <destination descriptor 1> # See "Modeling options" to learn about the
structure of this descriptor
    ...
    - <destination descriptor N> # See "Modeling options" to learn about the
structure of this descriptor
  instance:
    existing_destinations_policy: <policy> # optional, default value is "fail".
See "Existing destinations policy" for more details
    destinations:
    - <destination descriptor 1> # See "Modeling options" to learn about the
structure of this descriptor
    ...
    - <destination descriptor N> # See "Modeling options" to learn about the
structure of this descriptor
```

> ### Note
>
> Both the `subaccount` and `instance` sections are optional. They can both be present at the same time, or only one of them. They define the level on which the resulting destination is created.

Existing Destinations Policy

The `existing_destinations_policy` setting allows you to control what happens if a destination with the same name already exists. The possible values are:

- `fail`: Treat it as an error situation and fail the deployment. This is the default value of the setting.
- `ignore`: Keep what is currently saved in the Destination service, and skip deployment for this destination.
- `update`: Override what is currently saved in the Destination service.

Modeling Options

The `destinations` section represents an array of destination descriptors. Each of these array elements is converted to a destination and saved in the service on the respective level, based on the existing destination policy. The following options are available for modeling a destination descriptor via content deployment. They can be combined:

Destination Pointing to a Service Instance

This option lets you:

- Reference a service instance and a service key
- Specify a destination name
- Enter a description for the resulting destination (optional)
- Add additional properties and override default properties (optional)

As a result, a destination is created, based on the properties in the referenced service key.

> **Note**
>
> This function is equivalent to the Destinations Pointing to Service Instances [page 1489] template.

> **Caution**
>
> This feature is applicable for a selected set of the most commonly used services (from a Destination service perspective). If you would like to use this feature for a service which is not yet supported, let us know by opening a support ticket, see Connectivity Support [page 2362].
>
> In the meantime, you can follow the steps described in Create Destinations from Scratch [page 1475].

The descriptor may have the following structures:

Without client-provided service credentials:

```
Name: <name> # name of the destination
Description: <description> # optional, a description for the destination
Authentication: <auth> # optional for some services (the default is service
specific). Some services require the Authentication to be specified, like the
workflow service. The allowed authentication types are also service-specific
ServiceInstanceName: <instance name> # the name of the service instance to which
the destination will be created
ServiceKeyName: <service key name> # the service key of the instance targeted by
ServiceInstanceName which will be used as the source for the destination values
AdditionalProp1: <value1> # optional
...
AdditionalPropN: <valueN> # optional
```

Applicable for:

- Service bindings of type "x509" which contain both a public and private key in the service key credentials
- Service bindings of type "clientsecret"

With client-provided service credentials:

```
Name: <name> # name of the destination
Description: <description> # optional, a description for the destination
Authentication: <auth> # optional for some services (the default is service
specific). Some services require the Authentication to be specified, like the
workflow service. The allowed authentication types are also service-specific
ServiceInstanceName: <instance name> # the name of the service instance to which
the destination will be created
ServiceKeyName: <service key name> # the service key of the instance targeted by
ServiceInstanceName which will be used as the source for the destination values
ServiceCredentials: # the credentials of the service key which will be provided
by the client
  PrivateKey: <value>
  ...
```

```
AdditionalProp1: <value1> # optional
...
AdditionalPropN: <valueN> # optional
```

Applicable for:

- Service bindings of type "x509" which contain only the public key in the service key credentials

Destination Pointing to a Resource Protected by an XSUAA Service Instance

This option lets you:

- Reference a service instance and a service key
- Specify a destination name
- Enter a description for the resulting destination (optional)
- Specify the URL of the target resource
- Add additional properties and override default properties (optional)

As a result, a destination is created with the token service configuration based on the properties in the referenced service key, while the URL will be the one specified when modeling the destination.

The descriptor may have the following structures:

Without client-provided service credentials:

```
Name: <name> # name of the destination
Description: <description> # optional, a description for the destination
Authentication: <auth> # optional for some services (the default is service
specific). Some services require the Authentication to be specified, like the
workflow service. The allowed authentication types are also service-specific
URL: <url> # the URL of the target resource
TokenServiceInstanceName: <instance name> # the name of the service instance
used for protecting the target resource
TokenServiceKeyName: <service key name> # the service key of the instance
targeted by ServiceInstanceName which will be used as the source for the token
service values in the destination
AdditionalProp1: <value1> # optional
...
AdditionalPropN: <valueN> # optional
```

Applicable for:

- Service bindings of type "x509" which contain both a public and private key in the service key credentials
- Service bindings of type "clientsecret"

With client-provided service credentials:

```
Name: <name> # name of the destination
Description: <description> # optional, a description for the destination
Authentication: <auth> # optional for some services (the default is service
specific). Some services require the Authentication to be specified, like the
workflow service. The allowed authentication types are also service-specific
URL: <url> # the URL of the target resource
TokenServiceInstanceName: <instance name> # the name of the service instance
used for protecting the target resource
```

```
TokenServiceKeyName: <service key name> # the service key of the instance
targeted by ServiceInstanceName which will be used as the source for the token
service values in the destination
TokenServiceCredentials: # the credentials of the service key which will be
provided by the client
  PrivateKey: <value>
AdditionalProp1: value1 # optional
...
AdditionalPropN: valueN # optional
```

Applicable for:

- Service bindings of type "x509" which contain only the public key in the service key credentials

Example:

```
_schema-version: "3.2"
ID: example
version: 0.0.1
modules:
- name: myapp
  path: ./myapp
  type: javascript.nodejs
  requires:
  - name: xsuaa_service
  provides:
  - name: myapp-route
    properties:
      url: ${default-url} #generated during deployment
- name: destination-content
  type: com.sap.application.content
  requires:
  - name: xsuaa_service
    parameters:
      service-key:
        name: xsuaa_service-key
  - name: destination-service
    parameters:
      content-target: true
  - name: myapp-route
  build-parameters:
    no-source: true
  parameters:
    content:
      subaccount:
        existing_destinations_policy: update
        destinations:
        - Name: myappOauth
          URL: ~{myapp-route/url}
          Authentication: OAuth2ClientCredentials
          TokenServiceInstanceName: xsuaa_service
          TokenServiceKeyName: xsuaa_service-key
          myAdditionalProp: myValue
        - Name: workflowOauthJwtBearer
          Authentication: OAuth2JWTBearer
          ServiceInstanceName: workflow_service
          ServiceKeyName: workflow_service-key
      instance:
        existing_destinations_policy: update
        destinations:
        - Name: workflowBasicAuthentication
          Authentication: BasicAuthentication
          ServiceInstanceName: workflow_service
          ServiceKeyName: workflow_service-key
          myAdditionalProp: myValue
```

```
resources:
- name: xsuaa_service
  type: org.cloudfoundry.managed-service
  parameters:
    service: xsuaa
    service-name: xsuaa_service
    service-plan: application
    config:
      xsappname: "myApp"
- name: workflow_service
  type: org.cloudfoundry.managed-service
  parameters:
    service: workflow
    service-name: workflow_service
    service-plan: lite
- name: destination-service
  type: org.cloudfoundry.managed-service
  parameters:
    service: destination
    service-name: my-destination-service
    service-plan: lite
```

Back to Content [page 1497]

Create a Destination on Service Instance Creation

The MTA descriptor lets you create service instances and provide a JSON configuration for this operation. You can use this functionality to create a Destination service instance with a JSON, and include the required data to create or update destinations.

For more details, see Use a Config.JSON to Create or Update a Destination Service Instance [page 1525].

Back to Content [page 1497]

1.3.1.2.5 Create Destinations on Service Instance Creation

Use a JSON to create or update a destination when creating a Destination service instance.

When creating or updating a service instance of the Destination service, you can provide a JSON object with various configurations. One of the sections of this JSON lets you create or update destinations. Other operations, like deleting a destination, are not supported by this method.

For more information, see Use a Config.JSON to Create or Update a Destination Service Instance [page 1525].

1.3.1.3 Manage Destination Fragments

Destination fragments are objects used to override and extend destination properties through the Destination service REST API.

> **Note**
>
> Management of destination fragments for your application is currently only possible using the Destination Service REST API [page 1497].

You can use destination fragments to override and/or extend destination properties as result of the "Find a Destination" REST API request [page 1562]. These properties are key-value based objects which contain a name for identification and additional configurable properties.

The structure of the destination fragment is the following:

Property	Description
FragmentName	Mandatory. Name of the destination fragment. It must be unique for the level on which it is stored/maintained.
key1	Optional. Holds the respective value for key1.
key2	Optional. Holds the respective value for key2.
keyN	Optional. Holds the respective value for keyN.

> **Restriction**
>
> The destination fragment must not contain the properties `Name` or `Type`.

> **Caution**
>
> Only a single destination fragment can be used when requesting a destination via the "Find a Destination" REST API.

> **Caution**
>
> Destination fragments can be used only at runtime by the client of the Destination service when executing the request to the "Find a Destination" REST API.

Managing destination fragments for your application is supported only by the Destination service REST API. This API is documented in the SAP Business Accelerator Hub.

Related Information

Extending Destinations with Fragments [page 1590]

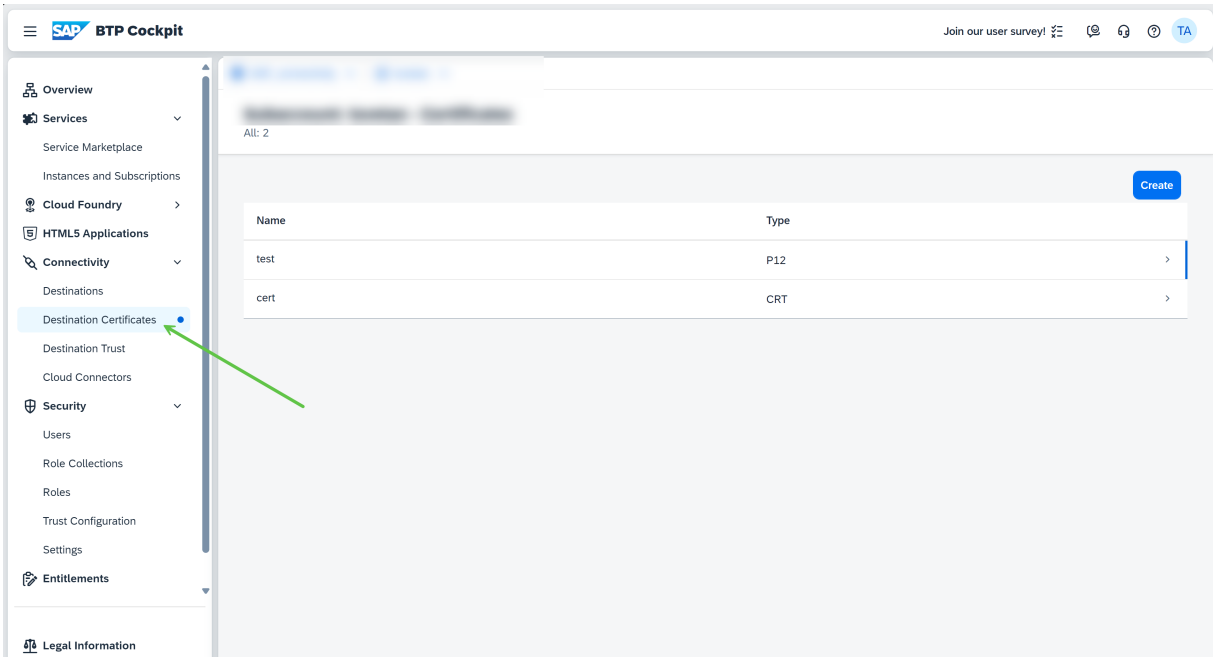
Calling the Destination Service REST API [page 1562]

1.3.1.4 Manage Destination Certificates

To use certificate-based authentication methods for a specific destination, you can maintain the corresponding certificates (X.509 client certificates, trusted certificates, CA certificates, trust store and key store certificates) in the **Destination Certificates** UI (SAP BTP cockpit).

Prerequisites

You have logged on to the cockpit and opened the *Certificate* view by choosing *Connectivity* > *Destination Certificates*.



Context

> **Note**
>
> You can manage certificates also via the Destination Service REST API [page 1497].

> ⚠ Caution
>
> Uploaded certificates are accessible via the Destination Service REST API [page 1497], including any private data they may contain.

You can maintain truststore and keystore certificates in the *Destinations* editor. You can upload, generate, view and delete certificates for your destinations and destination fragments.

- You can use JKS, PFX, PEM, CER, CRT, and P12 files.
- Certificates can both be used as standalone entities or referenced in various destination types.
- An uploaded certificate file must contain the entire certificate chain.

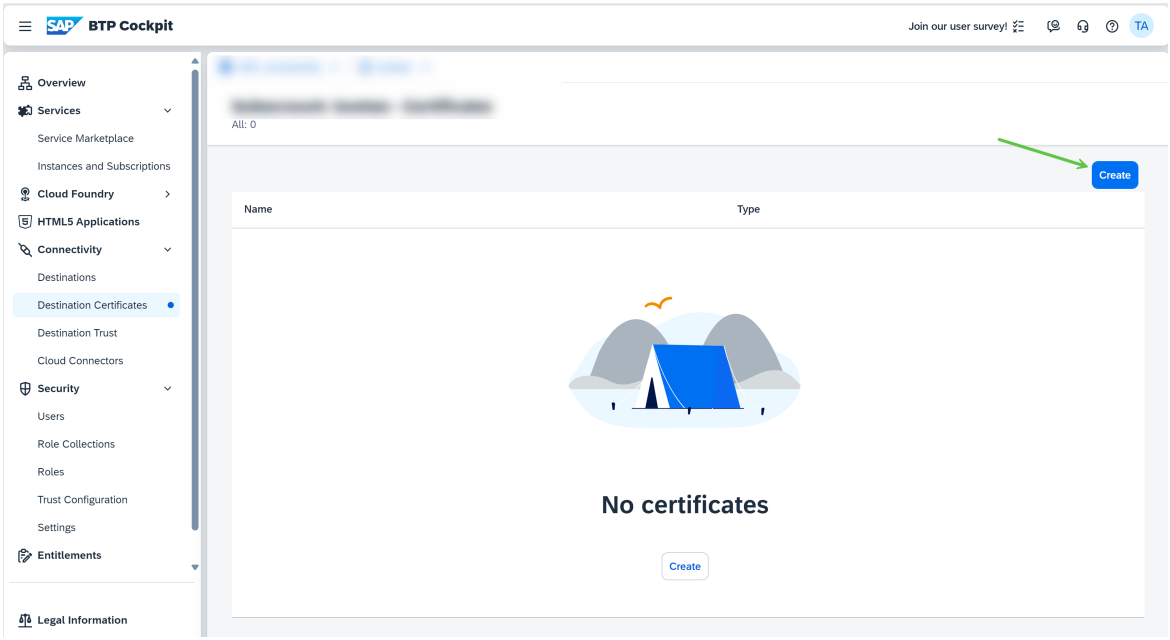
> ⓘ Note
>
> If you generate a certificate using the **Destination Certificates** UI or the **Destinations** editor in the cockpit, the certificate includes only the public key. You can obtain the private key from the Destination Service REST API [page 1497].

Procedure

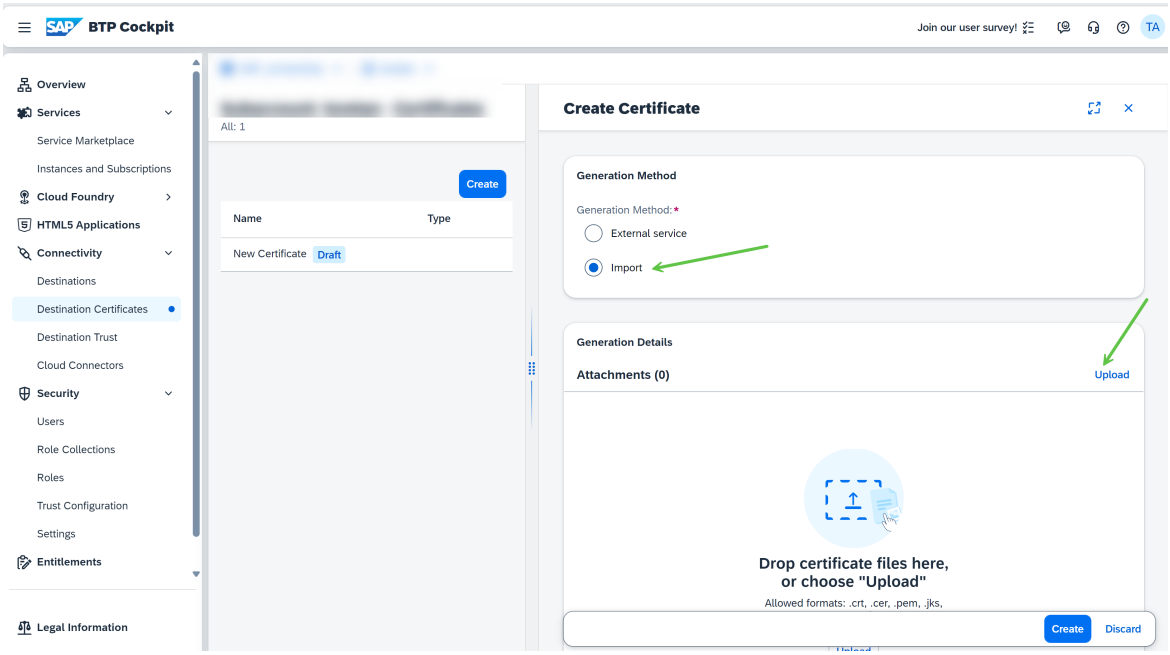
Upload Certificates

> ⚠ Caution
>
> Certificates added through the *Upload Certificate* option cannot be automatically renewed.

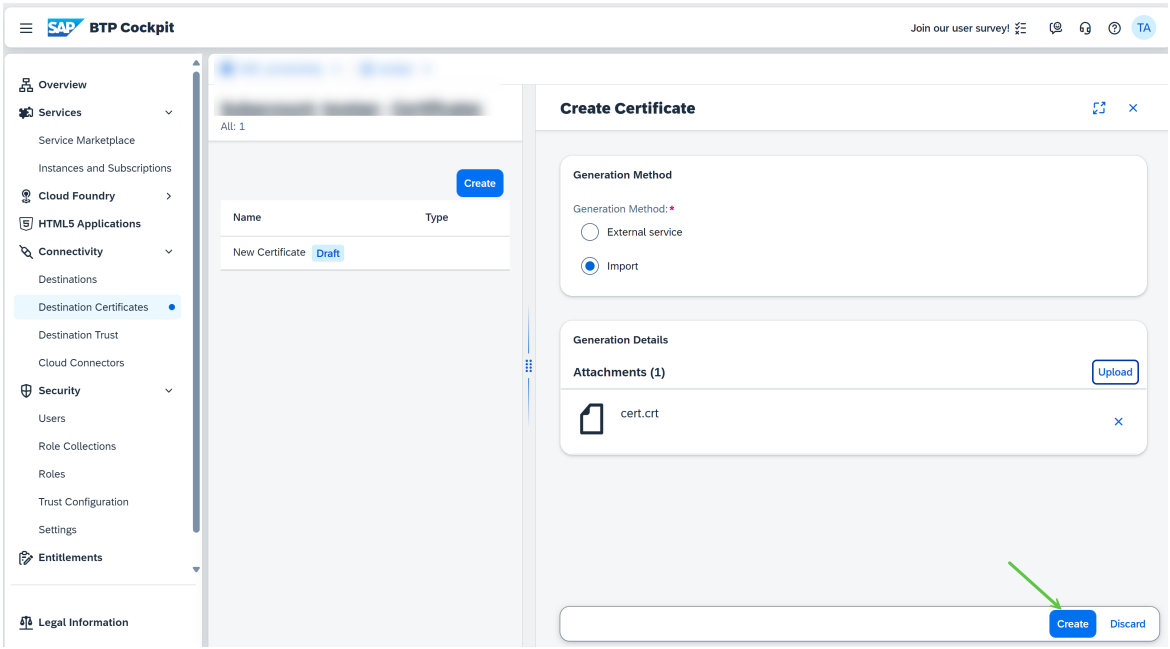
1. Choose *Create*.



2. Select *Import* and choose the *Upload* button.

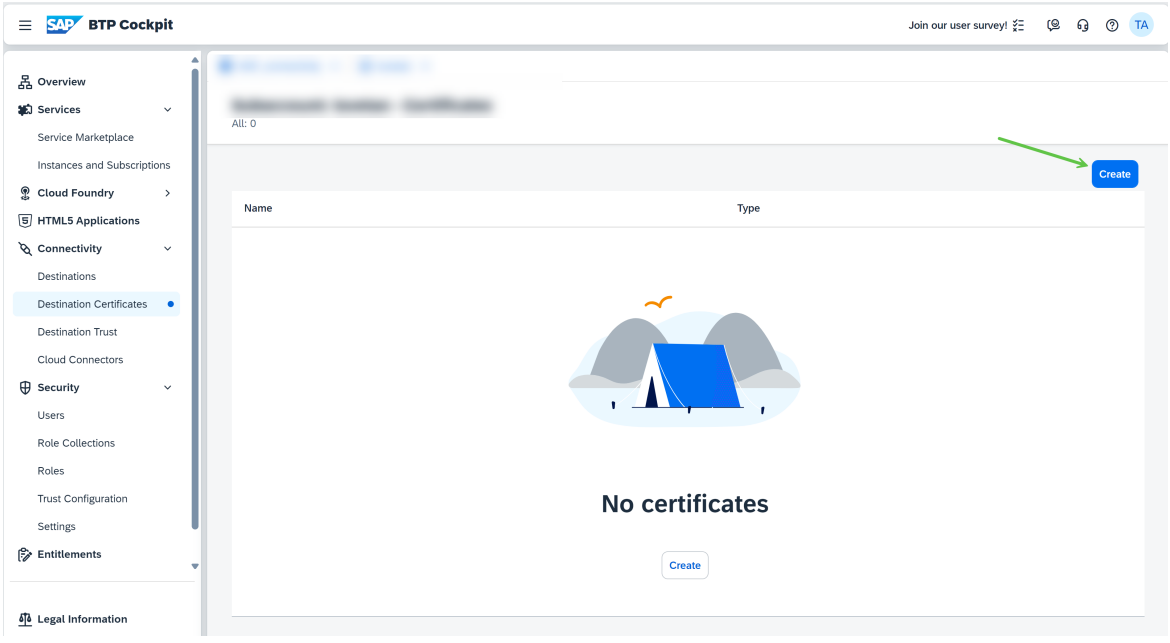


3. From your file system, select (or drag and drop) the certificate file you want to upload.
4. Choose *Create*.

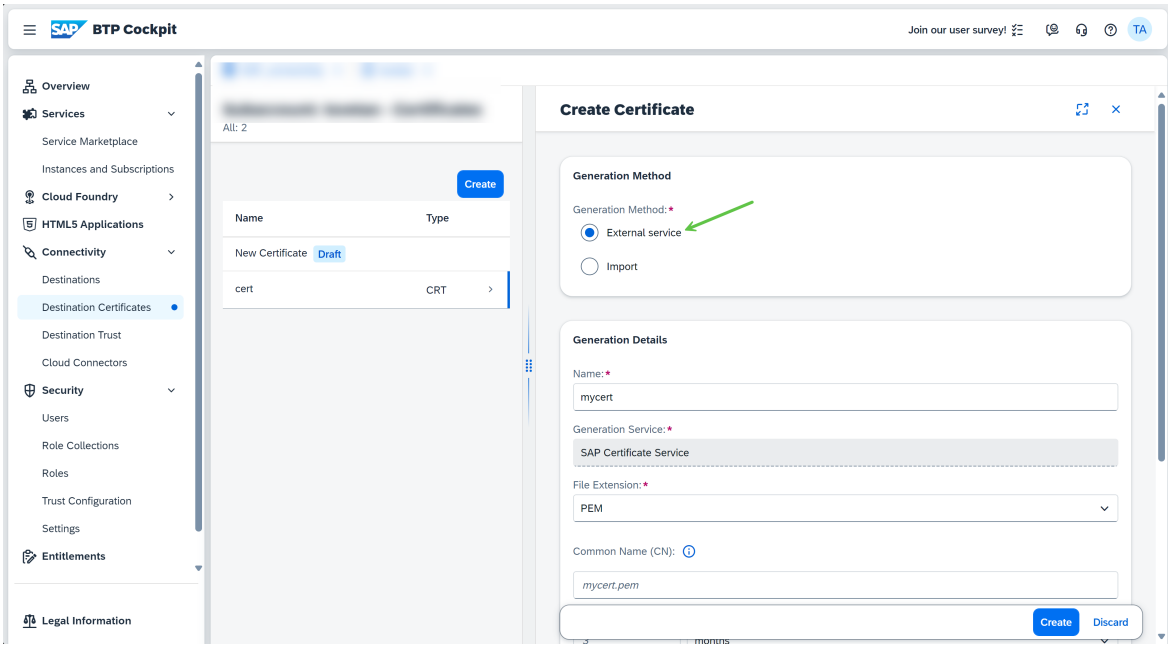


Generate an SAP Cloud PKI infrastructure Certificate

1. Choose *Create*.



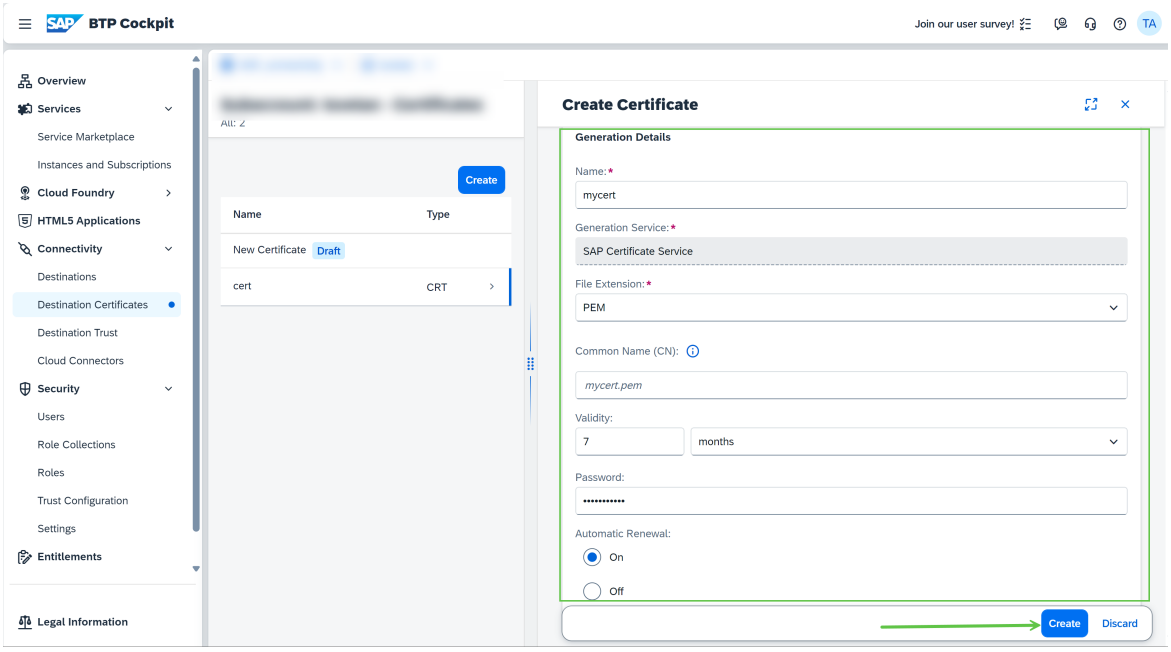
2. Select *External service* as method.



3. Enter certificate name and type. You can optionally enter the certificate CN and certificate validity. Also, you can optionally select the *Enable automatic renewal* checkbox to automatically renew the certificate when close to expiration. For more information, see Automatic Renewal of Generated Certificates [page 1513].

> ### Note
>
> In addition, you can optionally upload your own signed *certificate signing request*. In this case, the Destination service will still override the whole subject DN in the resulting certificate, but will not generate its own private key. Instead, you will be using your own local private key.

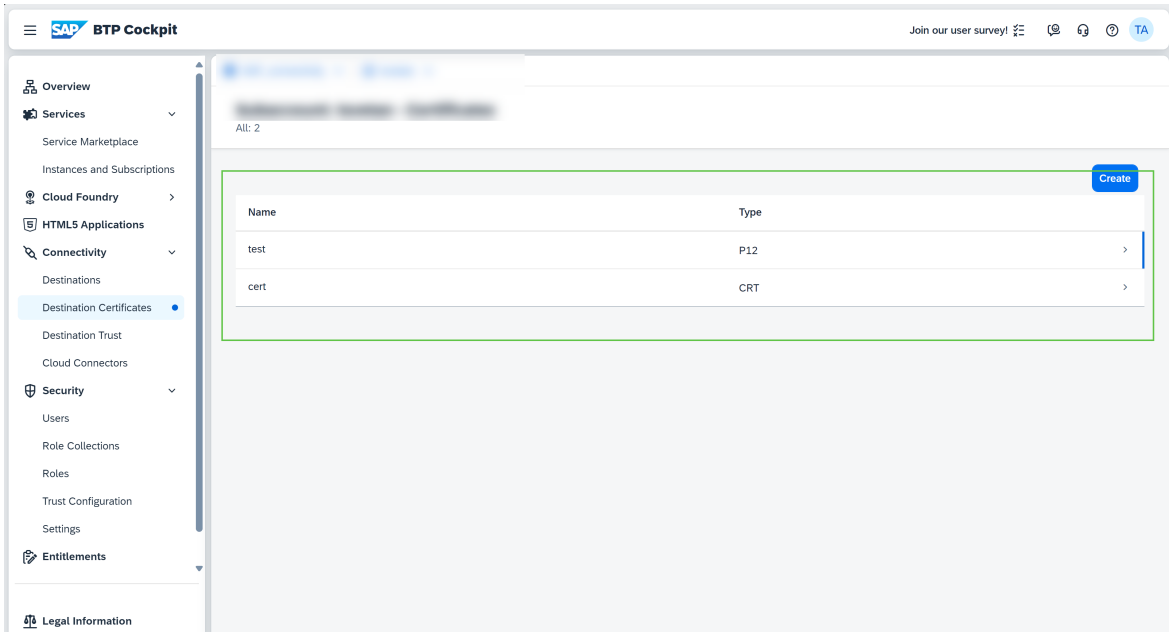
Choose *Create* to generate the desired certificate.



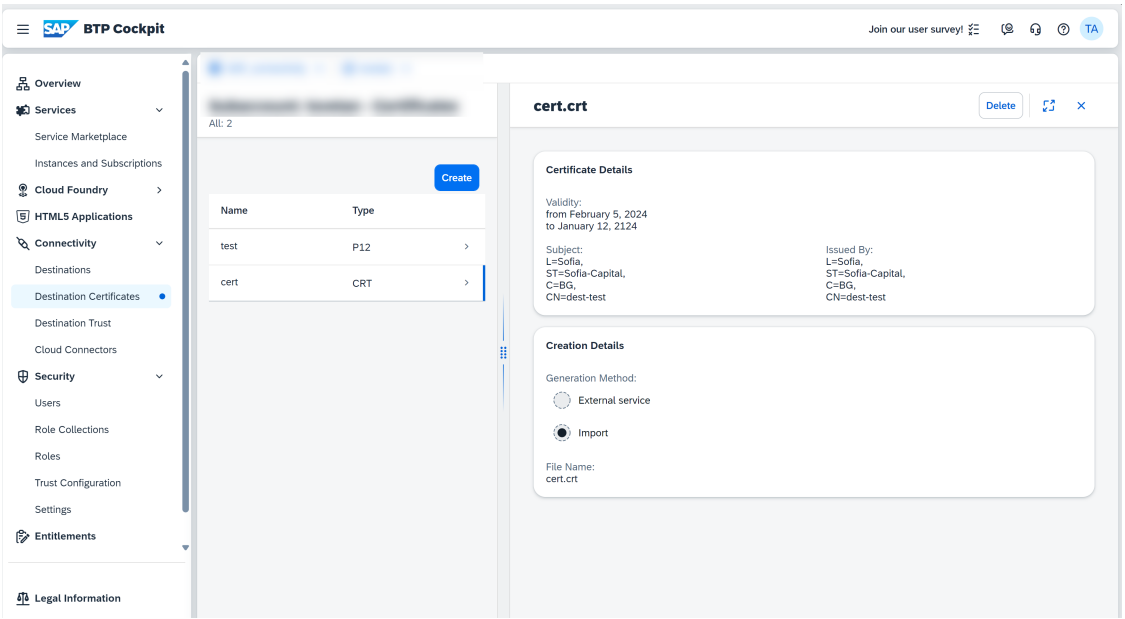
4. The certificate will be generated and its detail view is shown.

View a Certificate

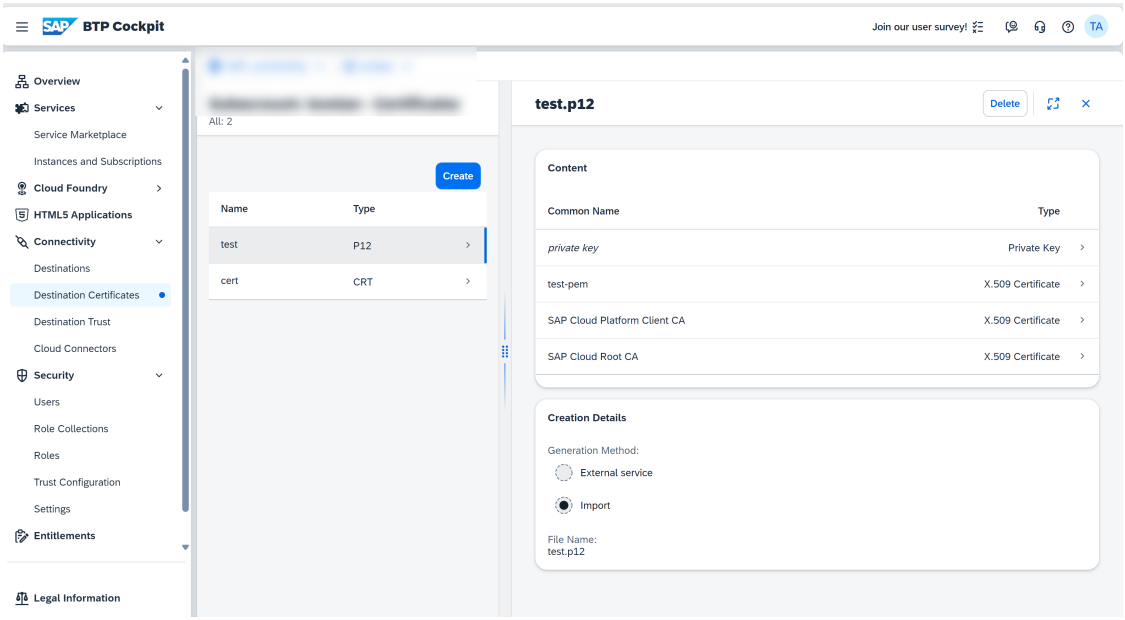
1. Once the certificate view is opened, it will display a table of all available certificates and keystores.



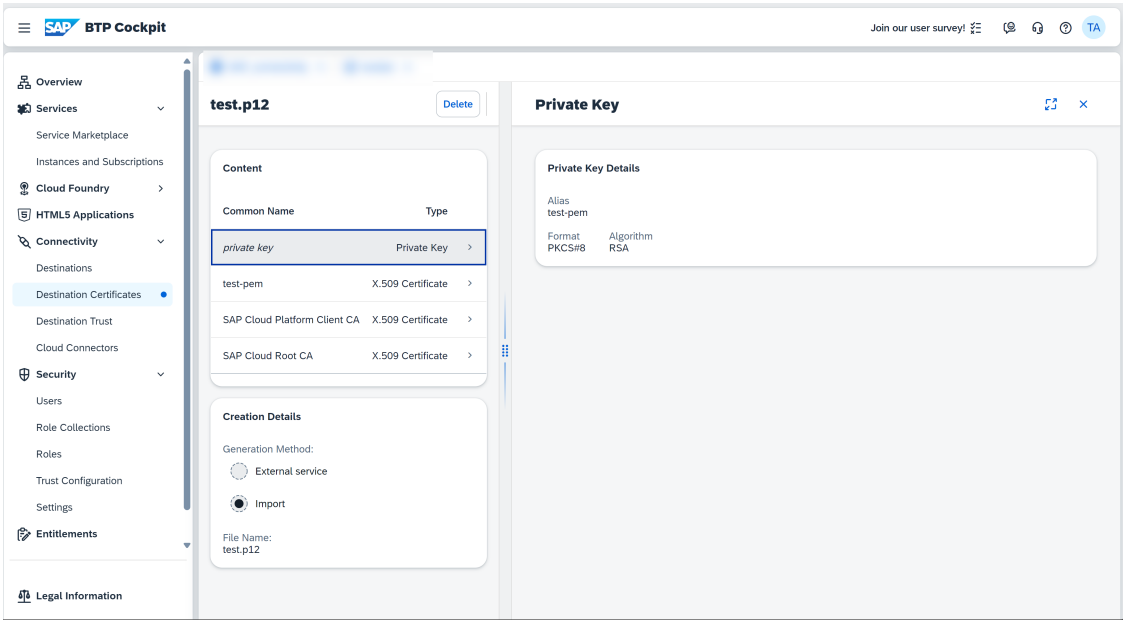
2. Click on the certificate you want to view. A panel to the right will open with the details.
3. Depending on the entity (single certificate or key store), the information is displayed differently:
    1. If it is a **single certificate**, you will see information about the validity and the subject and issuer DNs. In addition, you will also see the method used to create it.

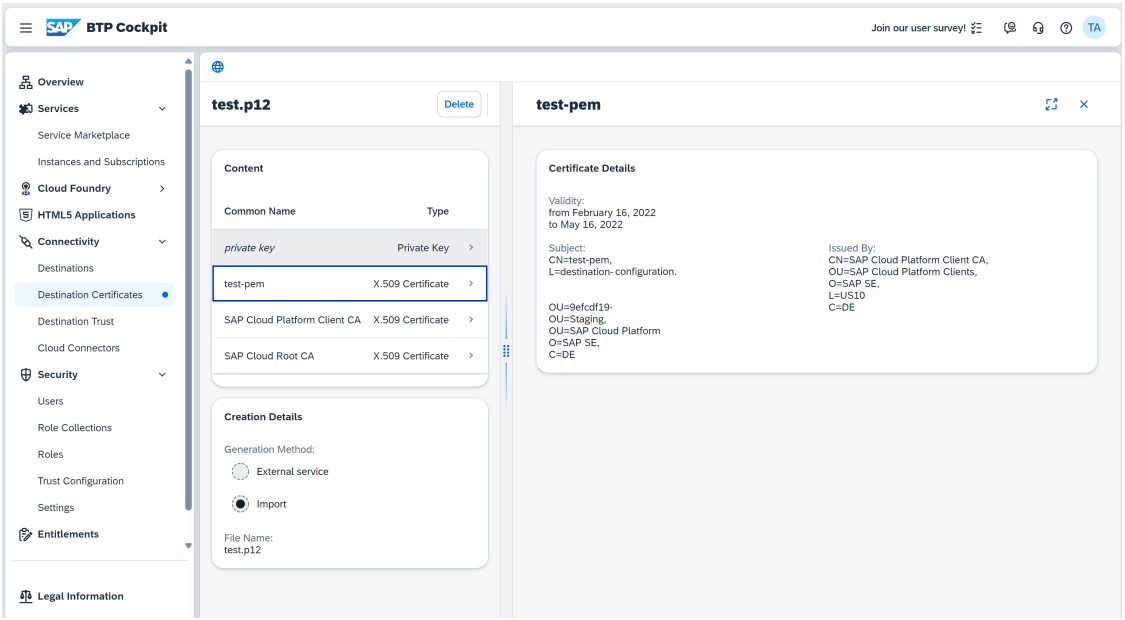


    2. If it is a **keystore**, you will see all entries in the keystore, and the method used to create it.



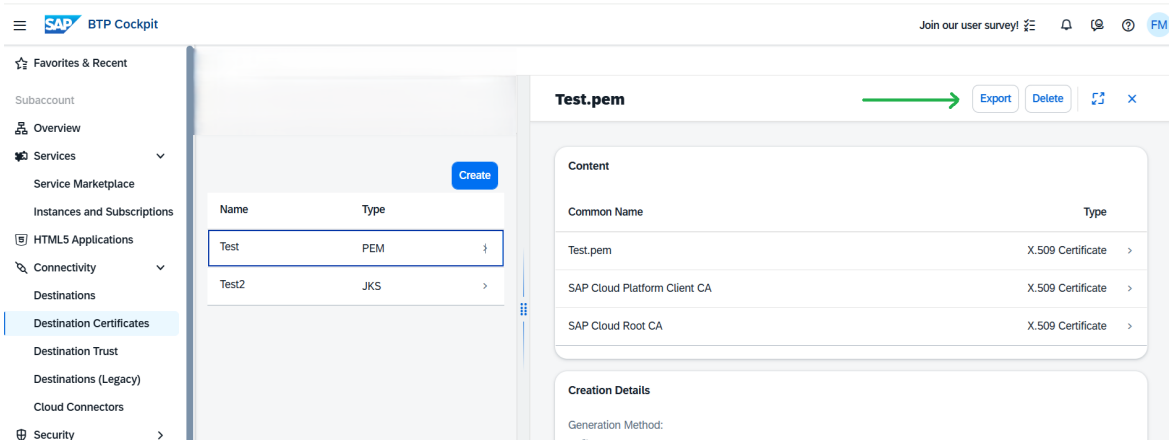
You can also click on each entry to view the details.





Export a Certificate

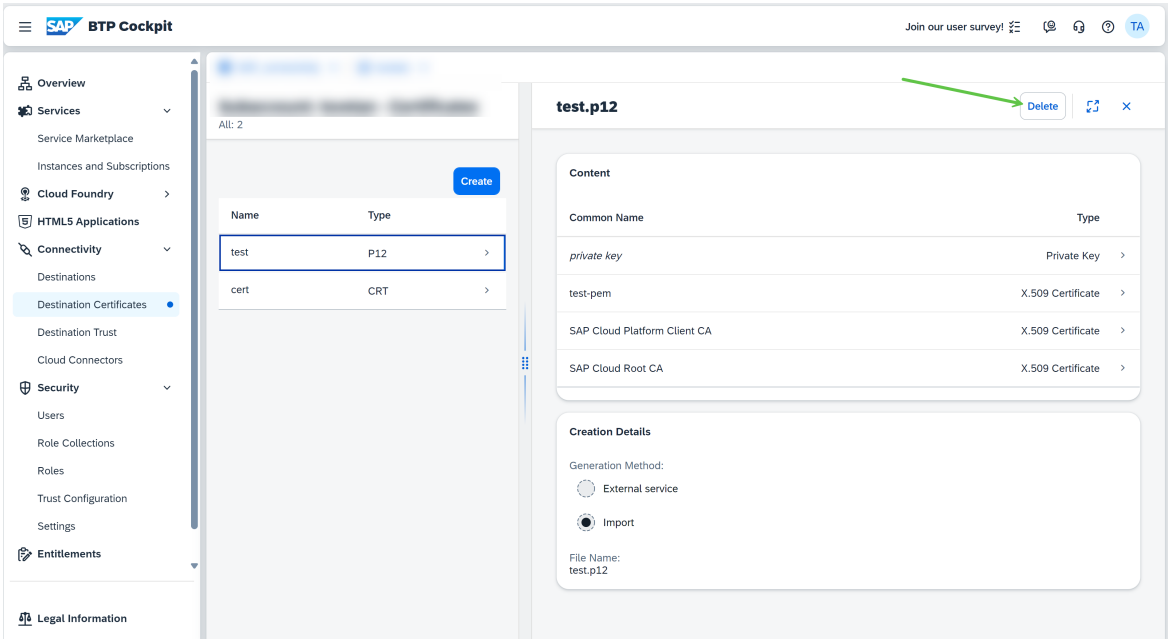
1. From the certificate list, select the one you want to export.
2. Choose *Export*.



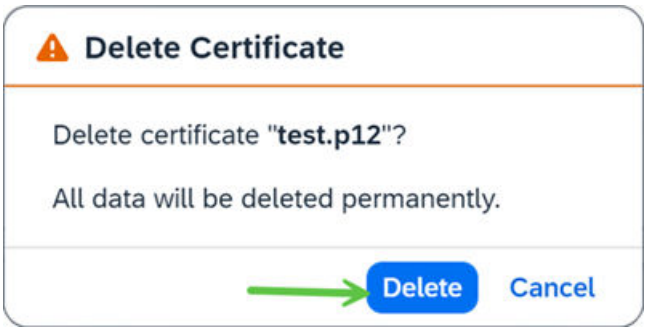
3. The certificate will be exported to your download folder.

Delete a Certificate

1. From the certificate list, select the one you want to delete.
2. Choose *Delete*.



3. Confirm deletion.



Related Information

Certificate Types [page 1512]

Manage Certificates Issued by the SAP Cloud Root CA [page 134]

Automatic Renewal of Generated Certificates [page 1513]

1.3.1.4.1 Certificate Types

Find information on different certificate types you can use to configure destinations in your SAP BTP subaccount.

Certificate Type	Description
Single (X.509) Certificate	Binds an identity to a public key using a digital signature. A certificate contains an identity (hostname, organization, or individual) and a public key. It is either signed by a certificate authority or is self-signed. Supported file formats: PEM, CER, CRT
Key Store Certificate	Certificate stored in a key store repository, containing the public and private key of the certificate. Supported file formats: JKS, PFX, P12 **⚠ Restriction** Key stores are not supported in environments based on FIPS (*Federal Information Processing Standard*).

1.3.1.4.2 Automatic Renewal of Generated Certificates

Configure automatic certificate renewal when generating a certificate.

When generating a certificate, you can choose automatic renewal when it gets close to expiration. This will result in a fresh certificate with the same subject and validity duration as the original one. The procedure will continue for the whole lifecycle of the certificate, allowing you to just trust it based on the root CA and the certificate subject on the recipient's side. As soon as configured, there is no more need for manual rotations and trust reestablishment.

> **ⓘ Note**
>
> The renewal is based on a periodic job that iterates the certificates and decides if they have passed the renewal threshold. Therefore, the exact renewal time may vary slightly, depending on the job schedule.

Find below the thresholds when renewal will be attempted for the first time. They are based on the overall validity of the certificate.

> **ⓘ Note**
>
> If for any reason the certificate can not be renewed at the time, it will be reattempted on the next job run.

Renewal Thresholds

Certificate Validity	Eligible for Renewal
1 day	12 hours before expiration
2–14 days	1 day before expiration
15–31 days	3 days before expiration
32–154 days	7 days before expiration
155–365 days	30 days before expiration
1 month	3 days before expiration
2–5 months	7 days before expiration
6–12 months	30 days before expiration
Any number of years	30 days before expiration

1.3.1.5 Manage Trust

Download and configure X.509 certificates as a prerequisite for user propagation from the multi-cloud environment.

Setting up a trust scenario for user propagation (also known as *principal propagation*) requires the exchange of public keys and certificates between the affected systems, as well as the respective trust configuration within these systems. This enables you to use an HTTP destination with authentication type `OAuth2SAMLBearerAssertion` or `SAMLAssertion` for the communication.

> **Note**
>
> The described procedures are focused on doing them via the *Destinations* editor in the cockpit. However, they are also fully automatable via the Destination Service REST API [page 1497]. References to the respective API endpoints are added to the relevant steps below.

A trust scenario can include user propagation from the multi-cloud environment to another SAP BTP environment, to another multi-cloud subaccount, or to a remote system outside SAP BTP, like SAP S/4HANA Cloud, SAP Sales Cloud, SAP Success Factors, and others.

Prerequisites

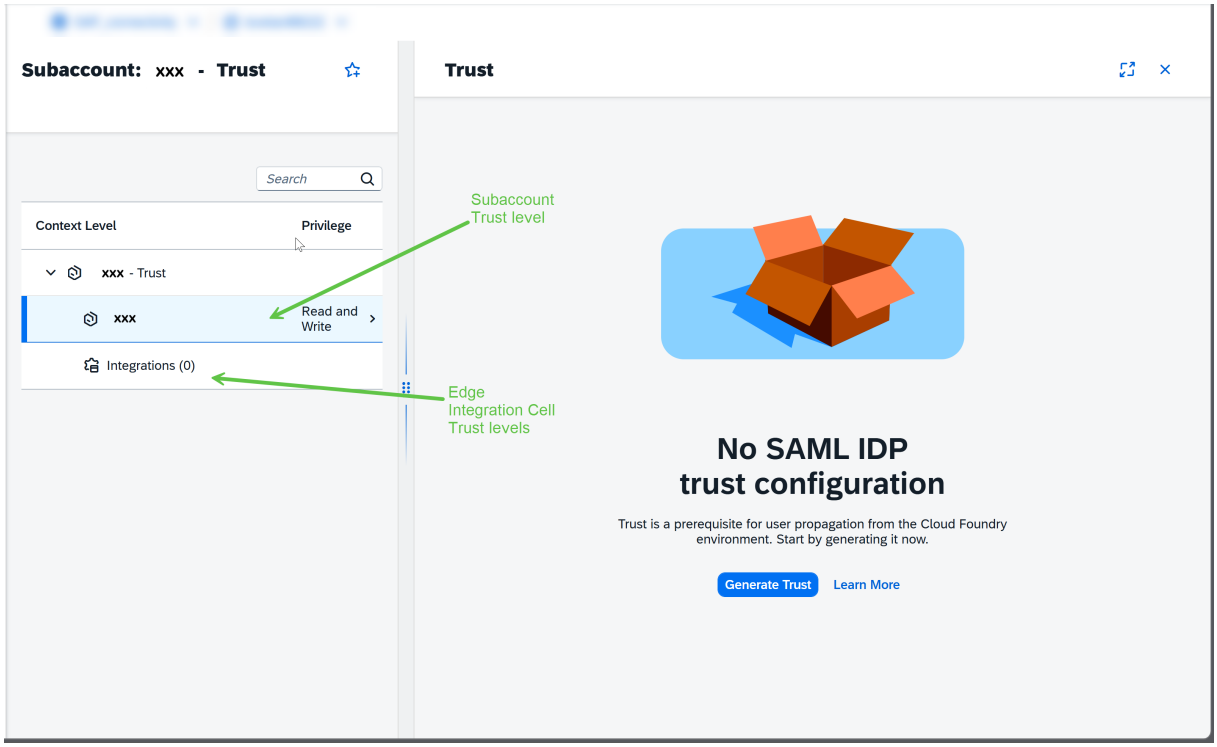
You have at least the relevant `Administrator` user role (*Destination Administrator* or *Destination Subaccount Trust Administrator*).

For more information, see User Roles [page 1301].

Choose a Context Level

In most cases, trust will be configured for the subaccount itself. This is the default *Context Level* when opening the *Destination Trust* screen in the cockpit.

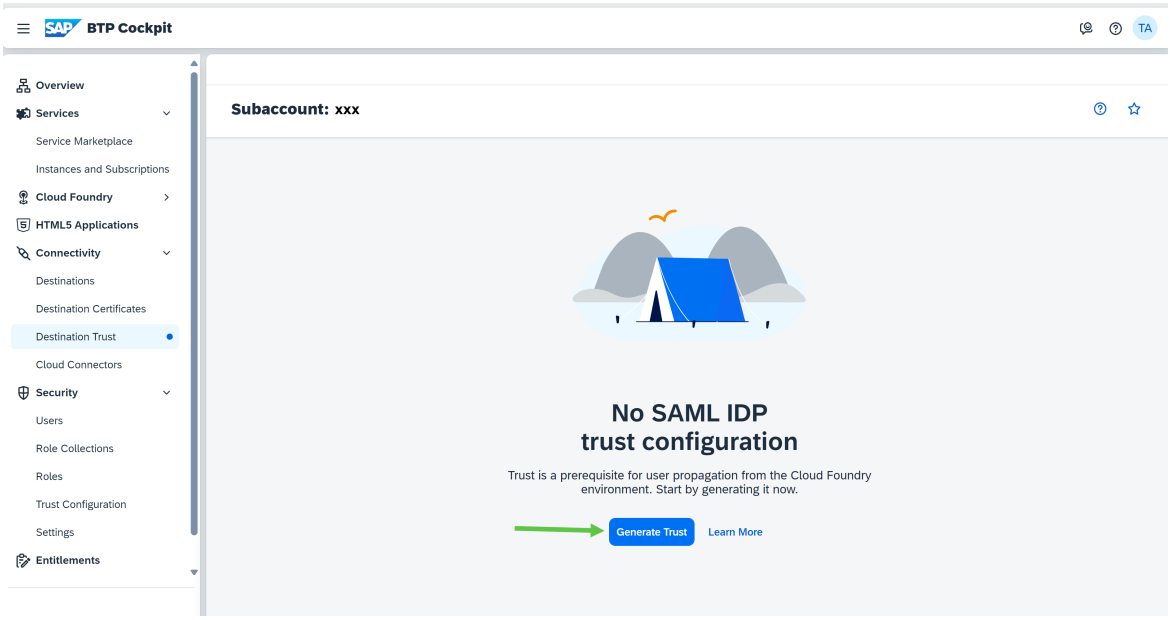
If you have connected Edge Integration Cell(s) to your subaccount, you will also see them listed in the *Integrations* section of the available context levels. This lets you manage specific SAML configurations for each of them.



Set Up a Certificate

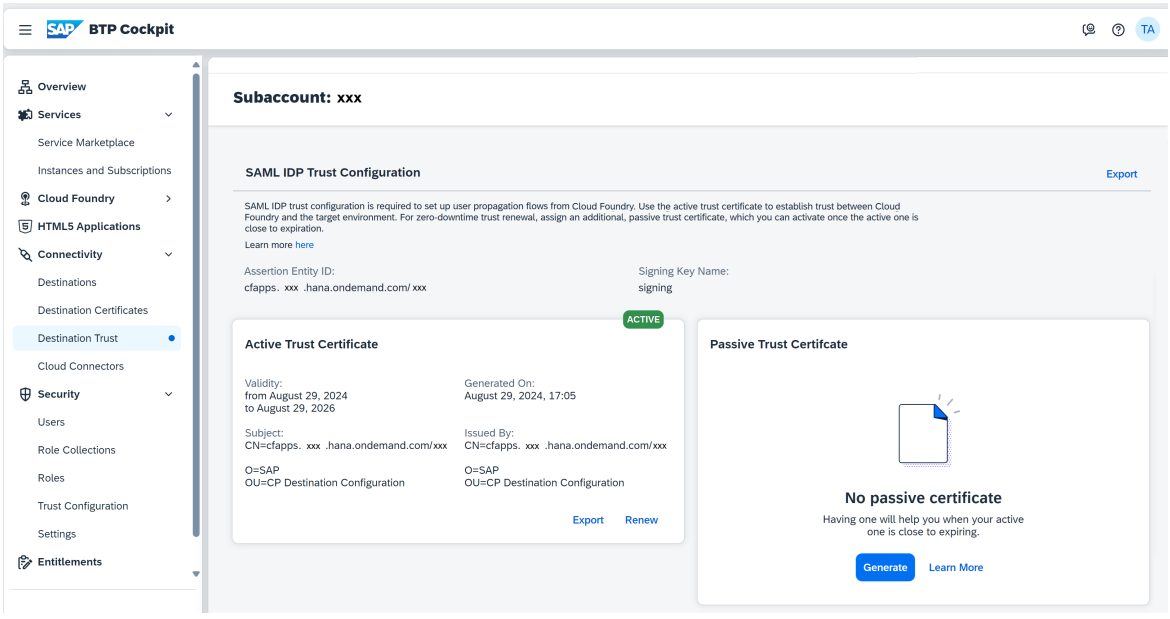
Download and save locally the X.509 certificate of the subaccount in the multi-cloud environment.

1. Log on to the cockpit and navigate to your subaccount in the multi-cloud environment.
2. From the left-side menu, choose *Connectivity* > *Destination Trust*.
3. If the subaccount does not yet have a *SAML Trust* configuration for the Destination service, choose *Generate Trust*:



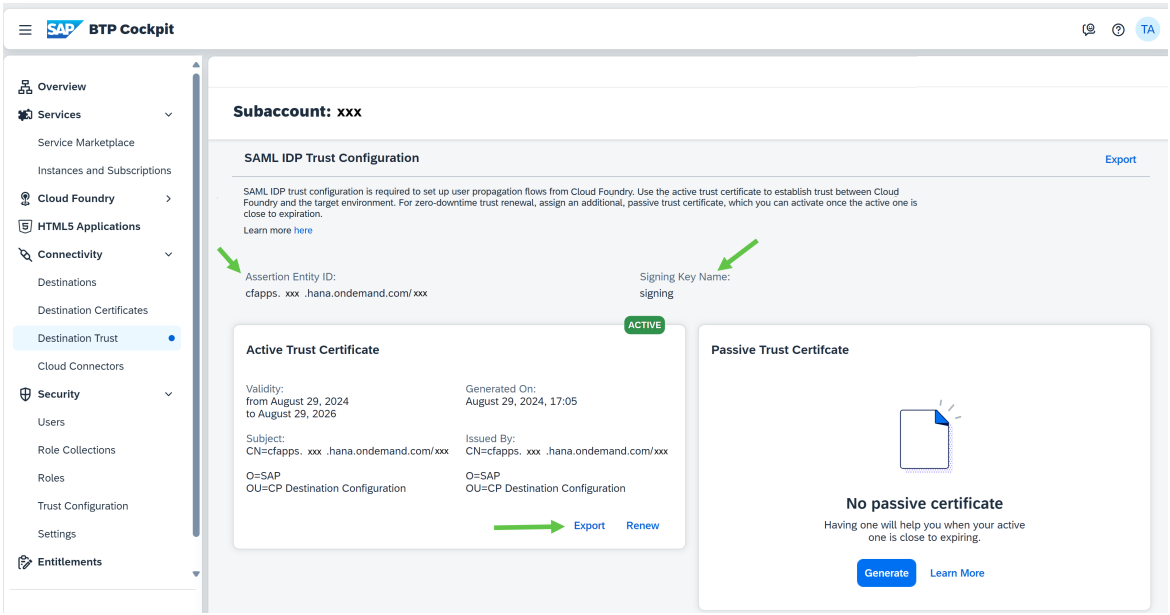
> **Note**
>
> Respective API endpoints:
>
> - Generate or renew the active X.509 trust certificate: `POST /saml2Metadata/certificate`.

4. Once generated, you have an active trust certificate, and its details are visible in the UI:



5. Export the public part of this certificate, which downloads a *.pem* certificate file:

> **→ Tip**
>
> We recommend that you also note down the values of *Assertion Entity ID* and *Signing Key Name* which may be required by the target system(s) when establishing trust.

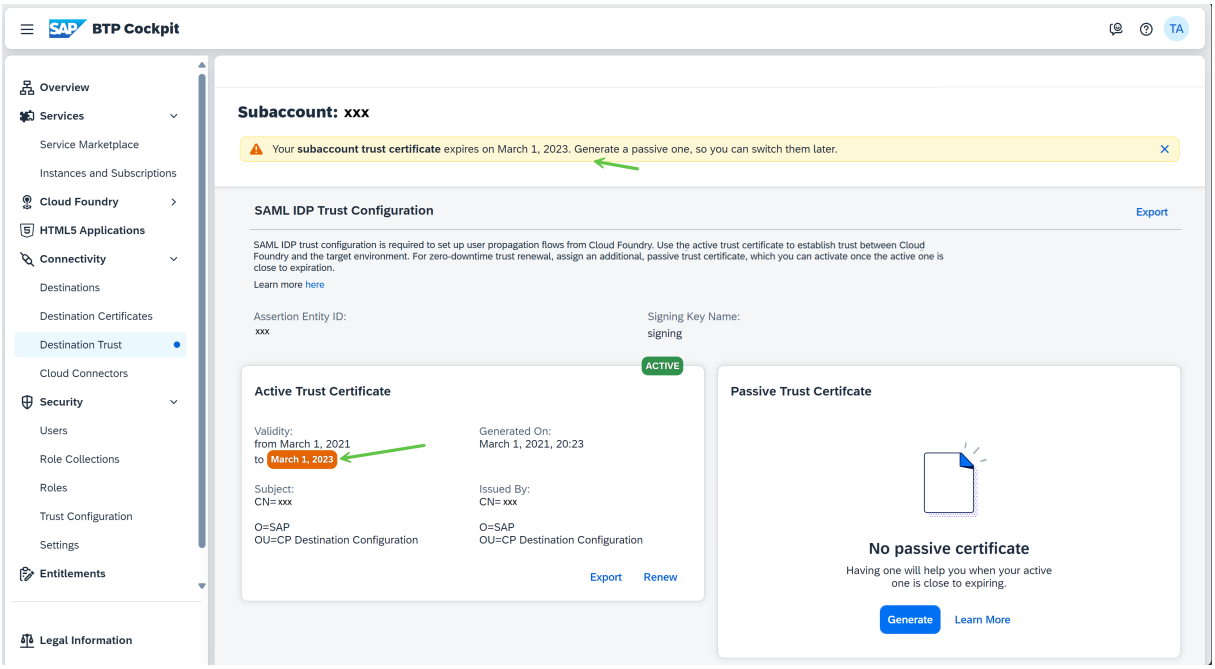


> ### Note
>
> Respective API endpoints:
>
> - Retrieve the active X.509 trust certificate: `GET /saml2Metadata/certificate`.

6. Configure the downloaded X.509 certificate **in the target system(s)** to which you want to propagate the user.

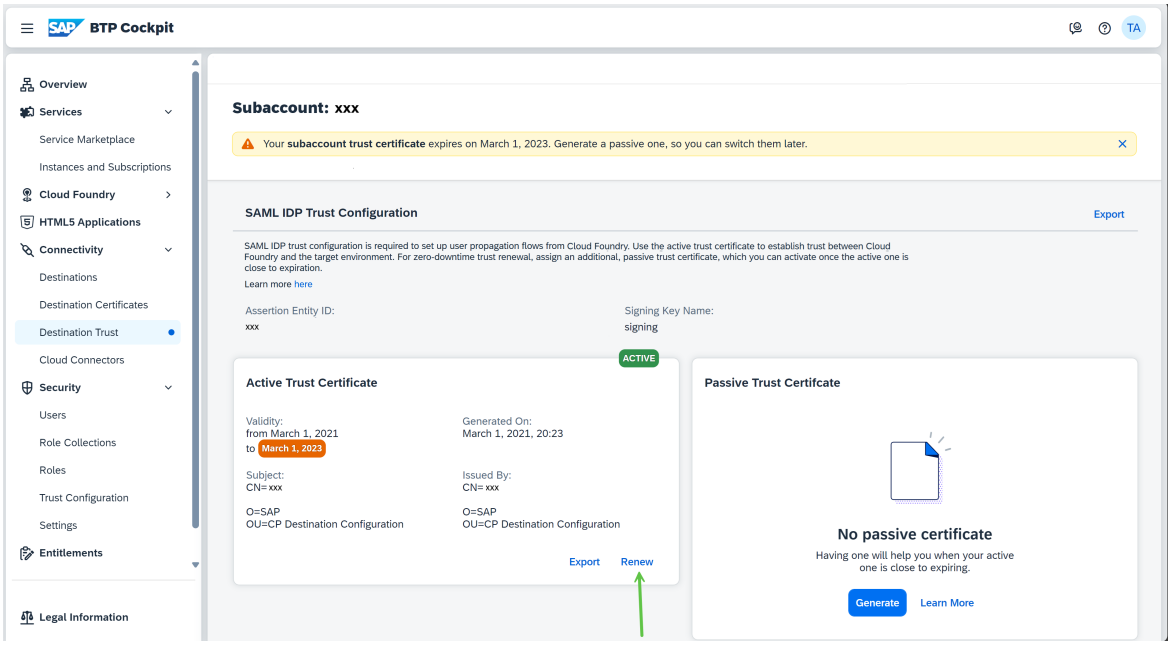
Renew a Certificate

If the X.509 certificate validity is about to expire, you can renew the certificate and extend its validity by another 2 years.



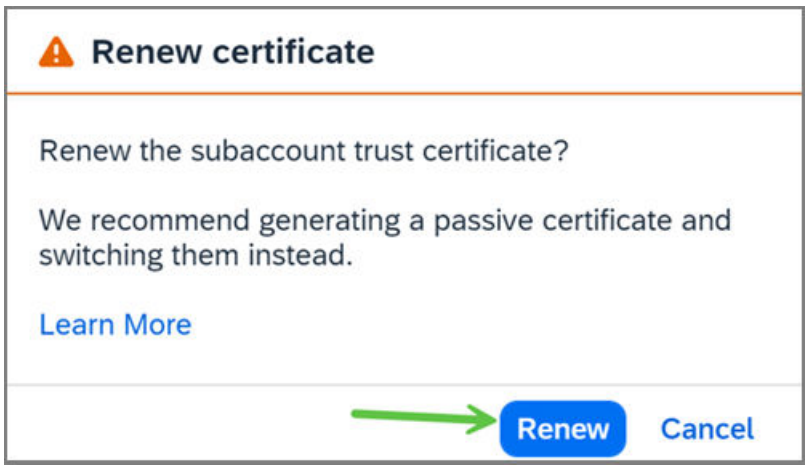
1. Log on to the cockpit and navigate to your subaccount in the multi-cloud environment.
2. From the left-side menu, choose *Connectivity* > *Destination Trust*.
3. Choose *Renew* for the certificate you want to update:

> ⚠ Caution
>
> It is not recommended to renew an active certificate directly. Consider using the Rotate Certificates [page 1520] feature below if you want a zero-downtime procedure.

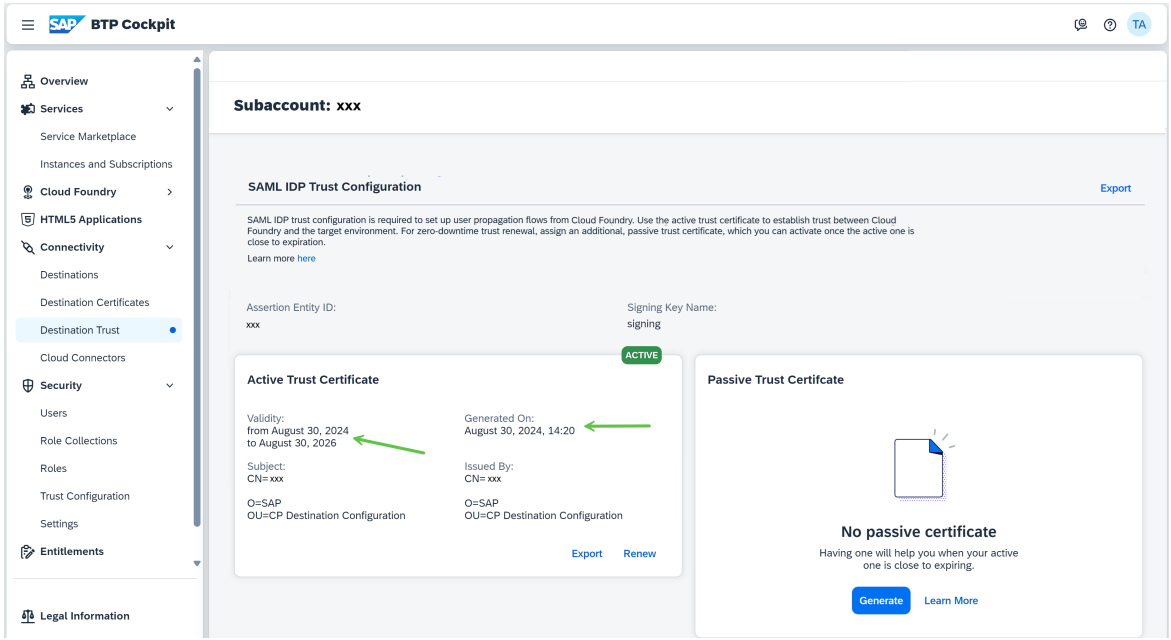


> **Note**
>
> Respective API endpoints:
>
> - Generate or renew the active X.509 trust certificate: `POST /saml2Metadata/certificate`.
> - Generate or renew the passive X.509 trust certificate: `POST /saml2Metadata/certificate/passive`.

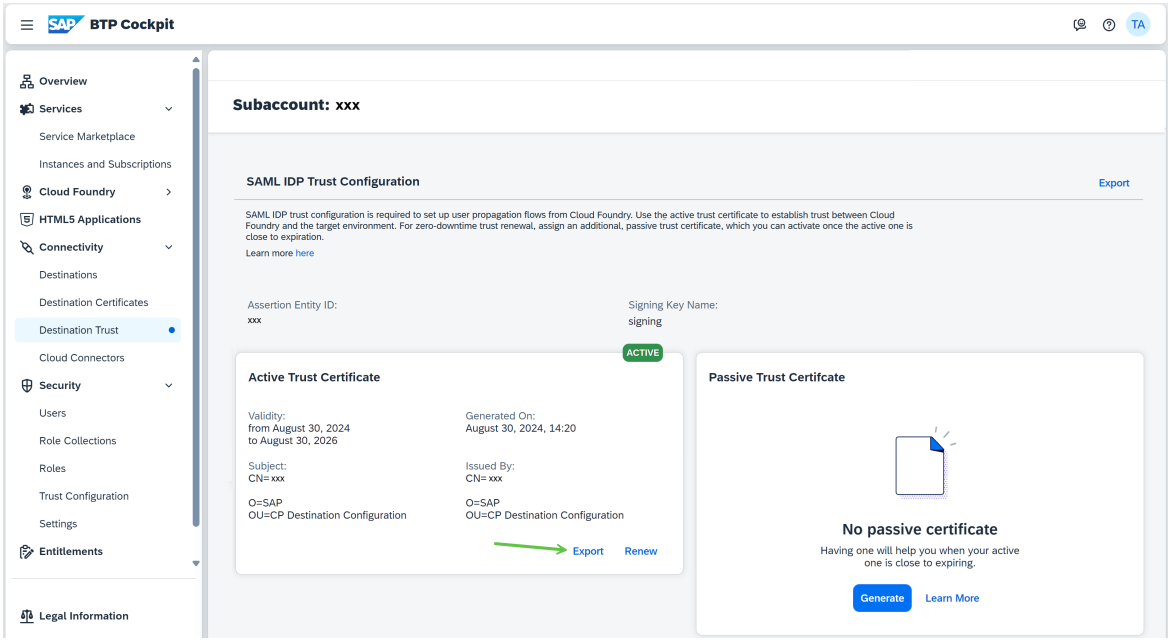
4. Confirm the renewal:



5. You will now see that the warnings are gone, and the certificate details reflect the new dates:



6. Get the required details to establish trust in the target system(s) based on the new certificate, by performing an *Export* of the certificate:



> ### Note
>
> Respective API endpoints:
>
> - Retrieve the active X.509 trust certificate: `GET /saml2Metadata/certificate`.
> - Retrieve the passive X.509 trust certificate: `GET /saml2Metadata/certificate/passive`.

7. Configure the renewed X.509 certificate **in the target system(s)** to which you want to propagate the user.

Rotate Certificates

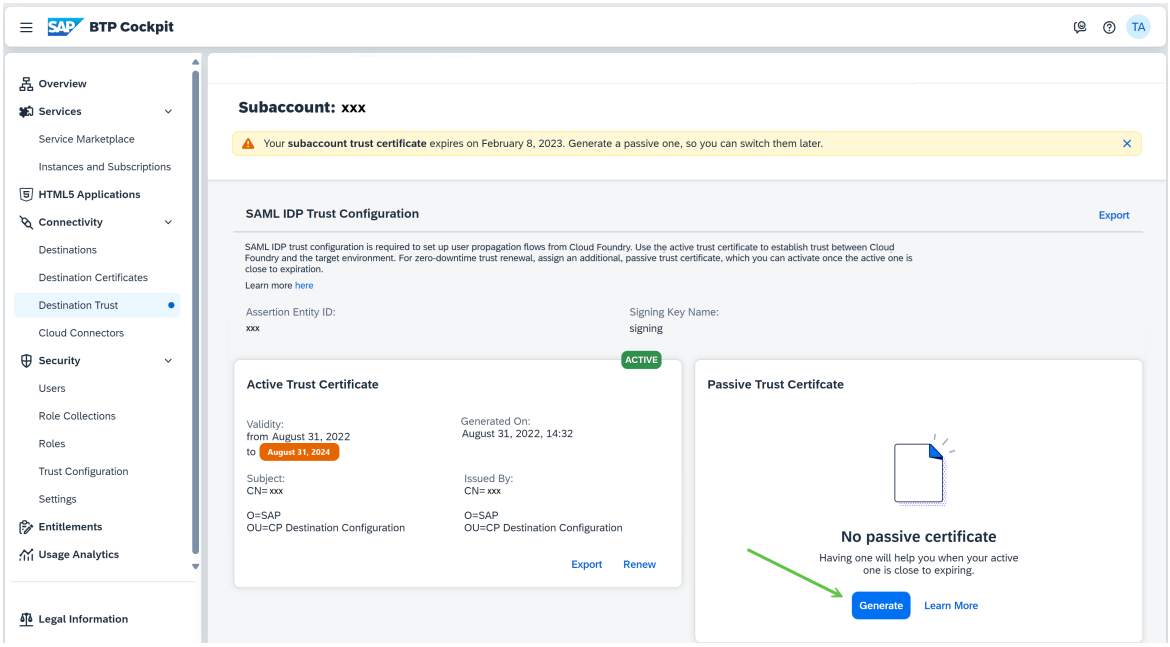
If you renew an active certificate, the Destination service will immediately start signing new SAML assertions with the new one. While this is fast and suitable for development scenarios, for productive scenarios you will have a downtime until you manage to re-establish the trust in all target systems. Therefore, we recommend that you perform the *rotate* procedure described in this section instead.

Rotation is done by creating a passive X.509 certificate for the subaccount, configuring it in the target system(s) to which you want to propagate the user, and rotating it with the active one. After rotation is performed, the active X.509 certificate becomes passive and the passive one active.

> ### Note
>
> You should execute this procedure *before* the expiration date of the currently active certificate.

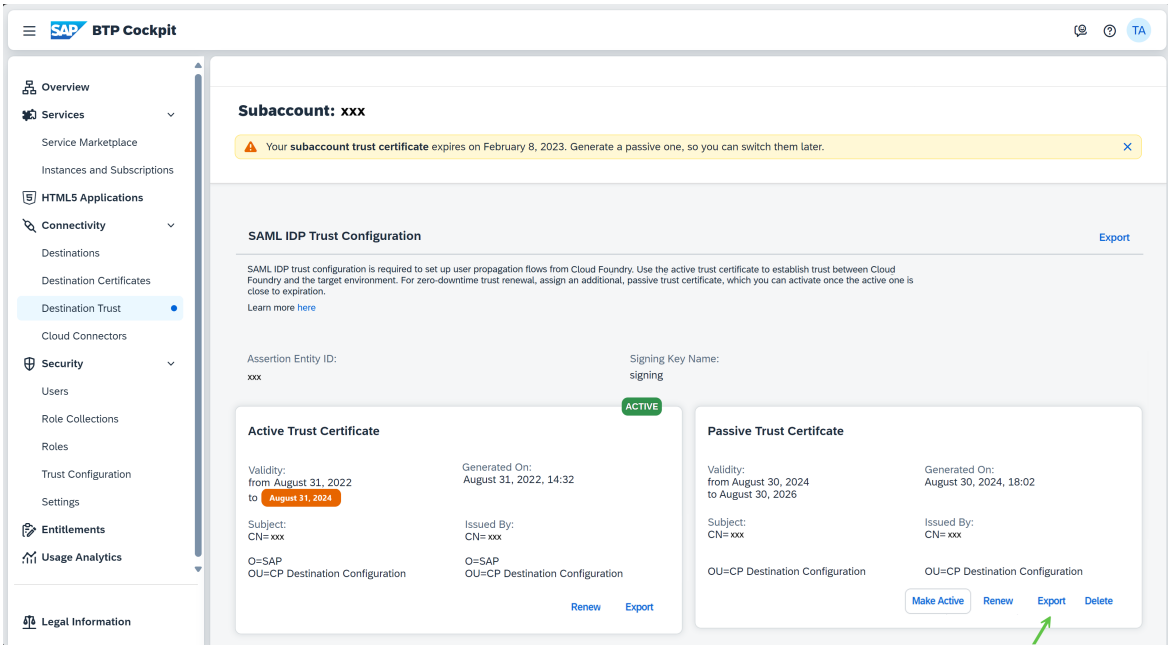
Procedure

1. Generate or renew the passive X.509 certificate:



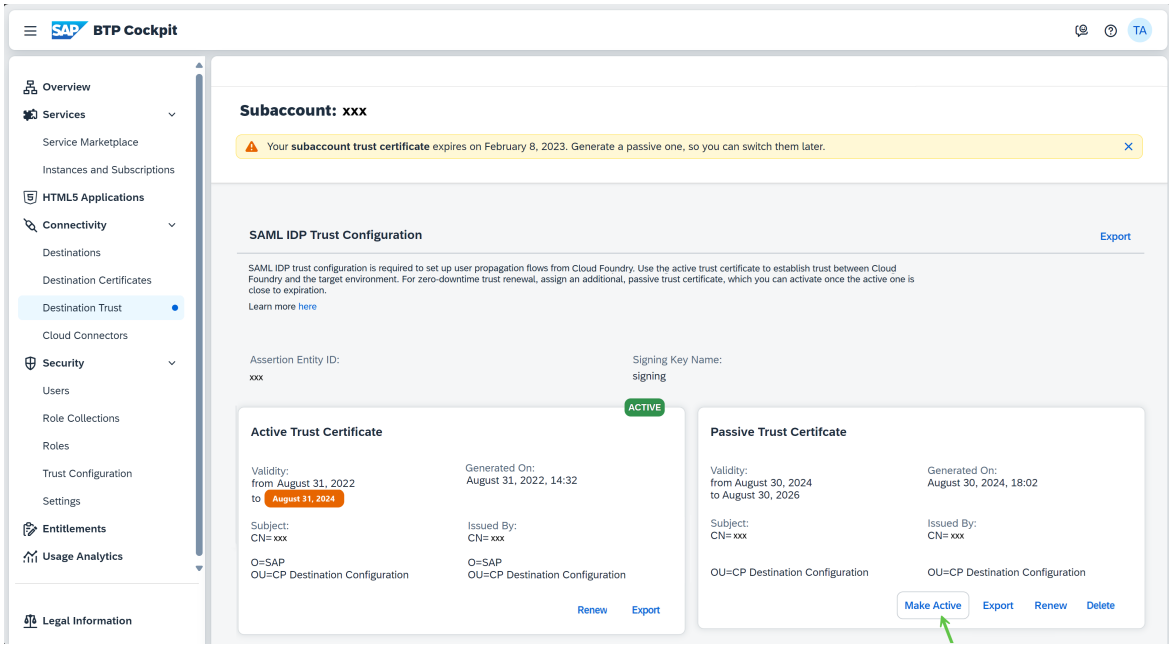
> ## Note
>
> Respective API endpoints:
>
> - Generate or renew the passive X.509 certificate: `POST /saml2Metadata/certificate/passive`.

2. Download and save locally the passive X.509 certificate:



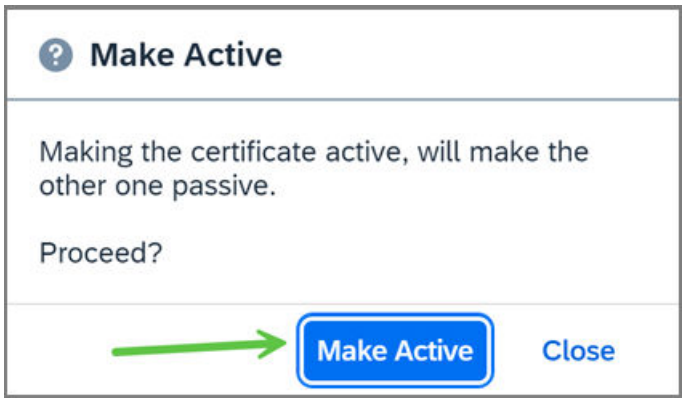
> **Note**
>
> Respective API endpoints:
>
> - Retrieve the passive X.509 certificate: `GET /saml2Metadata/certificate/passive`.

3. Configure the downloaded X.509 passive certificate **in the target system(s)** you want to propagate the user to.
4. Rotate the active certificate, making the active one passive and the passive one active:

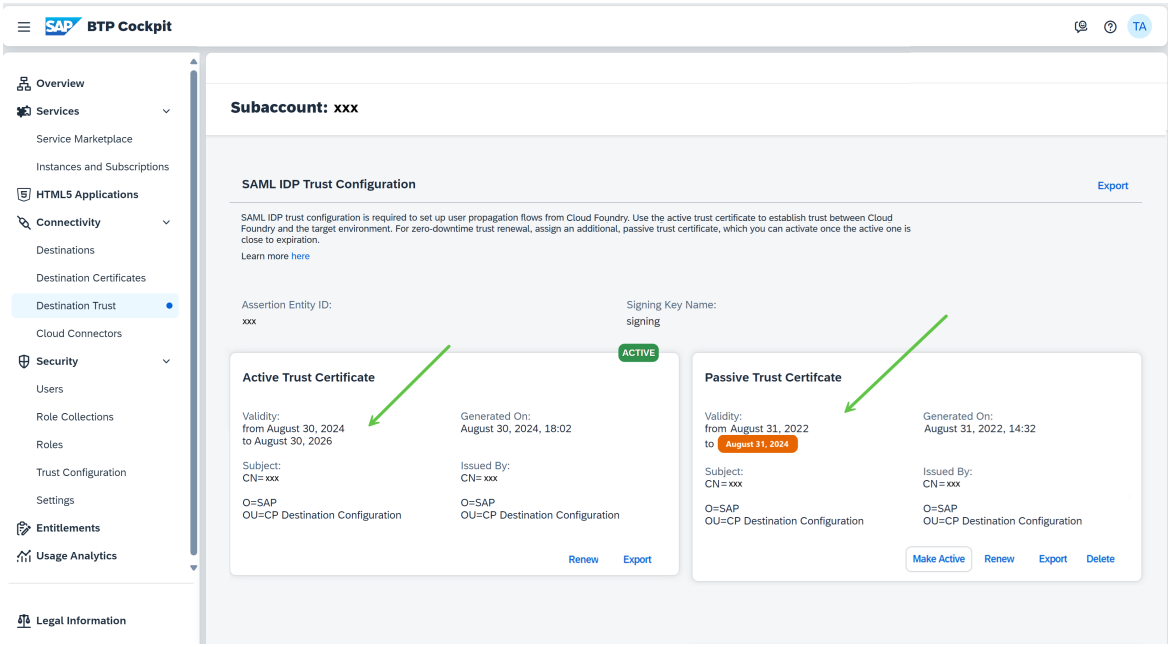


> **Note**
>
> Respective API endpoints:
>
> - Switch the roles of the active and passive certificates: `POST /saml2Metadata/rotateCertificate`.

5. Confirm the action:



6. Check if the two certificates have now switched places. Your scenarios should continue working with no disruption.



7. (Optional): Delete the passive X.509 certificate, which used to be active before rotation.

> **Note**
>
> Respective API endpoints:
>
> - Delete the passive X.509 certificate: `DELETE /saml2Metadata/certificate/passive`.

8. (Optional): Delete the passive X.509 certificate, which used to be active before rotation, from the target system(s).

Related Information

Principal Propagation [page 30]
SAP BTP Destination Service Expiring Certificate Notification

1.3.1.6 Create and Bind a Destination Service Instance

To use the Destination service in your application, you need an instance of the service.

Concept

To consume the Destination service, you must provide the appropriate credentials through a service instance and a service binding/service key. The Destination service is publicly visible and cross-consumable from

several environments and provides the service plan *lite* to all those environments. Provisioning a service instance and service key is done in the standard way for the respective environment, see:

- Cloud Foundry: Using Services in the Cloud Foundry Environment
- Kyma: Using SAP BTP Services in the Kyma Environment
- Kubernetes: Consuming SAP BTP Services from Kubernetes
- Other environments: Consuming Services in Other Environments Using the SAP Service Manager Instances

In all environments, the Destination service lets you provide a configuration JSON during instance creation or update.

Using a Configuration JSON

You can pass a configuration JSON during instance creation or update to modify some of the default settings of the instance and/or provide some content to be created during the operation.

The detailed structure of the configuration JSON is described in Use a Config.JSON to Create or Update a Destination Service Instance [page 1525].

Troubleshooting

If you get this failure message:

```
Failed to create all provided configurations. Will delete all on instance
level, for configurations on subaccount level you can set policies to handle
duplicates: "existing_destinations_policy" with value "update|fail|ignore" and
"existing_certificates_policy" with value "fail|ignore".
```

the root cause may be:

- A provided destination or certificate with the same name already exists in this subaccount. Solution: set policies on subaccount level to update or ignore in case of conflicts.

  ```
  "existing_destinations_policy": "update|fail|ignore"
  "existing_certificates_policy: "fail|ignore"
  ```

- A client input error occurred. Solution: check the input, apply correction and try again.
- An internal server error occurred. In this case, please try again later or report a support incident.

X.509 Bindings

> **Note**
>
> Older instances of the service do not support the X.509 credentials binding type and an attempt to create one will result in an error. To overcome this, you just need to update the service instance, so it picks up the X.509 support.

The service supports X.509 bindings as described in Retrieving Access Tokens with Mutual Transport Layer Security (mTLS). This lets you choose if your binding / service key will be with a client secret X.509 certificate generated by SAP or an X.509 certificate provided by you. To do so, add a config JSON when creating your binding:

```
cf bind-service <app-name> <service-name> [-c <config json>]
```

The structure of the config JSON should be like this:

```
{
    "xsuaa":
    {
        <X.509 properties>
    },
    <other config parameters>
}
```

For a list of the supported X.509 properties, see:

- Parameters for X.509 Certificates Managed by SAP Authorization and Trust Management Service
- Parameters for Self-Managed X.509 Certificates

> **Note**
>
> By default, without providing a config JSON, a binding with client secret will be created.

1.3.1.6.1 Use a Config.JSON to Create or Update a Destination Service Instance

Configure specific parameters in a `config.json` file to create or update a Destination service instance.

> **Note**
>
> By default, creating or updating a Destination service instance does not require passing any configuration settings.

When creating or updating a Destination service instance, you can optionally configure the following settings, which are part of the `config.json` input file (see Open Service Broker API), both via SAP BTP cockpit or Cloud Foundry command line interface (CLI):

Parameter	Value	Description
`HTML5Runtime_enabled`	Boolean	Optional. Indicates whether the SAP BTP HTML5 runtime should be enabled to work with the service instance on behalf of the HTML5 applications associated with it during deployment. **Note** Check if the SAP BTP HTML5 runtime is available for the region in which the service instance is being created or updated.
`init_data`	JSON	Optional. Typically, the data for the Destination service instance is managed via BTP cockpit or REST API. For more information, see Managing Destinations [page 1349]. This option allows to add data (destinations, certificates) to initialise or update the service instance with. The data can be stored on both *service instance* level and *subaccount* level.

Parameter	Value	Description
`subscriber_access`	JSON	Optional. Typically, the data for the Destination service instance is managed via BTP cockpit or REST API. For more information, see Managing Destinations [page 1349]. This option allows you to define the access level for your subscribers to their configurations stored on subscription level, which are shared with them. By default, subscribers have no access. You can specify separate scopes for both *read* and *write* operations for each entity type. If multiple scopes are defined for an operation, the subscriber must have all of them to perform the operation. The provided scopes will either be added during instance creation or used to override the existing scopes during instance update. Each section is optional. If a section is not provided, the existing values will remain unchanged. To delete scopes, you need to provide an empty array.

Find the `config.json` structure below:

Sample Code

```
{
    "HTML5Runtime_enabled" : true,
    "init_data" : {
        "subaccount" : {
            "existing_destinations_policy": "update|fail|ignore",
            "existing_certificates_policy": "update|fail|ignore",
            "destinations" : [
                {
                    ... // a destination entity in key-value JSON format, as
used in the service REST API
                }
            ],
            "certificates" : [
                {
                    ... // a certificate entity in key-value JSON format, as
used in the service REST API
                }
            ]
        },
        "instance" : {
            "existing_destinations_policy": "update|fail|ignore",
            "existing_certificates_policy": "update|fail|ignore",
```

```
            "destinations" : [
                {
                    ... // a destination entity in key-value JSON format, as
used in the service REST API
                }
            ],
            "certificates" : [
                {
                    ... // a certificate entity in key-value JSON format, as
used in the service REST API
                }
            ]
        }
    },
     "subscriber_access" : {
        "destination": {
             "read_scopes":[
               "my-saas-app!b3.ds.subscription.destination.read"
             ],
             "write_scopes": [
               "my-saas-app!b3.ds.subscription.destination.write"
             ]
        },
        "certificate": {
              "read_scopes":[
                "my-saas-app!b3.ds.subscription.certificate.read"
              ],
              "write_scopes": [
                "my-saas-app!b3.ds.subscription.certificate.write"
              ]
        }
    }
}
```

Related Information

HTTP Destinations [page 1350]

1.3.1.7 Destination Service Notifications

Subscribe to the SAP Alert Notification service to receive notifications and alerts for the Destination service on SAP BTP.

You can subscribe for notifications and alerts for different Destination service events via SAP Alert Notification Service for SAP BTP.

For more information, see What Is SAP Alert Notification Service for SAP BTP?.

For details on Destinaton service events, see SAP BTP Destination Service Events.

1.3.1.8 Configure Destination Service IPs in Firewall Rules

Include Destination service IP addresses in your target system firewall rules for incoming and outgoing traffic.

> ⚠ Caution
>
> The Destination service is planning to enable IPv4 and IPv6 dual stack ingress and egress for **AWS and Google Cloud-based regions**. Currently, only IPv4 is supported. Therefore, in our documentation we had only the respective addresses. We have now added also the **IPv6 addresses** that we plan to use. The enablement of the IPv6 addresses will happen no earlier than **February 2026** (AWS) and **April 2026** (Google Cloud). Until then, please **ensure any needed action is performed**, based on the information below.
>
> The Destination service *ingress* endpoints are the ones that are called by your HTTP client, accessing its REST API (you would not usually need to take action for this flow, unless you have productive calls to the service from your own network) and the Cloud Connector (only in the cases where automatic token retrieval is performed for an on-premise OAuth destination). If you make use of this on-premise token retrieval functionality and if the network in which your Cloud Connector is running does not support IPv6, no action is needed, as the IPv4 addresses are unaffected.
>
> If the network in which your Cloud Connector is running does support IPv6, then the Java virtual machine will usually prefer to use IPv4 instead of IPv6 (usually controlled via the `java.net.preferIPv6Addresses` system property). If this is the case for your Java virtual machine, again, no action is needed.
>
> **Required Action:**
>
> If your *JVM is configured to prefer IPv6*, you need to *verify if you have firewall rules in place* that limit outgoing traffic from the network. If that is the case, you would have previously allowed the certain IPv4 addresses to enable the Cloud Connector's communication with BTP. Now, you need to **add also the IPv6 addresses to the allowlist**.
>
> ---
>
> The Destination service *egress* IPs are the ones on behalf of which external resources are accessed either via *Check Connection* or automatic token retrieval for an Internet destination. In some cases, you might have an allowlist for IP addresses at the target system. If that is the case, please add the IPv6 addresses of the Destination service to the list.

> ⓘ Note
>
> As of version 2.18.1, the Cloud Connector shows the IPv6 status on the **About** screen.

NAT IPs

If the Destination service performs a connection check or automatic token retrieval for a configured destination on the Internet, the recipient of the request (a system or service configured through the properties **URL**,

`tokenServiceUrl`, and so on), should be reachable for the URLs listed below (depending on the Destination service region). If you restrict access to the target system for incoming traffic by allowlisting IPs in firewall rules, make sure you include the relevant IPs accordingly.

Region (Region Host)	IP Addresses (IPv4)	IP Addresses (IPv6)
Europe (Frankfurt) - SAP (cf.eu01.hana.ondemand.com)	130.214.173.207	
Europe (Rot) - SAP (cf.eu02.hana.ondemand.com)	130.214.170.39	
Europe (Frankfurt) - AWS (cf.eu10.hana.ondemand.com)	3.79.93.46 3.127.4.249 3.68.162.165	2a05:d014:ca2:3700::/56
Europe (Frankfurt) - AWS (cf.eu11.hana.ondemand.com)	3.77.2.199 3.125.182.25 3.75.8.16	2a05:d014:1b2c:b500::/56
Europe (Milan) - AWS (cf.eu13.hana.ondemand.com)	35.152.227.34 18.102.56.228 18.102.175.80	2a05:d01a:784:e200::/56
Europe (Netherlands) - Azure (cf.eu20.hana.ondemand.com)	40.115.20.192	
Europe (Frankfurt) - Azure (cf.eu22.hana.ondemand.com) **Note** Available as of January 30, 2026	135.220.43.209	
Europe (Frankfurt) - Google Cloud (cf.eu30.hana.ondemand.com)	34.107.98.49/32 35.198.112.96/32 34.107.83.17/32	2600:1900:40d0:8a0:0:0:0:0/64
UK South (London) - Azure (cf.uk20.hana.ondemand.com) **Note** Available as of January 30, 2026	172.166.225.229	

Region (Region Host)	IP Addresses (IPv4)	IP Addresses (IPv6)
US (Sterling) - SAP (cf.us01.hana.ondemand.com) **Note** Available as of January 30, 2026	130.214.109.51	
US West (Colorado) - SAP (cf.us02.hana.ondemand.com) **Note** Available as of December 30, 2025	130.214.68.26	
US East (VA) - AWS (cf.us10.hana.ondemand.com)	44.196.182.81 52.22.134.17 54.81.172.123	2600:1f18:ef:9b00::/56
US West (Oregon) - AWS (cf.us11.hana.ondemand.com)	100.21.196.19 44.239.115.54 35.155.159.17	2600:1f14:180c:7200::/56
US West (WA) - Azure (cf.us20.hana.ondemand.com)	52.151.19.92	
US East (VA) - Azure (cf.us21.hana.ondemand.com)	20.163.232.70	
US Central (IA) - Google Cloud (cf.us30.hana.ondemand.com)	34.67.55.101/32 34.72.92.245/32 34.29.83.5/32	2600:1900:4000:18b3:0:0:0:0/64
Brazil (São Paulo) - AWS (cf.br10.hana.ondemand.com)	18.228.74.68 18.231.84.86 54.232.114.228	2600:1f1e:c13:1500::/56
Brazil (São Paulo) - Azure (cf.br20.hana.ondemand.com)	4.201.180.215	

Region (Region Host)	IP Addresses (IPv4)	IP Addresses (IPv6)
Japan (Tokyo) - SAP (cf.jp01.hana.ondemand.com) **Note** Available as of January 15, 2026	130.214.52.51	
Japan (Tokyo) - AWS (cf.jp10.hana.ondemand.com)	18.177.52.4 57.181.89.180 54.150.191.131	2406:da14:1d30:cb00::/56
Japan (Tokyo) - Azure (cf.jp20.hana.ondemand.com)	48.210.28.199	
Japan (Osaka) - Google Cloud (cf.jp30.hana.ondemand.com)	34.97.201.156/32 34.97.188.12/32 34.97.65.243/32	2600:1900:41d0:611:0:0:0:0/64
Japan (Tokyo) - Google Cloud (cf.jp31.hana.ondemand.com)	34.84.61.30/32 34.180.69.131/32 35.221.82.66/32	2600:1900:4051:2b7:0:0:0:0/64
Australia (Sydney) - SAP (cf.ap01.hana.ondemand.com) **Note** Available as of December 30, 2025	130.214.141.233	
Australia (Sydney) - AWS (cf.ap10.hana.ondemand.com)	54.206.245.69 52.63.244.160 13.210.143.45	2406:da1c:339:e800::/56
Asia Pacific (Singapore) - AWS (cf.ap11.hana.ondemand.com)	52.220.52.143 13.215.24.185 18.143.71.37	2406:da18:500:5400::/56
Asia Pacific (Seoul) - AWS (cf.ap12.hana.ondemand.com)	43.202.18.57 13.209.44.214 52.78.55.157	2406:da12:ae7:cc00::/56

Region (Region Host)	IP Addresses (IPv4)	IP Addresses (IPv6)
Australia (Sydney) - Azure (cf.ap20.hana.ondemand.com)	13.75.143.163	
Singapore - Azure (cf.ap21.hana.ondemand.com)	23.98.89.110	
Australia (Sydney) - Google Cloud (cf.ap30.hana.ondemand.com)	34.151.67.240/32 35.189.1.23/32 34.40.245.56/32	2600:1900:40b0:eb::/64
Brazil (São Paulo) - Google Cloud (cf.br30.hana.ondemand.com)	35.247.212.99/32 34.39.174.112/32 34.95.181.127/32	2600:1900:40f0:a5e::/64
Canada (Montreal) - AWS (cf.ca10.hana.ondemand.com)	15.222.94.104 15.157.235.95 3.97.147.84	2600:1f11:b90:f900::/56
Canada Central (Toronto) - Azure (cf.ca20.hana.ondemand.com)	52.138.50.189	
Switzerland (Zurich) - Azure (cf.ch20.hana.ondemand.com)	51.103.232.173	
UAE (Dubai) - SAP (cf.ae01.hana.ondemand.com)	130.214.0.212	
KSA (Dammam) - Google Cloud public sector (cf.sa30.hana.ondemand.com)	34.166.65.101/32 34.166.162.197/32 34.166.10.123/32	2600:1900:5400:775:0:0:0:0/64
KSA (Dammam) - Google Cloud (cf.sa31.hana.ondemand.com)	34.166.227.227/32 34.166.78.37/32 34.166.221.54/32	2600:1900:5400:4b8:0:0:0:0/64
Israel (Tel Aviv) - Google Cloud (cf.il30.hana.ondemand.com)	34.165.245.143/32 35.252.1.206/32 34.165.224.138/32	2600:1901:8160:ac3:0:0:0:0/64

Region (Region Host)	IP Addresses (IPv4)	IP Addresses (IPv6)
India (Mumbai) - Google Cloud (cf.in30.hana.ondemand.com)	34.93.233.251/32 34.47.175.95/32 34.14.140.65/32	2600:1900:40a0:305:0:0:0:0/64

LB IPs

If the Destination service performs automatic token retrieval for an OAuth on-premise destination from a local token service, and there are restrictions on outgoing traffic for the network in which the Cloud Connector resides, you should configure a rule to allow traffic to these IPs (depending on the Destination service region), which lets the Cloud Connector establish the tunnel connection.

Region (Region Host)	IP Addresses (IPv4)	IP Addresses (IPv6)
Europe (Frankfurt) - SAP (cf.eu01.hana.ondemand.com)	130.214.124.217	
Europe (Rot) - SAP (cf.eu02.hana.ondemand.com)	130.214.170.43	
Europe (Frankfurt) - AWS (cf.eu10.hana.ondemand.com)	3.120.127.220 3.123.133.91 18.153.139.6	2a05:d014:ca2:3702::18, 2a05:d014:ca2:3705::18, 2a05:d014:ca2:3708::18
Europe (Frankfurt) - AWS (cf.eu11.hana.ondemand.com)	3.127.28.215 18.192.171.62 3.74.96.32	2a05:d014:1b2c:b502::18, 2a05:d014:1b2c:b508::18, 2a05:d014:1b2c:b505::18
Europe (Milan) - AWS (cf.eu13.hana.ondemand.com)	18.102.157.221 18.102.226.166 15.160.20.175	2a05:d01a:784:e205::18, 2a05:d01a:784:e208::18, 2a05:d01a:784:e202::18
Europe (Netherlands) - Azure (cf.eu20.hana.ondemand.com)	108.141.17.178	

Region (Region Host)	IP Addresses (IPv4)	IP Addresses (IPv6)
Europe (Frankfurt) - Azure (cf.eu22.hana.ondemand.com) **Note** Available as of January 30, 2026	131.189.188.171	
Europe (Frankfurt) - Google Cloud (cf.eu30.hana.ondemand.com)	35.246.171.255	2600:1900:40d0:8a0:8000:3::
UK South (London) - Azure (cf.uk20.hana.ondemand.com) **Note** Available as of January 30, 2026	4.158.194.40	
US (Sterling) - SAP (cf.us01.hana.ondemand.com) **Note** Available as of January 30, 2026	130.214.107.137	
US West (Colorado) - SAP (cf.us02.hana.ondemand.com) **Note** Available as of December 30, 2025	130.214.68.13	
US East (VA) - AWS (cf.us10.hana.ondemand.com)	54.243.28.3 35.173.121.86 52.71.181.35	2600:1f18:ef:9b05::18, 2600:1f18:ef:9b08::18, 2600:1f18:ef:9b02::18 **New** (as of Q2 2026, exact date will be announced via release note): **2600:1f18:ef:9b08::18,** **2600:1f18:ef:9b0b::18,** **2600:1f18:ef:9b02::18**
US West (Oregon) - AWS (cf.us11.hana.ondemand.com)	34.215.5.87 44.245.124.5 35.85.25.113	2600:1f14:180c:7205::18, 2600:1f14:180c:7208::18, 2600:1f14:180c:7202::18

Region (Region Host)	IP Addresses (IPv4)	IP Addresses (IPv6)
US West (WA) - Azure (cf.us20.hana.ondemand.com)	4.149.160.6	
US East (VA) - Azure (cf.us21.hana.ondemand.com)	135.234.230.67	
US Central (IA) - Google Cloud (cf.us30.hana.ondemand.com)	34.55.44.153	2600:1900:4000:18b3:8000:3::
Brazil (São Paulo) - AWS (cf.br10.hana.ondemand.com)	54.232.165.245 54.207.38.48 54.207.88.85	2600:1f1e:c13:1502::18, 2600:1f1e:c13:1508::18, 2600:1f1e:c13:1505::18
Brazil (São Paulo) - Azure (cf.br20.hana.ondemand.com)	4.228.11.82	
Brazil (São Paulo) - Google Cloud (cf.br30.hana.ondemand.com)	35.247.229.235	2600:1900:40f0:a5e:8000:3::
Japan (Tokyo) - SAP (cf.jp01.hana.ondemand.com) **Note** Available as of January 15, 2026	130.214.52.58	
Japan (Tokyo) - AWS (cf.jp10.hana.ondemand.com)	54.249.103.106 18.179.168.29 52.193.24.140	2406:da14:1d30:cb05::18, 2406:da14:1d30:cb08::18, 2406:da14:1d30:cb02::18
Japan (Tokyo) - Azure (cf.jp20.hana.ondemand.com)	74.176.24.90	
Japan (Osaka) - Google Cloud (cf.jp30.hana.ondemand.com)	34.97.81.151	2600:1900:41d0:611:8000:3::
Japan (Tokyo) - Google Cloud (cf.jp31.hana.ondemand.com)	34.85.115.68	2600:1900:4051:2b7:8000:3::

Region (Region Host)	IP Addresses (IPv4)	IP Addresses (IPv6)
Australia (Sydney) - SAP (cf.ap01.hana.ondemand.com) **Note** Available as of December 30, 2025	130.214.141.24	
Australia (Sydney) - AWS (cf.ap10.hana.ondemand.com)	13.54.39.245 13.238.213.186 13.54.25.95	2406:da1c:339:e808::18, 2406:da1c:339:e805::18, 2406:da1c:339:e802::18
Asia Pacific (Singapore) - AWS (cf.ap11.hana.ondemand.com)	47.130.15.124 18.141.62.28 18.138.210.183	2406:da18:500:5402::18, 2406:da18:500:5405::18, 2406:da18:500:5408::18
Asia Pacific (Seoul) - AWS (cf.ap12.hana.ondemand.com)	52.78.218.213 13.209.92.99 3.38.231.134	2406:da12:ae7:cc02::18, 2406:da12:ae7:cc05::18, 2406:da12:ae7:cc08::18
Australia (Sydney) - Azure (cf.ap20.hana.ondemand.com)	4.237.197.40	
Singapore - Azure (cf.ap21.hana.ondemand.com)	57.155.52.252	
Australia (Sydney) - Google Cloud (cf.ap30.hana.ondemand.com)	34.40.172.51	2600:1900:40b0:eb:8000:3::
Canada (Montreal) - AWS (cf.ca10.hana.ondemand.com)	15.222.109.206 3.96.0.52 3.97.184.18	2600:1f11:b90:f908::18, 2600:1f11:b90:f905::18, 2600:1f11:b90:f902::18
Canada Central (Toronto) - Azure (cf.ca20.hana.ondemand.com)	130.107.189.173	
Switzerland (Zurich) - Azure (cf.ch20.hana.ondemand.com)	20.250.185.103	
UAE (Dubai) - SAP (cf.ae01.hana.ondemand.com)	130.214.0.27	

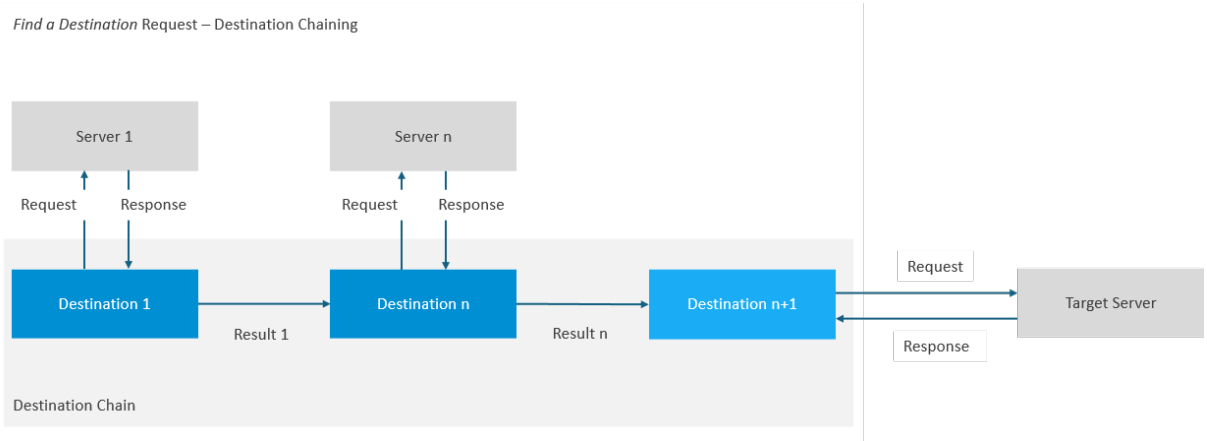
Region (Region Host)	IP Addresses (IPv4)	IP Addresses (IPv6)
KSA (Dammam) - Google Cloud public sector (cf.sa30.hana.ondemand.com)	34.166.138.63	2600:1900:5400:775:8000:3::
KSA (Dammam) - Google Cloud (cf.sa31.hana.ondemand.com)	34.166.145.125	2600:1900:5400:4b8:8000:4::
Israel (Tel Aviv) - Google Cloud (cf.il30.hana.ondemand.com)	34.165.163.225	2600:1901:8160:ac3:8000:3::
India (Mumbai) - Google Cloud (cf.in30.hana.ondemand.com)	34.93.203.8	2600:1900:40a0:305:8000:3::

1.3.1.9 Destination Chaining

Invoke multiple destinations with a single *Find a Destination* request.

Destination chaining is a mechanism that lets you invoke multiple destinations with a single *Find a Destination* request, automatically using the result of one destination as input for the next one, except for the final destination of the chain, which points to the actual target server.

Example



When calling this chain, the Destination service will first call **Server 1**, defined in **Destination 1**. It will use the output of this call to call **Server n**, defined in **Destination n**. It will then return the response of **Server n**, along with the contents of **Destination n+1** to you, so you can build and execute your request to the **Target Server**.

Process

You can use destination chaining by calling the *Find a Destination* API for a given destination and providing the HTTP header `X-chain-name`, which defines the chain in use.

The concrete chain might define more HTTP headers you need to provide when calling the API. They start with `X-chain-var-<variable_name>` and are specific for each chain. In order to check what headers you need to provide when you use destination chaining, check the dedicated documentation page of the specific chain.

Overview of the *Find a Destination* API request:

Sample Code

```
curl https://<destination-service-host>/destination-configuration/v1/
destinations/<final-destination-name> --request GET \
  --header 'Authorization: Bearer <destination-service-access-token>' \
  --header 'X-chain-name: <chain-name>' \
  --header 'X-chain-var-chainVariable1: <value of chainVariable1>'
```

Restriction

Destination chaining is only applicable for requests to a *Find a Destination* REST API endpoint.

Restriction

Currently, only predefined chains are supported by the Destination service.

Related Information

Calling the Destination Service REST API [page 1562]
Predefined Destination Chains [page 1540]
"Find a Destination" Response Structure [page 1565]
Token Retrieval Using IAS-Signed SAML2.0 Assertions [page 124]

1.3.1.9.1 Predefined Destination Chains

Find a list of predefined destination chains that can be consumed through requests to the Destination service.

Chain Name	Chain Description	Documentation
`com.sap.iasGeneratedOAuth2SamlBearerAssertion`	*IAS-generated SAML assertion to OAuth2 Bearer token* chain	IAS-generated SAML Assertion to OAuth2 Bearer Token Chain [page 1540]
`com.sap.sequentialUserPropagation`	*User-token-to-new-user-token* transformation and *new-user-token* consumption chain	Sequential User Propagation Chain [page 1542]

Related Information

Calling the Destination Service REST API [page 1562]
IAS-generated SAML Assertion to OAuth2 Bearer Token Chain [page 1540]
Token Retrieval Using IAS-Signed SAML2.0 Assertions [page 124]

1.3.1.9.1.1 IAS-generated SAML Assertion to OAuth2 Bearer Token Chain

Use a predefined destination chain to generate and consume IAS (Identity Authentication service)-signed SAML assertions.

This predefined destination chain simplifies the process of generating and consuming IAS-signed SAML assertions. The chain works in a 2-step process. The first step consumes one destination to generate the IAS-signed SAML assertion, while the second step uses a different destination to consume the generated assertion in order to retrieve a token from a token server.

> ⓘ Note
>
> For more information on identity authentication, see SAP Cloud Identity Services.

Chain Variable Consumption

To successfully consume the chain, the following headers must be included in the request to the *Find a Destination* REST API endpoint:

Header Name	Description
`X-chain-name`	Specifies the chain which will be applied to the request. In this case, the value must be `com.sap.iasGeneratedOAuth2SamlBearerAssertion`.
`X-chain-var-samlProviderDestinationName`	Specifies the name of the destination, which will be retrieved by the Destination service and whose properties will be used to generate the IAS-signed SAML assertion.
`X-chain-var-subjectToken`	Specifies the value of the subject token (an access token usually associated with the user principal), which will be used by Destination service to perform the token exchange for a IAS-signed SAML assertion.
`X-chain-var-subjectTokenType`	Specifes the type of the subject token, which will be used by Destination service to perform the token exchange for a IAS-signed SAML assertion.

Optional Headers

Header Name	Description
`X-chain-var-samlProviderFragmentName`	Specifies the name of a destination fragment that will be used to override and/or extend the properties of the destination specified by the `X-chain-var-samlProviderDestinationName` header. In case of overlapping properties, the values of the fragment properties take priority over the values of the destination properties.
`X-chain-var-samlProviderFragmentOptional`	Marks whether the destination fragment provided with the `X-chain-var-samlProviderFragmentName` is optional. If the fragment is marked as optional and it does not exist, the destination with its original properties will be consumed. If the fragment is not marked as optional and it does not exist, the request will fail. This header is only applicable if the `X-chain-var-samlProviderFragmentName` header is present. If this header is applicable but not provided, the default value will be *false*.

Related Information

Token Retrieval Using IAS-Signed SAML2.0 Assertions [page 124]

1.3.1.9.1.2 Sequential User Propagation Chain

Use a predefined destination chain to exchange a user token for another user token, which will then be used to perform a user propagation flow.

This predefined destination chain simplifies the process of transforming and consuming a user token. The chain works in a 2-step process:

1. The first step uses one destination to generate a user token using the user token provided by the user.
2. The second step uses the destination specified in the REST API path to consume the generated user token. How this token will be consumed is determined by the configuration properties of the destination specified in the REST API path.

Chain Variable Consumption

To successfully consume the chain, the following headers must be included in the request to the *Find a Destination* REST API endpoint:

Header Name	Description
`X-chain-name`	Specifies the chain which will be applied to the request. In this case, the value must be `com.sap.sequentialUserPropagation`.
`X-chain-var-tokenTransformationDestinationName`	Specifies the name of the destination that will be retrieved by the Destination service and whose properties will be used to perform the user-token-to-user-token exchange.
`X-chain-var-userToken`	The user token that will be used to perform the token exchange during the consumption of the destination specified by the `X-chain-var-tokenTransformationDestinationName` header.

Optional Headers

Header Name	Description
`X-chain-var-tokenTransformationFragmentName`	Specifies the name of a destination fragment that will be used to override and/or extend the properties of the destination specified by the `X-chain-var-tokenTransformationDestinationName` header. In case of overlapping properties, the values of the fragment properties take priority over the values of the destination properties.

Header Name	Description
`X-chain-var-tokenTransformationFragmentOptional`	Marks whether the destination fragment provided with the `X-chain-var-tokenTransformationFragmentName` is optional. If the fragment is marked as optional and it does not exist, the destination with its original properties will be consumed. If the fragment is not marked as optional and it does not exist, the request will fail. This header is only applicable if the `X-chain-var-tokenTransformationFragmentName` header is present. If this header is applicable but not provided, the default value will be *false*.

1.3.1.10 Labeling Destination Service Entities

You can assign labels to some of the configurations that you can manage within the Destination service. Currently, this is supported for destinations and certificates.

What are Labels?

Labels are **strictly non-sensitive** *key:multi-value pairs* that can be assigned to Destination service entities [page 16]. The exact key and values are custom, and based on your needs and preferences. The purpose of labels is to add additional semantics to the service entities. These semantics can be used for filtering, selecting, and grouping entities. It lets you assign metadata related to the modeling of the entities to improve efficiency in both administrative and runtime operations.

Labels vs Additional Properties

At first glance, labels may look somewhat similar to *additional properties*, which can be used in the SAP BTP cockpit to configure a destination. However, there are some key differences.

Additional properties can be interpreted as "additional information needed by the client to know how to connect to the remote system". They are considered part of the actual destination configuration and may be *sensitive* or *non-sensitive*. Therefore, they are stored encrypted. As a consquence, they are not suitable for searching, filtering, and grouping.

Labels are *outside* metadata, with clear guidance that **sensitive values should never be placed in them**. This allows to store them unencrypted, which enables searching, filtering and grouping on them. Also, labels are

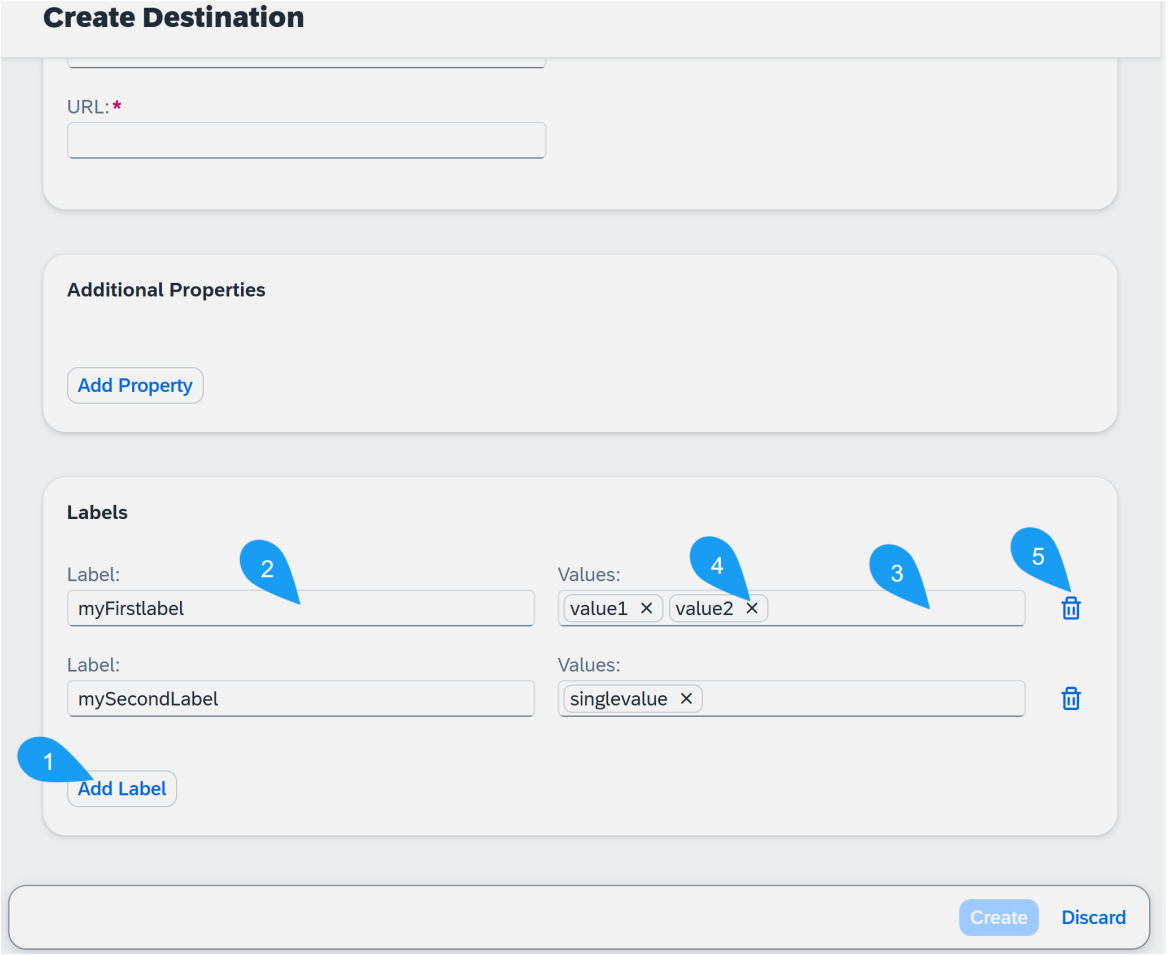
applicable for other service entities, as their purpose is organizational and not related to the concrete type of the entity.

Managing Labels

You can manage labels via REST API or SAP BTP cockpit.

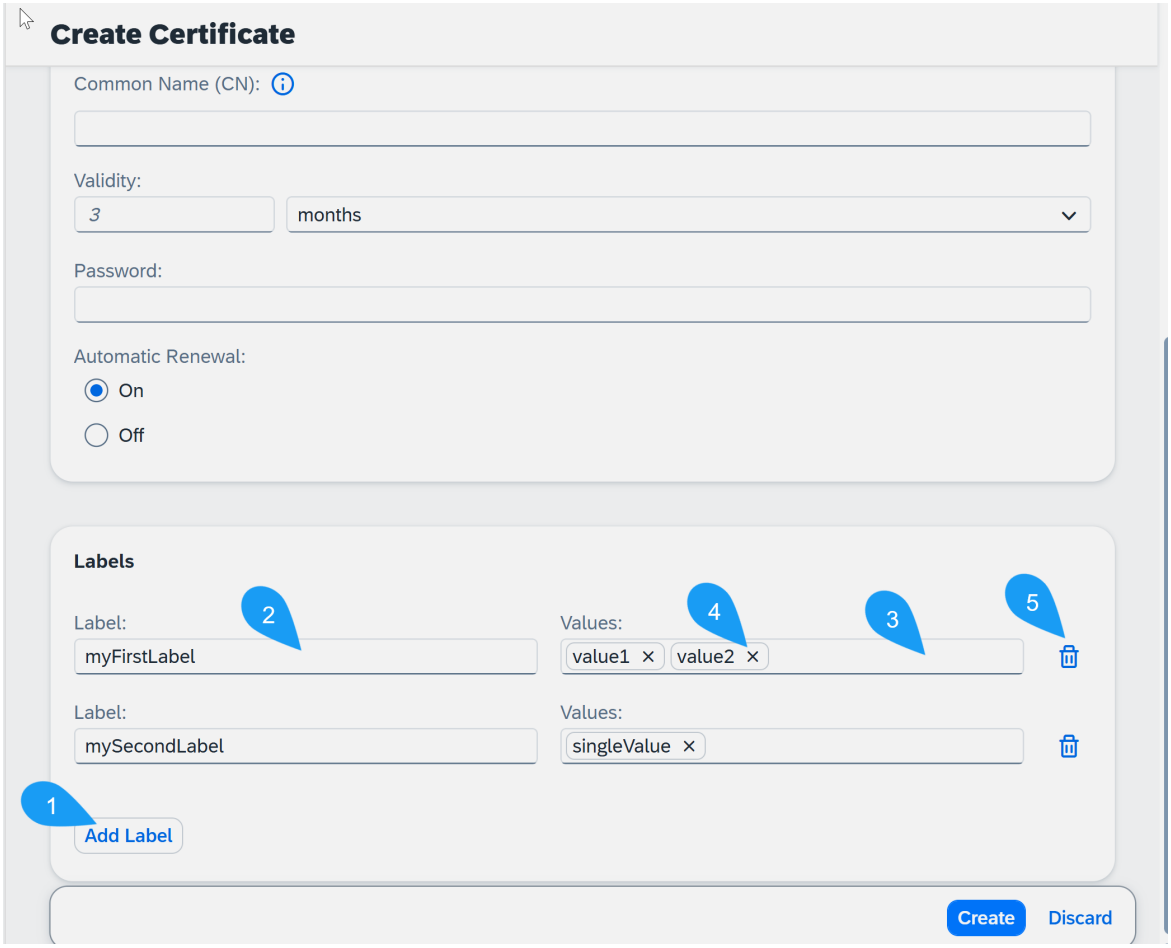
Destinations

- **REST API**:
  In the entity-specific APIs on the subaccount and instance/subscription levels, there are endpoints for reading all labels, overriding all labels and inserting/deleting individual labels.
  For more information on the REST API, see Calling the Destination Service REST API [page 1562].
- **Cockpit**:
  You can add labels via the UI both while creating destinations and while editing existing ones.
  Labels are shown in a separate section of the destination details panel. You can perform the following operations:
  1. Add new labels via the *Add Label* button.
  2. Type in a key for the label in the *Label* field (or change the previous key).
  3. Add values for the label. Type a value and press *Enter* when done. You can add more values (up to 10) for the same label.
  4. Remove an individual value from a label by pressing on its token in the text box.
  5. Remove the entire label by pressing the *Delete* button.



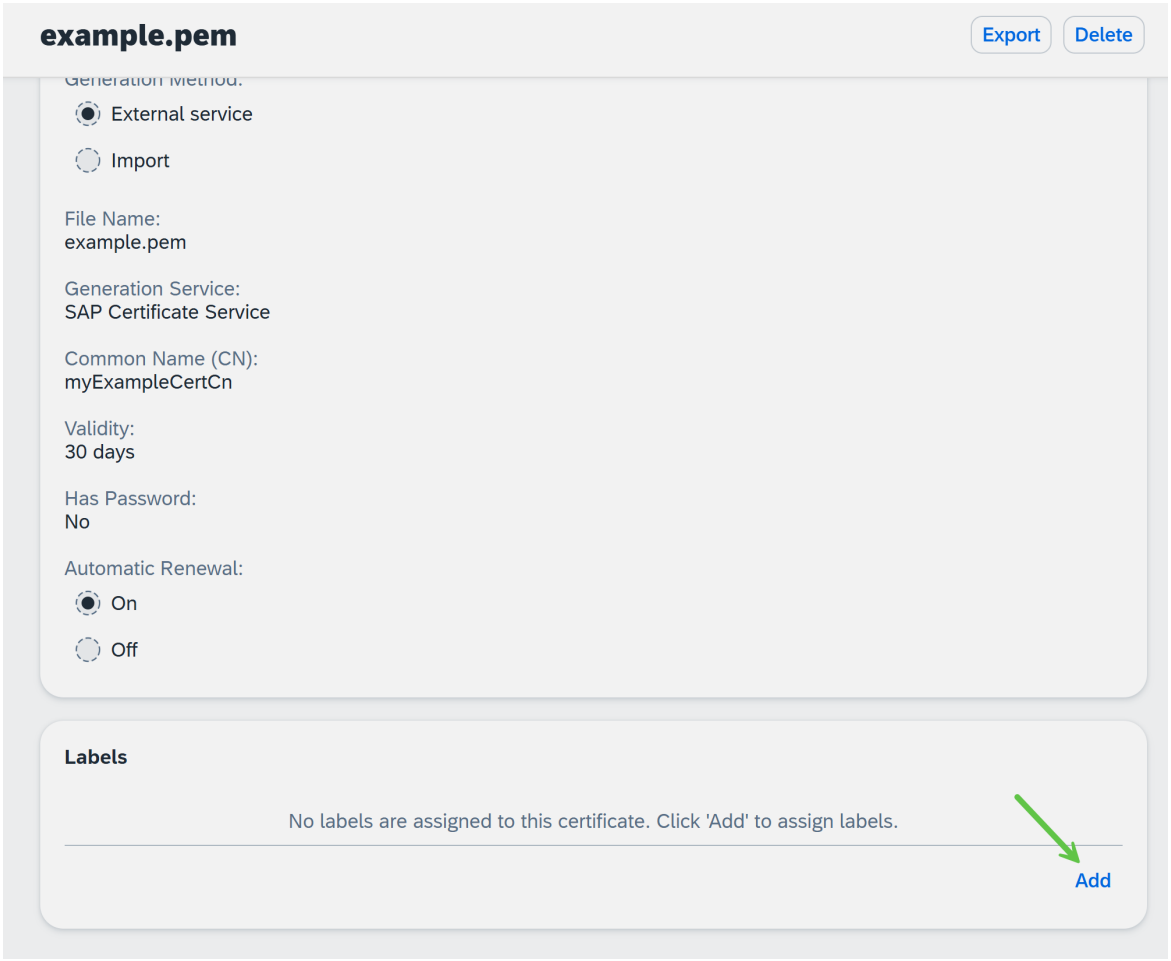
Certificates

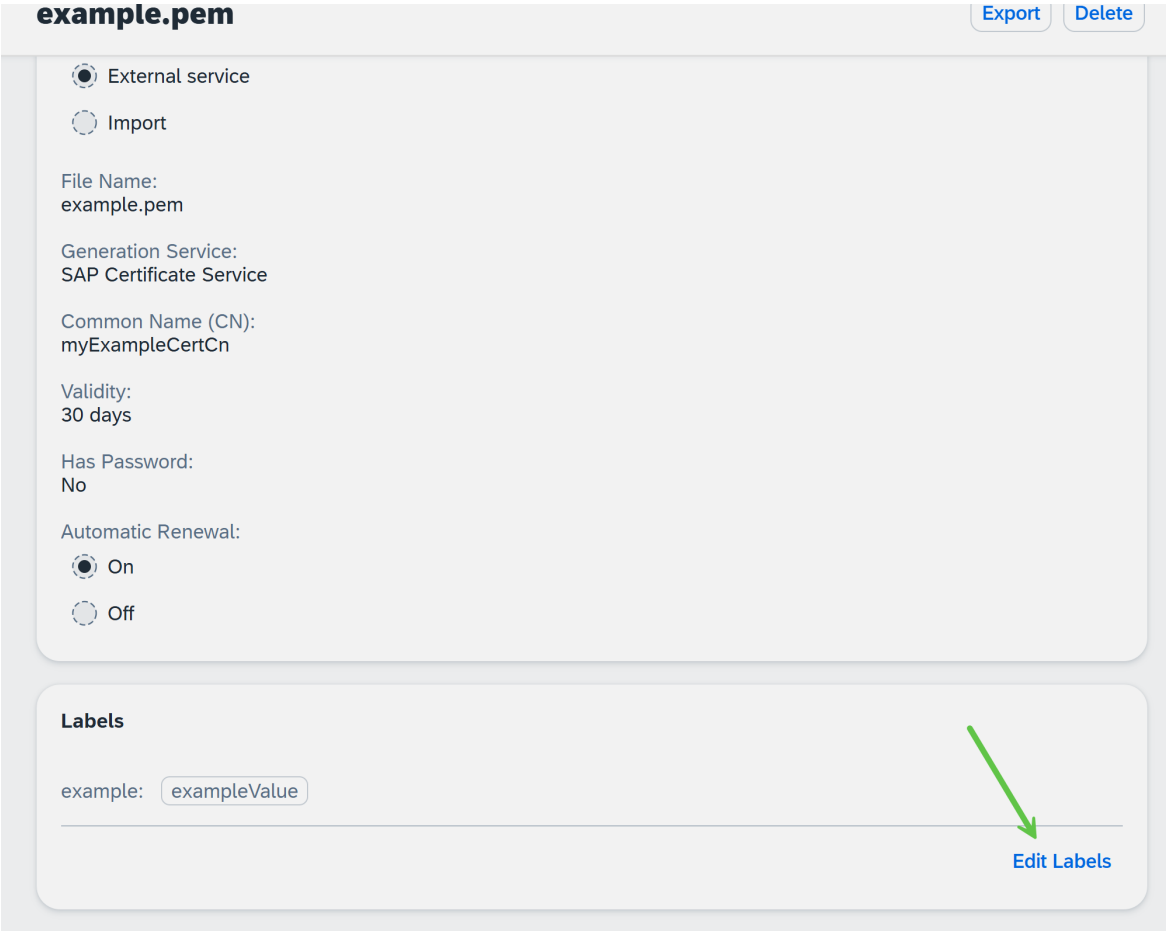
- **REST API**:
  In the entity-specific APIs on the subaccount and instance/subscription levels, there are endpoints for reading all labels, overriding all labels and inserting/deleting individual labels.
  For more information on the REST API, see Calling the Destination Service REST API [page 1562].
- **Cockpit**:
  You can add labels via the UI both while creating certificates and to an existing one.
  During Creation
  1. Add new labels via the *Add Label* button.
  2. Enter a key for the label in the *Label* field (or change the previous key).
  3. Add values for the label. Enter a value and press *Enter* when done. You can add more values this way for the same label (up to 10).
  4. Remove an individual value from a label by pressing on its token in the text box.
  5. Remove the whole label by pressing the *Delete* button.



Existing Certificate

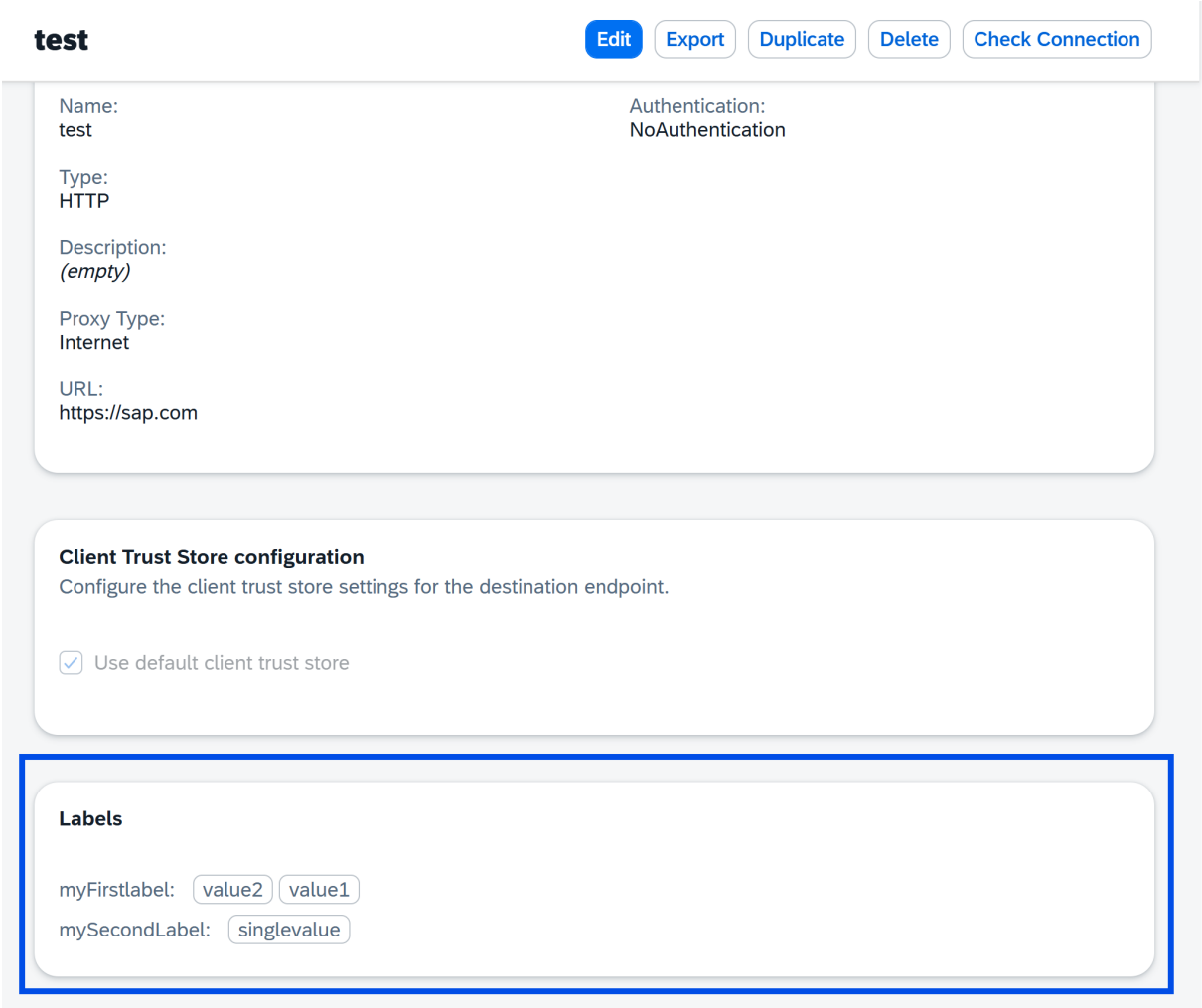
Labels are in a separate section of the *Certificate Details* panel. Choose *Add* (if there are no labels yet) or *Edit Labels* (if there are already some labels) and manage the labels in the same way as while creating the certificate. Choose *Save* once done.


example.pem
Export
Delete
Generation Method:
External service
Import
File Name:
example.pem
Generation Service:
SAP Certificate Service
Common Name (CN):
myExampleCertCn
Validity:
30 days
Has Password:
No
Automatic Renewal:
On
Off
Labels
No labels are assigned to this certificate. Click 'Add' to assign labels.
Add



Using Labels

Labels are visualized in the cockpit for managing the different entities that support them. This lets you understand better what the entity is used for, or what its status is. For example, you may have a *scenario* label and some destinations assigned to it with value *order_processing*, while others have the value *invoice_management* for the same label. Like this, the UI clearly shows the assignment of a destination to a specific scenario.



In addition, labels can be used in various REST APIs. This includes getting the labels as metadata during lookup and filtering in the *Get All* APIs based on having a label with desired value(s).

For more information on the REST API, see Calling the Destination Service REST API [page 1562].

Limits

- An individual entity can have up to 10 labels.
- An individual label can have up to 10 values.

1.3.1.11 Configuring Backup

To create a backup of your destination configurations, choose the procedure Export Destinations [page 1496].

1.3.2 Consuming the Destination Service

Retrieve and store externalized technical information about the destination to consume a target remote service from your application.

Tasks

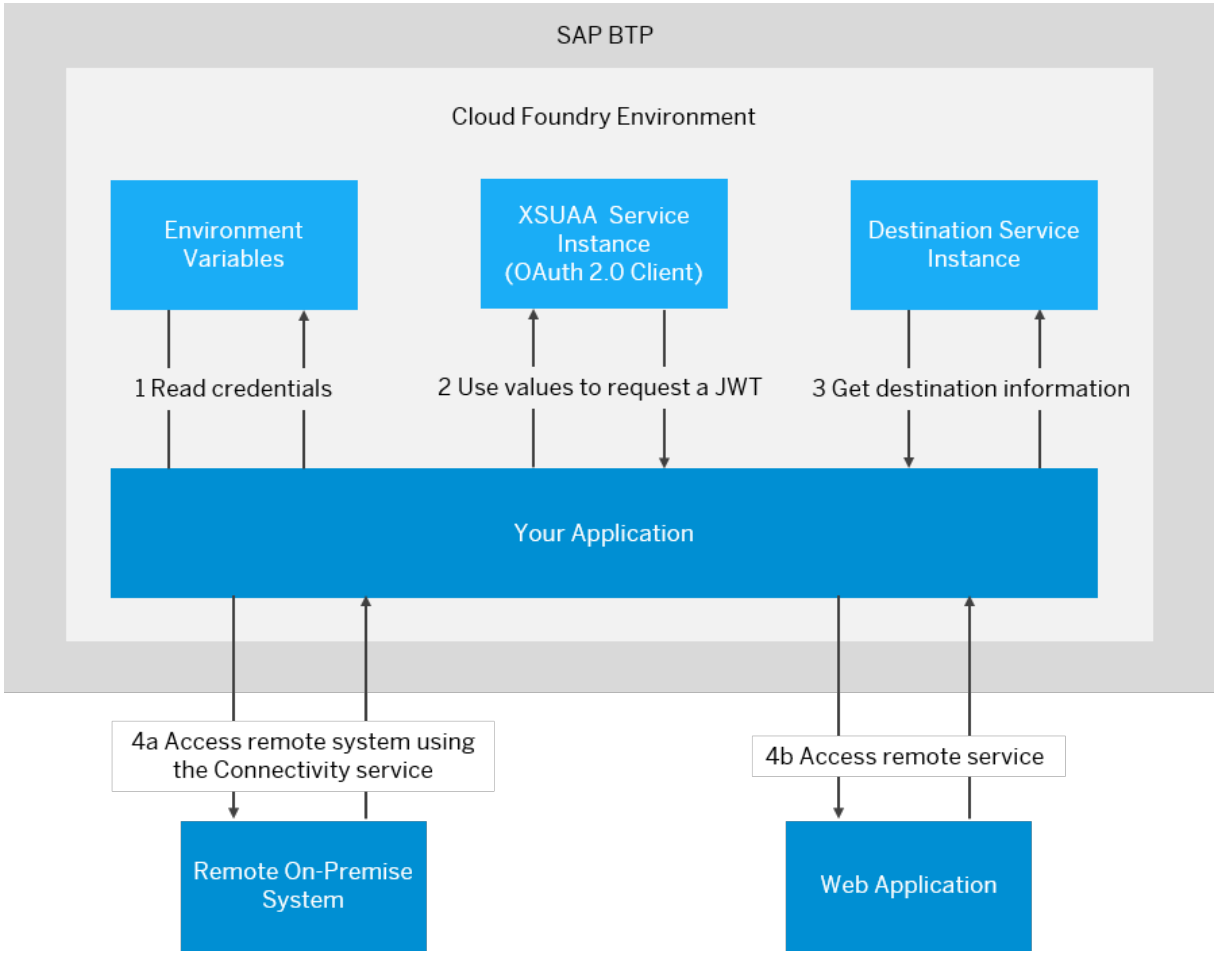
Task Type	Task
Operator and/or Developer	Overview [page 1550] Prerequisites [page 1551]
Developer	Steps [page 1552] 1. Read Credentials from the Environment Variables [page 1552] 2. Generate a JSON Web Token (JWT) [page 1553] 3. Call the Destination Service [page 1554]
Operator and/or Developer	Destination Configuration Attributes [page 1557]

Overview

The Destination service lets you find the destination information that is required to access a remote service or system from your cloud application.

- For the connection to an *on-premise system*, you can optionally use this service, together with (i.e. in addition to) the Connectivity service, see Consuming the Connectivity Service [page 1314].
- For the connection to any other *Web application* (remote service), you can use the Destination service without the Connectivity service.

Consuming the Destination Service includes user authorization via a JSON Web Token (JWT) that is provided by the *xsuaa* service.



Back to Tasks [page 1550]

Prerequisites

- To manage destinations and certificates on service instance level (all CRUD operations), you must be assigned to one of the following roles: `OrgManager`, `SpaceManager` or `SpaceDeveloper`.

  > ⓘ Note
  >
  > The role `SpaceAuditor` has only *Read* permission for destinations and certificates.

- To consume the Destination service from an application, you must create a service instance and bind it to the application. See Create and Bind a Destination Service Instance [page 1523].
- To generate the required JSON Web Token (JWT), you must bind the application to an instance of the *xsuaa* service using the service plan 'application'. The *xsuaa* service instance acts as an OAuth 2.0 client

and grants user access to the bound application. Make sure that you set the `xsappname` property when creating the instance. Find a detailed guide for this procedure in section *3. Creation of the Authorization & Trust Management Instance (aka XSUAA)* of the SCN blog How to use SAP BTP Connectivity and Cloud Connector in the Cloud Foundry environment.

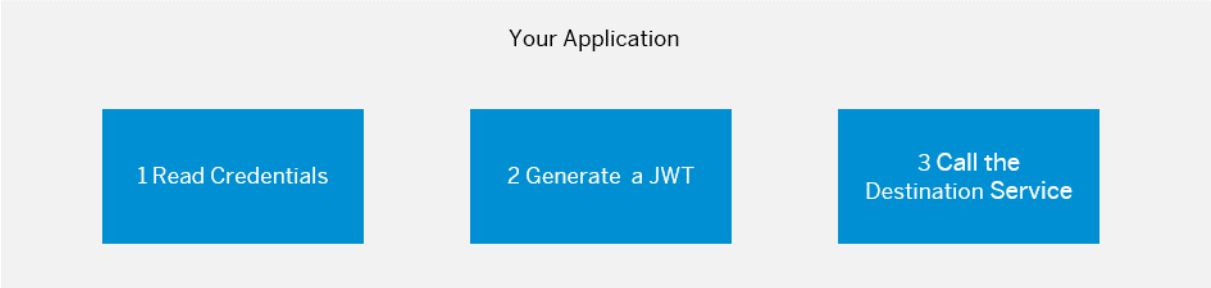
- You need at least one configured destination, otherwise there will be nothing to retrieve via the service. To access the *Destinations* editor in the cockpit, follow the steps in Access the Destinations Editor [page 1473].
  To manage destinations via REST API, see Destination Service REST API [page 1497].

Back to Tasks [page 1550]

Steps

To consume the Destination service from your application, perform the following basic steps:

1. Read Credentials from the Environment Variables [page 1552]
2. Generate a JSON Web Token (JWT) [page 1553]
3. Call the Destination Service [page 1554]



Back to Tasks [page 1550]

Read Credentials from the Environment Variables

The Destination service stores its credentials in the environment variables. To consume the service, you require the following information:

- The value of `clientid`, `clientsecret` and `uri` from the Destination service credentials.

- The values of `url` from the *xsuaa* credentials.

You can access this information as follows:

- From the CLI, the following command lists the environment variables of `<app-name>`:

  ```
  cf env <app-name>
  ```

- From within the application, the service credential can be accessed as described in Consuming the Connectivity Service [page 1314].

> ### Note
>
> Below, we refer to the *JSONObjects*, containing the instance credentials as `destinationCredentials` (for the Destination service) and `xsuaaCredentials` (for xsuaa).

Back to Tasks [page 1550]

Generate a JSON Web Token (JWT)

Your application must create an OAuth client using the attributes `clientid` and `clientsecret`, which are provided by the Destination service instance. Then, you must retrieve a new JWT from UAA and pass it in the *Authorization* HTTP header.

Two examples how to achieve this (Java and cURL):

Java:

For a Java application, you can use a library that implements the client credentials OAuth flow. Here is an example of how the `clientCredentialsTokenFlow` can be obtained using the XSUAA Token Client and Token Flow API :

> ### Caution
>
> Make sure you get the latest API version.

A sample Maven dependency declaration:

```
<dependency>
    <groupId>com.sap.cloud.security.xsuaa</groupId>
    <artifactId>token-client</artifactId>
    <version><latest version (e.g.: 2.7.7)></version>
</dependency>
```

→ Remember

The XSUAA Token Client library works with multiple HTTP client libraries. Make sure you have one as Maven dependency.

The following sample uses the Apache REST client:

Sample Code

```
// get value of "clientid" and "clientsecret" from the environment variables
String clientid = destinationCredentials.getString("clientid");
String clientsecret = destinationCredentials.getString("clientsecret");

// get the URL to xsuaa from the environment variables
URI xsuaaUri = new URI(xsuaaCredentials.getString("url"));

// use the XSUAA client library to ease the implementation of the user token
exchange flow
XsuaaTokenFlows tokenFlows = new XsuaaTokenFlows(new
DefaultOAuth2TokenService(), new XsuaaDefaultEndpoints(xsUaaUri.toString()),
new ClientCredentials(clientid, clientsecret));

String jwtToken =
tokenFlows.clientCredentialsTokenFlow().execute().getAccessToken();
```

For more information about caching, see also XSUAA Token Client and Token Flow API - Cache .

cURL:

Sample Code

```
curl -X POST \
  <xsuaa-url>/oauth/token \
  -H 'authorization: Basic <<clientid>:<clientsecret> encoded with Base64>' \
  -H 'content-type: application/x-www-form-urlencoded' \
  -d 'client_id=<clientid>&grant_type=client_credentials'
```

Back to Tasks [page 1550]

Call the Destination Service

When calling the Destination service, use the `uri` attribute, provided in `VCAP_SERVICES`, to build the request URLs.

Read a Destination by only Specifying its Name ("Find Destination") [page 1555]

Read a Destination Associated with a Subaccount [page 1555]

Get All Destinations Associated with a Subaccount [page 1556]

Response Codes [page 1557]

Read a Destination by only Specifying its Name ("Find Destination")

This lets you provide simply a name of the destination while the service will search for it. First, the service searches the destinations that are associated with the service instance. If none of the destinations match the requested name, the service searches the destinations that are associated with the subaccount.

- Path: `/destination-configuration/v1/destinations/<destination-name>`
- Example of a call (cURL):

  Sample Code

  ```
  curl "<uri>/destination-configuration/v1/destinations/<destination-name>" \
  -X GET \
  -H "Authorization: Bearer <jwtToken>"
  ```

- Example of a response (this is a destination found when going through the subaccount destinations):

  Sample Code

  ```
  {
  "owner":
  {
  "SubaccountId":<id>,
  "InstanceId":null
  },
  "destinationConfiguration":
  {
  "Name": "demo-internet-destination",
  "URL": "http://www.google.com",
  "ProxyType": "Internet",
  "Type": "HTTP",
  "Authentication": "NoAuthentication"
  }
  }
  ```

> **Note**
>
> The response from this type of call contains not only the configuration of the requested destination, but also some additional data. See "Find a Destination" Response Structure [page 1565].

Back to Call the Destination Service [page 1554]

Read a Destination Associated with a Subaccount

This lets you retrieve the configurations of a destination that is defined within a subaccount, by providing the name of the destination.

- Path: `/destination-configuration/v1/subaccountDestinations/<destination-name>`

- Example of a call (cURL):

  Sample Code

  ```
  curl "<uri>/destination-configuration/v1/subaccountDestinations/
  <destination name>" \
  -X GET \
  -H "Authorization: Bearer <jwtToken>"
  ```

- Example of a response:

  Sample Code

  ```
  {
  "Name": "demo-internet-destination",
  "URL": "http://www.google.com",
  "ProxyType": "Internet",
  "Type": "HTTP",
  "Authentication": "NoAuthentication"
  }
  ```

Back to Call the Destination Service [page 1554]

Get All Destinations Associated with a Subaccount

This lets you retrieve the configurations of all destinations that are defined within a subaccount.

- Path: `/destination-configuration/v1/subaccountDestinations`
- Example of a call (cURL):

  Sample Code

  ```
  curl "<uri>/destination-configuration/v1/subaccountDestinations" \
  -X GET \
  -H "Authorization: Bearer <jwtToken>"
  ```

- Example of a response:

  Sample Code

  ```
  [
  {
  "Name": "demo-onpremise-destination1",
  "URL": "http:/virtualhost:1234",
  "ProxyType": "OnPremise",
  "Type": "HTTP",
  "Authentication": "NoAuthentication"
  },
  {
  "Name": "demo-onpremise-destination2",
  "URL": "http:/virtualhost:4321",
  "ProxyType": "OnPremise",
  "Type": "HTTP",
  "Authentication": "BasicAuthentication",
  "User": "myname123",
  "Password": "123456"
  }
  ```

```
]
```

Back to Call the Destination Service [page 1554]

Response Codes

When calling the Destination service, you may get the following response codes:

- **200**: OK (Json of Destination)
- **401**: Unauthorized (Authentication Failed)
- **403**: Forbidden (Authorization Failed)
- **404**: The requested destination could not be found (not applicable to 'get all destinations associated with a subaccount')
- **500**: Internal Server Error

Back to Call the Destination Service [page 1554]

Back to Tasks [page 1550]

Destination Configuration Attributes

The JSON object that serves as the response of a successful request (value of the `destinationConfiguration` property for "Find destination") can have different attributes, depending on the authentication type and proxy type of the corresponding destination. See HTTP Destinations [page 1350].

Back to Tasks [page 1550]

Related Information

User Propagation via SAML 2.0 Bearer Assertion Flow [page 1558]
Destination Service REST API [page 1497]
Exchanging User JWTs via OAuth2JWTBearer Destinations [page 1579]
Use Cases [page 69]
Multitenancy in the Destination Service [page 1581]
Destination Java APIs [page 1583]
Extending Destinations with Fragments [page 1590]
Using an IAS User Token for Corporate IdP Token Principal Propagation to On-Premise Systems [page 1599]
Resolve Dynamic Token-Based Property Values [page 1599]

1.3.2.1 User Propagation via SAML 2.0 Bearer Assertion Flow

Learn about the process for automatic token retrieval, using the `OAuth2SAMLBearerAssertion` authentication type for HTTP destinations.

Tasks

Task Type	Task
Operator and/or Developer	Prerequisites [page 1558]
Developer	Automated Access Token Retrieval [page 1559] • Determine the Propagated User ID [page 1559] • Propagate User Attributes [page 1560] • Scenarios [page 1561]

Prerequisites

- You have configured an `OAuth2SAMLBearerAssertion` destination. See OAuth SAML Bearer Assertion Authentication [page 1363].
- The user's identity should be represented by a JSON Web token (JWT).

  > ### Note
  >
  > Though actually not being a strict requirement, it is likely that you need a user JWT to get the relevant information. See SAP Authorization and Trust Management Service.

- If you are using custom user attributes to determine the user, the JWT representing the user (that is passed to the Destination service) must have the `user_attributes` scope.

Back to Tasks [page 1558]

Automated Access Token Retrieval

For an `OAuth2SAMLBearerAssertion` destination, you can use the automated token retrieval functionality that is available via the "find destination" endpoint. See Destination Service REST API [page 1497].

Determine the Propagated User ID

There are currently two sources that can provide the propagated user ID. They are prioritized, meaning that the lookup always starts from the top-priority source and goes down the list. If the propagated user ID is not found at a given level, the next level is checked. If not found on any level, the operation would fail.

Find the available sources in the table below, in order of their priority.

Propagated User ID: Sources

Source	Procedure

Field in the JWT	In this case, the Destination service looks for the user ID as a field in the provided JWT. When you make the HTTP call to the Destination service, you must provide the `Authorization` header. The value must be a JWT in its encoded form (see RFC 7519). The procedure is as follows:
	• If the `userIdSource` property is configured in the destination, its value is the key of the JWT field that will be the user ID. There are 2 options:
	  • plain string: the exact match is searched on the root-level element keys of the JWT.
	  • JsonPath expression: lets you use non-root-level elements of the JWT.
	• If the `userIdSource` property is missing, the flow falls back to the destination property `nameIdFormat`. It must have one of the following two values (or not be set at all), otherwise an exception is thrown:
	  • **urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress**: the `email` element of the JWT is the user ID. If there is no such element in the JWT, an exception is thrown.
	  • **urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified** (or not set): the `user_name` element of the JWT is the user ID. If there is no such element in the JWT, an exception is thrown.

Source	Procedure
Custom User Attribute	Like *Field in the JWT*, this source must use the `Authorization` header. In this case, its value is used to retrieve the custom user attributes from the Identity Provider (XSUAA). One of those attributes can be used as the propagated user ID. The access token from the header must be a user JWT with the `user_attributes` scope. Otherwise the custom attributes cannot be retrieved, and the operation results in an error. • If the `userIdSource` property is configured in the destination, the same logic applies as for *Field in the JWT*, but this time on the JSON containing the custom user attributes. • If `userIdSource` is missing or the desired key is not found in the custom attributes, the operation fails (`user ID could not be determined`).

Back to Tasks [page 1558]

Propagate User Attributes

You can read additional user attributes from the identity provider (XSUAA), and propagate them as SAML attributes in the generated SAML bearer assertion.

These attributes are similar to the ones returned by the Cloud Foundry UAA user info endpoint. However, they may differ depending on the XSUAA behavior, and are specific to the identity provider you use.

For more details about these attributes and possible values, see https://docs.cloudfoundry.org/api/uaa/version/74.15.0/index.html#user-info .

> **Note**
>
> When adding the attributes, the following rules apply:
>
> - All root elements, except for `user_attributes`, are added "as is", that is, the attribute name and value are displayed as seen in the source (user info response structure).
> - Elements under the `user_attribute` key are parsed and added as attributes prefixed with '`user_attributes.`'. For example, having `{"user_attributes": { "my_param": "my_value" }}` will result in an attribute called '`user_attributes.my_param`' with value '`my_value`' in the SAML assertion. If you want to avoid this `user_attributes.` prefix, you can set the `skipUserAttributesPrefixInSAMLAttributes` additional property of the destination to **`true`**. If you do so, the above example will result in an attribute called `my_param` with value `my_value` in the SAML assertion.

In addition to identity provider (XSUAA) user info attributes, there are some more attributes which are read from the passed JWT. They are located via predefined `JsonPath` expressions and cannot be controlled by the end user:

- `$.['xs.system.attributes']['xs.saml.groups']`
- `$.['user_attributes']['xs.saml.groups']`

> **Note**
>
> The `'xs.saml.groups'` attribute, read from the passed JWT, is renamed to `'Groups'` in the resulting SAML assertion. See also Settings for Default SAML Federation Attributes of Identity Providers for Business Users.

Back to Tasks [page 1558]

Scenarios

Refer to the table below to find the JWT requirements for a specific scenario:

Scenario	Authorization Header
Propagate a **business user** principal, using an `OAuth2SAMLBearerAssertion` destination maintained in the **same subaccount** where the application is deployed. The business user is represented by a JWT that was **issued by the provider**.	The JWT, previously retrieved by the application • by exchanging the JWT (that represents the user) to another user access token via the client credentials of the Destination service instance (bound to the application). • using the provider's tenant-specific `Token Service URL`.
Propagate a **business user** principal, using an `OAuth2SAMLBearerAssertion` destination maintained in the **subscriber subaccount** where the application is deployed. The business user is represented by a JWT that was **issued by the subscriber**.	The JWT, previously retrieved from the application • by exchanging the JWT (that represents the user) to another user access token via the client credentials of the Destination service instance (bound to the application). • using the subscriber's tenant-specific `Token Service URL`.

Back to Tasks [page 1558]

1.3.2.2 Destination Service REST API

Destination service REST API specification for SAP BTP.

The Destination service provides a REST API that you can use to read and manage resources like destinations, certificates and destination fragments on all available levels. This API is documented in the SAP Business Accelerator Hub.

It shows all available endpoints, the supported operations, parameters, possible error cases and related status codes, etc. You can also execute requests using the credentials (for example, the service key) of your Destination service instance, see Create and Bind a Destination Service Instance [page 1523].

Related Information

Calling the Destination Service REST API [page 1562]
"Find a Destination" Response Structure [page 1565]
Referring Resources Using the Consume REST API [page 1569]
Calling the Destination Service REST API via IAS Token [page 1575]

1.3.2.2.1 Calling the Destination Service REST API

Prerequisites and steps to get access to the Destination service REST API.

Prerequisites

To call the Destination service REST API, you must have a Destination service instance inside your subaccount.

If you:

- **don't have** such instance, follow the next step to get it up and running.
- **have** such instance, you can skip the next step and go to Get Credentials [page 1563].

1. Set up a Destination Service Instance for your Subaccount

To create a Destination service instance inside your subaccount, see Creating Service Instances about creating service instances in the **BTP cockpit**, or from the **Cloud Foundry CLI**.

> ⓘ Note
>
> When creating a Destination service instance, you can refer to the following yaml segment for the basic information of the instance.

Basic information for the Destination service instance
• *Plan*: lite Currently, the Destination service offers only this plan. • *Runtime Environment*: Cloud Foundry Cloud Foundry must be enabled for your subaccount. • *Space*: <space_name> Choose the space in which the Destination service instance will reside. • *Instance Name*: <instance_name> Enter a name of your choice here for the instance.

> ### Note
>
> The yaml segment above contains Cloud Foundry-specific information. Other configurations are possible, for instance when running on Kyma. The important part is to deploy your instance in the target runtime and get the service key / binding credentials to access the Destination service.

2. Get Credentials

To access the Destination service REST API, you need an **access token**. To generate this token, you need credentials contained in a **service key** of the Destination service instance.

If you don't have any service keys in your Destination service instance, see Creating Service Keys about creating a service key for an instance in the **BTP cockpit** or from the **Cloud Foundry CLI**.

Once you have a service key for your Destination service instance, you open it and extract the following information:

Information to extract from the service key
• *clientid*: "<value_to_extract>" The client ID that will be used for authentication in the next step. • *clientsecret*: "<value_to_extract>" The client secret that will be used for authentication in the next step. • *url*: "<value_to_extract>" The authentication endpoint from which you get an access token for the Destination service. • *uri*: "<value_to_extract>" The URL of the Destination service.

> ### Note
>
> For demonstration purposes, we are using the `clientid` and `clientsecret` for authentication. We recommend that you use mTLS which uses an X.509 client certificate because it is a more secure way of authenticating.

3. Get an AccessToken

In this step, you will get an **access token** from XSUAA which you can then use to authenticate towards the Destination service REST API. For this step, you must use the values you extracted for **clientid**, clientsecret and **url** from the previous step.

Here is an example call using **curl**:

Sample Code

```
curl -X POST \
        "<url>/oauth/token" \
        -H "Content-Type: application/x-www-form-urlencoded" \
        -d "grant_type=client_credentials" --data-urlencode
"client_id=<client_id>" --data-urlencode "client_secret=<client_secret>"
```

where:

- `<url>` is the value of the URL from the previous step.
- `<client_id>` is the value of *clientid* from the previous step.
- `<client_secret>` is the value of *clientsecret* from the previous step.

The access token is shown in the **access_token** key in the response JSON. Make sure you save it because you will need it in the next step.

4. Call the Destination Service REST API

Now that you have an access token for the Destination service, you can call one of the Destination service REST API endpoints. To get the full list of available endpoints in the Destination service REST API, see the Destination Service REST API reference on SAP Business Accelerator Hub.

Here is an example of the call using **curl**:

Sample Code

```
curl -X GET \
        "<uri>/destination-configuration/v1/<endpoint>" \
        -H "Authorization: Bearer <access_token>"
```

where:

- `<uri>` is the value of *uri* from Get Credentials [page 1563].
- `<endpoint>` is the endpoint of the Destination service REST API that you want to call.
- `<access_token>` is the access token you saved from the previous section.

If, for example, you want to make a GET call towards the */subaccountDestinations* endpoint, the call would look like:

Sample Code

```
curl -X GET \
        "<uri>/destination-configuration/v1/subaccountDestinations" \
        -H "Authorization: Bearer <access_token>"
```

Example: Response from the Destination service

Sample Code

```
[
    {
        "Name": "no-authentication-destination",
        "Type": "HTTP",
        "URL": "https://sap.com",
        "Authentication": "NoAuthentication",
        "ProxyType": "Internet"
    },
    {
        "Name": "basic-authentication-destination",
        "Type": "HTTP",
        "URL": "https://sap.com",
        "Authentication": "BasicAuthentication",
        "ProxyType": "Internet",
        "User": "my-user",
        "Password": "my-password"
    }
]
```

1.3.2.2.2 "Find a Destination" Response Structure

Overview of data that are returned by the Destination service for the call type "Find a Destination".

Response Structure

When you use the "Find a Destination" call (read a destination by only specifying its name), the structure of the response includes four parts:

- The owner of the destination [page 1566].
- The actual destination configuration [page 1566].
- (Optional) Authentication tokens [page 1567] that are relevant to the destination.
- (Optional) Certificates [page 1568] that are relevant to the destination.

Each of these parts is represented in the JSON object as a key-value pair and their values are JSON objects, see Example [page 1569].

See also Call the Destination Service [page 1554].

Destination Owner

- **Key:** `owner`
  The JSON object that serves as the value of this property contains two properties itself: `SubaccountId` and `InstanceId`. `SubaccountId` has as its value the ID of the subaccount instance to which the destination belongs. Depending on where the destination was found (as a subaccount destination or a service instance destination) `InstanceId` can have the value `null`, or the ID of the service instance to which the destination belongs.
- **Example:**

  Sample Code

```
"owner": {
        "SubaccountId": "9acf4877-5a3d-43d2-b67d-7516efe15b11",
        "InstanceId": null
    }
```

Back to Response Structure [page 1565]

Destination Configuration

- **Key:** `destinationConfiguration`
  The JSON object that represents the value of this property contains the actual properties of the destination. To learn more about the available properties, see HTTP Destinations [page 1350].
  If a destination fragment was specified in the "Find a Destination" call, the value of this property is represented by the JSON object that contains the actual properties of the destination, merged with the JSON object that contains the properties of the destination fragment.
  For more information on how to extend a destination with a destination fragment, see Extending Destinations with Fragments [page 1590].
- **Example:**

  Sample Code

```
"destinationConfiguration": {
        "Name": "TestBasic",
        "Type": "HTTP",
        "URL": "http://sap.com",
        "Authentication": "BasicAuthentication",
        "ProxyType": "OnPremise",
        "User": "test",
```

```
        "Password": "pass12345"
    }
```

Back to Response Structure [page 1565]

Authentication Tokens

> **Note**
>
> This property is only applicable to destinations that use the following authentication types: *BasicAuthentication*, *OAuth2SAMLBearerAssertion*, *OAuth2ClientCredentials*, *OAuthUserTokenExchange*, *OAuth2JWTBearer*, *OAuth2Password*, *SAMLAssertion*, *OAuth2RefreshToken*, *OAuth2AuthorizationCode*, *OAuth2TechnicalUserPropagation*, *OAuth2TokenExchange*.

> **Restriction**
>
> The section will contain an error if the `tokenServiceUrl` is a private endpoint (like *localhost*) and `ProxyType` is **`Internet`**, as the automation cannot be performed on the server side in this case.

> **Note**
>
> This section will not be present if automatic token retrieval is skipped by setting the query parameter `$skipTokenRetrieval=true`.

- **Key:** `authTokens`
  The JSON array that represents the value of this property contains tokens that are required for authentication. These tokens are represented by JSON objects with these properties (expect more new properties to be added in the future):
  - `type`: the type of the token.
  - `value`: the actual token.
  - `http_header`: JSON object containing the prepared token in the correct format. The `<key>` field contains the key of the HTTP header. The `<value>` field contains the value of the header.
  - `expires_in` (only in OAuth2 destinations): The lifetime in seconds of the access token. For example, the value "3600" denotes that the access token will expire in one hour from the time the response was generated.
  - `error` (optional): if the retrieval of the token fails, the value of both `type` and `value` is an empty string and this property shows an error message, explaining the problem.
  - `scope` (optional) (only in OAuth2 destinations): The scopes issued with the token. The value of the scope parameter is expressed as a list of space-delimited strings. For example, `read write execute`.
  - `refresh_token` (optional) (only in OAuth2 destinations): A refresh token, returned by the OAuth service. It can be used to renew the access token via OAuth Refresh Token Authentication [page 1409].

- `issued_token_type` (optional) (present and required only in `OAuth2TokenExchange` destinations): Identifier for the representation of the issued security token (for example, "urn:ietf:params:oauth:token-type:access_token").

- **Example:**

  Sample Code

```
"authTokens": [
        {
            "type": "Basic",
            "value": "dGVzdDpwYXNzMTIzNDU=",
            "http_header": {
                "key":"Authorization",
                "value":"Basic dGVzdDpwYXNzMTIzNDU="
             }
        }
    ]
```

Back to Response Structure [page 1565]

Certificates

> **Note**
>
> This property is only applicable to destinations that use mTLS for token retrieval and automatic token retrieval is skipped, or destinations with the following authentication types: *ClientCertificateAuthentication*, *OAuth2SAMLBearerAssertion* (when default JDK trust store is not used).

- **Key:** `certificates`
  The JSON array that represents the value of this property contains the certificates, specified in the destination configuration. These certificates are represented by JSON objects with these properties (expect more new properties to be added in the future):
  - `type`
  - `content`: the encoded content of the certificate.
  - `name`: the name of the certificate, as specified in the destination configuration.
- **Example:**

  Sample Code

```
"certificates": [
        {
            "Name": "keystore.jks",
            "Content": "<value>"
            "Type": "CERTIFICATE"
        },
        {
            "Name": "truststore.jks",
            "Content": "<value>"
            "Type": "CERTIFICATE"
        }
```

```
    ]
```

Back to Response Structure [page 1565]

Example

Example of a full response for a destination using basic authentication:

Sample Code

```
{
    "owner": {
        "SubaccountId": "9acf4877-5a3d-43d2-b67d-7516efe15b11",
        "InstanceId": null
    },
    "destinationConfiguration": {
        "Name": "TestBasic",
        "Type": "HTTP",
        "URL": "http://sap.com",
        "Authentication": "BasicAuthentication",
        "ProxyType": "OnPremise",
        "User": "test",
        "Password": "pass12345"
    },
    "authTokens": [
        {
            "type": "Basic",
            "value": "dGVzdDpwYXNzMTIzNDU="
            "http_header": {
                "key":"Authorization",
                "value":"Basic dGVzdDpwYXNzMTIzNDU="
            }
        }
    ]
}
```

Back to Response Structure [page 1565]

1.3.2.2.3 Referring Resources Using the *Consume* REST API

Select a level for destinations and destination fragments when using the Destination service *Consume* REST API in a single-tenant or multi-tenant scenario.

The Destination service *Consume* REST API lets you retrieve a destination in a consumption-ready state. When calling the API, you must provide the level (see usage options below) where the destination is located (*subaccount* or *instance*). In multi-tenant scenarios, the destination can also be located on the levels *provider_subaccount* or *provider_instance*.

In addition to the destination level, you can also provide a fragment level. If not specified, the level of the destination is assumed.

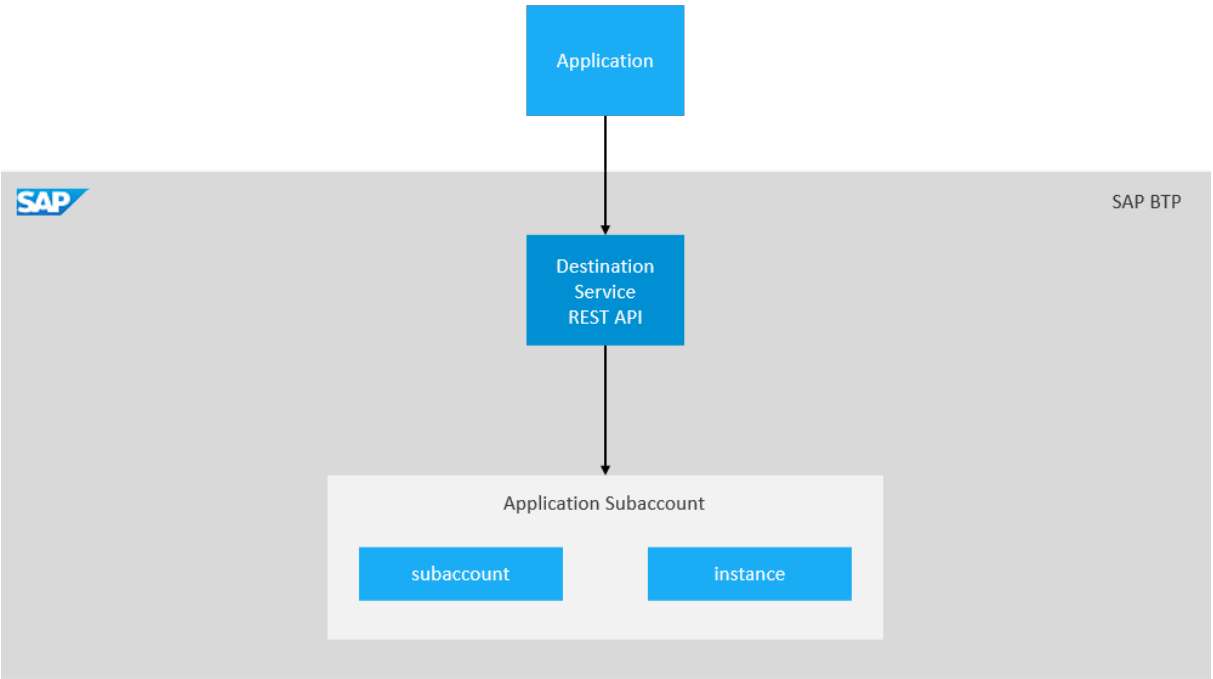
> ⚠ Caution
>
> In contrast to the *Find a Destination* API, a fallback between levels is not performed.

> ⚠ Caution
>
> Authorization with a token from the Identity Authentication service (IAS) is currently not supported.

Single-Tenant Usage

Supported Levels

- `subaccount`: resources accessible on subaccount level by applications using any Destination service instance in the subaccount
- `instance`: resources accessible on instance level by applications using the concrete Destination service instance



The following sample request shows how you can retrieve a destination with name `dest1` on `subaccount` level.

> {·} Sample Code
>
> ```
> curl --request GET
> ```

```
  --url https://<destination-service-url>/destination-configuration/v2/
destinations/dest1@subaccount
  --header 'Authorization: Bearer {{account token}}'
```

Support for Fragments

When calling the API, you can also provide the level where the fragment is located (`subaccount` or `instance`). The destination and fragment can either be on the same level, or on different levels.

The following sample request shows how you can retrieve a destination with name `dest1` on `subaccount` level, enriched with a fragment with name `fragment2` on `instance` level.

Sample Code

```
curl --request GET
  --url https://<destination-service-url>/destination-configuration/v2/
destinations/dest1@subaccount
  --header 'Authorization: Bearer {{account token}}'
  --header 'X-fragment-name: fragment2@instance'
```

Multi-Tenant Usage

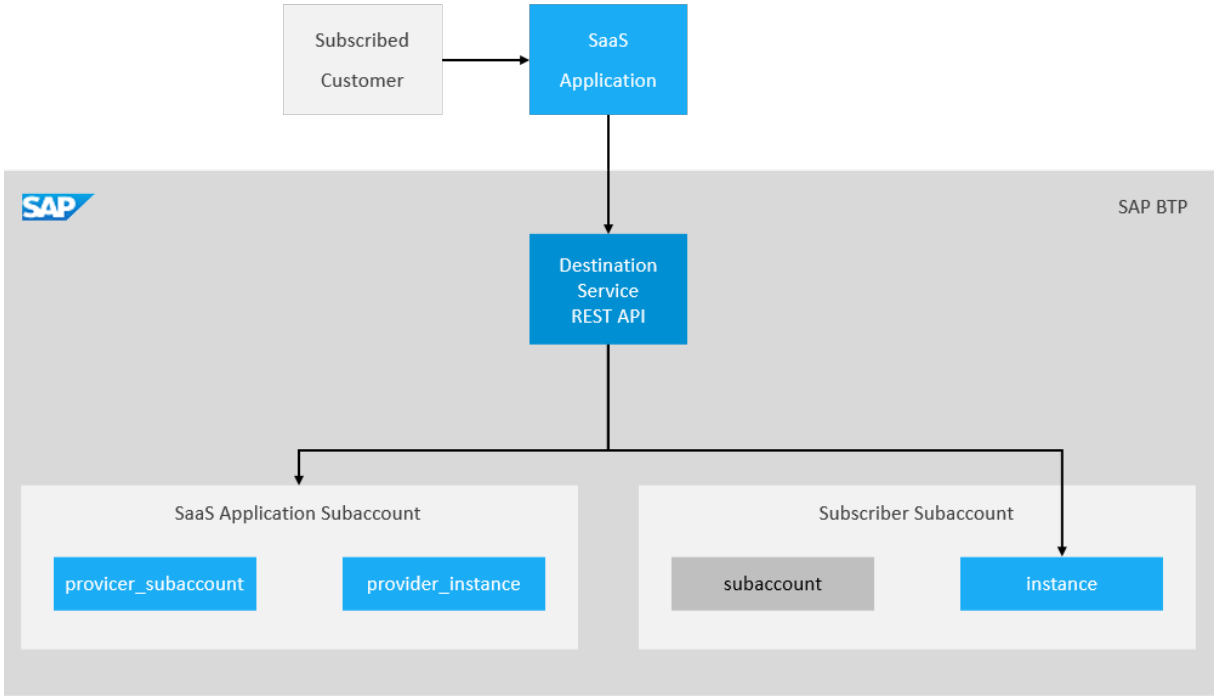
The API supports multi-tenant scenarios in which it is being called with an authorization token on behalf of the subscriber.

Supported Levels

- `provider_subaccount`: resources accessible on provider subaccount level
- `provider_instance`: resources accessible on provider instance level
- `subaccount`: resources accessible on subscriber subaccount level
- `instance`: resources accessible on subscriber instance level

Support for Fragments

Requesting Destination and Fragment on Different Levels



When calling the API, you can also provide the level where the fragment is located (`provider_subaccount`, `provider_instance` or `instance`). The destination and fragment can either be on the same level, or on different levels.

> ⚠ Caution
>
> Due to security considerations, retrieving one resource from subaccount level and another resource from any other level is forbidden.

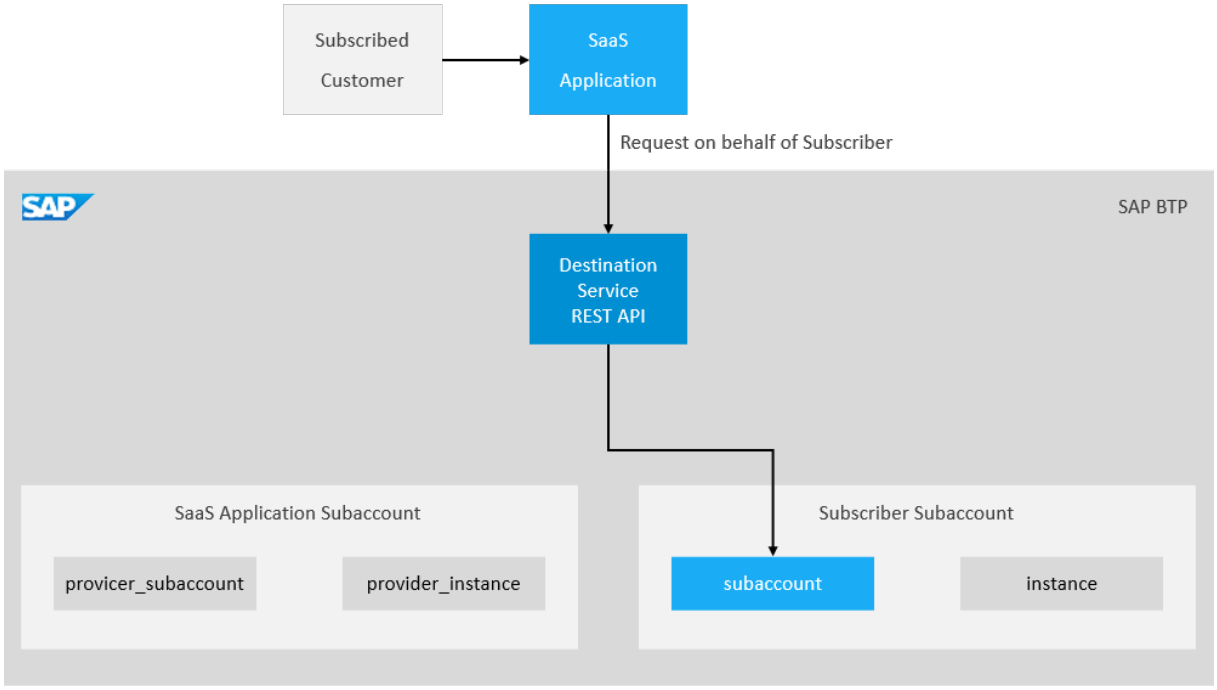
The following sample request shows how you can retrieve a destination with name `dest1` on `provider_subaccount` level, enriched with a fragment with name `fragment2` on `instance` level.

> {·} Sample Code

```
curl --request GET
  --url https://<destination-service-url>/destination-configuration/v2/
destinations/dest1@provider_subaccount
  --header 'Authorization: Bearer {{subscriber token}}'
  --header 'X-fragment-name: fragment2@instance'
```

Requesting resources on a single level has no restrictions.

Requesting Destination and Fragment on the Same Level



The following sample request shows how you can retrieve a destination with name `dest1` on `subaccount` level, enriched with a fragment with name `fragment2` on the same level.

Sample Code

```
curl --request GET
  --url https://<destination-service-url>/destination-configuration/v2/
destinations/dest1@subaccount
  --header 'Authorization: Bearer {{subscriber token}}'
  --header 'X-fragment-name: fragment2@subaccount'
```

Response Structure

The response structure of the *Consume* REST API is identical to the *Find a Destination* response structure with one difference: the destination owner. By default, the "owner" would not be present in the response, but if required, it can be included by providing the query parameter `includeMetadata` with value `owner`. The `owner` section (if requested) is located inside the `destinationConfiguration` section, under the `$metadata` key. The semantics remain the same, but since our final result could be constructed from resources located on different levels, the structure is extended, in order to accommodate multiple owners. Additionally, a reference was added to the name of the resource used for extracting.

Here is an example:

Sample Code

```
{
```

```
    "destinationConfiguration":
    {
        "Name": "dest1",
        "Type": "HTTP",
        "URL": "http://sap.com",
        "Authentication": "BasicAuthentication",
        "ProxyType": "OnPremise",
        "User": "test",
        "Password": "pass12345",
        "FragmentName": "fragment2",
        "$metadata":
        {
            "owner":
            {
                "destinations":
                [
                    {
                        "Name": "dest1",
                        "SubaccountId": "9acf4877-5a3d-43d2-
b67d-7516efe15b11",
                        "InstanceId": null
                    }
                ],
                "certificates":
                [
                    {
                        "Name": "cert1",
                        "SubaccountId": "9acf4877-5a3d-43d2-
b67d-7516efe15b11",
                        "InstanceId": null
                    }
                ],
                "fragments":
                [
                    {
                        "FragmentName": "fragment2",
                        "SubaccountId": "7516efe15b11-
b67d-43d2-5a3d-9acf4877",
                        "InstanceId": "255de9a7-64d9-496d-9a6d-2151ccf97264"
                    }
                ]
            }
        }
    },
    "authTokens":
    [
        {
            "type": "Basic",
            "value": "dGVzdDpwYXNzMTIzNDU=",
            "http_header":
            {
                "key": "Authorization",
                "value": "Basic dGVzdDpwYXNzMTIzNDU="
            }
        }
    ]
}
```

Related Information

Multitenancy in the Destination Service [page 1581]

Calling the Destination Service REST API [page 1562]
Extending Destinations with Fragments [page 1590]

1.3.2.2.4 Calling the Destination Service REST API via IAS Token

Find a step-by-step procedure to use an IAS token for calling available Destination service REST API endpoints.

Prerequisites

To call the Destination service REST API using an IAS token, you must have the following set up:

- Configured Trust between SAP Authorization and Trust Management service and SAP Cloud Identity Services
- A Destination service instance inside your subaccount
- An Identity service instance inside your subaccount

Set Up a Destination Service Instance for your Subaccount

To create a Destination service instance inside your subaccount, follow this documentation about creating service instances through the BTP cockpit or from the CF CLI: Creating Service Instances.

> **Note**
>
> When creating a Destination service instance, you can refer to the following *yaml* segment for the basic information of the instance.

Basic Information for the Destination Service instance

- Plan: *lite*
  # Currently, the Destination service offers only this plan.
- Runtime Environment: *Cloud Foundry*
  # You will need to have enabled Cloud Foundry for your subaccount.
- Space: <*space_name*>
  # Choose the space in which the Destination service instance will reside in.
- Instance Name: <*instance_name*>
  # Enter whatever name you want here for the instance.

Retrieving the Destination Service REST API Endpoint

The Destination service REST API endpoint can be retrieved from the credentials contained in a service key of the Destination Service instance. If you don't have any service keys in your Destination service instance, follow this BTP documentation about creating a service key for an instance from the BTP cockpit or through the CF CLI: Creating Service Keys .

Once you have a service key for your Destination service instance, you need to open it and extract the following information:

Information to Extract from the Service Key
• uri: *"<value_to_extract>"* # The URL of the Destination service

Set up a Cloud Identity Service Instance for your Subaccount

To create a Cloud Identity service instance inside your subaccount, follow this documentation about creating service instances through the BTP cockpit or from the CF CLI: Creating Service Instances.

> **Note**
>
> When creating a Cloud Identity service instance, you must provide the following basic information as well as the parameters in the *config.json* input file.

Basic Information for the Cloud Identity Service
• Plan: *application* # Currently, the Cloud Identity service offers only this plan. • Runtime Environment: *Cloud Foundry* # You must have enabled Cloud Foundry for your subaccount. • Space: *<space_name>* # Choose the space in which the Cloud Identity service instance will reside in. • Instance Name: *<instance_name>* # Enter any name for the instance.

config.json (Example)

> **Sample Code**

```
{
    "consumed-services":
    [
        {
            "service-instance-name": "<destination-service-instance-name>"
```

```
        }
    ]
}
```

where:

<*destination-service-instance-name*> is the value of `instance_name` from step Set Up a Destination Service Instance for your Subaccount [page 1575].

Getting the Credentials to Call the Destination Service REST API

To access the Destination service REST API, you need an access token. To generate such, you must get the credentials contained in a service key of the Cloud Identity service instance. If you don't have any service keys in your Cloud Identity service instance, follow this BTP documentation about creating a service key for an instance from the BTP cockpit or through the CF CLI: Creating Service Keys .

> ### Note
>
> The IAS access token is issued through mTLS, so when you are creating the service key in your Cloud Identity service, you must provide the following *config.json*:
>
> **X.509 Credential Type (Example)**
>
> #### Sample Code
>
> ```
> {
>     "credential-type": "X509_GENERATED"
> }
> ```

Once you have a service key for your Cloud Identity service instance, you need to open it and extract the following information:

Information to Extract from the Service Key
• clientid: *"<value_to_extract>"* # The client id which will be used for the authentication in the next step
• certificate: *"<value_to_extract>"* # The certificate which will be used for the authentication in the next step
• key: *"<value_to_extract>"* # The key which will be used for the authentication in the next step
• url: *"<value_to_extract>"* # The authentication endpoint from where an access token for the Cloud Identity service will be acquired

Acquire an Access Token from IAS to Access the Destination Service REST API

In this step, we will acquire an access token from IAS which we can then use to successfully authenticate towards the Destination service REST API. For this step, you must use the values you extracted for `clientid`, `certificate`, `key`, and `url` from the previous step.

Here is an example call using curl:

CURL Command to Acquire an Access Token for the Destination service

Sample Code

```
curl -X POST \
        "<url>/oauth2/token" \
        -H "Content-Type: application/x-www-form-urlencoded" \
        -d "grant_type=client_credentials" --data-urlencode
"client_id=<client_id>" --cert <certificate> --key <key>
```

where:

- <url> is the value of `url` from the previous step
- <client_id> is the value of `clientid` from the previous step
- <certificate> is the value of `certificate` from the previous step
- <key> is the value of the `key` from the previous step

The token which you will be using for the next step is provided under the `access_token` key in the response JSON. Make sure you save it because we will need it in the next step.

Call the Destination Service REST API

Now that you have an access token for the Destination service, you can finally call one of the Destination service REST API endpoints. To see the full list of available endpoints in the Destination service REST API and their responses, see Destination Service REST API reference.

Caution

Authentication to the Destination service via IAS token requires in addition mTLS.

Sample Code

```
curl --cert <IAS service instance certificate> --key <IAS service instance
private key> -X GET \
        "<uri>/destination-configuration/v1/<endpoint>" \
        -H "Authorization: Bearer <access_token>"
```

where:

- <uri> is the value of `uri` from Getting the Credentials to Call the Destination Service [page 1577]
- <endpoint> is the endpoint of the Destination service REST API which you want to call
- <access_token> is the access token you saved from Acquire an Access Token from IAS to Access the Destination Service REST API [page 1578]

For a more concrete example, if you want to make a GET call towards the */subaccountDestinations* endpoint, the call would look like this:

CURL Command for Calling the Destination Service

Sample Code

```
curl --cert cert.pem --key key.pem -X GET \
        "<uri>/destination-configuration/v1/subaccountDestinations" \
        -H "Authorization: Bearer <access_token>"
```

An example response from the Destination service would be:

Sample Code

```
[
    {
        "Name": "no-authentication-destination",
        "Type": "HTTP",
        "URL": "https://sap.com",
        "Authentication": "NoAuthentication",
        "ProxyType": "Internet"
    },
    {
        "Name": "basic-authentication-destination",
        "Type": "HTTP",
        "URL": "https://sap.com",
        "Authentication": "BasicAuthentication",
        "ProxyType": "Internet",
        "User": "my-user",
        "Password": "my-password"
    }
]
```

1.3.2.3 Exchanging User JWTs via OAuth2JWTBearer Destinations

Automatic token retrieval using the `OAuth2JWTBearer` authentication type for HTTP destinations.

Content

Prerequisites [page 1580]

Automated Access Token Retrieval [page 1580]

Scenarios [page 1580]

Prerequisites

You have configured an *OAuth2JWTBearer* destination. See OAuth JWT Bearer Authentication [page 1395].

Back to Content [page 1580]

Automated Access Token Retrieval

You can make use of the automated access token retrieval functionality, available via:

- The "Find a destination" [page 1497] endpoint.
  For information about the response structure, see "Find a Destination" Response Structure [page 1565].
- the "Consume a destination" [page 1569] endpoint.

The logic for controlling the token that gets exchanged is simple.

Using the X-user-token Header

If the `X-user-token` header is provided with the request to the Destination service and its value is not empty, it will be used with priority over the `Authorization` header for the token exchange operation. This means that the user token from this header will be used when determining the user and the tenant subdomain. It will also be the token for which token exchange is performed.

Using the Authorization Header

If `X-user-token header` is not provided with the request to the Destination service, or it is provided, but its value is empty, the token from the `Authorization` header will be used. The value of the header must be a user JWT (JSON Web token) in encoded form (see RFC 7519). Otherwise, token exchange will not work.

Back to Content [page 1580]

Scenarios

To achieve specific token exchange goals, you can use the following headers and values when calling the Destination service:

Goal	User Token Exchange Header	Authorization Header
Exchange a user token: • Issued on behalf of the **application provider tenant** • Using a destination in the **application provider tenant**	Not used	The user token to be exchanged, previously retrieved by the application via exchanging the initial user token, passed to the application (to another user access token) via the client credentials of the Destination service instance (bound to the application), using the provider tenant-specific token service URL.
Exchange a user token: • Issued on behalf of a tenant, **subscribed to your application** • Using a destination in the **application provider tenant**	<User token to be exchanged>	Access token retrieved via the client credentials of the Destination service instance (bound to the application), using the provider tenant-specific token service URL.
Exchange a user token: • Issued on behalf of a tenant, **subscribed to your application** • Using a destination in the **subscriber tenant**	<User token to be exchanged>	Access token retrieved via the client credentials of the Destination service instance (bound to the application), using the subscriber tenant-specific token service URL.

Back to Content [page 1580]

1.3.2.4 Multitenancy in the Destination Service

Establish multitenancy in the Destination service using subscription-level destinations.

Concept

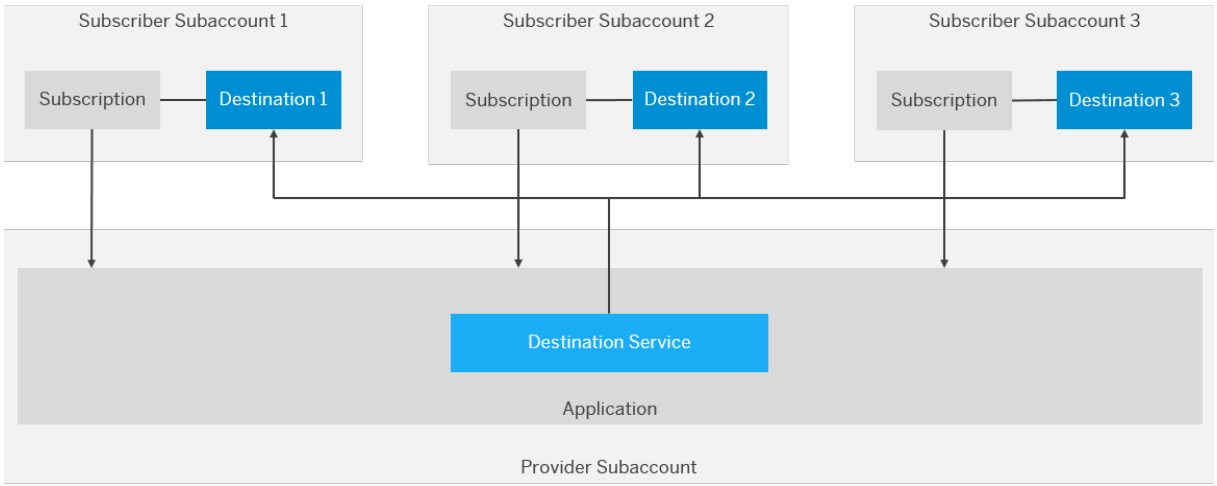
When developing a provider application (SaaS application) that consumes the Destination service, you can choose between the following destination levels:

Level	Who has Access and How?
`Subaccount`	*Any* application using *any* destination service instance in the subaccount context. This is the common level for all applications and service instances in the subaccount.

Level	Who has Access and How?
`Service Instance`	Any application using the concrete destination service instance in the context of the *provider* subaccount (the subaccount in which the instance is provisioned). Each service instance has its own level.
`Subscription`	*Any* application using the *concrete* destination service instance in the context of the *subscribed* subaccount. Each combination of service instance and subscriber subaccount is a unique level.

> ### Note
>
> The term *level* is used here to represent an area or visibility scope. The higher the level, the broader is the visibility scope.

If you, as an application provider, want to create a destination that is used at runtime *only by the subscriber* and that should be *visible and accessible only to the subscriber*, you can create a subscription-level destination for each subscriber subaccount (tenant):



Create a Subscription-Level Destination [page 1582]

Consume a Subscription-Level Destination [page 1583]

Create a Subscription-Level Destination

1. Retrieve an OAuth token from the subscriber token service URL using the OAuth client credentials from the destination service instance, for example:

   ```
   POST https://{subscriber subdomain}.authentication.{region host}/oauth/token
   ```

2. Use the retrieved token from step 1 to create (`POST`) a subscription-level destination in the Destination service, see *Destinations on service instance (subscription) level* in the REST API specification.

Back to Concept [page 1581]

Consume a Subscription-Level Destination

1. Retrieve an OAuth token from the subscriber token service URL using the OAuth client credentials from the destination service instance, for example:

   ```
   POST https://{subscriber subdomain}.authentication.{region host}/oauth/token
   ```

2. Use the token from step 1 to retrieve (`GET`) the destination stored on subscription level from the Destination service via:
   - *Find a destination* in the REST API specification
   - *Destinations on service instance (subscription) level* in the REST API specification

Back to Concept [page 1581]

Related Information

Multitenancy in SAP BTP Connectivity [page 18]

1.3.2.5 Destination Java APIs

Use Destination service Java APIs to optimize application development.

> ⚠ Caution
>
> The `ConnectivityConfiguration` and `AuthenticationHeaderProvider` APIs (the library described in this document) are not a recommended way of consuming the Destination service. If you, for some reason, have the need to use them, please open a support ticket on component BC-CP-DEST-CF to discuss your use case.
>
> For the Cloud Foundry environment, these APIs are in support-only mode, which means that no new development is done but bug fixes and security patches. They will not evolve with the service and no feature parity can be expected.
>
> For Kubernetes, we recommend that you use the Transparent Proxy as an alternative:
>
> - In the Kyma environment: Transparent Proxy in the Kyma Environment [page 2332]

- In any Kubernetes environment: Transparent Proxy for Kubernetes [page 2164]

For the Cloud Foundry environment, the recommendation is to use the SAP Cloud SDK instead.

When running your cloud application with *SAP Java Buildpack*, you can use the following Java APIs to optimize the application development:

- ConnectivityConfiguration API [page 1584]: Retrieve destination configurations.
- AuthenticationHeaderProvider API [page 1587]: Retrieve prepared authentication headers, ready to be used towards the remote target system.

Add the *Connectivity ApiExt* dependency in the `pom.xml` file:

```
<dependency>
    <groupId>com.sap.cloud.connectivity.apiext</groupId>
    <artifactId>com.sap.cloud.connectivity.apiext</artifactId>
    <version>${connectivity-apiext.version}</version>
    <scope>provided</scope>
</dependency>
```

For more information on *SAP Java Buildpack*, see also Developing Java in the Cloud Foundry Environment.

1.3.2.5.1 ConnectivityConfiguration API

Use the `ConnectivityConfiguration` API to retrieve destination configurations and certificate configurations.

Overview

The `ConnectivityConfiguration` API is visible by default from the web applications hosted on SAP Java Buildpack. You can access it via a JNDI lookup.

Besides managing destination configurations, you can also allow your applications to use their own managed HTTP clients. The `ConnectivityConfiguration` API provides you with direct access to the destination configurations of your applications.

To learn how to retrieve destination configurations, follow the procedure below.

Procedure

1. To consume a connectivity configuration using JNDI, you must define the `ConnectivityConfiguration` API as a resource in the *context.xml* file.

Example of a `ConnectivityConfiguration` resource named `connectivityConfiguration`, which is described in the *context.xml* file:
src/main/webapp/META-INF/context.xml

Sample Code

```
<?xml version='1.0' encoding='utf-8'?>

<Context>
    <Resource name="connectivityConfiguration" auth="Container"

type="com.sap.core.connectivity.api.configuration.ConnectivityConfiguration
"

factory="com.sap.core.connectivity.api.jndi.ServiceObjectFactory"/>
</Context>
```

2. You also need to enable Connectivity `ApiExt` with an environment variable and bind the appropriate services in `manifest.yml`:
manifest.yml

Sample Code

```
applications:
  - ...
    env:
      USE_CONNECTIVITY_APIEXT: true
    services:
      - xsuaa-instance
      - destination-instance
      - connectivity-instance
      ...
```

3. In your servlet code, you can look up the `ConnectivityConfiguration` API from the JNDI registry as follows:

Sample Code

```
import javax.naming.Context;
import javax.naming.InitialContext;
import
com.sap.core.connectivity.api.configuration.ConnectivityConfiguration;
...

// look up the connectivity configuration API "connectivityConfiguration"
Context ctx = new InitialContext();
ConnectivityConfiguration configuration = (ConnectivityConfiguration)
ctx.lookup("java:comp/env/connectivityConfiguration");
```

4. With the retrieved `ConnectivityConfiguration` API, you can read all properties of any destination defined on subscription, application, or subaccount level.

Sample Code

```
// get destination configuration for "myDestinationName"
DestinationConfiguration destConfiguration =
configuration.getConfiguration("myDestinationName");
```

```
// get a single destination property
String authenticationType =
destConfiguration.getProperty("Authentication");

// get all destination properties
Map<String, String> allDestinationPropeties =
destConfiguration.getAllProperties();
```

Note

If you have two destinations with the same name, for example, one configured on subaccount level and the other on service instance/subscription level, the `getConfiguration()` method will return the destination on instance/subscription level.

The preference order is:

1. Instance/subscription level
2. Subaccount level

5. If a trust store and a key store are defined in the corresponding destination, you can access them by using the methods `getKeyStore` and `getTrustStore`.

Sample Code

```
    // get destination configuration for "myDestinationName"
DestinationConfiguration destConfiguration =
configuration.getConfiguration("myDestinationName");

// get the configured keystore
KeyStore keyStore = destConfiguration.getKeyStore();

// get the configured truststore
KeyStore trustStore = destConfiguration.getTrustStore();

// create sslcontext
TrustManagerFactory tmf =
TrustManagerFactory.getInstance(TrustManagerFactory.getDefaultAlgorithm());
tmf.init(trustStore);

KeyManagerFactory keyManagerFactory =
KeyManagerFactory.getInstance(KeyManagerFactory.getDefaultAlgorithm());

// get key store password from destination
String keyStorePassword =
destConfiguration.getProperty("KeyStorePassword");
keyManagerFactory.init(keyStore, keyStorePassword.toCharArray());

SSLContext sslcontext = SSLContext.getInstance("TLSv1");
sslcontext.init(keyManagerFactory.getKeyManagers(),
tmf.getTrustManagers(), null);
SSLSocketFactory sslSocketFactory = sslcontext.getSocketFactory();

// get the destination URL
String value = destConfiguration.getProperty("URL");
URL url = new URL(value);

// use the sslcontext for url connection
URLConnection urlConnection = url.openConnection();
((HttpsURLConnection) urlConnection).setSSLSocketFactory(sslSocketFactory);
urlConnection.connect();
InputStream in = urlConnection.getInputStream();
...
```

1.3.2.5.2 AuthenticationHeaderProvider API

Use the `AuthenticationHeaderProvider` API for applications to retrieve prepared authentication headers that are ready to be used to towards a remote target system.

Overview

The `AuthenticationHeaderProvider` API is visible by default from the web applications hosted on *SAP Java Buildpack*. You can access it via a JNDI lookup.

This API lets your applications use their own managed HTTP clients, as it provides them with automated authentication token retrieval, making it easy to implement *single sign-on* (SSO) towards the remote target.

Procedure

1. To consume the authentication header provider API using JNDI, you need to define the `AuthenticationHeaderProvider` API as a resource in the *context.xml* file.
   Example of an `AuthenticationHeaderProvider` resource named `myAuthHeaderProvider`, which is described in the *context.xml* file:

   Sample Code

   ```
   <?xml version='1.0' encoding='utf-8'?>

   <Context>
       <Resource name="myAuthHeaderProvider" auth="Container"

   type="com.sap.core.connectivity.api.authentication.AuthenticationHeaderProv
   ider"

   factory="com.sap.core.connectivity.api.jndi.ServiceObjectFactory"/>
   </Context>
   ```

2. In your servlet code, you can look up the `AuthenticationHeaderProvider` API from the JNDI registry as follows:

   Sample Code

   ```
   import javax.naming.Context;
   import javax.naming.InitialContext;
   import
   com.sap.core.connectivity.api.authentication.AuthenticationHeaderProvider;
   ...
   ```

```
// look up the connectivity authentication header provider resource called
"myAuthHeaderProvider"
Context ctx = new InitialContext();
AuthenticationHeaderProvider authHeaderProvider
= (AuthenticationHeaderProvider) ctx.lookup("java:comp/env/
myAuthHeaderProvider");
```

Single Sign-On to On-Premise Systems

The Connectivity service supports a mechanism to enable SSO using the so-called *principal propagation* authentication type of a destination configuration. To propagate the logged-in user, the application can use the `AuthenticationHeaderProvider` API to retrieve a prepared HTTP header, which then embeds in the HTTP request to the on-premise system.

Prerequisites

To connect to on-premise systems and perform single sign-on, you must bind a Connectivity service instance to the cloud application.

References

For more information, see also:

- Principal Propagation SSO Authentication for HTTP [page 1359]
- Create and Bind a Connectivity Service Instance [page 1309]
- Consuming the Connectivity Service (Java) (Neo environment)

Example

Sample Code

```
String proxyHost = <connectivity_service_credentials_onPremiseProxyHost>;
int proxyPort =
Integer.parseInt(<connectivity_service_credentials_onPremiseProxyPort>);
String account = SecurityContext.getAccessToken().getZoneId();

// setup the on-premise HTTP proxy
HttpClient httpClient = new DefaultHttpClient();
httpClient.getParams().setParameter(ConnRoutePNames.DEFAULT_PROXY, new
HttpHost(proxyHost, proxyPort));

// look up the connectivity authentication header provider resource called
"authHeaderProvider" (must be defined in web.xml)
Context ctx = new InitialContext();
AuthenticationHeaderProvider authHeaderProvider =
(AuthenticationHeaderProvider) ctx.lookup("java:comp/env/authHeaderProvider");

// get header for principal propagation
AuthenticationHeader principalPropagationHeader =
authHeaderProvider.getPrincipalPropagationHeader();

//insert the necessary headers in the request
HttpGet request = new HttpGet("http://virtualhost:1234");
```

```
request.addHeader(principalPropagationHeader.getName(),
principalPropagationHeader.getValue());

// execute the request
HttpResponse response = httpClient.execute(request);
```

OAuth2 SAML Bearer Assertion

The Destination service provides support for applications to use the SAML Bearer assertion flow for consuming OAuth-protected resources. In this way, applications do not need to deal with some of the complexities of OAuth and can reuse user data from existing identity providers.

Users are authenticated by using a SAML assertion against the configured and trusted OAuth token service. The SAML assertion is then used to request an access token from an OAuth token service. This access token is returned by the API and can be injected in the HTTP request to access the remote OAuth-protected resources via SSO.

> → Tip
>
> The access tokens are cached by `AuthenticationHeaderProvider` and are auto-updated: When a token is about to expire, a new token is created shortly before the expiration of the old one.

The `AuthenticationHeaderProvider` API provides the following method to retrieve such headers:

```
List<AuthenticationHeader>
getOAuth2SAMLBearerAssertionHeaders(DestinationConfiguration
destinationConfiguration);
```

For more information, see also Principal Propagation SSO Authentication for HTTP [page 1359].

Client Credentials

The Destination service provides support for applications to use the OAuth client credentials flow for consuming OAuth-protected resources.

The client credentials are used to request an access token from an OAuth token service.

> → Tip
>
> The access tokens are cached by `AuthenticationHeaderProvider` and are auto-updated: When a token is about to expire, a new token is created shortly before the expiration of the old one.

The `AuthenticationHeaderProvider` API provides the following method to retrieve such headers:

```
AuthenticationHeader getOAuth2ClientCredentialsHeader (DestinationConfiguration
destinationConfiguration);
```

For more information, see:

- Principal Propagation SSO Authentication for HTTP [page 1359]
- OAuth SAML Bearer Assertion Authentication [page 1363]
- OAuth Client Credentials Authentication [page 1375]

Related Information

Principal Propagation [page 30]

1.3.2.6 Extending Destinations with Fragments

Use the “Find Destination” API to extend your destination with one or more destination fragments.

Overview

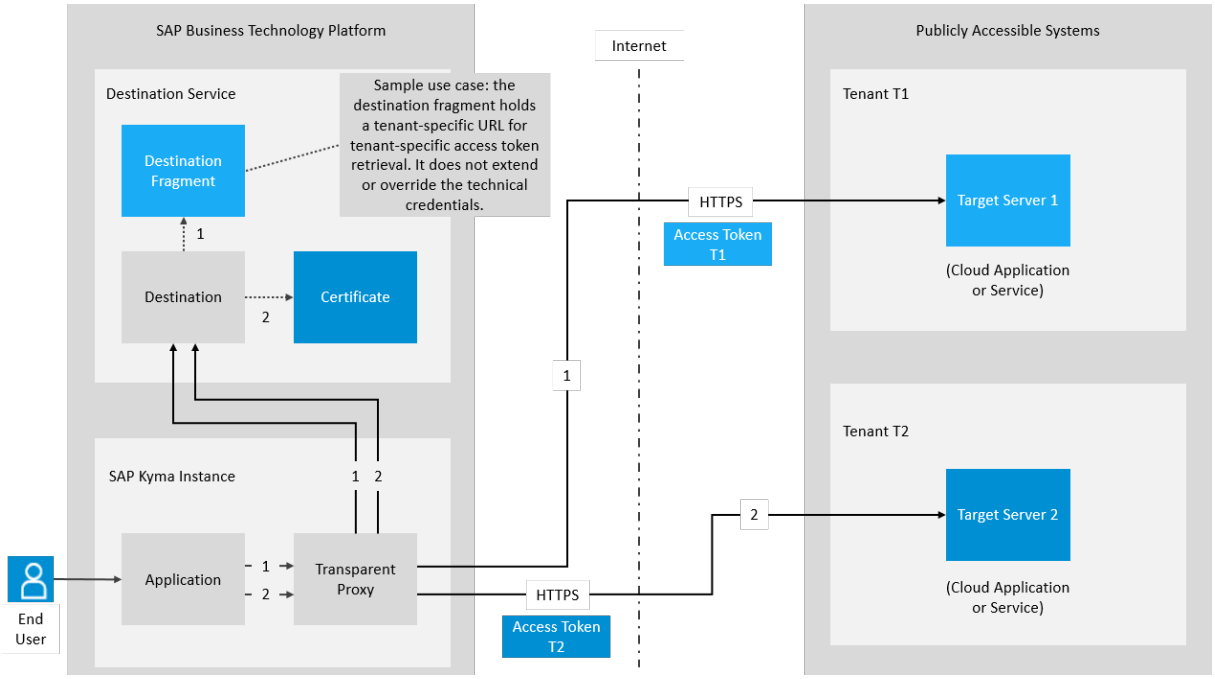
Destination fragments let you manage technical connection configurations (also known as destinations) in a more flexible and dynamic way compared to the regular usage of destinations. You can use the destination fragment to extend an existing destination with additional properties, and even to override some of the existing destination properties. This feature enables a variety of technical scenarios which solve higher-level business scenario needs.

For more information, see also Manage Destination Fragments [page 1503].

Sample Use Cases

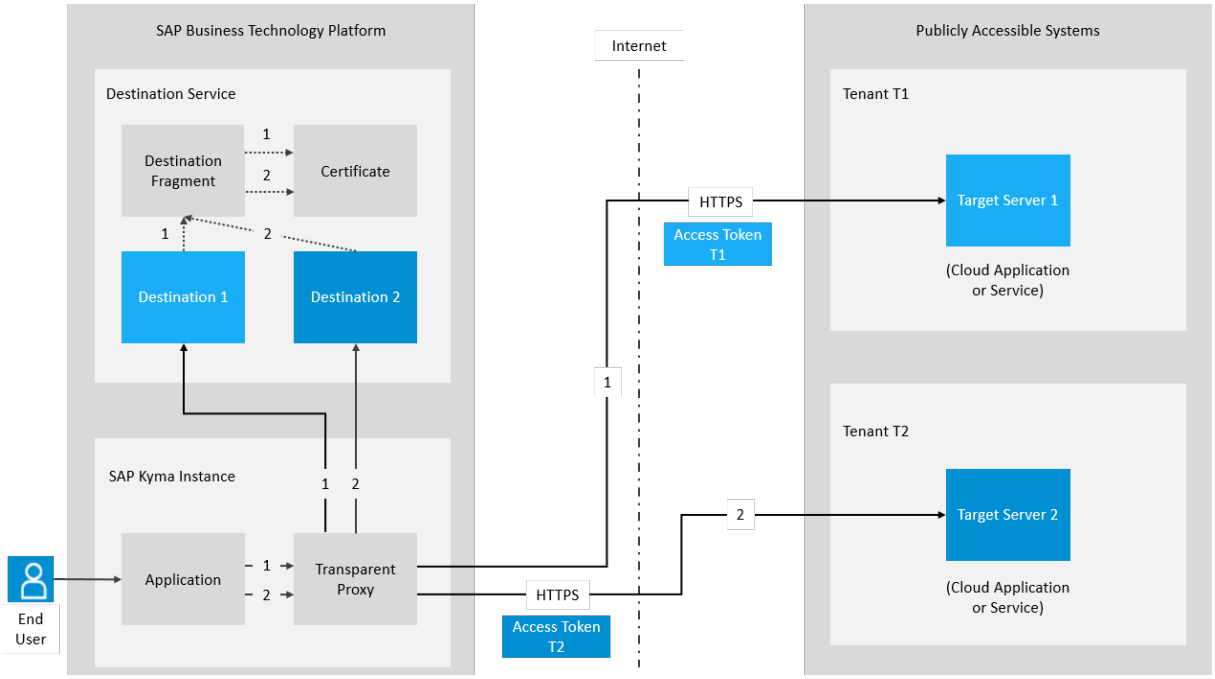
Find below some common use cases for destination fragments:

Reuse of technical credentials for multiple users or tenants



Option 1: Single destination, multiple destination fragments

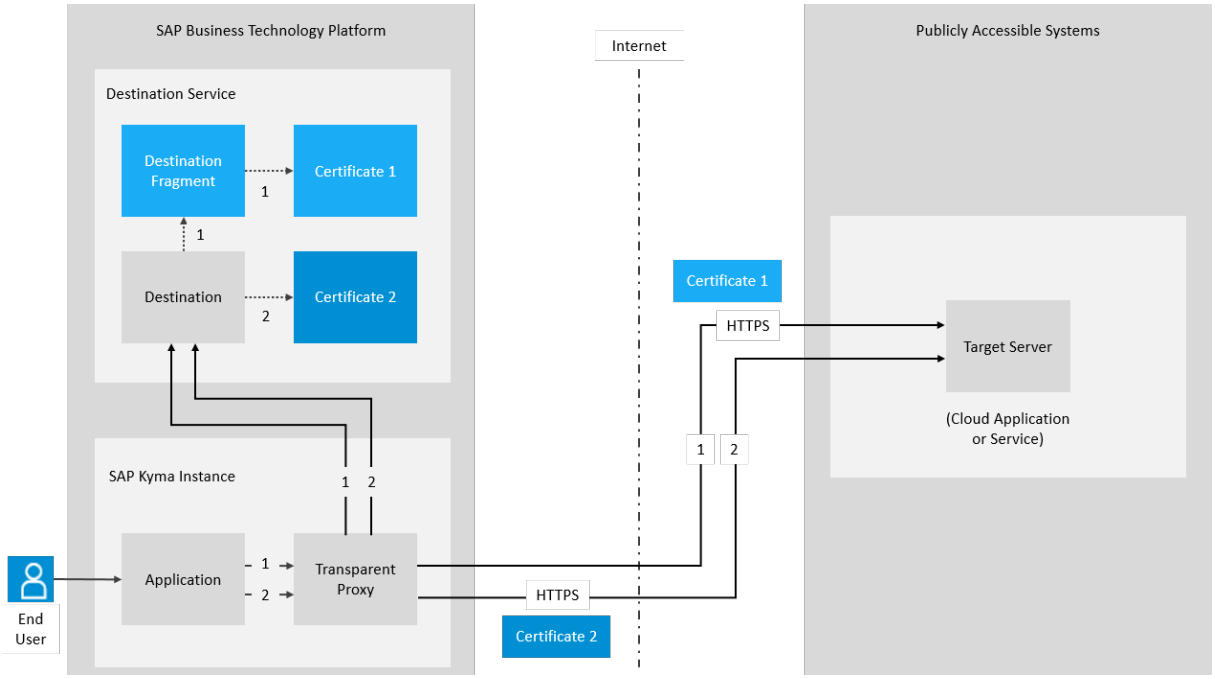
- In a regular destination, the administrator stores default properties like technical credentials, for example, *OAuth client* or *X.509 client certificate*.
- In a destination fragment, the administrator stores user or tenant-specific additional properties, for example, a URL with a specific path, a query parameter, and so on.
- At runtime, the application resolves the final destination using the regular destination lookup and in addition specifying the user or tenant-specific destination fragment via the optional HTTP header. For more information, see section *Use a Header to Specify the Destination Fragment* below.
- As a result, the application receives a user or tenant-specific destination, reusing the same technical credentials, this way optimising the technical connection configuration management, for example, without the need to duplicate technical credentials among multiple destinations. The latter reduces the risk of business interruptions during operations like credentials rotation.



Option 2: Multiple destinations, a single destination fragment

- In a each tenant-specific destination, the administrator stores properties like *URL* with a specific path, query parameter, and so on.
- In a single destination fragment, the administrator stores default properties like technical credentials, for example, *OAuth client* or *X.509 client certificate*.
- At runtime, the application resolves the final destination using the regular destination lookup and, optionally, if not already configured in the destination itself, in addition specifying the user or tenant-specific destination fragment via the optional HTTP header.
  For more information, see section *Use a Header to Specify the Destination Fragment* below.
- As a result, the application receives a user or tenant-specific destination, reusing the same technical credentials, this way optimising the technical connection configuration management, for example, without the need to duplicate technical credentials among multiple destinations. The latter reduces the risk of business interruptions during operations like credentials rotation.

Reuse of routing properties for multiple users or tenants



- In a regular destination, the administrator stores default properties like *URL*, *Authentication*, and so on.
- In a destination fragment, the administrator stores user or tenant-specific additional properties, for example, a tenant-specific URL path, an X.509 client certificate, and so on.
- At runtime, the application resolves the final destination using the regular destination lookup and in addition specifying the user or tenant-specific destination fragment via the optional HTTP header. For more information, see section *Use a Header to Specify the Destination Fragment* below.
- As a result, the application receives a destination, constructed by re-using the base destination properties but with a user or tenant-specific URL path and technical credentials, this way optimising the technical connection configuration management, for example, without the need to duplicate the base routing properties among multiple destinations. The latter reduces the risk of business interruptions during operations like the change of common or base destination properties.

How it Works

This extension involves merging the JSON object of the destination properties, together with the JSON object of the fragment properties.

> ### Note
>
> If one or more fragments were found and applied to the requested destination, the resulting destination properties will include `FragmentName`, containing a comma-separated list of all applied fragments.
>
> If any fragment property uses the same key name as a destination property, the combined JSON object will use the value of the fragment property.

The combined JSON of the destination and fragment will be returned in the response body as the value of the “destinationConfiguration” property. Additionally, this combined JSON will be used for retrieving tokens from authorization servers, if applicable.

> ### Note
>
> The combination of the destination JSON and the fragment JSON happens in the context of the “Find Destination” request. The actual destination stored in Destination service is not modified.

To apply this mechanism, you must use the following configurations:

- A destination – with any properties
- One or more destination fragments – with any properties

> ### Note
>
> If more than one fragment is used to extend the destination, the destination JSON will be merged with each fragment in the order of how the fragments are listed in the `X-Fragment-Name` request header.

> ### Note
>
> For more information on how to create and manage resources like destinations and destination fragments, see SAP Business Accelerator Hub.

Use a Header to Specify the Destination Fragment

Header	Value	Description
X-Fragment-Name	Names of the destination fragments	The names of the destination fragment as a comma-separated list. The fragments themselves must be maintained on the same level as the destination.
X-Fragment-Optional	true/false	To mark a destination fragment provided with the `X-Fragment-Name` as optional, you can use the header `X-Fragment-Optional`. If the fragment is marked as optional and it does not exist, the destination with its original properties will be returned.

> ### Caution
>
> Only one `X-Fragment-Name` header may be present in the “Find Destination” request.

> ⚠ Caution
>
> If multiple fragments are listed in the `X-Fragment-Name` request header, the `X-Fragment-Optional` header will be disabled and should not be included in the request. All of the specified fragments are expected to be maintained on the same level as the destination. If any fragment is not found, the request will fail.

Example: Using Destination Fragments to Add New Properties to a Destination

Destination:

> {·} Sample Code

```
Name=example-destination
Type=HTTP
URL=https://xxxx.example.com
ProxyType=Internet
Authentication=NoAuthentication
```

Destination Fragment:

> {·} Sample Code

```
FragmentName=example-fragment
example-property=example-value
```

cURL call to send “Find Destination” request:

> {·} Sample Code

```
curl --request GET --url 'https://<destination_service_host>/destination-
configuration/v1/destinations/example-destination' \
  --header 'Accept: */*' \
  --header 'Authorization: Bearer <access_token>' \
  --header 'X-Fragment-Name: example-fragment'
```

Response:

> {·} Sample Code

```
{
  "owner": {
    "SubaccountId": "<subaccount_id>",
```

```
    "InstanceId": null
  },
  "destinationConfiguration": {
    "Name": "example-destination",
    "Type": "HTTP",
    "URL": "https://xxxx.example.com",
    "Authentication": "NoAuthentication",
    "ProxyType": "Internet",
    "FragmentName": "example-fragment",
    "example-property": "example-value"
  }
}
```

Example: Using Destination Fragments to Override Properties of a Destination

Destination:

Sample Code

```
Name=example-destination
Type=HTTP
URL=https://xxxx.example.com
ProxyType=Internet
Authentication=OAuth2ClientCredentials
clientId=clientId
clientSecret=secret1234
tokenServiceURL=https://authserver1.example.com/oauth/token/
```

Destination Fragment:

Sample Code

```
FragmentName=example-fragment
clientId=clientId-2
clientSecret=secret2345
tokenServiceURL=https://authserver2.example.com/oauth/token/
```

cURL call to send “Find Destination” request:

Sample Code

```
curl --request GET \
  --url 'https://<destination_service_host>/destination-configuration/v1/
destinations/example-destination' \
  --header 'Accept: */*' \
  --header 'Authorization: Bearer <access_token>' \
  --header 'X-Fragment-Name: example-fragment'
```

Response:

Sample Code

```
{
  "owner": {
    "SubaccountId": "<subaccount_id>",
    "InstanceId": null
  },
  "destinationConfiguration": {
    "Name": "example-destination",
    "Type": "HTTP",
    "URL": "https://xxxx.example.com",
    "Authentication": "OAuth2ClientCredentials",
    "ProxyType": "Internet",
    "FragmentName": "example-fragment",
    "clientId": "clientId-2",
    "clientSecret": "secret2345",
    "tokenServiceURL": "https://authserver2.example.com/oauth/token/"
  },
  "authTokens": [
    {
      "type": "Bearer",
      "value": "eyJhbGciOiJSUzI1NiIsInR5cC...",
      "http_header": {
        "key": "Authorization",
        "value": "Bearer eyJhbGciOiJSUzI1NiIsInR5cC..."
      }
    }
  ]
}
```

Example: Using Multiple Destination Fragments to Add New Properties to a Destination

Destination:

Sample Code

```
Name=example-destination
Type=HTTP
URL=https://xxxx.example.com
ProxyType=Internet
Authentication=NoAuthentication
```

Destination Fragments:

Sample Code

```
FragmentName=example-fragment-1
example-property-1=example-value-1
```

```
conflicting-property=conflicting-value-1
FragmentName=example-fragment-2
example-property-2=example-value-2
conflicting-property=conflicting-value-2
```

cURL call to send “Find Destination” request:

Sample Code

```
curl --request GET --url 'https://<destination_service_host>/destination-
configuration/v1/destinations/example-destination' \
  --header 'Accept: */*' \
  --header 'Authorization: Bearer <access_token>' \
  --header 'X-Fragment-Name: example-fragment-1,example-fragment-2'
```

Response:

Sample Code

```
{
  "owner": {
    "SubaccountId": "<subaccount_id>",
    "InstanceId": null
  },
  "destinationConfiguration": {
    "Name": "example-destination",
    "Type": "HTTP",
    "URL": "https://xxxx.example.com",
    "Authentication": "NoAuthentication",
    "ProxyType": "Internet",
    "FragmentName": "example-fragment-1,example-fragment-2",
    "example-property-1": "example-value-1",
    "example-property-2": "example-value-2",
    "conflicting-property": "conflicting-value-2",
  }
}
```

Related Information

Calling the Destination Service REST API [page 1562]
"Find a Destination" Response Structure [page 1565]

1.3.2.7 Using an IAS User Token for Corporate IdP Token Principal Propagation to On-Premise Systems

Using an IAS user token for corporate IdP token principal propagation to on-premise systems allows automatic token exchange to the corporate IdP token.

Prerequisites

- You have a tenant of SAP Cloud Identity Services.
  For more information, see Get Your Tenant (*SAP Cloud Identity Services* documentation).
- You have configured a destination with proxy type `OnPremise` and authentication type `PrincipalPropagation`.

Scenario

When consuming a destination with principal propagation to an on-premise system using IAS user token that contains the `forward_corp_idp_token` claim, an automatic token exchange will be performed to the corporate IdP token on your behalf.

Depending on the value of the `forward_corp_idp_token` claim, the response in the *Authentication Tokens* section of "Find a Destination" Response Structure [page 1567] will be either an `access_token` or `id_token`. After that, the result may be passed to the Connectivity service.

For more information, see Configure Principal Propagation via OIDC Token [page 1333].

For detailed information on how to consume the Destination service using an IAS token, see Calling the Destination Service REST API via IAS Token [page 1575].

> ⚠ Caution
>
> Possible values of the `forward_corp_idp_token` user attribute are: `access_token` or `id_token`, and the IAS tenant must be additionally configured, so that this claim will be part of the IAS user token.
>
> For more information, see Configuring User Attributes from a Corporate Identity Provider.

1.3.2.8 Resolve Dynamic Token-Based Property Values

Learn how to resolve dynamic values in the additional properties of a destination from token claims.

This feature allows to resolve additional properties of a destination at retrieval time from the claims of the calling user's JWT. It lets you keep a single destination configuration whose additional properties adapt to the

user making the request - for example, forwarding the caller's user name, zone ID or a nested identity attribute to the target system.

> **Note**
>
> Resolution is offered **only for additional properties**. The core destination properties are not modified, even if they contain placeholder-like text.

Prerequisites

To use dynamic token-based property values, you need a configured destination with at least one additional property containing a placeholder.

Placeholder Syntax

A placeholder has the form:

`{<jsonpath>}`

where `<jsonpath>` is a `JsonPath` expression starting with `$`, evaluated against the JWT claims of the request. Bracket notation is used for claim names - the same convention as `userIdSource` - which also handles claim names that contain dots:

Placeholder	Resolves To
`{$.['user_name']}`	top-level `user_name` claim
`{$.['ext_attr']['subaccountid']}`	nested `ext_attr.subaccountid` claim
`{$.['xs.system.attributes']['xs.saml.groups']}`	nested claim whose keys contain dots

A value may contain static text and more than one placeholder:

```
"usernamePrefix": "prefix-{$.['user_name']}"
```

```
"id": "{$.['user_name']}@{$.['zid']}"
```

```
"grant_type": "urn:{$.['grant_type']}:token"
```

Which Token is Used

The claims are taken from the request headers in this order:

1. `X-user-token` - if present, its claims are used.
2. `Authorization` (Bearer) - used only when no `X-user-token` header is present.

Resolution Rules

- Known scalar claim (string, number, or boolean) - the placeholder is replaced with the claim value.
- Unknown claim - the placeholder is left literal (unchanged); the request does not fail.
- Malformed placeholder / invalid `JsonPath` - left literal; the request does not fail.
- Non-scalar claim - the placeholder is left literal (unchanged); only scalar claims are resolved.

Calling the "Find Destination" Endpoint

Path: `/destination-configuration/v1/destinations/<destination-name>`

Example of a Call (cURL)

Sample Code

```
curl "<uri>/destination-configuration/v1/destinations/<destination-name>" \
  -X GET \
  -H "Authorization: Bearer <jwtToken>" \
  -H "X-user-token: <userJwtToken>"
```

Example of a Configured Destination

Sample Code

```
 {
  "Name": "demo-destination",
  "URL": "https://example.com",
  "Type": "HTTP",
  "ProxyType": "Internet",
  "Authentication": "NoAuthentication",
  "customProp": "user-{$.['user_name']}"
}
```

Example of a Response

The `X-user-token` carries a JWT with claim `user_name = testUser`. The additional property `customProp` is resolved; core properties remain untouched:

Sample Code

```
{
  "owner": {
    "SubaccountId": "<id>",
    "InstanceId": null
  },
  "destinationConfiguration": {
    "Name": "demo-destination",
    "URL": "https://example.com",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "Authentication": "NoAuthentication",
    "customProp": "user-testUser"
  }
}
```

Calling the "Consume API Destination" Endpoint

Path: `/destination-configuration/v2/destinations/<destination-name>@subaccount`

The *Consume API* endpoint supports the same placeholder resolution behavior as the *Find Destination* endpoint. Provide an `X-user-token header` to resolve placeholders from a specific user's claims. Resolution rules, token priority and failure handling are identical between both endpoints.

If the token had no `user_name` claim, `customProp` would be returned as `'user-{$.['user_name']}'`.

1.4 Cloud Connector

Learn more about the Cloud Connector: features, scenarios and setup.

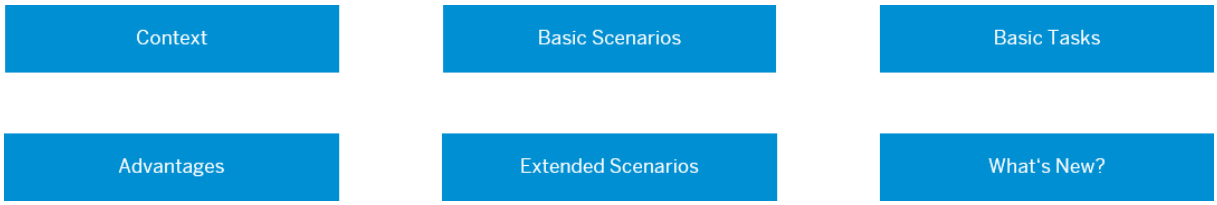
Note

This documentation refers to SAP BTP, multi-cloud foundation. If you are looking for information about the Neo environment, see Connectivity for the Neo Environment.

Content

In this Topic

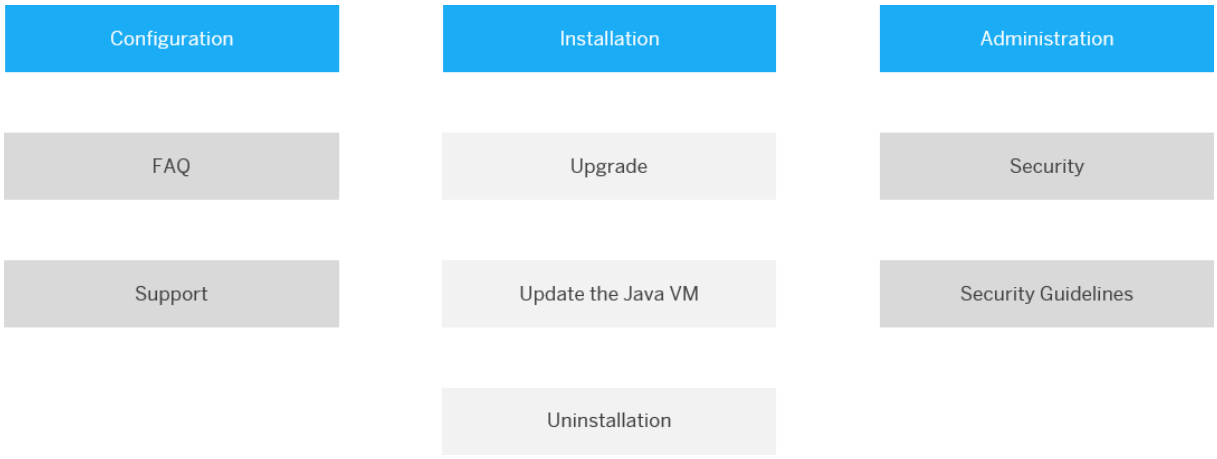
Hover over the elements for a description. Click an element for more information.



- Context [page 1604]
- Basic Scenarios [page 1605]
- Basic Tasks [page 1607]
- Advantages [page 1604]
- Extended Scenarios [page 1607]
- What's New? [page 1608]

In this Guide

Hover over the elements for a description. Click an element for more information.



- Configuration [page 1669]
- Installation [page 1608]
- Administration [page 1970]
- Frequently Asked Questions [page 2073]
- Upgrade [page 2069]
- Security [page 2057]
- Update the Java VM [page 2070]
- Uninstallation [page 2072]
- Connectivity Support [page 2362]
- Security Guidelines [page 2065]

Context

The Cloud Connector:

- Serves as a link between SAP BTP applications and on-premises systems.
  - Combines an easy setup with a clear configuration of the systems that are exposed to the SAP BTP.
  - Lets you use existing on-premises assets without exposing the entire internal landscape.
- Runs as on-premises agent in a secured network.
  - Acts as a reverse invoke proxy between the on-premises network and SAP BTP.
- Provides fine-grained control over:
  - On-premises systems and resources that can be accessed by cloud applications.
  - Cloud applications using the Cloud Connector.
- Lets you use the features that are required for business-critical enterprise scenarios.
  - Recovers broken connections automatically.
  - Provides audit logging of inbound traffic and configuration changes.
  - Can be run in a high-availability setup.

Back to Content [page 1602]

Advantages

Compared to the approach of opening ports in the firewall and using reverse proxies in the DMZ to establish access to on-premises systems, the Cloud Connector offers the following benefits:

- You don't need to configure the on-premises firewall to allow external access from SAP BTP to internal systems. For allowed outbound connections, no modifications are required.
- The Cloud Connector supports HTTP as well as additional protocols. For example, the RFC protocol supports native access to ABAP systems by invoking function modules.
- You can use the Cloud Connector to connect on-premises databases or BI tools to SAP HANA databases in the cloud.
- The Cloud Connector lets you propagate the identity of cloud users to on-premises systems in a secure way.
- Easy installation and configuration, which means that the Cloud Connector comes with a low TCO and is tailored to fit your cloud scenarios.
- SAP provides standard support for the Cloud Connector.

Back to Content [page 1602]

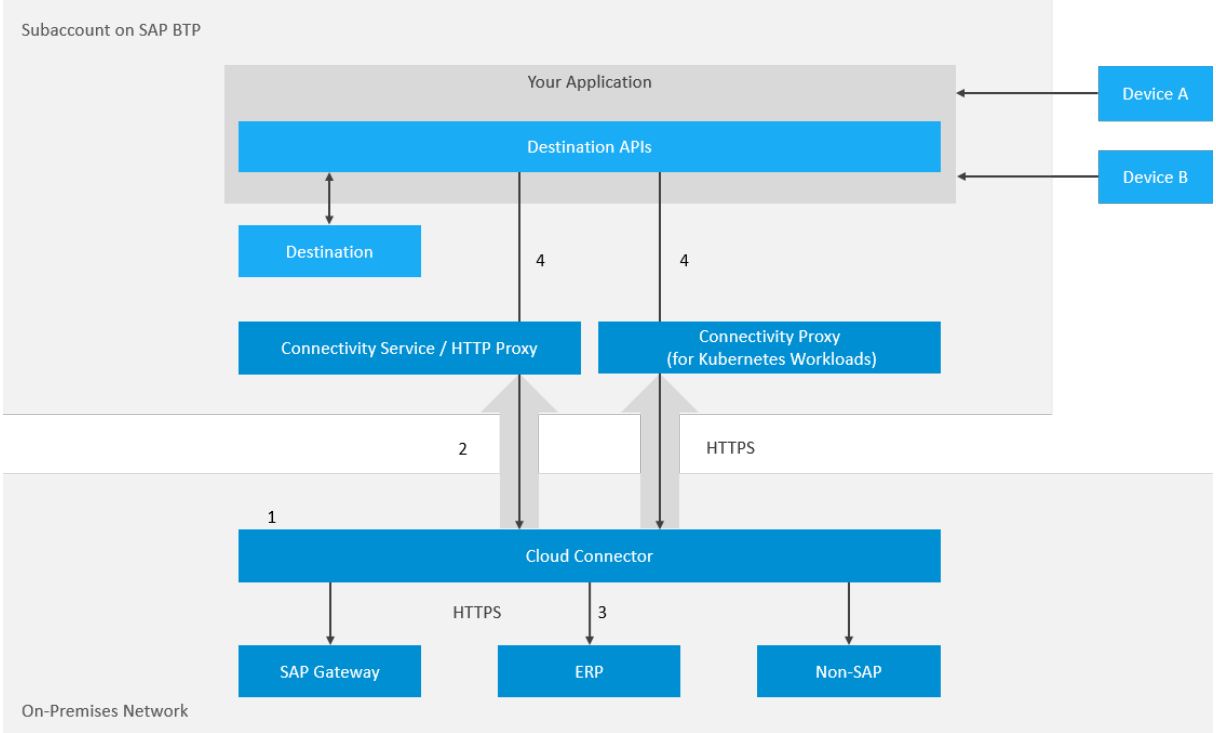
Basic Scenarios

Connecting Cloud Applications to On-Premises Systems [page 1605]

Connecting On-Premises Database Tools to SAP HANA Databases [page 1605]

> **Note**
>
> This section refers to the Cloud Connector installation in a standard on-premises network. Find setup options for other system environments in Extended Scenarios [page 1607].

Connecting Cloud Applications to On-Premises Systems



1. Install the Cloud Connector: Installation [page 1608]
2. Set up the connection between Cloud Connector, backend system and your SAP BTP subaccount : Initial Configuration [page 1670], Adding and Managing Subaccounts [page 1682]
3. Allow your cloud application to access a backend system on the intranet: Configure Access Control [page 1740]
4. Connect your cloud application to an on-premises system:
   Consuming the Connectivity Service [page 1314] (Cloud Foundry environment)
   Connectivity Proxy for Kubernetes [page 2088] (Kubernetes workloads / Kyma environment)

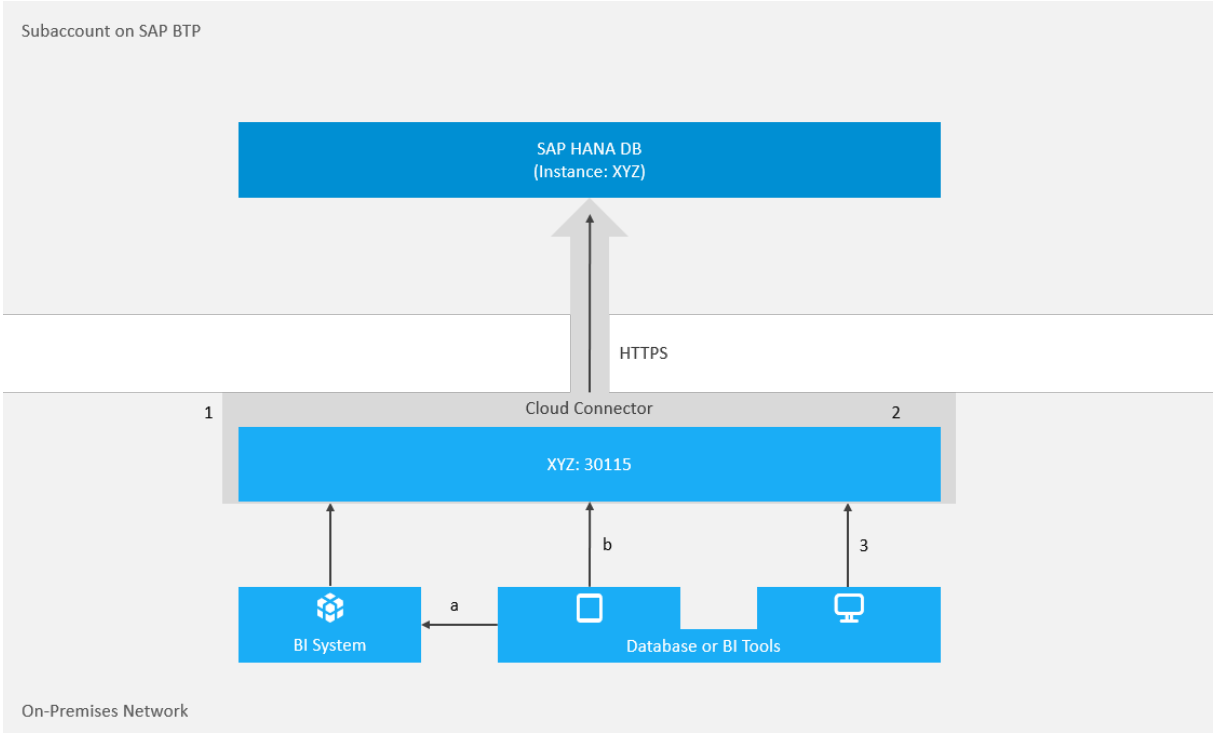
Back to Basic Scenarios [page 1605]

Back to Content [page 1602]

Connecting On-Premises Database Tools to SAP HANA Databases

⚠ Restriction

This scenario is only available for the Neo environment.



1. Install and configure the Cloud Connector: Installation, Initial Configuration, Adding and Managing Subaccounts
2. Access HANA databases on SAP BTP: Configure a Service Channel for an SAP HANA Database
3. Connect on-premises database or BI tools to a HANA database on SAP BTP: Connect DB Tools to SAP HANA via Service Channels

ⓘ Note

You can use service channels also for other purposes:

- Connect to a virtual machine on SAP BTP (**Neo** environment only).
- Configure an RFC connection from your on-premises system to S/4HANA Cloud (**Neo** and **multi-cloud** environment).
- Connect to a service in a Kubernetes cluster (**multi-cloud** environment only)

See also:

Using Service Channels [page 1949] (**multi-cloud** environment)

Using Service Channels (**Neo** environment)

Back to Basic Scenarios [page 1605]

Back to Content [page 1602]

Extended Scenarios

Besides the standard setup: *SAP BTP - Cloud Connector - on-premises system/network*, you can also use the Cloud Connector to connect SAP BTP applications to other cloud-based environments, as long as they are operated in a way that is comparable to an on-premises network from a functional perspective. This is particularly true for infrastructure (IaaS) hosting solutions.

Here's an overview of all environments in which you can or cannot set up the Cloud Connector:

Cloud Connector	Environment	Examples
Can be set up in: **Note** Within extended scenarios that allow a Cloud Connector setup, special procedures may apply for configuration. If so, they are mentioned in the corresponding configuration steps.	Customer on-premises network (see Basic Scenarios [page 1605])	SAP ERP, SAP S/4HANA
	SAP Hosting	SAP HANA Enterprise Cloud (HEC)
	Third-party **IaaS** providers (hosting)	Amazon Web Services (AWS), Microsoft Azure, Google Cloud
Cannot be set up in:	SAP **SaaS** solutions	SAP SuccessFactors, SAP Concur, SAP Ariba
	SAP cloud-based enterprise solutions	SAP S/4HANA Cloud (public edition), SAP C/4HANA
	Third-party **PaaS** providers	AWS, Azure, Google Cloud
	Third-party **SaaS** providers	

Back to Content [page 1602]

Basic Tasks

The following steps are required to connect the Cloud Connector to your SAP BTP subaccount:

- Install the Cloud Connector: Installation [page 1608]
- Perform the initial configuration for the Cloud Connector: Initial Configuration [page 1670]
- Register the Cloud Connector for your SAP BTP subaccount: Adding and Managing Subaccounts [page 1682]

Back to Content [page 1602]

What's New?

Follow the SAP BTP Release Notes to stay informed about Cloud Connector and Connectivity updates.

Back to Content [page 1602]

Related Information

Installation [page 1608]
Configuration [page 1669]
Administration [page 1970]
Security [page 2057]
Upgrade [page 2069]
Update the Java VM [page 2070]
Uninstallation [page 2072]
Frequently Asked Questions [page 2073]
REST APIs [page 2084]

1.4.1 Installation

Choose a procedure to install the Cloud Connector on your operating system.

Portable Version vs. Installer Version

On **Microsoft Windows** and **Linux**, two installation variants are available: a `portable` version and an `installer` version. On **Apple macOS**, only the `portable` version is available.

- `Portable` version: can be installed easily, by extracting a compressed archive into an empty directory. It does not require administrator or root privileges for the installation, and you can run multiple instances on the same host.
  Restrictions:
  - You cannot run it in the background as a Windows Service or Linux daemon (with automatic start capabilities at boot time).
  - The portable version does not support an automatic upgrade procedure. To update a portable installation, you must delete the current one, extract the new version, and then re-do the configuration.
  - Portable versions are meant for non-productive scenarios only.
  - The environment variable `JAVA_HOME` is relevant when starting the instance, and therefore must be set properly.

- `Installer` version: requires administrator or root permissions for the installation and can be set up to run as a Windows service or Linux daemon in the background. You can upgrade it easily, retaining all the configuration and customizing.

  > ### Note
  >
  > We strongly recommend that you use this variant for a productive setup.

> ### Caution
>
> Cloud Connector is based on Tomcat.
>
> Tomcat is a well-known and well-documented server, whose configuration can be adapted to one's own needs. However, we strongly recommend that you **do not modify** its configuration files like `conf\server.xml` with a text editor, because the Cloud Connector is making assumptions about the content of those files.
>
> If you still want to do custom modifications, you do so at your own risk. In this case, we can no longer guarantee that the Cloud Connector keeps working as expected. For any issues that can be traced back to such changes, we cannot provide support, in particular, if such changes cause trouble during upgrade to a newer version.

Prerequisites

- There are some general prerequisites you must fulfill to successfully install the Cloud Connector, see Prerequisites [page 1610].
- For OS-specific requirements and procedures, see section **Tasks** below.

Tasks

- Installation on Microsoft Windows OS [page 1657]
- Installation on Linux OS [page 1660]
- Installation on Apple macOS [page 1663]

Related Information

Sizing Recommendations [page 1649]
Recommendations for Secure Setup [page 1665]
Uninstallation [page 2072]

1.4.1.1 Prerequisites

Prerequisites for successful installation of the Cloud Connector.

Content

Section	Description
Connectivity Restrictions [page 1610]	General information about SAP BTP and connectivity restrictions.
Hardware [page 1610]	Hardware prerequisites for a physical or virtual machine.
Software [page 1611]	Required software download and installation.
JDKs [page 1611]	Java Development Kit (JDK) versions that you can use.
Product Availability Matrix [page 1612]	Availability of operating systems/versions for specific Cloud Connector versions.
Network [page 1613]	Required Internet connection to SAP BTP hosts per region.

> **Note**
>
> For additional system requirements, see also System Requirements [page 1647].

Connectivity Restrictions

For general information about SAP BTP restrictions, see Prerequisites and Restrictions.

For specific information about all Connectivity restrictions, see Connectivity: Restrictions [page 9].

Back to Content [page 1610]

Hardware

Hardware prerequisites, physical or virtual machine:

	Minimum	Recommended
CPU	Dual core 2 GHz, *x86-64*, *aarch64*, or *ppc64le* architecture-compatible	Multi core 3 GHz, *x86-64*, *aarch64*, or *ppc64le* architecture-compatible

	Minimum	Recommended
Memory (RAM)	4 GB	16 GB
Free disk space	2 GB	20 GB

Back to Content [page 1610]

Software

- You have downloaded the Cloud Connector installation archive from SAP Development Tools for Eclipse.
- A full JDK must be installed. Lightweight JRE installations are not sufficient. You can download a fitting up-to-date SAP JVM from SAP Development Tools for Eclipse.

> ⚠ Caution
>
> Do not use Apache Portable Runtime (APR) on the system on which you use the Cloud Connector. If you cannot avoid this restriction and want to use APR at your own risk, you need to manually adopt the `server.xml` configuration file in directory `<scc_installation_folder>/conf`. To do so, follow the steps in HTTPS port configuration for APR.

Back to Content [page 1610]

JDKs

JDK	Version	Cloud Connector Version
SAP JVM 64-bit (recommended)	8	2.7.2 and higher
Oracle JDK 64-bit	8	2.7.2 and higher
SapMachine 64-bit	11 (EOM)	2.14.0 up to 2.18.2
	17	2.15.0 up to 2.19.x
	21	2.17.0 and higher
	25	2.19.0 and higher

If you want to use SAP JVM, you can download its installer package from the SAP Development Tools page or a compressed SAR archive from the SAP Software Center.

Alternatively supported SapMachine JDK versions can be downloaded from the SapMachine homepage.

Back to Content [page 1610]

Product Availability Matrix

> **Note**
>
> Whether a concrete combination of JDK and operating system can actually be used depends on the availability of the JDK for the operating system.

Operating System Version	Architecture	Cloud Connector Version
Windows 7, Windows Server 2008 R2	x86_64	2.1.0 up to 2.17.2
SUSE Linux Enterprise Server 11, Red Hat Enterprise Linux 6	x86_64	2.1.0 up to 2.16.0
Mac OS X 10.7 (Lion), Mac OS X 10.8 (Mountain Lion)	x86_64	2.1.0 up to 2.12.5
Windows 8.1, Windows Server 2012, Windows Server 2012 R2	x86_64	2.5.1 up to 2.17.2
SUSE Linux Enterprise Server 12, Red Hat Enterprise Linux 7	x86_64	2.5.1 up to 2.17.2
Mac OS X 10.9 (Mavericks), Mac OS X 10.10 (Yosemite)	x86_64	2.5.1 up to 2.12.5
Windows 10	x86_64	2.7.2 and higher
Mac OS X 10.11 (El Capitan)	x86_64	2.8.1 up to 2.16.2
Windows Server 2016	x86_64	2.9.1 up to 2.19.x
Windows Server 2019	x86_64	2.11.3 and higher
macOS 10.12 (Sierra), macOS 10.13 (High Sierra)	x86_64	2.11.3 up to 2.16.2
macOS 10.14 (Mojave)	x86_64	2.11.3 up to 2.17.2
SUSE Linux Enterprise Server 15	x86_64	2.12.0 and higher
Red Hat Enterprise Linux 8	x86_64	2.12.2 and higher
SUSE Linux Enterprise Server 12, Red Hat Enterprise Linux 7	ppc64le	2.13.0 up to 2.17.2
SUSE Linux Enterprise Server 15, Red Hat Enterprise Linux 8	ppc64le	2.13.0 and higher
Windows Server 2022	x86_64	2.14.0 and higher
macOS 10.15 (Catalina)	x86_64	2.14.0 up to 2.17.2
Windows 11	x86_64	2.15.0 and higher
Red Hat Enterprise Linux 9	x86_64, ppc64le	2.15.0 and higher
Oracle Linux 8	x86_64	2.15.2 and higher
macOS 11 (Big Sur)	x86_64	2.16.0 up to 2.17.2
Oracle Linux 9	x86_64	2.16.0 and higher

Operating System Version	Architecture	Cloud Connector Version
macOS 12 (Monterey), macOS 13 (Ventura)	x86_64, aarch64	2.16.0 up to 2.18.2
macOS 14 (Sonoma)	x86_64, aarch64	2.17.0 up to 2.19.x
macOS 15 (Sequoia)	x86_64, aarch64	2.18.0 and higher
Windows Server 2025	x86_64	2.18.0 and higher
macOS 26 (Tahoe)	x86_64, aarch64	2.19.0 and higher
SUSE Linux Enterprise Server 15, Red Hat Enterprise Linux 9	aarch64	2.19.0 and higher
SUSE Linux Enterprise Server 16, Red Hat Enterprise Linux 10	x86_64, ppc64le, aarch64	2.19.0 and higher

Back to Content [page 1610]

Network

You must have Internet connection at least to the following Connectivity service hosts (depending on the region), to which you can connect your Cloud Connector. All connections to the hosts are TLS-based and connect to port 443.

> → Remember
>
> For some solutions of the BTP portfolio, you must include additional hosts to set up an on-premises connectivity scenario with the Cloud Connector. This applies, for example, to: SAP Data Intelligence, SAP HANA Cloud, Business Appilcation Studio, SAP Cloud Identity Services, and SAP Build Apps. Check the respective solution documentation for details.

> ⓘ Note
>
> To configure the Cloud Connector for SAP Datasphere, see also Configure Cloud Connector (SAP Datasphere documentation).

> ⓘ Note
>
> For general information on IP ranges per region, see Regions (Cloud Foundry and ABAP environment) or Regions and Hosts Available for the Neo Environment. Find detailed information about the region status and planned network updates on Platform Updates and Notifications.

> ⓘ Note
>
> The *Connectivity Region Host* in the table below is mentioned only if it differs from the usual *Region Host*.

Cloud Foundry Environment [page 1614]

ABAP Environment [page 1640]

Neo Environment [page 1641]

Trial [page 1643]

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
Cloud Foundry Environment			

> ⚠ Caution
>
> The Connectivity service is planning to enable IPv4 and IPv6 dual stack ingress for **AWS and Google Cloud-based regions**. Currently, only IPv4 is supported, therefore we had only listed the respective addresses here. We have now added also the **IPv6 addresses** that we plan to use. The enablement of the IPv6 addresses will happen no earlier than **February 2026** (AWS) and **April 2026** (Google Cloud). Until then, **please ensure any needed action is performed**, based on the information below.
>
> The Connectivity service ingress endpoints are the ones that are called by the Cloud Connector. If the network in which your Cloud Connector is running *does not* support IPv6, *no action is needed*, as the IPv4 addresses remain unaffected. If the network in which your Cloud Connector is running *does* support IPv6, then the Java virtual machine (JVM) will usually prefer using IPv4 instead of IPv6 (usually controlled via the system property `java.net.preferIPv6Addresses`). If this is the case for your JVM, again, *no action is needed*. If your *JVM is configured to prefer IPv6*, you need to *verify if you have firewall rules in place* that limit outgoing traffic from the network. If this is true, you would have previously allowed the Connectivity service's IPv4 addresses in order to enable the Cloud Connector's communication with BTP. Now, you need to *add also the IPv6 addresses* to the allowlist.
>
> As of version 2.18.1, the Cloud Connector shows the IPv6 status on the *About* screen.
>
> Regions for which we haven't added IPv6 address in the documentation are currently not planned for this activity.

> ⓘ Note
>
> In the Cloud Foundry environment, IPs are controlled by the respective IaaS provider - Amazon Web Services (AWS), Microsoft Azure (Azure), or Google Cloud. IPs may change due to network updates on the provider side. Any planned changes will be announced at least 4 weeks before they take effect. See also Regions.

Europe (Frankfurt) - SAP (EU Access Only) (**`cf.eu01.hana.ondemand.com`**) (mtls.eu01-l-c.uc-live-eu.shoot.live.k8s-hana.ondemand.com)	connectivitynotification.mtls.eu01-l-c.uc-live-eu.shoot.live.k8s-hana.ondemand.com	`130.214.124.220`
	connectivitycertsigning.mtls.eu01-l-c.uc-live-eu.shoot.live.k8s-hana.ondemand.com	`130.214.124.220`

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
Note Available as of September 19, 2025 **New domain (as of November 15, 2026):** **(eu01.uc.platform.cloud.sap)**	connectivitytunnel.eu01-l-d.uc-live-eu.shoot.live.k8s-hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu01.uc.platform.cloud.sap**	`130.214.124.213`	
Europe (Rot) - SAP (EU Access Only) (**`cf.eu02.hana.ondemand.com`**) (mtls.eu02-l-c.uc-live-eu.shoot.live.k8s-hana.ondemand.com) **Note** Available as of November 28, 2025 **New domain (as of November 15, 2026):** **(eu02.uc.platform.cloud.sap)**	connectivitynotification.mtls.eu02-l-c.uc-live-eu.shoot.live.k8s-hana.ondemand.com	`130.214.170.48`	
	connectivitycertsigning.mtls.eu02-l-c.uc-live-eu.shoot.live.k8s-hana.ondemand.com	`130.214.170.48`	
	connectivitytunnel.eu02-l-d.uc-live-eu.shoot.live.k8s-hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu02.uc.platform.cloud.sap**	`130.214.170.47`	
Europe (Frankfurt) - AWS (**`cf.eu10.hana.ondemand.com`**) **New domain (as of November 15, 2026):** **(eu10.uc.platform.cloud.sap)**	connectivitynotification.cf.eu10.hana.ondemand.com	`3.124.222.77`, `3.122.209.241`, `3.124.208.223`, `18.159.31.22`, `3.69.186.98`, `3.77.195.119` Additional IP addresses (as of January 19, 2025): 3.123.247.156, 18.184.184.134, 3.76.166.75	2a05:d014:ca2:3702::17, 2a05:d014:ca2:3705::17, 2a05:d014:ca2:3708::17

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
Enterprise & Trial	connectivitycertsigning.cf.eu10.hana.ondemand.com	`3.124.222.77`, `3.122.209.241`, `3.124.208.223`, `18.159.31.22`, `3.69.186.98`, `3.77.195.119` Additional IP addresses (as of January 19, 2025): 3.123.247.156, 18.184.184.134, 3.76.166.75	2a05:d014:ca2:3702::17, 2a05:d014:ca2:3705::17, 2a05:d014:ca2:3708::17
	connectivitytunnel.cf.eu10.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu10.uc.platform.cloud.sap**	`3.124.222.77`, `3.122.209.241`, `3.124.208.223`, `18.159.31.22`, `3.69.186.98`, `3.77.195.119` Additional IP addresses (as of January 19, 2025): 35.157.237.81, 52.58.116.80, 18.196.77.73	2a05:d014:e0d:f902::17, 2a05:d014:e0d:f908::17, 2a05:d014:e0d:f905::17
	connectivitytunnel.cf.eu10-002.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu10.uc.platform.cloud.sap**	`3.64.227.236`, `3.126.229.22`, `18.193.180.19`, `18.153.123.11`, `3.121.37.195`, `3.73.215.90` Additional IP addresses (as of January 19, 2025): 3.72.125.186, 18.199.43.118, 3.69.159.226 **Additional IP addresses (as of November 15, 2026):** **35.157.237.81, 52.58.116.80, 18.196.77.73**	2a05:d014:173b:8505::17, 2a05:d014:173b:8502::17, 2a05:d014:173b:8508::17 **Additional IP addresses (as of November 15, 2026):** **2a05:d014:e0d:f902::17,** **2a05:d014:e0d:f908::17,** **2a05:d014:e0d:f905::17**

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
	connectivitytunnel.cf.eu10-003.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu10.uc.platform.cloud.sap**	`3.127.77.3`, `3.64.196.58`, `18.156.151.247`, `18.197.252.154`, `3.79.137.29`, `52.58.93.50` Additional IP addresses (as of January 19, 2025): 3.65.23.248, 18.192.224.62, 35.159.136.14 **Additional IP addresses (as of November 15, 2026):** **35.157.237.81, 52.58.116.80, 18.196.77.73**	2a05:d014:1db0:1705::17, 2a05:d014:1db0:1702::17, 2a05:d014:1db0:1708::17 **Additional IP addresses (as of November 15, 2026):** **2a05:d014:e0d:f902::17, 2a05:d014:e0d:f908::17, 2a05:d014:e0d:f905::17**
	connectivitytunnel.cf.eu10-004.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu10.uc.platform.cloud.sap**	`3.65.185.47`, `3.70.38.218`, `18.196.206.8`, `3.73.109.100`, `3.73.8.210`, `52.59.18.183` Additional IP addresses (as of January 19, 2025): 3.125.225.167, 18.158.49.214, 35.157.19.103 **Additional IP addresses (as of November 15, 2026):** **35.157.237.81, 52.58.116.80, 18.196.77.73**	2a05:d014:1140:c905::17, 2a05:d014:1140:c908::17, 2a05:d014:1140:c902::17 **Additional IP addresses (as of November 15, 2026):** **2a05:d014:e0d:f902::17, 2a05:d014:e0d:f908::17, 2a05:d014:e0d:f905::17**

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
	connectivitytunnel.cf.eu10-005.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu10.uc.platform.cloud.sap**	`3.122.31.132`, `18.193.56.244`, `3.78.172.245` Additional IP addresses (as of January 19, 2025): 3.72.138.152, 18.195.38.125, 52.57.99.237 **Additional IP addresses (as of November 15, 2026):** **35.157.237.81, 52.58.116.80, 18.196.77.73**	2a05:d014:1e09:6a02::17, 2a05:d014:1e09:6a05::17, 2a05:d014:1e09:6a08::17 **Additional IP addresses (as of November 15, 2026):** **2a05:d014:e0d:f902::17,** **2a05:d014:e0d:f908::17,** **2a05:d014:e0d:f905::17**
Europe (Frankfurt) - AWS (`cf.eu11.hana.ondemand.com`) **New domain (as of November 15, 2026):** **(eu11.uc.platform.cloud.sap)**	connectivitynotification.cf.eu11.hana.ondemand.com	`3.124.207.41`, `18.157.105.117`, `18.156.209.198`, `3.66.26.249`, `3.72.216.204`, `3.74.99.245` Additional IP addresses (as of January 19, 2025): 3.65.41.79, 35.159.179.221, 52.59.124.48	2a05:d014:1b2c:b502::17, 2a05:d014:1b2c:b508::17, 2a05:d014:1b2c:b505::17
	connectivitycertsigning.cf.eu11.hana.ondemand.com	`3.124.207.41`, `18.157.105.117`, `18.156.209.198`, `3.66.26.249`, `3.72.216.204`, `3.74.99.245` Additional IP addresses (as of January 19, 2025): 3.65.41.79, 35.159.179.221, 52.59.124.48	2a05:d014:1b2c:b502::17, 2a05:d014:1b2c:b508::17, 2a05:d014:1b2c:b505::17

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
	connectivitytunnel.cf.eu11.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu11.uc.platform.cloud.sap**	`3.124.207.41`, `18.157.105.117`, `18.156.209.198`, `3.66.26.249`, `3.72.216.204`, `3.74.99.245` Additional IP addresses (as of January 19, 2025): 3.78.138.146, 3.78.214.251, 18.199.183.133	2a05:d014:62c:7a05::17, 2a05:d014:62c:7a08::17, 2a05:d014:62c:7a02::17
Europe (Milan) - AWS (**`cf.eu13.hana.ondemand.com`**) **Note** Available as of October 30, 2025	connectivitynotification.cf.eu13.hana.ondemand.com	`18.102.145.36`, `35.152.161.248`, `18.102.221.20`	2a05:d01a:784:e205::17, 2a05:d01a:784:e208::17, 2a05:d01a:784:e202::17
	connectivitycertsigning.cf.eu13.hana.ondemand.com	`18.102.145.36`, `35.152.161.248`, `18.102.221.20`	2a05:d01a:784:e205::17, 2a05:d01a:784:e208::17, 2a05:d01a:784:e202::17
New domain (as of November 15, 2026): **(eu13.uc.platform.cloud.sap)**	connectivitytunnel.cf.eu13.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu13.uc.platform.cloud.sap**	`18.102.227.45`, `15.161.193.78`, `18.102.173.150`	2a05:d01a:89a:3d02::17, 2a05:d01a:89a:3d08::17, 2a05:d01a:89a:3d05::17
Europe (Netherlands) - Azure (**`cf.eu20.hana.ondemand.com`**)	connectivitynotification.cf.eu20.hana.ondemand.com	`40.119.153.88` Additional IP address (as of January 19, 2025): 57.153.48.23	
New domain (as of November 15, 2026): **(eu20.uc.platform.cloud.sap)**	connectivitycertsigning.cf.eu20.hana.ondemand.com	`40.119.153.88` Additional IP address (as of January 19, 2025): 57.153.48.23	

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
	connectivitytunnel.cf.eu20.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu20.uc.platform.cloud.sap**	`40.119.153.88` Additional IP address (as of January 19, 2025): 57.153.52.157	
	connectivitytunnel.cf.eu20-001.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu20.uc.platform.cloud.sap**	`20.82.83.59` Additional IP address (as of January 19, 2025): 20.67.69.84 **Additional IP addresses (as of November 15, 2026):** **57.153.52.157**	
	connectivitytunnel.cf.eu20-002.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu20.uc.platform.cloud.sap**	`98.64.76.0` **Additional IP addresses (as of November 15, 2026):** **57.153.52.157**	
Europe (Frankfurt) - Azure (`cf.eu22.hana.ondemand.com`) **Note** Available as of January 30, 2026 **New domain (as of November 15, 2026):** **(eu22.uc.platform.cloud.sap)**	connectivitynotification.cf.eu22.hana.ondemand.com	`131.189.177.61`	
	connectivitycertsigning.cf.eu22.hana.ondemand.com	`131.189.177.61`	
	connectivitytunnel.cf.eu22.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu22.uc.platform.cloud.sap**	`9.141.129.192` **Additional IP addresses (as of November 15, 2026):** **4.185.195.58**	

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
Europe (Frankfurt) - Google Cloud (**cf.eu30.hana.ondemand.com**) **New domain (as of November 15, 2026):** **(eu30.uc.platform.cloud.sap)**	connectivitynotification.cf.eu30.hana.ondemand.com	35.198.143.110 Additional IP address (as of April 2026): 35.242.194.251	2600:1900:40d0:8a0:8000:2::
	connectivitycertsigning.cf.eu30.hana.ondemand.com	35.198.143.110 Additional IP address (as of April 2026): 35.242.194.251	2600:1900:40d0:8a0:8000:2::
	connectivitytunnel.cf.eu30.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu30.uc.platform.cloud.sap**	35.198.143.110 Additional IP address (as of April 2026): 34.107.120.89	2600:1900:40d0:1897:8000:1::
UK South (London) - Azure (**cf.uk20.hana.ondemand.com**) > Note > Available as of January 30, 2026 **New domain (as of November 15, 2026):** **(cf.uk20.hana.ondemand.com)**	connectivitynotification.cf.uk20.hana.ondemand.com	4.250.195.185	
	connectivitycertsigning.cf.uk20.hana.ondemand.com	4.250.195.185	
	connectivitytunnel.cf.uk20.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.uk20.uc.platform.cloud.sap**	4.250.195.80	
US (Sterling) - SAP (**cf.us01.hana.ondemand.com**) (mtls.us01-l-c.uc-live.shoot.live.k8s-hana.ondemand.com)	connectivitynotification.mtls.us01-l-c.uc-live.shoot.live.k8s-hana.ondemand.com	130.214.107.153	
	connectivitycertsigning.mtls.us01-l-c.uc-live.shoot.live.k8s-hana.ondemand.com	130.214.107.153	

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
> **Note** Available as of January 30, 2026 **New domain (as of November 15, 2026):** **(us01.uc.platform.cloud.sap)**	connectivitytunnel.us01-l-d.uc-live.shoot.live.k8s-hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.us01.uc.platform.cloud.sap**	`130.214.107.141`	
US West (Colorado) - SAP (**`cf.us02.hana.ondemand.com`**) (mtls.us02-l-c.uc-live.shoot.live.k8s-hana.ondemand.com) > **Note** Available as of December 30, 2025 **New domain (as of November 15, 2026):** **(us02.uc.platform.cloud.sap)**	connectivitynotification.mtls.us02-l-c.uc-live.shoot.live.k8s-hana.ondemand.com	`130.214.68.4`	
	connectivitycertsigning.mtls.us02-l-c.uc-live.shoot.live.k8s-hana.ondemand.com	`130.214.68.4`	
	connectivitytunnel.us02-l-d.uc-live.shoot.live.k8s-hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.us02.uc.platform.cloud.sap**	`130.214.68.30`	
US East (VA) - AWS (**`cf.us10.hana.ondemand.com`**) *Enterprise & Trial* **New domain (as of November 15, 2026):** **(us10.uc.platform.cloud.sap)**	connectivitynotification.cf.us10.hana.ondemand.com	`52.23.189.23`, `52.4.101.240`, `52.23.1.211`, `18.213.242.208`, `3.214.110.153`, `34.205.56.51` Additional IP addresses (as of January 19, 2025): 98.82.227.152, 107.20.252.91, 34.198.33.88	2600:1f18:ef:9b05::17, 2600:1f18:ef:9b08::17, 2600:1f18:ef:9b02::17 New (as of May 10, 2026): 2600:1f18:ef:9b08::17, 2600:1f18:ef:9b0b::17, 2600:1f18:ef:9b02::17

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
	connectivitycertsigning.cf.us10.hana.ondemand.com	`52.23.189.23`, `52.4.101.240`, `52.23.1.211`, `18.213.242.208`, `3.214.110.153`, `34.205.56.51` Additional IP addresses (as of January 19, 2025): 98.82.227.152, 107.20.252.91, 34.198.33.88	2600:1f18:ef:9b05::17, 2600:1f18:ef:9b08::17, 2600:1f18:ef:9b02::17 New (as of May 10, 2026): 2600:1f18:ef:9b08::17, 2600:1f18:ef:9b0b::17, 2600:1f18:ef:9b02::17
	connectivitytunnel.cf.us10.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.us10.uc.platform.cloud.sap**	`52.23.189.23`, `52.4.101.240`, `52.23.1.211`, `18.213.242.208`, `3.214.110.153`, `34.205.56.51` Additional IP addresses (as of January 19, 2025): 54.147.47.110, 18.214.162.12, 44.206.11.12	2600:1f18:12b5:c605::17, 2600:1f18:12b5:c602::17, 2600:1f18:12b5:c608::17 New (as of May 10, 2026): 2600:1f18:12b5:c60b::17, 2600:1f18:12b5:c602::17, 2600:1f18:12b5:c608::17
	connectivitytunnel.cf.us10-001.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.us10.uc.platform.cloud.sap**	`3.220.114.17`, `3.227.182.44`, `52.86.131.53`, `44.218.82.203`, `44.219.57.163`, `50.16.106.103` Additional IP addresses (as of January 19, 2025): 54.157.55.28, 3.234.87.202, 50.17.185.79 **Additional IP addresses (as of November 15, 2026):** **54.147.47.110, 18.214.162.12, 44.206.11.12**	2600:1f18:2d7e:2905::17, 2600:1f18:2d7e:2908::17, 2600:1f18:2d7e:2902::17 **Additional IP addresses (as of November 15, 2026):** **2600:1f18:12b5:c60b::17, 2600:1f18:12b5:c602::17, 2600:1f18:12b5:c608::17**

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
	connectivitytunnel.cf.us10-002.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.us10.uc.platform.cloud.sap**	`34.202.68.0`, `54.234.152.59`, `107.20.66.86`, `3.214.116.95`, `54.144.230.36`, `54.226.37.104` Additional IP addresses (as of January 19, 2025): 34.234.117.43, 34.237.168.235, 44.221.148.86 **Additional IP addresses (as of November 15, 2026):** **54.147.47.110, 18.214.162.12, 44.206.11.12**	2600:1f18:264b:5405::17, 2600:1f18:264b:5402::17, 2600:1f18:264b:5408::17 **Additional IP addresses (as of November 15, 2026):** **2600:1f18:12b5:c60b::17, 2600:1f18:12b5:c602::17, 2600:1f18:12b5:c608::17**
	connectivitytunnel.cf.us10-003.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.us10.uc.platform.cloud.sap**	`3.41.185.1`, `3.41.185.0`, `3.41.185.2` **Additional IP addresses (as of November 15, 2026):** **54.147.47.110, 18.214.162.12, 44.206.11.12**	**Additional IP addresses (as of November 15, 2026):** **2600:1f18:12b5:c60b::17, 2600:1f18:12b5:c602::17, 2600:1f18:12b5:c608::17**
US West (Oregon) - AWS (`cf.us11.hana.ondemand.com`) **New domain (as of November 15, 2026):** **(us11.uc.platform.cloud.sap)**	connectivitynotification.cf.us11.hana.ondemand.com	`34.211.82.149`, `100.20.19.69`, `35.95.238.236`, `54.69.228.173`, `35.161.174.123`, `44.230.137.133`, `54.201.79.229`, `34.217.172.244`, `44.235.33.61`	2600:1f14:180c:7205::17, 2600:1f14:180c:7208::17, 2600:1f14:180c:7202::17

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
	connectivitycertsigning.cf.us11.hana.ondemand.com	`34.211.82.149`, `100.20.19.69`, `35.95.238.236`, `54.69.228.173`, `35.161.174.123`, `44.230.137.133`, `54.201.79.229`, `34.217.172.244`, `44.235.33.61`	2600:1f14:180c:7205::17, 2600:1f14:180c:7208::17, 2600:1f14:180c:7202::17
	connectivitytunnel.cf.us11.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.us11.uc.platform.cloud.sap**	`34.211.82.149`, `100.20.19.69`, `35.95.238.236`, `54.69.228.173`, `35.161.174.123`, `44.230.137.133`, `54.201.79.229`, `34.217.172.244`, `44.235.33.61`	2600:1f14:2448:f202::17, 2600:1f14:2448:f205::17, 2600:1f14:2448:f208::17
US West (WA) - Azure (**`cf.us20.hana.ondemand.com`**) **New domain (as of November 15, 2026):** **(us20.uc.platform.cloud.sap)**	connectivitynotification.cf.us20.hana.ondemand.com	`40.91.120.100` Additional IP address (as of January 19, 2025): 52.137.100.108	
	connectivitycertsigning.cf.us20.hana.ondemand.com	`40.91.120.100` Additional IP address (as of January 19, 2025): 52.137.100.108	
	connectivitytunnel.cf.us20.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.us20.uc.platform.cloud.sap**	`40.91.120.100` Additional IP address (as of January 19, 2025): 52.137.108.200	
US East (VA) - Azure (**`cf.us21.hana.ondemand.com`**)	connectivitynotification.cf.us21.hana.ondemand.com	`40.88.52.17` Additional IP address (as of January 19, 2025): 48.216.153.6	

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
New domain (as of November 15, 2026): **(us21.uc.platform.cloud.sap)**	connectivitycertsigning.cf.us21.hana.ondemand.com	`40.88.52.17` Additional IP address (as of January 19, 2025): 48.216.153.6	
	connectivitytunnel.cf.us21.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.us21.uc.platform.cloud.sap**	`40.88.52.17` Additional IP address (as of January 19, 2025): 48.216.153.226	
	connectivitytunnel.cf.us21-001.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.us21.uc.platform.cloud.sap**	`20.241.255.189` **Additional IP addresses (as of November 15, 2026):** **48.216.153.226**	
US Central (IA) - Google Cloud (`cf.us30.hana.ondemand.com`)	connectivitynotification.cf.us30.hana.ondemand.com	`35.184.169.79` Additional IP address (as of April 2026): 34.57.177.99	2600:1900:4000:18b3:8000:2::
New domain (as of November 15, 2026): **(us30.uc.platform.cloud.sap)**	connectivitycertsigning.cf.us30.hana.ondemand.com	`35.184.169.79` Additional IP address (as of April 2026): 34.57.177.99	2600:1900:4000:18b3:8000:2::
	connectivitytunnel.cf.us30.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.us30.uc.platform.cloud.sap**	`35.184.169.79` Additional IP address (as of April 2026): 35.239.235.94	2600:1900:4001:b6d:8000:1::

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
Brazil (São Paulo) - AWS (**cf.br10.hana.ondemand.com**) **New domain (as of November 15, 2026):** **(br10.uc.platform.cloud.sap)**	connectivitynotification.cf.br10.hana.ondemand.com	18.229.91.150, 52.67.135.4, 54.232.179.204, 18.228.53.198, 52.67.149.240, 54.94.179.209 Additional IP addresses (as of January 19, 2025): 177.71.155.99, 177.71.245.233, 18.231.25.174	2600:1f1e:c13:1502::17, 2600:1f1e:c13:1508::17, 2600:1f1e:c13:1505::17
	connectivitycertsigning.cf.br10.hana.ondemand.com	18.229.91.150, 52.67.135.4, 54.232.179.204, 18.228.53.198, 52.67.149.240, 54.94.179.209 Additional IP addresses (as of January 19, 2025): 177.71.155.99, 177.71.245.233, 18.231.25.174	2600:1f1e:c13:1502::17, 2600:1f1e:c13:1508::17, 2600:1f1e:c13:1505::17
	connectivitytunnel.cf.br10.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.br10.uc.platform.cloud.sap**	18.229.91.150, 52.67.135.4, 54.232.179.204, 18.228.53.198, 52.67.149.240, 54.94.179.209 Additional IP addresses (as of January 19, 2025): 18.229.78.152, 54.232.119.169, 18.228.231.243	2600:1f1e:32e:2c05::17, 2600:1f1e:32e:2c08::17, 2600:1f1e:32e:2c02::17
Brazil (São Paulo) - Azure (**cf.br20.hana.ondemand.com**)	connectivitynotification.cf.br20.hana.ondemand.com	4.228.118.21 Additional IP address (as of January 19, 2025): 20.201.104.57	

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
New domain (as of November 15, 2026): **(br20.uc.platform.cloud.sap)**	connectivitycertsigning.cf.br20.hana.ondemand.com	`4.228.118.21` Additional IP address (as of January 19, 2025): 20.201.104.57	
	connectivitytunnel.cf.br20.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.br20.uc.platform.cloud.sap**	`4.228.118.21` Additional IP address (as of January 19, 2025): 20.201.105.31	
Brazil (São Paulo) - Google Cloud (**`cf.br30.hana.ondemand.com`**) **New domain (as of November 15, 2026):** **(br30.uc.platform.cloud.sap)**	connectivitynotification.cf.br30.hana.ondemand.com	`34.95.189.118` Additional IP address (as of April 2026): 34.151.240.141	2600:1900:40f0:a5e:8000:2::
	connectivitycertsigning.cf.br30.hana.ondemand.com	`34.95.189.118` Additional IP address (as of April 2026): 34.151.240.141	2600:1900:40f0:a5e:8000:2::
	connectivitytunnel.cf.br30.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.br30.uc.platform.cloud.sap**	`34.95.189.118` Additional IP address (as of April 2026): 34.39.192.15	2600:1900:40f0:648:8000:1::
Japan (Tokyo) - SAP (**`cf.jp01.hana.ondemand.com`**) (mtls.jp01-l-c.uc-live.shoot.live.k8s-hana.ondemand.com) **Note** Available as of January 15, 2026	connectivitynotification.mtls.jp01-l-c.uc-live.shoot.live.k8s-hana.ondemand.com	`130.214.52.41`	
	connectivitycertsigning.mtls.jp01-l-c.uc-live.shoot.live.k8s-hana.ondemand.com	`130.214.52.41`	

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
New domain (as of November 15, 2026): **(jp01.uc.platform.cloud.sap)**	connectivitytunnel.jp01-l-d.uc-live.shoot.live.k8s-hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.jp01.uc.platform.cloud.sap**	`130.214.52.46`	
Japan (Tokyo) - AWS (**`cf.jp10.hana.ondemand.com`**) **New domain (as of November 15, 2026):** **(jp10.uc.platform.cloud.sap)**	connectivitynotification.cf.jp10.hana.ondemand.com	`13.114.117.83`, `3.114.248.68`, `3.113.252.15`, `18.178.155.134`, `57.180.140.5`, `57.180.145.179` Additional IP addresses (as of January 19, 2025): 35.74.196.228, 35.75.166.160, 52.198.15.94	2406:da14:1d30:cb05::17, 2406:da14:1d30:cb08::17, 2406:da14:1d30:cb02::17
	connectivitycertsigning.cf.jp10.hana.ondemand.com	`13.114.117.83`, `3.114.248.68`, `3.113.252.15`, `18.178.155.134`, `57.180.140.5`, `57.180.145.179` Additional IP addresses (as of January 19, 2025): 35.74.196.228, 35.75.166.160, 52.198.15.94	2406:da14:1d30:cb05::17, 2406:da14:1d30:cb08::17, 2406:da14:1d30:cb02::17
	connectivitytunnel.cf.jp10.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.jp10.uc.platform.cloud.sap**	`13.114.117.83`, `3.114.248.68`, `3.113.252.15`, `18.178.155.134`, `57.180.140.5`, `57.180.145.179` Additional IP addresses (as of January 19, 2025): 13.114.120.142, 18.176.244.8, 54.92.104.72	2406:da14:191b:9005::17, 2406:da14:191b:9002::17, 2406:da14:191b:9008::17

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
Japan (Tokyo) - Azure (**cf.jp20.hana.ondemand.com**) **New domain (as of November 15, 2026):** **(jp20.uc.platform.cloud.sap)**	connectivitynotification.cf.jp20.hana.ondemand.com	20.43.89.91 Additional IP address (as of January 19, 2025): 48.218.89.49	
	connectivitycertsigning.cf.jp20.hana.ondemand.com	20.43.89.91 Additional IP address (as of January 19, 2025): 48.218.89.49	
	connectivitytunnel.cf.jp20.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.jp20.uc.platform.cloud.sap**	20.43.89.91 Additional IP address (as of January 19, 2025): 48.218.90.204	
Japan (Osaka) - Google Cloud (**cf.jp30.hana.ondemand.com**) **New domain (as of November 15, 2026):** **(jp30.uc.platform.cloud.sap)**	connectivitynotification.cf.jp30.hana.ondemand.com	34.97.168.169 Additional IP address (as of April 2026): 34.97.125.57	2600:1900:41d0:611:8000:2::
	connectivitycertsigning.cf.jp30.hana.ondemand.com	34.97.168.169 Additional IP address (as of April 2026): 34.97.125.57	2600:1900:41d0:611:8000:2::
	connectivitytunnel.cf.jp30.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.jp30.uc.platform.cloud.sap**	34.97.168.169 Additional IP address (as of April 2026): 34.97.69.170	2600:1900:41d0:713:8000:1::
Japan (Tokyo) - Google Cloud (**cf.jp31.hana.ondemand.com**)	connectivitynotification.cf.jp31.hana.ondemand.com	34.84.96.118 Additional IP address (as of April 2026): 34.85.108.120	2600:1900:4051:2b7:8000:2::

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
New domain (as of November 15, 2026): **(jp31.uc.platform.cloud.sap)**	connectivitycertsigning.cf.jp31.hana.ondemand.com	`34.84.96.118` Additional IP address (as of April 2026): 34.85.108.120	2600:1900:4051:2b7:8000:2::
	connectivitytunnel.cf.jp31.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.jp31.uc.platform.cloud.sap**	`34.84.96.118` Additional IP address (as of April 2026): 34.104.137.202	2600:1900:4050:248:8000:1::
Australia (Sydney) - SAP (`cf.ap01.hana.ondemand.com`) (mtls.ap01-l-c.uc-live.shoot.live.k8s-hana.ondemand.com) **Note** Available as of December 30, 2025 **New domain (as of November 15, 2026):** **(ap01.uc.platform.cloud.sap)**	connectivitynotification.mtls.ap01-l-c.uc-live.shoot.live.k8s-hana.ondemand.com	`130.214.141.27`	
	connectivitycertsigning.mtls.ap01-l-c.uc-live.shoot.live.k8s-hana.ondemand.com	`130.214.141.27`	
	connectivitytunnel.ap01-l-c.uc-live.shoot.live.k8s-hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.ap01.uc.platform.cloud.sap**	`130.214.141.21`	

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
Australia (Sydney) - AWS (`cf.ap10.hana.ondemand.com`) **New domain (as of November 15, 2026):** **(ap10.uc.platform.cloud.sap)**	connectivitynotification.cf.ap10.hana.ondemand.com	`13.236.220.84`, `13.211.73.244`, `3.105.95.184`, `13.55.188.95`, `3.105.212.249`, `3.106.45.106` Additional IP addresses (as of January 19, 2025): 3.107.130.66, 3.104.152.189, 3.105.214.80	2406:da1c:339:e808::17, 2406:da1c:339:e805::17, 2406:da1c:339:e802::17
	connectivitycertsigning.cf.ap10.hana.ondemand.com	`13.236.220.84`, `13.211.73.244`, `3.105.95.184`, `13.55.188.95`, `3.105.212.249`, `3.106.45.106` Additional IP addresses (as of January 19, 2025): 3.107.130.66, 3.104.152.189, 3.105.214.80	2406:da1c:339:e808::17, 2406:da1c:339:e805::17, 2406:da1c:339:e802::17
	connectivitytunnel.cf.ap10.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.ap10.uc.platform.cloud.sap**	`13.236.220.84`, `13.211.73.244`, `3.105.95.184`, `13.55.188.95`, `3.105.212.249`, `3.106.45.106` Additional IP addresses (as of January 19, 2025): 13.54.123.199, 13.55.164.105, 3.106.145.100	2406:da1c:b6a:b905::17, 2406:da1c:b6a:b908::17, 2406:da1c:b6a:b902::17

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
Asia Pacific (Singapore) - AWS (`cf.ap11.hana.ondemand.com`) **New domain (as of November 15, 2026):** **(ap11.uc.platform.cloud.sap)**	connectivitynotification.cf.ap11.hana.ondemand.com	`3.0.9.102`,`18.140.39.70`, `18.139.147.53`, `13.229.158.122`, `18.140.228.217`, `52.74.215.89` Additional IP addresses (as of January 19, 2025): 52.76.177.178, 13.213.177.227, 13.251.142.208	2406:da18:500:5402::17, 2406:da18:500:5405::17, 2406:da18:500:5408::17
	connectivitycertsigning.cf.ap11.hana.ondemand.com	`3.0.9.102`, `18.140.39.70`, `18.139.147.53`, `13.229.158.122`, `18.140.228.217`, `52.74.215.89` Additional IP addresses (as of January 19, 2025): 52.76.177.178, 13.213.177.227, 13.251.142.208	2406:da18:500:5402::17, 2406:da18:500:5405::17, 2406:da18:500:5408::17
	connectivitytunnel.cf.ap11.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.ap11.uc.platform.cloud.sap**	`3.0.9.102`, `18.140.39.70`, `18.139.147.53`, `13.229.158.122`, `18.140.228.217`, `52.74.215.89` Additional IP addresses (as of January 19, 2025): 13.215.4.190, 13.229.41.216, 52.76.219.61	2406:da18:6d5:c05::17, 2406:da18:6d5:c08::17, 2406:da18:6d5:c02::17

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
Asia Pacific (Seoul) - AWS (**cf.ap12.hana.ondemand.com**) **New domain (as of November 15, 2026):** **(ap12.uc.platform.cloud.sap)**	connectivitynotification.cf.ap12.hana.ondemand.com	3.35.255.45, 3.35.106.215, 3.35.215.12, 13.209.236.215, 43.201.194.105, 43.202.204.5 Additional IP addresses (as of January 19, 2025): 3.37.25.26, 43.202.209.5, 54.180.187.39	2406:da12:ae7:cc02::17, 2406:da12:ae7:cc05::17, 2406:da12:ae7:cc08::17
	connectivitycertsigning.cf.ap12.hana.ondemand.com	3.35.255.45, 3.35.106.215, 3.35.215.12, 13.209.236.215, 43.201.194.105, 43.202.204.5 Additional IP addresses (as of January 19, 2025): 3.37.25.26, 43.202.209.5, 54.180.187.39	2406:da12:ae7:cc02::17, 2406:da12:ae7:cc05::17, 2406:da12:ae7:cc08::17
	connectivitytunnel.cf.ap12.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.ap12.uc.platform.cloud.sap**	3.35.255.45, 3.35.106.215, 3.35.215.12, 13.209.236.215, 43.201.194.105, 43.202.204.5 Additional IP addresses (as of January 19, 2025): 3.37.1.194, 13.124.112.64, 54.180.67.191	2406:da12:21d:9308::17, 2406:da12:21d:9305::17, 2406:da12:21d:9302::17
Australia (Sydney) - Azure (**cf.ap20.hana.ondemand.com**) **New domain (as of November 15, 2026):** **(ap20.uc.platform.cloud.sap)**	connectivitynotification.cf.ap20.hana.ondemand.com	20.53.99.41 Additional IP address (as of January 19, 2025): 20.227.99.35	

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
	connectivitycertsigning.cf.ap20.hana.ondemand.com	`20.53.99.41` Additional IP address (as of January 19, 2025): 20.227.99.35	
	connectivitytunnel.cf.ap20.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.ap20.uc.platform.cloud.sap**	`20.53.99.41` Additional IP address (as of January 19, 2025): 20.227.102.91	
Singapore - Azure (`cf.ap21.hana.ondemand.com`) **New domain (as of November 15, 2026):** **(ap21.uc.platform.cloud.sap)** *Enterprise & Trial*	connectivitynotification.cf.ap21.hana.ondemand.com	`20.184.61.122` Additional IP address (as of January 19, 2025): 57.155.80.35	
	connectivitycertsigning.cf.ap21.hana.ondemand.com	`20.184.61.122` Additional IP address (as of January 19, 2025): 57.155.80.35	
	connectivitytunnel.cf.ap21.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.ap21.uc.platform.cloud.sap**	`20.184.61.122` Additional IP address (as of January 19, 2025): 57.155.80.232	
Australia (Sydney) - Google Cloud (`cf.ap30.hana.ondemand.com`) **New domain (as of November 15, 2026):** **(ap30.uc.platform.cloud.sap)**	connectivitynotification.cf.ap30.hana.ondemand.com	`35.244.71.16` Additional IP address (as of April 2026): 34.151.136.86	2600:1900:40b0:eb:8000:2::
	connectivitycertsigning.cf.ap30.hana.ondemand.com	`35.244.71.16` Additional IP address (as of April 2026): 34.151.136.86	2600:1900:40b0:eb:8000:2::

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
	connectivitytunnel.cf.ap30.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.ap30.uc.platform.cloud.sap**	`35.244.71.16` Additional IP address (as of April 2026): 35.201.15.255	2600:1900:40b0:f2b2:8000:1::
Canada (Montreal) - AWS (`cf.ca10.hana.ondemand.com`) **New domain (as of November 15, 2026):** **(ca10.uc.platform.cloud.sap)**	connectivitynotification.cf.ca10.hana.ondemand.com	`3.98.102.153`, `35.182.75.101`, `35.183.74.34`, `15.157.88.166`, `3.98.202.222`, `52.60.210.33` Additional IP addresses (as of January 19, 2025): 15.157.235.215, 15.222.67.204, 3.96.123.72	2600:1f11:b90:f908::17, 2600:1f11:b90:f905::17, 2600:1f11:b90:f902::17
	connectivitycertsigning.cf.ca10.hana.ondemand.com	`3.98.102.153`, `35.182.75.101`, `35.183.74.34`, `15.157.88.166`, `3.98.202.222`, `52.60.210.33` Additional IP addresses (as of January 19, 2025): 15.157.235.215, 15.222.67.204, 3.96.123.72	2600:1f11:b90:f908::17, 2600:1f11:b90:f905::17, 2600:1f11:b90:f902::17
	connectivitytunnel.cf.ca10.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.ca10.uc.platform.cloud.sap**	`3.98.102.153`, `35.182.75.101`, `35.183.74.34`, `15.157.88.166`, `3.98.202.222`, `52.60.210.33` Additional IP addresses (as of January 19, 2025): 3.97.216.231, 15.156.32.232, 52.60.236.134	2600:1f11:f08:3302::17, 2600:1f11:f08:3305::17, 2600:1f11:f08:3308::17

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
Canada Central (Toronto) - Azure (**cf.ca20.hana.ondemand.com**) **New domain (as of November 15, 2026):** **(ca20.uc.platform.cloud.sap)**	connectivitynotification.cf.ca20.hana.ondemand.com	4.172.17.29, 130.107.188.238	
	connectivitycertsigning.cf.ca20.hana.ondemand.com	4.172.17.29, 130.107.189.169	
	connectivitytunnel.cf.ca20.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.ca20.uc.platform.cloud.sap**	4.172.17.29, 130.107.189.169	
Switzerland (Zurich) - Azure (**cf.ch20.hana.ondemand.com**) **New domain (as of November 15, 2026):** **(ch20.uc.platform.cloud.sap)**	connectivitynotification.cf.ch20.hana.ondemand.com	20.208.56.83 Additional IP address (as of January 19, 2025): 74.242.192.152	
	connectivitycertsigning.cf.ch20.hana.ondemand.com	20.208.56.83 Additional IP address (as of January 19, 2025): 74.242.192.152	
	connectivitytunnel.cf.ch20.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.ch20.uc.platform.cloud.sap**	20.208.56.83 Additional IP address (as of January 19, 2025): 74.242.193.142	
India (Mumbai) - Google Cloud (**cf.in30.hana.ondemand.com**) **New domain (as of November 15, 2026):** **(in30.uc.platform.cloud.sap)**	connectivitynotification.cf.in30.hana.ondemand.com	34.93.125.74 Additional IP address (as of April 2026): 34.100.167.221	2600:1900:40a0:305:8000:2::
	connectivitycertsigning.cf.in30.hana.ondemand.com	34.93.125.74 Additional IP address (as of April 2026): 34.100.167.221	2600:1900:40a0:305:8000:2::

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
	connectivitytunnel.cf.in30.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.in30.uc.platform.cloud.sap**	`34.93.125.74` Additional IP address (as of April 2026): 34.14.155.31	2600:1900:40a0:151:8000:1::
Israel (Tel Aviv) - Google Cloud (**`cf.il30.hana.ondemand.com`**) **New domain (as of November 15, 2026):** **(il30.uc.platform.cloud.sap)**	connectivitynotification.cf.il30.hana.ondemand.com	`34.165.59.26` Additional IP address (as of April 2026): 34.165.53.182	2600:1901:8160:ac3:8000:2::
	connectivitycertsigning.cf.il30.hana.ondemand.com	`34.165.59.26` Additional IP address (as of April 2026): 34.165.53.182	2600:1901:8160:ac3:8000:2::
	connectivitytunnel.cf.il30.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.il30.uc.platform.cloud.sap**	`34.165.59.26` Additional IP address (as of April 2026): 34.165.135.0	2600:1901:8160:24b:8000:1::
China North - Azure (**`cf.cn20.hana.ondemand.com`**)	connectivitynotification.cf.cn20.hana.ondemand.com	`155.56.210.43`	
	connectivitycertsigning.cf.cn20.hana.ondemand.com	`155.56.210.43`	
	connectivitytunnel.cf.cn20.hana.ondemand.com	`155.56.210.43`	
China (Shanghai) - Alibaba Cloud (**`cf.cn40.platform.sapcloud.cn`**)	connectivitynotification.cf.cn40.platform.sapcloud.cn	`139.224.7.71`	
	connectivitycertsigning.cf.cn40.platform.sapcloud.cn	`139.224.7.71`	
	connectivitytunnel.cf.cn40.platform.sapcloud.cn	`139.224.7.71`	

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
KSA (Dammam) - Google Cloud (**cf.sa30.hana.ondemand.com**) **New domain (as of November 15, 2026):** **(sa30.uc.platform.cloud.sap)**	connectivitynotification.cf.sa30.hana.ondemand.com	34.166.32.46 Additional IP address (as of April 2026): 34.166.253.135	2600:1900:5400:775:8000:2::
	connectivitycertsigning.cf.sa30.hana.ondemand.com	34.166.32.46 Additional IP address (as of April 2026): 34.166.253.135	2600:1900:5400:775:8000:2::
	connectivitytunnel.cf.sa30.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.sa30.uc.platform.cloud.sap**	34.166.32.46 Additional IP address (as of April 2026): 34.166.158.8	2600:1900:5400:640:8000:2::
KSA (Dammam) - Google Cloud (**cf.sa31.hana.ondemand.com**) **New domain (as of November 15, 2026):** **(sa31.uc.platform.cloud.sap)**	connectivitynotification.cf.sa31.hana.ondemand.com	34.166.72.122 Additional IP address (as of April 2026): 34.166.211.2	2600:1900:5400:4b8:8000:2::
	connectivitycertsigning.cf.sa31.hana.ondemand.com	34.166.72.122 Additional IP address (as of April 2026): 34.166.211.2	2600:1900:5400:4b8:8000:2::
	connectivitytunnel.cf.sa31.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.sa31.uc.platform.cloud.sap**	34.166.72.122 Additional IP address (as of April 2026): 34.166.122.254	2600:1900:5400:1a4:8000:1::
UAE (Dubai) - SAP (**cf.ae01.hana.ondemand.com**) (mtls.ae01-l-c.uc-live.shoot.live.k8s-hana.ondemand.com)	connectivitynotification.mtls.ae01-l-c.uc-live.shoot.live.k8s-hana.ondemand.com	130.214.0.25	
	connectivitycertsigning.mtls.ae01-l-c.uc-live.shoot.live.k8s-hana.ondemand.com	130.214.0.25	

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
Note Available as of October 31, 2025 **New domain (as of November 15, 2026):** **(ae01.uc.plat-form.cloud.sap)**	connectivitytunnel.ae01-l-d.uc-live.shoot.live.k8s-hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.ae01.uc.plat-form.cloud.sap**	`130.214.0.22`	

Back to Network [page 1613]

Back to Content [page 1610]

ABAP Environment

Note

To enable the full scenario for the SAP BTP ABAP environment, the DNS names below are needed *in addition* to the IPs of the Connectivity service mentioned above. If you can only configure allowlists for *IP addresses* in your firewall, please note that these addresses may change at any time and that you need to validate the values with a DNS service of your choice regularly.

For more information, see Regions and API Endpoints for the ABAP Environment.

Region (Region Host)	Hosts	IP Addresses
Europe (Frankfurt) - AWS	*.abap.eu10.hana.ondemand.com	<dynamic>
Europe (Frankfurt) EU Access - AWS	*.abap.eu11.hana.ondemand.com	<dynamic>
Europe (Netherlands) - Azure	*.abap.eu20.hana.ondemand.com	<dynamic>
US East (VA) - AWS	*.abap.us10.hana.ondemand.com	<dynamic>
US East (VA) - Azure	*.abap.us21.hana.ondemand.com	<dynamic>
US West (WA) - Azure	*.abap.us20.hana.ondemand.com	<dynamic>
Brazil (São Paulo) - AWS	*.abap.br10.hana.ondemand.com	<dynamic>
Japan (Tokyo) - AWS	*.abap.jp10.hana.ondemand.com	<dynamic>
Japan (Tokyo) - Azure	*.abap.jp20.hana.ondemand.com	<dynamic>
Australia (Sydney) - AWS	*.abap.ap10.hana.ondemand.com	<dynamic>

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
Australia (Sydney) - Azure	*.abap.ap20.hana.ondemand.com	<dynamic>	
Asia Pacific (Singapore) - AWS	*.abap.ap11.hana.ondemand.com	<dynamic>	
Asia Pacific (Seoul) - AWS	*.abap.ap12.hana.ondemand.com	<dynamic>	
Singapore - Azure	*.abap.ap21.hana.ondemand.com	<dynamic>	
Canada (Montreal) - AWS	*.abap.ca10.hana.ondemand.com	<dynamic>	
Switzerland (Zurich) - Azure	*.abap.ch20.hana.ondemand.com	<dynamic>	

Back to Network [page 1613]

Back to Content [page 1610]

Neo Environment		
Region (Region Host)	**Hosts**	**IP Addresses**
Europe (Rot) (**hana.ondemand.com**) (**eu1.hana.ondemand.com**)	connectivitynotification.hana.ondemand.com	155.56.210.83
	connectivitycertsigning.hana.ondemand.com	155.56.210.43
	connectivitytunnel.hana.ondemand.com	155.56.210.84
Europe (Frankfurt) (**eu2.hana.ondemand.com**)	connectivitynotification.eu2.hana.ondemand.com	157.133.206.143
	connectivitycertsigning.eu2.hana.ondemand.com	157.133.205.174
	connectivitytunnel.eu2.hana.ondemand.com	157.133.205.233
Europe (Amsterdam) (**eu3.hana.ondemand.com**)	connectivitynotification.eu3.hana.ondemand.com	130.214.87.72
	connectivitycertsigning.eu3.hana.ondemand.com	130.214.87.76
	connectivitytunnel.eu3.hana.ondemand.com	130.214.86.212
United States East (Ashburn)	connectivitynotification.us1.hana.ondemand.com	130.214.181.204
	connectivitycertsigning.us1.hana.ondemand.com	130.214.181.48

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
(**us1.hana.ondemand.com**)	connectivitytunnel.us1.hana.ondemand.com	130.214.181.134	
United States West (Chandler) (**us2.hana.ondemand.com**)	connectivitynotification.us2.hana.ondemand.com	130.214.129.127	
	connectivitycertsigning.us2.hana.ondemand.com	130.214.129.140	
	connectivitytunnel.us2.hana.ondemand.com	130.214.129.88	
United States East (Sterling) (**us3.hana.ondemand.com**)	connectivitynotification.us3.hana.ondemand.com	130.214.32.144	
	connectivitycertsigning.us3.hana.ondemand.com	130.214.33.92	
	connectivitytunnel.us3.hana.ondemand.com	130.214.33.119	
US States West (Colorado Springs) (**us4.hana.ondemand.com**)	connectivitynotification.us4.hana.ondemand.com	130.214.183.46	
	connectivitycertsigning.us4.hana.ondemand.com	130.214.183.14	
	connectivitytunnel.us4.hana.ondemand.com	130.214.183.103	
Australia (Sydney) (**ap1.hana.ondemand.com**)	connectivitynotification.ap1.hana.ondemand.com	157.133.97.47	
	connectivitycertsigning.ap1.hana.ondemand.com	157.133.97.27	
	connectivitytunnel.ap1.hana.ondemand.com	157.133.97.46	
China (Shanghai) (**cn1.platform.sapcloud.cn**)	connectivitynotification.cn1.platform.sapcloud.cn	121.91.109.81	
	connectivitycertsigning.cn1.platform.sapcloud.cn	121.91.109.77	
	connectivitytunnel.cn1.platform.sapcloud.cn	121.91.109.82	
Japan (Tokyo) (**jp1.hana.ondemand.com**)	connectivitynotification.jp1.hana.ondemand.com	130.214.112.168	
	connectivitycertsigning.jp1.hana.ondemand.com	130.214.112.212	
	connectivitytunnel.jp1.hana.ondemand.com	130.214.112.164	

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
Canada (Toronto) (**ca1.hana.ondemand.com**)	connectivitynotification.ca1.hana.ondemand.com	130.214.174.201	
	connectivitycertsigning.ca1.hana.ondemand.com	130.214.174.220	
	connectivitytunnel.ca1.hana.ondemand.com	130.214.174.236	
Brazil (São Paulo) (**br1.hana.ondemand.com**)	connectivitynotification.br1.hana.ondemand.com	130.214.96.193	
	connectivitycertsigning.br1.hana.ondemand.com	130.214.96.195	
	connectivitytunnel.br1.hana.ondemand.com	130.214.96.173	
UAE (Dubai) (**ae1.hana.ondemand.com**)	connectivitynotification.ae1.hana.ondemand.com	130.214.80.206	
	connectivitycertsigning.ae1.hana.ondemand.com	130.214.80.208	
	connectivitytunnel.ae1.hana.ondemand.com	130.214.80.182	
KSA (Riyadh) (**sa1.hana.ondemand.com**)	connectivitynotification.sa1.hana.ondemand.com	130.214.209.241	
	connectivitycertsigning.sa1.hana.ondemand.com	130.214.209.207	
	connectivitytunnel.sa1.hana.ondemand.com	130.214.209.191	

Back to Network [page 1613]

Back to Content [page 1610]

Trial (Cloud Foundry Environment)

> **Note**
>
> In the Cloud Foundry environment, IPs are controlled by the respective IaaS provider (AWS, Azure, or Google Cloud). IPs may change due to network updates on the provider side. Any planned changes will be announced several weeks before they take effect. See also Regions.

Region (Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
Europe (Frankfurt) - AWS (**cf.eu10.hana.ondemand.com**) **New domain (as of November 15, 2026):** **(eu10.uc.platform.cloud.sap)**	connectivitynotification.cf.eu10.hana.ondemand.com	3.124.222.77, 3.122.209.241, 3.124.208.223, 18.159.31.22, 3.69.186.98, 3.77.195.119 Additional IP addresses (as of January 19, 2025): 3.123.247.156, 18.184.184.134, 3.76.166.75	2a05:d014:ca2:3702::17, 2a05:d014:ca2:3705::17, 2a05:d014:ca2:3708::17
	connectivitycertsigning.cf.eu10.hana.ondemand.com	3.124.222.77, 3.122.209.241, 3.124.208.223, 18.159.31.22, 3.69.186.98, 3.77.195.119 Additional IP addresses (as of January 19, 2025): 3.123.247.156, 18.184.184.134, 3.76.166.75	2a05:d014:ca2:3702::17, 2a05:d014:ca2:3705::17, 2a05:d014:ca2:3708::17
	connectivitytunnel.cf.eu10.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.eu10.uc.platform.cloud.sap**	3.124.222.77, 3.122.209.241, 3.124.208.223, 18.159.31.22, 3.69.186.98, 3.77.195.119 Additional IP addresses (as of January 19, 2025): 35.157.237.81, 52.58.116.80, 18.196.77.73	2a05:d014:e0d:f902::17, 2a05:d014:e0d:f908::17, 2a05:d014:e0d:f905::17

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
United States East (VA) - AWS (**cf.us10.hana.ondemand.com**) **New domain (as of November 15, 2026):** **(us10.uc.platform.cloud.sap)**	connectivitynotification.cf.us10.hana.ondemand.com	52.23.189.23, 52.4.101.240, 52.23.1.211, 18.213.242.208, 3.214.110.153, 34.205.56.51 Additional IP addresses (as of January 19, 2025): 98.82.227.152, 107.20.252.91, 34.198.33.88	2600:1f18:ef:9b05::17, 2600:1f18:ef:9b08::17, 2600:1f18:ef:9b02::17 New (as of May 10, 2026): 2600:1f18:ef:9b08::17, 2600:1f18:ef:9b0b::17, 2600:1f18:ef:9b02::17
	connectivitycertsigning.cf.us10.hana.ondemand.com	52.23.189.23, 52.4.101.240, 52.23.1.211, 18.213.242.208, 3.214.110.153, 34.205.56.51 Additional IP addresses (as of January 19, 2025): 98.82.227.152, 107.20.252.91, 34.198.33.88	2600:1f18:ef:9b05::17, 2600:1f18:ef:9b08::17, 2600:1f18:ef:9b02::17 New (as of May 10, 2026): 2600:1f18:ef:9b08::17, 2600:1f18:ef:9b0b::17, 2600:1f18:ef:9b02::17
	connectivitytunnel.cf.us10.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.us10.uc.platform.cloud.sap**	52.23.189.23, 52.4.101.240, 52.23.1.211, 18.213.242.208, 3.214.110.153, 34.205.56.51 Additional IP addresses (as of January 19, 2025): 54.147.47.110, 18.214.162.12, 44.206.11.12	2600:1f18:12b5:c605::17, 2600:1f18:12b5:c602::17, 2600:1f18:12b5:c608::17 New (as of May 10, 2026): 2600:1f18:12b5:c60b::17, 2600:1f18:12b5:c602::17, 2600:1f18:12b5:c608::17
Singapore - Azure (**cf.ap21.hana.ondemand.com**)	connectivitynotification.cf.ap21.hana.ondemand.com	20.184.61.122	
	connectivitycertsigning.cf.ap21.hana.ondemand.com	20.184.61.122	

Region (Region Host) (Connectivity Region Host)	Hosts	IP Addresses (IPv4)	IP Addresses (IPv6)
New domain (as of November 15, 2026): **(ap21.uc.platform.cloud.sap)**	connectivitytunnel.cf.ap21.hana.ondemand.com **New host (as of November 15, 2026):** **connectivitytunnel.ap21.uc.platform.cloud.sap**	`20.184.61.122`	

Back to Network [page 1613]

Back to Content [page 1610]

> ### Note
>
> If you install the Cloud Connector in a network segment that is isolated from the backend systems, make sure the exposed hosts and ports are still reachable and open them in the firewall that protects them:
>
> - for HTTP, the ports you chose for the HTTP/S server.
> - for LDAP, the port of the LDAP server.
> - for RFC, it depends on whether you use an SAProuter or not and whether load balancing is used:
>   - if you use an SAProuter, it is typically configured as visible in the network of the Cloud Connector and the corresponding *routtab* is exposing all the systems that should be used.
>   - without SAProuter, you must open the application server hosts and the corresponding gateway ports (33##, 48##). When using load balancing for the connection, you must also open the message server host and port.
>
> See also Network Zones [page 1648].

> ### Note
>
> For more information about the used ABAP server ports, see: Ports of SAP NetWeaver Application Server ABAP.
>
> For more information about additional IP addresses for SAP Business Application Studio, see SAP Business Application Studio Inbound IP Addresses.

Back to Network [page 1613]

Back to Content [page 1610]

Related Information

Installation on Microsoft Windows OS [page 1657]

Installation on Linux OS [page 1660]
Installation on Apple macOS [page 1663]
Recommendations for Secure Setup [page 1665]
Sizing Recommendations [page 1649]

1.4.1.1.1 System Requirements

Additional system requirements for installing and running the Cloud Connector.

Hardware Requirements

For an up-to-date list of hardware requirements, see Hardware [page 1610].

Software Requirements

For an up-to-date list with detailed Cloud Connector version information, see Prerequisites [page 1610].

Supported Browsers

The browsers that can be used for the Cloud Connector administration UI are the ones supported by SAP UI5. Currently, these are the following:

- Microsoft Edge (latest version)
- Mozilla Firefox (latest version and latest Extended Support Release (ESR))
- Safari 5.1 and higher (latest version)
- Google Chrome (latest versions)

For an up-to-date list of the supported SAP UI5 browsers, see:Browser and Platform Support.

Minimum Disk Space for Download and Installation

The minimum free disk space required to download and install a new Cloud Connector server is as follows:

- Size of downloaded Cloud Connector installation file (ZIP, TAR, MSI files): 120 MB
- Newly installed Cloud Connector server: 235 MB

- Total: 355 MB as a minimum

Additional Disk Space for Log and Configuration Files

The Cloud Connector writes configuration files, audit log files and trace files at runtime. We recommend that you reserve between 1 and 20 GB of disk space for those files.

Trace and log files are written to `<scc_dir>/log/` within the Cloud Connector `root` directory. The `scc_core.trc` file contains traces in general, communication payload traces are stored in `tunnel_traffic_*.trc` and `snc_traffic_*.trc`. These files may be used by SAP Support to analyze potential issues. The default trace level is `Information`, where the amount of written data is generally only a few KB per day. You can turn off these traces to save disk space. However, we recommend that you don't turn off this trace completely, but that you leave it at the default settings, to allow root cause analysis if an issue occurs. If you set the trace level to `All`, the amount of data can easily reach the range of several GB per day. Use trace level `All` only to analyze a specific issue. *Tunnel Traffic Trace* and *ABAP Cloud SNC Traffic Trace*, however, should normally be turned off, and used only for analysis by SAP Support.

> **Note**
>
> From an operations perspective, we recommend that you back up or delete written trace files regularly in order to clean up the used disk space.

Audit log files are written to `/log/audit/<subaccount-name>/audit-log_<subaccount-name>_<date>.csv` within the Cloud Connector root directory. By default, only security-related events are written in the audit log. You can change the audit log level using the administration UI, as described in Manage Audit Logs [page 2042].

To be compliant with the regulatory requirements of your organization and the regional laws, the audit log files must be persisted for a certain period of time for traceability purposes. Therefore, we recommend that you back up the audit log files regularly from the Cloud Connector file system and keep the backup for the length of time required.

Related Information

Prerequisites [page 1610]

1.4.1.1.2 Network Zones

Choose a network zone for your Cloud Connector installation.

A customer network is usually divided into multiple network zones or subnetworks according to the security level of the contained components. For example, the DMZ that contains and exposes the external-facing services of an organization to an untrusted network, usually the Internet, and there are one or more other network zones which contain the components and services provided in the company's intranet.

You can generally choose the network zone in which to set up the Cloud Connector:

- Internet access to the SAP BTP region host, either directly or via HTTPS proxy.
- Direct access to the internal systems it provides access to, which means that there is transparent connectivity between the Cloud Connector and the internal system.

The Cloud Connector can be set up either in the DMZ and operated centrally by the IT department, or set up in the intranet and operated by the appropriate line of business.

> ⓘ Note
>
> The internal network must allow access to the required ports; the specific configuration depends on the firewall software used.
>
> The default ports are **`80`** for HTTP and **`443`** for HTTPS. For RFC communication, you need to open a gateway port (default: **`33+<instance number>`** and an arbitrary message server port. For a connection to a HANA Database (on SAP BTP) via JDBC, you need to open an arbitrary *outbound* port in your network. Mail (SMTP) communication is not supported.

1.4.1.2 Sizing Recommendations

When installing a Cloud Connector, the first thing you need to decide is the sizing of the installation.

This section gives some basic guidance what to consider for this decision. The provided information includes the shadow instance, which should always be added in productive setups. See also Install a Failover Instance for High Availability [page 1996].

> ⓘ Note
>
> The following recommendations are based on current experiences. However, they are only a rule of thumb since the actual performance strongly depends on the specific environment. The overall performance of a Cloud Connector is impacted by many factors (number of hosted subaccounts, bandwidth, latency to the attached regions, network routers in the corporate network, used JVM, and others).

Restrictions

The sizing data refer to a single Cloud Connector installation.

> ⓘ Note
>
> Up until now, you cannot perform horizontal scaling directly. However, you can distribute the load statically by operating multiple Cloud Connector installations with different location IDs for all involved subaccounts. In this scenario, you can use multiple destinations with virtually the same configuration, except for the location ID. See also Adding and Managing Subaccounts [page 1682], step 4. Alternatively, each of the Cloud Connector instances can host its own list of subaccounts without any overlap in the respective lists. Thus, you can handle more load, if a single installation risks to be overloaded.

Related Information

Hardware Setup [page 1650]
Configuration Setup [page 1654]

1.4.1.2.1 Hardware Setup

How to choose the right sizing for your Cloud Connector installation.

Regarding the hardware, we recommend that you use different setups for master and shadow. One dedicated machine should be used for the master, another one for the shadow. Usually, a shadow instance takes over the master role only temporarily. During most of its lifetime, in the shadow state, it needs less resources compared to the master.

If the master instance is available again after a downtime, we recommend that you switch back to the actual master.

> ⓘ Note
>
> The sizing recommendations refer to the overall load across all subaccounts that are connected via the Cloud Connector. This means that you need to accumulate the expected load of all subaccounts and should not only calculate separately per subaccount (taking the one with the highest load as basis).

Related Information

Sizing for the Master Instance [page 1650]
Sizing for the Shadow Instance [page 1653]

1.4.1.2.1.1 Sizing for the Master Instance

Learn more about the basic criteria for the sizing of your Cloud Connector master instance.

For the master setup, keep in mind the expected load for communication between the SAP BTP and on-premises systems. The setups listed below differ in a mostly qualitative manner, without hard limits for each of them.

> ⓘ Note
>
> The mentioned sizes are considered as minimal configuration, larger ones are always ok. In general, the more applications, application instances, and subaccounts are connected, the more competition will exist for the limited resources on the machine.

Installation Size (S, M, L, XL)	CPU (x86_64)	Machine Memory / Heap / Direct Memory	Disk Space
S: The expected load is small (up to **1 million requests** per day, request concurrency and size is in average low) and only a **few subaccounts** with **some applications** are connected. In addition, only a **few service channels** are used, only **small data amount is replicated** to cloud systems. Setup can be a done in a virtual or physical machine.	2 cores 2.6 GHz	4GB RAM / 1GB / 2GB	10GB
M : The expected load is medium (up to **10 million requests** per day, request concurrency and size is in average medium) and **multiple subaccounts** with **multiple applications** are connected. In addition, **many service channels** are used, and a **medium data amount is replicated** to cloud systems. We recommend that you do the setup in a virtual or physical machine. A virtual machine should be on a host without overcommitted resources.	4 cores 3.0 Ghz	16GB RAM / 4GB / 8GB	20GB

Installation Size (S, M, L, XL)	CPU (x86_64)	Machine Memory / Heap / Direct Memory	Disk Space
L: The expected load is large (**more than 10 million requests** per day, request concurrency and size is in average medium or high) and multiple subaccounts with multiple applications are connected. In addition, many service channels are used, and a **large data amount is replicated** to cloud systems concurrently. We recommend that you do the setup in a virtual or physical machine. A virtual machine should be on a host without overcommitted resources.	8 cores 3.0 Ghz	32GB RAM / 8GB / 16GB	40GB

Installation Size (S, M, L, XL)	CPU (x86_64)	Machine Memory / Heap / Direct Memory	Disk Space
XL: The expected load is large (**more than 10 million requests** per day, request concurrency and size is mostly medium or high) and multiple subaccounts with multiple applications are connected. On some of them, **high traffic load** is expected. In addition, many service channels are used, and a **large data amount is replicated** to cloud systems concurrently. We recommend that you do the setup in a virtual or physical machine. A virtual machine should be on a host without overcommitted resources.	16 cores 3.0 Ghz	64GB RAM / 16GB / 32GB 10GBit/s network	40GB

Particularly the **heap size** is critical. If you size it too low for the load passing the Cloud Connector, at some point the Java Virtual Machine will execute full GCs (garbage collections) more frequently, blocking the processing of the Cloud Connector completely for multiple seconds, which massively slows down overall performance. If you experience such situations regularly, you should increase the heap size in the Cloud Connector UI (choose *Configuration* > *Advanced* > *JVM*). See also Configure the Java VM [page 1963].

> **Note**
>
> You should use the same value for both `<initial heap size>` and `<maximum heap size>`.

1.4.1.2.1.2 Sizing for the Shadow Instance

Learn more about the basic criteria for the sizing of your Cloud Connector shadow instance (high availability mode).

The shadow installation is typically not used in standard situations and hence does not need the same sizing, assuming that the time span in which it takes over the master role is limited.

> **Note**
>
> The shadow only acts as master, for example, during an upgrade or when an abnormal situation occurs on the master machine, and either the Cloud Connector or the full machine on OS level needs to be restarted.

While being in the shadow state, the resource consumption is very low, especially in productive environments, where typically only few configuration changes are required. Therefore, the machine sizing can usually be smaller than the one for the master. However, if you want to mitigate the risk of a longer outage of the master machine, you should increase the sizing of the shadow up to the master size:

Master	Shadow
S	**S** installation as virtual machine
M	**S** installation (with double memory) as virtual or physical machine **M** installation as virtual or physical machine for risk mitigation
L	**M** installation as virtual or physical machine **L** installation as virtual or physical machine for risk mitigation
XL	**L** installation as virtual or physical machine **XL** installation as virtual or physical machine for risk mitigation

1.4.1.2.2 Configuration Setup

Choose the right connection configuration options to improve the performance of the Cloud Connector.

This section provides detailed information how you can adjust the configuration to improve overall performance. This is typically relevant for an M or L installation (see Hardware Setup [page 1650]). For S installations, the default configuration will probably be sufficient to handle the traffic.

To change the connection parameters, proceed as follows:

- You can configure the number of physical connections through the Cloud Connector UI. See also Configure Advanced Connectivity [page 1962].
- In versions prior to 2.11, you have to modify the configuration files with an editor and restart the Cloud Connector to activate the changes.

In general, the Cloud Connector tunnel is multiplexing multiple virtual connections over a single physical connection. Thus, a single connection can handle a considerable amount of traffic. However, increasing the maximum number of physical connections allows you to make use of the full available bandwidth and to minimize latency effects.

If the bandwidth limit of your network is reached, adding additional connections doesn't increase the throughput, but will only consume more resources.

> **Note**
>
> Different network access parameters may impact and limit your configuration options: if the access to an external network is a 1 MB line with an added latency of 50 ms, you will not be able to achieve the same data volumes like with a 10 GB line with an added latency of < 1 ms. However, even if the line is good, for example 10 GB, but with an added latency of 100 ms, the performance might still be bad.

Optimal configuration strongly depends on your actual scenarios. A good approach is to try out different settings, if the current performance does not meet your expectations.

Related Information

Cloud-To-On-Premises Connections [page 1655]
On-Premises-To-Cloud Connections (Service Channels) [page 1656]

1.4.1.2.2.1 Cloud-To-On-Premises Connections

Configure the physical connections for cloud-to-on-premises calls in the Cloud Connector.

Adjusting the number of physical connections for this direction is possible both globally in the Cloud Connector UI (*Configuration* > *Advanced*), and for individual communication partners on cloud side (*Cloud To On-Premises* > *Applications*).

Connections are established for each defined and connected subaccount. The current number of opened connections is visible in the Cloud Connector UI via *<Subaccount>* > *Cloud Connections*.

The global default is 1 physical connection per connected subaccount. This value is used across all subaccounts hosted by the Cloud Connector instance and applies for all communication partners.

In general, the default should be sufficient for applications with low traffic. If you expect medium traffic for most applications, it may be useful to set the default value to 2.

> **Note**
>
> An exact traffic forecast is difficult to achieve. It requires a deep understanding of the use case and of the possible future load generated by different applications. For this reason, we recommend that you focus on subsequent configuration adjustments, using the Cloud Connector monitoring tools to recognize bottlenecks in time, and adjust Cloud Connector configuration accordingly.

Tunnel Worker Threads

In addition to the number of connections, you can configure the number of `<Tunnel Worker Threads>`. This value should be at least equal to the maximum of all individual application tunnel connections in all subaccounts, to have at least 1 thread available for each connection that can process incoming requests and outgoing responses.

Protocol Processor Worker Threads

The value for `<Protocol Processor Worker Threads>` is mainly relevant if RFC is used as protocol. Since its communication model towards the ABAP system is a blocking one, each thread can handle only one call at a time and cannot be shared. Hence, you should provide 1 thread per 5 concurrent RFC requests.

> **Note**
>
> The longer the RFC execution time in the backend, the more threads you should provide. Threads can be reused only after the response of a call was returned to SAP BTP.

Related Information

Configure Advanced Connectivity [page 1962]

1.4.1.2.2.2 On-Premises-To-Cloud Connections (Service Channels)

Configure the number of physical connections for a Cloud Connector service channel.

Service channels let you configure the number of physical connections to the communication partner on cloud side, see Using Service Channels [page 1949]. The default is 1. This value is used as well in versions prior to Cloud Connector 2.11, which did not offer a configuration option for each service channel. You should define the number of connections depending on the expected number of clients and, with lower priority, depending on the size of the exchanged messages.

If there is only a single RFC client for an S/4HANA Cloud channel or only a single HANA client for a HANA DB on SAP BTP side, increasing the number doesn't help, as each virtual connection is assigned to one physical connection. The following simple rule lets you to define the required number of connections per service channel:

- Per 10 concurrent clients, use one physical connection.
- If the transferred net data size is larger than 500k per request, make sure to add an additional connection per 2 of such clients.

Example

For a HANA system in the SAP BTP, data is replicated using 18 concurrent clients in the on-premises network. In average, about 5 of those clients are regularly sending 600k. For the number of clients, you should use 2 physical connections, for the 5 clients sending larger amounts add an additional 3, which sums up to 5 connections.

1.4.1.3 Installation on Microsoft Windows OS

Installing the Cloud Connector on a Microsoft Windows operating system.

Context

You can choose between a simple `portable` variant of the Cloud Connector and the MSI-based `installer`. The `installer` is the generally recommended version that you can use for both developer and productive scenarios. It lets you, for example, register the Cloud Connector as a Windows service and this way automatically start it after machine reboot.

> → Tip
>
> If you are a developer, you might want to use the `portable` variant as you can run the Cloud Connector after a simple unzip (archive extraction). You might want to use it also if you cannot perform a full installation due to lack of permissions, or if you want to use multiple versions of the Cloud Connector simultaneously on the same machine.

Prerequisites

- You have one of the supported 64-bit operating systems. For more information, see Product Availability Matrix [page 1612].
- You have downloaded either the `portable` variant as ZIP archive for Windows, or the MSI `installer` from the SAP Development Tools page.
- You must install Microsoft Visual Studio C++ 2019 runtime libraries (download `vc_redist.x64.exe`). For more information, see Microsoft Visual C++ Redistributable latest supported downloads .
- A supported Java version must be installed. For more information, see JDKs [page 1611].
  If you want to use SAP JVM, you can download its installer package from the SAP Development Tools page or a compressed SAR archive from the SAP Software Center.
  Alternatively supported SapMachine JDK versions can be downloaded from the SapMachine homepage.
- When using the `portable` variant, the environment variable `JAVA_HOME` must be set to the Java installation directory, so that the `bin` subdirectory can be found. Alternatively, you can add the relevant `bin` subdirectory to the `PATH` variable.

- If you want to use IPv6 with your Cloud Connector and you run on a dual stack setup, that is, both IPv4 and IPv6 are supported, you need to signal to the JDK that IPv6 should be preferred over IPv4.
  To do this, add `-Djava.net.preferIPv6Addresses=true` next to the other `-D` options in the `props.ini` file.
  For RFC connections to be established with IPv6, in addition the environment variable `SAP_IPv6_ACTIVE` must be set to **`1`** to make it visible for the Cloud Connector process.

Portable Scenario

1. Extract the `<sapcc-<version>-windows-x64.zip>` ZIP file to an arbitrary directory on your local file system.
2. Set the environment variable `<JAVA_HOME>` to the installation directory of the JDK that you want to use to run the Cloud Connector. Alternatively, you can add the `bin` subdirectory of the JDK installation directory to the `<PATH>` environment variable.
3. Go to the Cloud Connector installation directory and start it using the `go.bat` batch file.
4. Continue with the **Next Steps** section.

> ### Note
>
> The Cloud Connector is not started as a service when using the portable variant, and hence will not automatically start after a reboot of your system. Also, the portable version does not support the automatic upgrade procedure.sapcc-<version>-windows-x64.msi

Installer Scenario

1. Start the <sapcc-<version>-windows-x64.msi> installer by double-clicking it.
2. The installer informs you that you are now guided through the installation process. Choose *Next>*.
3. Navigate to the desired installation directory for your Cloud Connector and choose *Next>*. When doing the installation in the context of an upgrade, make sure you choose the previous installation directory again.
4. You can choose the port on which the administration UI is reachable. Either leave the default **`8443`** or choose a different port if needed. Then, choose *Next>*.
5. Select the JDK to be used for running the Cloud Connector. The installer displays a list of all usable JDKs that are installed on your machine. If the needed JDK is not listed in the drop-down box (for example, if it's an SAP JVM that is not registered in the Windows registry upon installation), you can browse to its installation directory and select it. We recommend that you use an up-to-date Java **8** installation to run the Cloud Connector.
6. Decide whether the Cloud Connector should be started immediately after finishing the setup. Then, choose *Next>*.
7. To start the installation, press the *Next>* button again.
8. After successful installation, choose *Close*.
9. Continue with the **Next Steps** section.

> ### Note
>
> The Cloud Connector is started as a Windows service in the productive use case. Therefore, installation requires administration permissions. After installation, manage this service under *Control Panel*

> *Administrative Tools* > *Services*. The service name is `SAP Cloud Connector` (formerly named `Cloud Connector 2.0`). Make sure the service is executed with a user that has limited privileges. Typically, privileges allowed for service users are defined by your company policy. Adjust the folder and file permissions to be manageable by only this user and system administrators.

> ### Note
>
> Using the SAP Cryptographic Library requires the environment variable SECUDIR to be set.
>
> If you want to make the variable visible only for the *Windows Service* of the Cloud Connector, add it to the registry in section `Computer\HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\SAP Cloud Connector` as element of the multi-string value `Environment`.

> ### Note
>
> For IPv6, the environment variable `SAP_IPv6_ACTIVE` must be set to **`1`**. If you want to make the variable visible only for the *Windows Service* of the Cloud Connector, add it to the registry in section `Computer\HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\SAP Cloud Connector` as element of the multi-string value `Environment`.

On Windows, the file `scc_service.log` is created and used by the Microsoft MSI installer (during Cloud Connector installation), and by the `scchost.exe` executable, which registers and runs the Windows service if you install the Cloud Connector as a Windows background job.

This log file is only needed if a problem occurs during Cloud Connector installation, or during creation and start of the Windows service, in which the Cloud Connector is running. You can find the file in the `log` folder of your Cloud Connector installation directory.

Starting the Cloud Connector

After installation, the Cloud Connector is registered as a Windows service that is configured to be started automatically after a system reboot. You can start and stop the service via shortcuts on the desktop ("Start Cloud Connector" and "Stop Cloud Connector"), or by using the `Windows Services` manager and look for the service **`SAP Cloud Connector`**.

Access the Cloud Connector administration UI at **`https://localhost:<port>`**, where the default port is **`8443`** (but this port might have been modified during the installation).

Next Steps

1. Open a browser and enter: **`https://<hostname>:8443`**. `<hostname>` is the host name of the machine on which you have installed the Cloud Connector. If you access the Cloud Connector locally from the same machine, you can simply enter **`localhost`**.
2. Continue with the initial configuration of the Cloud Connector, see Initial Configuration [page 1670].

Related Information

Recommendations for Secure Setup [page 1665]

1.4.1.4 Installation on Linux OS

Installing the Cloud Connector on a Linux operating system.

Context

You can choose between a simple `portable` variant of the Cloud Connector and the RPM-based `installer`. The `installer` is the generally recommended version that you can use for both the developer and the productive scenario. It registers, for example, the Cloud Connector as a daemon service and this way automatically starts it after machine reboot.

> → Tip
>
> If you are a developer, you might want to use the `portable` variant as you can run the Cloud Connector after a simple "`tar -xzof`" execution. You also might want to use it if you cannot perform a full installation due to missing permissions for the operating system, or if you want to use multiple versions of the Cloud Connector simultaneously on the same machine.

Prerequisites

- You have one of the supported 64-bit operating systems. For more information, see Product Availability Matrix [page 1612].
- The supported platforms are `x64`, `arm64`, and `ppc64le`, represented below by the variable `<platform>`. Variable `<arch>` is `x86_64`, `aarch64`, or `ppc64le` respectively.
- You have downloaded either the `portable` variant as `tar.gz` archive for Linux or the RPM `installer` contained in the ZIP for Linux, from the SAP Development Tools page.
- A supported Java version must be installed. For more information, see JDKs [page 1611].
  If you want to use SAP JVM, you can download its installer package from the SAP Development Tools page or a compressed SAR archive from the SAP Software Center.
  If you downloaded the RPM package, use the following command to install it:

  ```
  rpm -i sapjvm-<version>-linux-<platform>.rpm
  ```

  If you want to check the JVM version installed on your system, use the following command:

  ```
  rpm -qa | grep jvm
  ```

  An installed RPM package will be detected by the Cloud Connector and used for its runtime.

Alternatively supported SapMachine JDK versions can be downloaded from the SapMachine homepage.

- When using the **`tar.gz`** archive, the environment variable `<JAVA_HOME>` must be set to the Java installation directory, so that the `bin` subdirectory can be found. Alternatively, you can add the Java installation's `bin` subdirectory to the `<PATH>` variable.
- If you want to use IPv6 with your Cloud Connector and you run on a dual stack setup, that is, both IPv4 and IPv6 are supported, you need to signal to the JDK that IPv6 should be preferred over IPv4.
  To do this, add `-Djava.net.preferIPv6Addresses=true` next to the other `-D` options in the `props.ini` file.
  For RFC connections to be established with IPv6, in addition the environment variable `SAP_IPv6_ACTIVE` must be set to **`1`** to make it visible for the Cloud Connector process.

Portable Scenario

1. Extract the *tar.gz* file to an arbitrary directory on your local file system using the following command:

   ```
   tar -xzof sapcc-<version>-linux-<platform>.tar.gz
   ```

   > **Note**
   >
   > If you use the parameter "o", the extracted files are assigned to the user ID and the group ID of the user who has unpacked the archive. This is the default behavior for users other than the `root` user.

2. Go to this directory and start the Cloud Connector using the `go.sh` script.
3. Continue with the **Next Steps** section.

> **Note**
>
> In this case, the Cloud Connector is not started as a daemon, and therefore will not automatically start after a reboot of your system. Also, the `portable` version does not support the automatic upgrade procedure.

Installer Scenario

1. Extract the *sapcc-<version>-linux-<platform>.zip* archive to an arbitrary directory by using the following the command:

   ```
   unzip sapcc-<version>-linux-<platform>.zip
   ```

2. Go to this directory and install the extracted RPM using the following command. You can perform this step only as a `root` user.

   ```
   rpm -i com.sap.scc-ui-<version>.<arch>.rpm
   ```

3. Continue with the **Next Steps** section.

In the productive case, the Cloud Connector is started as a daemon. If you need to manage the daemon process, execute:

```
System V init distributions: service scc_daemon stop|restart|start|status
```

```
systemd distributions: systemctl stop|restart|start|status scc_daemon
```

> ⚠ Caution
>
> When adjusting the Cloud Connector installation (for example, restoring a backup), make sure the RPM package management is synchronized with such changes. If you simply replace files that do not fit to the information stored in the package management, lifecycle operations (such as upgrade or uninstallation) might fail with errors. Also, the Cloud Connector might get into unrecoverable state.
>
> **Example**: After a file system restore, the system files represent Cloud Connector 2.17.2 but the RPM package management "believes" that version 2.18.1.2 is installed. In this case, commands like **`rpm -U`** and **`rpm -e`** do not work as expected. Furthermore, avoid using the `--force` parameter as it may lead to an unpredictable state with two versions being installed concurrently, which is not supported.

Extending the Daemon

When using SNC for encrypting RFC communication, it might be required to provide some settings, for example, environment variables that must be visible for the Cloud Connector process. To achieve this, you must store a file named `scc_daemon_extension.sh` in the installation directory of the Cloud Connector (`/opt/sap/scc`), containing all commands needed for initialization without a shebang.

Example (SAP Cryptographic Library requires SECUDIR to be set, IPv6 support for RFC connectivity):

> {·} Sample Code
>
> ```
> export SECUDIR=/path/to/pse_directory
> export SAP_IPv6_ACTIVE=1
> ```

To activate it, you must reinstall the daemon.

> ⓘ Note
>
> Make sure `<JAVA_HOME>` is set to the JVM used (in the shell where the command is executed).

Execute the following command:

```
System V init distributions: /opt/sap/scc/daemon.sh reinstall
systemd distributions: /opt/sap/scc/daemon.sh reinstallSystemd
```

The daemon extension will survive Cloud Connector version updates.

Starting the Cloud Connector

After installation via RPM manager, the Cloud Connector process is started automatically and registered as a daemon process, which ensures the automatic restart of the Cloud Connector after a system reboot.

To start, stop, or restart the process explicitly, open a command shell and use the following commands, which require `root` permissions:

```
System V init distributions: service scc_daemon start|stop|restart
systemd distributions: systemctl start|stop|restart scc_daemon
```

Next Steps

1. Open a browser and enter: **`https://<hostname>:8443`**. `<hostname>` is the host name of the machine on which you installed the Cloud Connector.
   If you access the Cloud Connector locally from the same machine, you can simply enter **`localhost`**.
2. Continue with the initial configuration of the Cloud Connector, see Initial Configuration [page 1670].

Related Information

Recommendations for Secure Setup [page 1665]

1.4.1.5 Installation on Apple macOS

Installing the Cloud Connector on an Apple macOS operating system.

Prerequisites

> **Note**
>
> Apple macOS is not supported for productive scenarios. The developer version described below must not be used as productive version.

> **Caution**
>
> There are two different Cloud Connector portable versions available for running on Apple macOS with native support either for Apple M series CPUs based on the *aarch64* architecture, or with native support for INTEL x86 64-bit CPUs based on the *x64* architecture. Make sure you download and use the Cloud Connector version in combination with a JVM version, which both match your used hardware CPU architecture.

- You have one of the supported 64-bit operating systems. For more information, see Product Availability Matrix [page 1612].
- The supported platforms are *aarch64* and *x64*, represented below by the variable `<platform>`.
- You have downloaded the `tar.gz` archive for the developer use case on Apple macOS from the SAP Development Tools page.
- A supported Java version must be installed. For more information, see JDKs [page 1611].
  If you are running Apple macOS on an INTEL x86 64-bit CPU and you want to use SAP JVM, you can download its installer package from the SAP Development Tools page or a compressed SAR archive from the SAP Software Center.
  Alternatively supported SapMachine JDK versions for macOS on aarch64 and x64 processor architectures can be downloaded from the SapMachine homepage.

- Environment variable `<JAVA_HOME>` must be set to the Java installation directory so that the `bin` subdirectory can be found. Alternatively, you can add the Java installation's `bin` subdirectory to the `<PATH>` variable.
- If you want to use IPv6 with your Cloud Connector and you run on a dual stack setup, that is, both IPv4 and IPv6 are supported, you need to signal to the JDK that IPv6 should be preferred over IPv4.
  To do this, add `-Djava.net.preferIPv6Addresses=true` next to the other `-D` options in the `props.ini` file.
  For RFC connections to be established with IPv6, in addition the environment variable `SAP_IPv6_ACTIVE` must be set to **`1`** to make it visible for the Cloud Connector process. The easiest approach is setting it in the bash in which the Cloud Connector will be started with `export SAP_IPv6_ACTIVE=1`.

Procedure

1. Extract the `tar.gz` file to an arbitrary directory on your local file system using the following command:

   ```
   tar -xzof sapcc-<version>-macosx-<platform>.tar.gz
   ```

2. Go to this directory and start Cloud Connector using the `go.sh` script.
3. Continue with the **Next Steps** section.

> ### Note
>
> The Cloud connector is not started as a daemon, and therefore will not automatically start after a reboot of your system. Also, the macOS version of the Cloud Connector does not support the automatic upgrade procedure.

Next Steps

1. Open a browser and enter: **`https://<hostname>:8443`**. `<hostname>` is the host name of the machine on which you installed the Cloud Connector.
   If you access the Cloud Connector locally from the same machine, you can simply enter **`localhost`**.
2. Continue with the initial configuration of the Cloud Connector, see Initial Configuration [page 1670].

Related Information

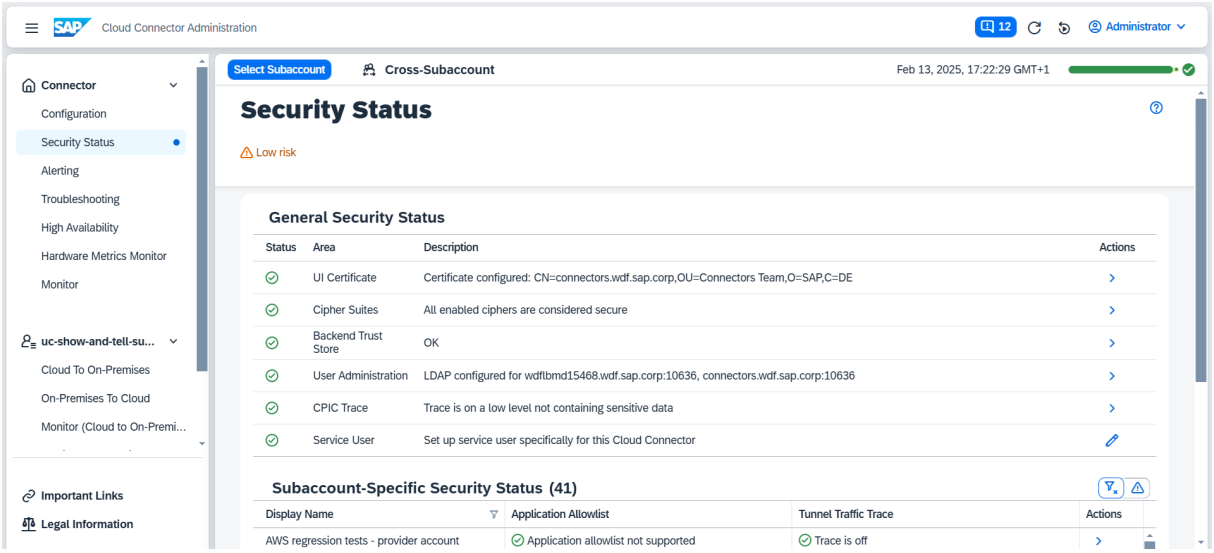
Recommendations for Secure Setup [page 1665]

1.4.1.6 Recommendations for Secure Setup

For the Connectivity service and the Cloud Connector, you should apply the following guidelines to guarantee the highest level of security for these components.

Security Status

From the *Connector* menu, choose *Security Status* to access an overview showing potential security risks and the recommended actions.



The *General Security Status* addresses security topics that are subaccount-independent.

- Choose any of the *Actions* icons in the corresponding line to navigate to the UI area that deals with that particular topic and view or edit details.

  > ### Note
  >
  > Navigation is not possible for the last item in the list (*Service User*).

- The service user is specific to the Windows operating system (see Installation on Microsoft Windows OS [page 1657] for details) and is only visible when running the Cloud Connector on Windows. It cannot be accessed or edited through the UI. If the service user was set up properly, choose *Edit* and check the corresponding checkbox.

The *Subaccount-Specific Security Status* lists security-related information for each and every subaccount.

> ### Note
>
> The security status only serves as a reminder to address security issues and shows if your installation complies with all recommended security settings.

UI Access

Upon installation, the Cloud Connector provides an initial user name and password for the administration UI, and forces the user (`Administrator`) to change the password. You must change the password immediately after installation.

The connector itself does not check the strength of the password. You should select a strong password that cannot be guessed easily.

> **Note**
>
> To enforce your company's password policy, we recommend that you configure the Administration UI to use an LDAP server for authorizing access to the UI.

The default user store is a local file store. It allows only one user, and only the *Administrator* role for this user. Using an LDAP server as user store lets you create various users to access the UI, and assign different roles to them. For more information on available roles, see Use LDAP for User Administration [page 1977].

The Cloud Connector administration UI can be accessed remotely via HTTPS. The connector uses a standard X.509 self-signed certificate as SSL server certificate. You can exchange this certificate with a specific certificate that is trusted by your company. See Exchange UI Certificates in the Administration UI [page 1971].

> **Note**
>
> Since browsers usually do not resolve *localhost* to the host name whereas the certificate usually is created under the host name, you might get a certificate warning. In this case, simply skip the warning message.

OS-Level Access

The Cloud Connector is a security-critical component that handles the external access to systems of an isolated network, comparable to a reverse proxy. We therefore recommend that you restrict the access to the operating system on which the Cloud Connector is installed to the minimal set of users who would administrate the Cloud Connector. This minimizes the risk of unauthorized users getting access to credentials, such as certificates stored in the secure storage of the Cloud Connector.

We also recommend that you use the machine to operate only the Cloud Connector and no other systems.

Administrator Privileges

To log on to the Cloud Connector administration UI, the `Administrator` user of the connector must not have an operating system (OS) user for the machine on which the connector is running. This allows the OS administrator to be distinguished from the Cloud Connector administrator. To make an initial connection between the connector and a particular SAP BTP subaccount, you need an SAP BTP user with the required permissions for the related subaccount. We recommend that you separate these roles/duties (that means, you have separate users for Cloud Connector administrator and SAP BTP).

> ⓘ Note
>
> We recommend that only a small number of users are granted access to the machine as *root* users.

Hard Drive Encryption

Hard drive encryption for machines with a Cloud Connector installation ensures that the Cloud Connector configuration data cannot be read by unauthorized users, even if they obtain access to the hard drive.

Supported Protocols

Currently, the protocols HTTP, HTTPS, RFC, RFC with SNC, LDAP, LDAPS, TCP, and TCP over TLS are supported for connections between the SAP BTP and on-premises systems when the Cloud Connector and the Connectivity service are used. The whole route from the application virtual machine in the cloud to the Cloud Connector is always SSL-encrypted.

The route from the connector to the back-end system can be TLS-encrypted or SNC-encrypted. See Configure Access Control (HTTP) [page 1744] and Configure Access Control (RFC) [page 1752].

Audit Log on OS Level

We recommend that you turn on the audit log on operating system level to monitor the file operations.

Audit Log on Cloud Connector Level

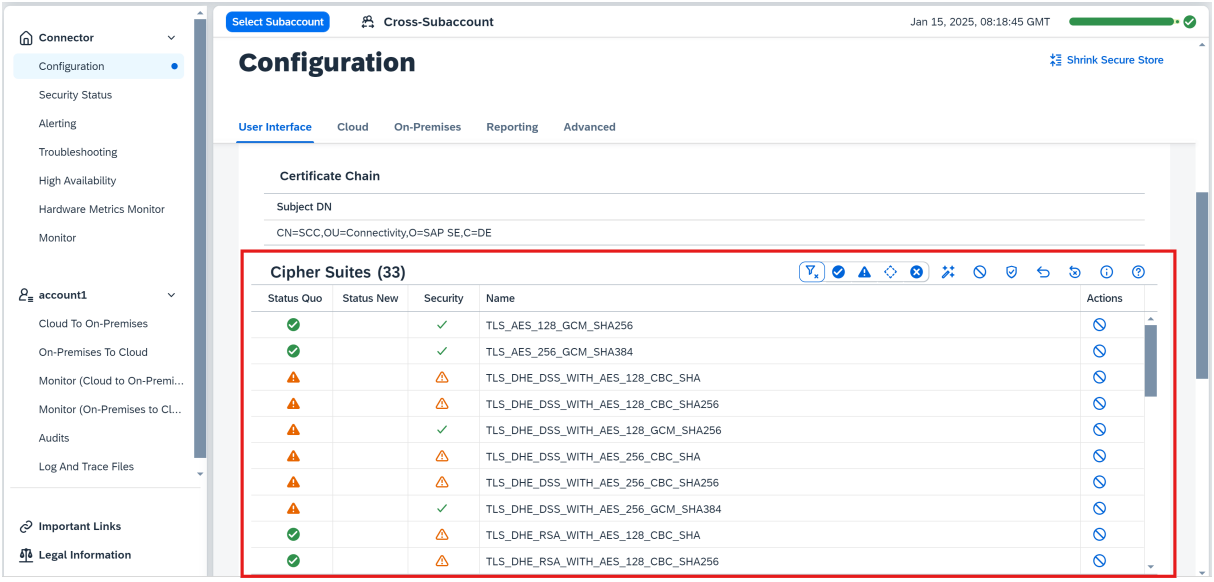
The Cloud Connector audit log must remain switched on during the time it is used with productive systems. The default audit level is **`SECURITY`**. Set it to **`ALL`** if required by your company policy. The administrators who are responsible for a running Cloud Connector must ensure that the audit log files are properly archived, to conform to the local regulations. You should switch on audit logging also in the connected back-end systems.

Encryption Ciphers

> → Tip
>
> Enable all cipher suites that are compatible with the current UI certificate and are deemed secure as per your company's and SAP's guidelines. Adapt the cipher suites whenever the UI certificate is exchanged or the JVM is updated.

Initially, a default set of encryption ciphers is enabled for HTTPS connections to the administration UI. This default set is determined by the JVM. Some of these ciphers may not be compliant with the UI certificate, and some of them may not conform to your or SAP's security standards. They should therefore be excluded.

To enable or disable ciphers, choose *Configuration* from the main menu and go to tab *User Interface*, section *Cipher Suites*.



The first column labeled **Status Quo** shows the current state of all available ciphers. The second column **Status New** shows the state the ciphers will have after a restart, if that state differs from the current one (that is, there is no entry in that column if the state remains the same after a restart).

Ciphers are either enabled or disabled, and they are either compatible or incompatible with the current UI certificate (that is, can potentially be used or not be used) . Consult the tooltips of the (four types of) icons for details. The third column shows SAP's security assessment of the ciphers *as per the time of release*. Enable or disable individual ciphers using the button in the **Action** column. Enable or disable certain groups of ciphers using the appropriate table button. Consult the tooltips for details.

> ### Note
>
> We recommend that you enable all cipher suites that are compatible with the UI whenever you plan to switch to another JVM. You can comfortably do so by using the first button to the right of the filter buttons.
>
> As the set of supported ciphers may differ, the selected ciphers may not be supported by the new JVM. In that case, the Cloud Connector will not start anymore, and you must fix the issue by manually adapting the file *conf/server.xml*. After a successful switch, you can adjust the list of eligible ciphers again.

> ### → Tip
>
> As of Cloud Connector version 2.17.0, you can run the script `resetCiphers` from the `scc` folder in the Cloud Connector installation directory to reset the ciphers if UI access is blocked due to wrong cipher settings.

Related Information

Connectivity via Reverse Proxy [page 2360]
Security [page 2057]

1.4.2 Configuration

Configure the Cloud Connector to make it operational for connections between your SAP BTP applications and on-premises systems.

Topic	Description
Initial Configuration [page 1670]	After installing the Cloud Connector and starting the Cloud Connector daemon, you can log on and perform the required configuration to make your Cloud Connector operational.
Adding and Managing Subaccounts [page 1682]	How to connect SAP BTP subaccounts to your Cloud Connector.
Authenticating Users against On-Premises Systems [page 1701]	Basic authentication and principal propagation (user propagation) are the authentication types currently supported by the Cloud Connector.
Configure Access Control [page 1740]	Configure access control or copy the complete access control settings from another subaccount on the same Cloud Connector.
Configuration REST APIs [page 1775]	Configure a newly installed Cloud Connector (initial configuration, subaccounts, access control) using the configuration REST API.
Using Service Channels [page 1949]	Service channels provide access from an external network to certain services on SAP BTP, which are not exposed to direct access from the Internet.
Configure Trust [page 1955]	Set up an allowlist for trusted cloud applications and a trust store for on-premises systems in the Cloud Connector.
Connect DB Tools to SAP HANA via Service Channels	How to connect database, BI, or replication tools running in the on-premises network to a HANA database on SAP BTP using the service channels of the Cloud Connector.
Configure Domain Mappings for Cookies [page 1958]	Map virtual and internal domains to ensure correct handling of cookies in client/server communication.
Configure Solution Management Integration [page 1959]	Activate Solution Management reporting in the Cloud Connector.
Configure Tunnel Connections [page 1960]	Adjust tunnel parameters for the Cloud Connector.
Configure Advanced Connectivity [page 1962]	Adapt connectivity settings that control the throughput and HTTP connectivity to on-premises systems.
Configure the Java VM [page 1963]	Adapt the JVM settings that control memory management.

Topic	Description
Configuration Backup [page 1964]	Backup and restore your Cloud Connector configuration.
Configure Login Screen Information [page 1966]	Add additional information to the login screen and configure its appearance.
Theming [page 1969]	Customize the Cloud Connector UI design by choosing from four Horizon themes.

1.4.2.1 Initial Configuration

After installing and starting the Cloud Connector, log on to the administration UI and perform the required configuration to make your Cloud Connector operational.

Tasks

Prerequisites [page 1670]

Log on to the Cloud Connector [page 1671]

Initial Setup [page 1672]

Adding the First Subaccount [page 1674]

Establish Connections to SAP BTP [page 1674]

Prerequisites

- You have downloaded and installed the Cloud Connector, see Installation [page 1608].
- You have assigned one of these roles/role collections to the subaccount user that you use for initial Cloud Connector setup, depending on the SAP BTP environment in which your subaccount is running:

Environment	Required Roles/Role Collections	More Information
Multi-Cloud	Assign at least one of these *default role collections* (all of them including the role `Cloud Connector Administrator`): • `Subaccount Administrator` • `Cloud Connector Administrator` • `Connectivity and Destination Administrator` Alternatively, you can assign a *custom role collection* to the user that includes the role `Cloud Connector Administrator`.	*Default Role Collections* Role Collections and Roles in Global Accounts, Directories, and Subaccounts
Neo	Assign at least one of these *default roles*: • `Cloud Connector Admin` • `Administrator` Alternatively, you can assign a *custom role* to the user that includes the permission `manageSCCTunnels`.	Managing Member Authorizations in the Neo Environment

After establishing the Cloud Connector connection, this user is not needed any more, since it serves only for initial connection setup. You may revoke the corresponding role assignment then and remove the user from the *Members* list (**Neo** environment), or from the *Users* list (**multi-cloud** environment).

> **Note**
>
> If the Cloud Connector is installed in an environment that is operated by SAP, SAP provides a user that you can add as member in your SAP BTP subaccount and assign the required role.

- We strongly recommend that you read and follow the steps described in Recommendations for Secure Setup [page 1665]. For operating the Cloud Connector securely, see also Security Guidelines [page 2065].

Back to Tasks [page 1670]

Log on to the Cloud Connector

To administer the Cloud Connector, you need a Web browser. To check the list of supported browsers, see Prerequisites and Restrictions → section *Browser Support*.

1. In a Web browser, enter: **`https://<hostname>:<port>`**

- `<hostname>` refers to the machine on which the Cloud Connector is installed. If installed on your machine, you can simply enter **`localhost`**.
- `<port>` is the Cloud Connector port specified during installation (the default port is **`8443`**).

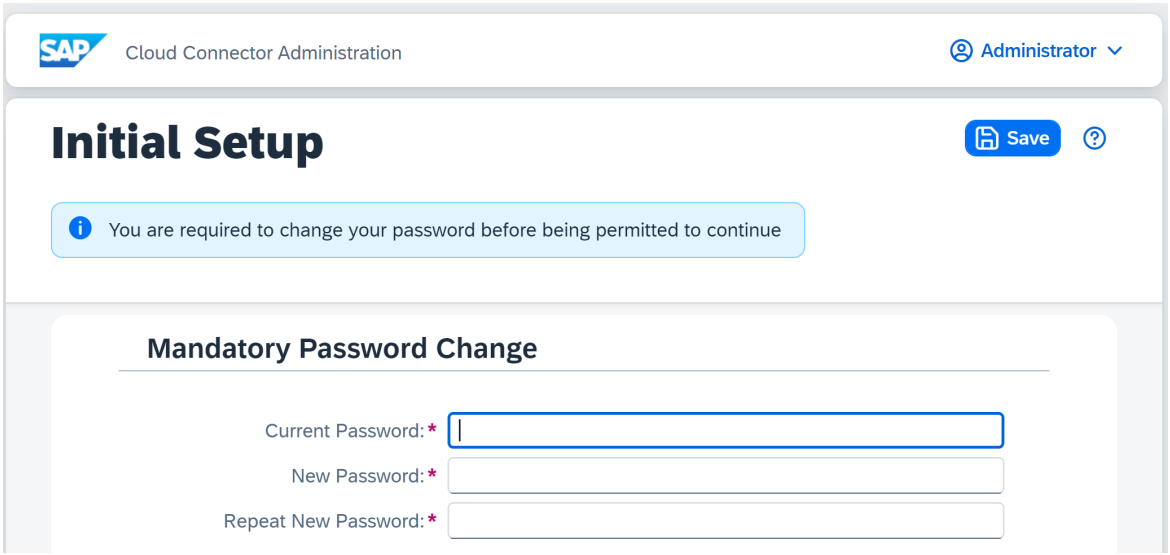
2. On the logon screen, enter **`Administrator`** / **`manage`** (case sensitive) for `<User Name>` / `<Password>`.

> **Note**
>
> By default, the Cloud Connector includes a self-signed UI certificate. Browsers may show a security warning because they don't trust the issuer of this certificate. In this case, you can skip the warning message.

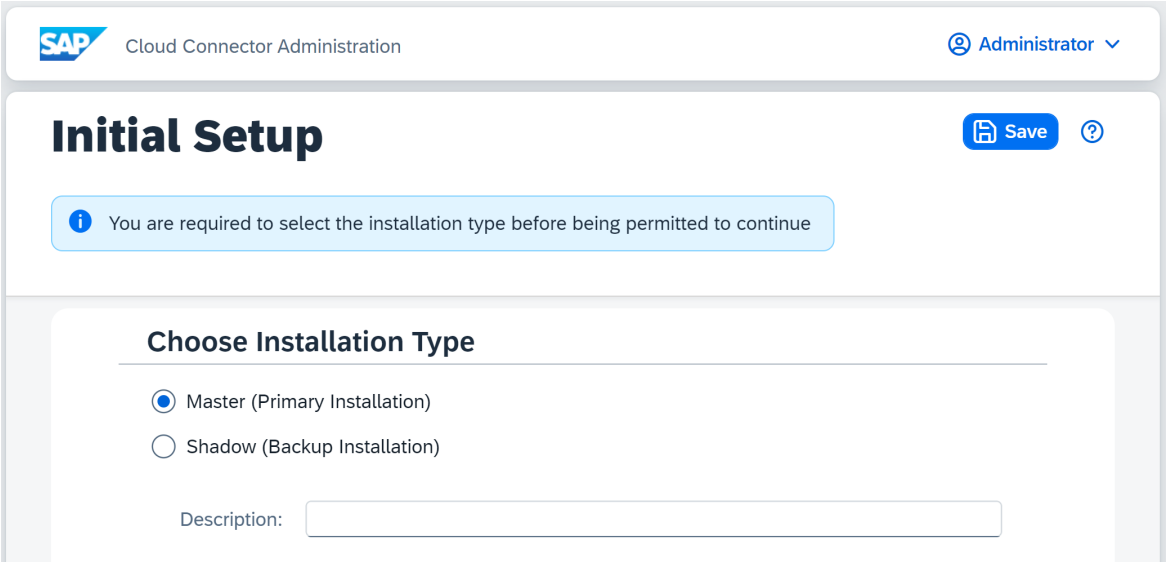
Back to Tasks [page 1670]

Initial Setup

1. When you first log in, you must change the password before you continue.



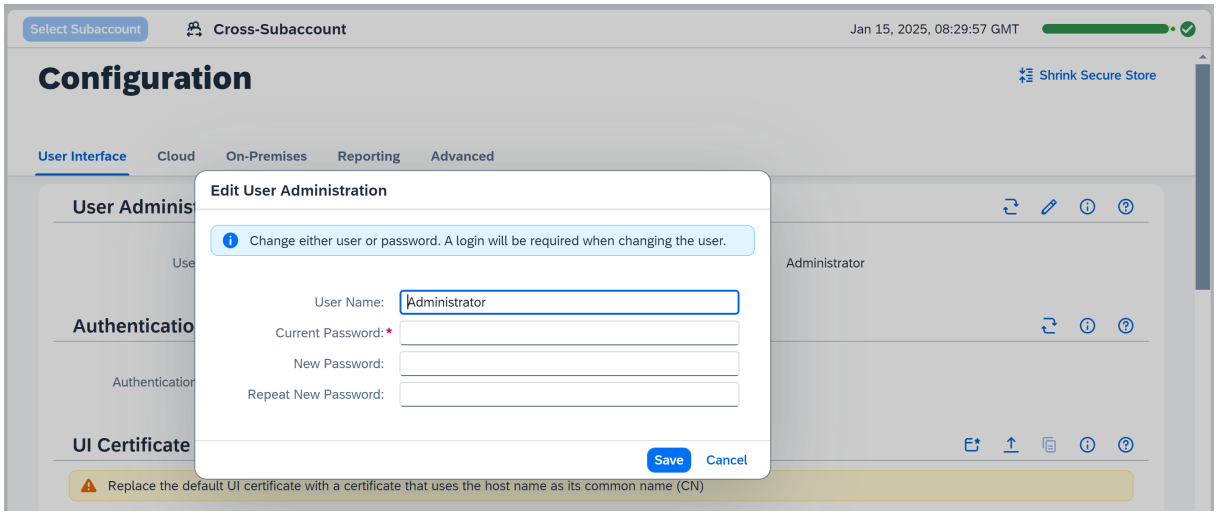
2. Afterwards, you can choose between master and shadow installation. Select *Master* if you are installing a single Cloud Connector instance or the main instance of a *high availability* setup. For more information, see High Availability Setup [page 1995].



3. (Optional): When configuring a master, you can provide a (free-text) *Description* for this Cloud Connector instance that helps you distinguish different Cloud Connectors. This information will also be shown in the *Cloud Connectors* view in the SAP BTP cockpit.
4. Afterwards, you are forwarded to the main page. In the top right corner you can always see how long your current session is still valid until you need to login again before proceeding.

User Administration

To edit the password for the **`Administrator`** user, choose *Configuration* from the main menu, tab *User Interface*, section *User Administration*:

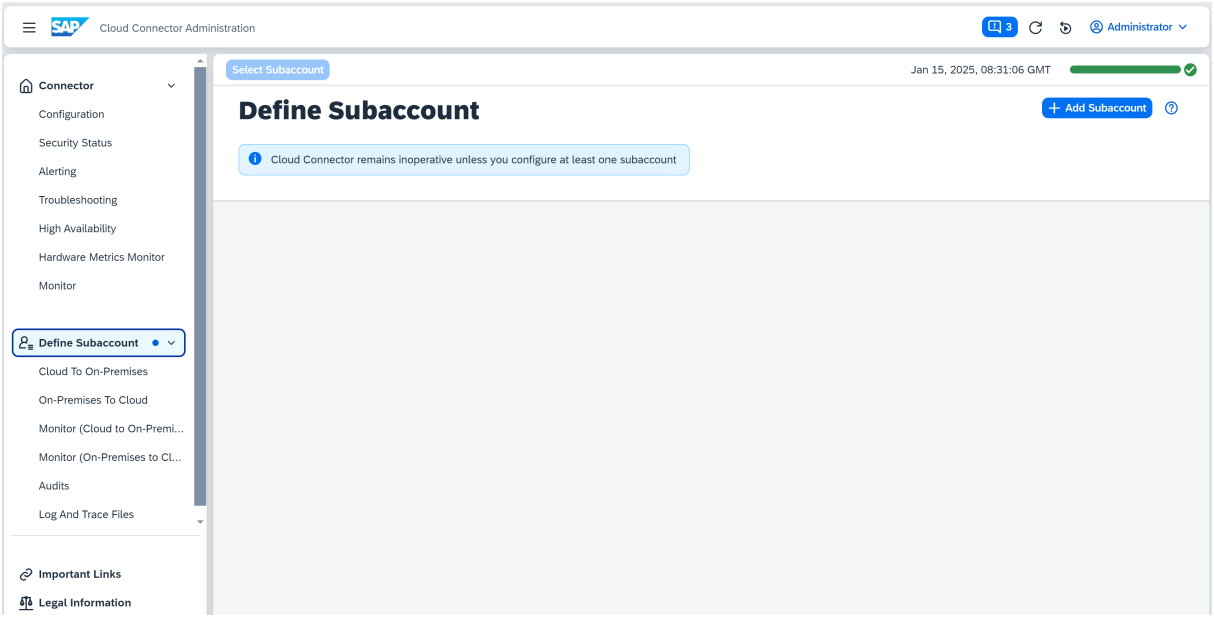


> **Note**
>
> User name and password cannot be changed at the same time. If you want to change the user name, you must enter only the current password in a first step. Do not enter values for `<New Password>` or `<Repeat New Password>` when changing the user name. To change the password in second step, enter the old password, the new one, and the repeated (new) password, but leave the user name unchanged.

Back to Tasks [page 1670]

Adding the First Subaccount

When logging in for the first time, the following screen is displayed every time you choose an option from the main menu that requires a configured subaccount:



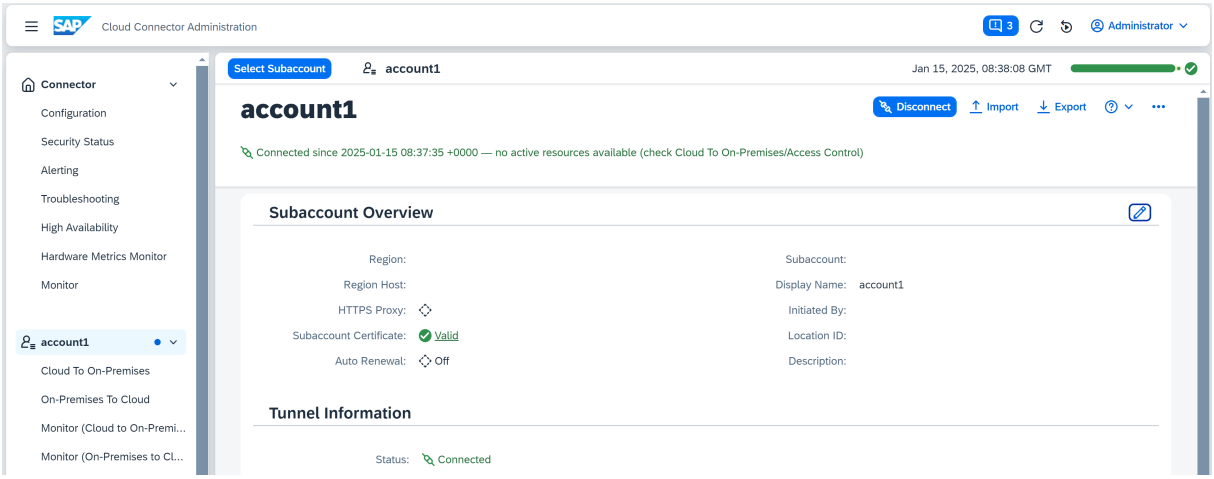
Press *Add Subaccount* to define a subaccount. For details, see Adding and Managing Subaccounts [page 1682].

Back to Tasks [page 1670]

Establish Connections to SAP BTP

As soon as the initial setup is complete, the tunnel to the cloud endpoint is open, but no requests are allowed to pass until you have performed the *Access Control* setup, see Configure Access Control [page 1740].

To manually close (and reopen) the connection to SAP BTP, choose your subaccount from the main menu and select the *Disconnect* button (or the *Connect* button to reconnect to SAP BTP).



- The green icon next to *Region Host* indicates that it is valid and can be reached.
- If an *HTTPS Proxy* is configured, its availability is shown the same way. In the screenshot, the grey diamond icon next to *HTTPS Proxy* indicates that connectivity is possible without proxy configuration.

In case of a timeout or a connectivity issue, these icons are yellow (warning) or red (error), and a tooltip shows the cause of the problem. *Initiated By* refers to the user that has originally established the tunnel. During normal operations, this user is no longer needed. Instead, a certificate is used to open the connection to a subaccount.

- The status of the certificate is shown next to *Subaccount Certificate*. It is shown as valid (green icon), if the expiration date is still far in the future, and turns to yellow if expiration approaches according to your alert settings. It turns red as soon as it has expired. This is the latest point in time, when you should Update the Certificate for Your Subaccount [page 1697].

> **Note**
>
> When connected, you can monitor the Cloud Connector also in the *Connectivity* section of the SAP BTP cockpit. There, you can track attributes like version, description and high availability set up. Every Cloud Connector configured for your subaccount automatically appears in the *Connectivity* section of the cockpit.

Back to Tasks [page 1670]

Related Information

Adding and Managing Subaccounts [page 1682]
Initial Configuration (HTTP) [page 1676]
Initial Configuration (RFC) [page 1678]
Configuring the Cloud Connector for LDAP [page 1682]
Managing Member Authorizations in the Neo Environment

Use a Custom IDP for Subaccount Configuration [page 1694]
Use LDAP for User Administration [page 1977]

1.4.2.1.1 Initial Configuration (HTTP)

Configure the Cloud Connector for HTTP communication.

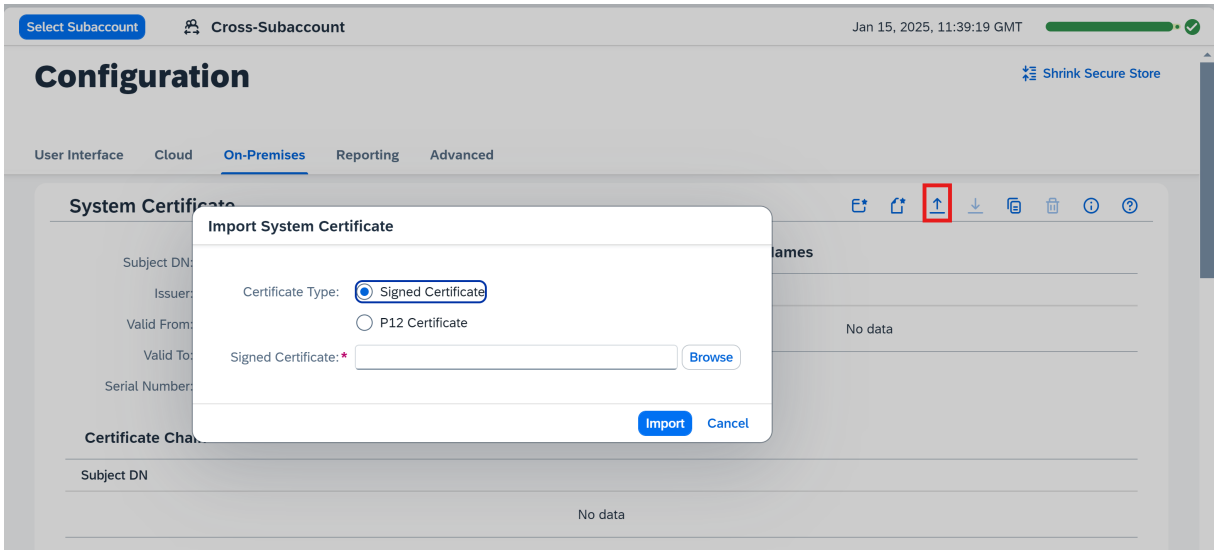
Install a System Certificate for Mutual Authentication

To set up a mutual authentication between the Cloud Connector and any backend system it connects to, you can import an X.509 client certificate into the Cloud Connector. The Cloud Connector then uses the so-called *system certificate* for all HTTPS requests to backends that request or require a client certificate. The certificate authority (CA) that signed the Cloud Connector's system certificate must be trusted by all backend systems to which the Cloud Connector is supposed to connect.

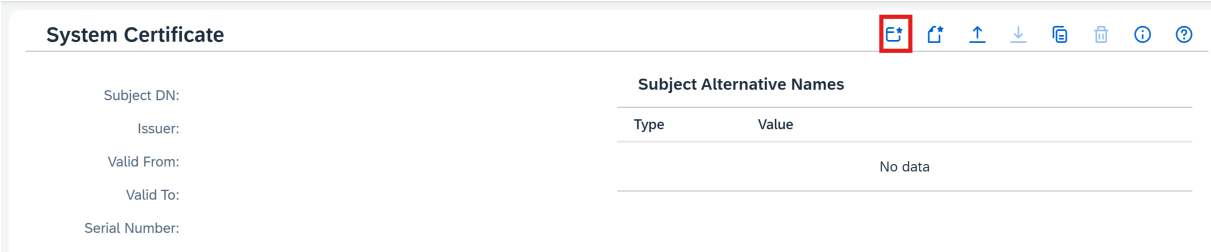
You must provide the system certificate as PKCS#12 file containing the client certificate, the corresponding private key and the CA root certificate that signed the client certificate (plus potentially the certificates of any intermediate CAs, if the certificate chain is longer than 2).

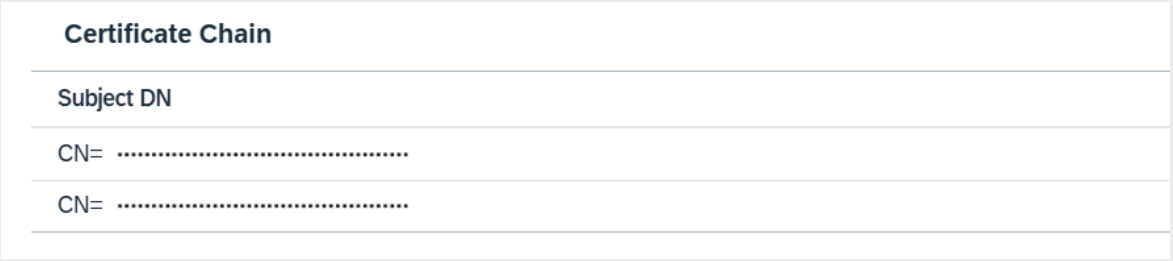
Procedure

From the left panel, choose *Configuration*. On the tab *On Premise*, choose *System Certificate* > *Import a certificate* to upload a certificate and provide its password:

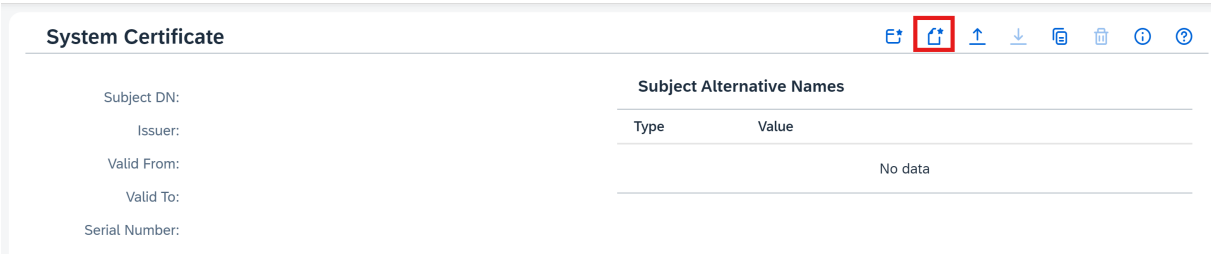


A second option is to start a *certificate signing request* procedure as described for the UI certificate in Exchange UI Certificates in the Administration UI [page 1971] and upload the resulting signed certificate.

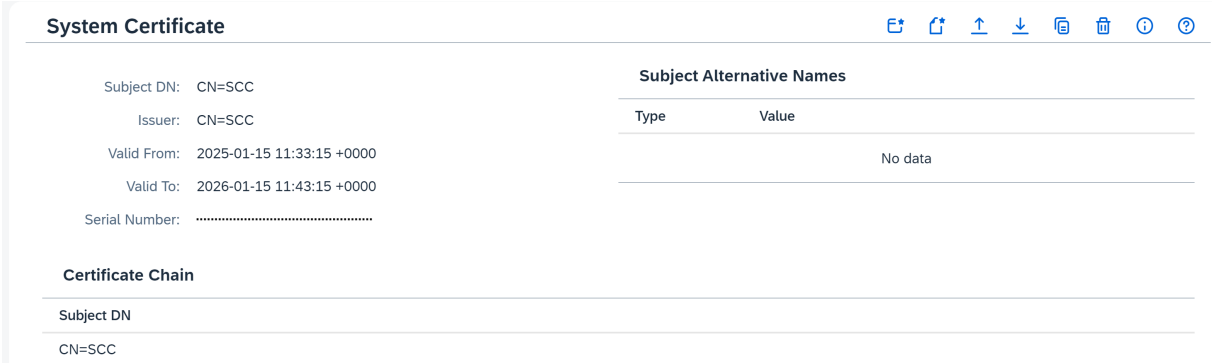


> ### Note
>
> If having intermediate certificates in place, make sure you import the complete chain (for example, as PKCS#7 file) so that mTLS connections always work properly.
>
> You can verify this in the **Certificate Chain** area of the **System Certificate** section:
>
> 
>

A third option is to generate a self-signed certificate. It might be useful if no CA is needed, for example, in a demo setup or if you want to use a dedicated CA. For this option, choose *Create and import a self-signed certificate*:



If a system certificate has been imported successfully, its distinguished name, the name of the issuer, and the validity dates are displayed:



Delete, Export, or Renew a System Certificate

You can delete a system certificate that is no longer required. To do this, use the respective button and confirm deletion.

If you need the public key for establishing trust with a server, you can export the full chain via the *Export* button.

To renew a certificate that is close to expiration, install the new certificate as described above. This will replace the expiring certificate.

Related Information

Configure Access Control (HTTP) [page 1744]

1.4.2.1.2 Initial Configuration (RFC)

Configure a Secure Network Connection (SNC) to set up the Cloud Connector for RFC communication to an ABAP backend system.

SNC Configuration for Mutual Authentication

To set up a mutual authentication between Cloud Connector and an ABAP backend system (connected via RFC), you can configure SNC for the Cloud Connector. It will then use the associated PSE for all RFC SNC requests. This means that the SNC identity, represented by this PSE, must:

- Be trusted by all backend systems to which the Cloud Connector is supposed to connect

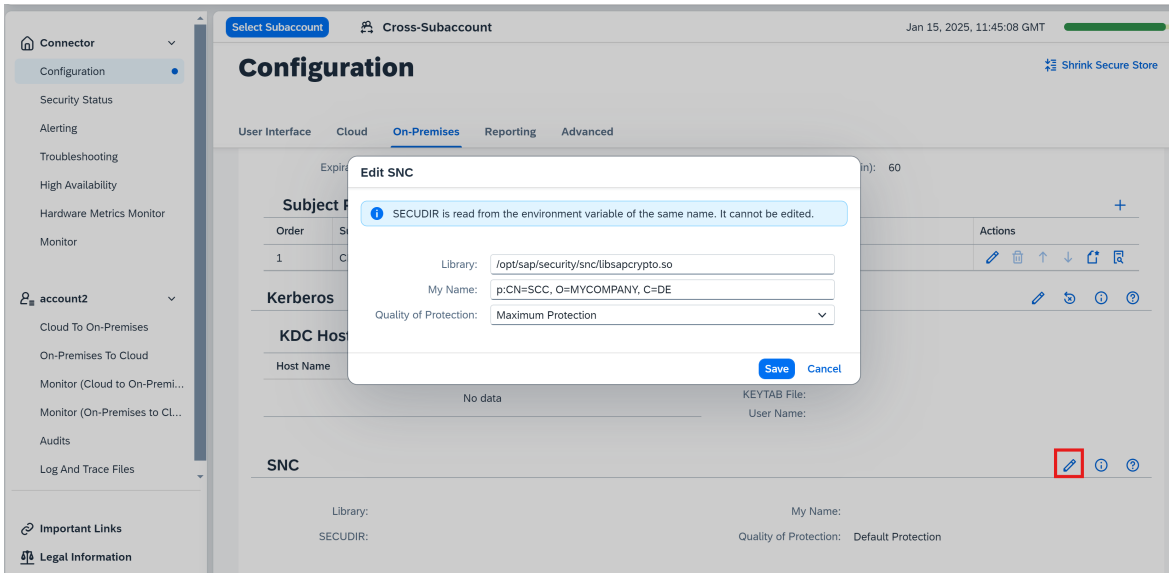
- Play the role of a trusted external system by adding the SNC name of the Cloud Connector to the SNCSYSACL table. You can find more details in the SNC configuration documentation for the release of your ABAP system.

Prerequisites

- You have configured your ABAP system(s) for SNC. For detailed information on configuring SNC for an ABAP system, see also Configuring SNC on AS ABAP.
- You have configured the ABAP System to trust the Cloud Connector's system SNC identity. To do this, or to establish trust for principal propagation, follow the steps described in Configure Identity Propagation for RFC [page 1721].

Configuration Steps

1. Logon to the Cloud Connector
2. Choose *Configuration* from the main menu and go to tab *On Premise*, section *SNC*.
3. Enter the corresponding values in the fields *Library Name*, *My Name*, and *Quality of Protection*
4. Press *Save*.
   Example:



- *Library Name*: Provides the location of the SNC library you are using for the Cloud Connector.

  > **Note**
  >
  > Bear in mind that you must use one and the same security product on both sides of the communication.

- *My Name*: The SNC name that identifies the Cloud Connector. It represents a valid scheme for the SNC implementation that is used.

- *Quality of Protection*: Determines the level of protection that you require for the connectivity to the ABAP systems.

> **Note**
>
> When using **CommonCryptoLibrary** as SNC implementation, note 1525059 will help you to configure the PSE to be associated with the user running the Cloud Connector process.

Using the SAP Cryptographic Library

> **Note**
>
> This procedure is available as of Cloud Connector version 2.14.1

When using the SAP Cryptographic Library as SNC implementation, you can use the interactive setup scripts which can be found in the Cloud Connector's installation folder to ease the setup process.

> **Note**
>
> Using the scripts mentioned below is not mandatory for using the SAP cryptographic library. If you are familiar with the necessary steps needed for running a process with the desired identity, you can also configure the SNC setup for the SAP cryptographic library on your own.

Linux/Mac scripts: *snc_create_pse.sh, snc_import_ca_response.sh*

Windows scripts: *snc_create_pse.bat, snc_import_ca_response.bat*

> **Note**
>
> These scripts are located in the Cloud Connector installation directory.

1. Download and extract the SAP Cryptographic Library from the Download Center (search for *sapcryptolib*).
2. Copy the respective scripts depending on your OS to the SAP Cryptographic Library folder.
3. Make sure the Cloud Connector process is running.
4. Make sure the environment variable `SECUDIR` is properly set.
    1. For Linux, you can set it solely for the Cloud Connector process by extending the daemon as described in Installation on Linux OS [page 1660].
5. Copy the partner's certificate you have exported to your `SECUDIR` folder. You must specify this to import it into your PSE.
6. Run the script *snc_create_pse* now. You should see the `SECUDIR` and the Cloud Connector user:
   SECUDIR D:\sec found.
   Cloud Connector is running with user NT AUTHORITY\SYSTEM
7. Specify the certificate name for the Cloud Connector:
   Specify the own certificate name in the format as CN=host,OU=org,O=comp,C=lang: CN=SCC
   Specify the PSE password: <enter secure PW here>

8. Specify the exported certificate of the partner from step 5:
   Specify the import certificate file of the partner contained in D:\sec: exported_abap_cert.cer
   Creating PSE now...
   ...
   PSE creation finished.
9. Now, a PSE with name `scc.pse` has been created in your `SECUDIR` folder. Additionally, a file `sccCertificateRequest.p10` has been created. This is the CSR you can use to get a signed certificate by your CA now. You can import the response certificate from your CA directly, or use the other script `import_ca_response` later. If you don't want to sign your certificate by a CA and use it as a self-signed certificate, choose *no*. Depending on your CA, you might need to provide, besides the signed certificate, all other intermediate certificates and the root certificate for the import. Copy them to the `SECUDIR` folder and specify them in the order shown below.
   Do you want to import a CA response now? [y/n] y
   Specify the CA response file contained in D:\sec: ca_response.crt
   Specify further Root CAs (PEM, Base64 or DER binary) in D:\sec needed to complete the chain (separated by blanks), otherwise press enter: intermediate.crt root.crt
   Importing CA response now...
   ...
   CA-Response successfully imported into PSE "D:\sec\scc.pse"
10. Now, all other required steps are done, such as creating the credentials file `cred_v2`, importing the partner certificate and exporting the Cloud Connector's certificate to `SECUDIR` with the name `scc.crt` (which must be imported at the partner's side).
    Create SSO server credentials...
    ...
    Creation finished.
    Export own certificate...
    ...
    Export finished.
    Import partner certificate into PSE file...
    ...
    Completed.
11. Restart the Cloud Connector and check the above screen if `SECUDIR` is set correctly. SNC setup for SAP Cryptographic Library should be complete now. In a next step, you must Configure Access Control [page 1752] and, if needed, Configure Identity Propagation for RFC [page 1721].

> ⚠ Caution
>
> The certificate in the PSE you generated in this process needs to be monitored separately, it is not under the control of Cloud Connector. Update it before expiration to make sure your scenarios keep working.

If you have further issues, check SAP note 1525059.

Related Information

Configure Identity Propagation for RFC [page 1721]

1.4.2.1.3 Configuring the Cloud Connector for LDAP

Configure the Cloud Connector to support LDAP in different scenarios (cloud applications using LDAP or Cloud Connector authentication).

Prerequisites

You have installed the Cloud Connector and done the basic configuration:

Installation [page 1608]

Initial Configuration [page 1670]

Steps

When using LDAP-based user management, you have to confgure the Cloud Connector to support this feature. Depending on the scenario, you need to perform the following steps:

Scenario 1: Cloud applications using LDAP for authentication. Configure the destination of the LDAP server in the Cloud Connector: Configure Access Control (LDAP) [page 1764].

Scenario 2: Internal Cloud Connector user management. Activate LDAP user management in the Cloud Connector: Use LDAP for User Administration [page 1977].

1.4.2.2 Adding and Managing Subaccounts

Add and connect your SAP BTP subaccounts to the Cloud Connector.

> ⓘ Note
>
> This topic refers to subaccount management in the Cloud Connector. If you are looking for information about managing subaccounts on SAP BTP (multi-cloud or Neo environment), see
>
> - Account Administration (multi-cloud environment)
> - Administration and Operations, Neo Environment

Context

You can connect to several subaccounts within a single Cloud Connector installation. Those subaccounts can use the Cloud Connector concurrently with different configurations. By selecting a subaccount from the

drop-down box, all tab entries show the configuration, audit, and state, specific to this subaccount. In case of audit and traces, cross-subaccount info is merged with the subaccount-specific parts of the UI.

> **Note**
>
> We recommend that you group only subaccounts with the same qualities in a single installation:
>
> - Productive subaccounts should reside on a Cloud Connector that is used for productive subaccounts only.
> - Test and development subaccounts can be merged, depending on the group of users that are supposed to deal with those subaccounts. However, the preferred logical setup is to have separate development and test installations.

Prerequisites

You have assigned one of these roles/role collections to the subaccount user that you use for initial Cloud Connector setup, depending on the SAP BTP environment in which your subaccount is running:

Environment	Required Roles/Role Collections	More Information
Multi-Cloud	Assign at least one of these *default role collections* (all of them including the role `Cloud Connector Administrator`): • `Subaccount Administrator` • `Cloud Connector Administrator` • `Connectivity and Destination Administrator` Alternatively, you can assign a *custom role collection* to the user that includes the role `Cloud Connector Administrator`.	Default Role Collections [page 1305] Role Collections and Roles in Global Accounts, Directories, and Subaccounts
Neo	Assign at least one of these *default roles*: • `Cloud Connector Admin` • `Administrator` Alternatively, you can assign a *custom role* to the user that includes the permission `manageSCCTunnels`.	Managing Member Authorizations in the Neo Environment

After establishing the Cloud Connector connection, this user is not needed any more, since it serves only for initial connection setup. You may revoke the corresponding role assignment then and remove the user from the *Members* list.

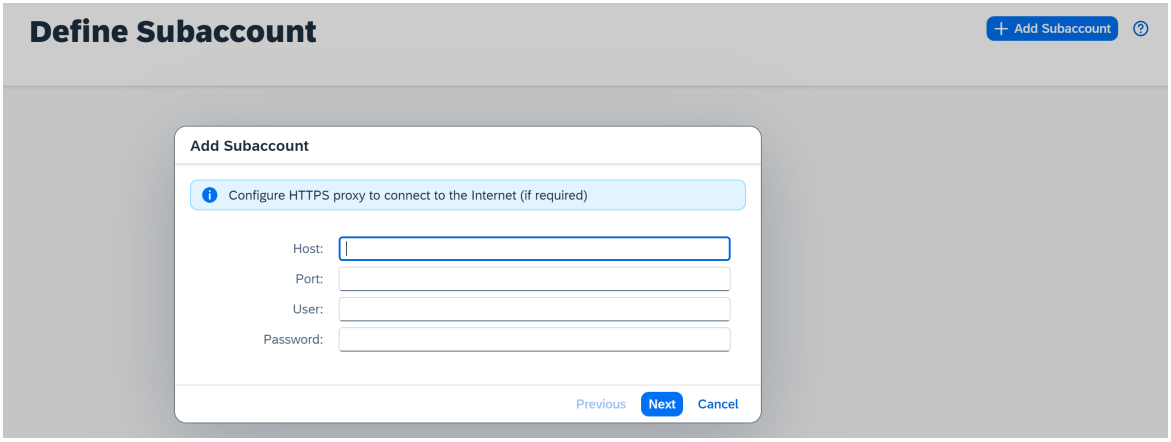
> **Note**
>
> If the Cloud Connector is installed in an environment that is operated by SAP, SAP provides a user that you can add as member in your SAP BTP subaccount and assign the required role.

Adding a Subaccount

Press *Add Subaccount* to define a subaccount. This will open a dialog or wizard through which the Cloud Connector collects the following optional and required information:

1. (Optional) Enter an HTTPS proxy. When in doubt, consult your network administrator to check if a proxy is required.

   > **Note**
   >
   > Up to Cloud Connector 2.17.0, the Cloud Connector sends HEAD requests during the configuration process to track the status of the proxy in the Cloud Connector UI. As a prerequisite, the proxy must be configured to allow those HEAD requests. Otherwise, it closes the connection and you get the message *Proxy is not operational (status code 503)*.
   >
   > As of Cloud Connector 2.17.1, HEAD requests will only be sent to the entered proxy in error situations, to create more meaningful error messages identifying the root cause of the error, for example, if the entered proxy is considered as unknown host, or if it does not run on the specified port.

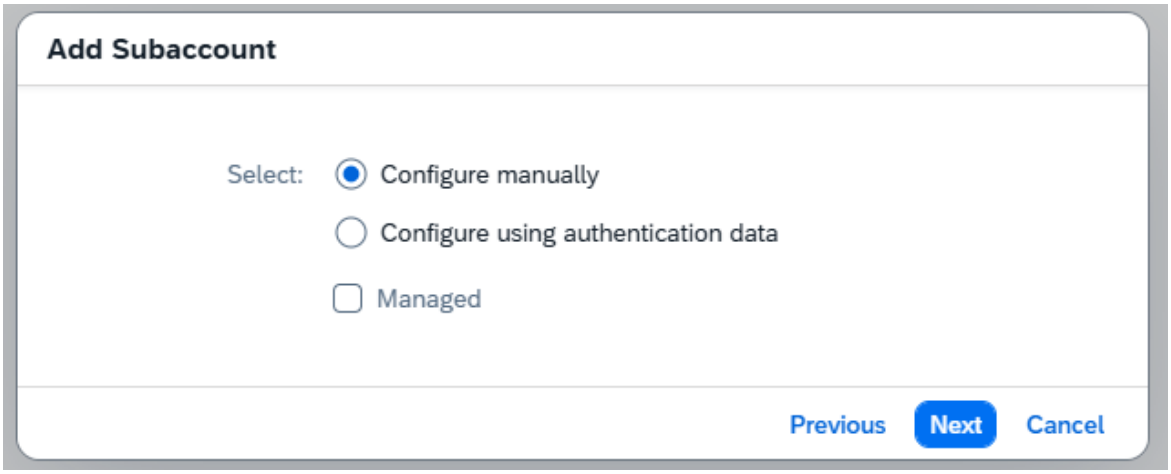
   

2. In the next step, you can choose between a *manual* configuration and a *file-based* configuration. *File-based* configuration lets you use an authentication data file downloaded from your subaccount on SAP BTP, multi-cloud foundation to simplify and accelerate subaccount configuration in the Cloud Connector.

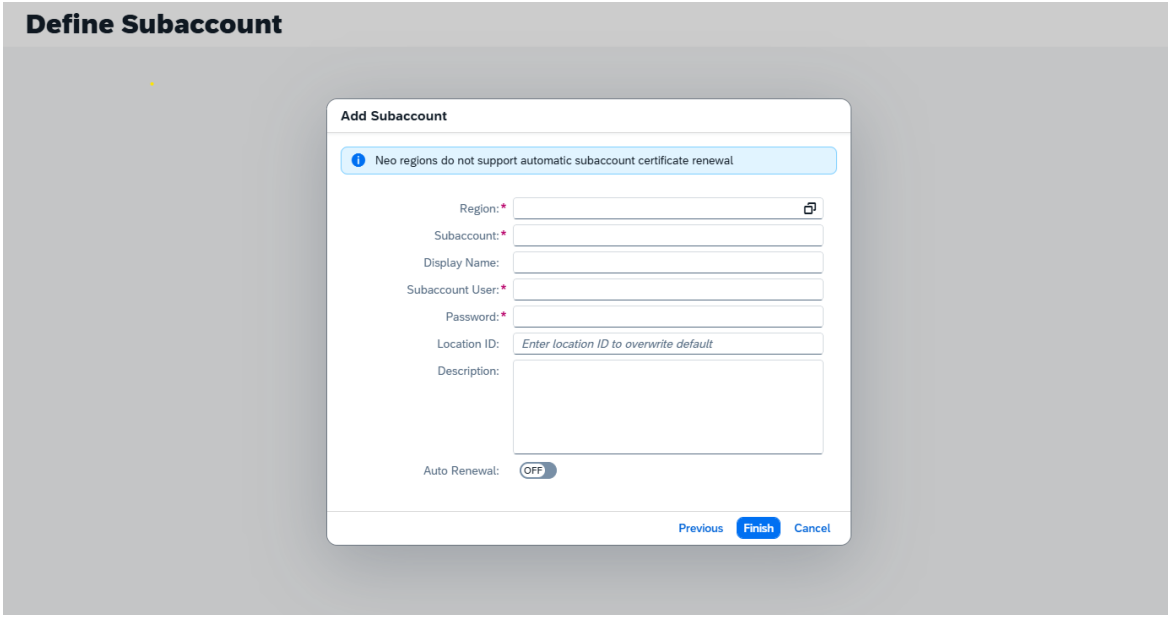
   > **Restriction**
   >
   > The option *File-based configuration* does not work for subaccounts in the **Neo** environment.

You can also choose whether the subaccount is to be *managed* or not.
A *managed* subaccount substantially restricts access for non-*Administrator* roles beyond the restrictions already present to the extent that a subaccount can only be configured and maintained by an administrator (read-only access as well as connecting/disconnecting is not affected).

> **Note**
>
> In a *managed* subaccount, only the *Administrator* role (default: *sccadmin*) can change the configuration of a single subaccount, while in a standard (non-managed) subaccount also the roles *Associate Administrator* (*sccassocadmin*) and *Subaccount Administrator* (*sccsubadmin*) are allowed to do so.
>
> For more information on role definitions, see also Use LDAP for User Administration [page 1977].
>
> The same applies for changes in the subaccount configuration via REST API: In a *managed* subaccount, only the *Administrator* role can perform POST, PUT, and DELETE tasks. GET tasks are not restricted.
>
> For detailed information, see User Roles [page 2086] and the subaccount-related APIs in Configuration REST APIs [page 1775].



3. (Skip if you have selected *file-based* configuration) For *manual* configuration, the following dialog is shown:



1. The <*Region*> field specifies the SAP BTP region that should be used such as *Europe (Rot)*, for example. Both a value help and a drop-down box with suggestions while typing are available.

   > → Remember
   >
   > The available regions and region domains depend on the SAP BTP environment you are using. For more information, see Regions (multi-cloud environment) or Regions and Hosts Available for the Neo Environment.

   > ⓘ Note
   >
   > You can also configure a region yourself, if it is not part of the standard list. Either insert the region host manually, or create a custom region, as described in Configure Custom Regions [page 1700].

2. For `<Subaccount>`, enter the value you obtained when you registered your subaccount on SAP BTP.

   > ⓘ Note
   >
   > For a subaccount in the **multi-cloud** environment, you must enter the subaccount **ID** as `<Subaccount>`, rather than its actual (technical) name. For information on getting the subaccount ID, see Find Your Subaccount ID (Multi-Cloud Environment) [page 1700].

3. `<Subaccount User>` and `<Password>` require dedicated values, depending on the type of identity provider (IDP) you are using:

   > ⓘ Note
   >
   > For more information on IDPs in the **multi-cloud** environment, see Trust and Federation with Identity Providers.

   - *SAP ID Service* (Default):
     - User/password from SAP ID Service must be used.

       > ⓘ Note
       >
       > For a subaccount in the **multi-cloud** environment, you must provide your **`Login E-mail`** as `<Subaccount User>` instead of a user ID. The user must be a member of the global account the subaccount belongs to.

     - The user must be a member of the subaccount, and the subaccount must have the correct (SAP ID Service) user base.

     > ⓘ Note
     >
     > Alternatively, you can add a new subaccount user in the SAP BTP cockpit, assign the required authorization (see section Prerequisites [page 1670]), and use the new user and password.
     >
     > For a **Neo** subaccount, you can also add a new subaccount user with the role `Cloud Connector Admin` from the *Members* tab in the SAP BTP cockpit and use the new user and password.

   - *Custom IDP* (IAS tenant):

- **Neo** subaccount:
  - User/password from the configured IAS tenant must be provided.
  - The user must be a member of the subaccount, and the subaccount must have the correct (IAS tenant) user base.
- **Multi-Cloud** subaccount:
  - For a subaccount in the **multi-cloud** environment, the Cloud Connector supports the use of a custom IDP via single sign-on (SSO) passcode.
    For more information, see Use a Custom IDP for Subaccount Configuration [page 1694].

> ### Note
>
> The Cloud Connector does not yet support *SAP Universal ID*. Please use your S-user or P-user credentials for the `<subaccount user>` and `<password>` fields instead.
>
> For more information, see SAP note 3085908.

For the **Neo** environment, see also Add Members to Your Neo Subaccount.
For the **multi-cloud** environment, see also Add Org Members.

> ### → Tip
>
> When using SAP Cloud Identity Services - Identity Authentication (IAS) as platform identity provider with two-factor authentication (2FA / MFA) for your subaccount, you can simply append the required token to the regular password. For example, if your password is "eX7?6rUm" and the one-time passcode is "123456", you must enter "eX7?6rUm123456" into the *<Password>* field.
>
> **Appending the two-factor authentication** (2FA / MFA) token to the regular password **is not required** when performing the initial configuration for a Cloud Connector subaccount in the **Neo environment**.

4. (Optional) You can define a `<Display Name>` that lets you easily recognize a specific subaccount in the UI compared to the technical subaccount name.
5. (Optional) You can define a `<Location ID>` identifying the location of this Cloud Connector for a specific subaccount. The location ID is used as routing information. It lets you connect multiple Cloud Connectors to a single subaccount. If you don't specify any value for `<Location ID>`, the default (empty string) is used. The location ID must be unique per subaccount and should be an identifier that can be used in a URI. To route requests to a Cloud Connector with a location ID, the location ID must be configured in the respective destinations.
6. (Optional) You can provide a `<Description>` of the subaccount that becomes a part of the subaccount's detail information.

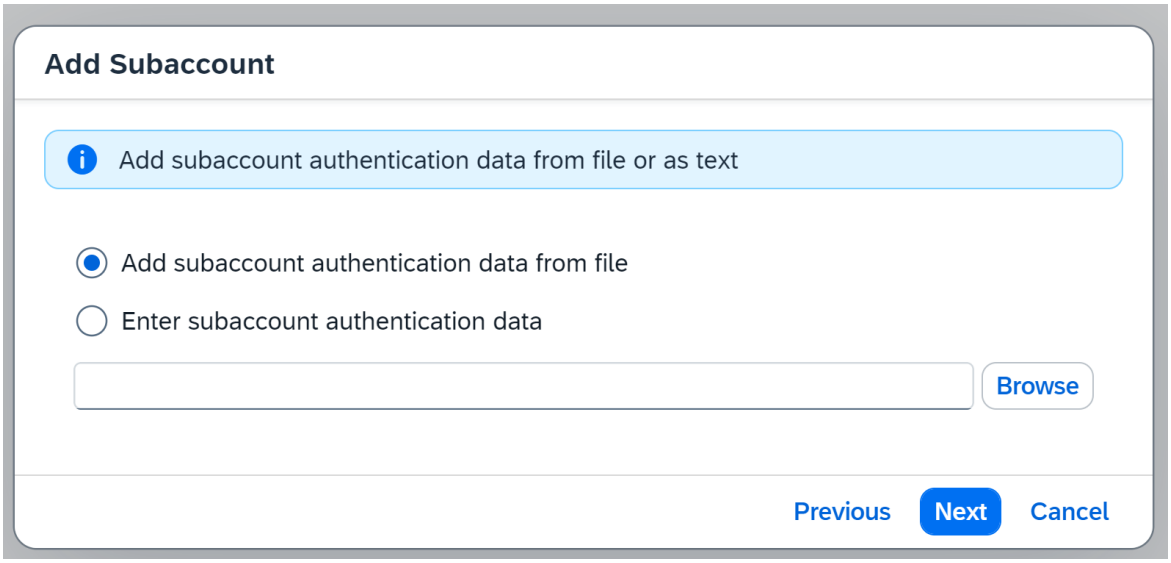
   > ### Note
   >
   > Location ID and description can be changed later on at any time. See Next Steps (Editing) [page 1691].

7. Select the automatic subaccount certificate renewal policy.

   > ### Note
   >
   > **Neo** regions do not support automatic subaccount certificate renewal. Choosing automatic renewal for a subaccount of a Neo region has no effect. For other regions, the policy can be changed later on at any time. See Next Steps (Editing) [page 1691].

8. Choose *Finish*.

4. (Skip if you have selected *manual* configuration) For the *file-based* approach using authentication data, the following dialog is shown:



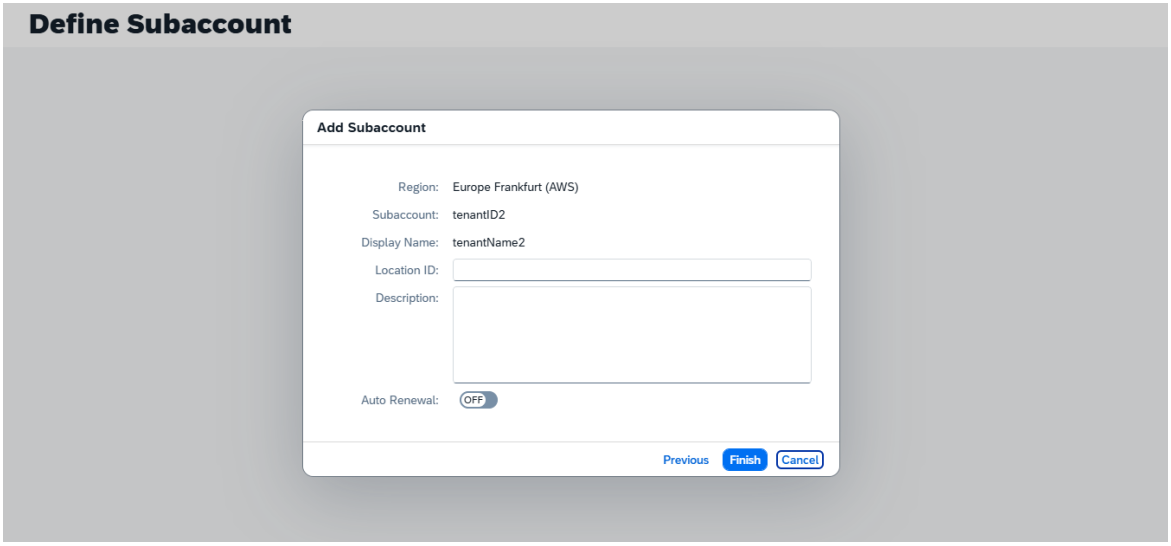
> **Note**
>
> The obtained subaccount authentication data you plan to apply is valid only for a few minutes. Therefore, you need to apply the subaccount authentication data immediately after fetching it from the cloud side.

To download the authentication data file **from your subaccount in the SAP BTP cockpit**, perform the following steps:

1. Log on to the SAP BTP cockpit and choose the subaccount you'd like to connect to.
2. Go to section *Connectivity* > *Cloud Connectors* and press *Download Authentication Data*.
3. Go back to the Cloud Connector *Add Subaccount* dialog and select *Add subaccount authentication data from file*.
4. Choose the file containing the authentication data and press *Next*.
   You can then review the data extracted from the file, as well as optionally enter a location ID, a description, and enable automatic subaccount certificate renewal (see steps 3e-g for details on those properties).

If you have the authentication data **available as a string from your SaaS solution**:

1. Select *Enter subaccount authentication data*.
2. Paste it as text.
3. Press *Next*.

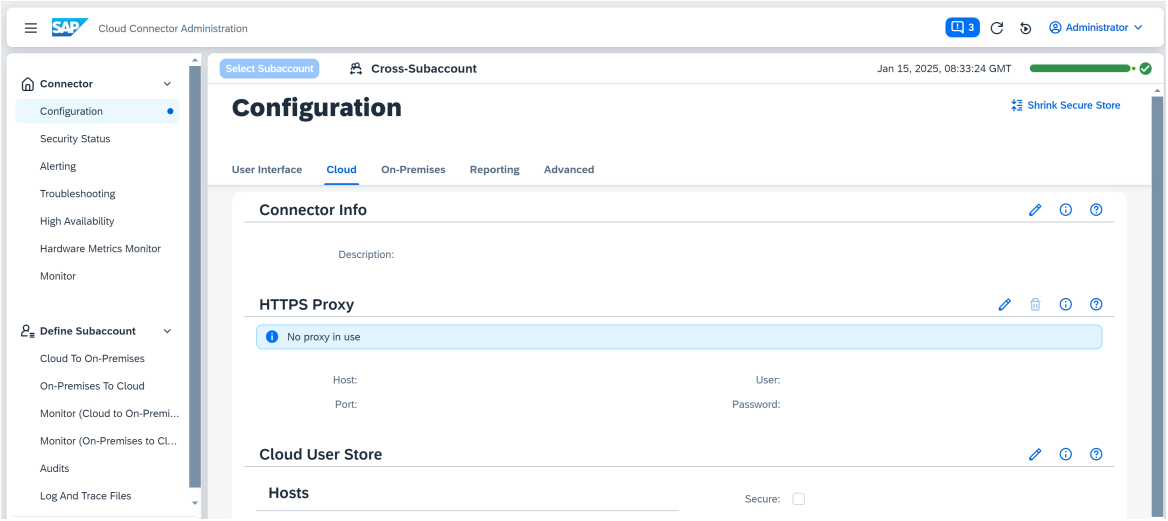


When done, press *Finish*.

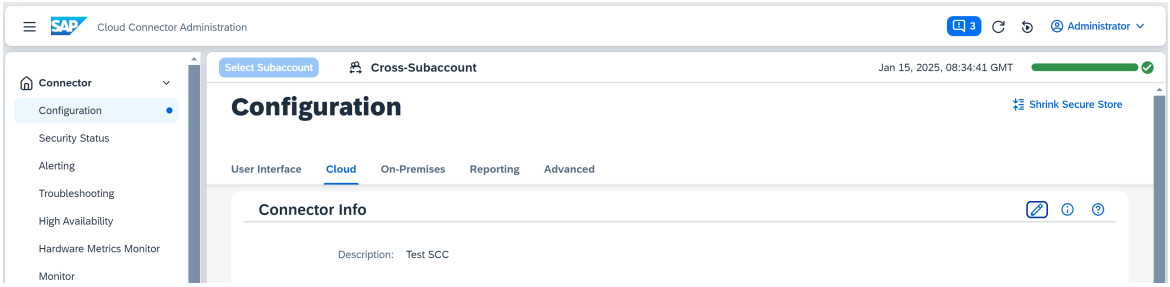
The Cloud Connector now starts a handshake with SAP BTP and attempts to establish a secure TLS tunnel to the server that hosts the subaccount in which your cloud applications are running. However, no requests are yet allowed to pass from the cloud side to any of your internal backend systems. To allow your cloud applications to access specific internal backend systems, proceed with Configure Access Control [page 1740].

> ### Note
>
> The internal network must allow access to the port. Specific configuration for opening the respective port(s) depends on the firewall software used. The default ports are **`80`** for HTTP and **`443`** for HTTPS. For RFC communication, you must open a gateway port (default: **`33+<instance number>`** and an arbitrary message server port. For a connection to a HANA Database (on SAP BTP) via JDBC, you must open an arbitrary *outbound* port in your network. Mail (SMTP) communication is not supported.

- If you later want to change your proxy settings (for example, because the company firewall rules have changed), choose *Configuration* from the main menu and go to the *Cloud* tab, section *HTTPS Proxy*. Some proxy servers require credentials for authentication. In this case, you must provide the relevant user/password information.

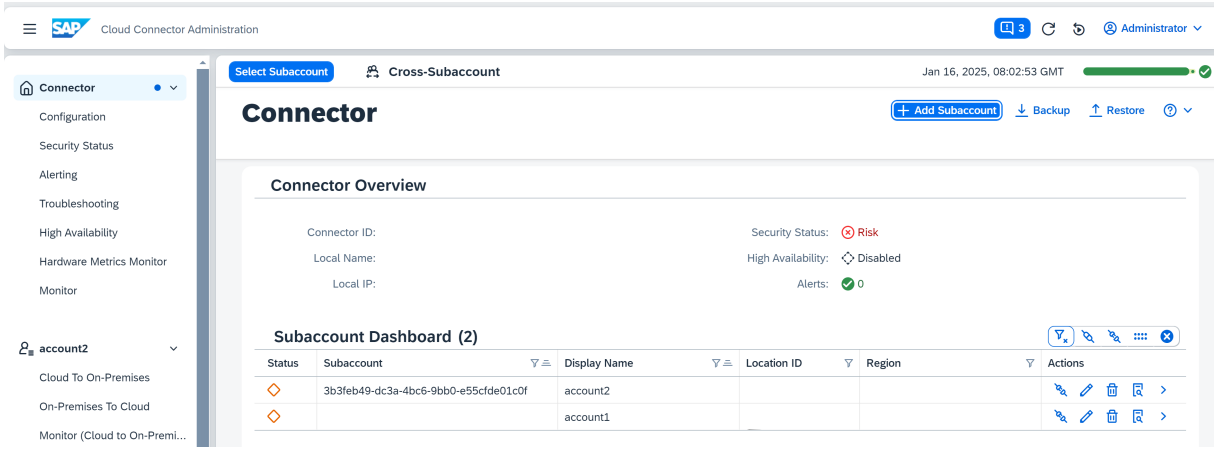


- If you want to change the description for your Cloud Connector (don't confuse with the description of subaccounts), choose *Configuration* from the main menu, go to the *Cloud* tab, section *Connector Info* and edit the description:



Subaccount Dashboard

In the subaccount dashboard, you can check the state of all subaccount connections managed by this Cloud Connector at a glance.



In the screenshot above, the subaccount with display name **account2** (the actual subaccount ID starts with *3b3feb49-*) is already connected, but has no active resources exposed. All other subaccounts are connected with exposed resources and are fully operational. In addition, depending on the connection state, the dashboard allows you to do disconnect and connect subaccounts by pressing the respective button in the **Actions** column. You may also view details of, delete, or select a subaccount and navigate to its overview page with the respective buttons from the **Actions** column. The sort buttons for the columns **Subaccount** and **Display Name** let you sort the entries of the respective column either ascending or descending. The filter buttons in the column headers allow to filter based on the entries of the respective column.

The filter buttons above the dashboard let you filter the shown subaccounts based on the connection status, showing either *all subaccounts*, *all connected ones*, *all disconnected ones*, *all subaccounts currently in connecting or reconnecting status*, or *all subaccounts for which establishing the connection has failed*.

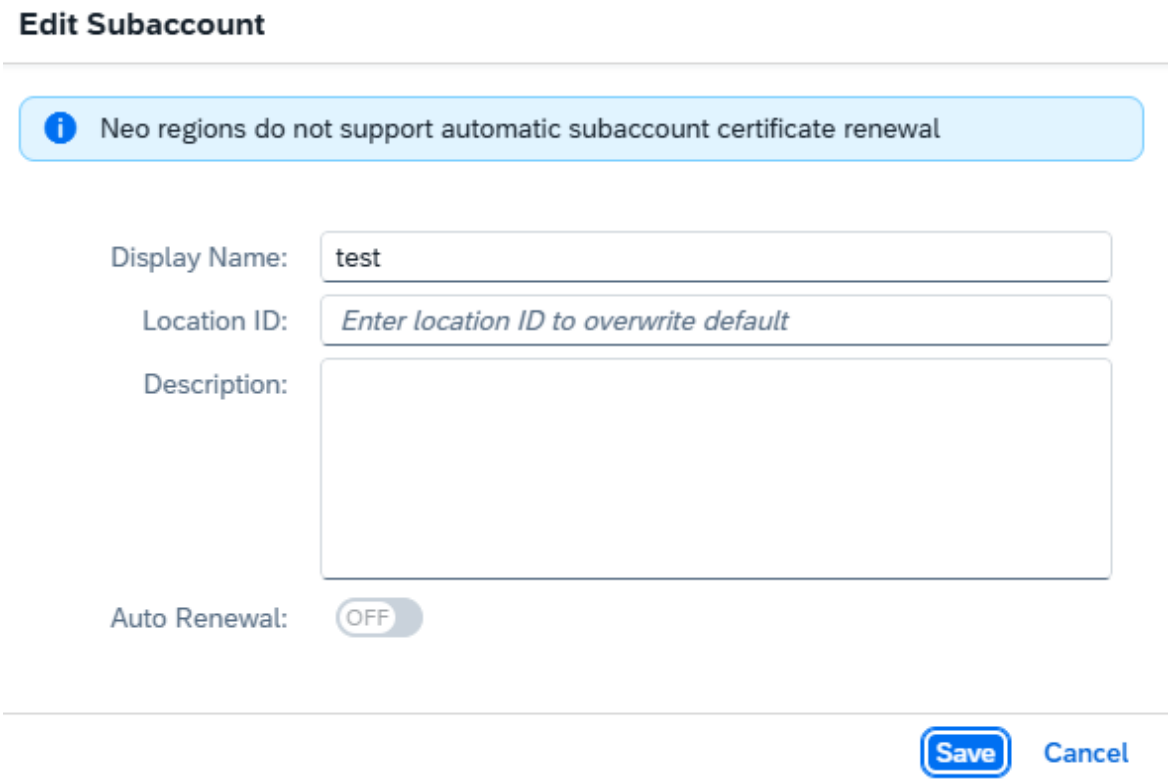
If you want to connect an additional subaccount with your on-premises landscape, press the *Add Subaccount* button.

→ Remember

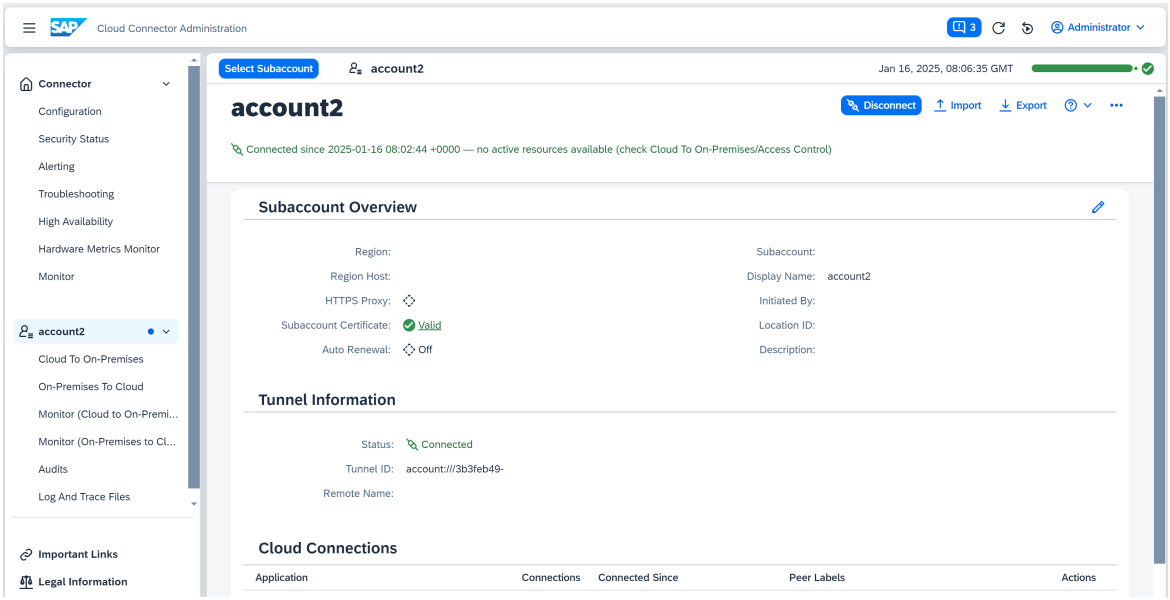
Keep in mind that the specification of an HTTPS proxy is only offered when establishing the first connection.

Next Steps

- To modify an existing subaccount, choose the *Edit* icon and change the `<Display Name>`, `<Location ID>`, `<Description>`, and the automatic subaccount certificate renewal policy. The latter is not available for Neo regions.



- You can also delete a subaccount from the list of connections.The subaccount will be disconnected and all configurations will be removed from the installation.
- Navigate to the overview page of a subaccount using the rightmost action button of the dashboard table to see further details and perform more tasks, such as exporting or importing the subaccount configuration, for example:



Related Information

Managing Member Authorizations in the Neo Environment
Copy a Subaccount Configuration [page 1692]
Renew the Certificate for a Subaccount [page 1697]
Find Your Subaccount ID (Multi-Cloud Environment) [page 1700]
Configure Custom Regions [page 1700]
Use a Custom IDP for Subaccount Configuration [page 1694]

1.4.2.2.1 Copy a Subaccount Configuration

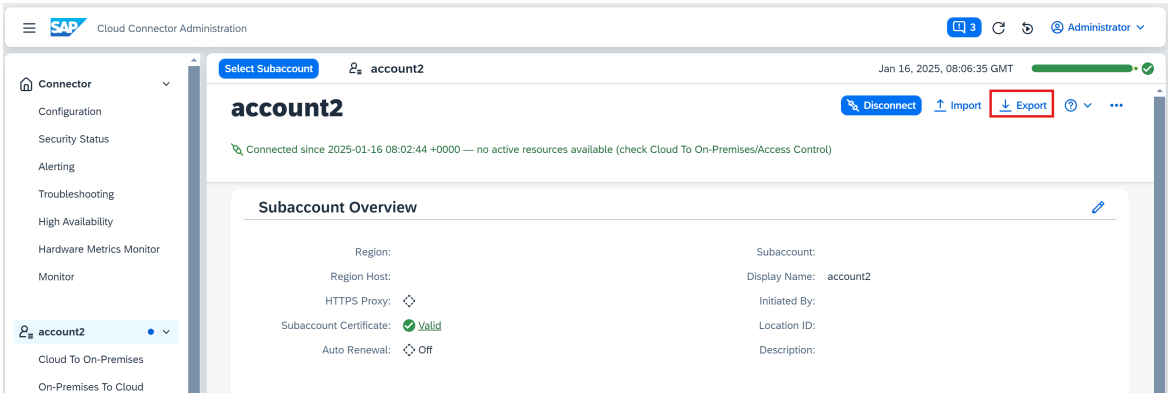
Copy an existing subcaccount configuration in the Cloud Connector to another subaccount.

You can copy the configuration of a subaccount's *Cloud To On-Premises* and *On-Premises To Coud* sections to a new subaccount, by using the export and import functions in the Cloud Connector administration UI.

> **Note**
>
> Principal propagation configuration (section *Cloud To On-Premises*) is not exported or imported, since it contains subaccount-specific data.

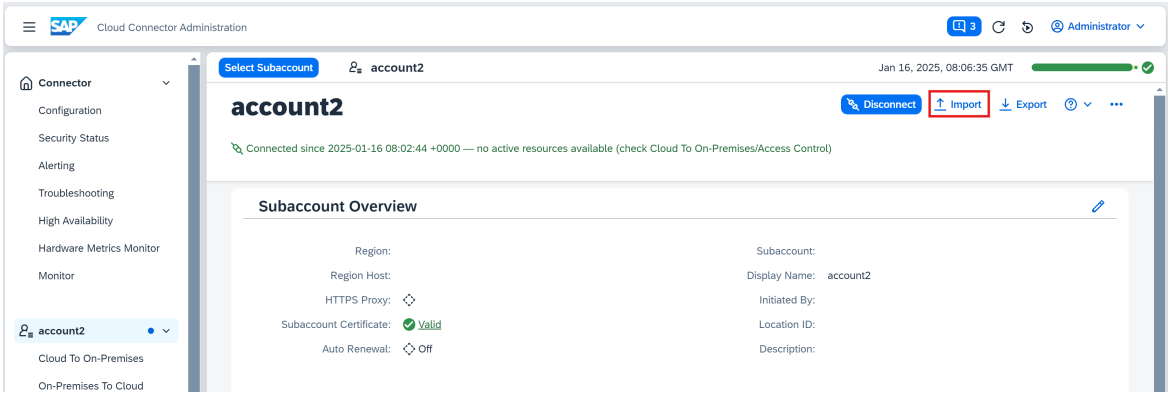
Procedure: Export an Existing Configuration

1. In the Cloud Connector administration UI, choose your subaccount from the navigation menu.
2. To export the existing configuration, choose the *Export* button in the upper right corner. The configuration is downloaded as a zip file to your local file system.

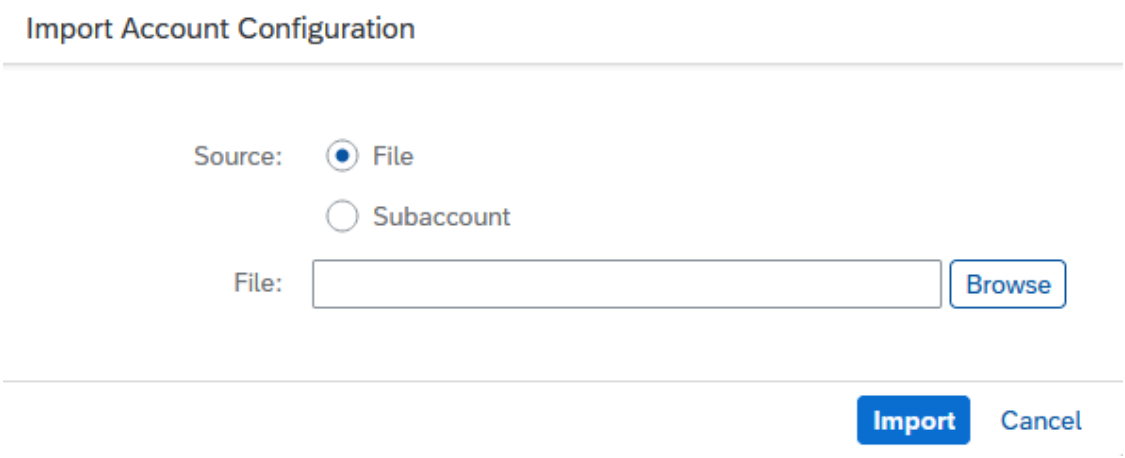


Procedure: Import an Existing Configuration

1. From the navigation menu, choose the subaccount to which you want to copy an existing configuration.
2. To import an existing configuration, choose the *Import* button in the upper right corner.



3. Select one of the following sources:
    1. *File*, if you want to copy the configuration from a previously downloaded zip file.
    2. *Subaccount*, if you want to copy the configuration directly from another existing subaccount.



1.4.2.2.2 Use a Custom IDP for Subaccount Configuration

Enable custom identity provider (IDP) authentication to configure a multi-cloud subaccount in the Cloud Connector by using a one-time passcode.

Content

Context [page 1694]

Get the URL [page 1696]

Get the One-Time Passcode [page 1696]

Context

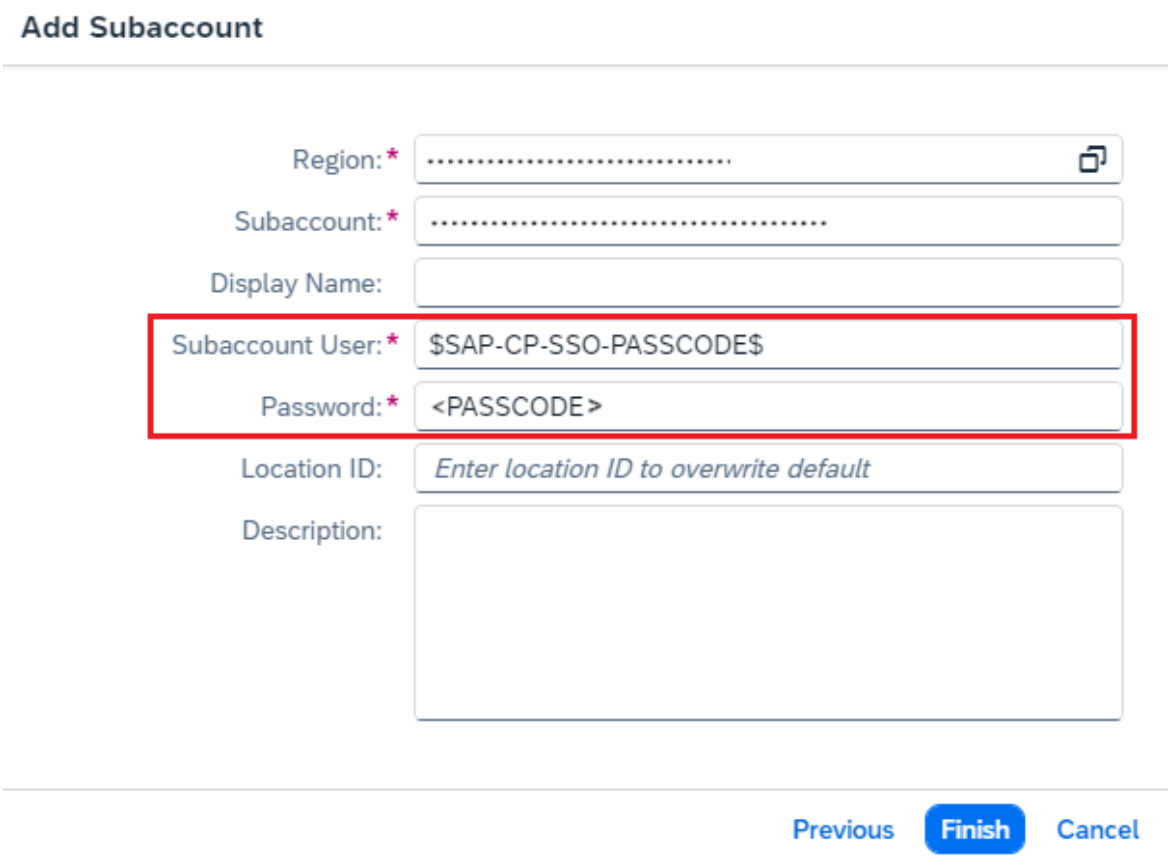
For a subaccount in the **multi-cloud** environment that uses a custom IDP, you can choose this IDP for authentication instead of the (default) SAP ID service when configuring the subaccount in the Cloud Connector.

Using custom IDP authentication, you can perform the following operations in the Cloud Connector:

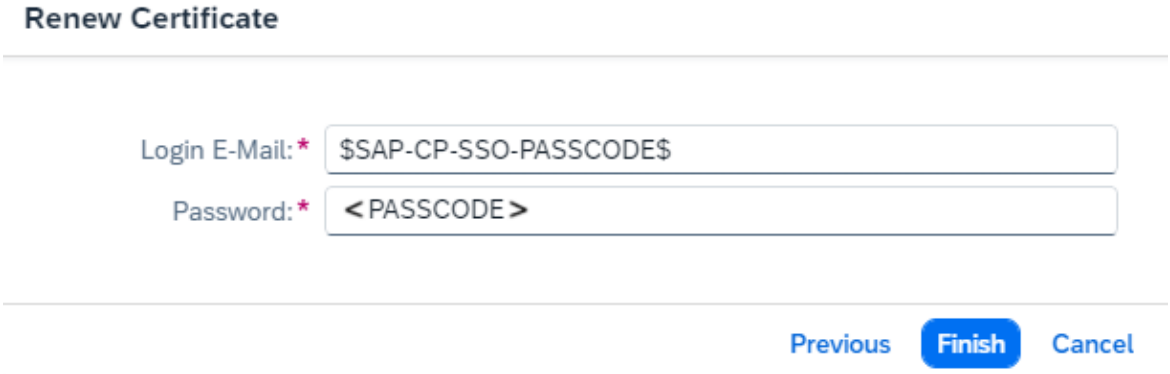
Operation	Description
Adding and Managing Subaccounts [page 1682]	Add a subaccount to a Cloud Connector installation.
Renew the Certificate for a Subaccount [page 1697]	Refresh a subaccount certificate's validity period.

To enable custom IDP authentication, for each of these operations you must enter the marker value **$SAP-CP-SSO-PASSCODE$** in the `<Subaccount User>` or `<User Name>` field, and a one-time generated passcode (known as *temporary authentication code*) in the `<Password>` field:

- When adding a subaccount to a Cloud Connector installation, enter the user name **$SAP-CP-SSO-PASSCODE$** and the passcode on the Cloud Connector's *Add Subaccount* screen:



- To **refresh a subaccount certificate**, enter the user name **$SAP-CP-SSO-PASSCODE$** and the passcode via the corresponding *<Subaccount>* screen (see also Renew the Certificate for a Subaccount [page 1697]):



To retrieve the one-time generated passcode, you must use the correct login URL for single sign-on (SSO) to access your custom IDP.

> → Tip
>
> As of Cloud Connector version 2.17, you can import subaccount authentication data to simplify the process when using custom IdPs. Instead of using the SSO passcode approach, we strongly recommend that you import an authentication data file.

For more information, see Adding and Managing Subaccounts [page 1682].

> ⚠ Caution
>
> Mind the required user rights described in the prerequisites for Initial Configuration [page 1670] and Adding and Managing Subaccounts [page 1682].

Back to Content [page 1694]

Get the URL

Choose one of the following options:

Option 1: Assemble the URL

The URL pattern is *https://<subdomain>.authentication.<btp-XSUAA-host>/passcode*.

1. Get the SAP BTP region host, for example, *eu10.hana.ondemand.com*.
2. Assemble the final URL to be used, in this case: *https://mysubdomain.authentication.eu10.hana.ondemand.com/passcode*.

Option 2: Get the URL Using the Connectivity Service Instance Credentials

1. Choose one of these steps to obtain the URL:
   - Create and Bind a Connectivity Service Instance [page 1309]
   - Create a service key
2. Get the value of the `token_service_url` attribute.
3. Append `/passcode` at the end of the obtained URL.

Next Step

Get the One-Time Passcode [page 1696]

Back to Content [page 1694]

Get the One-Time Passcode

1. Open the resulting URL in your browser to get the one-time passcode via SSO:
   - If *there is* an active user session, the passcode is generated automatically and returned right away.
   - If there is *no* active user session, you are asked to log on to the IDP manually. If several IDPs are configured, you can choose one from the available options.

2. Use the passcode to proceed with the subaccount configuration in the Cloud Connector UI.
Back to Content [page 1694]

1.4.2.2.3 Renew the Certificate for a Subaccount

Certificates used by the Cloud Connector are issued with a limited validity period. To prevent a downtime while refreshing the certificate, you can renew it for your subaccount directly from the administration UI.

> ⓘ Note
>
> As of version 2.18.0, the Cloud Connector provides an automatic renewal option for the *multi-cloud environment*.

Prerequisites

You must have the required subaccount authorizations on SAP BTP to renew certificates for your subaccount.

See:

- User Roles [page 1301] (**multi-cloud** environment)

Manual Renewal

> → Tip
>
> Check the validity of the subaccount certificate in section **Subaccount Overview** of the overview page of the respective subaccount. You can use the procedure described below even if the certificate has already expired.

Proceed as follows to renew your subaccount certificate:

1. Choose *Select Subaccount* at the top of the screen, select your subaccount, and navigate to the subaccount overview.
2. Choose *More (...)* > *Certificate* in the upper right corner of the screen. Depending on the cloud environment, you can renew using authentication data from a file or manually by providing the subaccount user and password, or only the latter. The respective dialog or wizard is shown.

> → Tip
>
> You can download an authentication data file from the *Cloud Connectors* view in your subaccount. Choose *Connectivity* > *Cloud Connectors* > *Download Authentication Data* from your subaccount menu. This feature is available for subaccounts in multi-cloud foundation only.

3. Enter `<Subaccount User>` and `<Password>` (or use the file with authentication data if the option is offered) and choose *Renew* (or *Finish*). The certificate assigned to your subaccount is renewed.

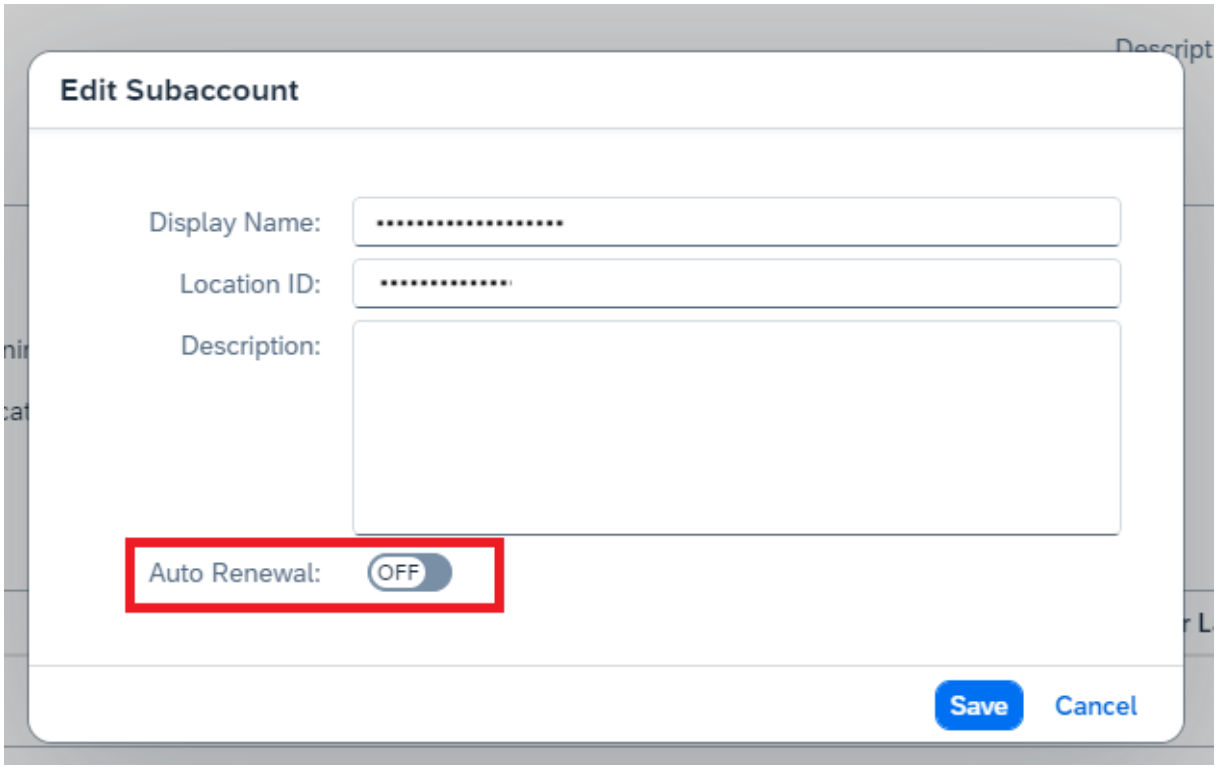
> → Tip
>
> When using SAP Cloud Identity Services - Identity Authentication (IAS) as platform identity provider with two-factor authentication (2FA / MFA) for your subaccount, you can simply append the required token to the regular password. For example, if your password is "eX7?6rUm" and the one-time passcode is "123456", you must enter "eX7?6rUm123456" into the `<Password>` field.
>
> **Appending the two-factor authentication** (2FA / MFA) token to the regular password **is not required** when updating the Cloud Connector subaccount certificate in the **Neo environment**.

Automatic Renewal

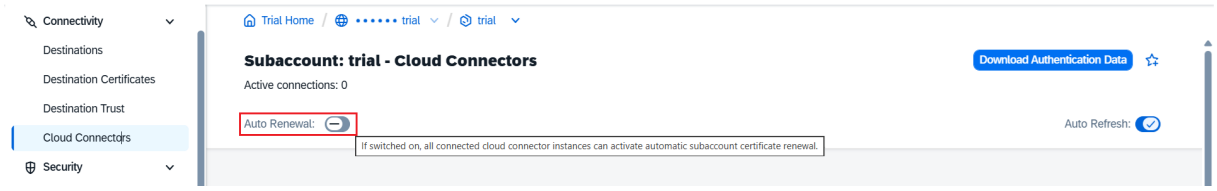
> ⓘ Note
>
> This feature is only supported for SAP BTP, *multi-cloud environment*.

As of Cloud Connector version 2.18.0, you can schedule the renewal of subaccount certificates to renew them automatically before they expire. This feature is subaccount-specific, therefore you must enable it for each subaccount you want to use it for. You must activate this feature for the respective subaccounts also on the cloud side (SAP BTP cockpit).

In the **Cloud Connector UI**, activate the feature for a subaccount by opening the *Edit* dialog for that subaccount and switching on *Auto Renewal*:



In the **SAP BTP cockpit**, activate the feature via *Connectivity* > *Cloud Connectors* > of the respective subacount by switching on *Auto Renewal*:



If automatic renewal is active, the date and time of the first renewal attempt is scheduled to occur n + 7 days before the certificate expires, where n is the minimal (remaining) validity period (in days) before an expiration alert is triggered. The minimal validity period is an alerting threshold that can be configured through the **Observation Configuration** of Alerting [page 2038]. A failed renewal attempt will be rescheduled and retried 12 hours later. If automatic renewal is not successful within 7 days it will be cancelled.

> ### ⚠ Restriction
>
> Currently, the automatic renewal is available only for subacounts in cloud regions that are running on the IaaS providers Amazon Web Services, Microsoft Azure, and SAP Cloud Infrastructure. For the time being, automatic renewal cannot be offered in cloud regions that are running on the IaaS providers Alibaba Cloud and Google Cloud.

1.4.2.2.4 Find Your Subaccount ID (Multi-Cloud Environment)

Get your subaccount ID to configure the Cloud Connector in the multi-cloud environment.

In order to set up your subaccount in the Cloud Connector, you must know the subaccount ID. Follow these steps to acquire it:

1. Open the SAP BTP cockpit.
2. Navigate to the subaccount list of the global account containing your subaccount: choose *Home* > *<Your Global Account>* > *Account Explorer*.
3. Find your subaccount in the list.
4. Choose *More Info* for the respective subaccount entry.

1.4.2.2.5 Configure Custom Regions

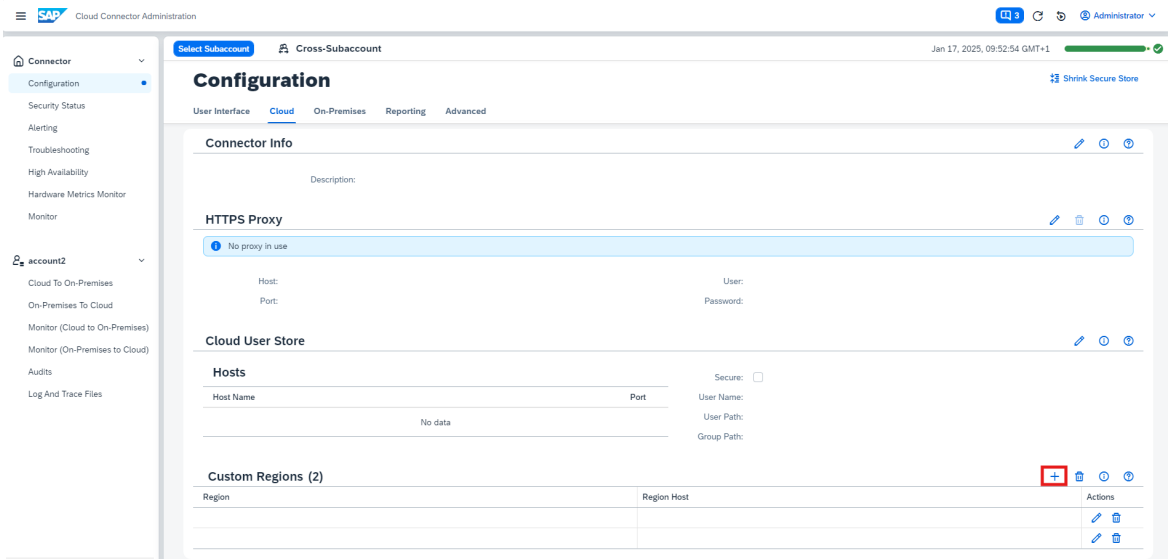
Configure regions that are not available in the standard selection.

> → Tip
>
> This procedure is useful in particular for regions introduced after the release of your current Cloud Connector version. Those regions are not included in the list of predefined regions.

If you want to use a custom region for your subaccount, you can configure regions in the Cloud Connector, which are not listed in the selection of standard regions.

To add a custom region, do the following:

1. From the Cloud Connector main menu, choose *Configuration* > *Cloud* and go to the *Custom Regions* section.
2. To add a region to the list, choose the *Add* icon.



3. In the *Add Region* dialog, enter the `<Region>` and `<Region Host>` you want to use.
4. Choose *Save*.
5. To edit a region from the list, select the corresponding line and choose the *Edit* icon.

1.4.2.3 Authenticating Users against On-Premises Systems

Authentication types supported by the Cloud Connector.

Currently, the Cloud Connector supports **basic authentication**, **principal propagation**, and **technical user propagation** as user authentication types towards internal systems. The destination configuration of the used cloud application defines which of these types is used for the actual communication to an on-premises system through the Cloud Connector, see Managing Destinations [page 1349].

- To use **basic authentication**, configure an on-premises system to accept basic authentication and to provide one or multiple service users. No additional steps are necessary in the Cloud Connector for this authentication type.
- To use **principal propagation** or **technical user propagation**, you must explicitly configure trust to those cloud entities from which user tokens are accepted as valid.
  For more information, see Configuring Principal Propagation [page 1702] and Configuring Technical User Propagation [page 1739].
- When using HTTP as communication protocol, you can also perform a logon using the Cloud Connector system certificate, if there is no principal provided by the cloud side. The access control entry must be configured for this.

> ### Note
>
> For HTTP, also some other token-based authentication methods might work between the cloud application and the backend, depending on the backend capabilities.

Related Information

Configuring Principal Propagation [page 1702]
Configuring Technical User Propagation [page 1739]

1.4.2.3.1 Configuring Principal Propagation

Use principal propagation to simplify the access of SAP BTP users to on-premises systems.

Tasks in this section:

Task	Description
Set Up Trust [page 1702]	Configure a trusted relationship in the Cloud Connector to support principal propagation. Principal propagation lets you forward the logged-on identity in the cloud to the internal system without requesting a password.
Configure a CA Certificate [page 1706]	Install and configure an X.509 certificate to enable support for principal propagation.
Configuring Identity Propagation to an ABAP System [page 1711]	Learn more about the different types of configuring and supporting principal propagation for a particular AS ABAP.
Configure Subject Patterns [page 1726]	Define a pattern identifying the user for the subject of the generated short-lived `X.509` certificate, as well as its validity period.
Configure a Secure Login Server [page 1731]	Configuration steps for Java Secure Login Server (SLS) support.
Configure Kerberos [page 1735]	The Cloud Connector lets you propagate users authenticated in SAP BTP via Kerberos against back-end systems. It uses the **Service For User and Constrained Delegation** protocol extension of Kerberos.
Configuring Identity Propagation to SAP NetWeaver AS for Java [page 1738]	Find step-by-step instructions on how to configure principal propagation to an application server Java (AS Java).

Related Information

Principal Propagation [page 30] (multi-cloud environment)

Principal Propagation (Neo environment)

1.4.2.3.1.1 Set Up Trust

Establish trust to an identiy provider to support principal propagation and technical user propagation.

Tasks

Configure Trusted Entities in the Cloud Connector [page 1703]

Configure an On-Premises System for Principal Propagation [page 1704]

Set up a Trust Store [page 1705]

Configure Trusted Entities in the Cloud Connector

> **Note**
>
> The information in this section applies to both principal propagation and technical user propagation.

You perform trust configuration to support principal propagation, that is, forwarding the *logged-on identity* in the cloud to the internal system without the need of providing the password. The same is done for technical user propagation, which logs on a *technical* user identified by an access token for an OAuth client without the need of providing the password.

By default, your Cloud Connector does not trust any entity that issues tokens for principal propagation. Therefore, the list of trusted identity providers (IdPs) is empty by default.

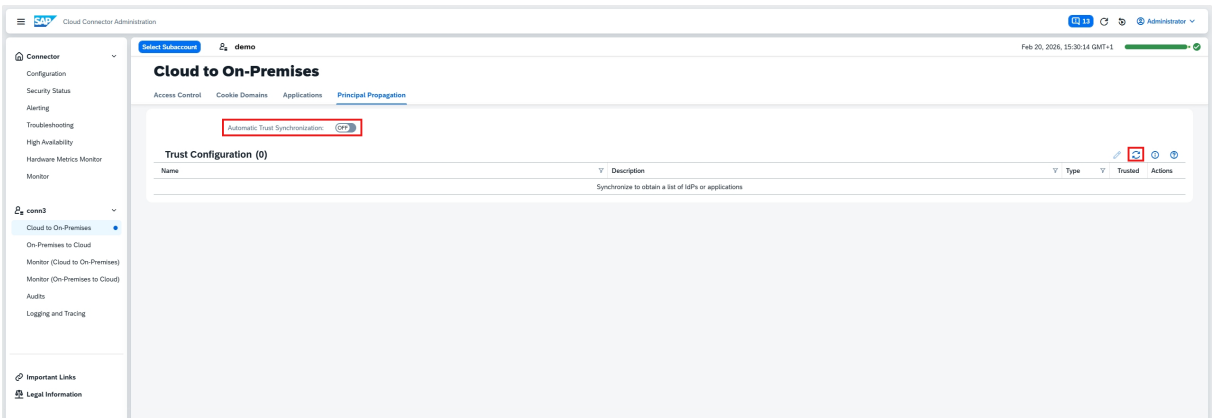
If you decide to use the principal propagation feature, you must establish trust to at least one IdP. The following IdP types are supported:

- **Neo** environment: *SAML* IdPs.

  > **Note**
  >
  > In the Neo environment, you can also trust *HANA instances* and *Java applications* to act as IdPs.

- **Multi-Cloud** environment: *OAuth* IdPs.

You can configure trust to one or more IdPs per subaccount. After you've configured trust in the cockpit for your subaccount, for example, to your own company's IdP(s), you can synchronize this list with your Cloud Connector.



From your subaccount menu, choose *Cloud to On-Premises* and go to the *Principal Propagation* tab. Choose the *Synchronize* button to store the list of existing identity providers and applications locally in your Cloud Connector.

Select an entry to see its details:

- `Name`: the name associated with the identity provider or application of the entry.
- `Description`: descriptive information about the entry.
- `Type`: trust type of the entry.
- `Trusted`: indicates whether the entry is trusted for principal propagation.
- `Actions`: Choose the *Show Certificate Information* icon in the *Actions* column to display detail information for the corresponding entry if such a certificate is available. The icon is disabled for applications.

The synchronization process can also be automated by enabling the *Automatic Trust Synchronization* switch, which instructs the Cloud Connector to synchronize the trust configuration at regular intervals. In particular, if the automatic signing key rotation is active you should consider to also enable the automatic synchronization.

You can decide for each entry, whether to trust it for the principal propagation use case by choosing *Edit* and (de)selecting the *Trusted* checkbox.

> ### Note
>
> Whenever you update an IdP configuration for a subaccount on cloud side, you must synchronize the trusted entities in the Cloud Connector. Otherwise the validation of the forwarded SAML assertion will fail with an exception (in case of SAML assertions containing an exception message similar to this: *Caused by: com.sap.engine.lib.xml.signature.SignatureException: Unable to validate signature -> java.security.SignatureException: Signature decryption error: javax.crypto.BadPaddingException: Invalid PKCS#1 padding: encrypted message and modulus lengths do not match!*).

For more information, see also Include Tokens from Corporate Identity Providers or Identity Authentication in Tokens of the SAP Authorization and Trust Management Service.

Back to Tasks [page 1703]

Configure an On-Premises System for Principal Propagation

Set up principal propagation from SAP BTP to your internal system that is used in a hybrid scenario.

1. Set up trust to an entity that is issuing an assertion for the logged-on user (see section above).
2. Set up the system identity for the Cloud Connector.
   - For HTTPS and WebSocket RFC, you must import a system certificate (X.509 certificate) into your Cloud Connector.
     For more information, see Install a System Certificate for Mutual Authentication [page 1676].
   - For RFC SNC, you must use an SNC PSE for your Cloud Connector.
3. Configure the target system to trust the Cloud Connector.
   There are two levels of trust:
   1. First, you must allow the Cloud Connector to identify itself with its system certificate (for HTTPS and WebSocket RFC), or with the SNC PSE (for RFC SNC).

2. Then, you must allow this identity to propagate the user accordingly:
    - For HTTPS, the Cloud Connector forwards the true identity in a short-lived `X.509` certificate in an HTTP header named **`SSL_CLIENT_CERT`**. The system must use this certificate for logging on the real user. The SSL handshake, however, is performed through the system certificate. For more information on identity forwarding, see Configure Access Control (HTTP) [page 1744].
    - For WebSocket RFC, the Cloud Connector will forward the true identity in a short-lived X.509 certificate in an HTTP header named `SSL_CLIENT_CERT` while performing the WebSocket upgrade when connecting to the back end. The TLS handshake, however, is performed through the system certificate. The trigger to use the forwarded certificate for logon is sent as part of the RFC protocol.
    - For RFC, the Cloud Connector forwards the true identity as part of the RFC protocol.

    For more information, see Configuring Identity Propagation to an ABAP System [page 1711].
4. Configure the user mapping in the target system. The `X.509` certificate contains information about the cloud user in its subject. Use this information to map the identity to the appropriate user in this system. This step applies for both HTTPS and RFC.

> **Note**
>
> If you have the following scenario: *Application1->AppToAppSSO->Application2->Principal Propagation->On premise Backend System* you must mark *Application2* as **`trusted`** by the Cloud Connector in tab *Principal Propagation*, section *Trust Configuration*.
>
> If you use an identity provider that issues unsigned assertions, you must mark all relevant applications as **`trusted`** by the Cloud Connector in tab *Principal Propagation*, section *Trust Configuration*.

Back to Tasks [page 1703]

Set up a Trust Store

Configure a trust store that acts as an allowlist for trusted on-premises systems. See Configure Trust [page 1955].

Back to Tasks [page 1703]

Related Information

Principal Propagation [page 30] (multi-cloud)

Principal Propagation (Neo)

1.4.2.3.1.2 Configure a CA Certificate

Install and configure an X.509 certificate to enable support for principal propagation in the Cloud Connector.

> ⓘ Note
>
> The information in this section applies to both principal propagation and technical user propagation.

Supported CA Mechanisms

You can enable support for principal propagation or technical user propagation with X.509 certificates by performing either of the following procedures:

- Using a *Local CA* in the Cloud Connector.

  > ⓘ Note
  >
  > Prior to version 2.7.0, this was the only option and the system certificate was acting both as client certificate and CA certificate in the context of principal propagation.

- Using a *Secure Login Server* (SLS) and delegate the CA functionality to it.

  > ⓘ Note
  >
  > You cannot use the *Secure Login Service* (cloud service) for this purpose.

The Cloud Connector uses the configured CA approach to issue short-lived certificates for logging on the same identity in the back end that is logged on in the cloud. For establishing trust with the back end, the respective configuration steps are independent of the approach that you choose for the CA.

Install a local CA Certificate

To issue short-lived certificates that are used for principal propagation to a back-end system, you can import an X.509 client certificate into the Cloud Connector. This CA certificate must be provided as *PKCS#12* file containing the (intermediate) certificate, the corresponding private key, and the CA root certificate that signed the intermediate certificate (plus the certificates of any other intermediate CAs, if the certificate chain includes more than those two certificates).

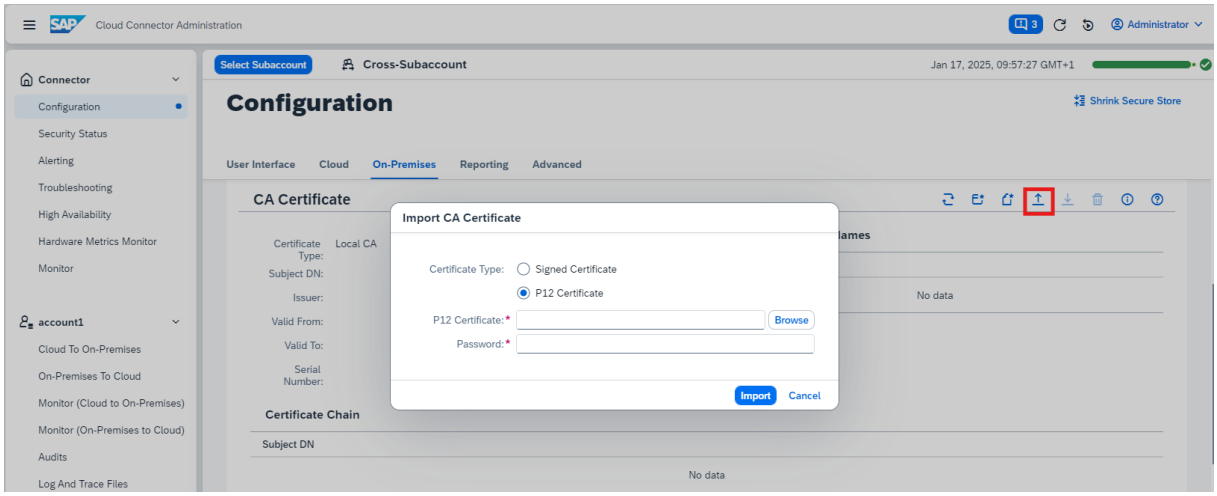
> ⚠ Caution
>
> Currently, only RSA certificates can be used as CA certificate. ECDSA certificates won't work at runtime, even if they could be imported successfully.

Use either of the following options to install a local CA certificate:

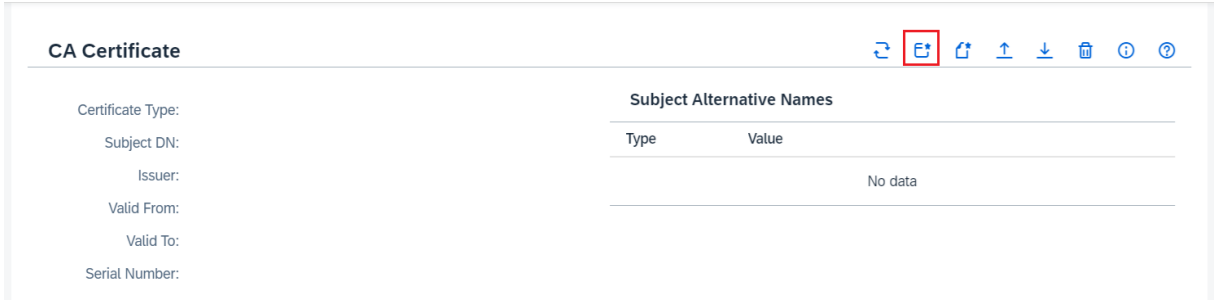
- Option 1: Choose the PKCS#12 file from the file system, using the file upload dialog. For the import process, you must also provide the file password.
- Option 2: Start a *Certificate Signing Request* (CSR) procedure like for the UI certificate, see Exchange UI Certificates in the Administration UI [page 1971].
- Option 3: Generate a self-signed certificate, which might be useful in a demo setup or if you need a dedicated CA. In particular for this option, it is useful to export the public key of the CA via the button *Download certificate in DER format*.

> ⚠ Caution
>
> The CA certificate should have the `KeyUsage` attribute `keyCertSign`. Many systems verify that the issuer of a certificate has this attribute and deny a client certificate, if this attribute is not present. When using the *Certificate Signing Request* procedure, the attribute will be requested for the CA certificate. Also, when generating a self-signed certificate, this attribute will be added automatically.

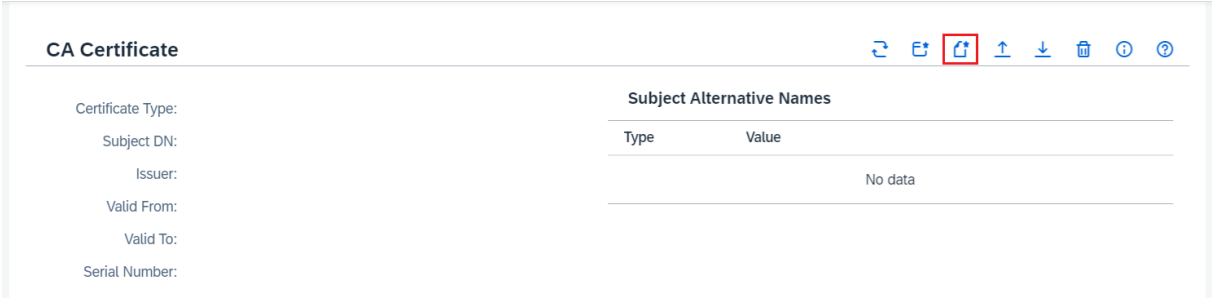
Choose *Import a certificate* for option 1:



Choose *Generate a certificate signing request* for option 2:



Choose *Create and import a self-signed certificate* if you want to use option 3:



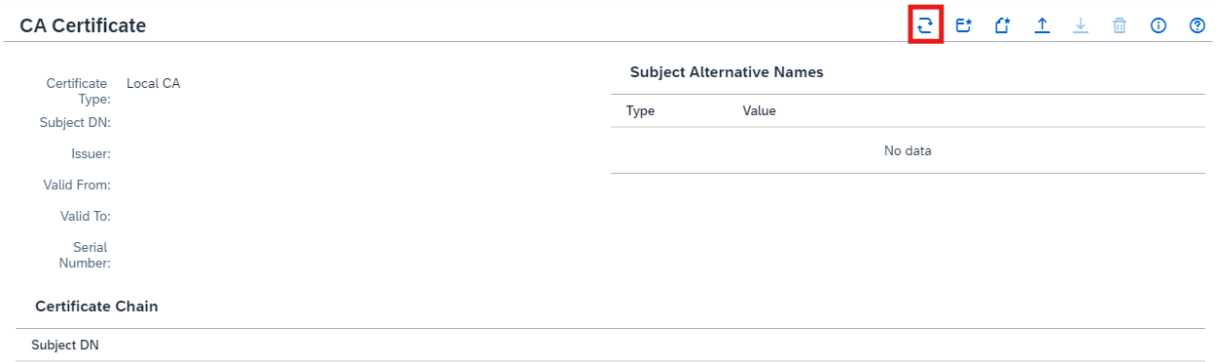
In particular for this option, it is useful to export the public key of the CA by choosing the respective button.

After successful import of the CA certificate, its distinguished name, the name of the issuer, and the validity dates are shown.

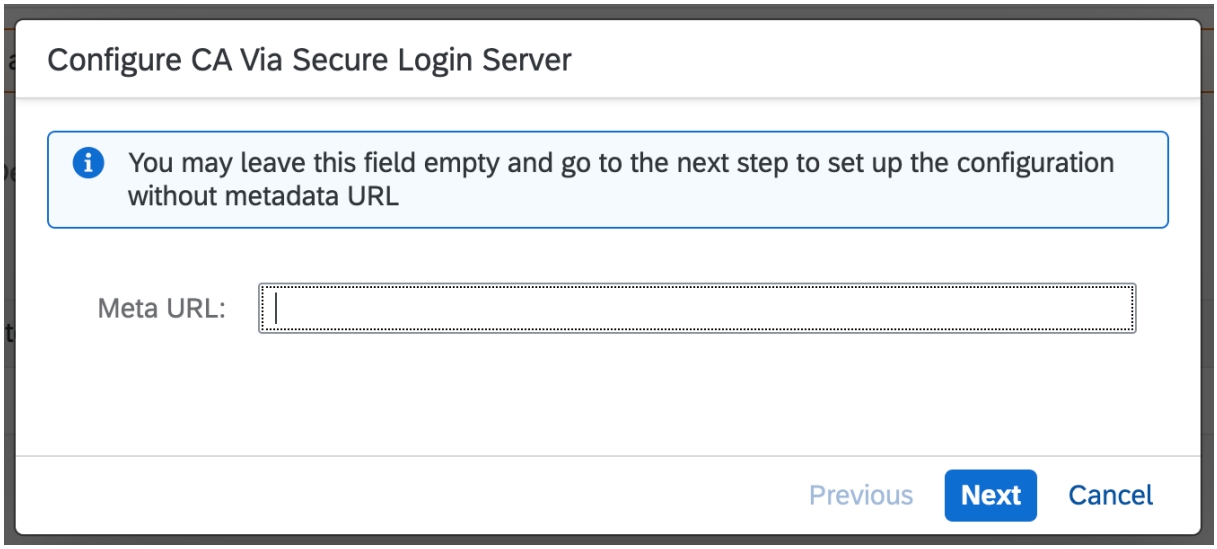
If a CA certificate is no longer required, you can delete it. Use the respective *Delete* button and confirm the deletion.

Configure a CA Hosted by a Secure Login Server

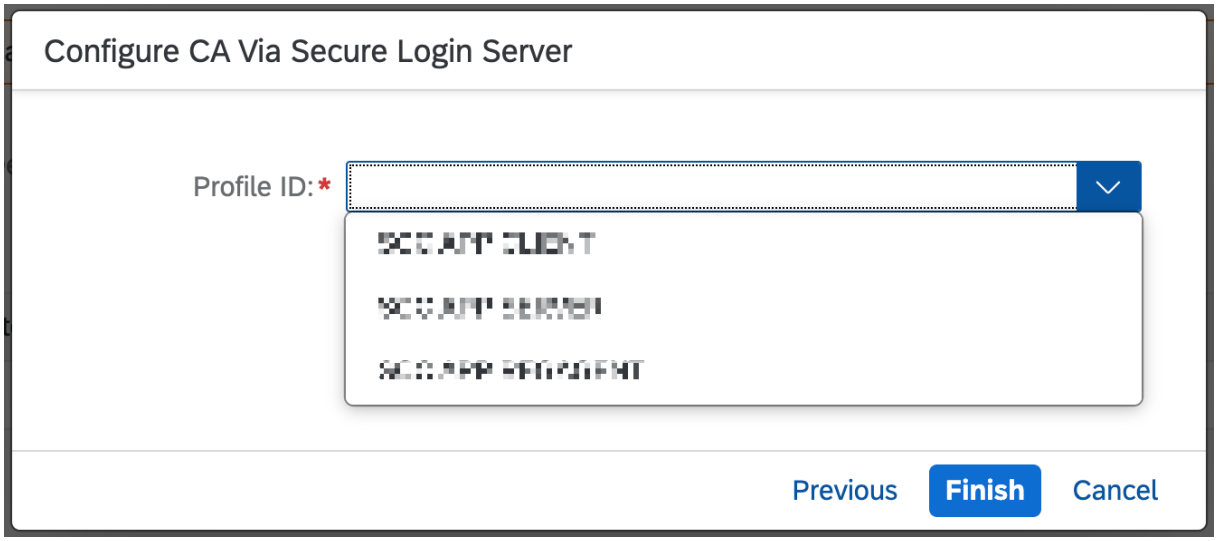
If you want to delegate the CA functionality to a *Secure Login Server* (SLS), choose the CA using the *Secure Login Server* option and configure the SLS as follows, after having configured the SLS as described in Configure a Secure Login Server [page 1731].



A wizard offers in a first step a quick configuration by metadata URL pointing to the SLS you'd like to use.



Using the metadata URL lets you fetch the most relevant data from SLS instance. You only have to choose the profile configured on SLS, which should be used for the generation of short-lived certificates. Choose *Finish* to save the configuration.



> **Note**
>
> The URL won't be stored in the Cloud Connector configuration.

If you don't have the metadata URL or would change the current configuration without metadata URL, you may keep the field empty and go to the next step.

In that case, you must provide all configuration details manually.

Enter the following:

- `API Version`: Version of the SLS configuration protocol.

- `Host Name`: Host on which your SLS is installed.
- `Port`: Port for communication with SLS during the configuration.
- `Profile ID`: SLS profile ID that allows to issue certificates as needed for principal propagation with the Cloud Connector.

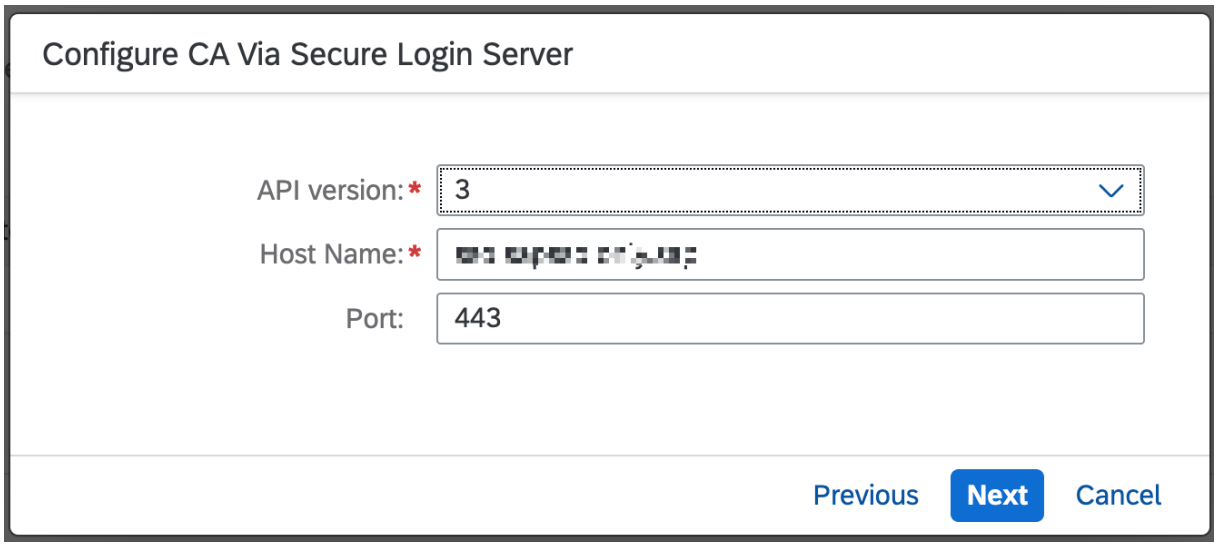
  > **Note**
  >
  > Used if SLS API *version 3* is configured.

- `Profile`: SLS profile that allows to issue certificates as needed for principal propagation with the Cloud Connector.

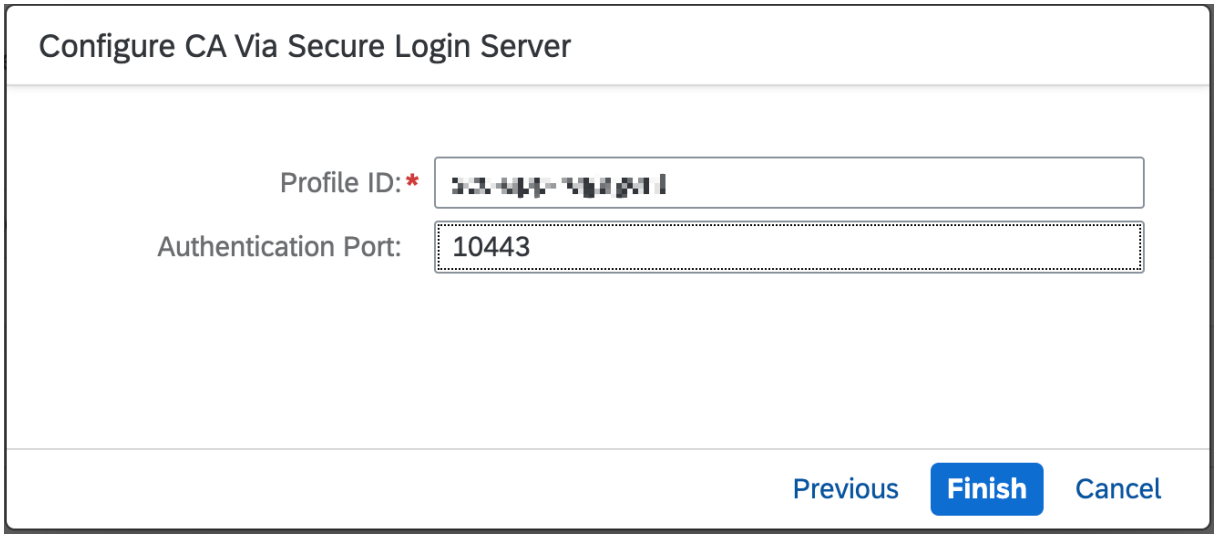
  > **Note**
  >
  > Used if SLS API *version 2* is configured.

- `Authentication Port`: Port over which the Cloud Connector is requesting the short-lived certificates from SLS.

  > **Note**
  >
  > For this privileged port, a client certificate authentication is required, for which the Cloud Connector's system certificate is used.



In the next step, you can finalize the configuration. The fields in the next step depend on the chosen SLS API version.



Choose *Finish* to save the configuration.

Related Information

Configure a Secure Login Server [page 1731]
Initial Configuration (HTTP) [page 1676]
Initial Configuration (RFC) [page 1678]

1.4.2.3.1.3 Configuring Identity Propagation to an ABAP System

Learn more about the different types of configuring and supporting principal propagation and technical user propagation for a particular AS ABAP.

> **Note**
>
> The information in this section applies to both principal propagation and technical user propagation.

Task	Description
Configure Identity Propagation for HTTPS [page 1712]	Step-by-step instructions to configure principal propagation to an ABAP server for HTTPS.
Configure Identity Propagation via SAP Web Dispatcher [page 1717]	Set up a trust chain to use principal propagation to an ABAP server for HTTPS via SAP Web Dispatcher.

Task	Description
Configure Identity Propagation for RFC [page 1721]	Step-by-step instructions to configure principal propagation to an ABAP server for RFC.
Rule-Based Mapping of Certificates [page 1724]	Map short-lived certificates to users in the ABAP server.

1.4.2.3.1.3.1 Configure Identity Propagation for HTTPS

Find step-by-step instructions to configure principal propagation to an ABAP server for HTTPS.

> **Note**
>
> The information in this section applies to both principal propagation and technical user propagation. These two types of user propagation are combined as *identity* propagation.

Example Data

The following data are used in this example:

- System certificate was issued by: **`CN=MyCompany CA,O=Trust Community,C=DE`**.
- It has the subject: **`CN=SCC`**, **`OU=BAIP Scenarios,O=Trust Community,C=DE`**.
- The short-lived certificate has the subject **`CN=P1234567890`**, where `P1234567890` is the platform user.

Tasks

Prerequisites [page 1712]

Configure an ABAP System to Trust the Cloud Connector's System Certificate [page 1713]

Map Short-Lived Certificates to Users [page 1715]

Access ICF Services [page 1715]

Prerequisites

- For identity propagation, mutual authentication (mTLS) is mandatory.
  To enable mTLS, you must configure a system certificate as described in this topic. Follow step 1 below to make sure the ABAP system trusts this certificate. You can use the connection check and the *Details* button to check if mTLS is working.

> ### Note
>
> The system certificate is only used to enable mTLS (that is, to establilsh trust between the ABAP system and the Cloud Connector). For the actual user propagation process (issuing short-lived certificates for the logged-on user identity in the Cloud Connector), you must also configure a CA certificate.
>
> For more information, see Configure a CA Certificate [page 1706].

- The access control entry (see Configure Access Control (HTTP) [page 1744]) must have specified the respective identity type to forward the identity correctly.

To perform the following steps, you must have the corresponding authorizations in the ABAP system for the transactions mentioned below (administrator role according to your specific authorization management) as well as an administrator user for the Cloud Connector.

Back to Tasks [page 1712]

Back to Example Data [page 1712]

1. Configure an ABAP System to Trust the Cloud Connector's System Certificate

This step includes two sub-steps:

Configure the ABAP system to trust the Cloud Connector's system certificate [page 1713]

Configure the Internet Communication Manager (ICM) to trust the system certificate for principal propagation [page 1714]

Configure the ABAP system to trust the Cloud Connector's system certificate:

1. Open the *Trust Manager* (transaction STRUST).
2. Double-click the *SSL-Server Standard* folder in the menu tree on the left.
3. In the displayed screen, click the *Import certificate* button.
4. In the dialog window, choose the certificate file representing the public key of the issuer of the system certificate, for example, in DER format. Typically, this is a CA certificate. If you decide to use a self-signed system certificate, it is the system certificate itself.
5. The details of this certificate are shown in the section above. Using the example certificate data, you would see C**`N=MyCompany CA`**, **`O=Trust Community`**, **`C=DE`** as subject.
6. If you are sure that you are importing the correct certificate, you can integrate the certificate into the certificate list by choosing the *Add to Certificate List* button.
7. Now, the CA certificate (**`CN=MyCompany CA`**, **`O=Trust Community`**, **`C=DE`**) is part of the certificate list.

Back to Step [page 1713]

Configure the Internet Communication Manager (ICM) to trust the system certificate for identity propagation:

1. Open the *Profile Editor* (transaction `RZ10`).
2. Select the profile you want to edit, for example, the DEFAULT profile.
3. Select the *Extended maintenance* radio button and choose the *Change* button.
4. Create the following parameter: `icm/trusted_reverse_proxy_<x> = SUBJECT="<subject>", ISSUER="<issuer>"`.
   - Select a free index for **`<x>`**.
   - **`<subject>`** is the subject of the system certificate (example data: **`CN=SCC`**, **`OU=BAIP Scenarios`**, **`O=Trust Community`**, **`C=DE`**).
   - **`<issuer>`** is the issuer of the system certificate (example data: **`CN=MyCompany CA`**, **`O=Trust Community`**, **`C=DE`**).
   - Example: `icm/trusted_reverse_proxy_2 = SUBJECT="CN=SCC, OU=BAIP Scenarios, O=Trust Community, C=DE", ISSUER="CN=MyCompany CA, O=Trust Community, C=DE"`.

   > ### Note
   >
   > If your ABAP system uses kernel 7.42 or lower, see SAP note 2052899 or set the following two parameters:
   >
   > - `icm/HTTPS/trust_client_with_issuer`: this is the issuer of the system certificate (example data: **`CN=MyCompany CA`**, **`O=Trust Community`**, **`C=DE`**).
   > - `icm/HTTPS/trust_client_with_subject`: this is the subject of the system certificate (example data: **`CN=SCC`**, **`OU=BAIP Scenarios`**, **`O=Trust Community`**, **`C=DE`**).

   > ### Caution
   >
   > In all kernel releases below 7.53, the ICM expects *blanks after the separating comma* for the issuer and subject elements. Even if the Cloud Connector administration UI shows a string without blanks, for example: *CN=SCC,OU=BAIP Scenarios,O=Trust Community,C=DE*, you must specify in ICM: *CN=SCC, OU=BAIP Scenarios, O=Trust Community, C=DE*.
   >
   > As of kernel release 7.53, this restriction does not exist any more. For more information, see SAP note 3335949.

   > ### Caution
   >
   > Make sure the parameter `icm/server_port` you are using has the attribute `VCLIENT`=1 (request certificate) or 2 (require certificate). The default value is 1.
   >
   > If `VCLIENT` is set to 0, trust cannot be established. In this case, you may change the value to 1, but beware that this may have a negative influence on the user experience. Alternatively, create a new parameter `icm/server_port` with `VCLIENT`=1 and configure the Cloud Connector to use this parameter.

5. Save the profile.
6. Open the *ICM Monitor* (transaction `SMICM`) and restart the ICM by choosing *Administration* > *ICM* > *Exit Hard* > *Global*.
7. Verify that the two profile parameters have been taken over by ICM by choosing *Goto* > *Parameters* > *Display*.

> ### Note
>
> If you have an SAP Web Dispatcher installed in front of the ABAP system, trust must be added in its configuration files with the same parameters as for the ICM. Also, you must add the system certificate of the Cloud Connector to the trust list of the Web dispatcher Server PSE. For more information, see Configure Identity Propagation via SAP Web Dispatcher [page 1717].

> ### Caution
>
> When using identity propagation with X.509 certificates, you cannot use the strict mode in certificate block management (transaction code: CRCONFIG) for the CRL checks within profile *SSL_SERVER*.

Back to Step [page 1713]

Back to Tasks [page 1712]

Back to Example Data [page 1712]

2. Map Short-Lived Certificates to Users

For systems later than SAP NetWeaver 7.3 EHP1 (7.31), you can use rule-based certificate mapping, which is the recommended way to create the required user mappings. For more information, see Rule-Based Mapping of Certificates [page 1724].

In older releases (for which this feature does not exist yet), you can do this manually in the system as described below, or use an identity management solution generating the mapping table for a more comfortable approach.

1. Open *Assignment of External ID to Users* (transaction `EXTID_DN`).
2. Switch to the edit mode.
3. Create a new entry. Specify the subject of the certificate as `External ID`. When using the example data, this is **`CN=P1234567890`**. In the *User* field, provide the appropriate ABAP user, for example **`JOHNSMITH`**.
4. Choose *Activate*.
5. Save the mapping.
6. Repeat the previous steps for all users that should be supported for the scenario.

Back to Tasks [page 1712]

Back to Example Data [page 1712]

3. Access ICF Services

To access the required ICF services for your scenario in the ABAP system, choose one of the following procedures:

For Cloud Connector version 2.15 or higher:

- To access ICF services via **certificate logon** (using the system certificate), select *Allow Principal Propagation*, choose the principal type **`X.509 Certificate`**, and select *System Certificate for Logon* in the corresponding system mapping.
  This setting lets you use the system certificate for trust in case of principal propagation as well as for user authentication. For details, see Configure Access Control (HTTP) [page 1744], step 7, 8 (*Procedure for Cloud Connector version 2.15 and higher*), and 9.
  Additionally, make sure that all required ICF services allow *Logon Through SSL Certificate* as logon method.
- To access ICF services via the logon method **Basic Authentication** (logon with user/password) and identity propagation, select *Allow Principal Propagation*, choose the principal type **`X.509 Certificate`**, and do *not* select *System Certificate for Logon* in the corresponding system mapping.
  This setting lets you use the system certificate for trust, but prevents its usage for user authentication. For details, see Configure Access Control (HTTP) [page 1744], step 7, 8 (*Procedure for Cloud Connector version 2.15 and higher*), and 9.
  Additionally, make sure that all required ICF services allow *Basic Authentication* and *Logon Through SSL Certificate* as logon method.
- If some of the ICF services require `Basic Authentication`, while others should be accessed via system certificate logon, perform these steps:
  1. In the Cloud Connector system mapping, select *Allow Principal Propagation*, choose the principal type **`X.509 Certificate`**, and select *System Certificate for Logon* in the corresponding system mapping as described above.
  2. In the ABAP system, choose transaction code SICF and go to *Maintain Services*.
  3. Select the service that requires `Basic Authentication` as logon method.
  4. Double-click the service and go to tab *Logon Data*.
  5. Switch to *Alternative Logon Procedure* and ensure that the `Basic Authentication` logon procedure is listed before `Logon Through SSL Certificate`.

For Cloud Connector versions below 2.15:

- To access ICF services via certificate logon, choose the principal type **`X.509 Certificate (general usage)`** in the corresponding system mapping. This setting lets you use the system certificate for trust as well as for user authentication.
  For details, see Configure Access Control (HTTP) [page 1744], step 8 (*Procedure for Cloud Connector versions below 2.15*).
  Additionally, make sure that all required ICF services allow `Logon Through SSL Certificate` as logon method.
- To access ICF services via the logon method `Basic Authentication` (logon with user/password) and principal propagation, choose the principal type **`X.509 Certificate (strict usage)`** in the corresponding system mapping. This setting lets you use the system certificate for trust, but prevents its usage for user authentication.
  For details, see Configure Access Control (HTTP) [page 1744], step 8 (*Procedure for Cloud Connector versions below 2.15*).
  Additionally, make sure that all required ICF services allow `Basic Authentication` and `Logon Through SSL Certificate` as logon methods.
- If some of the ICF services require `Basic Authentication`, while others should be accessed via system certificate logon, perform these steps:

1. In the Cloud Connector system mapping, choose the principal type **`X.509 Certificate (general usage)`** as described above.
2. In the ABAP system, choose transaction code SICF and go to *Maintain Services*.
3. Select the service that requires `Basic Authentication` as logon method.
4. Double-click the service and go to tab *Logon Data*.
5. Switch to *Alternative Logon Procedure* and ensure that the `Basic Authentication` logon procedure is listed before `Logon Through SSL Certificate`.

> **Note**
>
> If you are using SAP Web Dispatcher for communication, you must configure it to forward the SSL certficate to the ABAP backend system, see Forward SSL Certificates for X.509 Authentication (SAP Web Dispatcher documentation).

Back to Tasks [page 1712]

Back to Example Data [page 1712]

Related Information

Configure Identity Propagation via SAP Web Dispatcher [page 1717]

Rule-Based Mapping of Certificates [page 1724]

Configure Subject Patterns [page 1726]

Set Up Trust [page 1702]

Principal Propagation [page 30] (multi-cloud environment)

Principal Propagation (Neo environment)

Principal Propagation from SAP BAIP Applications to On-Premise Systems Using Corporate IdP Tokens (multi-cloud environment)

1.4.2.3.1.3.1.1 Configure Identity Propagation via SAP Web Dispatcher

Set up a trust chain to use identity propagation to an ABAP System for HTTPS via SAP Web Dispatcher.

> **Note**
>
> The information in this section applies to both principal propagation and technical user propagation. These two types of user propagation are combined as *identity* propagation.

Concept

If you are using an intermediate SAP Web Dispatcher to connect to your ABAP backend system, you must set up a trust chain between the involved components Cloud Connector, SAP Web Dispatcher, and ABAP backend system.

Before configuring the ABAP system (see Configure Identity Propagation for HTTPS [page 1712]), in a first step you must configure SAP Web Dispatcher to accept and forward user principals propagated from a cloud account to an ABAP backend.

To do this, follow the step-by-step instructions below.

Example Data

The following data and setup is used for this scenario:

- System certificate was issued by `CN=MyCompany CA, O=Trust Community, C=DE`
- It has the subject `CN=SCC, OU=BTP Scenarios, O=Trust Community, C=DE`

Tasks

- Prerequisites [page 1718]
- Configure the SAP Web Dispatcher to Trust the Cloud Connector's System Certificate [page 1719]
- Next Steps [page 1720]

Prerequisites

- Your SAP Web Dispatcher version is 7.53 or higher. See SAP note 908097 for information on recommended SAP Web Dispatcher versions.
- We recommend that you use a standalone Web Dispatcher deployment. To learn about deployment options, see SAP note 3115889.
- Make sure your SAP Web Dispatcher supports SSL. See Configure SAP Web Dispatcher to Support SSL.
- Ensure that TLS client certificates can be used for authentication in the backend system. See How to Configure SAP Web Dispatcher to Forward SSL Certificates for X.509 Authentication for step-by-step instructions.

> ⚠ Caution
>
> In case of identity propagation, make sure you do not use a *Web Application Firewall* (WAF) or a *Reverse Proxy* between the Cloud Connector and the SAP Web Dispatcher. This would terminate TLS, and disables functions like E2E trust and MITM attack prevention.

Back to Tasks [page 1718]

Back to Concept [page 1718]

Configure SAP Web Dispatcher to Trust the Cloud Connector's System Certificate

To allow Cloud Connector client certificates for authentication in the backend system, perform the following two steps:

1. **Configure SAP Web Dispatcher to trust the Cloud Connector's system certificate:**
   1. To import the system certificate to SAP Web Dispatcher, open the SAP Web Dispatcher administration interface in your browser.

      > ⓘ Note
      >
      > The interface is usually configured on `/sap/wdisp/admin`.

   2. In the menu, navigate to *SSL and Trust Configuration* and select *PSE Management*.
   3. In the **Manage PSE** section, select **`SAPSSLS.pse`** from the drop-down list. By default, SAPSSLS.pse contains the server certificate and the list of trusted clients that SAP Web Dispatcher trusts as a server.
   4. In the **Trusted Certificates** section, choose *Import Certificate*.
   5. Enter the certificate as *base64-encoded* into the text box. The procedure to export your certificate in such a format is described in Forward SSL Certificates for X.509 Authentication, step 1.

      > ⓘ Note
      >
      > Typically, this is a CA certificate. If you are using a self-signed system certificate, it's the system certificate itself.

   6. Choose *Import*.
   7. The certificate details are now shown in section **Trusted Certificates**.
2. **Configure SAP Web Dispatcher to trust the Cloud Connector's system certificate for identity propagation:**
   - Create or edit the following parameter in SAP Web Dispatcher:
     `icm/trusted_reverse_proxy_<x> = SUBJECT="<subject>", ISSUER="<issuer>"`
     - Select a free index for `<x>`.
     - `<subject>`: Subject of the system certificate (example data: `CN=SCC, OU=BTP Scenarios, O=Trust Community, C=DE`)

- `<issuer>`: Issuer of the system certificate (example data: `CN=MyCompany CA, O=Trust Community, C=DE`)

Example: `icm/trusted_reverse_proxy_0 = SUBJECT="CN=SCC, OU=BTP Scenarios, O=Trust Community, C=DE", ISSUER="CN=MyCompany CA, O=Trust Community, C=DE"`

> ⚠ Caution
>
> The ICM expects *blanks after the separating comma* for the issuer and subject elements. So, even if the Cloud Connector administration UI shows a string without blanks, for example: *CN=SCC,OU=BTP Scenarios,O=Trust Community,C=DE*, you must specify in ICM: *CN=SCC, OU=BTP Scenarios, O=Trust Community, C=DE*.

- **[Deprecated]** Create or edit the following two parameters in SAP Web Dispatcher:

> ⓘ Note
>
> Use the parameters below (instead of `icm/trusted_reverse_proxy_<x>`) only if your kernel release does not yet support parameter `icm/trusted_reverse_proxy_<x>`.

  - `icm/HTTPS/trust_client_with_issuer`: Issuer of the system certificate (example data: `CN=MyCompany CA, O=Trust Community, C=DE`)
  - `icm/HTTPS/trust_client_with_subject`: Subject of the system certificate (example data: `CN=SCC, OU=BTP Scenarios, O=Trust Community, C=DE`)

> ⓘ Note
>
> Make sure `icm/HTTPS/verify_client` is set to **`1`** (request certificate) or **`2`** (require certificate). If set to **`0`**, trust cannot be established. The default value is **`1`**, so it is OK if the parameter is not set at all.

Back to Tasks [page 1718]

Back to Concept [page 1718]

Next Steps

Now you can proceed with:

- Step 1 of the basic identity propagation setup for HTTPS, see Configure an ABAP System to Trust the Cloud Connector's System Certificate [page 1713]. However, when using SAP Web Dispatcher, the ABAP backend must trust the *SAP Web Dispatcher* instead of the *Cloud Connector*, see Forward SSL Certificates for X.509 Authentication, step 2 for details.

Then perform the remaining steps of the basic identity propagation setup for HTTPS as described here:

- Map Short-Lived Certificates to Users [page 1715]
- Access ICF Services [page 1715]

Back to Tasks [page 1718]

Back to Concept [page 1718]

1.4.2.3.1.3.2 Configure Identity Propagation for RFC

Find step-by-step instructions to configure principal propagation to an ABAP server for RFC.

Configuring principal propagation for RFC requires an SNC (Secure Network Communications) connection. To enable SNC, you must configure the ABAP system and the Cloud Connector accordingly.

The following example provides step-by-step instructions for the SNC setup.

> **Note**
>
> It is important that you use the same SNC implementation on both communication sides. Contact the vendor of your SNC solution to check the compatibility rules.

Example Data

The following data and setup is used:

> **Note**
>
> The parameters provided in this example are based on an SNC implementation that uses the **SAP Cryptographic Library**. Other vendors' libraries may require different values.

- An SNC identity has been generated and installed on the Cloud Connector host. Generating this identity for the SAP Cryptographic Library is typically done using the tool `SAPGENPSE`. For more information, see Configuring SNC for SAPCRYPTOLIB Using SAPGENPSE.
- The ABAP system is configured properly for SNC.

  > **Note**
  >
  > For the latest system releases, you can use the SSO wizard to configure SNC (transaction code: SNCWIZARD). System prerequisites are described in SAP note 2015966 .

- The Cloud Connector system identity's SNC name is **`p:CN=SCC, OU=SAP CP Scenarios, O=Trust Community, C=DE`**.
- The ABAP system's SNC identity name is **`p:CN=SID, O=Trust Community, C=DE`**. This value can typically be found in the ABAP system instance profile parameter `snc/identity/as` and hence is provided per application server.
- When using the SAP Cryptographic Library, we recommend that you use an ABAP system SNC identity and a Cloud Connector system identity signed by the same CA for mutual authentication.
- The example short-lived certificate has the subject **`CN=P1234567`**, where **`P1234567`** is the SAP BTP application user.

Tasks

1. Configure the ABAP System [page 1722]
2. Map Short-Lived Certificates to Users [page 1722]
3. Configure the Cloud Connector [page 1722]

1. Configure the ABAP System to Trust the Cloud Connector's System SNC identity

1. Open the *SNC Access Control List for Systems* (transaction `SNC0`).
2. As the Cloud Connector does not have a system ID, use an arbitrary value for `<System ID>` and enter it together with its SNC name: **`p:CN=SCC, OU=SAP CP Scenarios, O=Trust Community, C=DE`**.
3. Save the entry and choose the *Details* button.
4. In the next screen, activate the checkboxes for *Entry for RFC activated* and *Entry for certificate activated*.
5. Save your settings.

Back to Tasks [page 1722]

Back to Example Data [page 1721]

2. Map Short-Lived Certificates to Users

You can do this manually in the system as described below or use an identity management solution for a more comfortable approach. For example, for large numbers of users the rule-based certificate mapping is a good way to save time and effort. See Rule-Based Certificate Mapping.

1. Open *Assignment of External ID to Users* (transaction `EXTID_DN`).
2. Switch to the edit mode.
3. Create a new entry. Specify the subject of the certificate as `External ID`. Using the example data, this is CN=P1234567. In the `<User>` field, provide an appropriate ABAP user, for example **`JOHNDOE`**.
4. Save the mapping.
5. Repeat the previous steps for all users that should be supported for the scenario.

Back to Tasks [page 1722]

Back to Example Data [page 1721]

3. Configure the Cloud Connector

Prerequisites [page 1723]

Set up the Cloud Connector to Use the SNC Implementation [page 1723]

Create an RFC Hostname Mapping [page 1723]

Prerequisites

- The required security product for the SNC flavor that is used by your ABAP back-end systems, is installed on the Cloud Connector host.
- The Cloud Connector's system SNC identity is associated with the operating system user under which the Cloud Connector process is running.

> **Note**
>
> If you use SAP Cryptographic Library as SNC implementation, follow the steps described in Initial Configuration (RFC) [page 1678]. Additionally, SAP note 2642538 provides a good description to associate an SNC identity of SAP Cryptographic Library with a user running an external program that uses JCo. When using a different SNC offering, get in touch with the SNC library vendor for details.

Back to Step [page 1722]

Set up the Cloud Connector to Use the SNC Implementation

1. In the Cloud Connector UI, choose *Configuration* from the main menu, select the *On Premise* tab, and go to the *SNC* section.
2. Provide the fully qualified name of the SNC library (the security product's shared library implementing the GSS API), the SNC name of the above system identity, and the desired quality of protection by choosing the *Edit* icon.
   For more information, see Initial Configuration (RFC) [page 1678].

   > **Note**
   >
   > The example in Initial Configuration (RFC) [page 1678] shows the library location if you use the SAP Secure Login Client as your SNC security product. In this case (as well as for some other security products), *SNC My Name* is optional, because the security product automatically uses the identity associated with the current operating system user under which the process is running, so you can leave that field empty. (Otherwise, in this example it should be filled with **`p:CN=SCC, OU=SAP CP Scenarios, O=Trust Community, C=DE`**.)
   >
   > We recommend that you enter **`Maximum Protection`** for `<Quality of Protection>`, if your security solution supports it, as it provides the best protection.
3. Choose *Save* and *Close*.

Back to Step [page 1722]

Create an RFC Hostname Mapping

1. In the *Access Control* section of the Cloud Connector, create a hostname mapping corresponding to the cloud-side RFC destination. See Configure Access Control (RFC) [page 1752].
2. Make sure you choose **`RFC SNC`** as `<Protocol>` and **`ABAP System`** as `<Back-end Type>`. In the `<SNC Partner Name>` field, enter the ABAP system's SNC identitiy name, for example, **`p:CN=SID, O=Trust Community, C=DE`**.

3. Save your mapping.

Back to Step [page 1722]

Back to Tasks [page 1722]

Back to Example Data [page 1721]

Related Information

Secure Network Communications (SNC)

Using the SAP Cryptographic Library for SNC

Rule-Based Mapping of Certificates [page 1724]

Principal Propagation [page 30] (multi-cloud environment)

Principal Propagation (Neo environment)

1.4.2.3.1.3.3 Rule-Based Mapping of Certificates

Learn how to efficiently map short-lived certificates to users in the ABAP server.

> **Note**
>
> The information in this section applies to both principal propagation and technical user propagation.

To perform rule-based mapping of certificates in the ABAP server, proceed as follows:

1. Enter a dynamic parameter using transaction RZ10.
    1. Enter the `login/certificate_mapping_rulebased` parameter.
    2. Choose the *Change Value* button.
    3. Enter the value **1**.
    4. Save the value.

    > **Note**
    >
    > If dynamic parameters are disabled, enter the value using transaction RZ10 and restart the whole ABAP system.
2. Configure rule-based mapping
    1. Create a sample certificate with the Cloud Connector.
        1. Log on to the Cloud Connector UI.
        2. From the main menu, choose *Configuration*, tab *On Premise*.
        3. Go to the **Principal Propagation** section. In the area **Subject Patterns**, press *Add*.

4. Enter a subject pattern and choose *Save*.
5. The added item appears in the list of subject patterns. Select the item and choose **Generate a sample certificate**. The certificate will be downloaded to your machine.

2. To import the sample certificate into the ABAP server, choose transaction CERTRULE and select *Import certificate*.

> ### Note
>
> To access transaction CERTRULE, you need the corresponding authorizations (see: Assign Authorization Objects for Rule-based Mapping [page 1725]).

3. To create explicit rule mappings, choose the *Rule* button.
4. Choose *Save*.

> ### Note
>
> When you save the changes and return to transaction CERTRULE, the sample certificate which you imported in Step 2b will not be saved. This is just a sample editor view to see the sample certificates and mappings.

Related Information

Rule-Based Certificate Mapping

1.4.2.3.1.3.3.1 Assign Authorization Objects for Rule-based Mapping

Assign authorizations to access transaction CERTRULE.

To access transaction CERTRULE, you need the following authorizations:

- CC control center: System administration (S_RZL_ADM)
  - `Activity 03` grants display authorizations.
  - `Activity 01` grants change authorizations.
- User Master Maintenance: User Groups (S_USER_GRP)
  - `Activity 03` grants display authorizations.
  - `Activity 02` grants change authorizations.
  - `Class`: enter the names of user groups for which the administrator can maintain explicit mappings.

To assign these authorization objects, proceed as follows:

1. Create a *Single Role* using transaction PFCG.
2. To add the authorization objects **`S_RZL_ADM`** and **`S_USER_GRP`**, go to the *Authorizations* tab, choose *Change Authorization data* and select the *Manually* button .
3. To generate the profile, choose *Generate* and save the changes.

4. In the *User* tab, enter the user who should execute the transaction CERTRULE.
5. To match the generated profile to the users, choose *User comparison* .

1.4.2.3.1.4 Configure Subject Patterns

Define patterns identifying the user for the subject of a generated short-lived X.509 certificate.

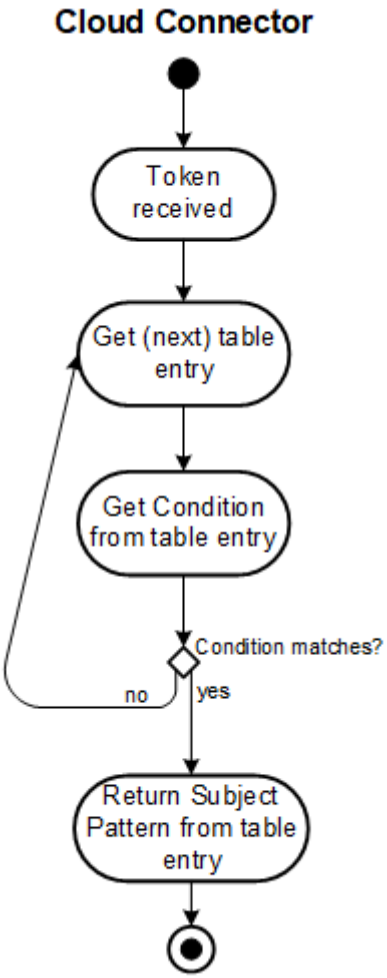
> **Note**
>
> The information in this section applies to both principal propagation and technical user propagation.

Using this configuration option, you can define different patterns identifying the user for the subject of the generated short-lived X.509 certificate, based on a specified condition. You can also specify the validity period and expiration tolerance.

Configure Subject Pattern Rules

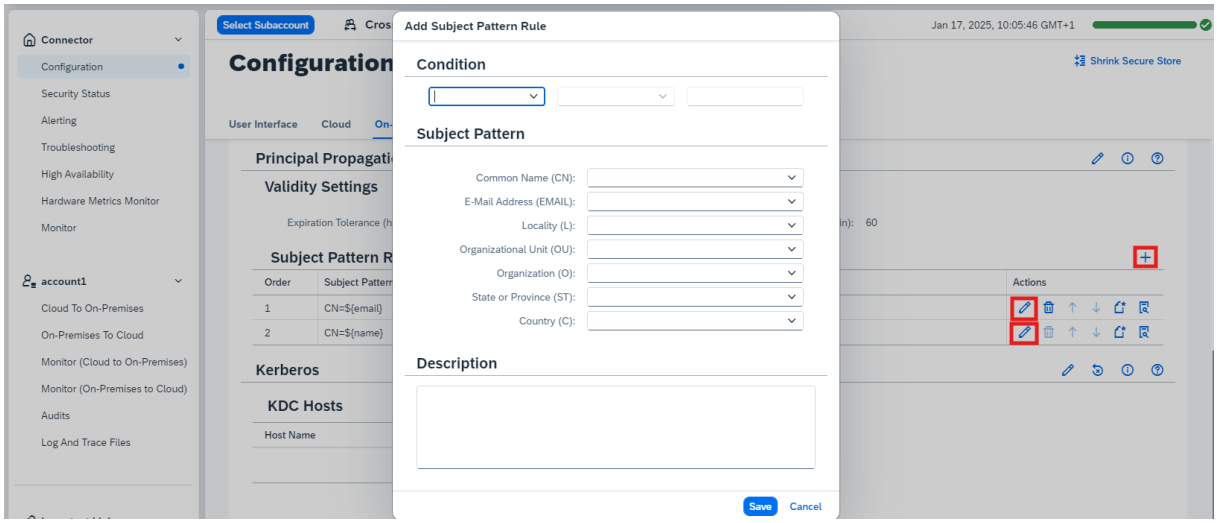
To configure a subject pattern rule, choose *Configuration* > *On Premise* > *Principal Propagation* . In the table shown, you can add or modify a rule consisting of a pattern and a condition.

This table represents an ordered list containing entries that have a specified condition, and the respective subject pattern. You can change the order for an entry by choosing the respective arrow buttons. The workflow in the Cloud Connector looks like this:



The last entry in the table is the default one having no condition (that is, it always matches as fallback). This entry can neither be moved nor deleted.

To modify or add table entries, choose the *Edit* or *Add* icon:



Specify a Condition

Use either of the following procedures to specify a condition based on the attributes of incoming tokens from cloud side:

- Enter the values in the subject pattern fields manually.
- Use the selection menu of the corresponding field to enter a predefined variable.
- Optionally, provide a description.

Using the selection menu, you can assign values for the following parameters:

- `${user_type}`
- `${name}`
- `${mail}`
- `${email}`
- `${user_uuid}`
- `${display_name}`
- `${login_name}`

Operators

In a next step, choose an operator:

- *exists*: This attribute must be present in the incoming token.
- *does not exist*: This attribute must not be present in the incoming token.
- *is*: This attribute must be present and equals to the value that has been entered in the third field afterwards.
- *is not*: This attribute must be present and is not equal to the value that has been entered in the third field afterwards.

> ### Note
>
> For the condition `${user_type}`, you can only switch between **`Technical`** or **`Business`**. The latter refers to the "classical" propagation of business user information, whereas **`Technical`** is the propagation of a technical user.
>
> For more information, see Configuring Technical User Propagation [page 1739].

> ### Note
>
> As of version 2.18.0, you can find the information about the last usage of a subject pattern rule by choosing the *Details* button.

Subject Pattern Details

Use either of the following procedures to define the subject's distinguished name (DN), for which the certificate will be issued:

- Enter the values in the subject pattern fields manually.
- Use the selection menu of the corresponding field to enter a predefined variable.

Using the selection menu, you can assign values for the following parameters:

- `${name}`
- `${mail}`
- `${display_name}`
- `${login_name}`

> ### Note
>
> `${name}` is a virtual variable of the token, it represents the principal name. When JWTs (JSON Web token) are used, there is no clear definition what to use and therefore the value is determined depending in the user type. Below you can find the order of variables that are checked, and the first one available will define the content of `${name}`:
>
> **Business**: `user_name`, `email`, `mail`, `user_uuid`, `sub`
>
> **Technical**: `client_id`, `cid`, `sub`, `azp`

> ### Note
>
> If the token provided by the Identity Provider contains additional values that are stored in attributes with different names, but you still want to use it for the subject pattern, you can edit the variable name to place the corresponding attribute value in the subject accordingly. For example, provide **`${email}`**, if a SAML assertion uses `email` instead of providing `mail`, or **`${user_uuid}`** if the attribute `user_uuid` representing the global user ID is contained in the assertion.
>
> When using a subaccount in the **multi-cloud** environment: The Cloud Connector also offers direct access to custom variables injected in the JWT (JSON Web token) by SAP BTP *Authorization & Trust Management* that were taken over from a SAML assertion, and are available in the *xs.user.attributes* section of the token. If such a custom variable has the same name as one on root level, the one on root level can still be accessed using `cp.` as a prefix to the name. *Example*: if `userId` is present on root level and in *xs.user.attributes*, then `${userId}` will contain the value of the variable in *xs.user.attributes*, and `${cp.userId}` the one of the variable on root level.

> ### Caution
>
> **Security Impact**
>
> Never use user attributes that can be modified by the user itself. Such attribute values can potentially be changed by attackers to try an injection attack.

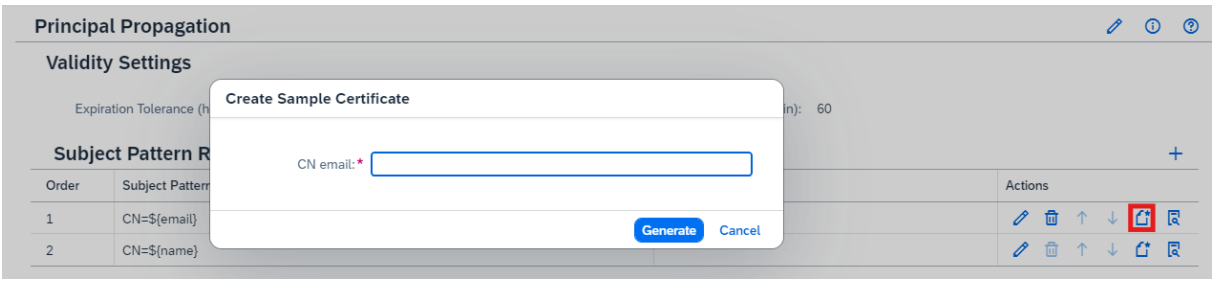
The values for these variables are provided by the trusted identity provider in the token which is passed to the Cloud Connector and specifies the user that has logged on to the cloud application.

By default, the following attributes are provided:

- `<CN>`: (common name) – the name of the certificate owner
- `<EMAIL>`: (e-mail address) - the e-mail address of the certificate owner
- `<L>`: (locality) – the certificate owner's location
- `<O>`: (organization) – the certificate owner's organization or company
- `<OU>`: (name of organizational unit) – the organizational unit to which the certificate owner belongs
- `<ST>`: (state of residence) – the state in which the certificate issuer resides
- `<C>`: (country of residence) – the country in which the certificate owner resides
- `<Expiration Tolerance (h)>`: The length of time in hours, that an application can use a principal issued for a user after the token from cloud side has expired. The minimum value for this attribute is 0, and the maximum 336 (14 days).
- `<Certificate Validity (min)>`: The length of time in minutes, that a certificate generated for principal propagation can authenticate against the back end. You can reuse a previously generated certificate to improve performance. The minimum value for this attribute is 0, and the maximum 43200 (30 days).

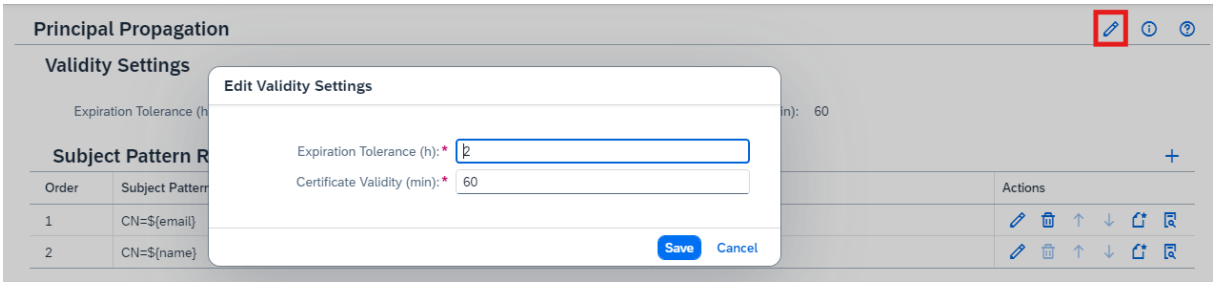
Sample Certificate

By choosing *Generate Sample Certificate* you can create a sample certificate that looks like one of the short-lived certificates created at runtime. You can use this certificate to, for example, generate user mapping rules in the target system, via transaction CERTRULE in an ABAP system. If your subject pattern contains variable fields, a wizard lets you provide meaningful values for each of them and eventually you can save the sample certificate in DER format.



Validity Settings

You can change the validity settings by choosing the *Edit* button.



Related Information

Principal Propagation SSO Authentication for HTTP [page 1359]

Trust and Federation with Identity Providers

1.4.2.3.1.5 Configure a Secure Login Server

Configuration steps for Java SLS support.

> **Note**
>
> The information in this section applies to both principal propagation and technical user propagation.

> **Note**
>
> The Secure Login Server mainstream maintenance ends on December 31, 2027. In particular, this is also relevant when using it with the Cloud Connector for the scenarios with principal propagation and technical user propagation.

Content

Overview [page 1732]

Requirements [page 1732]

Implementation [page 1733]

Overview

The Cloud Connector can use on-the-fly generated X.509 user certificates to log in to on-premises systems if the external user session is authenticated (for example by means of SAML). If you do not want to use the built-in certification authority (CA) functionality of the Cloud Connector (for example because of security considerations), you can connect SAP SSO 2.0 Secure Login Server (SLS) or higher.

> ### Note
>
> Make sure you use a version that is still supported, which is currently at least SAP SSO 3.0 Secure Login Server.

SLS is a Java application running on AS JAVA 7.20 or higher, which provides interfaces for certificate enrollment.

SLS supports the following formats:

- HTTPS
- REST
- JSON
- PKCS#10/PKCS#7

> ### Note
>
> Any enrollment requires a successful user or client authentication, which can be a single, multiple or even a multi factor authentication.

The following schemes are supported:

- LDAP/ADS
- RADIUS
- SAP SSO OTP
- ABAP RFC
- Kerberos/SPNego
- X.509 TLS Client Authentication

SLS lets you define arbitrary enrollment profiles, each with a unique profile UID in its URL, and with a configurable authentication and certificate generation.

Back to Content [page 1731]

Requirements

For user certification, SLS must provide a profile that adheres to the following:

- Cloud Connector client authentication by its X.509 system certificate
- Cloud Connector system certificate and SLS may live in different PKIs

- Cloud Connector hands over the full user´s certificate subject name

With SAP SSO 2.0 SP06, SLS provides the following required features:

- TLS Client Authentication-based enrollment with `SecureLoginModuleUserDelegationWithSSL` (available since SP04)
- multi-PKI support is implemented by all standard components of Application Server (AS) JAVA, AS ABAP, HANA, by importing trusted root CA certificates
- SLS allows `PKCS10:SUBJECT` in a profile´s certificate configuration (SP06)

Back to Content [page 1731]

Implementation

INSTALLATION

Follow the standard installation procedures for SLS. This includes the initial setup of a PKI (public key infrastructure).

> **Note**
>
> SLS allows you to set up one or more own PKIs with Root CA, User CA, and so on. You can also import CAs as PKCS#12 file or use a hardware security module (HSM) as "External User CA".

> **Note**
>
> You should only use HTTPS connections for any communication with SLS. AS JAVA / ICM supports TLS, and the default configuration comes with a self-signed sever certificate. You may use SLS to replace this certificate by a PKI certificate.

CONFIGURATION

SSL Ports

1. Open the NetWeaver Administrator, choose *Configuration* > *SSL* and define a new port with **`Client Authentication Mode = REQUIRED`**.

   > **Note**
   >
   > You may also define another port with **`Client Authentication Mode = Do not request`** if you did not do so yet.

2. Import the root CA of the PKI that issued your Cloud Connector system certificate.
3. Save the configuration and restart the Internet Communication Manager (ICM).

Authentication Policy

1. Open the NetWeaver Administrator (NWA, https://<host:port>/nwa).
2. From the top-level menu, choose *Configuration* > *Authentication and Single Sign-On* .
3. In the *Policy Configuration* table, switch to **`Type = Custom`**.

4. Choose *Add* to create a new policy and assign it a name, for example, **`SecureLoginCloudConnector`**.
5. Open *Edit* mode.
6. In the *Details* section of the authentication configuration, choose *Authentication Stack* > *Login Modules* and add **`SecureLoginModuleUserDelegationWithSSL`**.
7. In `<Rule1.subjectName>` and `<Rule1.issuerName>`, enter the respective certificate names of your Cloud Connector system certificate.
8. In the *Details* section of the authentication configuration, choose *Properties* and add the property `UserNameMapping` with value **`VirtualUser`**.
9. Save the policy.

Client Authentication Profile

1. Open the SLS Administration Console (SLAC, https://host:port/slac).
2. From the top-level menu, choose *Profile Management* > *Authentication Profiles*.
3. Create a new profile with `Client Type = Secure Login Client` and assign it a name, for example, **`Cloud Connector User Certificates`**.
4. Choose *User Authentication* > *Use Policy Configuration* and select `Policy Configuration Name = SecureLoginCloudConnector`.
5. Edit all required fields in the wizard according to your requirements.
6. In tab *Authentication Configuration*, check the box *Virtual User*.
7. Save your entries.
8. Select the new profile and open *Edit* mode.
9. Choose *Certificate Configuration* > *Certificate Name and Alternative Names* and set `Appendix Subject Name = (PKCS10:SUBJECT)`.
10. Leave all other fields in *Certificate Name and Alternative Names* empty.
11. On the *Enrollment Configuration* page, make sure that the `<Enrollment URL>` has the correct value, otherwise edit and fix it:
    1. full DNS name
    2. port *with* TLS Client Authentication (see port number in NWA SSL Configuration).
12. Save your entries.

User Profile Group

1. Open the SLS Administration Console (SLAC, https://host:port/slac).
2. Go to the top level menu and choose *Profile Management* > *User Profile Groups*.
3. Create a new profile group, make sure the `<Policy URL>` has the correct value:
    1. Full DNS name
    2. Port *without* TLS Client Authentication (see port number in NWA SSL Configuration).
4. In tab *Profiles*, add the profile `Cloud Connector User Certificates`.
5. Save your entries.

Root CA Certificate

1. Open SLS Administration Console (SLAC, https://host:port/slac).
2. Go to the top level menu and choose *Certificate Management*.
3. Select the Root CA certificate you are using in your profile.
4. Choose *Export entry* > *X.509 Certificate* and download the certificate file.

Cloud Connector

Follow the standard installation procedure of the Cloud Connector and configure SLS support as explained in Configuration of a CA hosted by a Secure Login Server:

1. Enter the policy URL that points to the SLS user profile group.
2. Select the profile, for example, **`Cloud Connector User Certificates`**.
3. Import the Root CA certificate of SLS into the Cloud Connector´s Trust Store.

On-Premises Target Systems

Follow the standard configuration procedure for Cloud Connector support in the corresponding target system and configure SLS support.

To do so, import the Root CA certificate of SLS into the system´s truststore:

- **AS ABAP**: choose transaction STRUST and follow the steps in Maintaining the SSL Server PSE's Certificate List.
- **AS Java**: open the Netweaver Administrator and follow the steps described in Configuring the SSL Key Pair and Trusted X.509 Certificates.

Back to Content [page 1731]

1.4.2.3.1.6 Configure Kerberos

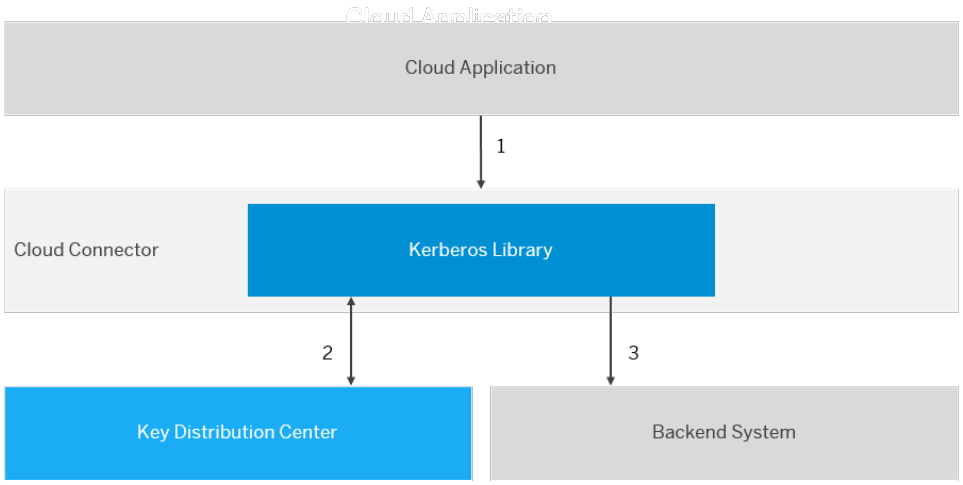
Context

The Cloud Connector allows you to propagate users authenticated in SAP BTP via Kerberos against backend systems. It uses the **Service For User and Constrained Delegation** protocol extension of Kerberos.

> **Note**
>
> This feature is not supported for ABAP backend systems. In this case, you can use the certificate-based principal propagation, see Configure a CA Certificate [page 1706].

The Key Distribution Center (KDC) is used for exchanging messages in order to retrieve Kerberos tokens for a certain user and backend system.

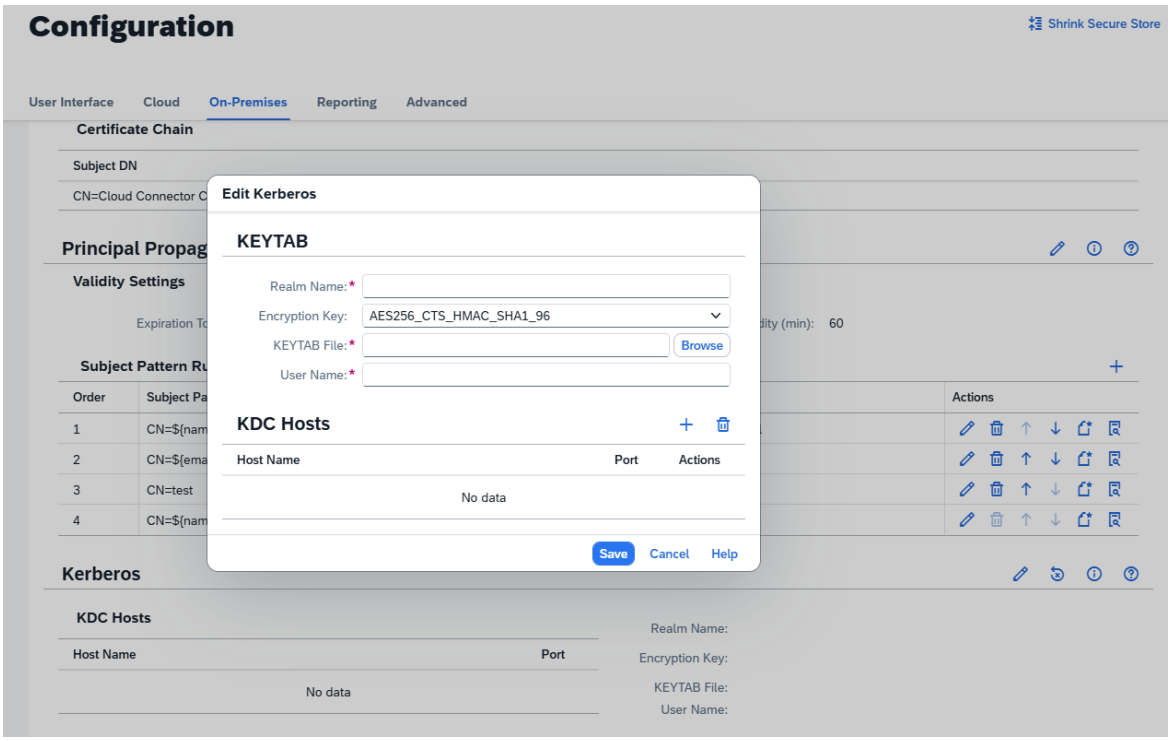
For more information, see Kerberos Protocol Extensions: Service for User and Constrained Delegation Protocol .



1. An SAP BTP application calls a backend system via the Cloud Connector.
2. The Cloud Connector calls the KDC to obtain a Kerberos token for the user propagated from the Cloud Connector.
3. The obtained Kerberos token is sent as a credential to the backend system.

Procedure

1. Choose *Configuration* from the main menu.
2. From the *Kerberos* section of the *On Premises* tab, choose *Edit*.



3. In the *KDC Hosts* field (press *Add* to display the field), enter the host name of your KDC in format <host>:<port>. The port is optional. If you do not specify a port, the default one, 88, will be used.
4. Choose the desired *Encryption Key* algorithm from the list of the 2 available ones: `RC4_HMAC` or `AES256_CTS_HMAC_SHA1_96`.
5. In the *Realm Name* field, enter the name of your Kerberos realm.
6. Upload a KEYTAB file that contains the secret keys of your service user. The KEYTAB file should fit to the chosen encryption key for your user.
7. In the *User Name* field, enter the name of the service user to be used for communication with the KDC. This service user should be allowed to request Kerberos tokens for other users for the back-end systems that you are going to access.
8. Choose *Save*.

Example

You have a backend system protected with SPNego authentication in your corporate network. You want to call it from a cloud application while preserving the identity of a cloud-authenticated user.

Define the following:

- A connectivity destination in SAP BTP, with `ProxyType` = **`OnPremise`**.
- A system mapping made in the Cloud Connector. (Choose *Cloud to On Premise* from your subaccount menu, Go to tab *Access Control* > *Add*, and for *Principal Type*, select **`Kerberos`**.)
- Kerberos configuration in the Cloud Connector, where the service user is allowed to delegate calls for your backend host service.

Result:

When you now call a backend system, the Cloud Connector obtains an SPNego token from your KDC for the cloud-authenticated user. This token is sent along with the request to the back end, so that it can authenticate the user and the identity to be preserved.

Related Information

Set Up Trust [page 1702]

1.4.2.3.1.7 Configuring Identity Propagation to SAP NetWeaver AS for Java

Find step-by-step instructions on how to set up an application server for Java (AS Java) to enable identity propagation for HTTPS.

> **Note**
>
> The information in this section applies to both principal propagation and technical user propagation. These two types of user propagation are combined as *identity* propagation.

Prerequisites

To perform the following steps, you must have the corresponding administrator authorizations in AS Java (SAP NetWeaver Administrator) as well as an administrator user for the Cloud Connector.

Configure AS Java to Trust the Cloud Connector's System Certificate

Procedure

1. Go to *SAP NetWeaver Administrator* > *Certificates and Keys* and import the Cloud Connector's system certificate into the *Trusted CAs* keystore view. See Importing Certificate and Key From the File System.
2. Configure the Internet Communication Manager (ICM) to trust the system certificate for identity propagation.
   a. Add a new SSL access point. See Adding New SSL Access Points.
   b. Generate a certificate signing request and send it to the CA of your choice. See Configuration of the AS Java Keystore Views for SSL.
   c. Import the certificates and save the configuration.

      Import the certificate signing response, the root X.509 certificate of the trusted CA, and the Cloud Connector's system certificate into the new SSL access point from step 2a. Save the configuration and restart the ICM. See Configuring the SSL Key Pair and Trusted X.509 Certificates.
   d. Make sure the ICM trusts the system certificate for identity propagation. For more information, see Using Client Certificates via an Intermediary Server.
   e. Restart the ICM and test the SSL connection. For more information, see Testing the SSL Connection.

Define Rules for User Mapping

Procedure

1. Add the *ClientCertLoginModule* to the policy configuration that the Cloud Connector connects to. See Configuring the Login Module on the AS Java.
2. Define the rules to map users authenticated with their certificate to users that exist in the User Management Engine. See Using Rules for User Mapping in Client Certificate Login Module.
   - To map the user ID of the certificate's subject name field to users, see Using Rules Based on Client Certificate Subject Names.
   - To map the user ID based on rules for the certificate V3 extension *SubjectAlternativeName*, see Using Rules Based on Client Certificate V3 Extensions.
   - To use client certificate filters, see Defining Rules for Filtering Client Certificates.

Related Information

Configuring Transport Layer Security on SAP NetWeaver AS for Java
Using X.509 Client Certificates
Configure Identity Propagation for HTTPS [page 1712]

1.4.2.3.2 Configuring Technical User Propagation

Use technical user propagation to provide access of technical users to on-premises systems.

Using technical user propagation, you can forward the identity of technical users that are identified using an OAuth client and act on behalf of this identity in an on-premises system.

> **Note**
>
> Technical user propagation is possible only in the Cloud Foundry environment.

Tasks in this section:

Task	Description
Set Up Trust [page 1702]	Configure a trusted relationship in the Cloud Connector to support technical user propagation. Technical user propagation lets you forward a technical user in the cloud to the internal system without requesting a password.

Task	Description
Configure a CA Certificate [page 1706]	Install and configure an X.509 certificate to enable support for technical user propagation.
Configuring Identity Propagation to an ABAP System [page 1711]	Learn more about the different types of configuring and supporting technical user propagation for a particular AS ABAP.
Configure Subject Patterns [page 1726]	Define a pattern identifying the user for the subject of the generated short-lived `X.509` certificate, as well as its validity period.
Configure a Secure Login Server [page 1731]	Configuration steps for Java Secure Login Server (SLS) support.
Configure Kerberos [page 1735]	The Cloud Connector lets you propagate users authenticated in SAP BTP via Kerberos against backend systems. It uses the **Service For User and Constrained Delegation** protocol extension of Kerberos.
Configuring Identity Propagation to SAP NetWeaver AS for Java [page 1738]	Find step-by-step instructions on how to configure technical user propagation to an application server Java (AS Java).

Related Information

OAuth Technical User Propagation Authentication [page 1424]

1.4.2.4 Configure Access Control

Specify the backend systems that can be accessed by your cloud applications.

To allow your cloud applications to access a certain backend system on the intranet, you must specify this system in the Cloud Connector. The procedure is specific to the protocol that you are using for communication.

Find the detailed configuration steps for each communication protocol here:

Configure Access Control (HTTP) [page 1744]

Configure Access Control (RFC) [page 1752]

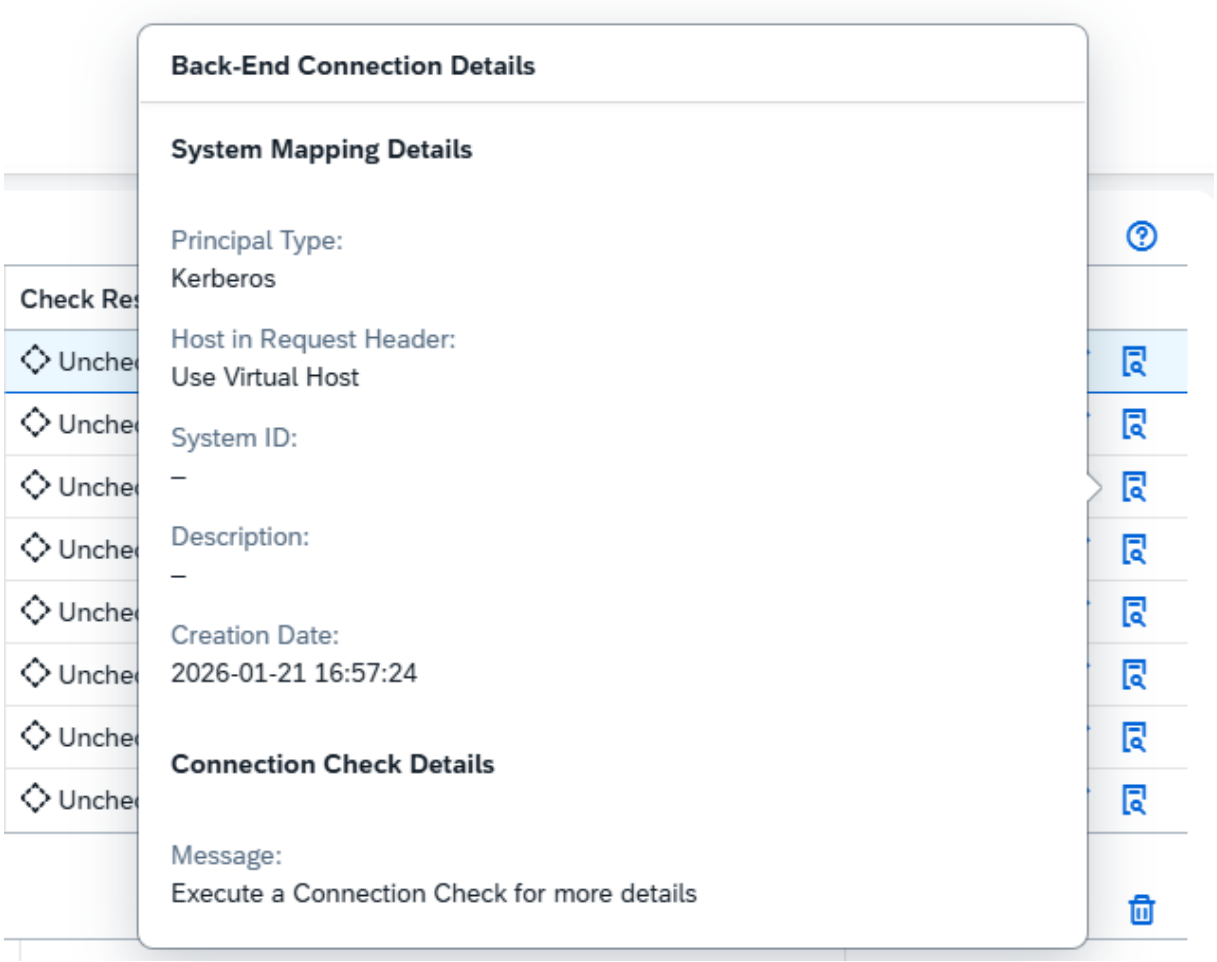
Configure Access Control (LDAP) [page 1764]

Configure Access Control (TCP) [page 1767]

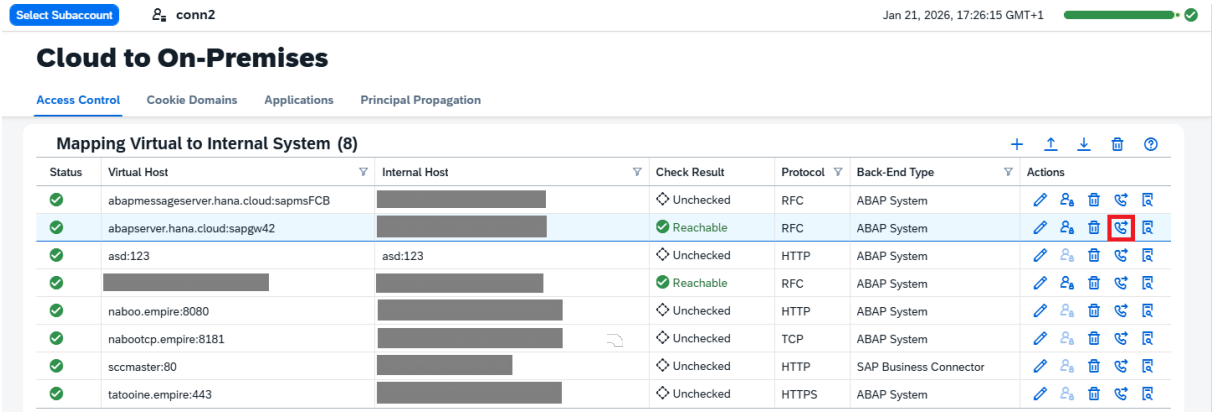
Copy Access Control Settings

Adding a new subaccount is not of any use unless you expose the systems you want to be available for this subaccount in the access control settings. You can copy the complete access control settings from another subaccount on the same Cloud Connector, or from a file, by using the import/export mechanism provided by the Cloud Connector.

You can find detailed information, like the creation date of an access control entry (also referred to as a *system mapping*) , by choosing the *Details* button in the **Actions** column.



When creating an access control entry, the connection to the respective internal host is checked immediately, and details of that check are available via the detailed information as explained above. Such a check can be repeated, or executed for the first time after restarting the Cloud Connector, by pressing the respective button in the **Actions** column:



> ### Note
>
> If an internal host and port combination is used by several access control entries, the check result will be updated for all those entries unless the protocol differs from the protocol where the check was triggered.

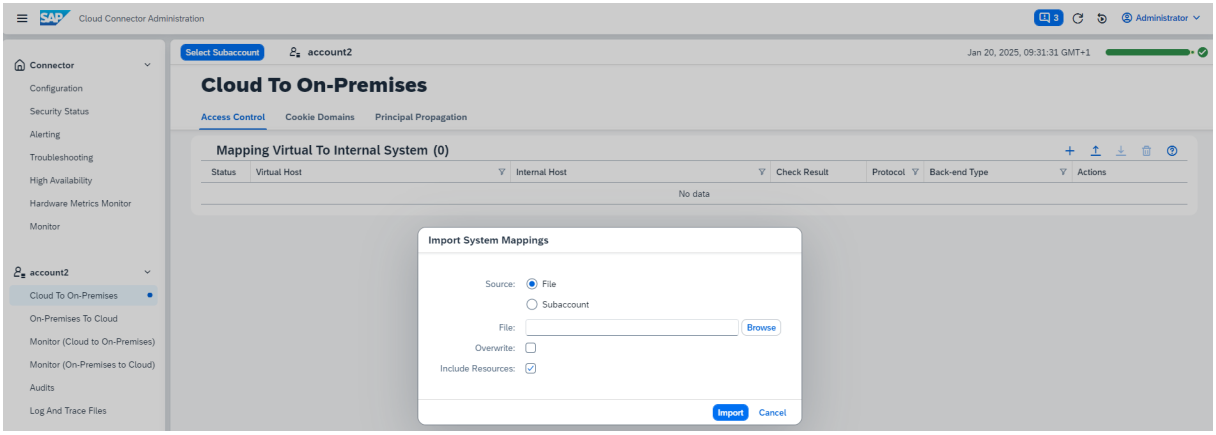
Export Access Control Settings

1. From your subaccount menu, choose *Cloud To On-Premises* and select the tab *Access Control*.
2. To store the current settings in a ZIP file, choose *Download* icon in the upper-right corner.
3. You can later import this file into a different Cloud Connector.

Import Access Control Settings

There are two locations from which you can import access control settings:

- A file that was previously exported from a Cloud Connector
- A different subaccount on the same Cloud Connector



Two additional options define the behavior of the import:

- *Overwrite*: Select this checkbox if you want to replace existing system mappings with imported ones. Do not select this checkbox if you want to keep existing mappings and only import the ones that are not yet available (default).

  > **Note**
  >
  > A system mapping is uniquely identified by the combination of virtual host and port.

- *Include Resources*: When this checkbox is selected (default), the resources that belong to an imported system are also imported. Otherwise no resources are imported, that is, imported system mappings do not expose any resources.

Back-End Types

When creating a new access control entry, you can select one of the following back-end types :

Type	Supported Protocols	Type-Specific Features
ABAP System	HTTP, HTTPS, RFC, RFC SNC, WebSocket RFC, TCP, TCP TLS	Supports: • ABAP-specific connection check features • URL mangling for BSPs
SAP HANA	HTTP, HTTPS, TCP, TCP TLS	
SAP Application Server Java	HTTP, HTTPS	
SAP Composition Environment	HTTP, HTTPS	
SAP Business Connector	HTTP, HTTPS	
SAP Process Integration	HTTP, HTTPS	
SAP Gateway	HTTP, HTTPS, RFC, RFC SNC, WebSocket RFC, TCP, TCP TLS	Supports: • URL mangling for BSPs
Other SAP System	HTTP, HTTPS, TCP, TCP TLS	
Non-SAP System	HTTP, HTTPS, TCP, TCP TLS, LDAP, LDAPS	

Related Information

Configure Access Control (HTTP) [page 1744]
Configure Access Control (RFC) [page 1752]
Configure Access Control (LDAP) [page 1764]
Configure Access Control (TCP) [page 1767]
Configure Accessible Resources [page 1771]
Configure Domain Mappings for Cookies [page 1958]

1.4.2.4.1 Configure Access Control (HTTP)

Specify the back-end systems that can be accessed by your cloud applications using HTTP.

To allow your cloud applications to access a certain back-end system on the intranet via HTTP, you must configure access control in the Cloud Connector accordingly.

> ⓘ Note
>
> Make sure that also redirect locations are configured as internal hosts.
>
> If the target server responds with a redirect HTTP status code (30x), the cloud-side HTTP client usually sends the redirect over the Cloud Connector as well. The Cloud Connector runtime then performs a reverse lookup to rewrite the location header that indicates where to route the redirected request.
>
> If the redirect location is ambiguous (that is, several mappings point to the same internal host and port), the first one found is used. If none is found, the location header stays untouched.

Tasks

Expose Intranet Systems for HTTP Requests [page 1744]

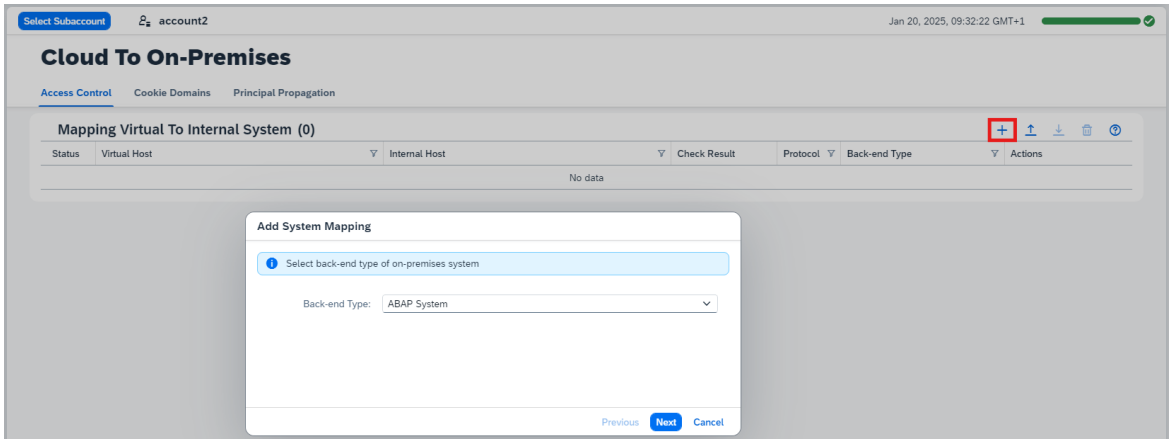
Limit the Accessible Services for HTTP(S) [page 1751]

Activate or Suspend Resources [page 1752]

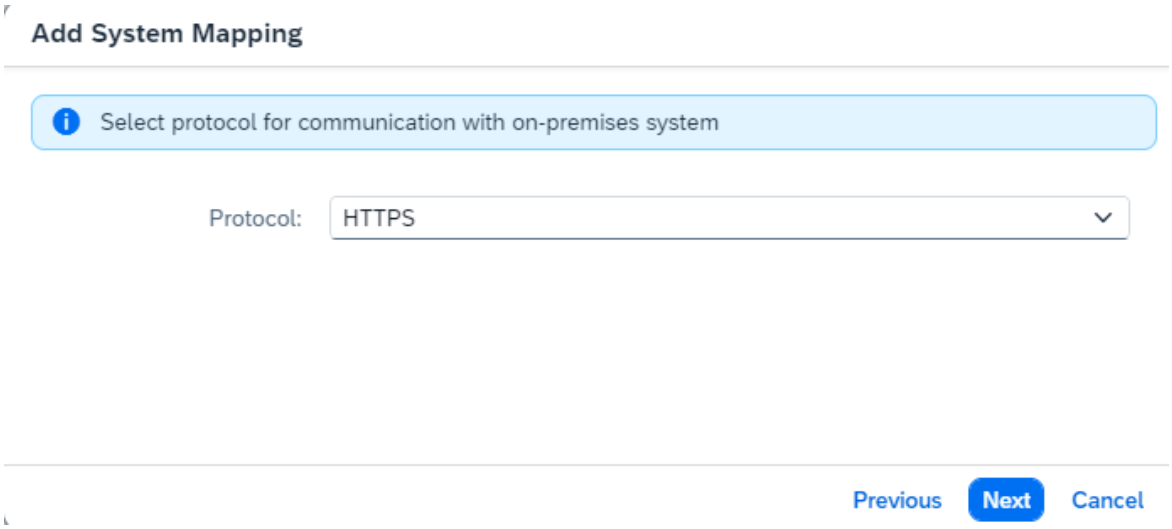
Expose Intranet Systems for HTTP Requests

Insert a new entry in the Cloud Connector access control management:

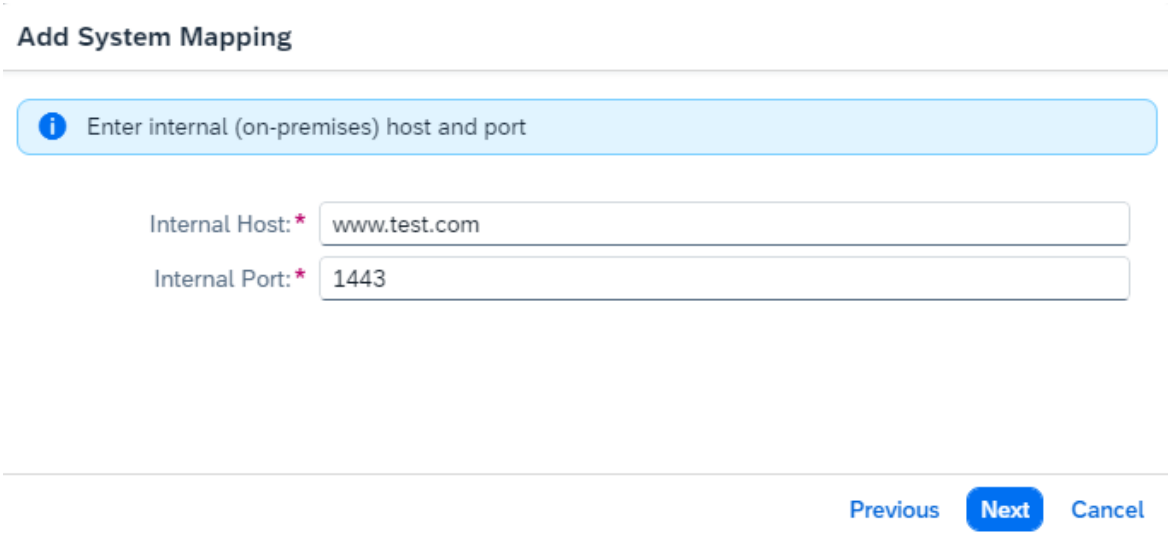
1. Choose *Cloud To On Premises* from your *Subaccount* menu and go to tab *Access Control*.
2. Choose *Add*. A wizard will open and ask for the required values.
3. *Back-End Type*: Select the description that matches best the addressed back-end system.
   When you are done, choose *Next*.



4. *Protocol*: This field allows you to decide whether the Cloud Connector should use **HTTP** or **HTTPS** for the connection to the back-end system. Note that this is completely independent from the setting on cloud side. Thus, even if the HTTP destination on cloud side specifies "`http://`" in its URL, you can select **HTTPS**. This way, you are ensured that the entire connection from the cloud application to the actual back-end system (provided through the TLS tunnel) is TLS-encrypted. The only prerequisite is that the back-end system supports HTTPS on that port. For more information, see Initial Configuration (HTTP) [page 1676].
   - If you specify HTTPS and there is a "system certificate" imported in the Cloud Connector, the latter attempts to use that certificate for performing a client-certificate-based logon to the back-end system.
   - If there is no system certificate imported, the Cloud Connector opens an HTTPS connection without client certificate.



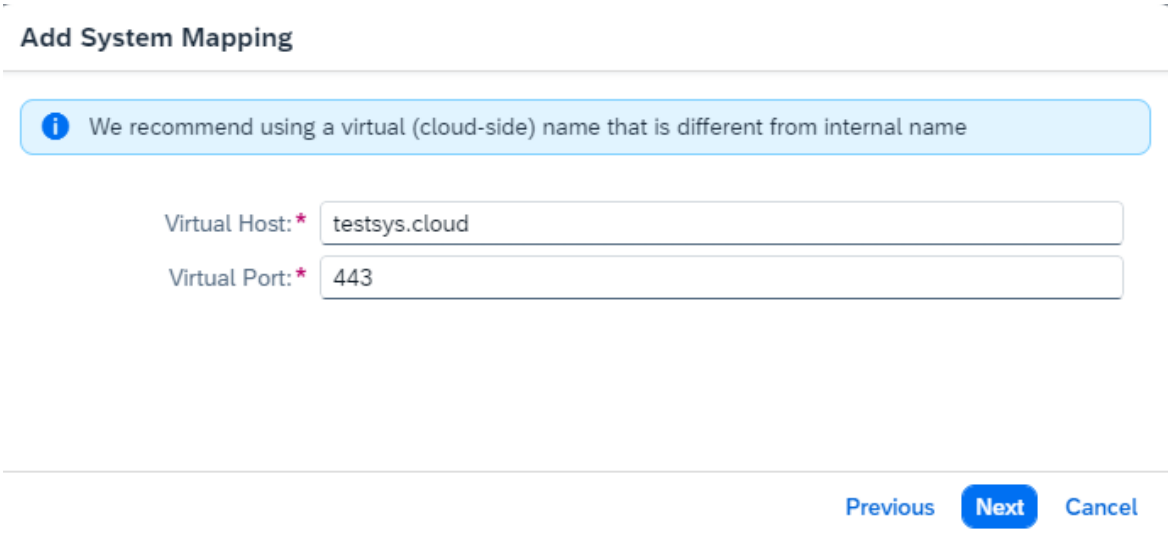
5. *Internal Host* and *Internal Port* specify the actual host and port under which the target system can be reached within the intranet. It needs to be an existing network address that can be resolved on the intranet and has network visibility for the Cloud Connector without any proxy. Cloud Connector will try to forward the request to the network address specified by the internal host and port, so this address needs to be real.



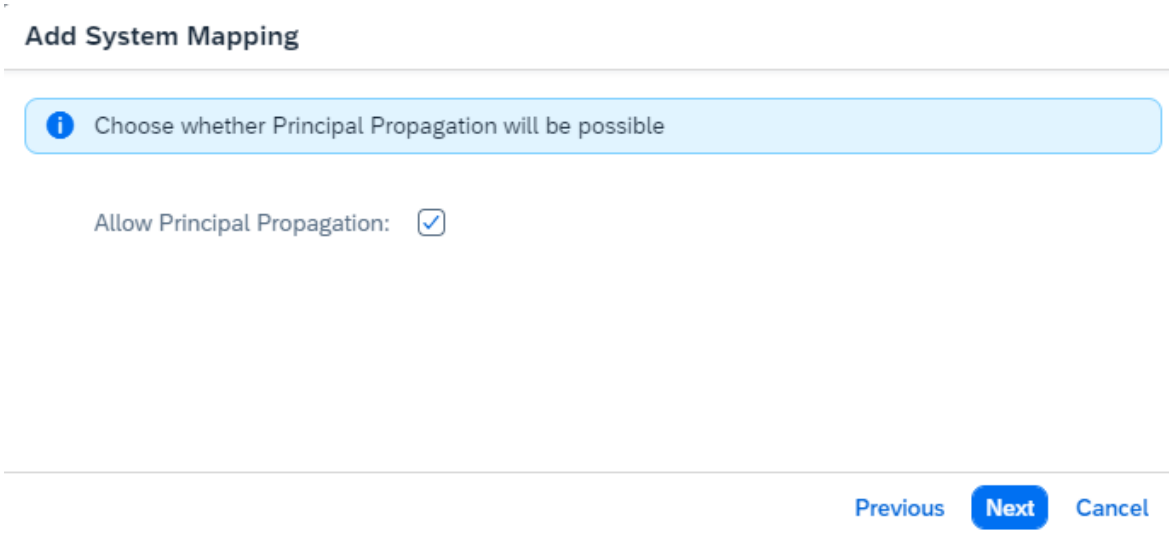
6. *Virtual Host* identifies the host name exactly as specified in the `<URL>` property of the HTTP destination configuration in SAP BTP.
See: Create HTTP Destinations [page 1476] (Cloud Foundry environment)
The virtual host can be a fake name and does not need to exist.

> ### Note
>
> In some scenarios, underscores ("_") may not be allowed for the virtual host name. In these cases, an *HTTP 400 Bad Request* error message will be shown.

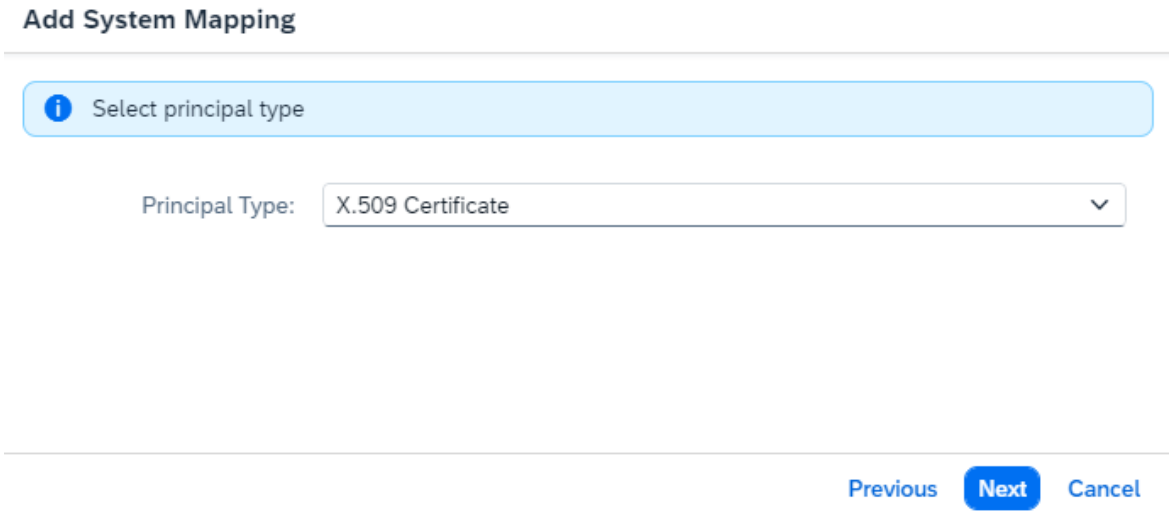
The *Virtual Port* allows you to distinguish between different entry points of your back-end system, for example, **`HTTP/80`** and **`HTTPS/443`**, and to have different sets of access control settings for them. For example, some non-critical resources may be accessed by HTTP, while some other critical resources are to be called using HTTPS only. The fields are prepopulated with the values of the *Internal Host* and *Internal Port*. If you don't modify them, you must provide your internal host and port also in the cloud-side destination configuration or in the URL used for your favorite HTTP client.



7. *Allow Principal Propagation* (available as of Cloud Connector 2.15): Defines if any kind of principal propagation should be allowed over this mapping. If not selected, go to step 9.



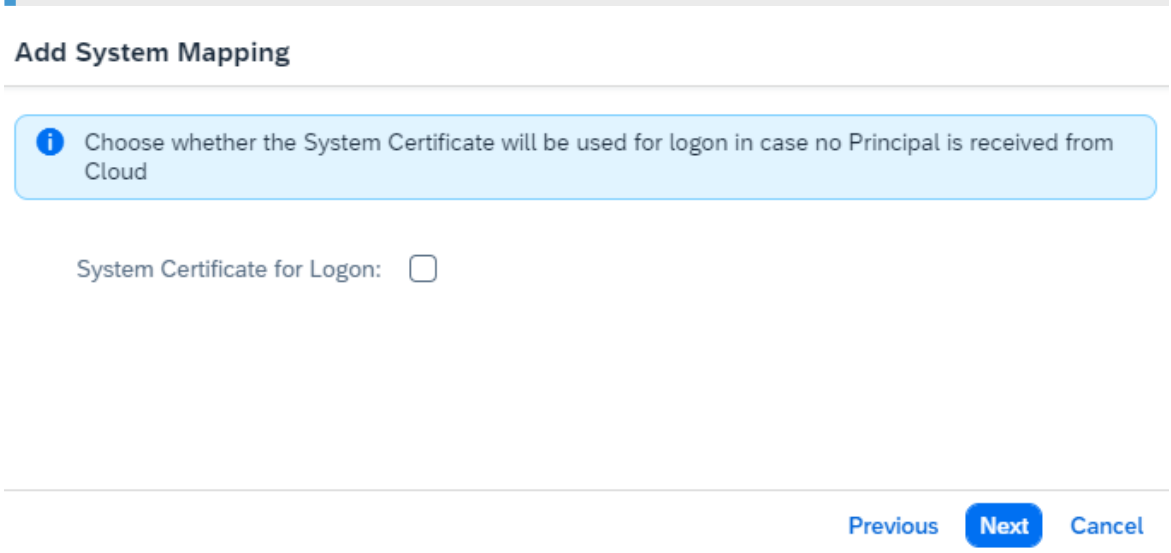
8. *Principal Type*
   Defines what kind of principal is sent to the back end within the HTTP request. For the principal type **`X.509 Certificate`** (if a principal is sent from the cloud side), the principal is injected as an HTTP header (`SSL_CLIENT_CERT`) and forwarded to the back end. There, depending on the configuration, it can be used for authentication.



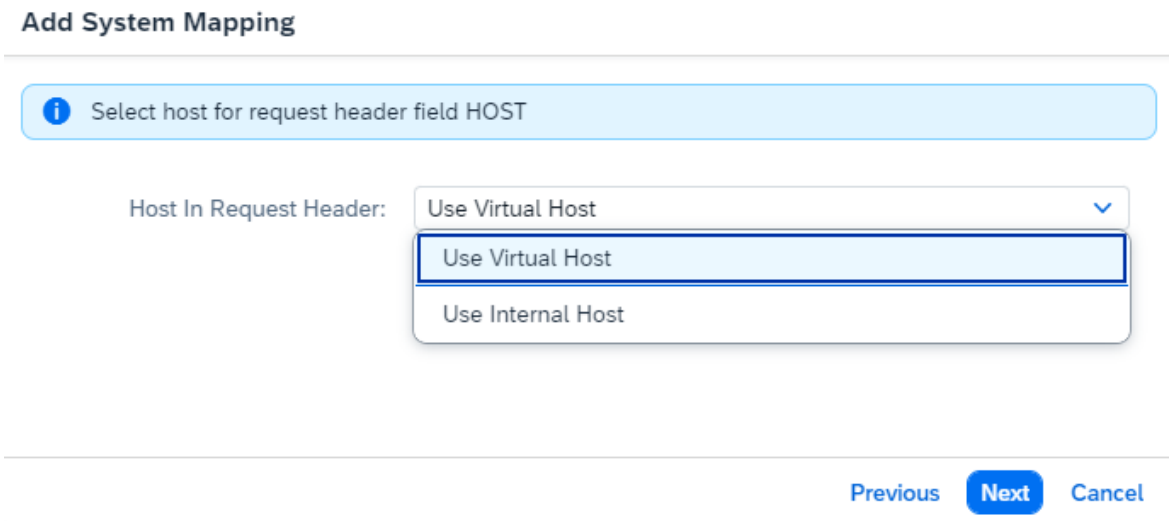
   For more information on principal propagation, see Configuring Principal Propagation [page 1702].
9. *System Certificate for Logon* (available as of Cloud Connector 2.15): Specifies if the Cloud Connector's system certificate should be used for authentication at the back end, if
   1. No principal is received, or
   2. Principal propagation is not allowed over this mapping at all.

   If *activated*, the system certificate of the TLS handshake used for trust is also used for authentication.
   If *not activated*, an additional HTTP header (`SSL_CLIENT_CERT`) is sent. It indicates to the target system that the system certificate used for trust must not be used for authentication.
   While unselected, another authentication method is used, for example, basic authentication.

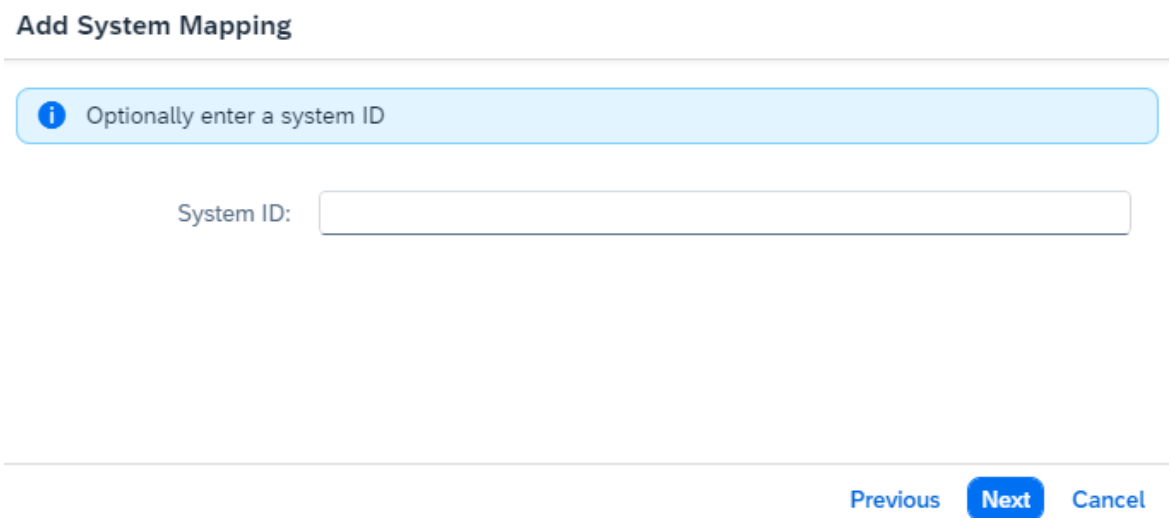
> **Note**
>
> We recommend that you **keep this option deactivated**, as this lets you use principal propagation and basic authentication over the same access control entry, regardless of the logon order settings in the target system.



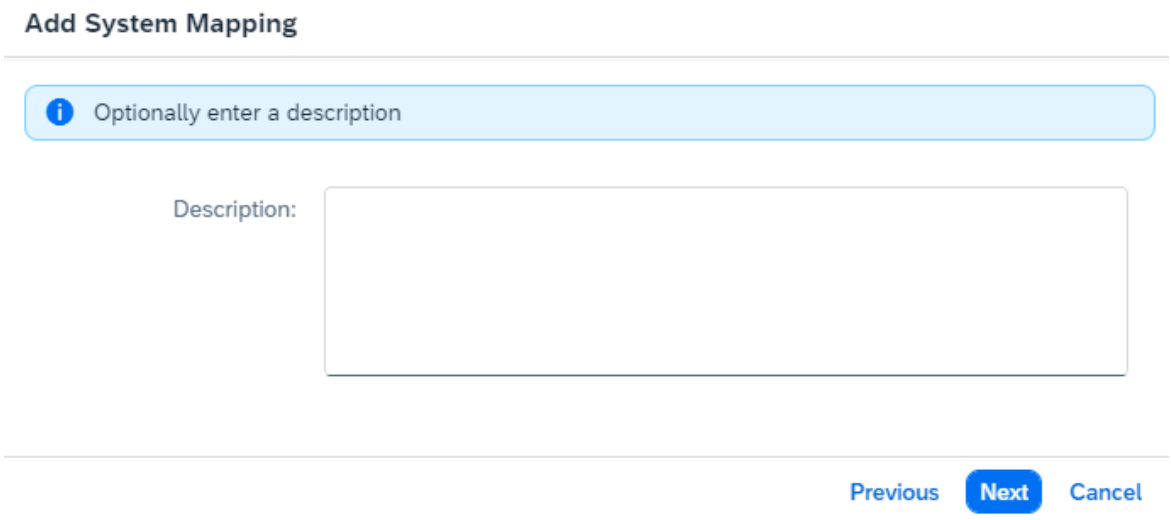
10. *Host In Request Header* lets you define, which host is used in the host header that is sent to the target server. By choosing **`Use Internal Host`**, the actual host name is used. When choosing **`Use Virtual Host`**, the virtual host is used. In the first case, the virtual host is also sent via the `X-Forwarded-Host` header.



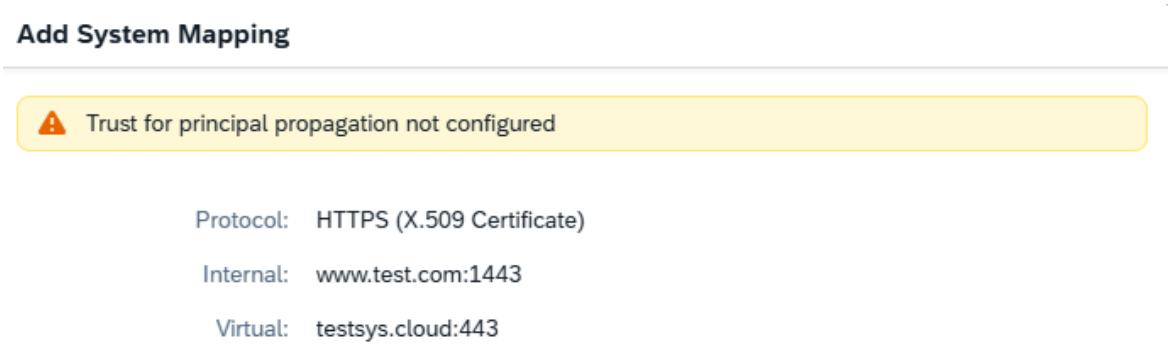
11. (Optional): You can enter the *System ID*:



12. You can enter an optional description at this stage. The respective description will be shown when pressing the *Info* button of the access control entry (table *Mapping Virtual to Internal System*).

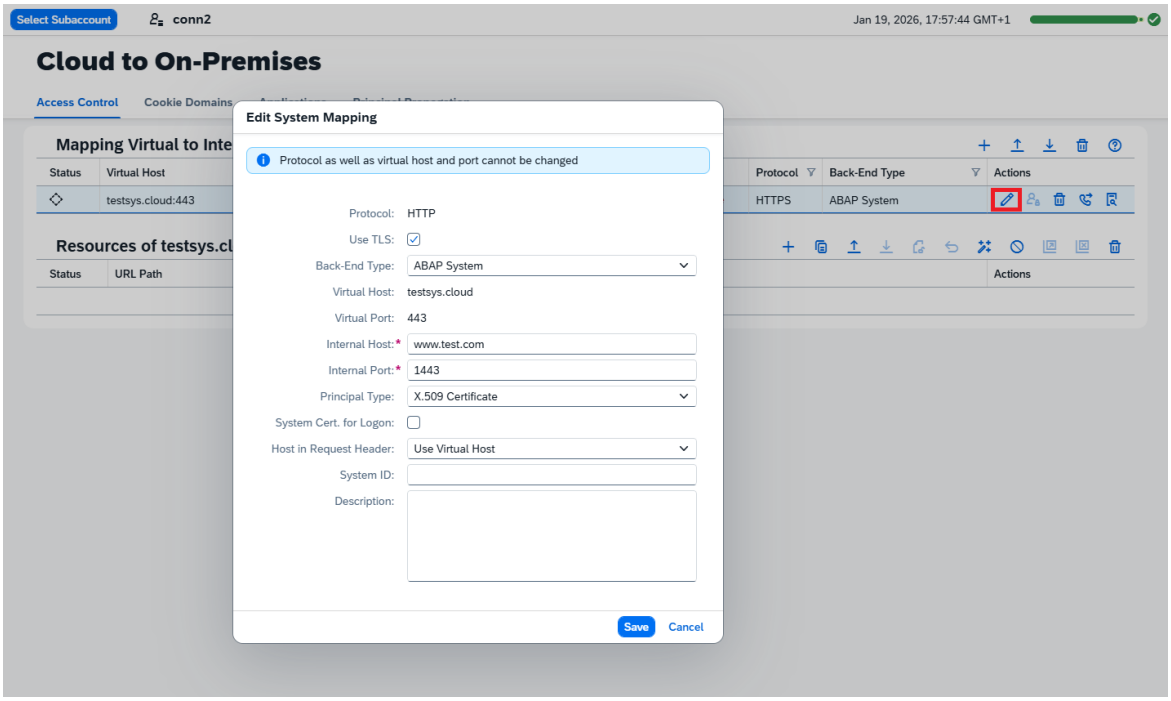


13. The summary shows information about the system to be stored. When saving the host mapping, a ping is triggered from the Cloud Connector to the internal host. This allows you to make sure the Cloud Connector can indeed access the internal system. That way, you can detect basic issues such as spelling mistakes or firewall problems between the Cloud Connector and the internal host at an early stage.
If the ping to the internal host is successful, the state `Reachable` is shown. If it fails, a warning is shown. You can view issue details by choosing the *Details* button, or check them in the log files.
You can execute such a check later at any time in the **Mapping Virtual To Internal System** overview by pressing **Check Availability of Internal Host** in column **Actions**.



> → Tip
>
> The connection check is done via a HEAD request to '/' on the respective internal host. Therefore, it's a good idea to either allow this request or to redirect it to a meaningful ping service. In case a web dispatcher is used, a dispatching entry needs to be added for '/' to avoid a connection failure.

14. Optional: You can later edit such a system mapping (via *Edit*), for example, to make the Cloud Connector route the requests for **`sales-system.cloud:443`** to a different back-end system. This can be useful if the system is currently down and there is a back-up system that can serve these requests in the meantime. However, you cannot edit the virtual name of this system mapping. If you want to use a different fictional host name in your cloud application, you must delete the mapping and create a new one.



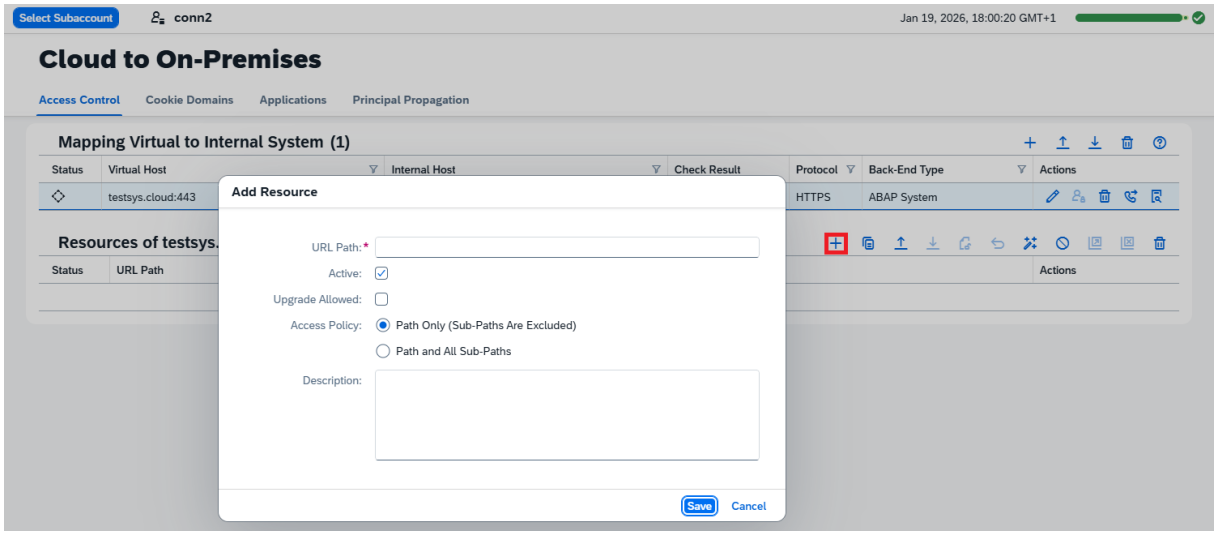
Back to Tasks [page 1744]

Limit the Accessible Services for HTTP(S)

In addition to allowing access to a particular host and port, you also must specify which URL paths (*Resources*) are allowed to be invoked on that host. The Cloud Connector uses very strict allowlists for its access control. Only those URLs for which you explicitly granted access are allowed. All other HTTP(S) requests are denied by the Cloud Connector.

To define the permitted URLs for a particular back-end system, choose the line corresponding to that back-end system and choose *Add* in section *Resources Accessible On...* below. A dialog appears prompting you to enter the specific URL path that you want to allow to be invoked.

> **Note**
>
> The path name is case sensitive.



The Cloud Connector checks that the path part of the URL (up to but not including a possible question mark (**?**) that may denote the start of optional CGI-style query parameters) is exactly as specified in the configuration. If it is not, the request is denied. If you select option *Path and all sub-paths*, the Cloud Connector allows all requests for which the URL path (not considering any query parameters) starts with the specified string.

The *Active* checkbox lets you specify, if that resource is initially enabled or disabled. See the section below for more information on enabled and disabled resources.

The *Upgrade Allowed* checkbox lets you specify, whether that resource allows a protocol upgrade, for example, upgrade an HTTPS connection to a WebSocket connection.

Back to Tasks [page 1744]

Activate or Suspend Resources

In some cases, it is useful for testing purposes to temporarily disable certain resources without having to delete them from the configuration. This allows you to easily reprovide access to these resources at a later point of time without having to type in everything once again.

- To suspend a resource, select it and choose the *Suspend* button:
  The status icon turns red, and from now on, the Cloud Connector will deny all requests coming in for this resource.

- To activate the resource again, select it and choose the *Activate* button.
- By choosing *Upgrade Allowed*, this is possible for the protocol upgrade setting as well.
- It is also possible to mark multiple lines and then suspend or activate all of them in one go by clicking the *Activate/Suspend* icons in the top row. The same is true for the corresponding *Upgrade Allowed* checkboxes.

Examples:

- */production/accounting* and *Path only (sub-paths are excluded)* are selected. Only requests of the form `GET /production/accounting` or `GET /production/accounting?name1=value1&name2=value2...` are allowed. (`GET` can also be replaced by `POST`, `PUT`, `DELETE`, and so on.)
- */production/accounting* and *Path and all sub-paths* are selected. All requests of the form `GET /production/accounting-plus-some-more-stuff-here?name1=value1...` are allowed.
- */* and *Path and all sub-paths* are selected. All requests to this server are allowed.

Back to Tasks [page 1744]

Related Information

Configure Domain Mappings for Cookies [page 1958]
Consume Backend Systems (Java EE 7 Web Profile TomEE 7)

1.4.2.4.2 Configure Access Control (RFC)

Specify the back-end systems that can be accessed by your cloud applications using RFC.

Tasks

Expose Intranet Systems [page 1753]

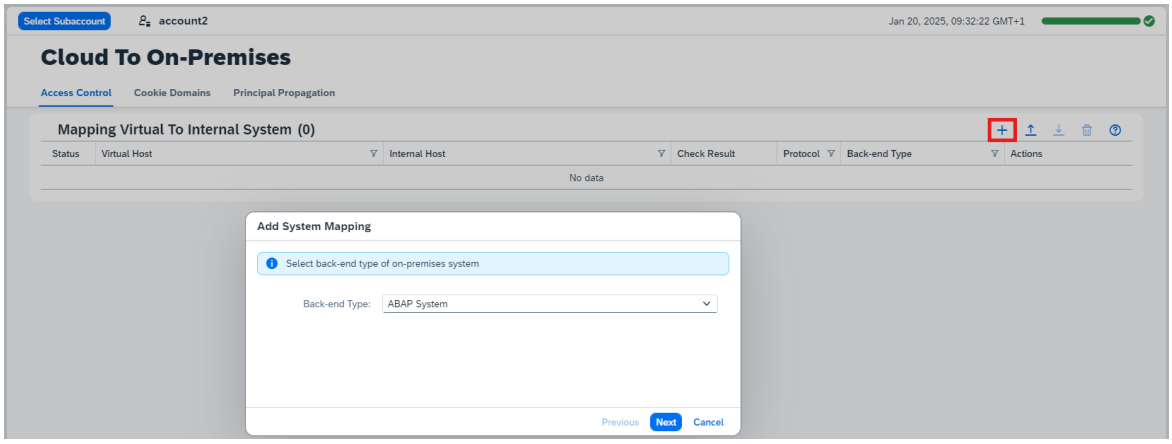
Limit the Accessible Resources for RFC [page 1759]

Configure WebSocket RFC [page 1760]

Expose Intranet Systems

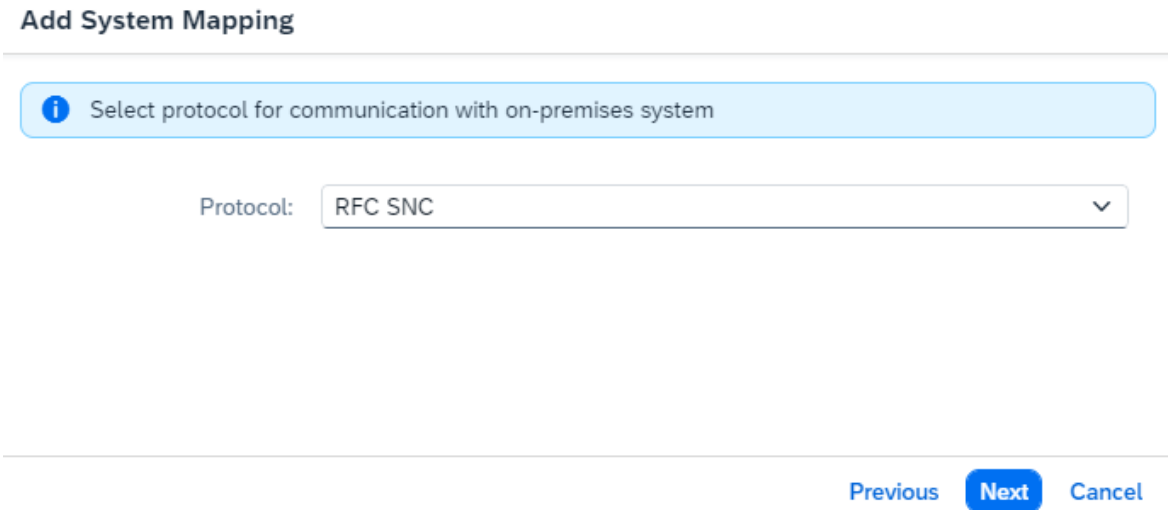
To allow your cloud applications to access a certain back-end system on the intranet, insert a new entry in the Cloud Connector *Access Control* management.

1. Choose *Cloud To On-Premises* from your *Subaccount* menu and go to tab *Access Control*.
2. Choose *Add*.
3. *Back-end Type*: Select the back-end system type (**`ABAP System`** or **`SAP Gateway`** for RFC).



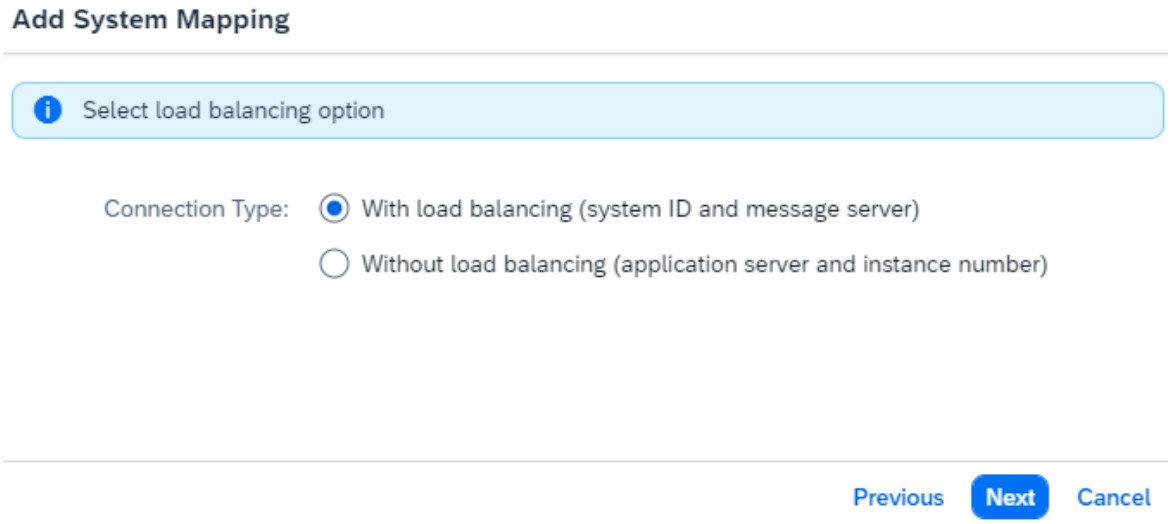
4. Choose *Next*.
5. *Protocol*: Choose **RFC** or **RFC SNC** for connecting to the back-end system. For WebSocket RFC (as of version 2.19), see section Configure WebSocket RFC [page 1760] below.

> ### Note
>
> The value **RFC SNC** is independent from your settings on the cloud side, since it only specifies the communication beween Cloud Connector and back-end system. Using **RFC SNC**, you can ensure that the entire connection from the cloud application to the actual back-end system (provided by the SSL tunnel) is secured, partly with SSL and partly with SNC. For more information, see Initial Configuration (RFC) [page 1678].

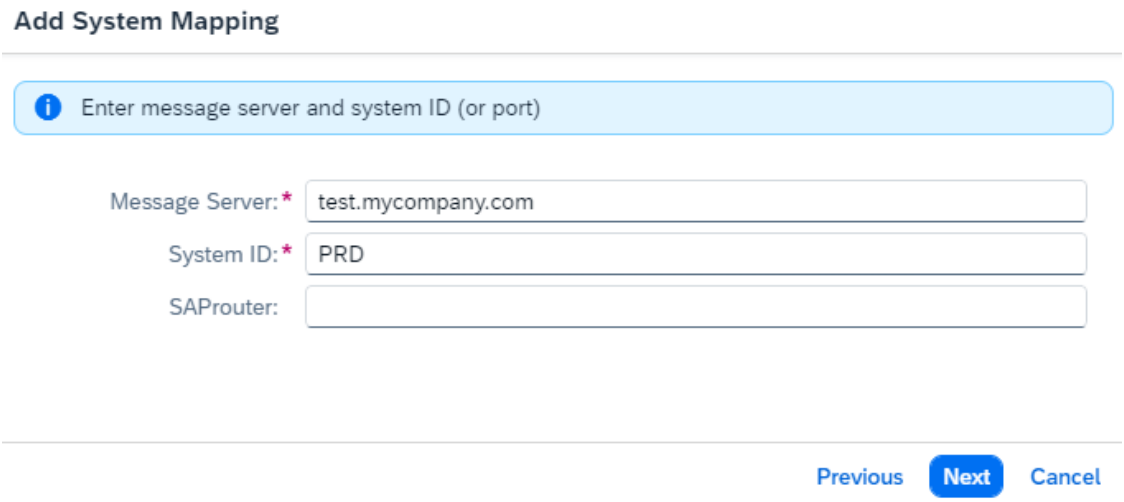


> **Note**
>
> - The back end must be properly configured to support SNC connections.
> - SNC configuration must be provided in the Cloud Connector.
>   For more information, see also Configure Identity Propagation for RFC [page 1721].

6. Choose *Next*.
7. Choose whether you want to configure a load balancing logon or connect to a specific application server.

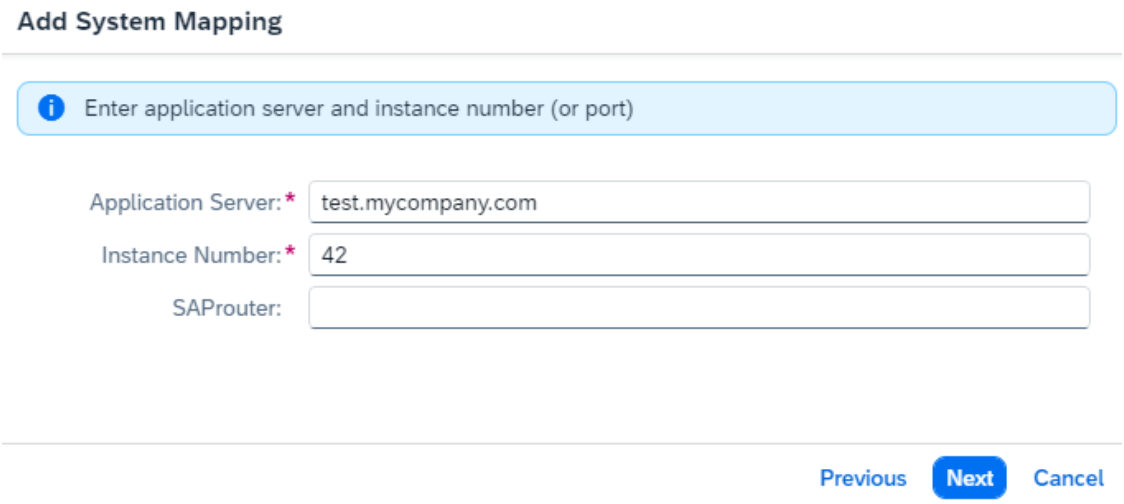


8. Specify the parameters of the back-end system. It needs to be an existing network address that can be resolved on the intranet and has network visibility for the Cloud Connector. If this is only possible using a valid SAProuter, specify the router in the respective field. The Cloud Connector will try to establish a connection to this system, so the address has to be real.
   - When using a load-balancing configuration, the *Message Server* field specifies the message server of the ABAP system. The system ID is a three-char identifier that is also found in the SAP Logon configuration. Alternatively, you can specify the message server port directly in the *System ID* field.

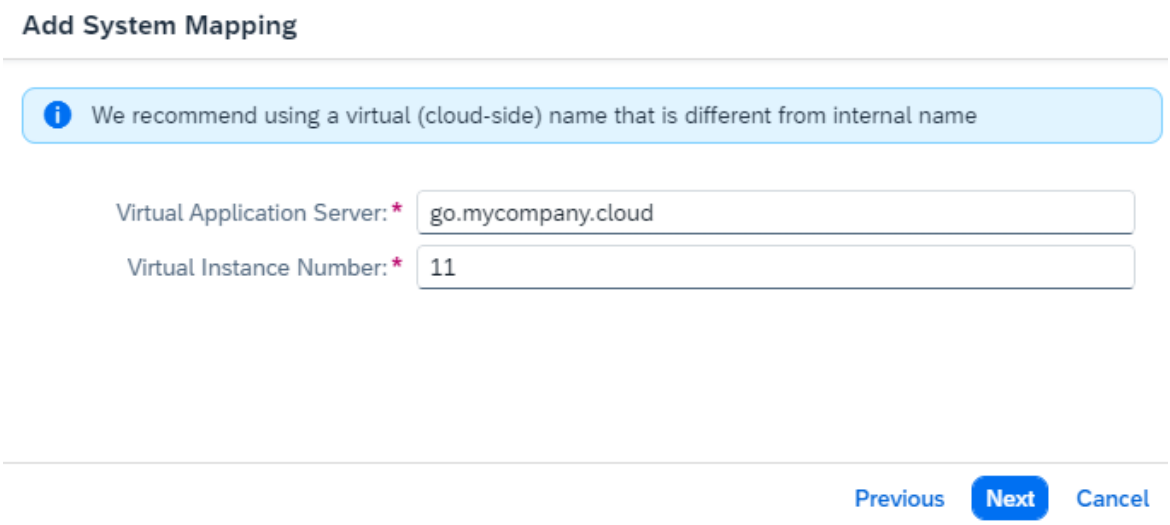


- When using direct logon, the *Application Server* specifies one application server of the ABAP system. The instance number is a two-digit number that is also found in the SAP Logon configuration. Alternatively, you can specify the gateway port directly in the *Instance Number* field.

> **Note**
>
> The default gateway port for RFC communication is **`33<instance number>`** (for example, **`3302`**). For RFC SNC, the gateway port is **`48<instance number>`**.
>
> In both cases (RFC and RFC SNC), the internal network must allow access to the required ports.



9. Optional: You can virtualize the system information in case you like to hide your internal host names from the cloud. The virtual information can be a fake name which does not need to exist. The fields will be pre-populated with the values of the configuration provided in *Message Server* and *System ID*, or *Application Server* and *Instance Number*.

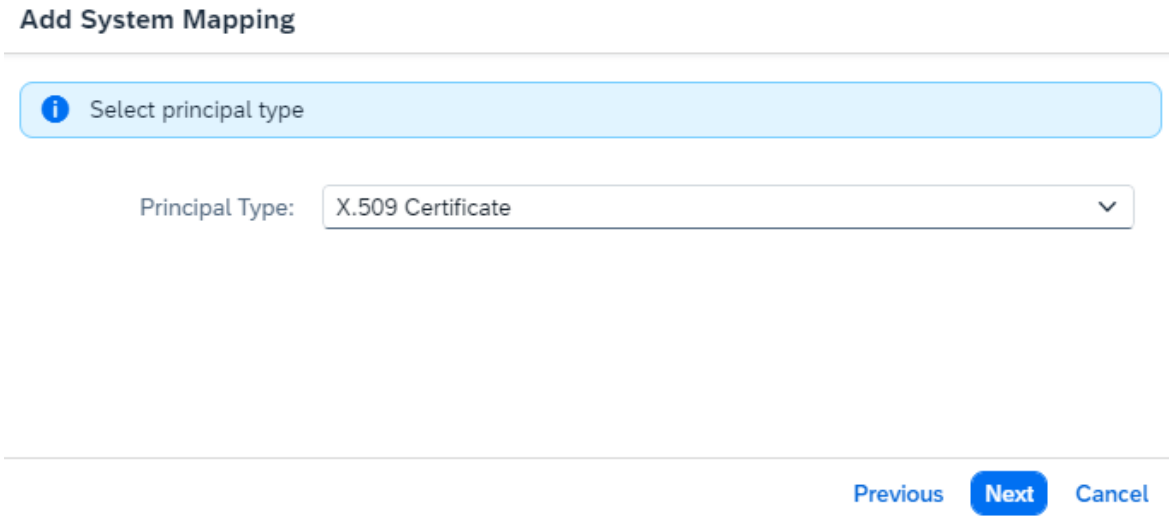


- *Virtual Message Server* - specifies the host name exactly as specified as the `jco.client.mshost` property in the RFC destination configuration in the cloud. The *Virtual System ID* allows you to distinguish between different entry points of your back-end system that have different sets of access control settings. The value needs to be the same like for the `jco.client.r3name` property in the RFC destination configuration in the cloud.
- *Virtual Application Server* - it specifies the host name exactly as specified as the `jco.client.ashost` property in the RFC destination configuration in the cloud. The *Virtual Instance Number* allows you to distinguish between different entry points of your back-end system that have different sets of access control settings. The value needs to be the same like for the `jco.client.sysnr` property in the RFC destination configuration in the cloud.

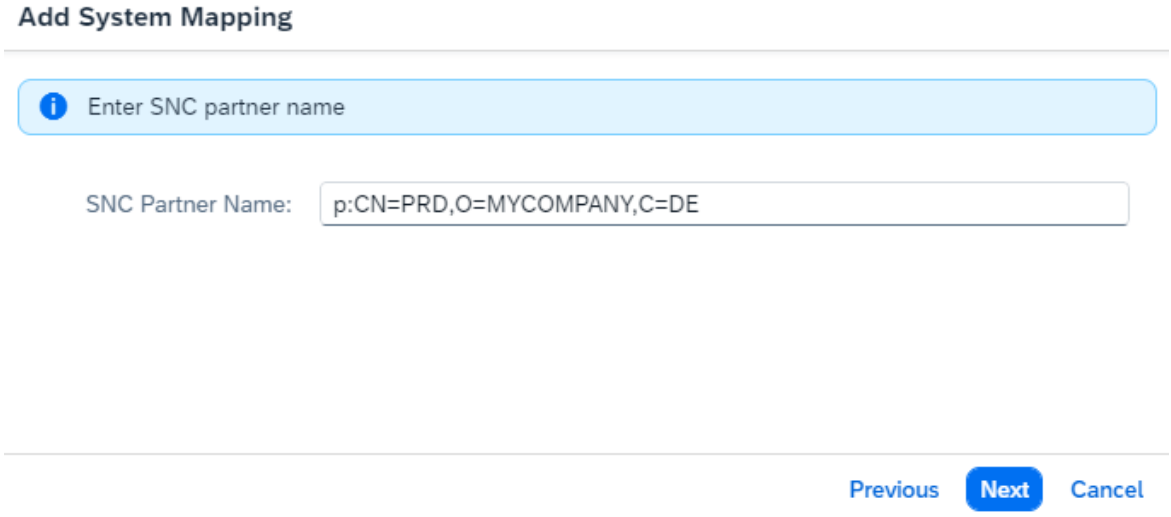
10. This step is only relevant if you have chosen **`RFC SNC`**. The `<Principal Type>` field defines what kind of principal is used when configuring a destination on the cloud side using this system mapping with authentication type `Principal Propagation`. No matter what you choose, make sure that the general configuration for the `<Principal Type>` has been done to make it work correctly. For destinations using different authentication types, this setting is ignored. In case you choose **`None`** as `<Principal Type>`, it is not possible to apply principal propagation to this system.

> **Note**
>
> If you use an RFC connection, you cannot choose between different principal types. Only the **`X.509`** certificate is supported. You need an SNC-enabled back-end connection to use it.

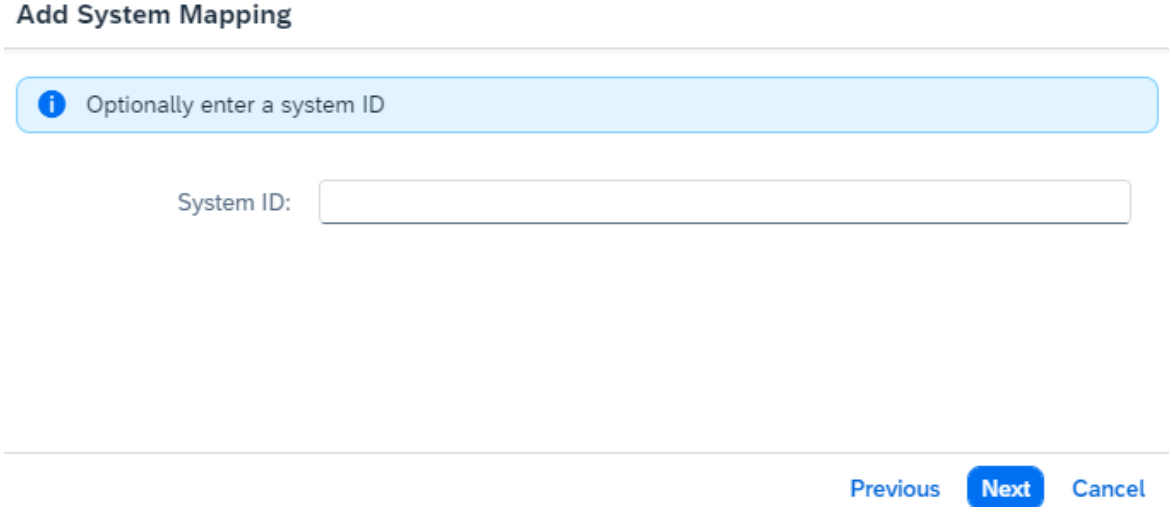
For more information on principal propagation, see Configuring Principal Propagation [page 1702].



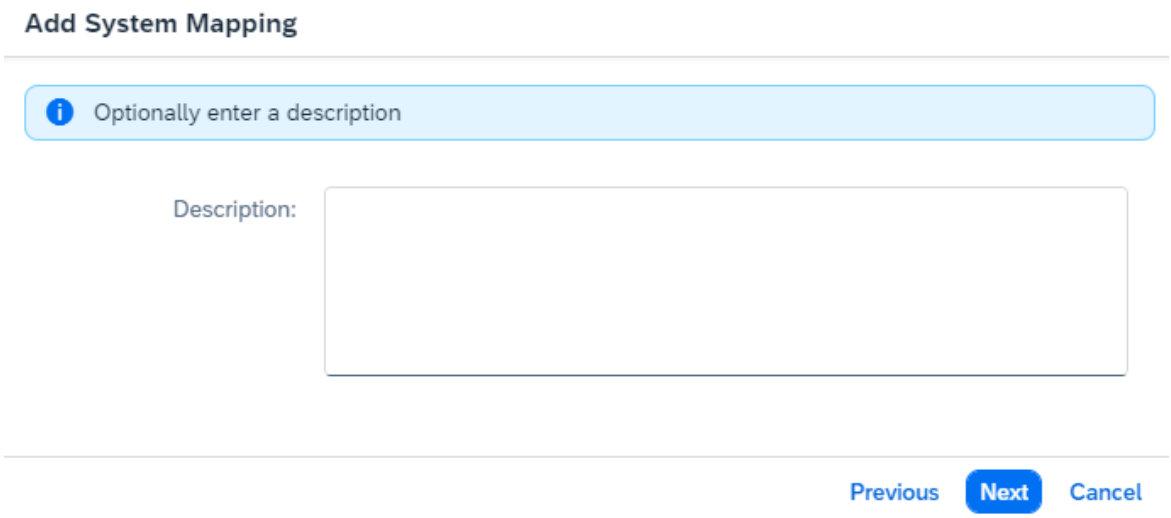
11. *SNC Partner Name*: This step will only come up if you have chosen **RFC** **SNC**. The SNC partner name needs to contain the correct SNC identification of the target system.



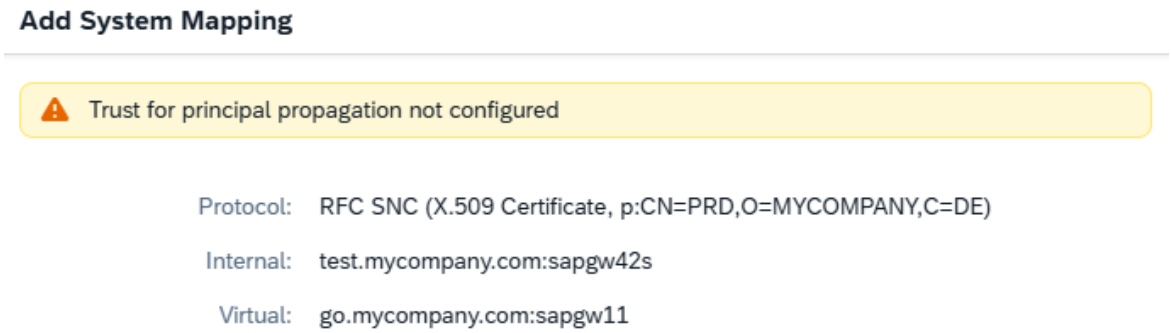
12. (Optional): You can enter the *System ID*:



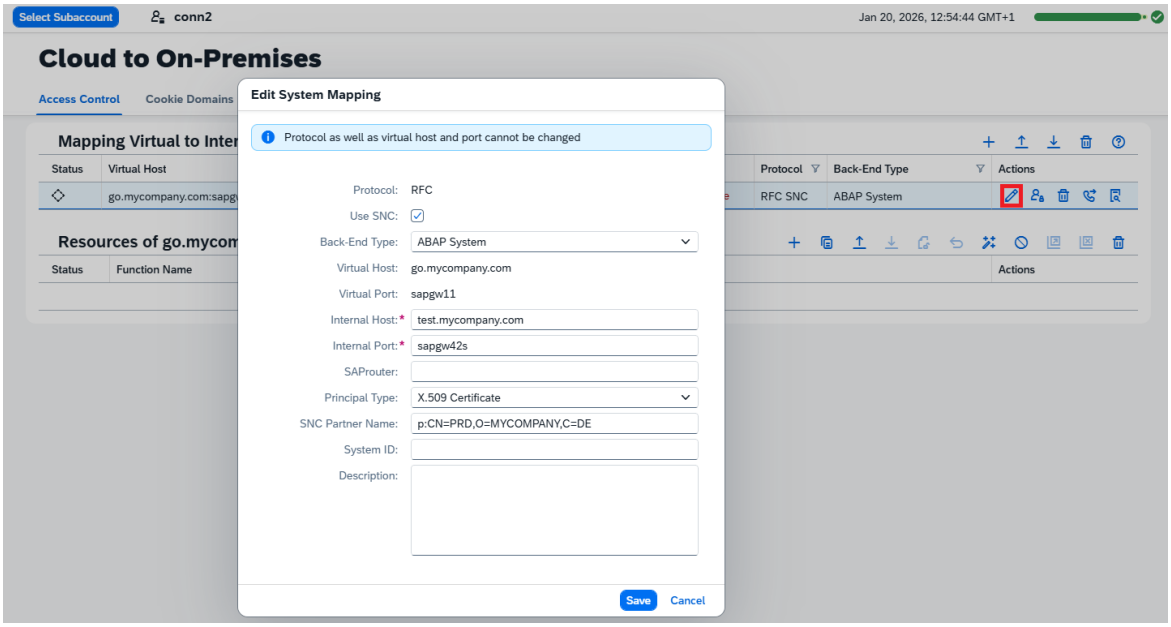
13. *Mapping Virtual to Internal System*: You can enter an optional description at this stage. The respective description will be shown as a rich tooltip when the mouse hovers over the entries of the virtual host column (table).



14. The summary shows information about the system to be stored. When saving the host mapping, a ping is triggered from the Cloud Connector to the internal host. This allows you to make sure the Cloud Connector can indeed access the internal system. That way, you can detect basic issues such as spelling mistakes or firewall problems between the Cloud Connector and the internal host at an early stage.
If the ping to the internal host is successful, the state `Reachable` is shown. If it fails, a warning is shown. You can view issue details by choosing the *Details* button, or check them in the log files.
You can execute such a check later at any time in the **Mapping Virtual To Internal System** overview by pressing **Check Availability of Internal Host** in column **Actions**.

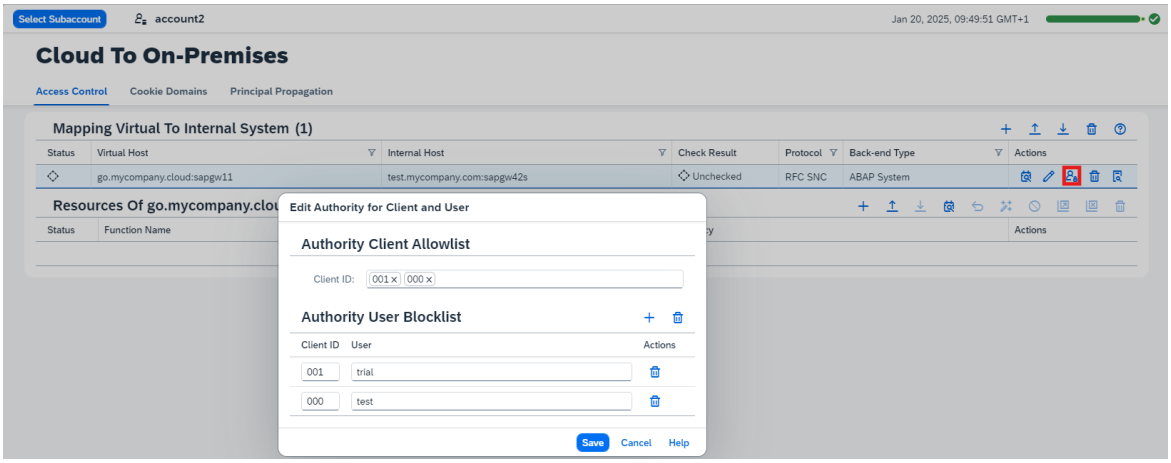


15. Optional: You can later edit a system mapping (choose *Edit*), for example, to make the Cloud Connector route the requests to a different back-end system. This can be useful if the system is currently down and there is a back-up system that can serve these requests in the meantime. However, you cannot edit the virtual name of this system mapping. If you want to use a different fictional host name in your cloud application, you must delete the mapping and create a new one.



16. *Optional*. You can later edit a system mapping to add more protection to your system when using RFC via theCloud Connector, by restricting the mapping to specified clients and users: in column *Actions*, choose the button *Maintain Authority Lists (only RFC)* to open an allowlist/blocklist dialog. In section *Authority Client Allowlist*, enter all clients of the corresponding system in the field `<Client ID>` that you want to allow to use the Cloud Connector connection. In section *Authority User Blocklist*, press the button *Add a user authority* (+) to enter all users you want to exclude from this connection. Each user must be assigned to a specified client. When you are done, press *Save*.

> **Note**
>
> This function applies for RFC/RFC SNC only.



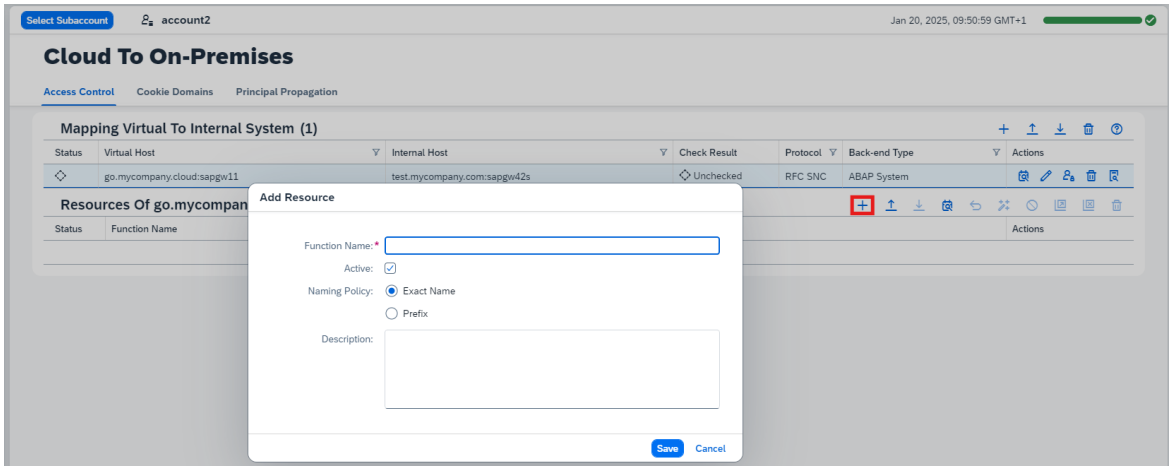
Back to Tasks [page 1753]

Limit the Accessible Resources for RFC

In addition to allowing access to a particular host and port, you also must specify which function modules (*Resources*) are allowed to be invoked on that host. You can enter an optional description at this stage.

The Cloud Connector uses very strict allowlists for its access control. Besides internally used infrastructure function modules, only function modules for which you explicitly granted access are allowed.

1. To define the permitted function modules for a particular back-end system, choose the row corresponding to that back-end system and press *Add* in section *Resources Accessible On...* below. A dialog appears, prompting you to enter the specific function module name whose invoking you want to allow.

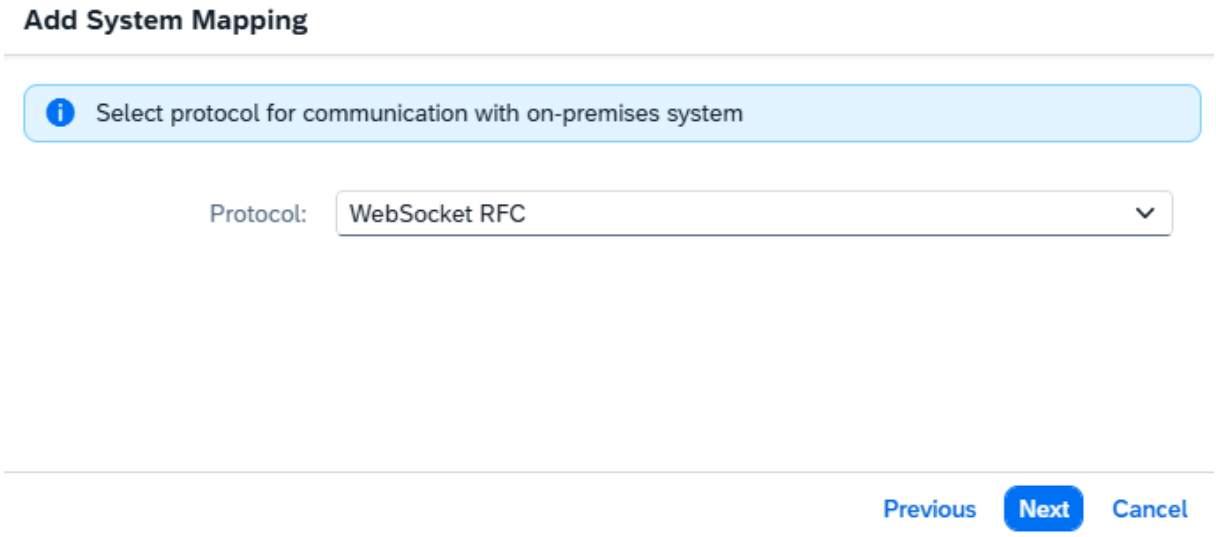


2. The Cloud Connector checks that the function module name of an incoming request is exactly as specified in the configuration. If it is not, the request is denied.
3. If you select the *Prefix* option, the Cloud Connector allows all incoming requests, for which the function module name begins with the specified string.
4. The *Active* checkbox allows you to specify whether that resource should be initially enabled or disabled.

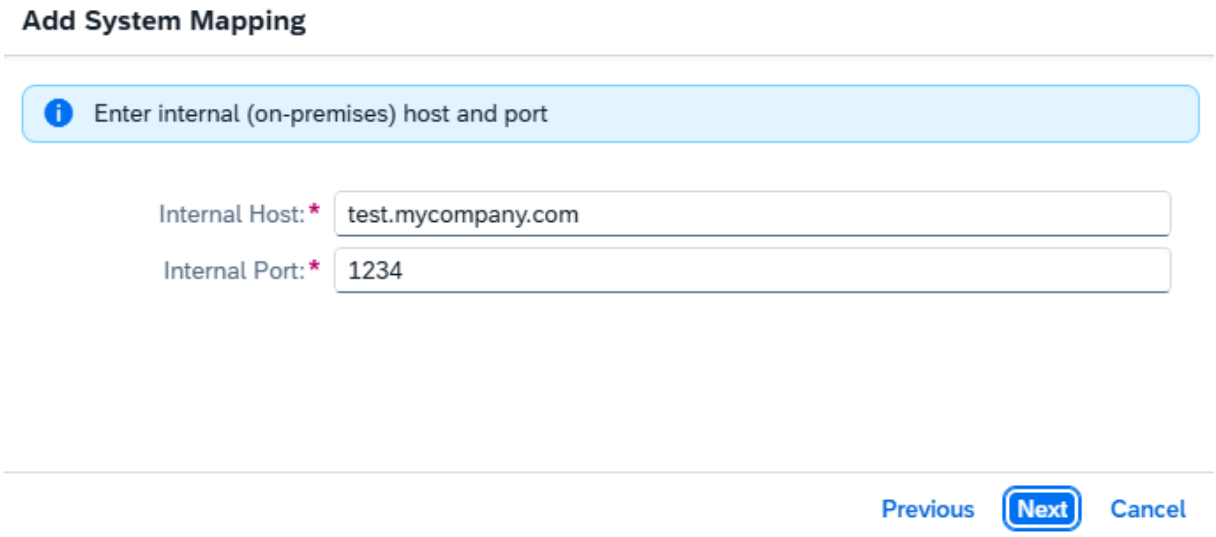
Back to Tasks [page 1753]

Configure WebSocket RFC

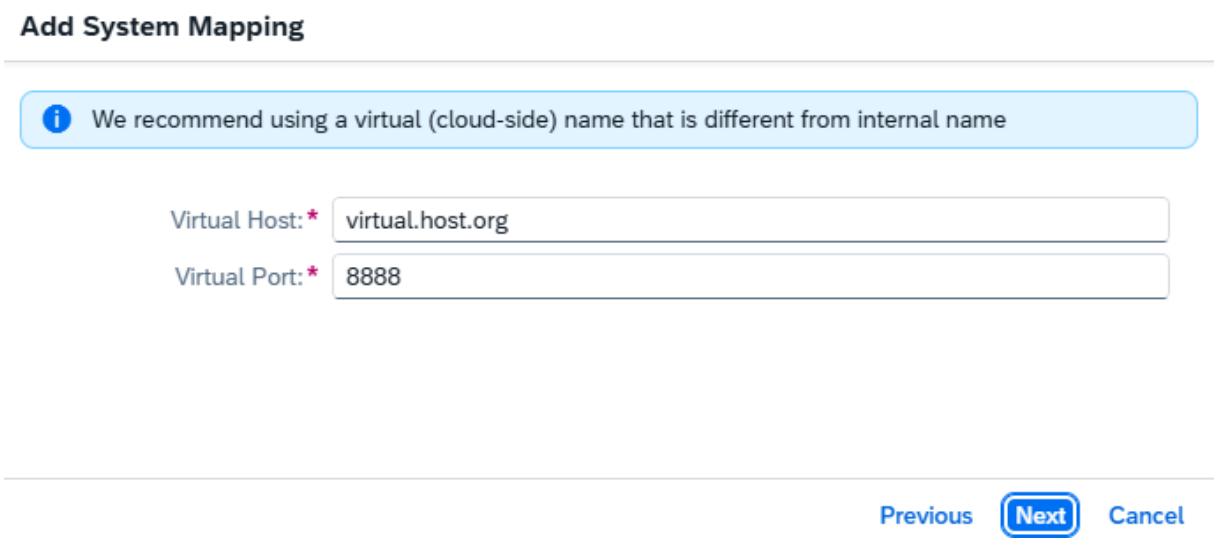
As of Cloud Connector version 2.19, you can employ *RFC via WebSocket Secure* (or *WebSocket RFC* for short). Since it is based on HTTP, its configuration differs from classic RFC configuration as shown above. The first steps up to selecting the protocol are the same. To use WebSocket RFC, select **`WebSocket RFC`** at this step:



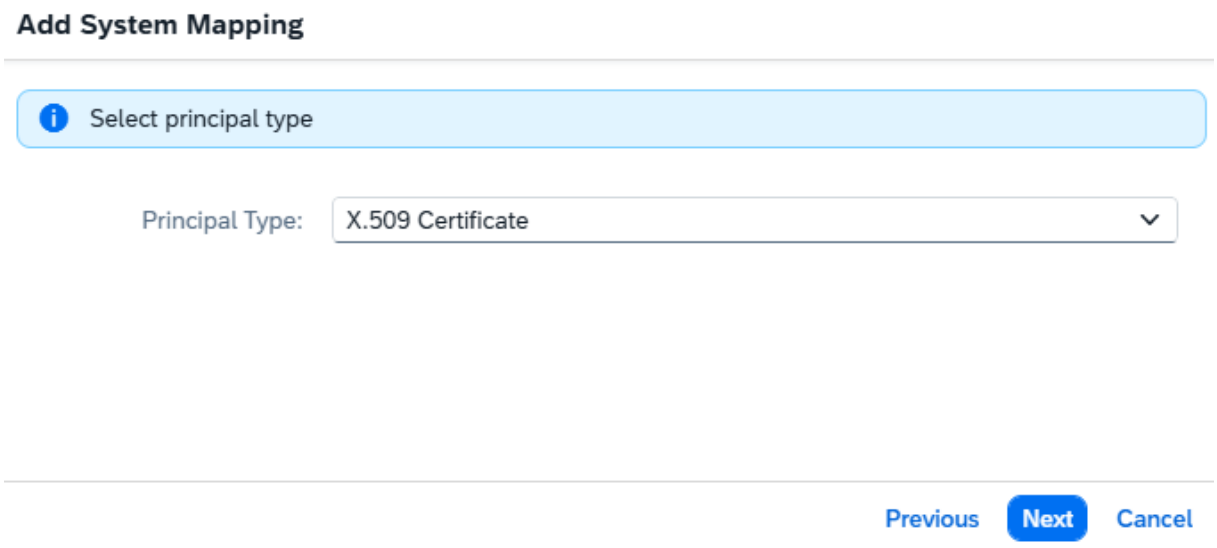
Similar to HTTP, internal and virtual host and port need to be provided next:



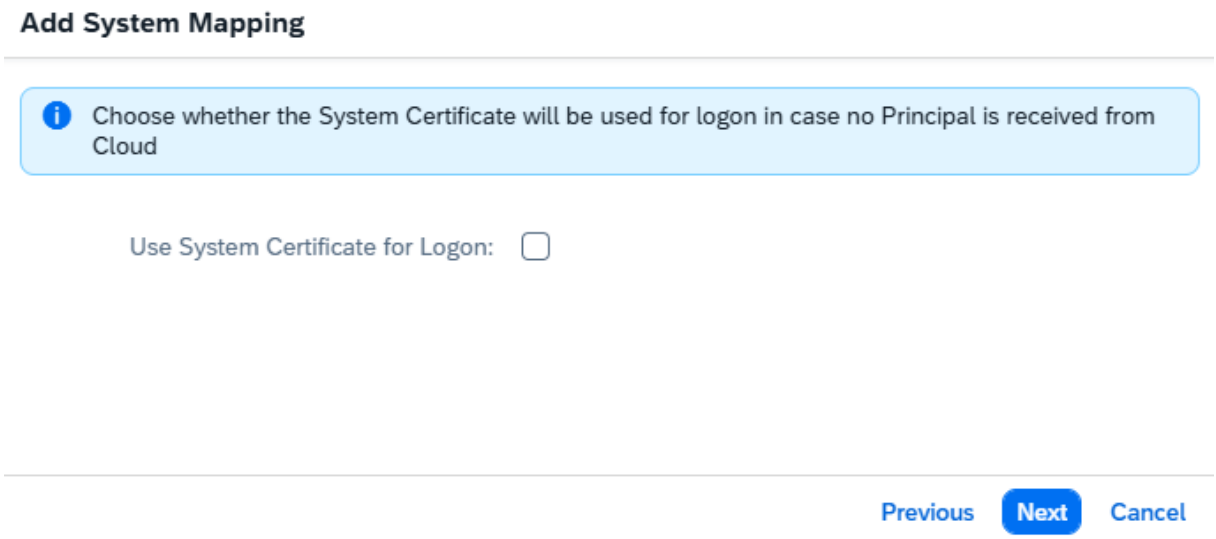
We recommend that you choose an arbitrary (but unique) virtual host name that differs from the internal host if only to hide internal network details or to avoid confusion when the internal host needs to be changed:



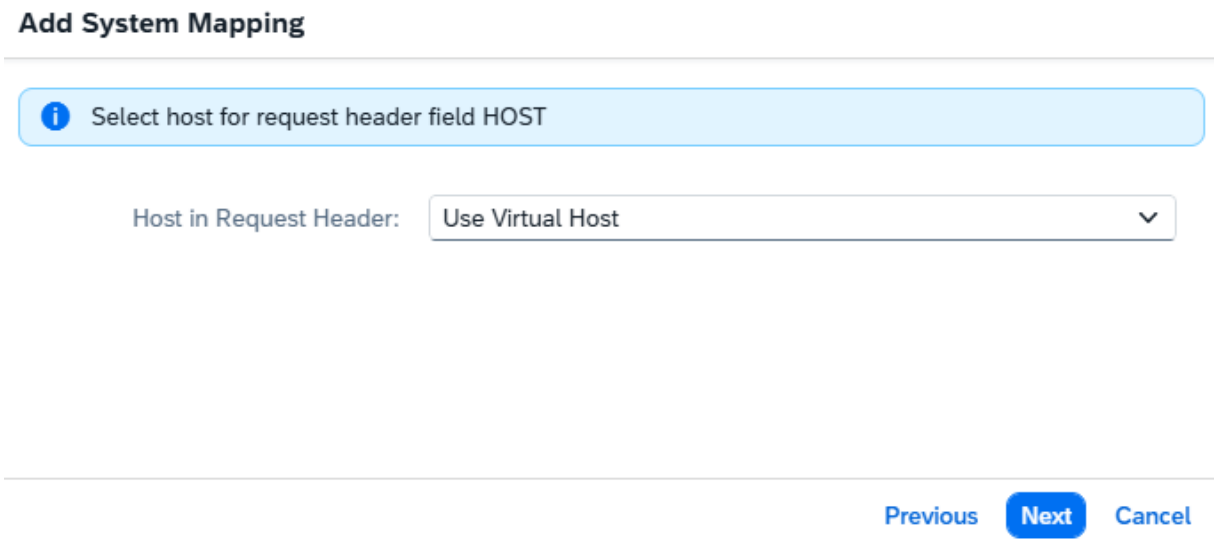
Next, select what kind of principal is sent to the back end if any:



If no principal is available, you can opt to use the system certificate for authentication at the back end:



Since WebSocket RFC is based on HTTP, you have the same choice as with HTTP regarding the value to be used for header `Host`. By choosing *Use Internal Host*, the actual host name is used, and when choosing *Use Virtual Host* the virtual one is used. In the first case, the virtual host is also sent via the `X-Forwarded-Host` header:



The final steps are optional and are the same as for classic RFC: providing a system ID and a description.

You can edit a WebSocket RFC entry at any time with the same kind of restrictions that apply to classic RFC or any access control entry.

> **Note**
>
> WebSocket RFC is per se secured. Therefore, there is no additional option similar to SNC for classic RFC.

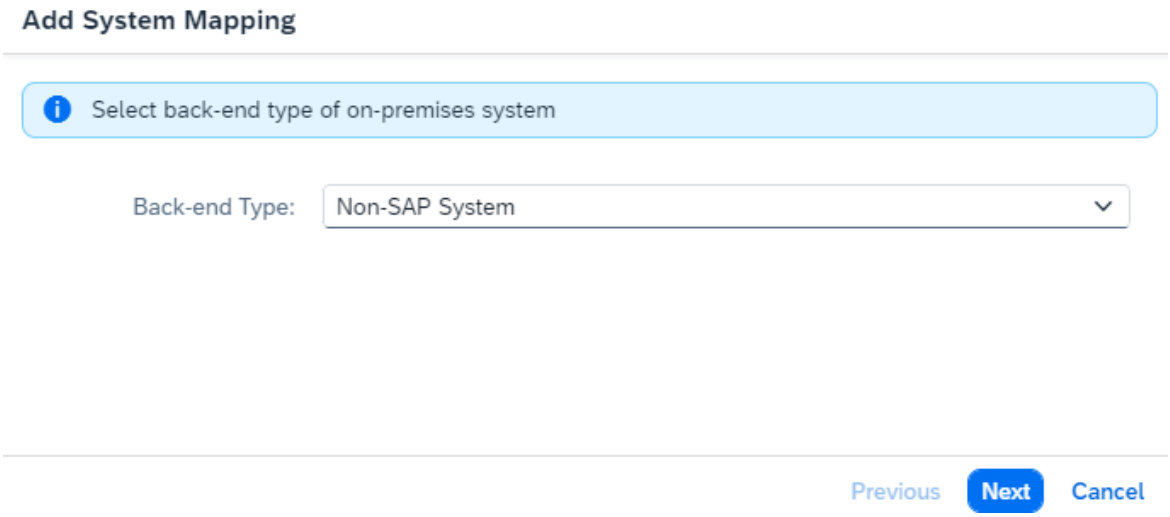
Back to Tasks [page 1753]

1.4.2.4.3 Configure Access Control (LDAP)

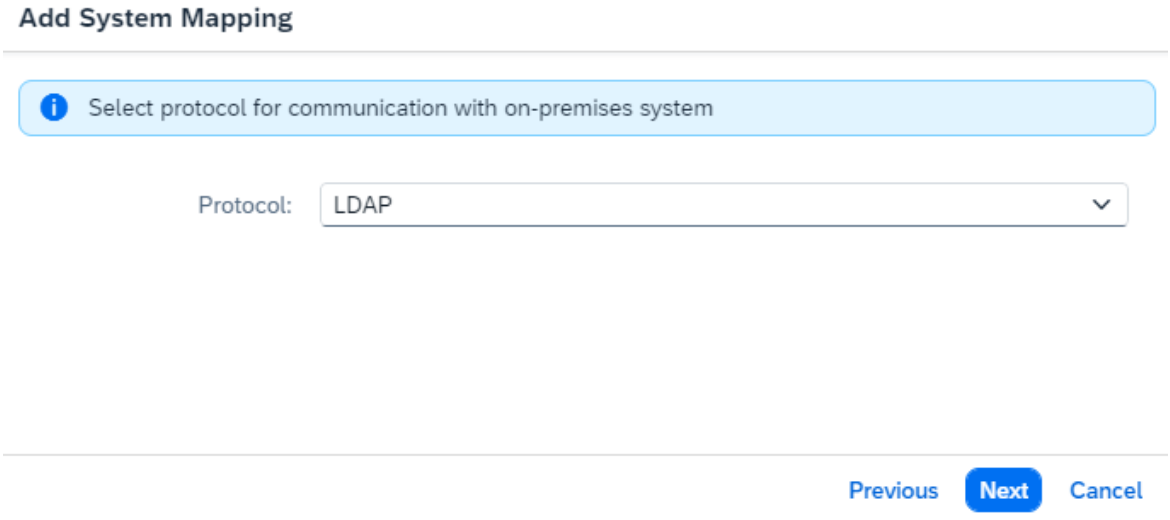
Add a specified system mapping to the Cloud Connector if you want to use an on-premises LDAP server or user authentication in your cloud application.

To allow your cloud applications to access an on-premises LDAP server, insert a new entry in the Cloud Connector access control management.

1. Choose *Cloud To On-Premises* from your *Subaccount* menu.
2. Choose *Add* (+). A wizard opens and asks for the required values.
3. *Back-End Type*: Select **`Non-SAP System`** from the drop down list. When you are done, choose *Next*.

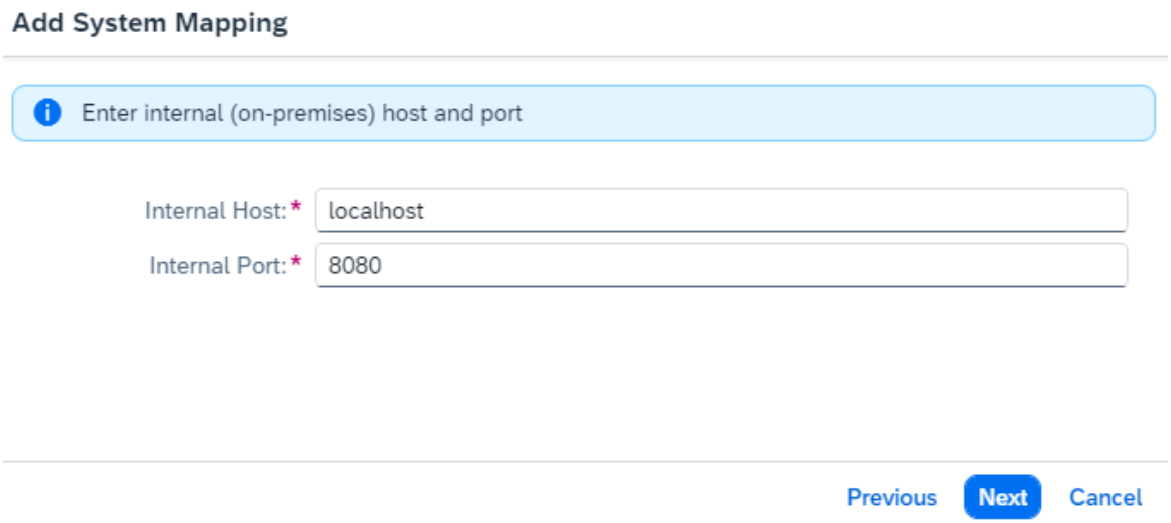


4. *Protocol*: Select **LDAP** or **LDAPS** for the connection to the back-end system. When you are done, choose *Next*.

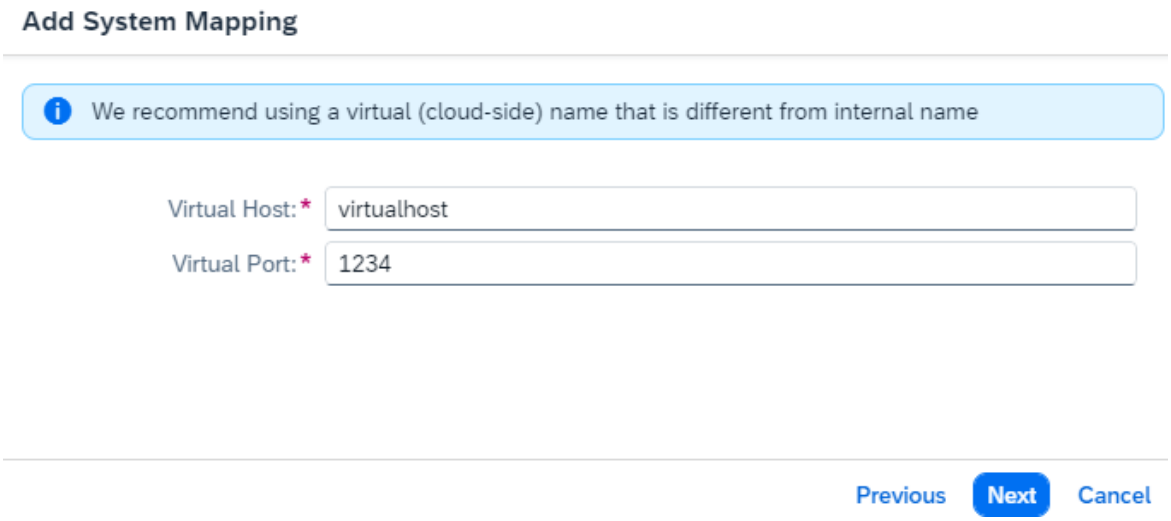


> ### Note
>
> If you specify **LDAPS**, and there is a "system certificate" imported in the Cloud Connector, the latter attempts to use this certificate for performing a client-certificate-based logon to the back-end system.

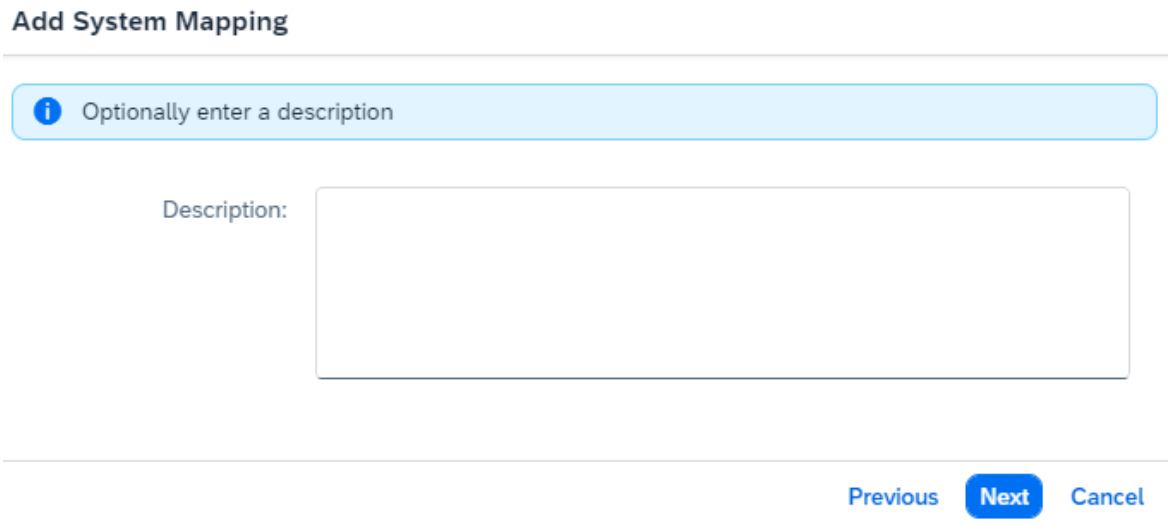
5. *Internal Host* and *Internal Port*: specify the host and port under which the target system can be reached within the intranet. It needs to be an existing network address that can be resolved on the intranet and has network visibility for the Cloud Connector. The Cloud Connector will try to forward the request to the network address specified by the internal host and port, so this address needs to be real.



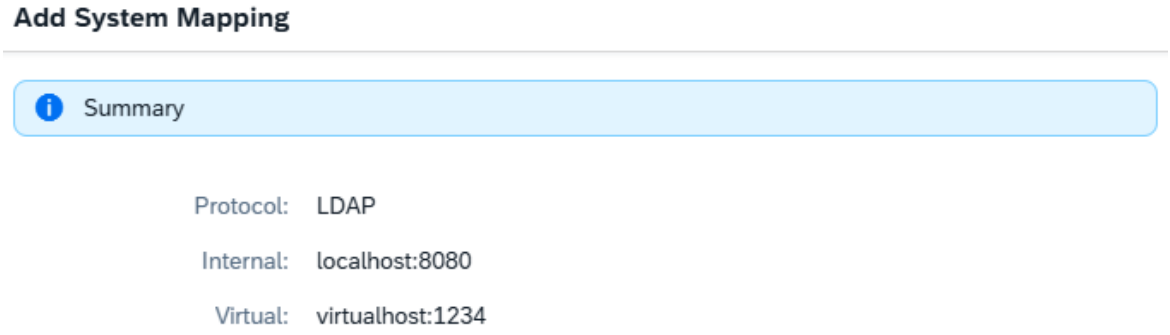
6. Enter a *Virtual Host* and *Virtual Port*. The virtual host can be a fake name and does not need to exist. The fields are pre-populated with the values of the *Internal Host* and *Internal Port*.



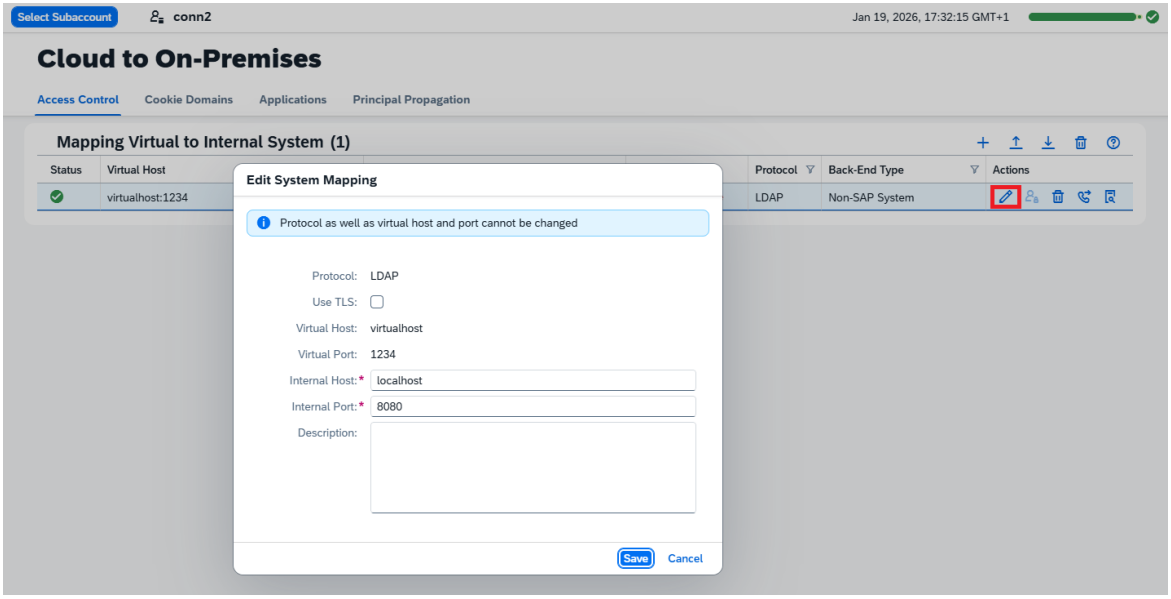
7. You can enter an optional description at this stage. The respective description will be shown as a tooltip when you press the button *Show Details* in column *Actions* of the *Mapping Virtual To Internal System* overview.



8. The summary shows information about the system to be stored. When saving the host mapping, a ping is triggered from the Cloud Connector to the internal host. This allows you to make sure the Cloud Connector can indeed access the internal system. That way, you can detect basic issues such as spelling mistakes or firewall problems between the Cloud Connector and the internal host at an early stage.
   If the ping to the internal host is successful, the state `Reachable` is shown. If it fails, a warning is shown. You can view issue details by choosing the *Details* button, or check them in the log files.
   You can execute such a check later at any time in the **Mapping Virtual To Internal System** overview by pressing **Check Availability of Internal Host** in column **Actions**.



9. Optional: You can later edit the system mapping (by choosing *Edit*) to make the Cloud Connector route the requests to a different LDAP server. This can be useful if the system is currently down and there is a back-up LDAP server that can serve these requests in the meantime. However, you cannot edit the virtual name of this system mapping. If you want to use a different fictional host name in your cloud application, you have to delete the mapping and create a new one.

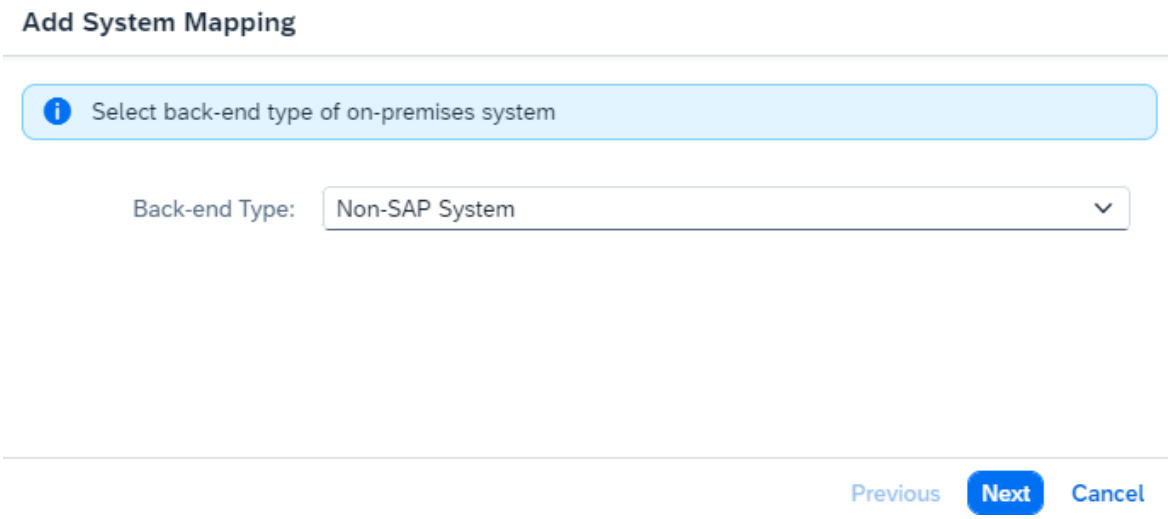


1.4.2.4.4 Configure Access Control (TCP)

Add a specified system mapping to the Cloud Connector if you want to use the TCP protocol for communication with a back-end system.

To allow your cloud applications to access a certain back-end system on the intranet via TCP, insert a new entry in the Cloud Connector access control management.

1. Choose *Cloud To On-Premises* from your *Subaccount* menu.
2. Choose *Add* (+). A wizard opens and asks for the required values.
3. *Back-End Type*: Select an appropriate system type, for example, **`Non-SAP System`**, from the drop-down list. When you are done, choose *Next*.

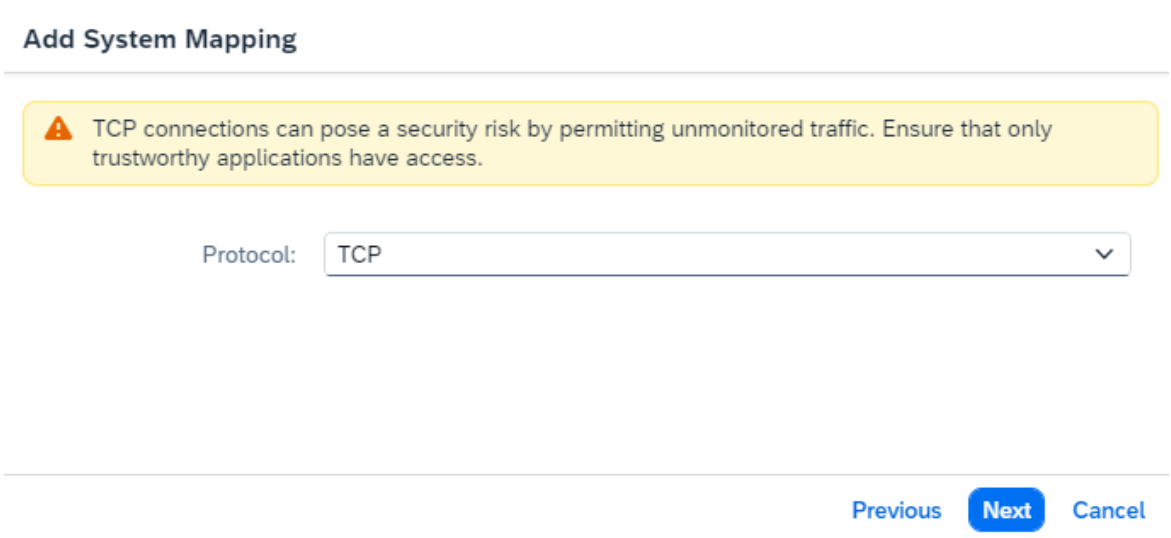


4. *Protocol*: Select **`TCP`** or **`TCP TLS`** for the connection to the back-end system. When choosing TCP, you can perform an end-to-end TLS handshake from the cloud client to the back end. If the cloud-side client

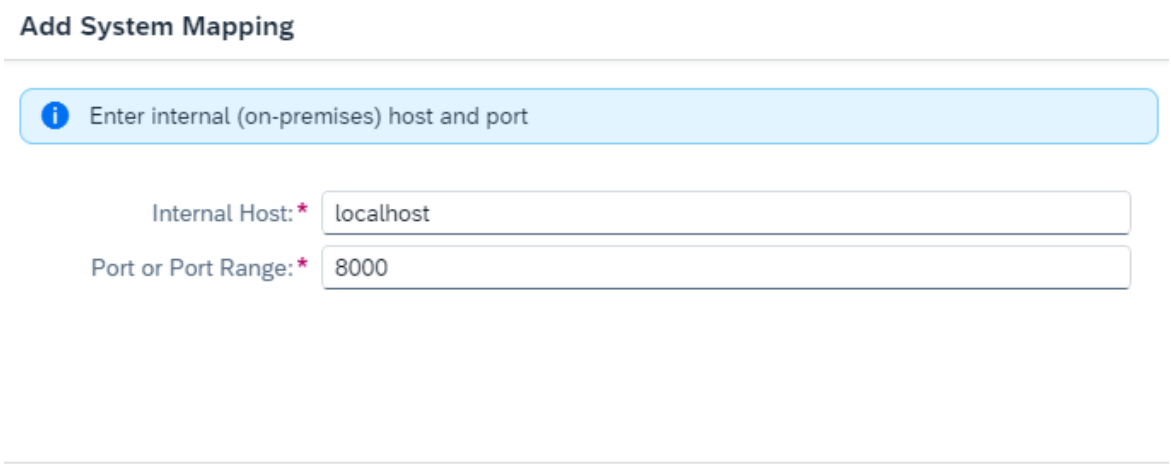
is using plain communication, but you still need to encrypt the hop between Cloud Connector and the back-end, choose **TCP** **TLS**. When you are done, choose *Next*.

> **Note**
>
> When selecting TCP as protocol, the following warning message is displayed: `TCP connections can pose a security risk by permitting unmonitored traffic. Ensure only trustworthy applications have access`. The reason is that using plain TCP, the Cloud Connector cannot see or log any detail information about the calls. Therefore, in contrast to HTTP or RFC (both running on top of TCP), the Cloud Connector cannot check the validity of a request. To minimize this risk, make sure you
>
> - deploy only trusted applications on SAP BTP.
> - configure an application allowlist in the Cloud Connector, see Set Up Trust [page 1702].
> - take the recommended security measures for your SAP BTP (sub)account. See section Security in the SAP BTP documentation.

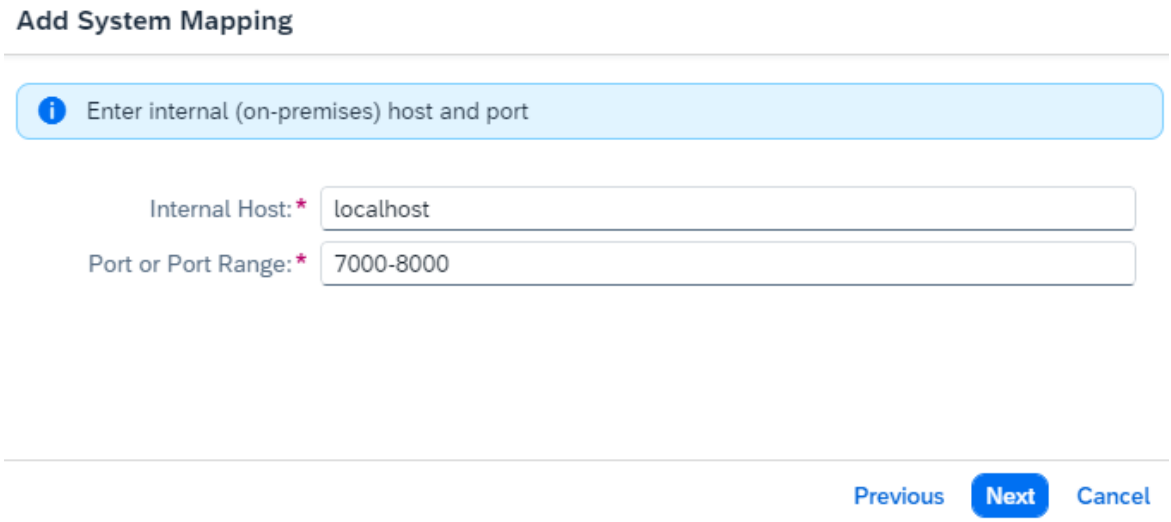
> **Note**
>
> If you specify **TCP** **SSL**, and there is a "system certificate" imported in the Cloud Connector, the latter attempts to use this certificate for performing a client-certificate-based logon to the back-end system.



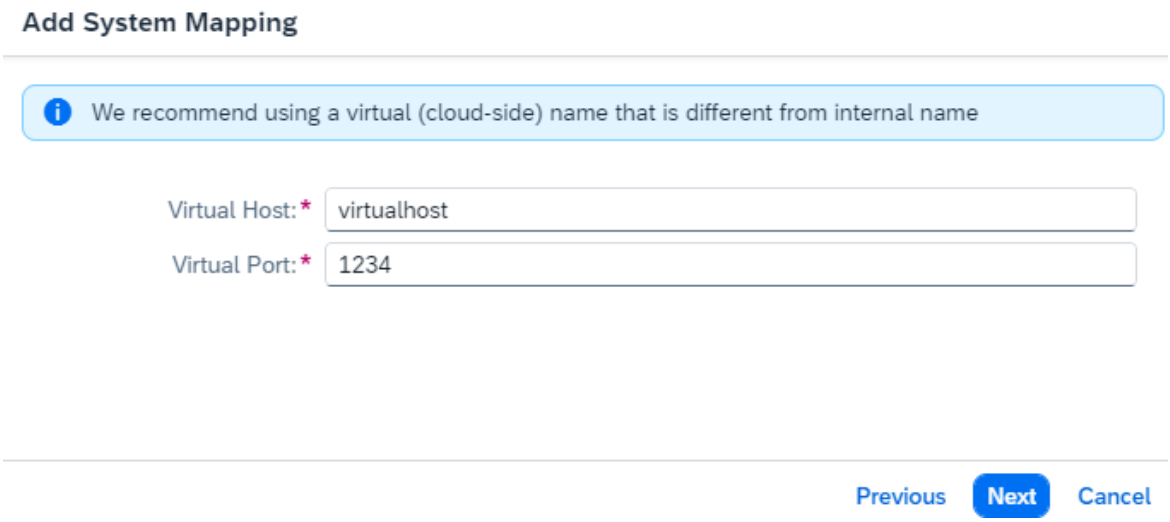
5. *Internal Host* and *Port or Port Range*: specify the host and port under which the target system can be reached within the intranet. It needs to be an existing network address that can be resolved on the intranet and has network visibility for the Cloud Connector. The Cloud Connector will try to forward the request to the network address specified by the internal host and port. That is why this address needs to be real.



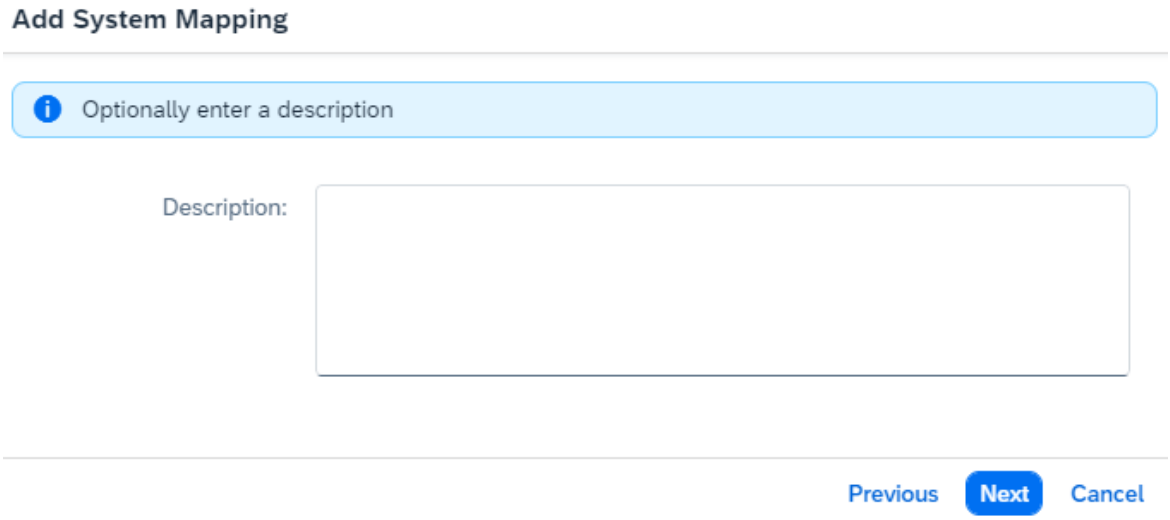
For TCP and TCP TLS, you can also specify a port range through its lower and upper limit, separated by a hyphen.



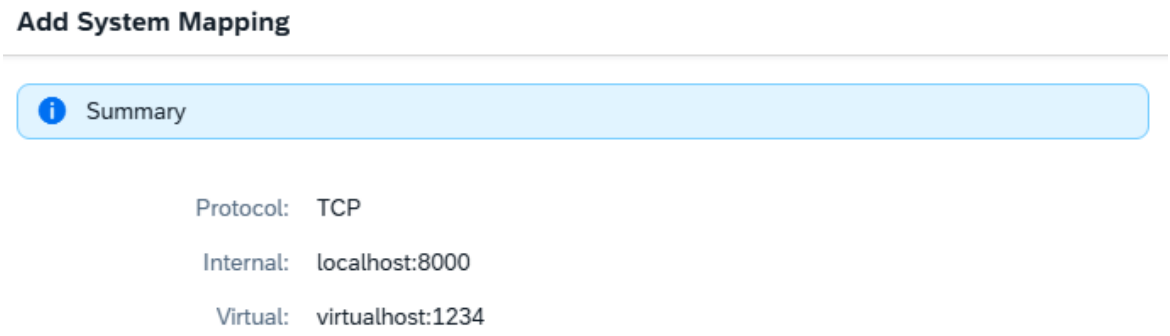
6. Enter a *Virtual Host* and *Virtual Port*. The virtual host can be an arbitrary host name that does not need to exist. The fields are pre-populated with the values of the *Internal Host* and *Port or Port Range*.



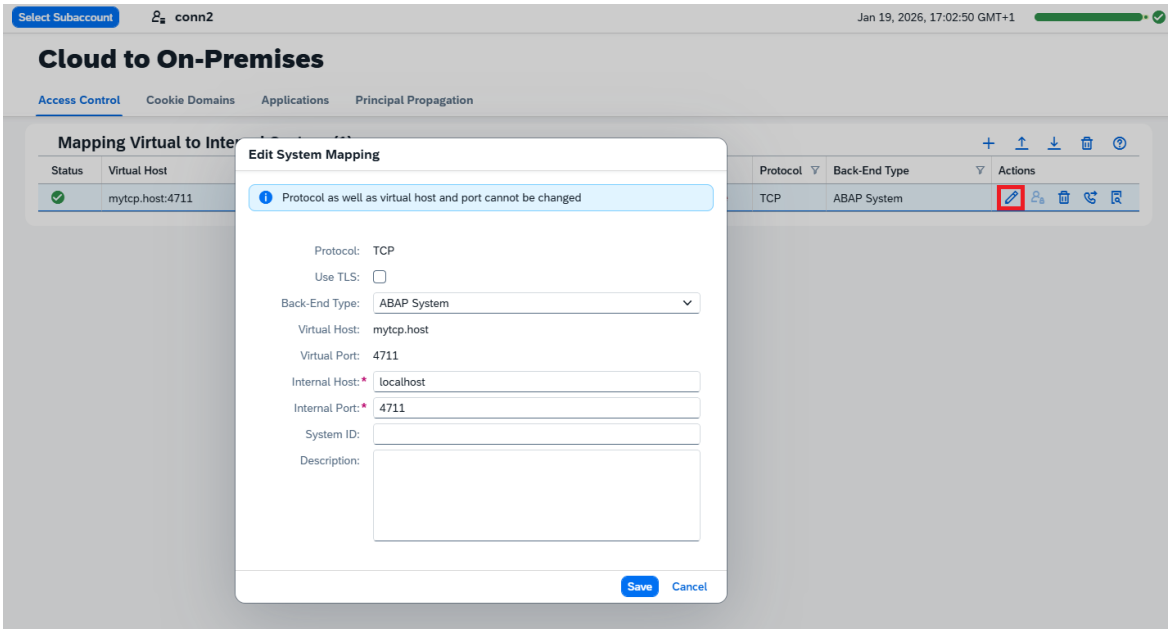
7. You can enter an optional description at this stage. The respective description will be shown as a tooltip when you press the button *Show Details* in column *Actions* of the *Mapping Virtual To Internal System* overview.



8. The summary shows information about the system to be stored. When saving the host mapping, a ping is triggered from the Cloud Connector to the internal host. This allows you to make sure the Cloud Connector can indeed access the internal system. That way, you can detect basic issues such as spelling mistakes or firewall problems between the Cloud Connector and the internal host at an early stage.
   If the ping to the internal host is successful, the state `Reachable` is shown. If it fails, a warning is shown. You can view issue details by choosing the *Details* button, or check them in the log files.
   This check also tries to perform client authentication if possible for TCP TLS, independent of the reachability of the host. With respect to that, you will also find additional information and hints by clicking the *Details* button, for example, if the system certificate which acts as a client certificate is correctly maintained and if the back end trusts it.
   You can execute such a check later at any time in the **Mapping Virtual To Internal System** overview by pressing **Check Availability of Internal Host** in column **Actions**.



9. Optional: You can later edit the system mapping (by choosing *Edit*) to make the Cloud Connector route the requests to a different back-end system. This can be useful if the system is currently down and there is a backup system that can serve these requests in the meantime. However, you cannot edit the virtual name nor port of this system mapping. If you want to use a different fictional host name in your cloud application, you must delete the mapping and create a new one. The same goes for port ranges. If a port range needs to be changed, you must delete the mapping and create it again with the desired port range.



1.4.2.4.5 Configure Accessible Resources

Configure back-end systems and resources in the Cloud Connector, to make them available for a cloud application.

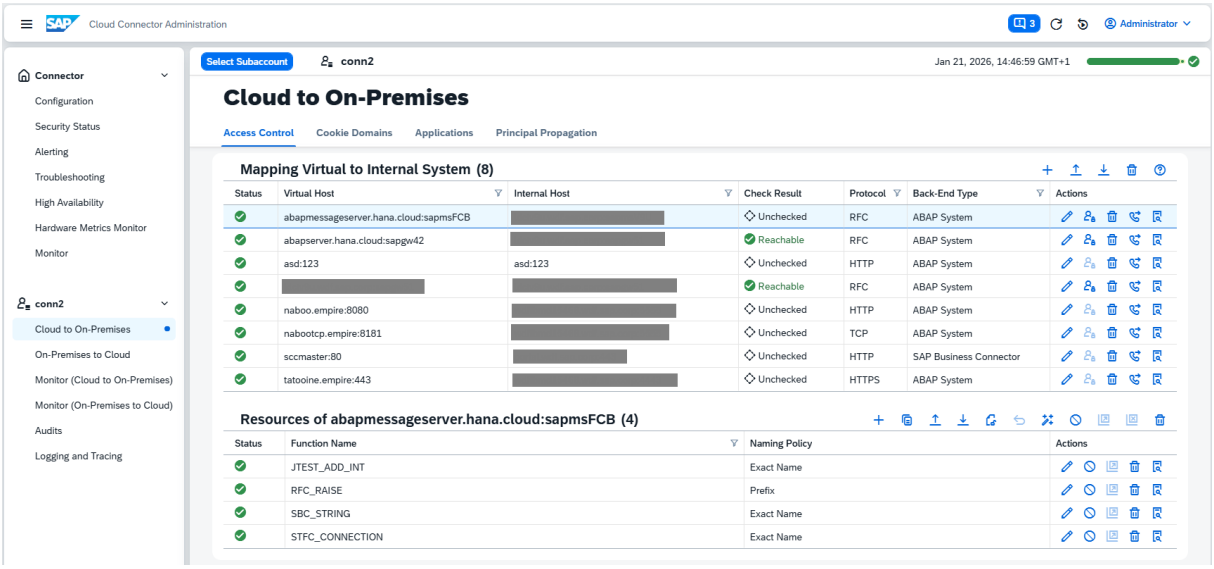
Tasks

Map Systems and Limit Access [page 1772]

Use Scenarios for Resources [page 1772]

Map Systems and Limit Access

Initially, after installing a new Cloud Connector, no network systems or resources are exposed to the cloud. You must configure each system and resource used by applications of the connected cloud subaccount. To do this, choose *Cloud To On-Premises* from your subaccount menu and go to tab *Access Control*:



The Cloud Connector supports any type of system (SAP or non-SAP system) that can be called via one of the supported protocols. For example, a convenient way to access an ABAP system from a cloud application is via SAP NetWeaver Gateway, as it allows the consumption of ABAP content via HTTP and open standards.

- For systems using HTTP communication, see: Configure Access Control (HTTP) [page 1744].
- For information on configuring RFC resources, see: Configure Access Control (RFC) [page 1752].

We recommend that you limit the access to only those back-end services and resources that are explicitly required by cloud applications. For example, instead of configuring a system and granting access to all its resources, grant access only to the specific resources that are needed by the respective cloud application. For example, define access to an HTTP service by specifying the service URL root path and allowing access to all its subpaths.

Back to Tasks [page 1771]

Use Scenarios for Resources

The Cloud Connector lets you define a set of resources as a scenario that you can export, and import into another Cloud Connector.

Scenarios are useful if you provide an application to many consumers, which invokes a large number of resources in an on-premises system. In this case, you must expose a system on your Cloud Connector that covers all required resources, which increases the risk of incorrect configuration.

Define and Export a Scenario [page 1773]

Import a Scenario [page 1773]

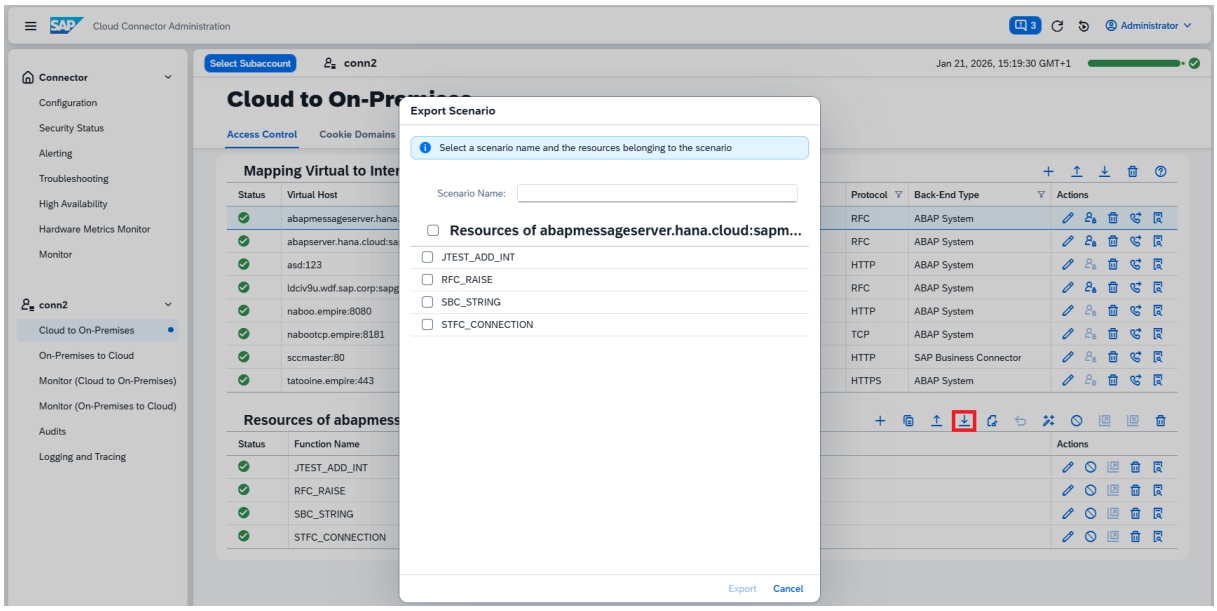
Remove a Scenario [page 1774]

Define and Export a Scenario

If you, as application owner, have implemented and tested a scenario, and configured a Cloud Connector accordingly, you can define the scenario as follows:

1. Choose the *Export Scenario* button.
2. In the dialog, select the resources that belong to the scenario.
3. In the field `<Scenario Name>`, choose a descriptive name, under which the set of resources can be identified in the consuming Cloud Connector.
4. Choose *Export* to store the scenario.

> **Note**
>
> For applications provided by SAP, default scenario definitions may be available. To verify this, check the corresponding application documentation.



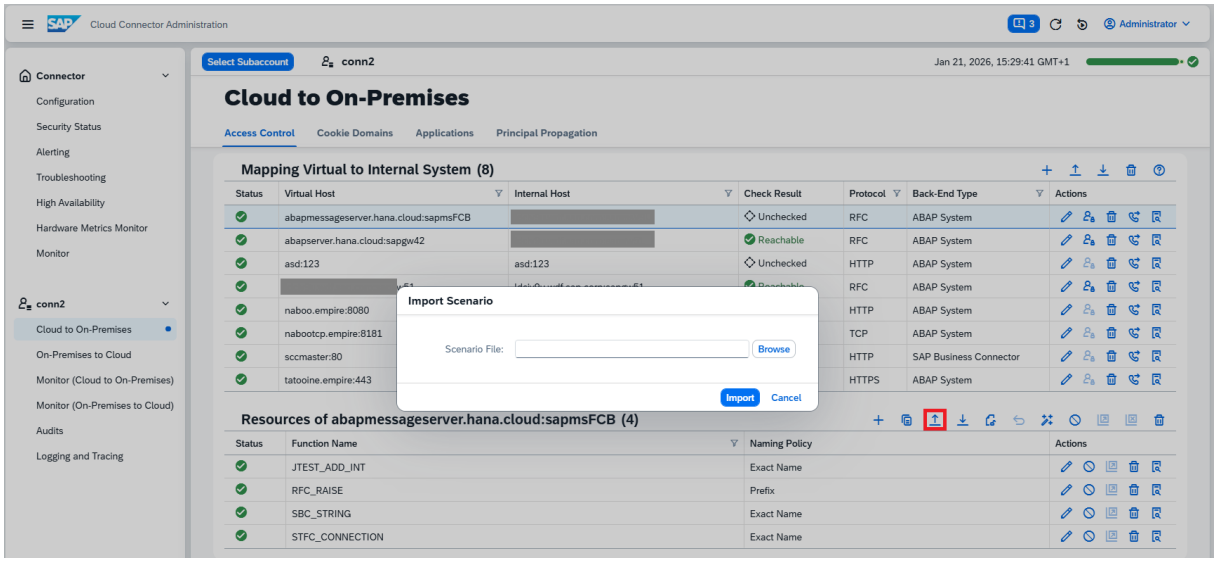
Back to Use Scenarios for Resources [page 1772]

Back to Tasks [page 1771]

Import a Scenario

As an administrator taking care for a scenario configuration in some other Cloud Connector, perform the following steps:

1. Choose the *Import Scenario* button to add all required resources to the desired access control entry.
2. In the dialog, navigate to the folder of the archive that contains the scenario definition.
3. Choose *Import*. The resources of the scenario are merged with the existing set of resources which are already available in the access control entry.



All resources belonging to a scenario are shown with an additional scenario icon in their status. When hovering over it, the assigned scenario(s) of this resource are listed.

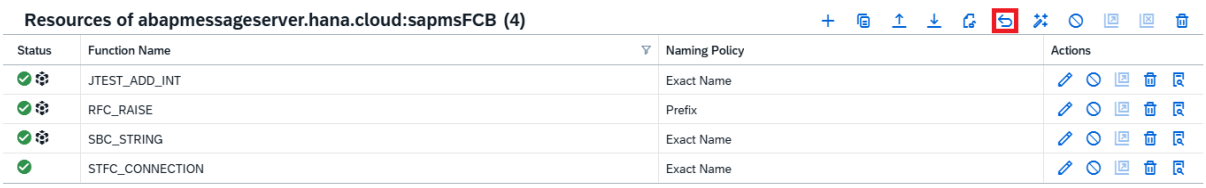
Back to Use Scenarios for Resources [page 1772]

Back to Tasks [page 1771]

Remove a Scenario

To remove a scenario:

1. Choose the *Remove Scenario* button.
2. In the dialog, choose the scenario(s) you want to remove.
3. Choose *Remove* to remove all resources associated with the chosen entry from the access control entry. Resources that existed before, or are assigned to another scenario as well, are not removed.



Back to Use Scenarios for Resources [page 1772]

Back to Tasks [page 1771]

1.4.2.5 Configuration REST APIs

You can use a set of APIs to perform the basic setup of the Cloud Connector.

Context

The Cloud Connector provides several REST APIs that let you configure a newly installed Cloud Connector. The configuration options correspond to the following steps:

- Initial Configuration [page 1670]
- Adding and Managing Subaccounts [page 1682]
- Configure Access Control [page 1740]

> **Note**
>
> All configuration APIs start with the following string: `/api/v1/configuration`.

For general information on the Cloud Connector REST APIs, see also REST APIs [page 2084].

Available APIs

- Common Properties [page 1776]
- Log and Trace Configuration [page 1778]
- High Availability Settings [page 1782]
- Proxy Settings [page 1793]
- Authentication and UI Settings [page 1796]
- Certificate Management for Back-End Communication [page 1816]
- Solution Management Configuration [page 1842]
- Backup [page 1844]
- Subaccount [page 1846]
- System Mappings [page 1862]
- System Mapping Resources [page 1867]
- Domain Mappings [page 1880]
- Subaccount Service Channels [page 1883]
- Access Control Entities [page 1907]
- Common On-Premises Connectivity Configuration [page 1913]
- Audit Log Configuration [page 1917]
- Alerting Configuration [page 1920]
- E-Mail Configuration [page 1923]

Related Information

Examples [page 1932]

1.4.2.5.1 Common Properties

Read and edit the Cloud Connector's common properties via API.

Get Common Properties

URI	`/api/v1/configuration/connector`
Method	*GET*
Request	
Response	`{ha, description}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

- **Response Properties**:
  - `ha`: Cloud Connector instance assigned to a high-availability role. Its value is either the string *master* or *shadow*.
  - `description`: Description of the Cloud Connector.

Get Version

> **Note**
>
> Available as of version 2.14.0.

URI	`/api/v1/connector/version`

Method	*GET*
Request	
Response	`{version}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

- **Response Properties**:
  `version`: A string; the Cloud Connector version.

Set Description (Master Only)

URI	`/api/v1/configuration/connector`
Method	*PUT*
Request	`{description}`
Response	`{ha, description}`
Errors	`INVALID_REQUEST`
Roles	Administrator, Associate Administrator

- **Request Properties**:
  `description`: A string; use an empty string to remove the description.
- **Response Properties**:
  - `ha`: Cloud Connector instance assigned to a high-availability role. Its value is either the string *master* or *shadow*.
  - `description`: Description of the Cloud Connector.
- **Errors**:
  `INVALID_REQUEST` ((400): The value of `description` is not a JSON string.

> **Note**
>
> `null` is not a JSON string.

Get Available Cipher Suites

URI	`/api/v1/configuration/availableCipherSuites`
Method	*GET*
Request	
Response	`[cipherSuite, ...]`
Errors	
Roles	Administrator, Subaccount Administrator, Display, Support, Monitoring

- **Response**:
  - list of:
    - `cipherSuite`: cipher suite name like TLS_AES_128_GCM_SHA256 (a string)

1.4.2.5.2 Log and Trace Configuration

Read and edit the Cloud Connector's log and trace settings via API.

The Cloud Connector provides subaccount-independent logs, also known as *cross-account settings*, and log settings related to subaccounts.

The location of API for cross-account settings is `/api/v1/configuration/connector/logAndTrace`. Subaccount-related settings can be done here: `/api/v1/configuration/connector/logAndTrace/subaccounts/<regionHost>/<subaccount>`.

Configuration changes are applied to the corresponding locations. The API method is PATCH, allowing you to set only relevant values. PUT and DELETE methods are not implemented.

- `Cloud Connector Loggers` are logged with output to `scc_core.trc`.
- `other Loggers` are logged with output to `scc_core.trc`.

Get Cross-Account Trace Configuration

URI	`/api/v1/configuration/connector/logAndTrace`
Method	*GET*
Request	
Response	`{sccLogLevel, otherLogLevel, tlsTrace, isTlsTraceActive, cpicTraceLevel, cleanup, rolloverThreshold, logAndTracePath}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `sccLogLevel`: log level for Cloud Connector log locations, one of OFF, ERROR, WARN, INFO, DEBUG, TRACE, ALL.
- `otherLogLevel`: log level for all other than Cloud Connector log locations, one of OFF, ERROR, WARN, INFO, DEBUG, TRACE, ALL.
- `tlsTrace`: Indicates if TLS trace is stored in configuration (true or false). If different from `isTlsTraceActive`, then a restart is required to activate the setting.

  > ### Note
  >
  > The `tlsTrace` setting is stored in the configuration file `props.ini`. When restoring a Cloud Connector installation from a backup, and the `tlsTrace` (or any other) setting in the backup `props.ini` file differs from the setting in the `props.ini` file of the original installation, the backup `props.ini` file will be added to the installation directory as `props.ini.restored` (for security reasons). Therefore, unless manually changed, the original `tlsTrace` setting will be kept after restoring from a backup.

- `isTlsTraceActive`: TLS trace currently active in runtime. If it differs from `tlsTrace`, then a restart is required.
- `cpicTraceLevel`: trace level of CPIC layer (0 - off, 1 - general information, 2 - detailed trace, 3 - verbose level)
- `cleanup`: number of days to keep the log files (0 means do not clean up the old log files)
- `rolloverThreshold`: size in megabytes for the scc_core.trc files. 20 files will be kept before the oldest is overwritten.
- `logAndTracePath`: path for log and trace files

> ### Note
>
> `otherLogLevel` should only be changed if requested by SAP support. The default value is `INFO`.

Change Cross-Account Configuration

URI	`/api/v1/configuration/connector/logAndTrace`
Method	*PATCH*
Request	`{sccLogLevel , otherLogLevel, tlsTrace, cpicTraceLevel, cleanup, rolloverThreshold }`
Response	
Errors	
Roles	Administrator, Support

Request Properties:

- `sccLogLevel`: log level for Cloud Connector log locations, one of OFF, ERROR, WARN, INFO, DEBUG, TRACE, ALL.
- `otherLogLevel`: log level for all other than Cloud Connector log locations, one of OFF, ERROR, WARN, INFO, DEBUG, TRACE, ALL.
- `tlsTrace`: Indicates if TLS trace is stored in configuration (true or false). When changing its value, a restart is required to activate the setting.

  > ### Note
  >
  > The `tlsTrace` setting is stored in the configuration file `props.ini`. When restoring a Cloud Connector installation from a backup, and the `tlsTrace` (or any other) setting in the backup `props.ini` file differs from the setting in the `props.ini` file of the original installation, the backup `props.ini` file will be added to the installation directory as `props.ini.restored` (for security reasons). Therefore, unless manually changed, the original `tlsTrace` setting will be kept after restoring from a backup.

- `cpicTraceLevel`: trace level of CPIC layer (0 - off, 1 - general information, 2 - detailed trace, 3 - verbose level)
- `cleanup`: number of days to keep the log files (0 means do not clean up the old log files)
- `rolloverThreshold`: size in megabytes for the scc_core.trc files. The unit MB must be included, i.e. a valid value is '75 MB' (with or without space), for instance. 20 files will be kept before the oldest gets overwritten.

> ### Note
>
> The request may contain only the properties to be changed. All other settings keep their values.

Get Subaccount-Related Trace Configuration

URI	`/api/v1/configuration/connector/logAndTrace/subaccounts/<regionHost>/<subaccount>`
Method	*GET*
Request	
Response	`{trafficTrace, payloadSncTrace}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `trafficTrace` : the traffic (payload) trace for the specified subaccount (true or false).
- `payloadSncTrace`: the SNC payload for the specified subaccount for Cloud ABAP communication (true or false).

Change Subaccount-Related Configuration

URI	`/api/v1/configuration/connector/logAndTrace/subaccounts/<regionHost>/<subaccount>`
Method	*PATCH*
Request	
Response	`{trafficTrace, payloadSncTrace}`
Errors	
Roles	Administrator, Support

Request Properties:

- `trafficTrace`: activates or deactivates the traffic (payload) trace for the specified subaccount (true or false).

- `payloadSncTrace`: activates or deactivates the snc payload for the specified subaccount for Cloud ABAP communication (true or false).

1.4.2.5.3 High Availability Settings

Read and edit the high availability settings of a Cloud Connector instance via API.

When installing a Cloud Connector instance, you usually define its high availability role (master or shadow instance) during initial configuration, see Change your Password and Choose Installation Type [page 1672].

If the high availability role was not defined before, you can set the master or shadow role via this API.

If a shadow instance is connected to the master, this API also lets you switch the roles: the master instance requests the shadow instance to take over the master role, and then takes the shadow role itself.

> ⓘ Note
>
> Editing the high availability settings is only allowed on the master instance, and supports only **shadow** as input.

Get High Availability Settings

URI	`/api/v1/configuration/connector/haRole`
Method	*GET*
Request	
Response	"<HA role>"
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Example

```
curl -i -k -u Administrator:<password> https://localhost:8443/api/v1/
configuration/connector/haRole
```

Set High Availability Settings

Use this API if you want to set the role of a fresh installation (no role assigned yet).

As of version 2.12.0, this API also allows to switch the roles if a shadow instance is connected to the master. In this case, the API is only allowed on the master instance and supports only the value **shadow** as input. The master instance requests the shadow instance to take over the master role and then assumes the shadow role itself.

URI	`/api/v1/configuration/connector/haRole`
Method	*POST*
Request	"master" or "shadow"
Response	
Errors	ILLEGAL_STATE, INVALID_REQUEST
Roles	Administrator

Errors:

- `INVALID_REQUEST` (400): If a high-availability role other than "master" or "shadow" is supplied.
- `ILLEGAL_STATE` (409): If changing the high-availability role is not possible; changing the role is only possible if no role has been assigned, or if the current role is master and a shadow system is connected.

Example

```
curl -i -k -H 'Content-Type: application/json' -d '"shadow"'  -u
Administrator:<password> -X POST https://localhost:8443/api/v1/configuration/
connector/haRole
```

Related Information

Master Instance Configuration [page 1784]
Shadow Instance Configuration [page 1788]

1.4.2.5.3.1 Master Instance Configuration

Read and edit the high availability settings for a Cloud Connector master instance via API.

> **Note**
>
> The APIs below are available as of Cloud Connector version 2.13.0.

> **Restriction**
>
> These APIs are only permitted on a Cloud Connector master instance. The shadow instance rejects the requests with error code *400 – Invalid Request*.

Get Configuration

URI	`/api/v1/configuration/connector/ha/master/config`
Method	*GET*
Request	
Response	`{haEnabled, haPort, allowedShadowHost}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `haEnabled`: Boolean value that indicates whether or not a shadow system is allowed to connect.
- `haPort`: Port on which the shadow instance can connect (restart required after change).
- `allowedShadowHost`: Name of the shadow host (a string) that is allowed to connect; an empty string signifies that any host is allowed to connect as shadow.

> **Note**
>
> `haPort` is an optional field in the HA shadow configuration, available as of version 2.17.0. Default value for the `haPort` parameter is the Cloud Connector's UI port.

Example

```
curl -i -k -u <user>:<password> -X GET https://host>:<port>/api/v1/configuration/
connector/ha/master/config
```

Set Configuration

URI	`/api/v1/configuration/connector/ha/master/config`
Method	*PUT*
Request	`{haEnabled, haPort, allowedShadowHost}`
Response	204 on success
Errors	INVALID_REQUEST
Roles	Administrator

Response Properties:

- `haEnabled`: Boolean value that indicates whether or not a shadow system is allowed to connect.
- `haPort`: Port on which the shadow instance can connect (restart required after change).
- `allowedShadowHost`: Name of the shadow host (a string) that is allowed to connect. An empty string means that any host is allowed to connect as shadow.

Errors:

- `INVALID_REQUEST` (400): if the name of the shadow host is not a valid host name

> **Note**
>
> `haPort` is an optional field in the HA shadow configuration, available as of version 2.17.0. Default value for the `haPort` parameter is the Cloud Connector's UI port.

Example

```
curl -i -k -u <user>:<password> -X PUT -H 'Content-Type: application/json'
-d '{"haEnabled":true}' https://<host>:<port>/api/v1/configuration/connector/ha/
master/config
```

Get State

URI	`/api/v1/configuration/connector/ha/master/state`
Method	*GET*
Request	
Response	`{state, shadowHost}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `state`: One of the following strings: ALONE, BINDING, CONNECTED or BROKEN.
- `shadowHost`: Connected shadow host (a string)

Example

```
curl -i -k -u <user>:<password> -X GET https://<host>:<port>/api/v1/
configuration/connector/ha/master/state
```

Set State

URI	`/api/v1/configuration/connector/ha/master/state`
Method	*POST*
Request	
Response	`{op}`
Errors	ILLEGAL_STATE, INVALID_REQUEST
Roles	Administrator

Request Properties:

The value of property `op` is one of the following strings:

- `SWITCH`: Switch roles with shadow
- `FORCE_SWITCH`: Take over the shadow role, even if shadow instance does not respond.

Errors:

- `INVALID_REQUEST` (400): Value of property `op` is neither SWITCH nor FORCE_SWITCH.
- `ILLEGAL_STATE` (409): Master system is in a state that does not permit the requested operation.

Example

```
curl -i -k -u <user>:<password> -X POST -H 'Content-Type: application/json' -d
'{"op":"SWITCH"}' https://<host>:<port>/api/v1/configuration/connector/ha/master/
state
```

Reset

A successful call to this API restores default values for all settings related to high availability on the master side.

> ⚠ Caution
>
> Do not perform this call if the shadow is connected to a master.

URI	`/api/v1/configuration/connector/ha/master/state`
Method	*DELETE*
Request	
Response	204 on success
Errors	ILLEGAL_STATE
Roles	Administrator

Errors:

- `ILLEGAL_STATE` (409): A shadow instance is connected to the master.

Example

```
curl -i -k -u <user>:<password> -X DELETE https://<host>:<port>/api/v1/
configuration/connector/ha/master/state
```

1.4.2.5.3.2 Shadow Instance Configuration

Read and edit the configuration settings for a Cloud Connector shadow instance via API (available as of Cloud Connector version 2.12.0, or, where mentioned, as of version 2.13.0).

> ⓘ Note
>
> The APIs below are only permitted on a Cloud Connector shadow instance. The master instance will reject the requests with error code `403 – FORBIDDEN_REQUEST`.

Get Configuration

URI	`/api/v1/configuration/connector/ha/shadow/config`
Method	*GET*
Request	
Response	`{masterHost, masterPort, ownHost, haPort, checkIntervalInSeconds, takeoverDelayInSeconds, connectTimeoutInMillis, requestTimeoutInMillis}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `masterHost`: Host name of the master instance (string)
- `masterPort`: Port of the master instance (string or number)
- `ownHost`: Host name of the shadow instance (string)
- `haPort`: Own port for HA communication (restart required after change).
- `checkIntervalInSeconds`: Time between two health checks against the master instance (number)

- `takeoverDelayInSeconds`: The time a master instance may stay unreachable before the shadow instance takes over (number)
- `connectTimeoutInMillis`: Timeout for connection attempts between shadow and master instance (number)
- `requestTimeoutInMillis`: Timeout for requests between shadow and master instance (number)

> ### Note
>
> This API may take some time to fetch the own hosts from the environment.

> ### Note
>
> `haPort` is a new field in the HA shadow configuration, available as of version 2.17.0. Default value for the `haPort` parameter is the Cloud Connector's UI port.

Example

```
curl -i -k -u <user>:<password> -X GET https://<host>:<port>/api/v1/
configuration/connector/ha/shadow/config
```

Set Configuration

URI	`/api/v1/configuration/connector/ha/shadow/config`
Method	*PUT*
Request	`{masterPort, masterHost, ownHost, haPort, checkIntervalInSeconds, takeoverDelayInSeconds, connectTimeoutInMillis, requestTimeoutInMillis}`
Response	`{masterPort, masterHost, ownHost, haPort, checkIntervalInSeconds, takeoverDelayInSeconds, connectTimeoutInMillis, requestTimeoutInMillis}`
Errors	
Roles	Administrator

Request Properties:

- `masterHost`: Host name of the master instance (string)
- `masterPort`: Port of the master instance (string or number)
- `ownHost`: Host name of the shadow instance (string)
- `haPort`: Own port for HA communication (restart required after change).
- `checkIntervalInSeconds`: Time between two health checks against the master instance (number)
- `takeoverDelayInSeconds`: The time a master instance may stay unreachable before the shadow instance takes over (number)
- `connectTimeoutInMillis`: Timeout for connection attempts between shadow and master instance (number)
- `requestTimeoutInMillis`: Timeout for requests between shadow and master instance (number)

Response Properties:

See *Request Properties*.

> **Note**
>
> `haPort` is an optional field in the HA shadow configuration, available as of version 2.17.0. Default value for the `haPort` parameter is the Cloud Connector's UI port.

Example

```
curl -i -k -u <user>:<password> -X PUT https://<host>:<port>/api/v1/
configuration/connector/ha/shadow/config
  -H 'Content-Type: application/json' -d '{"masterHost":"localhost",
"masterPort":"8443", "ownHost":"localhost",
      "checkIntervalInSeconds":30, "takeoverDelayInSeconds":10,
"connectTimeoutInMillis":1000,
      "requestTimeoutInMillis":12000}'
```

Get State

> **Note**
>
> Available as of version 2.13.0.

URI	`/api/v1/configuration/connector/ha/shadow/state`
Method	*GET*

Request	
Response	`{state, ownHosts, stateMessage, masterVersions}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `state`: Possible string values are: INITIAL, DISCONNECTED, DISCONNECTING, HANDSHAKE, INITSYNC, READY, or LOST.
- `ownHosts`: List of alternative host names for the shadow instance.
- `stateMessage`: Message providing details on the current state. This property may not always be present. Typically, this property is available if an error occurred (for example, a failed attempt to connect to the master instance).
- `masterVersions`: Overview of relevant component versions of the master system, including a flag (*property ok*) that indicates whether or not there are incompatibility issues because of differing master and shadow versions.

> **Note**
>
> This property is only available if the shadow instance is connected to the master instance, or if there has been a successful connection to the master system at some point in the past.

Example

```
curl -i -k -u <user>:<password> -X GET https://<host>:<port>/api/v1/
configuration/connector/ha/shadow/state
```

Change State

URI	`/api/v1/configuration/connector/ha/shadow/state`
Method	*POST*
Request	

Response	`{op, user, password}`
Errors	INVALID_REQUEST, ILLEGAL_STATE, RUNTIME_FAILURE
Roles	Administrator

Request Properties:

- `op`: String value representing the state change operation. Possible values are CONNECT or DISCONNECT.
- `user`: User for logon to the master instance
- `password`: Password for logon to the master instance

Errors:

- `INVALID_REQUEST` (400): Invalid or missing property values were supplied; this includes wrong user or password
- `ILLEGAL_STATE` (409): The requested operation cannot be executed given the current state of master and shadow instance. This typically means the master instance does not allow high availability.
- `RUNTIME_FAILURE` (500): communication error between shadow and master, e.g. handshake failure or master does not respond.

> **Note**
>
> The logon credentials are used for initial logon to master instance only. If a shadow instance is disconnected from its master instance, it will reconnect to the (same) master instance using a certificate. Hence, user and password can be omitted when reconnecting.

Example

```
curl -i -k -u <user>:<password> -X POST https://<host>:<port>/api/v1/
configuration/connector/ha/shadow/state
           -H 'Content-Type: application/json' -d '{"op":"CONNECT",
"user":"<user on master>", "password":"<password>"}'
curl -i -k -u <user>:<password> -X POST https://<host>:<port>/api/v1/
configuration/connector/ha/shadow/state
          -H 'Content-Type: application/json' -d '{"op":"DISCONNECT"}'
```

Reset

> **Note**
>
> Available as of version 2.13.0.

A successful call to this API deletes master host and port, and restores default values for all other settings related to a connection to the master.

> ⚠ Caution
>
> Do not perform this call if the shadow is connected to a master.

URI	`/api/v1/configuration/connector/ha/shadow/state`
Method	*DELETE*
Request	
Response	
Errors	ILLEGAL_STATE
Roles	Administrator

Errors:

- `ILLEGAL_STATE` (409): the shadow is currently connected to a master.

Example

```
curl -i -k -u <user>:<password> -X DELETE https://<host>:<port>/api/v1/
configuration/connector/ha/shadow/state
```

1.4.2.5.4 Proxy Settings

Read and edit the Cloud Connector's proxy settings via API.

Get Proxy Settings

URI	`/api/v1/configuration/connector/proxy`
Method	*GET*

Request	
Response	`{host, port, user}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `host`: the name of the proxy host (a string)
- `port`: the port of the proxy host (a string)
- `user`: the user name (a string)

Sample Code

```
curl -ik -u Administrator:<password> https://localhost:8443/api/v1/
configuration/connector/proxy
```

Set Proxy Settings (Master Only)

URI	`/api/v1/configuration/connector/proxy`
Method	*PUT*
Request	`{host, port, user, password}`
Response	
Errors	`INVALID_REQUEST`, `FORBIDDEN_REQUEST`
Roles	Administrator

Request Properties:

- `host`: the name of the proxy host (a string)
- `port`: the port of the proxy host (a string)
- `user`: the user name (a string)
- `password`: the password (a string - optional)

Errors:

- `INVALID_REQUEST` (400): invalid values were supplied, or mandatory values are missing.

- `FORBIDDEN_REQUEST` (403): the target of the call is a shadow instance.

The following example sets empty `user` and `password`.

Sample Code

```
curl -ik -u Administrator:<password> https://localhost:8443/api/v1/
configuration/connector/proxy -X PUT -H 'Content-Type: application/json' -d
'{"host":"proxy", "port":"8080"}'
```

This request removes the proxy configuration.

Sample Code

```
curl -ik -u Administrator:<password> https://localhost:8443/api/v1/
configuration/connector/proxy -X PUT -H 'Content-Type: application/json' -d
'{}'
```

Remove Proxy Settings (Master Only)

URI	`/api/v1/configuration/connector/proxy`
Method	*DELETE*
Request	
Response	204 on success
Errors	`FORBIDDEN_REQUEST`
Roles	Administrator

Errors:

- `FORBIDDEN_REQUEST` (403): the target of the call is a shadow instance.

Sample Code

```
curl -ik -u Administrator:<password> https://localhost:8443/api/v1/
configuration/connector/proxy -X DELETE
```

1.4.2.5.5 Authentication and UI Settings

Read and edit the Cloud Connector's authentication and UI settings via API.

Get Authentication Settings

URI	`/api/v1/configuration/connector/authentication`
Method	*GET*
Request	
Response	`{type, configuration}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `type`: The authentication type, which is one of the following strings: *basic* or *ldap*.
- `configuration`: The configuration of the active LDAP authentication. This property is only available if `type` is ldap. Its value is an object with properties that provide details on LDAP configuration.

Example

```
curl -i -k -H 'Accept:application/json'
-u Administrator:<password> -X GET https://<scchost>:8443/api/v1/configuration/
connector/authentication
```

Change Basic Authentication User

URI	`/api/v1/configuration/connector/authentication/basic`
Method	*PUT*

Request	`{password, user}`
Response	204 on success
Errors	INVALID_REQUEST
Roles	Administrator

Request Properties:

- `password`: The current password (a string)
- `user`: The new user name (a string), overwriting the current user name.

Errors:

- `INVALID_REQUEST` (400): Current password is wrong.

Change Basic Authentication Password

URI	`/api/v1/configuration/connector/authentication/basic`
Method	*PUT*
Request	`{oldPassword, newPassword}`
Response	204 on success
Errors	INVALID_REQUEST
Roles	Administrator

Request Properties:

- `oldPassword`: The current password, about to be changed (a string)
- `newPassword`: The new password (a string)

Errors:

- `INVALID_REQUEST` (400): Passwords are the same or current password is wrong.

Example

```
curl -i -k -H 'Content-Type:application/json' -d '{"oldPassword"="manage",
"newPassword"="test"}'
-u Administrator:manage -X PUT https://localhost:8443/api/v1/configuration/
connector/authentication/basic
```

LDAP Authentication Configuration

The *LDAP Configuration* APIs use the following objects to describe the LDAP server configuration:

`{id, config, hosts, user, password, customAdminRole , customDisplayRole, customMonitoringRole , customSupportRole}`

- `id`: identifier set by cloud connector runtime.
- `config`: LDAP query allowing narrow the use and role
- `hosts`: list of one or two host - primary and alternate LDAP host
  - `host`: LDAP host name
  - `port`: LDAP host port
  - `isSecure`: boolean indicating use or not secure (TLS) connection
- `user`: user to run LDAP queries (optional)
- `password`: password for the user, only required if user is set
- `customAdminRole`: custom role name for users having administrator permissions (optional)
- `customSubadminRole`: custom role name for users having subaccount administrator permissions (optional)
- `customDisplayRole`: custom role name for users having display only permissions (optional)
- `customMonitoringRole`: custom role name for user having monitoring permissions (optional)
- `customSupportRole`: custom role name for user having support permissions (optional)

> ⓘ Note
>
> The REST API client should not set the ID when creating new LDAP server configurations. The ID will be generated by the Cloud Connector runtime.

Change LDAP Authentication

> ⚠ Caution
>
> The Cloud Connector will restart if the request was successful and the configuration was changed. There is no test that confirms login will work afterwards. If you run into problems you can revert to basic authentication by executing the script `useFileUserStore` located in the root directory of your Cloud Connector installation.

URI	`/api/v1/configuration/connector/authentication/ldap`
Method	*PUT*
Request	`{enable, configuration}`
Response	204 on success
Errors	INVALID_REQUEST, INVALID_CONFIGURATION
Roles	Administrator

Request Properties:

- `enable`: Boolean flag that indicates whether or not to employ LDAP authentication.
- `configuration`: The LDAP configuration, a JSON object with the properties `{config, hosts, user, password, customAdminRole , customDisplayRole, customMonitoringRole , customSupportRole}`.
  - Property `hosts` is an array. Each element of the array defines a host, again specified through a JSON object, with the properties `{host, port, isSecure}`, accepting string, string (or number), and Boolean values, respectively.
  - All properties of the top-level object except `hosts` accept string values.
  - Properties `config` and `hosts` are mandatory. The array of hosts needs to have at least one element.
  - All other properties are optional.

Errors:

- `INVALID_REQUEST` (400): Configuration is invalid.
- `INVALID_CONFIGURATION` (409): LDAP server is not accessible (does not respond).

> **Note**
>
> In both error cases nothing is stored, and LDAP authentication is disabled.

Example

```
curl -i -k -H 'Content-Type: application/json' -u Administrator:<password>
-X PUT https://localhost:8443/api/v1/configuration/connector/authentication/
ldap -d '{"enable":true, "configuration":{"hosts":[{"host":"ldaphost",
"port":"10389", "isSecure":false}], "config":"roleBase=\"ou=groups,dc=scc\"
roleName=\"cn\" roleSearch=\"(uniqueMember={0})\"
userBase=\"ou=users,dc=scc\" userSearch=\"(uid={0})\"", "user":"ldapadmin",
"password":"<ldapadminpassword>"}}'
```

Authentication API v2

As of version 2.18.0, the Cloud Connector REST API supports configurations with multiple LDAP servers. To enable that, the data structures contain an array of LDAP servers instead of a single LDAP server configuration as in API v1.

Get Authentication Configuration v2

> **Note**
>
> Available as of version 2.18.0.

URI	`/api/v2/configuration/connector/authentication`
Method	*GET*
Request	
Response	`{type, orderedLdapConfigurations}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `type`: authentication type, which is one of the following strings: basic or ldap
- `orderedLdapConfigurations`: contains the list of LDAP server configurations in order. This value is only set if type is ldap.

Example

```
curl -i -k -H 'Accept:application/json'
-u Administrator:<password> -X GET https://<scchost>:8443/api/v2/configuration/
connector/authentication
```

Set LDAP Configuration v2

> **Note**
>
> Available as of version 2.18.0.

> **Caution**
>
> The Cloud Connector will restart if the request was successful and the configuration was changed. There is no test that confirms login will work afterwards. If you run into problems you can revert to basic authentication by executing the script `useFileUserStore` located in the root directory of your Cloud Connector installation.

URI	`/api/v2/configuration/connector/authentication/ldap`
Method	*PUT*
Request	`{enabled, orderedLdapConfigurations}`
Response	204 on success
Errors	INVALID_REQUEST, INVALID_CONFIGURATION
Roles	Administrator

Request Properties:

- `enabled`: Boolean flag that indicates whether or not to employ LDAP authentication
- `orderedLdapConfigurations`: contains the list of LDAP server configurations in order. This value is only set if type is ldap.

Errors:

- INVALID_REQUEST (400): configuration is invalid
- INVALID_CONFIGURATION (409): LDAP server is not accessible (does not respond)

> **Note**
>
> In both error cases nothing is stored, and LDAP authentication is disabled.

Example

```
curl -i -k -H 'Content-Type: application/json' -u Administrator:<password>
-X PUT https://localhost:8443/api/v2/configuration/connector/authentication/ldap
-d '{"enabled":true, "orderedLdapConfigurations":[{"hosts":[{"host":"ldaphost",
"port":"10389", "isSecure":false}], "config":"roleBase=\"ou=groups,dc=scc\"
roleName=\"cn\" roleSearch=\"(uniqueMember={0})\"
userBase=\"ou=users,dc=scc\" userSearch=\"(uid={0})\"", "user":"ldapadmin",
"password":"<ldapadminpassword>"}]}'
```

Get LDAP Configuration

> **Note**
>
> Available as of version 2.18.0.

URI	`/api/v2/configuration/connector/authentication/ldap`
Method	*GET*
Request	
Response	`{enabled, orderedLdapConfigurations}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `enabled`: if LDAP is active
- `orderedLdapConfigurations`: contains the list of LDAP server configurations in order.

Example

```
curl -i -k -H 'Accept:application/json'
-u Administrator:<password> -X GET https://<scchost>:8443/api/v2/configuration/
connector/authentication/ldap
```

Add LDAP Server Configuration

> **Note**
>
> Available as of version 2.18.0.

> **Note**
>
> No restart will be performed on the Cloud Connector. To activate the configuration, a manual restart will be required.

URI	`/api/v2/configuration/connector/authentication/ldap/servers`
Method	*POST*
Request	`{config, hosts, user, password, customAdminRole, customSubadminRole, customDisplayRole, customMonitoringRole , customSupportRole}` where hosts is array of `{host, port, isSecure}`
Response	201
Errors	INVALID_REQUEST, INVALID_CONFIGURATION
Roles	Administrator

Response Properties:

- `config`: LDAP query allowing narrow the use and role
- `hosts`: list of one or two host - primary and alternate LDAP host
  - `host`: LDAP host name
  - `port`: LDAP host port
  - `isSecure`: boolean indicating use or not secure (TLS) connection
- `user`: user to run LDAP queries (optional)
- `password`: password for the user, only required if user is set
- `customAdminRole`: custom role name for users having administrator permissions (optional)
- `customSubadminRole`: custom role name for users having subaccount administrator permissions (optional)
- `customDisplayRole`: custom role name for users having display only permissions (optional)
- `customMonitoringRole`: custom role name for user having monitoring permissions (optional)
- `customSupportRole`: custom role name for user having support permissions (optional)

Errors:

- INVALID_REQUEST (400): configuration is invalid
- INVALID_CONFIGURATION (409): LDAP server is not accessible (does not respond)

> **Note**
>
> In both error cases nothing is stored, and LDAP configuration remains unchanged.

Example

```
curl -i -k -H 'Accept:application/json'
-u Administrator:<password> -X POST https://<scchost>:8443/api/v2/configuration/
connector/authentication/ldap/servers
```

Get LDAP Server Configuration by ID

> **Note**
>
> Available as of version 2.18.0.

URI	`/api/v2/configuration/connector/authentication/ldap/servers/<id>`
Method	*GET*
Request	
Response	`LDAP server configuration`
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Example

```
curl -i -k -H 'Accept:application/json' -u Administrator:<password>
-X GET https://<scchost>:8443/api/v2/configuration/connector/authentication/ldap/
servers/0
```

Change LDAP Server Configuration by ID

> **Note**
>
> Available as of version 2.18.0.

> **Note**
>
> No restart will be performed on the Cloud Connector. To activate the configuration, a manual restart will be required.

URI	`/api/v2/configuration/connector/authentication/ldap/servers/<id>`
Method	*PUT*
Request	`{config, hosts, user, password, customAdminRole, customSubadminRole, customDisplayRole, customMonitoringRole, customSupportRole}`
Response	204
Errors	NOT_FOUND, INVALID_REQUEST, INVALID_CONFIGURATION
Roles	Administrator

Errors:

- NOT_FOUND(404): specified ID is not available
- INVALID_REQUEST (400): configuration is invalid
- INVALID_CONFIGURATION (409): LDAP server is not accessible (does not respond)

> **Note**
>
> In error cases nothing is stored, and LDAP configuration remains unchanged.

Example

```
curl -i -k -H 'Accept:application/json' -u Administrator:<password>
-X PUT https://<scchost>:8443/api/v2/configuration/connector/authentication/ldap/
servers/0
-d '{"id":"0", "hosts":[{"host":"ldaphost", "port":"10389",
"isSecure":false}], "config":"roleBase=\"ou=groups,dc=scc\" roleName=\"cn\"
```

```
roleSearch=\"(uniqueMember={0})\" userBase=\"ou=users,dc=scc\"
userSearch=\"(uid={0})\"", "user":"ldapadmin", "password":"<ldapadminpassword>"}'
```

Delete LDAP Server Configuration by ID

> **Note**
>
> Available as of version 2.18.0.

> **Note**
>
> No restart will be performed on the Cloud Connector. To activate the configuration, a manual restart will be required.

URI	`/api/v2/configuration/connector/authentication/ldap/servers/<id>`
Method	*DELETE*
Request	
Response	204
Errors	NOT_FOUND
Roles	Administrator

> **Note**
>
> Removal of the last remaining LDAP server is not allowed.

Example

```
curl -i -k -H 'Accept:application/json'
-u Administrator:<password> -X DELETE https://<scchost>:8443/api/v2/
configuration/connector/authentication/ldap/servers/1
```

Move LDAP Server Configuration by ID

> **Note**
>
> Available as of version 2.18.0.

> **Note**
>
> No restart will be performed on the Cloud Connector. To activate the configuration, a manual restart will be required.

URI	`/api/v2/configuration/connector/authentication/ldap/servers/<id>/move`
Method	*POST*
Request	`{position}`
Response	204
Errors	NOT_FOUND, INVALID_REQUEST
Roles	Administrator

Response Properties:

- `position`: position in the list starting with 0.

Example

```
curl -i -k -H 'Accept:application/json'
-u Administrator:<password> -X POST https://<scchost>:8443/api/v2/configuration/
connector/authentication/ldap/servers/1/move -d '{"position":0}'
```

Get Description for UI Certificate

This API returns a textual description for the system certificate.

> **Note**
>
> Available as of version 2.13.0.

URI	`/api/v1/configuration/connector/ui/uiCertificate`
Method	*GET*
Response	`{subjectDN, issuer, notBeforeTimeStamp, notAfterTimeStamp, subjectAltNames}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `subjectDN`: The subject distinguished name (a string)
- `issuer`: The issuer (a string)
- `notBeforeTimeStamp`: Timestamp of the beginning of the validity period (a UTC long number)
- `notAfterTimeStamp`: Timestamp of the end of the validity period (a UTC long number)
- `subjectAltNames`: Subject alternative names, as an array of objects with properties `type` and `value`. The value of property `type` is one of the following strings: IP, DNS, URI, or RFC822. The value of property `value` is the associated value (a string).

> ### Note
>
> `subjectAltNames` is not present if there are no subject alternative names.

Example

```
curl -i -k -H "Accept: application/json" -u <user>:<password> -X GET https://
<host>:<port>/api/v1/configuration/connector/ui/uiCertificate
```

Create a Self-Signed UI Certificate

> ### Note
>
> Available as of version 2.13.0.

URI	`/api/v1/configuration/connector/ui/uiCertificate`
Method	*POST*
Request	`{type, keySize, subjectDN, subjectAltNames}`
Response	201 on success
Errors	
Roles	Administrator, Associate Administrator

Request Properties:

- `type`: The string *selfsigned*
- `keySize`: Key size, which must be either 2048 or 4096 bits. Optional property. Default value is 4096.

  > ### Note
  >
  > This property is available as of version 2.15.0.

- `subjectDN`: The subject distinguished name (a string)
- `subjectAltNames`: Subject alternative names, as an array of objects with properties `type` and `value`. The value of property `type` is one of the following strings: IP, DNS, URI, or RFC822. The value of property `value` is the associated value (a string). This property is optional.

The UI certificate created this way has a validity of 1 year.

Example

```
curl -k -u Administrator:<password> -X POST -H  "Content-Type: application/json"
--data  '{"type":"selfsigned", "subjectDN":"CN=me"}'
https: //<host>:<port>/api/v1/configuration/connector/ui/uiCertificate
```

Create a Certificate Signing Request for a UI Certificate

> ### Note
>
> Available as of version 2.13.0.

URI	`/api/v1/configuration/connector/ui/uiCertificate`
Method	*POST*
Request	`{type, keySizesubjectDN, subjectAltNames}`
Response	PEM-encoded certificate request
Errors	
Roles	Administrator, Associate Administrator

Request Properties:

- `type`: The string *csr*
- `keySize`: Key size, which must be either 2048 or 4096 bits. Optional property. Default value is 4096.

  > **Note**
  >
  > This property is available as of version 2.15.0.

- `subjectDN`: The subject distinguished name (a string)
- `subjectAltNames`: Subject alternative names, as an array of objects with properties `type` and `value`. The value of property `type` is one of the following strings: IP, DNS, URI, or RFC822. The value of property `value` is the associated value (a string). This property is optional.

Example

```
curl -k -u Administrator:<password> -X POST -H  "Content-Type: application/json"
--data  '{"type":"csr", "subjectDN":"CN=me"}'
https: //<host>:<port>/api/v1/configuration/connector/ui/uiCertificate  -o
csr.pem
```

Upload a Signed Certificate Chain as UI Certificate

> **Note**
>
> Available as of version 2.13.0.

URI	`/api/v1/configuration/connector/ui/uiCertificate`
Method	*PATCH*
Request	multipart/form-data with the following form parameter: `signedCertificate`
Response	201 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator

Request Parameters:

- `signedCertificate`: The signed certificate and CA certificate chain (PEM-encoded)

Errors:

- `INVALID_REQUEST` (400): The certificate chain provided does not match the most recent certificate request, or it is not a certificate chain in the proper format (PEM-encoded).

Example

```
curl -i -k -u <user>:<password> -X PATCH -F signedCertificate=@<signedchain.pem>
https://<host>:<port>/api/v1/configuration/connector/ui/uiCertificate
```

Example

For test purposes, you can sign the certificate signing request with keytool.

```
keytool -genkeypair -keyalg RSA -keysize  1024 -alias mykey -dname  "cn=very
trusted, c=test" -validity  365 -keystore ca.ks -keypass testit -storepass testit
keytool -gencert -rfc -infile csr.pem -outfile signedcsr.pem -alias mykey
-keystore ca.ks -keypass testit -storepass testit
keytool -exportcert -rfc -file ca.pem -alias mykey -keystore ca.ks -keypass
testit -storepass testit
cat signedcsr.pem ca.pem > signedchain.pem
```

Upload a PKCS#12 Certificate as UI Certificate

> ### Note
>
> Available as of version 2.13.0.

URI	`/api/v1/configuration/connector/ui/uiCertificate`
Method	*PUT*

Request	multipart/form-data with the following form parameters:
	`pkcs12`
	`password`
	`keyPassword`

Response	204 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator

Request Parameters:

- `pkcs12`: Contents of PKCS#12 file
- `password`: Password for decrypting the PKCS#12 file
- `keyPassword`: Optional password for the private key

Errors:

- `INVALID_REQUEST` (400): Contents of PKCS#12 or password are invalid

> ### Note
>
> `keyPassword` is optional. If missing, `password` is used to decrypt the pkcs#12 file and the private key.

Example

```
curl -i -k -u <user>:<password> https://<host>:<port>/api/v1/configuration/
connector/ui/uiCertificate -X PUT -F 'password=<p12Password>' -F
pkcs12=@<p12file>
```

Example

For test purposes, you can create an own self-signed pkcs#12 certificate with keytool.

```
keytool -genkeypair -alias key -keyalg RSA -keysize 2048 -validity 365 -keypass
test20 -keystore test.p12 -storepass test20 -storetype PKCS12 -dname 'CN=test'
```

Get List of Available Cipher Suites for UI

> ⓘ Note
>
> Available as of version 2.15.0.

Returns a list of available cipher suites (as per JVM).

URI	`/api/v1/configuration/availableCipherSuites`
Method	*GET*
Request	
Response	`[name, ...]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

List of:

- `name`: Name of a cipher suite (a string).

  > ⓘ Note
  >
  > `name` is case-sensitive.

Example

```
curl -i -k -H "Accept: application/json" -u <user>:<password> -X GET https://
<host>:<port>/api/v1/configuration/availableCipherSuites
```

Get List of Enabled Cipher Suites for UI

> **Note**
>
> Available as of version 2.15.0.

Returns a list of enabled cipher suites.

URI	`/api/v1/configuration/connector/ui/cipherSuites`
Method	*GET*
Request	
Response	`[name, ...]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

List of:

- `name`: Name of a cipher suite (a string).

  > **Note**
  >
  `name` is case-sensitive.

Example

```
curl -i -k -H "Accept: application/json" -u <user>:<password> -X GET https://
<host>:<port>/api/v1/configuration/connector/ui/CipherSuites
```

Set List of Enabled Cipher Suites for UI

> **Note**
>
> Available as of version 2.15.0.

URI	`/api/v1/configuration/connector/ui/cipherSuites`
Method	*POST*
Request	`[name, ...]`
Response	204 on success
Errors	
Roles	Administrator

Request Properties:

List of:

- `name`: Name of a cipher suite (a string).

  > ### Note
  >
  > `name` is case-sensitive.

Example

```
curl -i -k -H "Content-Type: application/json" -u <user>:<password> --
data "['TLS_AES_128_GCM_SHA256', 'TLS_AES_256_GCM_SHA384']" -X POST https://
<host>:<port>/api/v1/configuration/connector/ui/cipherSuites
```

Enable default Cipher Suites for UI

> ### Note
>
> Available as of version 2.15.0.

Revert changes and enable the default list of cipher suites.

URI	`/api/v1/configuration/connector/ui/cipherSuites`

Method	*DELETE*
Request	
Response	204 on success
Errors	INVALID_REQUEST
Roles	Administrator

Response Properties:

- `name`: Name of cipher suite (a string).

  > ⓘ Note
  >
  > `name` is case-sensitive.

Example

```
curl -i -k -u <user>:<password> -X DELETE https://<host>:<port>/api/v1/
configuration/connector/ui/cipherSuites
```

1.4.2.5.6 Certificate Management for Back-End Communication

Manage a CA certificate for principal propagation or a system certificate via API.

> ⓘ Note
>
> The APIs below are available as of Cloud Connector version 2.13.

There are two similar sets of APIs for system certificate and CA certificate for principal propagation.

> ⓘ Note
>
> Some of the APIs list a parameter `subjectAltNames` (*subject alternative names* or SAN) for the request or response object. This parameter is an array of objects with the following properties:
>
> - `type`: one of the strings *DNS*, *URI*, *IP*, or *RFC822*.
> - `value`: the value associated with the chosen type.

- CA Certificate for Principal Propagation: APIs [page 1817]

- System Certificate APIs [page 1829]
- Truststore Back-End Certificates [page 1835]

1.4.2.5.6.1 CA Certificate for Principal Propagation: APIs

Manage a CA certificate for principal propagation via API.

> **Note**
>
> The APIs below are available as of Cloud Connector version 2.13.0.

> **Caution**
>
> The path component *onPremises* of the URI is available as of version 2.18.0. Older versions must use *onPremise*. The latter is currently accepted by all versions, but we recommend that you use *onPremises* as *onPremise* may be discontinued at some point.

Get Description for a CA Certificate for Principal Propagation

URI	`/api/v1/configuration/connector/onPremises/ppCaCertificate`
Method	*GET*
Header	Accept: application/json
Response	`{subjectDN, issuer, notBeforeTimeStamp, notAfterTimeStamp, subjectAltNames}`
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `subjectDN`: the subject distinguished name (a string)
- `issuer`: the issuer (a string)
- `notBeforeTimeStamp`: timestamp of the beginning of the validity period (a UTC long number)
- `notAfterTimeStamp`: timestamp of the end of the validity period (a UTC long number)
- `subjectAltNames`: subject alternative names (see Certificate Management for Back-End Communication [page 1816] for details).

Errors:

- `NOT_FOUND` (404): there is no CA certificate for principal propagation.

> **Note**
>
> `subjectAltNames` is not present if there are no subject alternative names.

Example

```
curl -i -k -H "Accept: application/json" -u <user>:<password> -X GET  https://
<host>:<port>/api/v1/configuration/connector/onPremises/ppCaCertificate
```

Get Binary Content of a CA Certificate for Principal Propagation

URI	`/api/v1/configuration/connector/onPremises/ppCaCertificate`
Method	*GET*
Header	Accept: application/pkix-cert
Response	Binary data of the certificate.
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `Success`: the binary data of the certificate; you can verify the downloaded certificate by storing it in file `ppca.crt`, for instance, and then running

  ```
  keytool -printcert -file ppca.crt
  ```

- `Failure`: an error in the usual JSON format; the content type of the response is *application/json* in this case.

Errors:

- `NOT_FOUND` (404): there is no CA certificate for principal propagation.

Example

```
curl -k -H "Accept: application/pkix-cert" -u <user>:<password> -X GET
--output sys.crt https://<host>:<port>/api/v1/configuration/connector/onPremises/
ppCaCertificate
```

Create a Self-Signed CA Certificate for Principal Propagation (Master Only)

URI	`/api/v1/configuration/connector/onPremises/ppCaCertificate`
Method	*POST*
Request	`{type, keySize, subjectDN, subjectAltNames}`
Response	201 on success
Errors	
Roles	Administrator, Associate Administrator

Request Properties:

- `type`: the string *selfsigned*
- `keySize`: Key size, which must be either 2048 or 4096 bits. Optional property. Default value is 4096.

  > ### Note
  >
  > This property is available as of version 2.15.0.

- `subjectDN`: the subject distinguished name (a string)
- `subjectAltNames`: subject alternative names (see Certificate Management for Back-End Communication [page 1816] for details). This property is optional.

The certificate created this way has a validity of 1 year.

Example

```
curl -i -k -H "Accept: application/json" -H "Content-Type: application/json" -u
<user>:<password> -X POST --data '{"type":"selfsigned", "subjectDN":"CN=me"}'
https://<host>:<port>/api/v1/configuration/connector/onPremises/ppCaCertificate
```

Create a Certificate Signing Request for a CA Certificate for Principal Propagation (Master Only)

URI	`/api/v1/configuration/connector/onPremises/ppCaCertificate`
Method	*POST*
Request	`{type, keySize, subjectDN, subjectAltNames}`
Response	PEM-encoded certificate request
Errors	
Roles	Administrator, Associate Administrator

Request Properties:

- `type`: the string *selfsigned*
- `keySize`: Key size, which must be either 2048 or 4096 bits. Optional property. Default value is 4096.

  > **Note**
  >
  > This property is available as of version 2.15.0.

- `subjectDN`: the subject distinguished name (a string)
- `subjectAltNames`: subject alternative names (see Certificate Management for Back-End Communication [page 1816] for details). This property is optional.

Example

```
curl -k -u <user>:<password> -X POST -H "Content-Type: application/json" --data
'{"type":"csr", "subjectDN":"CN=me"}'
https://<host>:<port>/api/v1/configuration/connector/onPremises/ppCaCertificate
-o csr.pem
```

Upload a Signed Certificate Chain as CA Certificate for Principal Propagation (Master Only)

URI	`/api/v1/configuration/connector/onPremises/ppCaCertificate`
Method	*PATCH*
Request	multipart/form-data with the following parameter: `signedCertificate`.
Response	201 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator

Request Properties:

- `signedCertificate`: the signed certificate and CA certificate chain (PEM-encoded)

Errors:

- `INVALID_REQUEST` (400): the certificate chain provided does not match the most recent certificate request, or it is not a certificate chain in the correct format (PEM-encoded).

Example

```
curl -i -k -u <user>:<password> -X PATCH -F signedCertificate=@<signedchain.pem>
https://<host>:<port>/api/v1/configuration/connector/onPremises/ppCaCertificate
```

Example: Sign The Certificate Signing Request

```
keytool -genkeypair -keyalg RSA -keysize 1024 -alias mykey -dname "cn=very
trusted, c=test" -validity 365 -keystore ca.ks -keypass testit -storepass testit
keytool -gencert -rfc -infile csr.pem -outfile signedcsr.pem -alias mykey
-keystore ca.ks -keypass testit -storepass testit
keytool -exportcert -rfc -file ca.pem -alias mykey -keystore ca.ks -keypass
testit -storepass testit
cat signedcsr.pem ca.pem > signedchain.pem
```

Upload a PKCS#12 Certificate as CA Certificate for Principal Propagation (Master Only)

URI	`/api/v1/configuration/connector/onPremises/ppCaCertificate`
Method	*PUT*
Request	Multipart/form-data with the following form parameters: `pkcs12`, `password`, `keyPassword`
Response	204 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator

Request Parameters:

- `pkcs12`: contents of PKCS#12 file
- `password`: password for decrypting PKCS#12 file
- `keyPassword`: optional password for the private key

Errors:

- `INVALID_REQUEST` (400): contents of PKCS#12 or password are invalid

> ⓘ Note
>
> `keyPassword` is optional. If it is missing, `password` is used to decrypt the pkcs#12 file and the private key.

Example

```
curl -i -k -u <user>:<password> https://<host>:<port>/api/v1/configuration/
connector/onPremises/ppCaCertificate -X PUT -F 'password=<p21Password>' -F
pkcs12=@<p12file>
```

Create an own self-signed pkcs#12 certificate for tests with:

```
keytool -genkeypair -alias key -keyalg RSA -keysize 2048 -validity 365 -keypass
test20 -keystore test.p12 -storepass test20 -storetype PKCS12 -dname 'CN=test'
```

Delete a CA Certificate for Principal Propagation (Master Only)

URI	`/api/v1/configuration/connector/onPremises/ppCaCertificate`
Method	*DELETE*
Request	
Response	204 on success
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator

Errors:

- `NOT_FOUND` (404): there is no CA certificate for principal propagation.

Example

```
curl -k -H "Accept: application/json" -u <user>:<password> --request DELETE
https://localhost:8443/api/v1/configuration/connector/onPremises/ppCaCertificate
```

1.4.2.5.6.1.1 Subject Pattern Rules

Read and edit the Cloud Connector's subject pattern rules via API.

Get All Subject Pattern Rules

URI	`/api/v1/configuration/connector/onPremises/subjectPatternRules`
Method	*GET*
Request	
Response	`[{description, condition, subjectPattern}]`
Errors	

Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

An array of objects with the following properties:

- `description`: description of the rule (a string)
- `condition`: condition of the rule, described in a human-readable format (a string)
- `subjectPattern`: object containing key-value pairs representing the configured subject pattern fields (CN, EMAIL, L, OU, O, ST, C)

Sample Code

```
curl -i -k -H 'Accept:application/json' -u <user>:<password> -X GET https://
<host>:<port>/api/v1/configuration/connector/onPremises/subjectPatternRules
```

Create a Subject Pattern Rule

URI	/api/v1/configuration/connector/onPremises/subjectPatternRules
Method	*POST*
Request	`{description, condition, subjectPattern}`
Response	201 on success
Errors	Bad Request (400)
Roles	Administrator, Associate Administrator

Request:

An object with the following properties:

- `description` (required): description of the rule (string).
- `condition` (required): object specifying when this rule should apply. It must include:
  - variable (required, string): variable name to evaluate (e.g., "user_type").
  - operator (required, string): operator to use for evaluation. Valid values:
    - "exist" – Variable must exist.
    - "exist_not" – Variable must not exist.

    - "is" – Variable must exactly match the provided value (requires value).
    - "is_not" – Variable must not match the provided value (requires value).
  - value (conditionally required, string): value to evaluate against. Required if operator is "is" or "is_not"; otherwise, must not be provided.
- `subjectPattern` (required): object containing key-value pairs representing the configured subject pattern fields ("CN", "EMAIL", "L", "OU", "O", "ST", "C"). Values may include placeholders in the format "${variable}".

Sample Code

```
curl -i -k -u <user>:<password> -X POST \
-H 'Content-Type: application/json' \
-d '{"description":"<description>","condition":
{"variable":"<variable>","operator":"<operator>","value":"<value>"},"subjectPa
ttern":{"<field>":"<value>", ...}}' \
https://<host>:<port>/api/v1/configuration/connector/onPremises/
subjectPatternRules
```

Change a Subject Pattern Rule

URI	`/api/v1/configuration/connector/onPremises/subjectPatternRules/{index}`
Method	*PUT*
Request	`{description, condition, subjectPattern}`
Response	204 on success
Errors	Bad Request (400)
Roles	Administrator, Associate Administrator

Request:

An object with the following properties:

- `description` (required): description of the rule (string).
- `condition` (required): object specifying when this rule should apply. It must include:
  - variable (required, string): variable name to evaluate (e.g., "user_type").
  - operator (required, string): operator to use for evaluation. Valid values:
    - "exist" – Variable must exist.
    - "exist_not" – Variable must not exist.

    - "is" – Variable must exactly match the provided value (requires value).
    - "is_not" – Variable must not match the provided value (requires value).
  - value (conditionally required, string): value to evaluate against. Required if operator is "is" or "is_not"; otherwise, must not be provided.
- `subjectPattern` (required): object containing key-value pairs representing the configured subject pattern fields ("CN", "EMAIL", "L", "OU", "O", "ST", "C"). Values may include placeholders in the format "${variable}".

Sample Code

```
curl -i -k -u <user>:<password> -X PUT \
-H 'Content-Type: application/json' \
-d '{"description":"<description>","condition":
{"variable":"<variable>","operator":"<operator>","value":"<value>"},"subjectPa
ttern":{"<field>":"<value>", ...}}' \
https://<host>:<port>/api/v1/configuration/connector/onPremises/
subjectPatternRules/<index>
```

Delete a Subject Pattern Rule

URI	/api/v1/configuration/connector/onPremises/subjectPatternRules/{index}
Method	DELETE
Request	
Response	`204 on success`
Errors	Bad Request (400)
Roles	Administrator, Associate Administrator

Request:

Path Parameter:

- `index` (required, integer): zero-based index of the subject pattern rule to delete.

Sample Code

```
curl -i -k -u <user>:<password> -X DELETE \
https://<host>:<port>/api/v1/configuration/connector/onPremises/
subjectPatternRules/<index>
```

Move a Subject Pattern Rule

URI	`/api/v1/configuration/connector/onPremises/subjectPatternRules/{id}/move`
Method	*POST*
Request	
Response	200 on success
Errors	Bad Request (400)
Roles	Administrator, Associate Administrator

Request:

Path Parameter:

- `id` (required, integer): zero-based index of the subject pattern rule to move.

Request Body: A JSON object containing:

- `position` (required, integer): new zero-based index to move the rule to.

Sample Code

```
curl -i -k -u <user>:<password> -X POST \
-H 'Content-Type: application/json' \
-d '{"position": <new_position>}' \
https://<host>:<port>/api/v1/configuration/connector/onPremises/
subjectPatternRules/<id>/move
```

Generate a Sample Certificate from a Subject Pattern Rule

URI	/api/v1/configuration/connector/onPremises/subjectPatternRules/{index}/sampleCertificate
Method	*GET*

Request	
Response	200 on success
Errors	
Roles	Administrator, Associate Administrator

Request:

Path Parameter:

- `index` (required, integer): zero-based index of the subject pattern rule used to generate the sample certificate.

Produces:

Format	Description
application/pkix-cert	DER-encoded certificate (binary)
application/pkix-cert	PEM-encoded certificate (Base64 with headers)

The response is a downloadable certificate file:

- **Filename**:
  - scc_sample_cert.der (for DER)
  - scc_sample_cert.pem (for PEM)

{·} Sample Code

Curl example for DER format

```
curl -i -k -u <user>:<password> -X GET \
-H 'Accept: application/pkix-cert' \
"https://<host>:<port>/api/v1/configuration/connector/
onPremises/subjectPatternRules/<index>/sampleCertificate?
<placeholder1>=<value1>&<placeholder2>=<value2>"
```

{·} Sample Code

Curl example for PEM format

```
curl -i -k -u <user>:<password> -X GET \
-H 'Accept: application/x-pem-file' \
"https://<host>:<port>/api/v1/configuration/connector/
onPremises/subjectPatternRules/<index>/sampleCertificate?
<placeholder1>=<value1>&<placeholder2>=<value2>"
```

> ⚠ Caution
>
> - All required placeholders must be provided. Otherwise, the request fails with *400 Bad Request*.
> - No extra query parameters are allowed beyond those matching the configured placeholders.
> - Format (DER or PEM) is selected automatically by the `Accept` HTTP header.

1.4.2.5.6.2 System Certificate APIs

Manage a system certificate via API.

> ⓘ Note
>
> The APIs below are available as of Cloud Connector version 2.13.0.

> ⚠ Caution
>
> The path component *onPremises* of the URI is available as of version 2.18.0. Older versions must use *onPremise*. The latter is currently accepted by all versions, but we recommend that you use *onPremises* as *onPremise* may be discontinued at some point.

Get Description for a System Certificate

URI	`/api/v1/configuration/connector/onPremises/systemCertificate`
Method	*GET*
Header	Accept: application/json
Response	`{subjectDN, issuer, notBeforeTimeStamp, notAfterTimeStamp, subjectAltNames}`
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `subjectDN`: the subject distinguished name (a string)
- `issuer`: the issuer (a string)

- `notBeforeTimeStamp`: timestamp of the beginning of the validity period (a UTC long number)
- `notAfterTimeStamp`: timestamp of the end of the validity period (a UTC long number)
- `subjectAltNames`: subject alternative names.

Errors:

- `NOT_FOUND` (404): there is no system certificate.

> ### Note
>
> `subjectAltNames` is not present if there are no subject alternative names.

Example

```
curl -i -k -H "Accept: application/json" -u <user>:<password> -X GET  https://
<host>:<port>/api/v1/configuration/connector/onPremises/systemCertificate
```

Get Binary Content of a System Certificate

URI	`/api/v1/configuration/connector/onPremises/systemCertificate`
Method	*GET*
Header	Accept: application/pkix-cert
Response	Binary data of the certificate.
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `Success`: the binary data of the certificate; you can verify the downloaded certificate by storing it in file `sys.crt`, for instance, and then running

  ```
  keytool -printcert -file sys.crt
  ```

- `Failure`: an error in the usual JSON format; the content type of the response is *application/json* in this case.

Errors:

- `NOT_FOUND` (404): there is no system certificate.

Example

```
curl -i -k -H "Accept: application/pkix-cert" -u <user>:<password> -X GET
--output sys.crt https://<host>:<port>/api/v1/configuration/connector/onPremises/
systemCertificate
```

Create a Self-Signed System Certificate (Master Only)

URI	`/api/v1/configuration/connector/onPremises/systemCertificate`
Method	*POST*
Header	CONTENT_TYPE: application/json
Request	`{type, keySize, subjectDN}`
Response	201 on success
Errors	
Roles	Administrator, Associate Administrator

Request Properties:

- `type`: the string *selfsigned*
- `keySize`: Key size, which must be either 2048 or 4096 bits. Optional property. Default value is 4096.

  > **Note**
  >
  > This property is available as of version 2.15.0.

- `subjectDN`: the subject distinguished name (a string)
- `subjectAltNames`: subject alternative names. This property is optional. As of version 2.16.1, the value is ignored. SAN on system certificate had never any effect.

The certificate created this way has a validity of 1 year.

Example

```
curl -i -k -H "Accept: application/json" -H "Content-Type: application/json" -u
<user>:<password> -X POST --data '{"type":"selfsigned", "subjectDN":"CN=me"}'
https://<host>:<port>/api/v1/configuration/connector/onPremises/systemCertificate
```

Create a Certificate Signing Request for a System Certificate (Master Only)

URI	`/api/v1/configuration/connector/onPremises/systemCertificate`
Method	*POST*
Header	CONTENT_TYPE: application/json
Request	`{type, keySize, subjectDN}`
Response	PEM-encoded certificate request
Errors	
Roles	Administrator, Associate Administrator

Request Properties:

- `type`: the string *csr*
- `keySize`: Key size, which must be either 2048 or 4096 bits. Optional property. Default value is 4096.

  > ### Note
  >
  > This property is available as of version 2.15.0.

- `subjectDN`: the subject distinguished name (a string)
- `subjectAltNames`: subject alternative names. This property is optional. As of version 2.16.1, the value is ignored. SAN on system certificate had never any effect.

Example

```
curl -k -u <user>:<password> -X POST -H "Content-Type: application/json" --data
'{"type":"csr", "subjectDN":"CN=me"}'
https://<host>:<port>/api/v1/configuration/connector/onPremises/
systemCertificate  -o csr.pem
```

Upload a Signed Certificate Chain as System Certificate (Master Only)

URI	`/api/v1/configuration/connector/onPremises/systemCertificate`
Method	*PATCH*
Request	multipart/form-data with the following parameter: `signedCertificate`.
Response	201 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator

Request Properties:

- `signedCertificate`: the signed certificate and CA certificate chain (PEM-encoded)

Errors:

- `INVALID_REQUEST` (400): the certificate chain provided does not match the most recent certificate request, or it is not a certificate chain in the correct format (PEM-encoded).

Example

```
curl -i -k -u <user>:<password> -X PATCH -F signedCertificate=@<signedchain.pem>
https://<host>:<port>/api/v1/configuration/connector/onPremises/systemCertificate
```

For test purposes, you can sign the certificate signing request with keytool:

```
keytool -genkeypair -keyalg RSA -keysize 1024 -alias mykey -dname "cn=very
trusted, c=test" -validity 365 -keystore ca.ks -keypass testit -storepass testit
keytool -gencert -rfc -infile csr.pem -outfile signedcsr.pem -alias mykey
-keystore ca.ks -keypass testit -storepass testit
keytool -exportcert -rfc -file ca.pem -alias mykey -keystore ca.ks -keypass
testit -storepass testit
cat signedcsr.pem ca.pem > signedchain.pem
```

Upload a PKCS#12 Certificate as System Certificate (Master Only)

URI	`/api/v1/configuration/connector/onPremises/systemCertificate`
Method	*PUT*
Request	Multipart/form-data with the following form parameters: `pkcs12`, `password`, `keyPassword`
Response	204 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator

Request Parameters:

- `pkcs12`: contents of PKCS#12 file
- `password`: password for decrypting PKCS#12 file
- `keyPassword`: optional password for the private key

Errors:

- `INVALID_REQUEST` (400): contents of PKCS#12 or password are invalid

> ### Note
>
> `keyPassword` is optional. If it is missing, `password` is used to decrypt the pkcs#12 file and the private key.

Example

```
curl -i -k -u <user>:<password> https://<host>:<port>/api/v1/configuration/
connector/onPremises/systemCertificate -X PUT -F password=<p21Password> -F
pkcs12=@<p12file>
```

Create an own self-signed pkcs#12 certificate for tests with:

```
keytool -genkeypair -alias key -keyalg RSA -keysize 2048 -validity 365 -keypass
test20 -keystore test.p12 -storepass test20 -storetype PKCS12 -dname 'CN=test'
```

Delete a System Certificate (Master Only)

URI	`/api/v1/configuration/connector/onPremises/systemCertificate`
Method	*DELETE*
Request	
Response	204 on success
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator

Errors:

- `NOT_FOUND` (404): there is no system certificate.

Example

```
curl -k -H "Accept: application/json" -u <user>:<password> --
request DELETE https://localhost:8443/api/v1/configuration/connector/onPremises/
systemCertificate
```

1.4.2.5.6.3 Truststore Back-End Certificates

Manage back-end certificates in the truststore via API.

Get Truststore Configuration

> ⓘ Note
>
> This API is available as of Cloud Connector version 2.15.0.

> ⚠ Caution
>
> The path component *onPremises* of the URI is available as of version 2.18.0. Older versions must use *onPremise*. The latter is currently accepted by all versions, but we recommend that you use *onPremises* as *onPremise* may be discontinued at some point.

URI	`/api/v1/configuration/connector/onPremises/truststore`
Method	*GET*
Header	
Response	`{trustAllBackends, trustedBackends}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `trustAllBackends`: flag (boolean) indicating whether all backends are trusted (**`true`**), or only the backends represented by certificates are trusted (**`false`**).
- `trustedBackendSystems`: list (array) of certificates representing the trusted backends. Each certificate is specified through an object with the following properties:

Certificate Properties:

- `alias`: alias of CA certificate (a string).
- `subjectDN`: subject DN (a string).
- `issuer`: the issuer (a string).
- `notAfterTimeStamp`: (a UTC long number).

> ⓘ Note
>
> No HTTP requests can be processed if `trustAllBackends` is **`false`** and the certificate list as per `trustedBackendSystems` is empty.

Example

```
curl -i -k -u <user>:<password> -X GET https://<host>:<port>/api/v1/
configuration/connector/onPremises/truststore
```

Change Truststore Configuration (Master Only)

> ⓘ Note
>
> This API is available as of Cloud Connector version 2.15.0.

URI	`/api/v1/configuration/connector/onPremises/truststore`
Method	*PATCH*
Header	Accept: application/json
Request	`{trustAllBackends}`
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator

Request Properties:

- `trustAllBackends`: flag (boolean) indicating whether all backends are trusted (**`true`**), or only the backends represented by certificates are trusted (**`false`**)

> ⚠ Caution
>
> We discourage trusting all backends and recommend that you trust only specific backends represented by their respective certificates.

Example

```
curl -k -u <user>:<password> -X PATCH  -H "Accept: application/json"
-d '{"trustAllBackends": true}' https://<host>:<port>/api/v1/configuration/
connector/onPremises/truststore
```

Add a Back-End Certificate to Truststore (Master Only)

> ⓘ Note
>
> This API is available as of Cloud Connector version 2.18.0.

URI	`/api/v1/configuration/connector/onPremises/truststore/certificates`

Method	*POST*
Header	CONTENT_TYPE: multipart/form-data
Request	`multipart/form-data with the following form parameter: certificate`
Response	201 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator

Request Properties:

- `certificates`: one or more certificates in DER or PEM format.

Errors:

- INVALID_REQUEST: if one of the certificates is already available.

> ### Note
>
> Prior to version 2.18, only *one* certificate was supported by that API with the parameter `certificate`. For compatibility reasons, this parameter is still supported. However, it should not be used anymore.

Example

```
curl -k -u <user>:<password> --request POST -F 'certificates=@<file>' https://
<host>:<port>/api/v1/configuration/connector/onPremises/truststore/certificates
```

Download All Back-End Certificates From Truststore (Master Only)

> ### Note
>
> This API is available as of Cloud Connector version 2.18.0.

URI	`/api/v1/configuration/connector/onPremises/truststore/certificates`
Method	*GET*

Header	Accept: application/x-pem-file Accept: application/pkix-cert
Request	
Response	CA certificate in binary format
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Request Header:

- `Accept: application/pkix-cert` forces binary output containing the specified certificates.

Errors:

- NOT_FOUND: if a certificate with a given alias is not available.

Example

```
curl -k -u <user>:<password> --request GET  -H "Accept: application/x-pem-
file" --output <pem file> https://<host>:<port>/api/v1/configuration/connector/
onPremises/truststore/certificates
```

Download a Back-End Certificate From Truststore (Master Only)

> **Note**
>
> This API is available as of Cloud Connector version 2.18.0.

URI	`/api/v1/configuration/connector/onPremises/truststore/certificates/<alias>`
Method	*GET*
Header	Accept: application/pkix-cert
Request	
Response	CA certificate in binary format

Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Request Header:

- `Accept: application/pkix-cert` forces binary output containing the specified certificate.

Errors:

- NOT_FOUND: if the certificate with the given alias is not available.

Example

```
curl -k -u <user>:<password> --request GET  -H "Accept: application/pkix-
cert" --output <cert file> https://<host>:<port>/api/v1/configuration/connector/
onPremises/truststore/certificates/<alias>
```

Delete a Back-End Certificate From Truststore (Master Only)

> **Note**
>
> This API is available as of Cloud Connector version 2.15.0.

URI	`/api/v1/configuration/connector/onPremises/truststore/certificates/<alias>`
Method	*DELETE*
Header	
Response	204 on success
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator

Errors:

- NOT_FOUND: if the certificate with the given alias is not available.

Example

```
curl -k -H "Accept: application/json" -u <user>:<password> --
request DELETE https://<host>:<port>/api/v1/configuration/connector/onPremises/
truststore/certificates/<alias>
```

Delete All CA Back-End Certificates From Truststore (Master Only)

> **Note**
>
> This API is available as of Cloud Connector version 2.15.0.

URI	`/api/v1/configuration/connector/onPremises/truststore/certificates`
Method	*DELETE*
Header	
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator

Example

```
curl -k -H "Accept: application/json" -u <user>:<password> --
request DELETE https://<host>:<port>/api/v1/configuration/connector/onPremises/
truststore/certificates
```

1.4.2.5.7 Solution Management Configuration

Manage the Cloud Connector's solution management configuration via API.

Get Solution Management Configuration

URI	`/api/v1/configuration/connector/solutionManagement`
Method	*GET*
Request	
Response	`{hostAgentPath, isEnabled, dsrEnabled}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `isEnabled`: flag indicating if reporting to solution management is active.
- `hostAgentPath`: path for host agent executable.
- `dsrEnabled`: indicating if DSR reporting is active.

Example

```
curl -ik -u <user>:<password>  https://<host>:<port>/api/v1/configuration/
connector/solutionManagement
```

Activate Solution Management Reporting

This API turns on the integration with the Solution Manager. The prerequisite is an available Host Agent. You can specify a path to the Host Agent executable, if you don't use the default path.

URI	`/api/v1/configuration/connector/solutionManagement`
Method	*POST*
Request	`{hostAgentPath, dsrEnabled}`
Response	
Errors	INVALID_CONFIGURATION, RUNTIME_FAILURE
Roles	Administrator, Support

Response Properties:

- `hostAgentPath`: path for host agent executable (string, optional).
- `dsrEnabled`: flag indicating if DSR reporting is active (boolean, optional)

Errors:

- `INVALID_CONFIGURATION` (409): host agent executable is not a file or does not exist.
- `RUNTIME_FAILURE` (500): failure when executing the host agent executable.

Example

```
curl -ik -u <user>:<password>  https://<host>:<port>/api/v1/configuration/
connector/solutionManagement -X POST
```

or, if configuration has to be changed:

```
curl -ik -u <user>:<password>  https://<host>:<port>/api/v1/configuration/
connector/solutionManagement -X POST -d "{\"hostAgentPath\":\"new/path/to/
hostAgent\"}" -H "Content-Type:application/json"
```

Deactivate Solution Management Reporting

URI	`/api/v1/configuration/connector/solutionManagement`
Method	*DELETE*

Request	
Response	
Errors	
Roles	Administrator, Support

Example

```
curl -ik -u <user>:<password>  https://<host>:<port>/api/v1/configuration/
connector/solutionManagement -X DELETE
```

Get Registration File

Generates a zip file containing the registration file for the solution management LMDB (Landscape Management Database).

> **Note**
>
> Available as of Cloud Connector version 2.12.0.

URI	`/api/v1/configuration/connector/solutionManagement/registrationFile`
Method	*GET*
Request	
Response	
Errors	
Roles	Administrator, Support

1.4.2.5.8 Backup

Manage the Cloud Connector's configuration backup via API.

Create Backup Configuration

URI	`/api/v1/configuration/backup`
Method	*POST*
Request	`{password}`
Response	ZIP archive (content type `application/zip`)
Errors	INVALID_REQUEST, FORBIDDEN
Roles	Administrator

Request Properties:

- `password`: password used to encrypt sensitive data.

Errors:

- `INVALID_REQUEST` (400): if no password was provided.
- `FORBIDDEN` (403): if the instance role is *shadow*.

> ### Note
>
> Only sensitive data in the backup are encrypted with an arbitrary password of your choice. The password is required for the restore operation. The returned ZIP archive itself is not password-protected.

> ### Sample Code
>
> ```
> curl -k -u Administrator:<password> https://<host>:<port>/api/v1/
> configuration/backup -X POST -H 'Content-Type: application/json' -d
> "{\"password\":\"<password>\"}" -o <backupfile>.zip
> ```

Restore Backup Configuration

> ### Caution
>
> A successful request triggers a restart of the Cloud Connector.

URI	`/api/v1/configuration/backup`

Method	*PUT*
Request	multipart/form-data with the following form parameters: `backup`, `password`.
Response	204 on success
Errors	INVALID_REQUEST, FORBIDDEN, INTERNAL_SERVER_ERROR
Roles	Administrator

Request Properties:

- `backup`: a backup file (produced through POST request).
- `password`: password chosen when creating the backup.

Errors:

- `INVALID_REQUEST` (400): if invalid or no backup file, or incorrect or missing password was provided.
- `FORBIDDEN` (403): if the instance role is *shadow*.
- `INTERNAL_SERVER_ERROR` (500): if an error happened during restore.

> ### Note
>
> Since this API uses a multipart request, it requires a multipart request header.

> ### Sample Code
>
> ```
> curl -k -u Administrator:<password> https://<host>:<port>/api/v1/
> configuration/backup -X PUT -F 'password=<password>' -F
> backup=@<backupfile>.zip
> ```

1.4.2.5.9 Subaccount

Manage the Cloud Connector's subaccount settings via API.

> ### Note
>
> Access to *managed* subaccounts is reserved for the role *Administrator* for all methods except for method GET.
>
> For more information on managed subaccounts, see Adding a Subaccount [page 1684] (step 2).

Operations

Subaccount	Get Subaccounts [page 1847] Get Corrupted Subaccounts [page 1848] Create Subaccount (Master Only) [page 1849] Delete Subaccount (Master Only) [page 1850] Edit Subaccount (Master Only) [page 1851] Connect/Disconnect Subaccount (Master Only) [page 1852] Renew Subaccount Certificate (Master Only) [page 1852] Get Subaccount Configuration [page 1853]
Recovery Subaccount ⚠ Caution This feature is deprecated. Due to the discontinuation of the *Enhanced Disaster Recovery Service*, the related functionality in the Cloud Connector has been dropped as of version 2.16. For more information, see What's New for SAP Business AI Platform.	Create Recovery Subaccount (Master Only) [page 1854] Delete Recovery Subaccount (Master Only) [page 1855] Refresh Certificate of Recovery Subaccount (Master Only) [page 1855] Activate/Deactivate Recovery Subaccount (Master Only) [page 1855] Takeover (Master Only) [page 1855]
Trust	Get Automatic Trust Synchronization Configuration [page 1856] Set Automatic Trust Synchronization Configuration [page 1857] Synchronize Trust List (Master Only) [page 1858] Get IDP Trust List [page 1858] Get Application Trust List [page 1859] Enable Or Disable Trust (Master Only) [page 1860] Get IDP Trust Properties [page 1860] Get Application Trust Properties [page 1861]

Get Subaccounts

URI	`/api/v1/configuration/subaccounts`
Method	*GET*
Request	
Response	`[{regionHost, subaccount, locationID}]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

An array of objects with the following properties:

- `regionHost`: region hosts (a string).
- `subaccount`: subaccount name (a string).
- `locationID`: location identifier for the Cloud Connector instance (a string); this property is not available if the default location ID is in use.

Back to Operations [page 1847]

Get Corrupted Subaccounts

> **Note**
>
> Available as of version 2.19.1.

URI	`/api/v1/configuration/subaccounts/corrupted`
Method	*GET*
Request	
Response	`[{regionHost, subaccount}]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

This will return an an array of corrupted subaccounts, that is, subaccounts whose configuration files were tampered and the Cloud Connector was not able to properly load them on startup.

An array of objects with the following properties:

- `regionHost`: region hosts (a string).
- `subaccount`: subaccount names (a string).

Back to Operations [page 1847]

Create Subaccount (Master Only)

Creates and connects a subaccount.

URI	`/api/v1/configuration/subaccounts`
Method	*POST*
Request	`{regionHost, subaccount, cloudUser, cloudPassword, locationID, autoCertRenewal, displayName, description, isManaged}`
Request (as of version 2.17.0)	`{authenticationData, locationID, autoCertRenewal, displayName, description, isManaged}`
Response	201, created subaccount entity: `{regionHost, subaccount, locationID, autoCertRenewal, displayName, description, tunnel}`
Errors	INVALID_REQUEST, INVALID_CONFIGURATION
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request Properties:

- `regionHost`: region host name (a string).
- `subaccount`: subaccount technical name (a string).
- `cloudUser`: user for the specified subaccount and region host.
- `cloudPassword`: password for the cloud user.
- `authenticationData`: subaccount authentication data, used instead of (that is, potentially overriding) `cloudUser`, `cloudPassword` and `regionHost`.

- `locationID`: location identifier for the Cloud Connector instance (a string; optional).
- `autoCertRenewal`: flag indicating the auto renewal of the subaccount certificate is allowed (boolean; optional, default is false) (as of 2.19).
- `displayName`: display name of the subaccount (a string; optional).
- `description`: subaccount description (a string; optional).
- `isManaged`: flag that indicates whether or not the subaccount to be created should be a managed subaccount (Boolean; optional, default is false)

Response Properties:

- `regionHost`: region host name (a string).
- `subaccount`: subaccount technical name (a string).
- `locationID`: location ID (a string); this property is not available if the default location ID is in use.
- `autoCertRenewal`: flag indicating whether auto renewal of the subaccount certificate is allowed (as of 2.19).
- `displayName`: display name of the subaccount (a string); this property is not available if there is no specified display name.
- `description`: subaccount description (a string); this property is not available if there is no description.
- `tunnel`: object outlining the current state of the tunnel.
- `isManaged`: a flag that indicates whether or not the subaccount is a managed subaccount (as of 2.19).

Errors:

- INVALID_REQUEST (400): one or more mandatory parameters are missing or invalid.
- INVALID_CONFIGURATION (409): the subaccount already exists.

Back to Operations [page 1847]

Delete Subaccount (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>`
Method	*DELETE*
Request	
Response	204 on success
Errors	NOT_FOUND, ILLEGAL_STATE
Roles	Administrator, Associate Administrator, Subaccount Administrator

Errors:

- `NOT_FOUND` (404): subaccount does not exist (in the specified region).
- `ILLEGAL_STATE` (409): there is at least one session that has access to the subaccount.

Back to Operations [page 1847]

Edit Subaccount (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>`
Method	*PUT*
Request	`{locationID, autoCertRenewal, displayName, description}`
Response	`{regionHost, subaccount, locationID, autoCertRenewal, displayName, description, tunnel}`
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request Properties:

- `locationID`: location identifier for the Cloud Connector instance (a string; optional); if this parameter is not supplied the location ID will not change. Revert to the default location ID by supplying the empty string.
- `autoCertRenewal`: flag indicating the auto renewal of the subaccount certificate is allowed (boolean; optional, default false) (as of 2.19).
- `displayName`: subaccount display name (a string; optional); if this parameter is not supplied the display name will not change. Clear the display name by using an empty string.
- `description`: subaccount description (a string; optional); if this parameter is not supplied the description will not change. Clear the description by using an empty string.

Response Properties:

- `regionHost`: region host name (a string).
- `subaccount`: subaccount technical name (a string).
- `locationID`: location identifier for the Cloud Connector instance (a string); this property is not available if the default location ID is in use.
- `autoCertRenewal`: flag indicating the auto renewal of the subaccount certificate is allowed (boolean; optional, default false) (as of 2.19).

- `displayName`: display name of the subaccount (a string); this property is not available if there is no specified display name.
- `description`: subaccount description (a string); this property is not available if there is no description.
- `tunnel`: object outlining the current state of the tunnel.

Errors:

- `NOT_FOUND` (404): subaccount does not exist (in the specified region).

Back to Operations [page 1847]

Connect/Disconnect Subaccount (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/state`
Method	*PUT*
Request	`{connected}`
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request Properties:

- `connected`: a Boolean value indicating whether the subaccount should be connected (true) or disconnected (false).

Back to Operations [page 1847]

Renew Subaccount Certificate (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/validity`
Method	*POST*

Request	`{user, password}`
Request (as of version 2.17.0)	`{authenticationData}`
Response	`{regionHost, subaccount, locationID, autoCertRenewal, displayName, description, tunnel}`
Errors (as of version 2.17.0)	BAD_REQUEST
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request Properties:

- `user`: (cloud) user for the specified region host and subaccount.
- `password`: password for the (cloud) user.
- `authenticationData`: subaccount authentication data, used instead of `user` and `password` (as of version 2.17.0).

Response Properties:

- `regionHost`: region host name (a string).
- `subaccount`: subaccount technical name (a string).
- `locationID`: location identifier for the Cloud Connector instance (a string); this property is not available if the default location ID is in use.
- `autoCertRenewal`: flag indicating that auto renewal of the subaccount certificate is allowed (boolean; optional, default false) (as of 2.19).
- `displayName`: display name of the subaccount (a string); this property is not available if there is no specified display name.
- `description`: subaccount description (a string); this property is not available if there is no description.
- `tunnel`: object outlining the current state of the tunnel.

Errors (as of version 2.17.0):

- `BAD_REQUEST` (400): the region in authentication data does not match the region of the subaccount.

Back to Operations [page 1847]

Get Subaccount Configuration

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>`
Method	*GET*
Request	
Response	`{regionHost, subaccount, locationID, autoCertRenewal, displayName, description, tunnel, isManaged}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `regionHost`: region host name (a string).
- `subaccount`: subaccount technical name (a string).
- `locationID`: location ID (a string); this property is not available if the default location ID is in use.
- `autoCertRenewal`: flag indicating that auto renewal of the subaccount certificate is allowed (as of version 2.19).
- `displayName`: display name of the subaccount (a string); this property is not available if there is no specified display name.
- `description`: description (a string); this property is not available if there is no description.
- `tunnel`: object outlining the current state of the tunnel.
- `isManaged`: flag that indicates whether or not the subaccount is a managed subaccount (as of version 2.19).

Back to Operations [page 1847]

Create Recovery Subaccount (Master Only)

> ⚠ Caution
>
> Disaster recovery discontinued as of version 2.16. A `POST` request to */api/v1/configuration/subaccounts/<regionHost>/<subaccount>/recovery* will trigger a 410 response.

Back to Operations [page 1847]

Delete Recovery Subaccount (Master Only)

> ⚠ Caution
>
> Disaster recovery discontinued as of version 2.16. A `DELETE` request to */api/v1/configuration/subaccounts/<regionHost>/<subaccount>/recovery* will trigger a 410 response.

Back to Operations [page 1847]

Refresh Certificate of Recovery Subaccount (Master Only)

> ⚠ Caution
>
> Disaster recovery discontinued as of version 2.16. A `POST` request to */api/v1/configuration/subaccounts/<regionHost>/<subaccount>/recovery/validity* will trigger a 410 response.

Back to Operations [page 1847]

Activate/Deactivate Recovery Subaccount (Master Only)

> ⚠ Caution
>
> Disaster recovery discontinued as of version 2.16. A `PUT` request to */api/v1/configuration/subaccounts/<regionHost>/<subaccount>/recovery/state* will trigger a 410 response.

Back to Operations [page 1847]

Takeover (Master Only)

> ⚠ Caution
>
> Disaster recovery discontinued as of version 2.16. A `POST` request to */api/v1/configuration/subaccounts/<regionHost>/<subaccount>/recovery/takeover* will trigger a 410 response.

Back to Operations [page 1847]

Get Automatic Trust Synchronization Configuration

> **Note**
>
> Available as of version 2.19.0.

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/trust`
Method	*GET*
Request	
Response	`{autoSyncTrustEnabled}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `autoSyncTrustEnabled`: Boolean - whether automatic trust configuration synchronization is enabled for this subaccount.

Errors:

- `NOT_FOUND`: no subaccount found for the specified path parameters.

> **Sample Code**

```
curl -i -k -H "Accept: application/json" -u <user>:<password> -X GET https://
<host>:<port>/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/trust
```

> **Sample Code**

Sample output: Trust synchronization configuration (GET)

```
{
  "autoSyncTrustEnabled": true
}
```

Back to Operations [page 1847]

Set Automatic Trust Synchronization Configuration

> **Note**
>
> Available as of version 2.19.0.

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/trust`
Method	*PUT*
Request	`{autoSyncTrustEnabled}`
Response	204 on success
Errors	INVALID_REQUEST, NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

- `autoSyncTrustEnabled`: Boolean - whether automatic trust configuration synchronization is enabled for this subaccount

Errors:

- `NOT_FOUND`: no subaccount found for the specified path parameters.
- `INVALID_REQUEST`:
  - empty or malformed input data
  - invalid property name
  - invalid value for `autoSyncTrustEnabled`
  - no trust configuration found for the specified subaccount

> **Sample Code**

```
curl -i -k -H "Content-Type: application/json" -u <user>:<password>
-X PUT https://<host>:<port>/api/v1/configuration/subaccounts/<regionHost>/
<subaccount>/trust -d "{\"autoSyncTrustEnabled\": true}"
```

> **Sample Code**

Sample input: Trust synchronization configuration (PUT)

```
{
  "autoSyncTrustEnabled": true
}
```

Back to Operations [page 1847]

Synchronize Trust List (Master Only)

> **Note**
>
> Available as of version 2.14.0.

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/trust`
Method	*POST*
Request	
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Back to Operations [page 1847]

Get IDP Trust List

> **Note**
>
> Available as of version 2.14.0.

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/trust/idps`
Method	*GET*
Request	
Response	`[{id, name, description, certificate, enabled}]`

Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

An array of objects, each of which represents a trusted IDP through the following properties:

- `id`: (unique) ID of the trusted IDP (a number).
- `name`: name of the trusted IDP (a string).
- `description`: description of the trusted IDP (a string).
- `certificate`: object with the following properties: `issuer` (a string), `subjectDN` (a string), `notBeforeTimeStamp` (a UTC long number), and `notAfterTimeStamp` (a UTC long number).
- `enabled`: flag that indicates whether the IDP is enabled or disabled (that is, whether the IDP is trusted or not).

Back to Operations [page 1847]

Get Application Trust List

> **Note**
>
> Available as of version 2.14.0.

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/trust/apps`
Method	*GET*
Request	
Response	`[{id, name, applicationType, enabled}]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

An array of objects, each of which represents a trusted application through the following properties:

- `id`: (unique) ID of the trusted application (a number).
- `name`: name of the trusted application (a string).

- `applicationType`: type of the application (a string, for example 'JAVA' or 'HANA').
- `enabled`: flag that indicates whether the application is enabled or disabled (that is, whether the application is trusted or not).

Back to Operations [page 1847]

Enable Or Disable Trust (Master Only)

Replace <type> with the type (either *apps* or *idps*) that belongs to the specified <id>.

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/trust/<type>/<id>`
Method	*PUT*
Request	`{enabled}`
Response	204 on success
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request Properties:

- `enabled`: true or false, indicating the specified trust source should be enabled or disabled.

Errors:

- `NOT_FOUND` (404): specified <id> does not exist for the given <type>.

Back to Operations [page 1847]

Get IDP Trust Properties

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/trust/idps/<id>`
Method	*GET*

Request	
Response	`{id, name, description, certificate, enabled}`
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator

Response Properties:

- `id`: (unique) ID of the trusted IDP (a number).
- `name`: name of the trusted IDP (a string).
- `description`: description of the trusted IDP (a string).
- `certificate`: object with the following properties: `issuer` (a string), `subjectDN` (a string), `notBeforeTimeStamp` (a UTC long number), and `notAfterTimeStamp` (a UTC long number).
- `enabled`: flag that indicates whether the IDP is enabled or disabled (that is, whether the IDP is trusted or not).

Errors:

- `NOT_FOUND` (404): There is no trusted IDP with the given <id>

Back to Operations [page 1847]

Get Application Trust Properties

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/trust/apps/<id>`
Method	*GET*
Request	
Response	`{id, name, applicationType, enabled}`
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator

Response Properties:

- `id`: (unique) ID of the trusted application (a number).

- `name`: name of the trusted application (a string).
- `applicationType`: type of the application (a string, for example 'JAVA' or 'HANA').
- `enabled`: flag that indicates whether the application is enabled or disabled (that is, whether the application is trusted or not).

Errors:

- `NOT_FOUND` (404): There is no trusted application with the given <id>.

Back to Operations [page 1847]

1.4.2.5.10 System Mappings

Manage the Cloud Connector's system mappings via API.

> **Note**
>
> Access to *managed* subaccounts is reserved for the role *Administrator* for all methods except for method GET.
>
> For more information on managed subaccounts, see Adding a Subaccount [page 1684] (step 2).

Get All System Mappings

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings`
Method	*GET*
Request	
Response	`[{virtualHost, virtualPort, localHost, localPort, protocol, backendType, authenticationMode, hostInHeader, description, ...}]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

An array of objects with the following properties:

- `virtualHost`: Virtual host used on the cloud side (a string)
- `virtualPort`: Virtual port used on the cloud side (a string)
- `localHost`: Host on the on-premises side (a string)
- `localPort`: Port on the on-premises side (a string)
- `protocol`: Protocol used when sending requests and receiving responses (a string)
- `backendType`: Type of the backend (a string)
- `authenticationMode`: Authentication mode used on the backend side (a string).
- `hostInHeader`: Policy for setting the host in the response header. Available for HTTP(S) protocols only.
- `totalResourcesCount`: the total number of resources
- `enabledResourcesCount`: the number of enabled resources
- `description`: Description for the system mapping (a string).
- `sncPartnerName`: SNC name of an ABAP Server, required for RFCS communication only.
- `sapRouter`: SAP router route, required only if an SAP router is used.
- `allowedClients`: Array of strings, describing the SAP clients allowing to execute the calls in this system. Valid clients are 3 letters long. If no clients are defined here, there is no restriction – every client is allowed. Only applicable for RFC-based communication.
- `blacklistedUsers`: Array of `{client, user}`, describing users that are not allowed to execute the call, even if the client is listed under allowed clients. Only applicable for RFC-based communication.

Get System Mapping

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>`
Method	*GET*
Request	
Response	`{virtualHost, virtualPort, localHost, localPort, protocol, backendType, authenticationMode, hostInHeader, description, ...}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

An array of objects with the following properties:

- `virtualHost`: Virtual host used on the cloud side (a string)
- `virtualPort`: Virtual port used on the cloud side (a string)
- `localHost`: Host on the on-premises side (a string)
- `localPort`: Port on the on-premises side (a string)
- `protocol`: Protocol used when sending requests and receiving responses (a string)
- `backendType`: Type of the backend (a string)
- `authenticationMode`: Authentication mode used on the backend side (a string).
- `hostInHeader`: Policy for setting the host in the response header (a string). Available for HTTP(S) protocols only.
- `totalResourcesCount`: the total number of resources
- `enabledResourcesCount`: the number of enabled resources
- `description`: Description for the system mapping (a string).
- `sncPartnerName`: SNC name of an ABAP Server, required for RFCS communication only.
- `sapRouter`: SAP router route, required only if an SAP router is used.
- `allowedClients`: Array of strings, describing the SAP clients allowing to execute the calls in this system. Valid clients are 3 letters long. If no clients are defined here, there is no restriction – every client is allowed. Only applicable for RFC-based communication.
- `blacklistedUsers`: Array of `{client, user}`, describing users that are not allowed to execute the call, even if the client is listed under allowed clients. Only applicable for RFC-based communication.

Create System Mapping (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings`
Method	*POST*
Request	`{virtualHost, virtualPort, localHost, localPort, protocol, backendType, authenticationMode, hostInHeader, description, ...}`
Response	201 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request Properties:

- `virtualHost`: Virtual host used on the cloud side (a string)
- `virtualPort`: Virtual port used on the cloud side (a string)
- `localHost`: Host on the on-premises side (a string)
- `localPort`: Port on the on-premises side (a string)
- `protocol`: Protocol used when sending requests and receiving responses, which must be one of the following strings: `HTTP, HTTPS, RFC, RFCS, LDAP, LDAPS, TCP, TCPS`.
- `backendType`: Type of the backend system. Valid values are `abapSys, netweaverCE, netweaverGW, applServerJava, BC, PI, hana, otherSAPsys, nonSAPsys`.
- `authenticationMode`: Authentication mode to be used on the backend side, which must be one of the following strings: `NONE, NONE_RESTRICTED, X509_GENERAL, X509_RESTRICTED, KERBEROS`.
- `hostInHeader`: Policy for setting the host in the response header. This property is applicable to HTTP(S) protocols only, and it is **optional**. If set, it must be one of the following strings: `internal, virtual`. The default is `virtual`. You may also use all capital letters, i.e. `INTERNAL` and `VIRTUAL`.
- `description`: Description for the system mapping (string, optional). The default is no description, i.e. the empty string.
- `sncPartnerName`: SNC name of an ABAP Server, required for RFCS communication only.
- `sapRouter`: SAP router route, required only if an SAP router is used.
- `allowedClients`: Array of strings, describing the SAP clients allowing to execute the calls in this system. Valid clients are 3 letters long. If no clients are defined here, there is no restriction – every client is allowed. Only applicable for RFC-based communication.
- `blacklistedUsers`: Array of `{client, user}`, describing users that are not allowed to execute the call, even if the client is listed under allowed clients. Only applicable for RFC-based communication.

> **Note**
>
> The authentication modes `NONE_RESTRICTED` and `X509_RESTRICTED` prevent the Cloud Connector from sending the system certificate in any case, whereas `NONE` and `X509_GENERAL` will send the system certificate if the circumstances allow it.

Delete System Mapping (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/`<virtualHost>:<virtualPort>
Method	*DELETE*
Request	
Response	204 on success

Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Delete All System Mappings (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings`
Method	*DELETE*
Request	
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Replace System Mapping (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>`
Method	*PUT*
Request	`{virtualHost, virtualPort, localHost, localPort, protocol, backendType, authenticationMode, hostInHeader, description, ...}`
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request Properties:

- `virtualHost`: Virtual host used on the cloud side (a string)
- `virtualPort`: Virtual port used on the cloud side (a string)
- `localHost`: Host on the on-premises side (a string)
- `localPort`: Port on the on-premises side (a string)
- `protocol`: Protocol used when sending requests and receiving responses, which must be one of the following strings: `HTTP, HTTPS, RFC, RFCS, LDAP, LDAPS, TCP, TCPS`.
- `backendType`: Type of the backend system. Valid values are `abapSys, netweaverCE, netweaverGW, applServerJava, BC, PI, hana, otherSAPsys, nonSAPsys`.
- `authenticationMode`: Authentication mode to be used on the backend side, which must be one of the following strings: `NONE, NONE_RESTRICTED, X509_GENERAL, X509_RESTRICTED, KERBEROS`.
- `hostInHeader`: Policy for setting the host in the response header; this property is **optional**. If set, it must be one of the following strings: `internal, virtual`. The default is `virtual`. You may also use all capital letters, i.e. `INTERNAL` and `VIRTUAL`.
- `description`: Description for the system mapping (string, optional). The default is no description, i.e. the empty string.
- `sncPartnerName`: SNC name of an ABAP Server, only set for RFCS communication.
- `sapRouter`: SAP router route, only set if an SAP router is used.
- `allowedClients`: Array of strings, describing the SAP clients allowing to execute the calls in this system. Valid clients are 3 letters long. If no clients are defined here, there is no restriction – every client is allowed. Only applicable for RFC-based communication.
- `blacklistedUsers`: Array of `{client, user}`, describing users, that are not allowed to execute the call, even if the client is listed under allowed clients. Only applicable for RFC-based communication.

> ### Note
>
> The authentication modes `NONE_RESTRICTED` and `X509_RESTRICTED` prevent the Cloud Connector from sending the system certificate in any case, whereas `NONE` and `X509_GENERAL` will send the system certificate if the circumstances allow it.

1.4.2.5.11 System Mapping Resources

Manage the Cloud Connector's system mapping resources via API.

> ### Note
>
> Access to *managed* subaccounts is reserved for the role *Administrator* for all methods except for method GET.
>
> For more information on managed subaccounts, see Adding a Subaccount [page 1684] (step 2).

Operations

System Mapping Resources	Get all System Mapping Resources [page 1868] Get System Mapping Resource [page 1869] Create System Mapping Resource [page 1870] Replace System Mapping Resource [page 1871] Delete System Mapping Resource [page 1872] Delete all System Mapping Resources [page 1872]
Allowed Clients	Get Allowed Clients for RFC System Mapping [page 1873] Set Allowed Clients for RFC System Mapping [page 1873] Add Allowed Clients for RFC System Mapping [page 1874] Delete an Allowed Client for RFC System Mapping [page 1875] Delete all Allowed Clients for RFC System Mapping [page 1876]
Blocked Users	Get Blocked Users for RFC System Mapping [page 1876] Set Blocked Users for RFC System Mapping [page 1877] Add Blocked Users for RFC System Mapping [page 1878] Remove one Blocked User for RFC System Mapping [page 1878] Remove all Blocked Users for RFC System Mapping [page 1879]

Get all System Mapping Resources

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/resources`
Method	*GET*
Request	

Response	`[{id, enabled, exactMatchOnly, websocketUpgradeAllowed, description}]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

An array of objects, each representing a resource through the following properties:

- `id`: The resource itself, which, depending on the owning system mapping, is either a URL path (or the leading section of it), or a RFC function name (prefix)
- `enabled`: Boolean flag indicating whether the resource is enabled.
- `exactMatchOnly`: Boolean flag determining whether access is granted only if the requested resource is an exact match.
- `websocketUpgradeAllowed`: Boolean flag indicating whether websocket upgrade is allowed; this property is of relevance only if the owning system mapping employs protocol HTTP or HTTPS.
- `description`: Description (a string); this property is not available unless explicitly set.

Back to Top [page 1867]

Get System Mapping Resource

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/resources/<encodedResourceId>`
Method	*GET*
Request	
Response	`{id, enabled, exactMatchOnly, websocketUpgradeAllowed, description}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `id`: The resource itself, which, depending on the owning system mapping, is either a URL path (or the leading section of it), or a RFC function name (prefix)
- `enabled`: Boolean flag indicating whether the resource is enabled.
- `exactMatchOnly`: Boolean flag determining whether access is granted only if the requested resource is an exact match.
- `websocketUpgradeAllowed`: Boolean flag indicating whether websocket upgrade is allowed; this property is of relevance only if the owning system mapping employs protocol HTTP or HTTPS.
- `description`: Description (a string); this property is not available unless explicitly set.

Back to Top [page 1867]

Create System Mapping Resource (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/resources`
Method	*POST*
Request	`{id, enabled, exactMatchOnly, websocketUpgradeAllowed, description}`
Response	201 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request Properties:

- `id`: The resource itself, which, depending on the owning system mapping, is either a URL path (or the leading section of it), or a RFC function name (prefix).
- `enabled`: Boolean flag indicating whether the resource is enabled (**optional**). The default value is `false`.
- `exactMatchOnly`: Boolean flag determining whether access is granted only if the requested resource is an exact match (**optional**). The default value is `false`.
- `websocketUpgradeAllowed`: Boolean flag indicating whether websocket upgrade is allowed (**optional**). The default value is `false`. This property is recognized only if the owning system mapping employs protocol HTTP or HTTPS.
- `description`: Description (a string, **optional**)

→ Tip

Encoded Resource ID

URI paths may contain the resource ID in order to identify the resource to be edited or deleted. A resource ID, however, may contain characters such as the forward slash that collide with the path separator of the URI and hence require an escape mechanism. We adopted the following simple escape or encoding method for a resource ID:

1. Replace all occurrences of character '+' with '+2B'.
2. Replace all occurrences of character '-' with '+2D'.
3. Replace all occurrences of character '/' with '-'.

Back to Top [page 1867]

Replace System Mapping Resource (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/resources/<encodedResourceId>`
Method	*PUT*
Request	`{enabled, exactMatchOnly, websocketUpgradeAllowed, description}`
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request Properties:

- `enabled`: Boolean flag indicating whether the resource is enabled.
- `exactMatchOnly`: Boolean flag determining whether access is granted only if the requested resource is an exact match.
- `websocketUpgradeAllowed`: Boolean flag indicating whether websocket upgrade is allowed; this property is of relevance only if the owning system mapping employs protocol HTTP or HTTPS.
- `description`: Description (a string); this property is not available unless explicitly set.

Back to Top [page 1867]

Delete System Mapping Resource (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/resources/<encodedResourceId>`
Method	*DELETE*
Request	
Response	204 on success
Errors	`NOT_FOUND`
Roles	Administrator, Associate Administrator, Subaccount Administrator

Errors:

`NOT_FOUND (404)`: Resource was not found

Back to Top [page 1867]

Delete all System Mapping Resources (Master Only)

URI	`/api/v1/configuration/subaccounts/<region>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/resources`
Method	*DELETE*
Request	
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Back to Top [page 1867]

Get Allowed Clients for RFC System Mapping

> **Note**
>
> Available as of version 2.14.

Returns the list of allowed ABAP clients for the system mapping definition.

URI	`/api/v1/configuration/subaccounts/<region>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/allowedClients`
Method	*GET*
Request	
Response	`[client, ...]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- Array of allowed clients:
  - `client`: ABAP client

> **Note**
>
> An empty list means that every client is allowed.

Back to Top [page 1867]

Set Allowed Clients for RFC System Mapping

> **Note**
>
> Available as of version 2.14.

URI	`/api/v1/configuration/subaccounts/<region>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/allowedClients`
Method	*POST*
Request	`[client, ...]`
Response	
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request Properties:

- Array of allowed clients:
  - `client`: ABAP client

The existing list of available clients will be cleaned and populated by the provided in request.

Back to Top [page 1867]

Add Allowed Clients for RFC System Mapping

> **Note**
>
> Available as of version 2.14.

Adds more ABAP clients to the allowed list for the system mapping definition.

URI	`/api/v1/configuration/subaccounts/<region>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/allowedClients`
Method	*PATCH*
Request	`[client, ...]`

Response	
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request Properties:

- Array of allowed clients:
  - `client`: ABAP client

Clients provided in this request will be added to the existing list. Clients that are already listed will be ignored.

Back to Top [page 1867]

Delete an Allowed Client for RFC System Mapping

> **Note**
>
> Available as of version 2.14.

URI	`/api/v1/configuration/subaccounts/<region>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/allowedClients/<client>`
Method	*DELETE*
Request	
Response	
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Back to Top [page 1867]

Delete all Allowed Clients for RFC System Mapping

> **Note**
>
> Available as of version 2.14.

URI	`/api/v1/configuration/subaccounts/<region>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/allowedClients`
Method	*DELETE*
Request	
Response	
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

> **Note**
>
> An empty list means that every client is allowed.

Back to Top [page 1867]

Get Blocked Users for RFC System Mapping

> **Note**
>
> Available as of version 2.14.

Returns the list of blocked ABAP users for the system mapping definition.

URI	`/api/v1/configuration/subaccounts/<region>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/blockedClientUsers`

Method	*GET*
Request	
Response	`[{client, user}, ...]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- Array of blocked users:
  - `client`: ABAP client
  - `user`: User name

Back to Top [page 1867]

Set Blocked Users for RFC System Mapping

> **Note**
>
> Available as of version 2.14.

Sets or replaces the list of blocked ABAP users for the system mapping definition.

URI	`/api/v1/configuration/subaccounts/<region>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/blockedClientUsers`
Method	*POST*
Request	`[{client, user}, ...]`
Response	
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request Properties:

- Array of blocked users:
  - `client`: ABAP client
  - `user`: User name

Back to Top [page 1867]

Add Blocked Users for RFC System Mapping

> **Note**
>
> Available as of version 2.14.

Adds the provided list of blocked ABAP users to the current list.

URI	`/api/v1/configuration/subaccounts/<region>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/blockedClientUsers`
Method	*PATCH*
Request	`[{client, user}, ...]`
Response	
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request Properties:

- Array of blocked users:
  - `client`: ABAP client
  - `user`: User name

Back to Top [page 1867]

Remove one Blocked User for RFC System Mapping

> **Note**
>
> Available as of version 2.14.

Removes one user from the list of blocked ABAP users for the system mapping definition.

URI	`/api/v1/configuration/subaccounts/<region>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/blockedClientUsers/<client>:<user>`
Method	*DELETE*
Request	
Response	
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Back to Top [page 1867]

Remove all Blocked Users for RFC System Mapping

> **Note**
>
> Available as of version 2.14.

Removes the list of blocked ABAP users for the system mapping definition.

URI	`/api/v1/configuration/subaccounts/<region>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/blockedClientUsers`
Method	*DELETE*
Request	
Response	
Errors	

Roles	Administrator, Associate Administrator, Subaccount Administrator

Back to Top [page 1867]

1.4.2.5.12 Domain Mappings

Manage the Cloud Connector's configuration for domain mappings via API.

> ### Note
>
> Access to *managed* subaccounts is reserved for the role *Administrator* for all methods except for method GET.
>
> For more information on managed subaccounts, see Adding a Subaccount [page 1684] (step 2).

Get Domain Mappings

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/domainMappings`
Method	*GET*
Request	
Response	`[{virtualDomain, internalDomain}]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

An array of objects, each representing a domain mapping through the following properties:

- `virtualDomain`: Domain used on the cloud side
- `internalDomain`: Domain used on the on-premises side

Create Domain Mapping (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/domainMappings`
Method	*POST*
Request	`{virtualDomain, internalDomain}`
Response	201
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Replace Domain Mapping (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/domainMappings/<internalDomain>`
Method	*PUT*
Request	`{virtualDomain, internalDomain}`
Response	204 on success
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

- `virtualDomain`: New virtual domain
- `internalDomain`: New internal domain

Errors:

- NOT_FOUND (404): Domain mapping does not exist.

> **Note**
>
> The internal domain in the URI path (i.e., <internalDomain>) is the current internal domain of the domain mapping that is to be edited. It may differ from the new internal domain set in the request.

Delete Domain Mapping (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/domainMappings/<internalDomain>`
Method	*DELETE*
Request	
Response	204 on success
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator

Errors:

- NOT_FOUND (404): Domain mapping does not exist.

Delete All Domain Mappings (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/domainMappings`
Method	*DELETE*
Request	
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

1.4.2.5.13 Subaccount Service Channels

Manage Cloud Connector service channels via API.

> **Note**
>
> Access to *managed* subaccounts is reserved for the role *Administrator* for all methods except for method GET.
>
> For more information on managed subaccounts, see Adding a Subaccount [page 1684] (step 2).

Service Channel for HANA Database	Get all HANA Service Channels [page 1884] Get Available HANA Channels [page 1885] Get HANA Service Channel [page 1886] Create HANA Service Channel (Master Only) [page 1887] Replace HANA Service Channel (Master Only) [page 1887]
Service Channel for Virtual Machine	Get all Service Channels for Virtual Machines [page 1888] Get Available Channels for Virtual Machines [page 1889] Get Service Channel for a Virtual Machine [page 1889] Create a Service Channel for a Virtual Machine (Master Only) [page 1890] Replace a Service Channel for a Virtual Machine (Master Only) [page 1891]
Service Channel for ABAP Cloud	Get all ABAP Cloud Service Channels without SNC [page 1892] Get ABAP Cloud Service Channel without SNC [page 1893] Create ABAP Cloud Service Channel without SNC (Master Only) [page 1894] Replace ABAP Cloud Service Channel without SNC (Master Only) [page 1894] Get all ABAP Cloud SNC-Encrypted Service Channels [page 1895] Get ABAP Cloud SNC-Encrypted Service Channel [page 1896] Create ABAP Cloud SNC-Encrypted Service Channel (Master Only) [page 1897] Replace ABAP Cloud SNC-Encrypted Service Channel (Master Only) [page 1898]

Service Channel for Kubernetes **Note** Available as of version 2.15.0.	Get All Kubernetes Cluster Service Channels [page 1899] Get Kubernetes Service Channel [page 1900] Create Kubernetes Service Channel (Master Only) [page 1900] Replace Kubernetes Service Channel (Master Only) [page 1901]
General	Enable/Disable Service Channel (Master Only) [page 1902] Delete Service Channel (Master Only) [page 1903] Delete all Service Channels (Master Only) [page 1903]
Deprecated	Get all Service Channels [page 1904] Get Service Channel [page 1905] Create Service Channel [page 1906] Replace Service Channel [page 1906]

Get all HANA Service Channels

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/HANA`
Method	*GET*
Request	
Response	`[{id, hanaInstanceName, instanceNumber, type, port, enabled, connections, state}]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

An array of objects, each of which represents a HANA service channel through the following properties:

- `id`: unique identifier for the service channel (a positive integer number, starting with 1). This identifier is unique across all types of service channels.
- `hanaInstanceName`: name of the HANA instance (a string).
- `instanceNumber`: instance number.

- `type`: string 'HANA'.
- `port`: port of the HANA service channel (a number).
- `enabled`: boolean flag indicating whether the channel is enabled and therefore should be open.
- `connections`: maximal number of open connections.
- `state`: current connection state; this property is only available if the channel is enabled (as per property `enabled`). The value of this property is an object with the properties:
  - `connected` (a boolean flag indicating whether the channel is connected),
  - `openedConnections` (the number of open, possibly idle connections), and
  - `connectedSinceTimeStamp` (the time stamp, a UTC long number, for the first time the channel was opened/connected).

Back to Top [page 1883]

Get Available HANA Channels

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/availableChannels/HANA`
Method	*GET*
Request	
Response	`[{hanaInstanceName, version, type}]`
Errors	RUNTIME_FAILURE
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `hanaInstanceName`: name of the HANA instance.
- `version`: version information.
- `type`: specific HANA database type (*not* the service channel type).

Errors:

- RUNTIME_FAILURE (500): the list of available HANA database instances could not be retrieved.

Back to Top [page 1883]

Get HANA Service Channel

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/HANA/<id>`
Method	*GET*
Request	
Response	`{id, hanaInstanceName, instanceNumber, type, port, enabled, connections, state, comment}`
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

:

- `id`: unique identifier for the service channel (a positive integer number, starting with 1). This identifier is unique across all types of service channels.
- `hanaInstanceName`: name of the HANA instance (a string).
- `instanceNumber`: instance number.
- `type`: string 'HANA'.
- `port`: port of the HANA service channel (a number).
- `enabled`: boolean flag indicating whether the channel is enabled and therefore should be open.
- `connections`: maximal number of open connections.
- `state`: current connection state; this property is only available if the channel is enabled (as per property `enabled`). The value of this property is an object with the properties:
  - `connected` (a boolean flag indicating whether the channel is connected),
  - `openedConnections` (the number of open, possibly idle connections), and
  - `connectedSinceTimeStamp` (the time stamp, a UTC long number, for the first time the channel was opened/connected).
- `comment`: comment or short description; this property is not supplied if no comment was provided.

Errors:

- NOT_FOUND (404): the HANA service channel with the given ID (that is, <id>) or the specified subaccount does not exist.

Back to Top [page 1883]

Create HANA Service Channel (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/HANA`
Method	*POST*
Request	`{hanaInstanceName, instanceNumber, connections}`
Response	201 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

Errors:

- INVALID_REQUEST (400): invalid or out of range values.

Back to Top [page 1883]

Replace HANA Service Channel (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/HANA/<id>`
Method	*PUT*
Request	`{hanaInstanceName, instanceNumber, connections}`
Response	204 on success
Errors	INVALID_REQUEST, NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

- `hanaInstanceName`: name of the HANA instance (a string).
- `instanceNumber`: instance number.
- `connections`: maximal number of open connections.

Errors:

- INVALID_REQUEST (400): invalid or out of range values.
- NOT_FOUND (404): the HANA service channel with the given ID (that is, <id>) or the specified subaccount does not exist.

Back to Top [page 1883]

Get all Service Channels for Virtual Machines

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/VirtualMachine`
Method	*GET*
Request	
Response	`[{id, name, endpointId, type, port, enabled, connections, state}]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `id`: unique identifier for the service channel (a positive integer number, starting with 1). This identifier is unique across all types of service channels.
- `name`: name of the virtual machine on the cloud platform (a string).
- `endpointId`: unique ID (GUID) of the virtual machine (a string).
- `type`: string 'VirtualMachine'.
- `port`: port of the service channel for the virtual machine (a number).
- `enabled`: boolean flag indicating whether the channel is enabled and therefore should be open.
- `connections`: maximal number of open connections.
- `state`: current connection state; this property is only available if the channel is enabled (as per property `enabled`). The value of this property is an object with the properties:
  - `connected` (a boolean flag indicating whether the channel is connected),

- `openedConnections` (the number of open, possibly idle connections), and
- `connectedSinceTimeStamp` (the time stamp, a UTC long number, for the first time the channel was opened/connected).

Back to Top [page 1883]

Get Available Channels for Virtual Machines

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/availableChannels/VirtualMachine`
Method	*GET*
Request	
Response	`[{endpointId, name}]`
Errors	RUNTIME_FAILURE
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `name`: name of the virtual machine on the cloud platform (a string).
- `endpointId`: unique ID (GUID) of the virtual machine (a string).

Errors:

- RUNTIME_FAILURE (500): the list of available virtual machines could not be retrieved.

Back to Top [page 1883]

Get Service Channel for a Virtual Machine

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/VirtualMachine/<id>`

Method	*GET*
Request	
Response	`{id, name, endpointId, type, port, enabled, connections, state}`
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `id`: unique identifier for the service channel (a positive integer number, starting with 1). This identifier is unique across all types of service channels.
- `name`: name of the virtual machine on the cloud platform (a string).
- `endpointId`: unique ID (GUID) of the virtual machine (a string).
- `type`: string 'VirtualMachine'.
- `port`: port of the service channel for the virtual machine (a number).
- `enabled`: boolean flag indicating whether the channel is enabled and therefore should be open.
- `connections`: maximal number of open connections.
- `state`: current connection state; this property is only available if the channel is enabled (as per property `enabled`). The value of this property is an object with the properties:
  - `connected` (a boolean flag indicating whether the channel is connected),
  - `openedConnections` (the number of open, possibly idle connections), and
  - `connectedSinceTimeStamp` (the time stamp, a UTC long number, for the first time the channel was opened/connected).

Errors:

- NOT_FOUND (404): the service channel for a virtual machine with the given ID (that is, <id>) or the specified subaccount does not exist.

Back to Top [page 1883]

Create a Service Channel for a Virtual Machine (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/VirtualMachine`
Method	*POST*

Request	`{name, endpointId, port, connections}`
Response	201 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

- `name`: name of the virtual machine on the cloud platform (a string).
- `endpointId`: unique ID (GUID) of the virtual machine (a string).
- `port`: port of the service channel for the virtual machine (a number).
- `connections`: maximal number of open connections.

Errors:

- INVALID_REQUEST (400): invalid or out of range values.

Back to Top [page 1883]

Replace Service Channel for a Virtual Machine (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/VirtualMachine/<id>`
Method	*PUT*
Request	`{name, endpointId, connections}`
Response	204 on success
Errors	INVALID_REQUEST, NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

- `name`: name of the virtual machine on the cloud platform (a string).
- `endpointId`: unique ID (GUID) of the virtual machine (a string).
- `port`: port of the service channel for the virtual machine (a number).

- `connections`: maximal number of open connections.

Errors:

- INVALID_REQUEST (400): invalid or out of range values.
- NOT_FOUND (404): the service channel for a virtual machine with the given ID (that is, <id>) or the specified subaccount does not exist.

Back to Top [page 1883]

Get all ABAP Cloud Service Channels without SNC

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/ABAPCloud`
Method	*GET*
Request	
Response	`[{id, abapCloudTenantHost, instanceNumber, type, port, enabled, connections, state, comment}]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

An array of objects, each of which represents an ABAP Cloud service channel through the following properties:

- `id`: unique identifier for the service channel (a positive integer number, starting with 1). This identifier is unique across all types of service channels.
- `abapCloudTenantHost`: the host name (a string).
- `instanceNumber`: instance number.
- `type`: string 'ABAPCloud'.
- `port`: port of the ABAPCloud service channel (a number).
- `enabled`: boolean flag indicating whether the channel is enabled and therefore should be open.
- `connections`: maximal number of open connections.
- `state`: current connection state; this property is only available if the channel is enabled (as per property `enabled`). The value of this property is an object with the properties:
  - `connected` (a boolean flag indicating whether the channel is connected),
  - `openedConnections` (the number of open, possibly idle connections), and

    - `connectedSinceTimeStamp` (the time stamp, a UTC long number, for the first time the channel was opened/connected).
- `comment`: optional comment or short description. This property is not supplied if no comment was provided.

Back to Top [page 1883]

Get ABAP Cloud Service Channel without SNC

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/ABAPCloud/<id>`
Method	*GET*
Request	
Response	`{id, abapCloudTenantHost, instanceNumber, type, port, enabled, connections, state, comment}`
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `id`: unique identifier for the service channel (a positive integer number, starting with 1). This identifier is unique across all types of service channels.
- `abapCloudTenantHost`: the host name (a string).
- `instanceNumber`: instance number.
- `type`: string 'ABAPCloud'.
- `port`: port of the ABAPCloud service channel (a number).
- `enabled`: boolean flag indicating whether the channel is enabled and therefore should be open.
- `connections`: maximal number of open connections.
- `state`: current connection state; this property is only available if the channel is enabled (as per property `enabled`). The value of this property is an object with the properties:
    - `connected` (a boolean flag indicating whether the channel is connected),
    - `openedConnections` (the number of open, possibly idle connections), and
    - `connectedSinceTimeStamp` (the time stamp, a UTC long number, for the first time the channel was opened/connected).
- `comment`: optional comment or short description. This property is not supplied if no comment was provided.

Errors:

- NOT_FOUND (404): the ABAP Cloud service channel with the given ID (that is, <id>) or the specified subaccount does not exist.

Back to Top [page 1883]

Create ABAP Cloud Service Channel without SNC (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/ABAPCloud`
Method	*POST*
Request	`{abapCloudTenantHost, instanceNumber, connections, comment}`
Response	201 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

- `abapCloudTenantHost`: host name (a string).
- `instanceNumber`: instance number.
- `connections`: maximal number of open connections.
- `comment`: optional comment or short description (a string)

Errors:

- INVALID_REQUEST (400): invalid or out of range values.

Back to Top [page 1883]

Replace ABAP Cloud Service Channel without SNC (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/ABAPCloud/<id>`
Method	*PUT*
Request	`{abapCloudTenantHost, instanceNumber, connections, comment}`
Response	204 on success
Errors	INVALID_REQUEST, NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

- `abapCloudTenantHost`: host name (a string).
- `instanceNumber`: instance number.
- `connections`: maximal number of open connections.
- `comment`: optional comment or short description (a string).

Errors:

- INVALID_REQUEST (400): invalid or out of range values.
- NOT_FOUND (404): the ABAP Cloud service channel with the given ID (that is, <id>) or the specified subaccount does not exist.

Back to Top [page 1883]

Get all ABAP Cloud SNC-Encrypted Service Channels

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/ABAPCloudSNC`
Method	*GET*
Request	
Response	`[{id, abapCloudTenantHost, instanceNumber, type, port, enabled, connections, state, comment}]`

Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

An array of objects, each of which represents an ABAP Cloud service channel (SNC-encrypted) through the following properties:

- `id`: unique identifier for the service channel (a positive integer number, starting with 1). This identifier is unique across all types of service channels.
- `abapCloudTenantHost`: the host name (a string).
- `instanceNumber`: instance number.
- `type`: string 'ABAPCloudSNC.
- `port`: port of the ABAPCloud service channel (a number).
- `enabled`: boolean flag indicating whether the channel is enabled and therefore should be open.
- `connections`: maximal number of open connections.
- `state`: current connection state; this property is only available if the channel is enabled (as per property `enabled`). The value of this property is an object with the properties:
  - `connected` (a boolean flag indicating whether the channel is connected),
  - `openedConnections` (the number of open, possibly idle connections), and
  - `connectedSinceTimeStamp` (the time stamp, a UTC long number, for the first time the channel was opened/connected).
- `comment`: optional comment or short description. This property is not supplied if no comment was provided.

Back to Top [page 1883]

Get ABAP Cloud SNC-Encrypted Service Channel

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/ABAPCloudSNC/<id>`
Method	*GET*
Request	
Response	`{id, abapCloudTenantHost, instanceNumber, type, port, enabled, connections, state, comment}`

Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `id`: unique identifier for the service channel (a positive integer number, starting with 1). This identifier is unique across all types of service channels.
- `abapCloudTenantHost`: the host name (a string).
- `instanceNumber`: instance number.
- `type`: string 'ABAPCloudSNC.
- `port`: port of the ABAPCloud service channel (a number).
- `enabled`: boolean flag indicating whether the channel is enabled and therefore should be open.
- `connections`: maximal number of open connections.
- `state`: current connection state; this property is only available if the channel is enabled (as per property `enabled`). The value of this property is an object with the properties:
  - `connected` (a boolean flag indicating whether the channel is connected),
  - `openedConnections` (the number of open, possibly idle connections), and
  - `connectedSinceTimeStamp` (the time stamp, a UTC long number, for the first time the channel was opened/connected).
- `comment`: optional comment or short description. This property is not supplied if no comment was provided.

Errors:

- NOT_FOUND (404): the ABAP Cloud service channel with the given ID (that is, <id>) or the specified subaccount does not exist.

Back to Top [page 1883]

Create ABAP Cloud SNC-Encrypted Service Channel (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/ABAPCloudSNC`
Method	*POST*
Request	`{abapCloudTenantHost, instanceNumber, connections, comment}`

Response	201 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

- `abapCloudTenantHost`: host name (a string).
- `instanceNumber`: instance number.
- `connections`: maximal number of open connections.
- `comment`: optional comment or short description (a string).

Errors:

- INVALID_REQUEST (400): invalid or out of range values.

Back to Top [page 1883]

Replace ABAP Cloud SNC-Encrypted Service Channel (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/ABAPCloudSNC/<id>`
Method	*PUT*
Request	`{abapCloudTenantHost, instanceNumber, connections, comment}`
Response	204 on success
Errors	INVALID_REQUEST, NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

- `abapCloudTenantHost`: host name (a string).
- `instanceNumber`: instance number.
- `connections`: maximal number of open connections.
- `comment`: optional comment or short description (a string).

Errors:

- INVALID_REQUEST (400): invalid or out of range values.
- NOT_FOUND (404): the ABAP Cloud service channel with the given ID (that is, <id>) or the specified subaccount does not exist.

Back to Top [page 1883]

Get All Kubernetes Cluster Service Channels

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/K8S`
Method	*GET*
Request	
Response	`[{id, type, port, k8sCluster, k8sService, enabled, connections, state, comment}]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

An array of objects, each of which represents a service channel for a virtual machine through the following properties:

- `id`: unique identifier for the service channel (a positive integer number, starting with 1); this identifier is unique across all types of service channels.
- `k8sCluster`: host name to access the Kubernetes cluster (a string).
- `k8sService`: host name providiing the service inside of Kubernetes cluster (a string).
- `type`: the string *'K8S'*.
- `port`: port of the service channel for the virtual machine (a number).
- `enabled`: Boolean flag indicating whether the channel is enabled and therefore should be open.
- `connections`: maximal number of open connections.
- `state`: current connection state; this property is only available if the channel is enabled (as per property `enabled`). The value of this property is an object with the properties `connected` (a Boolean flag indicating whether the channel is connected), `openedConnections` (the number of open, possibly idle connections), and `connectedSinceTimeStamp` (the time stamp, a UTC long number, for the first time the channel was opened/connected).
- `comment`: comment or short description; this property is not supplied if no comment was provided.

Back to Top [page 1883]

Get Kubernetes Service Channel

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/K8S/<id>`
Method	*GET*
Request	
Response	`[{id, type, port, k8sCluster, k8sService, enabled, connections, state, comment}]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `id`: unique identifier for the service channel (a positive integer number, starting with 1); this identifier is unique across all types of service channels.
- `k8sCluster`: host name to access the Kubernetes cluster (a string).
- `k8sService`: host name providiing the service inside of Kubernetes cluster (a string).
- `type`: the string *'K8S'*.
- `port`: port of the service channel for the virtual machine (a number).
- `enabled`: Boolean flag indicating whether the channel is enabled and therefore should be open.
- `connections`: maximal number of open connections.
- `state`: current connection state; this property is only available if the channel is enabled (as per property `enabled`). The value of this property is an object with the properties `connected` (a Boolean flag indicating whether the channel is connected), `openedConnections` (the number of open, possibly idle connections), and `connectedSinceTimeStamp` (the time stamp, a UTC long number, for the first time the channel was opened/connected).
- `comment`: comment or short description; this property is not supplied if no comment was provided.

Back to Top [page 1883]

Create Kubernetes Service Channel (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/K8S`
Method	*POST*
Request	`{k8sCluster, k8sService, port, connections, comment}`
Response	201
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

- `k8sCluster`: Kubernetes cluster host name, optionally with port separated by colon (a string).
- `k8sService`: service host inside the Kubernetes cluster (a string).
- `port`: local port.
- `connections`: maximal number of open connections.
- `comment`: optional comment or short description (a string).

Errors:

- INVALID_REQUEST (400): invalid or out-of-range values.

Back to Top [page 1883]

Replace Kubernetes Service Channel (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/K8S/<id>`
Method	*PUT*
Request	`{k8sCluster, k8sService, port, connections, comment}`
Response	INVALID_REQUEST, NOT_FOUND
Errors	

Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

- `k8sCluster`: Kubernetes cluster host name, optionally with port separated by colon (a string).
- `k8sService`: service host inside the Kubernetes cluster (a string).
- `port`: local port.
- `connections`: maximal number of open connections.
- `comment`: optional comment or short description (a string).

Errors:

- INVALID_REQUEST (400): invalid or out-of-range values.
- NOT_FOUND (404): the Kyma service channel with the given ID (that is, `<id>`) or the specified subaccount does not exist.

Back to Top [page 1883]

Enable/Disable Service Channel (Master Only)

Use one of the following channel types to replace <type> in the URI: HANA, VirtualMachine, or ABAPCloud.

> ⚠ Caution
>
> The URI variant without <type> still works, but it is obsolete and may be removed in a future release. We recommend that you move to using <type> as soon as possible.

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/<type>/<id>/state`
Method	*PUT*
Request	`{enabled}`
Response	
Errors	NOT_FOUND, RUNTIME_FAILURE
Roles	Administrator, Associate Administrator, Subaccount Administrator

Errors:

- NOT_FOUND (404): service channel (or subaccount) does not exist.
- RUNTIME_FAILURE (500): service channel could not be opened (when attempting to set `enabled:true`).

Back to Top [page 1883]

Delete Service Channel (Master Only)

> ⚠ Caution
>
> The URI variant without <type> still works, but it is obsolete and may be removed in a future release. We recommend to move to using <type> as soon as possible.

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/<type>/<id>`
Method	*DELETE*
Request	
Response	204 on success
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator

Errors:

- NOT_FOUND (404): service channel (or subaccount) does not exist.

Back to Top [page 1883]

Delete all Service Channels (Master Only)

> ⓘ Note
>
> Omit <type> to delete all service channels, regardless of the type. To delete all service channels of a particular type, replace <type> with one of the following channel types: HANA, VirtualMachine, or ABAPCloud.

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels[/<type>]`
Method	*DELETE*
Request	
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Back to Top [page 1883]

Get Service Channels

> ⚠ Caution
>
> **Obsolete**. This API is deprecated and may be removed in a future release. Use the getters for the specific service channel type (that is, HANA (database), Virtual Machine, or ABAP Cloud) which provide properties tailored for the respective channel type.

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels`
Method	*GET*
Request	
Response	`[{typeDesc, details, port, enabled, connected, connectionsCount, availableConnectionsCount, connectedSinceTimeStamp}]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

An array of objects, each of which represents a service channel through the following properties:

- `typeDesc`: an object specifying the service channel type through the properties `typeKey` and `typeName`.

- `details`
- `port`
- `enabled`
- `connected`
- `connectionsCount`
- `availableConnectionsCount`
- `connectedSinceTimeStamp`

Back to Top [page 1883]

Get Service Channel

> ⚠ Caution
>
> **Obsolete**. This API is deprecated and may be removed in a future release. Use the getters for the specific service channel type (that is, HANA (database), Virtual Machine, or ABAP Cloud) which provide properties tailored for the respective channel type.

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/<id>`
Method	*GET*
Request	
Response	`[{typeDesc, details, port, enabled, connected, connectionsCount, availableConnectionsCount, connectedSinceTimeStamp}]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `typeDesc`: an object specifying the service channel type through the properties `typeKey` and `typeName`.
- `details`
- `port`
- `enabled`
- `connected`
- `connectionsCount`
- `availableConnectionsCount`

- `connectedSinceTimeStamp`

Back to Top [page 1883]

Create Service Channel

> ⚠ Caution
>
> **Obsolete**. This API is deprecated and may be removed in a future release. Create a service channel using the API for the respective channel type, that is, HANA (database), Virtual Machine, or ABAP Cloud.

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels`
Method	*POST*
Request	`{typeKey,details,serviceNumber,connectionCount}`
Response	201 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request Properties:

- `typeKey`: type of service channel. Valid values are HANA_DB, HCPVM, RFC.
- `details`:
  - HANA instance name for HANA_DB
  - VM name for HCPVM
  - S/4HANA Cloud tenant host for RFC
- `serviceNumber`: service number, which is mapped to a port according to the type of service channel.
- `connectionCount`: number of connections for the channel.

Back to Top [page 1883]

Replace Service Channel

> ⚠ Caution
>
> **Obsolete**. This API is deprecated and may be removed in a future release. Replace a service channel using the API for the respective channel type, that is, HANA (database), Virtual Machine, or ABAP Cloud.

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/channels/<id>`
Method	*PUT*
Request	`{typeKey,details,serviceNumber,connectionCount}`
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Back to Top [page 1883]

1.4.2.5.14 Access Control Entities

Manage RFC-specific access control entities for the Cloud Connector via API.

Get List Of Allowed Clients [page 1908]

Get List Of Blocked Client/User Pairs [page 1908]

Create Allowed Clients (Master Only) [page 1909]

Extend Allowed Clients (Master Only) [page 1909]

Create Blocked Client/User Pairs (Master Only) [page 1910]

Extend Blocked Client/User Pairs (Master Only) [page 1911]

Delete All Allowed Clients (Master Only) [page 1911]

Delete All Blocked Client/User Pairs (Master Only) [page 1912]

Remove an Item from the List of Allowed Clients (Master Only) [page 1912]

Remove an Item from the List of Blocked Client/User Pairs (Master Only) [page 1913]

Get List Of Allowed Clients

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/allowedClients`
Method	*GET*
Request	
Response	`[{client}]`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

List of objects, each representing an allowed client:

- `client`: client name

Back to Top [page 1907]

Get List Of Blocked Client/User Pairs

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/blockedClientUsers`
Method	*GET*
Request	
Response	`[{client, user}]`
Errors	

Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

List of objects, each representing a blocked client and user:

- `client`: client name
  `user`: user name

Back to Top [page 1907]

Create Allowed Clients (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/allowedClients`
Method	*POST*
Request	`[client]`
Response	201 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

List of strings, each representing an allowed client.

Back to Top [page 1907]

Add Allowed Clients (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/allowedClients`
Method	*PATCH*
Request	`[client]`
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

List of strings, each representing an allowed client.

Back to Top [page 1907]

Create Blocked Client/User Pairs (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/blockedClientUsers`
Method	*POST*
Request	`[{client, user}]`
Response	201 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

List of objects, each representing a blocked client and user:

- `client`: ABAP client
- `user`: ABAP user

Back to Top [page 1907]

Add Blocked Client/User Pairs (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/blockedClientUsers`
Method	*PATCH*
Request	`[{client, user}]`
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

List of objects, each representing a blocked client and user:

- `client`: ABAP client
- `user`: ABAP user

Back to Top [page 1907]

Delete All Allowed Clients (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/allowedClients`
Method	*DELETE*
Request	
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Back to Top [page 1907]

Delete All Blocked Client/User Pairs (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/blockedClientUsers`
Method	*DELETE*
Request	
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator

Back to Top [page 1907]

Remove an Item from the List of Allowed Clients (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/allowedClients/<client>`
Method	*DELETE*
Request	
Response	204 on success
Errors	NOT_FOUND
Roles	Administrator, Subaccount Administrator

Back to Top [page 1907]

Remove an Item from the List of Blocked Client/User Pairs (Master Only)

URI	`/api/v1/configuration/subaccounts/<regionHost>/<subaccount>/systemMappings/<virtualHost>:<virtualPort>/blockedClientUsers/<client>:<user>`
Method	*DELETE*
Request	
Response	204 on success
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator

Back to Top [page 1907]

1.4.2.5.15 Common On-Premises Connectivity Configuration

Read the Cloud Connector's common on-premises connectivity settings via API.

Get Principal Propagation Configuration

URI	`/api/v1/configuration/connector/onPremises/ppSettings`
Method	*GET*
Request	
Response	`{expirationToleranceInHr, certificateValidityInMin}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `expirationToleranceInHr`: Integer - duration in hours a principal remains usable after the token has expired
- `certificateValidityInMin`: Integer - duration in minutes a certificate for principal propagation can be used for backend authentication

Sample Code

Sample Output

```
{
  "expirationToleranceInHr": 1,
  "certificateValidityInMin": 60
}
```

Sample Code

```
curl -i -k -H "Accept: application/json" -u <user>:<password> -X GET https://
<host>:<port>/api/v1/configuration/connector/onPremises/ppSettings
```

Set Principal Propagation Configuration

URI	`/api/v1/configuration/connector/onPremises/ppSettings`
Method	*PUT*
Request	`{expirationToleranceInHr, certificateValidityInMin}`
Response	204 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

- `expirationToleranceInHr`: Integer - duration in hours a principal remains usable after the token has expired
- `certificateValidityInMin`: Integer - duration in minutes a certificate for principal propagation can be used for backend authentication

Errors:

- `INVALID_REQUEST`:
  - empty input
  - values not set
  - invalid values
  - `expirationToleranceInHr` is not within the range 1-336 (inclusive) (14 days)
  - `certificateValidityInMin` is not within the range 1-43200 (inclusive) (30 days)

Sample Code

Sample Input

```
{
  "expirationToleranceInHr": 1,
  "certificateValidityInMin": 60
}
```

Sample Code

```
curl -i -k -H "Content-Type: application/json" -u <user>:<password> -X PUT
https://<host>:<port>/api/v1/configuration/connector/onPremises/ppSettings -d
"{\"expirationToleranceInHr\": 1, \"certificateValidityInMin\": 60}"
```

Modify Principal Propagation Configuration

URI	`/api/v1/configuration/connector/onPremises/ppSettings`
Method	*PATCH*
Request	`{<values to be changed>}`
Response	204 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator, Subaccount Administrator

Request:

One or several values of

- `expirationToleranceInHr`: Integer - duration in hours a principal remains usable after the token has expired
- `certificateValidityInMin`: Integer - duration in minutes a certificate for principal propagation can be used for backend authentication

Errors:

- `INVALID_REQUEST`:
  - empty input
  - values not set
  - invalid values
  - `expirationToleranceInHr` is not within the range 1-336 (inclusive) (14 days)
  - `certificateValidityInMin` is not within the range 1-43200 (inclusive) (30 days)

Sample Code

```
curl -i -k -H "Content-Type: application/json" -u <user>:<password> -X PATCH
https://<host>:<port>/api/v1/configuration/connector/onPremises/ppSettings
```

1.4.2.5.16 Audit Log Configuration

Read and edit the Cloud Connector's audit log settings via API.

Get Cross-Subaccount Audit Log Configuration

URI	`/api/v1/configuration/connector/auditLog`
Method	*GET*
Request	
Response	`{level, cleanup, logPath}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

Current cross-subaccount audit log configuration.

- `level`: OFF, SECURITY, ALL
- `cleanup`: `0` (logs will be retained indefinitely), or number of days the audit logs will be retained.
- `logPath`: path for audit logs

Example

```
curl -i -k -u <user>:<password> https://localhost:8443/api/v1/configuration/
connector/auditLog
```

Change Cross-Subaccount Audit Log Configuration

URI	`/api/v1/configuration/connector/auditLog`

Method	*PATCH*
Request	`{level, cleanup}`
Response	
Errors	
Roles	Administrator

Request:

Current subaccount independent audit log configuration.

- `level`: OFF, SECURITY, ALL
- `cleanup`: use **`0`** to never remove logs; otherwise 14, 30, 90,180, or 365 to retain logs for the respective number of days.

> ### Note
>
> `logPath` cannot be changed on a running Cloud Connector. To change the audit log path, use the script `changeAuditLogPath` located in your Cloud Connector installation directory.

Example

```
curl -i -k -u <user>:<password> -X PATCH https://localhost:8443/api/v1/
configuration/connector/auditLog --data '{"level"="SECURITY"}'
```

Get Subaccount-Related Audit Log Configuration

URI	`/api/v1/configuration/connector/auditLog/subaccounts/<regionHost>/<subaccount>`
Method	*GET*
Request	
Response	`{level}`

Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `level`: OFF, SECURITY, ALL

Example

```
curl -i -k -u <user>:<password> https://localhost:8443/api/v1/configuration/
connector/auditLog/subaccounts/<regionHost>/<subaccount>
```

Change Subaccount-Related Audit Log Configuration

URI	`/api/v1/configuration/connector/auditLog/subaccounts/<regionHost>/<subaccount>`
Method	*PATCH*
Request	`{level}`
Response	
Errors	
Roles	Administrator

Request:

- `level`: OFF, SECURITY, ALL

Example

```
curl -i -k -u <user>:<password> https://localhost:8443/api/v1/configuration/
connector/auditLog/subaccounts/<regionHost>/<subaccount> --data='{level="ALL"}'
```

1.4.2.5.17 Alerting Configuration

Read and edit the Cloud Connector's alerting settings via API.

Get Alerting Configuration

URI	`/api/v1/configuration/connector/alerting`
Method	*GET*
Request	
Response	`{thresholds, notification}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response Properties:

- `thresholds`: thresholds
  - `cpuLoad`: CPU load in percent
  - `cpuLoadViolationToleranceSec`: CPU load violation tolerance in seconds
  - `freeDiskSpaceMb`: free disc space in MB
  - `freeDiscForAuditLogMb`: (optional, only if external AuditLog is configured) free disc space on volume configured for external audit log
  - `freeDiscForTraceMb`: (optional, only if external Trace is configured) free disc space on volume configured for external trace
  - `certificateValidityDays`: End of validity period of certificates in days until expiration
- `notification`: Alerts categories to be reported via configured e-mail
  - `ha`: high availability: alerts related to HA communication and state management
  - `tunnels`: alerts related to SAP BTP cloud connectivity
  - `serviceChannels`: service channels
  - `CPU`: alerts related to CPU load
  - `freeDiskSpace`: alerts related to exhausted disk space
  - `certificates`: alerts related to expiring certificates
  - `troubleshooting`: alerts about new troubleshooting diagnose

Example

```
curl -i -k -H 'Accept:application/json' -u <user>:<password> -X GET https://
<host>:<port>/api/v1/configuration/connector/alerting
```

Set Alerting Configuration

URI	`/api/v1/configuration/connector/alerting`
Method	*PUT*
Request	`{thresholds, notification}`
Response	
Errors	
Roles	Administrator

Request Properties:

- `thresholds`: thresholds
  - `cpuLoad`: CPU load in percent
  - `cpuLoadViolationToleranceSec`: CPU load violation tolerance in seconds
  - `freeDiskSpaceMb`: free disc space in MB
  - `freeDiscForAuditLogMb`: (optional, only if external AuditLog is configured) free disc space on volume configured for external audit log
  - `freeDiscForTraceMb`: (optional, only if external Trace is configured) free disc space on volume configured for external trace
  - `certificateValidityDays`: End of validity period of certificates in days until expiration
- `notification`: Alerts categories to be reported via configured e-mail
  - `ha`: high availability: alerts related to HA communication and state management
  - `tunnels`: alerts related to SAP BTP cloud connectivity
  - `serviceChannels`: service channels
  - `CPU`: alerts related to CPU load
  - `freeDiskSpace`: alerts related to exhausted disk space
  - `certificates`: alerts related to expiring certificates
  - `troubleshooting`: alerts about new troubleshooting diagnose

Example

```
curl -i -k -u <user>:<password> -H 'Content-Type:application/json' -d
'{"notification":{<values>}, "thresholds":{<values>}}'
-X PUT https://<host>:<port>/api/v1/configuration/connector/alerting
```

Modify Alerting Configuration

URI	`/api/v1/configuration/connector/alerting`
Method	*PATCH*
Request	`{<values to be changed>}`
Response	
Errors	
Roles	Administrator

Request Properties:

One or several values of:

- `thresholds`: thresholds
  - `cpuLoad`: CPU load in percent
  - `cpuLoadViolationToleranceSec`: CPU load violation tolerance in seconds
  - `freeDiskSpaceMb`: free disc space in MB
  - `freeDiscForAuditLogMb`: (optional, only if external AuditLog is configured) free disc space on volume configured for external audit log
  - `freeDiscForTraceMb`: (optional, only if external Trace is configured) free disc space on volume configured for external trace
  - `certificateValidityDays`: End of validity period of certificates in days until expiration
- `notification`: Alerts categories to be reported via configured e-mail
  - `ha`: high availability: alerts related to HA communication and state management
  - `tunnels`: alerts related to SAP BTP cloud connectivity
  - `serviceChannels`: service channels
  - `CPU`: alerts related to CPU load
  - `freeDiskSpace`: alerts related to exhausted disk space
  - `certificates`: alerts related to expiring certificates
  - `troubleshooting`: alerts about new troubleshooting diagnose

Example

```
curl -i -k -u <user>:<password> -H 'Content-Type:application/json' -d
'{"notification":{"cpuLoad":90}}'
-X PATCH https://<host>:<port>/api/v1/configuration/connector/alerting
```

1.4.2.5.18 E-Mail Configuration

Read and edit the Cloud Connector's e-mail settings via API.

Get E-Mail Configuration

URI	`/api/v1/configuration/connector/email`
Method	*GET*
Request	
Response	`{enabled, trustOption, sendTo, sendCc, sendFrom, smtpHost, smtpPort, smtpUsername, additionalProperties}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `enabled`: Boolean - whether e-mails should be sent in case of an alert or not
- `trustOption`: String - one of the following levels of trust options for the e-mail server (see Configure Trust [page 1955]): 'tlsDisabled', 'emailTrustStore', 'jdkAndEmailTrustStore', 'trustAll'
- `sendTo`: String - e-mail addresses of the recipients in RFC 822 format
- `sendCc` (optional): String - e-mail addresses of the recipients that should get a copy of the e-mail in RFC 822 format
- `sendFrom`: String - e-mail address of the sender in RFC 822 format
- `smtpHost`: String - host of the mail server
- `smtpPort` (optional): String or Integer - port of the mail server
- `smtpUsername` (optional): String - username for authentication with the mail server
- `additionalProperties` (optional): Dictionary - additional properties that are supported by the java mail library

Sample Code

```
curl -i -k -H "Accept: application/json" -u <user>:<password> -X GET https://
<host>:<port>/api/v1/configuration/connector/email
```

Sample Code

Sample output: E-mail configuration (GET - without optional values)

```
{
  "enabled": true,
  "trustOption": "emailTrustStore",
  "sendTo": "recipient@corp.de",
  "sendFrom": "sender@othercorp.com",
  "smtpHost": "walldorf.corp.host"
}
```

Sample Code

Sample output: E-mail configuration (GET - with optional values)

```
{
  "enabled": true,
  "trustOption": "emailTrustStore",
  "sendTo": "recipient@corp.de",
  "sendCc": "John Doe <j.doe@company.com>, Jane Doe <j2.doe@company.com>",
  "sendFrom": "sender@othercorp.com",
  "smtpHost": "walldorf.corp.host",
  "smtpPort": 587,
  "smtpUsername": "username123",
  "additionalProperties": {
    "mail.smtp.someKey": "someValue"
  }
}
```

Set E-Mail Configuration

URI	`/api/v1/configuration/connector/email`
Method	*PUT*
Request	`{enabled, trustOption, sendTo, sendCc, sendFrom, smtpHost, smtpPort, smtpUsername, smtpPassword, additionalProperties}`

Response	204 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator

Request:

- `enabled`: Boolean - whether e-mails should be sent in case of an alert or not
- `trustOption`: String - one of the following levels of trust options for the e-mail server (see Configure Trust [page 1955]): 'tlsDisabled', 'emailTrustStore', 'jdkAndEmailTrustStore', 'trustAll'
- `sendTo`: String - e-mail addresses of the recipients in RFC 822 format
- `sendCc` (optional): String - e-mail addresses of the recipients that should get a copy of the e-mail in RFC 822 format
- `sendFrom`: String - e-mail address of the sender in RFC 822 format
- `smtpHost`: String - host of the mail server
- `smtpPort` (optional): String or Integer - port of the mail server
- `smtpUsername` (optional): String - username for authentication with the mail server
- `smtpPassword` (optional): String - password for authentication with the mail server
- `additionalProperties` (optional): Dictionary - additional properties that are supported by the java mail library

Errors:

- `INVALID_REQUEST`:
  - empty input
  - mandatory values are not set
  - invalid value for `trustOption`
  - format of the e-mail addresses is incorrect
  - format of host is wrong
  - port is not within the range 1-65535(inclusive)

> ⚠ Caution
>
> Validation of the configuration only happens if `enabled` is set to `true`.

> {·} Sample Code

```
curl -i -k -H "Content-Type: application/json" -u <user>:<password> -X PUT
https://<host>:<port>/api/v1/configuration/connector/email
```

Sample Code

Sample input: E-mail configuration (PUT- with optional values)

```
{
  "enabled": true,
  "trustOption": "emailTrustStore",
  "sendTo": "recipient@corp.de",
  "sendFrom": "sender@othercorp.com",
  "smtpHost": "walldorf.corp.host",
  "smtpPort": 587,
  "sendCc": "John Doe <j.doe@company.com>, Jane Doe <j2.doe@company.com>",
  "smtpUsername": "username123",
  "smtpPassword": "password123",
  "additionalProperties": {
    "mail.smtp.someKey": "someValue"
  }
}
```

Modify E-Mail Configuration

URI	`/api/v1/configuration/connector/email`
Method	*PATCH*
Request	`{<values to be changed>}`
Response	204 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator

Request:

One or several values of

- `enabled`: Boolean - whether e-mails should be sent in case of an alert or not
- `trustOption`: String - one of the following levels of trust options for the e-mail server (see Configure Trust [page 1955]): 'tlsDisabled', 'emailTrustStore', 'jdkAndEmailTrustStore', 'trustAll'
- `sendTo`: String - e-mail addresses of the recipients in RFC 822 format
- `sendCc` (optional): String - e-mail addresses of the recipients that should get a copy of the e-mail in RFC 822 format
- `sendFrom`: String - e-mail address of the sender in RFC 822 format
- `smtpHost`: String - host of the mail server
- `smtpPort` (optional): String or Integer - port of the mail server

- `smtpUsername` (optional): String - username for authentication with the mail server
- `smtpPassword` (optional): String - password for authentication with the mail server
- `additionalProperties` (optional): Dictionary - additional properties that are supported by the java mail library

Errors:

- `INVALID_REQUEST`:
  - empty input
  - mandatory values are not set
  - invalid value for `trustOption`
  - format of the e-mail addresses is incorrect
  - format of host is wrong
  - port is not within the range 1-65535(inclusive)

> ⚠ Caution
>
> Validation of the configuration only happens if `enabled` is set to `true`.

> {·} Sample Code

```
curl -i -k -H "Content-Type: application/json" -u <user>:<password> -X PATCH
https://<host>:<port>/api/v1/configuration/connector/email
```

Get Truststore Information

URI	`/api/v1/configuration/connector/email/truststore`
Method	*GET*
Request	
Response	`{trustedEmailCertificates}`
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Response:

- `trustedEmailCertificates`: A list of trusted e-mail server certificates with the following properties:

Certificate Properties:

- `alias`: String - the alias of a trusted certificate (a string)
- `subjectDN`: String - the subject DN (a string)
- `issuer`: String - the issuer (a string)
- `notAfterTimeStamp`: Long - the end date of the certificate's validity period, represented as the number of milliseconds since January 1, 1970, 00:00:00 GMT

Sample Code

```
curl -i -k -H "Accept: application/json" -u <user>:<password> -X GET https://
<host>:<port>/api/v1/configuration/connector/email/truststore
```

Sample Code

Sample output: Trusted e-mail certificates (GET Response)

```
{
  "trustedEmailCertificates": [
    {
      "alias": "trustedemailserver.1.1",
      "subjectDN": "CN=*.example.com, OU=IT, O=Example Corp, C=US",
      "issuer": "CN=Example Root CA, O=Example Corp, C=US",
      "notAfterTimeStamp": 2145916801000
    },
    {
      "alias": "trustedemailserver.1.2",
      "subjectDN": "CN=mail.internal.example.org, O=Example Org, C=US",
      "issuer": "CN=Example Intermediate CA, O=Example Org, C=US",
      "notAfterTimeStamp": 1966607187000
    },
    {
      "alias": "trustedemailserver.1.3",
      "subjectDN": "CN=selfsigned.example.local, O=Internal Systems, C=US",
      "issuer": "CN=selfsigned.example.local, O=Internal Systems, C=US",
      "notAfterTimeStamp": 2044533167000
    }
  ]
}
```

Get Single Truststore Certificate by Aalias

URI	`/api/v1/configuration/connector/email/truststore/certificates/<alias>`

Method	*GET*
Header	Accept: application/x-pem-file Accept: application/pkix-cert
Request	
Response	certificate in the specified format
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Request Header:

- `Accept`: application/x-pem-file - requests the certificate in PEM format (Base64-encoded)
- `Accept`: application/pkix-cert - requests the certificate in DER format (binary format)

Errors:

- `NOT_FOUND`): if the certificate with the specified alias could not be found

Sample Code

```
curl -i -k -H "Accept: application/x-pem-file" -u <user>:<password> -X GET
--output <pemFile> https://<host>:<port>/api/v1/configuration/connector/email/
truststore/certificates/<alias>
curl -i -k -H "Accept: application/pkix-cert"  -u <user>:<password> -X GET
--output <derFile> https://<host>:<port>/api/v1/configuration/connector/email/
truststore/certificates/<alias>
```

Get All Truststore Certificates

URI	`/api/v1/configuration/connector/email/truststore/certificates`
Method	*GET*
Header	Accept: application/x-pem-file Accept: application/pkix-cert
Request	

Response	certificates in the specified format
Errors	
Roles	Administrator, Associate Administrator, Subaccount Administrator, Display, Support, Monitoring

Request Header:

- `Accept`: application/x-pem-file - requests the certificate in PEM format (Base64-encoded)
- `Accept`: application/pkix-cert - requests the certificate in DER format (binary format)

Sample Code

```
curl -i -k -H "Accept: application/x-pem-file" -u <user>:<password> -X GET
--output <pemFile> https://<host>:<port>/api/v1/configuration/connector/email/
truststore/certificates
curl -i -k -H "Accept: application/pkix-cert"  -u <user>:<password> -X GET
--output <derFile> https://<host>:<port>/api/v1/configuration/connector/email/
truststore/certificates
```

Create a Truststore Certificate

URI	`/api/v1/configuration/connector/email/truststore/certificates`
Method	*POST*
Header	CONTENT_TYPE: multipart/form-data
Request	multipart/form-data with the following form parameter: `certificates`
Response	201 on success
Errors	INVALID_REQUEST
Roles	Administrator, Associate Administrator

Request Properties:

- `certificates`: one or multiple certificates in DER or PEM format

Errors:

- `INVALID_REQUEST`: if no or a duplicate certificate was provided

Sample Code

```
curl -i -k -u <user>:<password> -X POST -F "certificates=@<file>" https://
<host>:<port>/api/v1/configuration/connector/email/truststore/certificates
```

Delete Single Truststore Certificate by Alias

URI	`/api/v1/configuration/connector/email/truststore/certificates/<alias>`
Method	*DELETE*
Request	
Response	204 on success
Errors	NOT_FOUND
Roles	Administrator, Associate Administrator

Errors:

- `NOT_FOUND`: if the certificate with the specified alias could not be found

Sample Code

```
curl -i -k -u <user>:<password> -X DELETE https://<host>:<port>/api/v1/
configuration/connector/email/truststore/certificates/<alias>
```

Delete All Truststore Certificates

URI	`/api/v1/configuration/connector/email/truststore/certificates`
Method	*DELETE*

Request	
Response	204 on success
Errors	
Roles	Administrator, Associate Administrator

Sample Code

```
curl -i -k -u <user>:<password> -X DELETE https://<host>:<port>/api/v1/
configuration/connector/email/truststore/certificates
```

1.4.2.5.19 Examples

Find examples on how to use the Cloud Connector's configuration REST APIs.

Concept

The sample code in this section (download zip file) demonstrates how to use the REST APIs provided by Cloud Connector to perform various configuration tasks.

Starting with a freshly installed Cloud Connector, the samples include initial configuration of the Cloud Connector instance, connectivity setup, high availability configuration, and common tasks like backup/restore operations, as well as integration with solution management.

The examples are implemented in *Kotlin*, a simple Java VM-based language. However, even if you are using a different language, they still show the basic use of the APIs and their parameters for specific configuration purposes.

If you are not familiar with Kotlin, find a brief introduction and some typical statements below.

REST API Parameters [page 1933]

REST API Invocation [page 1933]

How To Use the Examples [page 1934]

REST API Parameters

In almost all requests and responses, structures are encoded in JSON format. To describe the parameter details, we use Kotlin data classes.

This class represents a structure that you can use as value in a request or response:

```
data class OnlyPropertiesNamesAreRelevant(
    val user: String,
    val password: String
)
```

The JSON representation for that class is

```
{"user":<userValue>, "password":<passwordValue>}
```

Encoded in Kotlin this string looks like

```
"""{"user":"$userValue", "password":"$passwordValue"}"""
```

or as an object:

```
val credentials = OnlyPropertiesNamesAreRelevant(userValue, passwordValue)
```

Back to Concept [page 1932]

REST API Invocation

```
(a)    Fuel.put(url)
(b)     .header("Connection", "close")
(c)        .authentication().basic(user, password)
(d)        .jsonBody(credentials)
(e)        .responseObject<OnlyPropertiesNamesAreRelevant> { _, _, result ->
            when (result) {
(f)                is Result.Failure -> processRequestError(result.error)
                   is Result.Success -> println("returned: ${result.get()}")
            }
        }
        .join()
```

- (a) - *Fuel* is an HTTP framework used in the examples. Important is the verb after *Fuel* - it is the REST API method.
- (b) - Adds a request header `Connection: close`, which forces a connection close after request. In the examples, this header is defined on *FuelManager* for all calls.

- (c) - Basic authentication is used for the call with user and password.
- (d) - HTTP requests with parameters require mostly JSON. The `jsonBody`-method adds the header `Content-Type: application/json` and serializes the provided object credentials to JSON. Requests without body like *DELETE* or *GET* omit body methods.
- (e) - The `responseObject<ClassType>`-method adds the header `Accept: application/json` and de-serializes the response to the specified class type. Some APIs do not have any response or non-JSON response. In such cases, the method `response` is used instead of `responseObject<ClassType>`.
- (f) - The way, how Kotlin decides between success (2xx) and failed responses. For details on the possible response status, see Configuration REST APIs [page 1775].

Back to Concept [page 1932]

How To Use the Examples

Some common details, used by different examples, were extracted to the `scenario.json` configuration file. This lets you use meaningful names like `config.master!!.user` in the examples. The file is loaded by

```
val config = loadScenarioConfiguration()
```

Each example also invokes

```
disableTrustChecks()
```

This method disables all SSL-related checks.

> ⚠ Caution
>
> `disableTrustChecks()` is only used for test purposes. *Do not* use it in a productive environment.

Both methods, as well as some common REST API parameter structures (data classes) are defined in the file Scenario Configuration [page 1936]. This is the only help class under `sources/`.

When using the examples, start with Initial Configuration [page 1937].

Prerequisite is a freshly installed Cloud Connector.

After a mandatory password change and defining the high availability role of the Cloud Connector instance (master or shadow), the example demonstrates how to provide a description for the instance and how to set up the UI and system certificates.

Once the initial configuration is done, you can optionally proceed with these steps:

- Connect to your subaccount on BTP (Subaccount Configuration [page 1939])
- Create or restore a configuration backup (Backup And Restore Configuration [page 1946])
- Configure and connect high availability instances (High Availability Settings [page 1944])
- Integrate the Cloud Connector with the solution management infrastructure (Solution Management Integration [page 1947])

Back to Concept [page 1932]

Related Information

scenario.json [page 1935]
Source Files [page 1935]

1.4.2.5.19.1 scenario.json

Sample Code

```
{
    "subaccount": {
        "regionHost": "cf.eu10.hana.ondemand.com",
        "subaccount": "11aabbcc-7821-448b-9ecf-a7d986effa7c",
        "user": "xxx",
        "password": "xxx"
    },
    "master": {
        "url": "https://localhost:8443",
        "user": "Administrator",
        "password": "test"
    },
    "shadow": {
        "url": "https://localhost:8444",
        "user": "Administrator",
        "password": "test"
    }
}
```

1.4.2.5.19.2 Source Files

Scenario Configuration [page 1936]

Initial Configuration [page 1937]

Subaccount Configuration [page 1939]

High Availability Settings [page 1944]

Backup And Restore Configuration [page 1946]

Solution Management Integration [page 1947]

1.4.2.5.19.2.1 Scenario Configuration

Sample Code

```
package com.sap.scc.examples
import com.github.kittinunf.fuel.core.FuelError
import com.google.gson.Gson
import java.io.File
import java.security.SecureRandom
import java.security.cert.X509Certificate
import javax.net.ssl.*
data class CloudConnector(
    val url: String,
    var user: String,
    var password: String
)
fun disableTrustChecks() {
    try {
        HttpsURLConnection.setDefaultHostnameVerifier { hostname: String,
session: SSLSession -> true }
        val context: SSLContext = SSLContext.getInstance("TLS")
        val trustAll: X509TrustManager = object : X509TrustManager {
            override fun checkClientTrusted(chain: Array<X509Certificate>,
authType: String) {}
            override fun checkServerTrusted(chain: Array<X509Certificate>,
authType: String) {}
            override fun getAcceptedIssuers(): Array<X509Certificate> {
                return arrayOf()
            }
        }
        context.init(null, arrayOf(trustAll), SecureRandom())
        HttpsURLConnection.setDefaultSSLSocketFactory(context.socketFactory)
    } catch (e: Exception) {
        e.printStackTrace()
    }
}
class ScenarioConfiguration {
    var subaccount: SubaccountParameters? = null
    var master: CloudConnector? = null
    var shadow: CloudConnector? = null
}
data class SubaccountParameters(
    val regionHost: String,
    val subaccount: String,
    val user: String,
    val password: String,
    var locationId: String? = null
)
data class SccCertificate(
    var subjectDN: String? = null,
    var issuer: String? = null,
    var notAfter: String? = null,
    var notBefore: String? = null,
    var subjectAltNames: List<SubjectAltName>? = null
)
data class SubjectAltName(
    var type: String? = null,
    var value: String? = null
)
internal fun loadScenarioConfiguration(): ScenarioConfiguration {
    println("scenario.json will be loaded from $
{File("scenario.json").absolutePath}")
    return Gson().fromJson(File("scenario.json").readText(),
ScenarioConfiguration::class.java)
}
```

```
internal fun processRequestError(error: FuelError) {
    println("failed with ${error.message} ${String(error.errorData)}")
    throw RuntimeException("Stop here. ")
}
```

1.4.2.5.19.2.2 Initial Configuration

Sample Code

```
package com.sap.scc.examples
//import com.sap.scc.examples.SccCertificate
import com.github.kittinunf.fuel.Fuel
import com.github.kittinunf.fuel.core.BlobDataPart
import com.github.kittinunf.fuel.core.FuelManager
import com.github.kittinunf.fuel.core.Method
import com.github.kittinunf.fuel.core.extensions.authentication
import com.github.kittinunf.fuel.gson.responseObject
import com.github.kittinunf.result.Result
import java.io.ByteArrayInputStream
import java.io.File
/*
 This example shows how to use REST APIs to perform the initial configuration
of a master instance
 after installing and starting the Cloud Connector.
 As a prerequisite you need to install and start the Cloud Connector.
 The example begins with changing the initial password, setting the instance
to the master role, edit the description,
 and upload UI and system certificates.
 For the certificates used by Cloud Connector in order to access the UI and
for the system certificate used to access backend systems,
 we simply upload the already available PKCS#12 certificates uiCert.p12 and
systemCert.p12 encrypted with the password "test1234".
 Cloud Connector also provides other options for certificate management,
please take a look at the documentation.
 The configuration details for master and shadow instances can be found in
scenario.json.
 */
fun main() {
    //Cloud Connector distribution generates only an untrusted self-signed
certificate.
    //So for this demonstration use case we need to deactivate all trust
checks.
    disableTrustChecks()
    //Use 'Connection: close' header, to make stateless communication more
efficient
    FuelManager.instance.baseHeaders = mapOf("Connection" to "close")
    //Add output of cURL commands for revision
    //FuelManager.instance.addRequestInterceptor(LogRequestAsCurlInterceptor)
    //Load configuration from property file
    val config = loadScenarioConfiguration()
    //Change the initial password
    Fuel.put("${config.master!!.url}/api/v1/configuration/connector/
authentication/basic")
        .authentication().basic(config.master!!.user, "manage")
        .body("""{"oldPassword":"manage", "newPassword":"$
{config.master!!.password}"}""")
        .response { _, _, result ->
            when (result) {
                is Result.Failure -> processRequestError(result.error)
```

```
                is Result.Success -> println("Password successfully set to $
{config.master!!.password}")
            }
        }
        .join()
    //Set the High-Availability Role to master, use "shadow" if you want to
set it to shadow
    Fuel.put("${config.master!!.url}/api/v1/configuration/connector/haRole")
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .body("master")
        .response { _, _, result ->
            when (result) {
                is Result.Failure -> processRequestError(result.error)
                is Result.Success -> println("high-availability role
successfully set to 'master'")
            }
        }
        .join()
    //Edit Common Description
    Fuel.put("${config.master!!.url}/api/v1/configuration/connector")
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .body("""{"description":"<description of Cloud Connector
instance>"}""")
        .responseObject<SccDescriptionResponse> { _, _, result ->
            when (result) {
                is Result.Failure -> processRequestError(result.error)
                is Result.Success -> println(result.get())
            }
        }
        .join()
    //Upload a PKCS#12 Certificate as UI Certificate
    val uiCertificateFormData = listOf("password" to "test1234",
"keyPassword" to "test1234")
    Fuel.upload(
        "${config.master!!.url}/api/v1/configuration/connector/ui/
uiCertificate",
        Method.PUT,
        uiCertificateFormData
    )
        .add(BlobDataPart(ByteArrayInputStream(File("uiCert.p12").readBytes())
, name = "pkcs12"))
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .response { _, _, result ->
            when (result) {
                is Result.Failure -> processRequestError(result.error)
                is Result.Success -> println("PKCS#12 Certificate
'uiCert.p12'  successfully uploaded")
            }
        }
        .join()
    //Upload a PKCS#12 Certificate as as System Certificate
    val systemCertificateFormData = listOf("password" to "test1234",
"keyPassword" to "test1234")
    Fuel.upload(
        "${config.master!!.url}/api/v1/configuration/connector/onPremise/
systemCertificate",
        Method.PUT,
        systemCertificateFormData
    )
        .add(BlobDataPart(ByteArrayInputStream(File("systemCert.p12").readByte
s()), name = "pkcs12"))
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .response { _, _, result ->
            when (result) {
```

```
                is Result.Failure -> processRequestError(result.error)
                is Result.Success -> println("PKCS#12 Certificate
'systemCert.p12'  successfully uploaded")
            }
        }
        .join()
}
//Data structures used by REST calls in this scenario
data class SccDescriptionResponse(
    var role: String,
    var description: String
)
```

1.4.2.5.19.2.3 Subaccount Configuration

Sample Code

```
package com.sap.scc.examples
import com.github.kittinunf.fuel.Fuel
import com.github.kittinunf.fuel.core.FuelManager
import com.github.kittinunf.fuel.core.extensions.authentication
import com.github.kittinunf.fuel.gson.jsonBody
import com.github.kittinunf.fuel.gson.responseObject
import com.github.kittinunf.result.Result
/*
 This example shows how to use REST APIs to configure and connect a
subaccount in cloud connector.
 The example begins with (1) connecting of the subaccount, then we create a
system (2) for an HTTP service
 and (3) for an RFC service.

 The configuration details for master and shadow instances can be found in
scenario.json.
 */
fun main() {
    //Cloud Connector distribution generates only an untrusted self-signed
certificate.
    //So for this demonstration use case we need to deactivate all trust
checks.
    disableTrustChecks()
    //Use 'Connection: close' header, to make stateless communication more
efficient
    FuelManager.instance.baseHeaders = mapOf("Connection" to "close")
    //Add output of cURL commands for revision
    //FuelManager.instance.addRequestInterceptor(LogRequestAsCurlInterceptor)
    //1.1. Create and connect subaccount
    //Load configuration from property file
    val config = loadScenarioConfiguration()
    //Some cloud regions require 2-Factor-Authentication
    println("Enter MFA (aka 2FA) token, if required: ")
    var token = readLine() ?: ""
    //Parameters required to establish the connection to the subaccount (aka
the secure tunnel)
    var subaccountCreateData = SubaccountConfiguration(
        config.subaccount!!.regionHost, config.subaccount!!.subaccount,
        config.subaccount!!.user, "" + config.subaccount!!.password + token,
locationId = config.subaccount!!.locationId
    )
    //Optional: Initialize the map to work with generated _links
```

```
    //If you don't like to use _links, you can easily compute the entity links
    var subaccountLinks: Map<String, HalLink> = mapOf()
    Fuel.post("${config.master!!.url}/api/v1/configuration/subaccounts")
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .jsonBody(subaccountCreateData)
        .responseObject<SubaccountInfo> { _, response, result ->
            when (result) {
                is Result.Success -> {
                    val subaccountLocation =
response.header("Location").first()
                    println("1.1. subaccount was created under:
$subaccountLocation")
                    println("subaccount: ${result.get()} ")
                    //GET details as SubaccountInfo every time possible by
invoking
                    //https://<SCC>/api/v1/configuration/subaccounts/
<regionHost>/<subaccountId>
                    subaccountLinks = result.get()._links
                }
                is Result.Failure -> processRequestError(result.error)
            }
        }
        .join()
    //1.2. From time to time it is necessary to extend the validity of the
subaccount - refresh subaccount
    //Again some cloud landscapes require 2-Factor-Authentication
    println("Enter MFA (aka 2FA) token for refresh, if required: ")
    token = readLine() ?: ""
    Fuel.post(subaccountLinks["validity"]!!.href)
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .jsonBody(SubaccountRefreshCred(config.subaccount!!.user, "" +
config.subaccount!!.password + token))
        .responseObject<SubaccountInfo> { _, _, result ->
            when (result) {
                is Result.Success -> println("1.2. validity of the subaccount
was extended")
                is Result.Failure -> processRequestError(result.error)
            }
        }
        .join()
    //2.1. Create a system mapping (aka access control) for an HTTP service
    val httpSystem = SystemMapping(
        "virtual.host", "vport", "local", "lport",
CommunicationProtocol.HTTPS,
        BackendType.abapSys, hostInHeader = HostInHeader.INTERNAL
    )
    var httpSystemLinks: Map<String, HalLink> = mapOf()
    Fuel.post(subaccountLinks["systemMappings"]!!.href)
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .jsonBody(httpSystem)
        .responseString { _, response, result ->
            when (result) {
                is Result.Success -> {
                    val httpSystemMappingLocation =
response.header("Location").first()
                    println("2.1. system mapping was created under:
$httpSystemMappingLocation")
                    //GET the details about the system mapping by invoking
                    //https://<SCC>/api/v1/configuration/subaccounts/
<regionHost>/<subaccountId>/systemMappings/<vhost>:<vport>
                    Fuel.get(httpSystemMappingLocation)
                        .authentication().basic(config.master!!.user,
config.master!!.password)
                        .responseObject<SystemMapping> { _, _, result ->
                            when (result) {
```

```
                                is Result.Success -> {
                                    println("system mapping: ${result.get()}")
                                    httpSystemLinks = result.get()._links
                                }
                                is Result.Failure ->
processRequestError(result.error)
                            }
                        }
                        .join()
                }
                is Result.Failure -> processRequestError(result.error)
            }
        }
        .join()
    //2.2 Add an allowed resource to the system mapping. Since this system
mapping points to an HTTP service, use HTTP resource parameters
    Fuel.post(httpSystemLinks["resources"]!!.href)
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .jsonBody(HttpResource("/", exactMatchOnly = false))
        .responseString { _, response, result ->
            when (result) {
                is Result.Success -> {
                    val httpResourceLocation =
response.header("Location").first()
                    println("2.2. http resource was created under:
$httpResourceLocation")
                    //GET the details about the resource by invoking
                    //https://<SCC>/api/v1/configuration/subaccounts/
<regionHost>/<subaccountId>/systemMappings/<vhost>:<vport>/<encoded-id>
                    //<encoded-id> -> replace '/' with '-'
                    Fuel.get(httpResourceLocation)
                        .authentication().basic(config.master!!.user,
config.master!!.password)
                        .responseObject<HttpResource> { _, _, result ->
                            when (result) {
                                is Result.Success -> println("http resource:
${result.get()}")
                                is Result.Failure ->
processRequestError(result.error)
                            }
                        }
                        .join()
                }
                is Result.Failure -> processRequestError(result.error)
            }
        }
        .join()
    //3.1 Create a system mapping for an RFC service
    var rfcSystemLinks: Map<String, HalLink> = mapOf()
    Fuel.post(subaccountLinks["systemMappings"]!!.href)
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .jsonBody(
            SystemMapping(
                "virtual.host",
                "rfcport",
                "local",
                "rfcport",
                CommunicationProtocol.RFCS,
                BackendType.abapSys
            )
        )
        .responseString { _, response, result ->
            when (result) {
                is Result.Success -> {
                    val rfcSystemMappingLocation =
response.header("Location").first()
```

```
                    println("3.1. system mapping was created under:
$rfcSystemMappingLocation")
                    //GET the details about the system mapping by invoking
                    //https://<SCC>/api/v1/configuration/subaccounts/
<regionHost>/<subaccountId>/systemMappings/<vhost>:<vport>
                    Fuel.get(rfcSystemMappingLocation)
                        .authentication().basic(config.master!!.user,
config.master!!.password)
                        .responseObject<SystemMapping> { _, _, result ->
                            when (result) {
                                is Result.Success -> {
                                    println("system mapping: ${result.get()}")
                                    rfcSystemLinks = result.get()._links
                                }
                                is Result.Failure ->
processRequestError(result.error)
                            }
                        }
                        .join()
                }
                is Result.Failure -> processRequestError(result.error)
            }
        }
        .join()
    //3.2 Now add the function module RFC_SYSTEM_INFO as an allowed resource
to the system mapping
    val rfcResource = RfcResource("RFC_SYSTEM_INFO")
    Fuel.post(rfcSystemLinks["resources"]!!.href)
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .jsonBody(rfcResource)
        .responseString { _, response, result ->
            when (result) {
                is Result.Success -> {
                    val resourceLocation = response.header("Location").first()
                    println("3.2. rfc resource was created under:
$resourceLocation")
                    //GET the details about the resource by invoking
                    //https://<SCC>/api/v1/configuration/subaccounts/
<regionHost>/<subaccountId>/systemMappings/<vhost>:<vport>/<encoded-id>
                    //<encoded-id> -> replace '/' with '-'
                    Fuel.get(resourceLocation)
                        .authentication().basic(config.master!!.user,
config.master!!.password)
                        .responseObject<RfcResource> { _, _, result ->
                            when (result) {
                                is Result.Success -> println("rfc resource: $
{result.get()}")
                                is Result.Failure ->
processRequestError(result.error)
                            }
                        }
                        .join()
                }
                is Result.Failure -> processRequestError(result.error)
            }
        }
        .join()
}
//Data structures used by REST calls in this scenario
//Nullable properties are optional
data class SubaccountConfiguration(
    var regionHost: String,
    var subaccount: String,
    var cloudUser: String,
    var cloudPassword: String,
    var displayName: String? = null,
    var locationId: String? = null
```

```
)
data class SubaccountInfo(
    val displayName: String,
    val regionHost: String,
    val subaccount: String,
    val tunnel: SubaccountTunnelInfo,
    val user: String,
    val _links: Map<String, HalLink>
)
data class SubaccountTunnelInfo(
    val state: String, //Connected
    val connectedSince: String, //timestamp formatted with client locale
    val connections: Int,
    val applicationConnections: List<String>,
    val serviceChannels: List<String>,
    val subaccountCertificate: X509CertificateInfo,
    )
data class X509CertificateInfo(
    val notAfter: String,
    val notBefore: String,
    val subjectDN: String,
    val issuer: String
)
data class HalLink(val href: String)
data class SubaccountRefreshCred(
    var cloudUser: String,
    var cloudPassword: String
)
data class SystemMapping(
    val virtualHost: String,
    val virtualPort: String,
    val localHost: String,
    val localPort: String,
    val protocol: CommunicationProtocol,
    val backendType: BackendType,
    val sncPartnerName: String? = null,
    val hostInHeader: HostInHeader? = null,
    val sapRouter: String? = null,
    val authenticationMode: AuthenticationMode = AuthenticationMode.NONE,
    val description: String = "",
) {
    val _links: Map<String, HalLink> = mapOf()
}
enum class CommunicationProtocol {
    HTTP, HTTPS, RFC, RFCS, LDAP, LDAPS, TCP, TCPS
}
enum class BackendType {
    abapSys, netweaverCE, applServerJava, BC, PI, hana, netweaverGW,
otherSAPsys, nonSAPsys
}
enum class HostInHeader {
    INTERNAL, VIRTUAL
}
enum class AuthenticationMode {
    NONE, X509_CERTIFICATE, X509_CERTIFICATE_LOCAL, KERBEROS
}
data class HttpResource(
    val id: String, //URI path
    val enabled: Boolean = true,
    val exactMatchOnly: Boolean = true,
    val webSocketUpgradeAllowed: Boolean = true,
    val description: String = ""
)
data class RfcResource(
    val id: String, //Function module or prefix for function module
    val enabled: Boolean = true,
    val exactMatchOnly: Boolean = true,
    val description: String = ""
```

```
)
```

1.4.2.5.19.2.4 High Availability Settings

Sample Code

```
package com.sap.scc.examples
import com.github.kittinunf.fuel.Fuel
import com.github.kittinunf.fuel.core.FuelManager
import com.github.kittinunf.fuel.core.Headers
import com.github.kittinunf.fuel.core.extensions.authentication
import com.github.kittinunf.fuel.gson.jsonBody
import com.github.kittinunf.fuel.gson.responseObject
import com.github.kittinunf.result.Result
import java.net.URL
/*
 This example shows how to use REST APIs to change high availability settings
of the Cloud Connector instances.
 As a prerequisite you need to install and start a shadow and a master
instance.
 Afterwards perform the initial configuration of the master instance (see
InitialConfiguration.kt).

 The configuration details for master and shadow instances can be found in
scenario.json.
 */
fun main() {
    //Cloud Connector distribution generates only an untrusted self-signed
certificate.
    //So for this demonstration use case we need to deactivate all trust
checks.
    disableTrustChecks()
    //Use 'Connection: close' header, to make stateless communication more
efficient
    FuelManager.instance.baseHeaders = mapOf("Connection" to "close")
    //Add output of cURL commands for revision
    //FuelManager.instance.addRequestInterceptor(LogRequestAsCurlInterceptor)
    //Load configuration from property file
    val config = loadScenarioConfiguration()
    //The high-availability role 'master' in Cloud Connector instance
'master' is already set in the initial configuration.
    //Set the high-availability role to 'shadow' in Cloud Connector instance
'shadow'
    Fuel.put("${config.shadow!!.url}/api/v1/configuration/connector/haRole")
        .authentication().basic(config.shadow!!.user,
config.shadow!!.password)
        .body("shadow")
        .response { _, _, result ->
            when (result) {
                is Result.Failure -> processRequestError(result.error)
                is Result.Success -> println("high-availability role
successfully set to 'shadow'")
            }
        }
        .join()
    //Enable HA in the master instance and define the allowed shadow hosts
    Fuel.put("${config.master!!.url}/api/v1/configuration/connector/ha/master/
config")
        .header(Headers.CONTENT_TYPE, "application/json")
```

```
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .body("""{"haEnabled":"true", "allowedShadowHost":"$
{URL(config.shadow!!.url).host}"}""")
        .responseObject<HAMasterConfiguration> { _, _, result ->
            when (result) {
                is Result.Failure -> processRequestError(result.error)
                is Result.Success -> {
                    val masterConfig = result.get()
                    println("Master configuration: $masterConfig ")
                }
            }
        }
        .join()
    //The configuration of the shadow instance
    var shadowConfig: HAShadowConfiguration? = null
    //Get the current configuration
    Fuel.get("${config.shadow!!.url}/api/v1/configuration/connector/ha/shadow/
config")
        .authentication().basic(config.shadow!!.user,
config.shadow!!.password)
        .responseObject<HAShadowConfiguration> { _, _, result ->
            when (result) {
                is Result.Failure -> processRequestError(result.error)
                is Result.Success -> shadowConfig = result.get()
            }
        }
        .join()
    //Edit the retrieved configuration and set it
    shadowConfig!!.masterPort = "${URL(config.master!!.url).port}"
    shadowConfig!!.masterHost = URL(config.master!!.url).host
    shadowConfig!!.ownHost = URL(config.master!!.url).host
    Fuel.put("${config.shadow!!.url}/api/v1/configuration/connector/ha/shadow/
config")
        .authentication().basic(config.shadow!!.user,
config.shadow!!.password)
        .jsonBody(shadowConfig!!)
        .responseObject<HAShadowConfiguration> { _, _, result ->
            when (result) {
                is Result.Failure -> processRequestError(result.error)
                is Result.Success -> shadowConfig = result.get()
            }
        }
        .join()
    //Set the HA link of both cloud connector instances to 'CONNECT'
    Fuel.post("${config.shadow!!.url}/api/v1/configuration/connector/ha/
shadow/state")
        .header(Headers.CONTENT_TYPE, "application/json")
        .authentication().basic(config.shadow!!.user,
config.shadow!!.password)
        .body("""{"op":"CONNECT", "user":"${config.master!!.user}",
"password":"${config.master!!.password}"}""")
        .response { _, _, result ->
            when (result) {
                is Result.Failure -> processRequestError(result.error)
                is Result.Success -> println("Master (${config.master!!.url})
and shadow (${config.shadow!!.url}) successfully connected")
            }
        }
        .join()
}
//Data structures used by REST calls in this scenario
//Structure used  to read the high availability settings for a Cloud
Connector master instance
data class HAMasterConfiguration(
    var haEnabled: Boolean? = null,
    var allowedShadowHost: String? = null
)
```

```
//Structure used  to read and edit the high availability settings for a Cloud
Connector shadow instance
data class HAShadowConfiguration(
    var masterPort: String,
    var masterHost: String,
    var ownHost: String? = null,
    var checkIntervalInSeconds: Int?,
    var takeoverDelayInSeconds: Int?,
    var connectTimeoutInMillis: Int?,
    var requestTimeoutInMillis: Int?
)
```

1.4.2.5.19.2.5 Backup And Restore Configuration

Sample Code

```
package com.sap.scc.examples
import com.github.kittinunf.fuel.Fuel
import com.github.kittinunf.fuel.core.BlobDataPart
import com.github.kittinunf.fuel.core.FuelManager
import com.github.kittinunf.fuel.core.Headers
import com.github.kittinunf.fuel.core.Method
import com.github.kittinunf.fuel.core.extensions.authentication
import com.github.kittinunf.result.Result
import java.io.ByteArrayInputStream
import java.io.File
/*
 This example shows how to use REST APIs to perform the Backup and Restore of
the Cloud Connector configuration.
 As a prerequisite you have a stable Cloud Connector configuration and you
want to save the backup for later use.

 The configuration details can be found in scenario.json.
 */
fun main() {
    //Cloud Connector distribution generates only an untrusted self-signed
certificate.
    //So for this demonstration use case we need to deactivate all trust
checks.
    disableTrustChecks()
    //Use 'Connection: close' header, to make stateless communication more
efficient
    FuelManager.instance.baseHeaders = mapOf("Connection" to "close")
    //Add output of cURL commands for revision
    //FuelManager.instance.addRequestInterceptor(LogRequestAsCurlInterceptor)
    //Load configuration from property file
    val config = loadScenarioConfiguration()
    //Create the backup configuration of the cloud connector "master".
    Fuel.post("${config.master!!.url}/api/v1/configuration/backup")
        .header(Headers.CONTENT_TYPE, "application/json")
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .body("""{"password":"my-very-secret-password"}""")
        .response { _, _, result ->
            when (result) {
                is Result.Failure -> processRequestError(result.error)
                is Result.Success ->
File("backup.zip").writeBytes(result.get())
            }
```

```
        }
        .join()
    //Restore the backup configuration if some fatal accidental changes
should be reverted
    val formData = listOf("password" to "my-very-secret-password")
    Fuel.upload("${config.master!!.url}/api/v1/configuration/backup",
Method.PUT, formData)
        .add(BlobDataPart(ByteArrayInputStream(File("backup.zip").readBytes())
, name = "backup"))
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .response { _, _, result ->
            when (result) {
                is Result.Failure -> processRequestError(result.error)
                is Result.Success -> println("Backup configuration
successfully restored in (${config.master!!.url}) ")
            }
        }
        .join()
}
```

1.4.2.5.19.2.6 Solution Management Integration

Sample Code

```
package com.sap.scc.examples
import com.github.kittinunf.fuel.Fuel
import com.github.kittinunf.fuel.core.FuelManager
import com.github.kittinunf.fuel.core.extensions.authentication
import com.github.kittinunf.fuel.gson.jsonBody
import com.github.kittinunf.fuel.gson.responseObject
import com.github.kittinunf.result.Result
import java.io.File
/*
 This example shows how to use REST APIs to configure the integration with
SAP solution management.
 In most cases it should be only necessary to pass a boolean flag in order to
enable or
 disable solution management integration. It is expected that SAP host agent
is already
 installed on the host as prerequisite for solution management integration.

 This example executes requests against master and shadow instances.
 The shadow instance is optional. Ignore the requests to shadow instance if
there is only
 a master instance in your environment.

 The configuration details for master and shadow instances can be found in
scenario.json.
 */
fun main() {
    //Cloud Connector distribution generates only an untrusted self-signed
certificate.
    //So for this demonstration use case we need to deactivate all trust
checks.
    disableTrustChecks()
    //Use 'Connection: close' header, to make stateless communication more
efficient
    FuelManager.instance.baseHeaders = mapOf("Connection" to "close")
```

```
    //Add output of cURL commands for revision
    //FuelManager.instance.addRequestInterceptor(LogRequestAsCurlInterceptor)
    //Load configuration from property file
    val config = loadScenarioConfiguration()
    //First we check the current configuration of solution management
    Fuel.get("${config.master!!.url}/api/v1/configuration/connector/
solutionManagement")
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .responseObject<SolutionManagementConfiguration> { _, _, result ->
            when (result) {
                is Result.Success -> println("response: ${result.get()}")
                is Result.Failure -> processRequestError(result.error)
            }
        }
        .join()
    //Configure the hostAgent path on the shadow instance
    //Invoking this API on a shadow instance changes only the configuration.
    //Only when invoking it on a master instance it also activates solution
management integration.
    //Note: This step is not required if host agent is installed at the
default location
    Fuel.post("${config.shadow!!.url}/api/v1/configuration/connector/
solutionManagement")
        .authentication().basic(config.shadow!!.user,
config.shadow!!.password)
        .jsonBody(SolutionManagementConfiguration(hostAgentPath = "/path/to/
hostAgent"))
        .response { _, _, result ->
            when (result) {
                is Result.Success -> println("new path to host agent was set")
                is Result.Failure -> processRequestError(result.error)
            }
        }
        .join()
    //Turns on the solution management integration.
    //Note: this operation is possible only on the master instance.
    //In case of an HA setup, the flag enabled is propagated to the shadow
automatically.
    //Note: optionally you can set here the new path for host agent and
enable DSR.
    Fuel.post("${config.master!!.url}/api/v1/configuration/connector/
solutionManagement")
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .jsonBody(SolutionManagementConfiguration())
        .response { _, _, result ->
            when (result) {
                is Result.Success -> println("solution management is
activated")
                is Result.Failure -> processRequestError(result.error)
            }
        }
        .join()
    //Turns off the solution management integration.
    //Note: this operation is possible only on the master instance.
    //In case of an HA setup, the flag not enabled is propagated to the
shadow automatically
    Fuel.delete("${config.master!!.url}/api/v1/configuration/connector/
solutionManagement")
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .response { _, _, result ->
            when (result) {
                is Result.Success -> println("solution management is
deactivated")
                is Result.Failure -> processRequestError(result.error)
            }
```

```
        }
        .join()
    //Registration file with LMDB model can be downloaded by following
request.
    //This file can be used for a manual upload to solution management.
    //If solution management integration is active,the updates are triggered
by configuration changes automatically.
    Fuel.get("${config.master!!.url}/api/v1/configuration/connector/
solutionManagement/registrationFile")
        .authentication().basic(config.master!!.user,
config.master!!.password)
        .response { _, _, result ->
            when (result) {
                is Result.Success ->
File("lmdbModel.zip").writeBytes(result.get())
                is Result.Failure -> processRequestError(result.error)
            }
        }
        .join()
}
//Data structures used by REST calls in this scenario
//Nullable properties are optional
data class SolutionManagementConfiguration(
    var hostAgentPath: String? = null,
    var enabled: Boolean? = null,
    var dsrEnabled: Boolean? = null
)
```

1.4.2.6 Using Service Channels

Configure Cloud Connector service channels to connect your on-premises network to specific services on SAP BTP or to an ABAP Cloud system.

Context

Cloud Connector service channels provide access from an external network to specific services on SAP BTP, or to an ABAP Cloud system. The called services are not exposed to direct access from the Internet. The Cloud Connector ensures that the connection is always available and communication is secured.

Service Channel Type	Description
RFC Connection to an ABAP Cloud system	The service channel for **RFC** supports calls from on-premises systems to an ABAP Cloud system using RFC.
K8s Cluster (TCP connection to a SAP BTP Kubernetes cluster)	Service channel to establish a connection to a service in a Kubernetes cluster on SAP BTP.

Next Steps

Configure a Service Channel for RFC [page 1950]

Configure a Service Channel for a Kubernetes Cluster [page 1952]

Related Information

Service Channels: Port Overview [page 1954]

1.4.2.6.1 Configure a Service Channel for RFC

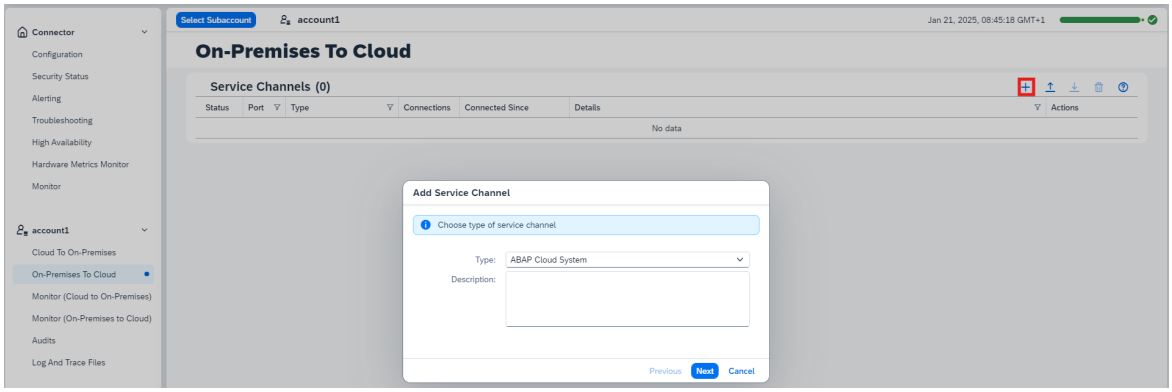
For scenarios that need to call from on-premises systems to SAP BTP ABAP environment using RFC, you can establish a connection to an ABAP Cloud tenant host. To do this, select *On-Premises to Cloud* > *Service Channels* in the Cloud Connector.

Prerequisites

- When using the default connectivity setup with the **multi-cloud** subaccount in which the system has been provisioned, you can use a service channel without additional configuration, as long as the system is a single-tenant system.
- When using connectivity via a **Neo** subaccount, you must create a communication arrangement for the scenario SAP_COM_0200. For more information, see Create a Communication Arrangement for Cloud Connector Integration (documentation for ABAP environment on SAP BTP).

Procedure

1. From your subaccount menu, choose *On Premise To Cloud*.
2. Choose the *Add* (+) icon.



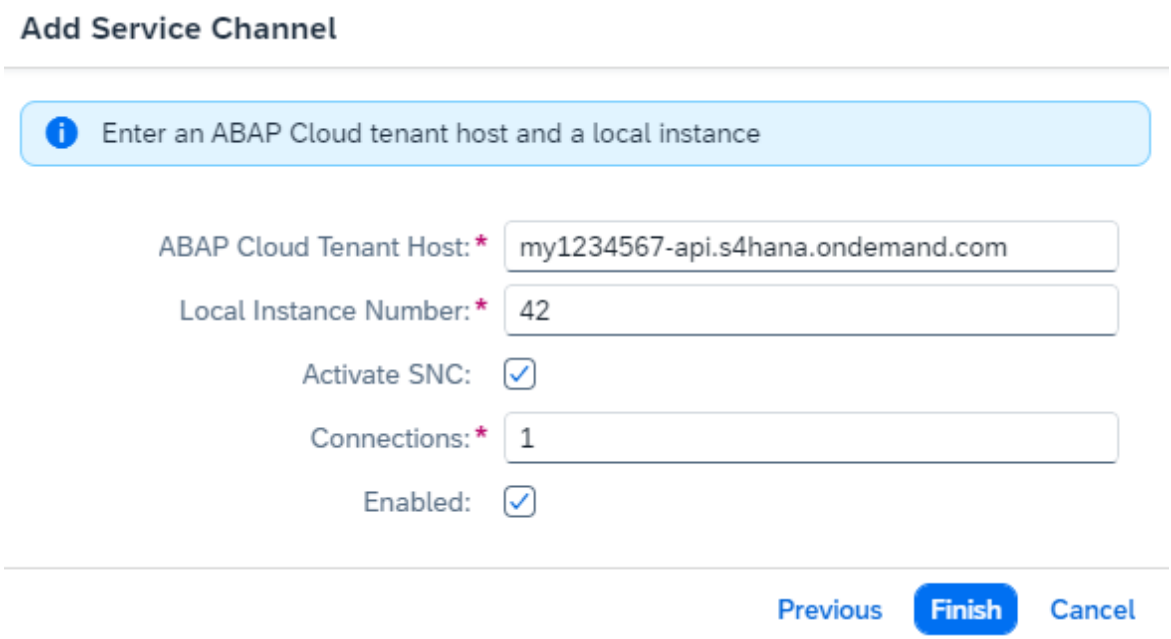
3. In the *Add Service Channel* dialog, select **`ABAP Cloud System`** from the drop-down list of supported channel types.
4. Optionally, provide a *Description* that explains what the ABAP Cloud service channel is used for.
5. Choose *Next*. The *ABAP Cloud System* dialog opens.
6. Enter the `<ABAP Cloud Cloud Tenant Host>` that you want to connect to.

> **Note**
>
> For SAP BTP ABAP environment, the tenant host is `<serviceinstanceguid>.abap.<region>.hana.ondemand.com` (note that this is not the frontend address with the abap-web subdomain). The region is, for example, **`eu10`** or **`us10`**.

7. In the same dialog window, define the `<Local Instance Number>` under which the ABAP Cloud system is reachable for the client systems. You can enter any instance number for which the port is not used yet on the Cloud Connector host. The port numbers result from the following pattern: **`33<LocalInstanceNumber>`**, for activated SNC (*Secure Network Connection*) **`48<LocalInstanceNumber>`**.
8. Use the checkbox *Activate SNC* to specify if the opened service channel port should listen to and terminate incoming SNC RFC connections instead of plain RFC connections. The SNC configuration used is the same as in section Initial Configuration (RFC) [page 1678]. In case of issues, you can use the trace *Cloud Connector loggers*, and also the *SNC payload* and *CPIC* traces.
   For more information on traces, see Monitoring, Logging, and Troubleshooting [page 2046].

> **Note**
>
> This SNC option is only supported for ABAP-based clients, not for any RFC connectors as JCo, NCo or NW RFC SDK. For these scenarios, please use the WebSocket RFC protocol without the Cloud Connector.

9. In the same dialog window, leave *Enabled* selected to establish the channel immediately after choosing *Finish*. Unselect it if you don't want to establish the channel immediately.



10. Choose *Finish*.

> **Note**
>
> When addressing an ABAP Cloud system in a **destination configuration**, you must enter the *Cloud Connector host* as application server host. As instance number, specify the `<Local Instance Number>` that you configured for the service channel. As user, you must provide the *business user* name but not the technical user name associated with the same.

> **Note**
>
> When using an RFC service channel in a high availability setup, you can put a TCP load balancer between the client ABAP system and the Cloud Connector master and shadow instances. In this case, you must configure the load balancer in the destination. The load balancer routes the request to the instance that is currently in the master role, either by simply trying one instance after the other or by checking regularly, which of the instances currently has the master role, and routing to this instance only.

1.4.2.6.2 Configure a Service Channel for a Kubernetes Cluster

Create a service channel to establish a connection to a service in a Kubernetes cluster in SAP BTP that is not directly exposed to external access.

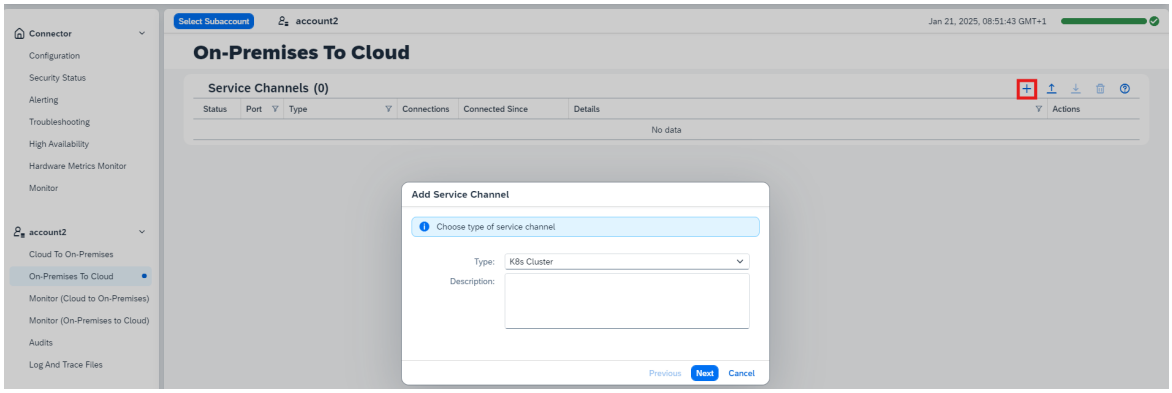
Follow the steps below to establish a service channel for a Kubernetes cluster (K8s cluster).

Prerequisites

- You have a service running in a Kubernetes cluster that is connected to your subaccount.
- You have exposed your application to the Connectivity Proxy for Kubernetes.
  For more information, see Service Channels: On-Premises-To-Cloud Connections [page 2112].

Procedure

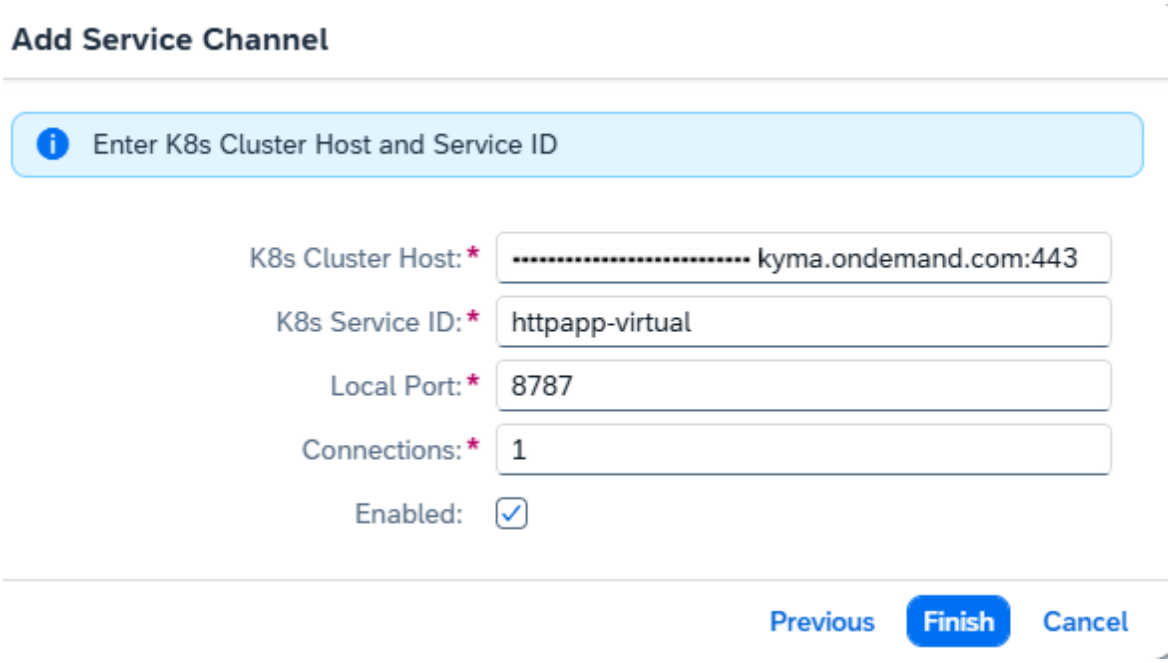
1. Choose *On-Premises to Cloud* > *Service Channels* from your subaccount menu.
2. Choose the *Add* button.



3. In the *Add Service Channel* dialog, select `K8s Cluster` from the drop-down list of supported channel types.
4. Optionally, provide a *Description* that explains what the Kubernetes cluster service channel is used for.
5. Choose *Next*. The *K8s Cluster* dialog opens.
6. Specify the cluster host to the Kubernetes cluster. You can also paste the whole string *[<protocol>://]<clusterHost>[:<port>]/<serviceID>* into the **K8s Cluster Host** field, after you have created the service mapping in your cluster:
   1. This can be retrieved via:
      ```
      kubectl get servicemappings.connectivityproxy.sap.com <name>
      -ojsonpath={.status.endpoint}
      ```
   2. It is automatically split into the fields **K8s Cluster Host** and **K8s Service ID**.

> ### Note
>
> This cluster host points to the Connectivity Proxy [page 2088] that exposes the service mapped by the Cloud Connector service channel.

7. Specify the service ID as configured in the service mapping in Service Channels: On-Premises-to-Cloud Connectivity [page 2112].
8. In the same dialog window, choose the local port and the number of connections. You can enter any port that is not used yet.
9. Leave the *Enabled* option selected to establish the channel immediately after clicking *Save*, or deselect it if the channel should not be established yet.



10. When you are done, choose *Finish*.

> **Note**
>
> If you enter the complete service string **`[<protocol>://]<host>[:<port>]/<service.host>`** into the *K8s Cluster Host* field, it is automatically split into the fields *K8s Cluster Host* and *K8s Service Host*.

Next Steps

Once you have established a service channel to the Kubernetes cluster, you can connect your client application by accessing `<Cloud_connector_host>:<local_port>`.

1.4.2.6.3 Service Channels: Port Overview

A service channel overview lets you see the details of all service channels that are used by a Cloud Connector installation.

The service channel port overview lists all service channels that are configured in the Cloud Connector. It lets you see at a glance, which server ports are used by a Cloud Connector installation.

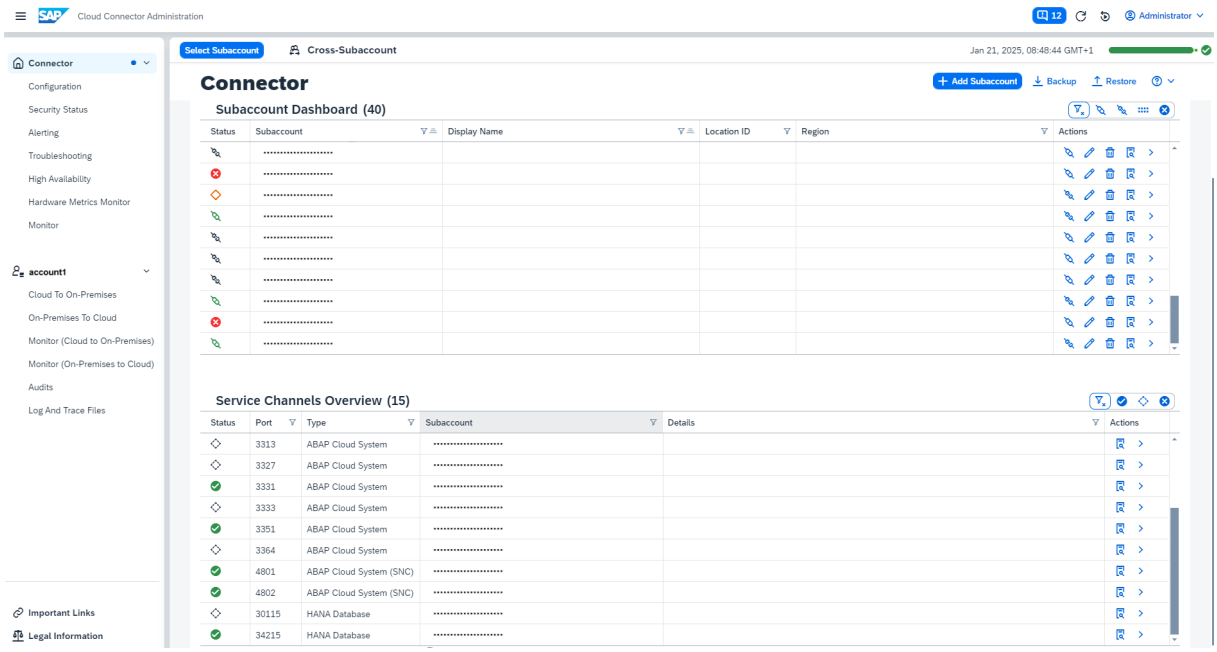
In addition, you can find the following information about each service channel:

- Status (`enabled`, `disabled`, `disconnected`)
- Service channel type (`ABAP Cloud System`, `Kubernetes Cluster`)

- Assigned subaccount (display name)
- Details (for example, the assigned ABAP Cloud tenant host)

From the *Actions* column, you can switch directly to the *On-Premises To Cloud* section of the corresponding subaccount and edit the selected service channel.

To find the overview list, choose *Connector* from the navigation menu and go to section *Service Channels Overview*:



The *filter buttons* above the overview table let you filter the shown service channels based on their status. You can select all service channels, all enabled ones, all disabled ones, or all service channels for which enabling has failed.

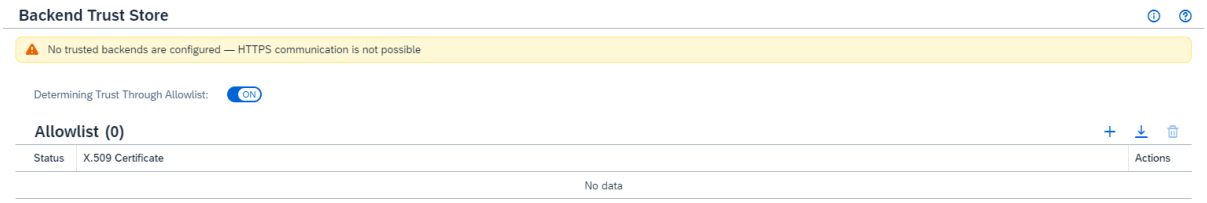
1.4.2.7 Configure Trust

Set up a trust store for on-premises systems in the Cloud Connector.

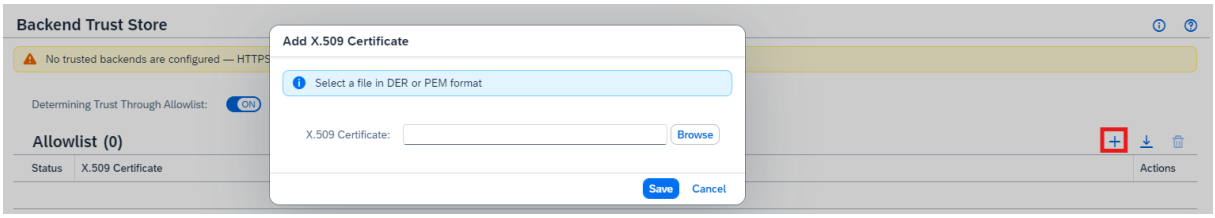
Configure the Trust Store

To configure the trust store, choose *Configuration* from the main menu, and go to tab *On Premises*, section *Backend Trust Store*.

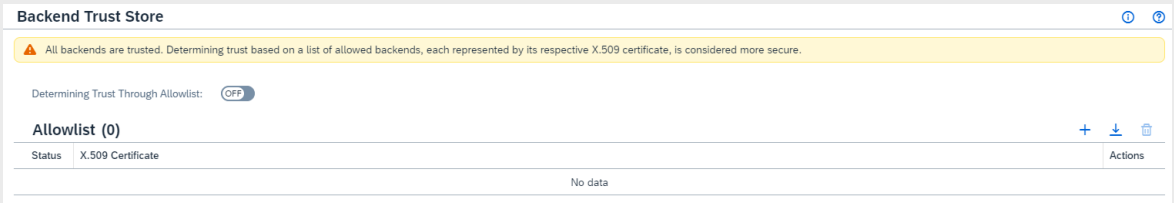
By default, the Cloud Connector **does not trust any** on-premises system when connecting to it via TLS:



To enable secured backend communication, you must add trusted certificates to the allowlist. Any TLS server certificate that holds one of these certificates in its certificate chain, as intermediate or root certificate, will be considered trusted. Otherwise, the server will be considered untrusted and the connection will fail.



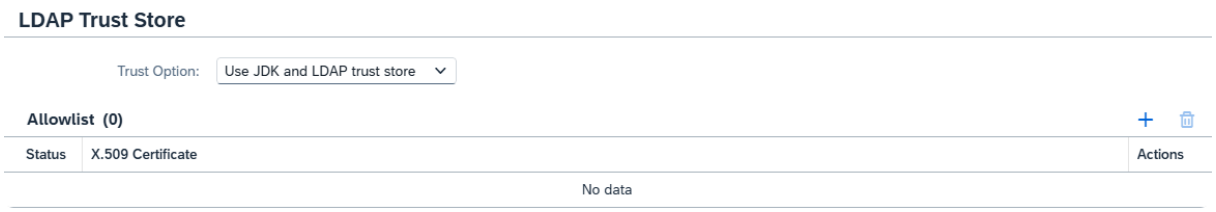
> ### Note
>
> You must provide the CA's X.509 certificates in **DER** or **PEM** format.

> ### Caution
>
> If you don't want to specify explicit certificates you are going to trust, but rather **trust all certificates used by your backends**, you can switch off the trust store, so the allowlist is being ignored. However, this is considered insecure, since any server certificate is trusted in this case.
>
> As a result, the risk of certain attacks on the hop between Cloud Connector and internal systems is increased. **Therefore, we strongly recommend that you** *don't do this in productive installations*.
>
> 
>
>
> By clicking on the *Download* button, you can download all or only specific CAs again.

LDAP Trust

When configuring LDAP for User Administration (see tab *User Interface* of *Configuration*), either in draft or active mode, the **LDAP Trust Store** can be set up by adding certificates to the **Allowlist** and choosing a *Trust Option*.

> **Note**
>
> The LDAP trust store is used by all LDAP user stores.

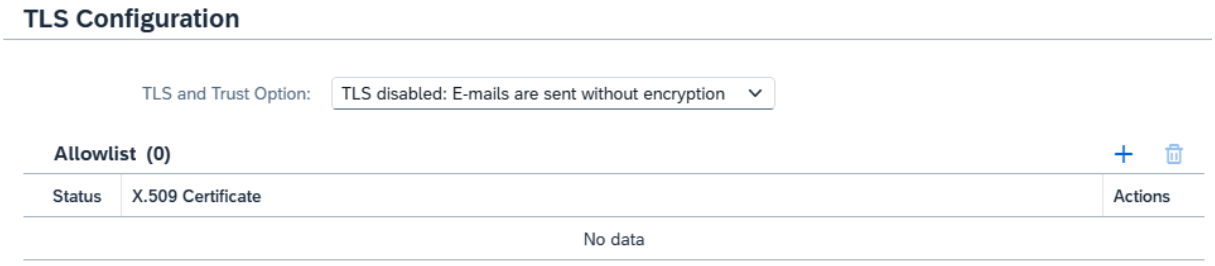


There are three trust options:

1. *Trust all*: All certificates are trusted. This is the least secure option which does not take into account any trusted certificates, in the allowlist or otherwise, and simply trusts all certificates.
2. *Use LDAP trust store only*: Only the certificates listed in the allowlist are trusted.
3. *Use JDK and LDAP trust store*: The certificates in the allowlist and those of the JDK are trusted.

E-Mail Trust

E-mail trust, or more precisely the **TLS Configuration**, is part of the **E-Mail Configuration** of *Alerting*. Choose the appropriate *TLS and Trust Option*, and add certificates to the **Allowlist** if required.



There are four TLS and trust options:

1. *TLS disabled: E-mails are sent without encryption*: E-mails are sent as plain text. No certificates are involved.
2. *TLS enabled: Trust all*: No trusted certificates are taken into account, and thus all certificates are trusted.
3. *TLS enabled: Use E-mail trust store only*: Only the certificates listed in the allowlist are trusted.
4. *TLS enabled: Use JDK and E-mail trust store*: The certificates in the allowlist and those of the JDK are trusted.

1.4.2.8 Configure Domain Mappings for Cookies

Context

Some HTTP servers return cookies that contain a `domain` attribute. For subsequent requests, HTTP clients should send these cookies to machines that have host names in the specified domain.

For example, if the client receives a cookie like the following:

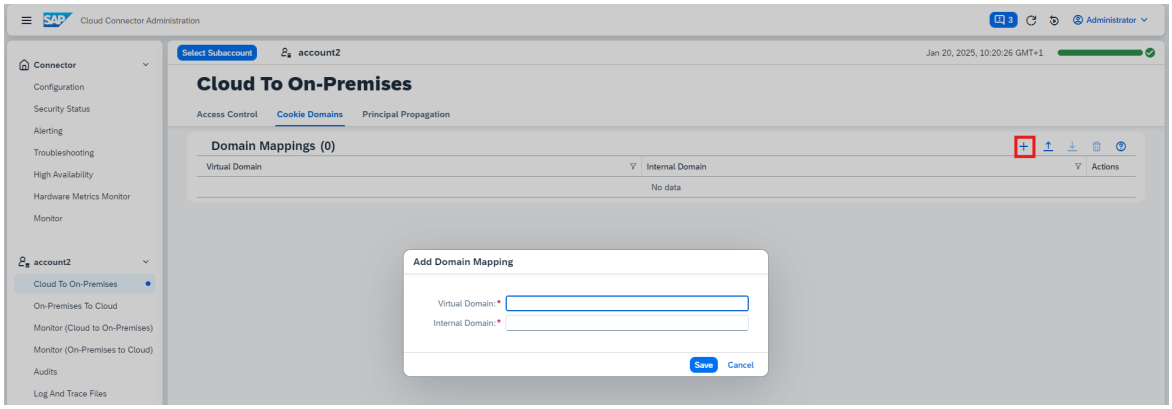
```
Set-Cookie: cookie-field=some-value; domain=mycompany.corp; path=...; ...
```

It returns the cookie in follow-up requests to all hosts like **`ecc60.mycompany.corp`**, **`crm40.mycompany.corp`**, and so on, if the other attributes like `path` and `attribute` require it.

However, in a Cloud Connector setup between a client and a Web server, this may lead to problems. For example, assume that you have defined a virtual host *www1.sales-system.cloud* and mapped it to the internal host name **`ecc60.mycompany.corp`**. The client "thinks" it is sending an HTTP request to the host name *www1.sales-system.cloud*, while the Web server, unaware of the above host name mapping, sets a cookie for the domain **`mycompany.corp`**. The client does not know this domain name and thus, for the next request to that Web server, doesn't attach the cookie, which it should do. The procedure below prevents this problem.

Procedure

1. From your subaccount menu, choose *Cloud To On-Premises*, and go to the *Cookie Domains* tab.
2. Choose *Add*.
3. Enter the virtual domain and the internal domain. In the example that would be **`cloud`** as the virtual and mycompany.corp as the internal domain.
4. Choose *Save*.



The Cloud Connector checks the Web server's response for **`Set-Cookie`** headers. If it finds one with an attribute `domain=intranet.corp`, it replaces it with `domain=sales.cloud` before returning the HTTP response to the client. Then, the client recognizes the domain name, and for the next request

against *www1.sales.cloud* it attaches the cookie, which then successfully arrives at the server on **`machine1.intranet.corp`**.

> **Note**
>
> Some Web servers use a syntax such as `domain=.intranet.corp` (RFC 2109), even though the newer RFC 6265 recommends using the notation without a dot.

> **Note**
>
> The value of the domain attribute may be an internal host name, in which case no extra domain mapping is necessary on the Cloud Connector. For example, if the server sets a cookie with `domain=machine1.intranet.corp`, the Cloud Connector automatically reverses the **`mapping machine1.intranet.corp`** to *www1.sales.cloud* and replaces the cookie domain accordingly. In other words, the system mappings of *Access Control* are employed as a fallback (see Configure Access Control [page 1740]).

Related Information

Configure Access Control [page 1740]

1.4.2.9 Configure Solution Management Integration

Activate Solution Management reporting in the Cloud Connector.

If you want to monitor the Cloud Connector with the SAP Solution Manager, you can install a host agent on the machine of the Cloud Connector and register the Cloud Connector on your system.

Prerequisites

- You have installed the SAP Diagnostics Agent and SAP Host Agent on the Cloud Connector host and connected them to the SAP Solution Manager. The RPM on Linux ensures that the host agent configuration is adjusted and that user groups are setup correctly.
  For more details about the host agent and diagnostics agent, see SAP Host Agent and the SCN Wiki SAP Solution Manager Setup/Managed System Checklist.
  See also SAP notes 2607632 (*SAP Solution Manager 7.2 - Managed System Configuration for SAP Cloud Connector*) and 1018839 (*Registering in the System Landscape Directory using sldreg*). For consulting, contact your local SAP partner.

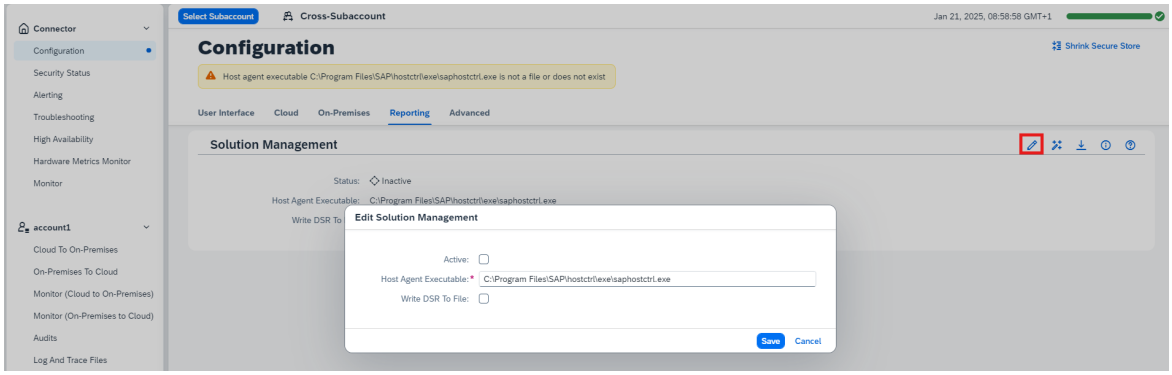
  > **Note**
  >
  > Linux OS: if you installed the host agent after installing the Cloud Connector, you can execute `enableSolMan.sh` in the installation directory to adjust the host agent configuration and user group setup. This action requires root permission.

- The SAP Solution Manager must be of release 7.2 SP06 or higher.

Procedure

To enable the integration, do the following:

1. From the Cloud Connector main menu, choose *Configuration* > *Reporting*. In section *Solution Management* of the *Reporting* tab, select *Edit*.



2. Select the *Active* checkbox.
3. In the field `<Host Agent>`, specify the location of the host agent as filepath.
4. If you want to store the reporting results (*Dynamic Statistical Records*), select *Write DSR To File*.
5. Choose *Save*.

> **Note**
>
> To download the registration file *ImdbModel.xml*, choose the icon *Download registration file* from the *Reporting* tab.

Related Information

Monitoring [page 2006]

1.4.2.10 Configure Tunnel Connections

Adjust tunnel parameters for the Cloud Connector.

If required, you can adjust the following parameters influencing the tunnel behavior by changing their default values:

- Application Tunnel Connections (default: 1): This parameter specifies the default value for the maximal number of tunnel connections *per application*. The value must be higher than 0.

- Tunnel Worker Threads (default: 10): The number of threads per subaccount that are available for processing incoming packages from the cloud.
- Protocol Processor Worker Threads (default: 20): The number of threads used for protocol-specific processing per subaccount.

  > **Note**
  >
  > The thread pool size for RFC is derived from that value. The thread pool size will be 5 times of the protocol worker thread size, meaning that at most this number of concurrent requests can be performed for the subaccount.

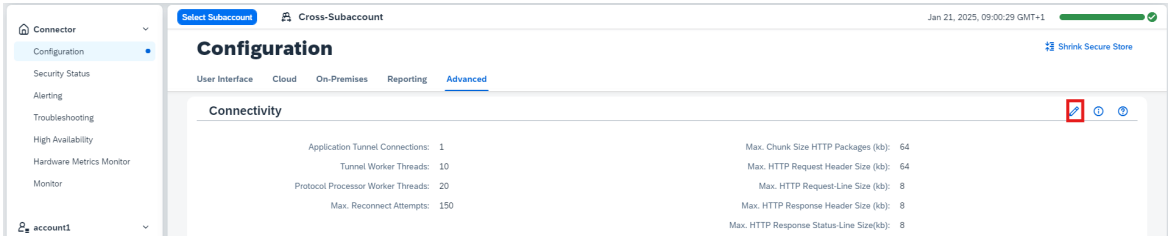
- Max. Reconnect Attempts (default: 150): Whenever the tunnel connection to a subaccount breaks, the Cloud Connector will perform that many reconnect attempts in order to re-establish the tunnel to the Connectivity service.

  > **Note**
  >
  > The reconnect procedure will be triggered only if the connection has been established successfully before. If the connection fails directly after startup, it will stay disconnected to avoid unnecessary connection attempts.

For detailed information on connection configuration requirements, see Configuration Setup [page 1654].

To change the parameter values, do the following:

1. From the Cloud Connector main menu, choose *Configuration* > *Advanced*. In section *Connectivity*, select *Edit*.



2. In the *Edit Connectivity Settings* dialog, change the parameter values as required.
3. Choose *Save*.

For the **Neo** environment only: Additionally, you can specify the number of allowed tunnel connections for each application that you have specified as a *trusted application*.

> **Note**
>
> If you don't change the value for a trusted application, it keeps the default setting specified above. If you change the value, it may be higher or lower than the default and must be higher than 0.

Related Information

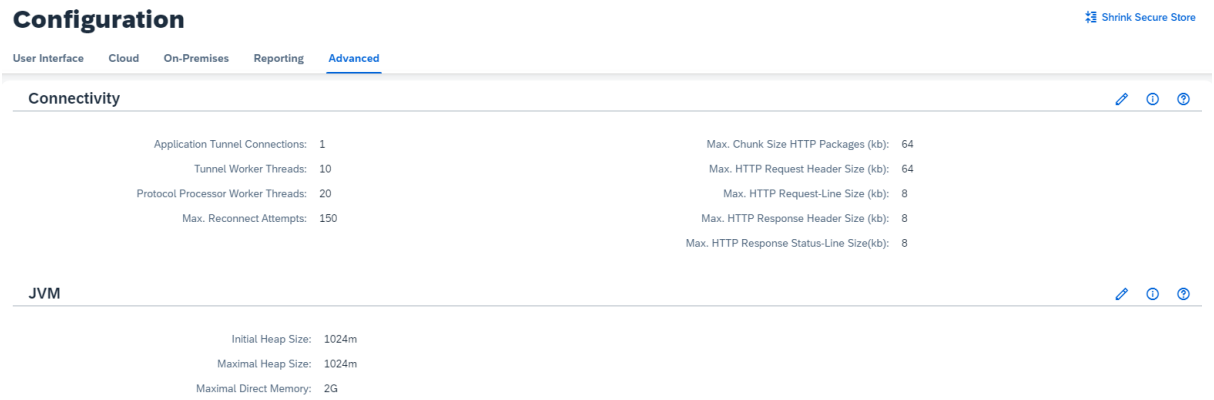
Configuration Setup [page 1654]

1.4.2.11 Configure Advanced Connectivity

Adapt connectivity settings that control the throughput and HTTP connectivity to on-premises systems.

> ⚠ Caution
>
> Do not change these parameters unless you are absolutely sure that changes are indispensable for your scenarios. Setting inadequate values may increase the risk of DoS (Denial of Service) attacks.

The Cloud Connector allows to set some sensible parameters controlling the HTTP connectivity to on-premises systems.



Find a brief description of these critical configuration parameters below that are related to HTTP processing:

Parameter	Description	Default Value
Max. Chunk Size HTTP Packages (kb)	Max. size of chunks transmitted in HTTP streaming. The chunk size affects the throughput of HTTP communication.	64kb
Max. HTTP Request Header Length (kb)	Max. size allowed for HTTP request headers. Headers containing authentication information in tokens, like SAML or JWT, may require this size.	64kb
Max. Size HTTP Request (kb)	Max. size for the request line of an HTTP request. HTTP body is not included.	8kb

Parameter	Description	Default Value
Max. HTTP Response Header Length (kb)	Max. size allowed for HTTP response headers.	8kb
Max. Size HTTP Response (kb)	Max. size for the response line of an HTTP response. HTTP body is not included.	8kb

Related Information

Cloud-To-On-Premises Connections [page 1655]

1.4.2.12 Configure the Java VM

Adapt the JVM settings that control memory management.

If required, you can adjust the following parameters for the Java VM by changing their default values:

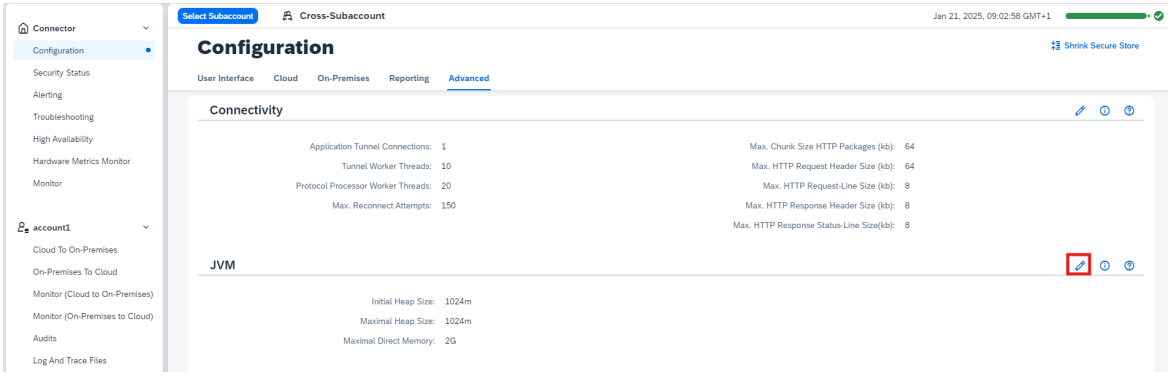
- Initial Heap Size (default: 1024 MB)
- Maximal Heap Size (default: 1024 MB)
- Maximal Direct Memory (default: 2 GB)

> ⓘ Note
>
> A restart is required when changing JVM settings. Unsuitable memory settings may result in a Cloud Connector start-up failure. In this case, you must manually adapt the file `props.ini` located in the Cloud Connector installation directory. Contact your system administrator when in doubt.

> → Tip
>
> We recommended that you set the initial heap size equal to the maximal heap size to avoid memory fragmentation.

To change the parameter values, proceed as follows:

1. From the Cloud Connector main menu, choose *Configuration* > *Advanced*. In section *JVM*, select *Edit*.



2. In the *Edit JVM Settings* dialog, change the parameter values as required.

> **Note**
>
> At most four digits are accepted. Providing a unit is mandatory. Use k or K for kilobyte, m or M for megabyte, or g or G for gigabyte. Do not insert spaces between digits and unit.

3. Choose *Save*.

Related Information

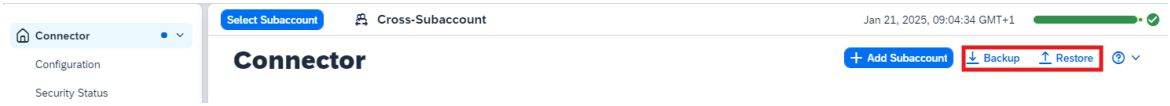
Sizing for the Master Instance [page 1650]

1.4.2.13 Configuration Backup

You can backup and restore your Cloud Connector configuration.

To backup or restore your Cloud Connector configuration, do the following:

1. From the Cloud Connector main menu, choose *Connector*.



2. To backup or restore your configuration, choose the respective icon in the upper right corner of the screen.
   1. To backup your configuration, enter and repeat a password in the *Backup* dialog and choose *Backup*.

> **Note**
>
> An archive containing a snapshot of the current Cloud Connector configuration is created and downloaded by your browser. For security reasons, some files are encrypted, using the password provided for the backup procedure. We recommend that you choose a strong password for the backup - a good password is long, uses multiple character classes, and is not predictable.

→ Tip

You can use this archive to restore the current state of the same installation or to move the current configuration to a new Cloud Connector installation, if the original instance can no longer be used.

You can restore the archive on a Cloud Connector instance

- With the same or higher version than the original one.
- Running on a different operating system.

However, you cannot restore it on an *older* Cloud Connector version.

2. To restore your configuration, enter the required *Archive Password* and the *Login Password* of the currently logged-in administrator user in the *Restore from Archive* dialog and choose *Restore*.

⚠ Caution

The restore action overwrites the current configuration of the Cloud Connector. Its current state and settings will be lost permanently unless you have created another backup before doing the restore operation. Upon successfully restoring the configuration backup, the Cloud Connector restarts automatically. All active sessions are then terminated.

ⓘ Note

- The `props.ini` file is treated in a special way. If the file from the backup archive differs from the one that is used in the current installation, it will be placed next to the original one as `props.ini.restored`. If you want to use the `props.ini.restored` file, replace the existing `props.ini` file by the restored one on OS level and restart the Cloud Connector afterwards.
- All custom path settings from the configuration backup archive, which are not valid or operable anymore, will be automatically reset to their respective default values upon restore operation. For example, this will be the case if a configured directory or file path from the backup archive does not exist or is not accessible anymore.
- Restored certificates from older backup archives might have expired and must be replaced with valid ones to get all configured scenarios to work again.
- If restoring a configuration backup archive on a host that is different from the original one, the restored UI certificate might not be issued for the new hostname, domain or IP address, and therefore would require a replacement with a matching UI server certificate as well.
- If you are using an external SNC library, neither the library itself is part of the configuration backup archive, nor any configuration that this library would load from its own external storage. As a result, the SNC configuration must be set up again after restoring a configuration backup archive on a different host.
- The same applies to used own CA certificates which are stored in the JVM's managed trust store, for example, if Email alerting or LDAP has been set up this way, with own certificates for establishing a secure communication. These externally stored CA certificates would also not be part of the configuration backup archive and must be imported into the local JVM's trust store manually.

⚠ Caution

Do not run multiple Cloud Connector instances with the same configuration simultaneously. The feature to restore a configuration backup archive on another Cloud Connector installation is not

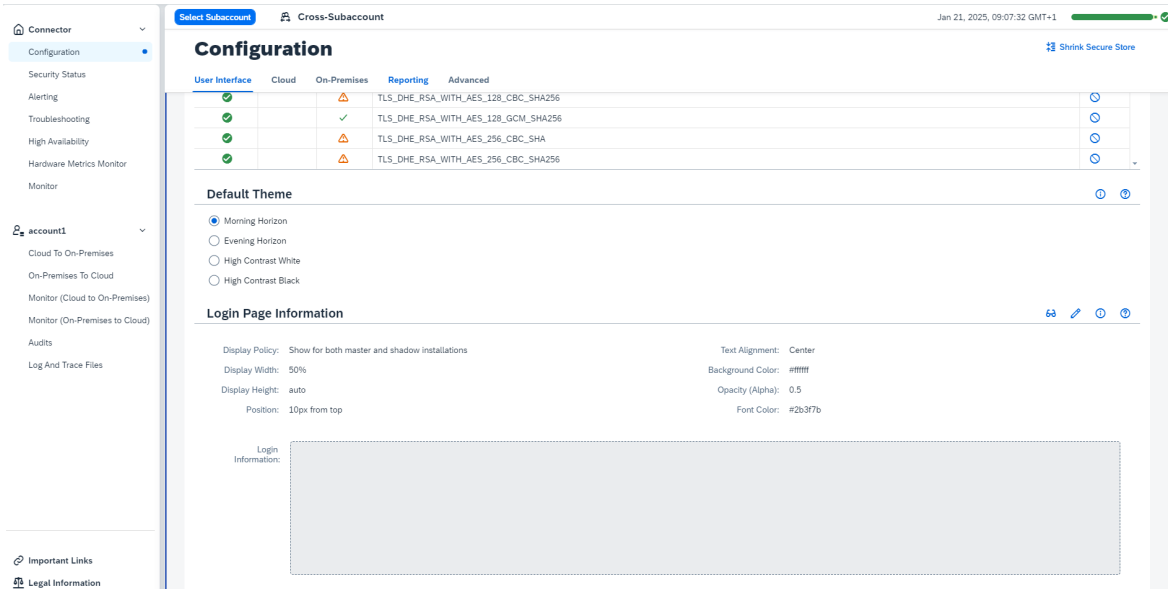
supposed to be used for cloning purposes, but only for supporting the move of a Cloud Connector instance from one host to another. After restoring a configuration backup archive on a different Cloud Connector installation, you must stop the original Cloud Connector instance *immediately*. Otherwise, running two or more instances with the same configuration will cause issues. Such a cloned setup is not supported.

1.4.2.14 Configure Login Screen Information

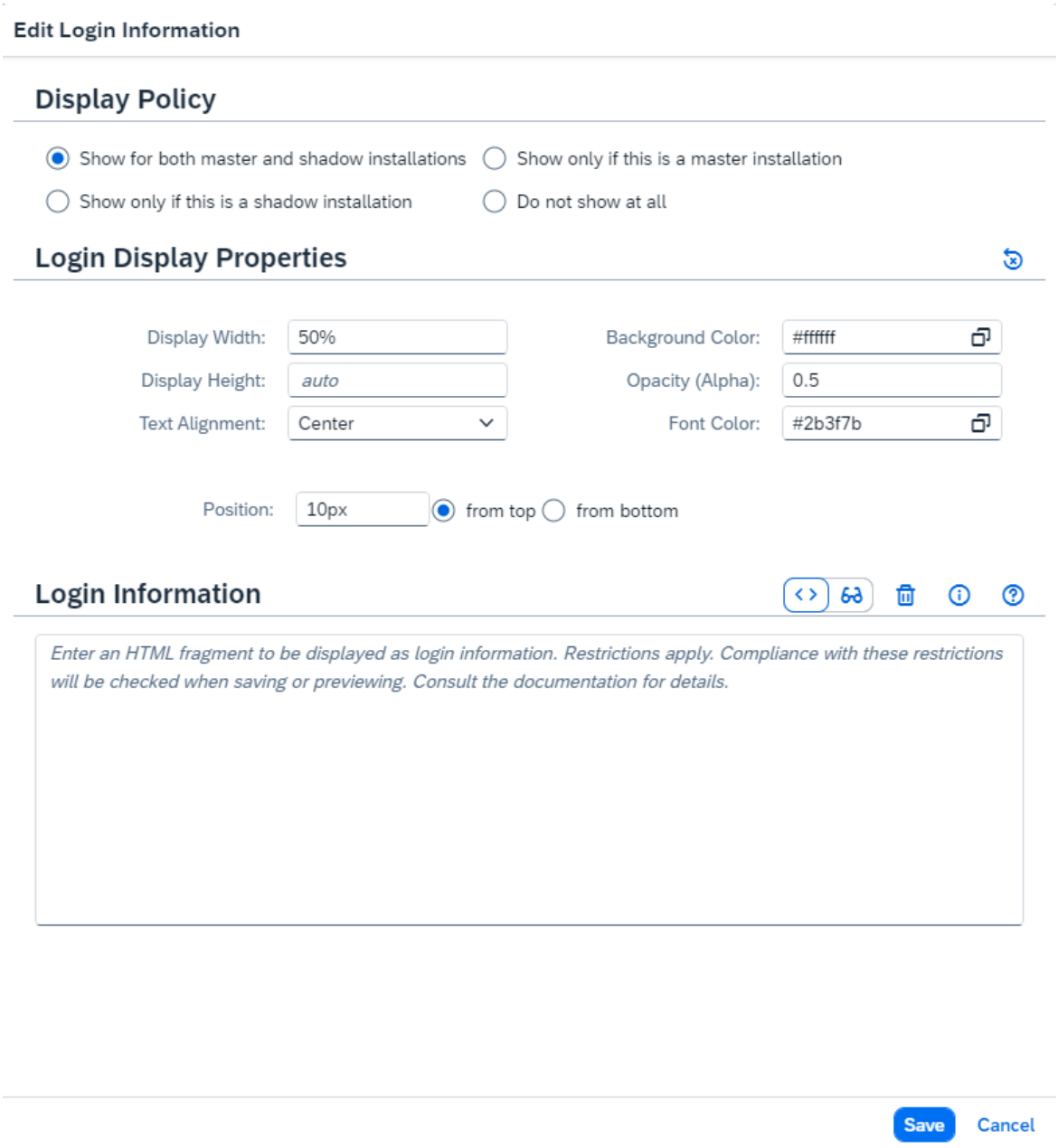
Add additional information to the login screen and configure its appearance.

To configure the login screen information, proceed as follows:

1. Go to *Configuration* > *User Interface* (Shadow instance: *Shadow Configuration* > *User Interface*) and press the *Edit* button in section *Login Screen Information* (at the bottom of the screen).



As a result, the following dialog is opened:



2. In section *Display Policy*, select a display policy for the login screen information. The display policy decides whether the information is shown if the instance is assuming the role mentioned in the policy.
3. In section *Display Properties*, specify your preferred display properties (appearance and position):
   - The login information is displayed in a box with rounded corners. You can specify its width and height in pixels (unit `px`) or as a percentage (unit `%`).

     > **Note**
     >
     > Omitting any value triggers the default or auto behavior.
     >
     > - For the width, the default behavior is equivalent to 100%.
     > - For the height, the default behavior sets the height to a value that accommodates the login information (that is, the given HTML fragment). For extensive information we therefore

recommend that you limit the height to a suitable pixel or percentage value to induce scrolling, and to prevent the box from growing beyond a reasonable height.

- When customizing the **background color** of the box, the **opacity** (of the background color), **font color**, or **text alignment**, then the section *Login Information* automatically switches to preview mode. That way you can follow live the changes made to the appearance of the box and of the information displayed inside it.

> ### Note
>
> You can hide the box and to show only the text of the login information by choosing an opacity value of `0` (opacity is the opposite of transparency. *No* opacity means *complete* transparency).

- You can **position the box** containing the login information at the top or bottom of the login page. To do this, set the field `<Position>` to the corresponding pixel or percentage value.

4. Enter the information to be displayed in section *Login Information*. The information must be supplied as an HTML fragment. There is a limited number of tags that can be used. Attributes available for these tags are subject to restrictions.

Available Tag	Attribute Restriction
p	Only attribute style is permitted, with property `text-align` set to a valid value (`left`, `right`, `center`, or `justified`)
ul	No attributes allowed
ol	No attributes allowed
li	No attributes allowed
br	No attributes allowed
h1	No attributes allowed
h2	No attributes allowed
h3	No attributes allowed
i	No attributes allowed
b	No attributes allowed
a	Must have exactly two attributes: • `href` (its value must be a `<URL>` with protocol `http` or `https`) • `target` (its value must be "`_blank`")

HTML syntax checking is strict. Attribute values must be enclosed by double quotes. Missing or unmatched opening or closing tags are not permitted.

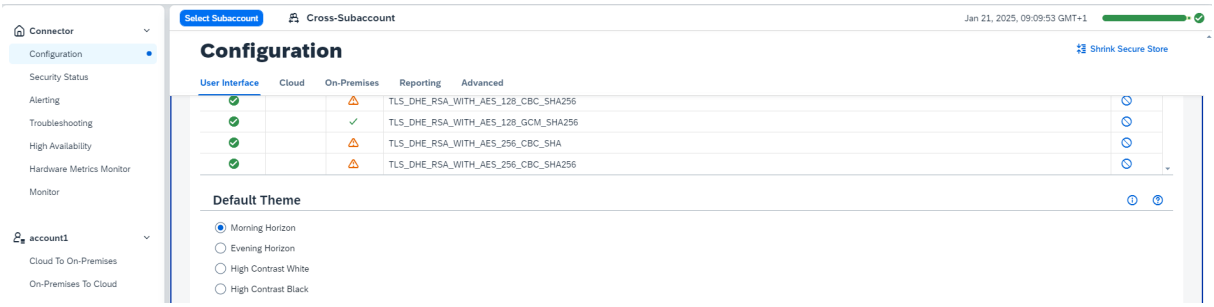
> ### Note
>
> Tag `br` does not require a closing tag as there cannot be any inner HTML.

1.4.2.15 Theming

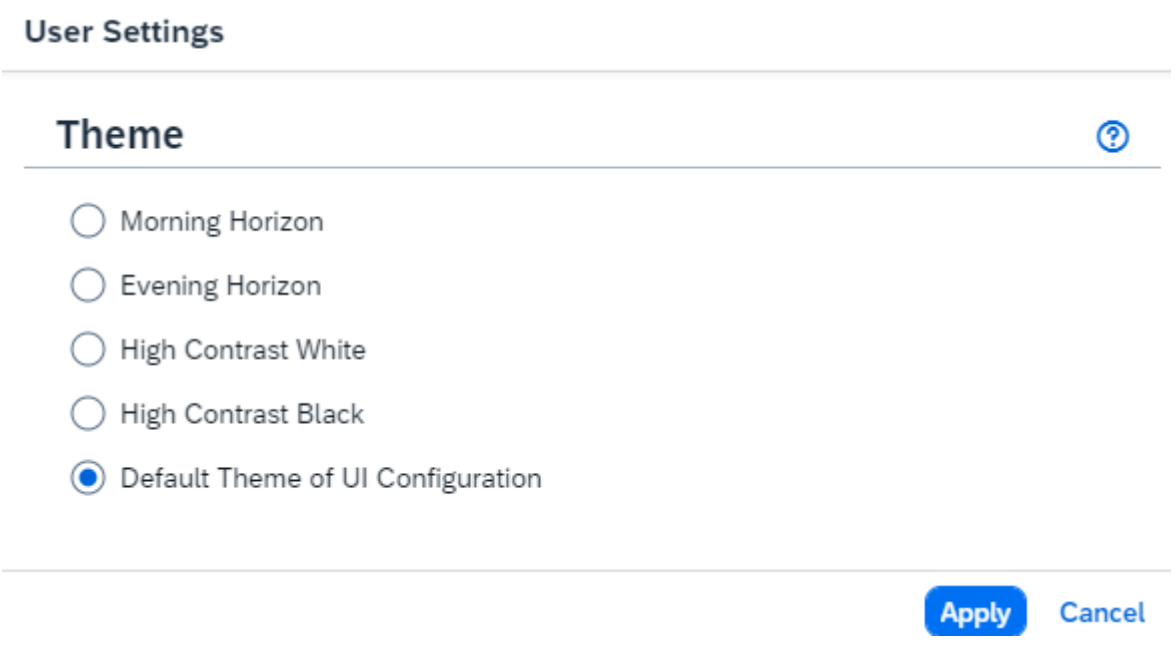
Theming in the Cloud Connector lets you customize UI design by choosing from four Horizon themes.

Theming or themes determine the general design or look of a UI. As of version 2.18.0, the Cloud Connector supports the Horizon themes *Morning Horizon*, *Evening Horizon*, as well as the two *High Contrast* variants *Black* and *White*. The default is *Morning Horizon*.

To change the default theme, choose *Configuration* from the main menu, go to tab *User Interface*, and select the desired theme from the *Default Theme* section (you may have to scroll down):



The default theme is applied to all pages that are not associated with a user (such as the login page) as well as pages associated with a user (that is, any page displayed while a user is logged in) unless you override the default theme through the respective user settings. To do this, open the drop-down button in the top right corner of the UI labeled with the user name and select *Settings*, which will open the following dialog:



Choose any of the four themes to override the default theme. You may revert to the default theme (chosen by the administrator) at any time.

> **Note**
>
> Changes to the default theme or to the user-specific theme that you made in one browser will not take effect in the UI of another browser unless you log out or press the *Reload* button of the browser (that is, the *Reload* button typically located in the top *left* corner, not to be confused with the *Reload* button of the *Cloud Connector UI* located in the top *right* corner).

1.4.3 Administration

Learn more about operating the Cloud Connector, using its administration tools and optimizing its functions.

Topic	Description
Exchange UI Certificates in the Administration UI [page 1971]	By default, the Cloud Connector includes a self-signed UI certificate. It is used to encrypt the communication between the browser-based user interface and the Cloud Connector itself. For security reasons, however, you should replace this certificate with your own one to let the browser accept the certificate without security warnings.
Logon to the Cloud Connector via Client Certificate [page 1973]	Switch from default logon to client certificate logon to access the Cloud Connector.
Configure Named Cloud Connector Users [page 1976]	If you operate an LDAP server in your system landscape, you can configure the Cloud Connector to use the named users who are available on the LDAP server instead of the default Cloud Connector users.
High Availability Setup [page 1995]	The Cloud Connector lets you install a redundant (shadow) instance, which monitors the main (master) instance.
Change the UI Port [page 2003]	Use the changeport tool (Cloud Connector version 2.6.0+) to change the port for the Cloud Connector administration UI. .
Connect and Disconnect a Cloud Subaccount [page 2003]	As a Cloud Connector administrator, you can connect the Cloud Connector to (and disconnect it from) the configured cloud subaccount.
Secure the Activation of Traffic Traces [page 2004]	Tracing of network traffic data may contain business critical information or security sensitive data. You can implement a "four-eyes" (double check) principle to protect your traces (Cloud Connector version 1.3.2+).
Monitoring [page 2006]	Use various views to monitor the activities and state of the Cloud Connector.
Alerting [page 2038]	Configure the Cloud Connector to send email alerts whenever critical situations occur that may prevent it from operating.

Topic	Description
Audit Logging [page 2041]	Use the auditor tool to view and manage audit log information (Cloud Connector version 2.2+).
Monitoring, Logging, and Troubleshooting [page 2046]	Information about monitoring the state of open tunnel connections in the Cloud Connector. Display different types of logs and traces that can help you troubleshoot connection problems.
Process Guidelines for Hybrid Scenarios [page 2054]	How to manage a hybrid scenario, in which applications running on SAP BTP require access to on-premises systems using the Cloud Connector.
Configuring Backup [page 2057]	Find an overview of backup procedures for the Cloud Connector.
Secure Store [page 2057]	Reduce the size of the Cloud Connector Secure Store.

1.4.3.1 Exchange UI Certificates in the Administration UI

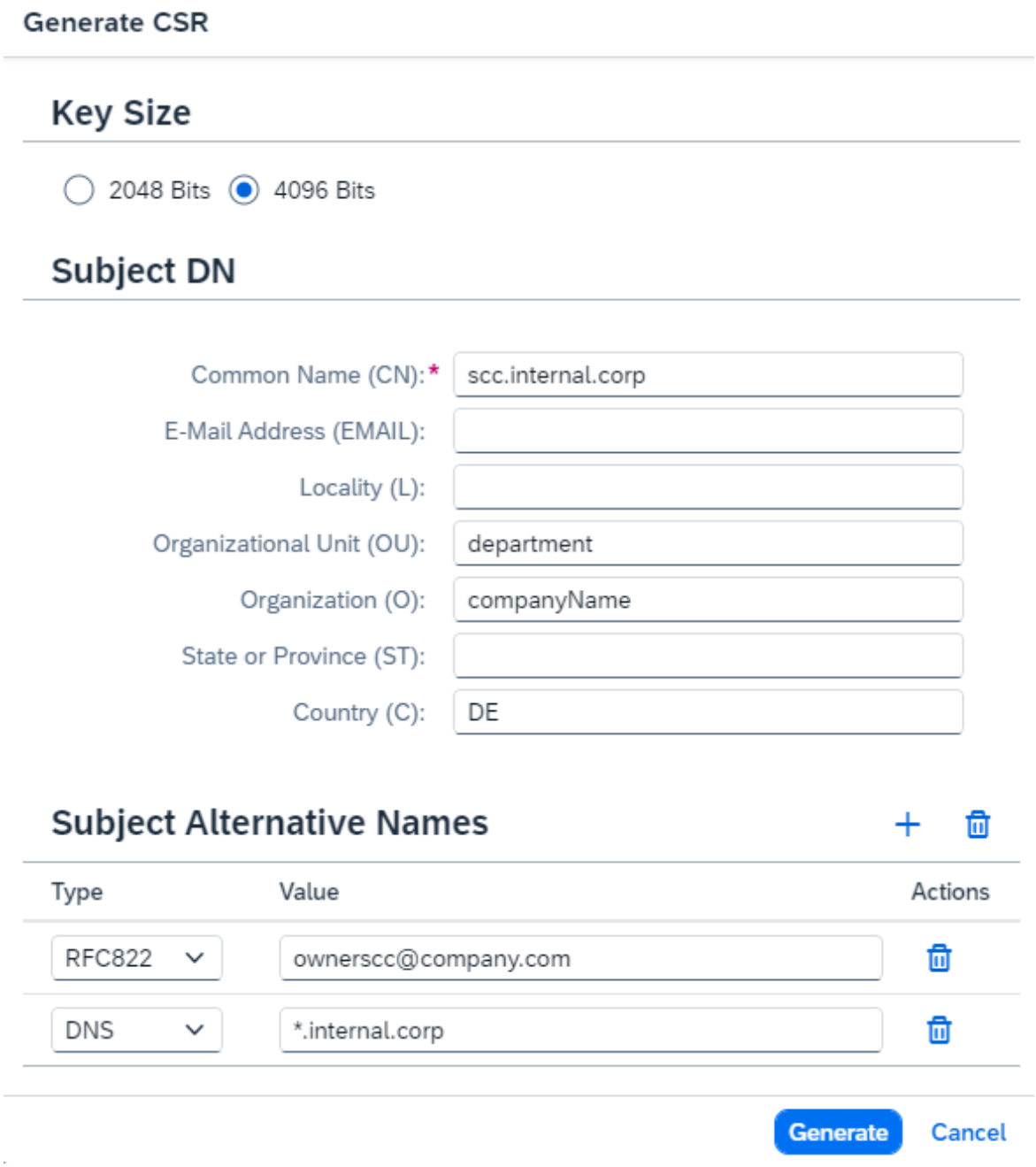
By default, the Cloud Connector includes a self-signed UI certificate. It is used to encrypt the communication between the browser-based user interface and the Cloud Connector itself. For security reasons, however, you should replace this certificate with your own one to let the browser accept the certificate without security warnings.

Procedure

Master Instance

1. From the main menu, choose *Configuration* and go to the *User Interface* tab.
2. In the *UI Certificate* section, start a certificate signing request procedure by choosing the icon *Generate a Certificate Signing Request*.
3. In the pop-up *Generate CSR*, specify a key size and a subject fitting to your host name.
   For host matching, you should use the available names within the `subjectAlternativeName` (SAN) extension, see RFC 2818 and *https://www.chromestatus.com/feature/4981025180483584*. A check verifies whether the host matches one of the entries in the SAN extension.
   In section **Subject Alternative Names**, you can add additional values by pressing the *Add* button. Choose one or more of the following SAN types and provide the matching values:
   - DNS: a specific host name (for example, `www.sap.com`) or a wildcard hostname (for example, `*.sap.com`).
   - IP: an IPv4 or IPv6 address.
   - RFC822 : an example for this type of value is a simple email address: for example, `donotreply@sap.com`.

- URI: a URI for which the certificate should be valid.



4. Press *Generate*.

> **Note**
>
> When starting the procedure, a previously triggered *Certificate Signing Request* will be invalidated and a response for it can no longer be imported, as a new key pair will be generated and the old private key will be deleted.

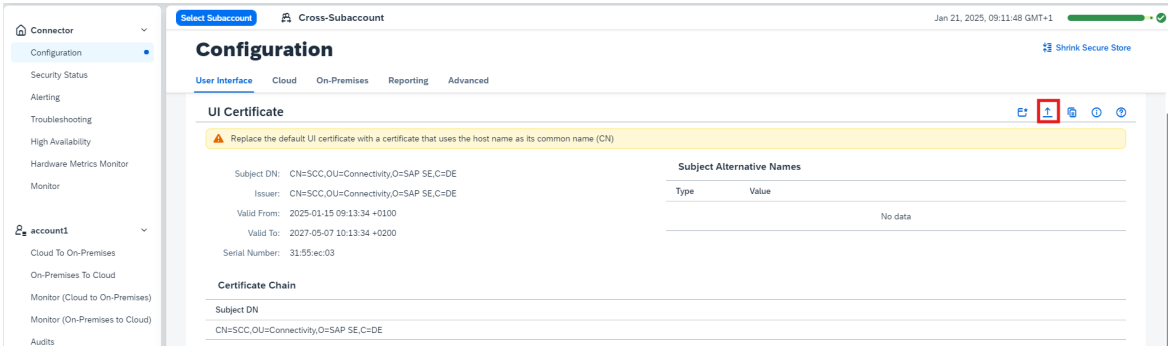
5. You are prompted to save the signing request in a file. The content of the file is the signing request in PEM format.

The signing request must be provided to a Certificate Authority (CA) - either one within your company or another one you trust. The CA signs the request and the returned response should be stored in a file.

> **Note**
>
> The response should be either an X.509 certificate or a PKCS#7 in PEM format.

6. To import the signing response, choose the *Upload* icon.

> **Note**
>
> You can also upload an existing PKCS#12 certificate directly (instead of generating a CSR).



7. Select *Browse* to locate the file and then choose the *Import* button.
8. Review the certificate details that are displayed.
9. Restart the Cloud Connector to activate the new certificate.

Shadow Instance

In a High Availability setup, perform the same operation on the shadow instance.

> **Caution**
>
> UI certificates are used for the secure communication between master and shadow instances. Replacing the UI certificate breaks the trust relationship and communication between master and shadow is not possible anymore.
>
> Please disconnect the shadow instance when you are going to replace UI certificate(s). Once the certificate update is done, connect the shadow instance again. You will be forced to enter user and password again to establish the trust relationship between master and shadow instances.

1.4.3.2 Logon to the Cloud Connector via Client Certificate

Switch from default logon to client certificate logon to access the Cloud Connector.

⚠ Caution

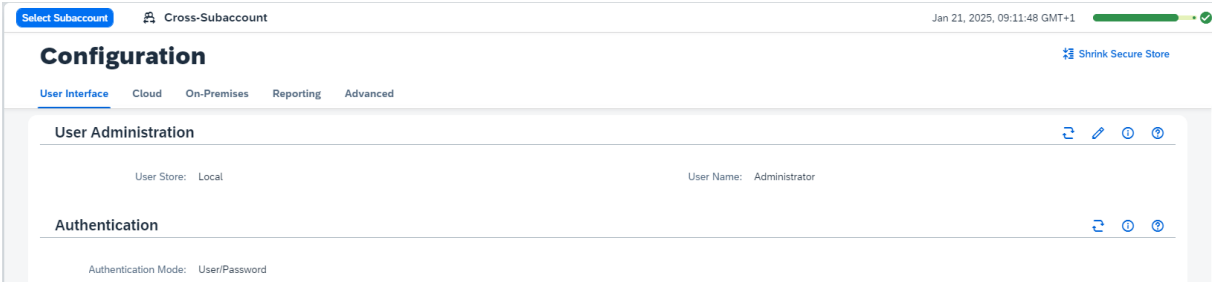
If *high availability* (HA) setup is active, this feature only works if a separate HA port has been specified (for information on how to specify a separate port for *high availability*, see Install a Failover Instance for High Availability [page 1996]).

Otherwise, a shadow will not be able to connect due to missing trust which will result in this error: *There is no trust with Cloud Connector on https://<host>:<port>. Reset shadow configuration and try to connect shadow again*.

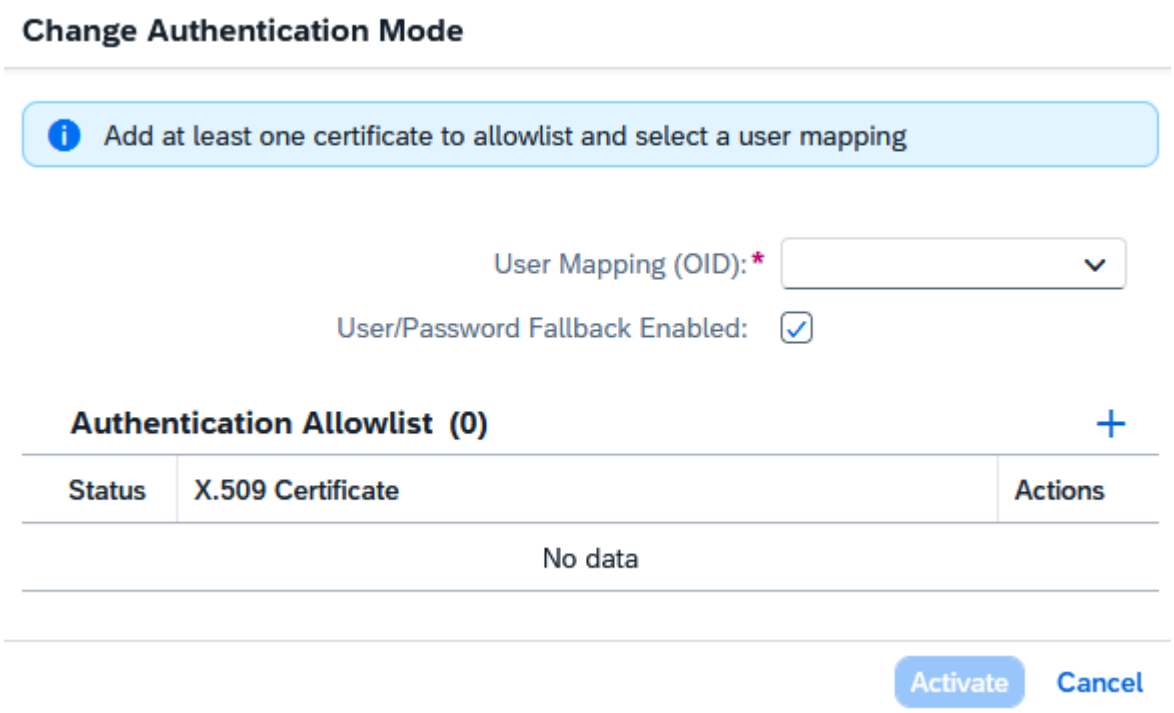
⚠ Restriction

Configuring the logon with a certificate is not allowed for SAP-operated Cloud Connectors, for example in the context of *Enterprise Cloud Services* or *S/4HANA Private Cloud Edition*.

Instead of authenticating with user and password as configured by default (see Initial Configuration [page 1670]), you can employ client certificate authentication and logon. To do so, choose *Configuration* (or *Shadow Configuration*) from the main menu, and tab *User Interface*, section *Authentication*.



In the pop-up dialog, specify an object identifier (OID) to extract the user name from subject of any given client certificate.



You must add at least one trusted certificate to the *Authentication Allowlist*. These certificates are used by Cloud Connector to validate an incoming client certificate. Use root or issuer or intermediate certificates of eligible client certificates, or the client certificates themselves if they are self-signed. Check the *User/Password Fallback Enabled* checkbox for a mixed mode that also permits user/password logon if no client certificate is supplied by the browser, or uncheck it for strict certificate logon without any recourse to basic authentication through user and password.

After activation, a restart is required.

The login works as follows:

1. (Prerequisite): Your browser (or your REST client) must provide a suitable client certificate. In particular, the user extracted from such a client certificate as per the selected *User Mapping (OID)* must be a member of the current user store (local or LDAP).
2. When accessing the Cloud Connector, an mTLS handshake is performed, that is, the Cloud Connector checks your selected client certificate against the CAs from the authentication allowlist.

   > ⚠ Caution
   >
   > If your client certificate is not self-signed, you must provide (one of) the issuer certificates in the *authentication allowlist*, not the client certificate from your browser.

3. If trust can be established, the Cloud Connector extracts the user from the subject as per the specified user mapping OID. This user will be logged on if it can be found in the current user store. If LDAP is used, the respective roles are checked and assigned.

If you select, for example, *CN* as OID, and have a client certificate with a subject containing *CN=Administrator*, the user *Administrator* is used after mutual trust was established.

> ### Note
>
> You can reset certificate-based authentication and revert to basic authentication with user and password by running the script *useBasicAuthentication* in the root directory of your Cloud Connector installation. The Cloud Connector needs to be restarted for basic authentication to take effect.

1.4.3.3 Configure Named Cloud Connector Users

Set up LDAP-based user management for the Cloud Connector.

LDAP-Based User Management

We recommend that you configure LDAP-based user management for the Cloud Connector to allow only named administrator users to log on to the administration UI.

This guarantees traceability of the Cloud Connector configuration changes via the Cloud Connector audit log. If you use the default and built-in `Administrator` user, you cannot identify the actual person or persons who perform configuration changes. Also, you will not be able to use different types of user groups.

Configuration

If you have an LDAP server in your landscape, you can configure the Cloud Connector to authenticate Cloud Connector users against the LDAP server.

For more information about the supported roles for LDAP, see Use LDAP for User Administration [page 1977].

Once configured, the default Cloud Connector `Administrator` user becomes inactive and can no longer be used to log on to the Cloud Connector.

Related Information

Use LDAP for User Administration [page 1977]
Logon to the Cloud Connector via Client Certificate [page 1973]

1.4.3.3.1 Use LDAP for User Administration

You can use LDAP (Lightweight Directory Access Protocol) to manage Cloud Connector users and authentication.

After installation, the Cloud Connector uses file-based user management by default. Alternatively, the Cloud Connector also supports LDAP-based user management. If you operate one or more LDAP servers in your landscape, you can configure the Cloud Connector to use the LDAP user base.

If LDAP authentication is active, you can assign users or user groups to the following default roles:

User Role	Authorization
`sccadmin`	Administrate the Cloud Connector (unrestricted access to all operations and settings).
`sccassocadmin`	• Administrate all subaccount-related settings unless a subaccount is *managed*. • Administrate certain cross-subaccount settings including, but not limited to, certificates and back-end trust.
`sccsubadmin`	• Administrate all subaccount-related settings unless a subaccount is *managed*.
`sccdisplay`	Access the Cloud Connector administration UI in read-only mode.
`sccsupport`	• Access the Cloud Connector administration UI in read-only mode. • Perform support-related tasks like setting trace levels or creating a thread dump.
`sccmonitoring`	Provides access to the monitoring APIs, and is particularly used by the SAP Solution Manager infrastructure. For more information, see Monitoring APIs [page 2016].

Authorization is checked by the Cloud Connector based on the user role retrieved by the LDAP server.

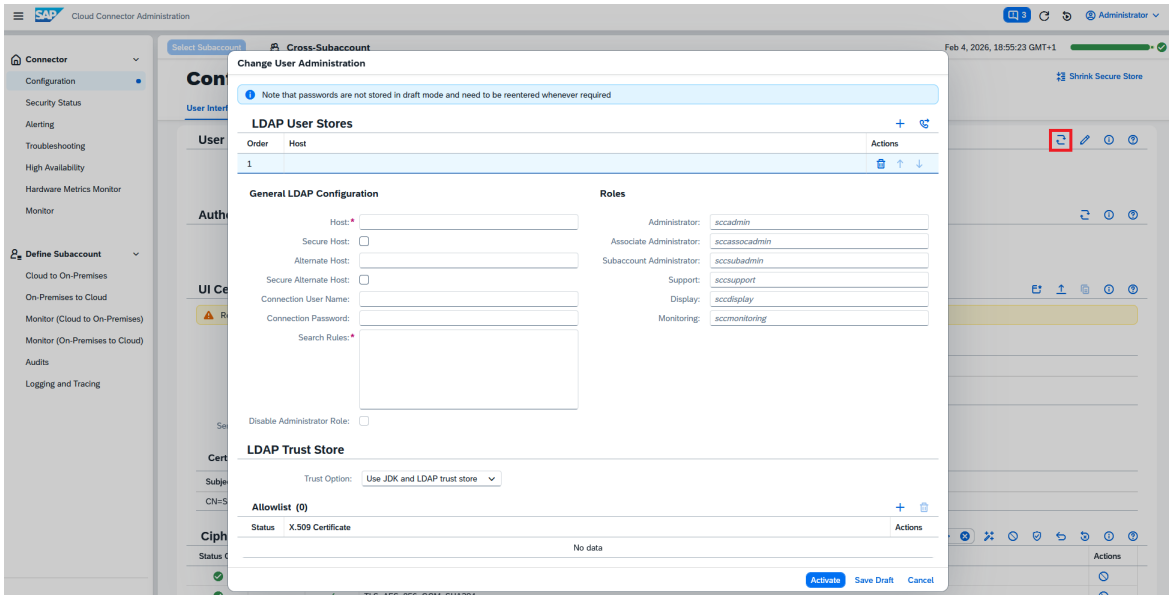
Setting LDAP Authentication

1. From the main menu, choose *Configuration* and go to the *User Interface* tab.
2. From the *User Administration* section, choose *Switch to LDAP*, and enter the required values for your LDAP user store. For detailed information, see LDAP Configuration: Best Practices [page 1981].

> **Note**
>
> The flag *Disable Administrator Role* is only active if you configure more than one LDAP user store. For all user stores except for the first one in the list (*Order* = 1) you can disable the *Administrator* role (default: `sccadmin`).

This option is relevant for managed Cloud Connector installations and lets you prevent the assignment of users or user groups to this role, which would grant full configuration rights for this Cloud Connector instance (like changing the configuration of managed subaccounts).

If you just leave the *Administrator* field in the *Roles* section empty (without checking the *Disable...* flag), the default value `sccadmin` would be assigned automatically to this role.



3. Add further LDAP user stores by pressing the *Add* button (plus icon) of the **LDAP User Stores** table at the top of the dialog.

> ### Note
>
> The number of LDAP user stores is limited to 3. The order of the stores is significant. During logon, the given user will be authenticated by the first LDAP server in that list that succeeds in doing so.

> ### Caution
>
> We strongly discourage you from configuring several LDAP servers to authenticate the same user, in particular with different roles, as this may cause undesirable effects, including security issues, if LDAP servers are temporarily unavailable.

4. Click on a row of the **LDAP User Stores** table to edit the respective LDAP configuration.
5. Column **Actions** lets you delete or move the respective LDAP configuration.
6. (Optional) To save an intermediate LDAP configuration, choose *Save Draft*. This lets you store the changes in the Cloud Connector without activation.
7. Usually, an LDAP server lists users in an LDAP node and user groups in another node. In this case, you can use the following template for LDAP configuration. Copy the template into the configuration text area:

```
roleBase="ou=groups,dc=scc"
roleName="cn"
roleSearch="(uniqueMember={0})"
userBase="ou=users,dc=scc"
userSearch="(uid={0})"
```

8. Change the `<ou>` and `<dc>` fields in `userBase` and `roleBase`, according to the configuration on your LDAP server, or use some other LDAP query.
9. Provide the LDAP server's host and port (port **389** is used by default) in the `<Host>` field. To use the secure protocol variant LDAPS based on TLS, select *Secure*.
10. (Optional) Provide a failover LDAP server's host and port (port **389** is used by default) in the `<Alternate Host>` field. To use the secure protocol variant LDAPS based on TLS, select `<Secure Alternate Host>`.
11. (Optional) Provide a service user and its password in the fields *Connection User Name* and *Connection Password*.
12. (Optional) You can override the roles in the *Custom Roles* section. If no custom role is provided, the Cloud Connector checks permissions for the corresponding default role name:
    - `<Administrator Role>` (default: `sccadmin`)
    - `<Associate Administrator Role>`: If not provided, the Cloud Connector will check permissions for the default role name `sccassocadmin`.
    - `<Sub-Administrator Role>` (default: `sccsubadmin`)
    - `<Support Role>` (default: `sccsupport`)
    - `<Display Role>` (default: `sccdisplay`)
    - `<Monitoring Role>` (default: `sccmonitoring`)
13. Set up the *LDAP Trust Store* as required.
    For more information, see Configure Trust [page 1955].
14. You can execute an authentication test by choosing the *Test LDAP Configuration* button. In the pop-up dialog, you must specify user name and password of a user who is allowed to logon after activating the configuration. The check verifies if authentication would be successful or not for the respective LDAP configuration.

    > **Note**
    >
    > We strongly recommend that you perform an authentication test. If authentication fails, login may not be possible anymore. The test dialog provides a test protocol that can be viewed and downloaded, which can be helpful for troubleshooting.
    >
    > Be advised that such a test queries the selected LDAP server only, not the entire list of servers (if there is more than one). Make sure that at least one of the LDAP user stores succeeds in authenticating a given user that you want to use for logon.

    For more information about how to set up LDAP authentication, see https://tomcat.apache.org/tomcat-9.0-doc/realm-howto.html .
    You can also configure LDAP authentication on the shadow instance in a high availability setup (master and shadow). From the main menu of the shadow instance, select *Shadow Configuration*, go to tab *User Interface*, and check the *User Administration* section.

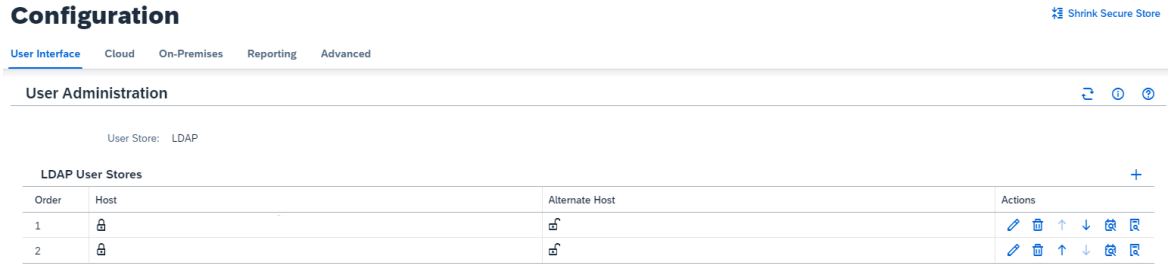
    > **Caution**
    >
    > An LDAP connection over TLS can cause TLS errors if the LDAP server uses a certificate that is not signed by a trusted CA. For information on how to specify a trust store, see Configure Trust [page 1955].

If you want to modify the JDK's trust storage, see the JDK documentation. Usually, the trust storage is `cacerts` inside the java directory (*jre/lib/security/cacerts*). If possible, you can use the *keytool* utility for import:

```
keytool -import -storepass changeit -file <certificate used by LDAP
server> -keystore cacerts -alias <e.g. LDAP_xyz>
```

For more information, see also https://docs.oracle.com/cd/E19830-01/819-4712/ablqw/index.html .

15. After finishing the configuration, choose *Activate*. Immediately after activating the LDAP configuration, a restart of the Cloud Connector server is enforced. After restart, log on to the Cloud Connector with the credentials as per your LDAP configuration.
16. LDAP user stores may be modified, deleted, added, or moved (that is, changed in their order) while LDAP-based user management is active. Navigate to the **LDAP User Stores** table and perform the required actions:



17. To switch back to file-based user management, choose the *Switch* icon in section *User Administration* again.

> ### Note
>
> If you have set up an LDAP configuration incorrectly, you may not be able to logon to the Cloud Connector again. In this case, revert to the file-based user store without the administration UI. For more information, see the next section.

Switching Back to File-Based User Store without the Administration UI

If your LDAP settings do not work as expected, you can use the *useFileUserStore* tool to revert to the file-based user store:

1. Change to the installation directory of the Cloud Connector and enter the following command:
   - **Microsoft Windows**: `useFileUserStore`
   - **Linux, Mac OS**: `./useFileUserStore.sh`

   The tool will inform you about the successful modification of the user store.
2. Restart the Cloud Connector service to activate the file-based user store:
   - **Microsoft Windows OS**: Open the Windows *Services* console and restart the `cloud connector` service.
   - **Linux OS**: Execute
     - System V init distributions: `service scc_daemon restart`

    - Systemd distributions: `systemctl restart scc_daemon`
- **Mac OS X**: Not applicable because no daemon exists (for Mac OS X, only a portable variant is available).

Related Information

LDAP Configuration: Best Practices [page 1981]

1.4.3.3.1.1 LDAP Configuration: Best Practices

Get background information on LDAP configuration for the Cloud Connector.

Introduction [page 1981]

Connect to the LDAP Server [page 1982]

TLS Issues [page 1983]

Authentication [page 1984]

User Selection [page 1987]

User Roles [page 1990]

Relationship between User and Group [page 1992]

Custom User Roles [page 1994]

Additional Notes [page 1995]

Introduction

Using an LDAP server for user management allows seamless integration of the Cloud Connector into the on-premises environment. It requires some configuration that must match the setup on your LDAP server, and therefore can't be generated automatically.

The configuration parameters are common for various products and mostly well known.

The Apache Tomcat project, which is used as underlying technology by the Cloud Connector, provides an excellent tutorial: tomcat.apache.org/tomcat-9.0-doc/realm-howto.html . It explains the LDAP configuration parameters and considers various LDAP directory setups, including their specific configuration.

However, some aspects may raise questions. For this reason, we show you how to configure LDAP and verify LDAP configuration, providing useful background information in this topic.

A basic understanding of LDAP and Tomcat's how-to guide is a prerequisite. As help tool, we are using the `ldapsearch` utility. You can use any LDAP client for this procedure.

Back to Top [page 1981]

Connect to the LDAP Server

In a first step, you must establish a connection to the LDAP server. Like an HTTP connection, the connection to LDAP can be secure (via TLS) or plain. It points to a host and port. The address looks like this:

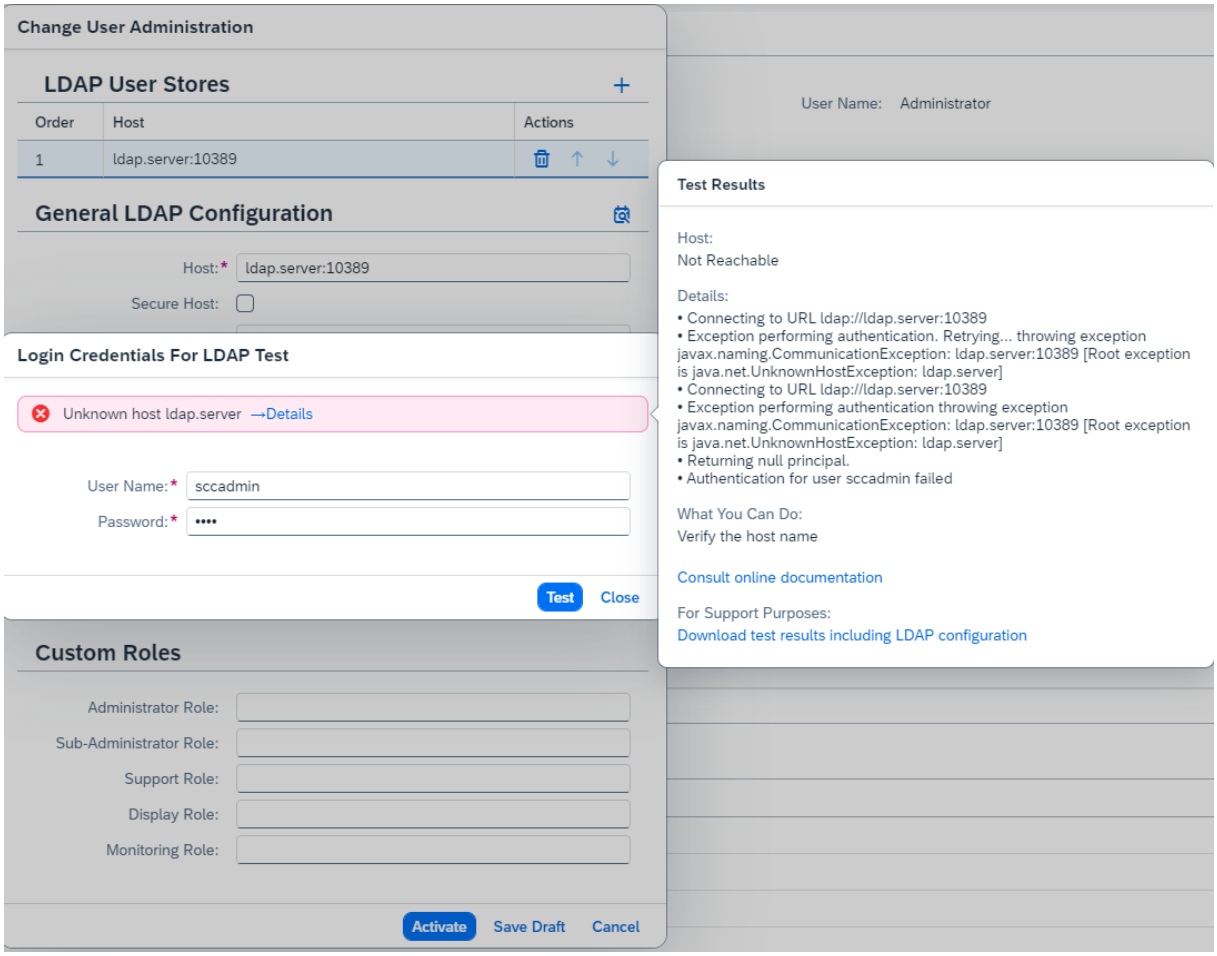
- TLS connection: `ldaps://<ldap.server.in.your.company>:<numeric port>`
- Plain connection: `ldap://<ldap.server.in.your.company>:<numeric port>`

Sample Code

```
ldapsearch -H ldap://<ldaphost>:<port>
```

The return value is `-1` if the address is not reachable. Before you go ahead, you need to know the address of your LDAP server. As soon as the `ldapsearch` utility returns a value other than `-1`, the address of the LDAP server is correct. More precisely, it indicates only that there is a server listening on this port, which is supposed to be the LDAP server.

Once the address is known, you can test the connection in the Cloud Connector. Enter the address and add a dummy configuration, for example, `x="x"`, to outwit the check. Then choose the test icon in the upper right corner. For a valid address, the LDAP configuration test in the Cloud Connector reports the following:



The message: *Connecting to URL ldap://<ldaphost>:<port>; Exception performing authentication* indicates that there is some general connection issue.

Back to Top [page 1981]

TLS Issues

An LDAP connection over TLS will run into TLS errors if the LDAP server uses an "untrusted" certificate. For information on how to specify a trust store, see Configure Trust [page 1955].

If you want to modify the JDK's trust storage, see the JDK documentation.

Usually, the trust storage location is *cacerts* inside the java directory (`jre/lib/security/cacerts`). You can use the `keytool` utility for import.

Sample Code

```
keytool -import -storepass changeit -file <certificate used by LDAP server>
-keystore cacerts -alias <e.g. LDAP_xyz>
```

See also: Working with Certificates and SSL (Java documentation).

Back to Top [page 1981]

Authentication

If the address of the LDAP server is correct and the Cloud Connector can establish a connection, you can proceed with the next step: the authentication by the LDAP directory.

The LDAP server may require authentication or not (anonymous connection), before a query can be executed. Authentication is done by user and password, specified by the `Connection User Name` and `Connection Password` properties.

Anonymous access is sufficient in most cases and provides the same level of security. However, you have to deal with the existing setup on the LDAP server.

Note

The `Connection User` is not the same user that is later used to logon on to the Cloud Connector. It is a specific user, which has permissions to query the LDAP directory. It can be stored in an LDAP directory separated from other users. We recommend that you specify the fully-qualified user name like `"uid=admin,ou=system"` for it.

Alternatively, *MS Active Directory* LDAP servers also allow the format *user@domain*, for example, *ldapUser@company.local*.

To verify the values, let's first check the authentication with `ldapsearch`:

Sample Code

```
ldapsearch -D "uid=admin,ou=system" -w <password> -H ldap://<ldaphost>:<port>
-b "x=x"
```

Note

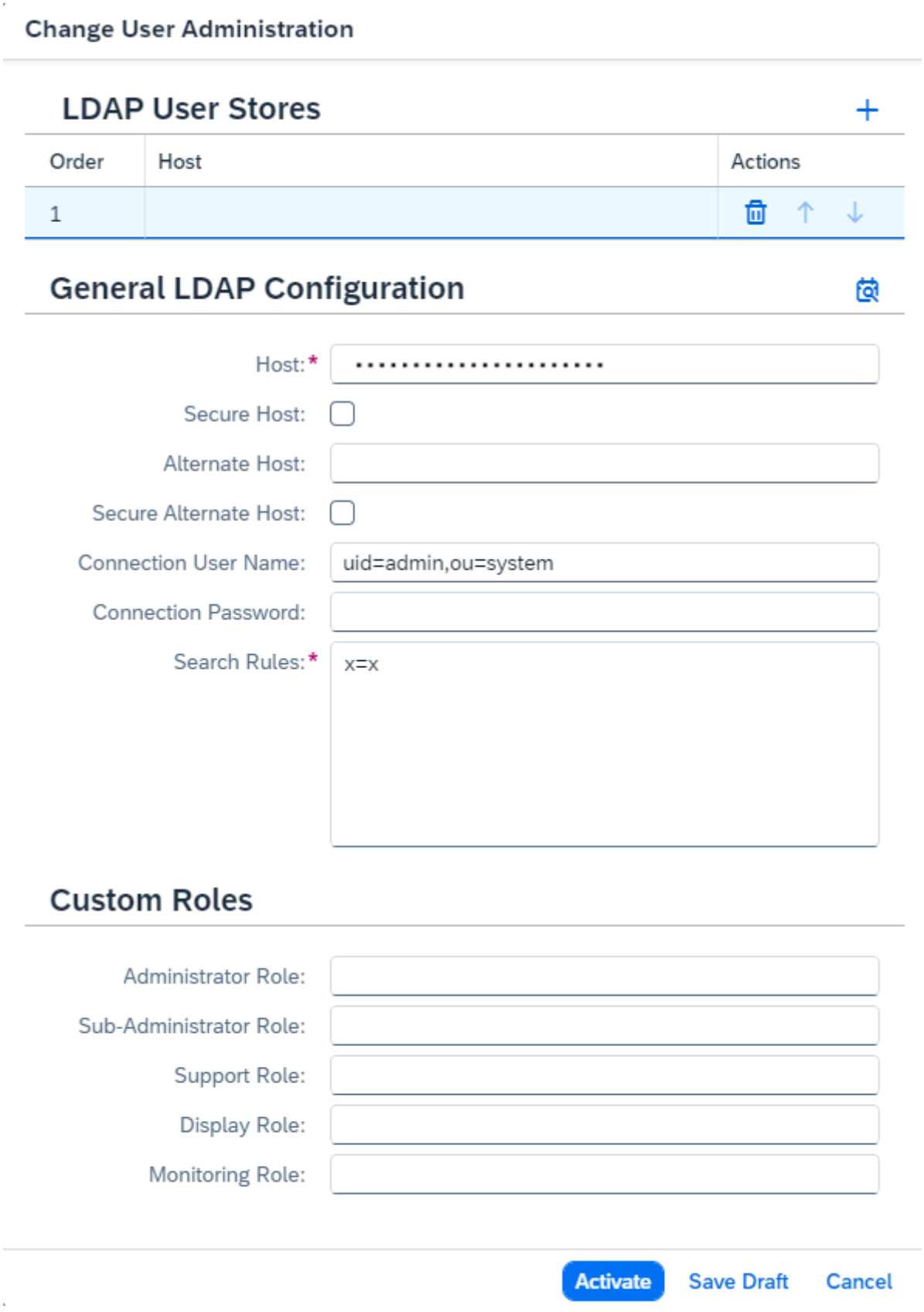
We added a non-existing user base `-b "x=x"` to prevent long output.

The only thing to check here is, if authentication is ok. If it is not, LDAP returns `INVALID_CREDENTIALS: Bind failed`:

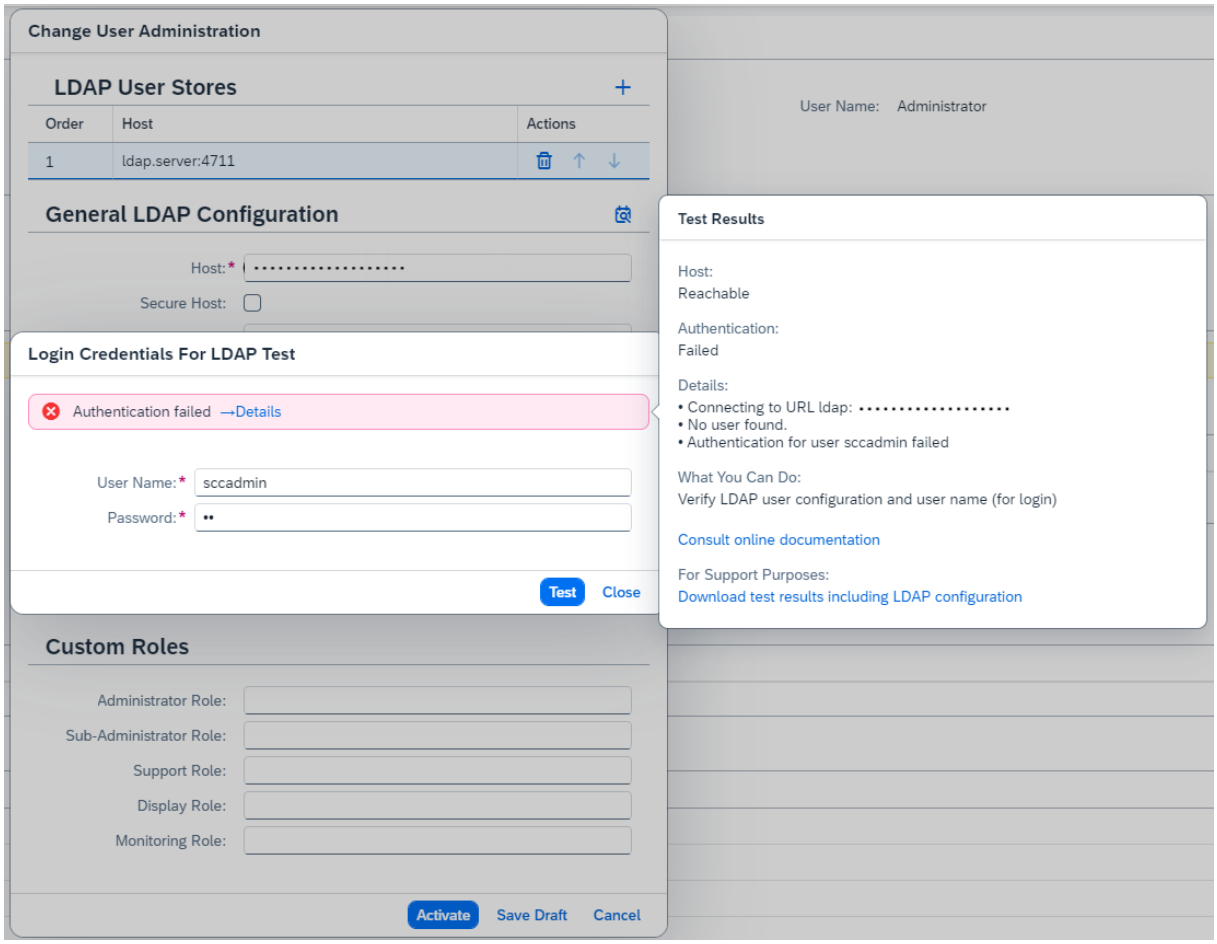
Sample Code

```
ldapsearch -D "uid=admin,ou=system" -w <wrong password> -H ldap://
<ldaphost>:<port> -b "x=x"
ldap_bind: Invalid credentials (49)
additional info: INVALID_CREDENTIALS: Bind failed: ERR_229 Cannot
authenticate user uid=admin,ou=system
```

You can perform the same test in the Cloud Connector UI:


Change User Administration
LDAP User Stores
Order
Host
Actions
1
General LDAP Configuration
Host:*
Secure Host:
Alternate Host:
Secure Alternate Host:
Connection User Name:
uid=admin,ou=system
Connection Password:
Search Rules:*
x=x
Custom Roles
Administrator Role:
Sub-Administrator Role:
Support Role:
Display Role:
Monitoring Role:
Activate
Save Draft
Cancel

Choose the test icon in the upper right corner, and enter something as `User Name` and `Password`:



Authentication failed in this check, but the connection to the LDAP server was successful.

If you get an *Exception performing authentication* message here, check the reply from the LDAP server and align your parameters until you can connect.

Back to Top [page 1981]

User Selection

Once authentication is checked, set the root node for users. User nodes are nodes containing user details. They are located somewhere in the LDAP tree. Sometimes they are all listed under one branch (parent node), but they may also be distributed across several branches. In any case, start with one branch that contains at least one user node.

List the user nodes with `ldapsearch`:

Sample Code

```
ldapsearch -D "uid=admin,ou=system" -w <password> -H ldap://<ldaphost>:<port>
-b "ou=users,dc=scc"
```

The output contains the nodes located under the specified base. Each node looks like this:

Sample Code

```
# Thor, users, scc
dn: uid=Thor,ou=users,dc=scc
sn: SCC Administrator
cn: sccadmin
objectClass: top
objectClass: inetOrgPerson
objectClass: person
objectClass: organizationalPerson
userPassword:: ...
uid: Thor
```

Basically, every unique parameter can be used as user ID. The user *Thor* in this example could also enter its DN (distinguished name) as user name. However, this is not very user-friendly. To not upset Thor, you can define the attribute containing the user ID that is used for logon.

`userSearch` selects the attribute containing the user ID. Together with the `userBase`, the configuration looks like this now:

Sample Code

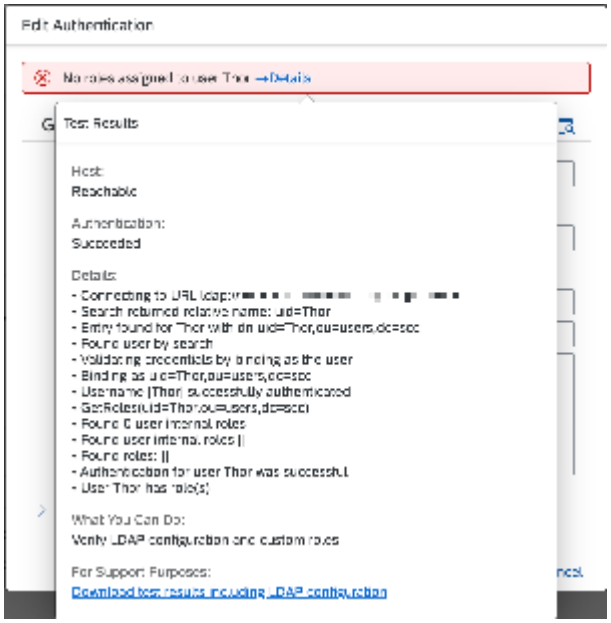
```
userBase="ou=users,dc=scc"
userSearch="(uid={0})"
```

The corresponding LDAP selection is:

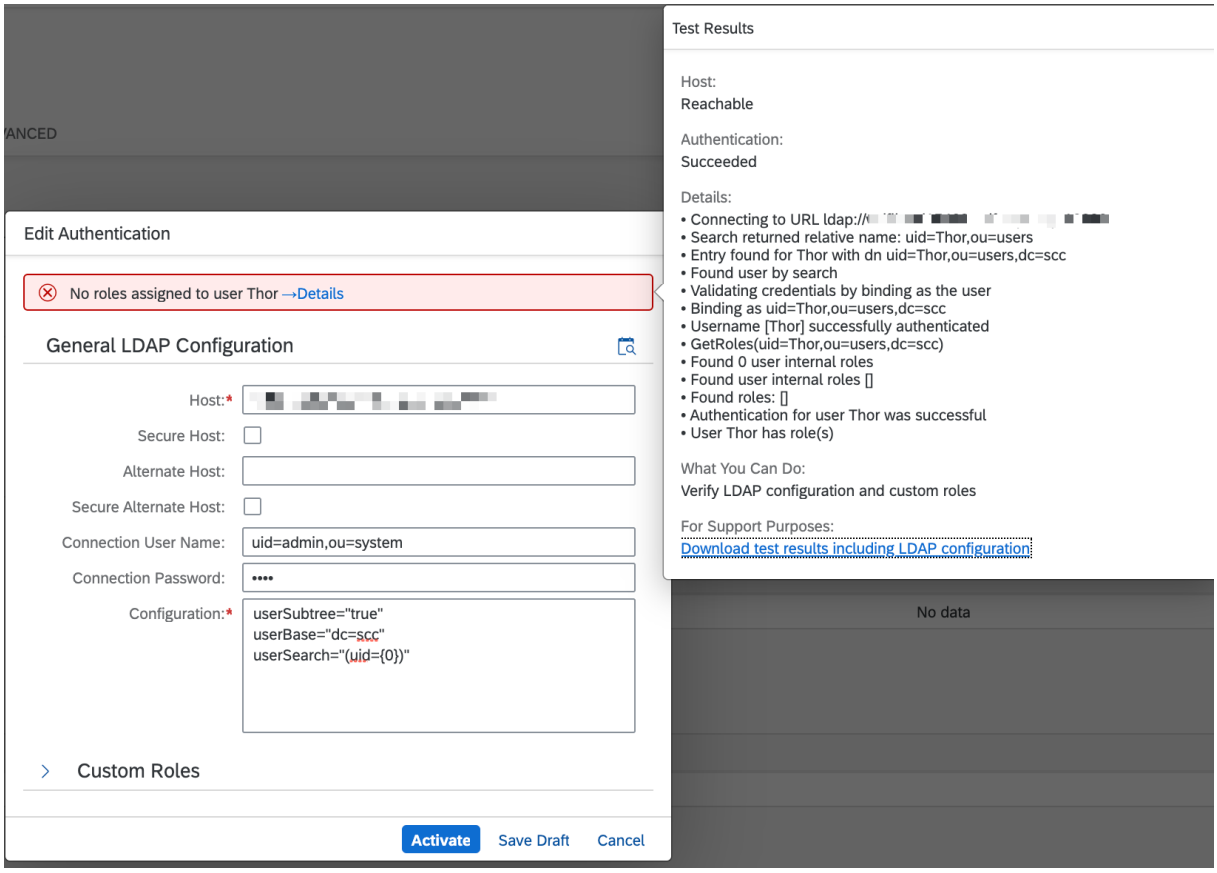
Sample Code

```
ldapsearch -D "uid=admin,ou=system" -w <password> -H ldap://<ldaphost>:<port>
-b "ou=users,dc=scc" "(&(uid=Thor))"
```

Now, we take a closer look now at the response. The user *Thor* was found, its password was successfully validated by the LDAP server, but there are no specific roles selected:



Let's assume that the users are not located under the same branch in the LDAP tree. For this case, you cannot define more than one *userBase*. Instead, you can set *userBase* for the corresponding parent node, which then includes all the user branches. To achieve this, add `userSubtree="true"` to the configuration.



Taking a look at the relative name returned by search, it is `uid=Thor,ou=users` now.

Besides `uid`, you are free to use every other attribute of the user node. For example, for *Active Directory*, the preferred attribute is often `sAMAccountName`, the corresponding `Search Rules` configuration is `userSearch="(sAMAccountName={0})"`.

> ### Note
>
> For *Active Directory*, it might be necessary to add `adCompat="true"` to the `Search Rules` configuration.

As mentioned above, you can use every attribute as user ID as long as it is *unique*. To verify this, check the query result for the respective attribute with `ldapsearch`. If it contains more than one node, the test in the Cloud Connector would report *User name [<userid>] has multiple entries, No user found, Authentication for user <userid> failed*. In our test, the CN (common name) attribute is not unique and the following search returns more than one entry.

> ### Sample Code
>
> ```
> ldapsearch -D "uid=admin,ou=system" -w <password> -H ldap://<ldaphost>:<port>
> -b "ou=users,dc=scc" "(&(cn=<userid>))"
> ```

> ### Note
>
> If `connectionUser` is located under `userBase` and its ID can be selected by the same `userSearch`, you can just enter the user ID in the `Connection User Name` field instead of its fully-qualified DN.

Back to Top [page 1981]

User Roles

At this point, the configuration let's you establish a connection to the LDAP server and authenticate a user.

In the next step, we configure authorizations, that is, the roles assigned to a user.

A role is a *group* in LDAP terms. The Cloud Connector provides the following roles: `sccadmin`, `sccsubadmin`, `sccsupport`, `sccmonitoring` and `sccdisplay`.

Most likely, your LDAP server does not define such groups. Here, the best practice is to create new groups for role assignment. Using these special groups for managing Cloud Connector users lets administrators easily grant permissions to the relevant users. For example, only users with administrator permissions would be added to the `sccadmin` group. Like this, you avoid side effects, and you can increase the security and stability levels of the Cloud Connector.

Reuse of already existing groups is also possible. Set these group names as custom roles in the Cloud Connector's LDAP configuration. However, keep in mind that every user in the existing group will automatically get permissions for the Cloud Connector. Even if at present all users in the available group should have permissions for Cloud Connector, this could cause issues at some point in the future. To avoid this, custom

roles should not be used for reuse of existing groups on your LDAP server. The main purpose of reused groups is to create group names that match your company's naming conventions.

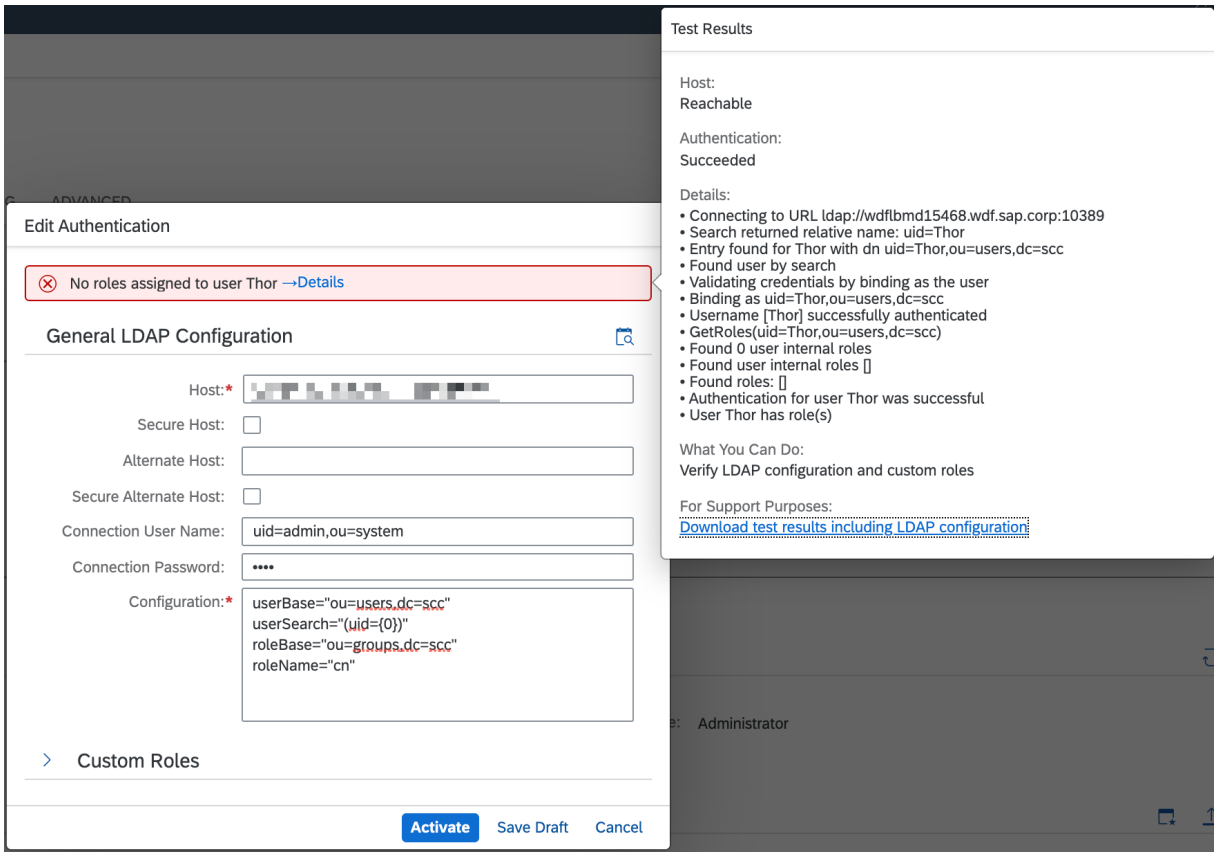
Like users, also groups are represented as nodes in an LDAP tree branch. The branch where the groups are located must be configured as `roleBase`.

The `roleName` defines, which of its attributes is taken as role name. Usually, you wouldn't use the fully-qualified distinguished name as role name.

Check if `ldapsearch` can find entries for the role:

Sample Code

```
ldapsearch -D "uid=admin,ou=system" -w <password> -H ldap://<ldaphost>:<port>
-b "<roleBase>" "(&(cn=<roleName>))"
```



When trying to define only `roleBase` and `roleName`, the test reports no roles for the specified user: *Found roles: []*. The empty brackets indicate an 'empty collection'. The reason is that the configuration does not define how users are related to the found groups. Find some background information on this in the next section.

Back to Top [page 1981]

Relationship between User and Group

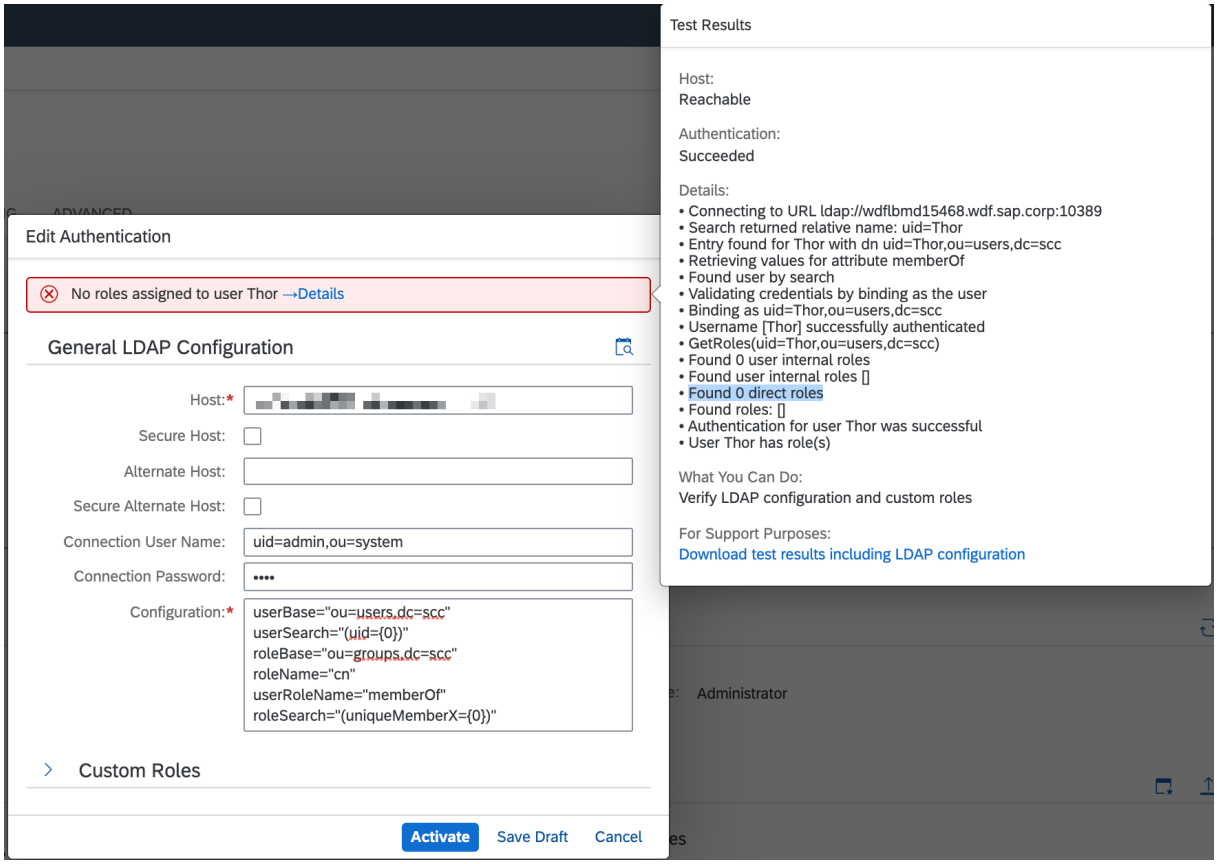
LDAP provides two ways to define the relationship between a user and its groups:

1. The group node contains one or more attributes `uniqueMember`, or
2. Uses one or more attributes `memberOf` in a user node.

- **Case 1**: Extend the current configuration by `roleSearch="(uniqueMember={0})"`.
  The Cloud Connector's test reports direct roles in this case. Roles selected by `roleSearch` are LDAP entries. Using configuration parameter `roleName`, you specify the LDAP entry attribute which will be the role name.
  In our example, we are using `roleName="cn"` with the following result in the test: *"Found direct role cn=sccadmin,ou=groups,dc=scc -> sccadmin"*. The selected value `sccadmin` is the value of the attribute `cn` located in the original LDAP entry.
- **Case 2**: Extend the current configuration by `userRoleName="memberOf"`.
  This is reported by the Cloud Connector's test as internal role. If no groups are defined by `memberOf`, the internal group list is empty.

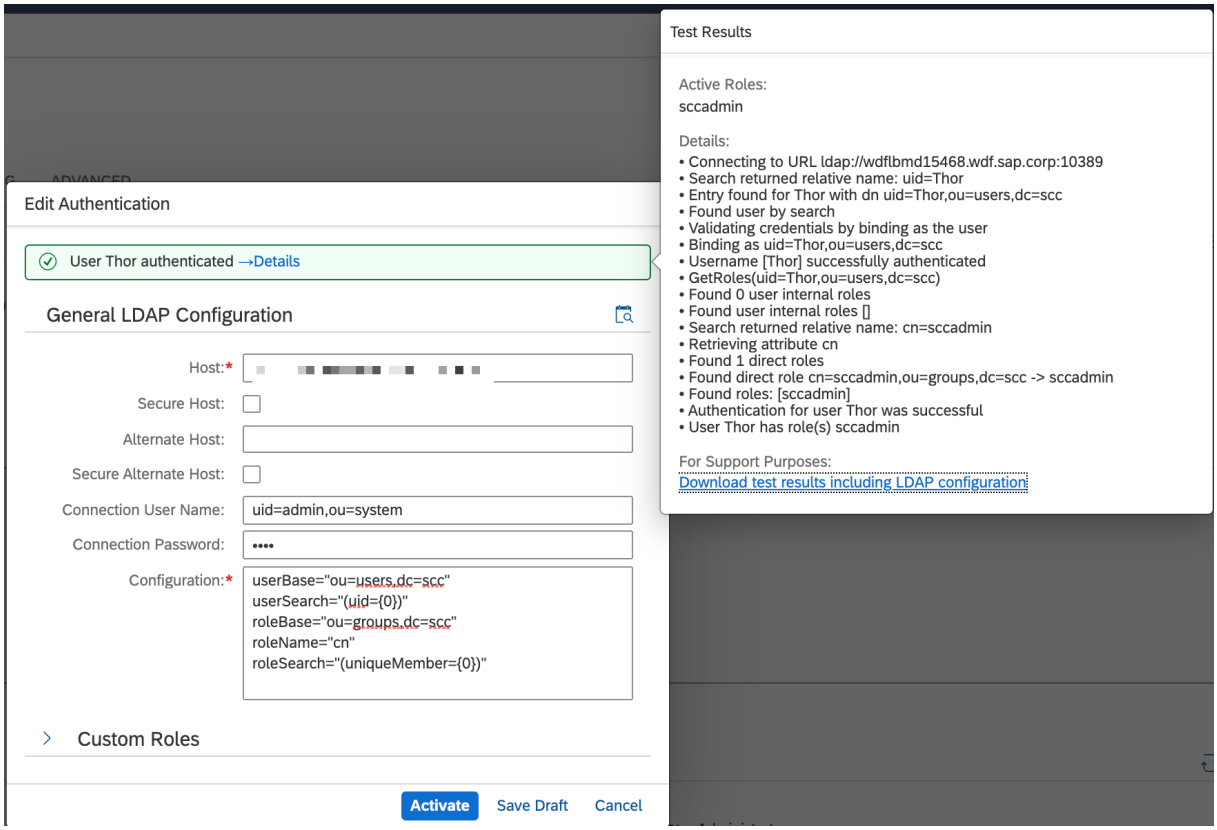
  > → Remember
  >
  > The roles specified by an attribute of a user entry are used as is, that is, the configuration parameter `roleName` is irrelevant in this case, and the role name is just set to the attribute value.

Below, the test report selected neither internal nor direct roles:



To demonstrate an empty result, the parameter `roleSearch` was set to a non-existing attribute here.

Below, the test reflecting LDAP configuration eventually reports a non-empty list of found roles, containing the role `sccadmin`:



> **Note**
>
> Like user nodes, also group nodes on the LDAP server may be located under several branches inside the "base" branch. In this case, add the boolean attribute `roleSubtree="true"`.

Back to Top [page 1981]

Custom User Roles

If you want to use non-default groups, you can set a group of your choice in the **Custom Roles** section. You can replace one or more roles.

> **Note**
>
> Keep in mind that the custom role definition replaces the standard role. So, once a custom role for *Administrator* is set, the standard one (`sccadmin`) is not effective anymore.

Back to Top [page 1981]

Additional Notes

There are some more configuration parameters available that are out of scope here. Most LDAP configurations are covered by the parameters discussed above.

For special purposes, you may have to add to your configuration:

- `adCompat="true"`, if you are using an *MS Active Directory* LDAP server and you encounter strange errors in the test report.
- `forceDnHexEscape="true"`, if you are using an *MS Active Directory* and there are non-standard characters in the DN.
- `connectionTimeout="x"`, if you want to change the default of 5s.

General Recommendations

- Don't use the `userPattern` configuration parameter. It invalidates SSL/TLS-based authentication and high availability setup would fail.
- If the user ID has **non-standard characters**, escape them with `\nn`.
- For **back-slash**, always use `\\`.

Back to Top [page 1981]

1.4.3.4 High Availability Setup

You can operate the Cloud Connector in a high availability mode, in which a master and a shadow instance are installed.

Context

In a failover setup, when the main instance should go down for some reason, a redundant one can take over its role. The main instance of the Cloud Connector is called **master** and the redundant instance is called the **shadow**. The shadow must be installed separately and connected to the master.

During high availability setup, the master pushes the entire configuration to the shadow. Later on, during normal operation, the master also pushes configuration updates to the shadow to keep both instances synchronized. The shadow pings the master regularly. If the master is not reachable for a while, the shadow takes over the master role and establishes the tunnel to SAP BTP.

> ⚠ Caution
>
> Master and shadow communicate outside the cloud (SAP BTP). Therefore, a communication breakdown between master and shadow may trigger a takeover although the original master is still operational, and in particular still connected to SAP BTP.

This may lead to a temporary *double-master* scenario that will be rectified as soon as both instances reestablish communication. During the blackout between the original master and the shadow that became the actual master instance, and the ensuing master-master situation, the behavior is undefined and may lead to issues.

It is therefore imperative to choose a **stable and reliable network environment** for the master and shadow instances.

> **Note**
>
> For detailed information about sizing of the master and the shadow instance, see also Sizing Recommendations [page 1649].

Related Information

Install a Failover Instance for High Availability [page 1996]
Master and Shadow Administration [page 1999]

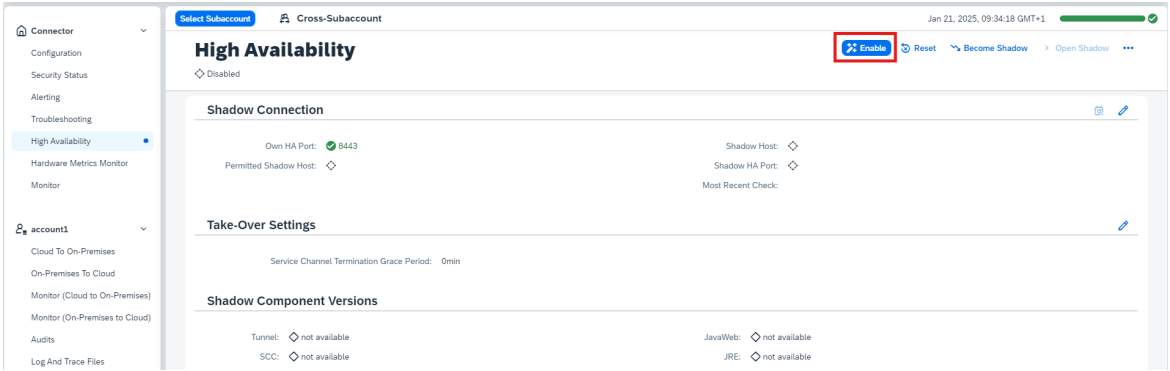
1.4.3.4.1 Install a Failover Instance for High Availability

Install a redundant Cloud Connector instance (shadow instance) that monitors the main instance.

Procedure

Prepare the Master Instance for High Availability

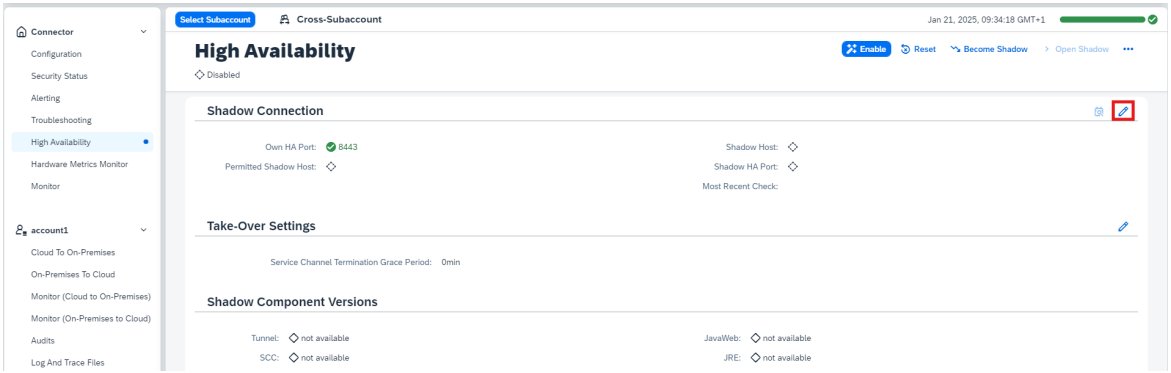
1. Open the Cloud Connector UI and go to the master instance.
2. From the main menu, choose *High Availability*.
3. Choose *Enable*.



If this flag is not activated, no shadow instance can connect to this Cloud Connector. Additionally, by providing a concrete *Shadow Host* you can ensure that only from this host a shadow instance can be connected.

Optional Configuration

1. Choose *Edit*.



2. You can now change the high availability (HA) port to a different one from where the Cloud Connector administration UI is accessible. This port is then used for the master/shadow communication only, that is, to check for failovers or push configuration. This is especially needed in case of a Logon to the Cloud Connector via Client Certificate [page 1973], so that master/shadow communication is still possible without having a client certificate for logon.
3. Additionally, by providing a concrete shadow host you can ensure that a shadow instance can be connected from this host only. The host name will be shown as *Permitted Shadow Host* on the HA screen if you have configured one.

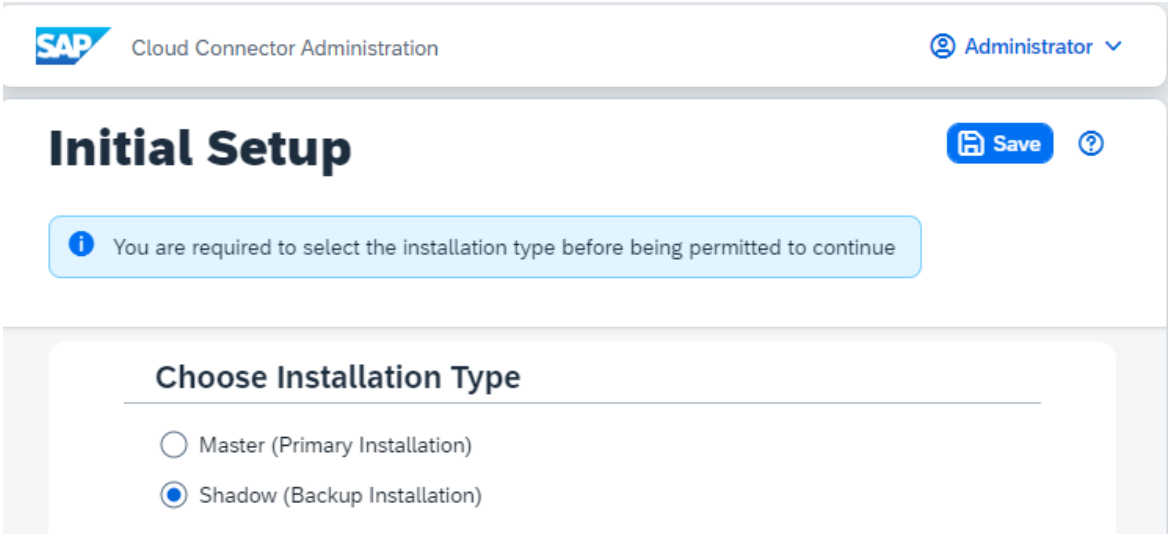
> ⚠ Caution
>
> Pressing the *Reset* button resets all high availability settings (besides the own port configuration) to their initial state. As a result, high availability is disabled and the shadow host is cleared. Reset only works if no shadow is connected. Also, editing the settings for HA will only work if no shadow is connected.

Install and Set Up a Shadow Instance

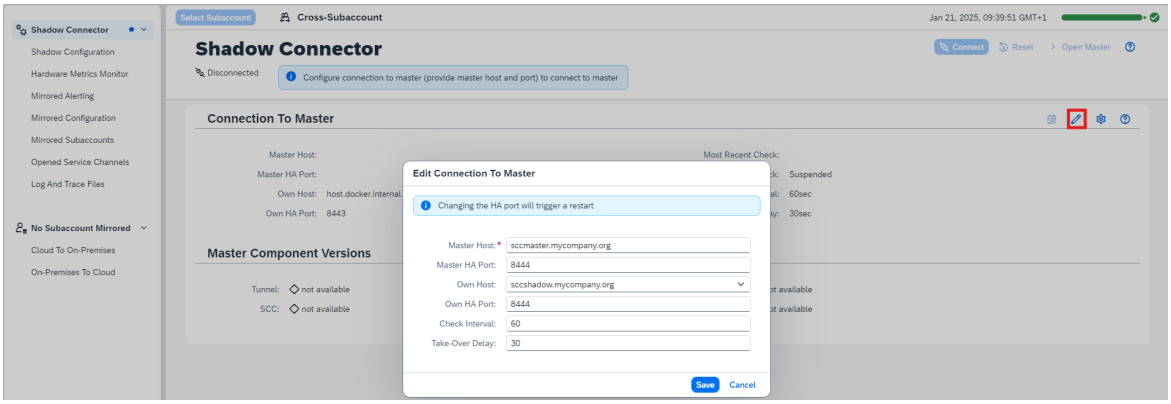
Install the shadow instance in the same network segment as the master instance. Communication between master and shadow via proxy is not supported. The same distribution package is used for master and shadow instance.

> ⓘ Note
>
> If you plan to use LDAP for the user authentication on both master and shadow, make sure you configure it **before** you establish the connection from shadow to master.

1. On first start-up of a Cloud Connector instance, a UI wizard asks you whether the current instance should be master or shadow. Choose *Shadow* and *Save*:



2. From the main menu, choose *Shadow Connector* and provide connection data for the master instance, that is, the master host and port. Optionally, you can change the high availability (HA) port to a different one from where the Cloud Connector administration UI is accessible. This port is then used for the master/shadow communication only, that is, to check for failovers or push configuration. This is especially needed in case of a Logon to the Cloud Connector via Client Certificate [page 1973] so that master/shadow communication is still possible without having a client certificate for logon.
   Additionally, you can choose from the list of known host names, to use the host name under which the shadow host is visible to the master. You can specify a host name manually, if the one you want is not on the list. For the first connection, you must log on to the master instance, using the user name and password for the master instance. The master and shadow instances exchange X.509 certificates, which will be used for mutual authentication.



> ### Note
>
> If you want to attach the shadow instance to a different master, press the *Reset* button. All your high availability settings (besides the own port configuration) will be removed, that is, reset to their initial state. This works only if the shadow is not connected.

3. Upon a successful connection, the master instance pushes the entire configuration plus some information about itself to the shadow instance. You can see this information in the UI of the shadow instance, but you can't modify it.
4. The UI on the master instance shows information about the connected shadow instance. To display this information, choose *High Availability* from the main menu.

5. If configuration changes have not been pushed successfully, alert messages are generated in the general **Alerting** section. This might happen, for example, if a temporary network failure occurs at the same time a configuration change is made. As an administrator, you can check if there is an inconsistency in the configuration data between master and shadow that could cause trouble if the shadow needs to take over. Typically, the master recognizes this situation and tries to push the configuration change at a later time automatically. If this is successful, all failure alerts are removed and replaced by a warning alert showing that there had been trouble before.
   If the master doesn't recover automatically, disconnect, then reconnect the shadow, which triggers a complete configuration transfer.

Related Information

Initial Configuration [page 1670]
Master and Shadow Administration [page 1999]

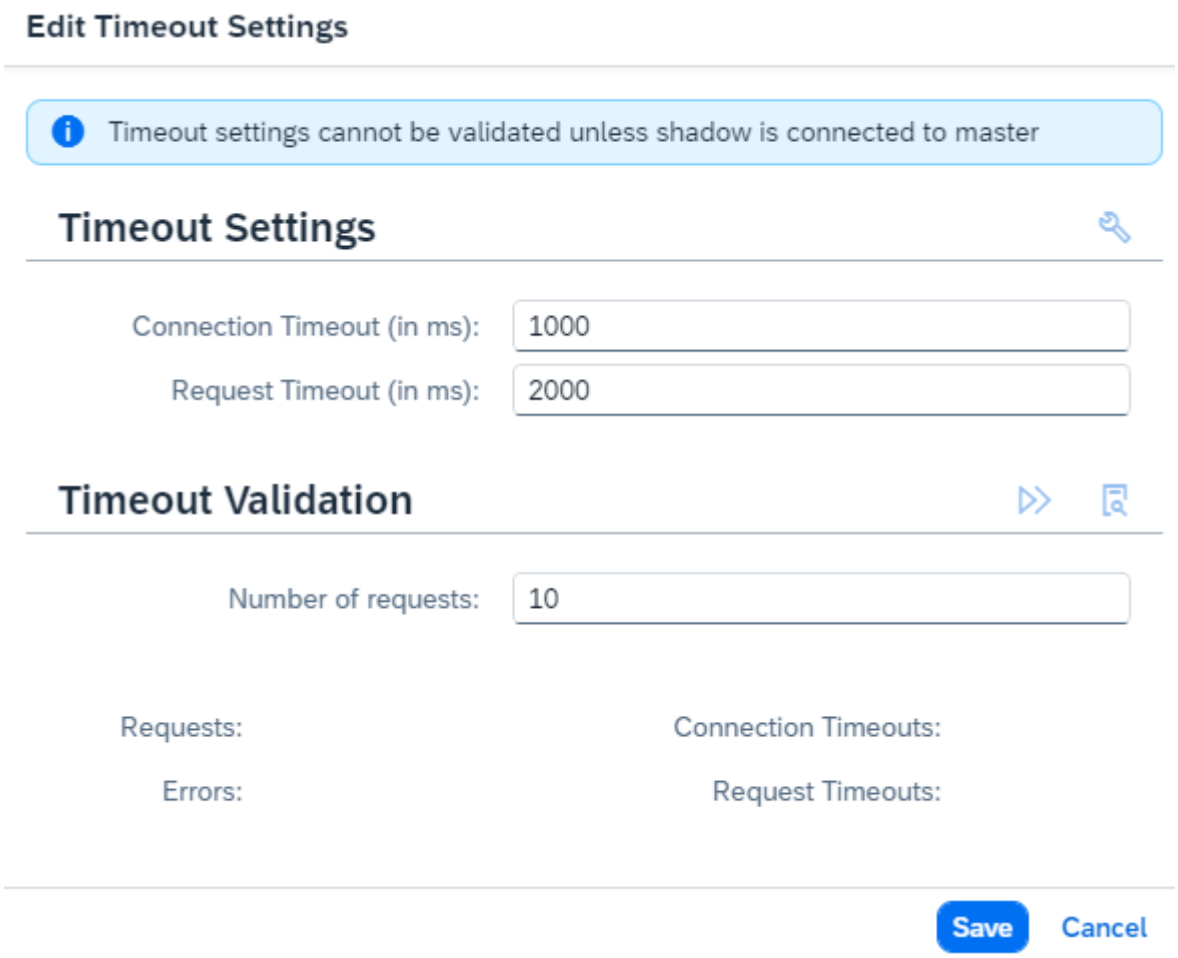
1.4.3.4.2 Master and Shadow Administration

Manage the Cloud Connector master and shadow instances in a high availability setup.

Administration of Shadow Instances

There are several administration activities you can perform on the shadow instance. All configuration of tunnel connections, host mappings, access rules, and so on, must be maintained on the master instance; however, you can replicate them to the shadow instance for display purposes. You may want to modify the **check interval** (time between checks of whether the master is still alive) and the **takeover delay** (time the shadow waits to see whether the master would come back online, before taking over the master role itself).

Also, you can configure the timeout for the connection check, by pressing the gear icon in the section *Connection To Master* of the shadow connector main page.



The `<Connection Timeout>` field specifies the maximum time allowed for establishing the technical connection to the master, the `<Request Timeout>` defines the maximum time allowed for executing the check over this connection. In the *Timeout Validation* section, you can check the current settings by pressing the *Execute* button. If the timeouts are hit, you might want to increase the settings accordingly.

Keep in mind the following points:

- The log level on master and shadow instances can be different.
- Configuration for check interval and takeover delay is maintained only on the shadow instance, and is transferred to the master for display purposes.
- Audit logs are only written on the master instance and are not transferred to the shadow. However, if the shadow becomes the master for some time, the audit log is potentially distributed over both master and shadow instances.

You can use the *Reset* button to drop all the configuration information on the shadow that is related to the master, but only if the shadow is not connected to the master.

As of version 2.17, the shadow instance lets you monitor Hardware Metrics [page 2007] just like the master instance.

Required Configuration on the Shadow Instance

Once connected to the master, the shadow instance receives the configuration from the master instance. Yet, there are some aspects you must configure on the shadow instance separately:

- **User administration** is configured separately on master and shadow instances. Generally, it is not required to have the same configuration on both instances. In most cases, however, it is suitable to configure master and shadow in the same way.
- The **UI certificate** is not shared. Each host can have its own certificate, so you must maintain the UI certificates on master and shadow. You can use the same certificate though.
- **SNC configuration**: If secure RFC communication or principal propagation for RFC calls is used, you must configure SNC on each instance separately.

Failover Process

The shadow instance regularly checks whether the master instance is still alive. If a check fails, the shadow instance first attempts to reestablish the connection to the master instance for the time period specified by the takeover delay parameter.

- If no connection becomes possible during the takeover delay time period, the shadow tries to take over the master role. At this point, it is still possible for the master to be alive and the trouble to be caused by a network issue between the shadow and master.
  The shadow instance next attempts to establish a tunnel to the given SAP BTP subaccount. If the connection attempt fails to all configured subaccounts (for whatever reason), the shadow instance remains in "shadow status", periodically pinging the master and trying to connect to the cloud, while the master is not yet reachable.
- Otherwise, if the the tunnel to the cloud side can be opened, the shadow instance will take over the master role. From this moment, the shadow instance displays the UI of a master instance and offers the usual operations of a master instance, for example, starting/stopping tunnels, modifying the configuration, an so on.
  This is, in particular, also the case if the master is still alive, but the network connection between shadow and master is corrupted. This leads to a *master-master* setup, which is automatically detected on the former master once the network between the two instances recovers. The former master will then automatically relinquish its master role and assume the shadow role to get back to the desired setup.

When the original master instance restarts, it first checks whether the registered shadow instance has taken over the master role. If it has, the master registers itself as a shadow instance on the former shadow (now master) instance. Thus, the two Cloud Connector installations, in fact, have switched their roles.

> ### Note
>
> Only one shadow instance is supported. Any further shadow instances that attempt to connect are declined by the master instance.

The master considers a shadow as lost, if no check/ping is received from that shadow instance during a time interval that is equal to three times the check period. Only after this much time has elapsed can another shadow system register itself.

> ### Note
>
> On the master instance, you can manually trigger failover (if a shadow instance is present) by selecting the *Switch Roles* button.
>
> A zero downtime is not guaranteed. Depending on various aspects and timings there can be short time slots in which establishing new connections fails. All requests currently being active on the master will be aborted when switching the role as the sockets will be closed.

> ### Note
>
> An attempt to switch roles will be rejected if none of the subaccounts can be connected on the shadow side as this may indicate that the shadow, which is to take over as master, will not be able to process requests. However, the inability to connect subaccounts may simply be due to expired subaccount certificates which is not necessarily an issue while subaccounts are connected, but will most certainly become an issue when attempting to connect. Therefore, make sure the subaccount certificates are valid before switching roles.

> ### Note
>
> On the master instance, you can force a role switch to assume the shadow role even if no shadow is connected. To do so, press the *Become Shadow* button (which is the re-labeled *Switch Roles* button). Enforce the role change only if you are absolutely sure that this is the correct procedure. Should you end up without any master instance after all, use the script *changerole* located in the root installation directory to fix the issue.

> ### Caution
>
> In the rare situation when both master and shadow are not functional, and the shadow instance is being started while the master instance is still not available, the shadow doesn't trigger the failover process for technical reasons. In this case you must (depending on the situation) either convert the shadow into master, or connect the shadow to the master manually once the the master instance is recovered.

Take-Over Settings

When the shadow instance takes over, by default service channels are immediately closed by the outgoing master instance. This may cause some disruption. It is possible to keep service channels open for a period of time that can be configured as **Service Channel Termination Grace Period** on the **High Availability** screen (master instance). Service channels will remain active during the configured grace period, and will be closed once the grace period expires.

1.4.3.5 Change the UI Port

Context

By default, the Cloud Connector uses port **8443** for its administration UI. If this port is blocked by another process, or if you want to change it after the installation, you can use the `changeport` tool, provided with Cloud Connector version **2.6.0** and higher.

> **Note**
>
> On Windows, you can also choose a different port during installation.

Procedure

1. Change to the installation directory of the Cloud Connector. To adjust the port and execute one of the following commands:
   - Microsoft Windows OS:

     ```
     changeport <desired_port>
     ```

   - Linux OS, Mac OS X:

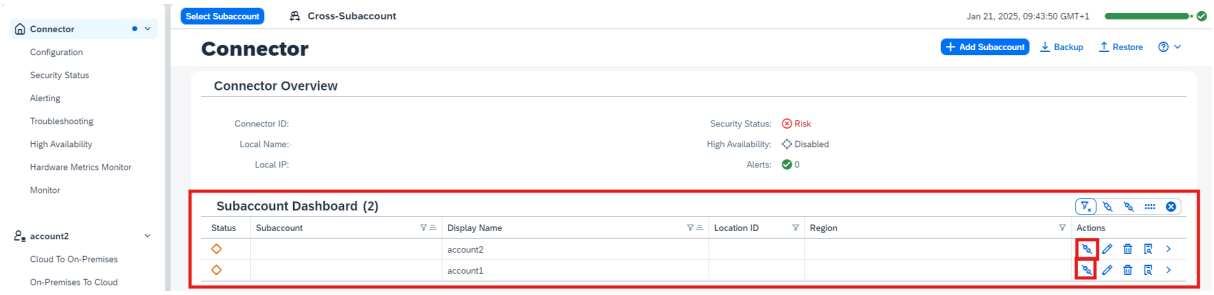
     ```
     ./changeport.sh <desired_port>
     ```

2. When you see a message stating that the port has been successfully modified, restart the Cloud Connector to activate the new port.

1.4.3.6 Connect and Disconnect a Cloud Subaccount

The major principle for the connectivity established by the Cloud Connector is that the Cloud Connector administrator should have full control over the connection to the cloud, that is, deciding if and when the Cloud Connector should be connected to the cloud, the accounts to which it should be connected, and which on-premises systems and resources should be accessible to applications of the connected subaccount.

Using the administration UI, the Cloud Connector administrator can connect and disconnect the Cloud Connector to and from the configured cloud subaccount. Once disconnected, no communication is possible, either between the cloud subaccount and the Cloud Connector, or to the internal systems. The connection state can be verified and changed by the Cloud Connector administrator on the Subaccount Dashboard tab of the UI.



> **Note**
>
> Once the Cloud Connector is freshly installed and connected to a cloud subaccount, none of the systems in the customer network are yet accessible to the applications of the related cloud subaccount. Accessible systems and resouurces must be configured explicitly in the Cloud Connector one by one, see Configure Access Control [page 1740].

A Cloud Connector instance can be connected to multiple subaccounts in the cloud. This is useful especially if you need multiple subaccounts to structure your development or to stage your cloud landscape into development, test, and production. In this case, you can use a single Cloud Connector instance for multiple subaccounts. However, we recommend that you do not use subaccounts running in productive scenarios and subaccounts used for development or test purposes within the same Cloud Connector. You can add or a delete a cloud account to or from a Cloud Connector using the *Add* and *Delete* buttons on the *Subaccount Dashboard* (see screenshot above).

Related Information

Adding and Managing Subaccounts [page 1682]

1.4.3.7 Secure the Activation of Traffic Traces

For support purposes, you can trace HTTP and RFC network traffic that passes through the Cloud Connector.

Context

Traffic data may include business-critical information or security-sensitive data, such as user names, passwords, address data, credit card numbers, and so on. Thus, by activating the corresponding trace level, a Cloud Connector administrator might see data that he or she is not meant to. To prevent this behavior, implement the four-eyes principle for your operating system as described below.

Once the four-eyes principle is applied, activating a trace level that dumps traffic data will require two separate users:

- An operating system user on the machine where the Cloud Connector is installed;
- An `Administrator` user of the Cloud Connector user interface.

By assigning these roles to two different people, you can ensure that both persons are needed to activate a traffic dump.

Four-Eyes Principle for Microsoft Windows OS

1. Create a file named **`writeHexDump`** in `<scc_install_dir>\scc_config`. The owner of this file must be a user other than the operating system user who runs the `cloud connector` process.

   > ### Note
   >
   > Usually, this file owner is the user which is specified in the *Log On* tab in the properties of the `cloud connector` service (in the Windows *Services* console). We recommend that you do not use the *Local System* user, but a dedicated OS user for the `cloud connector` service.

   - Only the file owner should have write permission for the file.
   - The OS user who runs the `cloud connector` process needs read-only permissions for this file.
   - Initially, the file should contain a line like `allowed=false`.
   - In the security properties of the file `scc_config.ini` (same directory), make sure that only the OS user who runs the `cloud connector` process has write/modify permissions for this file. The most efficient way to do this is simply by removing all other users from the list.
2. Once you've created this file, the Cloud Connector refuses any attempt to activate the *Tunnel Traffic Trace* or *ABAP Cloud SNC Traffic Trace* flag.
3. To set CPIC trace level 3 or activate any traffic trace, first the owner of **`writeHexDump`** must change the file content from `allowed=false` to `allowed=true`. Thereafter, the *Administrator* user can activate any traffic trace from the Cloud Connector administration screens.

Four-Eyes Principle for Linux OS/Mac OS X

1. Go to directory */opt/sap/scc/scc_config* and create a file with name **`writeHexDump`**. The owner of this file must be different from the *scctunnel* user (that is, the operating system user under which the Cloud Connector processes are running) and not a member of the operating system user group *sccgroup*.
   - Only the file owner should have write permission for the file.
   - The **`scctunnel`** user needs read-only permissions for this file.
   - Initially, the file should contain a line like `allowed=false`.
2. Once you've created this file, the Cloud Connector refuses any attempt to activate the *Tunnel Traffic Trace* or *ABAP Cloud SNC Traffic Trace* flag.
3. To set CPIC trace level 3 or activate any traffic trace, first the owner of the **`writeHexDump`** file mentioned above must change the file content from `allowed=false` to `allowed=true`. Then, the *Administrator* user can activate any traffic trace from the Cloud Connector administration screens.

1.4.3.8 Monitoring

Learn how to monitor the Cloud Connector from the SAP BTP cockpit and from the Cloud Connector administration UI.

Checking the Operational State

The simplest way to verify whether a Cloud Connector is running is to try to access its administration UI. If you can open the UI in a Web browser, the `cloud connector` process is running.

- On Microsoft Windows operating systems, the `cloud connector` process is registered as a Windows service, which is configured to start automatically after a new Cloud Connector installation. If the Cloud Connector server is rebooted, the `cloud connector` process should also auto-restart immediately. You can check the state with the following command:

  ```
  sc query "SAP Cloud Connector"
  ```

  The line state shows the state of the service.
- On Linux operating systems, the Cloud Connector is registered as a daemon process and restarts automatically each time the `cloud connector` process is down, for example, following a system restart. You can check the daemon state with the following command:

  ```
  service scc_daemon status
  ```

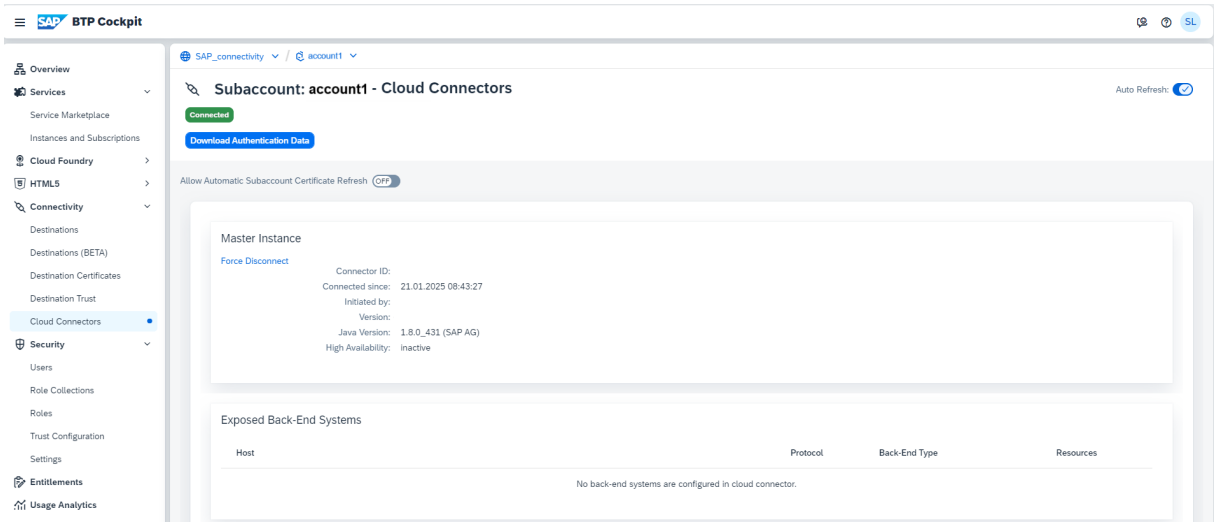
To verify if a Cloud Connector is connected to a certain cloud subaccount, log on to the Cloud Connector administration UI and go to the *Subaccount Dashboard*, where the connection state of the connected subaccounts is visible, as described in section Connect and Disconnect a Cloud Subaccount [page 2003].

Monitoring from the Cockpit

The cockpit includes a *Connectivity* section, where users can check the status of the Cloud Connector(s) attached in the current subaccount, if any, as well as information about the Cloud Connector ID, version, used Java runtime, high availability setup (master and shadow instance), and so on (choose *Connectivity* > *Cloud Connectors*).

Access to this view is granted to:

- Neo environment: Users with a role containing the permission `readSCCTunnels`, for example, the predefined role `Cloud Connector Admin`.
- Multi-Cloud environment: Users with a role containing the permission `readSCCTunnels`, for example, the predefined role `Cloud Connector Administrator`.



Monitoring from the Cloud Connector Administration UI

The Cloud Connector offers various views for monitoring its activities and state.

You can check the overall state of the Cloud Connector through its Hardware Metrics [page 2007]. Cross-subaccount performance data is provided by the Performance Monitor [page 2037], whereas subaccount, whereas subaccount-specific performance and usage data is available via Subaccount-Specific Monitoring [page 2009].

To provide external monitoring tools, you can use the Monitoring APIs [page 2016].

Related Information

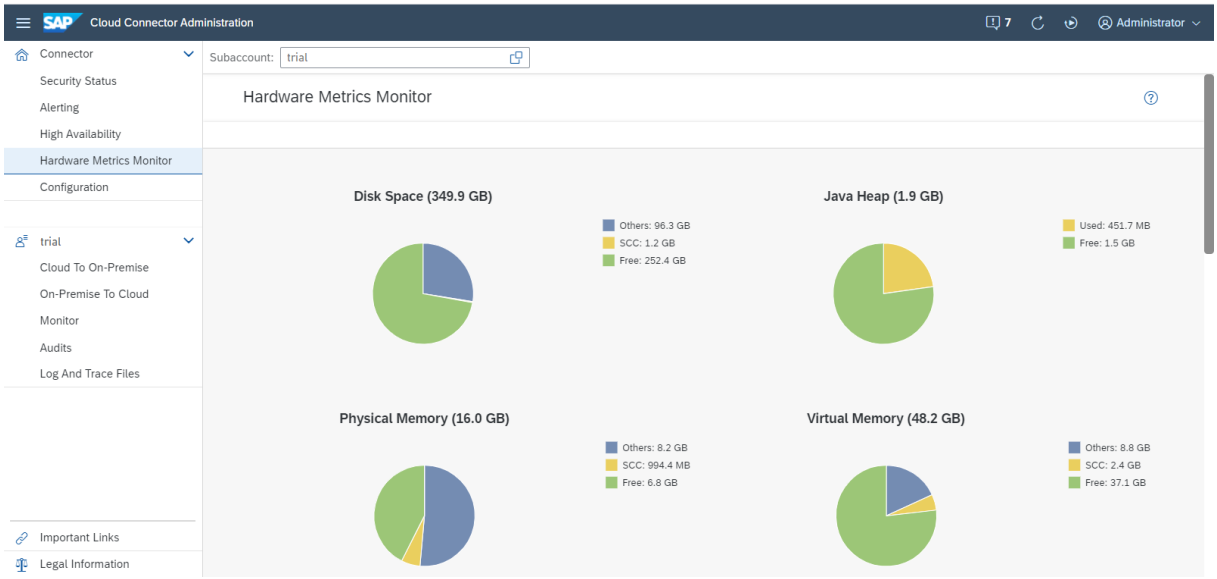
Configure Solution Management Integration [page 1959]
High Availability Setup [page 1995]

1.4.3.8.1 Hardware Metrics

Check the current state of critical system resources in the Cloud Connector.

You can check the current state of critical system resources (disc space, Java heap, physical memory, virtual memory) using pie charts.

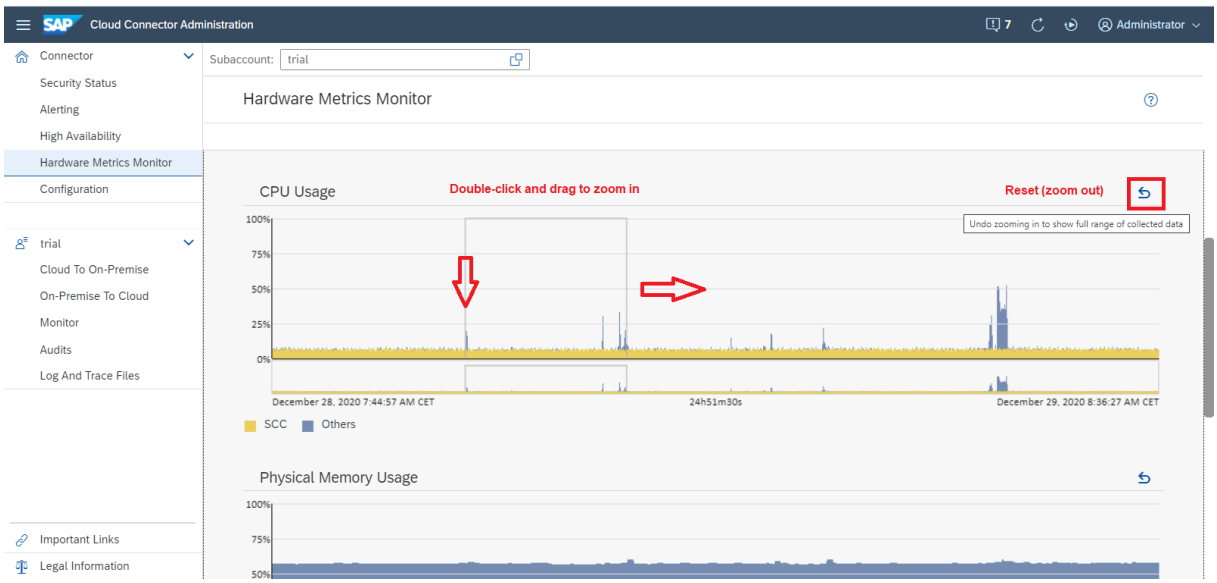
To access the monitor, choose *Hardware Metrics Monitor* from the main menu.



In addition, the history of CPU and memory usage (physical memory, Java heap) is shown in history graphs below the pie charts (recorded in intervals of 15 seconds), as well as the history of disk usage, recorded in intervals of 60 seconds. History data is retained for at most 24 hours.

You can view the usage data for a selected time period in each history graph:

- Double-click inside the main graph area to set the start (or end) point, and drag to the left or to the right to zoom in.
  - The entire timeline is always visible in the smaller bottom area right below the main graph.
  - A frame in the bottom area shows the position of the selected section in the overall timeline.
- Choose *Undo zooming in...* to reset the main graph area to the full range of available data.



> ⓘ Note
>
> Zooming, dragging, and undoing zooming is synchronized across all history graphs. All graphs always show the situation during the same time period.

History data is written to a file to avoid data loss when stopping the Cloud Connector. Upon restart, the history data is read from file. Downtime is represented by a gray rectangle indicating that no data is available during the respective period of time.

1.4.3.8.2 Subaccount-Specific Monitoring

Use different monitoring views in the Cloud Connector administration UI to check subaccount-specific activites and data.

The Cloud Connector provides various views for monitoring the activities associated with an account, such as HTTP requests and RFC calls (from cloud applications to backends as per access control settings) or data statistics for service channels. Choose one of the sub-menus *Monitor (Cloud to On-Premises)* or *Monitor (On-Premises to Cloud)*.

> ⚠ Caution
>
> The collected monitoring data is not part of the Cloud Connector's backup. After restoring a Cloud Connector instance, all monitoring collections are empty.

Monitoring (Cloud to On-Premises) [page 2009]

Monitoring (On-Premises to Cloud) [page 2015]

1.4.3.8.2.1 Monitoring (Cloud to On-Premises)

Monitor cloud to on-premises connections in the Cloud Connector.

Content

Performance Overview [page 2010]

Most Recent Requests [page 2011]

Resource Filter Settings [page 2012]

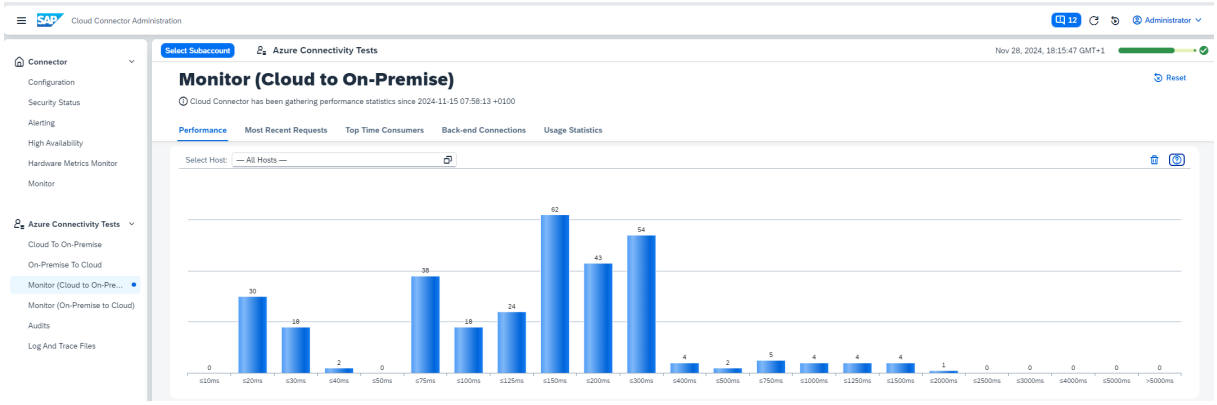
Top Time Consumers [page 2013]

Usage Statistics [page 2013]

Backend Connections [page 2014]

Performance Overview

All requests that travel through the Cloud Connector to a backend system, as specified through access control, take a certain amount of time. You can check the duration of requests in a bar chart.



The bar chart either shows the duration statistics for all virtual hosts or for a selected virtual host.

The requests are not shown individually, but are assigned to buckets, each of which represents a time range.

For example, the first bucket contains all requests that took 10ms or less, the second one the requests that took longer than 10ms, but not longer than 20ms. The last bucket contains all requests that took longer than 5000ms.

In case of latency gaps, you may try to adjust the influencing parameters: number of connections, tunnel worker threads, and protocol processor worker threads. For more information, see Configuration Setup [page 1654].

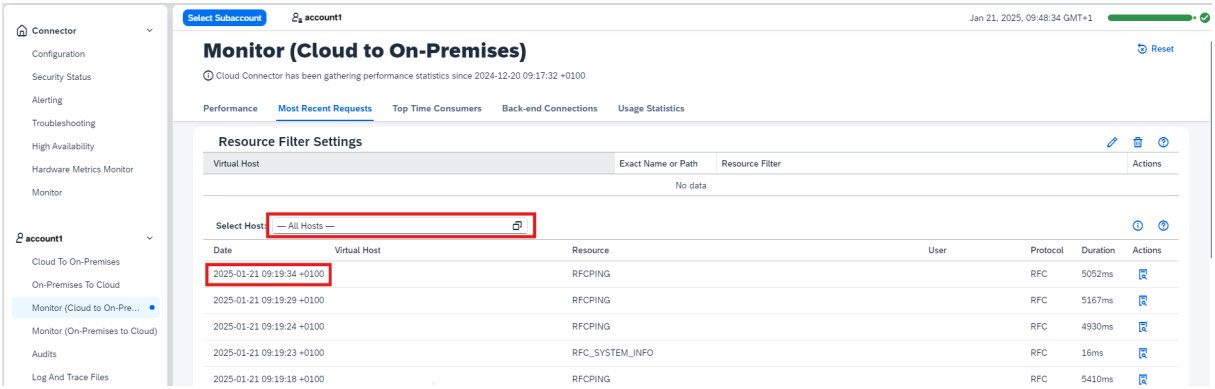
The collection of duration statistics starts as soon as the Cloud Connector is operational. You can delete all of these statistical records by selecting the button *Delete All*. After that, the collection of duration statistics starts over.

> **Note**
>
> *Delete All* deletes not only the list of most recent requests, but it also clears the top time consumers.

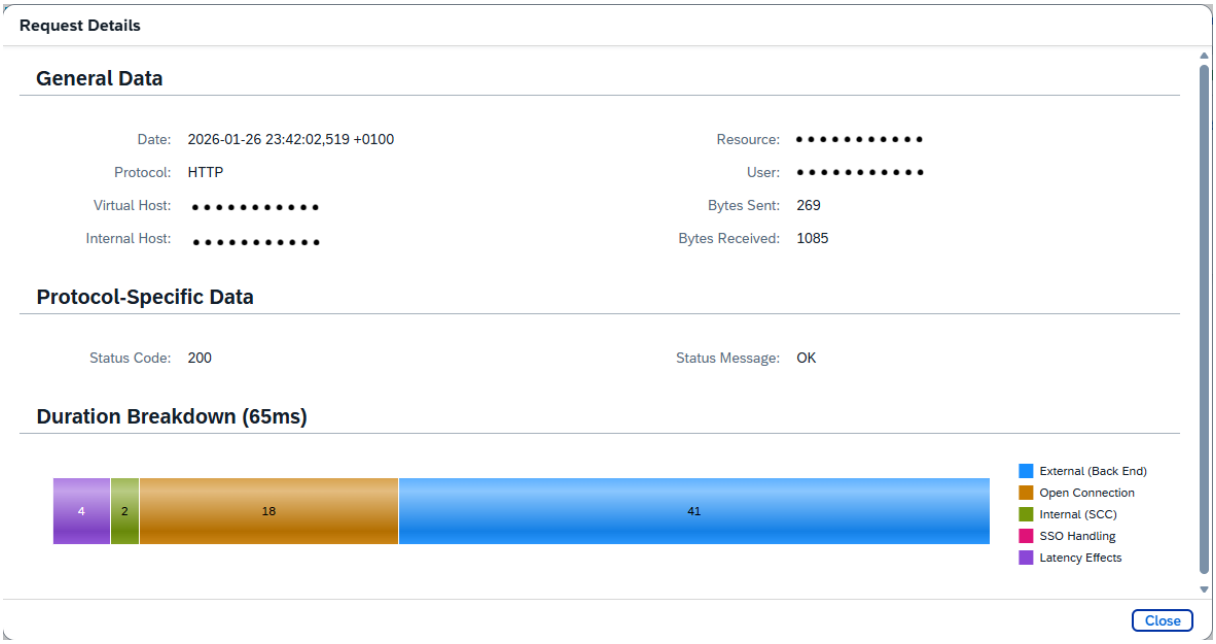
Back to Content [page 2009]

Most Recent Requests

This option shows the most recent requests:



The number of requests that are shown is limited to 50. You can either view all requests or only the ones destined for a certain virtual host, which you can select.You can select a row to see more detail.



For the **HTTP** protocol specifically, an additional section called *Protocol-Specific Data* is available as of Cloud Connector version 2.19. This section displays extended request information, such as *Status Code* and *Status Message* (200 and OK in the case shown above).

A horizontal stacked bar chart breaks down the duration of the request into several parts: *External (Back End)*, *Open Connection*, *Internal (SCC)*, *SSO Handling*, and *Latency Effects* (between SAP BTP and Cloud Connector). The numbers shown on the chart sections are milliseconds.

> ### Note
>
> Sections with a duration of less than 1ms are not included.

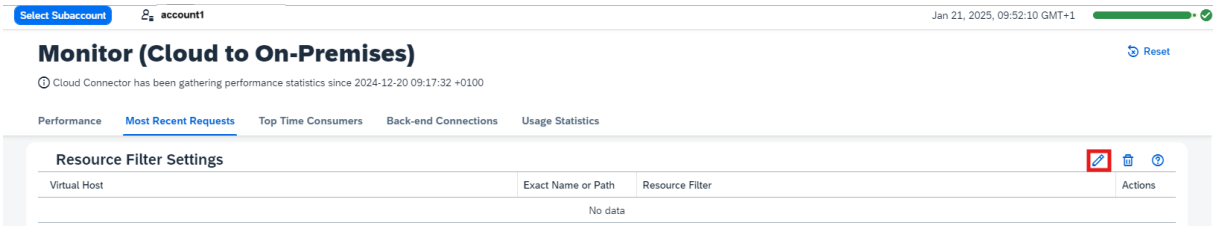
In the example, the selected request took 65ms, to which the Cloud Connector contributed 2ms. Opening the connection took 18ms. Processing at the back-end side consumed 41ms. Latency effects accounted for the remaining 4ms, while there was no SSO handling necessary and hence it took no time at all.

> ### Note
>
> The term "request" only refers to RFC and HTTP requests. TCP or LDAP traffic does not contribute to *Most Recent Requests* or *Top Time Consumers*.

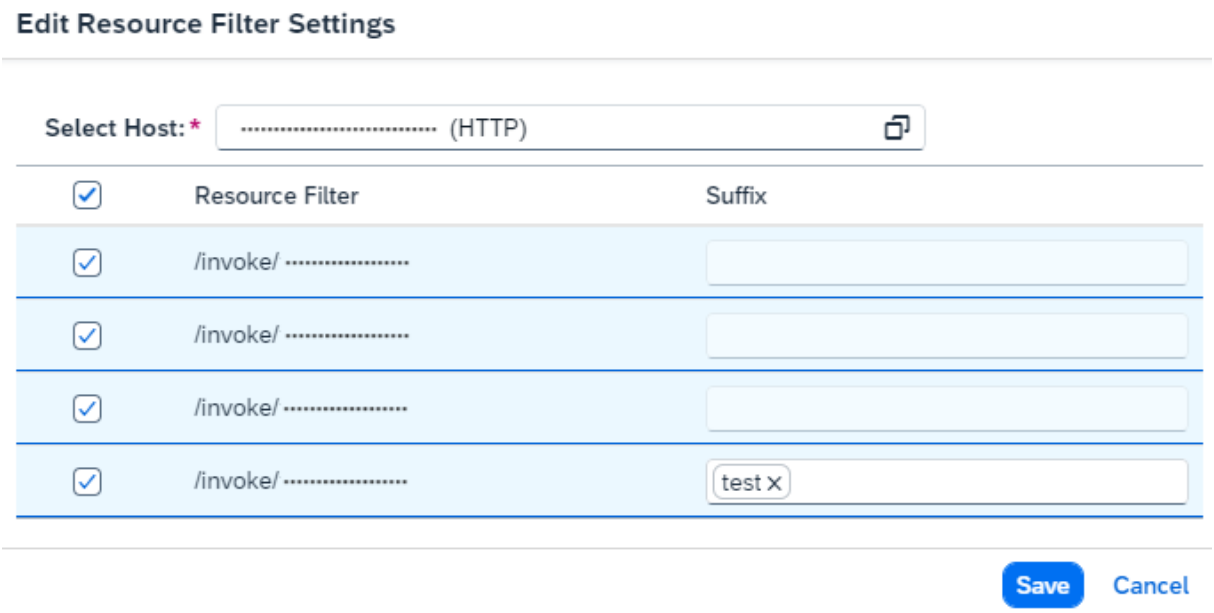
Back to Content [page 2009]

Resource Filter Settings

To further restrict the selection of the listed 50 most recent requests, you can edit the resource filter settings for each virtual host:



In the *Edit* dialog, select the virtual host for which you want to specify the resource filter and choose one or more of the listed accessible resources. This list includes all resources that have been exposed during access control configuration (see also: Configure Access Control [page 1740]). If the access policy for an accessible resource is set to `Path and all sub-paths`, you can further narrow the selection by adding one or more sub-paths to the resource as a suffix .



Each selected resource/sub-path is listed separately in the resource filter list.

> **Note**
>
> If you specify sub-paths for a resource, the request URL must match exactly one of these entries to be recorded. Without specified sub-paths (and the value `Path and all sub-paths` set for a resource), all sub-paths of a specified resource are recorded.

Back to Content [page 2009]

Top Time Consumers

This option is similar to *Most Recent Requests*; however, requests are not shown in order of appearance, but rather sorted by their duration (in descending order). Furthermore, you can delete top time consumers, which has no effect on most recent requests or the performance overview.

Back to Content [page 2009]

Usage Statistics

To view the statistical data regarding the traffic handled by each **virtual host**, you can select a virtual host from the table. The detail view shows the traffic handled by *each resource*, as well as a *24 hour overview* of

the throughput as a bar chart that aggregates the throughput (bytes received and bytes sent by a virtual host, respectively) on an hourly basis.

> ⓘ Note
>
> For communication via HTTP and RFC, also the number of calls or requests is recorded in the same way.

The data that is collected includes the number of bytes received *from* cloud applications and the number of bytes sent back *to* cloud applications. The time of the most recent access is shown for the virtual hosts, and in the detail view also for the resources. If no access has taken place yet, the most recent access is shown as `n.a.` (not available). Similarly, the number of bytes received and sent of a virtual host is the sum of bytes received and sent of its resources.

The tables listing usage statistics of virtual hosts and their resources let you delete unused virtual hosts or unused resources. Use action *Delete* to delete such a virtual host or resource.

> ⚠ Caution
>
> In Cloud Connector versions **before 2.14**, usage statistics are collected during runtime only and are not stored when stopping the Cloud Connector. That is, these statistics are lost when the Cloud Connector is stopped or restarted. Use care when taking the decision to delete a resource or virtual host based on its usage statistics.
>
> **As of version 2.14**, usage statistics are periodically stored to disk. The collected statistics are still available after a restart of the Cloud Connector.
>
> Actively deleted statistics are gone in either case.
>
> A backup of the Cloud Connector settings does not include collected usage data. After restoring a Cloud Connector instance, all monitoring collections are empty.

Using the *Reset* button, you can clean up all collected data.

> ⚠ Caution
>
> Statistical data will be deleted permanently.

For both virtual hosts and resources, you can use a classic *Filter* button to reduce the virtual hosts or resources to those that have never been used (since the Cloud Connector started). For the virtual hosts, a second filter type is available that selects only those virtual hosts that have been used, but *include resources never used*. This feature facilitates locating obsolete resources of otherwise active virtual hosts.

Back to Content [page 2009]

Backend Connections

This option shows a tabular overview of all active and idle connections, aggregated for each virtual host. By selecting a row (each of which represents a virtual host) you can view the details of all active connections

as well as a graphical summary of all idle connections. The graphical summary is an accumulative view of connections based on the time the connections have been idle.

The maximum idle time appears on the rightmost side of the horizontal axis. For any point `t` on that axis (representing a time value ranging between 0ms and the maximal idle time), the ordinate is the number of connections that have been idle for no longer than `t`. You can click inside the graph area to view the respective abscissa `t` and ordinate.

Back to Content [page 2009]

1.4.3.8.2.2 Monitoring (On-Premises to Cloud)

Monitor on-premises to cloud connections in the Cloud Connector.

Connections

This section shows a tabular overview of all currently opened logical connections, aggregated for each local port. You can identify if and how many connections are currently opened through this specific service channel.

Usage Statistics

Statistical data regarding the traffic handled by each port is shown in tabular form. From the table, you can select a service channel. The respective detail views show a 24 hour overview as a bar chart that aggregates the throughput (bytes received and bytes sent via a port, respectively) on an hourly basis.

The collected data comprises the number of bytes received from on-premises applications and the number of bytes sent to cloud applications.

The table listing usage statistics of ports lets you delete unused service channels. Use action *Delete* to delete a service channel.

> ⚠ Caution
>
> Usage statistics are collected at runtime only. They are not stored when stopping the Cloud Connector. These statistics are lost when the Cloud Connector is stopped or restarted. Be mindful of that fact, and use caution when taking the decision to delete a service channel based on its usage statistics.

Using the *Reset* button you can clean up all collected data. This might be helpful when monitoring a specific use case.

The table of service channels provides a filter button to reduce the service channels to those that have never been used (since the Cloud Connector started).

1.4.3.8.3 Monitoring APIs

Use the Cloud Connector monitoring APIs to include monitoring information in your own monitoring tool.

Context

You might want to integrate some monitoring information in the monitoring tool you use.

For this purpose, the Cloud Connector includes a collection of APIs that allow you to read various types of monitoring data.

> ⓘ Note
>
> This API set is designed particularly for monitoring the Cloud Connector via the SAP Solution Manager, see Configure Solution Management Integration [page 1959].

Before you start using these APIs, please also read the general introduction to REST APIs [page 2084] provided by Cloud Connector.

Prerequisites

You must use *Basic Authentication* or *form field* authentication to read the monitoring data via API. If certificate-based authentication has been configured for the Cloud Connector, a client certificate must be used instead.

Users must be assigned to the roles `sccmonitoring` or `sccadmin`.

> ⓘ Note
>
> The Health Check API does not require a specified user. Separate users are available through LDAP only.

Available APIs

The following APIs are currently available.

- Health Check [page 2017] (available as of version 2.6.0)
- Subaccount Data [page 2017] (as of 2.10.0)
- Open Connections to On-Premises Backend Systems [page 2018] (as of 2.10.0)
- Open Connections to Cloud Services [page 2020] (as of 2.15.0)

- Performance Data [page 2021] (as of 2.10.0)
- Top Time Consumers [page 2023] (as of 2.11.0)
- Memory Status [page 2025] (as of 2.13.0)
- Disk Status [page 2027] (as of 2.17.0)
- CPU Status [page 2027] (as of 2.17.0)
- Certificate Status [page 2028] (as of 2.13.0)
- Certificate Selection List [page 2030] (as of 2.13.0, extended in 2.17.0)
- Usage Statistics [page 2031] (as of 2.13.0)
- Master Role Check [page 2035] (as of 2.15.0, extended in 2.18.1)
- Alert Information [page 2036] (as of 2.19.0)

Health Check (available as of version 2.6.0)

Using the health check API, it is possible to recognize that the Cloud Connector is up and running. The purpose of this health check is only to verify that the Cloud Connector is not down. It does not check any internal state or tunnel connection states. Thus, it is a quick check that you can execute frequently:

URI	`/exposed?action=ping`
Method	`GET`
Request	
Response	
Errors	
Roles	All roles are accepted

Back to Available APIs [page 2016]

Back to Context [page 2016]

List of Subaccounts (available as of version 2.10.0)

> **Note**
>
> This API is relevant for the master instance only.

Using this API, you can read the list of all subaccounts connected to the Cloud Connector and view detail information for each subaccount:

URI	`/api/monitoring/subaccounts`
Method	GET
Request	

Response	`{subaccounts,version}`
Errors	
Roles	Administrator, Monitoring

Response Properties:

- `subaccounts`: array of subaccounts for which data is provided
  - `regionHost`: host of the region, in which the subaccount is residing
  - `subaccount`: name of subaccount
  - `displayName`: display name of the subaccount
  - `description`: description for the subaccount
  - `locationID`: identifying the location of this Cloud Connector for a specific subaccount
  - `tunnel`: array of connection tunnels used by the subaccount
    - `state`: *Connected*, *ConnectFailure*, or *Disconnected*
    - `connectedSinceTimeStamp`: connection start time as UTC timestamp
    - `connections`: number of subaccount connections
    - `applicationConnections`: array of connections to application instances
    - `serviceChannels`: type and state of the service channels used (types: HANA database, Virtual Machine or RFC)
    - `subaccountCertificate`: information on the subaccount certificate such as validity period, issuer and subject DN
- `version`: API version.

Example:

```
curl -k -H 'Accept:application/json' -u <user>:<password> -X GET https://
<scchost>:<sccport>/api/monitoring/subaccounts
```

Back to Available APIs [page 2016]

Back to Context [page 2016]

List of Open Connections to On-Premises Backend Systems (available as of version 2.10.0)

> ⓘ Note
>
> This API is relevant for the master instance only.

The list of connections lets you view all backend systems connected to the Cloud Connector and get detail information for each connection:

URI	`/api/monitoring/connections/backends`

Method	GET
Request	
Response	`{subaccounts, version}`
Errors	
Roles	Administrator, Monitoring

Response Properties:

- `subaccounts`: array of subaccounts for which data is provided
  - `regionHost`: host of the region, in which the subaccount is residing
  - `subaccount`: name of subaccount
  - `locationID`: identifying the location of this Cloud Connector for a specific subaccount
  - `backendConnections`: array of connections to a specified backend system
    - `virtualBackend`: virtual (external) backend URL
    - `internalBackend`: internal backend URL
    - `protocol`: type of protocol (RFC, HTTP, and so on)
    - `idle`: number of idle connections
    - `active`: number of active connections
- `version`: API version.

{·} Sample Code

```
curl -k -H 'Accept:application/json' -u <user>:<password> -X GET https://
<scchost>:<sccport>/api/monitoring/connections/backends
```

Example:

```
{
  "subaccounts": [
    {
      "backendConnections": [],
      "regionHost": "hana.ondemand.com",
      "subaccount": "a117",
      "locationID": ""
    },
    {
      "backendConnections": [
        {
          "virtualBackend": "abapserver.hana.cloud:",
          "internalBackend": "sap.corp:",
          "protocol": "RFC",
          "idle": 1,
          "active": 0
        }
      ],
      "regionHost": "hana.ondemand.com",
      "subaccount": "dev",
      "locationID": ""
    },
    {
      "backendConnections": [],
      "regionHost": "hanatrial.ondemand.com",
      "subaccount": "trial",
      "locationID": ""
    }
  ],
  "version": 1
}
```


Back to Available APIs [page 2016]

Back to Context [page 2016]

List of Open Connections to Cloud Services (available as of version 2.15.0)

> **Note**
>
> This API is relevant for the master instance only.

The list of connections opened for service channels:

URI	`/api/monitoring/connections/serviceChannels`
Method	GET

Request	
Response	`{subaccounts, version}`
Errors	
Roles	Administrator, Monitoring

Response Properties:

- `subaccounts`: array of subaccounts for which data is provided
  - `regionHost`: host of the region, in which the subaccount is residing
  - `subaccount`: name of subaccount
  - `locationID`: identifying the location of this Cloud Connector for a specific subaccount
  - `serviceChannelConnections`: array of connections opened by the specified service channel
    - `port`: port of the service channel
    - `typeDesc`:
      - `typeKey`: key name of the service channel, for example, *ABAPCloud*
      - `typeName`: human readable service channel type
    - `connections`: number of connections
- `version`: API version.

{·} Sample Code

```
curl -k -H 'Accept:application/json' -u <user>:<password> -X GET https://
<scchost>:<sccport>/api/monitoring/connections/serviceChannels
```

Back to Available APIs [page 2016]

Back to Context [page 2016]

Performance Monitor Data (available as of version 2.10.0)

ⓘ Note

This API is relevant for the master instance only.

Using this API, you can read the data provided by the Cloud Connector performance monitor:

URI	`/api/monitoring/performance/backends`
Method	GET
Request	

Response	`{subaccounts, version}`
Errors	
Roles	Administrator, Monitoring

Response Properties:

- `subaccounts`: array of subaccounts for which data is provided
  - `regionHost`: host of the region, in which the subaccount is residing
  - `subaccount`: name of subaccount
  - `locationID`: identifying the location of this Cloud Connector for a specific subaccount
  - `backendPerformance` given as array of:
    - `virtualHost`: host name of the backend system
    - `virtualPort`: port of the backend system
    - `protocol`: type of protocol (RFC, HTTP etc.)
    - `buckets`: array of performance data related to backend system
      - `numberOfCalls`: number of calls performed between Cloud Connector and backend system
      - `minimumCallDurationsMs`: minimum duration of the executed calls in miliseconds
  - `sinceTime`: start of performance measurement
- `version`: API version.

{·} Sample Code

```
curl -k -H 'Accept:application/json' -u <user>:<password> -X GET https://
<scchost>:<sccport>/api/monitoring/performance/backends
```

Example:

```
{
  "subaccounts": [
    {
      "backendPerformance": [],
      "sinceTime": "2017-05-11T15:48:42.084 +0200",
      "regionHost": "hana.ondemand.com",
      "subaccount": "a117",
      "locationID": ""
    },
    {
      "backendPerformance": [
        {
          "virtualHost": "abapserver.hana.cloud",
          "virtualPort": "2",
          "protocol": "RFC",
          "buckets": [
            {
              "numberOfCalls": 0,
              "minimumCallDurationMs": 0
            },
            {
              "numberOfCalls": 1,
              "minimumCallDurationMs": 10
            },
            {
              "numberOfCalls": 0,
              "minimumCallDurationMs": 20
            },
            {
              "numberOfCalls": 0,
              "minimumCallDurationMs": 30
            },
```


Back to Available APIs [page 2016]

Back to Context [page 2016]

Top Time Consumers (available as of version 2.11.0)

> **Note**
>
> This API is relevant for the master instance only.

Using this API, you can read the data of top time consumers provided by the Cloud Connector performance monitor:

URI	`/api/monitoring/performance/topTimeConsumers`
Method	GET
Request	
Response	`{subaccounts, version}`
Errors	
Roles	Administrator, Monitoring

Response Properties:

- `subaccounts`: array of subaccounts for which data is provided
  - `regionHost`: host of the region, in which the subaccount is residing
  - `subaccount`: name of subaccount
  - `locationID`: identifying the location of this Cloud Connector for a specific subaccount
  - `requests`: given as array of:
    - `protocol`: type of protocol (RFC, HTTP, and so on)
    - `virtualBackend`: virtual (external) backend URL
    - `internalBackend`: internal backend URL
    - `resource`: name of the request resource
    - `sentBytes`: number of sent bytes
    - `receivedBytes`: number of received bytes
    - `user`: name of the request user
    - `totalTime`: total request time in milliseconds
    - `externalTime`: in milliseconds
    - `genSsoTime`: in milliseconds
    - `openRemoteTime`: in milliseconds
    - `validationSsoTime`: time for SSO validation in milliseconds
    - `latencyTime`: latency in milliseconds
  - `sinceTime`: start of performance measurement
- `version`: API version.

Sample Code

```
curl -k -H 'Accept:application/json' -u <user>:<password> -X GET https://
<scchost>:<sccport>/api/monitoring/performance/topTimeConsumers
```

Example:

```
{
  "subaccounts": [
    {
      "sinceTime": "2017-06-26T16:57:21.615 +0200",
      "requests": [
        {
          "startTime": "2017-06-26T16:57:25.426 +0200",
          "id": 1639964397,
          "protocol": "RFC",
          "virtualBackend": "abapserver.hana.cloud:sapgw42",
          "internalBackend": "ldciv9u:sapgw51",
          "resource": "RFCPING",
          "sentBytes": 307,
          "receivedBytes": 441,
          "user": "JCOUSER",
          "totalTime": 21,
          "externalTime": 6,
          "genSsoTime": 0,
          "openRemoteTime": 5,
          "validateSsoTime": 0,
          "latencyTime": 6
        },
        { … } // 15 items
      ],
      "regionHost": "int.sap.hana.ondemand.com",
      "subaccount": "d039407sapdev",
      "locationID": "location"
    }
  ],
  "version": 1
}
```


Back to Available APIs [page 2016]

Back to Context [page 2016]

Memory Status (available as of version 2.13.0)

> **Note**
>
> This API is relevant for the master instance only.

This API provides a snapshot of the current memory status of the machine where the Cloud Connector is running:

URI	`/api/monitoring/memory`
Method	GET
Request	

Response	`{physicalKB, virtualKB, cloudConnectorHeapKB}`
Errors	
Roles	Administrator, Monitoring

Response Properties:

- `physicalKB`: usage of the physical memory, split into four categories (all sizes in KB):
  - `total`: total size of the physical memory
  - `CloudConnector`: size of the physical memory used by the Cloud Connector
  - `others`: size of the physical memory used by all other processes
  - `free`: size of the free physical memory
- `virtualKB` : usage of the virtual memory, split into four categories (all sizes in KB)
  - `total`: total size of the virtual memory
  - `CloudConnector`: size of the virtual memory used by the Cloud Connector
  - `others`: size of the virtual memory used by all other processes
  - `free`: size of the free virtual memory
- `cloudConnectorHeapKB` : usage of the Java heap, split into three categories (all sizes in KB):
  - `total`: total size of the Java heap
  - `used`: size of the Java heap used by the Cloud Connector
  - `free`: size of the free Java heap

Sample Code

```
curl -k -H 'Accept:application/json' -u <user>:<password> -X GET https://
<scchost>:<sccport>/api/monitoring/memory
```

Example:

```
{
  "physicalKB": {
    "total": 33380516,
    "CloudConnector": 633932,
    "others": 10960464,
    "free": 21786120
  },
  "virtualKB": {
    "total": 35477668,
    "CloudConnector": 1267504
    "others": 14174208,
    "free": 20035956
  },
  "cloudConnectorHeapKB": {
    "total": 983040,
    "used": 235457,
    "free": 747583
  }
}
```


Back to Available APIs [page 2016]

Back to Context [page 2016]

Disk Status (available as of version 2.17.0)

URI	`/api/monitoring/disk`
Method	GET
Request	
Response	`{total, CloudConnector, others, free}`
Errors	
Roles	Administrator, Monitoring

Response Properties:

- `total`: total size of the disk (in MB)
- `CloudConnector`: disk usage of the Cloud Connector (in MB)
- `others`: any other disk usage (in MB)
- `free`: free disk space (in MB)

Back to Available APIs [page 2016]

Back to Context [page 2016]

CPU Status (available as of version 2.17.0)

URI	`/api/monitoring/cpu`
Method	GET
Request	
Response	`{total, CloudConnector}`
Errors	
Roles	Administrator, Monitoring

Response Properties:

- `total`: total CPU usage (as percentage %)
- `CloudConnector`: CPU usage of the Cloud Connector (as percentage %)

Back to Available APIs [page 2016]

Back to Context [page 2016]

Certificate Status (available as of version 2.13.0)

> **Note**
>
> This API is relevant for the master instance only.

Using this API, you can get an overview of the certificates currently employed by the Cloud Connector:

URI	`/api/monitoring/certificates`
Method	GET
Request	
Response	`{expired, expiring, ok}`
Errors	
Roles	Administrator, Monitoring

Response Properties:

- `expired`: list of all expired certificates
- `expiring`: list of all certificates that will expire in less than N days, where N is the number of days specified in the alerting setup regarding certificates that are close to their expiration date
- `ok`: list of all certificates that continue to be valid for N days or more, where N is the number of days specified in the alerting setup regarding certificates that are close to their expiration date

A certificate in any of those lists is represented by a JSON object with the following properties:

- `type`: type of the certificate which can be one of the following strings:
  - `UI` (for the UI certificate)
  - `System` (for the system certificate)
  - `CA` (for the certificate used in connection with Principal Propagation/Certification Authority)
  - `subaccount` (for subaccount certificates)
- `validTo`: end date of the respective certificate's validty (as a long integer, that is, a UTC timestamp)
- `subjectDN`: subject DN of the respective certificate (included only for non-subaccount certificates)
- `issuerDN`: issuer DN of the respective certificate (included only for non-subaccount certificates)
- `serialNumber`: serial number of the respective certificate as hex-encoded string (included only for non-subaccount certificates)
- `subaccountName`: name of the subaccount (only for subaccount certificates)
- `subaccountRegion`: region or landscape host of the the subaccount (only for subaccount certificates)

Sample Code

```
curl -k -H 'Accept:application/json' -u <user>:<password> -X GET https://
<scchost>:<sccport>/api/monitoring/certificates
```

Example:

```
{
    "expired": [
        {
            "type": "CA",
            "subjectDN": "CN=Hugo, OU=CSI, O=SAP Trust Community, C=DE",
            "issuerDN": "CN=Hugo, OU=CSI, O=SAP Trust Community, C=DE",
            "serialNumber": "b8:24:67:d3",
            "validTo": 1712760831000
        }
    ],
    "expiring": [],
    "ok": [
        {
            "type": "UI",
            "subjectDN": "CN=cloudconnector.sap.org, OU=Connectivity, O=SAP, C=DE",
            "issuerDN": "CN=SAP Internal CA, O=SAP, L=Walldorf, C=DE",
            "serialNumber": "55:ad:40:13:de:84:f5:4f:18:da:a0:ef:72:88:88:14",
            "validTo": 1718277891000
        },
        {
            "type": "System",
            "subjectDN": "CN=FCB, OU=SAP Web AS, O=SAP-AG, C=DE",
            "issuerDN": "CN=CloudConnector, OU=Connectivity, O=SAP, C=DE",
            "serialNumber": "56:12:7b:5c",
            "validTo": 1759671294000
        },
        {
            "type": "subaccount",
            "subaccountName": "1fa142c4-c16b-423d-abdd-4c3d93ffdf01",
            "subaccountRegion": "cf.eu10.hana.ondemand.com",
            "validTo": 1743601324000
        }
    ]
}
```


Back to Available APIs [page 2016]

Back to Context [page 2016]

Certificate Selection List (available as of version 2.13.0, extended in 2.17.0)

> **Note**
>
> This API is relevant for the master instance only.

Using this API, you can obtain an overview of the certificates currently employed by the Cloud Connector:

URI	`/api/monitoring/certificates/{selection}`
Method	GET
Request	

Response	`[Array of certificates]`
Errors	
Roles	Administrator, Monitoring

Request:

- `selection` parameter
  - `expired`: an array holding the list of all expired certificates
  - `expiring`: an array holding the list of all certificates that will expire in less than N days, where N is the number of days specified in the alerting setup regarding certificates that are close to their expiration date
  - `ok`: an array holding the list of all certificates that continue to be valid for N days or more, where N is the number of days specified in the alerting setup regarding certificates that are close to their expiration date

Response Properties:

- `type`: type of the certificate which can be one of the following strings:
  - `UI` (for the UI certificate)
  - `System` (for the system certificate)
  - `CA` (for the certificate used in connection with Principal Propagation/Certification Authority)
  - `subaccount` (for subaccount certificates)
- `validTo`: end date of the respective certificate's validty (as a long integer, that is, a UTC timestamp)
- `subjectDN`: subject DN of the respective certificate (included only for non-subaccount certificates)
- `issuerDN`: issuer DN of the respective certificate (included only for non-subaccount certificates)
- `serialNumber`: serial number of the respective certificate as hex encoded string (included only for non-subaccount certificates)
- `subaccountName`: name of the subaccount (only for subaccount certificates)
- `subaccountRegion`: region or landscape host of the the subaccount (only for subaccount certificates)

Sample Code

```
curl -k -H 'Accept:application/json' -u <user>:<password> -X GET https://
<scchost>:<sccport>/api/monitoring/certificates/expired
```

Back to Available APIs [page 2016]

Back to Context [page 2016]

Usage Statistics (available as of version 2.13.0)

> ### Note
>
> This API is relevant for the master instance only.

This API provides usage statistics regarding the systems and resources available in the Cloud Connector:

URI	`/api/monitoring/usage`
Method	GET
Request	
Response	`{subaccounts, version}`
Errors	
Roles	Administrator, Monitoring

Response Properties:

- `subaccounts`: array of subaccounts for which data is provided
  - `regionHost`: host of the region, in which the subaccount is residing
  - `subaccount`: name of subaccount
  - `locationID`: identifying the location of this Cloud Connector for a specific subaccount
  - `usageStatistics`: backend usage statistics, given as an array of
    - `virtualHost`: host name of the backend system
    - `virtualPort`: port of the backend system
    - `protocol`: type of protocol (RFC, HTTP etc.)
    - `bytesReceived`: total number of bytes that were received through a call or request
    - `bytesSent`: total number of bytes sent back as a response
    - `calls`: total number of calls or requests
    - `mostRecentAccess`: time of the most recent access (that is, call or request) given as a UTC timestamp;

      > ### Note
      >
      > This property is only available if there has been at least one call or request.

    - `resources`: usage statistics per resource, given as an array (i.e. the distribution of bytes received, sent, as well as number of calls/requests, across the resources of the respective virtual host)
      - `resourceName`: name of the resource (that is, a URL path or the name of a remote function)
      - `enabled`: Boolean flag that indicates whether the resource is currently active (true) or suspended (false)
      - `bytesReceived`: total number of bytes that were received through a call or request and were handled by this resource
      - `bytesSent`: total number of bytes sent back as a response in the context of this resource

- `calls`: total number of calls or requests handled by this resource
- `mostRecentAccess`: time of the most recent access (that is, call or request) given as a UTC timestamp;

  > **Note**
  >
  > This property is only available if at least one call or request was handled by this resource.

- `serviceChannelUsageStatistics`: service channels usage statistics, given as an array of
  - `port`: port of the service channel
  - `typeDesc`:
    - `typeKey`: key name of the service channel, for example, *ABAPCloud*
    - `typeName`: human readable service channel type
  - `bytesReceived`: total number of bytes received through the service channel
  - `bytesSent`: total number of bytes sent back through the service channel
- `sinceTime`: start of performance measurement

- `version`: API version.

> **Sample Code**
>
> ```
> curl -k -H 'Accept:application/json' -u <user>:<password> -X GET https://
> <scchost>:<sccport>/api/monitoring/usage
> ```

Example:

```
{
    "subaccounts": [
        {
            "subaccountName": "lannister",
            "subaccountRegion": "westeros.com",
            "sinceTime": 1596810677689,
            "usageStatistics": [
                {
                    "virtualHost": "iron.islands",
                    "virtualPort": "1234",
                    "protocol": "RFC",
                    "bytesReceived": 0,
                    "bytesSent": 0,
                    "calls": 0,
                    "resources": [
                        {
                            "resourceName": "TEST_FCT_1",
                            "enabled": true,
                            "bytesReceived": 0,
                            "bytesSent": 0,
                            "calls": 0
                        },
                        {
                            "resourceName": "TEST_FCT_2",
                            "enabled": true,
                            "bytesReceived": 0,
                            "bytesSent": 0,
                            "calls": 0
                        }
                    ]
                },
                {
                    "virtualHost": "the.north",
                    "virtualPort": "8080",
                    "protocol": "HTTP",
                    "bytesReceived": 1482,
                    "bytesSent": 705,
                    "calls": 3,
                    "mostRecentAccess": 1596810970078,
                    "resources": [
                        {
                            "resourceName": "/invoke/giants",
                            "enabled": true,
                            "bytesReceived": 0,
                            "bytesSent": 0,
                            "calls": 0
                        },
                        {
                            "resourceName": "/invoke/whiteWalkers",
                            "enabled": true,
                            "bytesReceived": 0,
                            "bytesSent": 0,
                            "calls": 0
                        },
                        {
                            "resourceName": "/invoke/nightWatch",
                            "enabled": true,
                            "bytesReceived": 1482,
                            "bytesSent": 705,
                            "calls": 3,
                            "mostRecentAccess": 1596810970078
                        }
                    ]
                }
            ]
        }
```

Back to Available APIs [page 2016]

Back to Context [page 2016]

Master Role Check (available as of version 2.15.0, extended in 2.18.1)

With the master role check API, you can recognize if a Cloud Connector instance has currently the master role. The purpose of this master role check is only to recognize if the Cloud Connector instance is currently the master instance or not, without the need of providing credentials. It is a quick check that you can execute frequently.

URI	`/exposed?action=hasMasterRole`
Method	`GET`
Request	
Response	`{true, false}`
Errors	
Roles	All roles are accepted

Available as of version 2.18.1:

With the alternative master role check API below, you can also recognize whether the concrete Cloud Connector instance currently has the master role. If not, the HTTP status code will be shown as `409`.

URI	`/exposed?action=ensureMasterRole`
Method	`GET`
Request	
Response	`{true, false}`
Errors	`409` if the instance is not in master role.
Roles	All roles are accepted

Back to Available APIs [page 2016]

Back to Context [page 2016]

Alert Information (available as of version 2.19.0)

This API retrieves the alerts in a Cloud Connector instance: :

URI	`/api/monitoring/alerts`
Method	GET
Request	
Response	`[Array of alerts]`
Errors	
Roles	Administrator, Monitoring

Response Properties:

- `alerts`: array of alerts
  - `timestamp`: timestamp when the alert was generated
  - `status`: indicates the level of severity
  - `type`: the cause of the alert
  - `origin`: the HA role
  - `message`: message of the alert
  - `entity`: serves as a unique marker when multiple alerts of the same type can occur, otherwise empty
- `version`: API version

Sample Code

```
curl -k -H 'Accept:application/json' -u <user>:<password> -X GET https://
<scchost>:<sccport>/api/monitoring/alerts
```

Example:

```
{
  "alerts": [
    {
      "timestamp": 1739806853944,
      "status": "Warning",
      "type": "Disk",
      "origin": "Master",
      "message": "Free disk space dropped below configured threshold – 1.650.829 MB are free (Currently used by Cloud Connector: 260 MB)"
      ,
      "entity": ""
    },
    {
      "timestamp": 1739806853945,
      "status": "Warning",
      "type": "CPU",
      "origin": "Master",
      "message": "CPU load exceeded 0 percent for more than 0 seconds (Current usage all processes: 15%, Cloud Connector: 0%), check the
      hardware monitor for details",
      "entity": ""
    },
    {
      "timestamp": 1739806853982,
      "status": "Warning",
      "type": "UICertificate",
      "origin": "Master",
      "message": "UI certificate ('CN=SCC,OU=Connectivity,O=SAP SE,C=DE' issued by 'CN=SCC,OU=Connectivity,O=SAP SE,C=DE') expires
      2027-02-28 14:33:52 +0100",
      "entity": ""
    },
    {
      "timestamp": 1739806853993,
      "status": "Warning",
      "type": "SubaccountCertificate",
      "origin": "Master",
      "message": "Certificate of subaccount connectors_aws_integrate
      (44118284-0d31-419f-a303-41b86d08f576@cf.us10-integrate.hanavlab.ondemand.com) will expire 2025-11-11 14:06:07 +0100. Renew as
      soon as possible.",
      "entity": "44118284-0d31-419f-a303-41b86d08f576@cf.us10-integrate.hanavlab.ondemand.com"
    }
  ],
  "version": 1
}
```


Back to Available APIs [page 2016]

Back to Context [page 2016]

1.4.3.8.4 Performance Monitor

Monitor the cross-subaccount performance of the Cloud Connector.

The cross-subaccount performance monitor aggregates the performance data of all subaccounts for *cloud to on-premises* traffic, displayed in a single bar chart like the subaccount-specific counterpart (see Monitoring (Cloud to On-Premises) [page 2009]).

All contributing subaccounts are listed in the table below the bar chart. Use the buttons in column **Action** to view details of a subaccount, or navigate to its *cloud to on-premises* monitor, respectively.

1.4.3.9 Alerting

Configure the Cloud Connector to send e-mail messages when situations occur that may prevent it from operating correctly.

To configure alert e-mails, choose *Alerting* from the top-left navigation menu.

You must specify the receivers of the alert e-mails (*E-Mail Configuration*) as well as the Cloud Connector resources and components that you want to monitor (*Observation Configuration*). The corresponding *Alert Messages* are also shown in the Cloud Connector administration UI.

E-Mail Configuration

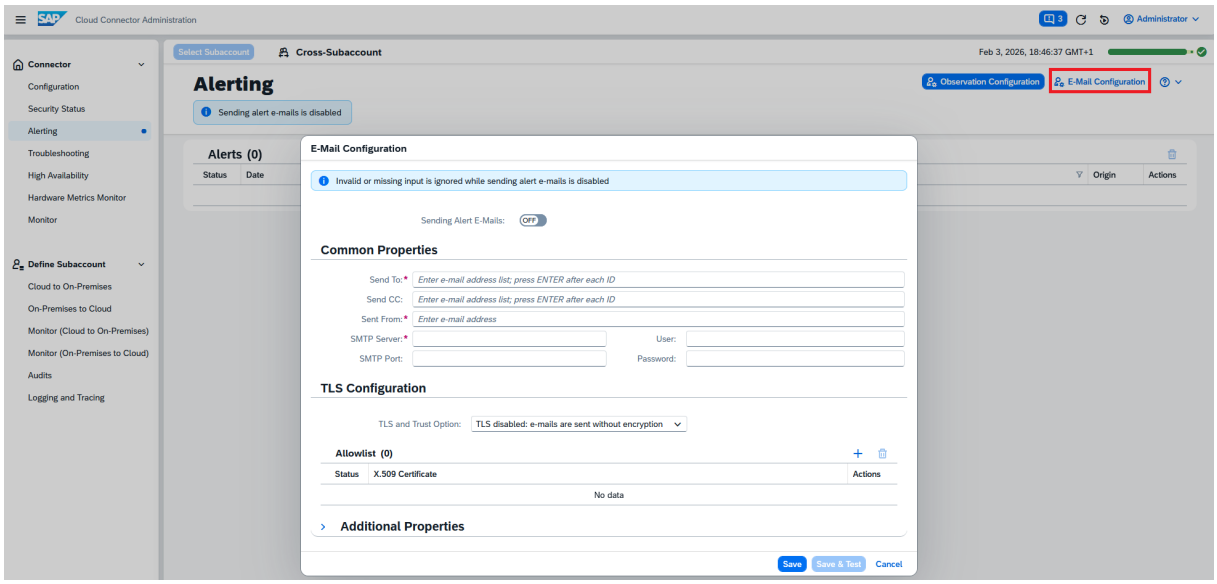
1. Select *E-Mail Configuration* to specify the list of e-mail addresses to which alerts should be sent (*Send To* and optionally *Send CC*).

   > **Note**
   >
   > You can assign e-mail addresses in compliance with RFC 2822. For example, john.doe@company.com or John Doe <j.doe@company.com> are valid e-mail addresses.

2. Enter the sender's e-mail address (`<Sent From>`).
3. In `<SMTP Server>` provide the host of the mail server.
4. You can specify an `<SMTP port>`, if the server is not using the default ports. For details, contact your e-mail administrator or provider.

5. If the SMTP server requires authentication, provide `<User>` and `<Password>`.
6. Set up TLS Configuration as required. The default is sending e-mails without encryption. For more information, see Configure Trust [page 1955].
7. In the *Additional Properties* section you can provide any property supported by the Java Mail library . Additionally, you can set `mail.debug` to `true` in order to get SMTP protocol debug output. All specified properties will be passed to the SMTP client.
8. Select *Save* to change the current configuration.

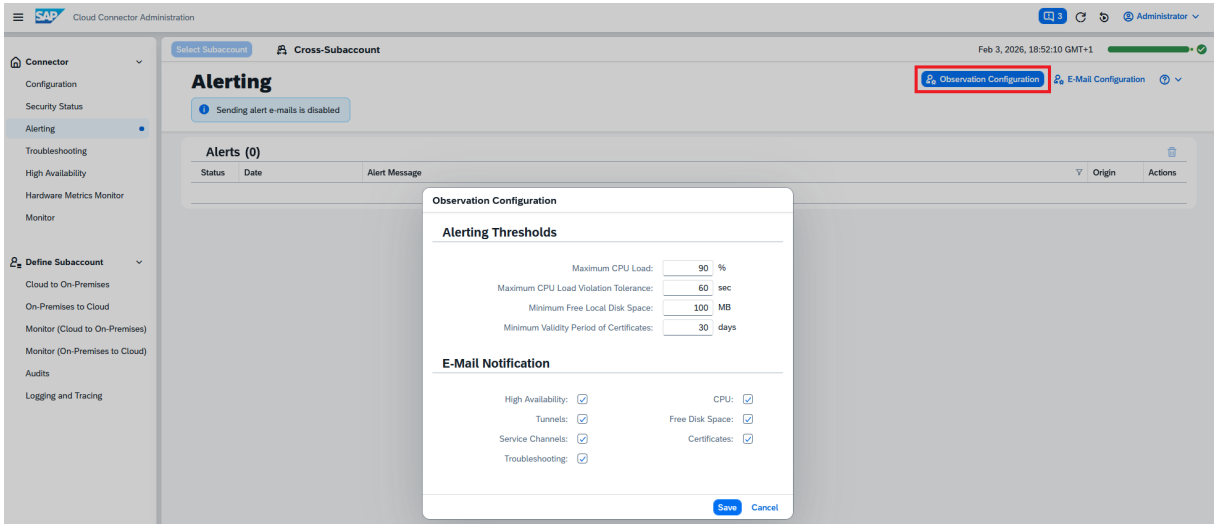
> ### Note
>
> Use the switch at the top of the dialog to enable or disable alert e-mails. Bear in mind that alert e-mails will not be sent out retroactively when switching on. Only alerts created while sending is enabled will trigger an e-mail.



> ### Note
>
> Connections to an SMTP server over TLS can cause TLS errors if the SMTP server uses an "untrusted" certificate. For more information on specifying a trust store, see Configure Trust [page 1955].
>
> If you want to modify the JDK's trust storage, see the JDK documentation. Usually, the trust storage is done in the file *cacerts* in the Java directory (`jre/lib/security/cacerts`). For import, you can use the *keytool* utility:
>
> ```
> keytool -import -storepass changeit -file <certificate used by SMTP server>
> -keystore cacerts -alias <for example, SMTP_xyz>
> ```
>
> For more information, see https://docs.oracle.com/cd/E19830-01/819-4712/ablqw/index.html .

Observation Configuration

In this section you can configure the surveillance of pivotal resources and components of the Cloud Connector: Alerts will be triggered as soon as any of the components or resources is deemed to malfunction, or is considered to be in a critical state. Additionally, e-mails will be sent as per e-mail configuration and the settings under *E-Mail Notification*.



- *High Availability* alerts can occur in the context of an active high availability setup, meaning a shadow system is connected.
- *Tunnels* and *Service Channels* refer to the state of the respective connections. Whenever such a connection cannot be opened or is lost, an alert is triggered.

  > ### Note
  >
  > These alerts are only triggered in case of an error or exception, but not upon intentional disconnect action.

- An excessively high *CPU* load over an extended period of time adversely affects performance and may be an indicator of serious issues that jeopardize the operability of the Cloud Connector. The CPU load is monitored and an alert is triggered whenever the CPU load exceeds and continues to exceed a given threshold percentage (the default is 90%) for more than a given period of time (the default is 60 seconds).
- Although the Cloud Connector does not require nor consume large amounts of disk space, running out of it is a circumstance that you should avoid as Cloud Connector configuration files could get corrupted. An alert is triggered if the **Free Disk Space** falls below a critical value (default is 100 megabytes).
- The Cloud Connector configuration contains various *Certificates*. Whenever one of those expires, scenarios might no longer work as expected. Therefore, it is important to get notified about the expiration (the default is 30 days).

  > ### Caution
  >
  > Trust store certificates are server certificates (issued by a CA) of systems that the Cloud Connector communicates with. As the Cloud Connector does not own those certificates, alerts are not triggered for the certificates in a trust store. Alerts are generated only for certificates, for which the Cloud Connector owns the private key as well.

- As of version 2.18.0, there is a Troubleshooting feature that detects certain issues and provides diagnoses consisting of possible causes and potential fixes to resolve the respective issues. An alert is triggered when adding certain kinds of diagnoses, typically those dealing with issues that occur in the background (for example, during requests processed by the Cloud Connector) as opposed to issues encountered on account of direct user interaction (for example, checking back-end connectivity).
  For more information, see Monitoring, Logging, and Troubleshooting [page 2046].

To configure the components to be monitored, proceed as follows:

1. Choose suitable values for the **Alerting Thresholds**.
2. Set up **E-Mail Notification** by selecting the observation categories for which you want to receive alert e-mails. By default, all categories are selected.
3. E-mails will be sent for all observation categories if **E-Mail Configuration** is set up to do so. Use the **Sending Alert E-Mails** switch (of the **E-Mail Configuration** dialog) if you want to enable or disable e-mail notification for all categories.
4. Press *Save* to change the current configuration.

Alert Messages

The Cloud Connector lists all alerts it triggers in a table on the main page, sorted by date (most recent alerts first). In general, theCloud Connector does not dispatch the same alert repeatedly. That is, recurring issues or persistent critical situations do not reproduce the same alert over and over. However, alerts pertaining to expiring or expired certificates are repeated with a reasonable frequency to remind you of the importance to attend to that matter. As soon as an issue or a critical situation has been resolved or overcome, an informational alert is generated, sent out and listed, replacing the original alert.

You can remove alerts using *Delete* or *Delete All*. If you delete active (unresolved) alerts, they reappear in the list after the next health check interval.

1.4.3.10 Audit Logging

Audit log data can alert Cloud Connector administrators to unusual or suspicious network and system behavior.

Additionally, the audit log data can provide auditors with information required to validate security policy enforcement and proper segregation of duties. IT staff can use the audit log data for root-cause analysis following a security incident.

The Cloud Connector includes an auditor tool for viewing and managing audit log information about access between the cloud and the Cloud Connector, as well as for tracking of configuration changes done in the Cloud Connector. The written audit log files are digitally signed by the Cloud Connector so that their integrity can be checked, see Manage Audit Logs [page 2042].

> **Note**
>
> We recommend that you permanently switch on Cloud Connector audit logging in productive scenarios.

- Under normal circumstances, set the logging level to **`Security`** (the default configuration value).
- If legal requirements or company policies dictate it, set the logging level to **`All`**. This lets you use the log files to, for example, detect attacks of a malicious cloud application that tries to access on-premises services without permission, or in a forensic analysis of a security incident.

We also recommend that you regularly copy the audit log files of the Cloud Connector to an external persistent storage according to your local regulations. The audit log files can be found in the Cloud Connector root directory `/log/audit/<subaccount-name>/audit-log_<timestamp>.csv`.

1.4.3.10.1 Manage Audit Logs

Configure audit log settings and verify the integrity of audit logs.

Configure Audit Logs in the Cloud Connector

Choose *Audit* from your subaccount menu and go to *Settings* to specify the type of audit events the Cloud Connector should log at runtime. You can currently select between the following *Audit Levels* (for either `<subaccount>` and `<cross-subaccount>` scope):

> ### Note
>
> The cross-subaccount scope includes all configuration activities in the Cloud Connector that are not related to a specific subaccount (that is, all settings which are not performed within the subaccount menu but under the **Connector** node of the Cloud Connector administration UI).

- **Security**: Default value. The Cloud Connector writes an audit entry (`Access Denied`) for each request that was blocked. It also writes audit entries, whenever an administrator changes one of the critical configuration settings, such as exposed back-end systems, allowed resources, and so on.

  > ### Note
  >
  > Find the full list of configuration activities that are considered critical in the list below, which shows the corresponding categories of audit log entries.

- **All**: The Cloud Connector writes one audit entry for each received request, regardless of whether it was allowed to pass or not (`Access Allowed` and `Access Denied`). It also writes audit entries that are relevant to the **Security** mode.
- **Off**: No audit entries are written.

> ### Caution
>
> To prevent a single person from being able to both change the audit log level, and delete audit logs, we recommend that the operating system administrator and the SAP BTP administrator are different persons. We also suggest that you turn on the audit log at the operating system level for file operations.

> → Tip
>
> We recommend that you don't log all events unless you are required to do so by legal requirements or company policies. Generally, logging security events only is sufficient.

To enable automatic cleanup of audit log files, choose a period (14 to 365 days) from the list in the field `<Automatic Cleanup>`.

Audit entries for configuration changes are written for the following categories representing the type of configuration that has been touched:

- `Account`: A subaccount configuration was changed.
- `Configuration`: A new subaccount was added or a disaster recovery switch happened.
- `BackendMapping`: Changes to the virtual to internal system mappings.
- `AllowedResource`: In a virtual system, changes in the accessible resources.
- `DomainMapping`: Changes to the domain mappings.
- `ServiceChannelConfiguration`: The configuration of a service channel was changed.
- `SCCPassword`: The Cloud Connector administration password was changed.
- `SCCUser`: The Cloud Connector administration user was changed.
- `LDAPConfiguration`: Changes to the LDAP settings.
- `EMailConfiguration`: The Email settings for alerts were changed.
- `AlertConfiguration`: The observation settings for alerts were changed.
- `ScimConfiguration`: Something changed in the settings for the cloud user store.
- `KerberosConfiguration`: The Kerberos configuration was changed.
- `SNCSettings`: SNC settings of Cloud Connector were changed.
- `ProxySettings`: The proxy settings were changed.
- `SystemCertificate`: The system certificate was changed.
- `PpcaCertificate`: The CA certificate was changed.
- `PrincipalPropagationConfiguration`: The principal propagation settings were changed.
- `TrustSynchronization`: The trust configuration for principal propagation was synchronized.
- `IdentityProviderTrust`: The trust configuration for a specific identity provider was changed.
- `ApplicationBasedIdentityProviderTrust`: Trust configuration for a specific application was changed (Neo environment only).
- `ApplicationTrust`: The trust configuration to applications was changed.
- `TrustedBackendCertificate`: The trust store certificate was added or removed.
- `SccCustomRoles`: Custom role name settings were changed.
- `BackendAuthority`: RFC-specific user and client settings were adjusted.
- `AdvancedConnectivity`: Advanced connectivity configuration was changed.
- `AdvancedJVM`: Advanced JVM configuration was changed.
- `ApplicationConfiguration`: Application-specific connection configuration was changed.
- `PayloadTrace`: Tunnel traffic trace was activated/deactivated.
- `PayloadSncTrace`: ABAP Cloud SNC traffic trace was activated/deactivated.
- `CPICTrace`: The CPIC trace level was changed.
- `AuditLogLevel`: The subaccount-specific audit log level was changed.

- `CrossAuditLogLevel`: The cross-subaccount audit log level was changed.
- `AuditLogCleanup`: The audit log cleanup setting was changed.
- `UiCertificate`: The Cloud Connector UI certificate was changed.
- `CloudConnectorLoggers`: The trace level for the *Cloud Connector Loggers* was changed.
- `OtherLoggers`: The trace level for the *Other Loggers* was changed.
- `TlsTrace`: TLS trace was activated/deactivated.

Other audit types do not have an additional subcategory that could group the audit entries.

In the *Audit Viewer* section, you can first define filter criteria, then display or download the selected audit entries.

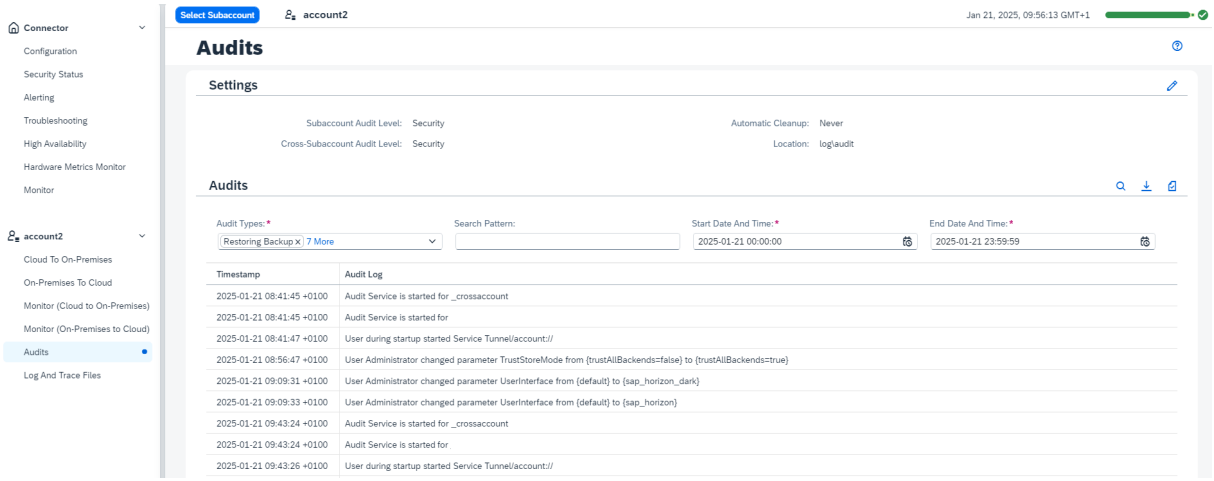
- In the *Audit Type* field, you can select the types of auditing events you are interested in.
- In the *Pattern* field, you can specify a certain string that the detail text of each selected audit entry must contain. The detail text may contain, for example, information about the user name, requested resource/URL, or the virtual `<host>:<port>`. Basic wildcards (glob patterns) are supported, that is, asterisk for any number of characters (including none) and question mark for a single character. Use this feature to do the following:
  - Filter the audit log for all requests that a particular HTTP user has made during a certain time frame.
  - Identify all users who attempted to request a particular URL.
  - Identify all requests to a particular back-end system.
  - Determine whether a user has changed a certain Cloud Connector configuration. For example, a search for string **`BackendMapping`** returns all `add-`, `delete-` or `modify-` operations on the *Mapping Virtual To Internal System* page.
- The *Time Range* settings specify the time frame to which you want to limit the eligible audit entries.

These filter criteria are combined with a logical `AND` so that all audit entries that match these criteria are shown. If you have modified the criteria, choose the *Search* button again to display the updated selection of audit events that match the new criteria.

Use the *Download* button to download the selected audit entries as a compressed (GZIP) CSV file that can be imported, for example, by Excel. The ZIP file also contains a second text file, a manifest or information, that provides the selection parameters as well as the date and time when the selection was extracted.

Example

In the following example, the *Audit Viewer* displays `Any` audit entries, at **`Security`** level, for the time frame between **Jan 21 2025, 00:00:00** and **Jan 21 2025, 23:59:59**:



Verify the Integrity of Audit Logs

To check the integrity of all or a part of the audit logs, go to `<scc_installation>/auditor`. This directory contains an executable `go` script file (respectively, `go.cmd` on Microsoft Windows and `go.sh` on other operating systems).

If you start the `go` file without specifying parameters from `<scc_installation>/auditor`, all available audit logs for the current Cloud Connector installation are verified.

The auditor tool is a Java application, and therefore requires a Java runtime, specified in JAVA_HOME, to execute:

- For Microsoft Windows OS, set *JAVA_HOME=<path-to-java-installation>*
- For Linux OS and Mac OS X, export: *JAVA_HOME=<path-to-java-installation>*

Alternatively, to execute Java, you can include the Java `bin` directory in the `PATH` variable.

You can check audit logs also in the UI, using the rightmost button of the **Audits** section.

> ### Note
>
> The selected date range determines the audit logs that are going to be checked (entire days only, ignoring the time of day).

Change the Location of Audit Logs

As of Cloud Connector 2.14 you can move audit logs to a different location. Standard location remains `log/audit`.

> ### Note
>
> Make sure there is enough space left on the device for the desired location and the Cloud Connector OS user has permission to write files to that location.

If you want to do this, proceed as follows:

1. Shut down the Cloud Connector.
2. Execute the respective script for the location change.
    1. For Microsoft Windows OS: `changeAuditLogPath.bat <desiredLocation>`.
    2. For Linux OS and Mac OS X: `./changeAuditLogPath.sh <desiredLocation>`.
3. The script checks if the target might be a network location. If this is assumed, the script asks for confirmation. Afterwards, it tries to move all existing audit logs to the new location. Only after successful move, the location change takes effect, otherwise it remains in the old place.
4. Start the Cloud Connector again.

> ⚠ Caution
>
> If you choose a network location while access to the file system is slow, overall processing performance of the Cloud Connector may decrease significantly.

1.4.3.11 Monitoring, Logging, and Troubleshooting

To troubleshoot connection problems, monitor the state of your open tunnel connections in the Cloud Connector, and view different types of logs and traces.

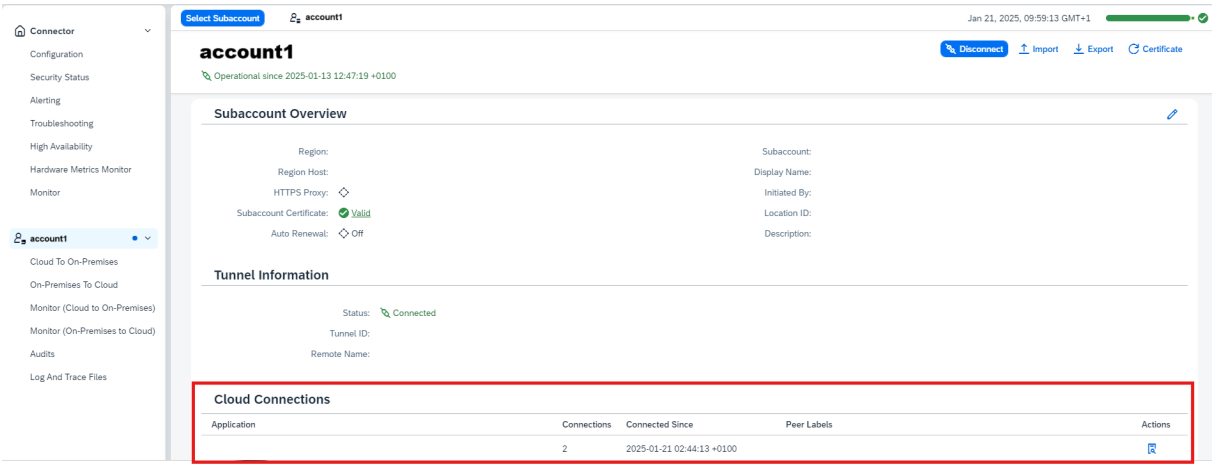
This section provides details on how to monitor the state of your open tunnel connections in the Cloud Connector. You can also view different types of logs and traces that may help you troubleshoot connection problems.

A dedicated **Troubleshooting** screen analyzes certain types of issues and proposes ways to resolve them.

> ⓘ Note
>
> For information about a specific problem or an error you have encountered, see also Connectivity Support [page 2362].

Monitoring

To view a list of all currently connected applications, choose your *Subaccount* from the left menu and go to section *Cloud Connections*:



The provided information includes:

- *Application name*: The name of the application, as also shown in the cockpit, for your subaccount
- *Connections*: The number of currently existing connections to the application
- *Connected Since*: The earliest start time of a connection to this application
- *Peer Labels*: The name of the application processes, as also shown for this application in the cockpit, for your subaccount

Log and Trace Settings

The *Log and Trace Files* page includes some files for troubleshooting that are intended primarily for SAP Support. These files include information about both internal Cloud Connector operations and details about the communication between the local and the remote (SAP BTP) tunnel endpoint.

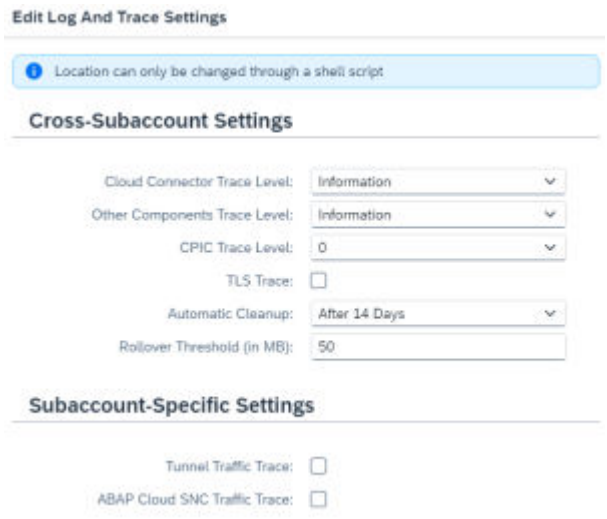
If you encounter problems that seem to be caused by some trouble in the communication between your cloud application and the on-premises system, choose *Log and Trace Files* from your *Subaccount* menu, go to section *Settings*, and activate the respective traces by selecting the *Edit* button:

- *Cloud Connector Trace Level* adjusts the levels for Java loggers directly related to Cloud Connector functionality.
- *Other Components Trace Level* adjusts the log level for all other Java loggers available at the runtime. Change this level only when requested to do so by SAP support. When set to a level higher than `Information`, it generates a large number of trace entries.
- *CPIC Trace Level* allows you to set the level between 0 and 3 and provides traces for the CPIC-based RFC communication with ABAP systems.
- When the *Tunnel Traffic Trace* is activated for a subaccount, all the traffic crossing the tunnels for that subaccount going through this Cloud Connector, is traced in files with names `tunnel_traffic_<account id>_on_<landscapehost>.trc`. This is helpful if you need to understand what documents have been exchanged between the involved systems, independent from the initiator of the call: Requests triggered by cloud applications are traced as well as the ones that are triggered by an application in the Cloud Connector network via a service channel in the subaccount.

- *TLS Trace* adds a TLS trace for all TLS-protected communication to the `scc_core.trc` file. A restart is required for the change to take effect. Only activate this trace when requested by SAP support. It has a significant impact on performance as it produces large amounts of traces.
- **ABAP Cloud SNC Traffic Trace**: When the *ABAP Cloud SNC traffic trace* is activated for a subaccount, all RFC SNC-based traffic crossing a service channel for that account (going through this Cloud Connector), is traced in files with names *snc_traffic_<account id>_on_<landscapehost>.trc*. This is helpful if you need to understand issues with SNC termination in the Cloud Connector.
- *SSL Trace*: When the SSL trace is activated, the `scc_core.trc` file includes information for SSL-protected communication. To activate a change of this setting, a restart is required. Activate this trace only when requested by SAP support. It has a high impact on performance as it produces a large amount of traces.

Apart from selecting trace levels for different types of traces there are further settings affecting the size and lifetime of trace files:

- *Automatic Cleanup* lets you remove old trace files that have not been changed for a period of time exceeding the configured interval. You can choose from a list of predefined periods. The default is `Never`.
- *Rollover Threshold* sets the maximal size of a `scc_core.trc` file (in MB) before the rollover mechanism kicks in.



> ⚠ Caution
>
> Use any **traffic and CPIC tracing at level 3** carefully, and only when requested to do so for support reasons. These traces may write sensitive information (such as payload data of HTTP/RFC requests and responses) to the trace files, and thus present a potential security risk. The Cloud Connector supports the implementation of a "four-eyes principle" for activating the trace levels that dump the network traffic into a trace file. This principle requires two users to activate a trace level that records traffic data.
>
> For more information, see Secure the Activation of Traffic Traces [page 2004].

Change the Location of Trace Files

As of Cloud Connector 2.14 you can move trace files to a different location.

> ### Note
>
> JVM-related files will remain in the standard location `log`.

> ### Note
>
> Make sure there is enough space left on the device for the desired location and the Cloud Connector OS user has permission to write files to that location.

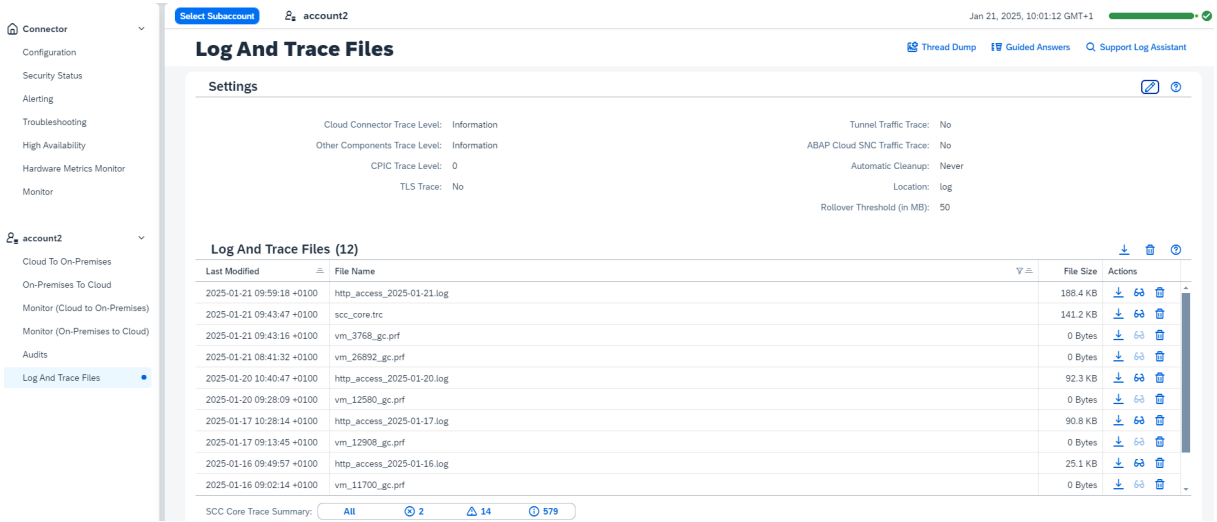
If you want to do this, proceed as follows:

1. Shut down the Cloud Connector.
2. Execute the respective script for the location change.
    1. For Microsoft Windows OS: `changeLogAndTracePath.bat <desiredLocation>`.
    2. For Linux OS and Mac OS X: `./changeLogAndTracePath.sh <desiredLocation>`.
3. The script checks if the target might be a network location. If this is assumed, the script asks for confirmation. Afterwards, it tries to move the existing current trace file to the new location. Only after successful move, the location change takes effect. Otherwise, the file remains in the old place.
4. Start the Cloud Connector again.

> ### Caution
>
> If you choose a network location while access to the file system is slow, overall processing performance of the Cloud Connector may decrease significantly.

Log and Trace Files

View all existing trace files and delete the ones that are no longer needed.



To prevent your browser from being overloaded when multiple large files are loaded simultaneously, the Cloud Connector loads only one page into memory. Use the page buttons to move through the pages.

Use the *Download*/*Download All* icons to create a ZIP archive containing one trace file or all trace files. Download it to your local file system for convenient analysis.

> **Note**
>
> If you want to download more than one file, but not all, select the respective rows of the table and choose *Download All*.

When running the Cloud Connector with SAP JVM or as of version 2.14 also with other JVMs, you can trigger the creation of a thread dump by choosing the *Thread Dump* button, which will be written to the JVM trace file *log/vm_$PID_trace.log* for SAP JVM and *log/vm_$PID_threads.log* for other JVMs. You may be asked by SAP support to create one, if considered helpful during incident analysis.

> **Note**
>
> From the UI, you can't delete trace files that are currently in use. You can delete them from the Linux OS command line; however, we recommend that you do not use this option to avoid inconsistencies in the internal trace management of the Cloud Connector.

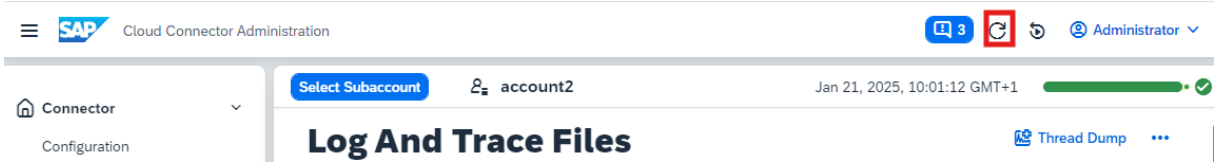
Two buttons may be helpful to solve issues on your own:

- *Guided Answers*: A new tab or window opens, showing the Cloud Connector section in Guided Answers. It helps you identify many issues that are classified through hierarchical topics. Once you found a matching issue, a solution is provided either directly, or by references to SAP Help Portal, Knowledge Base Articles (KBAs), and SAP notes.
- *Support Log Assistant*: Opens the support log assistant. There, you can upload Cloud Connector log files and have them analyzed. After triggering the scan, the tool lists all issues for which a solution can be identified.

  > **Note**
  >
  > The support log assistant analyzes the complete log. Therefore, also older issues may be found that are no longer relevant.

Once a problem has been identified, you should turn off the trace again by editing the trace and log settings accordingly to not flood the files with unnecessary entries.

Use the *Refresh* button to update the information that appears. For example, you can use this button because more trace files might have been written since you last updated the display.



Error Analysis and Support: Which Logs are Relevant?

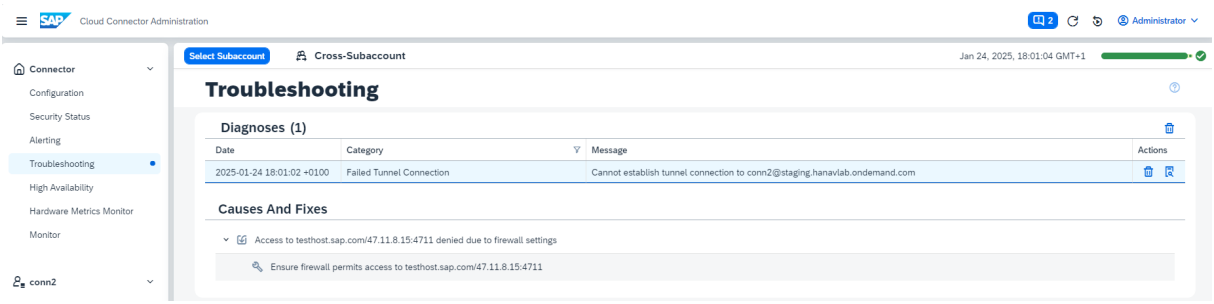
If you contact SAP support for help, please always attach the appropriate log files and provide the timestamp or period, when the reported issue was observed. Depending on the situation, different logs may help to find the root cause.

Some typical settings to get the required data are listed below:

- `<Cloud Connector Trace>` provides **details related to connections to SAP BTP and to backend systems as well as master-shadow communication in case of a high availability setup**. However, it does not contain any payload data. This kind of trace is written into `scc_core.trc`, which is the most relevant log for the Cloud Connector.
- `<Other Components Trace>` provides **details related to the tomcat runtime**, in which the Cloud Connector is running. The traces are written into `scc_core.trc` as well, but they are needed only in very special support situations. If you don't need these traces, leave the level on `Information` or even lower.
- **Tunnel traffic** is written into the traffic trace file for HTTP or RFC requests if the tunnel traffic trace is activated, or into the CPI-C trace file for RFC requests, if the CPI-C trace is set to level 3.
- **ABAP Cloud SNC traffic** is written into a ABAP Cloud SNC traffic file for incoming SNC RFC requests if ABAP Cloud SNC traffic trace is activated.
- `<TLS trace>` is helpful to **analyze TLS handshake failures** from Cloud Connector to Cloud or from Cloud Connector to backend. It should be turned off again as soon as the issue has been reproduced and recorded in the traces.
- Setting the audit log on level `ALL` for `<Subaccount Audit Level>` is the easiest way to **check if a request reached the the Cloud Connector and if it is being processed**.

Troubleshooting

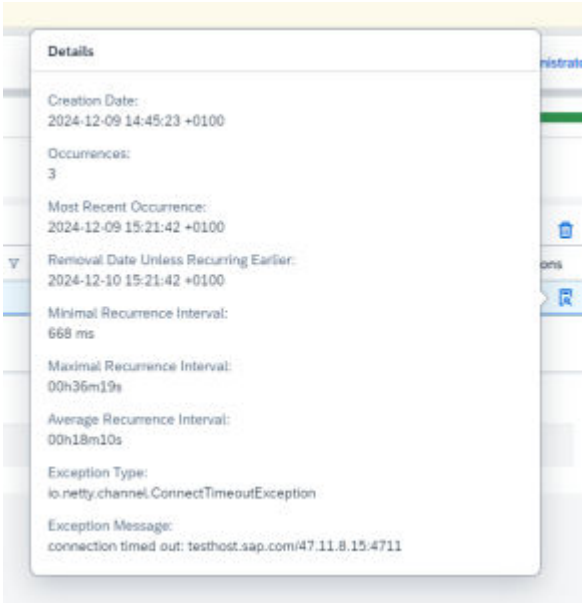
Certain types of issues are reported, analyzed, and in some cases recognized as configuration issues that can be resolved. Use the *Troubleshooting* screen to view the current list of issues, or more precisely the diagnoses of issues:



Certain types of issues are sent to and processed by the troubleshooting framework. The respective diagnoses are listed in the table **Diagnoses**. Select the row of the issue/diagnosis of interest to see the causes and fixes associated with it.

> ### Note
>
> The proposed causes and fixes are not guaranteed to explain or resolve an issue, and should therefore be regarded as hints rather than definite solutions.

A recurring issue will not create a new entry in the table. Instead, a counter for its occurrences will be incremented. You can view details like the number of occurrences in a pop-up dialog that you can open with the rightmost button of the **Actions** column:



Here, **Creation Date** refers to the first recorded occurrence. Some issues will be removed 24 hours after the most recent occurrence, as shown below **Removal Date Unless Recurring Earlier**. In other words, if an issue does not reoccur within 24 hours, its diagnosis will be discarded.

Some diagnoses will be removed as soon as the respective issues were resolved. In that case, there is no **Removal Date Unless Recurring Earlier**.

The *Minimal*, *Maximal*, and *Average Recurrence Interval* give a rough idea of the recurrence pattern. This information is meaningful and therefore shown only if an issue occurred more than once (within 24 hours).

For support purposes, the type and message of the technical exception behind each entry can be of interest. We recommend that you attach a screenshot of these details when creating a support ticket in this context.

Diagnoses can be deleted.

> ### Note
>
> Deletion will only be of a temporary nature if an issue reoccurs. Upon recurrence, it will be listed again with the occurrence counter reset to one.

> ### Note
>
> **Alerting**

Certain types of diagnoses, in particular those related to issues occurring in the background (for example, incoming requests) trigger an alert. Depending on your Alerting **E-Mail Configuration**, this may also mean that an e-mail is sent out to inform all recipients. Recurrent issues will not trigger alerts and therefore will also not trigger e-mails. If no e-mail is to be sent when a new diagnosis is created, unselect the checkbox for *Troubleshooting* of the **E-Mail Notification** (see Alerting [page 2040], section **Observation Configuration**).

Deleting a diagnosis does not affect the alert associated with it. If necessary, the alert can be removed manually. Note that is not always feasible to obtain the information that an issue was resolved, and that therefore there may not be a recovery notification in this context. In these cases, if an issue was resolved there is no automatic removal of the respective diagnosis, nor is the associated alert removed or modified.

Scripts for Configuration (Issues)

You can use the following scripts to perform configuration changes in case of issues, without accessing the Cloud Connector administration UI.

Script Name	Use
`changeAuditLogPath <newAuditlogPath>`	Changes the location for the audit log files.
`changeLogAndTracePath <newLogAndTracePath>`	Changes the location for the log and trace files.
`changePort <newPort>`	Changes the port for the UI access.
`changeRole <master|shadow>`	Changes the current high availability (HA) role.
`resetCiphers`	Reset all ciphers used for HTTPS communication to the administration UI to the default.
`snc_create_pse`	Setting up SNC with this guided wizard.
`snc_import_ca_response`	If not done with `snc_create_pse`, import a CA response to the existing PSE.
`useBasicAuthentication`	Use basic authentication and discard any stored certificate-based login settings.
`useFileUserStore`	Use local user store and discard any stored LDAP configuration.

Related Information

Getting Support

1.4.3.12 Process Guidelines for Hybrid Scenarios

A hybrid scenario is one, in which applications running on SAP BTP require access to on-premises systems. Define and document your scenario to get an overview of the required process steps.

Tasks

Document the Landscape of a Hybrid Solution [page 2054]

Document Administrator Roles [page 2055]

Document Communication Channels [page 2055]

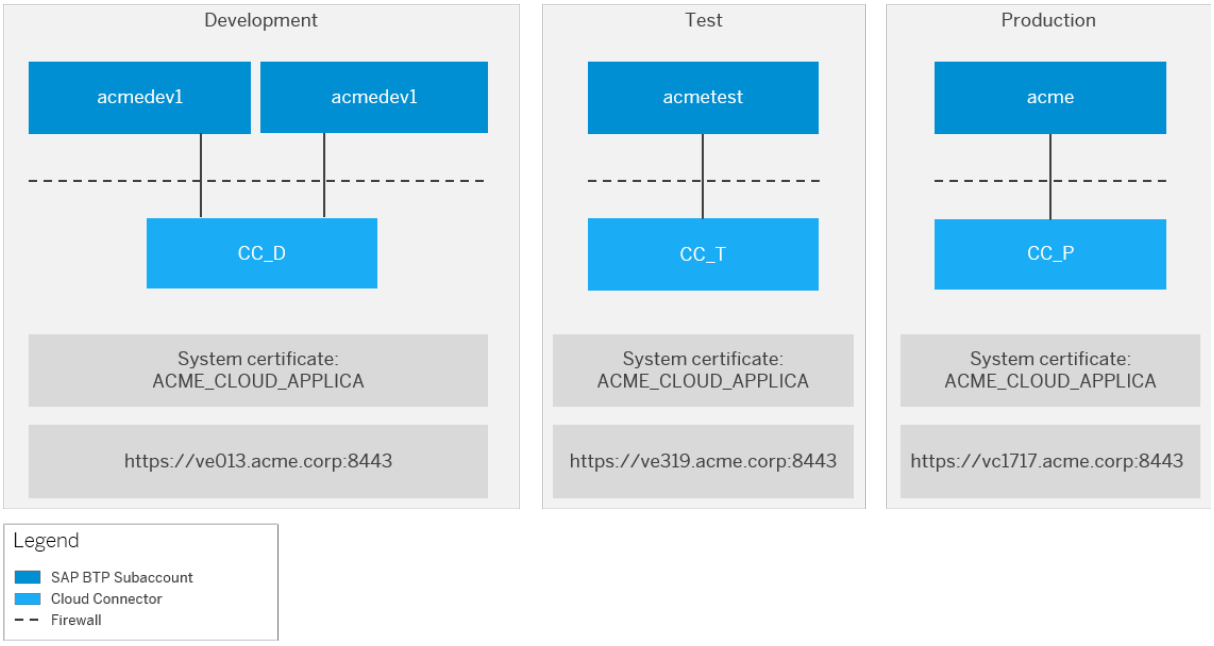
Define Project and Development Guidelines [page 2056]

Define How to Set a Cloud Application Live [page 2056]

Document the Landscape of a Hybrid Solution

To gain an overview of the cloud and on-premises landscape that is relevant for your hybrid scenario, we recommend that you diagrammatically document your cloud subaccounts, their connected Cloud Connectors and any on-premises backend systems. Include the subaccount names, the purpose of the subaccounts (dev, test, prod), information about the Cloud Connector machines (host, domains), the URLs of the Cloud Connectors in the landscape overview document, and any other details you might find useful to include.

An example of landscape overview documentation could look like this:



Back to Tasks [page 2054]

Document Administrator Roles

Document the users who have administrator access to the cloud subaccounts, to the Cloud Connector operating system, and to the Cloud Connector administration UI.

Such an administrator role documentation could look like following sample table:

Resource	bernardo@acme.com	mary@acme.com	smitha@acme.com	greg@acme.com
Cloud Subaccount (CA) Dev1	X			
CA Dev2		X		
CA Test			X	X
CA Prod				X
Cloud Connector Dev1 + Dev2	X	X		
Cloud Connector Test			X	X
Cloud Connector Prod				X
Cloud Connector Dev1 + Dev2 file system			X	X
Cloud Connector Test file system				X
Cloud Connector Prod file system				

Back to Tasks [page 2054]

Document Communication Channels

Create and document separate email distribution lists for both the cloud subaccount administrators and the Cloud Connector administrators.

An example of the documented communication channels could look like this:

Landscape	Distribution List
Cloud Subaccount Administrators	DL ACME BTP Subaccount Admins
Cloud Connector Administrators	DL ACME Cloud Connector Admins

Back to Tasks [page 2054]

Define Project and Development Guidelines

Define and document mandatory project and development guidelines for your SAP BTP projects. An example of such a guideline could be similar to the following.

Every SAP BTP project in this organization requires the following:

- Use Maven, Nexus, Git-&-Gerrit for the application development.
- Align with accountable manager in projects (including the names).
- Align with accountable security officer in projects (including the names).
- For externally developed source code, an official handover to the organization.
- Fulfill connection restrictions in a three-system landscape, that is, use a staged landscape for dev, test and prod, and, for example, the dev landscape connects only to dev systems, and so on.
- Productive subaccounts cannot use the same Cloud Connector as a dev or test subaccount.

Back to Tasks [page 2054]

Define How to Set a Cloud Application Live

Define and document how to set a cloud application live and how to configure needed connectivity for such an application.

For example, the following processes could be seen as relevant and should be defined and document in more detail:

1. Transferring application to production: Steps for transferring an application to the productive status on the SAP BTP.
2. Application connectivity: The steps for adding a connectivity destination to a deployed application for connections to other resources in the test or productive landscape.
3. Cloud Connector Connectivity: Steps for adding an on-premises resource to the Cloud Connector in the test or productive landscapes to make it available for the connected cloud subaccounts.
4. On-premises system connectivity: The steps for setting up a trusted relationship between an on-premises system and the Cloud Connector, and to configure user authentication and authorization in the on-premises system in the test or productive landscapes.
5. Application authorization: The steps for requesting and assigning an authorization that is available inside the SAP BTP application to a user in the test or productive landscapes.
6. Administrator permissions: Steps for requesting and assigning the administrator permissions in a cloud subaccount to a user in the test or productive landscape.

Back to Tasks [page 2054]

1.4.3.13 Configuring Backup

Find an overview of backup procedures for the Cloud Connector.

Configuration Backup [page 1964]	Backup and restore your Cloud Connector configuration via the administration UI.
Backup [page 1844]	Manage the Cloud Connector's configuration backup via REST API.
Backup And Restore Configuration [page 1946]	Example: Backup and restore the Cloud Connector configuration via REST API.

1.4.3.14 Secure Store

Reduce the size of the Cloud Connector Secure Store.

The Secure Store is a container for confidential information that is a part of the Cloud Connector configuration. The Secure Store resides in file `SSFS_SCC.DAT`, located in directory *scc_config*.

Every change affecting the Secure Store (such as changing the proxy password) is incremental, that is, it is appended to the Secure Store. As a consequence, the Secure Store will grow over time.

At some point it is sensible to remove obsolete entries to reduce the size of the Secure Store, and hence the size of the file `SSFS_SCC.DAT`.

To do this, choose *Configuration* > *Shrink Secure Store* (top right corner of the screen) from the Cloud Connector main menu.

1.4.4 Security

Learn how Cloud Connector features help you manage security.

Features

Security is a crucial concern for any cloud-based solution. It has a major impact on the business decision of enterprises whether to make use of such solutions. SAP BTP is a platform-as-a-service offering designed to run business-critical applications and processes for enterprises, with security considered on all levels of the on-demand platform:

Level	Features
Application Layer [page 2058]	• Frontend security • Security standard-based application development
Service Layer [page 2059]	• Identity and access management • Data protection • Regulatory compliance management
Cloud Infrastructure Layer [page 2064]	• Network infrastructure and communication • Sandboxing • Intrusion detection and prevention
Physical and Environmental Layer [page 2065]	• Strict physical access control • High availability

The Cloud Connector enables integration of cloud applications with services and systems running in customer networks, and supports database connections from the customer network to SAP HANA databases running on SAP BTP. As these are security-sensitive topics, this section gives an overview on how the Cloud Connector helps maintain security standards for the mentioned scenarios.

Target Audience

The content of this section concerns:

- System and IT administrators
- Technology consultants
- Solution architects

Related Information

Security Guidelines [page 2065]

1.4.4.1 Application Layer

On application level, the main tasks to ensure secure Cloud Connector operations are to provide appropriate frontend security (for example, validation of entries) and a secure application development.

Product Security Standard

Basically, you should follow the rules given in the product security standard, for example, protection against cross-site scripting (XSS) and cross-site request forgery (XSRF).

Scope and Design

The scope and design of security measures on application level strongly depend on the specific needs of your application.

1.4.4.2 Service Layer

You can use SAP BTP Connectivity to securely integrate cloud applications with systems running in isolated customer networks.

Overview

After installing the Cloud Connector as integration agent in your on-premises network, you can use it to establish a persistent TLS tunnel to SAP BTP subaccounts.

To establish this tunnel, the Cloud Connector administrator must authenticate himself or herself against the related SAP BTP subaccount of which he or she must be a member. Once estabilshed, the tunnel can be used by applications of the connected subaccount to remotely call systems in your network.

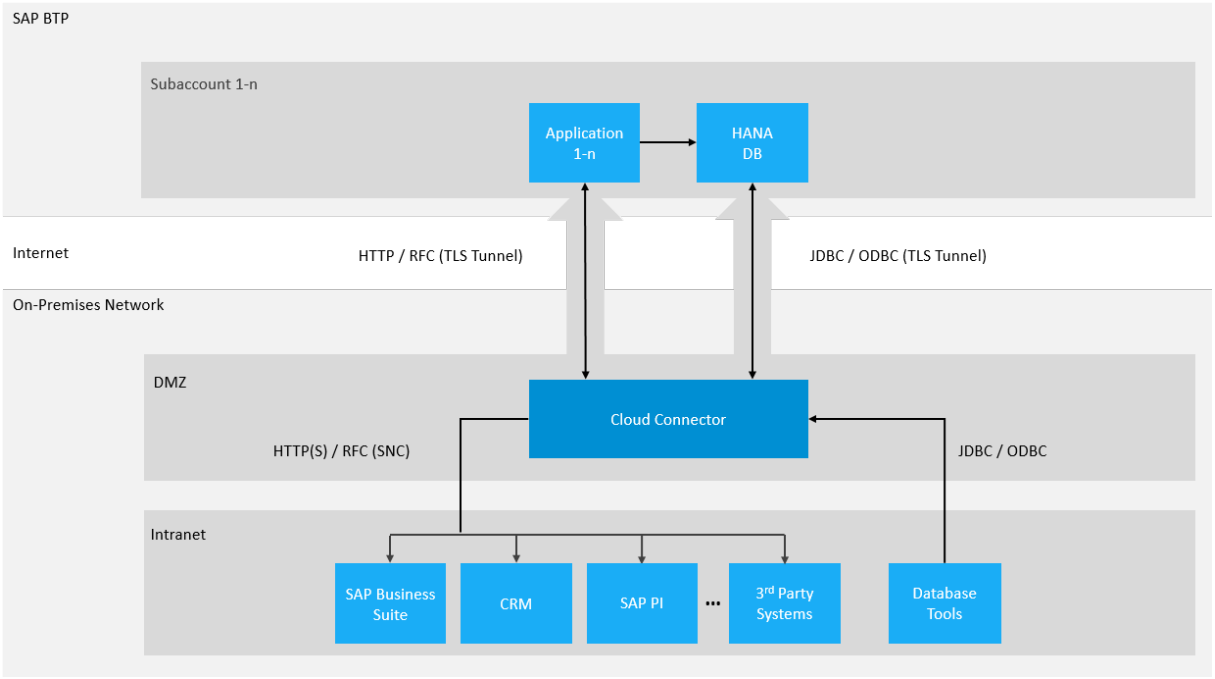
Architecture

The figure below shows a system landscape in which the Cloud Connector is used for secure connectivity between SAP BTP applications and on-premises systems.

- A single Cloud Connector instance can connect to multiple SAP BTP subaccounts, each connection requiring separate authentication and defining an own set of configuration.
- You can connect an arbitrary number of SAP and non-SAP systems to a single Cloud Connector instance.
- The on-premises system does not need to be touched when used with the Cloud Connector, unless you configure trust between the Cloud Connector and your on-premises system. A trust configuration is required, for example, for principal propagation (single sign-on), see Configuring Principal Propagation [page 1702].

- You can operate the Cloud Connector in a high availability mode. To achieve this, you must install a second (redundant) Cloud Connector (shadow instance), which takes over from the master instance in case of a downtime.
- The Cloud Connector also supports the communication direction from the on-premises network to the SAP BTP subaccount, using a database tunnel that lets you connect common ODBC/JDBC database tools to SAP HANA as well as other available databases in SAP BTP.

.



Related Information

Network Zones [page 2060]
Inbound Connectivity [page 2061]
Outbound Connectivity [page 2063]
Audit Log [page 2063]

1.4.4.2.1 Network Zones

Choosing a network zone for the Cloud Connector installation.

A company network is usually divided into multiple network zones according to the security level of the contained systems. The DMZ network zone contains and exposes the external-facing services of an organization to an untrusted network, typically the Internet. Besides this, there can be one or multiple other network zones which contain the components and services provided in the company's intranet.

You can set up the Cloud Connector either in the DMZ or in an inner network zone. Technical prerequisites for the Cloud Connector to work properly are:

- The Cloud Connector must have access to the SAP BTP landscape host, either directly or via HTTPS proxy (see also: Prerequisites [page 1610]).
- The Cloud Connector must have direct access to the internal systems it shall provide access to. I.e. there must be transparent connectivity between the Cloud Connector and the internal system.

It's a company's decision, whether the Cloud Connector is set up in the DMZ and operated centrally by an IT department or set up in the intranet and operated by the line of business.

Related Information

Network Zones [page 1648]

1.4.4.2.2 Inbound Connectivity

For inbound connections into the on-premises network, the Cloud Connector acts as a reverse invoke proxy between SAP BTP and the internal systems.

Exposing Resources

Once installed, none of the internal systems are accessible by default through the Cloud Connector: you must configure explicitly each system and each service and resource on every system to be exposed to SAP BTP in the Cloud Connector.

You can also specify a virtual host name and port for a configured on-premises system, which is then used in the cloud. Doing this, you can avoid that information on physical hosts is exposed to the cloud.

TLS Tunnel

The TLS (Transport Layer Security) tunnel is established from the Cloud Connector to SAP BTP via a so-called **reverse invoke** approach. This lets an administrator have full control of the tunnel, since it can't be established from the cloud or from somewhere else outside the company network. The Cloud Connector administrator is the one who decides when the tunnel is established or closed.

The tunnel itself is using TLS with strong encryption of the communication, and mutual authentication of both communication sides, the client side (Cloud Connector) and the server side (SAP BTP).

The X.509 certificates which are used to authenticate the Cloud Connector and the SAP BTP subaccount are issued and controlled by SAP BTP. They are kept in secure storages in the Cloud Connector and in the cloud.

Having encrypted and authenticated the tunnel, confidentiality and authenticity of the communication between the SAP BTP applications and the Cloud Connector is guaranteed.

Restricting Allowed Applications

As an additional level of control, the Cloud Connector optionally allows restricting the list of SAP BTP applications which are able to use the tunnel. This is useful in situations where multiple applications are deployed in a single SAP BTP subaccount while only particular applications require connectivity to on-premises systems.

Isolation on Subaccount Level

SAP BTP guarantees strict isolation on subaccount level provided by its infrastructure and platform layer. An application of one subaccount is not able to access and use resources of another subaccount.

Supported Protocols

The Cloud Connector supports inbound connectivity for HTTP and RFC, any other protocol is not supported.

- The payload sent via these protocols is encrypted on TLS/tunnel-level.
- For the route from the Cloud Connector to the on-premises systems, Cloud Connector administrators have the choice for each configured on-premises system whether to use HTTP, HTTPS, RFC or RFC over SNC.
- For HTTPS, you can configure a so-called system certificate in the Cloud Connector which is used for the trust relationship between the Cloud Connector and the connected on-premises systems.
- For RFC over SNC, you can configure an SNC PSE in the Cloud Connector respectively.

Principal Propagation

The Cloud Connector also supports principal propagation of the cloud user identity to connected on-premises systems (single sign-on). For this, the system certificate (in case of HTTPS) or the SNC PSE (in case of RFC) is mandatory to be configured and trust with the respective on-premises system must be established. Trust configuration, in particular for principal propagation, is the only reason to configure and touch an on-premises system when using it with the Cloud Connector.

Related Information

Configuring Principal Propagation [page 1702]

1.4.4.2.3 Outbound Connectivity

The Cloud Connector supports the communication direction from the on-premises network to SAP BTP, using a database tunnel.

The database tunnel is used to connect local database tools via JDBC or ODBC to the SAP HANA DB or other databases onSAP BTP, for example, SAP Business Objects tools like Lumira, BOE or Data Services.

- The database tunnel only allows JDBC and ODBC connections from the Cloud Connector into the cloud. A reuse for other protocols is not possible.
- The tunnel uses the same security mechanisms as for the inbound connectivity:
  - TLS-encryption and mutual authentication
  - Audit logging

To use the database tunnel, two different SAP BTP users are required:

- A platform user (member of the SAP BTP subaccount) establishes the database tunnel to the HANA DB.
- A HANA DB user is needed for the ODBC/JDBC connection to the database itself. For the HANA DB user, the role and privilege management of HANA can be used to control which actions he or she can perform on the database.

Related Information

Using Service Channels [page 1949]

1.4.4.2.4 Audit Log

As audit logging is a critical element of an organization's risk management strategy, the Cloud Connector provides audit logging for the complete record of access between cloud and Cloud Connector as well as of configuration changes done in the Cloud Connector.

Integrity Check

The written audit log files are digitally signed by the Cloud Connector so that they can be checked for integrity (see also: Manage Audit Logs [page 2042]).

Alerting

The audit log data of the Cloud Connector can be used to alert Cloud Connector administrators regarding unusual or suspicious network and system behavior.

Additional Use Cases

- The audit log data can provide auditors with information required to validate security policy enforcement and proper segregation of duties.
- IT staff can use the audit log data for root-cause analysis following a security incident.

Related Information

Audit Logging [page 2041]

1.4.4.3 Cloud Infrastructure Layer

Infrastructure and network facilities of the SAP BTP ensure security on network layer by limiting access to authorized persons and specific business purposes.

Isolated Network

The SAP BTP landscape runs in an isolated network, which is protected from the outside by firewalls, DMZ, and communication proxies for all inbound and outbound communications to and from the network.

Sandboxed Environments

The SAP BTP infrastructure layer also ensures that platform services, like the SAP BTP Connectivity, and applications are running isolated, in sandboxed environments. An interaction between them is only possible over a secure remote communication channel.

1.4.4.4 Physical and Environmental Layer

Learn about data center security provided for SAP BTP Connectivity.

SAP BTP runs in data centers which are compliant with regulatory requirements. The security measures include, for example:

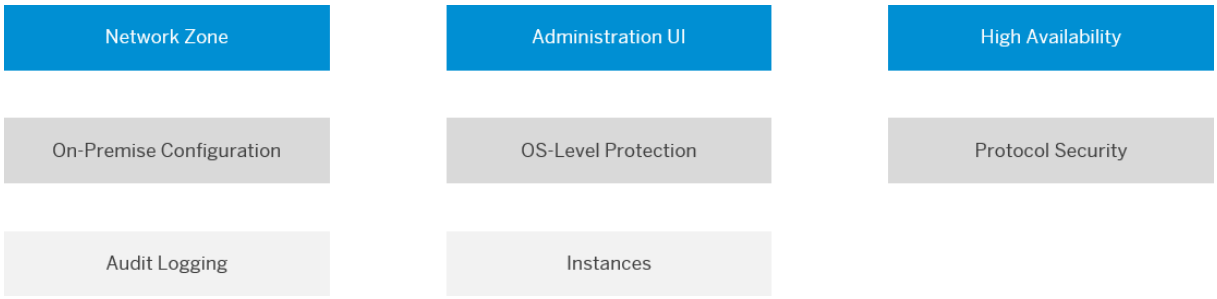
- strict physical access control mechanisms using biometrics, video surveillance, and sensors
- high availability and disaster recoverability with redundant power supply and own power generation

1.4.4.5 Security Guidelines

Find a checklist of recommended security measures for the Cloud Connector.

Topics

Hover over the elements for a description. Click an element to find the recommended actions in the table below.



- #unique_37/unique_37_Connect_42_network [page 2066]
- #unique_37/unique_37_Connect_42_ui [page 2066]
- #unique_37/unique_37_Connect_42_availability [page 2067]
- #unique_37/unique_37_Connect_42_protocols [page 2068]
- #unique_37/unique_37_Connect_42_os [page 2066]
- #unique_37/unique_37_Connect_42_oP [page 2068]
- #unique_37/unique_37_Connect_42_instances [page 2068]
- #unique_37/unique_37_Connect_42_audit [page 2067]

Recommended Actions

Topic	Description	Recommended Action
Network Zone Back to Topics [page 2065]	Depending on the needs of the project, the Cloud Connector can be either set up in the **DMZ** and operated centrally by the IT department or set up in the **intranet** and operated by the line-of-business.	To access highly secure on-premises systems, operate the Cloud Connector centrally by the IT department and install it in the DMZ of the company network. Set up trust between the on-premises system and the Cloud Connector, and only accept requests from trusted Cloud Connectors in the system.
OS-Level Protection Back to Topics [page 2065]	The Cloud Connector is a security-critical component that handles the inbound access from SAP BTP applications to systems of an on-premises network. **Methods to secure the operating system**, on which the Cloud Connector is running, should be applied.	Restrict access to the operating system on which the Cloud Connector is installed to the minimal set of users who should administrate the Cloud Connector. Use the machine which runs the Cloud Connector only for this purpose and don't reuse it for other scenarios. Use hard-drive encryption for the machine that runs the Cloud Connector. This ensures that the Cloud Connector configuration data cannot be read or modified by unauthorized users, even if they obtain access to the hard drive. Turn on the audit log on operating system level to monitor the file operations.
Administration UI Back to Topics [page 2065]	After installation, the Cloud Connector provides an initial user name and password and forces the user (`Administrator`) to **change the password** upon initial logon.	Change the password of the *Administrator* user immediately after installation. Choose a strong password for the user (see also Recommendations for Secure Setup [page 1665]). Configure a corporate LDAP system for the user management of the Cloud Connector administrator users. This guarantees that users of the Cloud Connector administration UI are named users and can be traced via the Cloud Connector audit log (see Use LDAP for User Administration [page 1977]).

Topic	Description	Recommended Action
	You can access the Cloud Connector administration UI remotely via HTTPS. After installation, it uses a self-signed **X.509 certificate** as TLS server certificate, which is not trusted by default by Web browsers.	Exchange the self-signed X.509 certificate of the Cloud Connector administration UI by a certificate that is trusted by your company and the company's approved Web browser settings (see Exchange UI Certificates in the Administration UI [page 1971]).
		For high-security scenarios, limit the access to the Cloud Connector administration UI to localhost (see also Recommendations for Secure Setup [page 1665]).
		Use a JVM that allows to limit the ciphers to a set considered safe (see Recommendations for Secure Setup [page 1665]). A list of cipher suites, which are currently considered safe, is available in Use Secure Cipher Suites .
Audit Logging Back to Topics [page 2065]	For end-to-end traceability of configuration changes in the Cloud Connector, as well as communication delivered by the Cloud Connector, **switch on audit logging** for productive scenarios.	Switch on audit logging in the Cloud Connector: set audit level to "All" (see Recommendations for Secure Setup [page 1665] and Manage Audit Logs [page 2042])
		Cloud Connector administrators must ensure that the audit log files are properly archived and are not lost, to conform to the local regulations.
		To gain end-to-end traceability, you should switch on audit logging also in the connected on-premises systems.
High Availability Back to Topics [page 2065]	To guarantee high availability of the connectivity for cloud integration scenarios, run productive instances of the Cloud Connector in high availability mode, that is, with a **second (redundant) Cloud Connector** in place.	Use the high availability feature of the Cloud Connector for productive scenarios (see Install a Failover Instance for High Availability [page 1996]).

Topic	Description	Recommended Action
Supported Protocols Back to Topics [page 2065]	**HTTP**, **HTTPS**, **RFC** and **RFC over SNC** are currently supported as protocols for the communication direction from the cloud to on-premises. The route from the application VM in the cloud to the Cloud Connector is always encrypted. You can configure the route from the Cloud Connector to the on-premises system to be encrypted or unencrypted.	The route from the Cloud Connector to the on-premises system should be encrypted using TLS (for HTTPS) or SNC (for RFC). Trust between the Cloud Connector and the connected on-premises systems should be established (see Set Up Trust [page 1702]).
Configuration of On-Premises Systems Back to Topics [page 2065]	When configuring the access to an internal system in the Cloud Connector, map physical host names to virtual host names to **prevent exposure of information** on physical systems to the cloud.	Use hostname mapping of exposed on-premises systems in the access control of the Cloud Connector (see Configure Access Control (HTTP) [page 1744] and Configure Access Control (RFC) [page 1752]).
	When configuring the access to an internal system, **restrict access** to those resources which are actually required by the cloud applications. Do not expose the complete system.	Narrow access to on-premises systems to resources required by the relevant cloud applications in the access control of the Cloud Connector (see Configure Access Control (HTTP) [page 1744] and Configure Access Control (RFC) [page 1752]).
	To allow **access only for trusted applications** of your SAP BTP subaccount to on-premises systems, configure the list of trusted applications in the Cloud Connector.	Narrow the list of cloud applications which are allowed to use the on-premises tunnel to the ones that need on-premises connectivity (see Set Up Trust [page 1702]).
Cloud Connector Instances Back to Topics [page 2065]	You can connect a single Cloud Connector instance to multiple SAP BTP subaccounts. Subaccounts can be created by SAP BTP users as a self service. Different subaccounts are often used to **separate development, test and production**. Do not mix productive Cloud Connector usages with development or test scenarios.	Use different Cloud Connector instances to separate productive and non-productive scenarios.

Related Information

Recommendations for Secure Setup [page 1665]
Secure the Activation of Traffic Traces [page 2004]

1.4.5 Upgrade

Upgrade your Cloud Connector and avoid connectivity downtime during the update.

The steps for upgrading your Cloud Connector are specific to the operating system that you use. Previous settings and configurations are automatically preserved.

> ⚠ Caution
>
> Upgrade is supported only for *installer* versions, not for *portable* versions, see Installation [page 1608]. Before upgrading, please check the Prerequisites [page 1610] and make sure your environment fits the new version. We recommend that you create a Configuration Backup [page 1964] before starting an upgrade.

Avoid Connectivity Downtime

If you have a single-machine Cloud Connector installation, a short downtime is unavoidable during the upgrade process. However, if you have set up a master and a shadow instance, you can perform the upgrade without downtime by executing the following procedure:

1. Shut down the shadow instance.
2. Perform the upgrade on the shadow instance. (Follow the relevant procedure below.)
3. Restart the shadow instance and wait until it has connected to the master instance.
4. Perform a *Switch Roles* operation by pressing the corresponding button in the master administration UI. The master instance has now changed into a shadow instance.

   > ⚠ Caution
   >
   > After upgrading the former shadow instance from a version prior to 2.13 and having switched its role to be the new master instance, reset high availability settings in *both* instances *now*, before continuing to upgrade the second instance from a version prior to 2.13 as well. The master-shadow connection must be re-established after both instances have been upgraded from versions prior to 2.13 to versions 2.13 or higher.

5. Shut down the new shadow instance and perform the upgrade procedure on it as well.
6. Restart the new shadow instance and wait until it has connected to the already upgraded current master instance.
7. Perform again the *Switch Roles* operation if you want the previous master instance to act as the new master instance again.

Result: Both instances have now been upgraded without connectivity downtime and without configuration loss.

For more information, see Install a Failover Instance for High Availability [page 1996].

Microsoft Windows OS

1. Uninstall the Cloud Connector as described in Uninstallation [page 2072] and make sure to retain the existing configuration.
2. Reinstall the Cloud Connector within the same directory. For more information, see Installation on Microsoft Windows OS [page 1657].
3. Before accessing the administration UI, clear your browser cache to avoid any unpredictable behavior due to the upgraded UI.

Linux OS

1. Execute the following command: :

   ```
   rpm -U com.sap.scc-ui-<version>.rpm
   ```

   > ⓘ Note
   >
   > **Daemon extensions (as of Cloud Connector version 2.12.3)**
   >
   > All extensions to the daemon provided via `scc_daemon_extension.sh` mechanism will survive a version update. An upgrade to version 2.12.3 will already consider an existing file, even though previous versions were not supporting that feature.

2. Before accessing the administration UI, clear your browser cache to avoid any unpredictable behavior due to the upgraded UI.

1.4.6 Update the Java VM

How to update the Java VM used by the Cloud Connector.

Sometimes you must update the Java VM used by the Cloud Connector, for example, because of expired TLS certificates contained in the JVM trust store, bug fixes, deprecated JVM versions, and so on.

- If you make a replacement in the same directory, shut down the Cloud Connector, upgrade the JVM, and restart the Cloud Connector when you are done.
- **If you change the installation directory** of the JVM, **follow the steps below** for your operating system.

Make sure that the JVM has been installed successfully.

> **Note**
>
> A Java Runtime Environment (JRE) is not sufficient. You must use a JDK or SAP JVM.

Microsoft Windows OS

1. Make sure that the current user has administrative privileges.
2. Shutdown the Cloud Connector (for example, by stopping the corresponding Windows service or by double-clicking the *Stop SAP Cloud Connector* shortcut).
3. Open the *registry editor* (regedit32) and locate the following registry entry: `HKEY_LOCAL_MACHINE\SOFTWARE\SAP\Cloud Connector`.
4. Change the value `JavaHome` to reflect the installation directory of the new SAP JVM or JDK.

   > **Note**
   >
   > The *bin* subdirectory must not be part of the `JavaHome` value.

   If the `JavaHome` value does not yet exist, create it here with a "String Value" (REG_SZ) and specify the full path of the Java installation directory, for example: `C:\Program Files\sapjvm`.
5. Close the registry editor and restart the Cloud Connector.

Linux OS

1. Open a shell command line prompt as root user.
2. In this shell, set the environment variable `JAVA_HOME`, pointing to the installation directory of the new SAP JVM or JDK, for example:
   - in csh/tcsh:

     ```
     setenv JAVA_HOME /opt/sap/sapjvm_8
     ```

   - in sh/bash/dash/zsh:

     ```
     export JAVA_HOME=/opt/sap/sapjvm_8
     ```

3. Execute the command

   ```
   System V init distributions: service scc_daemon stop
   systemd distributions: systemctl stop scc_daemon
   ```

4. Execute the command

   ```
   System V init distributions: /opt/sap/scc/daemon.sh reinstall
   systemd distributions: /opt/sap/scc/daemon.sh reinstallSystemd
   ```

5. Execute the command

   ```
   System V init distributions: service scc_daemon start
   ```

```
systemd distributions: systemctl start scc_daemon
```

Both Operating Systems

> **Note**
>
> Make sure the selected cipher suites are accepted by the JVM you are about to install. If in doubt, revert to the default selection prior to changing the JVM.

> **Note**
>
> If you use your own CA certificates for the e-mail configuration (see Alerting [page 2038]) or for LDAP (see Use LDAP for User Administration [page 1977]), and you imported these custom CA certificates into your JVM trust store, you must reimport them after a JVM update into the JVM trust store as described there.

> **→ Tip**
>
> If own CA certificates for e-mail alerting or LDAP user authorization setup are required, but these certificates are not necessary for other Java processes using the very same JVM installation, consider storing these custom CA certificates in the respective trust stores of the Cloud Connector instead of importing them into the JVM trust store. In this case, you do not need to reimport them into the JVM trust store after a JVM update, and you will benefit from these certificates being stored in Cloud Connector configuration backups.

After executing the above steps, the Cloud Connector should be running again and should have picked up the new Java version during startup. You can verify this by logging in to the Cloud Connector with your favorite browser, opening the *About* dialogue and checking that the field `<Java Details>` shows the version number and build date of the new Java VM. After you verified that the new JVM is indeed used by the Cloud Connector, delete or uninstall the old JVM.

1.4.7 Uninstallation

Uninstall an installer version or portable version of the Cloud Connector.

- If you have installed an installer variant of the Cloud Connector, follow the steps for your operating system to uninstall the Cloud Connector.
- To uninstall a developer version, proceed as described in section **Portable Variants**.

Microsoft Windows OS

1. In the Windows software administration tool, search for `Cloud Connector` (formerly named `SAP HANA cloud connector 2.x`).
2. Select the entry and follow the appropriate steps to uninstall it.
3. When you are uninstalling in the context of an upgrade, make sure to retain the configuration files.

Linux OS

To uninstall Cloud Connector 2.x, execute the following command:

```
rpm -e com.sap.scc-ui
```

> ⚠ Caution
>
> This command also removes the configuration files.

Mac OS X

There is no installer variant for Mac OS X, only a `portable` one.

Portable Variants

(Microsoft Windows OS, Linux OS, Mac OS X) If you have installed a portable version (zip or tgz archive) of the Cloud Connector, simply remove the directory in which you have extracted the Cloud Connector archive.

Related Information

Installation [page 1608]

1.4.8 Frequently Asked Questions

Answers to the most common questions about the Cloud Connector.

Technical Issues

Does the Cloud Connector send data from on-premises systems to SAP BTP or the other way around?

The connection is opened from the on-premises system to the cloud, but is then typically used in the other direction.

An on-premises system is, in contrast to a cloud system, normally located behind a restrictive firewall and its services aren't accessible thru the Internet. This concept follows a widely used pattern often referred to as *reverse invoke proxy*.

Is the connection between the SAP BTP and the Cloud Connector encrypted?

Yes, by default, TLS encryption is used for the tunnel between SAP BTP and the Cloud Connector.

If used properly, TLS is a highly secure protocol. It is the industry standard for encrypted communication and also, for example, as a secure channel in HTTPS.

Keep your Cloud Connector installation and JDK updated to avoid the use of weak and deprecated ciphers for TLS communication. Which cipher and TLS versions are actually used, is defined by both the cloud region setup and the JDK that is used for Cloud Connector. The TLS implementation used for the communication is the one of the JDK.

Can I use a TLS-terminating firewall between Cloud Connector and SAP BTP?

This is not possible. Basically, this is a desired man-in-the-middle attack, which does not allow the Cloud Connector to establish a mutual trust to the SAP BTP side.

Can I copy/clone a Cloud Connector installation and use it in parallel on a different machine?

This is not supported. You would face issues regularly, as those two instances will be considered as one, which is not expected by the cloud side.

If you just want to move the installation to a new machine, create a backup via the Configuration Backup [page 1964] feature, create a new installation, import the backup, and discard the previous installation.

What is the oldest version of SAP Business Suite that's compatible with the Cloud Connector?

The Cloud Connector can connect an SAP Business Suite system version 4.6C and newer.

Which Java versions are supported to run the Cloud Connector?

	Supported Java Version

		8	11	17	21	25
Cloud Connector Version	2.7.2 – 2.13.2	Yes	No	No	No	No
	2.14.0 – 2.14.2	Yes	Yes	No	No	No
	2.15.0 – 2.16.2	Yes	Yes	Yes	No	No
	2.17.0 – 2.18.2	Yes	Yes	Yes	Yes	No
	≥ 2.19.0	Yes	No	Yes	Yes	Yes

> ⚠ Restriction
>
> Support for Java 11 has been discontinued. For more information, see Prerequisites [page 1611].

> → Tip
>
> We recommend that you always use the latest patch level of the respective Java version.

Which configuration in the SAP BTP destinations do I need to handle the user management access to the Cloud User Store of the Cloud Connector?

See Configure an On-Premises User Store.

Is the Cloud Connector sufficient to connect the SAP BTP to an SAP ABAP back end or is SAP BTP Integration needed?

It depends on the scenario: For pure point-to-point connectivity to call on-premises functionality like BAPIs, RFCs, OData services, and so on, that are exposed via on-premises systems, the Cloud Connector might suffice.

However, if you require advanced functionality, for example, n-to-n connectivity as an integration hub, SAP BTP Cloud Integration – Process Integration (CI-PI) is a more suitable solution. SAP BTP CI can use the Cloud Connector as a communication channel.

How can I check whether the Cloud Connector is prepared for IPv6 support?

Check the *Prerequisites* section of the *Installation* chapter for your platform. As of version 2.18.1, the Cloud Connector shows on its *About* screen the IPv6 state that it sees in the respective environment.

How much bandwidth does the Cloud Connector consume?

The amount of bandwidth depends greatly on the application that is using the Cloud Connector tunnel. If the tunnel isn't currently used, but still connected, a few bytes per minute is consumed for the WebSocket ping/pong mechanism to keep the connection alive.

What happens to a response if there's a connection failure while a request is being processed?

The response is lost. The Cloud Connector only provides tunneling, it does not store and forward data when there are network issues.

Where should I install the Cloud Connector?

For productive instances, we recommend installing the Cloud Connector on a single purpose machine. This is relevant for Security [page 2057]. For more details on which network zones to choose for the Cloud Connector setup, see Network Zones [page 1648].

How does a disaster recovery setup look like for the Cloud Connector?

There is no explicit implementation of a disaster recovery setup for the Cloud Connector. However, it is actually not needed.

Instead, make sure you have machines available in some other data center than the one in which your productive setup is running. Also, make sure you regularly generate a Configuration Backup [page 1964].

If a disaster situation occurs, install the Cloud Connector again and restore the latest backup. Immediately after the restart that is required after restoring the backup, you are back to a running setup, as long as all the back-end systems are reachable from the new location.

How many servers do I need to deploy the Cloud Connector?

Actually, you need only a single instance, but we recommend that you use at least three servers, with the following purposes:

- Development
- Production master
- Production shadow

> **Note**
>
> Do not run the production master and the production shadow as VMs inside the same physical machine. Doing so removes the redundancy, which is needed to guarantee high availability. A QA (Quality Assurance) instance is a useful extension of a Cloud Connector landscape. For disaster recovery, you will need two additional instances: another master instance, and another shadow instance as a reserve for the disaster case.

What is the recommended upgrade procedure for a Cloud Connector landscape?

Assuming you run the tiers development, QA, and production, an upgrade in exactly this order is the recommended approach. Like this, you will recognize potential issues in less critical environments first. In exceptional cases, it might be needed to apply a newer version in production before applying it to development instances (for example, for important (hot)fixes), but this should not be the standard approach.

What are the hardware requirements to deploy the Cloud Connector?

See: Prerequisites [page 1610].

Is NTLM supported for authorization against the proxy server?

No, the Cloud Connector currently supports only basic authentication.

Which operating systems are supported by the Cloud Connector?

We currently support Windows and Linux versions with an installer for productive scenarios, in case of Linux for multiple processor architectures. Additionally, the portable variant of the Cloud Connector is available not only for those operating systems, but also for macOS.

You can find the full product availability matrix in Prerequisites [page 1610].

Which processor architectures are supported by the Cloud Connector?

We currently support 64-bit operating systems running only on an x86-64 processor (also known as x64, x86_64 or AMD64), and for Linux also on the *PowerPC Little Endian* variant (also known as *ppc64le*) as well as the ARM64 processor (also known as *aarch64*).

For macOS, also the ARM64 processor (also known as *aarch64*).

See Prerequisites [page 1610].

Can I use the Cloud Connector without an ABAP back end?

Yes, you should be able to connect almost any system that supports the HTTP Protocol, from SAP BTP, for example, Apache HTTP Server, Apache Tomcat, Microsoft IIS, or Nginx.

Can I authenticate with client certificates configured in SAP BTP destinations at HTTP services that are exposed via the Cloud Connector?

No, this is not possible. For client certificate authentication, an end-2-end TLS communication is required. This is not the case, because the Cloud Connector needs to inspect incoming requests in order to perform access control checks.

How can I do connection pooling for HTTP services that are exposed via the Cloud Connector?

The Cloud Connector itself does not perform connection pooling, but provides a 1-to-1 mapping for each logical connection received trough the tunnel.

By this mapping, a new connection to the back-end system is opened, and kept open until closed either by the back end or by the client on cloud side.

The actual connection pooling is defined by the application client on cloud side:

- If a connection is re-used in the client library, it is re-used on the Cloud Connector side as well.
- If it is closed immediately, also the mapped one on Cloud Connector side will be closed immediately.

In that context, it is important that you must use different physical connections for different access control entries on the Cloud Connector side.

Can I open two windows or tabs in a single browser instance to administrate the Cloud Connector?

No, this is not supported and may cause odd behavior on the different screens, in particular when trying to navigate through multiple subaccounts. If you like to open the administration UI twice, use two separate browser instances.

Does the Cloud Connector delete or modify HTTP headers?

Modifications of HTTP response headers are done if needed, in particular, for Set-Cookie domains are adjusted according to the configured domain and host mappings. Also, in case of redirects, the location header will be adjusted according to the host mappings. Modifications of HTTP request headers are also done if needed, which is currently only the case for the `Host` header content will be replaced by the internal host, if the host mapping configuration is setup accordingly.

The Cloud Connector does not delete any header that is sent by the cloud application. However, the Connectivity service will drop Connectivity service-specific headers, such as `SAP-Connectivity-Authentication` or `SAP-Connectivity-ConsumerAccount` so that those headers will neither reach the Cloud Connector nor the eventual back end.

Last but not least, the Cloud Connector will inject the HTTP header `SSL_CLIENT_CERT` containing a short-lived certificate representing a user for identity propagation purposes. If the current request coming in from the cloud does not contain a token representing an identity, and the access control entry is configured to not use the system certificate for logon purposes, the Cloud Connector injects an empty `SSL_CLIENT_CERT` header.

Is it possible to use the Cloud Connector as an HTTP proxy for outgoing traffic?

No, this is not possible. There are many HTTP proxy solutions out there that you can use in your network infrastructure.

Administration

Are there Audit Logs for changes in the Cloud Connector?

Yes, find more details here: Manage Audit Logs [page 2042].

Is it possible to split authorization?

When using the file based user store, there is only one role that allows complete administration of the Cloud Connector.

If you need multiple roles, setup your Cloud Connector to use LDAP for authentication as described in Configure Named Cloud Connector Users [page 1976].

Can I configure multiple administrative subaccounts?

Yes, to enable this, you must configure an LDAP server. See: Use LDAP for User Administration [page 1977].

How can I reset the Cloud Connector's administrator password when using the file-based user store for authentication?

Visit https://tools.hana.ondemand.com/#cloud to download the portable version of the Cloud Connector. Extract the *users.xml* file in the *config* directory to the *config* directory of your Cloud Connector installation, then restart the Cloud Connector.

This resets the password and user name to their default values.

You can manually edit the file; however, we strongly recommend that you use the *users.xml* file.

How do I create a backup of the Cloud Connector configuration?

Starting with Cloud Connector version 2.11, you can use a dedicated backup feature, either from the administration UI (see Configuration Backup [page 1964]) or via REST API (see Backup [page 1844]).

Can I create a backup of the complete installation?

Yes, but we don't recommend to do it. Instead, you should use the backup feature as described in the previous question. Only this type of backup will be supported. Restoring a file system-based backup would happen on your own risk.

You can create an archive file of the installation directory to create a full backup. Before you restore from a backup, note the following:

- If you restore the backup on a different host, the UI certificate will be invalidated.
- Before you restore the backup, you should perform a “normal” installation and then replace the files. This registers the Cloud Connector at your operating systems package manager.

Why do I need a user ID during configuration?

This user opens the tunnel and generates the certificates that are used for mutual trust later on.

The user is not part of the certificate that identifies the Cloud Connector.

In both the Cloud Connector UI and in the SAP BTP cockpit, this user ID appears as the one who performed the initial configuration (even though the user may have left the company).

What happens to a Cloud Connector connection if the user who created the tunnel leaves the company?

This does not affect the tunnel, even if you restart the Cloud Connector.

What do changes in major or minor version numbers mean?

The semantics of Cloud Connector versions are explained in detail here.

Does SAP provide support for older Cloud Connector versions?

SAP supports the latest 2 feature releases in parallel. For the latest feature release, the last 2 patch levels are supported. For the previous feature release, only its latest patch level is supported.

For more information on the Cloud Connector support strategy, see SAP Note 3302250.

What is the difference between “subaccount name” and “subaccount user”?

SAP BTP customers can purchase subaccounts and deploy applications into these subaccounts.

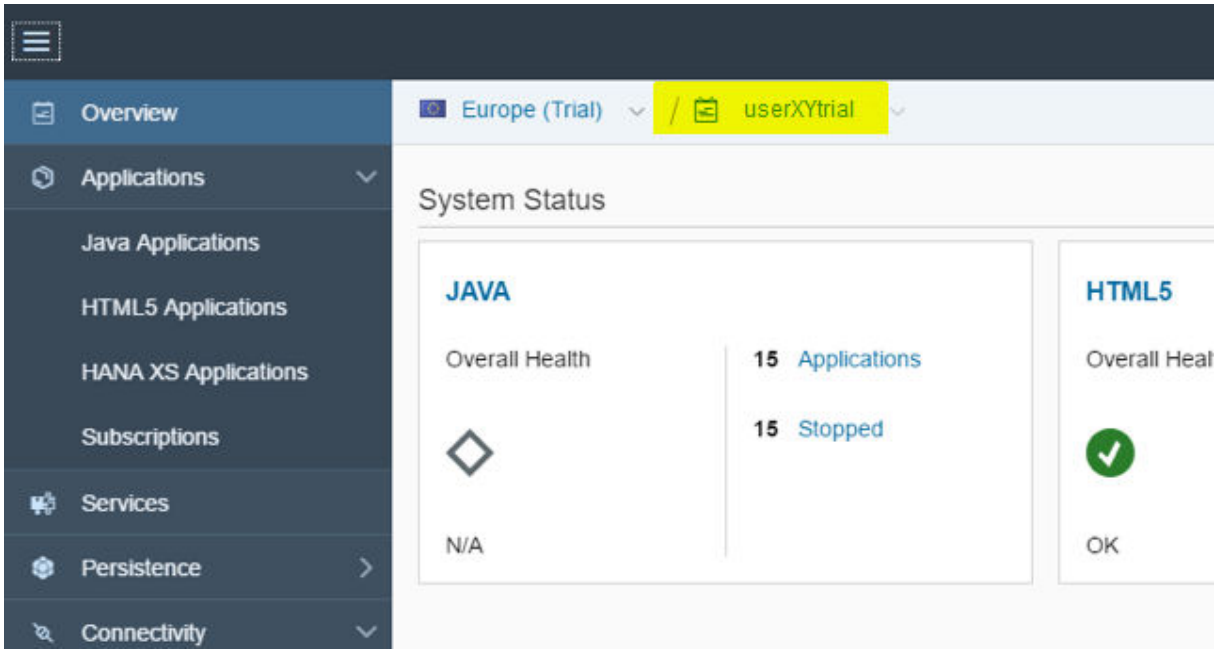
Additionally, there are users, who have a password and can log in to the cockpit and manage all subaccounts they have permission for.

- A single subaccount can be managed by multiple users, for example, your company may have several administrators.
- A single user can manage multiple subaccounts, for example, if you have multiple applications and want them (for isolation reasons) to be split over multiple subaccounts.

Find your account name by taking the following steps:

1. Open the SAP BTP cockpit.
2. Log in with your subaccount user.
3. You’ll see the subaccount name in the top left section of the screen.

For trial users, the account name is typically your user name, followed by the suffix “trial”:



Features

Does the Cloud Connector work with the SAP BTP multi-cloud environment?

Yes, the Cloud Connector can establish a connection to regions based on the SAP BTP multi-cloud environment.

Does the Cloud Connector work with SAP S/4HANA Cloud?

The Cloud Connector offers a Service Channel to S/4HANA Cloud instances, given that they are associated with the respective SAP BTP subaccount. For more information, see Using Service Channels [page 1949].

The Cloud Connector also supports S/4HANA Cloud communication scenarios invoking HTTP services or remote-enabled function modules (RFMs) in on-premises ABAP systems.

Does the Cloud Connector work with the SAP BTP ABAP environment?

The Cloud Connector supports communication from and to the SAP BTP ABAP environment using the **Cloud Foundry** Connectivity service, which requires a Cloud Connector version 2.12.3 or higher.

How do I bind multiple Cloud Connectors to one SAP BTP subaccount?

You can connect multiple Cloud Connectors to a single subaccount. This lets you assign multiple separate corporate network segments.

Those Cloud Connectors are distinguishable based on the location ID, which you must provide to the destination configuration on the cloud side.

Is WebSocket communication through the Cloud Connector supported?

Yes, this is possible.

Is there any plan to add traffic management functionality in Cloud Connector?

No, this functionality is not currently planned.

Can I use the Cloud Connector from cloud to on-premises for any protocol?

You can use the TCP channel of the Cloud Connector, if the client supports a SOCKS5 proxy to establish the connection and the protocol is TCP-based. However, only the HTTP and RFC protocols currently provide an additional level of access control by checking invoked resources.

Can I use the Cloud Connector from on-premises to cloud for any protocol?

This is possible only for a limited set of protocols, using Cloud Connector *service channels*:

Neo environment:

- You can use the Cloud Connector as a JDBC or ODBC proxy to access the HANA DB instance within your SAP BTP Neo subaccount (service channel). This is sometimes referred to as “HANA protocol”.
- Also, there are service channels for SSH access to SAP BTP Neo virtual machines.

Neo and **multi-cloud** environment:

- RFC access to ABAP cloud systems.

Multi-cloud environment:

- Access services in a Kubernetes deployment that hosts a *Connectivity Proxy*, in particular, also in Kyma clusters.

All of these service channels provide access to endpoints that are not visible in the Internet.

See also:

Using Service Channels [page 1949] (**multi-cloud** environment)

Using Service Channels (**Neo** environment)

For HTTP (**Neo** and **multi-cloud**), the endpoints that can be addressed are visible in the Internet. Therefore, you can simply use your normal network infrastructure that is prepared for accessing HTTPS endpoints in the Internet anyway.

Can I check the communication of the service channel?

No, the audit log monitors access only from SAP BTP to on-premises systems.

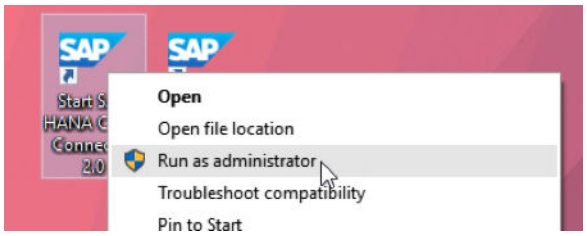
Troubleshooting

How do I fix the “Could not open Service Manager” error message?

You are probably seeing this error message due to missing administrator privileges. Right-click the cloud connector shortcut and select Run as administrator.

If you don’t have administrator privileges on your machine you can use the portable variant of the Cloud Connector.

> **Note**
>
> The portable variants of the Cloud Connector are meant for nonproductive scenarios only.



How do I set JAVA_HOME and PATH correctly?

`JAVA_HOME` must point to the installation directory of your SAP JVM or JDK while `PATH` must contain the *bin* folder inside the installation directory of your SAP JVM or JDK. This is relevant in particular for the portable versions. The installers will also detect JDKs in other places.

How do I use the various .bat-Batch or .sh-Shell script tools in the Cloud Connector directory?

Open a command line prompt with administrator privileges or with sufficient user privileges to read and write files in the Cloud Connector directory. Then, make sure the environment variable `JAVA_HOME` is set to the installation directory of the JDK used by the Cloud Connector.

Afterwards, switch to the Cloud Connector directory and call the appropriate batch or shell script tools via `<toolname>.bat` or `./<toolname>.sh`. If the respective tool script requires further input parameters, its usage syntax will be written to the console.

When I try to open the Cloud Connector UI, Google Chrome opens a *Save as* dialog, Firefox displays some cryptic signs, and Internet Explorer shows a blank page, how do I fix this?

This happens when you try to access the Cloud Connector over HTTP instead of HTTPS. HTTP is the default protocol for most browsers.

Adding “https://” to the beginning of your URL should fix the problem. For localhost, you can use `https://localhost:8443/`.

1.4.9 REST APIs

Find general information on the Cloud Connector REST APIs.

The Cloud Connector provides REST APIs to support automation of configuration and monitoring tasks. REST APIs are exposed on the same host and port that you use to access to the Cloud Connector.

Request and Response Payload Format [page 2084]

Security and Session [page 2085]

User Roles [page 2086]

Return Codes [page 2086]

Hypertext Application Language [page 2087]

Available REST APIs [page 2087]

Sample Code [page 2087]

Request and Response Payload Format

The payload (that is, the data transmitted in the body) of requests and responses is mostly coded in JSON format. The following example shows the request payload *{description:<value>}* coded in JSON:

```
{"description": "very helpful description"}
```

Values that represent a date are provided as a UTC long number, which is the number of milliseconds since 1 January 1970 00:00:00 UT (GMT+00).

In case of errors, the HTTP status code is *4xx* or *500*. Error details are supplied in the response body in JSON format:

```
{type: <type>, message: <message>}
```

where *type* can be one of the following strings:

```
INVALID_REQUEST, ILLEGAL_STATE, NOT_FOUND, INVALID_CONFIGURATION,
RUNTIME_FAILURE, SHADOW_UPDATE, FORBIDDEN_REQUEST
```

The request JSON is listed in the API descriptions in an abbreviated form, showing only the property names and omitting the values. Details on the values (data types and restrictions) are provided separately below the respective API descriptions.

The API documentation also includes possible error types, and details on those errors below the API description. These errors pertain to property values of the payload only. We do not claim this list of errors to be exhaustive. In particular, errors caused by obviously erroneous input, such as missing mandatory or malformed property values, may have been omitted. As a rule of thumb, missing or invalid values result in *INVALID_REQUEST*. Errors caused elsewhere (for example, inappropriate header field values) are not listed.

> **Note**
>
> Request bodies in JSON format require the header field *Content-Type application/json*. The API descriptions do not explicitly mention this fact. Header fields are shown only if they deviate from the default *Content-Type: application/json*.

Back to Top [page 2084]

Security and Session

The Cloud Connector supports the authentication types *basic authentication* and *form-based authentication*. Once authenticated, a client can keep the session and execute subsequent requests in the session. A session avoids the overhead caused by authentication. As usual, the session ID can be obtained from the response header field *Set-cookie* (as *JSESSIONID=<session Id>*), and must be sent in the request header *Cookie: JSESSIONID=<session Id>*.

The Cloud Connector employs CSRF tokens to counter CSRF (cross-site request forgery) attacks. Upon first request, a CSRF token is generated and sent back in the response header in field *X-CSRF-Token*. The client application must retain this token and send it in all subsequent requests as header field *X-CSRF-Token* together with the session cookie as explained above.

> **Note**
>
> No CSRF token is generated if request header field *Connection* has the value *close* (as opposed to *keep-alive*). In other words, if you want to make stateful, session-based REST calls, use *Connection: keep-alive*, extract the CSRF token from the first response header and subsequently include it in all request headers. Otherwise, use *Connection: close* and always submit user and password through *basic authentication*.

An inactive session will incur a timeout at some point, and will consequently be removed. A request using an expired session will receive a login page (*Content-type: text/html*). The status code of the response is 200 in this case. Hence, the only way to detect an expired session is to pay attention to the content type and status code. Content type *text/html* in a connection with status code 200 indicates an expired session.

For security reasons, a session should be closed or invalidated once it is not needed anymore. You can achieve this by including *Connection: close* in the header of the final call involving the session in question. As a result, the Cloud Connector invalidates the session. Subsequent attempts to send a request in the context of that session respond with a login page as explained above.

Back to Top [page 2084]

User Roles

The REST API supports different user roles. Depending on the role, an API grants or denies access. In default configuration, the Cloud Connector uses local user storage and supports the single user *Administrator* (administrator role). Using LDAP user storage, you can use various users:

Role	Technical Role Name	Authorization
Administrator	admin or sccadmin	All operations.
Associate Administrator (AssocAdministrator)	sccassocadmin	• All subaccount administration operations unless a subaccount is managed • Administrate certain cross-subaccount settings including, but not limited to, certificates and backend trust • Edit log and trace configuration
Subaccount Administrator (SubAdministrator)	sccsubadmin	• All subaccount administration operations unless a subaccount is managed • Edit log and trace configuration
Support	sccsupport	Edit log and trace configuration.
Display	sccdisplay	Read configuration.
Monitoring	sccmonitoring	Read monitoring information.

Back to Top [page 2084]

Return Codes

Successful requests return the code 200, or, if there is no content, 204. POST actions that create new entities return 201, with the location link in the header (that is, the value of the header field *location* is the full URI of the entity created).

The following general errors can be returned by each API:

- 400 – Invalid request. For example, if parameters are invalid or the API is not supported anymore, or an unexpected state occurs, as well as in case of other non-critical errors.
- 401 – Authentication is required and has failed or no credentials have been provided.
- 403 – Request is not allowed. Either the user does not have permission to invoke the API or the initial password has not been changed yet.

- 404 – The specified entity does not exist.
- 405 – An entity does not support the requested operation.
- 409 – Current state of the Cloud Connector does not allow a particular request as it conflicts with certain rules or violates certain constraints.
- 415 – Unexpected MIME type.
- 500 – Operation failed.

Most APIs return specific error details depending on the situation. Such errors are addressed by the respective API description.

> → Remember
>
> The Cloud Connector forbids simultaneous changes on a subaccount from different clients. A subaccount used by the Cloud Connector UI is blocked for changes made from other instances. Even just *selecting* a subaccount in the Cloud Connector dashboard (displaying the list of all subaccounts) blocks this subaccount. If you have selected more than one subaccount, the first one in the list is blocked.
>
> Operations on blocked resources raise an error. In this case, REST APIs return the code *409*.

Back to Top [page 2084]

Hypertext Application Language

Entities returned by the APIs contain links as suggested by the current draft *JSON Hypertext Application Language* (see https://tools.ietf.org/html/draft-kelly-json-hal-08).

Back to Top [page 2084]

Available REST APIs

The Cloud Connector provides APIs for:

Monitoring tasks [page 2016]

Configuration tasks [page 1775]

Back to Top [page 2084]

Sample Code

Some APIs include examples with command line invocations typically involving curl. The use of single and double quotes in those examples depends on the operating system and may have to be adapted.

Back to Top [page 2084]

1.5 Connectivity Proxy for Kubernetes

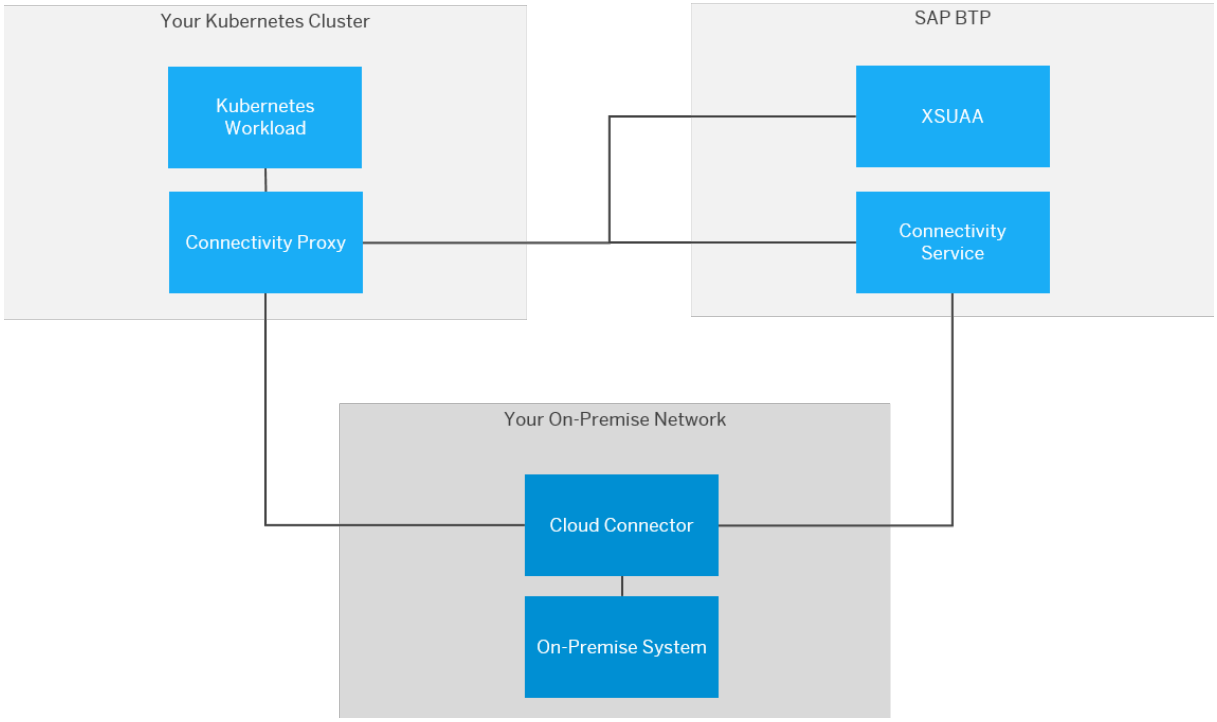
Use the Connectivity Proxy for Kubernetes to connect workloads on a Kubernetes cluster to on-premise systems.

> **Note**
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

The Connectivity Proxy is a Kubernetes component that connects workloads running on a Kubernetes cluster to on-premise systems, which are exposed via the Cloud Connector [page 1602]. The Connectivity Proxy must be paired to an SAP BTP region to grant access to the Cloud Connectors connected to that region. The SAP BTP domain model (subaccounts) is used to target a particular Cloud Connector.

The Connectivity Proxy is delivered as a Docker image and a Helm chart. You need to run the image on your Kubernetes cluster with appropriate configurations. The Helm chart simplifies the installation process. See Lifecycle Management [page 2120] for more details.

You can find information about new versions of the Connectivity Proxy via SAP BTP release notes. To request a new feature, you can use Influence SAP.



Related Information

Concepts [page 2089]
Lifecycle Management [page 2120]
Verification and Testing [page 2143]
Monitoring [page 2145]
Using the Connectivity Proxy [page 2148]
Troubleshooting [page 2152]
Frequently Asked Questions [page 2159]
Connectivity Proxy in the Kyma Environment [page 2160]

1.5.1 Concepts

Find an overview of important concepts for working with the Connectivity Proxy for Kubernetes.

How the Connectivity Proxy Works [page 2090]	Learn about the Connectivity Proxy for Kubernetes: Scenario and conguration steps.
Operational Modes [page 2093]	Details about the different operational modes of the Connectivity Proxy.
Mutual TLS [page 2097]	Use Transport Layer Security (TLS) with the Connectivity Proxy.
External Health Checking [page 2101]	Perform external health checks for the Connectivity Proxy.
High Availability [page 2105]	Run the Connectivity Proxy for Kubernetes in high availability mode.
Audit Logging [page 2107]	Using audit logging for the Connectivity Proxy.
Integration with SAP Services [page 2108]	Integrate the Connectivity Proxy with SAP services.
Automatic Pickup on Resource Changes [page 2111]	Configure automatic pickup on resource changes for the Connectivity Proxy.
Service Channels: On-Premise-to-Cloud Connectivity [page 2112]	Use the Connectivity Proxy to connect to an on-premise network via service channel.
Installing the Connectivity Proxy in Clusters with Istio [page 2114]	Install the Connectivity Proxy in a Kubernetes cluster that is configured for Istio.
Installing the Transparent Proxy as Subchart of the Connectivity Proxy [page 2116]	Install the Transparent Proxy as subchart of the Connectivity Proxy via Helm.
Installing the Connectivity Proxy in Multi-Region Mode [page 2116]	Use a single Connectivity Proxy installation for your global account.

1.5.1.1 How the Connectivity Proxy Works

Learn about the Connectivity Proxy for Kubernetes: Scenario and required configuration.

Glossary [page 2090]

Scenario [page 2090]

- Prerequisites [page 2091]
- Runtime Flow in Steps [page 2091]
- Involved Parties [page 2092]

Configuration [page 2092]

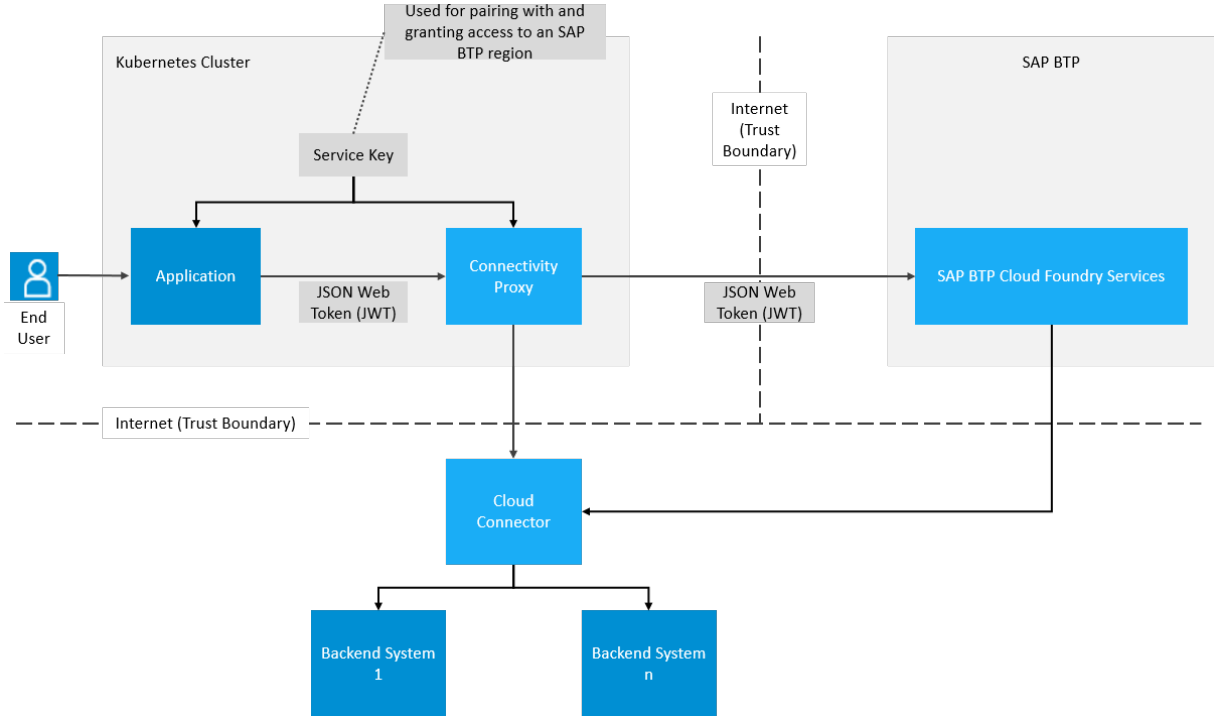
Glossary

- **SAP BTP**: SAP platform for cloud services (replaces *SAP Cloud Platform*, *SAP Business Technology Platform*).
- **SAP Connectivity service**: Core platform service, offering a secure tunnelling solution between your on-premise network and the cloud.
- **Cloud Connector**: On-premise client of the Connectivity service, deployed and lifecycle-managed in your local network. The initiator of the secure tunnel to the platform, that is, to the Connectivity service.
- **Connectivity proxy**: Software component (logically part of the Connectivity service), deployed locally to the consuming part (usually a cloud application or a service component). It can work in multiple operational modes, depending on the exact requirement of the consuming party.
- **SAP UAA** (aka XSUAA): SAP Authorization service, issues client credentials and access tokens, associated with the platform tenancy model.

Back to Top [page 2090]

Scenario

An end user works with a cloud application or solution. To complete the task, the application or solution needs to connect to an on-premise system (hosted either by the consumer tenant or the cloud application provider tenant). The system is not accessible directly via Internet, but securely exposed by the Cloud Connector. Only selected parts of the system functionality may be exposed to the cloud application. For more information, see Cloud Connector [page 1602].



Prerequisites

- The Connectivity Proxy is deployed and configured in the Kubernetes cluster (see Lifecycle Management [page 2120]).
- A cloud application is deployed on Kubernetes, next to the Connectivity Proxy, and it is configured to connect to the proxy (see Using the Connectivity Proxy [page 2148]). The cloud application is up and running and accessible by end users.
- The Cloud Connector is installed and configured in your local network and connected to the cloud (that is, to the Connectivity service), and stays in a ready-to-be-used mode (see Cloud Connector [page 1602]).
- The on-premise systems that the cloud application needs to connect to are properly exposed via / configured in the Cloud Connector (see Configure Access Control [page 1740]).

Back to Scenario [page 2090]

Back to Top [page 2090]

Runtime Flow in Steps

When all prerequisites are met, the cloud application can be properly used by end users:

- An end user works with a *client tool*, for example, a browser or a REST client.
- The *client tool* connects to a *cloud application*, in this case hosted in a Kubernetes cluster.
- The *cloud application* knows it needs to connect to the on-premise system, therefore it connects to the Connectivity Proxy. Depending on the specific requirements of the cloud application environment, the *cloud application* might need to obtain an access token which is used to authenticate the application as a client, as well as to authorize it to connect to the respective tenant-specific Cloud Connector exposing the target system (see Operational Modes [page 2093]).

- The *Connectivity Proxy* accepts the client proxy request (by the *cloud application*) and routes the traffic via a TLS secure tunnel, which has been already initiated by the Cloud Connector and successfully established.
- The Cloud Connector receives the *request data* from the *cloud application* and performs the related access control checks. A connection is established to the target on-premise system, and the request data is forwarded to the on-premise system.
- The *response data* from the on-premise system is routed back to the *cloud application*.
- The *cloud application* processes the response data retrieved by the on-premise system and shows the result to the end user.

Back to Scenario [page 2090]

Back to Top [page 2090]

Involved Parties

- **Cloud application**: Business workload initiated by end users (or a background job) of the business solution.
- **SAP BTP Services**: The Connectivity Proxy cannot operate on its own. It needs to connect to other services for key operations, namely:
  - *authorization - XSUAA*: Ensure any operation is properly secured.
  - *pairing/integration with SAP Connectivity service*: Secure access control to Cloud Connectors.
- **On-Premise systems or services**: The target system, securely hosted in your local network, usually behind a firewall, and exposed via the Cloud Connector.

.

Back to Scenario [page 2090]

Back to Top [page 2090]

Configuration

The Connectivity Proxy for Kubernetes is meant to

- Be deployed as a Kubernetes StatefulSet . As it is accessed both by other Kubernetes deployments and the Cloud Connector, you need to configure access accordingly.
- Connect to other (remote) services.
- Enforce authorization on its proxy endpoints.

For all these points, proper configuration is required. For more information, see Lifecycle Management [page 2120].

Back to Top [page 2090]

1.5.1.2 Operational Modes

Learn about the different operational modes of the Connectivity Proxy for Kubernetes.

The Connectivity Proxy can run in four different operational modes, based on two main categories:

- Trust for the surrounding environment and the callers of the proxy
- Tenant usage of the proxy

Find below the details for each operational mode:

Single-Tenant Usage: *Non-Trusted* Environment [page 2093]

Multi-Tenant Usage: *Non-Trusted* Environment [page 2094]

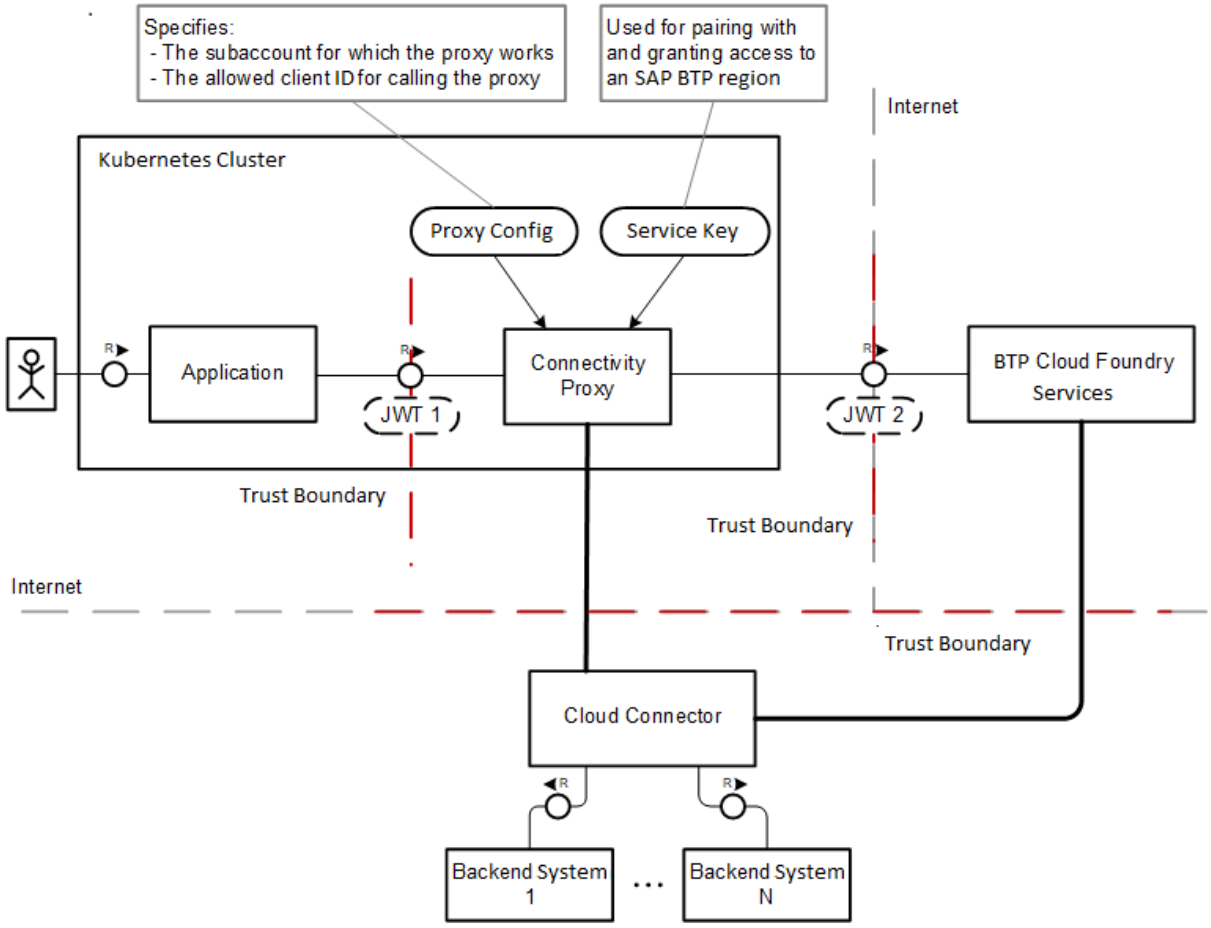
Single-Tenant Usage: *Trusted* Environment [page 2095]

Multi-Tenant Usage: *Trusted* Environment [page 2096]

Single-Tenant Usage: *Non-Trusted* Environment

The Connectivity Proxy operates on behalf of a single, statically configured tenant. Applications cannot be trusted. The Connectivity Proxy is configured as follows:

- Proxy authorization is *enabled*.
- Tenant mode is *dedicated*.
- Uses a statically configured service key of the `connectivity_proxy` service instance of the `connectivity` service.
- Connects to the central Connectivity service on behalf of the statically configured tenant.

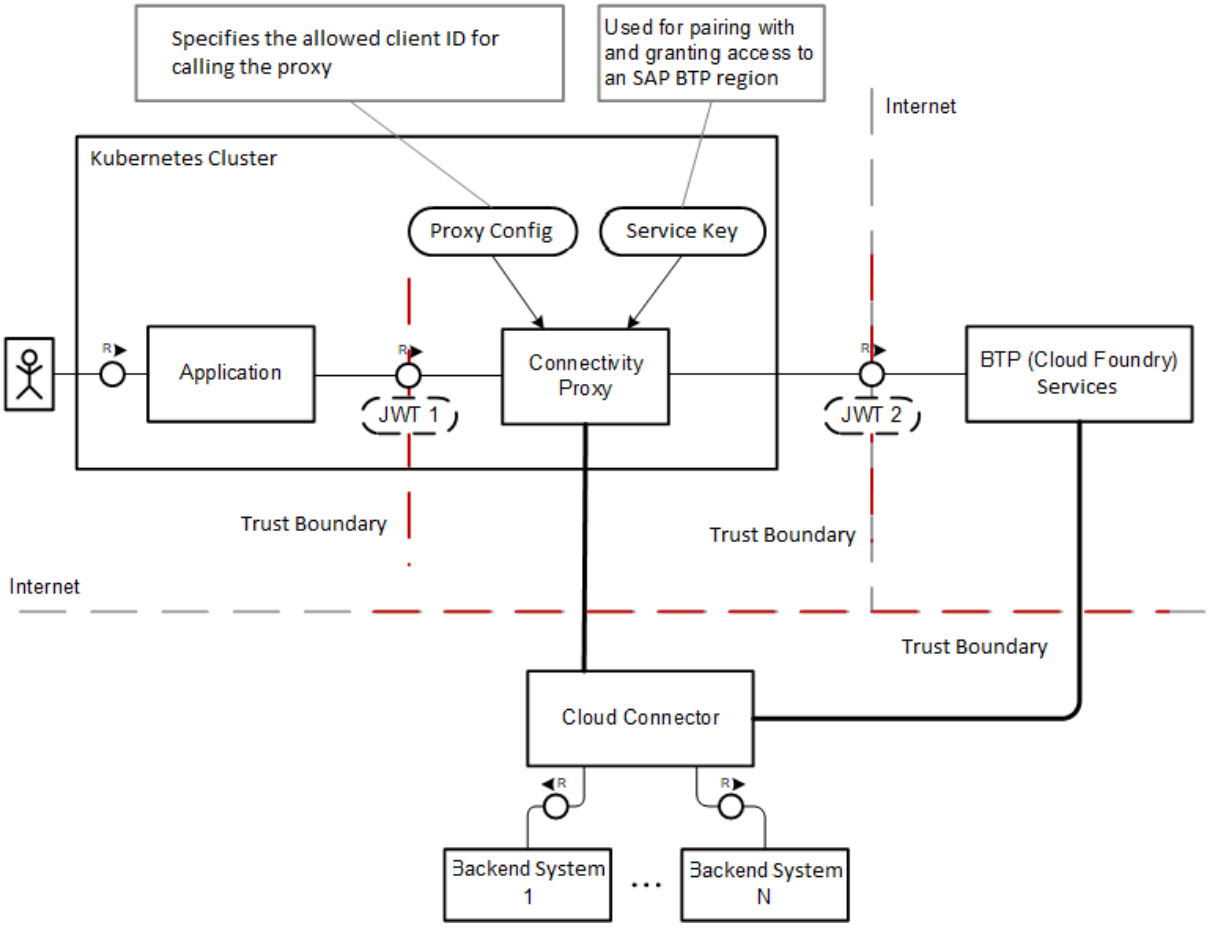


Back to Top [page 2093]

Multi-Tenant Usage: *Non-Trusted* **Environment**

The Connectivity Proxy operates on behalf of multiple tenants. Applications cannot be trusted. The Connectivity Proxy is configured as follows:

- Proxy authorization is *enabled*.
- Tenant mode is *shared*.
- Uses a statically configured service key of the `connectivity_proxy` service instance of the `connectivity` service.
- Connects to the central Connectivity service on behalf of the dynamically determined tenant, based on the OAuth access token (JWT) forwarded by the application.

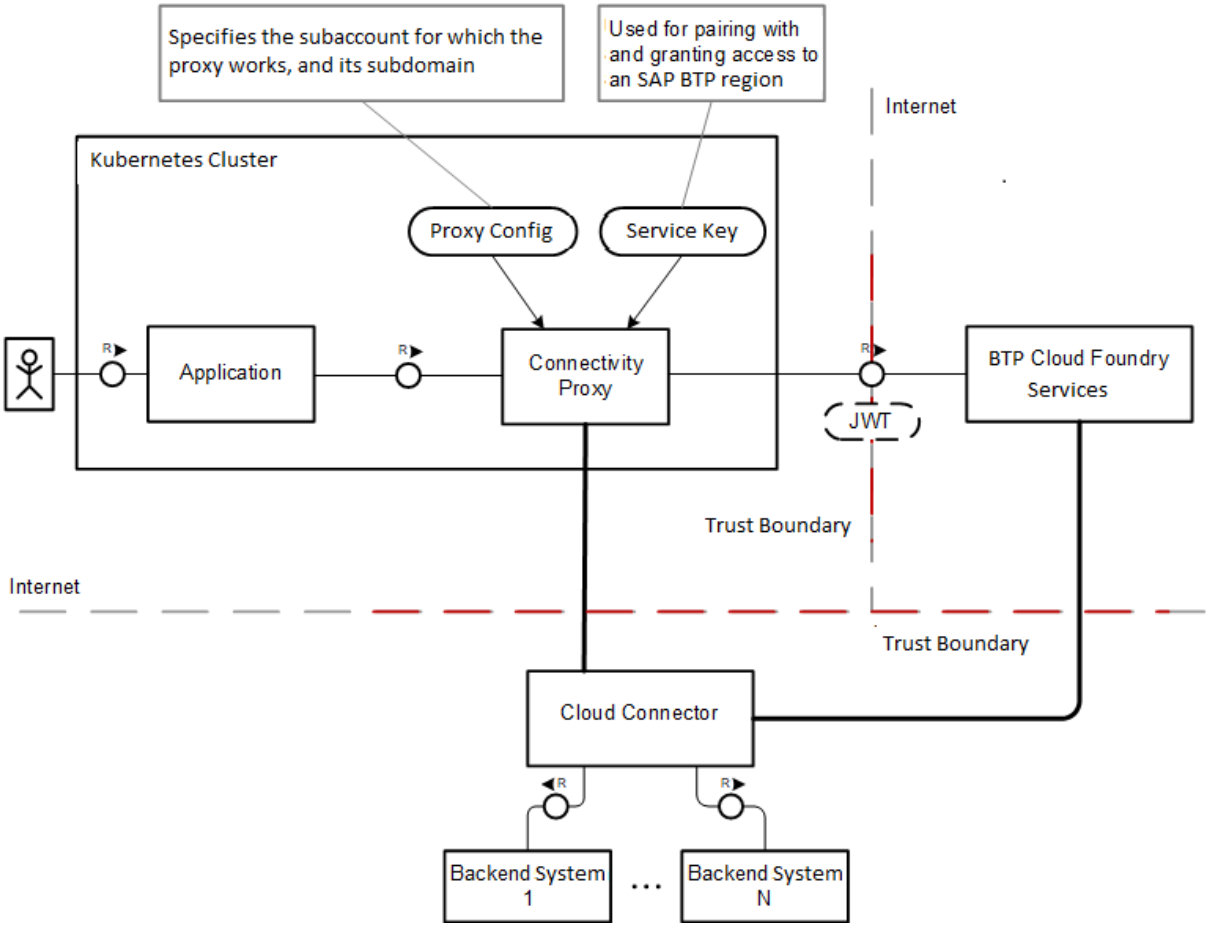


Back to Top [page 2093]

Single-Tenant Usage: *Trusted* Environment

The Connectivity Proxy operates on behalf of a single, statically configured tenant. Applications are trusted. The Connectivity Proxy is configured as follows:

- Proxy authorization is *disabled*.
- Tenant mode is *dedicated*.
- Uses a statically configured service key of the `connectivity_proxy` service instance of the `connectivity` service.
- Connects to the central Connectivity service on behalf of the statically configured tenant.

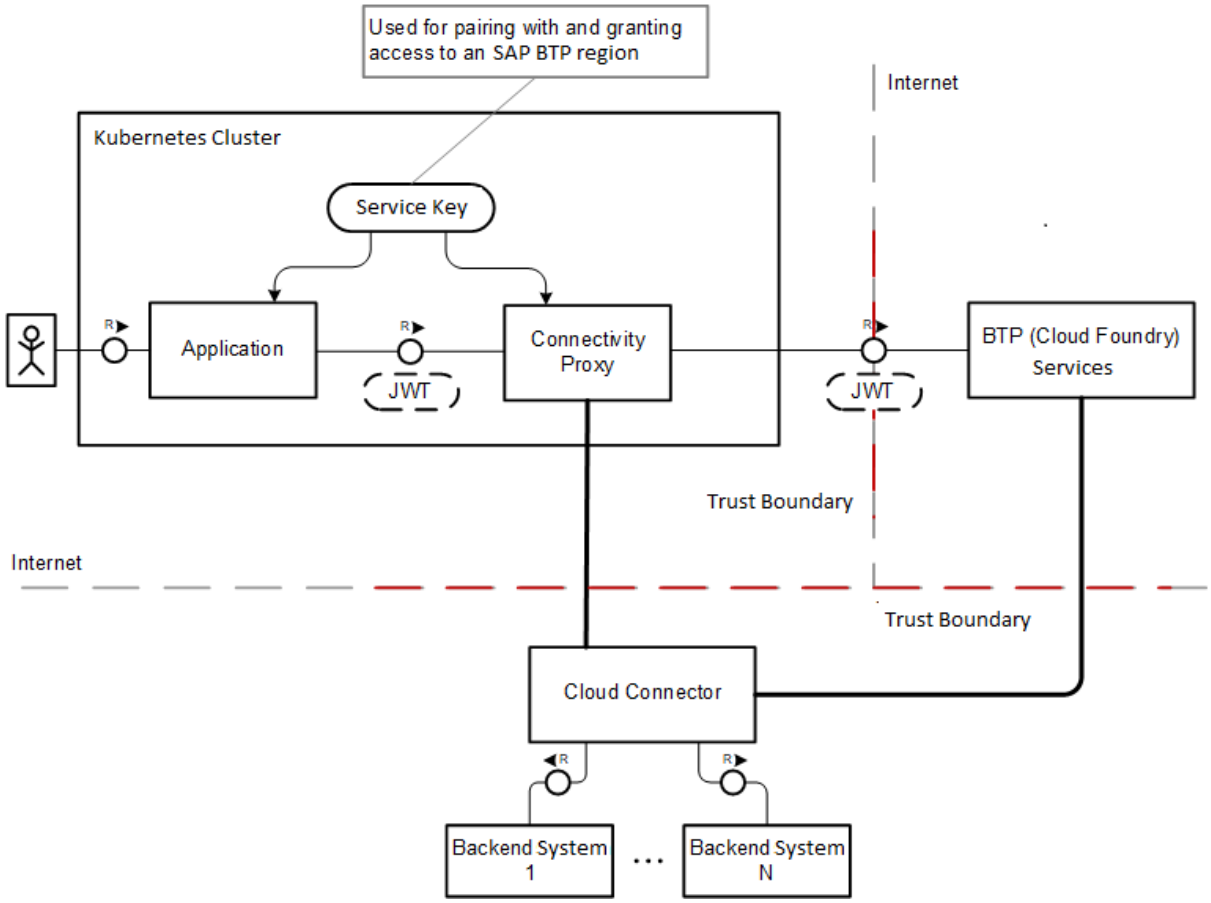


Back to Top [page 2093]

Multi-Tenant Usage: *Trusted* Environment

The Connectivity Proxy operates on behalf of multiple tenants. Applications are trusted. Applications use service keys of the `connectivity_proxy` service instance of the `connectivity` service. The Connectivity Proxy is configured as follows:

- Proxy authorization is *disabled*.
- Tenant mode is *shared*.
- Connects to the central Connectivity service on behalf of the dynamically determined tenant, using the OAuth access token (JWT) forwarded by the application.



Back to Top [page 2093]

Related Information

Installing the Connectivity Proxy in Multi-Region Mode [page 2116]
Configuration Guide [page 2128]

1.5.1.3 Mutual TLS

Use Transport Layer Security (TLS) for the Connectivity Proxy for Kubernetes.

TLS encrypts the connection between client and server, following the TLS specification. When using mutual TLS, both the TLS client and the TLS server authenticate each other through X.509 certificates.

In an on-premise network, the TLS client is represented by the Cloud Connector. On the cloud side, the direct TLS server may be:

- The Kubernetes Ingress: In this case, the Ingress terminates the TLS connection and establishes a new TLS connection to the Connectivity Proxy.

- The Connectivity Proxy: In this case, the Ingress does not terminate the TLS, but transparently forwards the TLS traffic to the Connectivity Proxy.

Connectivity proxy deployment provides two options to configure end-to-end mutual TLS, that is, the TLS communication between Cloud Connector and Connectivity Proxy:

- *With* TLS termination in the Ingress [page 2098]: TLS configuration has to be made on both the Ingress controller (TLS client) and Connectivity Proxy (TLS server).
- *Without* TLS termination in the Ingress [page 2100]: TLS configuration has to be made only on the Connectivity Proxy side (TLS server).

End-to-End Mutual TLS *with* Termination of the TLS Connection in the Ingress

Perform the following steps:

1. Enable the TLS communication on the Connectivity Proxy (TLS server) side, adding the following configuration in the `values.yaml` file:

   ```
   config:
     servers:
       businessDataTunnel:
         enableTls: true
   ```

2. Add the required TLS configuration for the Connectivity Proxy (TLS server). There are two options to do this:
   - If you already have an appropriate secret for this purpose in the cluster, add the following configuration in the `values.yaml` file:

     ```
     secretConfig:
       servers:
         businessDataTunnel:
           secretName: <secret name>
     ```

     > ### Note
     >
     > The Kubernetes secret must contain the following properties, used for authentication to the Ingress resource:
     >
     > - `tls.key`: Base 64-encoded private key in PEM format.
     > - `tls.crt`: Base 64-encoded certificate in PEM format.
     > - `ca.crt`: Base 64-encoded full certificate authority (CA) chain in PEM format.

   - If you don't have such a Kubernetes secret, it can be automatically generated. Add the following configuration in the `values.yaml` file:

     ```
     secretConfig:
       servers:
         businessDataTunnel:
           secretName: <secret name>
     ```

```
      secretData:
        key: <base 64-encoded private key in PEM format>
        certificate: <base 64-encoded certificate in PEM format>
        caCertificate: <base 64-encoded full Certificate Authority chain
in PEM format>
```

> **Note**
>
> The certificate must be issued for the external host specified on the configuration path `config.servers.businessDataTunnel.externalHost` or for the domain to which the external host belongs. For example, if the external host is "*ingress.mycluster.com*", the certificate CN or SAN must contain "*ingress.mycluster.com*" or "**.mycluster.com*".

3. Add the required TLS configuration for the Ingress. There are three options to do this:
   - If you already have an appropriate Kubernetes secret for this purpose in the cluster, add the following configuration in the `values.yaml` file:

     ```
     ingress:
       tls:
         proxy:
           secretName: <secret name>
     ```

     > **Note**
     >
     > The Kubernetes secret must contain the following properties, used for authentication to the Ingress resource:
     >
     > - `tls.key`: Base 64-encoded private key in PEM format.
     > - `tls.crt`: Base 64-encoded certificate in PEM format.
     > - `ca.crt`: Base 64-encoded full certificate authority (CA) chain in PEM format.

   - If you don't have such a secret, it can be automatically generated. Add the following configuration in the `values.yaml` file:

     ```
     ingress:
       tls:
         proxy:
           secretName: <secret name>
           secretData:
             key: <base 64-encoded private key in PEM format>
             certificate: <base 64-encoded certificate in PEM format>
             caCertificate: <base 64-encoded full Certificate Authority chain
     in PEM format>
     ```

   - If you didn't add any TLS configuration for the Ingress, the TLS configuration of the Connectivity Proxy is reused for the Ingress, that is, the Connectivity Proxy and the Ingress will use the same TLS configuration to communicate to each other.

     > **Note**
     >
     > In this case, the specified certificate must be issued by the specified certificate authority.

> **Note**
>
> The certificate must be issued for the external host specified on the configuration path `config.servers.businessDataTunnel.externalHost` or for the domain to which the external

host belongs. For example, if the external host is "*ingress.mycluster.com*", the certificate CN or SAN must contain "*ingress.mycluster.com*" or "**.mycluster.com*".

> ### Note
>
> In this case, the **minimum required version** for the NGINX ingress controller is 0.31.0.

Back to Top [page 2097]

End-to-End Mutual TLS *without* Termination of the TLS Connection in the Ingress

Perform the following steps:

1. Enable the TLS communication on the Connectivity Proxy (TLS server) side, adding the following configuration in the `values.yaml` file:

   ```
   config:
     servers:
       businessDataTunnel:
         enableTls: true
   ```

2. Add the required TLS configuration for the Connectivity Proxy (TLS server). There are two options to do this:
   - If you already have an appropriate secret for this purpose in the cluster, add the following configuration in the `values.yaml` file:

     ```
     secretConfig:
       servers:
         businessDataTunnel:
           secretName: <secret name>
     ```

     > ### Note
     >
     > The Kubernetes secret must contain the following properties, used for authentication to the Ingress resource:
     >
     > - `tls.key`: Base 64-encoded private key in PEM format.
     > - `tls.crt`: Base 64-encoded certificate in PEM format.
     >
     > For Connectivity proxy release < 2.4.0, an additional field is required:
     >
     > - `ca.crt`: Base 64-encoded full **Connectivity** certificate authority (CA) chain in PEM format.

   - If you don't have such a secret, it can be automatically generated. Add the following configuration in the `values.yaml` file:

     ```
     secretConfig:
       servers:
     ```

```
    businessDataTunnel:
      secretName: <secret name>
      secretData:
        key: <base 64-encoded private key in PEM format>
        certificate: <base 64-encoded certificate in PEM format>
```

> **Note**
>
> For Connectivity proxy release < 2.4.0 an additional field is required:
>
> ```
> secretConfig:
>   servers:
>     businessDataTunnel:
>       ...
>       secretData:
>         ...
>         caCertificate: <base 64-encoded full Connectivity Certificate
> Authority chain in PEM format>
> ```

> **Note**
>
> The certificate must be issued for the external host specified on the configuration path `config.servers.businessDataTunnel.externalHost` or for the domain to which the external host belongs. For example, if the external host is "*ingress.mycluster.com*", the certificate CN or SAN must contain "*ingress.mycluster.com*" or "**.mycluster.com*".

3. Skip the TLS configuration on the Ingress resource, that is, skip the configuration path `ingress.tls` in the `values.yaml` file.

   > **Note**
   >
   > In this case, you must set the configuration parameter `config.servers.businessDataTunnel.strictSniEnabled` to **`false`**, because this strict SNI configuration is relevant for a TLS termination in the Ingress. In case of direct TLS connection, strict SNI is supported by default.

> **Note**
>
> By default, the NGINX ingress controller does not support communication without termination of the TLS connection. To support such communication, the `<--enable-ssl-passthrough>` flag must be part of the configuration with which the ingress controller is started. For more information, see Command line arguments (Kubernetes documentation on github).

Back to Top [page 2097]

1.5.1.4 External Health Checking

Perform external health checks for the Connectivity Proxy for Kubernetes.

External health checking lets you check and monitor the health of the Connectivity Proxy using an external application. To achieve this, you must call one of the following URLs:

- `https://healthcheck.<config.servers.businessDataTunnel.externalHost>/healthcheck`
- `https://healthcheck.<config.servers.businessDataTunnel.externalHost>`, that will redirect the request to the first URL.

The external health checking configuration depends on the mutual TLS configuration. Currently, the Connectivity Proxy deployment provides three mutual TLS configuration options:

- Mutual TLS *only to the Ingress* [page 2102]
- End-to-end mutual TLS *with* termination of the TLS connection in the Ingress [page 2102]
- End-to-end mutual TLS *without* termination of the TLS connection in the Ingress [page 2103]

For more information, see Mutual TLS [page 2097].

Mutual TLS *Only to the Ingress* **or End-to-End Mutual TLS** *with* **Termination of the TLS Connection in the Ingress**

In these cases, you have two options to configure the external health checking:

- Reuse the TLS secret that is used for communication with the Cloud Connector. In this case, you don't need to do any extra configuration.

  > ### Note
  >
  > The certificate from the TLS secret must be issued for both the external host specified on the configuration path `<config.servers.businessDataTunnel.externalHost>` and the host `healthcheck.<config.servers.businessDataTunnel.externalHost>`. For example, if the external host is "*ingress.mycluster.com*", the certificate CN could be "*ingress.mycluster.com*" and certificate SAN must contain "*healthcheck.ingress.mycluster.com*".

- Use a specific TLS secret only for the communication with the external health checking application. In this case, there are two options to do this:
  - If you already have an appropriate secret for this purpose in the cluster, add the following configuration in the `values.yaml` file:

    ```
    ingress:
      healthcheck:
        tls:
          secretName: <secret name>
    ```

    > ### Note
    >
    > The Kubernetes secret must contain the following properties, used for authentication to the Ingress resource:
    >
    > - `tls.key`: Base 64-encoded private key in PEM format.
    > - `tls.crt`: Base 64-encoded certificate in PEM format.

- If you don't have such a Kubernetes secret, it can be automatically generated. Add the following configuration in the `values.yaml` file:

```
ingress:
  healthcheck:
    tls:
      secretName: <secret name>
      secretData:
        key: <base 64 encoded private key in PEM format>
        certificate: <base 64 encoded certificate in PEM format>
```

> ### Note
>
> The certificate from the TLS secret must be issued for the host `healthcheck.<config.servers.businessDataTunnel.externalHost>`. For example, if the external host is "*ingress.mycluster.com*", then the certificate CN or SAN must contain "*healthcheck.ingress.mycluster.com*".

> ### Note
>
> The certificate chain of the CA that issues the certificate from the TLS secret must be imported to the trust store of the external health checking application.

Back to Top [page 2101]

End-to-end Mutual TLS *without* Termination of the TLS Connection in the Ingress

In this case, execute the following steps:

1. Download the *plugins* archive and unzip it (for more information, see Lifecycle Management [page 2120]).
2. Navigate to the *plugins* folder and install *connectivity-certificate-plugin* by executing the following command:

   ```
   helm plugin install connectivity-certificate-plugin
   ```

3. Generate a TLS secret by executing the following command:

   ```
   helm get-connectivity-certificates <region_domain> <subaccount> <user>
   <password> <namespace> <secret_name>
   ```

   > ### Note
   >
   > - `region_domain`: BTP region to which you are pairing the Connectivity Proxy.
   > - `subaccount`: BTP subaccount, on whose behalf the Connectivity Proxy is running.
   > - `user`: BTP subaccount user.

- `password`: Password of the BTP subaccount user.
- `namespace`: Kubernetes namespace to which the secret is created.
- `secret_name`: Name of the Kubernetes secret to be created.

4. Specify a TLS secret that will be used for the communication with the external health checking application. There are two options to do this:
   - If you already have an appropriate secret for this purpose in the cluster, add the following configuration in the `values.yaml` file:

     ```
     ingress:
       healthcheck:
         tls:
           secretName: <secret name>
     ```

     > ### Note
     >
     > The Kubernetes secret must contain the following properties, used for authentication to the Ingress resource:
     >
     > - `tls.key`: Base 64-encoded private key in PEM format.
     > - `tls.crt`: Base 64-encoded certificate in PEM format.

   - If you don't have such a Kubernetes secret, it can be automatically generated. Add the following configuration in the `values.yaml` file:

     ```
     ingress:
       healthcheck:
         tls:
           secretName: <secret name>
           secretData:
             key: <base 64 encoded private key in PEM format>
             certificate: <base 64 encoded certificate in PEM format>
     ```

   > ### Note
   >
   > The certificate from the TLS secret must be issued for the host `healthcheck.<config.servers.businessDataTunnel.externalHost>`. For example, if the external host is "*ingress.mycluster.com*", then the certificate CN or SAN must contain "*healthcheck.ingress.mycluster.com*".

   > ### Note
   >
   > The certificate chain of the CA that issues the certificate from the TLS secret must be imported to the trust store of the external health checking application.

5. Specify the TLS secret that was generated in step 3, to be used for the communication between the Ingress controller and the Connectivity Proxy. Add the following configuration in the `values.yaml` file:

   ```
   ingress:
     healthcheck:
       tls:
         proxy:
           secretName: <name of the secret generated on step 3>
   ```

Back to Top [page 2101]

1.5.1.5 High Availability

Run the Connectivity Proxy for Kubernetes in a high-availability setup.

Content

Overview [page 2105]

High-Availability Modes [page 2106]

Advantages and Drawbacks [page 2106]

High-Availability Mode: *Path* [page 2106]

High-Availability Mode: *Subdomain* [page 2107]

Overview

The Connectivity Proxy can work in an active-active high-availability setup. In this setup, there are at least two Connectivity Proxy instances, both running actively and simultaneously.

The main purpose of an active-active deployment is to provide high availability and allow zero-downtime maintenance as well as horizontal-scaling capabilities.

The Kubernetes service exposing the Connectivity Proxy pods distributes and load-balances the traffic from the workloads across all running Connectivity Proxy pods.

> ⓘ Note
>
> The load balancing strategy for distributing traffic to the Connectivity Proxy pods depends on the *kube-proxy* mode. The strategies used by the different *kube-proxy* modes are described in the Kubernetes documentation . This configuration is done on cluster level.

> → Tip
>
> MultiAZ with Pod Anti-Affinity and PodDisruptionBudget are supported out of the box in the Helm chart.

Back to Content [page 2105]

High-Availability Modes

There are two technical options (modes) for the active-active deployment:

- *Path*
- *Subdomain*

The difference between the two modes is in the way the routing information of the current Connectivity Proxy (which is being used for the current connection) is passed to the Cloud Connector.

Depending on the perspective, as well as on the concrete requirements and boundaries you have, both modes may have advantages and drawbacks, so you should carefully choose the one which best suits your needs.

The number of Connectivity Proxy instances that run simultaneously depends on the *replicaCount* configuration in the `values.yml` file.

```
deployment:
  replicaCount: <number_of_replicas>
```

Back to Content [page 2105]

Advantages and Drawbacks

High-Availability Mode	Path	Subdomain
Compatibility with end-to-end mutual TLS without termination	No	Yes
Compatibility with secure and inexpensive certificates	Yes	No

Back to Content [page 2105]

High-Availability Mode: *Path*

To configure the Connectivity Proxy for the high-availability mode *Path*, add the following configuration in the `values.yml` file:

```
config:
  highAvailabilityMode: "path"
```

Back to Content [page 2105]

High-Availability Mode: *Subdomain*

To configure the Connectivity Proxy for the high-availability mode *Subdomain*, add the following configuration in the `values.yml file`:

```
config:
  highAvailabilityMode: "subdomain"
```

> ⓘ Note
>
> High-Availability mode *Subdomain* uses either wildcard certificates or SAN (Subject Alternative Name) certificates. Wildcard certificates cover all possible subdomains. However, they are insecure since when the certificate key that is installed on one subdomain gets compromised, it will be compromised on all of the subdomains it is installed on. A safer option is to use SAN certificates, although they may be expensive and require each subdomain to be specified explicitly.

Back to Content [page 2105]

1.5.1.6 Audit Logging

Use audit logging for the Connectivity Proxy for Kubernetes.

At runtime, the Connectivity Proxy produces audit messages for the relevant auditable events. These audit messages help you gather audit data which can be useful for identifying possible attacks or unwanted access attempts. The audit messages are written on the standard output and it is up to the operator of the Connectivity Proxy to configure the surrounding infrastructure in a way that these audit messages are collected, stored, protected from tampering, rotated when necessary, and so on.

> ⓘ Note
>
> When written to the standard system output, the audit log messages are preceded by the prefix `[AUDIT-EVENT]` to distinguish them from the other logs.

Audit Messages

In the table below, the left column shows a description of the security event, the right column shows the corresponding audit message. These messages are written on behalf of the consumer tenant.

> ### Note
>
> The `<message>` field of the audit message should be treated as case insensitive.

Operation	Audit Message
`[Security Event]` **Successfully opened tunnel** from Cloud Connector to Connectivity Proxy.	`{"user":"YYY","data":"{"action":"open connectivity tunnel","message":"tunnel handshake success","tunnelId":"NNN","objectName":"connectivity tunnel management","clientId":"FFF","loggedByClass":"com.sap.core.connectivity.tunnel.server.TunnelServerHandshaker","handshakeDetails":"success"}","tenant":"TTT"}`
`[Security Event]` **Failed attempt to open tunnel** from Cloud Connector to Connectivity Proxy.	`{"user":"YYY","data":"{"action":"open connectivity tunnel","message":"tunnel handshake failure","tunnelId":"NNN","objectName":"connectivity tunnel management","clientId":"FFF","loggedByClass":"com.sap.core.connectivity.tunnel.server.TunnelServerHandshaker","handshakeDetails":"UUU"}","tenant":"TTT"}`

In the table above, there are multiple keys with UPPER CASE values. At runtime, these keys hold real scenario values:

- YYY: User executing the operation, if known. If the request is not done on behalf of a user, the value will be `fallback_for_missing_user`.
- NNN: Secure tunnel between the Connectivity Proxy and the Cloud Connector.
- TTT: Tenant on whose behalf the operation is executed.
- FFF: Remote client (unique identifier of a particular installation) of the Cloud Connector.
- UUU: Additional information about the event.

1.5.1.7 Integration with SAP Services

Integrate the Connectivity Proxy for Kubernetes with SAP services.

Connectivity Service [page 2109]	Use the Connectivity Proxy for Kubernetes with the Connectivity service to connect to on-premise systems.
Security (Proxy Authorization) [page 2109]	Perform proxy authorization for the Connectivity Proxy.

1.5.1.7.1 Connectivity Service

Use the Connectivity Proxy for Kubernetes with the Connectivity service to connect to on-premise systems.

On-premise systems are exposed via the Cloud Connector. The Cloud Connector is connected to (registered in) the SAP Connectivity service and ready to serve. The Connectivity Proxy is the software component to which, at runtime, the Cloud Connector connects and establishes the business data tunnel.

To use the Connectivity Proxy, it must be paired with the SAP Connectivity service, that is, configured to connect to the central service to which initially the Cloud Connector was connected. This enables the Connectivity Proxy to dynamically (on-demand) serve its purpose, that is, to securely establish business data tunnel connections to the Cloud Connector, hosted in an on-premise network next to the backend systems.

Prerequisites

You have an account on SAP BTP, Cloud Foundry environment, and your SAP BTP account user has the authorization to create service instances.

Procedure

1. Create a Connectivity service instance with service plan `connectivity_proxy`.
   For more information, see Creating Service Instances in Cloud Foundry.
2. Create a service key of the created service instance.

   > → Tip
   >
   > You can create the service key with x.509 credentials instead of a client secret, but only in the SAP-managed certificate case. If you do so, you must take care to rotate the key before the certificate expires.

   For more information, see Service Keys.
3. Follow the instructions under Lifecycle Management [page 2120] and set up the Kubernetes secret for pairing with the central SAP Connectivity service, providing the content of the created service key.
4. Check the Configuration Guide [page 2128] for `config.integration.connectivityService*` parameters.

1.5.1.7.2 Security (Proxy Authorization)

Perform proxy authorization for the Connectivity Proxy for Kubernetes.

An important aspect of using the Connectivity Proxy is the proxy authorization, that is, how the access to the proxy is controlled (protected), enabling the callers to reach out to on-premise systems exposed via the Cloud Connector.

As described in Operational Modes [page 2093], there are two major categories for consideration: The level of trust in the environment, and the *type of tenancy* (single vs multi-tenant). This helps you prevent unwanted callers and is especially useful in scenarios where calls to the proxy can originate from workloads outside of your control, or if you simply want to apply stricter rules.

The authorization of the Connectivity Proxy is based on OAuth and it is provided via integration with the SAP UAA (aka XSUAA) service acting as an OAuth server. For a successful authorization request, a JSON Web token (JWT) must be sent to the Connectivity Proxy, for which the following is valid:

- Issued by XSUAA
- Not expired
- Passes the signature verification (the Connectivity Proxy takes care to get the token keys from XSUAA for offline JWT verification).
- Its `client_id` (the client ID of the OAuth client which is part of the respective XSUAA service instance credentials) matches the allowed client ID in the Connectivity Proxy configuration.

```
config:
  servers:
    proxy:
      authorization:
        oauth:
          allowedClientId: "the client id which is allowed as JWT issuer for
proxy authorization"
```

> → Tip
>
> The service instance used to protect the Connectivity Proxy can be any XSUAA service instance.

> ⓘ Note
>
> Based on the above info, the Connectivity Proxy can be protected with the very same service instance you use to perform the user login flow for your application. In this case, you can reuse the login token. However, to achieve separation of concerns, we recommend that you use a dedicated service instance to protect the Connectivity Proxy.

Multi-Tenancy and Subscriptions

An important aspect of the proxy authorization is that, when enabled, it is also used to pass the tenant context for the request to the Connectivity Proxy. When you fetch a token for proxy authorization, make sure you do so on behalf of the *correct* tenant, over which you want to call the Connectivity Proxy.

In cases of multi-tenancy, or single-tenancy where the proxy-protecting instance is created in a tenant that is *different* from the one for which the proxy is dedicated, you must *ensure subscriptions* between this tenant and the proxy-protecting instance to fetch a token on behalf of the correct tenant.

1.5.1.8 Automatic Pickup on Resource Changes

Configure automatic pickup on resource changes for the Connectivity Proxy.

The Connectivity Proxy can automatically pick up changes in secrets and configmaps. This is achieved by performing a rolling restart when a resource is modified.

Restart Watcher

The restart watcher is responsible for picking up changes in the above-mentioned resources, and for restarting the Connectivity Proxy if given conditions are met.

It will be informed for changes in a secret or configmap containing the label *connectivityproxy.sap.com/restart*.

To enable the restart watcher, you must configure the following in the `values.yml` file.

```
deployment:
  restartWatcher:
    enabled: true
```

> ### Note
>
> The restart watcher is unique for every Helm installation of the Connectivity Proxy and will be deployed as a part of it.

> ### Caution
>
> If the Connectivity Proxy:
>
> - is with initial installation version < 2.5.0
> - is upgraded via Helm to version 2.5.0 or higher
> - has the restart watcher enabled
> - and has replica count 2
>
> the restart operation which is performed by the restart watcher may not be a ZDM (zero-downtime migration).
>
> To avoid such situation, you should perform a fresh Helm installation. This is inevitable because a change in an immutable field of a Kubernetes resource is required.

Conditions for Restart

The restart watcher checks for two possible values of the label *connectivityproxy.sap.com/restart*:

- if the value is an empty string, the restart watcher will perform a rolling update on the Connectivity Proxy, based on respective Helm installation.
- if the value has the form `<helm_installation_name>.<namespace>`, the restart watcher will perform a rolling restart only if the values of `<helm_Installation_name>` and `<namespace>` match the watcher's installation and namespace.
- for all other possible values, the restart watcher will not perform a restart.

Examples

- You have a Helm installation in the *default* namespace and a secret with label *connectivityproxy.sap.com/restart: ""* (with value empty string) in namespace *test*. In this case, the Connectivity Proxy installation in namespace *default* will be restarted for every change of the secret.
- You have a Helm installation with name *connectivity-proxy* in namespace *test*, and a configmap with name *test-configmap* and label *connectivityproxy.sap.com/restart* : *connectivity-proxy.test*. In this case, the Connectivity Proxy will be restarted for every change of the configmap. However, if you have another installation in the same namespace *test* but with name *connectivity-proxy2*, restarts will not be triggered on it based on changes of the *configmap test-configmap*.

1.5.1.9 Service Channels: On-Premise-to-Cloud Connectivity

Use the Connectivity Proxy to connect to an on-premise network via a Cloud Connector service channel.

If you want to consume a cloud application running in a Kubernetes cluster, and the application is not accessible via Internet, it can be securely exposed by the Connectivity Proxy and consumed from the on-premise network via a Cloud Connector service channel.

For more information on configuring service channels in the Cloud Connector, see Using Service Channels [page 1949].

Perform the following steps to expose an application through the Connectivity Proxy:

1. Install the Connectivity Proxy in the cluster where the cloud application is running.
2. Create a Kubernetes resource of type `ServiceMapping` for every cloud application that you want to expose through the Connectivity Proxy.

   Sample Code

   ```
   apiVersion: connectivityproxy.sap.com/v1
   kind: ServiceMapping
   ```

```
metadata:
  name: <required>
spec:
  type: <required>
  subaccountId: <required>
  serviceId: <required>
  tenantId: <optional>
  internalAddress: <required>
  locationIds: <optional>
```

- `name`: Required. Name of the `ServiceMapping` that is created.
- `type`: Required. Type of the `ServiceMapping`. The type defines the connection protocol used to expose the cloud application. Currently, the supported type is **TCP**.
- `subaccountId`: Required for multi-tenant modes and optional for single-tenant modes. This is the ID of the tenant on whose behalf the cloud application is exposed. In single-tenant modes, the configured dedicated tenant is used by default.
- `serviceId`: Required. This is the virtual host used to expose the cloud application.
- `tenantId`: Optional. If the Connectivity Proxy consumer's flow has a tenancy domain model different from the SAP BTP tenancy domain model, you can specify the tenant identifier of the custom domain model here.
- `internalAddress`: Required. This field is composed of the host and port for the exposed application. The Connectivity Proxy will use the internal address for opening a connection to the application. This usually is the DNS name of a Kubernetes service and a port.
- `locationIds`: Optional. List of location identifiers for which service channels can be opened. By default, if the list of location identifiers is not provided, only the default location identifier is supported. If more identifiers should be supported and the default one is amongst them, you can add them as "" (empty string) in the list.
  For more information about location identifiers, see Set up Connection Parameters and HTTPS Proxy (step 4).
  Example (Service Mapping):

  Sample Code

  ```
  spec:
    locationIds:
      - "locationId1"
      - "locationId2"
      - ""
  ```

  Note

  The combination of `type`, `subaccountId` and `serviceId` for a s ervice mapping resource must be unique.

Note

The combination of `type`, `subaccountId`, and `serviceId` for a `ServiceMapping` resource should be unique.

3. Once the `ServiceMapping` resource is successfully created, it is picked up and loaded in the Connectivity Proxy.

> ⚠ Caution
>
> The `ServiceMapping` is a cluster-scoped Kubernetes resource. That is, the created `ServiceMapping`s are not bound to a specific namespace. They will be picked up from all namespaces of the Kubernetes cluster and loaded in the Connectivity Proxy.

> → Remember
>
> Picking up and loading a newly created `ServiceMapping` resource in the Connectivity Proxy may take a few seconds, as it is done by Kubernetes-native mechanisms.

4. After the `ServiceMapping` is loaded in the Connectivity Proxy, the cloud application can be consumed via the Cloud Connector as of version 2.15.0.
5. To stop exposing the cloud application through the Connectivity Proxy, delete the corresponding `ServiceMapping` resource from the cluster.

1.5.1.10 Installing the Connectivity Proxy in Clusters with Istio

Install the Connectivity Proxy in a Kubernetes cluster that is configured for Istio.

> ⚠ Restriction
>
> Currently, in clusters with Istio configured, only the high availability mode "path" is supported. Also, you cannot configure end-to-end mutual TLS (mTLS).
>
> By default, the traffic between Cloud Connector and the ingress gateway is secured by mTLS. The traffic between the ingress gateway pod and the Connectivity Proxy pod is also encrypted by Istio by default.

Required Configuration

1. The ingress `className` must be changed to "istio". You can specify this in the `values.yaml` file:

> {·} Sample Code
>
> ```
> ingress:
>   className: "istio"
> ```

> ⚠ Caution
>
> The default value of `ingress.className` is "nginx". It is mandatory to change it if you have Istio configured.

2. By default, Istio is usually installed in the namespace "istio-system". However, you can install it in a different namespace. The Connectivity Proxy needs to know Istio's installation namespace because it creates secrets used by the Istio ingress gateway when performing the TLS handshake with the Cloud Connector.
   The Connectivity Proxy will create those secrets in the default namespace "istio-system". However, if you have installed Istio in a different namespace, you can specify this in the `values.yaml` file:

   Sample Code

   ```
   ingress:
     istio:
       namespace: <istio-installation-namespace>
   ```

   Caution

   The secret that will be used for TLS configuration of the ingress-gateway should be created in the installation namespace of Istio. This secret is specified with the field `ingress.tls.secretName` in the `values.yaml`. For more information, see Configuration Guide [page 2128].

3. The Connectivity Proxy uses the selector which is configured in the Istio ingress gateway to apply a gateway configuration on the gateway pods. When installing Istio with default configurations, it uses "istio: ingressgateway" as default value for the selector.
   The Connectivity Proxy also has this value configured as default one. If Istio was installed with different configurations and you need to change the selector, you can specfify this in the `values.yaml` file:

   Sample Code

   ```
   ingress:
     istio:
       gateway:
         selector:
           "key1" : "value1"
           "key2" : "value2"
   ```

Caution

Istio AuthorizationPolicy

If you have configured an Istio authorization policy that could affect traffic to the Connectivity Proxy, you may need to add the following rules (or the whole authorization policy):

Sample Code

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: <authorization-policy-name>
  namespace: <istio-installation-namespace>
spec:
  selector:
    matchLabels:
      app: connectivity-proxy
  action: ALLOW
  rules:
```

```
  - to:
    - operation:
        ports: ["8042", "20001", "20003", "20004"]
```

1.5.1.11 Installing the Transparent Proxy as Subchart of the Connectivity Proxy

If you install the Connectivity Proxy with Helm, you can have the Transparent Proxy installed as a subchart. For more information about Helm subcharts, see the Helm documentation .

1. The Transparent Proxy subchart is disabled by default. To enable it, set the `enabled` flag to **`true`** in the `transparent-proxy` section in the `values.yaml` file.

   Sample Code

   ```
   transparent-proxy:
     enabled: true
   ```

2. You can modify the Transparent Proxy version from the `requirements.yaml` file in the Connectivity Proxy release.

Related Information

Transparent Proxy for Kubernetes [page 2164]
Configuration Guide [page 2128]

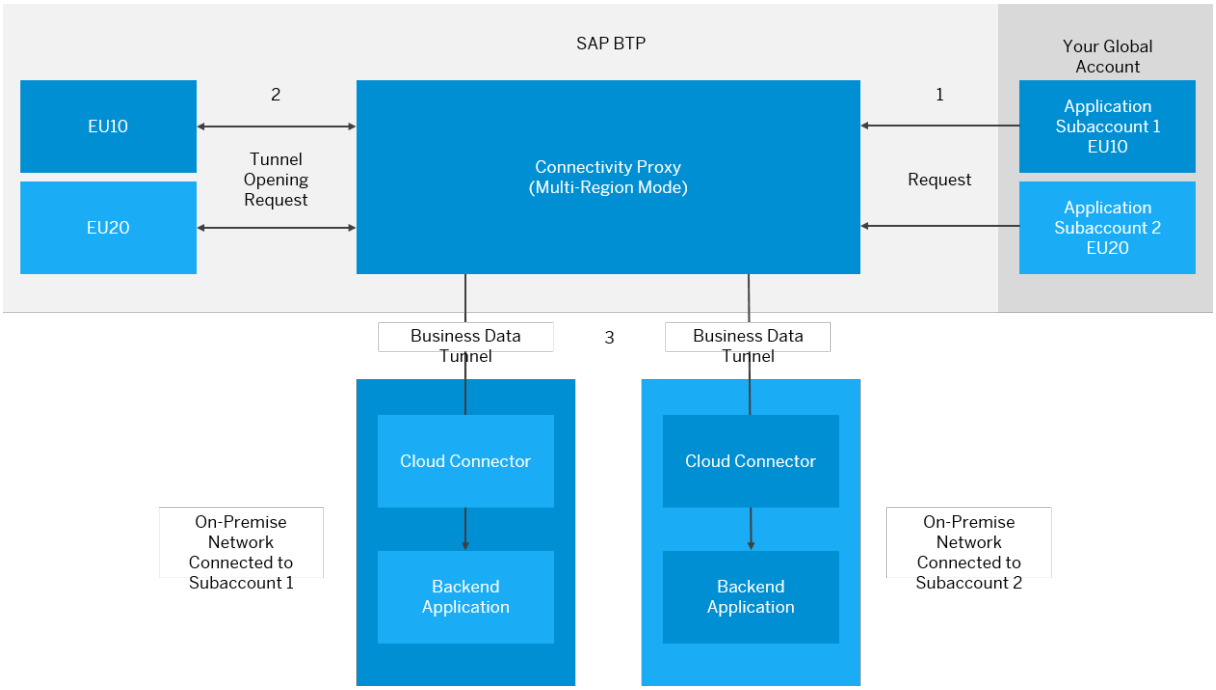
1.5.1.12 Installing the Connectivity Proxy in Multi-Region Mode

Use a single Connectivity Proxy installation for your Kubernetes cluster.

Overview

The Connectivity Proxy can operate in multi-region mode.

SAP BTP regions represent locations (of data centers) like eu10, eu20, us10, us20, and so on. Using the multi-region mode, a single Connectivity Proxy installation can operate in different regions which, however, must be part of the same landscape (global account). Otherwise, the uniqueness of subaccount IDs is not guaranteed.



Why is the Multi-Region Mode Valuable?

Using the multi-region mode, you can install *a single Connectivity Proxy* for your global account. Applications - deployed in different regions or in different subaccounts within the same region - can consume the Connectivity service through this single Connectivity Proxy instance.

Without multi-region mode, you must install *one Connectivity Proxy for every region* where applications on cloud side need to consume the Connectivity service. This setup requires considerably more maintenance effort and consumes a lot more resources than just a single (multi-region) instance.

Install the Connectivity Proxy in Multi-Region Mode

1. *Without multi-region mode*, the service keys for integration with other services are provided through the *values.yaml* file in the following way:

```
secretConfig:
  integration:
    connectivityService:
      secretData: <base64 encoded service key>
```

In **multi-region mode**, you can assign different regions to a single Connectivity Proxy instance. Consequently, the service keys for each region are needed. In this case, you cannot configure the keys through the *values.yaml* file. Instead, you must put them into secrets, and then specify them in a *ConfigMap*.
Let's create a *ConfigMap* containing two region configurations.
First, we configure the two secrets holding the service keys for each region:

```
apiVersion: v1
kind: Secret
metadata:
  name: <secret-name-region1>
data:
  serviceKey: <base64 encoded service key for region1>
```

```
apiVersion: v1
kind: Secret
metadata:
  name: <secret-name-region2>
data:
  serviceKey: <base64 encoded service key for region2>
```

> ⚠ Caution
>
> The `serviceCredentialsKey` property for each service type specified under `config.integration` must be used in all secrets that will hold service keys for the respective service type. For example, if the following `serviceCredentialsKey` is configured for the Connectivity service in the *values.yaml*...
>
> ```
> config:
>   integration:
>     connectivityService:
>       serviceCredentialsKey: serviceKey
> ```
>
> ...then it must be used in all secrets holding service keys for the Connectivity service.

In a second step, we create a *ConfigMap* that contains two region configuration IDs (*region1* and *region2*), which may have any value. Each region configuration contains the Connectivity service as dependency, and the secret name holding the specified service key.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: regionConfigurations
data:
  region1: "{\"dependencies\":{\"connectivity\": \"<secret-name-region1>\"}}"
  region2: "{\"dependencies\":{\"connectivity\": \"<secret-name-region2>\"}}"
```

> ⓘ Note
>
> Region configuration IDs do not require any specific naming pattern.

2. To enable the multi-region mode, add the following in the *values.yaml* file and specify the name of the *ConfigMap* containing the region configurations:

```
config:
  tenantMode: shared
```

```
  multiRegionMode:
    enabled: true
    configMapName: "regionConfigurations"
```

> ⓘ Note
>
> When multi-region mode is enabled, the Connectivity Proxy operates in multi-tenant mode. Therefore, you must set the `tenantMode` property to *shared*.

> ⚠ Caution
>
> If the Connectivity Proxy is installed in non-trusted mode [page 2093] (or proxy authorization is enabled), the allowed client IDs [page 2109] for all region configurations must be specified in the *ConfigMap*:
>
> ```
> apiVersion: v1
> kind: ConfigMap
> metadata:
>   name: regionConfigurations
> data:
>   region1: "{\"allowedClientId\":\"<allowed-client-id>\", \"dependencies\":
> {\"connectivity\": \"<secret-name>\"}}"
>   region2: "{\"allowedClientId\":\"<allowed-client-id>\", \"dependencies\":
> {\"connectivity\": \"<secret-name>\"}}"
> ```

> ⚠ Caution
>
> The connectivity poxy cannot be installed if both **multi-region mode** and **service channels [page 2112]** are enabled.

Consume the Connectivity Proxy in Multi-Region Mode

If multi-region mode is enabled, you must send the header `SAP-Connectivity-Region-Configuration-Id` with every request made to the Connectivity Proxy.

> → Remember
>
> Keep in mind that the correct token for each region configuration must be sent. Sending an access token/ authorization token to the proxy is required with every request, because the Connectivity Proxy is working in multi-tenant mode.

An example for a request with curl through the HTTP protocol:

> {··} Sample Code
>
> ```
> curl -x connectivity-proxy:20003 "http://<virtual_host>:<virtual_port>/
> <path>" --header "SAP-Connectivity-Region-Configuration-Id:
> region1" --header "SAP-CP-Connectivity-Service-Token:
> <base64_encoded_jwt_for_the_subaccount_in_the_specified_region_configuration>"
> ```

In this case, the request is for a region configuration with ID *region1* in the *ConfigMap*.

> **Note**
>
> The access token must be issued for the subaccount that corresponds to the region configuration, identified by the region ID that has been sent with the request.

> **Caution**
>
> If the proxy is installed in non-trusted mode, the authorization token must be provided via the `Proxy-Authorization` header.
>
> Example:
>
> **Sample Code**
>
> ```
> curl -x connectivity-proxy:20003 "http://<virtual_host>:<virtual_port>/
> <path>" --header "SAP-Connectivity-Region-Configuration-Id: region1" --
> header "Proxy-Authorization: Bearer
> <base64_encoded_jwt_for_the_subaccount_in_the_specified_region_configuratio
> n>"
> ```

Change Region Configuration

As an operator of the Connectivity Proxy, at some point you might need to make a change in the region configurations *ConfigMap* or in any of the secrets containing service credentials. For example, add or remove a region configuration, or change the service credentials in a secret. This can be done any time, and no further actions are needed to make the Connectivity Proxy start working with the performed changes:

- After a short period of time, the changes will be picked up automatically by the Connectivity Proxy.
- A potentially required restart is also done automatically.
- The restart of the Connectivity Proxy's pods is rolling.
- If high availability mode is enabled, no downtime is expected.

1.5.2 Lifecycle Management

Use the Connectivity Proxy image and the Connectivity Proxy Helm chart to manage the life cycle of the Connectivity Proxy for Kubernetes.

The Connectivity Proxy delivery includes the following main components:

- Connectivity Proxy Image [page 2121]: the functional component that packages all the binaries and utilities.

- Connectivity Proxy Helm Chart [page 2122]: used for configuring and managing the life cycle of the Connectivity Proxy via the popular Helm package manager. For more information, see Operations via Helm [page 2123].

Connectivity Proxy Image

The Connectivity Proxy image is a standard Docker image containing all the required binaries for the Connectivity Proxy. This includes the Connectivity Proxy binaries themselves, a JVM (SapMachine), and other important utilities.

Connectivity Proxy Image in DockerHub

The Connectivity Proxy image is available via the DockerHub image repository, see https://hub.docker.com/r/sapse/connectivity-proxy .

Pull information (for the latest version):

- **Registry**: docker.io
- **Repository**: sapse/connectivity-proxy
- **Tag**: 2.20.0

Example:

Sample Code

```
docker pull docker.io/sapse/connectivity-proxy:2.20.0
```

Connectivity Proxy Image in the Repository-Based Shipment Channel (RBSC)

The Connectivity Proxy image is available via the RBSC docker image repository. This requires having an S-user with associated licenses.

Note

Contact your account manager to clarify access to the repository.

Pull information (for the latest version):

- **Registry**: 73554900100900005672.dockersrv.cdn.repositories.cloud.sap
- **Repository**: com.sap.cloud.connectivity/connectivity-proxy
- **Tag**: 2.20.0
- **Authorization**: see Manage Technical Users in SAP Repositories Management (RBSC documentation).

Example:

```
docker pull 73554900100900005672.dockersrv.cdn.repositories.cloud.sap/
com.sap.cloud.connectivity/connectivity-proxy:2.20.0
```

Back to Top [page 2]

Connectivity Proxy Helm Chart

The Connectivity Proxy delivery also includes a Helm chart that you can use for life cycle management. It is the recommended way to perform life cycle management. The Connectivity Proxy Helm provides full configuration capabilities via the standard Helm method of a `values.yaml` file (see Configuration Guide [page 2128]).

Connectivity Proxy Helm Chart in the Repository-Based Shipment Channel (RBSC)

The Connectivity Proxy Helm chart is available via the RBSC Helm repository. This requires having an S-user with associated licenses.

> ⓘ Note
>
> Contact your account manager to clarify access to the repository.

Pull information (for the latest version):

- **Registry**: 73554900100900005672.helmsrv.cdn.repositories.cloud.sap
- **Repository**: com.sap.cloud.connectivity/connectivity-proxy
- **Tag**: 2.20.0
- **Authorization**: see Manage Technical Users in SAP Repositories Management (RBSC documentation).

Example:

```
helm repo add connectivity https://
73554900100900005672.helmsrv.cdn.repositories.cloud.sap --username <user> --
password <pass>
helm pull connectivity/connectivity-proxy --version=
```

Back to Top [page 2]

Related Information

Operations via Helm [page 2123]
Operations via Separate YAML Files [page 2126]

Configuration Guide [page 2128]
Additional Security Aspects [page 2141]
Sizing Recommendations [page 2142]

1.5.2.1 Operations via Helm

Use the Helm chart to configure and manage the life cycle of the Connectivity Proxy for Kubernetes.

> **Note**
>
> Out of the box, the Helm chart only supports the NGINX Ingress Controller (default) and the Istio Ingress Gateway (for more information, see Installing the Connectivity Proxy in Clusters with Istio [page 2114]).

Deploy [page 2123]
Update / Upgrade / Downgrade [page 2124]
Undeploy [page 2125]

Deploy

> **Note**
>
> For this procedure, you must have a generated public/private TLS key pair as a prerequisite. For generating the TLS key pair, you can use Gardener Certificate resources , *openssl*.

To deploy the Connectivity Proxy on a cluster that does not have the Helm chart yet, for example, in a new namespace, follow these steps:

1. Get the Connectivity Proxy Helm chart, as described in Lifecycle Management [page 2120].
2. Create a Kubernetes secret from the generated public/private TLS key pair (for the Ingress public endpoint). Depending on the Connectivity Proxy release version, the secret might require additional fields:
    1. Connectivity Proxy release < 2.4.1:
    Download the list of trusted CAs for the BTP region to which you are pairing the Connectivity Proxy.
    Example:

```
# download the list of trusted CAs
curl https://connectivity.cf.{region_domain}/api/v1/CAs -o
connectivity_ca.crt
# create a k8s secret
kubectl create secret generic connectivity-tls --from-
file=ca.crt=connectivity_ca.crt --from-file=tls.key=private.key --from-
file=tls.crt=public.crt --namespace my-namespace
```

Where `private.key` is the private key and `public.crt` is the public key of a TLS certificate, generated for the Ingress public endpoint of the Connectivity Proxy (the one which the Cloud Connector connects to).

> → Remember
>
> The content of `/api/v1/CAs` may change over time. Make sure you update it regularly.

2. Connectivity Proxy release >= 2.4.1:
   Download of the trusted CAs is automated. They are saved in a secret in the respective name space. The secret's name is formed in the following way: *connectivity-ca-<helm_installation_name>*. For example, if the helm installation name is *connectivity-proxy*, the generated secret name is *connectivity-ca-connectivity-proxy*.

   > ⓘ Note
   >
   > If a secret with the same name pattern, as mentioned above, is present whenever the Connectivity Proxy is installed/upgraded, its content is overridden.

   Example:

   ```
   # create a k8s secret
   kubectl create secret generic connectivity-tls --from-
   file=tls.key=private.key --from-file=tls.crt=public.crt --namespace my-
   namespace
   ```

   Where `private.key` is the private key and `public.crt` is the public key of a TLS certificate, generated for the Ingress public endpoint of the Connectivity Proxy (the one which the Cloud Connector connects to).
3. Prepare the `values.yaml` file for your your scenario, as described in Configuration Guide [page 2128].
4. Use the Helm CLI to deploy the Connectivity Proxy. Example:

   ```
   helm install connectivity-proxy ./connectivity-proxy-<version>.tgz -f
   values.yaml --namespace my-namespace
   ```

Back to Top [page 2123]

Update / Upgrade / Downgrade

When you have a Connectivity Proxy deployed on the cluster, you may want to maintain it by changing configurations and/or changing it's version. To do so, follow these steps:

1. Get the Connectivity Proxy Helm chart, as described in Lifecycle Management [page 2120]. It can be the same version as the one currently installed or a different version, when you want to upgrade or downgrade.
2. Prepare the `values.yaml` file for your your scenario, as described in Configuration Guide [page 2128]. Here you can just modify the one you used previously by applying the changes you desire.
3. Use the Helm CLI to upgrade the Connectivity Proxy. Example:

   ```
   helm upgrade connectivity-proxy ./connectivity-proxy-<version>.tgz -f
   values.yaml --namespace my-namespace
   ```

> ### Note
>
> Each Helm upgrade will result in a restart of the Connectivity Proxy pod(s). This is done to ensure that configuration changes are picked up immediately.

Back to Top [page 2123]

Undeploy

If you need to remove the Connectivity Proxy from you cluster or from a namespace, you can do it almost completely via normal Helm tools. However, there are some additional actions required. Please follow these steps:

1. Use the Helm CLI to undeploy the Connectivity Proxy. Example:

   ```
   helm uninstall connectivity-proxy --namespace my-namespace
   ```

2. Delete the Kubernetes secret representing the TLS certificate for the Connectivity Proxy public endpoint. Example:

   ```
   kubectl delete secret connectivity-tls --namespace my-namespace
   ```

> ### Note
>
> For Connectivity Proxy releases >= 2.4.1, the secret containing the trusted CAs is removed automatically when `helm uninstall` is executed.

> ### Caution
>
> As of Connectivity Proxy release 2.8, trying to execute `helm uninstall`, while there are still resources of type *ServiceMapping*, would result in an error (*job failed: BackoffLimitExceeded.* Detailed error message can be obtained after looking into the logs of the newly created pod `servicemapping-cleanup-verifier-job`), and the proxy would not be uninstalled. Also, `helm upgrade` operations would no longer be available.
>
> To proceed, remove all resources of type *ServiceMapping* and retry the uninstallation .

Back to Top [page 2123]

1.5.2.2 Operations via Separate YAML Files

Using separate YAML files to configure and manage the life cycle of the Connectivity Proxy for Kubernetes.

> ⚠ Caution
>
> As operating with separate YAML files that you modify and maintain is considered error prone, we do not recommend this method. Consider using Operations via Helm [page 2123] instead. If you do need to use separate YAML files, we recommend that you generate them via Helm template.

Deploy [page 2126]

Update / Upgrade / Downgrade [page 2127]

Undeploy [page 2127]

Deploy

To deploy the Connectivity Proxy on a cluster that does not have it yet, for example, in a new namespace, follow these steps:

1. Get the Connectivity Proxy YAML files via Helm template
2. Download the list of trusted CAs for the SAP BTP region to which you want to pair the Connectivity Proxy. Create a Kubernetes secret from it together with a previously generated public/private TLS key pair (for the Ingress public endpoint). For generating the TLS key pair, you can use Gardener Certificate resources , *openssl*, or some other tool. Example:

   ```
   # download the list of trusted CAs
   curl https://connectivity.cf.{region_domain}/api/v1/CAs -o connectivity_ca.crt
   # create a Kubernetes secret
   kubectl create secret generic connectivity-tls --from-
   file=ca.crt=connectivity_ca.crt --from-file=tls.key=private.key --from-
   file=tls.crt=public.crt --namespace my-namespace
   ```

   Where `private.key` is the private key and `public.crt` is the public key of a TLS certificate, generated for the Ingress public endpoint of the Connectivity Proxy (the one which the Cloud Connector connects to).

   > → Remember
   >
   > The content of `/api/v1/CAs` may change over time. Make sure you update it regularly.

3. Use the `kubectl` CLI to deploy the Connectivity Proxy. Example:

   ```
   kubectl apply -f my_changed_resource.yaml --namespace my-namespace
   ```

Back to Top [page 2126]

Update / Upgrade / Downgrade

When you have a Connectivity Proxy deployed on the cluster, you may want to maintain it by changing configurations and/or changing it's version. To do so, follow these steps:

1. Get the YAML files (or only those you want to modify) you used to deploy the Connectivity Proxy. You can also export them from the cluster via `kubectl`.
2. Make the desired changes. For example, you can modify the subaccount ID in the `config` map or the Connectivity Proxy version in the deployment.
3. Use the `kubectl` CLI to apply your changes. Example:

```
kubectl apply -f my_changed_resource.yaml --namespace my-namespace
```

> → Tip
>
> You can also use `kubectl edit` to directly modify the resources on the cluster.

> ⓘ Note
>
> When updating secrets or config maps, you must restart the pod(s) to activate the changes. You can do this by running `kubectl rollout restart statefulset/connectivity-proxy`.

Back to Top [page 2126]

Undeploy

If you need to remove the Connectivity Proxy from you cluster or from a namespace, follow these steps:

1. Get the YAML files you used to deploy the Connectivity Proxy. You can also export them from the cluster via `kubectl`.
2. Use the `kubectl` CLI to undeploy the Connectivity Proxy. Example:

```
kubectl delete -f ./dir_with_yaml_files --namespace my-namespace
```

> → Tip
>
> You can also use `kubectl delete <resource type> <resource name>`.

3. Delete the Kubernetes secret representing the TLS certificate for the Connectivity Proxy public endpoint. Example:

```
kubectl delete secret connectivity-tls --namespace my-namespace
```

Back to Top [page 2126]

1.5.2.3 Configuration Guide

Find an overview of configuration parameters for the Connectivity Proxy for Kubernetes.

Refer to the table below for the configurations available in the `values.yaml` file of the Connectivity Proxy.

> **Note**
>
> Make sure you become familiar with the semantics, restrictions and interoperability aspects of each property before using it.

Parameter Overview [page 2128]

Example [page 2138]

Parameter Overview

Parameter	Desccription	Default
additionalCAs.secretName	This property allows to specify a list of CA certificates which are to be trusted, in addition to the default ones for the outbound communication of the Connectivity Proxy components. The value is the name of a secret in the same namespace in which the proxy is installed. It must contain a field with key "ca-certificates.crt" and with the value being all CAs to be trusted, concatenated together one after the other.	None
chart.nameSuffix	Custom string used to suffix your resources.	"" (empty string)
compliance.fipsModeEnabled	Configures the Connectivity Proxy and its helper components to run in a mode that is compliant with FIPS (*Federal Information Processing Standard*). It will use *Bouncy Castle FIPS* as its Java security provider.	false
config.displayName	Display name for the Connectivity Proxy installation. It is displayed in the Cloud Connector UI in Cloud Connections section.	None

Parameter	Desccription	Default
config.integration.connectivityService.serviceCredentialsKey	Key in the Connectivity service secret resource that holds the base64-encoded value of the Connectivity Proxy service key.	"service_key"
config.logging.format	Specifies the logging format for the Connectivity Proxy and it's components. The possible values are "console" and "json".	"console"
config.servers.businessDataTunnel.enableTls	Enables end-to-end mutual TLS. See Mutual TLS [page 2097] for details.	false
config.servers.businessDataTunnel.externalHost	External ingress host of the business data tunnel server.	None
config.servers.businessDataTunnel.externalPort	External ingress port of the business data tunnel server.	443
config.servers.businessDataTunnel.strictSniEnabled	Flag for enabling and disabling strict SNI verification in the business data tunnel server.	true
config.servers.businessDataTunnel.port	Port on which to start the business data tunnel server.	8042
config.servers.businessDataTunnel.threadPoolSize	Number of worker threads for the tunnel server. If not set, the value will be calculated, based on the available resources. The value has a lower limit of 20.	None
config.servers.proxy.authorization.oauth.allowedClientId	OAuth client ID that is authorized to pass through the Connectivity Proxy when authorization is enabled.	None
config.servers.proxy.clientConnection.writeTimeoutSeconds	Write timeout for the client connections to the proxy (from the proxy's point of view).	10
config.servers.proxy.http.allowRemoteConnections	Flag for enabling and disabling calls from outside the pod to the HTTP proxy server.	true
config.servers.proxy.http.enabled	Flag for enabling and disabling the HTTP proxy server.	false
config.servers.proxy.http.enableProxyAuthorization	Flag for enabling and disabling the HTTP proxy authorization. See Operational Modes [page 2093] for details.	true
config.servers.proxy.http.port	Port on which to start the HTTP proxy server.	20003
config.servers.proxy.socks5.allowRemoteConnections	Flag for enabling and disabling calls from outside the pod to the SOCKS5 proxy server.	true
config.servers.proxy.socks5.enabled	Flag for enabling and disabling the SOCKS5 proxy server.	true

Parameter	Desccription	Default
config.servers.proxy.socks5.enable-ProxyAuthorization	Flag for enabling and disabling the SOCKS5 proxy authorization. See Operational Modes [page 2093] for details.	true
config.servers.proxy.socks5.port	Port on which to start the SOCKS5 proxy server.	20004
config.servers.proxy.threadPoolSize	Number of worker threads for the proxy server. If not set, the value will be calculated, based on the available resources. The value has a lower limit of 20.	None
config.jvm.errorFilePath	Directory for JVM error logs.	"/var/log/connectivity"
config.jvm.heapDumpPath	Directory for JVM heap dumps.	"/var/log/connectivity"
config.jvm.memory.maxDirectSize	Maximum direct memory size to be set to the java process of the Connectivity Proxy. If not set, the value will be calculated, based on the available resources. If the available resources do not set a limit, a default value is used.	512m
config.jvm.memory.maxHeapSize	Maximum heap size to be set to the java process of the Connectivity Proxy. If not set, the value will be calculated, based on the available resources. If the available resources do not set a limit, a default value is used.	512m
config.jvm.memory.minHeapSize	Minimum heap size to be set to the java process of the Connectivity Proxy. If not set, the value will be calculated, based on the available resources. If the available resources do not set a limit, a default value is used.	128m
config.highAvailabilityMode	High availability mode in which the Connectivity Proxy works. The possible values are `off`, `subdomain`, and `path`. See High Availability [page 2105] for details.	"off"
config.subaccountId	ID of the subaccount, on whose behalf the Connectivity Proxy is running. This is mandatory only for the *dedicated* tenant mode. It is ignored when running in *shared* tenant mode.	None
config.subaccountSubdomain	Subdomain of the subaccount, on whose behalf the Connectivity Proxy is running. This is mandatory only for the *dedicated* tenant mode when there is at least one proxy with disabled authorization. It is ignored when running in *shared* tenant mode or if authorization is enabled for all proxies.	None

Parameter	Desccription	Default
config.tenantMode	Mode in which the Connectivity Proxy works. The possible values are *dedicated* (single-tenant mode) and *shared* (multi-tenant mode). See Operational Modes [page 2093] for details.	"dedicated"
config.serviceChannels.enabled	Enables or disables the service channels which allow *on-premise to cloud* connectivity to services running in a Kubernetes cluster. **⚠ Caution** Only one Connectivity Proxy per Kubernetes cluster can have service channels enabled. For more information, see Service Channels: On-Premise-to-Cloud Connectivity [page 2112].	true
config.multiRegionMode.enabled	Enables or disables the multi-region mode.	false
config.multiRegionMode.configMapName	The name of the config map holding all region configurations which will be used by the Connectivity Proxy if multi-region mode is enabled.	None
config.metrics.prometheus.enabled	Enables the collection and exposure of metrics in the Prometheus format through an HTTP endpoint.	false
config.metrics.prometheus.port	Specifies the port on which the Prometheus metrics will be exposed.	9494
config.metrics.prometheus.jvmMetrics.enabled	Enables the collection of JVM-related metrics (e.g., memory usage, garbage collection, thread count, etc.).	false
config.metrics.prometheus.jmxMetrics.enabled	Enables the collection of metrics from JMX that are exposed by the Connectivity Proxy.	false
secretConfig.integration.connectivityService.secretName	Name of the secret, which will hold the credentials for the Connectivity service.	"connectivity-proxy-service-key"
secretConfig.integration.connectivityService.secretData	base64-encoded value of the service key, obtained from the Connectivity Proxy service instance.	None

Parameter	Desccription	Default
secretConfig.servers.businessDataTunnel.secretName	Name of the secret that is used for TLS handshake with The Ingress proxy or Cloud Connector. See Mutual TLS [page 2097] for details.	None
secretConfig.servers.businessDataTunnel.secretData.caCertificate	CA certificate part of the secret that is used for TLS handshake with the Ingress proxy or Cloud Connector. See Mutual TLS [page 2097] for details.	None
secretConfig.servers.businessDataTunnel.secretData.certificate	Certificate part of the secret that is used for TLS handshake with the Ingress proxy or Cloud Connector. SeeMutual TLS [page 2097] for details.	None
secretConfig.servers.businessDataTunnel.secretData.key	Key part of the secret that is used for TLS handshake with the Ingress proxy or Cloud Connector. See Mutual TLS [page 2097] for details.	None
deployment.affinity.podAntiAffinity.topologyKey	Configuration of the topology key for the built-in pod anti-affinity setting of the Connectivity Proxy. More info: Assigning Pods to Nodes | Kubernetes .	"topology.kubernetes.io/zone"
deployment.nodeSelector	Label(s) which are configured for the `nodeSelector` property for all Connectivity Proxy-associated pods. More info: https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/#nodeselector	None
deployment.podDisruptionBudget.minAvailable	Configuration for the minimum required available Pods during voluntary disruptions in the Connectivity Proxy *Pod Disruption Budget*. Can be specified as percentage (e.g. "50%") or as an absolute number (e.g. "3").	"50%"
deployment.podDisruptionBudget.unhealthyPodEvictionPolicy	Configuration for eviction policy of unhealthy pods in the Connectivity Proxy *Pod Disruption Budget*. One of "IfHealthyBudget", "AlwaysAllow". For more information, see: Kubernetes Pod Disruption Budget .	"IfHealthyBudget"
deployment.image.pullPolicy	One of **`Always`**, **`Never`**, **`IfNotPresent`**. For more information, see: Updating images (Kubernetes documentation).	"IfNotPresent"
deployment.image.pullSecret	Secret used for authentication against the repository.	None
deployment.image.registry	Name of the registry from which the Connectivity deployment is downloaded.	<the image repository>

Parameter	Desccription	Default
deployment.image.repository	Name of the repository.	"com.sap.cloud.connectivity/connectivity-proxy"
deployment.image.tag	Version of the Connectivity Proxy to be deployed.	1
deployment.image.digest	Digest the Connectivity Proxy's image to be deployed. If both digest and tag are provided, the value of the digest has precedence.	None
deployment.utilityImage.pullPolicy	One of *Always*, *Never*, *IfNotPresent*. More info: https://kubernetes.io/docs/concepts/containers/images#updating-images	"IfNotPresent"
deployment.utilityImage.pullSecret	Pull secret for the utility image, used for authenticating against the repository.	None
deployment.utilityImage.registry	Name of the registry from which the utility image will be downloaded.	"docker.io"
deployment.utilityImage.repository	Name of the repository of the utility image. **⚠ Caution** The repository must point to an image, which is a version of the Docker official Alpine image (see Alpine Docker Image). Usage of any other type of container image is not recommended and can result in broken or dysfunctional Connectivity Proxy deployment.	"alpine"
deployment.utilityImage.tag	Version of the utility image to be deployed.	"3.18.0"
deployment.utilityImage.digest	Digest of the utility image to be deployed. If both digest and tag are provided, the value of the digest has precedence.	None
deployment.replicaCount	Amount of connectivity proxies to start from this deployment.	1
deployment.resources.maxFileDescriptorCount	Maximum number of file descriptors that will be used. This value should be based on the overall scenario in which the Connectivity Proxy is used and on the load of the proxy (user requests to the proxy).	64000

Parameter	Desccription	Default
deployment.resources.limits.cpu	The *kubelet* enforces the limit so that the running container is not allowed to use more CPU resource than set as limit. If the limit is crossed, the process is throttled.	1
deployment.resources.limits.memory	The *kubelet* enforces the limit so that the running container is not allowed to use more memory than set as limit. If the limit is crossed, the process is terminated with an out-of-memory (OOM) error.	1024M
deployment.resources.requests.cpu	The Kubernetes scheduler uses this information to decide which node to place the pod on. If there are no nodes with the specified amount of CPU resources, the pod won't be scheduled (and therefore not started).	0.1
deployment.resources.requests.memory	The Kubernetes scheduler uses this information to decide which node to place the pod on. If there are no nodes with the specified amount of memory, the pod won't be scheduled (and therefore not started).	256M
deployment.regionConfigurationsController.resources.limits.cpu	The kubelet enforces the limit so that the running container of the Region Configurations Controller utility microservice is not allowed to use more CPU resource than the set limit. In case the limit is crossed, the process would be throttled. If omitted, nothing is enforced.	None
deployment.regionConfigurationsController.resources.limits.memory	The kubelet enforces the limit so that the running container of the Region Configurations Controller utility microservice is not allowed to use more memory than the set limit. In case the limit is crossed, the process would be terminated with an out of memory (OOM) error. If omitted, nothing is enforced.	None
deployment.regionConfigurationsController.resources.requests.cpu	The Kubernetes scheduler uses this information to decide which node to place the Region Configurations Controller utility microservice Pod on. If there are no nodes with the specified amount of CPU resources, the pod won't be scheduled (and therefore started). If omitted, nothing is enforced.	None

Parameter	Desccription	Default
deployment.regionConfigurationsController.resources.requests.memory	The Kubernetes scheduler uses this information to decide which node to place the Region Configurations Controller utility microservice Pod on. If there are no nodes with the specified amount of memory, the pod won't be scheduled (and therefore started). If omitted, nothing is enforced.	None
deployment.autoscaling.horizontal.enabled	Enables or disables the *Horizontal Pod Autoscaler* mechanism, see Horizontal Pod Autoscaler (Kubernetes documentation).	256M
deployment.autoscaling.horizontal.maxReplicaCount	Upper limit for the number of Connectivity Proxy replicas to which the autoscaler can scale up. It should be higher than `deployment.replicaCount`.	2
deployment.autoscaling.horizontal.metrics.cpuAverageUtilization	Target value of the average CPU metric across all Connectivity Proxy pods, represented as a percentage of the requested value of the CPU for the pods.	80
deployment.autoscaling.horizontal.metrics.memoryAverageUtilization	Target value of the average memory metric across all Connectivity Proxy pods, represented as a percentage of the requested value of the memory for the pods.	80
deployment.autoscaling.vertical.enabled	Enables or disables the *Vertical Pod Autoscaler* mechanism. **⚠ Caution** To take effect, the *Vertical Pod Autoscaling* mechanism for the cluster must be enabled. See Vertical Pod Auto-Scaling for details (Github Gardener Kubernetes documentation).	false
deployment.autoscaling.vertical.updateMode	Mode in which the *Vertical Pod Autoscale* should operate. See Vertical Pod Autoscaler Quick start for details (Github Kubernetes documentation).	"Off"

Parameter	Desccription	Default
deployment.restartWatcher.enabled	Enables or disables the restart watcher which watches for changes in secrets and configmaps containing the label *connectivityproxy.sap.com/restart*, and restarts the Connectivity Proxy if needed. For more information, see Automatic Pickup on Resource Changes [page 2111].	false
ingress.annotations	Annotations to add to the Ingress resource.	{}
ingress.className	Class name of the ingress resource. The possible values are "nginx" and "istio".	"nginx"
ingress.healthcheck.tls.proxy.secretName	Name of the secret that is used for TLS handshake with the Connectivity Proxy. See External Health Checking [page 2101] for details.	None
ingress.healthcheck.tls.proxy.secretData.caCertificate	CA certificate part of the secret that is used for TLS handshake with the Connectivity Proxy. See External Health Checking [page 2101] for details.	None
ingress.healthcheck.tls.proxy.secretData.certificate	Certificate part of the secret that is used for TLS handshake with the Connectivity Proxy. See External Health Checking [page 2101] for details.	None
ingress.healthcheck.tls.proxy.secretData.key	Key part of the secret that is used for TLS handshake with the Connectivity Proxy. See External Health Checking [page 2101] for details.	None
ingress.healthcheck.tls.secretName	Name of the secret that is used for TLS handshake with external health checking application. See External Health Checking [page 2101] for details.	None
ingress.healthcheck.tls.secretData.certificate	Certificate part of the secret that is used for TLS handshake with external health checking application. See External Health Checking [page 2101] for details.	None
ingress.healthcheck.tls.secretData.key	Key part of the secret that is used for TLS handshake with external health checking application. See External Health Checking [page 2101] for details.	None
ingress.istio.namespace	Installation namespace of Istio. For more information, see Installing the Connectivity Proxy in Clusters with Istio [page 2114].	"istio-system"

Parameter	Desccription	Default
ingress.istio.gateway.selector	Selector that is configured in the Istio ingress gateway (used in the gateway configuration which is applied on the gateway pods). For more information, see Installing the Connectivity Proxy in Clusters with Istio [page 2114].	{ istio: ingressgateway }
ingress.istio.tls.ciphers	Allowlist of enabled TLS ciphers specified as an array of strings.	As determined by Istio: https://istio.io/latest/docs/reference/config/networking/gateway/#ServerTLSSettings , see the `cipherSuites` setting.
ingress.nginx.tls.ciphers	Enabled TLS ciphers specified in OpenSSL format.	As determined by nginx: https://nginx.org/en/docs/http/ngx_http_ssl_module.html#ssl_ciphers
ingress.timeouts.proxy.connect	Connect timeout (in seconds), for the Ingress controller.	10
ingress.timeouts.proxy.read	Read timeout (in seconds), for the Ingress controller. **⚠ Caution** Make sure the load balancer exposing the Ingress controller is also configured to allow longer times. For example, for NGINX with AWS, you need to add a special annotation for the LB service of the Ingress controller.	120
ingress.timeouts.proxy.send	Send timeout (in seconds), for the Ingress controller.	120
ingress.tls.proxy.secretName	Name of the secret that is used for TLS handshake with the Connectivity Proxy. See Mutual TLS [page 2097] for details.	None
ingress.tls.proxy.secretData.caCertificate	CA certificate part of the secret that is used for TLS handshake with the Connectivity Proxy. See Mutual TLS [page 2097] for details.	None
ingress.tls.proxy.secretData.certificate	Certificate part of the secret that is used for TLS handshake with the Connectivity Proxy. See Mutual TLS [page 2097] for details.	None
ingress.tls.proxy.secretData.key	Key part of the secret that is used for TLS handshake with the Connectivity Proxy. See Mutual TLS [page 2097] for details.	None

Parameter	Desccription	Default
ingress.tls.secretName	Name of the secret that is used for TLS handshake with the Cloud Connector.	None
ingress.tls.secretData.caCertificate	CA certificate part of the secret that is used for TLS handshake with the Cloud Connector.	None
ingress.tls.secretData.certificate	Certificate part of the secret that is used for TLS handshake with the Cloud Connector.	None
ingress.tls.secretData.key	Key part of the secret that is used for TLS handshake with the Cloud Connector.	None
ipv6.preference.enabled	If enabled, the JVM will prefer using IPv6 over IPv4 when connecting to outside resources like BTP endpoints.	false
transparent-proxy.enabled	Flag for enabling and disabling Transparent Proxy installation as a subchart.	false

Back to Top [page 2128]

Example

> ⚠ Caution
>
> The following example shows the full structure and does not represent a productive `values.yaml` file. Many properties listed here are mutually exclusive and should *not be used together* in real situations.

values.yaml structure example

```
chart:
  nameSuffix: "a string to append to resource names"

config:
  integration:
    auditlog:
      mode: "service"/"console"
      serviceCredentialsKey: "specifies the filename of the mounted auditlog
secret"
    connectivityService:
      serviceCredentialsKey: "specifies the filename of the mounted connectivity
service secret"
  servers:
    businessDataTunnel:
      enableTls: true/false
      externalHost: "myconnproxy.mycluster.com"
      externalPort: 443
      strictSniEnabled: true/false
      port: 8042
    proxy:
```

```
      authorization:
        oauth:
          allowedClientId: "the client id which is allowed as JWT issuer for
proxy authorization"
      http:
        allowRemoteConnections: true/false
        enabled: true/false
        enableProxyAuthorization: true/false
        port: 20003
      rfcAndLdap:
        allowRemoteConnections: true/false
        enabled: true/false
        enableProxyAuthorization: true/false
        port: 20001
      socks5:
        allowRemoteConnections: true/false
        enabled: true/false
        enableProxyAuthorization: true/false
        port: 20004
      clientConnection:
        writeTimeoutSeconds: 10

  jvm:
    errorFilePath: "directory for JVM error logs"
    heapDumpPath: "directory for JVM heap dumps"
    memory:
      maxHeapSize: 256m
      minHeapSize: 16m
  highAvailabilityMode: "off"/"subdomain"/"path"
  subaccountId: "id of the subaccount, for which the proxy is running"
  subaccountSubdomain: "subdomain of the subaccount, for which the proxy is
running"
  tenantMode: dedicated/shared
  serviceChannels:
    enabled: true/false

secretConfig:
  integration:
    auditlogService:
      secretData: "base64 encoded auditlog service key"
      secretName: "name of the secret resource, holding the auditlog secret"
    connectivityService:
      secretData: "base64 encoded connectivity_proxy service key"
      secretName: "name of the secret resource, holding the connectivity service
secret"
  servers:
    businessDataTunnel:
      secretData:
        caCertificate: "base64 encoded CA certificate"
        certificate: "base64 encoded certificate"
        key: "base64 encoded private key"
      secretName: "name of the secret"

deployment:
  affinity:
    podAntiAffinity:
      topologyKey: "topology.kubernetes.io/zone"
  image:
    pullPolicy: "IfNotPresent"
    pullSecret: "name of the pull secret fot the registry"
    registry: "the official SAP image repo"
    repository: "com.sap.cloud.connectivity/connectivity-proxy"
    tag: X.Y.Z
  replicaCount: 1
  regionConfigurationsController:
    resources:
      limits:
        cpu: 1
```

```
        memory: 1024M
      requests:
        cpu: 0.1
        memory: 256M
  resources:
    maxFileDescriptorCount: 64000
    limits:
      cpu: 1
      memory: 1024M
    requests:
      cpu: 0.1
      memory: 256M
  autoscaling:
    horizontal:
      enabled: true/false
      maxReplicaCount: 2
      metrics:
        cpuAverageUtilization: 80
        memoryAverageUtilization: 80
    vertical:
      enabled: true/false
      updateMode: "Off"

ingress:
  annotations:
    annotation1: value1
    annotation2: value2
  healthcheck:
    tls:
      proxy:
        secretData:
          caCertificate: "base64 encoded CA certificate"
          certificate: "base64 encoded certificate"
          key: "base64 encoded private key"
        secretName: "name of the secret"
      secretData:
        certificate: "base64 encoded certificate"
        key: "base64 encoded private key"
      secretName: "name of the secret"
  timeouts:
    proxy:
      connect: 10
      read: 120
      send: 120
  tls:
    proxy:
      secretData:
        caCertificate: "base64 encoded CA certificate"
        certificate: "base64 encoded certificate"
        key: "base64 encoded private key"
      secretName: "name of the secret"
    secretData:
      caCertificate: "base64 encoded CA certificate"
      certificate: "base64 encoded certificate"
      key: "base64 encoded private key"
    secretName: "name of the secret"
  nginx:
    tls:
      ciphers: "HIGH:!aNULL:!MD5"
  istio:
    tls:
      ciphers:
      - ECDHE-RSA-AES128-GCM-SHA256
      - ECDHE-RSA-AES256-GCM-SHA384

transparent-proxy:
  enabled: true/false
```

Back to Top [page 2128]

1.5.2.4 Additional Security Aspects

Considerations on security for the traffic flow and configuration of the Connectivity Proxy for Kubernetes.

Security of Traffic Flow *within* a Cluster

This section refers to the traffic flow between a workload from a Kubernetes cluster and the Connectivity Proxy running in the same cluster.

The traffic between a workload running in the cluster and the HTTP and LDAP/RFC proxies is not encrypted. The traffic to the SOCKS5 proxy can be SSL-encrypted if the client application initiates an SSL connection. The HTTP and LDAP/RFC proxies are disabled by default for a Connectivity Proxy installation. If you want to enable them, add the following configuration in the `values.yml` file:

```
config:
  servers:
    proxy:
      rfcAndLdap:
        enabled: true
      http:
        enabled: true
```

> ⓘ Note
>
> All business data that is transmitted between the Connectivity Proxy in the cluster and the Cloud Connector is sent over an SSL-encrypted tunnel.

Security of Traffic Flow *from Outside* a Cluster

This section refers to the traffic flow between the Cloud Connector and the Connectivity Proxy running in a Kubernetes cluster.

By default, the connections between the Cloud Connector and the Ingress load balancer in the Kubernetes cluster are SSL-encrypted. If you want to enable mutual TLS for the connections from the Ingress to the Connectivity Proxy or have ent-to-end mutual TLS between the Cloud Connector and the Connectivity Proxy, follow the procedures described in Mutual TLS [page 2097].

Security of Connectivity Proxy Configuration Data

This section refers to any security-sensitive configuration data which is required for the Connectivity Proxy.

The Connectivity Proxy installation requires some configuration data to be supplied via a Kubernetes secret.

By default, the Kubernetes secrets are stored as unencrypted base64-encoded strings and are transmitted in base64-encoded format, so they are basically accessible by persons with access to the cluster.

See the Kubernetes documentation for details on how to secure the usage of secrets in a cluster.

1.5.2.5 Sizing Recommendations

Find basic sizing guidance for the Connectivity Proxy for Kubernetes.

Sizing Options

The following table gives basic sizing guidance for different usage scenarios. The values listed in the **CPU** and **Memory**, and **Thread Pools** columns correspond to the properties you should define in the `values.yaml` file (for more information, see Configuration Guide [page 2128]):

Size (S/M/L)	CPU	Memory	Thread Pools
S: The expected load is small - request concurrency and size is low	• deployment.resources.requests.cpu: 0.1 • deployment.resources.limits.cpu: 1	• deployment.resources.requests.memory: 256 M • deployment.resources.limits.memory: 1024 M	• config.servers.businessDataTunnel.threadPoolSize: 20 • config.servers.proxy.threadPoolSize: 20
M: The expected load is medium - request concurrency and size is medium	• deployment.resources.requests.cpu: 1 • deployment.resources.limits.cpu: 2	• deployment.resources.requests.memory: 512 M • deployment.resources.limits.memory: 2048 M	• config.servers.businessDataTunnel.threadPoolSize: 20 • config.servers.proxy.threadPoolSize: 20

Size (S/M/L)	CPU	Memory	Thread Pools
L: The expected load is large - request concurrency and size is medium or high	• deployment.resources.requests.cpu: 2 • deployment.resources.limits.cpu: 4	• deployment.resources.requests.memory: 1024 M • deployment.resources.limits.memory: 4096 M	• config.servers.businessDataTunnel.threadPoolSize: 20 • config.servers.proxy.threadPoolSize: 20

Relation to Operational Modes

The Connectivity Proxy can operate in multiple Operational Modes [page 2093].

A major criteria is the type of tenancy used: single or multi-tenant. If multiple tenants are served, we recommend that you choose a slightly bigger size to make sure each tenant is served without precedence to any other tenant at the same time. Based on the chosen tenancy mode, the following general recommendations apply:

Tenancy Mode	Sizes to Consider
Single-Tenant	S, M
Multi-Tenant	M, L

> ### Note
>
> The above-mentioned sizing recommendations are related to the Connectivity Proxy software component, also acting as a tunnel server to which Cloud Connector instances connect. This means the Cloud Connector must be sized properly as well, see Sizing Recommendations [page 1649] (Cloud Connector).

> ### → Remember
>
> These sizing recommendation are just a direction point. There are many factors that affect the performance of the tunneling between Cloud Connector and Connectivity Proxy. They are closely related to the specifics of your scenario, expected regular and intermittent load, and so on.

1.5.3 Verification and Testing

Check if the Connectivity Proxy for Kubernetes is operational.

Once you have installed the Connectivity Proxy in your cluster, you can perform the following checks to verify it is running successfully.

> ### Note
>
> Before starting the checks, you must wait for a few seconds until all the components are started and can be consumed.

1. Execute the following command and verify that the status of the pod (pods) is **running**.

   ```
   kubectl run verify-connectivity-proxy --image=nginx --restart=Never -it --rm
   -- curl -v -x "connectivity-proxy:20003" "http://virtual:1234/"
   ```

2. Call the external health check endpoint of the Connectivity Proxy to verify it is returning a successful response. You can call the endpoint in a web browser or execute the following command from the command line:

   ```
   kubectl run verify-connectivity-proxy --image=nginx --restart=Never -it --rm
   -- curl -v -x "connectivity-proxy:20003" "http://virtual:1234/" -H "Proxy-
   Authorization: Bearer <proxy-authorization-jwt-value>"
   ```

   > ### Caution
   >
   > If the host of the Ingress in your cluster is configured with a self-signed certificate, add the `-k` flag to curl to disable the SSL certificate verification.

3. Make sure you have connected a Cloud Connector to your cloud subaccount. For more information, see Adding and Managing Subaccounts [page 1682].
   1. If you have deployed the Connectivity Proxy in a **single-tenant trusted mode** and have **enabled** the HTTP proxy in the `values.yml` file, execute the following command:

      ```
      kubectl run verify-connectivity-proxy --image=nginx --restart=Never -it --
      rm -- curl -v -x "connectivity-proxy:20003" "http://virtual:1234/"
      ```

   2. If you have deployed the Connectivity Proxy in a **single-tenant non-trusted** or **multi-tenant non-trusted mode** and have **enabled** the HTTP proxy in the `values.yml` file, execute the following command:

      ```
      kubectl run verify-connectivity-proxy --image=nginx --restart=Never -it
      --rm -- curl -v -x "connectivity-proxy:20003" "http://virtual:1234/" -H
      "Proxy-Authorization: Bearer <proxy-authorization-jwt-value>"
      ```

   3. If you have deployed the Connectivity Proxy in a **single-tenant trusted mode** and have **enabled** the HTTP proxy in the `values.yml` file, execute the following command:

      ```
      kubectl run verify-connectivity-proxy --image=nginx --restart=Never -it --
      rm -- curl -v -x "connectivity-proxy:20003" "http://virtual:1234/" -H "SAP-
      CP-Connectivity-Service-Token: <connectivity-service-jwt-value>"
      ```

      For more informaion on the process of fetching a JWT, see Consuming the Connectivity Service [page 1314].
      For more informaion on the different operational modes, see Operational Modes [page 2093].

   If the Connectivity Proxy is working as expected, you get one of these responses:

   - `Access denied to system virtual:1234`: If this was a valid request, make sure you expose the system correctly in your Cloud Connector.

> ### Note
>
> This response indicates that the business data from the cluster is successfully reaching your Cloud Connector, but the system `virtual:1234` is not exposed there.

- Response from your backend system if you have exposed it in the Cloud Connector.

  > ### Note
  >
  > This response confirms that the business data from the cluster is successfully reaching your backend system exposed in the Cloud Connector.

If you encounter problems with any of the above steps, please refer to Troubleshooting [page 2152] and Recommended Actions [page 2155] for further investigation.

1.5.4 Monitoring

Check operability, scenarios and metrics of the Connectivity Proxy for Kubernetes.

Basic Availability Monitoring

The basic availability check is the minimal verification you can do to make sure the Connectivity Proxy is alive. This check only shows if the process of the Connectivity Proxy is running and if it is able to handle requests. This check is also what is configured as the *liveness probe* for the Kubernetes deployment resource.

You can perform this check on-demand by invoking a simple HTTP GET request to the healthcheck endpoint of the Connectivity Proxy. If the response is `200 OK`, the check was *successful*. Any other response means the check *failed*. There are two ways to perform this:

- From **within the cluster**: Call `connectivity-proxy-tunnel:8042/healthcheck` and observe the result
- From **outside the cluster**: Call `https://healthcheck.<ingress host of the Connectivity Proxy>/healthcheck` and observe the result. See External Health Checking [page 2101] for more details.

Scenario Monitoring

On top of the availability monitoring of the component itself, it is useful to also monitor entire scenarios. This, however, cannot be provided out of the box by the Connectivity Proxy as it is specific to the way you use the component. Some options you can explore for this are:

- Monitor the failure rate of requests to the Connectivity Proxy.
- Monitor the amount of error logs in the Connectivity Proxy.

- Set up a scheduled execution of a full scenario that performs end-to-end verification, including the operations done via the Connectivity Proxy.

Metrics

The Connectivity Proxy supports the collection and exposure of metrics in *Prometheus* format.

The metrics are exposed through an HTTP endpoint on the port specified in the configuration (default: 9494).

To enable the collection and exposure of metrics, set `config.metrics.prometheus.enabled` to true.

```
config:
  metrics:
    prometheus:
      enabled: true
```

To obtain the collected metrics, send an HTTP GET request to the `/metrics` endpoint of the Connectivity Proxy.

```
curl http://connectivity-proxy-prometheus-metrics:9494/metrics
```

The metrics collected in the Connectivity Proxy without any additional configuration include:

- **connectivity_tunnel_opening_timeout_count_total**: This metric tracks the total number of tunnel openings that have timed out.
  By default, if a tunnel opening takes longer than 10 seconds, it is deemed to have timed out.
  This metric reflects the total count of such timeout occurrences. Additionally, a tunnel ID is included as a label in the metric.

  Sample Code

  ```
  # HELP connectivity_tunnel_opening_timeout_count_total Total number of
  tunnel openings that timed out
  # TYPE connectivity_tunnel_opening_timeout_count_total counter
  connectivity_tunnel_opening_timeout_count_total{tunnel_id="account:///
  83039ea2-e213-46e5-9723-fbfa2d0f7ee3"} 1.0
  ```

- **connectivity_business_data_tunnels_count**: This metric tracks the total number of active business data tunnels (connections between the Connectivity Proxy and different Cloud Connectors).

  Sample Code

  ```
  # HELP connectivity_business_data_tunnels_count Total number of active
  business data tunnels
  # TYPE connectivity_business_data_tunnels_count gauge
  connectivity_business_data_tunnels_count 5.0
  ```

The Connectivity Proxy also supports the collection of JVM-related metrics by setting `config.metrics.prometheus.jvmMetrics.enabled` to true:

```
config:
  metrics:
    prometheus:
      enabled: true
      jvmMetrics:
        enabled: true
```

This will expose the following JVM-related metrics:

- JVM Buffer Pool Metrics
- JVM Class Loading Metrics
- JVM Compilation Metrics
- JVM Garbage Collector Metrics
- JVM Memory Metrics
- JVM Memory Pool Allocation Metrics
- JVM Runtime Info Metric
- JVM Thread Metrics
- Process Metrics

The full list of JVM-related and process-related metrics with descriptions can be found in the Prometheus Java Client documentation .

There is also support for collecting metrics from JMX that are exposed by the Connectivity Proxy.

To enable this, set `config.metrics.prometheus.jmxMetrics.enabled` to true:

```
config:
  metrics:
    prometheus:
      enabled: true
      jmxMetrics:
        enabled: true
```

The JMX metrics collected by the Connectivity Proxy include:

- `connectivity_proxy_open_tunnel_call_health_status` - Health status of the open tunnel call.
  This metric indicates the health status of the open tunnel call.
  It reports a value of 1 if the open tunnel call is considered healthy and 0 if it is not.
  A failure may occur for various reasons, such as a network issue or an internal error.
  A call is considered a failure if either the last five open tunnel calls failed or if at least 60% of the last fifteen open tunnel calls failed.

```
# HELP connectivity_proxy_open_tunnel_call_health_status Open tunnel call health
status of the Connectivity Proxy
# TYPE connectivity_proxy_open_tunnel_call_health_status gauge
connectivity_proxy_open_tunnel_call_health_status 1.0
```

> **Note**
>
> The standard Prometheus format allows for integration with various monitoring and alerting tools, such as Grafana, Prometheus, Dynatrace, and others.

1.5.5 Using the Connectivity Proxy

Use the Connectivity Proxy for Kubernetes with different communication protocols and principal propagation (SSO).

Overview [page 2148]

TCP [page 2150]

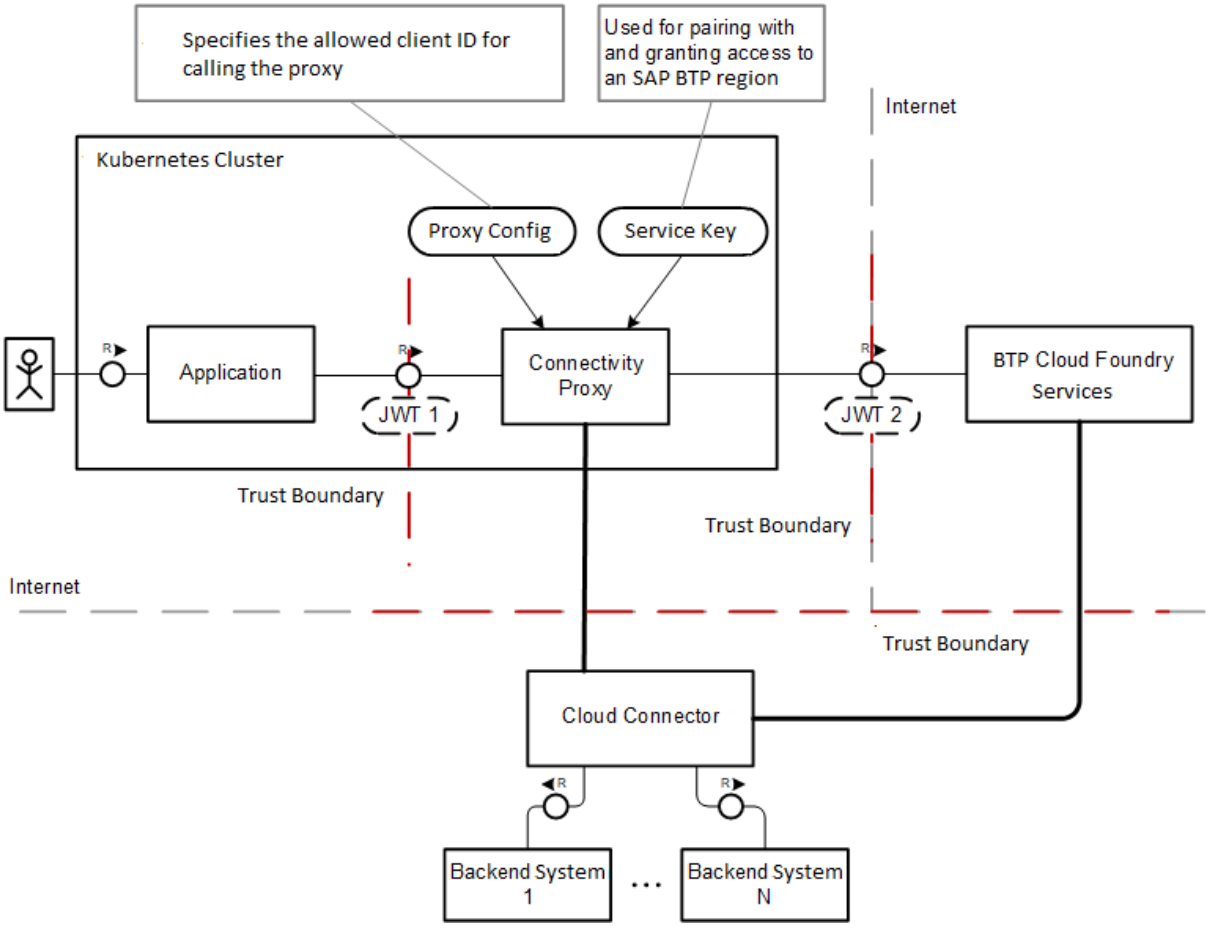
HTTP [page 2150]

Principal Propagation [page 2150]

Technical User Propagation [page 2151]

Overview

The Connectivity Proxy offers multiple proxy endpoints which are communication protocol-specific: TCP (via SOCKS5), HTTP, RFC (invoking ABAP functions) and LDAP. Depending on the operational mode, the usage may involve technical authorization when connecting to the proxy, see Operational Modes [page 2093].



Basically, the application needs to connect to the target on-premise system, and select the respective host and port as a protocol-standard proxy (with optional authorization) to get the request routed to the target on-premise system via the Cloud Connector.

> ### Note
>
> - The tunnel channel (between Cloud Connector and Connectivity Proxy) always uses TLS, that is, it is encrypted.
> - The actual connection from the application to the Connectivity Proxy may be encrypted or not, depending on the exact scenario used.
>
>   > ### Note
>   >
>   > If a TLS connection is attempted by the application, the SOCKS5 proxy endpoint must be used.
>
> - The actual connection from the Cloud Connector towards the on-premise system is established and controlled in the Cloud Connector. As a result, TLS can be enabled or disabled, regardless of the connection from the application to the Connectivity Proxy.

Back to Top [page 2]

TCP

TCP connectivity is achieved via the SOCKS5 proxy protocol. As the authorization is based on OAuth, we provide a custom authorization scheme for SOCKS5 that lets the application pass the required OAuth tokens and establish technical authorization with the Connectivity Proxy. For more information, see Using the TCP Protocol for Cloud Applications [page 1335].

Back to Top [page 2]

HTTP

Uses a standard HTTP proxy, just like using a corporate proxy to reach out to the Internet. For more information, see Authentication to the On-Premise System [page 1322].

```
curl --proxy "connectivity-proxy-host:20003" /
http://my-virtual-host:1234/my-path#my-fragment?my-query=my-value /
--header "Proxy-Authorization: Bearer <token>"
```

Back to Top [page 2]

Principal Propagation

Principal propagation, also known as *user propagation*, lets you perform single sign-on (SSO) authentication of the cloud user towards an on-premise system.

The cloud user identity is passed as a token represented by a JSON Web token (JWT). It is forwarded via the Connectivity Proxy to the Cloud Connector, which validates and further processes it to establish SSO with the on-premise system.

For more information, see Authenticating Users against On-Premises Systems [page 1701] and Set Up Trust [page 1702].

As of Connectivity Proxy release 2.1.1, support for principal propagation with IAS tokens is added.

Prerequisites:

- Cloud Connector 2.13 (or higher) must be used.
- The Cloud Connector must be connected to a subaccount, that is, configured with the IAS tenant issuing the tokens (for more information, see Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services).

Principal propagation is supported for both HTTP and RFC communication protocols and can be used in all Operational Modes [page 2093] of the Connectivity Proxy:

- **Single-tenant in trusted environment**
  The user should be propagated via `SAP-Connectivity-Authentication` header.
- **Multi-tenant in trusted environment**
  The user should be propagated only via `SAP-Connectivity-Authentication` header.
  The `SAP-CP-Connectivity-Service-Token` header *must* be sent for handling the tenant context.
- **Single or multi-tenant in non-trusted environment**
  The user should be propagated via either `Proxy-Authorization` or `SAP-Connectivity-Authentication` header, exclusively. If `Proxy-Authorization` is used, it must hold a JWT that results from a token exchange (see JWT Bearer Grant Type), to include both the context for technical access toward the Connectivity Proxy as well as the user identity.

  > **Note**
  >
  > If IAS tokens are used, they can *only* be provided via the `SAP-Connectivity-Authentication` header.

> **Note**
>
> If in a **non-trusted environment** the user context is provided via the `Proxy-Authorization` header, the `SAP-Connectivity-Authentication` *must not* be sent.

> **Caution**
>
> In principal propagation scenarios, you must omit the `Authorization` header.

For more information, see also XSUAA Token Client and Token Flow API .

Back to Top [page 2]

Technical User Propagation

Technical user propagation lets you perform single sign-on (SSO) authentication of a cloud technical user towards an on-premise system. It is very similar to *principal propagation*, but instead of forwarding the identity of business users, the identity of technical users is forwarded.

The technical user is represented by a token in form of a JSON web token (JWT) that is usually obtained via the `client_credentials` OAuth flow. It is forwarded via the Connectivity Proxy to the Cloud Connector, which validates and processes it to perform SSO with the on-premise system.

Technical user propagation is supported as of Connectivity Proxy version 2.6.1. As a prerequisite, Cloud Connector version 2.15 or higher must be used.

Technical user propagation is supported for both HTTP and RFC communication protocols and can be used in all Operational Modes [page 2093].

The token representing the technical user must be sent to the Connectivity Proxy via the `SAP-Connectivity-Technical-Authentication` HTTP request header.

> ⚠ Caution
>
> In *technical user propagation* scenarios, you must omit the `Authorization` header.

For more information, see Configuring Principal Propagation [page 1702] and XSUAA Token Client and Token Flow API .

Back to Top [page 2]

1.5.6 Troubleshooting

Find procedures to troubleshoot issues with the Connectivity Proxy for Kubernetes.

Get Logs of the Connectivity Proxy [page 2152]

Changing Log Level(s) of the Connectivity Proxy [page 2152]

Viewing Logger(s) of the Connectivity Proxy [page 2153]

Troubleshooting the Cloud Connector [page 2154]

Common Issues and Solutions [page 2154]

Get Logs of the Connectivity Proxy

As the Connectivity Proxy workload is represented as a standard Kubernetes StatefulSet , fetching the logs is done in the standard Kubernetes way. Example via `kubectl`:

```
kubectl logs statefulset/connectivity-proxy
```

Back to Top [page 2]

Changing Log Level(s) of the Connectivity Proxy

When the default logging level is not sufficient for debugging the issue you are facing, you can change the log level to get more insight about the problem.

> **Note**
>
> Changing a log level to something more verbose will have a negative impact on the performance of the Connectivity Proxy. Thus, we recommend that you do not keep such a log level for a long period of time.

Changing a log level is done without any downtime and requires no restarts. All you need to do is invoke a simple command on a pod of the Connectivity Proxy. Here are some examples:

Put all loggers on DEBUG (full command)

```
kubectl exec <pod> -it -- connectivity-proxy-operations logging change-log-level
DEBUG
```

Put all loggers on DEBUG (shortcut)

```
kubectl exec <pod> -it -- change-log-level DEBUG
```

Put only some loggers on DEBUG (shortcut)

```
kubectl exec <pod> -it -- change-log-level com.sap.core.connectivity.tunnel.k8s
DEBUG # Will affect all loggers with names, starting with
com.sap.core.connectivity.tunnel.k8s
```

Restore log level to default (shortcut)

```
kubectl exec <pod> -it -- change-log-level INFO
```

For more information, check the help text of the command:

```
kubectl exec <pod> -it -- change-log-level help
```

Back to Top [page 2]

Viewing Logger(s) of the Connectivity Proxy

List all loggers (full command)

```
kubectl exec <pod> -it -- connectivity-proxy-operations logging list-loggers
```

List all loggers (shortcut)

```
kubectl exec <pod> -it -- list-loggers
# or
```

```
kubectl exec <pod> -it -- get-loggers
```

List only some specific loggers (shortcut)

```
kubectl exec <pod> -it -- list-loggers com.sap.core.connectivity.tunnel.k8s
# Will show all loggers with names, starting with
com.sap.core.connectivity.tunnel.k8s
```

For more information, check the help text of the command:

```
kubectl exec <pod> -it -- list-loggers help
```

Back to Top [page 2]

Troubleshooting the Cloud Connector

For troubleshooting the Cloud Connector, see Monitoring, Logging, and Troubleshooting [page 2046] (Cloud Connector).

Back to Top [page 2]

Common Issues and Solutions

Issue	Solution
You use the HTTP proxy (port 20003) and get a 405 response.	Make sure the URL you are calling is `http://<virtual host>:<virtual port>`and not `https://<virtual host>:<virtual port>`.
You use the HTTP proxy (port 20003) and get a 407 response (in a non-trusted environment / proxy authorization turned on).	Make sure you set the `Proxy-Authorization` header with a value in the format "`Bearer <valid JWT token>`".
You use the HTTP proxy (port 20003) and get a 503 response stating that there is no Cloud Connector for your subaccount.	• Make sure the Cloud Connector you are targeting is still connected. • Make sure it's location ID matches the one used in the request. • Make sure the Cloud Connector is connected to the same subaccount, on whose behalf the token for the request is issued.

Issue	Solution
Out of nowhere, the Connectivity Proxy stops working / monitors indicate a failure (considered an outage if it happens in production).	See Recommended Actions [page 2155].
An SSL error is shown in the Cloud Connector when trying to send a request through the Connectivity Proxy.	Assuming the SSL error occurs during the call from the Cloud Connector to the public endpoint of the Connectivity Proxy, there are two options: • Error is due to a connection termination during the SSL handshake. In this case, check you intermediate components (proxies, firewalls, etc.). • The JVM on which the Cloud Connector is running does not trust the TLS certificate of the public endpoint of the Connectivity Proxy. In this case, import the certificate (or its CA) into the trust store of the JVM.

Back to Top [page 2]

Related Information

Recommended Actions [page 2155]

1.5.6.1 Recommended Actions

Find procedures to resolve an outage of the Connectivity Proxy for Kubernetes functionality.

> ⚠ Caution
>
> Before performing any of the steps below, make sure there is really an outage of the Connectivity Proxy and not just a general problem on the entire cluster.

Pre-Intervention Steps [page 2156]

Recovery Attempt [page 2156]

Request Help from SAP [page 2158]

Request Root Cause Analysis (RCA) [page 2158]

Pre-Intervention Steps

Before doing any restarts or modifications, it is important that you collect all relevant information.

1. Check the status of the Connectivity Proxy pods and collect the outcome. Example via `kubectl`:

   ```
   kubectl get pods | grep 'connectivity-proxy'
   ```

2. Perform basic availability monitoring from *within* the cluster, as described in Monitoring [page 2145], and collect the outcome. This can be done from an existing container or by spinning up a container for the check. Example:

   ```
   kubectl run perform-hc --image=curlimages/curl -it --rm --restart=Never --
   curl -vvv 'connectivity-proxy-tunnel:8042/healthcheck'
   ```

3. Perform basic availability monitoring from *outside* the cluster, as described in Monitoring [page 2145], and collect the outcome. This can be done from your browser or via REST clients like cUrl or Postman. Example:

   ```
   curl -vvv 'https://healthcheck.connectivitytunnel.ingress.mycluster.com/
   healthcheck'
   ```

4. Collect the logs of the Connectivity Proxy (see Troubleshooting [page 2152]).
5. Proceed to **Recovery Attempt**.

Back to Top [page 2155]

Recovery Attempt

The exact action to take here depends on the check result in steps 2 and 3 of section Pre-Intervention Steps [page 2156]. Choose one of the four options below, according to the outcome of your checks.

> → Tip
>
> Check if the cause of the outage might be insufficient resources. For more information, see Sizing Recommendations [page 2142]. If this is the possible cause, try scaling the Connectivity Proxy vertically and/or horizontally (see Configuration Guide [page 2128]).

- Option 1: Check *Succeeds* from within the Cluster and *Fails* from outside the Cluster [page 2156]
- Option 2: Check *Fails* from within the Cluster and *Succeeds* from outside the Cluster [page 2157]
- Option 3: Check *Fails* from within the Cluster and *Fails* from outside the Cluster [page 2157]
- Option 4: Check *Succeeds* from within the Cluster and *Succeeds* from outside the Cluster [page 2157]

Check *Succeeds* from within the Cluster and *Fails* from outside the Cluster

This indicates some sort of issue with the Ingress configuration. Some possible reasons:

- TLS certificate has expired.
- Something went wrong with the Ingress controller.

Such a situation is likely not an issue with the Connectivity Proxy. Next steps should be to stop following the steps here and shift focus towards the Ingress configuration and Ingress controller.

Back to Recovery Attempt [page 2156]

Back to Top [page 2155]

Check *Fails* from within the Cluster and *Succeeds* from outside the Cluster

This indicates some sort of issue with the exposure of the Connectivity Proxy to internal pods. Some possible reasons:

- Some unwanted network policy came into effect, preventing calls to the Connectivity Proxy from where you are executing them.

Such a situation is likely not an issue with the Connectivity Proxy. Next steps should be to stop following the steps here and shift focus towards cluster configurations and the network policies that affect access to the Connectivity Proxy.

Back to Recovery Attempt [page 2156]

Back to Top [page 2155]

Check *Fails* from within the Cluster and *Fails* from outside the Cluster

This indicates that the Connectivity Proxy itself is indeed having issues. Please perform the following steps:

1. Restart the Connectivity Proxy deployment. Example via `kubectl`:

   ```
   kubectl rollout restart statefulset/connectivity-proxy
   ```

2. Collect logs from the Connectivity Proxy after the restart completes.
3. Check if outage is still ongoing:
   1. If no, issue is resolved, proceed with Request Root Cause Analysis (RCA) [page 2158].
   2. If yes, issue is not resolved, proceed with Request Help from SAP [page 2158].

Back to Recovery Attempt [page 2156]

Back to Top [page 2155]

Check *Succeeds* from within the Cluster and *Succeeds* from outside the Cluster

This indicates that the Connectivity Proxy is currently considered operational, however it might still have trouble when used for real scenarios (depends on how you detect the outage).

1. Check if the outage is still ongoing:
   - If no, issue is resolved, proceed with Request Root Cause Analysis (RCA) [page 2158].
   - If yes, issue is not resolved, proceed with the next step.

2. Restart the Connectivity Proxy deployment. Example via `kubectl`:

```
kubectl rollout restart statefulset/connectivity-proxy
```

3. Collect logs from the Connectivity Proxy after the restart completes.
4. Check if outage is still ongoing:
   - If no, issue is resolved, proceed with Request Root Cause Analysis (RCA) [page 2158].
   - If yes, issue is not resolved, proceed with Request Help from SAP [page 2158].

Back to Recovery Attempt [page 2156]

Back to Top [page 2155]

Request Help from SAP

If you cannot resolve the issue and require help from SAP, follow this procedure:

1. Open an incident on the support component (see Connectivity Support [page 2362]) for the Connectivity Proxy (with the appropriate priority and impact stated).
2. Provide all the collected information in the incident, including all the logs, timestamps of the events that occurred, summary of the taken actions, version of the Connectivity Proxy you are using, and so on.
3. Engage your SAP contacts to help with this.
4. Continue working in parallel to identify as much information as possible or to fine a temporary measure to mitigate the outage.

Back to Top [page 2155]

Request Root Cause Analysis (RCA)

Once the issue is resolved, the next step is figuring out what exactly caused the issue and if there is something that can be done to prevent it from happening in the future. Follow this procedure for requesting RCA for the issue you experienced:

1. Open an incident on the support component for the Connectivity Proxy (see Connectivity Support [page 2362]).
2. Provide all the collected information in the incident, including all the logs, timestamps of the events that occurred, summary of the taken actions, version of the Connectivity Proxy you are using, and so on.
3. The incident will be handled according to the SAP incident SLAs.

Back to Top [page 2155]

1.5.7 Frequently Asked Questions

Answers to the most common questions about the Connectivity Proxy for Kubernetes.

When using one of the two untrusted operational modes, what is the purpose of the allowedClientId property? What token should I provide in the Proxy-Authorization header?

The `Proxy-Authorization` header serves as a way for workloads calling the proxy to authenticate against it. To do this, they need to provide this header with a JSON Web token (JWT) as value.

To accept the JWT, it must be issued by the OAuth client, specified via the `allowedClientId`. By configuring the `allowedClientId`, you determine which OAuth client protects the proxy endpoints.

The OAuth client must be XSUAA-based.

How is the Proxy-Authorization header verified?

There are several aspects that are being verified:

- The signature is being verified by calling XSUAA to get the public keys for the tenant, on behalf of which the JWT is issued, and using those keys to check if it is valid.
- The token validity is also verified and expired tokens are rejected.
- Also, we allow only tokens issued by a specified client ID (via the `allowedClientId` configuration).

Why do I need the connectivity:connectivity_proxy service key?

The Connectivity Proxy is a distributed software component that needs to connect to an instance of the central Connectivity service to function.

This pairing is achieved via a `connectivity:connectivity_proxy service` key, which contains both routing information and credentials for this pairing.

Why do I need a public endpoint for the Connectivity Proxy?

To preserve the integrity of an on-premise landscape and not expose anything from there to the Internet, the flow for establishing the connection between Connectivity Proxy and Cloud Connector is initiated by the Cloud Connector.

The public endpoint is used by the Cloud Connector to call the Connectivity Proxy and enable the data exchange.

What is the relation of the Connectivity Proxy to the Connectivity service in SAP BTP?

The Connectivity Proxy is a distributed component that must be paired to an instance of the Connectivity service in SAP BTP in order to function.

Cloud Connectors would still connect to the Connectivity service on SAP BTP and the Connectivity Proxy will make use of those Cloud Connectors via the established pairing.

What is the relation of the Connectivity Proxy to the Destination service in SAP BTP?

There is no dependency or tight integration.

You can use the Destination service to store and retrieve on-premise destination configurations which can then be used to construct a request to on-premise systems through the Connectivity Proxy.

Are there any client libraries that I can use with the Connectivity Proxy?

Please check Using the Connectivity Proxy [page 2148].

When do I need a subscription to the connectivity:connectivity_proxy instance?

Please check Connectivity Service [page 2109].

Can I port a cloud SDK application from the Cloud Foundry environment to Kubernetes and use the Connectivity Proxy?

It is a bit clunky, but yes.

If you set up the Connectivity Proxy in an untrusted operational mode, the way the SDK works is well suited for it.

However, since the SDK is created with the Cloud Foundry environment in mind, you would need to simulate the `VCAP_SERVICES` environment to get it working.

Is there an equivalent to the lite plan from the Cloud Foundry environment? Is the lite plan relevant for the Connectivity Proxy?

The lite plan is only relevant for the Cloud Foundry environment.

There is no lite plan for the Connectivity Proxy. Instead, you use an XSUAA-based OAuth client of your choice to protect the Connectivity Proxy, and use it to issue tokens.

1.5.8 Connectivity Proxy in the Kyma Environment

Find information on installing Connectivity Proxy in the the Kyma environment, providing easier installation and setup.

Prerequisites

- The service plan *"connectivity_proxy"* of the *"connectivity"* service is assigned to your subaccount as an entitlement.
  For more information, see Configure Entitlements and Quotas for Subaccounts.

  > **Note**
  >
  > For subaccounts created after the February 15, 2024, this entitlement is assigned automatically.

- You have added the *btp-operator* module.
  For more information, see Add and Delete a Kyma Module.

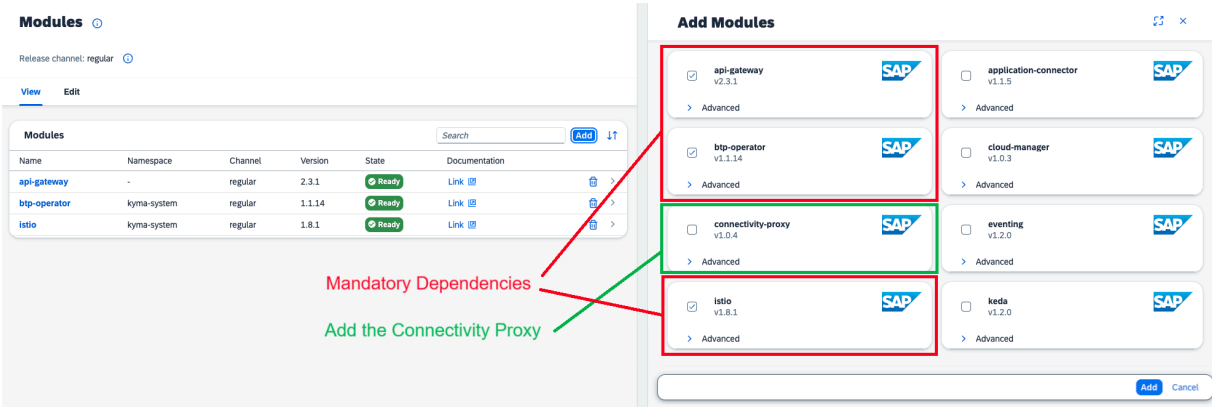
Context

The *connectivity-proxy* Kyma module installs the needed components to establish a secure tunnel between the Kyma environment and the systems in your on-premise network, exposed via Cloud Connector [page 1602]. The module is based on the Connectivity Proxy for Kubernetes [page 2088]. It supports both *on-premise-to-cloud* and *cloud-to-on-premise* scenarios.

Add the Connectivity Proxy module

The Connectivity Proxy is a standard Kyma module. You can add the module as described in Add and Delete a Kyma Module.

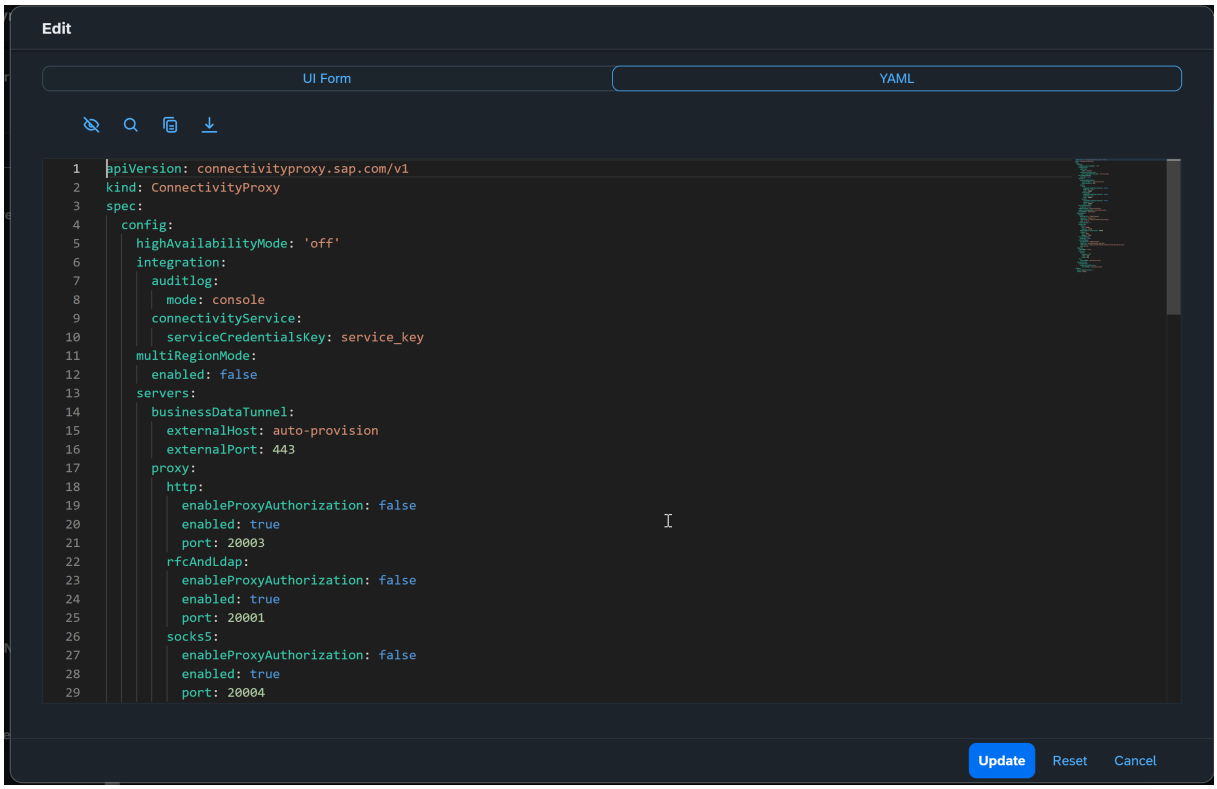
> **Note**
>
> The modules `api-gateway`, `btp-operator` and `istio` are required dependencies. They must be added to make the Connectivity Proxy work properly.



The Kubernetes resource that facilitates this installation in the background is a `ConnectivityProxy` resource. This is a custom resource, defined by the module. It holds the configuration for the Connectivity Proxy components.

For more information, see Configuration Guide [page 2128].

> **Note**
>
> Not all configuration options of the Connectivity Proxy are supported by the Kyma module.
>
> See Limitations [page 2163] below for more details.



> **Caution**
>
> **Do not** create additional resources of type `ConnectivityProxy`. Only one Connectivity Proxy installation per cluster is supported.

Result

The module will be added and will result in an installation of the Connectivity Proxy for Kubernetes [page 2088] and its supporting workloads.

As part of this installation, a `ServiceInstance` and a `ServiceBinding` resource will be created in the cluster. These are *BTP Operator* resources which result in the creation of a *Connectivity* service instance with service plan *connectivity_proxy*, and in a service binding in your subaccount. This is needed to pair the Connectivity Proxy with the SAP BTP Connectivity service.

For more information, see Connectivity Service [page 2109].

> ⚠ Caution
>
> Do not interact with the created service instance and service binding, as they are only meant for usage by the Connectivity Proxy. Any modification may cause a loss of functionality.

The installation also exposes the public facing endpoint of the Connectivity Proxy via the built-in Istio Ingress controller (TLS certificates are generated automatically during this process). This endpoint is used by the Cloud Connector to reach the proxy.

The `Ingress` endpoint is propagated transparently for *cloud-to-on-premise* scenarios.

For *on-premise-to-cloud* connections, you must look it up manually. You can find it in the created `Gateway` resource of the *kyma-system* namespace. Look for the *Istio* `Gateway` resource with name `connectivity-proxy-tunnel`. The exact auto-provisioned host will be shown in the `Servers` section.

Capabilities

If no settings are modified, the following specifics and features are available in the *connectivity_proxy* Kyma module:

- All proxy servers are enabled (HTTP, RFC, LDAP, SOCKS5).
- The Service Channels: On-Premise-to-Cloud Connectivity [page 2112] feature is enabled.
- Operational mode is single tenant, trusted [page 2095].
- Automatic Pickup on Resource Changes [page 2111] is enabled.
- High Availability [page 2105] is set to *off* by default, but can be activated in "path" mode.
- Auto-scaling (only applicable if *High Availability* is turned on) is disabled by default, but can be activated.

Limitations

The following features of the Connectivity Proxy are not available via the *connectivity-proxy* Kyma module:

- Multi-Region Mode [page 2116]
- Support for Ingress controllers other than Istio

Consume an On-Premise System Through the Connectivity Proxy

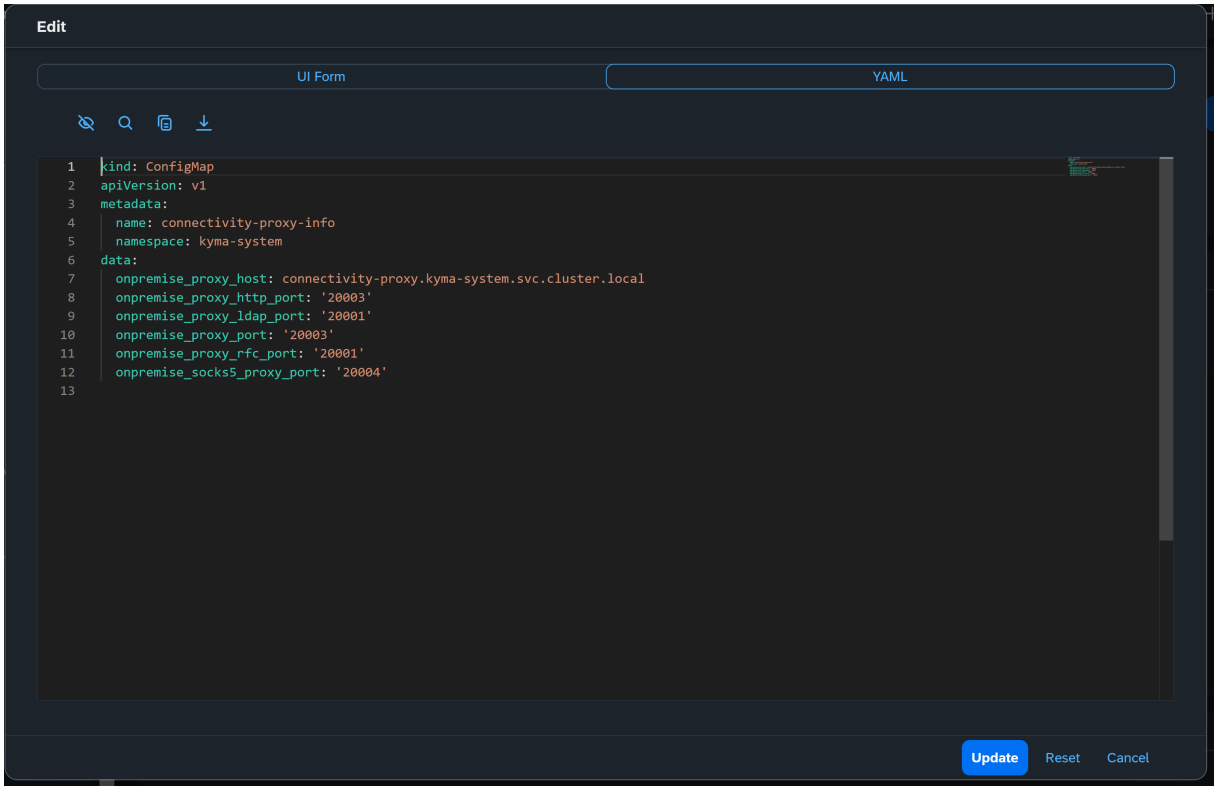
Integration with the Transparent Proxy

We recommend that you consume an on-premise system through a combination of SAP BTP Connectivity software components and services. While the Connectivity Proxy is used to establish the connection to the on-premise system/VPC, the Transparent Proxy provides an abstraction enabling seamless consumption of the target system. Transparent Proxy and Connectivity Proxy integrate automatically with each other when both are available as modules in a Kyma instance.

For more information, see Transparent Proxy in the Kyma Environment [page 2332].

Standalone Usage of the Connectivity Proxy

As part of the installation, a special `ConfigMap` will be created in the *kyma-system* namespace, called *connectivity-proxy-info*. It contains the host of the Connectivity Proxy and all its proxy ports.



To use the Connectivity Proxy, you must set the appropriate host and port as proxy configuration.

For more information, see Using the Connectivity Proxy [page 2148].

> ⚠ Caution
>
> Every workload using the Connectivity Proxy to call the on-premise system must have the Istio sidecar proxy injection enabled. Otherwise, the Connectivity Proxy does not work correctly.

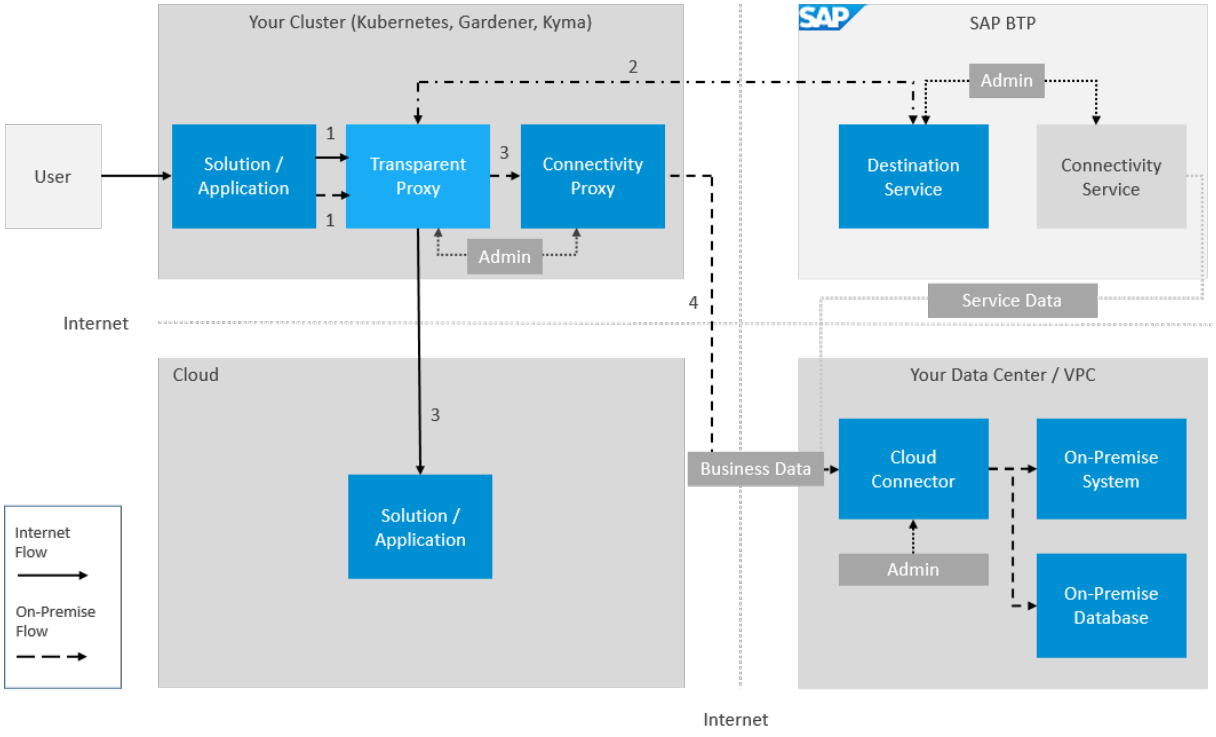
1.6 Transparent Proxy for Kubernetes

Use the Transparent Proxy for Kubernetes to connect workloads on a Kubernetes cluster to Internet and on-premise applications.

> ⓘ Note
>
> The on-premise use cases described in this guide are also applicable to virtual private cloud (VPC) environments.

The Transparent Proxy routes to SAP BTP destinations configured in the Destination service. On-premise applications must be exposed via Cloud Connector [page 1602] (installed in the same network right next to the on-premise system) and Connectivity Proxy for Kubernetes [page 2088] (installed in the Kubernetes cluster).

The Transparent Proxy is delivered as Docker images and a Helm chart. You need to run the image on your Kubernetes cluster with appropriate configurations. The Helm chart simplifies the installation process.



The Transparent Proxy handles HTTP, LDAP, MAIL (SMTP, IMAP, POP3), and TCP protocols for both Internet and on-premise destinations. It also supports IPv4, IPv6 and Dual-Stack client workloads, Kubernetes environments, and target systems.

The Transparent Proxy creates a Kubernetes service for each SAP BTP destination configuration exposed by a *destination custom resource*.

When an application tries to reach the desired system defined as a destination configuration, the Transparent Proxy:

- Intercepts the traffic (1)
- Calls the SAP BTP Destination service (or uses the cached response for the destination value) to obtain the configuration for the requested destination (2)
- Enriches the request by providing a handshake with the Connectivity Proxy and authentication mechanism, and routes the traffic to the desired remote system:
  - Directly for Internet-accessible solutions/applications/systems (3)
  - Via the Connectivity Proxy (3) and the Cloud Connector (4) for on-premise systems

Related Information

Using the Transparent Proxy [page 2166]
Integration with SAP BTP Connectivity [page 2247]
Lifecycle Management [page 2252]
Monitoring [page 2294]
Troubleshooting [page 2297]
Recommended Actions [page 2326]
Resilience [page 2328]
Verification and Testing [page 2330]
Transparent Proxy in the Kyma Environment [page 2332]
Concepts [page 2336]
Local Development [page 2356]
Transparent Proxy Operator [page 2358]
Security Considerations [page 2359]

1.6.1 Using the Transparent Proxy

Use the Transparent Proxy for Kubernetes in different SAP BTP communication scenarios.

The Transparent Proxy lightens the way your Kubernetes workloads connect to remote systems using BTP destinations of type *Internet* and *OnPremise*. It provides authentication, principal propagation, SOCKS5 handshake, and easy access to the destination target systems by exposing them as Kubernetes Services .

To target a given destination for handling by the Transparent Proxy, you should create a custom Kubernetes resource of type *destinations.destination.connectivity.api.sap* that references the given destination. If you do not create a custom Kubernetes resource of type destinations.destination.connectivity.api.sap for a given destination, the destination in question will not be handled by the Transparent Proxy.

> **Note**
>
> There should be no Kubernetes service in the namespace of the destination custom resource or in the namespace where the Transparent Proxy is installed, with a name equal to the destination custom resource. The Transparent Proxy handles the creation of Kubernetes services based on the information stored in these custom resources.

Internet Connectivity

The Transparent Proxy handles the HTTP(s) communication protocol for Internet destinations. The application developer needs to create a BTP destination of Type “HTTP” and ProxyType “Internet”, for example:

Internet Destination

Sample Code

```
{
    "Name": "example-dest-client-cert",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": <application-url>,
    "Authentication": "ClientCertificateAuthentication",
    "KeyStorePassword": <key-store-password>,
    "KeyStoreLocation": <key-store-location>
}
```

To target the destination with the name “example-dest-client-cert” for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice:

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: example-dest
spec:
  destinationRef:
    name: "example-dest-client-cert"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Once done, the application could start consuming the destination from within the Kubernetes cluster, for example:

Sample Code

```
curl example-dest.<destination-cr-namespace>
```

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

On-Premise Connectivity

In order to use on-premise connectivity, you should have installed the Connectivity Proxy and a Cloud Connector.

The Connectivity Proxy is a Kubernetes component that connects workloads running on a Kubernetes cluster to on-premise systems, which are exposed via the Cloud Connector. The Connectivity Proxy must be paired to an SAP BTP region to grant access to the Cloud Connectors connected to that region. The SAP BTP domain model (subaccounts) is used to target a particular Cloud Connector.

1. Install the Connectivity Proxy either via Helm or as a Kyma module.
   For more information, see Operations via Helm [page 2123] and Connectivity Proxy in the Kyma Environment [page 2160].
2. Install the Cloud Connector.
   For more information, see Installation [page 1608].
3. Connect your subaccount to the Cloud Connector and expose your systems.
   For more information, see Configuration [page 1669].
4. Integrate your Connectivity Proxy in the Transparent Proxy configuration:

   Sample Code

   ```
   integration:
     connectivityProxy:
       serviceName: connectivity-proxy.<connectivity-proxy-namespace>
   ```

Note

To use the Cloud Connector with a specified *Location ID*, you have two options:

- Pass the Location ID via the HTTP header "SAP-Connectivity-SCC-Location_ID"
- Use the property "CloudConnectorLocationId" in the referenced SAP BTP destination.

If both methods are used, the value in the HTTP header will take precedence.

For non-HTTP SAP BTP destinations, only the "CloudConnectorLocationId" property is supported.

Caution

For installation in Kyma environment using the Transparent Proxy module, the integration between the Transparent Proxy and the Connectivity Proxy is automatic.

Then the application developer needs to create a BTP destination of `ProxyType` “OnPremise”, for example:

Sample Code

```
{
    "Name": "example-dest-on-prem",
    "Type": "HTTP",
    "ProxyType": "OnPremise",
    "URL": <virtual-host>:<port>,
    "Authentication": "PrincipalPropagation"
}
```

To target the destination with the name “example-dest-onprem” for handling by the Transparent Proxy, you should create a destination custom resource in a namespace of your choice:

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
```

```
  name: example-dest
spec:
  destinationRef:
    name: "example-dest-on-prem"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Once done, the application could start consuming the destination from within the Kubernetes cluster, for example:

Sample Code

```
curl example-dest.<destination-cr-namespace>
```

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

Destination Fragments

Additionally, you can use destination fragments.

For more information, see Extending Destinations with Fragments [page 1590].

Sample Code

```
{
    "FragmentName": "example-fragment",
    "URL": <application-url>
}
```

The destination fragment can be referenced in two ways: statically through a Destination Custom Resource [page 2342], or dynamically by using Destination Gateway (Dynamic Lookup of Destinations) [page 2349].

You can specify whether a fragment is optional or not by using the destination fragments optionality feature. The destination fragment optionality can be referenced in two ways: statically through a Destination Custom Resource [page 2342], or dynamically by using Destination Gateway (Dynamic Lookup of Destinations) [page 2349].

Destination Chaining

The destination chaining can be referenced in two ways: statically through a Destination Custom Resource [page 2342], or dynamically by adding the related chain headers during the HTTP request.

For more information, see Destination Chaining [page 1538].

Troubleshooting

In case of errors, always look for detailed error messages and hints in 'x-error-message', 'x-error-origin', 'x-proxy-server' and 'x-internal-error-code'. Also, you could use 'x-request-id' as correlation ID to find more about the error in the Transparent Proxy HTTP pods.

For more information, see Troubleshooting [page 2297].

Related Information

HTTP Destinations [page 2170]
Mail Destinations [page 2211]
LDAP Destinations [page 2241]
TCP Destinations [page 2244]

1.6.1.1 HTTP Destinations

Configure HTTP destinations for the Transparent Proxy for Kubernetes.

The Transparent Proxy simplifies access to target systems which use HTTP protocol. It efficiently handles HTTP traffic for both Internet-based and on-premise destinations.

HTTP Authentication

The Transparent Proxy enriches your request by adding credentials configured in the destination configuration. Depending on the authentication of the destination configuration, the Transparent Proxy propagates them via `Authorization` or `SAP-Connectivity-Authentication` header. For more information, see Configuring Authentication [page 1351].

Mandatory Destination Configuration Fields

Property	Description
Name	Destination name
Type	Destination type. Use HTTP for all HTTP(S) destinations.
URL	URL of the application. For on-premise destinations, use the virtual URL of the protected on-premise application.
Authentication	One of the listed Authentication types here.
ProxyType	Internet or OnPremise

Additional Destination Configuration Fields

Property	Description
URL.connectionTimeoutInSeconds	The period of time in which the HTTP client expects a confirmation after it initiated a new TCP connection. It must be in the range of 1 second to 5 minutes (300 seconds). If you do not use it, its default is 5 seconds. If you decide to use it, the total timeout of the initial request could be the sum of the timeout configured and the time needed to obtain the destination configuration data from the Destination service. You could set custom timeouts for calls to Destination service and Connectivity Proxy as Helm properties. See the Configuration Guide [page 2265].
URL.socketReadTimeoutInSeconds	The period of time the HTTP client will wait for receiving a response (data) from the server. It must be in the range of 1 second to 12 hours (43200 seconds). If you do not use it, its default is 30 seconds. If you decide to use it, the total timeout of the initial request could be the sum of the timeout configured and the time needed to obtain the destination configuration data from the Destination service. You could set custom timeouts for calls to Destination service and Connectivity Proxy as Helm properties. See the Configuration Guide [page 2265].

Property	Description

URL.headers.<header-key>	A static key prefix is used as a namespace grouping of URL's HTTP headers which values to be sent to the target endpoint. For each HTTP header's key must be added a ' URL.headers' prefix separated by dot-delimiter. For example:
	
	Sample Code
	`{`
	`  ...`
	`  "URL.headers.<header-key-1>" : "<header-value-1>",`
	`  ...`
	`  "URL.headers.<header-key-N>": "<header-value-N>",`
	`}`
	
	Caution
	If there are duplicate header keys passed from the request and contained in the destination, they are both added to the request against the target system.
	
	Example:
	
	Sample Code
	`{`
	`/// other destination properties`
	`  "URL.headers.foo": "bar"`
	`}`
	
	Sample Code
	`curl -H "foo: bar2" targetsystem -vvv`
	`*   Trying <some ip>...`
	`* Connected to targetsystem (<some ip>) port 80 (#0)`
	`> GET / HTTP/1.1`
	`> Host: targetsystem`
	`> User-Agent: curl/7.84.0`
	`> foo: bar2`
	`> foo: bar`

Property	Description

URL.queries.<query-key>	A static key prefix is used as a namespace grouping of URL's query parameters which values to be sent to the target end-point. For each query parameter's key must be added to a 'URL.queries' prefix separated by dot-delimiter. For example:
	
	Sample Code
	`{ ... "URL.queries.<query-key-1>" : "<query-value-1>", ... "URL.queries.<query-key-N>": "<query-value-N>", }`
	
	Caution
	If there are duplicate query parameter keys passed from the request and contained in the destination, they are appended to the request against the target system.
	
	Example:
	
	Sample Code
	`{ /// other destination properties "URL.queries.foo": "bar" }`
	
	Sample Code
	`curl targetsystem?foo=bar2 // will result to the following final URL : targetsystem? foo=bar2&foo=bar`

Related Information

Basic Authentication [page 2174]
Client Certificate Authentication [page 2175]
No Authentication [page 2178]
OAuth [page 2179]

User Propagation [page 2203]
SAML Assertion Authentication [page 2209]

1.6.1.1.1 Basic Authentication

The Transparent Proxy handles the HTTP communication protocol for both Internet and on-premise destinations protected with *Basic Authentication*.

Prerequisites

To integrate this functionality, you must create an SAP BTP destination. This destination should have `Type` "HTTP" and `Authentication` "BasicAuthentication", for example:

Basic Authentication Destination

Sample Code

```
{
    "Name": "example-dest-basic",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": <application-url>,
    "Authentication": "BasicAuthentication"
    "User": <username>,
    "Password": <password>
}
```

To target the destination with the name "example-dest-basic-auth" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for Basic Authentication

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-basic"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Consumption

For consumption of destination with authentication type BasicAuthentication no additional Authorization headers have to be provided to the Transparent Proxy as all necessary data for authorization is in the destination.

> **Note**
>
> `<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

Basic Authentication Pseudocode Snippet

> **Sample Code**

```
function callHttpDestinationBasic() {
  url = '<destination-cr-name>.<destination-cr-namespace>'
  headers = {
            // X-Tenant-Subdomain is required only when Transparent Proxy is
in shared tenant mode
            'X-Tenant-Subdomain': '<tenant-where-destination-is-located>'
        };

  response = http.get(url, headers)
  print(response)
}
```

Basic Authentication Bash Snippet

> **Sample Code**

```
# Call HTTP destination protected with Basic authentication:
curl <destination-cr-name>.<destination-cr-namespace> -H 'X-Tenant-Subdomain:
<tenant-where-destination-is-located>'
```

Related Information

Client Certificate Authentication [page 1355]

1.6.1.1.2 Client Certificate Authentication

The Transparent Proxy handles HTTP communication protocol for both Internet and on-premise destinations protected with *Client Certificate Authentication*.

For this authentication type, the Transparent Proxy manages the client certificate addition (client certificates and CA certificates) by getting them from the destination and adding them to the HTTP client used to establish the communication between the client and server.

Prerequisites

To integrate this functionality, you must create an SAP BTP destination. This destination should have `Type` "HTTP" and `Authentication` "ClientCertificateAuthentication", for example:

Client Certificate Destination

Sample Code

```
{
    "Name": "example-dest-client-certificate",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": <application-url>,
    "Authentication": "ClientCertificateAuthentication"
    "KeyStorePassword": <key-store-password>,
    "TrustStoreLocation": <trust-store-location>,
    "KeyStoreLocation": <key-store-location>
}
```

Restriction

Limitations for usage:

- Supported cert store types are p12, pfx, pem, der, cer, crt.
- Only RSA pkcs1(starting with header RSA PRIVATE KEY and having headers with encryption information) keys are currently supported for decryption from PEM format. The following encryption algorithms are supported DES-CBC, DES-EDE3-CBC, AES-128-CBC, AES-192-CBC, and AES-256-CBC. All key types should be supported if no encryption is present.
- KeyStore.Source=ClientProvided is not supported as the Transparent Proxy can manage only certificates stored in the Destination Service.

For more information, see *Client Certificate Authentication*.

To target the destination with the name "example-dest-client-certificate" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for a Client Certificate Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
```

```
spec:
  destinationRef:
    name: "example-dest-client-certificate"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Consumption

For consumption of destination with authentication type *ClientCertificateAuthentication*, no additional `Authorization` headers need to be provided to the Transparent Proxy as all necessary data for authorization is in the destination.

> **Note**
>
> `<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

Client Certificate Authentication Pseudocode Snippet

Sample Code

```
function callHttpsDestinationClientCertificateAuthentication() {
  url = '<destination-cr-name>.<destination-cr-namespace>'
  headers = {
            // X-Tenant-Subdomain is required only when Transparent Proxy is
in shared tenant mode
            'X-Tenant-Subdomain': '<tenant-where-destination-is-located>'
        }

  response = http.get(url, headers)
  print(response)
}
```

Client Certificate Authentication Bash Snippet

Sample Code

```
# Call HTTP destination protected with Client Certificate authentication:
curl <destination-cr-name>.<destination-cr-namespace> -H 'X-Tenant-
Subdomain': '<tenant-where-destination-is-located>'
```

Related Information

Client Certificate Authentication [page 1355]

1.6.1.1.3 No Authentication

The Transparent Proxy handles HTTP and TCP communication protocols for both Internet and on-premise destinations without authentication.

Prerequisites

To integrate this functionality, you must create an SAP BTP destination. This destination should have `Type` "HTTP" and `Authentication` "NoAuthentication", for example:

No Authentication Destination

Sample Code

```
{
    "Name": "example-dest-no-auth",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": <application-url>,
    "Authentication": "NoAuthentication"
}
```

To target the destination with the name "example-dest-basic-auth" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for No Authentication Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-no-auth"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Consumption

> **Note**
>
> `<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

No Authentication Pseudocode Snippet

> **Sample Code**

```
function callHttpDestinationNoAuth() {
  url = '<destination-cr-name>.<destination-cr-namespace>'
  headers = {
            // X-Tenant-Subdomain is required only when Transparent Proxy is
in shared tenant mode
            'X-Tenant-Subdomain': '<tenant-where-destination-is-located>'
        };

  response = http.get(url, headers)
  print(response)
}
```

No Authentication Bash Snippet

> **Sample Code**

```
# Call HTTP destination without authentication:
curl <destination-cr-name>.<destination-cr-namespace> -H 'X-Tenant-Subdomain:
<tenant-where-destination-is-located>'
```

Related Information

Client Certificate Authentication [page 1355]

1.6.1.1.4 OAuth

Use OAuth authentication for the Transparent Proxy for Kubernetes.

Related Information

IAS-Signed SAML Bearer Assertion [page 2180]
OAuth Authorization Code Authentication [page 2183]
OAuth Client Credentials Authentication [page 2186]

OAuth JWT Bearer Authentication [page 2188]
OAuth Password Authentication [page 2190]
OAuth Refresh Token Authentication [page 2192]
OAuth SAML Bearer Assertion Authentication [page 2195]
OAuth Token Exchange Authentication [page 2197]
OAuth User Token Exchange Authentication [page 2200]

1.6.1.1.4.1 IAS-Signed SAML Bearer Assertion

The Transparent Proxy supports Token Retrieval Using IAS-Signed SAML2.0 Assertions [page 124] for the HTTP communication protocol for both *Internet* and *On-Premise* destinations.

Preparation

Provider Destination

Sample Code

```
{
    "Name": "samlAssertionProviderDest",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": "<application-url>",
    "Authentication": "OAuth2TokenExchange",
    "tokenServiceURLType": "Dedicated",
    "tokenServiceURL": "<ias-token-service-url>",
    "clientId": "<ias-client-id>",
    "clientSecret": "<ias-client-secret>",
    "tokenServiceURL.queries.requested_token_type":
"urn:ietf:params:oauth:token-type:saml2",
    "tokenServiceURL.queries.resource":
"urn:sap:identity:application:provider:name:<ias-saml-dependency-name>"
}
```

The example below uses XSUAA on region cf-ap10 as a token service:

Consumer Destination

Sample Code

```
{
    "Name": "samlAssertionConsumerDest",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": "<application-url>",
    "Authentication": "OAuth2SAMLBearerAssertion",
    "tokenServiceURLType": "Dedicated",
    "tokenServiceURL":
"https://<tenant-subdomain>.authentication.ap10.hana.ondemand.com/oauth/token/
alias/<tenant-subdomain>.cf-ap10",
```

```
    "clientKey": "<token-service-user>",
    "tokenServiceUser": "<token-service-user>",
    "tokenServicePassword": "<token-service-password>",
    "SAMLAssertionProvider": "ClientProvided"
}
```

To target the destination with the name "samlAssertionConsumerDest" for handling by the Transparent Proxy, you should create a destination custom resource in a namespace of your choice.

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "samlAssertionConsumerDest"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
  chainRef:
    name: com.sap.iasGeneratedOAuth2SamlBearerAssertion // can be omitted or
overridden if 'X-Chain-Name' header is passed
    variables:
    - name: samlProviderDestinationName // can be omitted or overridden if
'X-Chain-var-samlProviderDestinationName' header is passed
      value: samlAssertionProviderDest
    - name: subjectTokenType // can be omitted or overridden if 'X-Chain-var-
subjectTokenType' header is passed
      value: urn:ietf:params:oauth:token-type:access_token
```

Consumption

For authentication type *OAuth2SAMLBearerAssertion*, the cloud user identity must passed to the Transparent Proxy as a token represented by a JSON Web token (JWT) via an *Authorization Header* of scheme *Bearer*.

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

OAuth 2 SAML Bearer Assertion Pseudocode Snippet

Sample Code

```
function callOAuth2SAMLBearerAssertionHttpDestination() {
    accessToken = getPasswordAccessToken()

    url = '<destination-cr-name>.<destination-cr-namespace>'
    headers: {
            // X-Tenant-Subdomain is required only when transparent proxy is
in shared tenant mode
```

```
            'X-Tenant-Subdomain': '<tenant-where-destination-is-located>',
            'Authorization': 'Bearer ' + accessToken,
            // 'X-Chain' headers are required only if the are not present
as configuration in the Destination CR. If passed they will override the
configuration from the CR
            'X-Chain-Name': 'com.sap.iasGeneratedOAuth2SamlBearerAssertion' \
            'X-Chain-Var-samlProviderDestinationName': '<provider-destination-
name>'   \
            'X-Chain-Var-subjectTokenType': 'urn:ietf:params:oauth:token-
type:access_token' \
    }

    response = performHttpGetRequest(url, headers)
    print(response)
}

function getPasswordAccessToken() {
    url = 'https://<ias-tenant>.accounts400.ondemand.com/oauth2/token'
    headers = {
                'Content-Type': 'application/x-www-form-urlencoded',
                'Authorization': 'Basic <auth>',
    }

    postData = {
            'username': '<username>',
            'password': '<password>',
            'grant_type': 'password',
            'client_id': '<client-id>',
            'token_format': 'access_token'
    }

    response = performHttpPostRequest(url, headers, postData)

    try {
        passwordAccessToken = parseJSON(response.body)['access_token']
        return passwordAccessToken
    } catch (err) {
        return fail('Cannot parse password access token: ' + err)
    }
}
```

OAuth 2 SAML Bearer Assertion Bash Snippet

Sample Code

```
# 1. Retrieve the user identity, which is the password access token (the
'access_token' value from the response body):
curl --request POST \
  --url https://<ias-tenant>.accounts400.ondemand.com/oauth2/token \
  --header 'Authorization: Basic <basic-auth>' \
  --header 'Content-Type: application/x-www-form-urlencoded' \
  --data username=<username> \
  --data password=<password> \
  --data grant_type=password \
  --data client_id=<oidc-app-client-id> \
  --data token_format=access_token

# 2.1 Call HTTP destination protected with OAuth 2 SAML Bearer Assertion
using the password access token retrieved in step 1:
curl \
  -H "X-Tenant-Subdomain: <tenant-where-destination-is-located>" \
  -H "Authorization: Bearer <password-access-token>" \
  '<destination-cr-name>.<destination-cr-namespace>'
# 2.2 If chainRef.name and variables are not specified in the Destination CR
they have to be provided as headers. Call the HTTP destination protected with
```

```
OAuth 2 SAML Bearer Assertion using the password access token retrieved in
step 1:
curl \
  -H "X-Tenant-Subdomain: <tenant-where-destination-is-located>" \
  -H "Authorization: Bearer <password-access-token>" \
  -H "X-Chain-Name: com.sap.iasGeneratedOAuth2SamlBearerAssertion" \
  -H "X-Chain-Var-samlProviderDestinationName: <provider-destination-name>"
\
  -H "X-Chain-Var-subjectTokenType: urn:ietf:params:oauth:token-
type:access_token" \
  '<destination-cr-name>.<destination-cr-namespace>'
```

1.6.1.1.4.2 OAuth Authorization Code Authentication

The Transparent Proxy handles the HTTP communication protocol for both Internet and on-premise destinations protected with the *OAuth Authorization Code* flow.

Prerequisites

To integrate this functionality, you must create an SAP BTP destination. This destination should have `Type` "HTTP" and `Authentication` "OAuth2AuthorizationCode", for example:

OAuth 2 Authorization Code Destination

Sample Code

```
{
    "Name": "example-dest-oauth-2-auth-code",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": <application-url>,
    "Authentication": "OAuth2AuthorizationCode",
    "tokenServiceURLType": "Dedicated",
    "clientId": <client-id>,
    "clientSecret": <client-secret>,
    "tokenServiceURL": <token-service-URL>
}
```

To target the destination with the name "example-dest-oauth-2-auth-code" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for an OAuth 2 Authhroziation Code Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
```

```
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-oauth-2-auth-code"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Consumption

For consumption of a destination with authentication type `OAuth2AuthorizationCode`, the code of the business user from the authorization server has to be passed to the Transparent Proxy via `Authorization` header of scheme `Bearer`. You can also pass the optional headers 'X-redirect-uri' and 'X-code-verifier' to the request. They can also be configured in the destination configuration. If any or both of the headers are present in the request as well as in the destination configuration, the Transparent Proxy attaches the request headers to the request to the Destination service.

For more information, see OAuth Authorization Code Authentication [page 1416].

> ### Note
>
> `<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

OAuth 2 Authorization Code Pseudocode Snippet

> ### Sample Code

```
function callHttpDestinationWithOAuth2AuthorizationCode() {
    passwordAccessToken = getPasswordAccessToken()
    authorizationCode = getAuthorizationCode(accessToken);

    url = '<destination-cr-name>.<destination-cr-namespace>'
    headers: {
            // X-Tenant-Subdomain is required only when Transparent Proxy is
in shared tenant mode
            'X-Tenant-Subdomain': '<tenant-where-destination-is-located>',
            'Authorization': 'Bearer ' + authorizationCode,
    }

    response = performHttpGetRequest(url, headers)

    print(response)
}

function getAuthorizationCode(accessToken) {
    url
= '<tenant-for-retrieving-auth-code>.authentication.ap10.hana.ondemand.com/
oauth/authorize?client_id=<client-id>&response_type=code'
    headers = {
                'Authorization': 'Bearer ' + accessToken,
    }

    response = performHttpGetRequest(url, headers);
```

```
    try {
        locationHeaderValue = response.headers['location'];
        code = locationHeaderValue.split('code=')[1];
        return code;
    } catch (err) {
        return fail('Cannot retrieve authorization code: ' + err);
    }
}

function getPasswordAccessToken() {
    url =
'<tenant-for-retrieving-password-token>.authentication.ap10.hana.ondemand.com/
oauth/token?grant_type=password&response_type=token'
    headers = {
                'Content-Type': 'application/x-www-form-urlencoded',
                'Authorization': 'Basic <auth>',
    }

    postData = {
            'username': '<username>',
            'password': '<password>',
    }

    response = performHttpPostRequest(url, headers, postData)

    try {
        passwordAccessToken = parseJSON(response.body)['access_token']
        return passwordAccessToken
    } catch (err) {
        return fail('Cannot parse password access token: ' + err)
    }
}
```

OAuth 2 Authorization Code Bash Snippet

Sample Code

```
# 1. Retrieve the user identity, which is the password access token (the
'access_token' value from the response body):
curl -X POST \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -H "Authorization: Basic <auth>" \
  -d "username=<username>&password=<password>" \

'<tenant-for-retrieving-password-token>.authentication.ap10.hana.ondemand.com/
oauth/token?grant_type=password&response_type=token'

# 2. Get authorization code (the 'location' header value from the response)
using the password access token retrieved in step 1:
curl \
  -H "Authorization: Bearer <password-access-token>" \
  '<tenant-for-retrieving-auth-code>.authentication.ap10.hana.ondemand.com/
oauth/authorize?client_id=<client-id>&response_type=code'

# 3. Call HTTP destination protected with OAuth 2 Authorization Code using
the authorisation code retrieved in step 2:
curl \
  -H "X-Tenant-Subdomain: <tenant-where-destination-is-located>" \
  -H "Authorization: Bearer <authorization-code>" \
  '<destination-cr-name>.<destination-cr-namespace>'
```

Related Information

OAuth Authorization Code Authentication [page 1416]

1.6.1.1.4.3 OAuth Client Credentials Authentication

The Transparent Proxy handles the HTTP communication protocol for both Internet and on-premise destinations protected with the *OAuth Client Credentials* flow.

Prerequisites

To integrate this functionality, you must create an SAP BTP destination. This destination should have `Type` "HTTP" and `Authentication` "OAuth2ClientCredentials", for example:

OAuth 2 Client Credentials Destination

Sample Code

```
{
    "Name": "example-dest-oauth-2-client-credentials",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": <application-url>,
    "Authentication": "OAuth2ClientCredentials"
    "tokenServiceURLType": "Common",
    "clientId": <client-id>,
    "clientSecret": <client-secret>,
    "tokenServiceURL": <token-service-url>
}
```

To target the destination with the name "example-dest-oauth-2-client-credentials" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for an OAuth 2 Client Credentials Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-oauth-2-client-credentials"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Consumption

For authentication type `OAuth2ClientCredentials`, no additional `Authorization` headers have to be provided to the Transparent Proxy as all necessary data for authorization is in the destination.

> ### Note
>
> `<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

> ### Caution
>
> The `X-Token-Service-Tenant` header represents the subdomain of the tenant on behalf of which to fetch an access token using the configured credentials. When `tokenServiceURLType` in the BTP Destination is `Common`, this header is required and its value will be used to replace the tenant placeholder in the URL or added as a subdomain if no placeholder exists. When `tokenServiceURLType` is `Dedicated`, this header is forbidden and an error will be returned if provided.
>
> If this header is not provided when `tokenServiceURLType` is `Common`, an error will be returned.

OAuth 2 Client Credentials Pseudocode Snippet

> ### Sample Code

```
function callHttpsDestinationOAuth2ClientCredentials() {
  url = '<destination-cr-name>.<destination-cr-namespace>'
  headers = {
            // X-Tenant-Subdomain is required only when transparent proxy is
in shared tenant mode
            'X-Tenant-Subdomain': '<tenant-where-destination-is-located>'
            // Only when the tokenServiceURLType in the BTP Destination is
'Common'
            'X-Token-Service-Tenant': '<tenant-to-retrieve-oauth-token>'
  };

  response = performHttpGetRequest(url, headers)
  print(response)
}
```

OAuth 2 Client Credentials Bash Snippet

> ### Sample Code

```
# Call HTTP destination protected with OAuth 2 Client Credentials:
curl <destination-cr-name>.<destination-cr-namespace> -H 'X-Tenant-
Subdomain': '<tenant-where-destination-is-located>'
```

Related Information

OAuth Client Credentials Authentication [page 1375]

1.6.1.1.4.4 OAuth JWT Bearer Authentication

The Transparent Proxy handles the HTTP communication protocol for both Internet and on-premise destinations protected with the *OAuth JWT Bearer* flow.

Prerequisites

To integrate this functionality, you must create an SAP BTP destination. This destination should have `Type` "HTTP" and `Authentication` "OAuth2JWTBearer", for example:

OAuth 2 JWT Bearer Destination

Sample Code

```
{
    "Name": "example-dest-oauth-2-jwt-bearer",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": <application-url>,
    "Authentication": "OAuth2JWTBearer",
    "tokenServiceURLType": "Common",
    "clientId": <client-id>,
    "clientSecret": <client-secret>,
    "tokenServiceURL": <token-service-URL>
}
```

To target the destination with the name "example-dest-oauth-2-jwt-bearer" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for an OAuth 2 JWT Bearer Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-oauth-2-jwt-bearer"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Consumption

For authentication type `OAuth2JWTBearer`, the cloud user identity has to be passed to the Transparent Proxy as a token represented by a JSON Web token (JWT) via `Authorization` header of scheme `Bearer`. Then, the token will be passed as an 'X-User-Token' to the Destination service, which then will return `authTokens`, processed by the Transparent Proxy and attached to the HTTP request as an `Authorization` header.

> ### Note
>
> `<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

> ### Caution
>
> The `X-Token-Service-Tenant` header represents the subdomain of the tenant on behalf of which to fetch an access token using the configured credentials. When `tokenServiceURLType` in the BTP Destination is `Common`, this header can be used to specify the tenant if the `X-User-Token` does not contain tenant information. The header value will be used to replace the tenant placeholder in the URL or added as a subdomain if no placeholder exists. When `tokenServiceURLType` is `Dedicated`, this header is forbidden and an error will be returned if provided.
>
> - If `X-Token-Service-Tenant` is not provided but `X-User-Token` contains tenant information, the tenant information will be automatically extracted from the JWT (JSON web token).
> - If neither the header nor tenant information in the JWT is available, an error will be returned.

OAuth 2 JWT Bearer Pseudocode Snippet

> ### Sample Code

```
function callHttpsDestinationOAuth2JWTBearer() {
    accessToken = getPasswordAccessToken()

    url = '<destination-cr-name>.<destination-cr-namespace>'
    headers: {
            // X-Tenant-Subdomain is required only when transparent proxy is
in shared tenant mode
            'X-Tenant-Subdomain': '<tenant-where-destination-is-located>',
            // Only when the tokenServiceURLType in the BTP destination is
'Common'
            'X-Token-Service-Tenant': '<tenant-to-retrieve-oauth-token>',
            'Authorization': 'Bearer ' + accessToken,
        }

    response = performHttpGetRequest(url, headers)
    print(response)
}

function getPasswordAccessToken() {
    url =
'<tenant-for-retrieving-password-token>.authentication.ap10.hana.ondemand.com/
oauth/token?grant_type=password&response_type=token'
```

```
    headers = {
                'Content-Type': 'application/x-www-form-urlencoded',
                'Authorization': 'Basic <auth>',
            }

    postData = {
            'username': '<username>',
            'password': '<password>',
    }

    response = performHttpPostRequest(url, headers, postData)

    try {
        passwordAccessToken = parseJSON(response.body)['access_token']
        return passwordAccessToken
    } catch (err) {
        return fail('Cannot parse password refresh token: ' + err)
    }
}
```

OAuth 2 JWT Bearer Bash Snippet

Sample Code

```
# 1. Retrieve the user identity, which is the password access token (the
'access_token' value from the response body):
curl -X POST \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -H "Authorization: Basic <auth>" \
  -d "username=<username>&password=<password>" \

'<tenant-for-retrieving-password-token>.authentication.ap10.hana.ondemand.com/
oauth/token?grant_type=password&response_type=token'

# 2. Call HTTP destination protected with OAuth 2 JWT Bearer using the
password access token retrieved in step 1:
curl \
  -H "X-Tenant-Subdomain: <tenant-where-destination-is-located>" \
  -H "Authorization: Bearer <password-access-token>" \
  '<destination-cr-name>.<destination-cr-namespace>'
```

Related Information

OAuth JWT Bearer Authentication [page 1395]

1.6.1.1.4.5 OAuth Password Authentication

The Transparent Proxy handles the HTTP communication protocol for both Internet and on-premise destinations protected with the *OAuth Password* flow.

Prerequisites

To integrate this functionality, you must create an SAP BTP destination. This destination should have Type "HTTP" and Authentication "OAuth2Password", for example:

OAuth 2 Password Destination

Sample Code

```
{
    "Name": "example-dest-oauth2-password",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": <application-url>,
    "Authentication": "OAuth2Password",
    "clientId": <client-id>,
    "clientSecret": <client-secret>,
    "tokenServiceURL": <token-service-URL>,
    "User": <user>,
    "Password": <password>
}
```

To target the destination with the name "example-dest-oauth-2-password" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice

Destination CR for an OAuth 2 Password Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-oauth-2-password"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Consumption

For authentication type `OAuth2Password`, no additional `Authorization` headers have to be provided to the Transparent Proxy as all necessary data for authorization is in the destination.

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

OAuth 2 Password Pseudocode Snippet

Sample Code

```
function callHttpsDestinationOAuth2Password() {
  url = '<destination-cr-name>.<destination-cr-namespace>'
  headers = {
            // X-Tenant-Subdomain is required only when Transparent Proxy is
in shared tenant mode
            'X-Tenant-Subdomain': '<tenant-where-destination-is-located>'
        }

  response = performHttpGetRequest(url, headers)
  print(response)
}
```

OAuth 2 Password Bash Snippet

Sample Code

```
# Call HTTP destination protected with OAuth 2 Password:
curl <destination-cr-name>.<destination-cr-namespace> -H 'X-Tenant-
Subdomain': '<tenant-where-destination-is-located>'
```

Related Information

OAuth Password Authentication [page 1391]

1.6.1.1.4.6 OAuth Refresh Token Authentication

The Transparent Proxy handles the HTTP communication protocol for both Internet and on-premise destinations protected with the *OAuth Refresh Token* flow.

Prerequisites

To integrate this functionality, you must create an SAP BTP destination. This destination should have `Type` "HTTP" and `Authentication` "OAuth2RefreshToken", for example:

OAuth 2 Refresh Token Destination

Sample Code

```
{
    "Name": "example-dest-oauth-2-refresh-token",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": <application-url>,
    "Authentication": "OAuth2RefreshToken",
    "tokenServiceURLType": "Dedicated",
    "clientId": <client-id>,
    "clientSecret": <client-secret>,
    "tokenServiceURL": <token-service-url>
}
```

To target the destination with the name "example-dest-oauth-2-refresh-token" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for an OAuth 2 Refresh Token Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-oauth-2-refresh-token"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Consumption

For authentication type `OAuth2RefreshToken`, a refresh token has to be passed to the Transparent Proxy via `Authorization` header of scheme `Bearer`.

For more information, see OAuth Refresh Token Authentication [page 1409].

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

OAuth 2 Refresh Token Pseudocode Snippet

Sample Code

```
function callOAuth2RefreshTokenHttpDestination() {
    refreshToken = getPasswordRefreshToken()
```

```
    url = '<destination-cr-name>.<destination-cr-namespace>'
    headers: {
            // X-Tenant-Subdomain is required only when Transparent Proxy is
in shared tenant mode
            'X-Tenant-Subdomain': '<tenant-where-destination-is-located>',
            'Authorization': 'Bearer ' + refreshToken,
    }

    response = performHttpGetRequest(url, headers)
    print(response)
}

function getPasswordRefreshToken() {
    url =
'<tenant-for-retrieving-password-token>.authentication.ap10.hana.ondemand.com/
oauth/token?grant_type=password&response_type=token'
    headers = {
                'Content-Type': 'application/x-www-form-urlencoded',
                'Authorization': 'Basic <auth>',
    }

    postData = {
            'username': '<username>',
            'password': '<password>',
    }

    response = performHttpPostRequest(url, headers, postData)

    try {
        passwordRefreshToken = parseJSON(response.body)['refresh_token']
        return passwordRefreshToken
    } catch (err) {
        return fail('Cannot parse password refresh token: ' + err)
    }
}
```

OAuth 2 Refresh Token Bash Snippet

Sample Code

```
# 1. Get password refresh token  (the 'refresh_token' value from the response
body):
curl -X POST \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -H "Authorization: Basic <auth>" \
  -d "username=<username>&password=<password>" \

'<tenant-for-retrieving-password-token>.authentication.ap10.hana.ondemand.com/
oauth/token?grant_type=password&response_type=token'

# 2. Call HTTP destination protected with OAuth 2 Refresh Token using the
refresh token retrieved in step 1:
curl \
  -H "X-Tenant-Subdomain: <tenant-where-destination-is-located>" \
  -H "Authorization: Bearer <refresh-token>" \
  '<destination-cr-name>.<destination-cr-namespace>'
```

Related Information

OAuth Refresh Token Authentication [page 1409]

1.6.1.1.4.7 OAuth SAML Bearer Assertion Authentication

The Transparent Proxy handles the HTTP communication protocol for both Internet and on-premise destinations protected with the *OAuth SAML Bearer Assertion* flow.

Prerequisites

OAuth 2 SAML Bearer Assertion Destination

Sample Code

```
{
    "Name": "example-dest-oauth-2-saml-bearer-assertion",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": <application-url>,
    "Authentication": "OAuth2SAMLBearerAssertion",
    "tokenServiceURLType": "Dedicated",
    "tokenServiceURL": <token-service-url>,
    "clientKey": <client-key>,
    "apiKey": <api-key>,
    "audience": <audience-url>,
    "authnContextClassRef":
"urn:oasis:names:tc:SAML:2.0:ac:classes:PreviousSession"
}
```

To target the destination with the name "example-dest-oauth-2-saml-bearer-assertion" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for an OAuth 2 SAML Bearer Assertion Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-oauth-2-saml-bearer-assertion"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Consumption

For authentication type `OAuth2SAMLBearerAssertion`, the cloud user identity has to be passed to the Transparent Proxy as a token represented by a JSON Web token (JWT) via `Authorization` header of scheme `Bearer`.

> **Note**
>
> `<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

> **Caution**
>
> The `X-Token-Service-Tenant` header represents the subdomain of the tenant on behalf of which to fetch an access token using the configured credentials. When `tokenServiceURLType` in the BTP Destination is `Common`, this header can be used to specify the tenant if the `X-User-Token` does not contain tenant information. The header value will be used to replace the tenant placeholder in the URL or added as a subdomain if no placeholder exists. When `tokenServiceURLType` is `Dedicated`, this header is forbidden and an error will be returned if provided.
>
> - If `X-Token-Service-Tenant` is not provided but `X-User-Token` contains tenant information, the tenant information will be automatically extracted from the JWT (JSON web token).
> - If neither the header nor tenant information in the JWT is available, an error will be returned.

OAuth 2 SAML Bearer Assertion Pseudocode Snippet

> **Sample Code**

```
function callOAuth2SAMLBearerAssertionHttpDestination() {
    accessToken = getPasswordAccessToken()

    url = '<destination-cr-name>.<destination-cr-namespace>'
    headers: {
            // X-Tenant-Subdomain is required only when transparent proxy is
in shared tenant mode
            'X-Tenant-Subdomain': '<tenant-where-destination-is-located>',
            // Only when the tokenServiceURLType in the BTP destination is
'Common'
            'X-Token-Service-Tenant': '<tenant-to-retrieve-oauth-token>',
            'Authorization': 'Bearer ' + accessToken,
    }

    response = performHttpGetRequest(url, headers)
    print(response)
}

function getPasswordAccessToken() {
    url =
'<tenant-for-retrieving-password-token>.authentication.ap10.hana.ondemand.com/
oauth/token?grant_type=password&response_type=token'
    headers = {
```

```
                'Content-Type': 'application/x-www-form-urlencoded',
                'Authorization': 'Basic <auth>',
    }

    postData = {
            'username': '<username>',
            'password': '<password>',
    }

    response = performHttpPostRequest(url, headers, postData)

    try {
        passwordAccessToken = parseJSON(response.body)['access_token']
        return passwordAccessToken
    } catch (err) {
        return fail('Cannot parse password access token: ' + err)
    }
}
```

OAuth 2 SAML Bearer Assertion Bash Snippet

Sample Code

```
# 1. Retrieve the user identity, which is the password access token (the
'access_token' value from the response body):
curl -X POST \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -H "Authorization: Basic <auth>" \
  -d "username=<username>&password=<password>" \

'<tenant-for-retrieving-password-token>.authentication.ap10.hana.ondemand.com/
oauth/token?grant_type=password&response_type=token'

# 2. Call HTTP destination protected with OAuth 2 SAML Bearer Assertion using
the password access token retrieved in step 1:
curl \
  -H "X-Tenant-Subdomain: <tenant-where-destination-is-located>" \
  -H "Authorization: Bearer <password-access-token>" \
  '<destination-cr-name>.<destination-cr-namespace>'
```

Related Information

OAuth SAML Bearer Assertion Authentication [page 1363]

1.6.1.1.4.8 OAuth Token Exchange Authentication

The Transparent Proxy handles the HTTP communication protocol for both Internet and on-premise destinations protected with the *OAuth Token Exchange* flow.

Prerequisites

To integrate this functionality, you must create an SAP BTP destination. This destination should have `Type` "HTTP" and `Authentication` "OAuth2TokenExchange", for example:

> **Note**
>
> All examples below assume usage of an IAS token service URL, but any authorization server that supports RFC-8963 can be used.

OAuth 2 Token Exchange Destination

> **Sample Code**

```
{
    "Name": "example-dest-oauth-2-token-exchange",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": <application-url>,
    "Authentication": "OAuth2TokenExchange",
    "tokenServiceURLType": "Dedicated",
    "clientId": <client-id>,
    "clientSecret": <client-secret>,
    "tokenServiceURL": <ias-token-service-URL>
}
```

To target the destination with the name "example-dest-oauth-2-token-exchange" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for an OAuth 2 Token Exchange Destination

> **Sample Code**

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-oauth-2-token-exchange"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Consumption

For authentication type `OAuth2TokenExchange`, the cloud user identity has to be passed to the Transparent Proxy as a token represented by a JSON Web token (JWT) via an `Authorization` header of scheme `Bearer`.

> ### Note
>
> `<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

OAuth 2 Token Exchange Pseudocode Snippet

> ### Sample Code

```
function callOAuth2TokenExchangeHttpsDestination() {
    accessToken = getPasswordAccessToken()
    url = '<destination-cr-name>.<destination-cr-namespace>'
    headers: {
            // X-Tenant-Subdomain is required only when Transparent Proxy is
in shared tenant mode
            'X-Tenant-Subdomain': '<tenant-where-destination-is-located>',
            'Authorization': 'Bearer ' + accessToken,
            'X-Subject-Token-Type': 'urn:ietf:params:oauth:token-type:jwt',
            'X-Actor-Token: <actor-token>',        # Optional: Include if an
actor token is available
            'X-Actor-Token-Type: urn:ietf:params:oauth:token-type:jwt',  #
Optional: Specify the token type if using 'X-Actor-Token'
    }

    response = performHttpGetRequest(url, headers)
    print(response)
}

function getPasswordAccessToken() {
    url = 'https://<tenant-domain>/oauth2/token'

    headers = {
                'Content-Type': 'application/x-www-form-urlencoded',
                'Authorization': 'Basic <auth>',
    }

    postData = {
            'username': '<username>',
            'password': '<password>',
            'grant_type' : 'password',
            'client_id' : '<client_id>',
            'token_format' : 'jwt',
    }

    response = performHttpPostRequest(url, headers, postData)

    try {
        passwordAccessToken = parseJSON(response.body)['access_token']
        return passwordAccessToken
    } catch (err) {
        return fail('Cannot parse password access token: ' + err)
    }
}
function getActorAccessToken() {
    const url = 'https://<tenant-domain>/oauth2/token';

    const headers = {
        'Content-Type': 'application/x-www-form-urlencoded',
        'Authorization': 'Basic <actor-basic-auth>',
    };

    const postData = {
        'grant_type': 'client_credentials',
```

```
        'client_id': '<actor_client_id>',
        'token_format': 'jwt',
    };

    const response = performHttpPostRequest(url, headers, postData);

    try {
        const actorAccessToken = parseJSON(response.body)['access_token'];
        return actorAccessToken;
    } catch (err) {
        return fail('Cannot parse actor access token: ' + err);
    }
}
```

OAuth 2 Token Exchange Bash Snippet

Sample Code

```
# Get password access token (the 'access_token' value from the response body):
curl --request POST \
  --url https://<tenant-domain>/oauth2/token \
  --header 'Authorization: Basic <basic-auth>' \
  --header 'Content-Type: application/x-www-form-urlencoded' \
  --data username=<username> \
  --data password=<password> \
  --data grant_type=password \
  --data client_id=<subject_client_id> \
  --data token_format=jwt

# Optional - Get token representing the identity of the acting party on
behalf of the subject.
curl --request POST \
  --url https://<tenant-domain>/oauth2/token \
  --header 'Authorization: Basic <actor-basic-auth>' \
  --header 'Content-Type: application/x-www-form-urlencoded' \
  --data grant_type=client_credentials \
  --data client_id=<actor_client_id> \
  --data token_format=jwt

# Call HTTP destination protected with OAuth 2 Token Exchange:
curl \
  -H "X-Tenant-Subdomain: <tenant-where-destination-is-located>" \
  -H "Authorization: Bearer <password-access-token>" \
  -H "X-Subject-Token-Type: urn:ietf:params:oauth:token-type:jwt" \
  -H "X-Actor-Token: <actor-token>" \        # Optional: Include if an actor
token is available
  -H "X-Actor-Token-Type: urn:ietf:params:oauth:token-type:jwt" \  #
Optional: Specify the token type if using 'X-Actor-Token'
  '<destination-cr-name>.<destination-cr-namespace>'
```

1.6.1.1.4.9 OAuth User Token Exchange Authentication

The Transparent Proxy handles the HTTP communication protocol for both Internet and on-premise destinations protected with the *OAuth User Token Exchange* flow.

Prerequisites

To integrate this functionality, you must create an SAP BTP destination. This destination should have `Type` "HTTP" and `Authentication` "OAuth2UserTokenExchange", for example:

OAuth 2 User Token Exchange Destination

Sample Code

```
{
    "Name": "example-dest-oauth-2-user-token-exchange",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": <application-url>,
    "Authentication": "OAuth2UserTokenExchange",
    "tokenServiceURLType": "Common",
    "clientId": <client-id>,
    "clientSecret": <client-secret>,
    "tokenServiceURL": <token-service-URL>
}
```

To target the destination with the name "example-dest-user-oauth-2-token-exchange" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for an OAuth 2 User Token Exchange Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-oauth-2-user-token-exchange"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Consumption

For authentication type `OAuth2UserTokenExchange`, the cloud user identity has to be passed to the Transparent Proxy as a token represented by a JSON Web token (JWT) via `Authorization` header of scheme `Bearer`.

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

Caution

The `X-Token-Service-Tenant` header represents the subdomain of the tenant on behalf of which to fetch an access token using the configured credentials. When `tokenServiceURLType` in the BTP Destination is `Common`, this header can be used to specify the tenant if the `X-User-Token` does not contain tenant information. The header value will be used to replace the tenant placeholder in the URL or added as a subdomain if no placeholder exists. When `tokenServiceURLType` is `Dedicated`, this header is forbidden and an error will be returned if provided.

- If `X-Token-Service-Tenant` is not provided but `X-User-Token` contains tenant information, the tenant information will be automatically extracted from the JWT (JSON web token).
- If neither the header nor tenant information in the JWT is available, an error will be returned.

OAuth 2 User Token Exchange Pseudocode Snippet

Sample Code

```
function callOAuth2TokenExchangeHttpsDestination() {
    accessToken = getPasswordAccessToken()
    url = '<destination-cr-name>.<destination-cr-namespace>'
    headers: {
            // X-Tenant-Subdomain is required only when transparent proxy is
in shared tenant mode
            'X-Tenant-Subdomain': '<tenant-where-destination-is-located>',
            // Only when the tokenServiceURLType in the BTP destination is
'Common'
            'X-Token-Service-Tenant': '<tenant-to-retrieve-oauth-token>',
            'Authorization': 'Bearer ' + accessToken,
    }

    response = performHttpGetRequest(url, headers)
    print(response)
}

function getPasswordAccessToken() {
    url =
'<tenant-for-retrieving-password-token>.authentication.ap10.hana.ondemand.com/
oauth/token?grant_type=password&response_type=token'
    headers = {
                'Content-Type': 'application/x-www-form-urlencoded',
                'Authorization': 'Basic <auth>',
    }

    postData = {
            'username': '<username>',
            'password': '<password>',
    }

    response = performHttpPostRequest(url, headers, postData)

    try {
        passwordAccessToken = parseJSON(response.body)['access_token']
        return passwordAccessToken
```

```
    } catch (err) {
        return fail('Cannot parse password access token: ' + err)
    }
}
```

OAuth 2 User Token Exchange Bash Snippet

Sample Code

```
# 1. Get password access token (the 'access_token' value from the response
body):
curl -X POST \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -H "Authorization: Basic <auth>" \
  -d "username=<username>&password=<password>" \

'<tenant-for-retrieving-password-token>.authentication.ap10.hana.ondemand.com/
oauth/token?grant_type=password&response_type=token'

# 2. Call HTTP destination protected with OAuth 2 User Token Exchange:
curl \
  -H "X-Tenant-Subdomain: <tenant-where-destination-is-located>" \
  -H "Authorization: Bearer <password-access-token>" \
  '<destination-cr-name>.<destination-cr-namespace>'
```

Related Information

OAuth User Token Exchange Authentication [page 1384]

1.6.1.1.5 User Propagation

Configure user propagation for the Transparent Proxy for Kubernetes.

Related Information

Principal Propagation [page 2203]
Technical User Propagation [page 2207]

1.6.1.1.5.1 Principal Propagation

Configure *principal propagation* for the Transparent Proxy for Kubernetes.

Principal propagation, also known as *user propagation*, lets you perform a single sign-on (SSO) of the cloud user towards an on-premise system.

Prerequisites

The Transparent Proxy handles HTTP communication protocol for on-premise destinations which use *principal propagation*.

To integrate this functionality, you must:

- Fulfill the on-premise/private cloud connectivity prerequisites
- Create an SAP BTP Destination

This destination should have `Type` "HTTP" and `Authentication` "PrincipalPropagation", for example:

Principal Propagation Destination

Sample Code

```
{
    "Name": "example-dest-principal-propagation",
    "Type": "HTTP",
    "ProxyType": "OnPremise",
    "URL": <virtual-host>:<port>,
    "Authentication": "PrincipalPropagation"
}
```

To target the destination with the name "example-dest-principal-propagation" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for a Principal Propagation Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-principal-propagation"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Consumption

For the consumption of a destination with *principal propagation*, the cloud user identity is passed to the Transparent Proxy as a token represented by a JSON Web token (JWT) via an `Authorization` header of scheme `Bearer`. It is forwarded via the Transparent Proxy to the Connectivity Proxy for Kubernetes [page 2088] and then to the Cloud Connector [page 1602], which validates and further processes it to establish SSO with the on-premise system.

For more information, see Authenticating Users against On-Premises Systems [page 1701] and Set Up Trust [page 1702].

> **Note**
>
> `<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

Principal Propagation Pseudocode Snippet

> **Sample Code**

```
function callHttpDestinationPrincipalPropagation() {
    passwordAccessToken = getPasswordAccessToken()
    assertionToken = getAssertionToken(passwordAccessToken);

    url = '<destination-cr-name>.<destination-cr-namespace>'
    headers = {
                // X-Tenant-Subdomain is required only when Transparent Proxy
is in shared tenant mode
                'X-Tenant-Subdomain': '<tenant-where-destination-is-located>',
                'Authorization': 'Bearer ' + assertionToken,
    }

    response = performHttpGetRequest(url, headers)
    print(response)
}

function getAssertionToken(accessToken) {
    host = '<tenant-to-retrieve-assertion-
token>.authentication.ap10.hana.ondemand.com'
    path = '/oauth/token?grant_type=urn:ietf:params:oauth:grant-type:jwt-
bearer&response_type=token&assertion=' + accessToken
    url = host + path
    headers = {
                'Authorization': 'Basic <auth>',
    }

    response = performHttpGetRequest(url, headers);

    try {
        assertionToken = parseJSON(response.body)['access_token']
        return assertionToken
    } catch (err) {
        return fail('Cannot parse assertion token token: ' + err)
    }
}

function getPasswordAccessToken() {
```

```
    url =
'<tenant-for-retrieving-password-token>.authentication.ap10.hana.ondemand.com/
oauth/token?grant_type=password&response_type=token'
    headers = {
                'Content-Type': 'application/x-www-form-urlencoded',
                'Authorization': 'Basic <auth>',
            }

    postData = {
            'username': '<username>',
            'password': '<password>',
    }

    response = performHttpPostRequest(url, headers, postData)

    try {
        passwordAccessToken = parseJSON(response.body)['access_token']
        return passwordAccessToken
    } catch (err) {
        return fail('Cannot parse password access token: ' + err)
    }
}
```

Principal Propagation Bash Snippet

Sample Code

```
# 1. Get password access token (the 'access_token' value from the response
body):
curl -X POST \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -H "Authorization: Basic <auth>" \
  -d "username=<username>&password=<password>" \

'<tenant-for-retrieving-password-token>.authentication.ap10.hana.ondemand.com/
oauth/token?grant_type=password&response_type=token'

# 2. Get assertion token (the 'access_token' value from the response body)
using the password access token retrieved in step 1:
curl \
-H 'Authorization: Basic <auth>' \
'https://<tenant-to-retrieve-assertion-
token>.authentication.ap10.hana.ondemand.com/oauth/
token?grant_type=urn:ietf:params:oauth:grant-type:jwt-
bearer&response_type=token&assertion=<passwordAccessToken>'

# 3. Call HTTP destination protected with Principal Propagation using the
assertion token retrieved in step 2:
curl \
-H 'X-Tenant-Subdomain: <tenant-where-destination-is-located>' \
-H 'Authorization: Bearer <assertionToken>' \
'https://<destination-cr-name>.<destination-cr-namespace>'
```

Related Information

Principal Propagation [page 30]

1.6.1.1.5.2 Technical User Propagation

Configure *technical user propagation* for the Transparent Proxy for Kubernetes.

Technical user propagation is similar to *principal propagation*, but in this case, a technical user is propagated instead of a business user. The retrieval of the access token performs the OAuth 2.0 client credentials flow, according to the token service configurations in the destination.

Prerequisites

The Transparent Proxy handles HTTP communication protocol for on-premise destinations which use *technical user propagation*.

To integrate this functionality, you must:

- Fulfill the on-premise/private cloud connectivity prerequisites
- Create an SAP BTP Destination

This destination should have `Type` "HTTP" and `Authentication` "OAuth2TechnicalUserPropagation", for example:

Technical User Propagation Destination

Sample Code

```
{
    "Name": "example-dest-technical-user-propagation",
    "Type": "HTTP",
    "ProxyType": "OnPremise",
    "URL": <virtual-host>:<port>,
    "Authentication": "OAuth2TechnicalUserPropagation"
    "tokenServiceURLType": <token-service-url-type>,
    "clientId": <client-id>,
    "clientSecret": <client-secret>,
    "tokenServiceURL": <token-service-url>
}
```

To target the destination with the name "example-dest-technical-user-propagation" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for a Technical User Propagation Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
```

```
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-technical-user-propagation"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Consumption

For the consumption of a destination with *technical user propagation*, the technical user is obtained from the destination by the Transparent Proxy, forwarded to the Connectivity Proxy for Kubernetes [page 2088], and then to the Cloud Connector [page 1602], which validates and further processes it to establish SSO with the on-premise system.

For more information, see Authenticating Users against On-Premises Systems [page 1701] and Configuring Technical User Propagation [page 1739].

> **Note**
>
> `<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

Technical User Propagation Pseudocode Snippet

Sample Code

```
function callHttpDestinationTechnicalUserPropagation() {
  url = '<destination-cr-name>.<destination-cr-namespace>'
  headers = {
            // X-Tenant-Subdomain is required only when Transparent Proxy is
in shared tenant mode
            'X-Tenant-Subdomain': '<tenant-where-destination-is-located>',
        }

  response = performHttpGetRequest(url, headers)
  print(response)
}
```

Technical User Propagation Bash Snippet

Sample Code

```
# Call HTTP destination protected with Technical User Propagation:
curl <destination-cr-name>.<destination-cr-namespace> -H 'X-Tenant-Subdomain:
<tenant-where-destination-is-located>' -H 'X-Token-Service-Tenant: <tenant-to-
retrieve-oauth-token>'
```

1.6.1.1.6 SAML Assertion Authentication

The Transparent Proxy handles the HTTP communication protocol for both Internet and on-premise destinations protected with the *SAML Assertion* flow.

Prerequisites

SAML Assertion Destination

Sample Code

```
{
    "Name": "example-dest-saml-assertion",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": <application-url>,
    "Authentication": "SAMLAssertion",
    "audience": <audience-url>,
    "authnContextClassRef": "urn:oasis:names:tc:SAML:2.0:ac:classes:X509"
}
```

To target the destination with the name "example-dest-saml-assertion" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for a SAML Assertion Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-saml-assertion"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Consumption

For authentication type `SAMLAssertion`, the cloud user identity has to be passed to the Transparent Proxy as a token represented by a JSON Web token (JWT) via `Authorization` header of scheme `Bearer`.

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

Caution

The `X-Token-Service-Tenant` header represents the subdomain of the tenant on behalf of which to fetch an access token using the configured credentials. When `tokenServiceURLType` in the BTP Destination is `Common`, this header can be used to specify the tenant if the `X-User-Token` does not contain tenant information. The header value will be used to replace the tenant placeholder in the URL or added as a subdomain if no placeholder exists. When `tokenServiceURLType` is `Dedicated`, this header is forbidden and an error will be returned if provided.

- If `X-Token-Service-Tenant` is not provided but `X-User-Token` contains tenant information, the tenant information will be automatically extracted from the JWT (JSON web token).
- If neither the header nor tenant information in the JWT is available, an error will be returned.

SAML assertion Pseudocode Snippet

Sample Code

```
function callSAMLAssertionHttpDestination() {
    accessToken = getPasswordAccessToken()

    url = '<destination-cr-name>.<destination-cr-namespace>'
    headers: {
            // X-Tenant-Subdomain is required only when transparent proxy is
in shared tenant mode
            'X-Tenant-Subdomain': '<tenant-where-destination-is-located>',
            // Only when the tokenServiceURLType in the BTP destination is
'Common'
            'X-Token-Service-Tenant': '<tenant-to-retrieve-oauth-token>',
            'Authorization': 'Bearer ' + accessToken,
    }

    response = performHttpGetRequest(url, headers)
    print(response)
}

function getPasswordAccessToken() {
    url =
'<tenant-for-retrieving-password-token>.authentication.ap10.hana.ondemand.com/
oauth/token?grant_type=password&response_type=token'
    headers = {
                'Content-Type': 'application/x-www-form-urlencoded',
                'Authorization': 'Basic <auth>',
    }

    postData = {
            'username': '<username>',
            'password': '<password>',
    }

    response = performHttpPostRequest(url, headers, postData)

    try {
        passwordAccessToken = parseJSON(response.body)['access_token']
```

```
        return passwordAccessToken
    } catch (err) {
        return fail('Cannot parse password access token: ' + err)
    }
}
```

SAML Assertion Bash Snippet

Sample Code

```
# 1. Retrieve the user identity, which is the password access token (the
'access_token' value from the response body):
curl -X POST \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -H "Authorization: Basic <auth>" \
  -d "username=<username>&password=<password>" \

'<tenant-for-retrieving-password-token>.authentication.ap10.hana.ondemand.com/
oauth/token?grant_type=password&response_type=token'

# 2. Call HTTP destination protected with SAML Assertion using the password
access token retrieved in step 1:
curl \
  -H "X-Tenant-Subdomain: <tenant-where-destination-is-located>" \
  -H "Authorization: Bearer <password-access-token>" \
  '<destination-cr-name>.<destination-cr-namespace>'
```

Related Information

SAML Assertion Authentication [page 1402]

1.6.1.2 Mail Destinations

Configure Mail destinations for the Transparent Proxy for Kubernetes.

On-Premise Mail Destinations

The Transparent Proxy handles SMTP, POP3 and IMAP communication protocols for on-premise Mail destinations. The Transparent Proxy performs the SOCKS5 handshake with the Connectivity Proxy for Kubernetes [page 2088] to enable the consumption of that (otherwise complex) functionality out-of-the-box for the application developer.

For more information about SOCKS5, see TCP Destinations [page 2244].

In addition, the Transparent Proxy supports *Basic Authentication*, *OAuth2ClientCredentials*, *OAuth2RefreshToken* and *OAuth2AuthorizationCode* for on-premise Mail destinations. Whether Basic

Authentication/OAuth2ClientCredentials/OAuth2AuthorizationCode/OAuth2RefreshToken is used or not, the TCP connection remains secure due to the SOCKS5 communication.

Internet Mail Destinations

The Transparent Proxy handles SMTP, POP3, IMAP, SMTPS, POP3S and IMAPS communication protocols for Internet Mail destinations. The Transparent Proxy supports *Basic Authentication*, *OAuth2ClientCredentials*, *OAuth2RefreshToken* and *OAuth2AuthorizationCode* for Internet Mail destinations.

Related Information

IMAP [page 2212]
IMAPS [page 2218]
POP3 [page 2222]
POP3S [page 2227]
SMTP [page 2231]
SMTPS [page 2237]
MAIL Destinations [page 1467]

1.6.1.2.1 IMAP

Consume a mail system via IMAP through the Transparent Proxy for Kubernetes and the Destination service.

Preparation

To integrate this functionality, you must:

- For On-Premise/Private Cloud connectivity, fulfill the prerequisites [page 2166].
- Create an SAP BTP Destination.

Example: OnPremise IMAP Mail Destination with BasicAuthentication

Sample Code

```
{
        "Name": "example-dest-imap",
        "Type": "MAIL",
        "Authentication": "BasicAuthentication",
```

```
        "mail.user": <username>,
        "mail.password": <password>,
        "mail.transport.protocol": "imap4",
        "mail.imap4.host": "imap4",
        "mail.imap4.auth": "true",
        "mail.imap4.port": "143",
        "ProxyType": "OnPremise"
}
```

Example: OnPremise IMAP Mail Destination with OAuth2ClientCredentials

Sample Code

```
{
        "Name": "example-dest-imap",
        "Type": "MAIL",
        "Authentication": "OAuth2ClientCredentials",
        "mail.user": <username>,
        "mail.transport.protocol": "imap4",
        "mail.imap4.host": "imap4",
        "mail.imap4.auth": "true",
        "mail.imap4.port": "143",
        "ProxyType": "OnPremise",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: OnPremise IMAP Mail Destination with OAuth2RefreshToken

Sample Code

```
{
        "Name": "example-dest-imap",
        "Type": "MAIL",
        "Authentication": "OAuth2RefreshToken",
        "mail.user": <username>,
        "mail.transport.protocol": "imap4",
        "mail.imap4.host": "imap4",
        "mail.imap4.auth": "true",
        "mail.imap4.port": "143",
        "ProxyType": "OnPremise",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: OnPremise IMAP Mail Destination with OAuth2AuthorizationCode

Sample Code

```
{
        "Name": "example-dest-imap",
        "Type": "MAIL",
        "Authentication": "OAuth2AuthorizationCode",
        "mail.user": <username>,
```

```
        "mail.transport.protocol": "imap4",
        "mail.imap4.host": "imap4",
        "mail.imap4.auth": "true",
        "mail.imap4.port": "143",
        "ProxyType": "OnPremise",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: Internet IMAP Mail Destination with BasicAuthentication

Sample Code

```
{
        "Name": "example-dest-imap",
        "Type": "MAIL",
        "Authentication": "BasicAuthentication",
        "mail.user": <username>,
        "mail.password": <password>,
        "mail.transport.protocol": "imap4",
        "mail.imap4.host": "imap4",
        "mail.imap4.auth": "true",
        "mail.imap4.port": "143",
        "ProxyType": "Internet"
}
```

Example: Internet IMAP Mail Destination with OAuth2ClientCredentials

Sample Code

```
{
        "Name": "example-dest-imap",
        "Type": "MAIL",
        "Authentication": "OAuth2ClientCredentials",
        "mail.user": <username>,
        "mail.transport.protocol": "imap4",
        "mail.imap4.host": "imap4",
        "mail.imap4.auth": "true",
        "mail.imap4.port": "143",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: Internet IMAP Mail Destination with OAuth2RefreshToken

Sample Code

```
{
        "Name": "example-dest-imap",
        "Type": "MAIL",
        "Authentication": "OAuth2RefreshToken",
        "mail.user": <username>,
```

```
        "mail.transport.protocol": "imap4",
        "mail.imap4.host": "imap4",
        "mail.imap4.auth": "true",
        "mail.imap4.port": "143",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: Internet IMAP Mail Destination with OAuth2AuthorizationCode

Sample Code

```
{
        "Name": "example-dest-imap",
        "Type": "MAIL",
        "Authentication": "OAuth2AuthorizationCode",
        "mail.user": <username>,
        "mail.transport.protocol": "imap4",
        "mail.imap4.host": "imap4",
        "mail.imap4.auth": "true",
        "mail.imap4.port": "143",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

To target the destination with the name "example-dest-imap" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Example: Destination CR for an IMAP Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-imap"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy will create a Kubernetes Service for that destination on port 143, the default IMAP port (unless another port is specified in the destination CR). The Transparent Proxy handles the basic authentication with the mail server using the credentials from the respective destination.

Multitenant Usage

The Transparent Proxy supports multitenancy for `MAIL` destinations. To consume a target system in a multitenant manner, describe all tenants in the *transparent-proxy.connectivity.api.sap/tenant-subdomains* annotation:

Example: Destination CR for a MAIL Destination in Transparent Proxy *Shared* Mode

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
  annotations:
    transparent-proxy.connectivity.api.sap/tenant-subdomains: '["tenant-
subdomain", "another-tenant-subdomain"]'
  destinationRef:
    name: "example-dest-imap"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy will create a Kubernetes service for each tenant described in the *transparent-proxy.connectivity.api.sap/tenant-subdomains* annotation. The names of the Kubernetes services will follow this format: <destination-cr-name>-<tenant-subdomain>.

Consumption

Once done, the application can start consuming the destination from within the Kubernetes cluster.

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

IMAP Consumption - Bash Snippet

Sample Code

```
telnet <destination-cr-name>.<destination-cr-namespace> 143

2 LIST "" "*"

3 SELECT INBOX

5 FETCH 1 BODY[TEXT]
```

IMAP Consumption for Authentication Types OAuth2RefreshToken/OAuth2AuthorizationCode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>.<destination-cr-namespace> 143

// Token Format: "user=<username>\x01auth=Bearer <access_token>\x01\x01"
where access_token is the refresh token for OAuth2RefreshToken or the
authorization code for OAuth2AuthorizationCode and username is your mail user
1 AUTHENTICATE XOAUTH2 <base64-encoded-token>

2 LIST "" "*"

3 SELECT INBOX

5 FETCH 1 BODY[TEXT]
```

IMAP Consumption in *Shared* (Multitenant) Mode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>-<tenant-subdomain>.<destination-cr-namespace> 143

2 LIST "" "*"

3 SELECT INBOX

5 FETCH 1 BODY[TEXT]
```

IMAP Consumption for Authentication Types OAuth2RefreshToken/OAuth2AuthorizationCode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>-<tenant-subdomain>.<destination-cr-namespace> 143

// Token Format: "user=<username>\x01auth=Bearer <access_token>\x01\x01"
where access_token is the refresh token for OAuth2RefreshToken or the
authorization code for OAuth2AuthorizationCode and username is your mail user
1 AUTHENTICATE XOAUTH2 <base64-encoded-token>

2 LIST "" "*"

3 SELECT INBOX

5 FETCH 1 BODY[TEXT]
```

Related Information

Create MAIL Destinations [page 1484]

1.6.1.2.2 IMAPS

Consume a mail system via IMAPS through the Transparent Proxy for Kubernetes and the Destination service.

Preparation

To integrate this functionality, you must create a BTP destination in a first step.

Example: Internet IMAPS Destination with BasicAuthentication

Sample Code

```
{
        "Name": "example-dest-imaps",
        "Type": "MAIL",
        "Authentication": "BasicAuthentication",
        "mail.user": <username>,
        "mail.password": <password>,
        "mail.transport.protocol": "imap4s",
        "mail.imap4s.host": "imap.gmail.com",
        "mail.imap4s.auth": "true",
        "mail.imap4s.port": "993",
        "ProxyType": "Internet"
}
```

Example: Internet IMAPS Mail Destination with OAuth2ClientCredentials

Sample Code

```
{
        "Name": "example-dest-imaps",
        "Type": "MAIL",
        "Authentication": "OAuth2ClientCredentials",
        "mail.user": <username>,
        "mail.transport.protocol": "imap4s",
        "mail.imap4s.host": "imap4s",
        "mail.imap4s.auth": "true",
        "mail.imap4s.port": "993",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: Internet IMAPS Mail Destination with OAuth2RefreshToken

Sample Code

```
{
```

```
        "Name": "example-dest-imaps",
        "Type": "MAIL",
        "Authentication": "OAuth2RefreshToken",
        "mail.user": <username>,
        "mail.transport.protocol": "imap4s",
        "mail.imap4s.host": "imap4s",
        "mail.imap4s.auth": "true",
        "mail.imap4s.port": "993",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: Internet IMAPS Mail Destination with OAuth2AuthorizationCode

Sample Code

```
{
        "Name": "example-dest-imaps",
        "Type": "MAIL",
        "Authentication": "OAuth2AuthorizationCode",
        "mail.user": <username>,
        "mail.transport.protocol": "imap4s",
        "mail.imap4s.host": "imap4s",
        "mail.imap4s.auth": "true",
        "mail.imap4s.port": "993",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

To target the destination with the name "example-dest-imaps" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] (CR) in a namespace of your choice.

Example: Destination CR for an IMAPS Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-imaps"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy will create a Kubernetes Service for that destination on port 993, the default IMAPS port (unless another port is specified in the destination CR). The Transparent proxy handles basic authentication with the mail server using the credentials from the respective destination.

Multitenant Usage

The Transparent Proxy supports multitenancy for MAIL destinations. To consume a target system in a multitenant manner, describe all tenants in the `transparent-proxy.connectivity.api.sap/tenant-subdomains` annotation:

Example: Destination CR for a MAIL Destination in Transparent Proxy *Shared* Mode

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
  annotations:
    transparent-proxy.connectivity.api.sap/tenant-subdomains: '["tenant-
subdomain", "another-tenant-subdomain"]'
  destinationRef:
    name: "example-dest-imaps"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy will create a Kubernetes Service for each tenant described in the `transparent-proxy.connectivity.api.sap/tenant-subdomains` annotation. The names of the Kubernetes services will follow this format: `<destination-cr-name>-<tenant-subdomain>`.

Consumption

Once done, the application can start consuming the destination from within the Kubernetes cluster:

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

IMAPS Consumption - Bash Snippet

Sample Code

```
telnet <destination-cr-name>.<destination-cr-namespace> 993

2 LIST "" "*"

3 SELECT INBOX

5 FETCH 1 BODY[TEXT]
```

IMAPS Consumption for Authentication Types OAuth2RefreshToken/OAuth2AuthorizationCode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>.<destination-cr-namespace> 993

// Token Format: "user=<username>\x01auth=Bearer <access_token>\x01\x01"
where access_token is the refresh token for OAuth2RefreshToken or the
authorization code for OAuth2AuthorizationCode and username is your mail user
1 AUTHENTICATE XOAUTH2 <base64-encoded-token>

2 LIST "" "*"

3 SELECT INBOX

5 FETCH 1 BODY[TEXT]
```

IMAPS Consumption in *Shared* (Multitenant) Mode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>-<tenant-subdomain>.<destination-cr-namespace> 993

2 LIST "" "*"

3 SELECT INBOX

5 FETCH 1 BODY[TEXT]
```

IMAPS Consumption for Authentication Types OAuth2RefreshToken/OAuth2AuthorizationCode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>-<tenant-subdomain>.<destination-cr-namespace> 993

// Token Format: "user=<username>\x01auth=Bearer <access_token>\x01\x01"
where access_token is the refresh token for OAuth2RefreshToken or the
authorization code for OAuth2AuthorizationCode and username is your mail user
1 AUTHENTICATE XOAUTH2 <base64-encoded-token>

2 LIST "" "*"

3 SELECT INBOX

5 FETCH 1 BODY[TEXT]
```

Related Information

Create MAIL Destinations [page 1484]

1.6.1.2.3 POP3

Consume a mail system via POP3 through the Transparent Proxy for Kubernetes and the Destination service.

Preparation

To integrate this functionality, you must:

- For On-Premise/Private Cloud connectivity, fulfill the prerequisites [page 2166].
- Create an SAP BTP Destination.

Example: OnPremise POP3 Mail Destination with BasicAuthentication

Sample Code

```
{
        "Name": "example-dest-pop3",
        "Type": "MAIL",
        "Authentication": "BasicAuthentication",
        "mail.user": <username>,
        "mail.password": <password>,
        "mail.transport.protocol": "pop3",
        "mail.pop3.host": "pop3",
        "mail.pop3.auth": "true",
        "mail.pop3.port": "110",
        "ProxyType": "OnPremise"
}
```

Example: OnPremise POP3 Mail Destination with OAuth2ClientCredentials

Sample Code

```
{
        "Name": "example-dest-pop3",
        "Type": "MAIL",
        "Authentication": "OAuth2ClientCredentials",
        "mail.user": <username>,
        "mail.transport.protocol": "pop3",
        "mail.pop3.host": "pop3",
        "mail.pop3.auth": "true",
        "mail.pop3.port": "110",
        "ProxyType": "OnPremise",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: OnPremise POP3 Mail Destination with OAuth2RefreshToken

Sample Code

```
{
        "Name": "example-dest-pop3",
        "Type": "MAIL",
        "Authentication": "OAuth2RefreshToken",
        "mail.user": <username>,
        "mail.transport.protocol": "pop3",
        "mail.pop3.host": "pop3",
        "mail.pop3.auth": "true",
        "mail.pop3.port": "110",
        "ProxyType": "OnPremise",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: OnPremise POP3 Mail Destination with OAuth2AuthorizationCode

Sample Code

```
{
        "Name": "example-dest-pop3",
        "Type": "MAIL",
        "Authentication": "OAuth2AuthorizationCode",
        "mail.user": <username>,
        "mail.transport.protocol": "pop3",
        "mail.pop3.host": "pop3",
        "mail.pop3.auth": "true",
        "mail.pop3.port": "110",
        "ProxyType": "OnPremise",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: Internet POP3 Mail Destination with BasicAuthentication

Sample Code

```
{
        "Name": "example-dest-pop3",
        "Type": "MAIL",
        "Authentication": "BasicAuthentication",
        "mail.user": <username>,
        "mail.password": <password>,
        "mail.transport.protocol": "pop3",
        "mail.pop3.host": "pop3",
        "mail.pop3.auth": "true",
        "mail.pop3.port": "110",
        "ProxyType": "Internet"
}
```

Example: Internet POP3 Mail Destination with OAuth2ClientCredentials

Sample Code

```
{
        "Name": "example-dest-pop3",
        "Type": "MAIL",
        "Authentication": "OAuth2ClientCredentials",
        "mail.user": <username>,
        "mail.transport.protocol": "pop3",
        "mail.pop3.host": "pop3",
        "mail.pop3.auth": "true",
        "mail.pop3.port": "110",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: Internet POP3 Mail Destination with OAuth2RefreshToken

Sample Code

```
{
        "Name": "example-dest-pop3",
        "Type": "MAIL",
        "Authentication": "OAuth2RefreshToken",
        "mail.user": <username>,
        "mail.transport.protocol": "pop3",
        "mail.pop3.host": "pop3",
        "mail.pop3.auth": "true",
        "mail.pop3.port": "110",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: Internet POP3 Mail Destination with OAuth2AuthorizationCode

Sample Code

```
{
        "Name": "example-dest-pop3",
        "Type": "MAIL",
        "Authentication": "OAuth2AuthorizationCode",
        "mail.user": <username>,
        "mail.transport.protocol": "pop3",
        "mail.pop3.host": "pop3",
        "mail.pop3.auth": "true",
        "mail.pop3.port": "110",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

To target the destination with the name "example-dest-pop3" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Example: Destination CR for a POP3 Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-pop3"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy manager will create a service for that destination on port 110, the default POP3 port (unless another port is specified in the destination CR). The Transparent Proxy handles the basic authentication with the mail server using the credentials from the respective destination.

Multitenant Usage

The Transparent Proxy supports multitenancy for `MAIL` destinations. To consume a target system in a multitenant manner, describe all tenants in the *transparent-proxy.connectivity.api.sap/tenant-subdomains* annotation:

Example: Destination CR for a MAIL Destination in Transparent Proxy *Shared* Mode

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
  annotations:
    transparent-proxy.connectivity.api.sap/tenant-subdomains: '["tenant-
subdomain", "another-tenant-subdomain"]'
  destinationRef:
    name: "example-dest-pop3"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy will create a Kubernetes Service for each tenant described in the *transparent-proxy.connectivity.api.sap/tenant-subdomains* annotation. The names of the Kubernetes services will follow this format: <destination-cr-name>-<tenant-subdomain>.

Consumption

Once done, the application can start consuming the destination from within the Kubernetes cluster.

> **Note**
>
> `<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

POP3 Consumption - Bash Snippet

> **Sample Code**

```
telnet <destination-cr-name>.<destination-cr-namespace> 110
list
retr 1
retr 2
quit
```

POP3 Consumption for Authentication Types OAuth2RefreshToken/OAuth2AuthorizationCode - Bash Snippet

> **Sample Code**

```
telnet <destination-cr-name>.<destination-cr-namespace> 110
// Token Format: "user=<username>\x01auth=Bearer <access_token>\x01\x01"
where access_token is the refresh token for OAuth2RefreshToken or the
authorization code for OAuth2AuthorizationCode and username is your mail user
AUTH XOAUTH2 <base64-encoded-token>
list
retr 1
retr 2
quit
```

POP3 Consumption in *Shared* (Multitenant) Mode - Bash Snippet

> **Sample Code**

```
telnet <destination-cr-name>-<tenant-subdomain>.<destination-cr-namespace> 110
list
retr 1
retr 2
quit
```

c

Related Information

Create MAIL Destinations [page 1484]

1.6.1.2.4 POP3S

Consume a mail system via POP3S through the Transparent Proxy for Kubernetes and the Destination service.

Preparation

To integrate this functionality, you must create a BTP destination in a first step.

Example: Internet POP3S Destination with BasicAuthentication

Sample Code

```
{
        "Name": "example-dest-pop3s",
        "Type": "MAIL",
        "Authentication": "BasicAuthentication",
        "mail.user": <username>,
        "mail.password": <password>,
        "mail.transport.protocol": "pop3s",
        "mail.pop3s.host": "pop.gmail.com",
        "mail.pop3s.auth": "true",
        "mail.pop3s.port": "995",
        "ProxyType": "Internet"
}
```

Example: Internet POP3S Mail Destination with OAuth2ClientCredentials

Sample Code

```
{
        "Name": "example-dest-pop3s",
        "Type": "MAIL",
        "Authentication": "OAuth2ClientCredentials",
        "mail.user": <username>,
        "mail.password": <password>,
        "mail.transport.protocol": "pop3s",
        "mail.pop3s.host": "pop3s",
        "mail.pop3s.auth": "true",
        "mail.pop3s.port": "995",
```

```
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: Internet POP3S Mail Destination with OAuth2RefreshToken

Sample Code

```
{
        "Name": "example-dest-pop3s",
        "Type": "MAIL",
        "Authentication": "OAuth2RefreshToken",
        "mail.user": <username>,
        "mail.password": <password>,
        "mail.transport.protocol": "pop3s",
        "mail.pop3s.host": "pop3s",
        "mail.pop3s.auth": "true",
        "mail.pop3s.port": "995",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: Internet POP3S Mail Destination with OAuth2AuthorizationCode

Sample Code

```
{
        "Name": "example-dest-pop3s",
        "Type": "MAIL",
        "Authentication": "OAuth2AuthorizationCode",
        "mail.user": <username>,
        "mail.password": <password>,
        "mail.transport.protocol": "pop3s",
        "mail.pop3s.host": "pop3s",
        "mail.pop3s.auth": "true",
        "mail.pop3s.port": "995",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

To target the destination with the name "example-dest-pop3s" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] (CR) in a namespace of your choice.

Example: Destination CR for a POP3S Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-pop3s"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy will create a Kubernetes Service for that destination on port 995, the default POP3S port (unless another port is specified in the destination CR). The Transparent proxy handles basic authentication with the mail server using the credentials from the respective destination.

Multitenant Usage

The Transparent Proxy supports multitenancy for MAIL destinations. To consume a target system in a multitenant manner, describe all tenants in the `transparent-proxy.connectivity.api.sap/tenant-subdomains` annotation:

Example: Destination CR for a MAIL Destination in Transparent Proxy *Shared* Mode

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
  annotations:
    transparent-proxy.connectivity.api.sap/tenant-subdomains: '["tenant-
subdomain", "another-tenant-subdomain"]'
  destinationRef:
    name: "example-dest-pop3s"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy will create a Kubernetes Service for each tenant described in the `transparent-proxy.connectivity.api.sap/tenant-subdomains` annotation. The names of the Kubernetes services will follow this format: `<destination-cr-name>-<tenant-subdomain>`.

Consumption

Once done, the application can start consuming the destination from within the Kubernetes cluster:

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

POP3S Consumption - Bash Snippet

Sample Code

```
telnet <destination-cr-name>.<destination-cr-namespace> 995

list

retr 1

retr 2

quit
```

POP3S Consumption for Authentication Types OAuth2RefreshToken/OAuth2AuthorizationCode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>.<destination-cr-namespace> 995

// Token Format: "user=<username>\x01auth=Bearer <access_token>\x01\x01"
where access_token is the refresh token for OAuth2RefreshToken or the
authorization code for OAuth2AuthorizationCode and username is your mail user
AUTH XOAUTH2 <base64-encoded-token>

list

retr 1

retr 2

quit
```

POP3S Consumption in *Shared* (Multitenant) Mode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>-<tenant-subdomain>.<destination-cr-namespace> 995

list

retr 1

retr 2

quit
```

POP3S Consumption for Authentication Types OAuth2RefreshToken/OAuth2AuthorizationCode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>-<tenant-subdomain>.<destination-cr-namespace> 995
```

```
// Token Format: "user=<username>\x01auth=Bearer <access_token>\x01\x01"
where access_token is the refresh token for OAuth2RefreshToken or the
authorization code for OAuth2AuthorizationCode and username is your mail user
AUTH XOAUTH2 <base64-encoded-token>

list

retr 1

retr 2

quit
```

Related Information

Create MAIL Destinations [page 1484]

1.6.1.2.5 SMTP

Consume a mail system via SMTP through the Transparent Proxy for Kubernetes and the Destination service.

Preparation

To integrate this functionality, you must:

- For On-Premise/Private Cloud connectivity, fulfill the prerequisites [page 2166].
- Create an SAP BTP Destination.

Example: OnPremise SMTP Mail Destination with BasicAuthentication

Sample Code

```
{
        "Name": "example-dest-smtp",
        "Type": "MAIL",
        "Authentication": "BasicAuthentication",
        "mail.user": <username>,
        "mail.password": <password>,
        "mail.transport.protocol": "smtp",
        "mail.smtp.host": "smtp",
        "mail.smtp.auth": "true",
        "mail.smtp.port": "587",
        "ProxyType": "OnPremise"
}
```

Example: OnPremise SMTP Mail Destination with OAuth2ClientCredentials

Sample Code

```
{
        "Name": "example-dest-smtp",
        "Type": "MAIL",
        "Authentication": "OAuth2ClientCredentials",
        "mail.user": <username>,
        "mail.transport.protocol": "smtp",
        "mail.smtp.host": "smtp",
        "mail.smtp.auth": "true",
        "mail.smtp.port": "587",
        "ProxyType": "OnPremise",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: OnPremise SMTP Mail Destination with OAuth2RefreshToken

Sample Code

```
{
        "Name": "example-dest-smtp",
        "Type": "MAIL",
        "Authentication": "OAuth2RefreshToken",
        "mail.user": <username>,
        "mail.transport.protocol": "smtp",
        "mail.smtp.host": "smtp",
        "mail.smtp.auth": "true",
        "mail.smtp.port": "587",
        "ProxyType": "OnPremise",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: OnPremise SMTP Mail Destination with OAuth2AuthorizationCode

Sample Code

```
{
        "Name": "example-dest-smtp",
        "Type": "MAIL",
        "Authentication": "OAuth2AuthorizationCode",
        "mail.user": <username>,
        "mail.transport.protocol": "smtp",
        "mail.smtp.host": "smtp",
        "mail.smtp.auth": "true",
        "mail.smtp.port": "587",
        "ProxyType": "OnPremise",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
```

```
}
```

Example: Internet SMTP Mail Destination with BasicAuthentication

Sample Code

```
{
        "Name": "example-dest-smtp",
        "Type": "MAIL",
        "Authentication": "BasicAuthentication",
        "mail.user": <username>,
        "mail.password": <password>,
        "mail.transport.protocol": "smtp",
        "mail.smtp.host": "smtp",
        "mail.smtp.auth": "true",
        "mail.smtp.port": "587",
        "ProxyType": "Internet"
}
```

Example: Internet SMTP Mail Destination with OAuth2ClientCredentials

Sample Code

```
{
        "Name": "example-dest-smtp",
        "Type": "MAIL",
        "Authentication": "OAuth2ClientCredentials",
        "mail.user": <username>,
        "mail.transport.protocol": "smtp",
        "mail.smtp.host": "smtp",
        "mail.smtp.auth": "true",
        "mail.smtp.port": "587",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: Internet SMTP Mail Destination with OAuth2RefreshToken

Sample Code

```
{
        "Name": "example-dest-smtp",
        "Type": "MAIL",
        "Authentication": "OAuth2RefreshToken",
        "mail.user": <username>,
        "mail.transport.protocol": "smtp",
        "mail.smtp.host": "smtp",
        "mail.smtp.auth": "true",
        "mail.smtp.port": "587",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
```

```
}
```

Example: Internet SMTP Mail Destination with OAuth2AuthorizationCode

Sample Code

```
{
        "Name": "example-dest-smtp",
        "Type": "MAIL",
        "Authentication": "OAuth2AuthorizationCode",
        "mail.user": <username>,
        "mail.transport.protocol": "smtp",
        "mail.smtp.host": "smtp",
        "mail.smtp.auth": "true",
        "mail.smtp.port": "587",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

To target the destination with the name "example-dest-smtp" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Example: Destination CR for an SMTP Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-smtp"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy manager will create a service for that destination on port 587, the default SMTP port (unless another destination is specified in the destination CR). The Transparent Proxy handles the basic authentication with the mail server using the credentials from the respective destination.

Multitenant Usage

The Transparent Proxy supports multitenancy for `MAIL` destinations. To consume a target system in a multitenant manner, describe all tenants in the *transparent-proxy.connectivity.api.sap/tenant-subdomains* annotation:

Example: Destination CR for a MAIL Destination in Transparent Proxy *Shared* Mode

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
  annotations:
    transparent-proxy.connectivity.api.sap/tenant-subdomains: '["tenant-
subdomain", "another-tenant-subdomain"]'
  destinationRef:
    name: "example-dest-smtp"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy will create a Kubernetes Service for each tenant described in the *transparent-proxy.connectivity.api.sap/tenant-subdomains* annotation. The names of the Kubernetes services will follow this format: <destination-cr-name>-<tenant-subdomain>.

Consumption

Once done, the application can start consuming the destination from within the Kubernetes cluster.

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

Note

In the following code snippets, a single dot on a line by itself indicates the end-of-data marker in SMTP. This notifies the server that the message body is complete.

SMTP Consumption - Bash Snippet

Sample Code

```
telnet <destination-cr-name>.<destination-cr-namespace> 587

MAIL FROM:sender@transparentproxy.com

RCPT TO:receiver@transparentproxy.com

DATA

Some test data.
.
```

SMTP Consumption for Authentication Types OAuth2RefreshToken/OAuth2AuthorizationCode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>.<destination-cr-namespace> 587

// Token Format: "user=<username>\x01auth=Bearer <access_token>\x01\x01"
where access_token is the refresh token for OAuth2RefreshToken or the
authorization code for OAuth2AuthorizationCode and username is your mail user
AUTH XOAUTH2 <base64-encoded-token>

MAIL FROM:sender@transparentproxy.com

RCPT TO:receiver@transparentproxy.com

DATA

Some test data.

.
```

SMTP Consumption in *Shared* (Multitenant) Mode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>-<tenant-subdomain>.<destination-cr-namespace> 587

MAIL FROM:sender@transparentproxy.com

RCPT TO:receiver@transparentproxy.com

DATA

Some test data.
.
```

SMTP Consumption for Authentication Types OAuth2RefreshToken/OAuth2AuthorizationCode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>-<tenant-subdomain>.<destination-cr-namespace> 587

// Token Format: "user=<username>\x01auth=Bearer <access_token>\x01\x01"
where access_token is the refresh token for OAuth2RefreshToken or the
authorization code for OAuth2AuthorizationCode and username is your mail user
AUTH XOAUTH2 <base64-encoded-token>

MAIL FROM:sender@transparentproxy.com

RCPT TO:receiver@transparentproxy.com

DATA

Some test data.

.
```

Related Information

Create MAIL Destinations [page 1484]

1.6.1.2.6 SMTPS

Consume a mail system via SMTPS through the Transparent Proxy for Kubernetes and the Destination service.

Preparation

To integrate this functionality, you must create a BTP destination in a first step:

Example: Internet SMTPS Destination with BasicAuthentication

Sample Code

```
{
        "Name": "example-dest-smtps",
        "Type": "MAIL",
        "Authentication": "BasicAuthentication",
        "mail.user": <username>,
        "mail.password": <password>,
        "mail.transport.protocol": "smtps",
        "mail.smtps.host": "smtp.gmail.com",
        "mail.smtps.auth": "true",
        "mail.smtps.port": "465",
        "ProxyType": "Internet"
}
```

Example: Internet SMTPS Mail Destination with OAuth2ClientCredentials

Sample Code

```
{
        "Name": "example-dest-smtps",
        "Type": "MAIL",
        "Authentication": "OAuth2ClientCredentials",
        "mail.user": <username>,
        "mail.transport.protocol": "smtps",
        "mail.smtps.host": "smtps",
        "mail.smtps.auth": "true",
        "mail.smtps.port": "465",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: Internet SMTPS Mail Destination with OAuth2RefreshToken

Sample Code

```
{
        "Name": "example-dest-smtps",
        "Type": "MAIL",
        "Authentication": "OAuth2RefreshToken",
        "mail.user": <username>,
        "mail.transport.protocol": "smtps",
        "mail.smtps.host": "smtps",
        "mail.smtps.auth": "true",
        "mail.smtps.port": "465",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

Example: Internet SMTPS Mail Destination with OAuth2AuthorizationCode

Sample Code

```
{
        "Name": "example-dest-smtps",
        "Type": "MAIL",
        "Authentication": "OAuth2AuthorizationCode",
        "mail.user": <username>,
        "mail.transport.protocol": "smtps",
        "mail.smtps.host": "smtps",
        "mail.smtps.auth": "true",
        "mail.smtps.port": "465",
        "ProxyType": "Internet",
        "tokenServiceURL": <token-service-url>,
        "clientId": <client-id>,
        "clientSecret": <client-secret>,
        "tokenServiceURLType": "Dedicated"
}
```

To target the destination with the name "example-dest-smtps" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] (CR) in a namespace of your choice.

Example: Destination CR for an SMTPS Destination

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-smtps"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy will create a Kubernetes Service for that destination on port 465, the default SMTPS port (unless another port is specified in the destination CR). The Transparent proxy handles basic authentication with the mail server using the credentials from the respective destination.

Multitenant Usage

The Transparent Proxy supports multitenancy for MAIL destinations. To consume a target system in a multitenant manner, describe all tenants in the `transparent-proxy.connectivity.api.sap/tenant-subdomains` annotation:

Example: Destination CR for a MAIL Destination in Transparent Proxy *Shared* Mode

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
  annotations:
    transparent-proxy.connectivity.api.sap/tenant-subdomains: '["tenant-
subdomain", "another-tenant-subdomain"]'
  destinationRef:
    name: "example-dest-smtps"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy will create a Kubernetes Service for each tenant described in the `transparent-proxy.connectivity.api.sap/tenant-subdomains` annotation. The names of the Kubernetes services will follow this format: `<destination-cr-name>-<tenant-subdomain>`.

Consumption

Once done, the application can start consuming the destination from within the Kubernetes cluster:

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

Note

In the following code snippets, a single dot on a line by itself indicates the end-of-data marker in SMTPS. This notifies the server that the message body is complete.

SMTPS Consumption - Bash Snippet

Sample Code

```
telnet <destination-cr-name>.<destination-cr-namespace> 465

MAIL FROM:sender@transparentproxy.com

RCPT TO:receiver@transparentproxy.com

DATA

Some test data.

.
```

SMTPS Consumption for Authentication Types OAuth2RefreshToken/OAuth2AuthorizationCode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>.<destination-cr-namespace> 465

// Token Format: "user=<username>\x01auth=Bearer <access_token>\x01\x01"
where access_token is the refresh token for OAuth2RefreshToken or the
authorization code for OAuth2AuthorizationCode and username is your mail user
AUTH XOAUTH2 <base64-encoded-token>

MAIL FROM:sender@transparentproxy.com

RCPT TO:receiver@transparentproxy.com

DATA

Some test data.

.
```

SMTPS Consumption in *Shared* (Multitenant) Mode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>-<tenant-subdomain>.<destination-cr-namespace> 465

MAIL FROM:sender@transparentproxy.com

RCPT TO:receiver@transparentproxy.com

DATA

Some test data.

.
```

SMTPS Consumption for Authentication Types OAuth2RefreshToken/OAuth2AuthorizationCode - Bash Snippet

Sample Code

```
telnet <destination-cr-name>-<tenant-subdomain>.<destination-cr-namespace> 465

// Token Format: "user=<username>\x01auth=Bearer <access_token>\x01\x01"
where access_token is the refresh token for OAuth2RefreshToken or the
authorization code for OAuth2AuthorizationCode and username is your mail user
AUTH XOAUTH2 <base64-encoded-token>

MAIL FROM:sender@transparentproxy.com

RCPT TO:receiver@transparentproxy.com

DATA

Some test data.

.
```

Related Information

Create MAIL Destinations [page 1484]

1.6.1.3 LDAP Destinations

The Transparent Proxy simplifies access to target systems defined as LDAP destinations. It handles the LDAP protocol for both Internet and on-premise destinations.

For on-premise connections, the Transparent Proxy performs the connectivity handshake with the Connectivity Proxy for Kubernetes [page 2088] to enable the consumption of that (otherwise complex) functionality out-of-the-box for the application developer. If the destination is configured with authentication type *BasicAuthentication*, the Transparent Proxy executes LDAP simple authentication bind with the on-premise system using the username and password provided in the destination on behalf of the client.

Mandatory Destination Configuration Fields

Property	Description
Name	Destination name
Type	Destination type. Use LDAP.
ldap.url	<virtual_host>:<virtual_port> of the on-premise application or <host>:<port> of the Internet application
ldap.proxyType	OnPremise or Internet
ldap.authentication	Use "BasicAuthentication" or "NoAuthentication".
ldap.user	Mandatory when ldap.authentication is "BasicAuthentication"

Property	Description
ldap.password	Mandatory when ldap.authentication is "BasicAuthentication"

Prerequisites

To integrate this functionality, you must:

- Fulfill the connectivity prerequisites if you are using an on-premise/private cloud connection. For more information, see On-Premise Connectivity [page 2167].
- Create an SAP BTP Destination.

Example: LDAP On-Premise Destination

Sample Code

```
{
        "Name": "example-dest-ldap",
        "Type": "LDAP",
        "ldap.user": <username>,
        "ldap.password": <password>,
        "ldap.authentication": "BasicAuthentication",
        "ldap.url": "ldap://<virtual_host>:<virtual_port>",
        "ldap.proxyType": "OnPremise"
}
```

Example: LDAP Internet Destination

Sample Code

```
{
        "Name": "example-dest-ldap",
        "Type": "LDAP",
        "ldap.user": <username>,
        "ldap.password": <password>,
        "ldap.authentication": "BasicAuthentication",
        "ldap.url": "ldap://<host>:<port>",
        "ldap.proxyType": "Internet"
}
```

To target the destination with the name "example-dest-ldap" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for an LDAP destination in Transparent Proxy *dedicated* mode

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
```

```
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-ldap"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy will create a Kubernetes Service for that destination on port 389, the default LDAP port (unless another port is specified in the destination CR). The Transparent Proxy handles the basic authentication with the LDAP server using the credentials from the respective destination.

Multitenant Usage

The Transparent Proxy supports multitenancy for LDAP destinations. To consume an end-system in a multitenant manner, describe all tenants in the *transparent-proxy.connectivity.api.sap/tenant-subdomains* annotation:

Destination CR for an LDAP destination in Transparent Proxy *shared* mode

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
  annotations:
    transparent-proxy.connectivity.api.sap/tenant-subdomains: '["tenant-
subdomain", "another-tenant-subdomain"]'
  destinationRef:
    name: "example-dest-ldap"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy will create a Kubernetes Service for each of the tenants described in the *transparent-proxy.connectivity.api.sap/tenant-subdomains* annotation. The names of the Kubernetes services will be: <destination-cr-name>-<tenant-subdomain>.

Consumption

Once done, the application ca start consuming the destination from within the Kubernetes cluster.

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

LDAP Consumption Bash Snippet

Sample Code

```
ldapsearch -x -H ldap://<destination-cr-name>.<destination-cr-namespace>:389
-b dc=example,dc=org
```

LDAP Consumption in *Shared* (Multitenant) Mode Bash Snippet

Sample Code

```
ldapsearch -x -H ldap://<tenant-subdomain>-<destination-cr-name>.<destination-
cr-namespace>:389 -b dc=example,dc=org
```

1.6.1.4 TCP Destinations

The Transparent Proxy simplifies access to target systems defined as TCP destinations. It handles the TCP protocol for both Internet and on-premise destinations.

The Connectivity Proxy for Kubernetes [page 2088] provides a SOCKS5 proxy interface that you can use to access on-premise systems via TCP-based protocols. SOCKS5 is the industry standard for proxying TCP-based traffic. For more information, see RFC 1928 .

The Transparent Proxy performs the SOCKS5 handshake with the Connectivity Proxy to enable the consumption of that (otherwise complex) functionality out-of-the-box for the application developer.

Mandatory Destination Configuration Fields

Property	Description
Name	Destination name
Type	Destination type. Use TCP.
Address	<virtual_host>:<virtual_port> of the on-premise application or <host>:<port> of the Internet application
ProxyType	OnPremise or Internet

Prerequisites

To integrate this functionality, you must:

- Fulfill the connectivity prerequisites if you are using an on-premise/private cloud connection. For more information, see On-Premise Connectivity [page 2167].
- Create an SAP BTP Destination

Example: TCP On-Premise Destination

Sample Code

```
{
        "Name": "example-dest-tcp",
        "Type": "TCP",
        "Address": "tcp://<virtual_host>:<virtual_port>",
        "ProxyType": "OnPremise"
}
```

Example: TCP Internet Destination

Sample Code

```
{
        "Name": "example-dest-tcp",
        "Type": "TCP",
        "Address": "tcp://<host>:<port>",
        "ProxyType": "Internet"
}
```

To target the destination with the name "example-dest-tcp" for handling by the Transparent Proxy, you should create a Destination Custom Resource [page 2342] in a namespace of your choice.

Destination CR for a TCP destination in Transparent Proxy *dedicated* mode

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: "example-dest-tcp"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy will create a Kubernetes Service for that destination on port 20004 (unless another port is specified in the destination CR).

Multitenant Usage

The Transparent Proxy supports multitenancy for TCP destinations. To consume a target system in a multitenant manner, describe all tenants in the *transparent-proxy.connectivity.api.sap/tenant-subdomains* annotation:

Destination CR for a TCP destination in Transparent Proxy *shared* mode

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
  annotations:
    transparent-proxy.connectivity.api.sap/tenant-subdomains: '["tenant-
subdomain", "another-tenant-subdomain"]'
  destinationRef:
    name: "example-dest-tcp"
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
```

The Transparent Proxy will create a Kubernetes Service for each of the tenants described in the *transparent-proxy.connectivity.api.sap/tenant-subdomains* annotation. The names of the Kubernetes services will be: <destination-cr-name>-<tenant-subdomain>.

Consumption

Once done, the application could start consuming the destination from within the Kubernetes cluster.

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

TCP Consumption Bash Snippet

Sample Code

```
telnet <destination-cr-name>.<destination-cr-namespace>
```

TCP Consumption in *Shared* (Multitenant) Mode Bash Snippet

Sample Code

```
telnet <destination-cr-name>-<tenant-subdomain>.<destination-cr-namespace>
```

Note

This is just an example, you should choose the right tool based on the system you want to consume.

1.6.2 Integration with SAP BTP Connectivity

Integrate the Transparent Proxy with other SAP BTP Connectivity services.

Destination Service Integration [page 2247]	Integrate the Transparent Proxy for Kubernetes with the SAP BTP Destination service.
Connectivity Proxy Integration [page 2249]	Integrate the Transparent Proxy with the connectivity proxy for Kubernetes.

1.6.2.1 Destination Service Integration

Integrate the Transparent Proxy for Kubernetes with the SAP BTP Destination service.

The Destination service lets you declaratively model a technical connection as a destination configuration, and via its REST APIs find the destination configuration that is required to access a remote service or system from your Kubernetes workload. To integrate the Transparent Proxy with the Destination service, you must:

1. Create a Destination service instance or reuse an existing one.
2. Create a service key of the service instance.
3. Prepare Destination service instance credentials
   - Option 1. Reference a Kubernetes secret
     - You can create a Kubernetes secret holding your Destination service instance key by executing this command:
       Create a Kubernetes Secret

       Sample Code

       ```
       kubectl create secret generic <secret-name> --from-literal=<secret-key>=<destination-service-instance-key> -n <namespace>
       ```

     - Reference the Kubernetes secret by providing `config.integration.destinationService.instances[n].serviceCredentials.secretName`. The secret data format can be either single-key or multi-key (e.g., secrets created by the BTP Operator for services and bindings inside a Kubernetes cluster).
       - If the secret data key format is single-key, you should also provide the value of that key as `config.integration.destinationService.instances[n].serviceCredentials.secretKey`.
       - If the secret is not in the same namespace as the Transparent Proxy, also provide `config.integration.destinationService.instances[n].serviceCredentials.secretNamespace`.
   - Option 2. Pass a secret holding the Destination service key directly in the Helm values.yaml
     - Get the service key of the Destination service instance.
     - Base64 encode the service key and place it in the values.yaml for config.integration.destinationService.instances[n].serviceCredentials.secretData. Check the Configuration Guide [page 2265].

> ### Note
>
> The base64 encoded value of the service key can only be specified during Installation with Helm [page 2253].

> ### Note
>
> For x509-based service keys with self-signed certificates prepare a Kubernetes secret holding the private key for the certificate.
>
> Example: *kubectl create secret generic x509-svc-key-private-key --from-file=pk.pem*
>
> If using another secret name or internal key in the secret, provide the required parameters (`config.integration.destinationService.instances[n].serviceCredentials.privateKey.secretName` and
>
> `config.integration.destinationService.instances[n].serviceCredentials.privateKey.secretInternalKey`) in the `values.yaml`.

4. Check the Configuration Guide [page 2265] for the `config.integration.destinationService.instances` property.

Destination Service Integration

> ### Sample Code

```
config:
  integration:
    destinationService:
      ## Name of the default destination service instance to be used when
'destinationServiceInstanceName' is not provided in the Destination Custom
Resource spec.
      #defaultInstanceName: dest-service-instance
      connectionTimeoutSeconds: 5
      readTimeoutSeconds: 10
      instances:
          - name: dest-service-instance
            serviceCredentials:
              secretKey: <secret-key>
              secretName: <secret-name>
              secretNamespace: <secret-namespace>
```

IAS Authorization

The Destination service lets you authorize using IAS as well:

1. Create a Destination service instance or reuse an existing one.
2. Create a service key for the Destination service instance.
3. Create a Cloud Identity service instance or reuse an existing one.
4. Create a service key for the Cloud Identity service instance.

5. Prepare instance credentials:
   - Option 1: Prepare Destination service instance credentials. See step.3 above.
   - Option 2: Prepare Cloud Identity service instance credentials. See step 3 above.
6. Check the Configuration Guide [page 2265] for the parameter `config.integration.destinationService.instances`.

Sample Code

```
config:
  integration:
    destinationService:
      ## Name of the default destination service instance to be used when
'destinationServiceInstanceName' is not provided in the Destination Custom
Resource spec.
      #defaultInstanceName: dest-service-instance
      connectionTimeoutSeconds: 5
      readTimeoutSeconds: 10
      instances:
          - name: dest-service-instance
            serviceCredentials:
              secretKey: <secret-key>
              secretName: <secret-name>
              secretNamespace: <secret-namespace>
```

1.6.2.2 Connectivity Proxy Integration

Integrate the Transparent Proxy with the Connectivity Proxy for Kubernetes.

The Connectivity Proxy is a Kubernetes component that connects workloads running on a Kubernetes cluster to on-premise systems exposed via the Cloud Connector [page 1602]. The Transparent Proxy itself doesn't provide direct connections to on-premise systems, that is, the Connectivity Proxy integration is mandatory for the Transparent Proxy to route to on-premise destinations. To integrate the Transparent Proxy with the Connectivity Proxy, you must:

1. Install a Connectivity Proxy instance or reuse an existing one. It must be installed in the same Kubernetes cluster where the Transparent Proxy is installed.
   For more information, see Lifecycle Management [page 2120].
2. Follow the instructions under Lifecycle Management [page 2252] and set up the `values.yaml` for integrating with the Connectivity Proxy, providing the `config.integration.connectivityProxy.serviceName`.
   For more information, see Configuration Guide [page 2265].
3. If the Connectivity Proxy runs in multi-region mode, you can link a Transparent Proxy configuration for a Destination service instance with a Connectivity Proxy local region configuration. This can be done at design time by adding an association. Doing this, you won't need to provide the HTTP header `SAP-Connectivity-Region-Configuration-Id` on each request - the Transparent Proxy will automatically pass it to the Connectivity Proxy:

> ### Note
>
> To use the Cloud Connector with a specified *Location ID*, you have two options:
>
> - Pass the Location ID via the HTTP header "SAP-Connectivity-SCC-Location_ID"
> - Use the property "CloudConnectorLocationId" in the referenced SAP BTP destination.
>
> If both methods are used, the value in the HTTP header will take precedence.
>
> For non-HTTP SAP BTP destinations, only the "CloudConnectorLocationId" property is supported.

Connectivity Proxy Integration

> ### Sample Code
>
> ```
> config:
>   integration:
>     connectivityProxy:
>       serviceName: <conn-proxy-service-name>.<conn-proxy-service-namespace>
>       connectionTimeoutSeconds: 1
> ```

Association of a local region of Connectivity Proxy with a Destination service instance in Transparent Proxy

> ### Sample Code
>
> ```
> config:
>   integration:
>     destinationService:
>       instances:
>       - name: <local-instance-name>
>         serviceCredentials:
>           secretKey: <secret-key>
>           secretName: <secret-name>
>           secretNamespace: <secret-namespace>
>         associateWith:
>           connectivityProxy:
>             locallyConfiguredRegionId: <locally-configured-conn-proxy-region-
> id>
> ```

Connectivity Proxy in Untrusted Mode

The Connectivity Proxy can be configured to require authentication on every incoming request. To do so, configure the following flags on the Connectivity Proxy CR in Kyma, or in the Connectivity Proxy Helm values when deployed on any other Kubernetes cluster:

- `config.servers.proxy.http.enableProxyAuthorization`: when `true`, the Transparent Proxy automatically mints a Bearer token for the provider subaccount and injects it on every outbound HTTP request to the Connectivity Proxy.

- `config.servers.proxy.socks5.enableProxyAuthorization`: when `true`, the Transparent Proxy automatically mints a Bearer token for the provider subaccount and uses it for SOCKS5 authentication on every outbound TCP request to the Connectivity Proxy.
- `config.servers.proxy.rfcAndLdap.enableProxyAuthorization`: when `true`, the Transparent Proxy automatically mints a Bearer token for the provider subaccount and includes it in the LDAP/RFC handshake to the Connectivity Proxy.

The flags can be changed at runtime without restarting the Transparent Proxy. The new value propagates to all Transparent Proxy pods within a few seconds.

Prerequisite: allowedClientIds

Before enabling any of these flags, you *must* add the Transparent Proxy's `client_id` to the Connectivity Proxy's allowlist. Otherwise, the Connectivity Proxy rejects every Transparent Proxy request and all on-premise traffic fails. The Transparent Proxy `client_id` is the *provider subaccount* `client_id` *of the Connectivity Proxy instance* used by the Transparent Proxy.

The allowlist is stored in the `connectivity-proxy-region-configurations` Secret in the Connectivity Proxy namespace, under `allowedClientIds`. The exact path depends on whether the Connectivity Proxy is configured in *multi-region* or *single-region* mode. Each region has its own `allowedClientIds` list, so you must add the provider subaccount `client_id` to every region to which the Transparent Proxy can route:

Example: Allowed Client IDs in Multi-Region Mode

Sample Code

```
{
   "default": {
     "allowedClientIds": ["<provider-subaccount-client-id>"],
     "dependencies": { ... }
   },
   "eu10": {
     "allowedClientIds": ["<provider-subaccount-client-id>"],
     "dependencies": { ... }
   }
 }
```

In *single-region* mode, the JSON has a flat top-level structure (no region keys). In this case, add the provider subaccount `client_id` to the single top-level `allowedClientIds` list:

Example: Allowed Client IDs in Single-Region Mode

Sample Code

```
{
   "allowedClientIds": ["<provider-subaccount-client-id>"],
   "dependencies": { ... }
 }
```

After updating the Secret, restart the Connectivity Proxy *StatefulSet* to make it pick up the new allowlist.

HTTP: customer-supplied authorization (optional)

If your application already has a Connectivity Proxy-compatible Bearer token, it can supply it explicitly with the header `connectivity-proxy-authorization`: Bearer <jwt>.

The Transparent Proxy honors this value and forwards it to the Connectivity Proxy. If the header is absent, the Transparent Proxy automatically mints a token as described above. The header name avoids a collision with the application's use of the standard `Proxy-Authorization` header.

Related Information

Connectivity Proxy for Kubernetes [page 2088]

1.6.3 Lifecycle Management

Find informationn on installation, configuration, and sizing of the Transparent Proxy for Kubernetes.

Related Information

Installation [page 2252]
Configuration Guide [page 2265]
Sizing Recommendations [page 2290]

1.6.3.1 Installation

Install the Transparent Proxy for Kubernetes using a Helm chart or the Transparent Proxy operator.

Related Information

Installation with Helm [page 2253]
Installation with Operator [page 2261]

1.6.3.1.1 Installation with Helm

Use the Helm chart to configure and manage the lifecycle of the Transparent Proxy.

> ⓘ Note
>
> Current version of the Transparent Proxy is 1.10.2.

The Transparent Proxy delivery includes a Helm chart that you can use for lifecycle management. The Helm allows full configuration via the standard Helm method of a "values.yaml" file .

> ⓘ Note
>
> If Istio is not present in the cluster, you must manually set up the *certificate manager*. Alternatively, you can disable encryption within the cluster by setting `encryptionEnabled` to `false`. However, the latter is not recommended for use in production environments.

The Transparent Proxy Helm chart is available via the RBSC (*repository-based shipment channel*) Helm repository and DockerHub.

Prerequisites

1. Prepare credentials for the Destination service instance : see Destination Service Integration [page 2247] (step 3).
2. Fill all other properties in `values.yaml` for your scenario, as described in Configuration Guide [page 2265].

   > ⚠ Caution
   >
   > Do not extract the archive and modify the default `values.yml` file included there. Instead, use your own `values.yml` and only include fields that should be overridden.

   > ⓘ Note
   >
   > By default, mTLS encryption is enabled using cert-manager . You should check the Configuration Guide [page 2265] to modify the configuration settings according to your setup, for example, referencing your *cert-manager* Issuer/ClusterIssuer .

   > → Tip
   >
   > Encryption for the Transparent Proxy components can also be disabled for test purposes or if you have implemented your own mTLS sidecar solution, for example, Istio.

   Example: Transparent Proxy values.yaml Snippet

Sample Code

```
compliance:
  ## Enables or disables FIPS mode for the Transparent Proxy components.
When enabled, all cryptographic operations performed by the Transparent
Proxy components will use FIPS-validated algorithms and libraries.
  fipsModeEnabled: false
additionalCAs:
  ## The name of the secret that allows to specify a list of CA
certificates which are to be trusted, in addition to the default ones for
the outbound communication of the Transparent Proxy components.
  #secretName: <secret-name>
  ## The key in the additionalCAs.secretName that defines the CAs to be
trusted, concatenated together one after the other.
  #secretKey: <secret-key>
  ## The namespace of the existing secret to be used, which holds the
additional CA certificates. the Destination service. Should be specified
when the additionalCAs.secretName is in a different namespace to the
Transparent Proxy's installation namespace.
  #secretNamespace: <secret-namespace>
deployment:
  image:
    ## The name of the registry from which the transparent proxy will be
downloaded.
    registry: docker.io
    ## The name of the repository from which the transparent proxy will be
downloaded.
    repository: sapse
    ## One of Always, Never, IfNotPresent. For more information, see
https://kubernetes.io/docs/concepts/containers/images#updating-images.
    pullPolicy: IfNotPresent
    ## The secret used for authenticating against the repository. (Not
required when using the DockerHub registry).
    #pullSecret: ""
    ## The version of the transparent proxy images that are to be
deployed. By default, it is the chart version
    #tag:
    ## The full image used by the manager and healthcheck config-init
containers.
    initContainer: docker.io/library/alpine:3.22.2
  replicas:
    ## The amount of transparent HTTP proxy pods to start.
    http: 1
    ## The amount of transparent TCP proxy pods to start.
    tcp: 1
  resources:
    http:
      requests:
        ## The K8s scheduler uses this information to decide which node to
place the pod on. If there are no nodes with the specified amount of CPU
resources, the pod won't be scheduled (and therefore started).
        cpu: 0.05
        ## The K8s scheduler uses this information to decide which node
to place the Pod on. If there are no nodes with the specified amount of
memory, the pod won't be scheduled (and therefore started).
        memory: 192M
      limits:
        ## The Kubelet enforces the limit so that the running container
is not allowed to use more CPU resources than the set limit. In case the
limit is crossed, the process would be throttled.
        cpu: 0.05
        ## The Kubelet enforces the limit so that the running container is
not allowed to use more memory than the set limit. In case the limit is
crossed, the process would be terminated with an out-of-memory (OOM) error.
        memory: 192M
    tcp:
      requests:
```

```
        ## The K8s scheduler uses this information to decide which node to
place the pod on. If there are no nodes with the specified amount of CPU
resources, the pod won't be scheduled (and therefore started).
        cpu: 0.01
        ## The K8s scheduler uses this information to decide which node
to place the Pod on. If there are no nodes with the specified amount of
memory, the pod won't be scheduled (and therefore started).
        memory: 64M
      limits:
        ## The Kubelet enforces the limit so that the running container
is not allowed to use more CPU resources than the set limit. In case the
limit is crossed, the process would be throttled.
        cpu: 0.01
        ## The Kubelet enforces the limit so that the running container is
not allowed to use more memory than the set limit. In case the limit is
crossed, the process would be terminated with an out-of-memory (OOM) error.
        memory: 64M
    manager:
      requests:
        ## The K8s scheduler uses this information to decide which node to
place the pod on. If there are no nodes with the specified amount of CPU
resources, the pod won't be scheduled (and therefore started).
        cpu: 0.4
        ## The K8s scheduler uses this information to decide which node
to place the Pod on. If there are no nodes with the specified amount of
memory, the pod won't be scheduled (and therefore started).
        memory: 192M
      limits:
        ## The Kubelet enforces the limit so that the running container
is not allowed to use more CPU resources than the set limit. In case the
limit is crossed, the process would be throttled.
        cpu: 0.4
        ## The Kubelet enforces the limit so that the running container is
not allowed to use more memory than the set limit. In case the limit is
crossed, the process would be terminated with an out-of-memory (OOM) error.
        memory: 192M
  autoscaling:
    http:
      vertical:
        ## Enables or disables the Vertical Pod Autoscaler mechanism for
HTTP proxy pods. For this configuration to take effect, the Vertical Pod
Autoscaling mechanism for the cluster should be enabled. See Vertical Pod
Auto-Scaling for details.
        ## Cannot enable both horizontal and vertical autoscaling. If you
want vertical autoscaling deployment.autoscaling.http.horizontal.enabled
should not be set to true
        enabled: false
        ##  The mode in which the Vertical Pod Autoscaler should operate.
For more details see https://github.com/kubernetes/autoscaler/tree/master/
vertical-pod-autoscaler#quick-start
        updateMode: "Off"
        ## Specifies the minimal amount of CPU/memory that will be
recommended for each HTTP proxy pod. By default, there is no minimum.
There are 2 containers in each HTTP proxy pod.
        #minAllowed:
          #cpu: 0.25
          #memory: 128M
        ## Specifies the minimal amount of CPU/memory that will be
recommended for each HTTP proxy pod. By default, there is no minimum.
There are 2 containers in each HTTP proxy pod.
        #maxAllowed:
          #cpu: 0.25
          #memory: 128M
      horizontal:
        ## Enables or disables the Horizontal Pod Autoscaler mechanism
see  Horizontal Pod Autoscaler(Kubernetes documentation) for HTTP proxy
pods.
```

```
        ## Cannot enable both horizontal and vertical autoscaling.If you
want horizontal autoscaling deployment.autoscaling.http.vertical.enabled
should not be set to true
        enabled: false
        ## Upper limit for the number of HTTP transparent proxy replicas
to which the autoscaler can scale up. It should be higher than
deployment.replicas.http.
        maxReplicaCount: 2
        metrics:
          ## Target value of the average CPU metric across all transparent
HTTP proxy pods, represented as a percentage of the requested value of the
CPU for the pods.
          cpuAverageUtilization: 80
          ## Target value of the average memory metric across all
transparent HTTP proxy pods, represented as a percentage of the requested
value of the memory for the pods.
          memoryAverageUtilization: 80
    tcp:
      horizontal:
        ## Enables or disables the Horizontal Pod Autoscaler mechanism.
See Horizontal Pod Autoscaler(Kubernetes documentation) for TCP proxy pods.
        ## Cannot enable both horizontal and vertical autoscaling. If you
want horizontal autoscaling deployment.autoscaling.tcp.vertical.enabled
should not be set to true
        enabled: false
        ## Upper limit for the number of TCP transparent proxy replicas
to which the autoscaler can scale up. It should be higher than
deployment.replicas.tcp.
        maxReplicaCount: 2
        metrics:
          ## Target value of the average CPU metric across all transparent
TCP proxy pods for a given destination instance, represented as a
percentage of the requested value of the CPU for the pods.
          cpuAverageUtilization: 80
          ## Target value of the average memory metric across transparent
TCP proxy pods for a given destination instance, represented as a
percentage of the requested value of the memory for the pods.
          memoryAverageUtilization: 80
      vertical:
        ## Enables or disables the Vertical Pod Autoscaler mechanism for
TCP proxy pods. In order for this configuration to take effect, the
Vertical Pod Autoscaling mechanism for the cluster should be enabled. See
Vertical Pod Auto-Scaling for details.
        ## Cannot enable both horizontal and vertical autoscaling. If you
want vertical autoscaling deployment.autoscaling.tcp.horizontal.enabled
should not be set to true
        enabled: false
        updateMode: "Off"
        ## Specifies the minimal amount of CPU/memory that will be
recommended for each TCP proxy pod. By default, there is no minimum.
        #minAllowed:
          #cpu: 0.025
          #memory: 64M
        ## Specifies the minimal amount of CPU/memory that will be
recommended for each TCP proxy pod. By default, there is no minimum.
        #maxAllowed:
          #cpu: 0.025
          #memory: 64M
  priority:
    ##     Specifies the pod priority of the TP components. By default,
there is no priority. For more details about pod priority see https://
kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/
    value: 0
config:
  helm:
    ## Skips validation of existing Kubernetes resources during Helm
installation/upgrade. When set to true, the chart will not verify the
existence of referenced resources (e.g., Connectivity Proxy ConfigMaps,
```

```
cert-manager Issuers, additional CA Secrets). This is useful for testing
helm template generation, CI/CD pipelines, or deploying in environments
where referenced resources may not exist yet. Default is false (validation
enabled).
    skipExistingResourceValidation: false
  http:
    retry:
      maxRetries: 0
      statusCodes: []
  metrics:
    prometheus:
      enabled: false
  logging:
    ## The initial log level across all transparent proxy components.
Accepted log levels are: trace, debug, info, warn, error, fatal.
    ## The log levels can be changed dynamically, see
https://help.sap.com/docs/connectivity/sap-btp-connectivity-cf/transparent-
proxy-troubleshooting?locale=en-US&q=transparent%20proxy#change-log-levels-
of-the-transparent-proxy-components
    level: info
    requestContext:
      ## Enables additional INFO logs for destination request flow
details, including destination, fragment, level, and tenant information.
      enabled: false
    ## Defines the tenant mode in which transparent proxy is working in.
Default value "dedicated" means that only destinations in the subaccount
defined in Destination Service key could be exposed and accessed via TP.
    ## The other option "shared" shows that the proxy could work with
different subscribed tenants to the provider in the service key. Then, on
each request "X-Tenant-Subdomain" or "X-Tenant-Id" header becomes required
in this mode.
  tenantMode: dedicated
  security:
    accessControl:
      destinations:
        ## Defines the default scope of Destination
CRs. Possible values are "namespaced" or "clusterWide".
See https://help.sap.com/docs/connectivity/sap-btp-connectivity-cf/
destination-custom-resource-scope?locale=en-US&q=transparent%20proxy
        defaultScope: "namespaced"
    communication:
      internal:
        ## Enables/Disables mTLS communication between the transparent
proxy components. Make sure to install cert-manager in advance.
        ## You can disable the encryption at your own risk. It is
advisable to disable it only in test environments.
        encryptionEnabled: true
        ## Certificate management types cert-manager.io(https://cert-
manager.io/docs/) and cert.gardener.cloud(https://github.com/gardener/cert-
management) are currently supported
        certManager:
            # issuerRef:
            ##  The name of the installed cert-manager issuer.
            #   name:
            ##  The cert-manager custom k8s resource type of the CA
- should be one of (Issuer or ClusterIssuer). Only applicable for cert-
manager.io.
            #   kind:
            ##  The namespace of the installed cert-manager issuer(only
applicable for cert.gardener.cloud).
            #   namespace:
            ## Certificate properties are only applicable for cert-
manager.io. If specified when using cert.gardener.cloud - will be ignored.
            certificate:
              privateKey:
                ## Check cert-manager private key
docs - https://cert-manager.io/docs/reference/api-docs/#cert-manager.io/
v1.CertificatePrivateKey
```

```
                algorithm: ECDSA
                ## Check cert-manager private key
docs - https://cert-manager.io/docs/reference/api-docs/#cert-manager.io/
v1.CertificatePrivateKey
                encoding: PKCS8
                ## Check cert-manager private key
docs - https://cert-manager.io/docs/reference/api-docs/#cert-manager.io/
v1.CertificatePrivateKey
                size: 256
              ## The duration for which the certificates will be valid.
Minimum accepted duration is 1 hour. Value must be in units accepted by Go
time.ParseDuration https://golang.org/pkg/time/#ParseDuration
              duration: 720h
              ## How long before the currently issued certificate's expiry
cert-manager should renew the certificate. Value must be in units accepted
by Go time.ParseDuration https://golang.org/pkg/time/#ParseDuration
              renewBefore: 120h
  ## The mode in which transparent proxy operates with the destinations in
the cluster. Possible values are "all" or "labelSelector".
  ## If "labelSelector" set, TP operates only in namespaces labeled with
"transparent-proxy.connectivity.api.sap/namespace:<namespace where TP is
installed in>"
  managedNamespacesMode: "all"
  manager:
    ## The execution interval on which the destination resource updates
are fetched and applied to the cluster.
    ## It is not advisable to set this value higher than 5 if there are
frequent updates on the destinations.
    executionIntervalMinutes: 3
  integration:
    ## Integration with service mesh. Currently, only Istio service mesh
is supported. The only acceptable value is 'enabled'. If it is set, the
Transparent proxy components will be integrated in the mesh, otherwise
they will be exclusively removed from the mesh.
    #serviceMesh:
      #istio:
        #istio-injection: enabled
    destinationService:
      ## Name of the default destination service instance to be used when
'destinationServiceInstanceName' is not provided in the Destination Custom
Resource spec.
      #defaultInstanceName: dest-service-instance
      ## The connect timeout used when getting access tokens and
Destination service clients. [1, 60] seconds allowed.
      connectionTimeoutSeconds: 5
      ## The read timeout used when getting access tokens and Destination
service clients. [1, 60] seconds allowed.
      readTimeoutSeconds: 10
      instances: []
    ##     Local name of the Destination service
instance that is described. It could be used as
config.integration.destinationService.defaultInstanceName or it can
be referenced in the destination custom resource spec as
destinationServiceInstanceName.
    #      - name: dest-service-instance
    #        serviceCredentials:
    ##         The key in the Destination service secret resource, which
holds the base64 encoded value of the Destination service key.
    #          secretKey: secret
    ##         The name of the existing secret, which holds the
credentials for the Destination service.
    #          secretName: dest-svc-secret
    ##         The base64 encoded value of the service key, obtained from
the Destination service instance. Required when Transparent Proxy should
create secret and not required when describing an existing secret.
    #          secretData:
    ##         The namespace of the existing secret to be used, which
holds the credentials for the Destination service.
```

```
    #          secretNamespace:
    #          privateKey:
    ##           The name of K8s secret, which holds the private key
to authenticate if using an x.509-based service key to the Destination
service.
    #            secretName: x509-svc-key-private-key
    ##           The name of the internal key inside the K8s secret
holding the private key to authenticate if using an x.509-based service
key to the Destination service.
    #            secretInternalKey: pk.pem
    ##           The namespace of the existing secret to be used, which
holds the private key.
    #            secretNamespace:
    connectivityProxy:
      ## Connectivity Proxy topology: 'inCluster' (default) or 'remote'.
      ## In remote mode the CP is reached through an Istio Egress Gateway
(or any TLS-terminating
      ## gateway) at serviceName:443.
      #type: inCluster
      ## In-cluster mode: K8s Service name of the connectivity proxy
workload.
      ## Remote mode: DNS name of the egress gateway (e.g. istio-
egressgateway.istio-system.svc.cluster.local).
      #serviceName: connectivity-proxy
      ## TCP port of the remote Connectivity Proxy endpoint. Applies only
in remote mode; ignored in in-cluster mode.
      #servicePort: 443
      ## The connect timeout is to be used when accessing an HTTP system
with ProxyType=OnPremise through the connectivity proxy. [1, 60] seconds
allowed.
      connectionTimeoutSeconds: 1
    ztis:
      ## Defines the K8s socket path for the SPIRE agent connection.
This path is where the SPIRE agent exposes its UNIX domain socket. The
transparent proxy components use this socket to request certificates and
verify identities.
      socketPath: /run/spire/agent-sockets/spire-agent.sock
```

Deploy (DockerHub)

The Transparent Proxy Helm chart is available in DockerHub.

You can directly install it with one command:

```
helm install transparent-proxy oci://registry-1.docker.io/sapse/transparent-
proxy --version <version of helm chart> --namespace <namespace> -f <path-to-
values.yaml>
```

Deploy from Repository Based Shipment Channel (RBSC)

Registry: 73554900100900006891.helmsrv.cdn.repositories.cloud.sap

Tag: 1.10.2

Authorization: See RBSC documentation.

```
helm repo add transparent-proxy-helm https://
73554900100900006891.helmsrv.cdn.repositories.cloud.sap --username <user> --
password <pass>
helm pull transparent-proxy-helm/transparent-proxy --version=<version of helm
chart>
helm repo update
```

(Optional) Prepare the Docker image configuration for the repository authentication if want to use a registry different from *docker.io*:

- Create docker secret:

  ```
  kubectl create secret docker-registry docker-secret --docker-
  server=73554900100900006891.dockersrv.cdn.repositories.cloud.sap --docker-
  username=<rbsc-user> --docker-password=<password>
  ```

- Place the name of secret it in the `values.yaml` for the `deployment.image.pullSecret` property. For more information, see Configuration Guide [page 2265].

Update / Upgrade / Downgrade

> ⚠ Caution
>
> Changing any of the `values.yaml` configurations using *helm upgrade* may result in the restart of some or all Transparent Proxy components.

When you have a Transparent Proxy deployed on the cluster, you may want to maintain it by changing configurations and/or changing its version. To do so, follow these steps:

1. Get the Transparent Proxy Helm chart. It can be the same version as the one currently installed or a different version that you want to upgrade or downgrade.
2. Prepare the `values.yaml` file for your scenario, as described in Configuration [page 2265]. Here you can just modify the one you used previously by applying the changes you desire.
3. Use the Helm CLI to upgrade the Transparent Proxy:

   ```
   helm upgrade transparent-proxy oci://registry-1.docker.io/sapse/transparent-
   proxy --version <version of helm chart> --namespace <namespace> -f <path-to-
   new-values.yaml>
   ```

Undeploy

If you need to remove the Transparent Proxy from your cluster or from a namespace, you can delete all resources installed by Helm except for the Custom Resource Definition (CRD) of type `destinations.destination.connectivity.api.sap` using the Helm CLI:

```
helm uninstall transparent-proxy
```

If you need to remove the CRD of type `destinations.destination.connectivity.api.sap`, you can do it manually using kubectl .

```
kubectl delete crd destinations.destination.connectivity.api.sap
```

This will delete the CRD `destinations.destination.connectivity.api.sap` as well as all custom resources of type `destination.connectivity.api.sap` in the cluster.

Deploy/Update/Undeploy using Landscaper

To deploy/update/undeploy the Transparent Proxy on a Kubernetes cluster via Landscaper, look at the following Guided Tour with Landscaper . You will need an LAAS instance that you can request from the corresponding team, or launch a local one using the Landscaper CLI.

1.6.3.1.2 Installation with Operator

Use the operator to install the Transparent Proxy for Kubernetes.

Prerequisites

Integration with Destination Service

You need an existing Destination service instance so the Transparent Proxy can link to it. If the SAP Business AI Platform Service Operator is installed in your Kubernetes cluster, the Transparent Proxy will automatically create a Destination service instance in your Kubernetes cluster and link to it.

Encryption

Istio injection is enabled by default. If Istio is present in the cluster, traffic between the workloads will be encrypted, making your installation more secure. The communication with the Transparent Proxy will be secure, as well as the communication from the Transparent Proxy to the Connectivity Proxy, if a Connectivity Proxy is present.

If Istio is not present, you should configure encryption with the cert-manager . Check the Configuration Guide [page 2265] to modify the configuration settings according to your setup, for example, referencing your cert-manager `Issuer` or `ClusterIssuer`.

Deploy/Upgrade

The Transparent Proxy with operator [page 2358] can be installed/upgraded in any Kubernetes cluster as follows:

Prerequisites:

- A Kubernetes cluster
- *kubectl* installed on your machine

The installation will result in Transparent Proxy operator and Transparent Proxy installed with predefined configurations in your Kubernetes cluster. The Transparent Proxy configuration is located in the Transparent Proxy custom resource and can be found in the *sap-transp-proxy-system* namespace after installation. The default configuration is the following:

```
apiVersion: operator.kyma-project.io/v1alpha1
kind: TransparentProxy
metadata:
  name: transparent-proxy
  namespace: sap-transp-proxy-system
spec:
  config:
    security:
      communication:
        internal:
          encryptionEnabled: true
    integration:
      destinationService:
        defaultInstanceName: ${SERVICE_INSTANCE_NAME}
        instances:
          - name: ${SERVICE_INSTANCE_NAME}
            serviceCredentials:
              secretKey: ${SECRET_KEY}
              secretName: ${SECRET_NAME}
              secretNamespace: ${SECRET_NAMESPACE}
      serviceMesh:
        istio:
          istio-injection: enabled
```

Deploy/Upgrade Using the SAP BTP Service Operator

This operation creates *service instance* and *service binding* custom resources in the *sap-transp-proxy-system* namespace, resulting in a destination service instance created in SAP BTP. This destination service instance is loaded in the Transparent Proxy configuration with name *sap-transp-proxy-default*.

> **Note**
>
> To upgrade to a newer version, run the same command in a Kubernetes cluster where you have already used this script to install the Transparent Proxy.

> ### Note
>
> The upgrade operation will only upgrade to the newer version of the Transparent Proxy deployments without creating an additional destination service instance or changing configurations.

Prerequisites:

- SAP Business AI Platform Service Operator set up in your Kubernetes cluster

```
sh -c "$(curl -fsSL https://raw.githubusercontent.com/sap-software/btp-transparent-proxy/refs/heads/main/deploy.sh)"
```

Deploy/Upgrade Using a Destination Service Instance Key in Plain JSON Format

The automation script creates a Kubernetes secret containing the service instance and loads it into the Transparent Proxy configuration.

> ### Note
>
> To upgrade, run the same command in a Kubernetes cluster where you have already used this script to install the Transparent Proxy.

> ### Note
>
> The upgrade operation will update the Transparent Proxy to the latest version. If you select the name of an existing destination service instance configuration during this process, that configuration will be updated. All other configurations will remain unchanged.

Prerequisites:

- Destination service instance created

```
sh -c "$(curl -fsSL https://raw.githubusercontent.com/sap-software/btp-transparent-proxy/refs/heads/main/deploy.sh)" -- --destination-service-instance-name <value> --destination-service-key '{...}'
```

Before executing the script, ensure you have the necessary values ready to replace the placeholders in the command. The following table explains each argument you'll need to provide:

Argument	Description	Required
--destination-service-instance-name (-dsin)	The local name of the Destination service instance present in the Transparent Proxy configuration. It can be later used to reference it in destination custom resources.	True
--destination-service-key (-dsk)	The service instance key obtained from the destination service instance.	True

Deploy/Upgrade Using a Kubernetes Secret Holding a Destination Service Instance Key

The automation script loads the Kubernetes secret into the Transparent Proxy configuration.

> ### Note
>
> To upgrade, run the same command in a Kubernetes cluster where you have already used this script to install the Transparent Proxy.

> ### Note
>
> The upgrade operation will update the Transparent Proxy to the latest version. If you select the name of an existing destination service instance configuration during this process, that configuration will be updated. All other configurations will remain unchanged.

Prerequisites:

- A Kubernetes secret in your cluster that holds a destination service instance key

```
sh -c "$(curl -fsSL https://raw.githubusercontent.com/sap-software/btp-transparent-proxy/refs/heads/main/deploy.sh)" -- --destination-service-instance-name <value> --destination-service-secret-name <secret-name> --destination-service-secret-key <secret-key> --destination-service-secret-namespace <namespace>
```

Before executing the script, ensure you have the necessary values ready to replace the placeholders in the command. The following table explains each argument you'll need to provide:

Argument	Description	Required
--destination-service-instance-name (-dsin)	The local name of the Destination service instance present in the Transparent Proxy configuration. It can be later used to reference it in destination custom resources. **Note** If not specified, the script uses the name of the Kubernetes secret.	False
--destination-service-secret-name (-dssn)	The name of the existing secret, which holds the credentials for the Destination service.	True
--destination-service-secret-key (-dssk)	The key in the Destination service secret resource, which holds the base64-encoded value of the destination service key.	True
--destination-service-secret-namespace (-dssn)	The namespace of the existing secret to be used, which holds the credentials for the Destination service.	True

Undeploy

To undeploy the Transparent Proxy and its operator, run:

```
sh -c "$(curl -fsSL https://raw.githubusercontent.com/sap-software/btp-transparent-proxy/refs/heads/main/undeploy.sh)"
```

> ⚠ Caution
>
> This action will delete all Transparent Proxy resources and instances that were previously created by the script or the operator.

Troubleshooting

In case of issues, refer to Transparent Proxy Custom Resource Conditions [page 2321].

1.6.3.2 Configuration Guide

Use the parameters below to configure the Transparent Proxy for Kubernetes.

> ⚠ Caution
>
> Changing any of the properties below using *helm upgrade* may result in the restart of some or all Transparent Proxy components.

> ⓘ Note
>
> [n] in the table below means that the described property is part of an array.

Configuration

Parameter	Description	Default Value	Example	Required
`additionalCAs.secretName`	Name of the secret that allows to specify a list of CA certificates which are to be trusted, in addition to the default ones for the outbound technical communication of the Transparent Proxy components.			False
`additionalCAs.secretKey`	Key in the *additionalCAs.secretName* that defines the CAs to be trusted, concatenated together one after the other.			False
`additionalCAs.secretNamespace`	Namespace of the existing secret to be used, which holds the CA certificates to the Destination service and XSUAA. Should be specified when the *additionalCAs.secretName* is in a different namespace to the Transparent Proxy's installation namespace.			False
`compliance.fipsModeEnabled`	Enables or disables FIPS (*Federal Information Processing Standard*) mode for the Transparent Proxy components. When enabled, all cryptographic operations performed by the Transparent Proxy components will use FIPS-validated algorithms and libraries.	false		False

Parameter	Description	Default Value	Example	Required
`config.helm.skipExistingResourceValidation`	Skips validation of existing Kubernetes resources during Helm installation/upgrade. When set to `true`, the chart will not verify the existence of referenced resources (for example, Connectivity Proxy ConfigMaps, cert-manager Issuers, additional CA Secrets). This is useful for testing helm template generation, CI/CD pipelines, or deploying in environments where referenced resources may not exist yet. This property is configurable only for Helm installations.	false		False

Parameter	Description	Default Value	Example	Required
`config.http.retry.maxRetries`	Defines how many additional attempts the Transparent Proxy should make for an HTTP request after the initial attempt fails due to a retry-eligible condition. The value is a non-negative integer (0 = no retries; N = up to N retry attempts after the first try). Retries are only performed when the failure matches the retry policy (for example, transport errors like timeouts/connection resets and/or responses with codes listed in `config.http.retry.statusCodes`). **Note** Keep in mind that retrying non-idempotent requests (such as POST without idempotency guarantees) can cause duplicate side effects.	0		False

Parameter	Description	Default Value	Example	Required
`config.http.retry.statusCodes`	Lists the HTTP response status codes that should trigger a retry attempt. The value is an array of integers (e.g., [408, 429, 500, 502, 503, 504]). When a response returns one of these codes, the proxy may retry the request up to `config.http.retry.maxRetries` times. If this list is empty or not set, the proxy will not retry based on HTTP response codes.	[]	[502]	False
`config.tenantMode`	Defines the tenant mode in which the Transparent Proxy is working. Default value *"dedicated"* specifies that only destinations defined in the subaccount in a service key can be exposed and accessed via the Transparent Proxy. The other option *"shared"* specifies that the proxy can work with different tenants subscribed to the provider in the service key. In this mode, on each request the *"X-Tenant-Subdomain"* or *"X-Tenant-Id"* header is required. The tenant mode *"shared"* works only for destinations of type HTTP.	"dedicated"		True
`config.integration.destinationService.connectionTimeoutSeconds`	Connect timeout that would be used when getting access tokens and Destination service clients. [1, 60] seconds allowed.	5	20	False

Parameter	Description	Default Value	Example	Required
`config.integration.destinationService.readTimeoutSeconds`	Read timeout that would be used when getting access tokens and Destination service clients. [1, 60] seconds allowed.	10	20	False
`config.integration.connectivityProxy.connectionTimeoutSeconds`	The connect timeout must be used when accessing an HTTP system with `ProxyType=OnPremise` through the Connectivity Proxy. [1, 60] seconds allowed.	1	20	False
`config.integration.connectivityProxy.serviceName`	Kubernetes service name and namespace associated with the Connectivity Proxy workload.	testservice.testnamespace	`connectivity-proxy.<namespace>`	False
`config.integration.connectivityProxy.http.port`	Port on which the HTTP interface of the Connectivity Proxy is started.	20003		False
`config.integration.connectivityProxy.tcp.port`	Port on which the TCP interface of the Connectivity Proxy is started.	20004		False
`config.integration.serviceMesh.istio.istio-injection`	Integration with Istio service mesh. The only acceptable value is 'enabled'. If set, the Transparent Proxy components will be integrated in the mesh, otherwise they will be exclusively removed from the mesh. If the configuration is changed, it triggers a restart.		"enabled"	False
`config.integration.ztis.socketPath`	The UNIX socket path to the ZTIS Spire Agent. For more information, see ZTIS documentation .	/run/spire/agent-sockets/spire-agent.sock		False

Parameter	Description	Default Value	Example	Required
`config.logging.level`	The initial log level across all Transparent Proxy components. The log levels can be changed dynamically, see Troubleshooting .	info		False
`config.logging.requestContext.enabled`	Enables additional INFO logs for destination request flow details, including destination, fragment, level, and tenant information.	false		False
`config.managedNamespacesMode`	The mode in which Transparent Proxy operates with the destinations in the cluster. Possible values are "all" or "labelSelector". If "labelSelector" is set, the Transparent Proxy operates only in namespaces labeled with *transparent-proxy.connectivity.api.sap/namespace:<namespace where Transparent Proxy is installed in>*.	all		False
`config.metrics.prometheus.enabled`	Enables the collection and exposure of metrics in the Prometheus format through an HTTP endpoint. When enabled it creates Kubernetes services for each Transparent Proxy component which can be configured for scrape in various tools understanding the OpenMetrics format.	false		False

Parameter	Description	Default Value	Example	Required
`config.manager.executionIntervalMinutes`	The execution interval in minutes, on which the destination resource updates are fetched and applied to the configurations. **→ Tip** We recommend that you do not set this value higher than 5, if there are frequent updates on destinations.	3		False
`config.security.accessControl.destinations.defaultScope`	Defines the default scope of destination custom resources. Possible values are “namespaced” or “clusterWide”. See Restrict Access Using Scoping [page 2348].	"namespaced"		True
`config.security.communication.internal.encryptionEnabled`	Enables/disables mTLS communication between the Transparent Proxy components. It should be disabled only in test environments or if you implement your own mTLS solution like Istio for example. **ⓘ Note** Make sure you install the cert-manager in advance.	true		False

Parameter	Description	Default Value	Example	Required
`config.security.communication.internal.certManager.issuerRef.kind`	The *cert-manager* custom Kubernetes resource type of the CA. Must be one of (`Issuer` or `ClusterIssuer`). Only applicable for cert-manager.io (https://cert-manager.io/docs/).		Issuer	False when encryption is disabled
`config.security.communication.internal.certManager.issuerRef.name`	The name of the installed cert-manager issuer.		sap-transp-proxy-issuer	False when encryption is disabled
`config.security.communication.internal.certManager.issuerRef.namespace`	The namespace of the installed cert-manager issuer. Only applicable for cert.gardener.cloud (https://github.com/gardener/cert-management).		sap-transp-proxy-issuer	False when encryption is disabled
`config.security.communication.internal.certManager.certificate.privateKey.algorithm`	Check cert-manager private key docs . Only applicable for cert-manager.io (https://cert-manager.io/docs/).	ECDSA		False
`config.security.communication.internal.certManager.certificate.privateKey.encoding`	Check cert-manager private key docs . Only applicable for cert-manager.io (https://cert-manager.io/docs/).	ECDSA		False
`config.security.communication.internal.certManager.certificate.privateKey.size`	Check cert-manager private key docs . Only applicable for cert-manager.io (https://cert-manager.io/docs/).	256		False

Parameter	Description	Default Value	Example	Required
`config.security.communication.internal.certManager.certificate.duration`	The duration for which the certificates will be valid. Minimum accepted duration is 1 hour. Value must be in units accepted by Go time.ParseDuration . Only applicable for cert-manager.io (https://cert-manager.io/docs/).	720h		False
`config.security.communication.internal.certManager.certificate.renewBefore`	How long before expiry the *cert-manager* should renew the currently issued certificate. Value must be in units accepted by Go time.ParseDuration . Only applicable for cert-manager.io (https://cert-manager.io/docs/).	240h		False
`config.integration.destinationService.defaultInstanceName`	Name of the default destination service instance to be used when `destinationServiceInstanceName` is not provided in the destination custom resource specification.			False
`config.integration.destinationService.instances[n].name`	Name of the destination service instance that is described. You can use it as `config.integration.destinationService.defaultInstanceName` or reference it in the destination custom resource specification as `destinationServiceInstanceName`.			True

Parameter	Description	Default Value	Example	Required
`config.integration.destinationService.instances[n].serviceCredentials.secretName`	The name of the existing secret, which holds the credentials for the Destination service or the name of the secret to be created based on the "secretName", "secretData", "secretKey" fields.			True
`config.integration.destinationService.instances[n].serviceCredentials.secretKey`	The key in the Destination service secret resource, which holds the base64-encoded value of the destination service key. **Note** Make sure you provide the right key for an existing secret.			True
`config.integration.destinationService.instances[n].serviceCredentials.secretData`	The base64-encoded value of the service key, obtained from the Destination service instance. Required when Transparent Proxy should create secret and not required when describing an existing secret. **Note** The base64 encoded value of the service key can only be specified during Installation with Helm [page 2253].			False

Parameter	Description	Default Value	Example	Required
`config.integration.destinationService.instances[n].serviceCredentials.secretNamespace`	The namespace of the existing secret to be used, which holds the credentials for the Destination service. This field should not be present if there is no existing secret and the Transparent Proxy would create one based on the "secretName", "secretData", "secretKey" fields.			False
`config.integration.destinationService.instances[n].serviceCredentials.privateKey.secretName`	The name of the Kubenetes secret, which holds the private key to authenticate if using an x.509 or x.509_attested service key to the Destination service.			False
`config.integration.destinationService.instances[n].serviceCredentials.privateKey.secretInternalKey`	The name of the internal key inside the Kubenetes secret holding the private key to authenticate if using an x.509 or x.509_attested service key to the Destination service.			False
`config.integration.destinationService.instances[n].serviceCredentials.privateKey.secretNamespace`	Namespace of the existing secret to be used, holds the private key to authenticate if using an x.509 or x.509_attested service key to the Destination service.			

Parameter	Description	Default Value	Example	Required
`config.integration.destinationService.instances[n].associateWith.connectivityProxy.locallyConfiguredRegionId`	The locally configured region id from the Connectivity Proxy multi region configurations, that will be used by default when consuming destinations from the given destination service instance. Will be overridden for HTTP scenarios if header `SAP-Connectivity-Region-Configuration-Id` is passed.			False
`deployment.image.registry`	Name of the registry from which the Transparent Proxy will be downloaded.	docker.io		False
`deployment.image.repository`	Name of the repository.	sapse		False
`deployment.image.pullPolicy`	One of `Always`, `Never`, `IfNotPresent`. For more information, see Updating Images (Kubernetes documentation).	"IfNotPresent"		False
`deployment.image.pullSecret`	The secret used for authenticating against the repository (not required when using the Docker registry).			False
`deployment.image.tag`	Version of the Transparent Proxy images to be deployed.	<Transparent Proxy version>	1.0.0	False
`deployment.replicas.http`	Amount of Transparent HTTP Proxy pods to start.	1		False
`deployment.replicas.tcp`	Amount of Transparent TCP Proxy pods to start.	1		False

Parameter	Description	Default Value	Example	Required
`deployment.resources.http.request.cpu`	The Kubernetes scheduler uses this information to decide which node to place the pod on. If there are no nodes with the specified amount of CPU resources, the pod won't be scheduled (and therefore started).	0.2		False
`deployment.resources.http.request.memory`	The Kubernetes scheduler uses this information to decide which node to place the pod on. If there are no nodes with the specified amount of memory, the pod won't be scheduled (and therefore started).	256M		False
`deployment.resources.http.limits.cpu`	The Kubelet enforces the limit so that the running container is not allowed to use more CPU resources than the set limit. If the limit is crossed, the process is throttled.	0.4		False
`deployment.resources.http.limits.memory`	The Kubelet enforces the limit so that the running container is not allowed to use more memory than the set limit. If the limit is crossed, the process is terminated with an out of memory (OOM) error.	512M		False
`deployment.resources.tcp.request.cpu`	The Kubernetes scheduler uses this information to decide which node to place the pod on. If there are no nodes with the specified amount of CPU resources, the pod won't be scheduled (and therefore started).	0.05		False

Parameter	Description	Default Value	Example	Required
`deployment.resources.tcp.request.memory`	The Kubernetes scheduler uses this information to decide which node to place the Pod on. If there are no nodes with the specified amount of memory, the pod won't be scheduled (and therefore started).	32M		False
`deployment.resources.tcp.limits.cpu`	The Kubernetes enforces the limit so that the running container is not allowed to use more CPU resources than the set limit. In case the limit is crossed, the process would be throttled.	0.1		False
`deployment.resources.tcp.limits.memory`	The Kubelet enforces the limit so that the running container is not allowed to use more memory than the set limit. In case the limit is crossed, the process would be terminated with an out-of-memory (OOM) error.	64M		False
`deployment.resources.manager.request.cpu`	The Kubernetes scheduler uses this information to decide which node to place the pod on. If there are no nodes with the specified amount of CPU resources, the pod won't be scheduled (and therefore started).	0.4		False
`deployment.resources.manager.request.memory`	The Kubernetes scheduler uses this information to decide which node to place the Pod on. If there are no nodes with the specified amount of memory, the pod won't be scheduled (and therefore started).	192M		False

Parameter	Description	Default Value	Example	Required
`deployment.resources.manager.limits.cpu`	The Kubernetes enforces the limit so that the running container is not allowed to use more CPU resources than the set limit. In case the limit is crossed, the process would be throttled.	0.4		False
`deployment.resources.manager.limits.memory`	The Kubelet enforces the limit so that the running container is not allowed to use more memory than the set limit. In case the limit is crossed, the process would be terminated with an out-of-memory (OOM) error.	192M		False
`deployment.autoscaling.http.horizontal.enabled`	Enables or disables the *Horizontal Pod Autoscaler* mechanism see Horizontal Pod Autoscaler (Kubernetes documentation) for HTTP proxy pods. **Note** You cannot enable both horizontal and vertical autoscaling. If you want to use horizontal autoscaling, `deployment.autoscaling.http.vertical.enabled` must not be set to `true`.	False		False

Parameter	Description	Default Value	Example	Required
`deployment.autoscaling.http.horizontal.maxReplicaCount`	Upper limit for the number of HTTP Transparent Proxy replicas to which the autoscaler can scale up. It should be higher than `deployment.replicas.http`.	2		False
`deployment.autoscaling.http.horizontal.metrics.cpuAverageUtilization`	Target value of the average CPU metric across all Transparent HTTP Proxy pods, represented as a percentage of the requested value of the CPU for the pods.	80		False
`deployment.autoscaling.http.horizontal.metrics.memoryAverageUtilization`	Target value of the average memory metric across all Transparent HTTP Proxy pods, represented as a percentage of the requested value of the memory for the pods.	80		False

Parameter	Description	Default Value	Example	Required
`deployment.autoscaling.http.vertical.enabled`	Enables or disables the *Vertical Pod Autoscaler* mechanism for HTTP proxy pods. To take effect, the vertical pod autoscaling mechanism for the cluster should be enabled. See Vertical Pod Auto-Scaling for details. **Note** You cannot enable both horizontal and vertical autoscaling. If you want to use vertical autoscaling, `deployment.autoscaling.http.horizontal.enabled` must not be set to `true`.	false		False
`deployment.autoscaling.http.vertical.updateMode`	The mode in which the *Vertical Pod Autoscaler* should operate. See Vertical Pod Autoscaler Quick start for details.	"Off"		False
`deployment.autoscaling.http.vertical.minAllowed.cpu`	Specifies the minimal amount of CPU that will be recommended for each HTTP proxy pod. By default there is no minimum. **Note** There are 2 containers in each HTTP proxy pod.		0.2	False

Parameter	Description	Default Value	Example	Required
`deployment.autoscaling.http.vertical.minAllowed.memory`	Specifies the minimal amount of memory that will be recommended for each HTTP proxy pod. By default, there is no minimum. **Note** There are 2 containers in each HTTP proxy pod.		256M	False
`deployment.autoscaling.http.vertical.maxAllowed.cpu`	Specifies the maximum amount of CPU that will be recommended for each HTTP proxy pod. By default, there is no maximum. **Note** There are 2 containers in each HTTP proxy pod.		0.4	False
`deployment.autoscaling.http.vertical.maxAllowed.memory`	Specifies the maximum amount of memory that will be recommended for each HTTP proxy pod. By default, there is no maximum. **Note** There are 2 containers in each HTTP proxy pod.		512M	False

Parameter	Description	Default Value	Example	Required
`deployment.autoscaling.tcp.horizontal.enabled`	Enables or disables the *Horizontal Pod Autoscaler* mechanism see Horizontal Pod Autoscaler (Kubernetes documentation) for TCP proxy pods. **Note** You cannot enable both horizontal and vertical autoscaling. If you want to use horizontal autoscaling, `deployment.autoscaling.tcp.vertical.enabled` must not be set to `true`.	False		False
`deployment.autoscaling.tcp.horizontal.maxReplicaCount`	Upper limit for the number of TCP Transparent Proxy replicas to which the autoscaler can scale up. It should be higher than `deployment.replicas.tcp`.	2		False
`deployment.autoscaling.tcp.horizontal.metrics.cpuAverageUtilization`	Target value of the average CPU metric across all Transparent TCP Proxy pods for a given destination instance, represented as a percentage of the requested value of the CPU for the pods.	80		False
`deployment.autoscaling.tcp.horizontal.metrics.memoryAverageUtilization`	Target value of the average memory metric across Transparent TCP Proxy pods for a given destination instance, represented as a percentage of the requested value of the memory for the pods.	80		False

Parameter	Description	Default Value	Example	Required
`deployment.autoscaling.tcp.vertical.enabled`	Enables or disables the *Vertical Pod Autoscaler* mechanism for TCP proxy pods. To take effect, the vertical pod autoscaling mechanism for the cluster should be enabled. See Vertical Pod Auto-Scaling for details. **Note** You cannot enable both horizontal and vertical autoscaling. If you want to use vertical autoscaling, `deployment.autoscaling.tcp.horizontal.enabled` must not be set to `true`.	false		False
`deployment.autoscaling.tcp.vertical.updateMode`	The mode in which the *Vertical Pod Autoscaler* should operate. See Vertical Pod Autoscaler Quick start for details.	"Off"		False
`deployment.autoscaling.tcp.vertical.minAllowed.cpu`	Specifies the minimal amount of CPU that will be recommended for each TCP proxy pod. By default there is no minimum.		0.2	False

Parameter	Description	Default Value	Example	Required
`deployment.autoscaling.tcp.vertical.minAllowed.memory`	Specifies the minimal amount of memory that will be recommended for each TCP proxy pod. By default there is no minimum.		256M	False
`deployment.autoscaling.tcp.vertical.maxAllowed.cpu`	Specifies the maximum amount of CPU that will be recommended for each TCP proxy pod. By default there is no maximum.		0.4	False
`deployment.autoscaling.tcp.vertical.maxAllowed.memory`	Specifies the maximum amount of memory that will be recommended for each TCP proxy pod. By default there is no maximum.		512M	False
`deployment.priority.value`	Specifies the pod priority of the Transparent Proxy components. By default in Kyma it is 2000000 (same as most other Kyma modules) and in non-Kyma it is 0. For more details about pod priorirty see Pod Priority and Preemption .	Kyma: 2000000 Other environments: 0		False

If you want to pull docker images from the *SAP Repository-Based Shipment Channel* (RBSC), use the following configuration:

- `deployment.image.registry` = `73554900100900006891.dockersrv.cdn.repositories.cloud.sap`
- `deployment.image.repository` = `com.sap.core.connectivity.transparent-proxy`
- `deployment.image.pullSecret` = See Lifecycle Management [page 2252].

1.6.3.2.1 Secure Communication Between Internal Components

Use the certificate manager or the Istio service mesh for secure internal communication of the Transparent Proxy for Kubernetes.

Related Information

Certificate Manager [page 2287]
Istio Service Mesh Integration [page 2289]

1.6.3.2.1.1 Certificate Manager

Use the certificate manager for secure internal communication of the Transparent Proxy for Kubernetes.

If you do not integrate the Transparent Proxy with the Istio service mesh, you must encrypt the traffic between the micro-components. Transparent proxy can use either cert-manager.io, or cert.gardener.cloud to encrypt the communication via mTLS between the Transparent Proxy components.

Prerequisites

Internal communication between Transparent Proxy components can be made secure by setting the Helm property `.Values.config.security.communication.internal.encryptionEnabled` to **`true`**.

Sample Code

```
config:
  security:
    communication:
      internal:
        ## Enables/Disables mTLS communication between the Transparent Proxy
components. Make sure to install cert-manager in advance.
        #It should be disabled only in test environments or if you implement
your own mTLS solution like Istio for example.
        encryptionEnabled: true
```

By default, `.Values.config.security.communication.internal.encryptionEnabled` is **`true`**. This is recommended for all productive scenarios.

Integration

To integrate a certificate manager, follow these steps:

1. Enable secure communication between internal components by setting `.Values.config.security.communication.internal.encryptionEnabled` to **`true`**.
2. Specify `.Values.config.security.communication.internal.certManager.issuerRef.name`. Certificate management types cert-manager.io and cert.gardener.cloud are currently supported.
3. Depending on the certificate management type (either cert-manager.io or `cert.gardener.cloud), specify the corresponding additional settings:
    1. When using cert-manager.io, you should specify `.Values.config.security.communication.internal.certManager.issuerRef.kind`.
    2. When using cert.gardener.cloud, you should specify `.Values.config.security.communication.internal.certManager.issuerRef.namespace`.

Certificate Manager Integration

Sample Code

```
config:
  security:
    communication:
      internal:
        ## Enables/Disables mTLS communication between the Transparent Proxy
components. Make sure to install cert-manager in advance.
        #It should be disabled only in test environments or if you implement
your own mTLS solution like Istio for example.
        encryptionEnabled: true
        ## Certificate management types cert-manager.io(https://cert-
manager.io/docs/) and cert.gardener.cloud(https://github.com/gardener/cert-
management) are currently supported
        certManager:
          issuerRef:
            ##  The name of the installed cert-manager issuer.
            name: <issuer-ref-name>
            ##  The cert-manager custom k8s resource type of the CA - should
be one of (Issuer or ClusterIssuer). Only applicable for cert-manager.io.
            #   kind: <issues-ref-kind>
            ##  The namespace of the installed cert-manager issuer(only
applicable for cert.gardener.cloud).
            #   namespace: <issuer-ref-namespace>
            ## Certificate properties are only applicable for cert-
manager.io. If specified when using cert.gardener.cloud - will be ignored.
            certificate:
              privateKey:
                ## Check cert-manager private key docs - https://cert-
manager.io/docs/reference/api-docs/#cert-manager.io/v1.CertificatePrivateKey
                algorithm: <private-key-algorithm>
                ## Check cert-manager private key docs - https://cert-
manager.io/docs/reference/api-docs/#cert-manager.io/v1.CertificatePrivateKey
                encoding: <private-key-encoding>
                ## Check cert-manager private key docs - https://cert-
manager.io/docs/reference/api-docs/#cert-manager.io/v1.CertificatePrivateKey
                size: <private-key-duration>
              ## The duration for which the certificates will be valid.
Minimum accepted duration is 1 hour. Value must be in units accepted by Go
time.ParseDuration https://golang.org/pkg/time/#ParseDuration
```

```
              duration: <certificate-validity-duration>
              ## How long before the currently issued certificate's expiry
cert-manager should renew the certificate. Value must be in units accepted by
Go time.ParseDuration https://golang.org/pkg/time/#ParseDuration
              renewBefore: <renew-before-duration>
```

> ### Note
>
> When configuring `cert-manager`, only the communication between Transparent Proxy micro-components is encrypted. To also encrypt outer communication, such as from applications on the cluster to the Transparent Proxy, or from observability services on the cluster (for example, Prometheus) to the Transparent Proxy, use a service mesh (for example, Istio).

> ### Note
>
> For more details about configuring the Transparent Proxy, check the Configuration Guide [page 2265].

1.6.3.2.1.2 Istio Service Mesh Integration

Transparent Proxy supports integration with the Istio service mesh for mTLS communication between the Transparent Proxy components.

Prerequisites

Before proceeding with the integration, it is crucial to ensure that Istio is installed on your cluster, as it is a mandatory requirement. If you are operating within a Kyma environment, Istio can be automatically installed by adding the Istio module.

> ### Caution
>
> Transparent proxy will not check whether Istio is working properly.

Integration

To integrate the Transparent Proxy with the Istio mesh, you need to enable the 'istio-injection' setting in the Transparent Proxy configuration.

Example Istio Integration

Sample Code

```
config:
    integration:
        ## Integration with service mesh. Currently, only Istio service mesh
is supported. The only acceptable value is 'enabled'. If it is set, the
Transparent Proxy components will be   integrated in the mesh, otherwise they
will be exclusively removed from the mesh.
        serviceMesh:
        istio:
            istio-injection: enabled
```

If `.Values.config.security.communication.internal.encryptionEnabled` is set to **`true`**, the Transparent Proxy is configured to integrate in the Istio service mesh and Istio is present on the cluster, integration with the certificate manager becomes optional, otherwise it is mandatory. It is possible to have both integration with Istio service mesh as well as with the certificate manager.

Note

If Transparent Proxy operator is available and `.Values.config.security.communication.internal.encryptionEnabled` is set to **`false`** but there is integration in the Istio service mesh, the Transparent Proxy CR will become in "Ready" state with message "installation is ready. Although encryptionEnabled is set to false, the traffic will be encrypted by Istio.".

Note

If you have set `config.metrics.prometheus.enabled` to **`true`**, the Prometheus endpoint will also be part of the istio service mesh.

Note

For more details about configuring the Transparent Proxy, check the Configuration Guide [page 2265].

1.6.3.3 Sizing Recommendations

When installing a transpartent proxy for Kubernetes, the first thing you need to decide is the sizing of the installation.

Overview

To get the most out of the Transparent Proxy, you must configure it in a suitable way, fitting the scenarios in which it plays a role.

Sizing Options

The following table gives basic sizing guidance. The values listed in the `CPU` and `Memory` sections correspond to the properties, which should be defined in the configuration. For more information, see Configuration Guide [page 2265]:

HTTP Proxy Sizing

Size	CPU	Memory	Max. Number of Requests per Second (RPS)	Concurrent Users	Bytes per Request (KB)
S:	• `deployment.resources.http.requests.cpu: 0.05` • `deployment.resources.http.limits.cpu: 0.05`	• `deployment.resources.http.requests.memory: 192 M` • `deployment.resources.http.limits.memory: 192 M`	10	1	1
M:	• `deployment.resources.http.requests.cpu: 0.225` • `deployment.resources.http.limits.cpu: 0.225`	• `deployment.resources.http.requests.memory: 256 M` • `deployment.resources.http.limits.memory: 256 M`	100	10	3
L:	• `deployment.resources.http.requests.cpu: 0.8` • `deployment.resources.http.limits.cpu: 0.8`	• `deployment.resources.http.requests.memory: 320 M` • `deployment.resources.http.limits.memory: 320 M`	500	50	5

Size	CPU	Memory	Max. Number of Requests per Second (RPS)	Concurrent Users	Bytes per Request (KB)
XL:	• `deployment.resources.http.requests.cpu: 1.9` • `deployment.resources.http.limits.cpu: 1.9`	• `deployment.resources.http.requests.memory: 384 M` • `deployment.resources.http.limits.memory: 384 M`	1000	100	8

TCP Proxy Sizing

Size	CPU	Memory	Max. Number of Requests per Second (RPS)	Concurrent Users	Bytes per Request (KB)
S:	• `deployment.resources.tcp.requests.cpu: 0.01` • `deployment.resources.tcp.limits.cpu: 0.01`	• `deployment.resources.tcp.requests.memory: 64 M` • `deployment.resources.tcp.limits.memory: 64 M`	10	1	1
M:	• `deployment.resources.tcp.requests.cpu: 0.025` • `deployment.resources.tcp.limits.cpu: 0.025`	• `deployment.resources.tcp.requests.memory: 80 M` • `deployment.resources.tcp.limits.memory: 80 M`	100	10	3

Size	CPU	Memory	Max. Number of Requests per Second (RPS)	Concurrent Users	Bytes per Request (KB)
L:	• `deployment.resources.tcp.requests.cpu: 0.08` • `deployment.resources.tcp.limits.cpu: 0.08`	• `deployment.resources.tcp.requests.memory: 96 M` • `deployment.resources.tcp.limits.memory: 96 M`	500	50	5
XL:	• `deployment.resources.tcp.requests.cpu: 0.2` • `deployment.resources.tcp.limits.cpu: 0.2`	• `deployment.resources.tcp.requests.memory: 128 M` • `deployment.resources.tcp.limits.memory: 128 M`	1000	100	8

> ### Note
>
> **Relation to Connectivity Proxy**
>
> The above-mentioned sizing recommendations for the TCP proxies are related to the Connectivity Proxy software component, which also acts as a tunnel server to the on-premise systems it connects to. This means that the Connectivity Proxy has to be properly sized as well, see Sizing Recommendations [page 2142] (Connectivity Proxy for Kubernetes).

> ### Note
>
> **General Note**
>
> These sizing recommendations are just a direction point. There are many factors that affect the performance offered by the Transparent Proxy, related to the specifics of your concrete scenarios, expected regular and intermittent load, and so on.
>
> The tests were made with the Transparent Proxy in *dedicated* mode with 50 HTTP or TCP destination custom resources applied respectively.

> ### Note
>
> **General Note**

These sizing recommendations are aligned with the *Pod Quality* of *Service Classes*, specifically the *Guaranteed* class. The Transparent Proxy components have the strictest resource limits, making them the least likely to face eviction.

1.6.4 Monitoring

Check the availability, status, and destination custom resources of the Transparent Proxy for Kubernetes.

Availability Monitoring

The availability check of the Transparent Proxy is the minimal verification that can be done to ensure that the Transparent Proxy is alive and working. This is done by a health check application deployed to the cluster during installation of the Transparent Proxy.

Status Check

The `/status` check performs the following :

- If there are HTTP custom resources defined → is the `sap-transp-proxy-http` alive and running?
- If there are non-HTTP custom resources defined → is the `sap-transp-proxy-tcp` pod alive and running?
- Is the `sap-transp-proxy-manager` alive and running?

The overall status provided by the `/status` endpoint is formed in the following way:

- if all used components are in `ok` status→ overall status is `ok`.
- if some, but not all components are in `critical` status → overall status is `warning`.
- if all components are in `critical` status → overall status is `critical`.

Destination Custom Resources Check

The `/destinationCRs` check provides information for all destination custom resources in the namespace where the Transparent Proxy is installed.

To find an example, see Verification and Testing [page 2330].

Monitoring

The Transparent Proxy supports the collection and exposure of metrics in *OpenMetrics* format.

The metrics are exposed through an HTTP endpoint on port **9494**.

To enable the collection and exposure of metrics, set `config.metrics.prometheus.enabled` to true.

```
config:
  metrics:
    prometheus:
      enabled: true
```

> ### Note
>
> The Transparent Proxy requires Prometheus version v2.0.0 or higher.

To obtain the collected metrics, send an `HTTP GET` request to the `/metrics` endpoint on any of the Transparent Proxy components.

```
curl http://sap-transp-proxy-manager-metrics.<transparent-proxy-namespace>:9494/
metrics // Obtain metrics for sap-transp-proxy-manager
curl http://sap-transp-proxy-healthcheck-metrics.<transparent-proxy-
namespace>:9494/metrics // Obtain metrics for sap-transp-proxy-healthcheck
curl http://sap-transp-proxy-tcp-metrics.<transparent-proxy-namespace>:9494/
metrics // Obtain metrics for sap-transp-proxy-tcp
curl http://sap-transp-proxy-http-metrics.<transparent-proxy-namespace>:9494/
metrics // Obtain metrics for sap-transp-proxy-http
```

Each component exposes metrics for CPU and memory :

> ### Sample Code
>
> ```
> # HELP tp_cpu_percent Process CPU usage percentage
> # TYPE tp_cpu_percent gauge
> tp_cpu_percent{otel_scope_name="system-metrics",otel_scope_version=""}
> 0.025630845500773086
> # HELP tp_memory_bytes Process memory usage
> # TYPE tp_memory_bytes gauge
> tp_memory_bytes{otel_scope_name="system-metrics",otel_scope_version=""}
> 4.9000448e+07
> ```

Custom metrics for sap-transp-proxy-healthcheck:

- **tp_destination_crs** - This metric tracks the total number of destination custom resources (CRs) divided by label *Available* which can be either *True* or *False* depending on the status of the destination CR.

> ### Sample Code
>
> ```
> # HELP tp_destination_crs Total Destination Crs
> # TYPE tp_destination_crs gauge
> ```

```
tp_destination_crs{Available="False",otel_scope_name="sap-transp-proxy-
healthcheck",otel_scope_version=""} 0
tp_destination_crs{Available="True",otel_scope_name="sap-transp-proxy-
healthcheck",otel_scope_version=""} 18
```

Custom metrics for sap-transp-proxy-http:

- **tp_received_requests_total** - This metric tracks the total number of requests that the transparent HTTP proxy has received.

  Sample Code

  ```
  # HELP tp_received_requests_total Total Received Requests
  # TYPE tp_received_requests_total counter
  tp_received_requests_total{otel_scope_name="sap-transp-proxy-conn-
  handler",otel_scope_version=""} 112
  ```

- **tp_target_system_requests_total** - This metric tracks the total number of requests that have been made through the whole chain to the target system.

  Sample Code

  ```
  # HELP tp_target_system_requests_total Total Target Systems Requests
  # TYPE tp_target_system_requests_total counter
  tp_target_system_requests_total{otel_scope_name="sap-transp-proxy-conn-
  handler",otel_scope_version=""} 82
  ```

Custom metrics for sap-transp-proxy-tcp:

- **tp_open_tcp_connections** - This metric tracks the total number of opened connections at the given time.

  Sample Code

  ```
  # HELP tp_open_tcp_connections Opened tcp connections
  # TYPE tp_open_tcp_connections gauge
  tp_open_tcp_connections{otel_scope_name="sap-transp-proxy-
  tcp",otel_scope_version=""} 26
  ```

- **tp_tcp_connections_total** - This metric tracks the total number of connections to the transparent TCP proxy.

  Sample Code

  ```
  # HELP tp_tcp_connections_total Total tcp connections
  # TYPE tp_tcp_connections_total counter
  tp_tcp_connections_total{otel_scope_name="sap-transp-proxy-
  tcp",otel_scope_version=""} 15
  ```

> ### Note
>
> The standard *OpenMetrics* format allows for integration with various monitoring and alerting tools, such as *Prometheus*, *Grafana*, *Dynatrace* and others.

1.6.5 Troubleshooting

Find troubleshooting information for the Transparent Proxy for Kubernetes.

Related Information

Basic CPU and Memory Monitoring [page 2297]
Common Issues and Solutions [page 2297]
Get Logs and Change Log Levels [page 2323]

1.6.5.1 Basic CPU and Memory Monitoring

Perform CPU and memory monitoring for the Transparent Proxy for Kubernetes.

You can monitor the CPU and memory of the Transparent Proxy pods in the following way:

```
kubectl top pods -l 'transparent-proxy.connectivity.api.sap/component in (http-proxy,tcp-proxy,manager)' -n <installation-namespace>
```

> ### Note
>
> For more info about interacting with pods using kubectl, see Interacting with running Pods .

1.6.5.2 Common Issues and Solutions

Find troubleshooting information for the Transparent Proxy for Kubernetes.

Related Information

Destination Custom Resource Conditions [page 2298]
Error Response Headers [page 2308]
Installation and Configuration [page 2319]
Transparent Proxy Custom Resource Conditions [page 2321]

1.6.5.2.1 Destination Custom Resource Conditions

Find an overview of destination custom resource conditions for the Transparent Proxy for Kubernetes.

Issue	Possible Solution
The call to the Kubernetes service referencing your destination returns 404.	Make sure the URL configured in the destination is valid. Wait for the next Transparent Proxy manager completion.
The call to the Kubernetes service referencing your destination returns response with status code 400. Check also 'x-error-message', 'x-error-origin' and 'x-internal-error-code' response headers for more concrete information.	Check your request headers or OAuth Configuration fields in the referenced Destination. Example for 'authorization' header of scheme - 'Authorization: Bearer <base64-encoded-token>'
The call to the Kubernetes service referencing your destination returns response with status code 400 and has response header 'x-error-message' with value ''x-token-service-tenant' request header is missing. It is required when 'tokenServiceURLType' property in the destination is of type Common.' .	Make sure you add a valid 'x-token-service-tenant' request header to each destination that has 'tokenServiceURLType' property of type Common.
The call to the Kubernetes service referencing your destination returns response with status code 400 and has response header 'x-error-message' with value 'Cannot mix header 'x-client-assertion-destination-name' with headers 'x-client-assertion' and 'x-client-assertion-type'.' .	Make sure you pass either 'x-client-assertion-destination-name' or 'x-client-assertion' and 'x-client-assertion-type' request headers and not the combination of all of them.
The call to the Kubernetes service referencing your destination returns response with status code 400 and has response header 'x-error-message' with value 'Destination service returned unsuccessful status code. Check your related request headers.' and 'Destination service' value in the 'x-error-origin' header.	Make sure you pass a valid 'authorization' header value that is appropriate for the referenced destination authentication type.
The call to the Kubernetes service referencing your destination returns response with status code 400 and has response header 'x-error-message' with value 'In multi-tenant operational mode, either 'x-tenant-id' or 'x-tenant-subdomain' header is mandatory to be provided.'.	Make sure you pass either x-tenant-id or x-tenant-subdomain header in shared mode.
The call to the Kubernetes service referencing your destination returns response with status code 400 and has response header 'x-error-message' with value 'Both 'x-tenant-id' and 'x-tenant-subdomain' headers are passed. Only one should be provided.'.	Make sure you pass either x-tenant-id or x-tenant-subdomain header but not both at the same time in shared mode.

Issue	Possible Solution
The call to the Kubernetes service referencing your destination returns response with status code 400 and has response header 'x-error-message' with value ''x-destination-name' header is missing but required for an endpoint exposing multiple destinations via the same Destination CR.'.	In case the spec.destinationRef.name property is "*" the 'x-destination-name' header is mandatory provide reference to a valid destination.
The call to the Kubernetes service referencing your destination returns response with status code 400 and has response header 'x-error-message' with value 'The value of the header 'x-destination-name' exceeds the maximum allowed length of 200'.	The provided header value is wrong since the Destination Service allow maximum length for a destination name to be 200 symbols.
The call to the Kubernetes service referencing your destination returns response with status code 400 and has response header 'x-error-message' with value 'The value of the header 'x-destination-name' contains characters that are not allowed. Check the destination name restrictions in the Destination service documentation.'.	The provided header value for 'x-destination-name' has invalid characters. Check the name restrictions in the Destination service documentation.
The call to the Kubernetes service referencing your destination returns response with status code 400 and has response header 'x-error-message' with value 'The value of the header 'x-fragment-name' exceeds the maximum allowed length of 200'.	The provided header value is wrong since the Destination Service allow maximum length for a fragment name to be 200 symbols.
The call to the Kubernetes service referencing your destination returns response with status code 400 and has response header 'x-error-message' with value 'The value of the header 'x-fragment-name' contains characters that are not allowed. Check the destination name restrictions in the Destination service documentation.'.	The provided header value for 'x-destination-name' has invalid characters. Check the name restrictions in the Destination service documentation.
The call to the Kubernetes service referencing your destination returns response with status code 400 and has response header 'x-error-message' with value 'The requested endpoint is bound to a concrete destination via the respective Destination CR. 'x-destination-name' header is not supported for this endpoint. Contact the Transparent Proxy Administrator.'.	The Destination Custom Resource is bound to a specific destination via spec.destinationRef.name property. Either change the spec.destinationRef.name property in the CR to "*" or do not pass the 'x-destination-name' header.
The call to the Kubernetes service referencing your destination returns response with status code 400 and has response header 'x-error-message' with value 'The requested endpoint is bound to a concrete destination via the respective Destination CR. 'x-fragment-name' header is not supported for this endpoint. Contact the Transparent Proxy Administrator.'.	The Destination Custom Resource is bound to a specific destination via spec.destinationRef.name property. Either change the spec.destinationRef.name property in the CR to "*" or set spec.fragmentRef.name to the desired fragment.
The call to the Kubernetes service referencing your destination returns response with status code 502 and has response header 'x-error-message' with value 'Destination '<name>' referred in Destination CR '<name>' is not found in tenant '<tenant-subdomain>'. Inspect the Destination CR or contact the Destination Administrator.' and 'Destination service' value in the 'x-error-origin' header.	Check if your referenced destination exists in the specified Destination Service instance or contact the Destination Administrator.

Issue	Possible Solution
The call to the Kubernetes service referencing your destination returns response with status code 502 and has response header 'x-error-message' with value 'Destination '<name>' with '<fragmentName>' referred in Destination CR '<name>' is not found in tenant '<tenant-subdomain>'. Inspect the Destination CR or contact the Destination Administrator.' and 'Destination service' value in the 'x-error-origin' header.	Check if your referenced destination/fragment exists in the specified Destination Service instance or contact the Destination Administrator.
The call to the Kubernetes service referencing your destination returns response with status code 502 and has response header 'x-error-message' with value 'Received invalid configuration from Destination Service, requested via destination '<destinationName>'.' and 'Destination service' value in the 'x-error-origin' header.	There could be an issue with the configuration fields of the destination. Contact the Destination Administrator.
The call to the Kubernetes service referencing your destination returns response with status code 502 and has response header 'x-error-message' with value 'Received invalid configuration from Destination Service, requested via destination '<destinationName>' and fragment '<fragmentName>'.' and 'Destination service' value in the 'x-error-origin' header.	There could be an issue with the configuration fields of the destination/fragment. Contact the Destination Administrator.
The call to the Kubernetes service referencing your destination returns response with status code 502 and has response header 'x-error-message' with value 'Invalid OAuth configuration in destination '<name>' referred in Destination CR '<name>'. Contact the Destination Administrator.' and 'Destination service' value in the 'x-error-origin' header.	Check Auth Configuration fields or your request headers in the referenced destination.
The call to the Kubernetes service referencing your destination returns response with status code 502 and has response header 'x-error-message' with value 'Invalid OAuth configuration in destination '<name>' with fragment '<fragmentName>' referred in Destination CR '<name>'. Contact the Destination Administrator.' and 'Destination service' value in the 'x-error-origin' header.	Check Auth Configuration fields or your request headers in the referenced destination/fragment.
The call to the Kubernetes service referencing your destination returns response with status code 502 and has response header 'x-error-message' with value 'On-Premise technical connectivity is not configured. There is no value provided for Connectivity Proxy service name.' and 'Transparent Proxy' value in the 'x-error-origin' header.	Make sure you provide correct Connectivity Proxy serviceName in Helm Configurations.
The call to the Kubernetes service referencing your destination returns response with status code 502, and has response header 'x-error-message' with value 'The Destination service instance name in the Destination CR '%s' is not configured as expected.' and 'Transparent Proxy' value in the 'x-error-origin' header.	Check if you pass a valid Destination Service instance in Destination CR or contact the Kubernetes cluster administrator to check the Helm configuration.

Issue	Possible Solution
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ReferencedDestination-NotFound".	Create a destination with the name "testdest". Wait for the next Transparent Proxy Manager completion.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice" and references fragment called "testfragment". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ReferencedFragment-NotFound".	Create a fragment with name "testfragment". Wait for the next Transparent Proxy Manager completion.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice" and references fragment called "testfragment". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ReferencedFragment-NotFound".	Create a fragment with name "testfragment". Wait for the next Transparent Proxy Manager completion.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ReferencedDestination-TypeNotSupported".	Update the destination with the name "testdest" Type field with supported values: "HTTP", "TCP". Wait for the next Transparent Proxy Manager completion.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has status "False" with the reason "ConfigurationError-ReferencedDestination-ProxyTypeNotSupported".	Update the destination with the name "testdest" ProxyType field with supported values: "OnPremise", "Internet". Wait for the next Transparent Proxy Manager completion.

Issue	Possible Solution
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice" and references fragment called "testfragment". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has status "False" with the reason "ConfigurationError-ReferencedDestinationWithFragment-ProxyTypeNotSupported".	Update the destination with the name "testdest" or the fragment with name "testfragment" ProxyType field with supported values: "OnPremise", "Internet". Wait for the next Transparent Proxy Manager completion.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ReferencedDestination-UrlNotSupported".	Update the destination with the name "testdest" URL field with supported values: Should not be empty; If type is HTTP, URL should start with "http://" or "https://" or without protocol. Wait for the next Transparent Proxy Manager completion.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice" and references fragment called "testfragment". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ReferencedDestinationWithFragment-UrlNotSupported".	Update the destination with the name "testdest" or the fragment with name "testfragment" URL field with supported values: Should not be empty; If type is HTTP, URL should start with "http://" or "https://" or without protocol. Wait for the next Transparent Proxy Manager completion.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice". The destination is of type TCP and ProxyType OnPremise and there is no k8s service with that name created.The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ReferencedDestination-TcpAddressNotSupported".	Update the destination with the name "testdest" Address field with supported values: Should not be empty; It should start with "tcp://" or without protocol. Wait for the next Transparent Proxy Manager completion.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice" and references fragment called "testfragment". The destination is of type TCP and ProxyType OnPremise and there is no k8s service with that name created.The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ReferencedDestinationWithFragment-TcpAddressNotSupported".	Update the destination with the name "testdest" or the fragment with name "testfragment" Address field with supported values: Should not be empty; It should start with "tcp://" or without protocol. Wait for the next Transparent Proxy Manager completion.

Issue	Possible Solution
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ReferencedDestination-LoopbackUrlNotSupported".	Update the destination with the name "testdest" URL field with supported values: Should not be empty; URL should not be loopback (loopback URLs are those, which contain exactly one of the following hosts: "localhost", "127.0.0.1", "::1", "0:0:0:0:0:0:0:1"); Wait for the next Transparent Proxy Manager completion.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice" and references fragment called "testfragment". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ReferencedDestinationWithFragment-LoopbackUrlNotSupported".	Update the destination with the name "testdest" or the fragment with name "testfragment" URL field with supported values: Should not be empty; URL should not be loopback (loopback URLs are those, which contain exactly one of the following hosts: "localhost", "127.0.0.1", "::1", "0:0:0:0:0:0:0:1"); Wait for the next Transparent Proxy Manager completion.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ReferencedDestination-KeyStoreInvalid".	Update the destination with the name "testdest" KeyStoreLocation field with supported values. Supported cert store types are p12, pfx, pem, der, cer, crt.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ReferencedDestination-TrustStoreInvalid".	Update the destination with the name "testdest" TrustStoreLocation field with supported values. Supported cert store types are p12, pfx, pem, der, cer, crt.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ReferencedDestination-HttpUrlWithTrustStoreOrKeyStoreNotSupported".	Update the destination with the name "testdest" so that it has not URL that starts with "http://" and has TrustStoreLocation or KeyStoreLocation in the same time.

Issue	Possible Solution
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice" and references fragment called "testfragment". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ReferencedDestination-HttpUrlWithTrustStoreOrKeyStoreNotSupported".	Update the destination with the name "testdest" or the fragment with name "testfragment" so that it has not URL that starts with "http://" and has TrustStoreLocation or KeyStoreLocation in the same time.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationFailed."	Wait for the next Transparent Proxy Manager completion. If the issue remains, see Recommended Actions [page 2326].
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice" and is a dynamic custom resource since it refers spec.destinationRef.name to "*". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-FragmentSpecification-NotSupported" with message: "Technical connectivity is not configured. Static declaration of a fragment is not supported for dynamic destination custom resources."	Dynamic custom resource should not have statically declared fragments in spec.fragmentRef.name. Fragment name should be provided only through the "x-fragment-name" header.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice". The call to the "testservice" Kubernetes service returns an unsuccessful status code or the Kubernetes service does not exist. The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ServiceCreationFailed.	Wait for the next Transparent Proxy Manager completion. If the issue remains, see Recommended Actions [page 2326].
The call to the Kubernetes service referencing your destination returns an unsuccessful status code or the Kubernetes service does not exist.	Wait for the next Transparent Proxy Manager completion. If the issue remains, see Recommended Actions [page 2326].
The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-DuplicateDestinationCustomResource". There is a destinations.destination.connectivity.api.sap custom resource with the same name in another namespace.	Change the name of your destinations.destination.connectivity.api.sap custom resource or remove the other one if you have control of it.

Issue	Possible Solution
The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ProxyTenantModeMismatch" and message "Technical connectivity is not configured. Connectivity Proxy tenant mode \"dedicated\" is not compatible with Transparent Proxy tenant mode \"shared\"".	The Connectivity Proxy should be configured with the same tenant mode as the Transparent Proxy.
The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-ProxyTenantModeMismatch" and message "Technical connectivity is not configured. Connectivity Proxy tenant mode \"shared\" is not compatible with Transparent Proxy tenant mode \"dedicated\"".	The Connectivity Proxy should be configured with the same tenant mode as the Transparent Proxy.
The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-MultitenancyAnnotation-Invalid" and message "Technical connectivity is not configured. \"transparent-proxy.connectivity.api.sap/tenant-subdomains\" annotation is invalid".	Use proper json array formatting when specifying the TCP tenants in your "transparent-proxy.connectivity.api.sap/tenant-subdomains" annotation.
The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "FailedToApplyConfiguration" and message "Configuration failed for tenants: [\"Tenant1\", \"Tenant2\"...]" where Tenant1, Tenant2, etc. are tenant subdomains specified in the "transparent-proxy.connectivity.api.sap/tenant-subdomains" annotation.	Check the Transparent Proxy manager for error logs. The tenants Tenant1, Tenant2 could not be onboarded because either the specified destination is not found in the destination service for the given tenants, is not valid in the destinations service or creating k8s components for the given destination/tenant failed.
The call to the Kubernetes service referencing your destination returns response with status code 500, response header 'x-error-message' with value "Tenant subdomain '<tenantSubdomain>' could not be determined as valid tenant for Destination service instance with name '<destinationServiceInstanceName>' as provided in the Transparent Proxy deployment configuration. Contact the Tenant administrator." response header 'x-error-origin' with value "XSUAA", response header 'x-internal-error-code' with value "404".	The passed tenant in x-tenant-subdomain is not found. Make sure you pass existing tenant in x-tenant-subdomain.
The call to the Kubernetes service referencing your destination returns response with status code 500, response header 'x-error-message' with value "Tenant id '<tenantSubdomain>' could not be determined as valid tenant for Destination service instance with name '<destinationServiceInstanceName>' as provided in the Transparent Proxy deployment configuration. Contact the Tenant administrator." response header 'x-error-origin' with value "XSUAA", response header 'x-internal-error-code' with value "404".	The passed tenant in x-tenant-id is not found. Make sure you pass existing tenant in x-tenant-id.

Issue	Possible Solution
The call to the Kubernetes service referencing your destination returns response with status code 500, response header 'x-error-message' with value "Tenant subdomain '<tenantSubdomain>' is not subscribed to respective provider application using Destination service instance with name '<destinationServiceInstanceName>', as provided in the Transparent Proxy deployment configuration, under provider tenant subdomain '<providerTenantSubdomain>'. Contact the Destination Administrator." response header 'x-error-origin' with value "XSUAA", response header 'x-internal-error-code' with value "401".	The passed tenant in x-tenant-subdomain is not subscribed to the provider. Make sure you pass subscribed to the provider tenant in x-tenant-subdomain.
The call to the Kubernetes service referencing your destination returns response with status code 500, response header 'x-error-message' with value "Tenant id '<tenantId>' is not subscribed to respective provider application using Destination service instance with name '<destinationServiceInstanceName>', as provided in the Transparent Proxy deployment configuration, under provider tenant subdomain '<providerTenantSubdomain>'. Contact the Destination Administrator." response header 'x-error-origin' with value "XSUAA", response header 'x-internal-error-code' with value "401".	The passed tenant in x-tenant-id is not subscribed to the provider. Make sure you pass subscribed to the provider tenant in x-tenant-id.
The call to the Kubernetes service referencing your destination returns response with status code 502, response header 'x-error-message' with value "Destination '<destinationName>' referred in Destination CR '<crName'> is not found in tenant '<tenantName>'. Inspect the Destination CR or contact the Destination Administrator." response header 'x-error-origin' with value "Destination Service", response header 'x-internal-error-code' with value "404".	The referenced destination is not found in the destination service for the given tenant.
The call to the Kubernetes service referencing your destination returns response with status code 502, response header 'x-error-message' with value "Destination '<destinationName>' with fragment '<fragmentName>' referred in Destination CR '<crName'> is not found in tenant '<tenantName>'. Inspect the Destination CR or contact the Destination Administrator." response header 'x-error-origin' with value "Destination Service", response header 'x-internal-error-code' with value "404".	The referenced destination or fragment is not found in the destination service for the given tenant.
The call to the Kubernetes service referencing your destination returns response with status code 502, response header 'x-error-message' with value "Destination '<destinationName>' not found in tenant '<tenantName>'. Contact the Destination Administrator." response header 'x-error-origin' with value "Destination Service", response header 'x-internal-error-code' with value "404".	The referenced destination is not found in the destination service for the given tenant.

Issue	Possible Solution
The call to the Kubernetes service referencing your destination returns response with status code 502, response header 'x-error-message' with value "Destination '<destinationName>' with fragment '<fragmentName>' not found in tenant '<tenantName>'. Contact the Destination Administrator." response header 'x-error-origin' with value "Destination Service", response header 'x-internal-error-code' with value "404".	The referenced destination or fragment is not found in the destination service for the given tenant.
The call to the Kubernetes service referencing your destination returns response with status code 400, response header 'x-error-message' with value "The '<headerKey>' header should be passed only once. Check your request parameters." response header 'x-error-origin' with value "Transparent Proxy",	The provided header should be passed only once.
The destinations.destination.connectivity.api.sap custom resource referencing your destination with the name "testdest" has the name "testservice". The last condition of type "Available" of the destinations.destination.connectivity.api.sap custom resource has the status "False" with the reason "ConfigurationError-K8sResourceWithNameAlreadyPresent".	There is a Kubernetes resource with the name of the Destination custom resource present in the same namespace. Rename one of the resources to remove the collision.
The call to the Kubernetes service referencing your destination returns response with status code 502, response header 'x-error-message' with value "Region with ID <regionId> is not found in the Connectivity Proxy configuration.", response header 'x-error-origin' with value " Transparent Proxy".	The region id <regionId> passed as SAP-Connectivity-Region-Configuration-Id header or statically configured for the given destination service instance in the Transparent Proxy configurations is not found in the Connectivity Proxy multi region configurations. Make sure that the used region id is existing in the Connectivity Proxy multi region configurations.
The call to the Kubernetes service referencing your destination returns response with status code 400, response header 'x-error-message' with value "Connectivity Proxy region ID is not specified. Either pass it as a 'SAP-Connectivity-Region-Configuration-Id' or set it in the Transparent Proxy configuration.", response header 'x-error-origin' with value " Transparent Proxy".	You should either pass SAP-Connectivity-Region-Configuration-Id header or statically configure the region for the given destination service instance in the Transparent Proxy configurations.
The call to the Kubernetes service referencing your destination returns response with status code 502, response header 'x-error-message' with value "Invalid Connectivity Proxy instance configuration. Contact the local Kubernetes cluster administrator to inspect the Connectivity Proxy deployment configuration.", response header 'x-error-origin' with value " Transparent Proxy".	The Connectivity Proxy multi-region config map does not exist.
The call to the Kubernetes service referencing your destination returns response with status code 502, response header 'x-error-message' with value "The referenced Connectivity Proxy instance configuration for region named <regionName> is not found or is invalid.".	The secret referenced by the Connectivity Proxy region configuration named <regionName> does not exist or is invalid.

1.6.5.2.2 Error Response Headers

When an error occurs, the Transparent Proxy returns a response containing one or more of the following error headers:

- **x-error-message**
  Provides a descriptive message associated with an error that has occurred.
- **x-error-origin**
  Describes the origin of the error. The value can be one of the following options:
  - Transparent Proxy
  - XSUAA
  - Destination Service
  - XSUAA/Destination Service
- **x-internal-error-code**
  It represents a status code generated from an internal call, such as to the Destination Service, XSUAA, or Connectivity Proxy.
- **x-request-id**
  The x-request-id header is used by Transparent Proxy to uniquely identify a request.
- **x-proxy-server**
  Specifies the proxy server that processes the user's request. The expected value for this header is 'Transparent Proxy'.

You can find a possible solution by examining the response status code along with the values in the x-error-message, x-error-origin, and x-internal-error-code headers using the following table:

Status Code	X-Error-Message	X-Error-Origin	X-Internal-Error-Code	Possible Solution
404 Not Found				Make sure the URL configured in the destination is valid. Wait for the next Transparent Proxy Manager completion.

Status Code	X-Error-Message	X-Error-Origin	X-Internal-Error-Code	Possible Solution
400 Bad Request				Check your request headers or OAuth Configuration fields in the referenced Destination. Example for 'authorization' header of scheme - 'Authorization: Bearer <base64-encoded-token>'. Check also 'x-error-message', 'x-error-origin' and 'x-internal-error-code' response headers for more concrete information.
400 Bad Request	"x-subject-token-type" request header is required for destination with Authentication: OAuth2TokenExchange	Transparent Proxy		Make sure you pass "x-subject-token-type" header when using OAuth2TokenExchange destinations.
400 Bad Request	"x-actor-token-type" request header is required when "x-actor-token" header is provided in the request	Transparent Proxy		Make sure you pass "x-actor-token-type" when "x-actor-token" is also provided in the request headers.
400 Bad Request	"x-actor-token" request header is required when "x-actor-token-type" header is provided in the request	Transparent Proxy		Make sure you pass "x-actor-token" when "x-actor-token-type" is also provided in the request headers.
400 Bad Request	'x-token-service-tenant' request header is missing. It is required when 'tokenServiceURLType' property in the destination is of type Common.	Transparent Proxy		Make sure you add a valid 'x-token-service-tenant' request header to each destination that has 'tokenServiceURLType' property of type Common.
400 Bad Request	Cannot mix header ‘x-client-assertion-destination-name‘ with headers ‘x-client-assertion‘ and ‘x-client-assertion-type	Destination Service		Make sure you pass either ‘x-client-assertion-destination-name‘ or ‘x-client-assertion‘ and ‘x-client-assertion-type‘ request headers and not the combination of all of them.

Status Code	X-Error-Message	X-Error-Origin	X-Internal-Error-Code	Possible Solution
400 Bad Request	Destination service returned unsuccessful status code. Check your related request headers.	Destination Service		Make sure you pass a valid 'authorization' header value that is appropriate for the referenced destination authentication type.
400 Bad Request	In multi-tenant operational mode, either ‘x-tenant-id‘ or ‘x-tenant-subdomain‘ header is mandatory to be provided.	Transparent Proxy		Make sure you pass either x-tenant-id or x-tenant-subdomain header in shared mode.
400 Bad Request	Both ‘x-tenant-id‘ and ‘x-tenant-subdomain‘ headers are passed. Only one should be provided.	Transparent Proxy		Make sure you pass either x-tenant-id or x-tenant-subdomain header but not both at the same time in shared mode.
400 Bad Request	'x-destination-name' header is missing but required for an endpoint exposing multiple destinations via the same Destination CR.	Transparent Proxy		In case the spec.destinationRef.name property is "*" the 'x-destination-name' header is mandatory provide reference to a valid destination.
400 Bad Request	The value of the header 'x-destination-name' exceeds the maximum allowed length of 200	Transparent Proxy		The provided header value is wrong since the Destination Service allow maximum length for a destination name to be 200 symbols.
400 Bad Request	The value of the header 'x-destination-name' contains characters that are not allowed. Check the destination name restrictions in the Destination service documentation.	Transparent Proxy		The provided header value for 'x-destination-name' has invalid characters. Check the name restrictions in the Destination service documentation.
400 Bad Request	The value of the header 'x-fragment-name' exceeds the maximum allowed length of 200	Transparent Proxy		The provided header value is wrong since the Destination Service allow maximum length for a fragment name to be 200 symbols.

Status Code	X-Error-Message	X-Error-Origin	X-Internal-Error-Code	Possible Solution
400 Bad Request	The value of the header 'x-fragment-name' contains characters that are not allowed. Check the fragment name restrictions in the Destination service documentation.	Transparent Proxy		The provided header value for 'x-destination-name' has invalid characters. Check the name restrictions in the Destination Service documentation.
400 Bad Request	The value of the header 'x-fragment-optional' should be \"true\" or \"false\".	Transparent Proxy		The provided header value for 'x-fragment-optional' has invalid characters. The value should be "true" or "false".
400 Bad Request	Destination level is invalid in the request. It should be one of the following: provider_subaccount, provider_instance, subaccount, instance.	Transparent Proxy		The provided header value for 'x-destination-level' has invalid characters. The value should be one of the following: provider_subaccount, provider_instance, subaccount, instance.
400 Bad Request	Fragment level is invalid in the request. It should be one of the following: provider_subaccount, provider_instance, subaccount, instance.	Transparent Proxy		The provided header value for 'x-fragment-level' has invalid characters. The value should be one of the following: provider_subaccount, provider_instance, subaccount, instance.

Status Code	X-Error-Message	X-Error-Origin	X-Internal-Error-Code	Possible Solution
400 Bad Request	The requested endpoint is bound to a concrete destination, statically declared in the respective Destination CR. 'x-destination-name' header is not supported for this endpoint. Contact the Transparent Proxy Administrator.	Transparent Proxy		The Destination Custom Resource is linked to a specific destination via the spec.destinationRef.name property, meaning the spec.destinationRef.name cannot be overridden with an HTTP header. You can either change the spec.destinationRef.name property in the CR to "*" to convert it to a destination gateway, or configure it directly in the spec.destinationRef.name property.
400 Bad Request	The requested endpoint is bound to a concrete destination, statically declared in the respective Destination CR. 'x-fragment-optional' header is not supported for this endpoint. Contact the Transparent Proxy Administrator.	Transparent Proxy		The Destination Custom Resource is linked to a specific destination via the spec.destinationRef.name property, meaning the spec.fragmentRef.optional cannot be overridden with an HTTP header. You can either change the spec.destinationRef.name property in the CR to "*" to convert it to a destination gateway, or configure it directly in the spec.fragmentRef.optional property.

Status Code	X-Error-Message	X-Error-Origin	X-Internal-Error-Code	Possible Solution
400 Bad Request	The requested endpoint is bound to a concrete destination, statically declared in the respective Destination CR. 'x-fragment-name' header is not supported for this endpoint. Contact the Transparent Proxy Administrator.	Transparent Proxy		The Destination Custom Resource is linked to a specific destination via the spec.destinationRef.name property, meaning the spec.fragmentRef.name cannot be overridden with an HTTP header. You can either change the spec.destinationRef.name property in the CR to "*" to convert it to a destination gateway, or configure it directly in the spec.fragmentRef.name property.
400 Bad Request	The requested endpoint is bound to a concrete destination, statically declared in the respective Destination CR. 'x-destination-level' header is not supported for this endpoint. Contact the Transparent Proxy Administrator.	Transparent Proxy		The Destination Custom Resource is linked to a specific destination via the spec.destinationRef.name property, meaning the spec.destinationRef.level cannot be overridden with an HTTP header. You can either change the spec.destinationRef.name property in the CR to **"*"** to convert it to a destination gateway, or configure it directly in the spec.destinationRef.level property.

Status Code	X-Error-Message	X-Error-Origin	X-Internal-Error-Code	Possible Solution
400 Bad Request	The requested endpoint is bound to a concrete destination, statically declared in the respective Destination CR. 'x-fragment-level' header is not supported for this endpoint. Contact the Transparent Proxy Administrator.	Transparent Proxy		The Destination Custom Resource is linked to a specific destination via the `spec.destinationRef.name` property, meaning the `spec.destinationRef.level` cannot be overridden with an HTTP header. You can either change the `spec.destinationRef.name` property in the CR to `"*"` to convert it to a destination gateway, or configure it directly in the `spec.fragmentRef.level` property.
400 Bad Request	'x-fragment-level' header is not supported when 'x-fragment-name' header is not provided.	Transparent Proxy		The 'x-fragment-level' header could only be provided when 'x-fragment-name' level is provided.
502 Bad Gateway	Destination '<name>' referred in Destination CR '<name>' is not found in tenant '<tenant-subdomain>'. Inspect the Destination CR or contact the Destination Administrator.	Destination Service		Check if your referenced destination exists in the specified Destination Service instance or contact the Destination Administrator.
502 Bad Gateway	Destination '<name>' with '<fragmentName>' referred in Destination CR '<name>' is not found in tenant '<tenant-subdomain>'. Inspect the Destination CR or contact the Destination Administrator.	Destination Service		Check if your referenced destination/fragment exists in the specified Destination Service instance or contact the Destination Administrator.

Status Code	X-Error-Message	X-Error-Origin	X-Internal-Error-Code	Possible Solution
502 Bad Gateway	Received invalid configuration from Destination Service, requested via destination '<destinationName>'.'	Destination Service		There could be an issue with the configuration fields of the destination. Contact the Destination Administrator.
502 Bad Gateway	Received invalid configuration from Destination Service, requested via destination '<destinationName>' and fragment '<fragmentName>'.	Destination Service		There could be an issue with the configuration fields of the destination/fragment. Contact the Destination Administrator.
502 Bad Gateway	Invalid OAuth configuration in destination '<name>' referred in Destination CR '<name>	Destination Service		Check Auth Configuration fields or your request headers in the referenced destination.
502 Bad Gateway	Invalid OAuth configuration in destination '<name>' with fragment '<fragmentName>' referred in Destination CR '<name>	Destination Service		Check Auth Configuration fields or your request headers in the referenced destination/fragment.
502 Bad Gateway	On-Premise technical connectivity is not configured. There is no value provided for Connectivity Proxy service name.	Transparent Proxy		Make sure you provide correct Connectivity Proxy serviceName in Helm Configurations.
502 Bad Gateway	Tenant subdomain '<tenantSubdomain>' could not be determined as valid tenant for Destination service instance with name '<destinationServiceInstanceName>' as provided in the Transparent Proxy deployment configuration. Contact the Tenant administrator.	Transparent Proxy		Check if you pass a valid Destination Service instance in Destination CR or contact the Kubernetes cluster administrator to check the Helm configuration.

Status Code	X-Error-Message	X-Error-Origin	X-Internal-Error-Code	Possible Solution
500 Internal Server Error	Tenant id '<tenantId>' could not be determined as valid tenant for Destination service instance with name '<destinationServiceInstanceName>' as provided in the Transparent Proxy deployment configuration. Contact the Tenant administrator.	XSUAA	404 Not Found	The passed tenant in x-tenant-id is not found. Make sure you pass existing tenant in x-tenant-id.
500 Internal Server Error	Tenant subdomain '<tenantSubdomain>' is not subscribed to respective provider application using Destination service instance with name '<destinationServiceInstanceName>', as provided in the Transparent Proxy deployment configuration, under provider tenant subdomain '<providerTenantSubdomain>'. Contact the Destination Administrator.	XSUAA	401 Unauthorized	The passed tenant in x-tenant-subdomain is not subscribed to the provider. Make sure you pass subscribed to the provider tenant in x-tenant-subdomain.
500 Internal Server Error	Tenant id '<tenantId>' is not subscribed to respective provider application using Destination service instance with name '<destinationServiceInstanceName>', as provided in the Transparent Proxy deployment configuration, under provider tenant subdomain '<providerTenantSubdomain>'. Contact the Destination Administrator.	XSUAA	401 Unauthorized	The passed tenant in x-tenant-id is not subscribed to the provider. Make sure you pass subscribed to the provider tenant in x-tenant-id.
500 Internal Server Error		XSUAA		The referenced destination is not found in the Destination Service for the given tenant.

Status Code	X-Error-Message	X-Error-Origin	X-Internal-Error-Code	Possible Solution
502 Bad Gateway	Destination '<destinationName>' referred in Destination CR '<crName'> is not found in tenant '<tenantName>'. Inspect the Destination CR or contact the Destination Administrator.	Destination Service	404 Not Found	The referenced destination is not found in the Destination Service for the given tenant.
502 Bad Gateway	Destination '<destinationName>' with fragment '<fragmentName>' referred in Destination CR'<crName'> is not found in tenant '<tenantName>'. Inspect the Destination CR or contact the Destination Administrator.	Destination Service	404 Not Found	The referenced destination or fragment is not found in the Destination Service for the given tenant.
502 Bad Gateway	Destination '<destinationName>' not found in tenant '<tenantName>'. Contact the Destination Administrator.	Destination Service	404 Not Found	The referenced destination is not found in the Destination Service for the given tenant.
502 Bad Gateway	Destination '<destinationName>' with fragment '<fragmentName>' not found in tenant '<tenantName>'. Contact the Destination Administrator.	Destination Service	404 Not Found	The referenced destination or fragment is not found in the Destination Service for the given tenant.
400 Bad Request	The following headers: [<headerKey1>, ...] should be passed only once. Check your request parameters.	Transparent Proxy		The provided headers should be passed only once.

Status Code	X-Error-Message	X-Error-Origin	X-Internal-Error-Code	Possible Solution
502 Bad Gateway	Region with ID <regionId> is not found in the Connectivity Proxy configuration.	Transparent Proxy		The region id <regionId> passed as SAP-Connectivity-Region-Configuration-Id header or statically configured for the given Destination Service instance in the Transparent Proxy configurations is not found in the Connectivity Proxy multi region configurations. Make sure that the used region id is existing in the Connectivity Proxy multi region configurations.
400 Bad Request	Connectivity Proxy region ID is not specified. Either pass it as a 'SAP-Connectivity-Region-Configuration-Id' or set it in the Transparent Proxy configuration.	Transparent Proxy		You should either pass SAP-Connectivity-Region-Configuration-Id header or statically configure the region for the given Destination Service instance in the Transparent Proxy configurations.
500 Internal Server Error	Invalid Connectivity Proxy instance configuration. Contact the local Kubernetes cluster administrator to inspect the Connectivity Proxy deployment configuration.	XSUAA/IAS		Check the provided header "connectivity-proxy-authorization" or Connectivity Proxy configuration secret for valid credentials.
502 Bad Gateway	Invalid Connectivity Proxy instance configuration. Contact the local Kubernetes cluster administrator to inspect the Connectivity Proxy deployment configuration.	Transparent Proxy		The Connectivity Proxy multi region config map does not exist.
502 Bad Gateway	The referenced Connectivity Proxy instance configuration for region named '<regionName>' is not found or is invalid.	Transparent Proxy		The secret referenced by the Connectivity Proxy region configuration named <regionName> does not exist or is invalid.

Status Code	X-Error-Message	X-Error-Origin	X-Internal-Error-Code	Possible Solution
502 Bad Gateway	Failed to establish connection to the Connectivity Proxy. Check the configuration and the Connectivity Proxy status.	Connectivity Proxy		Check if the Connectivity Proxy service name provided through integration.connectivity-Proxy.serviceName exists. Additionaly check if "connectivity-proxy" config map exists in the namespace where the connectivity-proxy is installed.
400 Bad Request	‘x-tenant-id‘ cannot be used when the authorization server for the given Destination service instance is IAS.	Transparent Proxy		Passing 'x-tenant-id' header when the authorization to the Destination service is done using IAS is not supported.
400 Bad Request	Authorization header cannot be set when authentication method is OAuth2TechnicalUserPropagation.	Transparent Proxy		Omit the 'Authorization' header when the authentication method is OAuth2TechnicalUserPropagation.

1.6.5.2.3 Installation and Configuration

Find troubleshooting information for installation and configuration of the Transparent Proxy for Kubernetes.

Installation Issue	Possible Solution
Applying helm configuration fails with "When .Values.config.security.communication.internal.encryptionEnabled is true, .Values.config.security.communication.internal.certManager.issuerRef.name cannot be empty".	Set value to .Values.config.security.communication.internal.certManager.issuerRef.name when .Values.config.security.communication.internal.encryptionEnabled is true.
Applying helm configuration fails with ".Values.config.security.communication.internal.certManager.issuerRef.namespace and .Values.config.security.communication.internal.certManager.issuerRef.kind are both passed. Use .Values.config.security.communication.internal.certManager.issuerRef.kind when your issuer is of type cert-manager.io and .Values.config.security.communication.internal.certManager.issuerRef.namespace when your issuer is of type cert.gardener.cloud".	Pass either .Values.config.security.communication.internal.certManager.issuerRef.kind or .Values.config.security.communication.internal.certManager.issuerRef.namespace, not both, when .Values.config.security.communication.internal.encryptionEnabled is true.

Installation Issue	Possible Solution
Applying helm configuration fails with "Neither .Values.config.security.communication.internal.certManager.issuerRef.namespace or .Values.config.security.communication.internal.certManager.issuerRef.kind is passed. Use .Values.config.security.communication.internal.certManager.issuerRef.kind when your issuer is of type cert-manager.io and .Values.config.security.communication.internal.certManager.issuerRef.namespace when your issuer is of type cert.gardener.cloud".	One of .Values.config.security.communication.internal.certManager.issuerRef.kind and .Values.config.security.communication.internal.certManager.issuerRef.namespace is required when .Values.config.security.communication.internal.encryptionEnabled is true.
Applying helm configuration fails with "Issuer of version cert.gardener.cloud/v1alpha1 with name 'test' in namespace 'test' not found".	Make sure that Issuer of version cert.gardener.cloud/v1alpha1 with name 'test' in namespace 'test' exists
Applying helm configuration fails with "Issuer of version cert-manager.io/v1 with name 'test' in namespace 'test' not found".	Make sure that Issuer of version cert-manager.io/v1 with name 'test' in namespace 'test' exists
Applying helm configuration fails with "ClusterIssuer of version cert-manager.io/v1 with name 'test' not found".	Make sure that ClusterIssuer of version cert-manager.io/v1 with name 'test' exists
Applying helm configration fails with "Istio not installed on the cluster. Revise your Istio integration configuration."	The configuration for integration in Istio service mesh is provided but Istio is not installed on the cluster. Either install Istio on the cluster or remove the configuration for integration in Istio service mesh.
Applying helm configuration fails with "An error occurred because certain necessary configuration settings are missing. Ensure that either 'config.security.communication.internal.encryptionEnabled' or 'config.integration.serviceMesh.istio.istio-integration' is provided.".	It is mandatory to integrate either with Istio or cert-manager. This is done by configuring at least one of the two properties:: config.integration.serviceMesh.istio.istio-integration and config.security.communication.internal.encryptionEnabled. config.integration.serviceMesh.istio.istio-integration makes config.security.communication.internal.encryptionEnabled optional. If "encryptionEnabled" is set to true and config.integration.serviceMesh.istio.istio-integration is missing then config.security.communication.internalCommunication.certManager.issuerRef.name and config.security.communication.internalCommunication.certManager.issuerRef.kind have to be provided.
Applying helm configuration fails with 'Connectivity Proxy tenant mode "shared" is not compatible with Transparent Proxy tenant mode "dedicated". Both must be in the same mode.'.	Make sure that the Transparent Proxy and the referenced Connectivity Proxy are in the same tenant mode.
Applying helm configuration fails with 'Connectivity Proxy tenant mode "dedicated" is not compatible with Transparent Proxy tenant mode "shared". Both must be in the same mode.'.	Make sure that the Transparent Proxy and the referenced Connectivity Proxy are in the same tenant mode.
Applying helm configuration fails with "Connectivity Proxy multi region ConfigMap with name <connectivityProxyMultiRegionConfigMapName> in namespace <connectivityProxyMultiRegionConfigMapNamespace> does not exist!".	The Connectivity Proxy multi region config map does not exist.

1.6.5.2.4 Transparent Proxy Custom Resource Conditions

When installing the Transparent Proxy via the Transparent Proxy Operator or adding the Transparent Proxy module in Kyma environment, a Transparent Proxy custom resource is created in the 'sap-transp-proxy-system' namespace. In this custom resource, you can define the configuration for the Transparent Proxy and also receive information in the status conditions whether the installation is successful or not. If the installation is not successful, use the table below to find a solution for your problem.

Common Issues and Solutions

For custom resources having Warning state refer to this table with issue/solution to the particular misconfiguration.

Check the Configuration guide after identifying your misconfiguration in the Transparent Proxy custom resource conditions.

Custom Resource Condition Messages	Solution
An error occurred because certain necessary configuration settings are missing. Ensure that either 'config.security.communication.internal.encryptionEnabled' or 'config.integration.serviceMesh.istio.istio-integration' is provided.	It is mandatory to integrate either with Istio or cert-manager. This is done by configuring at least one of the two properties: config.integration.serviceMesh.istio.istio-integration and config.security.communication.internal.encryptionEnabled. config.integration.serviceMesh.istio.istio-integration makes config.security.communication.internal.encryptionEnabled optional. If "encryptionEnabled" is set to true and config.integration.serviceMesh.istio.istio-integration is missing then config.security.communication.internalCommunication.certManager.issuerRef.name and config.security.communication.internalCommunication.certManager.issuerRef.kind have to be provided.
Custom Resource with name "<name>" already exists in this namespace.	There could be only 1 Transparent Proxy Custom resource in a single namespace. Delete the unnecessary custom resource or move it to another namespace.
When config.security.communication.internal.encryptionEnabled is true, config.security.communication.internal.certManager.issuerRef.name cannot be empty	If config.security.communication.internalCommunication.encryptionEnabled is set to true then config.security.communication.internal.certManager.issuerRef.name cannot be empty.
config.security.communication.internal.certManager.issuerRef.namespace and config.security.communication.internal.certManager.issuerRef.kind are both passed. Use config.security.communication.internal.certManager.issuerRef.kind when your issuer is of type cert-manager.io and config.security.communication.internal.certManager.issuerRef.namespace when your issuer is of type cert.gardener.cloud	If config.security.communication.internalCommunication.encryptionEnabled is set to true, you must pass either config.security.communication.internal.certManager.issuerRef.kind or config.security.communication.internal.certManager.issuerRef.namespace, not both.

Custom Resource Condition Messages	Solution
Neither config.security.communication.internal.certManager.issuerRef.namespace or config.security.communication.internal.certManager.issuerRef.kind is passed. Use config.security.communication.internal.certManager.issuerRef.kind when your issuer is of type cert-manager.io and config.security.communication.internal.certManager.issuerRef.namespace when your issuer is of type cert.gardener.cloud.	If config.security.communication.internalCommunication.encryptionEnabled is set to true, you must pass either config.security.communication.internal.certManager.issuerRef.kind or config.security.communication.internal.certManager.issuerRef.namespace, both cannot be empty.
"certificates.cert-manager.io" custom resource definition not found.	If config.security.communication.internalCommunication.encryptionEnabled is set to true and config.security.communication.internalCommunication.certManager.issuerRef.kind is not empty then cert-manager-io has to be installed in the cluster.
"certificates.cert.gardener.cloud" custom resource definition not found.	If config.security.communication.internalCommunication.encryptionEnabled is set to true and config.security.communication.internalCommunication.certManager.issuerRef.namespace is not empty then cert-manager-gardener has to be installed in the cluster.
Resource with apiVersion cert-manager.io/v1 and kind "<Issuer|ClusterIssuer>" with name "<name>" not found.	The referenced Issuer or ClusterIssuer with apiVersion cert-manager.io/v1 is not found in the cluster. In case it is of type Issuer check the Transparent Proxy namespace and name. For ClusterIssuer check the name.
Resource with apiVersion cert.gardener.cloud/v1alpha1 and kind Issuer in namespace "<namespace>" with name "<name>" not found.	The referenced Issuer with apiVersion cert.gardener.cloud/v1alpha1 is not found. Check the specified namespace to see if the resource exists.
Provide at least one Destination service instance in config.integration.destinationServiceIntegration.instances, or create and bind an instance in the sap-transp-proxy-system namespace using the SAP BTP Service Operator.	The Transparent Proxy should have at least one destination service instance configured in config.integration.destinationServiceIntegration.instances or resources of api version "services.cloud.sap.com/v1" and kind "ServiceInstance" should exist in "sap-transp-proxy-system" namespace.
Destination service instance name should be provided.	Some of the entries in config.integration.destinationService.instances does not have name specified.
secretName should be provided for destination service instance: "<name>".	The given destination service instance should have valid reference to a secret holding the service credentials for consuming the Destination Service.

Custom Resource Condition Messages	Solution
Secret with name "<name>" not found in namespace: "<namespace>".	The provided secret cannot be found in the referenced namespace. Examples: • The referenced secret holding the service credentials for the Destination Service is not found for the referenced destination service instance. Check the secretName and secretNamespace properties provided in the serviceCredentials section for the given instance in config.integration.destinationService.instances. • The referenced secret holding the service credentials for the Connectivity Proxy is not found. Check the secretName and secretNamespace properties provided in the serviceCredentials section for the given instance in config.integration.connectivityProxy.serviceCredentials.
secretName should be provided for Connectivity Proxy service instance in shared mode.	config.integration.connectivityProxy.serviceCredentials.secretName should be provided when config.integration.connectivityProxy.serviceName is not blank and Connectivity Proxy referenced by config.integration.connectivityProxy.serviceName is in shared mode.
configMap "connectivity-proxy" not found in namespace "<namespace>".	The given config map is not present in the same namespace as the service defined in config.integration.connectivityProxy.serviceName. If the service name contains a namespace the config map should be in that namespace otherwise it should be in the same namespace as the Transparent Proxy CR.
Key "connectivity-proxy-config.yml" not found in configMap`s Data from configMap: "connectivity-proxy" in namespace "<namespace>".	The referenced config map does not contain the expected "connectivity-proxy-config.yml" Data key.
Connectivity Proxy tenant mode "<tenant_mode>" is not compatible with Transparent Proxy tenant mode "<tenant_mode>".	The tenant mode defined in the Connectivity Proxy`s configmap is different from the one defined in config.tenantMode of the Transparent Proxy CR. The tenant modes of the Connectivity Proxy and Transparent Proxy must be equal (shared & shared or dedicated & dedicated).
Istio not installed on the cluster. Revise your Istio integration configuration.	The configuration for integration in Istio service mesh is provided but Istio is not installed on the cluster. Either install Istio on the cluster or remove the configuration for integration in Istio service mesh.

1.6.5.3 Get Logs and Change Log Levels

Get logs and change log levels for the Transparent Proxy for Kubernetes.

Get Logs of the Transparent Proxy Components

The Transparent Proxy consists of a Transparent Proxy Manager, Transparent HTTP Proxy, Transparent TCP Proxy, Transparent Proxy Health Check, and Transparent Proxy Operator.

1. To get the logs of the Transparent Proxy Manager, execute:

```
kubectl logs -l transparent-proxy.connectivity.api.sap/component=manager --
tail=-1 -n <installation-namespace> > transparent-proxy-manager.log
```

2. To get the logs of the Transparent HTTP Proxy, execute:

```
kubectl logs -l transparent-proxy.connectivity.api.sap/component=http-proxy
--all-containers --tail=-1 -n <installation-namespace> > transparent-http-
proxy.log
```

3. To get the logs of the Transparent TCP Proxy, execute:

```
kubectl logs -l transparent-proxy.connectivity.api.sap/component=tcp-proxy --
tail=-1 -n <installation-namespace> > transparent-tcp-proxy.log
```

4. To get the logs of the Transparent Proxy Health Check, execute:

```
kubectl logs -l transparent-proxy.connectivity.api.sap/component=healthcheck
--tail=-1 -n <installation-namespace> > transparent-proxy-health-check.log
```

5. To get the logs of the Transparent Proxy Operator (not installed when Transparent Proxy is deployed via Helm) execute:

```
kubectl logs -l transparent-proxy.connectivity.api.sap/component=operator --
tail=-1 -n <installation-namespace> > transparent-proxy-operator.log
```

Files created by the above commands:

- `transparent-proxy-manager.log`
- `transparent-http-proxy.log`
- `transparent-tcp-proxy.log`
- `transparent-proxy-health-check.log`
- `transparent-proxy-operator.log`

can be used for investigation purposes. For more information, see Recommended Actions [page 2326].

Get Status of *destinations.destination.connectivity.api.sap* Custom Resources

```
kubectl describe destinations -n <installation-namespace>
```

Change Log Levels of the Transparent Proxy Components

When the default logging level is not sufficient for debugging the issue you are facing, you can change the log level to get more insight about the problem.

Changing a log level is done without any downtime and requires no restarts. All you need to do is invoke a simple command in the namespace of the Transparent Proxy to gain more insight about a component. Here are some examples:

Prerequisites:

- Kubectl version: Client v1.25+ (Recommended: v1.28+).
- Cluster version: Kubernetes v1.25+ (ephemeral containers must be enabled).
- Version compatibility: The kubectl client version must be within +/- 1 minor version of the Kubernetes server version.
  - Example: If the server is v1.33, the client must be v1.32, v1.33, or v1.34.

    > **Note**
    >
    > Larger version gaps cause API mismatches that break the `--profile` flag.

1. To change the log level of the Transparent Proxy Manager, execute:

```
kubectl debug -it <pod-name> \
        --namespace=<installation-namespace> \
        --image=alpine \
        --target=sap-transp-proxy-manager \
        --profile=sysadmin \
        -- sh -c "printf 'log:\n  level: <log-level>' > /proc/1/root/etc/
logging/logger-config.yaml"
```

2. To change the log level of the Transparent HTTP Proxy, execute:

```
kubectl debug -it <pod-name> \
        --namespace=<installation-namespace> \
        --image=alpine \
        --target=sap-transp-proxy-http \
        --profile=sysadmin \
        -- sh -c "printf 'log:\n  level: <log-level>' > /proc/1/root/etc/
logging/logger-config.yaml"
```

3. To change the log level of a Transparent TCP Proxy, execute:

```
kubectl debug -it <pod-name> \
        --namespace=<installation-namespace> \
        --image=alpine \
        --target=sap-transp-proxy-tcp \
        --profile=sysadmin \
        -- sh -c "printf 'log:\n  level: <log-level>' > /proc/1/root/etc/
logging/logger-config.yaml"
```

4. To change the log level of the Transparent Proxy Health Check, execute:

```
kubectl debug -it <pod-name> \
        --namespace=<installation-namespace> \
        --image=alpine \
        --target=sap-transp-proxy-healthcheck \
```

```
        --profile=sysadmin \
        -- sh -c "printf 'log:\n  level: <log-level>' > /proc/1/root/etc/
logging/logger-config.yaml"
```

5. To change the log level of the Transparent Proxy Operator (installed only when Transparent Proxy is enabled as a Kyma Module in the Kyma environment [page 2332]) execute:

```
kubectl debug -it <pod-name> \
        --namespace=<installation-namespace> \
        --image=alpine \
        --target=sap-transp-proxy-operator \
        --profile=sysadmin \
        -- sh -c "printf 'log:\n  level: <log-level>' > /proc/1/root/etc/
logging/logger-config.yaml"
```

Accepted log levels are: `trace`, `debug`, `info`, `warn`, `error`, `fatal`. The default log level across all Transparent Proxy components is info.

1.6.6 Recommended Actions

To resolve issues with the Transparent Proxy for Kubernetes, follow the recommendations below.

Pre-Intervention Steps

Before doing any restarts or modifications, it is important that you collect all relevant information.

1. Check the status of the Transparent Proxy pods and collect the outcome. Example via `kubectl`:
    1. Get the status of the Transparent Proxy manager executions:

    ```
    kubectl get pods -l transparent-proxy.connectivity.api.sap/
    component=manager –n <installation-namespace>
    ```

    2. Get the status of the Transparent HTTP Proxy pods:

    ```
    kubectl get pods -l transparent-proxy.connectivity.api.sap/component=http-
    proxy –n <installation-namespace>
    ```

    3. Get the status of the Transparent TCP Proxy pods:

    ```
    kubectl get pods -l transparent-proxy.connectivity.api.sap/component=tcp-
    proxy –n <installation-namespace>
    ```

2. Perform basic availability monitoring from within the cluster as described in Monitoring [page 2294], and collect the outcome.
3. Collect the logs of all Transparent Proxy components (see Troubleshooting [page 2297]).
4. Collect the status of all `destinations.destination.connectivity.api.sap` custom resources (see Troubleshooting [page 2297]).
5. Proceed with Recovery Attempt [page 2327] below.

Recovery Attempt

The exact action to take here depends on the check result in step 2 of section Pre-Intervention Steps [page 2326]. Choose one of the two options below, according to the outcome of your checks.

- Option 1: Check Succeeds [page 2327]
- Option 2: Check Fails [page 2327]

Check Succeeds

This indicates that the Transparent Proxy is currently considered operational. However, it might still have trouble when used for real scenarios (depending on how you detect the outage).

1. Check if the outage is still ongoing:
    1. If no, issue is resolved, proceed with Request Root Cause Analysis(RCA) [page 2328].
    2. If yes, issue is not resolved, proceed with the next step.
2. Wait for the next Transparent Proxy manager completion.
3. Check if the outage is still ongoing:
    1. If no, issue is resolved, proceed with Request Root Cause Analysis(RCA) [page 2328].
    2. If yes, issue is not resolved, proceed with the next step.
4. Restart the Transparent HTTP Proxy and Transparent TCP Proxy instances. Example via `kubectl`:

    ```
    kubectl delete po -l transparent-proxy.connectivity.api.sap/component=http-
    proxy -n <installation-namespace>
    kubectl delete po -l transparent-proxy.connectivity.api.sap/component=tcp-
    proxy -n <installation-namespace>
    ```

5. Collect the logs from the Transparent Proxy components after the restart completes.
6. Check if outage is still ongoing:
    1. If no, issue is resolved, proceed with Request Root Cause Analysis(RCA) [page 2328].
    2. If yes, issue is not resolved, proceed with Request Help from SAP [page 2328].

Check Fails

This indicates that the Transparent Proxy itself is indeed having issues. Please perform the following steps:

1. Wait for the next Transparent Proxy manager completion.
2. Check if the outage is still ongoing:
    1. If no, issue is resolved, proceed with Request Root Cause Analysis(RCA) [page 2328].
    2. If yes, issue is not resolved, proceed with the next step.

3. Restart the Transparent HTTP Proxy and Transparent TCP Proxy instances. Example via `kubectl`:

```
kubectl delete po -l transparent-proxy.connectivity.api.sap/component=http-
proxy -n <installation-namespace>
kubectl delete po -l transparent-proxy.connectivity.api.sap/component=tcp-
proxy -n <installation-namespace>
```

4. Collect logs from the Transparent Proxy after the restart completes.
5. Check if outage is still ongoing:
   1. If no, issue is resolved, proceed with Request Root Cause Analysis(RCA) [page 2328].
   2. If yes, issue is not resolved, proceed with Request Help from SAP [page 2328].

Request Help from SAP

If you cannot resolve the issue and require help from SAP, follow this procedure:

1. Open an incident on the support component BC-CP-CON-K8S-TP for the Transparent Proxy (with the appropriate priority and impact stated).
2. Provide all the collected information in the incident, including all the logs, timestamps of the events that occurred, a summary of the taken actions, the version of the Transparent Proxy you are using, and so on.
3. Engage your SAP contacts to help with this.
4. Continue working in parallel to identify as much information as possible or to find a temporary measure to mitigate the outage.

Request Root Cause Analysis (RCA)

Once the issue is resolved, the next step is figuring out what exactly caused the issue and if there is something that can be done to prevent it from happening in the future.

Follow this procedure for requesting RCA for the issue you experienced:

1. Open an incident on the support component for the Transparent Proxy on the support component BC-CP-CON-K8S-TP.
2. Provide all the collected information in the incident, including all the logs, timestamps of the events that occurred, a summary of the taken actions, the version of the Transparent Proxy you are using, and so on.

The incident will be handled according to the SAP incident SLAs.

1.6.7 Resilience

Improve resilience of the Transparent Proxy for Kubernetes.

Even though the Transparent Proxy is not a cloud service, it offers options to improve resilience and robustness of the whole system benefiting from the Transparent Proxy functionality.

The software architecture of the Transparent Proxy in terms of HTTP and TCP proxies is loosely decoupled, meaning that if there are failures in one of them it would not affect the other. Thus they are independent of each other and errors regarding for example the Transparent HTTP Proxy should not affect the work of the Transparent TCP Proxy instances.

To achieve additional resilience, see High Availability [page 2329].

1.6.7.1 High Availability

Run the Transparent Proxy for Kubernetes in a high-availability setup.

Overview

The Transparent Proxy can work in an active-active high-availability setup. In this setup, there are at least two Transparent Proxy instances, both running actively and simultaneously.

The main purpose of an active-active deployment is to provide high availability and allow zero-downtime maintenance as well as horizontal-scaling capabilities.

> **Note**
>
> The load balancing strategy for distributing traffic to the Connectivity Proxy pods depends on the kube-proxy mode. The strategies used by the different kube-proxy modes are described in the Kubernetes documentation. This configuration is done on cluster level.

Scaling

If you need to scale your Transparent Proxy, you can do so by updating your Helm chart. Follow these steps:

1. For horizontal scaling of the TCP and HTTP proxy, update the `values.yaml`, and more concretely `deployment.replicas.tcp` for HTTP and `deployment.replicas.tcp` for TCP, as described in the Configuration [page 2265] section.
2. For vertical scaling, update the respective `deployment.resources.limits` and `deployment.resources.requests` properties for *CPU* and *Memory* described in the Configuration [page 2265] section.
3. To enable autoscaling of the Transparent Proxy, update the respective `deployment.autoscaling.*` properties.

> ### Note
>
> Horizontal and vertical autoscaling cannot be activated simultaneously. You have to choose one or the other.

Multi-AZ

If the Kubernetes cluster is configured with two or more nodes running in different availability zones (AZ), the Transparent Proxy automatically deploys and runs in the AZs for the Kubernetes cluster that matches the nodes.

1.6.8 Verification and Testing

Check the Transparent Proxy for Kubernetes after installation.

Once you have installed the Transparent Proxy in your cluster, you can perform the following steps to verify it is running successfully.

> ### Note
>
> You may have to wait for a few seconds before all the components are started and can be consumed.

To check the status of the Transparent Proxy's components, execute the following, replacing the namespace placeholder:

```
kubectl run perform-hc --image=radial/busyboxplus:curl -it --rm --restart=Never
-- curl -w "\n" 'sap-transp-proxy-int-healthcheck.{installation-namespace}/
status'
```

or the following commands when your transaparent proxy is started within *Istio Service Mesh*:

```
kubectl run perform-hc --image=radial/busyboxplus:curl --
labels="sidecar.istio.io/inject=true" -it --rm
curl -w '\n' sap-transp-proxy-int-healthcheck.{installation-namespace}/status
exit
```

and observe the result.

Example:

```
{
    "Status": "ok", // possible values: "ok|warning|critical"
    "Components": {
```

```
        "sap-transp-proxy-http": {
            "Status": "ok", // possible values: "ok|warning|critical"
            "Message":  "" // optional if status is not ok.
        },
        "sap-transp-proxy-tcp": {
            "Status": "ok", // possible values: "ok|warning|critical"
            "Message":  "", // optional if status is not ok.
            "AffectedDeployments": []// optional if status is not ok. Lists all
sap-transp-proxy-tcp deployments that have unready pods.
        },
        "sap-transp-proxy-manager": {  "Status": "ok", // possible values: "ok|
warning|critical"
            "Message":  "" // optional if status is not ok.
        }
    }
}
```

To check if all destination custom resources are configured properly, execute:

```
kubectl run perform-hc --image=curlimages/curl -it --rm --restart=Never
-- curl -w "\n" 'sap-transp-proxy-int-healthcheck.{installation-namespace}/
destinationCRs'
```

and observe the result.

Example:

```
{
    "Status": "ok",
    "CustomResources": [{
        "Name": "httpexample",
        "DestinationRef":  {
            "name" : "httpdestination"
        },
        "CreationTimestamp": "2022-08-26T11:05:30Z",
        "Available": "True",
        "Message": "Technical connectivity is configured. Kubernetes service
with name httpexample is created.",
        "Reason": "ConfigurationSuccessful"
    }, {
        "Name": "tcpexample",
        "DestinationRef":  {
            "name" : "tcpdestination"
        },
        "CreationTimestamp": "2022-08-26T11:05:34Z",
        "Available": "True",
        "Message": "Technical connectivity is configured. Kubernetes service
with name tcpexample is created.",
        "Reason": "ConfigurationSuccessful"
    }]
}
```

If you encounter problems with any of the above steps, see Troubleshooting [page 2297] and Recommended Actions [page 2326] for further investigation.

Now you can proceed with consuming the Transparent Proxy from your Kubernetes applications.

1.6.9 Transparent Proxy in the Kyma Environment

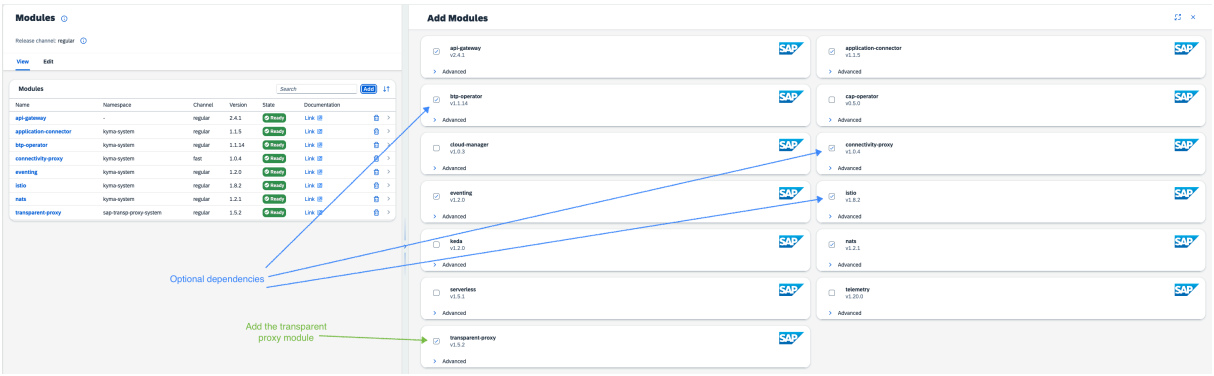
Use the Transparent Proxy in the Kyma environment.

This documentation serves as a reference for the Transparent Proxy installation in the Kyma environment and the respective Transparent Proxy Operator [page 2358].

Add the Transparent Proxy Module

To add the Transparent Proxy module, refer to Add and Delete a Kyma Module.

Navigate to the module management form and opt-in for the Transparent Proxy module.



> ⚠ Caution
>
> The **sap-transp-proxy-system** namespace is intended for the monitoring of the Transparent Proxy resources, do not use it for deploying workloads there.

> ⚠ Caution
>
> All resources created in **sap-transp-proxy-system** will be deleted (and also the namespace) with the deletion of the Transparent Proxy itself - this includes the default Destination service instance and the Transparent Proxy module created initially.

Once the module is successfully added, a Transparent Proxy custom resource gets created in the "sap-transp-proxy-system" namespace. You can modify this resource to suit your requirements. For more information, check Transparent Proxy custom resource section here: Installation with Operator [page 2261].

Dependencies

The optional dependencies automate the installation and configuration of the Transparent Proxy:

- With the BTP Operator in place, the Transparent Proxy will create a Destination service instance in your Kyma cluster and link itself to it.
- Istio will encrypt the traffic between the workloads, making your Kyma instance more secure. The communication with the Transparent Proxy will be secure, as well as the communication from the Transparent Proxy to the Connectivity Proxy, if a Connectivity Proxy is present in the Kyma instance.
- The Connectivity Proxy becomes mandatory if you want to consume an on-premise system.

For more comprehensive information about the Transparent Proxy dependencies, see Transparent Proxy Operator [page 2358].

Connectivity Proxy and On-Premise Connectivity

If you want to connect to an on-premise system, you will need the Connectivity Proxy module added in your Kyma instance. The Transparent Proxy will automatically connect with the Connectivity Proxy if the Connectivity Proxy module is added. If you choose to manually link to a Connectivity Proxy, the automatic connection will not override your configuration.

Consume a System Through the Transparent Proxy

For more information about consumption of target systems through the Transparent Proxy, see Using the Transparent Proxy [page 2166].

Troubleshooting

In case of issues, please refer to Transparent Proxy Custom Resource Conditions [page 2321].

Related Information

Create a Destination Custom Resource in the Kyma Environment [page 2334]

1.6.9.1 Create a Destination Custom Resource in the Kyma Environment

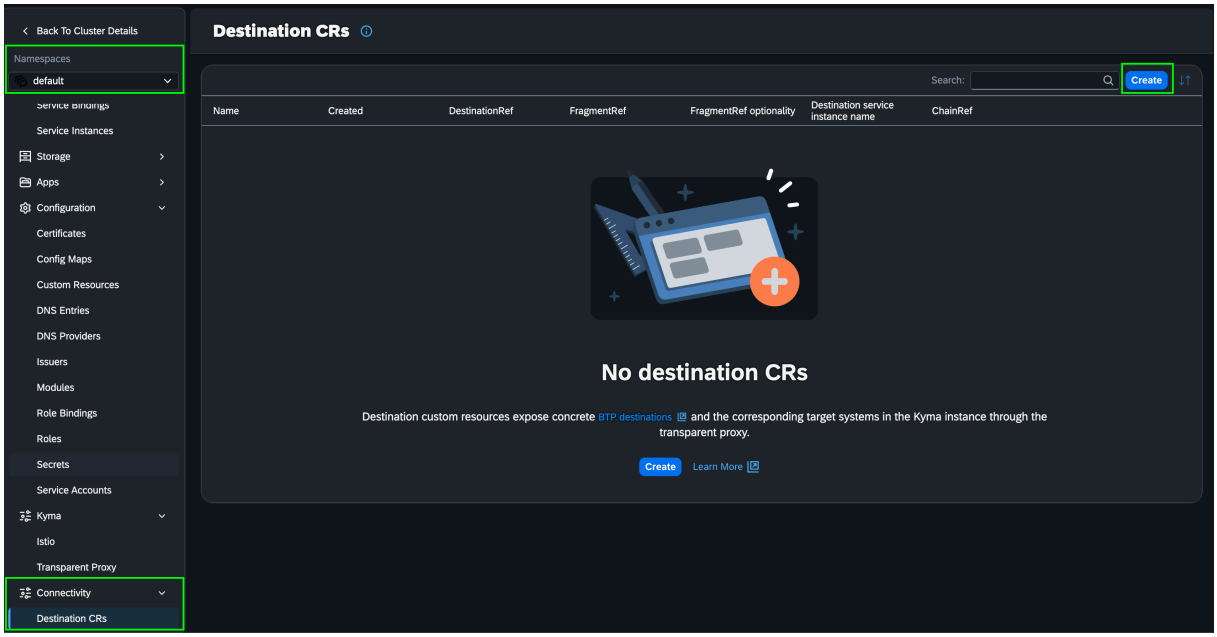
Create a Destination Custom Resource in the Kyma environment to expose SAP BTP destinations as Kyma-internal endpoints.

Prerequisites

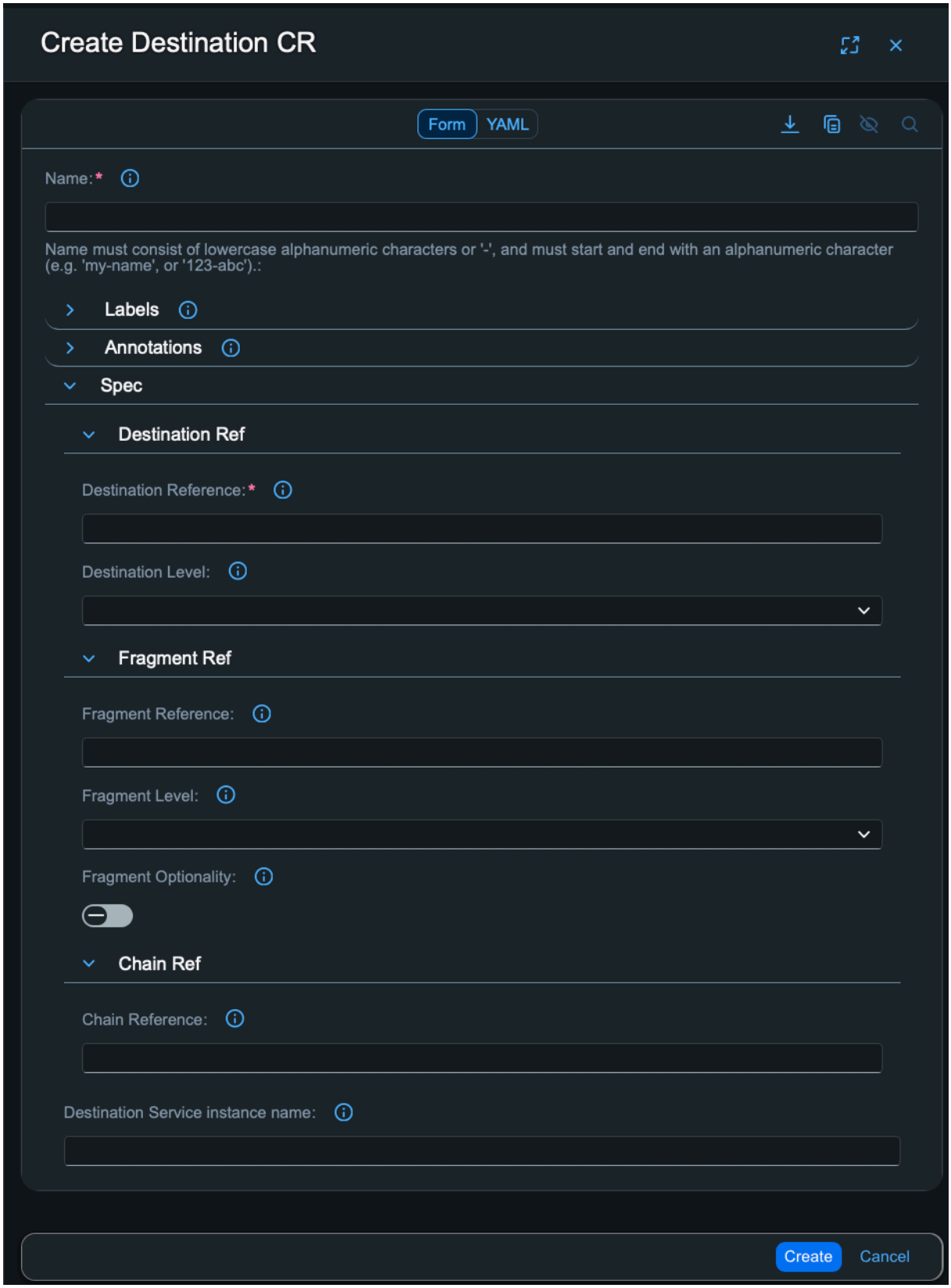
You have added Transparent Proxy module in the Kyma instance.

For more information, see Add the Transparent Proxy module [page 2332].

Procedure



1. After adding the Transparent Proxy module, navigate to a namespace of your choice in the Kyma instance.
2. In the left navigation bar, choose and expand *Connectivity*.
3. Select *Destination CRs* and choose *Create*.
4. In the open form you can enter the following properties:



Mandatory fields:

- `Name`: The name of the Destination CR. Must consist of lowercase alphanumeric characters or '-', and must start and end with an alphanumeric character (e.g. 'my-name', or '123-abc').
- `Destination Reference`: Reference an SAP BTP destination or add a wildcard symbol "*" to lookup one or more destinations dynamically.

Optional fields:

- `Fragment Reference`: Extend a destination with a fragment. Destination fragments enable more flexible technical connection configuration. For more information, see Extending Destinations with Fragments [page 1590].
- `Destination Level`: Specify a certain level from which the destination will be retrieved. For more information, see Referring Resources Using the Consume REST API [page 1569].
- `Fragment Level`: Specify a certain level from which the fragment will be retrieved. For more information, see Referring Resources Using the Consume REST API [page 1569].
- `Fragment Optionality`: Specify whether the fragment is optional (true) or mandatory (false).
- `Destination Service Instance Name`: The local name of the Destination service instance, provided in the Transparent Proxy configuration. For more information, see Configuration Guide [page 2265].
- `Chain Reference`: Reference a chain of destinations. For more information, see Destination Chaining [page 1538].

1.6.10 Concepts

Find information on basic concepts of the Transparent Proxy for Kubernetes.

Related Information

Actions [page 2337]
Cache [page 2341]
Destination Custom Resource [page 2342]
Destination Gateway (Dynamic Lookup of Destinations) [page 2349]
Managed Namespaces Mode [page 2352]
Multiple Destination Service Instances [page 2353]
Multitenancy [page 2355]

1.6.10.1 Actions

Actions let the Transparent Proxy attach a client-side authentication flow to a destination. When an action is configured, the Transparent Proxy obtains a token at request time and sets it on the request forwarded to the target system.

Depending on the action, the token is set either as the `Authorization` header (for tokens meant for the target system) or as the `SAP-Connectivity-Technical-Authentication` header (for a technical token meant for the Connectivity Proxy).

Actions are used to enable *Identity Authentication service* (IAS) principal propagation and technical access directly from the Transparent Proxy, without additional server-side configuration on the destination.

Supported Actions

Action	Description
`OAuth2JWTBearer`	*Principal propagation*. Exchanges the caller's IAS token for a new IAS access token using the JWT bearer grant, so the target system receives a token that represents the calling user. The minted token is set as the `Authorization` header.
`OAuth2ClientCredentials`	*Technical access*. Obtains a technical (machine-to-machine) IAS access token using the client credentials grant. The caller's identity is not used.
`OAuth2TechnicalUserPropagation`	*Technical access for on-premise connectivity*. Obtains a technical (machine-to-machine) IAS access token and sets it as the `SAP-Connectivity-Technical-Authentication` header, which is consumed by the Connectivity Proxy rather than forwarded to the target system. This action only applies to `OnPremise` destinations. The caller's identity is not used, and any caller-supplied `Authorization` header is removed so it cannot be forwarded.

Configuring an Action

Actions are configured under *spec.actions* of a destination custom resource. Each action has:

- `name`: the action type, one of `OAuth2JWTBearer`, `OAuth2ClientCredentials`, or `OAuth2TechnicalUserPropagation`.
- `secretRef`: a reference to a *Kubernetes Secret* that holds the credentials used to obtain the token. `name` and `key` are required; `namespace` is optional and defaults to the namespace of the destination.

- `triggers` (optional): conditions that control when the action is applied. See section *Triggers* below.

The secret referenced by `secretRef` must contain, at the given `key`, a JSON object with the credentials (for example, client ID and client secret, or a client certificate and key).

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
  kind: Destination
  metadata:
    name: jwtbearer-example
  spec:
    destinationRef:
      name: http
    actions:
      - name: OAuth2JWTBearer
        secretRef:
          name: jwtbearer-credentials
          key: credentials
        triggers:
          - name: exactMatch
            matches:
              - destinationProperty: HTML5.ForwardAuthToken
                value: "true"
          - name: exist
            destinationProperties:
              - "tokenService.body.resource"
```

Sample Code

OAuth2TechnicalUserPropagation

```
apiVersion: destination.connectivity.api.sap/v1
  kind: Destination
  metadata:
    name: technicaluserpropagation-example
  spec:
    destinationRef:
      name: onpremise-http
    actions:
      - name: OAuth2TechnicalUserPropagation
        secretRef:
          name: technicaluserpropagation-credentials
          key: credentials
```

Triggers

By default, an action is applied to every request. Add one or more triggers to apply an action only when the destination meets certain conditions. When multiple triggers are configured, *all* of them must be given for the action to run.

Two trigger types are available:

exist

Applies the action when the listed destination properties and/or destination labels are present, regardless of their values.

- `destinationProperties` – a list of destination property names that must exist.
- `destinationLabelKeys` – a list of destination label keys that must exist.

At least one of `destinationProperties` or `destinationLabelKeys` must be provided.

exactMatch

Applies the action only when every listed condition matches. Provide the conditions under `matches`. Each condition targets either a destination property or a destination label:

Fields	Matches when
`destinationProperty` + value	the destination property equals value.
`OAuth2ClientCredentials`	*Technical access*. Obtains a technical (machine-to-machine) IAS access token using the client credentials grant. The caller's identity is not used. The minted token is set as the `Authorization` header.
`OAuth2TechnicalUserPropagation`	*Technical access for on-premise connectivity*. Obtains a technical (machine-to-machine) IAS access token and sets it as the `SAP-Connectivity-Technical-Authentication` header, which is consumed by the Connectivity Proxy rather than forwarded to the target system. This action only applies to `OnPremise` destinations. The caller's identity is not used, and any caller-supplied `Authorization` header is removed so it cannot be forwarded.

Compatibility with the Destination Authentication Type

An action can only be applied when it does not conflict with the credentials the destination already provides. An action is allowed when the destination's authentication is `NoAuthentication`, `ClientHandledAuthentication`, or the same type as the action itself; any other authentication type is rejected. The following combinations are allowed:

Destination Authentication	OAuth2JWTBearer	OAuth2ClientCredentials	OAuth2TechnicalUserPropagation
`NoAuthentication`	yes	yes	yes
`ClientHandledAuthentication`	yes	yes	yes
`OAuth2JWTBearer`	yes	no	no

Destination Authentication	OAuth2JWTBearer	OAuth2ClientCredentials	OAuth2TechnicalUserPropagation
`OAuth2ClientCredentials`	no	yes	no
`OAuth2TechnicalUserPropagation`	no	no	yes
Any other authentication type	no	no	no

If an action is not compatible with the destination's authentication type, the destination is not configured and requests to it fail. The destination's status reports that the authentication type is not compatible with the configured action.

Additional Requirements for OAuth2TechnicalUserPropagation

Beyond authentication-type compatibility, `OAuth2TechnicalUserPropagation` has two runtime requirements that are enforced when a request is processed:

- The destination must have `ProxyType` `OnPremise`. If it does not, the request fails and the destination status reports that the action requires an `OnPremise` destination.
- The destination must provide a non-empty `tokenServiceURL`. If it is not present, the request fails and the status reports the missing token service configuration.

Unlike the other actions, `OAuth2TechnicalUserPropagation` does not set the `Authorization` header. It sets the `SAP-Connectivity-Technical-Authentication` header (overwriting any existing value) and removes any caller-supplied `Authorization` header from the forwarded request. Its token is not fed to any subsequent `Authorization`-based action in the same destination.

Prerequisites for IAS Actions

All actions require IAS configuration on the SAP BTP destination:

- `OAuth2JWTBearer` requires the IAS dependency name (`tokenService.body.resource`).
  For more information, see OAuth JWT Bearer Authentication [page 2188].
- `OAuth2ClientCredentials` requires the IAS dependency name (`tokenService.body.resource`) and the token service URL (`tokenServiceURL`).
  For more information, see OAuth Client Credentials Authentication [page 2186].
- `OAuth2TechnicalUserPropagation` requires the IAS dependency name (`tokenService.body.resource`) and the token service URL (`tokenServiceURL`). The destination must use proxy type `OnPremise`.
  For more information, see Technical User Propagation [page 2207].

These values are taken from the destination configuration returned by the Destination service. If a required value is missing, the destination is not configured and its status indicates the missing information.

Validation and status

When a Destination custom resource is created or changed, the Transparent Proxy validates its actions and reports the result in the resource's status conditions. Configuration problems that prevent an action from being applied include:

- The referenced Secret does not exist, cannot be read, or does not contain the specified key.
- A trigger is malformed (unknown trigger type or an invalid combination of fields).
- The action is not compatible with the destination's authentication type.
- Required IAS configuration is missing from the destination.
- For `OAuth2TechnicalUserPropagation`: the destination is not an `OnPremise` destination, or it is missing `tokenServiceURL`.

While such a condition is present, the destination is not usable and requests to it fail. Correct the configuration and the destination becomes available again.

1.6.10.2 Cache

Find informationn on cache types for the Transparent Proxy for Kubernetes.

Destination Service Access Token Cache

- When a request is made to a specific Destination service instance, the associated *SAP Authorization and Trust Management* service access token is stored by the Transparent Proxy. For future requests, if the stored token is still valid, it is reused, avoiding the need to call the *SAP Authorization and Trust Management* service again. If the stored token is invalid or missing, a new token is retrieved from the access token service and cached.
- Expiration time: expiration time of the respective OAuth token

Destination Cache

- SAP BTP destinations are cached by the Transparent Proxy. Incoming requests are handled more efficiently if the corresponding destination is already cached, eliminating the need to call the Destination

service API. If the destination is not in the cache, the Destination service API is called, and the fetched destination is cached for subsequent use.

- Expiration time: 2 minutes

Destination Tokens Cache

- Tokens for specific destination types (such as OAuth Authorization code, OAuth Refresh Token, OAuth User Token Exchange, OAuth JWT Bearer, OAuth SAML Bearer Assertion, SAML Assertion, SAP Assertion SSO) are cached during a request to avoid unnecessary exchanges in the future. If a request comes in and the appropriate destination is already in the cache, it is used directly, bypassing the need to call the Destination service API. If the tokens are not in the cache, a call to the Destination service API is made, and the resulting tokens are then cached.
- Expiration time: expiration time of the respective OAuth token

> **Note**
>
> If the Destination service or *SAP Authorization and Trust Management* service becomes unavailable, the Transparent Proxy will continue to use the information it has previously cached, about the destinations and their associated tokens. The cached data will be used until the connection with the Destination service or *SAP Authorization and Trust Management* service can be reestablished or until the tokens are valid. Once the connection is back, the proxy will update its cache with the most recent data. This way, it provides more resilience to temporary failures related to external services.

1.6.10.3 Destination Custom Resource

A destination custom resource represents an SAP BTP destination from the Destination service that can be used by the Transparent Proxy.

For each destination custom resource, the Transparent Proxy creates two Kubernetes Services : one in the Transparent Proxy's installation namespace and the other in the destination custom resource's namespace. Both services can be used to access the destination.

> **Note**
>
> We recommend that you create the destination custom resource in the same namespace as the application workload. By doing so, you can access the destination without specifying the namespace in the URL.

Prerequisites

To target the destination with the name “example-dest” for handling by the Transparent Proxy, you should create the following Kubernetes resource in a namespace of your choice:

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: <destination-name>
  destinationServiceInstanceName: dest-service-instance-example // can be
ommited if config.destinationService.defaultInstanceName is provided
```

Destination Fragments

You can reference a fragment as follows:

- Directly in the destination custom resource:
  Destination CR with Static Reference to a Destination Fragment

  Sample Code

  ```
  apiVersion: destination.connectivity.api.sap/v1
  kind: Destination
  metadata:
    name: <destination-cr-name>
  spec:
    destinationRef:
      name: <destination-name>
    fragmentRef:
      name: <fragment-name>
    destinationServiceInstanceName: <dest-service-instance-name> // can be
  ommited if config.destinationService.defaultInstanceName is provided
  ```

  ⚠ Caution

  Static reference of a fragment is only applicable for a destination custom resource that references a particular destination. The Dynamic Lookup of Destinations [page 2349] feature is not compatible with this approach.

- Dynamically passing a fragment as an HTTP header is only compatible with the Dynamic Lookup of Destinations [page 2349] approach. You can check the examples given there.

The Transparent Proxy monitors the available *destinations.destination.connectivity.api.sap* Kubernetes resources. Once a new SAP BTP destination is created for which *destinations.destination.connectivity.api.sap* resource exists, the Transparent Proxy changes its configuration.

After the Transparent Proxy executes successfully all necessary operations, the status of the *destinations.destination.connectivity.api.sap* Kubernetes resource will be updated as follows and the DNS record <destination-cr-name> will route to the configured URL in the destination:

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: <destination-name>
  fragmentRef:
    name: <fragment-name>
destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
status:
  conditions:
  - lastUpdateTime: "2022-09-28T07:26:46Z"
    message: Transparent Proxy is configured and Kubernetes service with name
"example-dest" is created.
    reason: ConfigurationSuccessful
    status: "True"
    type: Available
```

Destination/Fragment Levels

Destination/fragment levels enable specifying a certain level from which the destination/fragment will be retrieved. For more information, see Referring Resources Using the Consume REST API [page 1569].

You can choose a destination or fragment level as follows:

- Directly in the Destination custom resource:

  Sample Code

  ```
  apiVersion: destination.connectivity.api.sap/v1
  kind: Destination
  metadata:
    name: <destination-cr-name>
  spec:
    destinationRef:
      name: <destination-name>
      level: <destination-level>
    fragmentRef:
      name: <fragment-name>
      level: <fragment-level>
    destinationServiceInstanceName: <dest-service-instance-name> // can be
  ommited if config.destinationService.defaultInstanceName is provided
  ```

  Caution

  Static reference of destination/fragment levels is only applicable for a destination custom resource that references a particular destination/fragment. The Dynamic Lookup of Destinations [page 2349] feature is not compatible with this approach.

- Dynamically passing the destination/fragment level as HTTP headers. This approach is only compatible with the Dynamic Lookup of Destinations [page 2349] approach. You can check the examples given there.

Destination Fragments: Optionality

You can choose whether a fragment is optional in two ways::

- Directly in the destination custom resource:

  Sample Code

  ```
  apiVersion: destination.connectivity.api.sap/v1
  kind: Destination
  metadata:
    name: <destination-cr-name>
  spec:
    destinationRef:
      name: <destination-name>
    fragmentRef:
      name: <fragment-name>
      optional: true/false
    destinationServiceInstanceName: <dest-service-instance-name> // can be
  ommited if config.destinationService.defaultInstanceName is provided
  ```

  Caution

  Static reference of a fragment optionality is only applicable for a destination custom resource that references particular destination. The Dynamic Lookup of Destinations [page 2349] feature is not compatible with this approach.

- Dynamically passing whether a fragment is optional as an HTTP header is only compatible with the Dynamic Lookup of Destinations [page 2349] approach. You can check the examples given there.

Destination Chaining

You can choose to specify a destination chain name and variables statically in your destination custom resource.

For more information, see Destination Chaining [page 1538].

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: <destination-name>
  chainRef:
    name: <chain-name> // e.g - com.sap.iasGeneratedOAuth2SamlBearerAssertion
    variables:
    - name: <chain-var-key> // e.g - samlProviderDestinationName , will
result to header key - "X-Chain-Var-samlProviderDestinationName".
      value: <chain-var-value> // e.g. the destination name of the provider
destination in case "com.sap.iasGeneratedOAuth2SamlBearerAssertion" chain is
used.
```

```
    - name: <chain-var-key2>
      value: <chain-var-value2>
```

⚠ Caution

Passing headers that match the chain name or variable name will override the configuration from the destination custom resource.

For example, passing the "X-Chain-Name" header will override `spec.chainRef.name`, or passing "X-Chain-Var-key" will override a `chainRef` variable with name: "key".

Configure a Custom Service Port

In the destination custom resource spec you can describe on which port you would like to consume your destination:

{·} Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: <destination-name>
  destinationServiceInstanceName: <dest-service-instance-name> // can be
ommited if config.destinationService.defaultInstanceName is provided
  service:
    port: <service-port>
```

Configurable Target-System Timeouts

The destination custom resource supports two optional integer fields that set default timeouts for calls to the target system:

- `spec.readTimeout`: the socket read timeout, in seconds (allowed range **`1–86400`**, that is, up to 24 h).
- `spec.connectTimeout`: the connection timeout, in seconds (allowed range **`1–60`**).

These are defaults that apply to every tenant consuming the destination.

Precedence (highest first):

1. the value in the individual destination configuration (`URL.socketReadTimeoutInSeconds` / `URL.connectionTimeoutInSeconds`), if set.
2. the CR-level `spec.readTimeout` / `spec.connectTimeout`, if set.
3. the built-in default
   - read timeout: 30 s
   - connect timeout: the transparent proxy's configured destination service connection timeout, 5 s by default

Both fields apply to **HTTP destinations only**; they have no effect on TCP destinations. Values outside the allowed range are rejected (the destination CR reports `ReadTimeoutNotSupported` / `ConnectTimeoutNotSupported` in its status). Both fields can also be set from the Kyma dashboard.

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: <destination-name>
  destinationServiceInstanceName: <dest-service-instance-name> // can be
omitted if config.destinationService.defaultInstanceName is provided
  readTimeout: 200
  connectTimeout: 100
```

Consumption

Once done, the application can start consuming the destination from within the Kubernetes cluster.

Note

`<destination-cr-namespace>` can be omitted if the destination custom resource is created in the same namespace as the application workload.

Example without Additional Parameters

Sample Code

```
curl <destination-cr-name>.<destination-cr-namespace>
```

Example with Path Parameters

Sample Code

```
curl <destination-cr-name>.<destination-cr-namespace>/<param-1>/<param-2>/
<param-3>
```

Example with Path and Query Parameters

Sample Code

```
curl <destination-cr-name>.<destination-cr-namespace>/<param-1>/<param-2>/
<param-3>?<key-1>=<value-1>&<key-2>=<value-2>
```

Example with Custom Service Port

Sample Code

```
curl <destination-cr-name>.<destination-cr-namespace>:<custom-service-port>
```

Related Information

Restrict Access Using Scoping [page 2348]

1.6.10.3.1 Restrict Access Using Scoping

Define the access control scope of the destination custom resources for the Transparent Proxy for Kubernetes.

The scoping of access to destinations exposed via *destination custom resources* (destination CRs) is based on the *network policies* concept in Kubernetes.

Prerequisites

Network policies are implemented by a CNI Plugin. To apply network policies, you must use a networking solution that supports the *NetworkPolicy* Kubernetes resource.

Note

Creating a *NetworkPolicy* resource without having a corresponding controller running in the cluster, will have **no effect**. In this case, access scoping will not work.

For more information, see Network Policies .

Procedure

Global Configuration

You can set a default scope on all destination custom resources by setting `.Values.config.security.accessControl.destinations.defaultScope` in the `values.yaml` file.

- If .`Values.config.security.accessControl.destinations.defaultScope` is set to **`namespaced`**, the destinations exposed via destination custom resources that were created in a namespace, are accessible only by the applications running in this namespace of the destination custom resource.
- If `.Values.config.security.accessControl.destinations.defaultScope` is set to **`clusterWide`**, the destinations exposed via destination custom resources are accessible from any namespace in the cluster.
- The default value is **`namespaced`**.

Destination CR Configuration

You can also optionally set scope for the destination CR directly, which will override the global setting from the values.yaml. for that particular CR only.

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: <destination-cr-name>
spec:
  destinationRef:
    name: <destination-name>
  accessControl:
    scope: "namespaced|clusterWide"
```

1.6.10.4 Destination Gateway (Dynamic Lookup of Destinations)

Create a single custom resource to look up one ore more destinations dynamically with the Transparent Proxy for Kubernetes.

The Transparent Proxy lets you perform a dynamic lookup of one or multiple destinations from a destination service instance and its tenants. To consume a destination in that way, you only have to create a single custom resource.

Example: Destination Custom Resource

```
apiVersion: destination.connectivity.api.sap/v1
kind: Destination
metadata:
  name: destination-gateway
spec:
  destinationRef:
    name: "*"
  destinationServiceInstanceName: <destination-service-instance-local-
name> //not mandatory
```

In the example above, a destination custom resource has `spec.destinationRef.name = "*"`, which indicates that this destination would accept dynamic lookup and only a single Kubernetes service will be created with name `dynamic-destination` which works as an entry point for all destinations from `destinationServiceInstanceName` with `name: dest-service-instance`.

The Transparent Proxy identifies the destination for which a request should be configured and dispatched by obtaining the destination name (destination identifier) from a custom header called `X-Destination-Name`.

> **Note**
>
> This header is specifically intended to identify a destination within a Destination service instance and its tenants.

The destination can be combined with a fragment by optionally providing a custom header called `X-Fragment-Name`. You can choose whether the fragment is optional or mandatory by passing the `X-Fragment-Optional` header, which accepts values `true` or `false`.

If `X-Fragment-Optional=true` and the referenced fragment exists, the destination and fragment are combined. If the referenced fragment does not exist, the base destination is used. Conversely, if `X-Fragment-Optional=false` and the referenced fragment exists, the destination and fragment are combined. If the referenced fragment does not exist, an error is returned.

Example Destination:

```
{
    "Name": "example-dest",
    "Type": "HTTP",
    "ProxyType": "Internet",
    "URL": "<application-url>",
    "Authentication": "ClientCertificateAuthentication",
    "KeyStorePassword": "<key-store-password>",
    "KeyStoreLocation": "<key-store-location>"
}
```

Example Fragment:

```
{
    "FragmentName": "example-fragment",
    "URL": "<another-application-url>"
}
```

If you want to call the `example-dest` destination through the Kubernetes service named `destination-gateway`, you can execute the command below:

Dynamic Lookup of Destinations - Bash Snippet

```
curl destination-gateway.<destination-cr-namespace> -H "X-Destination-Name:
example-dest"
```

If you want to consume `example-dest` destination with path and query parameters through the Kubernetes service named `destination-gateway`, you can execute the command below:

Dynamic Lookup of Destinations with Path and Query Parameters - Bash Snippet

```
curl destination-gateway.<destination-cr-namespace>/<param-1>/<param-2>/
<param-3>?<key-1>=<value-1>&<key-2>=<value-2> -H "X-Destination-Name: example-
dest"
```

If you want to call `example-dest` destination, enriching it with the properties from fragment `example-fragment` through the Kubernetes service named `dynamic-destination`, you can execute the command below:

Dynamic Lookup of Destinations with Fragment - Bash Snippet

```
curl destination-gateway.<destination-cr-namespace> -H "X-Destination-Name:
example-dest" -H "X-Fragment-Name: example-fragment"
```

If you want to call `example-dest` destination, enriching it with the properties from fragment `example-fragment` through the Kubernetes service named `destination-gateway` and specify that the fragment is optional, you can execute the command below:

Dynamic Lookup of Destinations with Fragment (Optional) - Bash Snippet

```
curl destination-gateway.<destination-cr-namespace> -H "X-Destination-Name:
example-dest" -H "X-Fragment-Name: example-fragment" -H "X-Fragment-Optional:
true"
```

If you want to call `example-dest` destination, enriching it with the properties from fragment `example-fragment` through the Kubernetes service named `destination-gateway` and specify a destination level `provider_subaccount`, you can execute the command below:

Dynamic Lookup of Destinations with Destination Level - Bash Snippet

```
curl destination-gateway.<destination-cr-namespace> -H "X-Destination-Name:
example-dest" -H "X-Destination-Level: provider_subaccount"
```

If you want to call `example-dest` destination, enriching it with the properties from fragment `example-fragment` through the Kubernetes service named `destination-gateway` and specify a fragment level `provider_instance`, you can execute the command below:

Dynamic Lookup of Destinations with Fragment and Fragment Level - Bash Snippet

```
curl destination-gateway.<destination-cr-namespace> -H "X-Destination-Name:
example-dest" -H "X-Fragment-Name: example-fragment" -H "X-Fragment-Level:
provider_instance"
```

If you want to call `example-dest` destination, enriching it with the properties from fragment `example-fragment` through the Kubernetes service named `destination-gateway` and specify both, destination and fragment levels, you can execute the command below:

Dynamic Lookup of Destinations with Destination Level and Fragment Level - Bash Snippet - Bash Snippet

```
curl destination-gateway.<destination-cr-namespace> -H "X-Destination-Name:
example-dest" -H "X-Fragment-Name: example-fragment" -H "X-Destination-Level:
provider_subaccount" -H "X-Fragment-Level: subaccount"
```

If you want to call `example-dest` destination, enriching it with destination chaining properties through the Kubernetes service named `destination-gateway`, you can execute the command below:

Dynamic Lookup of Destinations Using Destination Chaining- Bash Snippet

```
curl destination-gateway.<destination-cr-namespace> -H "X-Destination-Name:
example-dest" -H "X-Chain-Name: <chain-name>" -H "X-Chain-Var-key1: <val1>" -H
"X-Chain-Var-key2: <val2>" // and etc for each x-chan-var header
```

If you want to call `example-dest` destination, enriching it with the properties from fragment `example-fragment` through the Kubernetes service named `destination-gateway` and specify that the fragment is not optional, you can execute the command below:

Dynamic Lookup of Destinations with Fragment (Not Optional) - Bash Snippet

```
curl destination-gateway.<destination-cr-namespace> -H "X-Destination-Name:
example-dest" -H "X-Fragment-Name: example-fragment" -H "X-Fragment-Optional:
false"
```

1.6.10.5 Managed Namespaces Mode

Manage namespaces for the Transparent Proxy for Kubernetes.

The Transparent Proxy can operate in either:

- All namespaces in the cluster, or
- Only in namespaces labeled in the following way: *transparent-proxy.connectivity.api.sap/namespace:<namespace where Transparent Proxy is installed>*.

To configure this feature, set the Helm property `config.managedNamespacesMode` to either:

- **`all`**
  This is the default setting, indicating the proxy operates with destination custom resources across all namespaces in the cluster.
- **`labelSelector`**
  Restricting the proxy to operate only in namespaces with the specific label.

For more information, see Configuration Guide [page 2265].

1.6.10.6 Multiple Destination Service Instances

The Destination service lets you declaratively model a technical connection as a destination configuration, and find via its REST APIs the destination configuration that is required to access a remote service or system from your Kubernetes workload.

To integrate the Transparent Proxy with the Destination Service, you must:

1. Create a Destination service instance or reuse an existing one.
2. Create a service key for the service instance.
3. Follow the instructions in step Prepare Destination service instance credentials [page 2247] to pair with the Destination service. This involves referencing a secret that contains the base64-encoded service key for the Destination service.
4. Check the Configuration Guide [page 2265] for *config.integration.destinationService.instances*.

The Transparent Proxy can consume SAP BTP destinations from multiple Destination service instances. They are specified in the Transparent Proxy configuration.

Integration with Multiple Destination Service Instances

Sample Code

```
config:
  integration:
    destinationService:
       ## Name of the default destination service instance to be used when
'destinationServiceInstanceName' is not provided in the Destination Custom
Resource spec.
       #defaultInstanceName: dest-service-instance
       instances:
    ##     Name of the destination service instance that is described.
It could be used as config.integration.destinationService.defaultInstanceName
or it can be referenced in the Destination Custom Resource spec as
destinationServiceInstanceName.
          - name: dest-service-instance
            serviceCredentials:
    ##         The key in the Destination service secret resource, which
holds the base64 encoded value of the destination service key. (info)
Make sure to provide the right key for an existing secret. Required when
Transparent Proxy should create secret and not required when describing an
existing secret.
              secretKey: <secret-key>
    ##         The name of the existing secret, which holds the credentials
for the Destination service or the name of the secret to be created based on
"secretName", "secretData", "secretKey" fields.
              secretName: <secret-name>
    ##         The base64 encoded value of the service key, obtained from the
Destination service instance. Required when Transparent Proxy should create
secret and not required when describing an existing secret.
              secretData: <secret-data>
    ##         The namespace of the existing secret to be used, which holds
the credentials for the Destination service. This field should not be present
if there is no existing secret and Transparent Proxy would create one based
on "secretName", "secretData", "secretKey" fields
    #          secretNamespace: <secret-namespace>
    #          privateKey:
    ##           The name of K8s secret, which holds the private key to
authenticate if using an x509-based service key to the Destination service.
    #            secretName: <x509-svc-key-secret-name>
```

```
    ##           The name of the internal key inside the K8s secret holding
the private key to authenticate if using an x509-based service key to the
Destination service.
    #            secretInternalKey: <key>
    ##     Name of the destination service instance that is described.
It could be used as config.integration.destinationService.defaultInstanceName
or it can be referenced in the Destination Custom Resource spec as
destinationServiceInstanceName.
          - name: dest-service-instance-2
            serviceCredentials:
    ##         The key in the Destination service secret resource, which
holds the base64 encoded value of the destination service key. (info)
Make sure to provide the right key for an existing secret. Required when
Transparent Proxy should create secret and not required when describing an
existing secret.
              secretKey: <secret-key>
    ##         The name of the existing secret, which holds the credentials
for the Destination service or the name of the secret to be created based on
"secretName", "secretData", "secretKey" fields.
              secretName: <secret-name>
    ##         The namespace of the existing secret to be used, which holds
the credentials for the Destination service. This field should not be present
if there is no existing secret and Transparent Proxy would create one based
on "secretName", "secretData", "secretKey" fields
    #          secretNamespace: <secret-namespace>
    #          privateKey:
    ##           The name of K8s secret, which holds the private key to
authenticate if using an x509-based service key to the Destination service.
    #            secretName: <x509-svc-key-secret-name>
    ##           The name of the internal key inside the K8s secret holding
the private key to authenticate if using an x509-based service key to the
Destination service.
    #            secretInternalKey: <key>
```

A specific instance of the Destination service will be selected by the Transparent Proxy based on the following rules:

If the `defaultInstanceName` is provided in the Transparent Proxy configuration, the `destinationServiceInstanceName` field in the destination custom resource specification becomes optional.

In cases where two different Destination service instance names are specified, the one in the destination custom resource specification will be prioritized.

Example with a Default Destination Service Instance

Sample Code

```
integration:
  destinationService:
    ## Local name of the default Destination service instance to be used when
'destinationServiceInstanceName' is not provided in the destination custom
resource spec.
    defaultInstanceName: dest-service-instance
```

Example with a Destination Service Instance Name

Sample Code

```
apiVersion: destination.connectivity.api.sap/v1
```

```
kind: Destination
metadata:
  name: example-dest
spec:
  destinationRef:
    name: "example-dest-client-cert"
  destinationServiceInstanceName: dest-service-instance-example // can be
ommited if config.integration.destinationService.defaultInstanceName is
provided
```

> ⓘ Note
>
> For more details about configuring the Transparent Proxy, check the Configuration Guide [page 2265].

1.6.10.7 Multitenancy

Use multitenancy for the Transparent Proxy for Kubernetes.

The Transparent Proxy works in two tenant modes: *dedicated* and *shared*. You can change the mode by modifying the Helm property `.config.tenantMode`.

For more information, see Configuration Guide [page 2265].

- In the *dedicated* tenant mode, the Transparent Proxy exposes destinations only from the linked Destination service instance and its subaccount. Requests to the Transparent Proxy remain unchanged.
- In the *shared* tenant mode, the Transparent Proxy exposes destinations from any subscribed tenant.

HTTP

When requesting a destination through the Transparent Proxy in shared tenant mode, you must pass the `X-Tenant-Subdomain: <tenant-subdomain>` or `X-Tenant-Id: <tenant-id>` header.

A tenant is successfully onboarded when it is visible in the destination custom resource status. The onboarding of a tenant is dynamic and happens during the first request on behalf of that tenant.

Non-HTTP

Tenants are configured in the destination custom resource (CR) as an annotation with the key `transparent-proxy.connectivity.api.sap/tenant-subdomains`. The value should be a JSON array of tenant subdomains:

```
"transparent-proxy.connectivity.api.sap/tenant-subdomains":
'["tenantSubdomain1", "tenantSubdomain2", ...]'
```

The Transparent Proxy will create a Kubernetes Service for each of the tenants described in the `transparent-proxy.connectivity.api.sap/tenant-subdomains` annotation. The names of the Kubernetes services will be: `<destination-cr-name>-<tenant-subdomain>`.

Each tenant is successfully onboarded when it is visible in a Destination CR status.

1.6.11 Local Development

Find a local development guide for the Transparent Proxy for Kubernetes.

Prerequisites

Before you begin, ensure the following prerequisites are set up:

1. *kubectl* (or an equivalent client) installed on your machine to communicate with the Kubernetes cluster's control plane.
2. One or more Destination service instances that the Transparent Proxy can work with.
3. An SAP BTP destination created within one of the above Destination service instances, which references the end system.
4. A destination custom resource created for that destination.

> **Note**
>
> Transparent proxy will create a Kubernetes Service for every destination custom resource which it processes. Each service is an entry point to its corresponding destination.

Guide

This guide outlines the steps to set up port forwarding and use a Transparent Proxy service from your local environment. Follow the steps below to list the services, forward ports, and consume services.

1. Find the Kubernetes Service that match your destination custom resource name and namespace.

Search by labels containing destination custom resource attributes(e.g. name=example-dest, namespace=client-namespace, tp-namespace=sap-transp-proxy-system):

Sample Code

```
kubectl get svc -n <tp-
namespace> -l transparent-proxy.connectivity.api.sap/
parent-destination-cr-name=<name>,transparent-proxy.connectivity.api.sap/
parent-destination-cr-namespace=<namespace> -o yaml
```

Sample Code

```
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: "2025-11-22T21:44:48Z"
  labels:
    transparent-proxy.connectivity.api.sap/parent-destination-cr-name:
example-dest
    transparent-proxy.connectivity.api.sap/parent-destination-cr-
namespace: client-namespace
  name: example-dest-d3q12
  namespace: sap-transp-proxy-system
...
```

2. Port forward the selected service to your local machine. Replace `<local-port>` with any available port number on your local machine (for example, `8042`), and `<k8s-svc-port>` with the port number used by the Kubernetes service (for example, `80`).

   Sample Code

   ```
   kubectl port-forward svc/<cluster-ip-service-name> <local-port>:<k8s-svc-
   port> -n <transparent-proxy-namespace>
   ```

   For example, to port forward `example-dest-no-auth-d3q12` service to local port `8042`, execute:

   Sample Code

   ```
   kubectl port-forward svc/example-dest-d3q12 8042:80 -n <transparent-proxy-
   namespace>
   ```

3. When making requests from your local environment, **include the Host header** with the `ClusterIP` service name.

   Consumption Using curl

   Sample Code

   ```
   curl localhost:<local-port> -H "Host: <cluster-ip-service-name>"
   ```

   Consumption of example-dest-no-auth on Port 8042 Using curl

   Sample Code

   ```
   curl localhost:8042 -H "Host: example-dest-d3q12"
   ```

> ⚠ Caution
>
> If you encounter an error response from the Transparent Proxy, refer to the Error Response Headers [page 2308] page for detailed information and troubleshooting guidance.

1.6.12 Transparent Proxy Operator

Use the Transparent Proxy Operator for the Transparent Proxy for Kubernetes.

The Transparent Proxy Operator continuously observes the state of the system and the desired state defined by the Transparent Proxy custom resource. It then makes necessary adjustments to the system (like creating, updating, or deleting resources) to achieve the desired state, and regularly monitors the health of the Transparent Proxy, ensuring it runs optimally according to the configurations defined in the custom resource.

Automatic Tracking and Revival of Mandatory Resources

The Transparent Proxy Operator continuously monitors Transparent Proxy components within the cluster and makes necessary adjustments in order to achieve the desired state.

If a mandatory resource fails or is accidentally deleted, the operator automatically:

- Detects the missing resource
- Attempts to revive or recreate the resource based on its last known configuration
- Ensures the resource is properly integrated back into the cluster

This feature maintains the stability and reliability of essential cluster components without manual intervention.

Dependencies

Traffic Encryption

The dependencies below are interchangeable and up to your preference:

- Istio Service Mesh Integration [page 2289]
- Certificate Manager [page 2287]

Destination Service Integration

If the BTP Operator is available in the cluster, a Destinations service instance is created/loaded depending on the setup, automating the configuration. To manually integrate with the Destination service, see Integration with SAP BTP Connectivity [page 2247].

Automatic Creation of a Destination Service Instance

If no resources of api version "services.cloud.sap.com/v1" and kind "ServiceInstance" exist in namespace "sap-transp-proxy-system" and no destination service instances are directly specified in the Transparent Proxy CR, a default, empty destination service instance with name "sap-transp-proxy-default" will be created in namespace "sap-transp-proxy-system" and loaded as destination service instance in the Transparent Proxy CR.

Automatic Loading of a Destination Service Instances

If the service instance and service bindings for a certain Destination service instance are created in the Transparent Proxy namespace, they will be automatically loaded by the Transparent Proxy. The Transparent Proxy will load all resources of api version "services.cloud.sap.com/v1" and kind "ServiceInstance" having "spec.serviceOfferingName: destination", created in namespace "sap-transp-proxy-system", as destination service instances.

1.6.13 Security Considerations

Recommended security measures for the Transparent Proxy for Kubernetes.

Kubernetes Resources Managed by Transparent Proxy

The Transparent Proxy manages *Transparent Proxy Custom Resources*, *Destination Custom Resources*, and *Config Map* resources. All Kubernetes resources require comprehensive protection to keep your cluster secure against both external threats and insider misuse, by enforcing strong RBAC, encryption, and auditing.

Recommended Security Practices

Implement these cluster-wide security measures for comprehensive protection for the resources above:

Access Control

- **Enforce least privilege** for all users, service accounts, and controllers
- Configure proper RBAC policies to control the access for all resources managed by the Transparent Proxy
- Regularly review and audit permissions

Encryption & Communication

- **Enable TLS everywhere** - API server, *kubelet*, and *etcd* communications
- **Turn on** *etcd* **encryption at rest** for all cluster data
- Use secure communication channels for all cluster operations

Monitoring & Auditing

- **Enable comprehensive audit logging** for all API operations
- **Ship logs to a centralized system** for monitoring and analysis

- Set up monitoring dashboards for configuration changes
- Implement alerting for unusual activity patterns

Kubernetes Watch Events

The Transparent Proxy listens for **ADDED**, **MODIFIED**, and **DELETED** events for its relevant resources, which are emitted by the Kubernetes API server when resource changes occur.

The Transparent Proxy implements efficient reconciliation which first compares the relevant fields of the resource to the current state. If no meaningful change has occurred, it exits early without performing expensive operations.

Kubernetes has built-in rate limiting that automatically restricts how many API requests can be processed at once. In case of excessive API calls, Kubernetes will automatically throttle the requests, protecting applications from being flooded with too many events to handle.

Recommended Security Practices

- Define default CPU and memory limits for the Transparent Proxy deployment. This contains the blast radius of any potential resource exhaustion, ensuring the controller cannot cause a node-level DoS.
- Configure proper RBAC policies to control all Kubernetes resources managed by the Transparent Proxy.

1.7 Connectivity via Reverse Proxy

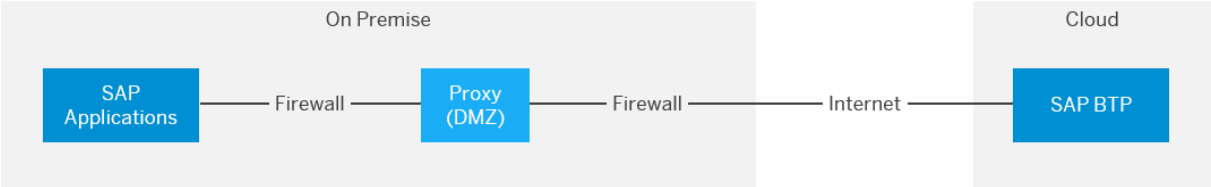
The text discusses the use of a reverse proxy as an alternative approach to connect on-premise services to SAP BTP. While it allows for reuse of existing network infrastructure, it exposes services to potential attacks and requires significant involvement from your IT department. The Cloud Connector is recommended as a more secure and efficient solution, providing TLS tunneling and fine-grained access control.

An alternative approach compared to the TLS tunnel solution that is provided by the Cloud Connector is to expose on-premise services and applications via a reverse proxy to the Internet. This method typically uses a reverse proxy setup in a customer's "demilitarized zone" (DMZ) subnetwork. The reverse proxy setup does the following:

- Acts as a mediator between SAP BTP and the on-premise services
- Provides the services of an Application Delivery Controller (ADC) to, for example, encrypt, filter, route, or check inbound traffic

The figure below shows the minimal overall network topology of this approach.

On-premise services that are accessible via a reverse proxy are callable from SAP BTP like other HTTP services available on the Internet. When you use destinations to call those services, make sure the configuration of the `ProxyType` parameter is set to **`Internet`**.



Advantages

Depending on your scenario, you may benefit from the reverse proxy:

- Network infrastructure (such as a reverse proxy and ADC services): since it already exists in your network landscape, you can reuse it to connect to SAP BTP. There's no need to set up and operate new components on your (customer) side.
- A reverse proxy is independent of the cloud solution you are using.
- It acts as single entry point to your corporate network.

Disadvantages

- The reverse proxy approach leaves exposed services generally accessible via the Internet. This makes them vulnerable to attacks from anywhere in the world. In particular, **Denial-of-Service attacks** are possible and difficult to protect against. To prevent attacks of this type and others, you must implement the highest security in the DMZ and reverse proxy. For the productive deployment of a hybrid cloud/on-premise application, this approach usually requires intense involvement of the customer's IT department and a longer period of implementation.
- If the reverse proxy allows filtering, or restricts accepted source IP addresses, you can set only one IP address to be used for all SAP BTP outbound communications.
  A reverse proxy does not exclusively restrict the access to cloud applications belonging to a customer, although it does filter any callers that are not running on the cloud. Basically, any application running on the cloud would pass this filter.
- The SAP-proprietary RFC protocol is supported only if WebSocket RFC can be used for communication with the ABAP system. WebSocket RFC is available as of S/4HANA release 1909. A cloud application cannot call older on-premise ABAP systems directly without using application proxies on top of ABAP in between.
- No easy support of principal propagation authentication, which lets you forward the cloud user identity to on-premise systems.
- You cannot implement projects close to your line of business (LoB).

> ### Note
>
> Using the Cloud Connector mitigates all of these issues. As it establishes the TLS tunnel to SAP BTP using a reverse invoke approach, there is no need to configure the DMZ or external firewall of a customer network for inbound traffic. Attacks from the Internet are not possible. With its simple setup and fine-grained access control of exposed systems and resources, the Cloud Connector allows a high level of security and fast

productive implementation of hybrid applications. It also supports multiple application protocols, such as HTTP and RFC.

1.8 Connectivity Support

Support information for SAP BTP Connectivity and the Cloud Connector.

Troubleshooting

Locate the problem or error you have encountered and follow the recommended steps:

- Frequently Asked Questions [page 2073] (Cloud Connector)
- Administration [page 1970] (Cloud Connector)
- Getting Support (SAP Support Portal, SAP BTP community)

SAP Support Information

If you cannot find a solution to your issue, collect and provide the following specific, issue-relevant information to SAP Support:

- The Java EE code that throws an error (if any)
- A screenshot of the error message for the failed operation or the error message from the `HttpResponse` body
- Access credentials for your cloud or on-premise location

You can submit this information by creating a customer ticket in the SAP CSS system using the following components:

Component	Purpose
Connectivity Service	
BC-CP-CON	For *cloud-side* issues with **cloud to on-premise** connectivity, where: • The **environment is unknown** or • The issue is **not related** to a **specific environment**
BC-CP-CON-CF	For *cloud-side* issues with **cloud to on-premise** connectivity in the SAP BTP **Cloud Foundry environment**.
BC-CP-CON-S4HC	For *cloud-side* issues with **cloud to on-premise** connectivity in an **S/4HANA Cloud** system.

Component	Purpose
BC-CP-CON-K8S-PROXY	For cloud-side issues with **cloud to on-premise** connectivity in a Kubernetes cluster (or Kubernetes-based product), using the connectivity proxy software component.
BC-CP-CON-ABAP	For *cloud-side* issues with **cloud to on-premise** connectivity in the SAP BTP **ABAP environment**.
BC-NEO-CON	For *cloud-side* issues with **cloud to on-premise** connectivity in the SAP BTP **Neo environment**.
Destinations	
BC-CP-DEST	For issues with **destination configurations**, where: • The **environment is unknown** or • The issue is **not related** to a **specific environment**
BC-CP-DEST-CF	For **general issues** with the Destination service in the SAP BTP **Cloud Foundry environment**, like: • REST API • Instance creation, etc.
BC-CP-DEST-CF-CLIBS	For **client library issues** with the Destination service in the SAP BTP **Cloud Foundry environment**.
BC-CP-DEST-CF-TOOLS	For issues with the management of **destination configurations via tools** like the SAP BTP cockpit (**Cloud Foundry environment**).
BC-CP-DEST-NEO	For issues with **destination configurations** or: • Management tools • Client libraries, etc. related to destinations in the SAP BTP **Neo environment**.
Cloud Connector	
BC-MID-SCC	For connectivity issues related to installing and configuring the **Cloud Connector**, configuring tunnels, connections, and so on.

If you experience a more serious issue that cannot be resolved using only traces and logs, SAP Support may request access to the Cloud Connector. Follow the instructions in these SAP notes:

- To provide access to the Administration UI via a browser, see 592085.
- To provide SSH access to the operating system of the Linux machine on which the connector is installed, see 1275351.

Related Information

Release and Maintenance Strategy [page 2364]

1.8.1 Release and Maintenance Strategy

Find information about SAP BTP Connectivity releases, versioning and upgrades.

Release Cycles

Updates of the Connectivity service are published as required, within the regular, bi-weekly SAP BTP release cycle.

New releases of the Cloud Connector are published when new features or important bug fixes are delivered, available on the Cloud Tools page.

Cloud Connector Versions

Cloud Connector versions follow the `<major>.<minor>.<micro>` versioning schema. The Cloud Connector stays fully compatible within a major version. Within a minor version, the Cloud Connector will stay with the same feature set. Higher minor versions usually support additional features compared to lower minor versions. Micro versions generally consist of patches to a `<master>.<minor>` version to deliver bug fixes.

For each supported major version of the Cloud Connector, only one `<major>.<minor>.<micro>` version will be provided and supported on the Cloud Tools page. This means that users must upgrade their existing Cloud Connectors to get a patch for a bug or to make use of new features.

For detailed support strategy information, check SAP note 3302250.

Cloud Connector Upgrade

New versions of the Cloud Connector are announced in the Release Notes of SAP BTP. We recommend that Cloud Connector administrators regularly check the release notes for Cloud Connector updates. New versions of the Cloud Connector can be applied by using the Cloud Connector upgrade capabilities. For more information, see Upgrade [page 2069].

> **Note**
>
> We recommend that you first apply upgrades in a test landscape to validate that the running applications are working.

There are no manual user actions required in the Cloud Connector when the SAP BTP is updated.

1.9 Submitting Improvement Requests

You can submit your improvement request for SAP BTP Connectivity. It's a single central place for all activities related to product improvement requests.

Context

You can also browse improvement requests, comment, vote, receive updates, and browse who has voted.

> **Note**
>
> Before you submit your improvement, check if a similar improvement hasn't already been submitted requested. If yes, please vote for it, instead of submitting a new one.
>
> For more information, see also SAP note 11.

Procedure

1. Access the SAP Customer Influence site from the following link: https://influence.sap.com/sap/ino/#/campaign/2277
2. Log on with your *S user*.
3. Choose *Improvement Request* on the left panel.
4. Enter the details for the improvement request you want to submit.

Field	Notes
Project	The name of the current product session. It can't be edited.
Title	Enter your title here clearly stating your request and area of improvement.
IR Category	Select *Connectivity* from the drop-down list.
Description	Provide details about your idea.
Tags	Use at least one tag that matches your request.

   You can also provide links and attachments, or add co-authors.

5. Choose the *Submit* button when you are done.

Important Disclaimers and Legal Information

Hyperlinks

Some links are classified by an icon and/or a mouseover text. These links provide additional information.
About the icons:

- Links with the icon : You are entering a Web site that is not hosted by SAP. By using such links, you agree (unless expressly stated otherwise in your agreements with SAP) to this:
  - The content of the linked-to site is not SAP documentation. You may not infer any product claims against SAP based on this information.
  - SAP does not agree or disagree with the content on the linked-to site, nor does SAP warrant the availability and correctness. SAP shall not be liable for any damages caused by the use of such content unless damages have been caused by SAP's gross negligence or willful misconduct.
- Links with the icon : You are leaving the documentation for that particular SAP product or service and are entering an SAP-hosted Web site. By using such links, you agree that (unless expressly stated otherwise in your agreements with SAP) you may not infer any product claims against SAP based on this information.

Videos Hosted on External Platforms

Some videos may point to third-party video hosting platforms. SAP cannot guarantee the future availability of videos stored on these platforms. Furthermore, any advertisements or other content hosted on these platforms (for example, suggested videos or by navigating to other videos hosted on the same site), are not within the control or responsibility of SAP.

Beta and Other Experimental Features

Experimental features are not part of the officially delivered scope that SAP guarantees for future releases. This means that experimental features may be changed by SAP at any time for any reason without notice. Experimental features are not for productive use. You may not demonstrate, test, examine, evaluate or otherwise use the experimental features in a live operating environment or with data that has not been sufficiently backed up.
The purpose of experimental features is to get feedback early on, allowing customers and partners to influence the future product accordingly. By providing your feedback (e.g. in the SAP Community), you accept that intellectual property rights of the contributions or derivative works shall remain the exclusive property of SAP.

Example Code

Any software coding and/or code snippets are examples. They are not for productive use. The example code is only intended to better explain and visualize the syntax and phrasing rules. SAP does not warrant the correctness and completeness of the example code. SAP shall not be liable for errors or damages caused by the use of example code unless damages have been caused by SAP's gross negligence or willful misconduct.

Bias-Free Language

SAP supports a culture of diversity and inclusion. Whenever possible, we use unbiased language in our documentation to refer to people of all cultures, ethnicities, genders, and abilities.

www.sap.com/contactsap

© 2026 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company. The information contained herein may be changed without prior notice.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors. National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. All other product and service names mentioned are the trademarks of their respective companies.

Please see https://www.sap.com/about/legal/trademark.html for additional trademark information and notices.